

Video and Vision Processing Suite IP User Guide

Updated for Quartus® Prime Design Suite: **25.1**

Contents

1. About the Video and Vision Processing Suite.....	12
1.1. Video and Vision Processing IPs Features.....	13
1.2. Device Family Support.....	14
1.3. Video and Vision Processing IPs Release Information.....	14
1.4. Lite versus Full IP Variants.....	15
1.5. Glossary of Video and Vision Terminology.....	16
2. Getting Started with the Video and Vision Processing IPs.....	18
2.1. Video and Vision Processing IP Evaluation Time-out Behavior.....	18
2.2. Generating a Video and Vision Processing IP.....	18
2.3. Simulating the Video and Vision Processing IPs.....	20
3. Video and Vision Processing IPs Functional Description.....	21
3.1. Auxiliary Control Packets.....	24
3.2. Latency.....	25
3.3. IP Debugging Features.....	25
3.4. Stall Behavior and Error Recovery.....	26
3.5. Avalon Streaming Video Protocol Versus Altera Streaming Video Protocol.....	27
4. Video and Vision Processing IP Interfaces.....	30
5. Video and Vision Processing IP Registers	32
5.1. Video and Vision Processing IP Control Examples.....	35
6. Video and Vision Processing IPs Software Programming Model.....	38
7. Protocol Converter IP.....	46
7.1. About the Protocol Converter IP.....	46
7.1.1. Protocol Converter IP Performance and Resources.....	46
7.2. Protocol Converter IP Parameters.....	47
7.3. Protocol Converter IP Functional Description.....	48
7.4. Protocol Converter IP Interfaces.....	54
7.5. Protocol Converter IP Registers.....	56
7.6. Protocol Converter IP Software API.....	61
8. 1D LUT IP.....	68
8.1. About the 1D LUT IP.....	68
8.1.1. 1D LUT IP Performance and Resources.....	68
8.2. 1D LUT IP Parameters.....	69
8.3. 1D LUT IP Interfaces.....	71
8.4. 1D LUT IP Registers.....	72
8.5. 1D LUT IP Software API.....	74
9. 3D LUT IP.....	85
9.1. About the 3D LUT IP.....	85
9.1.1. 3D LUT IP Features.....	85
9.1.2. 3D LUT IP Performance IP and Resource Information.....	85
9.2. 3D LUT IP Parameters.....	86
9.3. 3D LUT IP Block Description.....	88

9.3.1. 3D LUT IP Interfaces.....	89
9.3.2. 3D LUT IP Latency.....	90
9.4. 3D LUT IP Registers.....	90
9.5. 3D LUT IP Software API.....	93
10. Adaptive Noise Reduction IP.....	99
10.1. About the Adaptive Noise Reduction IP.....	99
10.1.1. Adaptive Noise Reduction IP Performance and Resources.....	99
10.2. Adaptive Noise Reduction IP Parameters.....	100
10.3. Adaptive Noise Reduction IP Functional Description.....	101
10.3.1. Adaptive Noise Reduction IP Interfaces.....	103
10.4. Adaptive Noise Reduction IP Registers.....	104
10.5. Adaptive Noise Reduction IP Software API.....	107
11. Advanced Test Pattern Generator IP.....	117
11.1. About the Advanced Test Pattern Generator IP.....	117
11.1.1. Advanced Test Pattern Generator IP Performance and Resources.....	117
11.2. Advanced Test Pattern Generator IP Parameters.....	118
11.3. Advanced Test Pattern Generator IP Functional Description.....	120
11.3.1. Test Patterns.....	121
11.3.2. Dimensions, Offsets, and Alpha Settings.....	125
11.3.3. Run-time Settings.....	126
11.3.4. Advanced Test Pattern Generator IP Interfaces.....	127
11.4. Advanced Test Pattern Generator IP Registers.....	128
11.5. Advanced Test Pattern Generator IP Software API.....	137
12. AXI-Stream Broadcaster IP.....	158
12.1. About the AXI-Stream Broadcaster IP.....	158
12.1.1. AXI-Stream Broadcaster IP Features.....	158
12.1.2. AXI-Stream Broadcaster IP Performance and Resources.....	159
12.2. AXI-Stream Broadcaster IP Parameters.....	159
12.3. AXI-Stream Broadcaster IP Interfaces.....	160
13. Bits per Color Sample Adapter IP.....	162
13.1. About the Bits per Color Sample Adapter IP.....	162
13.1.1. Bits per Color Sample Adapter IP Performance and Resources.....	163
13.2. Bits per Color Sample Adapter IP Parameters.....	163
13.3. Bits per Color Sample Adapter IP Dithering Functional Description.....	165
13.3.1. Bits per Color Sample Adapter IP Interfaces.....	169
13.4. Bits per Color Sample Adapter IP Registers.....	170
13.5. Bits per Color Sample Adapter Software API.....	175
14. Black Level Correction IP.....	184
14.1. About the Black Level Correction IP.....	184
14.1.1. Black Level Correction IP Performance and Resources.....	184
14.2. Black Level Correction IP Parameters.....	185
14.3. Black Level Correction IP Functional Description.....	186
14.3.1. Black Level Correction IP Interfaces.....	188
14.4. Black Level Correction IP Registers.....	189
14.5. Black Level Correction IP Software API.....	193

15. Black Level Statistics IP.....	210
15.1. About the Black Level Statistics IP.....	210
15.1.1. Black Level Statistics IP Performance and Resources.....	210
15.2. Black Level Statistics IP Parameters.....	211
15.3. Black Level Statistics IP Interfaces.....	212
15.4. Black Level Statistics IP Registers.....	213
15.5. Black Level Statistics IP Software API.....	216
16. Chroma Key IP.....	231
16.1. About the Chroma Key IP.....	231
16.1.1. Chroma Key IP Performance and Resources.....	231
16.2. Chroma Key IP Parameters.....	232
16.3. Chroma Key IP Interfaces.....	233
16.4. Chroma Key Replacement.....	234
16.5. Chroma Key IP Registers.....	235
16.6. Chroma Key IP Software API.....	242
17. Chroma Resampler IP.....	253
17.1. About the Chroma Resampler IP.....	253
17.1.1. Chroma Resampler Performance and Resources.....	253
17.2. Chroma Resampler IP Parameters.....	253
17.3. Chroma Resampler IP Functional Description.....	255
17.3.1. Chroma Resampler IP Interfaces.....	260
17.4. Chroma Resampler IP Registers.....	261
17.5. Chroma Resampler IP Software API.....	265
18. Clipper IP.....	271
18.1. About the Clipper IP.....	271
18.1.1. Clipper IP Performance and Resources.....	271
18.2. Clipper IP Parameters.....	271
18.3. Clipper IP Interfaces.....	273
18.4. Clipper IP Registers.....	275
18.5. Clipper IP Software API.....	279
19. Clocked Video Input IP.....	291
19.1. About the Clocked Video Input IP.....	291
19.1.1. Clocked Video Input IP Features.....	294
19.1.2. Clocked Video Input IP Performance and Resources.....	294
19.2. Initializing the Clocked Video Input IP.....	295
19.3. Clocked Video Input IP Parameters.....	295
19.4. Clocked Video Input IP Interfaces.....	296
19.5. Clocked Video Input IP Registers.....	298
19.6. Clocked Video Input IP Software API.....	305
20. Clocked Video to Full-Raster Converter IP.....	330
20.1. About the Clocked Video to Full-Raster Converter IP.....	330
20.1.1. Clocked Video to Full-Raster Converter Features.....	330
20.1.2. Clocked Video to Full-Raster Converter Performance and Resources.....	330
20.2. Clocked Video to Full-Raster Converter Parameters.....	331
20.3. Clocked Video to Full-Raster Converter Block Description.....	335
20.3.1. Clocked Video to Full-Raster Converter Interfaces.....	336

20.4. Clocked Video Converter to Full-Raster IP Registers.....	340
21. Clocked Video Output IP.....	342
21.1. About the Clocked Video Output IP.....	342
21.1.1. Clocked Video Output IP Performance and Resources.....	343
21.1.2. Clocked Video Output IP Features.....	343
21.2. Clocked Video Output IP Parameters.....	343
21.3. Clocked Video Output IP Functional Description.....	347
21.4. Clocked Video Output IP Interfaces.....	348
21.5. Clocked Video Output IP Registers.....	352
21.6. Clocked Video Output IP Software API.....	359
22. Color Plane Manager IP.....	375
22.1. About the Color Plane Manager IP.....	375
22.1.1. Color Plane Manager IP Performance and Resources.....	375
22.2. Color Plane Manager IP Parameters.....	376
22.3. Color Plane Manager IP Functional Description.....	381
22.3.1. Color Plane Manager IP Interfaces.....	387
22.4. Color Plane Manager IP Registers.....	388
22.5. Color Plane Manager Software API.....	394
23. Color Space Converter IP.....	401
23.1. About the Color Space Converter IP.....	401
23.1.1. Color Space Converter IP Features.....	401
23.1.2. Color Space Converter IP Performance and Resources.....	401
23.2. Color Space Converter IP Parameters.....	402
23.3. Color Space Converter IP Functional Description.....	404
23.3.1. Color Space Converter IP Interfaces.....	407
23.4. Color Space Converter IP Registers.....	408
23.5. Color Space Converter IP Software API.....	414
24. Defective Pixel Correction IP.....	425
24.1. About the Defective Pixel Correction IP.....	425
24.1.1. Defective Pixel Correction IP Performance and Resources.....	425
24.2. Defective Pixel Correction IP Parameters.....	426
24.3. Defective Pixel Correction IP Functional Description.....	427
24.3.1. Defective Pixel Correction IP Interfaces.....	429
24.4. Defective Pixel Correction IP Registers.....	430
24.5. Defective Pixel Correction IP Software API.....	432
25. Deinterlacer IP.....	445
25.1. About the Deinterlacer IP.....	445
25.1.1. Deinterlacer Performance and Resources.....	445
25.2. Deinterlacer Parameters.....	446
25.3. Deinterlacer IP Functional Description.....	449
25.3.1. Deinterlacer Interfaces.....	451
25.4. Deinterlacer IP Registers.....	453
25.5. Deinterlacer IP Software API.....	458
26. Demosaic IP.....	467
26.1. About the Demosaic IP.....	467
26.1.1. Demosaic IP Performance and Resources.....	467

26.2. Demosaic IP Parameters.....	468
26.3. Demosaic IP Functional Description.....	469
26.3.1. Demosaic IP Interfaces.....	471
26.4. Demosaic IP Registers.....	472
26.5. Demosaic IP Software API.....	474
27. FIR Filter IP.....	482
27.1. About the FIR Filter.....	482
27.1.1. FIR Filter IP Performance and Resources.....	482
27.2. FIR Filter Parameters.....	482
27.3. FIR Filter IP Operation.....	484
27.3.1. FIR Filter Processing.....	484
27.3.2. FIR Filter Precision.....	485
27.3.3. FIR Coefficient Specification.....	485
27.3.4. FIR Filter Symmetry.....	487
27.3.5. Result to Output Data Type Conversion.....	489
27.4. FIR Filter Interfaces.....	490
27.5. FIR Filter Registers.....	492
27.6. FIR Filter IP Software API.....	498
28. Frame Cleaner IP.....	508
28.1. About the Frame Cleaner IP.....	508
28.1.1. Frame Cleaner IP Performance and Resources.....	508
28.2. Frame Cleaner IP Parameters.....	508
28.3. Frame Cleaner IP Functional Description.....	509
28.4. Frame Cleaner IP Interfaces.....	511
28.5. Frame Cleaner IP Registers.....	512
28.6. Frame Cleaner IP Software API.....	519
29. Full-Raster to Clocked Video Converter IP	527
29.1. About the Full-Raster to Clocked Video Converter	527
29.1.1. Full-Raster to Clocked Video Converter Features.....	527
29.1.2. Full-Raster to Clocked Video Converter Performance and Resource Utilization.....	527
29.2. Full Raster to Clocked Video Converter Parameters.....	528
29.3. Full-Raster to Clocked Video Converter Block Description.....	531
29.3.1. Full Raster to Clocked Video Converter Interfaces.....	532
29.4. Full-Raster to Clocked Video Converter Registers.....	535
30. Full-Raster to Streaming Converter IP.....	538
30.1. About the Full-Raster to Streaming Converter IP.....	538
30.1.1. Full-Raster to Streaming Converter Features.....	538
30.1.2. Full-Raster to Streaming Converter Performance and Resources.....	538
30.2. Full-Raster to Streaming Converter Parameters.....	539
30.3. Full-Raster to Streaming Converter Block Description.....	540
30.3.1. Full-Raster to Streaming Converter Interfaces.....	541
31. Genlock Controller IP.....	543
31.1. About the Genlock Controller IP.....	543
31.1.1. Genlock Controller IP Performance and Resources.....	544
31.2. Genlock Controller IP Parameters.....	544
31.3. Genlock Controller IP Functional Description.....	546

31.3.1. Genlock Controller IP Interfaces.....	548
31.4. Using Genlock Controller IP Software.....	549
31.4.1. Achieving Genlock Controller Free Running (for Initialization or from Lock to Reference Clock N).....	549
31.4.2. Locking to Reference Clock N (from Genlock Controller IP free running).....	550
31.4.3. Setting the VCXO hold over.....	551
31.4.4. Restarting the Genlock Controller IP.....	551
31.4.5. Locking to Reference Clock N New (from Locking to Reference Clock N Old).....	551
31.4.6. Changing to Reference Clock or VCXO Base Frequencies (switch between p50 and p59.94 video formats and vice-versa).....	551
31.4.7. Disturbing a Reference Clock (a cable pull).....	552
31.5. Genlock Controller IP Registers.....	552
31.6. Genlock Controller IP Software API.....	559
32. Generic Crosspoint IP.....	584
32.1. About the Generic Crosspoint IP.....	584
32.1.1. Generic Crosspoint IP Features.....	585
32.1.2. Generic Crosspoint Performance and Resources.....	585
32.2. Generic Crosspoint IP Parameters.....	585
32.3. Generic Crosspoint IP Interfaces.....	586
32.4. Generic Crosspoint IP Registers.....	587
32.5. Generic Crosspoint IP Software API.....	588
33. Genlock Signal Router IP.....	591
33.1. About the Genlock Signal Router IP.....	591
33.1.1. Genlock Signal Router IP Features.....	592
33.1.2. Genlock Signal Router IP Performance and Resources.....	592
33.2. Genlock Signal Router IP Parameters.....	592
33.3. Genlock Signal Router IP Interfaces.....	594
33.4. Genlock Signal Router IP Registers.....	595
33.5. Genlock Signal Router IP Software API.....	598
34. Guard Bands IP.....	603
34.1. About the Guard Bands IP.....	603
34.1.1. Guard Bands IP Performance and Resources.....	603
34.2. Guard Bands IP Parameters.....	603
34.3. Guard Bands IP Interfaces.....	604
34.4. Guard Bands IP Registers.....	605
34.5. Guard Bands IP Software API.....	610
35. Histogram Statistics IP.....	616
35.1. About the Histogram Statistics IP.....	616
35.1.1. Histogram Statistics IP Performance and Resources.....	616
35.2. Histogram Statistics IP Parameters.....	617
35.3. Histogram Statistics IP Interfaces.....	619
35.4. Histogram Statistics IP Registers.....	620
35.5. Histogram Statistics IP Software API.....	623
36. Interlacer IP.....	637
36.1. About the Interlacer IP.....	637
36.1.1. Interlacer IP Performance and Resources.....	637
36.2. Interlacer IP Parameters.....	637

36.3. Interlacer IP Functional Description.....	638
36.3.1. Interlacer IP Interfaces.....	640
36.4. Interlacer IP Registers.....	641
36.5. Interlacer IP Software API.....	645
37. Mixer IP.....	652
37.1. About the Mixer IP.....	652
37.1.1. Mixer IP Performance and Resources.....	652
37.2. Mixer IP Parameters.....	652
37.3. Mixer IP Functional Description.....	654
37.3.1. Mixer IP Interfaces.....	659
37.4. Mixer IP Registers.....	661
37.5. Mixer IP Software API.....	666
38. Pixels in Parallel Converter IP.....	679
38.1. About the Pixels in Parallel Converter IP.....	679
38.1.1. Pixels in Parallel Converter IP Performance and Resources.....	679
38.2. Pixels in Parallel Converter IP Parameters.....	680
38.3. Pixels in Parallel Converter Interfaces.....	680
38.4. Pixels in Parallel Converter Registers.....	682
38.5. Pixels in Parallel Converter IP Software API.....	683
39. Scaler IP.....	687
39.1. About the Scaler.....	687
39.1.1. Scaler IP Performance and Resources.....	687
39.2. Scaler IP Parameters.....	688
39.2.1. Scaler Maximum Resolution.....	690
39.3. Scaler IP Functional Description.....	690
39.3.1. Coefficient Selection.....	693
39.3.2. Coefficient Quantization.....	695
39.3.3. Filter Behavior at Edge Boundaries.....	696
39.3.4. Partial Frame Scaling.....	697
39.3.5. 422 and 420 Chroma Sampled Data Scaling.....	698
39.3.6. Scaler IP Interfaces.....	699
39.4. Scaler IP Registers.....	700
39.5. Scaler IP Software API.....	712
40. Stream Cleaner IP.....	740
40.1. About the Stream Cleaner IP.....	740
40.1.1. Stream Cleaner IP Performance and Resources.....	740
40.2. Stream Cleaner IP Parameters.....	740
40.3. Stream Cleaner IP Functional Description.....	741
40.3.1. Stream Cleaner IP Interfaces.....	743
41. Switch IP.....	745
41.1. About the Switch IP.....	745
41.1.1. Switch IP Performance and Resources.....	745
41.2. Switch IP Parameters.....	746
41.3. Switch IP Functional Description.....	747
41.3.1. Switch IP Latency.....	749
41.3.2. Switch IP Interfaces.....	750
41.4. Switch IP Registers.....	752

41.5. Switch IP Software API.....	755
42. Text Box IP.....	763
42.1. About the Text Box IP.....	763
42.1.1. Text Box IP Performance and Resources.....	763
42.2. Text Box IP Parameters.....	763
42.3. Text Box IP Functional Description.....	766
42.3.1. Writing Text at Run Time with Double Buffering.....	768
42.3.2. Writing Text at Run Time without Double Buffering.....	769
42.3.3. Text Box IP Interfaces.....	769
42.4. Text Box IP Registers.....	770
42.5. Text Box IP Software API.....	775
43. Tone Mapping Operator IP.....	791
43.1. About the Tone Mapping Operator IP.....	791
43.1.1. TMO IP Features.....	792
43.1.2. TMO IP Performance and Resource Utilization.....	793
43.2. TMO IP Parameters.....	794
43.3. TMO IP Block Description.....	795
43.3.1. TMO IP Interfaces.....	798
43.3.2. TMO IP Latency.....	803
43.4. TMO IP Registers.....	803
43.5. TMO IP Software API.....	805
44. Test Pattern Generator IP.....	817
44.1. About the Test Pattern Generator IP	817
44.1.1. Test Pattern Generator IP Performance and Resources.....	817
44.2. Test Pattern Generator IP Parameters.....	818
44.3. Test Pattern Generator IP Functional Description.....	820
44.3.1. Test Patterns.....	821
44.3.2. Output Subsampling and Color Space.....	825
44.3.3. Run-time Settings.....	826
44.3.4. Test Pattern Generator IP Interfaces.....	827
44.4. Test Pattern Generator IP Registers.....	828
44.5. Test Pattern Generator IP Software API.....	836
45. Unsharp Mask IP.....	854
45.1. About the Unsharp Mask IP.....	854
45.1.1. Unsharp Mask IP Performance and Resources.....	854
45.2. Unsharp Mask IP Parameters.....	855
45.3. Unsharp Mask IP Functional Description.....	856
45.3.1. Unsharp Mask IP Interfaces.....	858
45.4. Unsharp Mask IP Registers.....	859
45.5. Unsharp Mask IP Software API.....	860
46. Video and Vision Monitor IP.....	866
46.1. About the Video and Vision Monitor IP.....	866
46.1.1. Video and Vision Monitor IP Performance and Resources.....	866
46.2. Video and Vision Monitor IP Parameters.....	866
46.3. Video and Vision Monitor IP Functional Description.....	867
46.3.1. Video and Vision Monitor IP Interfaces.....	868
46.4. Video and Vision Monitor IP Registers.....	869

46.5. Video and Vision Monitor Software API.....	873
47. Video Frame Buffer IP.....	879
47.1. About the Video Frame Buffer IP.....	879
47.1.1. Video Frame Buffer Performance and Resources.....	879
47.2. Video Frame Buffer IP Parameters.....	880
47.3. Video Frame Buffer IP Functional Description.....	883
47.3.1. Video Frame Buffer IP Interfaces.....	886
47.4. Video Frame Buffer IP Registers.....	888
47.5. Video Frame Buffer IP Software API.....	894
48. Video Frame Reader IP.....	905
48.1. About the Video Frame Reader IP.....	905
48.1.1. Video Frame Reader IP Performance and Resources.....	905
48.2. Video Frame Reader IP Parameters.....	906
48.3. Video Frame Reader IP Functional Description.....	907
48.3.1. Video Frame Reader IP Interfaces.....	911
48.4. Video Frame Reader IP Registers.....	912
48.5. Video Frame Reader IP Software API.....	919
49. Video Frame Writer IP.....	940
49.1. About the Video Buffer Writer IP.....	940
49.1.1. Video Frame Writer IP Performance and Resources.....	940
49.2. Video Frame Writer IP Parameters.....	941
49.3. Video Frame Writer IP Functional Description.....	943
49.3.1. Video Frame Writer IP Interfaces.....	945
49.4. Video Frame Writer IP Registers.....	946
49.5. Video Frame Writer IP Software API.....	954
50. Video Streaming FIFO IP.....	967
50.1. About the Video Streaming FIFO IP.....	967
50.1.1. Video Streaming FIFO IP Performance and Resources.....	967
50.2. Video Streaming FIFO IP Parameters.....	967
50.3. Video Streaming FIFO IP Interfaces.....	968
51. Video Timing Generator IP.....	970
51.1. About the Video Timing Generator IP.....	970
51.1.1. Video Timing Generator IP Features.....	972
51.1.2. Video Timing Generator IP Performance and Resources.....	972
51.2. Video Timing Generator IP Parameters.....	973
51.3. Video Timing Generator IP Functional Description.....	979
51.4. Video Timing Generator IP Interfaces.....	982
51.5. Video Timing Generator IP Registers.....	984
51.6. Video Timing Generator IP Software API.....	993
52. Vignette Correction IP.....	1013
52.1. About the Vignette Correction IP.....	1013
52.1.1. Vignette Correction IP Performance and Resources.....	1013
52.2. Vignette Correction IP Parameters.....	1014
52.3. Vignette Correction IP Interfaces.....	1016
52.4. Vignette Correction IP Registers.....	1017
52.5. Vignette Correction IP Software API.....	1024

53. Warp IP.....	1044
53.1. About the Warp IP.....	1044
53.1.1. Warp IP Features.....	1044
53.1.2. Warp IP Performance and Resource Utilization.....	1045
53.2. Warp IP Parameters.....	1048
53.3. Warp IP Block Description.....	1051
53.3.1. Block Cache Tool.....	1059
53.3.2. Warp IP Interfaces.....	1063
53.3.3. Warp IP Latency.....	1065
53.3.4. External Memory for Warp IP.....	1068
53.4. Warp IP Registers.....	1079
53.5. Warp IP Software API.....	1085
53.5.1. Using Warp IP Software.....	1116
53.5.2. Warp IP Software Code Examples.....	1116
54. White Balance Correction IP.....	1121
54.1. About the White Balance Correction IP.....	1121
54.1.1. White Balance Correction IP Performance and Resources.....	1121
54.2. White Balance Correction IP Parameters.....	1122
54.3. White Balance Correction IP Functional Description.....	1123
54.3.1. White Balance Correction IP Interfaces.....	1125
54.4. White Balance Correction IP Registers.....	1126
54.5. White Balance Correction IP Software API.....	1128
55. White Balance Statistics IP.....	1141
55.1. About the White Balance Statistics IP.....	1141
55.1.1. White Balance Statistics IP Performance and Resources.....	1141
55.2. White Balance Statistics IP Parameters.....	1142
55.3. White Balance Statistics IP Interfaces.....	1144
55.4. White Balance Statistics IP Registers.....	1145
55.5. White Balance Statistics IP Software API.....	1149
56. Design Security.....	1171
56.1. Memory Subsystems.....	1171
56.2. Considering Design Security.....	1171
57. Document Revision History for Video and Vision Processing Suite User Guide.....	1173



1. About the Video and Vision Processing Suite

The Video and Vision Processing Suite is the next-generation suite of IPs for video, image and vision processing. The IPs transport video using the Altera® Streaming Video Protocol, which uses the industry-standard AXI4-Stream protocol. A protocol converter IP allows interoperability with the Avalon Streaming video standard and the existing video and image processing IP or other IPs compliant with the Avalon streaming video protocol.

The IPs support Quartus® Prime Pro Edition only.

Related Information

- [About the Protocol Converter IP](#) on page 46
- [About the 1D LUT IP](#) on page 68
- [About the 3D LUT IP](#) on page 85
- [About the Adaptive Noise Reduction IP](#) on page 99
- [About the Advanced Test Pattern Generator IP](#) on page 117
- [About the AXI-Stream Broadcaster IP](#) on page 158
- [About the Bits per Color Sample Adapter IP](#) on page 162
- [About the Black Level Correction IP](#) on page 184
- [About the Black Level Statistics IP](#) on page 210
- [About the Chroma Key IP](#) on page 231
- [About the Chroma Resampler IP](#) on page 253
- [About the Clipper IP](#) on page 271
- [About the Clocked Video Input IP](#) on page 291
- [About the Clocked Video Output IP](#) on page 342
- [About the Clocked Video to Full-Raster Converter IP](#) on page 330
- [About the Color Space Converter IP](#) on page 401
- [About the Color Plane Manager IP](#) on page 375
- [About the Defective Pixel Correction IP](#) on page 425
- [About the Deinterlacer IP](#) on page 445
- [About the Demosaic IP](#) on page 467
- [About the FIR Filter](#) on page 482
- [About the Frame Cleaner IP](#) on page 508
- [About the Full-Raster to Clocked Video Converter](#) on page 527
- [About the Full-Raster to Streaming Converter IP](#) on page 538

- [About the Genlock Controller IP](#) on page 543
- [About the Generic Crosspoint IP](#) on page 584
- [About the Genlock Signal Router IP](#) on page 591
- [About the Guard Bands IP](#) on page 603
- [About the Histogram Statistics IP](#) on page 616
- [About the Interlacer IP](#) on page 637
- [About the Mixer IP](#) on page 652
- [About the Pixels in Parallel Converter IP](#) on page 679
- [About the Scaler](#) on page 687
- [About the Stream Cleaner IP](#) on page 740
- [About the Switch IP](#) on page 745
- [About the Text Box IP](#) on page 763
- [About the Tone Mapping Operator IP](#) on page 791
- [About the Test Pattern Generator IP](#) on page 817
- [About the Unsharp Mask IP](#) on page 854
- [About the Video and Vision Monitor IP](#) on page 866
- [About the Video Frame Buffer IP](#) on page 879
- [About the Video Frame Reader IP](#) on page 905
- [About the Video Buffer Writer IP](#) on page 940
- [About the Video Streaming FIFO IP](#) on page 967
- [About the Video Timing Generator IP](#) on page 970
- [About the Vignette Correction IP](#) on page 1013
- [About the Warp IP](#) on page 1044
- [About the White Balance Correction IP](#) on page 1121
- [About the White Balance Statistics IP](#) on page 1141
- [Altera Streaming Video Protocol Specification](#)
- [AMBA 4 AXI4-Stream Protocol](#)

1.1. Video and Vision Processing IPs Features

- Altera Streaming Video data interfaces for video I/O, based on the industry-standard AXI4-Stream protocol
- Avalon memory-mapped agent interfaces for run-time control and Avalon memory-mapped host interface for external memory usage allowing push-button conversion in Platform Designer to industry-standard AXI4 or AXI4-Lite memory-mapped interfaces if required.
- Device utilization and Fmax on Agilex™ 7 FPGAs allow 8K60 processing on Agilex 7 platforms with four pixels in parallel at 600 MHz.
- Processing of 1 to 8 pixels in parallel

- Support for 1 to 4 color symbols per pixel and RGB and YCbCr 444, 422 and 420 color spaces.
- Data precision from 8 to 16 bits per symbol.
- Video fields with 1 to 16384 pixels in both height and width.

1.2. Device Family Support

The device support is the same for all video and vision processing IPs.

Altera offers the following device support levels for Altera FPGA IP:

- **Advance support**—the IP is available for simulation and compilation for this device family. FPGA programming file (.pof) support is not available for Quartus Prime Pro Edition software and as such IP timing closure cannot be guaranteed. Timing models include initial engineering estimates of delays based on early post-layout information. The timing models are subject to change as silicon testing improves the correlation between the actual silicon and the timing models. You can use this IP for system architecture and resource utilization studies, simulation, pinout, system latency assessments, basic timing assessments (pipeline budgeting), and I/O transfer strategy (data-path width, burst depth, I/O standards tradeoffs).
- **Preliminary support**—Altera verifies the IP with preliminary timing models for this device family. The IP core meets all functional requirements, but might still be undergoing timing analysis for the device family. You can use it in production designs with caution.
- **Final support**—Altera verifies the IP with final timing models for this device family. The IP meets all functional and timing requirements for the device family. You can use it in production designs.

Table 1. Video and Vision Processing IP Device Family Support

Device Family	Support
Agilex 3	Advance
Agilex 5 E-series	Advance
Agilex 7	Final
Arria® 10	Final
Cyclone® 10	Final
Stratix® 10	Final

Related Information

[Timing Model, Power Model, and Device Status](#)

1.3. Video and Vision Processing IPs Release Information

Altera IP versions match the Quartus Prime Design Suite software versions until v19.1. Starting in Quartus Prime Design Suite software version 19.2, the IP has a new versioning scheme.

The IP version (X.Y.Z) number can change with each Quartus Prime software version. A change in:

- X indicates a major revision of the IP. If you update the Quartus Prime software, you must regenerate the IP.
- Y indicates the IP includes new features. Regenerate your IP to include these new features.
- Z indicates the IP includes minor changes. Regenerate your IP to include these changes.

1.4. Lite versus Full IP Variants

Most video and vision processing IPs support both full and lite variants of the Altera Streaming Video Protocol. You should choose which protocol variant (and therefore which parameterization of the IPs) is most suitable for your end application. Some of the IPs only use the lite variant of the protocol. Therefore, they do not pass control information along the streaming interface.

For the full variants, the IPs transmit some control information together with video data over the streaming interfaces. For the lite variants, write all the control information to IPs using a processor interface.

You may still select run-time control via a processor interface when using the full variant of IPs (for example, to update clipper offsets or mixer settings). The control information about the size and type of incoming video is always handled automatically using streamed control packets.

You can turn on or off **lite mode** in the GUI for most of the Altera video and vision processing IPs. When off, the IP is a full variant. You can interface between full and lite variants of IPs using the protocol converter IP. However, system design is easier if a video pipeline standardizes on one version of the protocol.

When choosing between full and lite consider the following advantages:

For full IP variants:

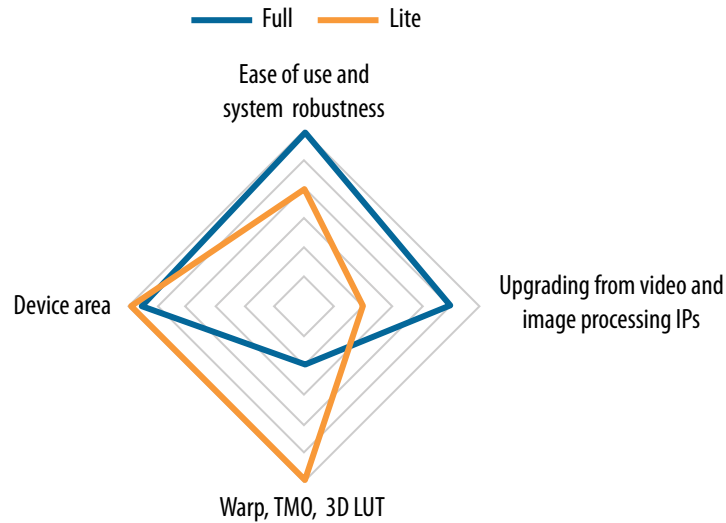
- Ease of use. Full variants of the IPs handle more of the system control aspects automatically.
- Video and Image Processing Suite interoperability or upgradeability. Interfacing or modifying existing systems based around the Video and Image Processing Suite IPs to the full variants is easier, as these handle video in a similar way.

For lite IP variants:

- Building a system primarily using one of the Warp, 3D LUT or TMO IPs or a system requiring interoperability with third-party AXI-Stream video steaming IP. The Warp, 3D LUT, and TMO IPs use the lite variant of the protocol and do not pass control along the streaming interface.
- Device area. Expect to save 100-200 ALMs per IP for lite variants.

Figure 1. Full and lite benefits radar diagram

For example, lite variants are better for device area (lower area).



Regenerate IPs with either version of the protocol and compare overall system f_{MAX} and area before deciding.

1.5. Glossary of Video and Vision Terminology

Table 2. Glossary of Video and Vision Terminology

Agent	Part of a memory-mapped interface, which receives transactions from a host interface.
Auxiliary control packets	A term for any Altera Streaming Video Protocol control packet with ID>=2. Optionally available to use for a variety of purposes, such as timestamping, register updates, IP synchronization and non-video data such as closed-captioning.
Avalon streaming interface	Altera streaming standard, similar to AXI4-Stream. Used by VIP IPs.
Avalon memory-mapped interface	Altera memory-mapped interface standard, similar to AXI4-Lite.
Field	Either a progressive frame of video, or a field of interlaced video.
Frame	A progressive frame of video.
Full variant	The variant of the IPs when you turn off Lite mode of the Altera Streaming Video Protocol and which comprises both control and data packets.
Full-raster variant	The variant of the IP of the Altera Streaming Video Protocol that comprises both blanking and active data packets.
Host	Part of a memory-mapped interface, which sends transactions to one or more agent interfaces.
Interlaced	Interlaced video comprises alternating fields with either the odd or even lines comprising a video frame. A 1080i video comprises one field of 540 odd-numbered lines followed by one field of 540 even-numbered lines.
Lite variant	The variant of the IPs when you turn on lite mode for the Altera Streaming Video Protocol and which comprises only data packets.
Metapackets	Either an image information packet, an end-of-field packet, or an auxiliary control packet.
continued...	

Progressive	Video that is not interlaced. Lines are transmitted in order.
Symbol	Pixels are comprised of 1 to 4 symbols. A symbol comprises between 8 and 16 bits of data, representing one color from an RGB triplet (red, blue, green), or a luma or chroma sample from YCbCr pixels, or some other color space component.
Undefined behavior	If an IP experiences a stimulus that falls outside of its design parameters, its behavior is unspecified. The IP may fail gracefully, or lock-up, or continue working. Altera does not specify the outcome.
VIP	Video and image processing. Altera's previous generation of video processing IPs from the Video and Image Processing Suite.



2. Getting Started with the Video and Vision Processing IPs

2.1. Video and Vision Processing IP Evaluation Time-out Behavior

The free Altera FPGA IP Evaluation Mode allows you to evaluate Altera FPGA IPs in simulation and hardware before purchasing a license. For IP generated in this mode, the IP operates indefinitely while tethered (e.g., with a download cable). The untethered timeout is about one hour, after which the IP stops functioning.

You must purchase the license and generate a full production license key before you can generate an unrestricted device programming file. During Altera FPGA IP Evaluation Mode, the IP only generates a time-limited device programming file (`<projectname>_time_limited.sof`) that expires at the time limit.

All IPs in a device time out simultaneously when the most restrictive evaluation time is reached. If a design has more than one IP, the time-out behavior of the other IP may mask the time-out behavior of a specific IP.

When the evaluation time expires, the video and vision processing IP stops working.

Related Information

[AN 320: Using the Altera FPGA IP Evaluation Mode](#)

2.2. Generating a Video and Vision Processing IP

To include the IP in a design, generate the IP in Platform Designer.

1. Create a New Quartus Prime project
2. Open Platform Designer and create a project.
The video and vision processing IPs are only available in Platform Designer.
3. Select **DSP > Video and Vision Processing > <IP name>IP** and click **Add**
4. Enter a name for your IP variant and click **Create**.

The name is for both the top-level RTL module and the corresponding `.ip` file.

The parameter editor for this IP appears.

5. Choose your parameters.
6. Click **Generate HDL**.

Quartus Prime generates the RTL and the files necessary to instantiate the IP in your design and synthesize it.

Related Information

- [Protocol Converter IP Parameters](#) on page 47

- [1D LUT IP Parameters](#) on page 69
- [3D LUT IP Parameters](#) on page 86
- [Adaptive Noise Reduction IP Parameters](#) on page 100
- [Advanced Test Pattern Generator IP Parameters](#) on page 118
- [AXI-Stream Broadcaster IP Parameters](#) on page 159
- [Bits per Color Sample Adapter IP Parameters](#) on page 163
- [Black Level Correction IP Parameters](#) on page 185
- [Black Level Statistics IP Parameters](#) on page 211
- [Chroma Key IP Parameters](#) on page 232
- [Chroma Resampler IP Parameters](#) on page 253
- [Clipper IP Parameters](#) on page 271
- [Clocked Video Input IP Parameters](#) on page 295
- [Clocked Video to Full-Raster Converter Parameters](#) on page 331
- [Clocked Video Output IP Parameters](#) on page 343
- [Color Plane Manager IP Parameters](#) on page 376
- [Color Space Converter IP Parameters](#) on page 402
- [Defective Pixel Correction IP Parameters](#) on page 426
- [Deinterlacer Parameters](#) on page 446
- [Demosaic IP Parameters](#) on page 468
- [FIR Filter Parameters](#) on page 482
- [Frame Cleaner IP Parameters](#) on page 508
- [Full Raster to Clocked Video Converter Parameters](#) on page 528
- [Full-Raster to Streaming Converter Parameters](#) on page 539
- [Generic Crosspoint IP Parameters](#) on page 585
- [Genlock Controller IP Parameters](#) on page 544
- [Genlock Signal Router IP Parameters](#) on page 592
- [Guard Bands IP Parameters](#) on page 603
- [Histogram Statistics IP Parameters](#) on page 617
- [Interlacer IP Parameters](#) on page 637
- [Mixer IP Parameters](#) on page 652
- [Pixels in Parallel Converter IP Parameters](#) on page 680
- [Scaler IP Parameters](#) on page 688
- [Test Pattern Generator IP Parameters](#) on page 818
- [TMO IP Parameters](#) on page 794
- [Unsharp Mask IP Parameters](#) on page 855
- [Video and Vision Monitor IP Parameters](#) on page 866
- [Video Frame Buffer IP Parameters](#) on page 880
- [Video Frame Reader IP Parameters](#) on page 906

- [Video Streaming FIFO IP Parameters](#) on page 967
- [Video Timing Generator IP Parameters](#) on page 973
- [Vignette Correction IP Parameters](#) on page 1014
- [Warp IP Parameters](#) on page 1048
- [White Balance Correction IP Parameters](#) on page 1122
- [White Balance Statistics IP Parameters](#) on page 1142

2.3. Simulating the Video and Vision Processing IPs

You can simulate IPs in any of the supported simulators listed in the Quartus Prime Pro Edition user guide.

Related Information

[Simulating IP Cores](#)



3. Video and Vision Processing IPs Functional Description

Video and vision processing IPs conform to the Altera Streaming Video Protocol.

Reset Behavior

IPs employ a synchronous reset and system resets must have a minimum duration of 256 clock cycles. In accordance with the AXI specification, all `TVALID` and `TREADY` signals from components drive low during reset and for at least one cycle after you deassert reset.

TUSER usage

The protocol specifies a `TUSER` width of `TDATA/8` where `TDATA` is at least 16 bits and is always divisible by 8. The 2 LSBs of `TUSER` indicate whether a packet is control (full variants only) or data (bit 1) and indicate the start of a new field of video (bit 0). Altera video and vision processing IPs do not drive any unused bits (bit 2 upwards). Quartus Prime optimizes them away during synthesis. The IPs ignore and do not propagate any data you drive on bits 2 and upwards of `TUSER`.

TVALID and TREADY usage

The protocol specifies that an input interface can wait for `TVALID` to be asserted before asserting the corresponding `TREADY`. However, Altera video and vision processing IP sinks assert `TREADY` independently of whether the input `TVALID` is asserted. If a third-party IP drives a video and vision processing IP sink and it does not respect this AXI rule for sources, the video pipe still operates correctly.

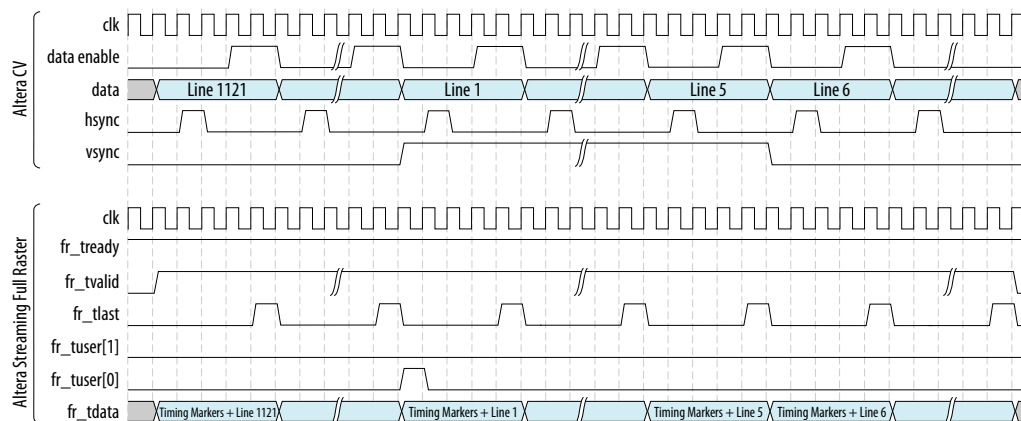
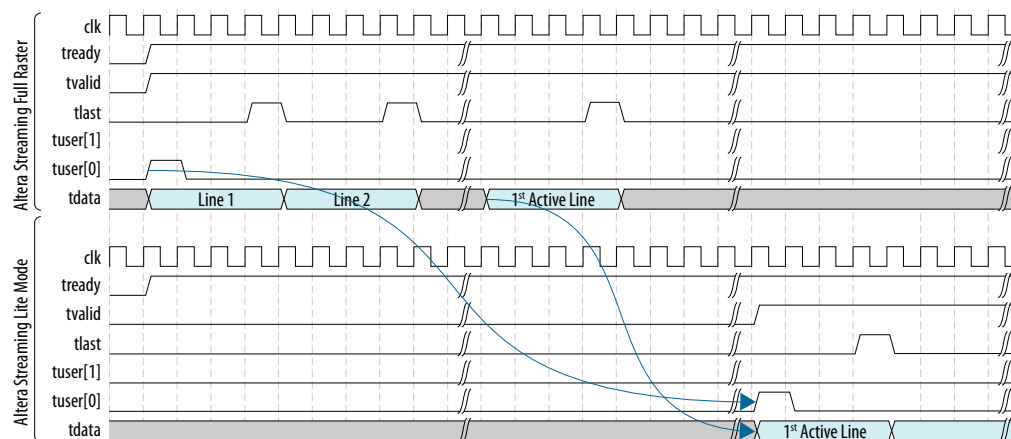
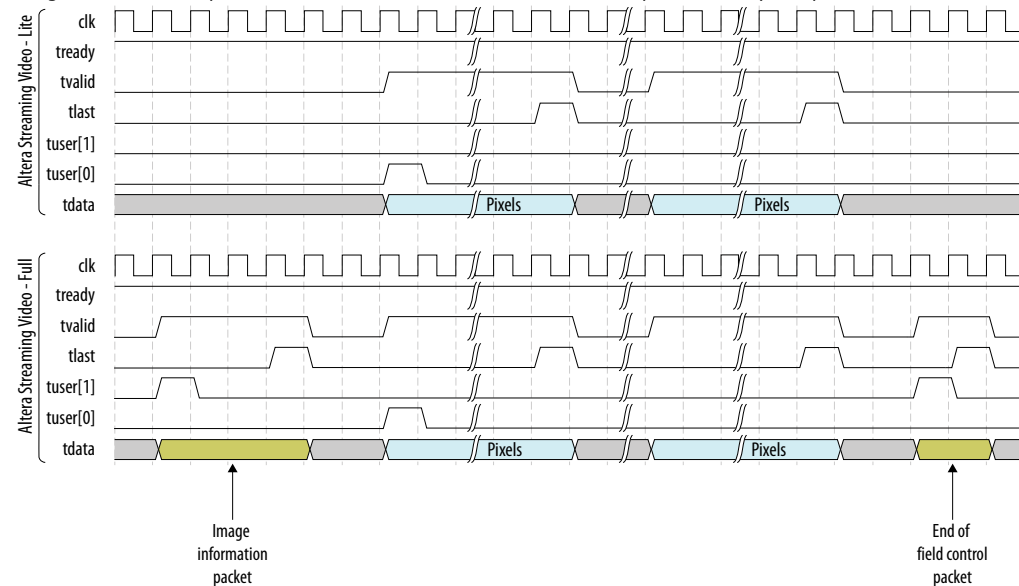
Figure 3. Conversion of Altera clocked Video to full-raster video data.

Figure 4. Conversion of full-raster to streaming video data.


Figure 5. Conversion of lite to full Altera Streaming Video data.

The figure shows the optional conversion from lite to full variants as performed by the protocol converter.



The Altera Streaming Video Protocol states that IPs transmit video fields in packets of pixel data. One packet carries each line of video, with the start of field indicated by `tuser[0]`. The protocol converter supplements the pixel data packets with image information packets and end of field packets. The IP gains image information packet information from the protocol converter's control registers

Most IPs update their behavior after the end of the current field, switching to any new control settings if required. Full variant IPs detect the end of the current field by the presence of the end of field packet. Lite variants IPs, without the benefit of metapackets, need to count the number of lines and compare this count with the value in the `IMG_INFO_HEIGHT` register. Alternatively, the IPs wait until they detect `tuser[0]`, marking the start of the next field.

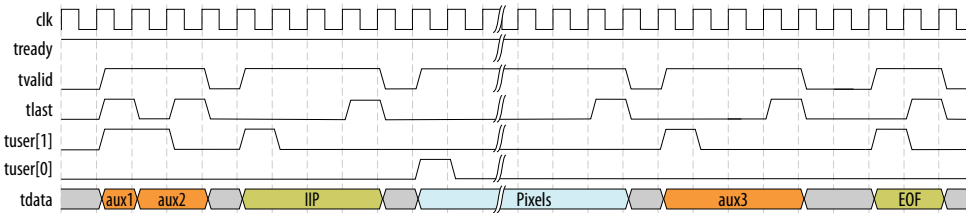
3.1. Auxiliary Control Packets

Full variant video and vision processing IPs may transmit other types of auxiliary control packets, such as timestamp packets to synchronize with audio or other types of auxiliary control packets. You can define auxiliary control packets, which contain information such as active format description (AFD) or closed captioning.

IPs propagate auxiliary control packets with the video data packets. IPs such as the video frame buffer observe rules that determine how to handle auxiliary control packets.

Figure 6. Auxiliary control packet positions.

The figure shows an example where two auxiliary control packets precede an image information packet and a third precedes an end of frame packet.



Auxiliary control packets in between image information packets and video packets, or in between adjacent data packets contravene the Altera Streaming Video Protocol. IPs presented with metapackets in these locations exhibit undefined behavior and the system may break.

In the figure, the video field described by the image information packets, video and end of field packets has three auxiliary control packets. If the frame buffer receives these packets, the IP processes them in the same way as the video field. The IP potentially drops or repeats them.

Auxiliary control packets only occupy the low 16 bits of the TDATA bus.

For TDATA widths greater than 16 bits, to adapt to a different interface, do not use standard AXI4-Stream converters. Use the protocol converter IPs.

3.2. Latency

You can use the latency information of video and vision processing IPs to predict the approximate latency between the input and the output of your video processing pipeline.

The latency is described using one or more of the following measures:

- The number of progressive frames or interlaced fields
- The number of lines when less than a field of latency
- A number of cycles

For all latency metrics, assume that other functions on the datapath are not stalling the IP and the `tready` signal is high. If not stated for an IP, the approximate latency from the video data input to the video data output for typical usage modes of the IPs is of the order of a small number of cycles, for example between 1 and 20.

3.3. IP Debugging Features

If you turn on **Memory-mapped control interface** for a video and vision processing IP, you can also turn on **Debug features**.

For lite variants with **Debug features** on, you can read back any values you write to the control `IMG_INFO` registers.

Debugging features with full variants allow you to query details of received image information packets. Every time an IP receives an image information packet, the image information fields are stored in a debug `IMG_INFO` register bank. You can read back this information over the memory-mapped control interface.

In a system that is not behaving as expected the debugging features are a useful debugging tool. If required, when the system is working, the IP can be regenerated in Platform Designer with debugging features off so that the design does not add the area cost of the debugging register bank.

Table 3. Relative ALM consumption for full variant, lite variant, and debug features (Clipper IPs)

The table shows the relative ALM consumption of the clipper IP, with and without debugging features. The clipper parameters are 2 pixels in parallel, 8 bits per color sample, 2 color planes and clipping with offsets. For comparison, the table includes the lite variant Clipper and the VIP Clipper II IP. The table includes timing slack for a clocking target of 333 MHz. The designs use default settings, the Quartus Prime Pro Edition version 21.4, and target a Cyclone 10 GX 10CX220YF78015G device. The table shows debugging features slightly affect f_{MAX} .

IP	ALMs	Slack when targeting 333 MHz (ns)
Clipper IP (full)	493	0.481
Clipper IP (full with debug features)	616	0.494
Clipper IP (lite)	394	0.451
VIP Clipper II	670	0.359

Table 4. Relative timing slack for Clipper IPs

The table shows an artificially aggressive target frequency for the Clipper IP for an Cyclone 10 GX device of 525 MHz. Turning on **Debug features** reduces F_{max} to 518 MHz. For comparison with a full variant clipper, the table includes a lite variant Clipper and the VIP Clipper II IP. All figures using default settings with Quartus Prime Pro Edition release 21.4, targeting an Cyclone 10 GX 10CX220YF78015G device

IP	ALMs	Slack when targeting 525 MHz	
		ns	MHz
Clipper IP (full variant)	494	0.001	525
Clipper IP (full variant with debug features)	614	-0.028	518
Clipper IP (lite variant)	395	0.065	545
VIP Clipper II	672	-0.659	390

3.4. Stall Behavior and Error Recovery

The video and vision processing IPs do not continuously process data. They use flow-controlled streaming interfaces, which allow them to stall the data while they perform internal calculations.

During metapacket processing full variant IPs might stall frequently and read or write less than once per clock cycle. During data processing, the IPs generally process one input or output per clock cycle. The IPs have some stalling cycles. Typically, the stalling cycles are for internal calculations between data packets and between fields. When stalled, an IP drives `t_ready` low on its inputs to indicate that it is not ready to receive data. The time spent in the stalled state varies between IPs and their parameterizations. In general, it is a few cycles between data packets and a few more between frames.

If data is not available at the input when required, IPs stall and do not output data.

When IPs receive a `tlast` signal or `TUSER[0]` strobe unexpectedly (early or late), they recover from the error and prepare for the next valid packet (control or data).

Errors fall into one of three categories:

- Low-level violations of the underlying AXI4-S protocol.
- Violations of the Altera Streaming Video Protocol
- Higher-level violations of the Altera Streaming Video Protocol.

Low-level protocol violations (such as `TLAST` stuck at zero or a `TVALID` or `TREADY` handshake fault) give system failure and possibly lockup. The IPs have no mechanism to protect against these violations.

Altera Streaming Video Protocol violations include:

- IPs receiving malformed control packets, which results in undefined behavior and likely result in system lock-up
- Video packets with incorrect pixel-packing are processed by IPs successfully, but output data is likely to be incorrect or incorrectly packed.

Examples of high-level errors are:

- Early or late `TLAST` signaling at the end of packets on data packets, where a line is shorter or longer than expected
- Early or late `TUSER[0]` marking a start of frame, where a frame contains less or more lines than expected

You should expect these high-level errors sometimes in a running system. Therefore, IPs accept invalid video fields and they may also generate them without breaking the specification.

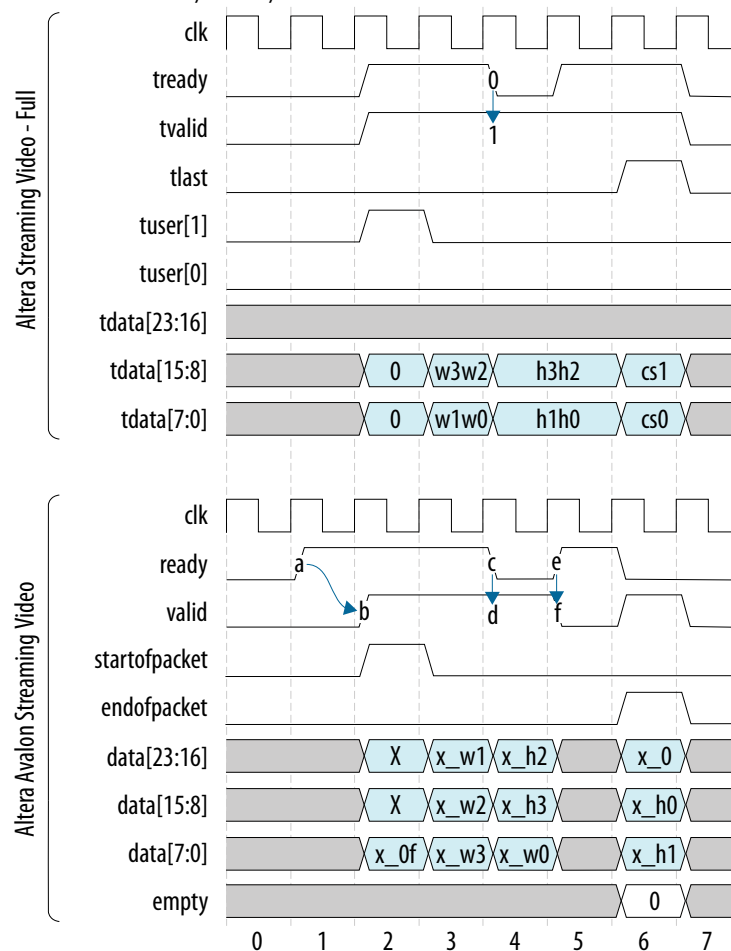
3.5. Avalon Streaming Video Protocol Versus Altera Streaming Video Protocol

Altera offers IPs to convert between the two protocols. This topic compares the differences between control and data packets from the Avalon Streaming Video protocol to metapackets and video packets from the Altera Streaming Video Protocol.

Control and Image Information Packets

Figure 7. Control Packets for Both Protocols

The Avalon streaming video protocol has a ready latency of 1. The figure shows a transition a to b where the first valid cycle of the packet occurs in cycle 2, one clock cycle after the sink raises its ready. The Altera Streaming Video Protocol has a ready latency of 0.



The Avalon streaming video protocol indicates control packets with the value `0xf` in the low nibble of the first beat of the transaction. The control packet payload is then packed over subsequent beats into the low nibbles of each byte, across the whole width of the data bus. The Altera Streaming Video Protocol indicates image information packets by setting `tuser[1]` and `0x0` in the low 5 bits of the first beat. The protocol uses the remaining 11 bits of the first beat and the low 16 bits of subsequent beats to pack the remainder of the image information control packet.

Avalon streaming video carries data for the width field of the control packets in 4 nibbles spread across multiple bytes and often over multiple beats on the bus (`w3:w0` in the figure). The Altera Streaming Video Protocol always contains the 16 width field bits in the second beat of the transaction (`w3:w0` in cycle 3 in the figure). Both protocols process the height field in these ways.

Interlace nibble codes have the same semantics in both protocols. Avalon streaming video carries interface nibble codes in the low nibble of the last byte of the packet. Altera Streaming Video packs them in positions 8 down to 5 of the first beat of the packet.

The Avalon streaming video protocol makes use of the Avalon streaming `empty` signal to indicate any empty symbols in the last beat. The Altera Streaming Video Protocol does not use the AXI4-Stream `TKEEP` or `TSRB` signals.

Data packets

Data packets in the Avalon streaming video protocol represent one entire field or frame of video. The Altera Streaming Video Protocol transports each line of video as individual data packet. Data packets are also ready latency 1 in the Avalon streaming video protocol.

The protocols have other differences for pixel packing and empty symbols. The protocol converter IP manages these differences, so you do not need to understand the difference between the two standards.

4. Video and Vision Processing IP Interfaces

The IPs have the main clock, an optional control clock, and some IPs have an optional memory interface clock.

Table 5. Altera video stream input interface

Signal name	Direction	AXI4-Stream Wire Signal	Width
axi4s_vid_in_tdata	Input	TDATA	Number of data bytes * 8
axi4s_vid_in_tlast	Input	TLAST	1
axi4s_vid_in_tuser	Input	TUSER	Number of data bytes
axi4s_vid_in_tvalid	Input	TVALID	1
axi4s_vid_in_tready	Output	TREADY	1

Table 6. Altera video stream output interface

Signal name	Direction	AXI4-Stream Wire Signal	Width
axi4s_vid_out_tdata	Output	TDATA	Number of data bytes * 8
axi4s_vid_out_tlast	Output	TLAST	1
axi4s_vid_out_tuser	Output	TUSER	Number of data bytes
axi4s_vid_out_tvalid	Output	TVALID	1
axi4s_vid_out_tready	Input	TREADY	1

Number of data bytes = $\max(2, \text{ceil}(\text{bits per sample} * \text{number of color planes} / 8)) * \text{pixels in parallel}$

Related Information

- [1D LUT IP Interfaces](#) on page 71
- [3D LUT IP Interfaces](#) on page 89
- [Adaptive Noise Reduction IP Interfaces](#) on page 103
- [Advanced Test Pattern Generator IP Interfaces](#) on page 127
- [AXI-Stream Broadcaster IP Interfaces](#) on page 160
- [Bits per Color Sample Adapter IP Interfaces](#) on page 169
- [Black Level Correction IP Interfaces](#) on page 188
- [Black Level Statistics IP Interfaces](#) on page 212
- [Protocol Converter IP Interfaces](#) on page 54
- [Chroma Key IP Interfaces](#) on page 233
- [Chroma Resampler IP Interfaces](#) on page 260

- [Clipper IP Interfaces](#) on page 273
- [Clocked Video Input IP Interfaces](#) on page 296
- [Clocked Video to Full-Raster Converter Interfaces](#) on page 336
- [Color Plane Manager IP Interfaces](#) on page 387
- [Color Space Converter IP Interfaces](#) on page 407
- [Defective Pixel Correction IP Interfaces](#) on page 429
- [Deinterlacer Interfaces](#) on page 451
- [Demosaic IP Interfaces](#) on page 471
- [FIR Filter Interfaces](#) on page 490
- [Frame Cleaner IP Interfaces](#) on page 511
- [Full Raster to Clocked Video Converter Interfaces](#) on page 532
- [Full-Raster to Streaming Converter Interfaces](#) on page 541
- [Generic Crosspoint IP Interfaces](#) on page 586
- [Genlock Controller IP Interfaces](#) on page 548
- [Genlock Signal Router IP Interfaces](#) on page 594
- [Guard Bands IP Interfaces](#) on page 604
- [Histogram Statistics IP Interfaces](#) on page 619
- [Interlacer IP Interfaces](#) on page 640
- [Mixer IP Interfaces](#) on page 659
- [Pixels in Parallel Converter Interfaces](#) on page 680
- [Scaler IP Interfaces](#) on page 699
- [Stream Cleaner IP Interfaces](#) on page 743
- [Switch IP Interfaces](#) on page 750
- [TMO IP Interfaces](#) on page 798
- [Test Pattern Generator IP Interfaces](#) on page 827
- [Unsharp Mask IP Interfaces](#) on page 858
- [Video and Vision Monitor IP Interfaces](#) on page 868
- [Video Frame Buffer IP Interfaces](#) on page 886
- [Video Frame Reader IP Interfaces](#) on page 911
- [Video Frame Writer IP Interfaces](#) on page 945
- [Video Streaming FIFO IP Interfaces](#) on page 968
- [Video Timing Generator IP Interfaces](#) on page 982
- [Vignette Correction IP Interfaces](#) on page 1016
- [Warp IP Interfaces](#) on page 1063
- [White Balance Correction IP Interfaces](#) on page 1125
- [White Balance Statistics IP Interfaces](#) on page 1144
- [Altera Streaming Video Protocol Specification](#)

5. Video and Vision Processing IP Registers

Each IP has an optional memory-mapped control agent interface. When you turn **Memory-mapped control interface** on in the GUI, the interface connects to your host processor via an Avalon memory-mapped host interface (or via an AXI4 memory-mapped bus through conversion in Platform Designer). It provides a unified method of controlling the IPs.

Every IP has a standard address map when you turn on **Memory-mapped control interface**. If you turn off **Memory-mapped control interface**, the IP always processes video data when it is available.

Generally, video and vision processing IP register maps have these distinctive areas:

- A common area, which contains parameterization information. You can read from IPs to determine their parameters, which allows portability of software and binaries between different video and vision processing platforms.
- An IP-specific video and vision processing area, which contains functional configuration information for the specific IP.
- An optional control and debug register area, which allows you to write video field information in lite variants, read video field information in full variants, and to perform other control functions.

Control interfaces use Avalon memory-mapped interfaces. Platform Designer natively supports AXI4-Lite protocols and can automatically adapt to and from Avalon memory-mapped interfaces. Memory interfaces also use Avalon memory-mapped interfaces.

Table 7. Register Map for Full Variant Video and Vision Processing IPs

With some exceptions (for example coefficient banks) in lite variants, register writes usually take effect at the start of the next field.

Address	Register
Parameterization Registers	
0x0000	VID_ID
0x0004	VERSION
0x0008 to 0x00FF	IP-specific parameterization registers.
0x0100 to 0x011F	Reserved for future use.
Reserved for optional Control and debug registers	
0x0120	IMG_INFO_WIDTH
0x0124	IMG_INFO_HEIGHT
0x0128	IMG_INFO_INTERLACE
0x012C	Reserved for future use.
<i>continued...</i>	

Address	Register
0x0130	IMG_INFO_COLORSPACE
0x0134	IMG_INFO_SUBSAMPLING
0x0138	IMG_INFO_COSITING
0x013C	IMG_INFO_FIELD_COUNT
IP-specific registers	
Read-only registers	
0x0140	STATUS
0x0144 to RW_BASE	Optional additional read-only registers
Read-write registers	
RW_BASE	Optional COMMIT register (must be present if any other read-write registers are present)
RW_BASE+4 onwards	Optional additional read-write registers

Parameterization registers

The parameterization registers address space occupies from the base address of the IP to a maximum address of 0x011F. The first two registers in this section are the product ID and version registers, so you can identify the IP and the structure of its register map.

The next registers are an optional set of further parameterization information which IPs may populate to inform software on how to control the IP.

For example, a clipper describes clipping using either offsets from the edge of the video or using a top-left corner offset with required height and width. Control software can interrogate the Clipper IP's parameterization registers to determine which is correct and drive clipping control register accesses accordingly. You can use the same control software across different video pipeline configurations.

The parameterization registers section is followed by an unused area from 0x0100 to 0x011F.

Control and debug registers

You control the IPs in two ways:

- Provide the input video field information. For example the type of video packets you send to the IP with respect to height, width, or interlacing.
- Instruct the IP. For example which clipper offsets, mixer offsets, or scaling behavior to apply.

Full variants send image information packets down the video processing pipeline to communicate video field information. Image information packets handle the communication of any resolution changes to downstream IPs. You can control IP functions for full variants via the IP-specific registers.

Communicate video field information and control IP functions for lite variants via the control and IP-specific registers, using the register interface.

Registers 0x0120 to 0x0138 (IMG_INFO_WIDTH to IMG_INFO_COSITING) are the image information registers. For lite variants, write to these to set the dimensions and properties of incoming video fields. For full variants, ignore these registers. However, if you turn on debug features, you can read these registers to retrieve details of the image information fields that the IP receives via the last received image information packet.

Register 0x013C is the IMG_INFO_FIELD_COUNT register. Read this register for full variants when you turn on **Debug features** for the field count field of the last received image information packet.

Register 0x0140 is the STATUS register. IPs hold the LSB of this register high while they process video packets. The LSB returns low in-between video fields. Bit 1 of this register is the pending register updates bit. The IP sets this bit when you make writes to the IP's register map. The IP automatically clears this bit.

The COMMIT register is at address RW_BASE or later. Write to this register after writing new values to any of the registers in the IP-specific portion of the address map. The IP sets the pending register updates bit of the STATUS register in response and clears it when the new settings take effect.

IP-specific registers

This section of the register map is different for every IP and comprises read and write registers required to control the IP, such as clipping, mixing offsets, color space, or scaling coefficients.

IPs without a memory-mapped control interface

If you turn off **Memory-mapped control interface**, the IP always processes video data when it is available.

Related Information

- [1D LUT IP Registers](#) on page 72
- [3D LUT IP Registers](#) on page 90
- [Adaptive Noise Reduction IP Registers](#) on page 104
- [Advanced Test Pattern Generator IP Registers](#) on page 128
- [Bits per Color Sample Adapter IP Registers](#) on page 170
- [Black Level Correction IP Registers](#) on page 189
- [Black Level Statistics IP Registers](#) on page 213
- [Chroma Key IP Registers](#) on page 235
- [Chroma Resampler IP Registers](#) on page 261
- [Clipper IP Registers](#) on page 275
- [Clocked Video Input IP Registers](#) on page 298
- [Clocked Video Converter to Full-Raster IP Registers](#) on page 340
- [Clocked Video Output IP Registers](#) on page 352
- [Color Plane Manager IP Registers](#) on page 388
- [Color Space Converter IP Registers](#) on page 408

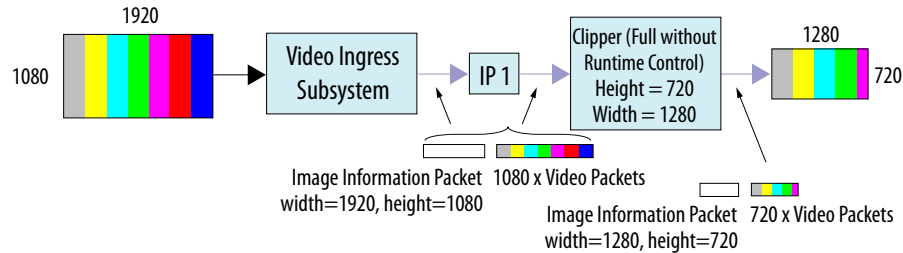
- [Defective Pixel Correction IP Registers](#) on page 430
- [Deinterlacer IP Registers](#) on page 453
- [Demosaic IP Registers](#) on page 472
- [FIR Filter Registers](#) on page 492
- [Frame Cleaner IP Registers](#) on page 512
- [Full-Raster to Clocked Video Converter Registers](#) on page 535
- [Generic Crosspoint IP Registers](#) on page 587
- [Genlock Controller IP Registers](#) on page 552
- [Genlock Signal Router IP Registers](#) on page 595
- [Guard Bands IP Registers](#) on page 605
- [Histogram Statistics IP Registers](#) on page 620
- [Interlacer IP Registers](#) on page 641
- [Mixer IP Registers](#) on page 661
- [Pixels in Parallel Converter Registers](#) on page 682
- [Protocol Converter IP Registers](#) on page 56
- [Scaler IP Registers](#) on page 700
- [Switch IP Registers](#) on page 752
- [Test Pattern Generator IP Registers](#) on page 828
- [TMO IP Registers](#) on page 803
- [Unsharp Mask IP Registers](#) on page 859
- [Video and Vision Monitor IP Registers](#) on page 869
- [Video Frame Buffer IP Registers](#) on page 888
- [Video Frame Reader IP Registers](#) on page 912
- [Video Frame Writer IP Registers](#) on page 946
- [Video Timing Generator IP Registers](#) on page 984
- [Vignette Correction IP Registers](#) on page 1017
- [Warp IP Registers](#) on page 1079
- [White Balance Correction IP Registers](#) on page 1126
- [White Balance Statistics IP Registers](#) on page 1145

5.1. Video and Vision Processing IP Control Examples

Consider which control method best suits your application. Configure the IP according to one of the following methods.

Figure 8. Full variant with no memory-mapped control interface, fixed operation

The figure shows a video pipeline with a video ingress system (for example, HDMI, Displayport, PCIe) and two IPs, the second IP is the Clipper.



In this system, the full variant clipper IP is parameterized to perform a fixed clip of 1080p HD video clipped down to 720p HD video and has no memory-mapped control interface. The clipper parameters always clip to a height and width of 1280x720 pixels. No further control is necessary as information about the incoming video fields is carried in image information control packets (refer to the *Altera Streaming Video Protocol Specification*). So, if the input resolution changed to 720p, the clipper reads that information from the image information packets and performs no additional clipping.

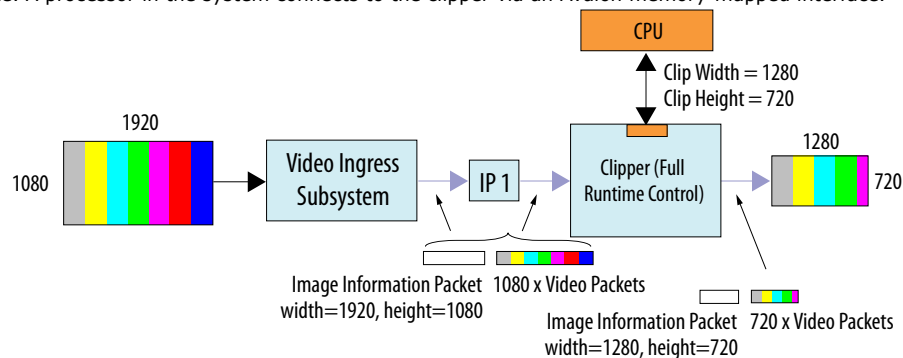
The figure shows the video protocol packets from the video ingress subsystem. The packets stream into IP1, which doesn't change the dimensions of the video. It sends the same image information packets and the same number of data packets into the clipper IP.

The figure shows an image information packet with the video field information for a progressive frame of 1920x1080 pixels. Each video packet carries one line of 1920 pixels.

The image information packet from the clipper contains the new field dimensions of 1280x720 and the clipper only outputs 720 video packets, each of 1280 pixels.

Figure 9. Full variant with memory-mapped control interface

The figure shows the same system but with a full variant clipper and with a memory-mapped run-time control interface. A processor in the system connects to the clipper via an Avalon memory-mapped interface.



Most systems need to control IPs so that you can apply different amounts of clipping, scaling or mixing. In this system, the full variant of the clipper IP now includes a memory-mapped control interface. The clipper IP includes a register map and the processor can read the clipper's parameter information and set clipping offsets for any clipping style that you want. When the processor writes any changes to clipping

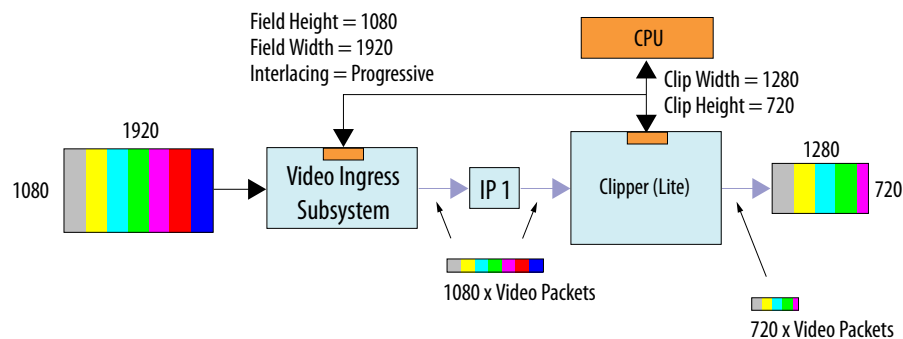
requirements, a final write to the `COMMIT` register (or receiving a `commit` auxiliary control packet) ensures they apply when the IP receives the next image information packet to mark the start of the next field.

The processor can also read the dimensions of the incoming video and the count of incoming fields if you turn on **Debug features**.

Altera recommends this method of control, which is the simplest and most flexible.

Figure 10. Lite variants with mandatory memory-mapped control interface

The figure shows the system with the lite variant clipper and a memory-mapped control interface. A processor connects to both the video ingress system and the clipper via Avalon memory-mapped interfaces.



In this example, the lite variant clipper IP is parameterized with the mandatory memory-mapped control interface. In lite variant IPs, the processor writes the field's properties to the clipper's `IMAGE_INFO` registers. The streaming interface just contains video data. Otherwise, control operates as for full variant IPs with memory-mapped control. The processor also writes desired clipping dimensions to the clipper's register map.

Related Information

[Altera Streaming Video Protocol Specification](#)



6. Video and Vision Processing IPs Software Programming Model

Most video and vision processing IPs are configurable at run-time. Memory-mapped run-time control interfaces expose a set of control registers to change the processing behavior and status registers to query the states of the processing functions. The set of available registers and the width in binary bits of each register varies with each control interface. You must assume that writable control registers and readable registers power up in an undefined state.

HAL Device Drivers for Nios II SBT

For software control and run-time reconfiguration, the Altera video and vision processing IPs have an associated software driver. The video IPs do not fit any of the generic device models provided by the Nios II HAL. Consequently, the bare-metal drivers only provide partial HAL support with a few dedicated accessor functions to the control and status registers. The video functions cannot be accessed indirectly through the generic HAL API or the ANSI C standard library.

Using these drivers in your application is optional. If you do not use drivers, Altera recommends you use the definitions provided by the register header files to control the IP components (*_regs.h) and avoid using unnamed constants and addresses to access the run-time register map from your application.

Refer to the *Nios® II Software Developer's Handbook* to get started with the Nios® II development tools.

Common Software API

Many Altera video and vision processing IPs expose a register map with common registers located at the same location (refer to the *Video and Vision Processing Registers*). To limit code footprint, many video and vision processing device structures (C struct) include a common device structure as their first member. This shared base allows the definition of a set of common functions to query the VID_PID and VERSION registers, respectively at addresses 0x0000 and 0x0004, and the set of IMG_INFO registers in the range 0x0120-0x013C.

Common API reference

Register definition header file: `intel_vvp_core_regs.h`

Include file: `intel_vvp_core.h`

Table 8. Common API reference

Name	Description
intel_vvp_core_init	Initialize the instance.
intel_vvp_core_get_vendor_id	Returns the vendor ID.
intel_vvp_core_get_product_id	Returns the product ID.
intel_vvp_core_get_qpds_major	Returns the version major number.
intel_vvp_core_get_qpds_update	Returns the version update number.
intel_vvp_core_get_qpds_patch	Returns the version patch number.
intel_vvp_core_get_register_map_version	Returns the version number of the register map.
intel_vvp_core_get_img_info_width	Returns the width read from the IMG_INFO_WIDTH register.
intel_vvp_core_get_img_info_height	Returns the height read from the IMG_INFO_HEIGHT register.
intel_vvp_core_get_img_info_interlace	Returns the interlace nibble read from the IMG_INFO_INTERLACE register.
intel_vvp_core_get_img_info_colorspace	Returns the colorspace code read from the IMG_INFO_COLORSPACE register.
intel_vvp_core_get_img_info_subsampling	Returns the subsampling code read from the IMG_INFO_SUBSAMPLING register.
intel_vvp_core_get_img_info_cositing	Returns the cositing code read from the IMG_INFO_COSITING register.
intel_vvp_core_get_img_info_field_count	Returns the field count code read from the IMG_INFO_FIELD_COUNT register.
intel_vvp_core_set_img_info_width	Writes a width to the IMG_INFO_WIDTH register.
intel_vvp_core_set_img_info_height	Writes a height to the IMG_INFO_HEIGHT register.
intel_vvp_core_set_img_info_interlace	Writes an interlace nibble to the IMG_INFO_INTERLACE register.
intel_vvp_core_set_img_info_colorspace	Writes a colorspace code to the IMG_INFO_COLORSPACE register.
intel_vvp_core_set_img_info_subsampling	Writes a subsampling code to the IMG_INFO_SUBSAMPLING register.
intel_vvp_core_set_img_info_cositing	Writes a cositing code to the IMG_INFO_COSITING register.

intel_vvp_core_init

Prototype

```
eIntelVvpCoreErrors
intel_vvp_core_init(intel_vvp_core_instance *instance,
intel_vvp_core_base base, uint16_t expected_product_id);
```

Description

Initializes the common members intel_vvp_core_instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match. Otherwise, the function proceeds to read and store the version numbers in the second register

Arguments

instance – pointer to the intel_vvp_core software driver instance structure

base – base address of the register map

expected_product_id – the expected product id

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product_id does not match with the expectation

kIntelVvpCoreInstanceErr if the instance is a null pointer

intel_vvp_core_get_vendor_id

Prototype `uint16_t intel_vvp_core_get_vendor_id(void *instance);`

Description Query the vendor_id of an initialized instance

Arguments instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it

Returns the vendor_id field in the intel_vvp_core_instance instance

intel_vvp_core_get_product_id

Prototype `uint16_t intel_vvp_core_get_product_id(void *instance);`

Description Query the product_id of an initialized instance

Arguments instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it

Returns the product_id field in the intel_vvp_core_instance instance

intel_vvp_core_get_qpds_major

Prototype `uint8_t intel_vvp_core_get_qpds_major(void *instance);`

Description Query the major version number of an instance. The instance must have been initialized successfully.

Arguments instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it

Returns the major version number field in the intel_vvp_core_instance instance

intel_vvp_core_get_qpds_update

Prototype	<pre>uint8_t intel_vvp_core_get_qpds_update(void *instance);</pre>
Description	Query the minor version number of an instance. The instance must have been initialized successfully.
Arguments	instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it
Returns	the update (minor) version number field in the intel_vvp_core_instance instance

intel_vvp_core_get_qpds_patch

Prototype	<pre>uint8_t intel_vvp_core_get_qpds_patch(void *instance);</pre>
Description	Query the patch version number of an instance. The instance must have been initialized successfully.
Arguments	instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it
Returns	the patch (build) version number field in the intel_vvp_core_instance instance

intel_vvp_core_get_register_map_version

Prototype	<pre>uint8_t intel_vvp_core_get_register_map_version(void *instance);</pre>
Description	Query the register map version number of an instance. The instance must have been initialized successfully.
Arguments	instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it
Returns	the register map version number field in the intel_vvp_core_instance instance

intel_vvp_core_get_img_info_width

Prototype	<pre>uint32_t intel_vvp_core_get_img_info_width(void *instance);</pre>
Description	Read the IMG_INFO_WIDTH register of an instance. The instance must have been initialized successfully and the register readable.
Arguments	instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it

Returns the value returned from a read to the image info width register

`intel_vvp_core_get_img_info_height`

Prototype `uint32_t intel_vvp_core_get_img_info_height(void *instance);`

Description Read the IMG_INFO_HEIGHT register of an instance. The instance must have been initialized successfully and the register readable.

Arguments instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it

Returns the value returned from a read to the image info height register

`intel_vvp_core_get_img_info_interlace`

Prototype `uint8_t intel_vvp_core_get_img_info_interlace(void *instance);`

Description Read the IMG_INFO_INTERLACE register of an instance. The instance must have been initialized successfully and the register readable.

Arguments instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it

Returns the value returned from a read to the image info interlace register

`intel_vvp_core_get_img_info_colorspace`

Prototype `uint8_t intel_vvp_core_get_img_info_colorspace(void *instance);`

Description Read the IMG_INFO_COLORSPACE register of an instance. The instance must have been initialized successfully and the register readable.

Arguments instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it

Returns the value returned from a read to the image info colorspace register

`intel_vvp_core_get_img_info_subsampling`

Prototype `uint8_t intel_vvp_core_get_img_info_subsampling(void *instance);`

<i>Description</i>	Read the IMG_INFO_SUBSAMPLING register of an instance. The instance must have been initialized successfully and the register readable.
<i>Arguments</i>	instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it
<i>Returns</i>	the value returned from a read to the image info subsampling register

`intel_vvp_core_get_img_info_cositing`

Prototype `uint8_t intel_vvp_core_get_img_info_cositing(void *instance);`

<i>Description</i>	Read the IMG_INFO_COSITING register of an instance. The instance must have been initialized successfully and the register readable.
<i>Arguments</i>	instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it
<i>Returns</i>	the value returned from a read to the image info cositing register

`intel_vvp_core_get_img_info_field_count`

Prototype `uint16_t intel_vvp_core_get_img_info_field_count(void *instance);`

<i>Description</i>	Read the IMG_INFO_FIELD_COUNT register of an instance. The instance must have been initialized successfully and the register readable.
<i>Arguments</i>	instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it
<i>Returns</i>	the value returned from a read to the image info field count register

`intel_vvp_core_set_img_info_width`

Prototype `eIntelVvpCoreErrors intel_vvp_core_set_img_info_width(void *instance, uint32_t width);`

<i>Description</i>	Write a new width to the IMG_INFO_WIDTH register of an instance. The instance must have been initialized successfully and the register writable.
<i>Arguments</i>	instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it

width – new width

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

intel_vvp_core_set_img_info_height

Prototype

```
eIntelVvpCoreErrors intel_vvp_core_set_img_info_height(void
*instance, uint32_t height);
```

Description Write a new height to the IMG_INFO_HEIGHT register of an instance. The instance must have been initialized successfully and the register writable.

Arguments instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it
height – new height

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

intel_vvp_core_set_img_info_interlace

Prototype

```
eIntelVvpCoreErrors intel_vvp_core_set_img_info_interlace(void
*instance, uint8_t height);
```

Description Write a new interlace to the IMG_INFO_INTERLACE register of an instance. The instance must have been initialized successfully and the register writable.

Arguments instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it
height – new height

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

intel_vvp_core_set_img_info_colorspace

Prototype

```
eIntelVvpCoreErrors intel_vvp_core_set_img_info_colorspace
(void *instance, uint8_t colorspace);
```

Description Write a new colorspace to the IMG_INFO_COLORSPACE register of an instance. The instance must have been initialized successfully and the register writable.

Arguments instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it

colorspace – new colorspace

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

`intel_vvp_core_set_img_info_subsampling`

Prototype

```
eIntelVvpCoreErrors  
intel_vvp_core_set_img_info_subsampling(void *instance,  
uint8_t subsampling);
```

Description Write a new subsampling to the IMG_INFO_SUBSAMPLING register of an instance. The instance must have been initialized successfully and the register writable.

Arguments instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it

subsampling – new subsampling

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

`intel_vvp_core_set_img_info_cositing`

Prototype

```
eIntelVvpCoreErrors intel_vvp_core_set_img_info_cositing (void  
*instance, uint8_t cositing);
```

Description Write a new cositing to the IMG_INFO_COSITING register of an instance. The instance must have been initialized successfully and the register writable.

Arguments instance – pointer to the initialized intel_vvp_core software driver instance structure or any software driver structure derived from it

cositing – new cositing

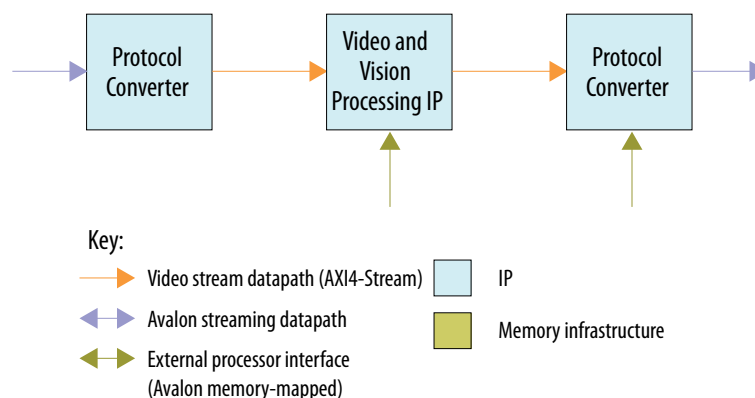
Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

7. Protocol Converter IP

7.1. About the Protocol Converter IP

The IP converts between three interface protocols: Avalon streaming video, Altera Streaming Video lite variant and Altera Streaming Video full variant. You can select any of the three protocols for the input and output interface. The IP allows you to create systems with IPs from both Altera video IP libraries. The Video and Vision Processing IPs use the AXI4-Stream based Altera Streaming Video Protocol to receive and transmit streaming video data at their interfaces. The Video and Image Processing IPs use the Avalon Streaming Video protocol. The Video and Vision Processing IPs do not directly connect to IPs from the Video and Image Processing library.

Figure 11. Protocol Converter Example



7.1.1. Protocol Converter IP Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family and the number of supported bits per color sample, colors per pixel, and pixels in parallel.

Table 9. Protocol Converter IP Performance and Resources

The table shows ALM usage and f_{MAX} for a design with 10 bits per color sample, 3 color planes, 4 pixels in parallel, and converting Altera Streaming Video full to Avalon streaming video.

Target device	ALMs	f_{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	1,231	688
Arria 10 speed grade 1 (10AS066H1F34E1HG)	980	447
Cyclone 10 GX speed grade 5 (10CX220YF78015G)	979	389
Stratix 10 GX speed grade 1 (1SX280LN2F43E1VG)	1,258	518

7.2. Protocol Converter IP Parameters

The IP offers compile-time parameters

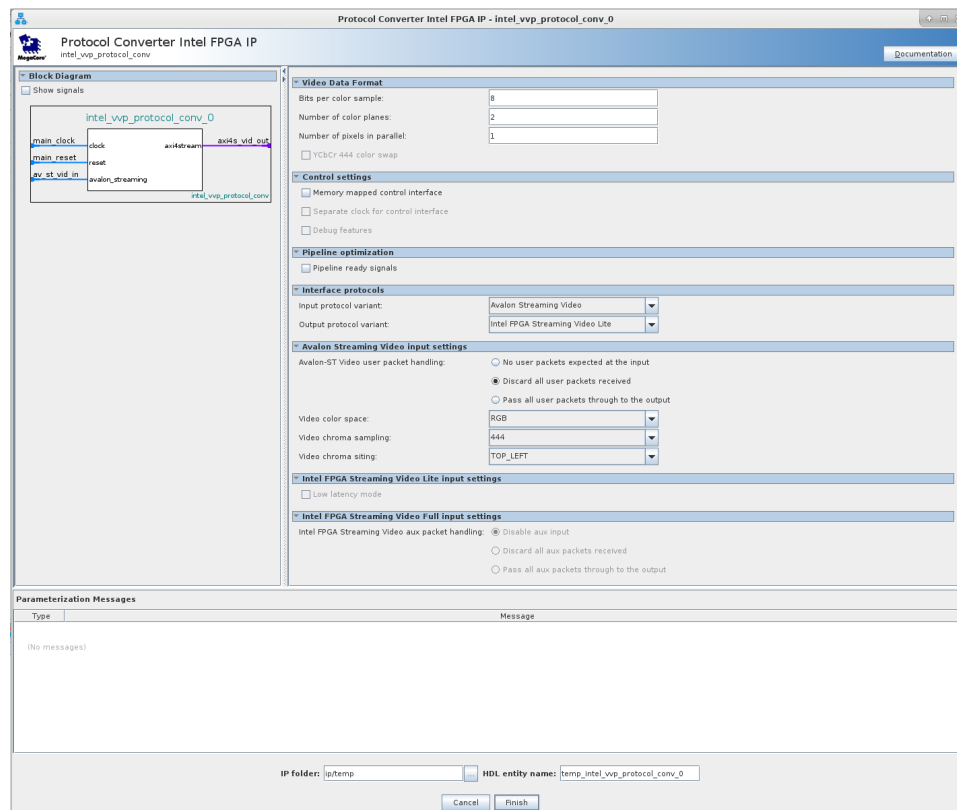
Table 10. Parameters

The table lists the parameters that are available to configure the IP in Platform Designer.

Name	Values	Description
Bits per color sample	8 to 16	Number of bits that represent each color sample
Number of color planes	1 to 4	Number of colors per pixel
Number of pixels in parallel	1 to 8	Number of pixels transmitted per clock cycle. If either the input or output interface use the Avalon Streaming Video protocol, the number of pixels in parallel must be a power of two.
YCbCr 444 color swap	On or off	Turn on to automatically correct for the color plane ordering differences between Avalon Streaming Video and Altera Streaming Video when transmitting YCbCr data with 4:4:4 chroma
Control settings		
Memory-mapped control interface	On or off	Turn on to allow the Avalon memory-mapped control agent interface to update settings at run time
Separate clock for control interface	On or off	Turn on for a separate clock domain for the Avalon memory-mapped control agent interface
Debug features	On or off	Turn on for the debugging features of the Avalon memory-mapped control agent interface
Pipeline Optimization		
Pipeline ready signals	On or off	Turn on to add extra pipeline registers to the AXI4-Stream or Avalon Streaming ready signals. Turning on this option may make it easier to close timing for the protocol converter and achieve a higher operation clock frequency, but may contribute to additional ALM usage.
Interface Protocols		
Input protocol variant	Avalon Streaming Video, Altera Streaming Video Full, or Altera Streaming Video Lite	Select the protocol for the input interface
Output protocol variant	Avalon Streaming Video, Altera Streaming Video Full, or Altera Streaming Video Lite	Select the protocol for the output interface
Avalon Streaming Video Input Settings		
How Avalon-ST Video user packets are handled	No user packets expected at the input or Discard all user packets received	Select how the IP processes user packets if the input protocol is Avalon Streaming Video. If you do not expect the input stream to contain any user packets, you can select No user packets expected at the input and save the ALM resources required to discard these packets
Video color space	RGB or YCbCr	If the input protocol is Avalon Streaming Video and you do not turn on Avalon memory-mapped control agent interface , you must specify the color space of the incoming video
<i>continued...</i>		

Name	Values	Description
Video chroma sampling	444, 422 or 420	If the input protocol is Avalon Streaming Video and you turn off Avalon memory-mapped control agent interface , you must specify the chroma sampling of the incoming video
Altera Streaming Video Lite input settings		
Enable low latency mode	On or off	If the input protocol is Altera Streaming Video this parameter determines the behavior of the Protocol Converter at the end of each video frame.
Altera Streaming Video Full input settings		
How Altera Streaming Video aux packets are handled	Disable aux input, Discard all aux packets received, Pass all aux packets through to the output	Select how the IP processes auxiliary packets if the input protocol is Altera Streaming Video.

Figure 12. Protocol Converter GUI



7.3. Protocol Converter IP Functional Description

Protocol Converter - Avalon Streaming Video to Altera Streaming Video Lite

The IP converts the protocol in steps:

1. Changes the Avalon Streaming ready latency from 1 to 0.

Avalon Streaming Video specifies a ready latency of 1. The IP converts the ready latency to 0 to match the ready-valid handshake mechanism specified for AXI4-Stream.

2. Removes all packets other than video packets from the stream.

Avalon Streaming Video specifies a mechanism to assign a type identifier (a number between 0 and 15) to each packet in the stream. Type-0 packets are frames of pixel data and all other packet types indicate nonvideo data. Packets of type-15 (referred to as metapackets) contain metadata to specify the width, height, and interlacing properties of subsequent type-0 video packets. Altera Streaming Video lite does not allow for metapackets in the stream. The IP discards all packets with a type that is greater than 0. The IP does not propagate type-15 metapackets that specify the properties of the video. However, it parses them during the discard process to extract the expected width of the video frames that follow. The IP uses this information in the next step of the conversion.

3. Splits frame packets into line packets.

Avalon Streaming Video specifies that each video packet contains one cycle of header data followed by all of the pixels required for an interlaced frame or progressive frame of video. The header data specifies the packet type. Altera Streaming Video requires that each packet is one line of video data (with no header information). The IP strips out incoming Avalon Streaming Video frame packets and then splits them into multiple packets. Each packet contains a single video line. The IP extracts the expected width of the video frame from the discarded metapacket that precedes the frame. The IP uses this value to determine where the incoming frame packet should be split to make each output line packet. The IP replaces the Avalon Streaming `startofpacket` and `endofpacket` signals with the AXI4-Stream `tlast` signal and creates the `tuser` signal. The IP reformats the incoming Avalon Streaming `data` to create byte-aligned AXI4-Stream `tdata`.

Avalon streaming video to Altera Streaming Video full

Converting Avalon streaming video to Altera Streaming Video full is similar to converting to Altera Streaming Video lite. Steps 1 and 3 are the same, but step 2 is different because Altera Streaming Video full supports nonvideo packets. The nonvideo data in the Avalon streaming video input can be retained and does not need to be discarded.

Nonvideo packets in Altera Streaming Video full are indicated by asserting bit 1 of the `tuser` field on the first cycle of the packet. As with Avalon streaming video, each nonvideo packet in Altera Streaming Video full has an associated packet type, which is indicated in the five LSBs of the first beat of data.

Altera Streaming Video full requires an image information packet (type 0) that precedes the packets for each video field and followed by an end-of-field packet (type 1). The IP adds these to the incoming stream. The image information packet is similar to the control packet in the Avalon streaming video input. It contains information about the width, height, and interlaced format of the following field, which can be sourced directly from the Avalon streaming video control packet. Further information about the input color space, chroma sampling and siting, which is not in the incoming control packet comes either from the register map or parameters, depending on how you configure the IP.

You can select how the IP processes user packets (types 1-14) in the Avalon streaming video input via the **How Avalon-ST Video user packets are handled** parameter. You can select that these types of packets never exist, in which case no provision is made in the IP to process them (if any are received the IP may lock-up). You can select that the IP discards these packets, in which case the IP adds some additional logic to detect and discard them, as is the case in conversion to Altera Streaming Video lite.

The final option that is not available when converting to Altera Streaming Video lite but is available when converting to Altera Streaming Video full, is to propagate the incoming user packets. However, Altera Streaming Video full cannot insert the user packet directly into the main video stream as nonvideo packets are restricted to only use 16 bits of the data signal, and to have a maximum length of 4 data beats.

To process larger nonvideo packets, Altera Streaming Video includes the option for a separate auxiliary interface, which follows the same protocol as the main video interface. The video packets on the video interface (those with `tuser` bit 1 set to 0) contain video lines. The data packets on the auxiliary interface may contain any nonvideo data you require. When you select **Pass all user packets through to the output** for **Avalon streaming video user packet handling**, the header beat of the Avalon streaming video user packet (which contains the packet type) is clipped from the packet, and the remainder of the data is routed to the auxiliary output interface. To allow the packets on the auxiliary interface to be synchronized with the main video interface, a single beat nonvideo packet is included in the main stream at the point where the user packet was present in the original Avalon streaming video input. Altera Streaming Video full sets aside the nonvideo packet types in the range 16-31 for free use by the system. The IP modifies the original Avalon streaming video packet type (in the range 1-14) by adding 16 to move it into the range allowed by Altera Streaming Video full, and the packet created uses this type. The newly created packet also sets a flag in bit 5 to indicate that a packet on the auxiliary interface is associated with this nonvideo packet. You should read it at this point in the video stream.

Protocol Converter - Altera Streaming Video Lite to Avalon Streaming Video

The IP converts the protocol in 3 steps.

1. Combines line packets into a single frame packet.

Altera Streaming Video specifies that you transmit video data with one video line per packet. Avalon Streaming Video requires that you transmit all the pixels in a frame in a single packet. The incoming line packets must merge to form one frame packet. If you transmit a single pixel per clock cycle, bit 0 of the incoming `tuser` signal marks the first pixel of each frame. The IP concatenates packets until bit 0 of `tuser` is asserted. If the number of pixels per line is not a multiple of the pixels per clock, the extra pixels in the final clock cycle of data are effectively empty and you must ignore them. You must specify the width of the incoming video frame via the register map. The IP uses this width information to determine which (if any) pixels it should ignore at the end of each line when concatenating the packets.

2. Adds the frame packet header and the control packet

Avalon Streaming Video requires that each frame packet begins with a one cycle header specifying a packet type of 0. The IP adds this header to the frame packet created previously. Avalon Streaming Video also recommends that each frame packet is preceded by a control packet (of type-15) that specifies the width, height, and interlacing scheme of the following frame. You supply the width and height via the register map, as the initial value for interlacing specifier. The IP

uses these values to control Avalon Streaming control packets that it adds to the stream. If you select an interlacing specifier for progressive video, the IP uses this value for all control packets. If the interlacing specifier identifies an interlaced scheme, the IP toggles the f0/f1 bit automatically in the outgoing control packets.

3. Converts Avalon Streaming ready latency 0 to 1

The IP replaces the AXI4-Stream `tlast` signal with the Avalon Streaming `startofpacket` and `endofpacket` signals. The IP creates the `empty` signal (if the number of pixels per clock cycle is greater than 1). The IP reformats the AXI4-Stream byte aligned `tdata` to non-byte aligned Avalon Streaming data. The interface is now compliant with the Avalon Streaming protocol, but with a ready latency of 0. Avalon Streaming Video requires that the ready latency is 1, so the IP converts the ready latency from 0 to 1.

Altera Streaming Video full to Avalon streaming video

Converting Altera Streaming Video full to Avalon streaming video is similar to converting from Altera Streaming Video lite. Steps 1 and 3 are the same, but step 2 is different because the Altera Streaming Video full supports nonvideo packets.

You can extract the information required to populate the Avalon streaming video control packet from the image info packet in the Altera Streaming Video stream. The Avalon memory-mapped control agent interface is not required.

All Altera Streaming Video nonvideo packets with a type in the range 0-15 have specified or reserved functions and cannot be translated to Avalon streaming video. The IP removes these packets from the stream regardless of the value you select for the **How Altera Streaming Video aux packets are handled** parameter. If you select **Discard all user packets received**, the IP discards all user-defined auxiliary control packets received from the Altera Streaming Video input. If you select **Discard all user packets received**, the optional auxiliary streaming input interface turns on. The IP discards any packets received from this interface, as this interface can carry associated payloads for auxiliary control packets.

If you select the option to pass through auxiliary packets, the IP must discard in-band packets with types 0-15 as these do not translate to Avalon streaming video. The IP must discard types 16, 28 and 31 as they map to reserved types in Avalon streaming video. For the remaining packet types, the IP looks at the value of the flag in bit 5 of the first data beat indicating if a packet on the auxiliary interface accompanies the packet. If the flag is not set, the IP creates an output packet containing just the header beat with the packet type minus 16. If the flag is set, the IP propagates a packet from the auxiliary interface to the output, with a single beat header appended to the front containing the in-band packet type minus 16. This process is the reverse of propagating user packets when converting from Avalon streaming video to Altera Streaming Video full.

The in-band packets that indicate a packet exists on the auxiliary interface may have between 1 and 4 beats of data. However, the IP only reads the data in the first beat, as it ignores all other data in the conversion process. The auxiliary interface conforms to the Altera Streaming Video Protocol and can carry all packet types. The IP discards all image information, end of field, and all auxiliary control packets. The IP forwards all other packets, as these are data packets for any auxiliary control packets received on the main Altera Streaming Video input.

Altera Streaming Video full allows auxiliary control packets to arrive in the stream both before the field data (the line packets), and between the final lines packet and the end of frame packet (both before and after the field). Avalon streaming video only allows user packets before the field packet. The IP considers any packets after the field as part of the next field. If the IP encounters auxiliary control packets after the field in Altera Streaming Video full, it still converts them to Avalon streaming video, but considers them part of the following frame. You should avoid the use of auxiliary control packets with types 17-31 after the field if this behavior is not acceptable.

Altera Streaming Video lite to Altera Streaming Video full

Converting from the lite variant to the full variant of Altera Streaming Video is a light-weight process because both variants use the same data format and packetization to transmit lines of video. The conversion process adds an information packet before the first line of each field and an end-of-field packet after the final line. The IP takes the information required to populate the image information packet from the register map. The broken field flag is the only information required for the end-of-field packet. The broken flag is set if the number of lines received for the given frame does not match the field height set in the register map and the field count. The field count is set from a counter that you can reset to zero via the register map at any time, and which increments at the end of each field.

Altera Streaming Video full to Altera Streaming Video lite

When converting from the full variant to the lite variant of Altera Streaming Video both variants use the same data format and packetization to transmit lines of video. To convert, the IP removes any image information, end of field, or auxiliary control packets (marked by asserting bit 1 of `tuser` for the first beat of the packet) from the input stream.

Pixel data format

The pixel data format for Avalon Streaming Video and Altera Streaming Video is almost identical. Altera Streaming Video requires that the width of each pixel is rounded up to the next whole number of bytes. Any extra bits required can be filled with zeros, ones, or any random data. Avalon Streaming Video has no such requirement and uses only the required bits for each pixel. The Protocol Converter adds the required extra bits when converting from Avalon Streaming Video to Altera Streaming Video. It removes them when converting from Altera Streaming Video to Avalon Streaming Video.

Avalon Streaming Video and Altera Streaming Video both specify how the color planes in each pixel should be arranged for RGB and YCbCr formatted data. For YCbCr data, the protocols specify the color plane ordering for 4:4:4, 4:2:2 and 4:2:0 chroma sampling. The color plane ordering is almost identical between the two protocols, apart from swapping Y and Cr planes in the case of YCbCr 4:4:4. The Protocol Converter IP can implement the swap, but you must specify the color space and chroma sampling for each frame. You can specify either via the parameters or the register map accessed through an Avalon memory-mapped agent interface.

You can turn on or turn off the Avalon memory-mapped agent interface via a parameter. If the Avalon memory-mapped agent interface is turned on, specify the color space and chroma sampling at run time via the register map. If the Avalon memory-mapped agent interface is not turned on, specify the color space and chroma sampling in the **Video color space** and **Video chroma sampling** parameters respectively.

If the Protocol Convert IP converts from Altera Streaming Video to Avalon Streaming Video, turn on the Avalon memory-mapped agent interface and do not use the parameters. If the Protocol Convert IP converts from Avalon Streaming Video to Altera Streaming Video, the Avalon memory-mapped agent interface is optional. If you know the color space and chroma sampling are fixed for the system, you can opt to turn off the agent interface and specify the color space and chroma sampling via the parameters. If the color space and chroma sampling may vary at run time, turn on the agent interface and specify the values in the register map.

For conversions both ways, the IP gates the color plane swap for YCbCr 4:4:4 formatted data by the **YCbCr 444 color swap** parameter. You must turn on this option for the IP to apply the color plane swap.

4:2:0 chroma sampled video

Avalon streaming video requires you to declare a fixed number of color planes per pixel. The packing of 4:2:0 color planes does not fully align with this definition of a pixel. 4 luma samples exist for every pair of Cb and Cr samples, so the effective number of color planes per pixel is 1.5 (1 luma sample and ½ a chroma sample). You cannot configure Avalon streaming video to have 1.5 colors per pixel, so the IP packs two luma samples with each chroma to make an atom of transport on the Avalon streaming video interface. The Avalon streaming video protocol considers this group of three color planes to be a pixel, even though it contains 2 luma samples and is effectively 2 pixels. The width value reported in the control packet counts the number of 3 color plane groups that interface treats as a pixel, so the width value reported is always half the actual field width for 4:2:0 sampled data.

Both variants of Altera Streaming Video Protocol pack two luma samples into an atom of transport that you consider a pixel in 4:4:4 or 4:2:2 sampling. The protocol similarly defines chroma sampling. The field width specified in the image information packets (when using the full variant of Altera Streaming Video) and the register map is always the true field width.

The protocol converter IP requires you to specify the chroma sampling and automatically doubles or halves the reported widths where required.

End of field detection in Altera Streaming Video lite

For Avalon Streaming Video, the `endofpacket` signal marks the end of each video frame that is asserted on the final pixel of the video data packet.

In Avalon Streaming Video, you cannot transmit the final pixel of each frame until you are certain that it is the final pixel, otherwise you risk driving the `endofpacket` signal incorrectly.

For Altera Streaming Video the end of each frame is inferred by receiving the start-of-frame marker for the next frame, which is explicitly indicated in the protocol. You might see potential latency issues when converting from Altera Streaming Video to Avalon Streaming Video. The IP cannot transmit the final pixel of each frame at the output until it receives the first pixel of the next frame at the input. When converting to Altera Streaming Video full, the IP cannot send the EOF packet for a given frame until it sees the first pixel of the next frame.

If the video application has no significant blanking (delay) between the last pixel of one frame and the first pixel of the next frame, the IP gives little or no delay in sending out the final pixel of each frame (Avalon streaming video) or end of frame

packet (Altera Streaming Video full). If the application does have significant blanking, the delay to transmit the final pixel or end of frame may be too long. The Protocol Converter IP includes an option to remove this delay.

If you turn on **Low latency mode**, the Protocol Converter IP transmits the Avalon Streaming Video frame `endofpacket` or Altera Streaming Video full EOF packet according to the number of lines it expects in each frame, as you specify in the register map. The Altera Streaming Video Protocol transmits each line of video data as a packet, so the IP terminates the output frame at the end of the input packet for the specified number of lines. If the IP receives any additional lines, the IP discards them and does not transmit them at the output.

7.4. Protocol Converter IP Interfaces

Name	Direction	Width	Description
Clocks and resets			
main_clock_clk	In	1	AXI4-S or Avalon Streaming processing clock.
main_reset_rst	In	1	AXI4-S or Avalon Streaming processing reset.
agent_clock_clk	In	1	Clock for the Avalon memory-mapped control agent interface. Only if you select Separate clock for control interface .
agent_reset_rst	In	1	Reset for the Avalon memory-mapped control agent interface. Only if you select Separate clock for control interface .
Control interfaces			
av_mm_control_agent_address	In	7	Avalon memory-mapped agent address.
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write.
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data.
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable.
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read.
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data.
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read.
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request.
Altera Streaming Video interfaces			
axi4s_vid_in_tdata	In		AXI4-S data in.
axi4s_vid_in_tvalid	In	1	AXI4-S data valid.
axi4s_vid_in_tuser	In		AXI4-S tuser. tuser[0] indicates start of video frame when asserted
<i>continued...</i>			

- (1) The equation shows all tdata widths in these interfaces:

$$\max(\text{ceil}((\text{bits per color sample} \times \text{number of color planes} \times \text{pixels in parallel}) / 8) \times 8, 16)$$
- (2) The equation gives all tuser widths in these interfaces

$$N = \text{ceil}(\text{tdata width} / 8)$$

Name	Direction	Width	Description
			tuser[1] indicates the start of a non-video packet when asserted
axi4s_vid_in_tlast	In	1	AXI4-S end of packet.
axi4s_vid_in_tready	Out	1	AXI4-S data ready.
axi4s_aux_in_tdata	In	(1)	AXI4-S data in.
axi4s_aux_in_tvalid	In	1	AXI4-S data valid.
axi4s_aux_in_tuser	In	(2)	AXI4-S tuser tuser[0] indicates start of video frame when asserted tuser[1] indicates the start of a non-video packet when asserted.
axi4s_aux_in_tlast	In	1	AXI4-S end of packet.
axi4s_aux_in_tready	Out	1	AXI4-S data ready.
axi4s_vid_out_tdata	Out	(1)	AXI4-S data in.
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid.
axi4s_vid_in_tuser	Out	(2)	AXI4-S tuser tuser[0] indicates start of video frame when asserted. tuser[1] indicates the start of a non-video packet when asserted.
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet.
axi4s_vid_out_tready	In	1	AXI4-S data ready.
Avalon streaming video interfaces			
av_st_vid_in_valid	In	1	Avalon streaming valid.
av_st_vid_in_ready	out	1	Avalon streaming ready.
av_st_vid_in_startofpacket	In	1	Avalon streaming start of packet.
av_st_vid_in_endofpacket	In	1	Avalon streaming end of packet.
av_st_vid_in_data	In		Avalon streaming data.
av_st_vid_in_empty	In		Avalon streaming empty.
av_st_vid_out_valid	Out	1	Avalon streaming valid.
av_st_vid_out_ready	In	1	Avalon streaming ready.
av_st_vid_out_startofpacket	Out	1	Avalon streaming start of packet.
<i>continued...</i>			

- (3) The equation shows all data widths in these interfaces:
bits per color sample x number of color planes x pixels in parallel
- (4) The empty signal is only used if pixels in parallel > 1. Where applicable, the equation gives all empty widths in these interfaces:
 $\log_2(\text{number of color planes} \times \text{pixels in parallel})$

Name	Direction	Width	Description
av_st_vid_out_endofpacket	Out	1	Avalon streaming end of packet.
av_st_vid_out_data	Out	(3)	Avalon streaming data.
av_st_vid_out_empty	Out	(4)	Avalon streaming empty.

Related Information

Video and Vision Processing IP Interfaces on page 30

7.5. Protocol Converter IP Registers

Read and write access to the register map is via the Avalon memory-mapped compliant `av_mm_control_agent` interface. Turn on **Memory-mapped control interface**, for access to this interface and access to the register map.

Table 11. Protocol Converter IP Parameterization Registers

In the software API these register names appear with a prefix of `INTEL_VVP`, `INTEL_VVP_CORE` or `INTEL_VVP_PROTOCOL_CONV` as appropriate and with an optional `REG` suffix

Address	Register	Access	Description
0x0000	VID_PID	RO	Read this register to retrieve the protocol converter vendor ID and product ID. This register always returns 6AF7_023D.
0x0004	VERSION	RO	Read this register for the IP version information.
0x0008	CONVERSION_MODE	RO	A read to this register returns a value that specifies the input and output protocols for this instance of the Protocol Converter IP. 0 = Avalon streaming video to Altera Streaming Video Lite 1 = Altera Streaming Video Lite to Avalon streaming video 2 = Avalon streaming video to Altera Streaming Video Full 3 = Altera Streaming Video Full to Avalon streaming video 4 Altera Streaming Video Lite to Altera Streaming Video Full 5 = Altera Streaming Video Full to Altera Streaming Video Lite
0x000C	DEBUG_ENABLED	RO	A read to this register returns the value you select for the Enable debug parameter in this instance of the Protocol converter. The host software can read this value to determine which registers you can read
0x0010 to 0x011F	-	-	Unused.

Table 12. Protocol Converter IP Control and Debug Registers

In the software API these register names appear with a prefix of INTEL_VVP, INTEL_VVP_CORE or INTEL_VVP_PROTOCOL_CONV as appropriate and with an optional REG suffix. For more information, refer to *Control Packets*.

Address	Register	Access ⁽⁵⁾		Description
		Not Full	Full	
0x0120	IMG_INFO_WIDTH	RW	RO	When the input protocol is Altera Streaming Video lite, use this register to set the expected width of incoming video fields. When the input protocol is Altera Streaming Video full and you turn on Debug features , this register returns the width that the protocol converter derives from information in the image information packet. This register is unused if the input protocol is Avalon streaming video.
0x0124	IMG_INFO_HEIGHT	RW	RO	When the input protocol is Altera Streaming Video lite, use this register to set the expected height of incoming video fields. When the input protocol is Altera Streaming Video full and you turn on Debug features , this register returns the height that the protocol converter derives from information in the image information packet. This register is unused if the input protocol is Avalon streaming video.
0x0128	IMG_INFO_INTERLACE	RW	RO	When the input protocol is Altera Streaming Video lite, use this register to set the expected interlacing of incoming video fields. When the input protocol is Altera Streaming Video full and you turn on Debug features , this register returns the interlace code that the protocol converter derives from information in the image information packet. This register is unused if the input protocol is Avalon streaming video.
0x012C	Reserved	-	-	Reserved.
0x0130	IMG_INFO_COLORSPACE	RW	RO	When the input protocol is Altera Streaming Video lite or Avalon streaming video, use this register to set the expected color space of incoming video fields. When the input protocol is Altera Streaming Video full and you turn on Debug features , this register returns the color space that the protocol converter derives from information in the image information packet.
0x0134	IMG_INFO_SUBSAMPLING	RW	RO	When the input protocol is Altera Streaming Video lite or Avalon streaming video, use this register to set the expected chroma sampling of incoming video fields. When the input protocol is Altera Streaming Video full and you turn on Debug features , this register returns the chroma sampling that the protocol converter derives from information in the image information packet.
0x0138	IMG_INFO_COSITING	RW	RO	When the input protocol is Altera Streaming Video lite or Avalon streaming video, use this register to set the expected co-siting of incoming video fields. When the input protocol is Altera Streaming Video full and you turn on Debug features , this register returns the cositing that the protocol converter derives from information in the image information packet.
continued...				

- ⁽⁵⁾ The function of each of these registers and the access permission, varies depending on which protocol you select for the input interface. The registers are read only (RO) if the input interface is Altera Streaming Video full, and RW if the input interface is either Altera Streaming Video lite or Avalon streaming video. If the registers are RW, they are only readable if you turn on **Debug features** parameter.

Address	Register	Access ⁽⁵⁾		Description
		Not Full	Full	
0x013C	IMG_INFO_FIELD_COUNT	-	RO	When the input protocol is Altera Streaming Video full and you turn on Debug features , this register returns the field count that the protocol converter derives from information in the image information packet. Unused if the input protocol is Avalon streaming video or Altera Streaming Video lite.
0x0140	STATUS	RO		The value you read from this register indicates the processing status of the IP.
0x0144	FIELD_COUNT	RO		Only used if the output protocol is Altera Streaming Video full. A read to this register returns the value of the internal field count, which increments after each output field. The IP uses it to populate the field count values in the output image info and end of frame packets.
0x0148	VIP_WIDTH	RO		If you select Avalon Streaming Video , for Input protocol variant , a read to this register returns the frame width in the most recently received control packet. The width that the IP reports is a literal decode of the information in the control packet. If the data the IP processes is 4:2:0 chroma sampled, the width that the IP reports is half the actual frame or frame width.
0x014C	VIP_HEIGHT	RO		If you select Avalon Streaming Video for Input protocol variant , a read to this register returns the frame height specified in the most recently received control packet.
0x0150	VIP_INTERLACE	RO		If you select Avalon Streaming Video for Input protocol variant , a read to this register returns the interlace nibble specified in the most recently received control packet.
0x0154	CTRL	RW		Writes to this register instruct the IP to start processing video frames, or to stop processing at the next frame boundary. Write a 1 to <code>bit[0]</code> of this register to start the IP. Write a 0 to <code>bit[0]</code> to stop at the next frame boundary. If the IP is already at a frame boundary or is between frames when the write to stop occurs, it stops immediately and does not begin the next frame. The value of this register resets to 0, so if the <code>av_mm_control_agent</code> interface is turned on, the IP resets into the stopped state and you must write a 1 to <code>bit[0]</code> to begin processing.
0x0158	FIELD_COUNT_RESET	RW		Any write to this register resets the value of the internal field count to zero. The value you write is unimportant. Only if the output protocol is Altera Streaming Video full.
0x015C	Reserved	-		Reserved.

Register Bit Descriptions

Table 13. VID_PID

Name	Bits	Description
Protocol converter vendor and product ID	31:0	This register always returns 6AF7_023D. 15:0 is the product ID and always returns 6AF7_023D 31:16 is the vendor ID and is always going to be 0x6AF7

⁽⁵⁾ The function of each of these registers and the access permission, varies depending on which protocol you select for the input interface. The registers are read only (RO) if the input interface is Altera Streaming Video full, and RW if the input interface is either Altera Streaming Video lite or Avalon streaming video. If the registers are RW, they are only readable if you turn on **Debug features** parameter.

Table 14. VERSION

Name	Bits	Description
Register map version	7:0	Register map version.
IP patch revision	15:8	-
IP update revision	23:16	Updated when the IP version changes.
IP major revision	31:24	Updated when the IP version changes.

Table 15. CONVERSION_MODE

Name	Bits	Description
Conversion mode	31:0	Returns the protocol conversion mode. 0 = Avalon streaming video to Altera Streaming Video lite 1 = Altera Streaming Video lite to Avalon streaming video 2 = Avalon streaming video to Altera Streaming Video full 3 = Altera Streaming Video full to Avalon streaming video 4 = Altera Streaming Video lite to Altera Streaming Video full 5 = Altera Streaming Video full to Altera Streaming Video lite

Table 16. DEBUG_ENABLED

Name	Bits	Description
Debug features parameterization bit	31:0	Returns 1 if you turn on Debug features and 0 otherwise.

Table 17. IMG_INFO_WIDTH

Name	Bits	Description
Width bits	15:0	For parameterizations with Altera Streaming Video lite input, write to this register to set the expected width of the incoming video fields. For parameterizations with Altera Streaming Video full input and with debug features on, this register returns the <code>width-1</code> field from the most recently received <code>image information packet</code> and adds 1 to return a value for <code>width</code> .
unused	31:16	Unused.

Table 18. IMG_INFO_HEIGHT

Name	Bits	Description
Height bits	15:0	For parameterizations with Altera Streaming Video lite input, write to this register to set the expected height of the incoming video fields. For parameterizations with Altera Streaming Video full input and with debug features on, this register reads the <code>height-1</code> field from the most recently received <code>image information packet</code> and adds 1 to return a value for <code>height</code> .
unused	31:16	Unused.

Table 19. IMG_INFO_INTERLACE

Name	Bits	Description
IntlaceNibble bits	3:0	For parameterizations with Altera Streaming Video lite input, write to this register to set the expected interlacing of the incoming video fields.
<i>continued...</i>		

		For parameterizations with Altera Streaming Video full input and with debug features enabled, this register returns the <code>intlaceNibble</code> field from the most recently received image information packet.
unused	31:4	Unused.

Table 20. IMG_INFO_COLORSPACE

Name	Bits	Description
CSP code bits	6:0	For parameterizations with Altera Streaming Video lite or Avalon streaming video input, write to this register to set the expected color space of the incoming video fields. For parameterizations with Altera Streaming Video full input and with debug features on, this register returns the 7 bit <code>CSP</code> field from the most recently received image information packet.
unused	31:7	

Table 21. IMG_INFO_SUBSAMPLING

Name	Bits	Description
SubSa code bits	1:0	For parameterizations with Altera Streaming Video lite or Avalon streaming video input, write to this register to set the expected subsampling of the incoming video fields. For parameterizations with Altera Streaming Video full input and with debug features on, this register returns the <code>SUBSA</code> field from the most recently received image information packet.
unused	31:2	Unused.

Table 22. IMG_INFO_COSITING

Name	Bits	Description
Cosite code bits	1:0	For parameterizations with Altera Streaming Video lite or Avalon streaming video input, write to this register to set the expected co-siting of the incoming video fields. For parameterizations with Altera Streaming Video full input and with debug features on, this register returns the <code>COSITE</code> field from the most recently received image information packet.
unused	31:2	Unused.

Table 23. IMG_INFO_FIELD_COUNT

Name	Bits	Description
Count bits	6:0	For parameterizations with Altera Streaming Video full input and with debug features on, this register returns the 7 bit <code>FIELD_COUNT</code> field from the most recently received image information packet.
unused	31:7	Unused.

Table 24. STATUS

Name	Bits	Description
Status bit	0	This bit indicates the IP is currently processing a frame. A value of 1 indicates that the IP is busy processing, a value of 0 indicates that it is idle. When the input uses the Altera Streaming Video lite protocol, the interpretation of bit 0 depends on if you turn on Enable low latency mode . If you turn on Enable low latency mode , the IP sets bit 0 to 1 when it receives the first pixel of the frame and sets it to 0 when it receives the number of lines specified in register map (address 0x124). The IP holds bit 0 at 0 while it flushes any additional lines. If you do not turn on Enable low latency mode , the IP sets bit 0 to 1 at the start of the first frame received and it remains high until you reset the IP.

continued...

		When the input uses the Altera Streaming Video full protocol or Avalon streaming video, the end of a frame sequence is clearly marked by the protocol and bit 0 of the status can be deasserted between frames
Frame processed bit	1	This bit indicates if the IP has fully processed at least one frame since the last reset. A 1 indicates that the IP has processed at least one, a 0 indicates that the IP has processed no frames.
unused	2	This bit indicates if the last frame the IP receives has the expected number of pixels. A 0 indicates that the frame matched the expected width and height. A 1 indicates that the frame had too many or too few pixels according to these settings.

Table 25. FIELD_COUNT

Name	Bits	Description
Field count	31:0	Current value of the internal field counter.

Table 26. VIP_WIDTH

Name	Bits	Description
VIP width	15:0	Field width specified in the last Avalon streaming video control packet received at the input.
unused	31:16	Unused.

Table 27. VIP_HEIGHT

Name	Bits	Description
VIP height	15:0	Field height specified in the last Avalon streaming video control packet received at the input.
unused	31:16	Unused.

Table 28. VIP_INTERLACE

Name	Bits	Description
VIP interlace	3:0	Interlace nibble specified in the last Avalon streaming video control packet received at the input.
unused	31:4	Unused.

Table 29. CTRL

Name	Bits	Description
Start / stop bit	0	Write 1 to this bit to start the IP, write 0 to stop the IP.
unused	31:1	Unused.

Table 30. FIELD_COUNT_RESET

Name	Bits	Description
unused	31:0	Unused.

7.6. Protocol Converter IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: intel_vvp_protocol_conv_regs.h

Include file: intel_vvp_protocol_conv.h

Table 31. Protocol Converter IP API reference

Name	Description
intel_vvp_protocol_conv_init	Initialize the Protocol Converter.
intel_vvp_core_*	Accessors defined in <i>Video and Vision Processing IPs Software API</i> . Writable when Lite is on, readable when Lite is off and Debug features is on. Unused when the input is video and image processing IP.
intel_vvp_protocol_conv_get_conversion_mode	Returns the configured conversion.
intel_vvp_protocol_conv_conv_get_debug_enabled	Returns if debug features is on.
intel_vvp_protocol_conv_is_running	Returns if the IP is processing data.
intel_vvp_protocol_conv_has_received_field	Returns if the IP processed a field.
intel_vvp_protocol_conv_was_last_field_broken	Returns status of the previous field.
intel_vvp_protocol_conv_get_status	Reads the status register.
intel_vvp_protocol_conv_get_vip_width	Reads the VIP width.
intel_vvp_protocol_conv_get_vip_height	Reads the VIP height.
intel_vvp_protocol_conv_get_vip_interlace	Reads the VIP interlace nibble.
intel_vvp_protocol_conv_enable	Starts or stops the processing.
intel_vvp_protocol_conv_start	Starts the processing.
intel_vvp_protocol_conv_stop	Stops the processing.
intel_vvp_protocol_conv_get_field_count	Reads the field counter.
intel_vvp_protocol_conv_reset_field_count	Resets the field counter.

enum eIntelVvpProtocolConvConversionMode

Members

- kIntelVvpProtocolConvVipToVvpLite (0) – VIP to Lite conversion
- kIntelVvpProtocolConvVvpLiteToVip (1) – Lite to VIP conversion
- kIntelVvpProtocolConvVipToVvpFull (2) – VIP to Full conversion
- kIntelVvpProtocolConvVvpFullToVip (3) – Full to VIP conversion
- kIntelVvpProtocolConvVvpLiteToVvpFull (4) – Lite to Full conversion
- kIntelVvpProtocolConvVvpFullToVvpLite (5) – Full to Lite conversion
- kIntelVvpProtocolConvInvalid (-1) – Invalid conversion mode

intel_vvp_protocol_conv_init

Prototype

```
int  
intel_vvp_protocol_conv_init(intel_vvp_protocol_conv_instance  
*instance, intel_vvp_core_base base);
```

Description

Initializes a Protocol Converter instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and should not be used further by the application if returning a non-zero error code. Beware that the IP is left in a stopped state after successful initialization.

Arguments

instance – pointer to the intel_vvp_protocol_conv_instance software driver instance structure

base – base address of the register map

Returns

kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the IP is not the Altera vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product_id does not match with the Protocol Converter product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpProtocolConvRegMapVersionErr if the register map is not supported

intel_vvp_protocol_conv_get_conversion_mode

Prototype

```
eIntelVvpProtocolConvConversionMode  
intel_vvp_protocol_conv_get_conversion_mode(intel_vvp_protocol_conv_instance  
*instance);
```

Description

Returns the value of the CONVERSION_MODE register. The instance must be a valid intel_vvp_protocol_conv_instance fully initialized

Arguments

instance – pointer to the intel_vvp_protocol_conv_instance software driver instance structure

Returns

the static conversion configured at IP generation time

intel_vvp_protocol_conv_get_debug_enabled

Prototype

```
bool  
intel_vvp_pip_get_debug_enabled(intel_vvp_protocol_conv_instance  
*instance);
```

- Description** Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_protocol_conv_instance fully initialized.
- Arguments** instance – pointer to the intel_vvp_protocol_conv_instance software driver instance structure
- Returns** true if the IP is parameterized with debug features enabled

intel_vvp_protocol_conv_is_running

Prototype

```
bool intel_vvp_protocol_conv_is_running(intel_vvp_pip_conv_instance* instance);
```

- Description** Reads and returns the running bit of the STATUS register. The instance must be a valid intel_vvp_protocol_conv_instance fully initialized.
- Arguments** instance – pointer to the intel_vvp_protocol_conv_instance software driver instance structure
- Returns** True is the IP is currently generating an output field, false when disabled and/or between two fields

intel_vvp_protocol_conv_has_received_field

Prototype

```
bool intel_vvp_protocol_conv_has_received_field(intel_vvp_protocol_conv_instance* instance);
```

- Description** Reads and returns the field received bit of the STATUS register. The instance must be a valid intel_vvp_protocol_conv_instance fully initialized.
- Arguments** instance – pointer to the intel_vvp_protocol_conv_instance software driver instance structure
- Returns** True is the IP is currently generating an output field, false when disabled and/or between two fields

intel_vvp_protocol_conv_was_last_field_broken

Prototype

```
bool intel_vvp_protocol_conv_was_last_field_broken(intel_vvp_protocol_conv_instance* instance);
```

- Description** Reads and returns the field broken bit of the STATUS register. The instance must be a valid intel_vvp_protocol_conv_instance fully initialized.
- Arguments** instance – pointer to the intel_vvp_protocol_conv_instance software driver instance structure

Returns True if an anomaly was detected on the last received field

`intel_vvp_protocol_conv_get_status`

Prototype

```
uint8_t  
intel_vvp_protocol_conv_get_status(intel_vvp_protocol_conv_instance*  
instance);
```

Description Reads and returns the STATUS register. The instance must be a valid `intel_vvp_protocol_conv_instance` fully initialized.

Arguments instance – pointer to the `intel_vvp_protocol_conv_instance` software driver instance structure

Returns The value read from the status register

`intel_vvp_protocol_conv_get_vip_width`

Prototype

```
uint32_t  
intel_vvp_protocol_conv_get_vip_width(intel_vvp_protocol_conv_instance*  
instance);
```

Description Retrieves the last width received from a VIP control packet. The instance must be a valid `intel_vvp_protocol_conv_instance` fully initialized and configured to convert the VIP protocol to either the full or lite variant.

Arguments instance – pointer to the `intel_vvp_protocol_conv_instance` software driver instance structure

Returns The width in the last control packet received

`intel_vvp_protocol_conv_get_vip_height`

Prototype

```
uint32_t  
intel_vvp_protocol_conv_get_vip_height(intel_vvp_protocol_conv_instance*  
instance);
```

Description Retrieves the last height received from a VIP control packet. The instance must be a valid `intel_vvp_protocol_conv_instance` fully initialized and configured to convert the VIP protocol to either the full or lite variant.

Arguments instance – pointer to the `intel_vvp_protocol_conv_instance` software driver instance structure

Returns The height in the last control packet received

intel_vvp_protocol_conv_get_vip_interlace

Prototype

```
uint8_t
intel_vvp_protocol_conv_get_vip_interlace(intel_vvp_protocol_conv_instance*
instance);
```

Description Retrieves the last interlace nibble received from a VIP control packet. The instance must be a valid intel_vvp_protocol_conv_instance fully initialized and configured to convert the VIP protocol to either the full or lite variant.

Arguments instance – pointer to the intel_vvp_protocol_conv_instance software driver instance structure

Returns The interlace nibble in the last control packet received

intel_vvp_protocol_conv_enable

Prototype

```
intel_vvp_protocol_conv_enable
(intel_vvp_protocol_conv_instance* instance, bool enabled);
```

Description Starts/stops the core

Arguments instance – pointer to the intel_vvp_protocol_conv_instance software driver instance structure

enabled– whether the instance should be started or stopped

Returns kIntelVvpCoreOk in case of success, negative error code otherwise

intel_vvp_protocol_conv_start

Prototype

```
int intel_vvp_protocol_conv_start
(intel_vvp_protocol_conv_instance* instance);
```

Description Starts the IP

Arguments instance – pointer to the intel_vvp_protocol_conv_instance software driver instance structure

Returns kIntelVvpCoreOk in case of success, negative error code otherwise

intel_vvp_protocol_conv_stop

Prototype

```
int intel_vvp_protocol_conv_stop
(intel_vvp_protocol_conv_instance* instance);
```

Description Stops the IP

Arguments instance – pointer to the intel_vvp_protocol_conv_instance software driver instance structure

Returns kIntelVvpCoreOk in case of success, negative error code otherwise

intel_vvp_protocol_conv_get_field_count

```
uint16_t  
intel_vvp_protocol_conv_get_field_count(intel_vvp_protocol_conv_instance  
*instance)
```

Description Reads and returns the FIELD_COUNT register. The instance must be a valid intel_vvp_protocol_conv_instance fully initialized. When the input is the Full variant, the field count register is set from the last end-of-field packet received and the call returns an invalid value if none was ever received

Arguments instance – pointer to the intel_vvp_protocol_conv_instance software driver instance structure

Returns The value read from the field_count register

intel_vvp_protocol_conv_reset_field_count

Prototype

```
int_t  
intel_vvp_protocol_conv_reset_field_count(intel_vvp_protocol_conv_instance*  
instance);
```

Description Resets the field counter register. The instance must be a valid intel_vvp_protocol_conv_instance fully initialized and configured with Lite or Vip input. The call is invalid when the input is the Full variant because the field counter register is updated every field with the field count value received in end-of-field control packets. The field counter value is also used in control packets when the output is the full variant.

Arguments instance – pointer to the intel_vvp_protocol_conv_instance software driver instance structure

Returns kIntelVvpCoreOk in case of success, negative error code otherwise

kIntelVvpCoreInstanceErr – the instance is invalid or configured with the Full variant input

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

8. 1D LUT IP

8.1. About the 1D LUT IP

The 1D LUT IP uses a software configurable look-up table to apply a transfer function to a streaming video. For every pixel, the IP converts its set of color values to a different set. The IP implements various opto-optical, opto-electrical, and electro-optical transfer functions defined for video standards. Also it performs legacy gamma compression or decompression. You may change the look-up table content arbitrarily to apply artistic effects to the video stream.

The IP only supports lite variants. For more information about full and lite variants, refer to the *Altera Streaming Video Protocol Specification*

Related Information

[Altera Streaming Video Protocol Specification](#)

8.1.1. 1D LUT IP Performance and Resources

Altera provides resource and utilization data for guidance only. The IP resource usage depends on the device family, compilation software version, compilation seed randomization, and other parameterization options.

Table 32. 1D LUT IP Performance and Resources

For 10-bit LUT and 3 Color Channels

Device	Pixels in Parallel	Bits per Color Sample	Max Resolution	ALM	M20k	DSP	f _{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	1	10	2k	969	3	3	807
	1	16	2k	2,203	6	3	818
	4	10	8k	3,169	12	12	803
	4	16	8k	8,114	24	12	773
Arria 10 speed grade 1 (10AS066H1F34E1HG)	1	10	2k	853	7	3	621
	1	16	2k	1,793	28	3	546
	2	10	4k	1,505	13	6	598
	2	16	4k	3377	55	6	508
Cyclone 10 GX speed grade 5 (10CX220YF672E5G)	1	10	2k	852	7	3	572
	1	16	2k	1,788	28	3	445
Stratix 10 speed grade 1 (1SX280LN2F43E1VG)	1	10	2k	1,029	3	3	705
continued...							

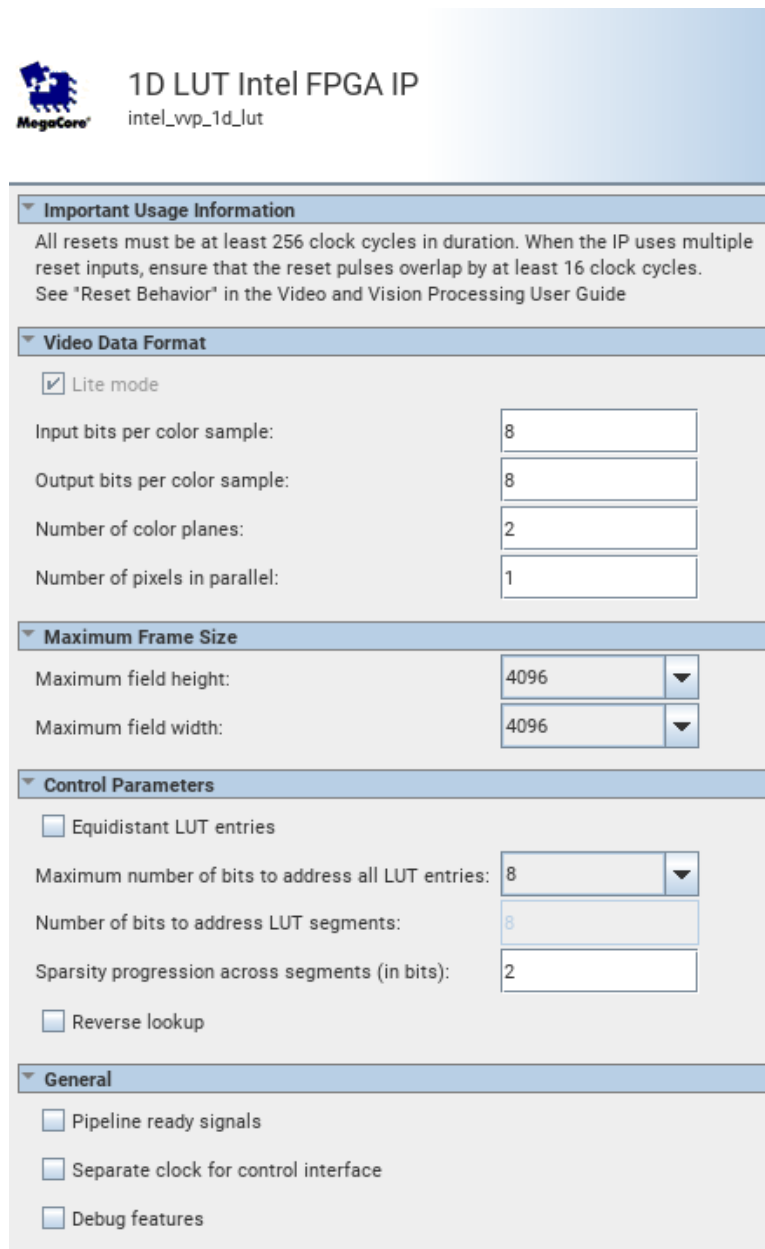
Device	Pixels in Parallel	Bits per Color Sample	Max Resolution	ALM	M20k	DSP	f _{MAX} (MHz)
	1	16	2k	2085	7	3	674
	2	10	4k	1764	6	6	675
	2	16	4k	4308	13	6	691

8.2. 1D LUT IP Parameters

The IP offers run-time and compile-time parameters.

Parameter	Values	Description
Video Data Format		
Lite mode	On	The IP only supports lite mode
Input bits per color sample	8 to 16	Select the number of input bits per color sample.
Output bits per color sample	8 to 16	Select the number of output bits per color sample.
Number of color planes	1 to 4	Number of color planes processed per pixel.
Number of pixels in parallel	1 to 8	Select the number of pixels in parallel.
Maximum Frame Size		
Maximum field height	1k, 2k, 4k, 8k, 16k	Specify the maximum height of incoming frames
Maximum field width	2k, 4k, 8k, 16k	Specify the maximum width of incoming frames
Control Parameters		
Equidistant LUT Entries	On or off	Turn on to distribute LUT entries equally across the dynamic range. When off, the IP divides the LUT into segments. When on, the segments at the lower end of the dynamic range are denser. The ones at the upper end are sparser.
Maximum number of bits to address all LUT entries	8 to 12	Maximum number of LUT entries in bits. This value cannot exceed the bits per color sample value.
Number of bits to address LUT segments	Derived	When Equidistant LUT Entries is on, the IP divides the LUT into segments. The IP derives this parameter from input bits per color sample, maximum number of LUT address bits, and sparsity bit parameters. Only visible when Equidistant LUT Entries is off.
Sparsity progression across segments (in bits)	1 to 2	The sparsity progression factor between neighboring LUT segments in number of address bits. Only available when Equidistant LUT Entries is off.
Reverse LUT	On or off	Allows you to use the LUT entries in reverse order.
General		
Pipeline ready signal	On or off	Turn on to add extra pipeline registers to the AXI4-S <code>tready</code> signals
Separate clock for control interface	On or off	Turn on for a separate clock for the control agent interface
Debug features	On or off	Turn on to read back frame information registers and debugging information registers via the control agent interface.

Figure 13. 1D LUT IP GUI



1D LUT Intel FPGA IP
intel_vvp_1d_lut

Important Usage Information
All resets must be at least 256 clock cycles in duration. When the IP uses multiple reset inputs, ensure that the reset pulses overlap by at least 16 clock cycles. See "Reset Behavior" in the Video and Vision Processing User Guide

Video Data Format

☒ Lite mode

Input bits per color sample: 8

Output bits per color sample: 8

Number of color planes: 2

Number of pixels in parallel: 1

Maximum Frame Size

Maximum field height: 4096

Maximum field width: 4096

Control Parameters

☐ Equidistant LUT entries

Maximum number of bits to address all LUT entries: 8

Number of bits to address LUT segments: 8

Sparsity progression across segments (in bits): 2

☐ Reverse lookup

General

☐ Pipeline ready signals

☐ Separate clock for control interface

☐ Debug features

8.3. 1D LUT IP Interfaces

Table 33. 1D LUT IP Clock and Reset Interfaces

Name	Direction	Width	Description
main_clock_clk	In	1	AXI4-S processing clock.
main_reset_rst	In	1	AXI4-S processing reset.
agent_clock_clk	In	1	Clock for the memory-mapped control agent interface. Only if you select Separate clock for control interface .
agent_reset_reset	In	1	Reset for the memory-mapped control agent interface. Only if you select Separate clock for control interface .

Table 34. 1D LUT IP Control Interfaces

Name	Direction	Width	Description
av_mm_control_agent_address	In	(6)	Avalon memory-mapped agent address
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write.
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data.
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable.
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read.
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data.
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read.
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request.

Table 35. 1D LUT Altera Streaming Video Interfaces

Name	Direction	Width	Description
Input			
axi4s_vid_in_tdata	In	(7)	AXI4-S data in.
axi4s_vid_in_tvalid	In	1	AXI4-S data valid.
axi4s_vid_in_tuser	In	(8)	[0] AXI4-S start of video frame. [N-1:1] Unused.
axi4s_vid_in_tlast	In	1	AXI4-S end of packet.
axi4s_vid_in_tready	Out	1	AXI4-S data ready.
Output			
axi4s_vid_out_tdata	Out	(7)	AXI4-S data out.
<i>continued...</i>			

(6) This equation gives address with of this interface: (maximum LUT address bits) + ceil(log2(1+(number of color planes)))

(7) This equation gives all tdata widths sizes in these interfaces: pixels in parallel × max (round (bits per color sample × number of color planes, 8), 16)

(8) This equation gives all tuserwidths sizes in these interfaces: N = ceil (tdata width / 8)

Name	Direction	Width	Description
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid.
axi4s_vid_out_tuser	Out	(8)	[0] AXI4-S start of video frame. [N-1:1] Unused.
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet.
axi4s_vid_out_tready	In	1	AXI4-S data ready

Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

8.4. 1D LUT IP Registers

Each register is either read-only (RO), read-write (RW) or write-only (WO).

Table 36. 1D LUT IP Registers

In the software API the register names appear with a prefix of INTEL_VVP, INTEL_VVP_CORE or INTEL_VVP_1DLUT as appropriate and with an optional REG suffix.

Address	Register	Access		Description
		Lite ⁽⁹⁾	Full	
Parameterization registers				
0x0000	VID_PID	RO	N/A	Read this register to retrieve the ID of the 1D LUT IP. This register always returns 0x6FA7_017D.
0x0004	VERSION	RO	N/A	Read this register for the IP version information.
0x0008	LITE_MODE	RO	N/A	Read this register to determine if lite mode is on. This register always returns 1.
0x000C	DEBUG_ENABLED	RO	N/A	Read this register to determine if Debug features are on. This register returns 0 for off and 1 for on.
0x0010	BPS_IN	RO	N/A	Read this register to determine the bits per symbol for the input data.
0x0014	BPS_OUT	RO	N/A	Read this register to determine the bits per symbol for the output data.
0x0018	NUM_COLOR	RO	N/A	Read this register to determine the number of color planes.
0x001C	PIP	RO	N/A	Read this register to determine the number of pixels in parallel.
0x0020	MAX_WIDTH	RO	N/A	Read this register to determine the maximum supported input field width.
0x0024	MAX_HEIGHT	RO	N/A	Read this register to determine the maximum supported input field height.
0x0028	EQUIDISTANT	RO	N/A	Read this register to determine if Equidistant LUT Entries are on.
0x002C	BITS_LUT	RO	N/A	Read this register to determine the number of maximum bits to address the whole LUT.
continued...				

⁽⁹⁾ Registers are RW only if you also turn on **Debug features**, otherwise they are WO.

Address	Register	Access		Description
		Lite ⁽⁹⁾	Full	
0x0030	BITS_SEG	RO	N/A	Read this register to determine the number of bits allocated for address bits of each LUT segment. Only relevant if Equidistant LUT Entries are off.
0x0034	BITS_STEP	RO	N/A	Read this register to determine the number of bits allocated as a sparsity factor across segments. Only relevant if Equidistant LUT Entries are off.
0x0038	REVERSE_LUT	RO	N/A	Read this register to determine if the LUT entries and their readouts are reversed.
0x003C to 0x011F	-	-	-	Reserved
Control, debug, and statistics registers				
0x0120	IMG_INFO_WIDTH	RW	N/A	The expected width of the incoming video fields.
0x0124	IMG_INFO_HEIGHT	RW	N/A	The expected height of the incoming video fields.
0x0128 to 0x013F	-	-	-	Reserved
0x0140	STATUS	RO	N/A	Read this register for information about the 1D LUT IP status.
0x0144	FRAME_STATS	RO	N/A	Read this register for some frame statistics.
0x0148	-	-	-	Reserved
0x014c	CONTROL	RW	N/A	Control bits and fields of 1D LUT IP
0x0150 to 0x01FF	-	-	-	Reserved
0x0200 to 0x0200 + 4 x $2^{\text{BITS_LUT}} - 1$	CP0_LUT[i] ⁽¹⁰⁾	WO	N/A	LUT entries for color plane 1. [BPS_IN-1:0]: Lower lookup value [2xBPS_IN-1:BPS_IN]: Upper lookup value [31:2xBPS_IN]: Reserved
0x0200 + 4 x $2^{\text{BITS_LUT}}$ to 0x0200 + 8 x $2^{\text{BITS_LUT}} - 1$	CP1_LUT[i] ⁽¹⁰⁾	WO	N/A	LUT entries for color plane 2. [BPS_IN-1:0]: Lower lookup value [2xBPS_IN-1:BPS_IN]: Upper lookup value [31:2xBPS_IN]: Reserved These registers are reserved or not implemented if the number of color planes is smaller than 2. Reserved if the address is within the range of the memory-mapped control interface, otherwise not implemented.
0x0200 + 8 x $2^{\text{BITS_LUT}}$ to 0x0200 + 12 x $2^{\text{BITS_LUT}} - 1$	CP2_LUT[i] ⁽¹⁰⁾	WO	N/A	LUT entries for color plane 3. [BPS_IN-1:0]: Lower lookup value [2xBPS_IN-1:BPS_IN]: Upper lookup value [31:2xBPS_IN]: Reserved

continued...

⁽⁹⁾ Registers are RW only if you also turn on **Debug features**, otherwise they are WO.

Address	Register	Access		Description
		Lite ⁽⁹⁾	Full	
				These registers are reserved or not implemented if the number of color planes is smaller than 3. Reserved if the address is within the range of the memory-mapped control interface, otherwise not implemented.
0x0200 + 12 x $2^{\text{BITS_LUT}}$ to 0x0200 + 16 x $2^{\text{BITS_LUT}} - 1$	CP3_LUT[i] ⁽¹⁰⁾	WO	N/A	LUT entries for color plane 4. [BPS_IN-1:0]: Lower lookup value [2xBPS_IN-1:BPS_IN]: Upper lookup value [31:2xBPS_IN]: Reserved These registers are reserved or not implemented if the number of color planes is smaller than 4. Reserved if the address is within the range of the memory-mapped control interface, otherwise not implemented.
0x0200 + 16 x $2^{\text{BITS_LUT}}$ to end of memory	–	–	–	Reserved or not implemented.

Register Bit Descriptions

Table 37. STATUS

Name	Bits	Description
Reserved	31:1	Reserved.
Running	0	When 1, the IP is processing data.

Table 38. FRAME_STATS

Name	Bits	Description
Reserved	31:8	Reserved.
Checksum	7:0	A simple checksum of the frame.

Table 39. CONTROL

Name	Bits	Description
Reserved	31:1	Reserved
Bypass	0	Set to bypass 1D LUT. When set, 1D LUT passes pixel values unprocessed.

8.5. 1D LUT IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

⁽⁹⁾ Registers are RW only if you also turn on **Debug features**, otherwise they are WO.

⁽¹⁰⁾ Index *i* from 0 to $2^{\text{BITS_LUT}} - 1$

Register definition header file: intel_vvp_1d_lut_regs.h

Include file: intel_vvp_1d_lut.h

Table 40. 1D-LUT IP Software API

Name	Description
intel_vvp_1d_lut_init	Initialize an IP instance
intel_vvp_core_*	Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i>
intel_vvp_1d_lut_get_lite_mode	Returns if Lite mode is on
intel_vvp_1d_lut_get_debug_enabled	Returns if Debug features is on
intel_vvp_1d_lut_get_bits_per_sample_in	Returns the number of bits per color sample for the streaming input interface
intel_vvp_1d_lut_get_bits_per_sample_out	Returns the number of bits per color sample value for the streaming output interface
intel_vvp_1d_lut_get_num_color_planes_in	Returns the number of color planes of the streaming input interface
intel_vvp_1d_lut_get_num_color_planes_out	Returns the number of color planes of the streaming output interface
intel_vvp_1d_lut_get_pixels_in_parallel	Returns the pixels in parallel of the streaming input and output interfaces
intel_vvp_1d_lut_get_max_width	Returns the maximum number of pixels that the IP supports on the horizontal dimension of an image or video frame
intel_vvp_1d_lut_get_max_height	Returns the maximum number of pixels that the IP supports on the vertical dimension of an image or video frame
intel_vvp_1d_lut_get_equidistant	Returns if equidistant lookup table mode is enabled
intel_vvp_1d_lut_get_bits_lut	Returns the number of bits required to address all lookup table entries
intel_vvp_1d_lut_get_bits_seg	Returns the number of bits required to address one segment of the lookup table
intel_vvp_1d_lut_get_bits_step	Returns if the IP is running. Returns the number of bits that represent the sparsity change across neighboring segments
intel_vvp_1d_lut_get_reverse_lut	Returns if the reverse lookup table mode is enabled
intel_vvp_1d_lut_is_running	Reads if the IP is running
intel_vvp_1d_lut_get_status	Reads the status register
intel_vvp_1d_lut_get_frame_stats	Reads the frame statistics register
intel_vvp_1d_lut_get_bypass	Reads the bypass bit from the status register
intel_vvp_1d_lut_set_bypass	Writes the bypass bit of the status register
intel_vvp_1d_lut_write_data	Write a single value to the lookup table
intel_vvp_1d_lut_write_data_array	Writes all entries of the lookup table from an array of values

intel_vvp_1d_lut_init

Prototype

```
int
intel_vvp_1d_lut_init(intel_vvp_1d_lut_instance* instance,
intel_vvp_core_base base);
```

Description Initialize an IP instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

base - base address of the register map

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7)

kIntelVvpCorePidErr if the product_id does not match with the expected product ID (0x017D)

kIntelVvp1dLutRegMapVersionErr if the register map is not supported

intel_vvp_1d_lut_get_lite_mode

Prototype

```
bool
intel_vvp_1d_lut_get_lite_mode(intel_vvp_1d_lut_instance*
instance);
```

Description Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

Returns true if the IP is parameterized in lite mode

intel_vvp_1d_lut_get_debug_enabled

Prototype

```
bool  
intel_vvp_1d_lut_get_debug_enabled(intel_vvp_1d_lut_instance*  
instance);
```

Description Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

Returns true if the IP is parameterized with debug features enabled

intel_vvp_1d_lut_get_bits_per_sample_in

Prototype

```
uint8_t  
intel_vvp_1d_lut_get_bits_per_sample_in(intel_vvp_1d_lut_instance*  
instance);
```

Description Returns the value of the BPS_IN register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

Returns the input bits per color sample parameter used to generate the IP

intel_vvp_1d_lut_get_bits_per_sample_out

Prototype

```
uint8_t  
intel_vvp_1d_lut_get_bits_per_sample_out(intel_vvp_1d_lut_instance*  
instance);
```

Description Returns the value of the BPS_OUT register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

Returns the output bits per color sample parameter used to generate the IP

intel_vvp_1d_lut_get_num_color_planes_in

Prototype

```
uint8_t
intel_vvp_1d_lut_get_num_color_planes_in(intel_vvp_1d_lut_instance*
instance);
```

Description Returns the value of the NUM_COLOR_IN register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

Returns the input number of color planes parameter used to generate the IP

intel_vvp_1d_lut_get_num_color_planes_out

Prototype

```
uint8_t
intel_vvp_1d_lut_get_num_color_planes_out(intel_vvp_1d_lut_instance*
instance);
```

Description Returns the value of the NUM_COLOR_OUT register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

Returns the output number of color planes parameter used to generate the IP

intel_vvp_1d_lut_get_pixels_in_parallel

Prototype

```
uint8_t
intel_vvp_1d_lut_get_pixels_in_parallel(intel_vvp_1d_lut_instance*
instance);
```

Description Returns the value of the PIXELS_IN_PARALLEL register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

Returns the pixels in parallel parameter used to generate the IP

intel_vvp_1d_lut_get_max_width

Prototype

```
uint32_t  
intel_vvp_1d_lut_get_max_width(intel_vvp_1d_lut_instance*  
instance);
```

Description Returns the value of the MAX_WIDTH register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

Returns the maximum field width parameter used to generate the IP

intel_vvp_1d_lut_get_max_height

Prototype

```
uint32_t  
intel_vvp_1d_lut_get_max_height(intel_vvp_1d_lut_instance*  
instance);
```

Description Returns the value of the MAX_HEIGHT register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

Returns the maximum field height parameter used to generate the IP

intel_vvp_1d_lut_get_equidistant

Prototype

```
bool  
intel_vvp_1d_lut_get_equidistant(intel_vvp_1d_lut_instance*  
instance);
```

Description Returns the value of the EQUIDISTANT register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

Returns the equidistant parameter used to generate the IP

intel_vvp_1d_lut_get_bits_lut

Prototype

```
uint8_t
intel_vvp_1d_lut_get_bits_lut(intel_vvp_1d_lut_instance*
instance);
```

Description Returns the value of the BITS_LUT register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

Returns the number of lookup table address bits parameter used to generate the IP

intel_vvp_1d_lut_get_bits_seg

Prototype

```
uint8_t
intel_vvp_1d_lut_get_bits_seg(intel_vvp_1d_lut_instance*
instance);
```

Description Returns the value of the BITS_SEG register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

Returns the number of lookup table segment address bits parameter used to generate the IP

intel_vvp_1d_lut_get_bits_step

Prototype

```
uint8_t
intel_vvp_1d_lut_get_bits_step(intel_vvp_1d_lut_instance*
instance);
```

Description Returns the value of the BITS_STEP register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

Returns the number of step bits parameter used to generate the IP

intel_vvp_1d_lut_get_reverse_lut

Prototype

```
bool  
intel_vvp_1d_lut_get_reverse_lut(intel_vvp_1d_lut_instance*  
instance);
```

Description Returns the value of the REVERSE_LUT register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

Returns the reverse lookup table parameter used to generate the IP

intel_vvp_1d_lut_is_running

Prototype

```
bool  
intel_vvp_1d_lut_is_running(intel_vvp_1d_lut_instance*  
instance);
```

Description Returns the value of the IP Running bit from the STATUS register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

Returns the running bit from the status register

intel_vvp_1d_lut_get_status

Prototype

```
uint32_t  
intel_vvp_1d_lut_get_status(intel_vvp_1d_lut_instance*  
instance);
```

Description Returns the value of the STATUS register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

Returns the value of the status register

intel_vvp_1d_lut_get_frame_stats

Prototype

```
int
intel_vvp_1d_lut_get_frame_stats(intel_vvp_1d_lut_instance*
instance, uint32_t* stats_out);
```

Description Reads the value of the FRAME_STATS register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

stats_out - pointer of a variable to return the statistics register value

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvp1dLutPointerErr if stats_out is a null pointer

intel_vvp_1d_lut_get_bypass

Prototype

```
bool
intel_vvp_1d_lut_get_bypass(intel_vvp_1d_lut_instance*
instance);
```

Description Returns the value of the bypass bit from the CONTROL register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

Returns the value of the bypass bit form the control register

intel_vvp_1d_lut_set_bypass

Prototype

```
int
intel_vvp_1d_lut_set_bypass(intel_vvp_1d_lut_instance*
instance, bool bypass);
```

Description Writes to the bypass bit of the CONTROL register. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

bypass - new value of the bypass bit

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

intel_vvp_1d_lut_write_data

Prototype

```
int
intel_vvp_1d_lut_write_data(intel_vvp_1d_lut_instance*
instance, uint32_t data,
uint32_t addr);
```

Description Writes into a single entry of the lookup table. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

data - value of the lookup table entry

addr - address of the lookup table entry

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvp1dLutOutOfBoundsErr if the address is out of range

intel_vvp_1d_lut_write_data_array

Prototype

```
int
intel_vvp_1d_lut_write_data_array(intel_vvp_1d_lut_instance*
instance,
const uint32_t* data_array, uint32_t
length);
```

Description Writes all entries of the lookup table from an array of values. The instance must be a valid intel_vvp_1d_lut_instance fully initialized.

Arguments instance - pointer to the intel_vvp_1d_lut_instance software driver instance structure

data_array - pointer of the array containing all lookup table entries

length - the length of the data_array

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

 kIntelVvpCoreInstanceErr if the instance is a null pointer

 kIntelVvp1dLutPointerErr if data_array is a null pointer

 kIntelVvp1dLutOutOfBoundsErr if the length is longer than the lookup table size

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

9. 3D LUT IP

9.1. About the 3D LUT IP

The IP maps a video stream's color space to another using interpolated values from a lookup table where you can build in or dynamically program the values.

You can configure the 3D LUT's size, bits per color and pixels in parallel. You can enable double buffering, add an output alpha channel, and initialize the LUT from a built-in file.

Typical applications include:

- Color space conversion
- Chroma keying
- Dynamic range conversion
- Artistic effects (e.g. sepia, hue rotation, color volume adjustment)

The IP only supports lite variants. For more information about full and lite variants, refer to the *Altera Streaming Video Protocol Specification*

Related Information

[Altera Streaming Video Protocol Specification](#)

9.1.1. 3D LUT IP Features

- Avalon memory-mapped CPU interface for control and LUT upload
- LUT sizes of 9^3 , 17^3 , 33^3 , and 65^3
- Tetrahedral interpolation
- Range of 8 to 16 bits per color
- Independent parameters for input, output, and LUT bits per color
- Up to 8 pixels in parallel
- Dynamic update of LUT via CPU interface
- Double buffered LUT option allows for seamless run-time switching
- Optional output alpha channel
- Subframe fixed latency
- Very small ALM footprint ($\sim 2K$ ALMs @ 2 pixels in parallel)

9.1.2. 3D LUT IP Performance IP and Resource Information

Altera provides resource and utilization data for guidance.

Table 41. 3D LUT Performance and Resource Usage

The numbers are for a design targeting an Arria 10 device with a design f_{MAX} of 300 MHz.

Pixel in parallel	Bits per color sample	LUT Size	Double buffer	ALMs	Memory (M20K)	DSP Blocks
1	8	17	no	810	17	6
1	8	17	yes	937	25	6
2	10	17	no	1,640	33	12
2	10	17	yes	1,681	49	12
2	10	33	no	1,649	120	12
2	10	33	yes	1,685	224	12
2	10	65	no	2,575	830	12
2	10	65	yes	4,035	1,622	12

9.2. 3D LUT IP Parameters

The 3D LUT IP offers compile-time parameters.

Table 42. 3D LUT IP compile-time parameters

Name	Values	Description
Video data format		
Number of pixels in parallel	1 to 8	Number of pixels transmitted every clock cycle.
Input bits per color sample	8 to 16	Number of bits per color sample at the input.
Output bits per color sample	8 to 16	Number of bits per color sample at the output.
Control settings		
Separate clock for control interface	On or off	Turn on to run the run-time control interface on a different clock domain.
LUT read interface	On or off	Allows you to read LUT contents via the CPU interface.
Enabled out of reset	On or off	- Turn on to enable the LUT out of reset - Turn off to bypass the LUT out of reset
LUT settings		
Size	9, 17, 33, 65	Size of each LUT dimension.
Bits per color	8 to 16	The number of bits per color in the LUT (<code>LUT_DEPTH</code>).
Output alpha channel	On or off	Turn on to add alpha channel to the LUT (RGBA).
Bypass alpha	0 to $2^{LUT_DEPTH}-1$	The alpha value output in bypass mode.
Double buffered	On or off	Double the memory for seamless LUT programming and switching: - On to instantiate the second buffer - Off for single buffer only
Buffer 0 and Buffer 1		
<i>continued...</i>		

Name	Values	Description
Initialize from file	On or off	Turn on to initialize the LUT from the initialization file.
Init file	user file	Optional initialization file.
Init file type	normalized, integer	Type of coefficients in the initialization file: - Normalized floating- or fixed-point numbers between 0.0 and 1.0 - Integer numbers between 0 and $2^{\text{LUT_DEPTH}-1}$

Figure 14. 3D LUT GUI

LUT Initialization File

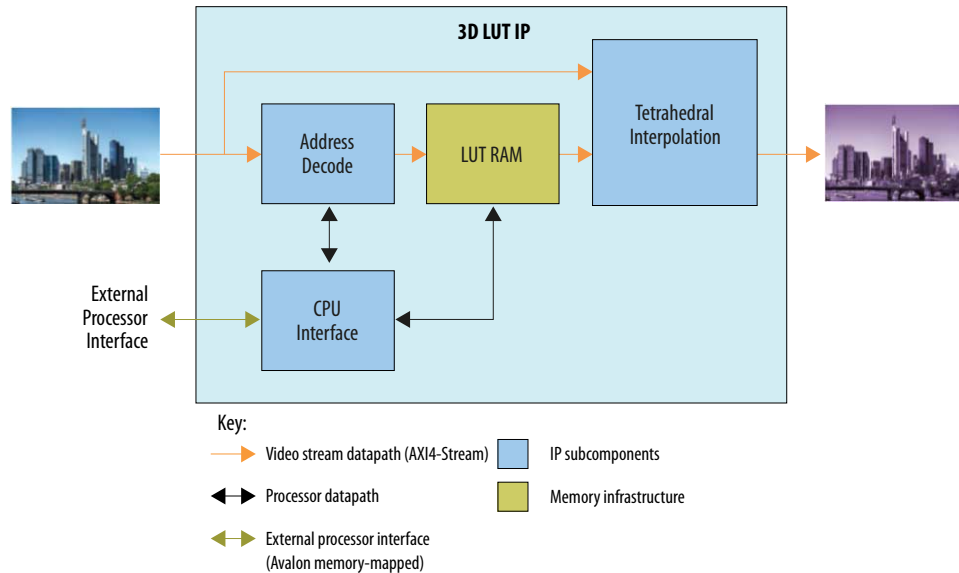
You can initialize each buffer of the LUT from reset by providing a compatible 3D LUT file to **Init file** in the GUI. The IP generation process converts the LUT file into RAM initialization `.hex` files that get built into the firmware during compilation. The script can read `.cube` format files, or any 3D LUT files that follow these conventions:

- RGB component order (must match the video stream's order)
- Components change first from left to right, i.e. R first, G second, B third
- If you turn on alpha, you append the alpha value as a fourth component (RGBA)
- The data type must match the IP GUI parameter and may either be:
 - ☐ normalized fixed- or floating-point numbers between 0.0 to 1.0
 - ☐ integers between 0 and $2^{\text{LUT_DEPTH}-1}$ (e.g. 10-bit: 0 to 1023)
- The data type must be the same for the whole file
- Lines starting with `#` or any letter are ignored

9.3. 3D LUT IP Block Description

The 3D LUT IP accepts RGB-format video input from its Altera video streaming interface. It uses the most significant bits (MSBs) of the 3 color component inputs to retrieve data values from the contents of the LUT and the least significant bits (LSBs) to interpolate the final output value. An Avalon Memory-Mapped compatible CPU interface handles the run-time control and LUT programming.

Figure 15. 3D LUT IP block diagram



The address decoder converts the MSBs of the three input color components into read addresses for the LUT. If you turn on **Double buffered**, the IP adds a page offset to the address when selecting the second buffer via the CPU interface. Page-flip double buffering allows for instantaneous switching between LUTs.

The LUT RAM instantiates the on-chip memory containing the LUT. The 3D LUT cube vertices are divided across eight sub-RAMs to output the target sub-cube vertices in parallel. Enabling the second buffer doubles the memory depth of the LUT. Both buffers' contents are programmable via the CPU interface and can also be pre-initialized in the firmware via the 3D LUT IP GUI.

The tetrahedral interpolator uses a DSP-efficient method to interpolate four of the LUT subcube vertices using the input LSBs. Part of the input MSBs determines which of the six tetrahedra in the target sub-cube contains the pixel.

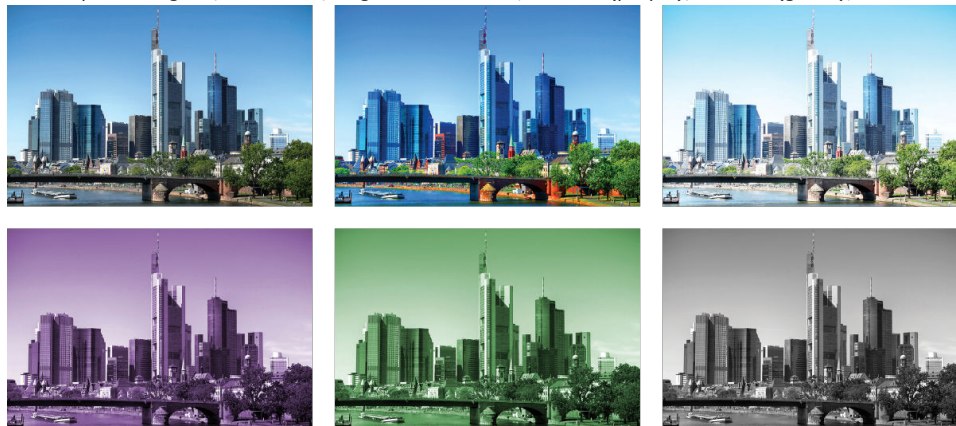
The control register in the run-time control register map allows you to switch between the interpolated output and the bypass output.

Consider these points when integrating into a streaming video pipeline:

- The IP controls buffer selection and output enable and only updates them at the start of each new frame.
- The internal pipeline forwards control signals and is unaffected by changes to video resolution.

Figure 16. 3D LUT color transform examples

From top left: original, saturation, brightness increase, colorize (purple), colorize (green), desaturation



9.3.1. 3D LUT IP Interfaces

The IP has three functional interfaces.

The interfaces are:

- Altera video stream input interface
- Altera video stream output interface
- Avalon Memory-Mapped compatible CPU interface

The 3D LUT IP control interface uses Avalon Memory-Mapped protocol to access control and RAM interface registers.

Clocks

The 3D LUT IP has two clock domains, each with a corresponding reset signal.

Table 43. Clock domains

Clock name	Description
cpu_clock	CPU interface clock domain
vid_clock	Video processing clock domain

The CPU interface uses little bandwidth and therefore does not impose a minimum clock frequency. The video clock frequency depends on the video resolution, frame rate, and the 3D LUT IP's number of pixels in parallel. For example, a 300 MHz clock at 2 pixels in parallel supports active video resolutions up to 4096x2160 at 60 Hz.

All RTL-based blocks that transfer or receive data from a different clock domain include clock domain crossing (CDC) circuits for both, single bit and data bus signal cases. The CDC safely allows exchange of data between the two asynchronous clock domains. This principle applies to the control signals from the CPU interface to the main video datapath. The 3D LUT IP includes a .sdc file to constrain this CDC.

Resets

Table 44. Resets associated with clock domains

Both resets are synchronous active-high

Reset name	Description
cpu_reset	CPU interface clock domain reset.
vid_reset	Video processing clock domain reset.

Related Information

Video and Vision Processing IP Interfaces on page 30

9.3.2. 3D LUT IP Latency

The latency information can predict the approximate latency between the input and the output of your video processing pipeline.

Table 45. 3D LUT IP operation mode latency

The table shows latency as a number of valid clock cycles. Altera measures the latency assuming that other functions on the datapath are not stalling the IP, i.e. the output ready signal is high.

Device	Latency (cycles)
Arria 10 Cyclone 10 GX	21
Agilex 7 Stratix 10	22

9.4. 3D LUT IP Registers

The 3D LUT IP allows run-time control and LUT programming via the CPU interface.

The register map provides access to:

- Build parameters such as LUT size and bits per color.
- A control interface that allows you to enable or bypass the LUT. Also, it allows you to toggle buffers when you turn on **Double buffered** for the LUT.
- A RAM interface that allows programming of the LUT's 8 sub-RAMs during run time and reading their contents if you turn on **LUT read interface**.

Table 46. 3D LUT IP register map

Unless stated, all the registers are 32-bit wide.

Register Name	Byte Address Offset	Access	Description
vid_pid	0x000	RO	Vendor ID and Product ID
version_number	0x004	RO	Version number
-	0x008	RO	Reserved
pixels_in_parallel	0x00C	RO	Video data format Number of pixels in parallel parameter
input_bps	0x010	RO	Video data format Input bits per color sample parameter
continued...			

Register Name	Byte Address Offset	Access	Description
output_bps	0x014	RO	Video data format Output bits per color sample parameter
lut_alpha	0x018	RO	LUT settings Output alpha channel parameter
lut_depth	0x01C	RO	LUT settings Bits per color parameter
lut_dimension	0x020	RO	LUT settings Size parameter
lut_double_buffered	0x024	RO	LUT settings Double buffered parameter
lut_cpu_readable	0x028	RO	Control settings LUT read interface parameter
-	0x02C - 0x147	RO	Reserved
Control	0x148	RW	Control interface: enable and buffer select
	0x14C - 0x17F	RO	Reserved
RAM n Control	0x180 + 0x10*n	RW	RAM n interface: address and write enable
	0x184 + 0x10*n	RW	Reserved
RAM n Data Lower	0x188 + 0x10*n	RW	RAM n interface: data, lower 32 bits
RAM n Data Upper	0x18C + 0x10*n	RW	RAM n interface: data, upper 32 bits (if applicable)

Table 47. vid_pid Register

Name	Bits	Description
PID	15:0	3D LUT Product ID: 0x0165
VID	31:16	Altera Vendor ID: 0x6AF7

Table 48. version_number Register

Name	Bits	Description
Minor	15:0	Minor version number for this release of the 3D LUT IP
Major	31:16	Major version number for this release of the 3D LUT IP

Table 49. pixels_in_parallel Register

Name	Bits	Description
Pixels in Parallel	31:0	Video data format Number of pixels in parallel parameter

Table 50. input_bps Register

Name	Bits	Description
Input BPS	31:0	Video data format Input bits per color sample parameter

Table 51. output_bps Register

Name	Bits	Description
Output BPS	31:0	Video data format Output bits per color sample parameter

Table 52. lut_alpha Register

Name	Bits	Description
LUT alpha	31:0	LUT settings Output alpha channel parameter

Table 53. lut_depth Register

Name	Bits	Description
LUT depth	31:0	LUT settings Bits per color parameter

Table 54. lut_dimension Register

Name	Bits	Description
LUT dimension	31:0	LUT settings Size parameter

Table 55. lut_double_buffered Register

Name	Bits	Description
LUT double buffered	31:0	LUT settings Double buffered parameter

Table 56. lut_cpu_readable Register

Name	Bits	Description
LUT CPU readable	31:0	Control settings LUT read interface parameter

Table 57. Control Register

Name	Bits	Description
Enable	0	0: bypass 1: enable LUT
Buffer select	1	0: buffer 0 1: buffer 1 (if Double buffered is enabled)
	31:2	Reserved

Table 58. RAM n Control Register

Name	Bits	Description
Address	16:0	RAM <i>n</i> address to write data to or read data from
	27:17	Reserved
Write enable	28	Write enable (clears to 0 automatically)
	31:29	Reserved

Table 59. RAM n Data Lower Register

Name	Bits	Description
Data	31:0	LUT data, lower 32 bits Write access: first write the new LUT entry data, then set the target address with the write enable asserted in RAM <i>n</i> Control Read access: if you turn on LUT read interface, retrieve the data from RAM <i>n</i> at the address set in RAM <i>n</i> Control

Table 60. RAM *n* Data Upper Register

Name	Bits	Description
Data	31:0	LUT data, upper 32 bits Only present when the LUT data width is greater than 32, i.e.: (lut_alpha + 3) * lut_depth > 32

9.5. 3D LUT IP Software API

The IP includes a software driver that configures and controls all the necessary parameters of the IP.

Figure 17. Software driver usage example

```
intel_vvp_3d_lut_instance lut_0;
ret = intel_vvp_3d_lut_init(&lut_0, (intel_vvp_core_base)LUT_0_BASE);
if (ret == 0)
{
    /* Load LUT buffer 0 by software */
    if (load_3d_lut(&lut_0, 0) == 0)
    {
        /* Enable LUT processing */
        intel_vvp_3d_lut_enable(&lut_0, true);
    }
    else
    {
        printf("Error loading LUT data: %d\n");
    }
}
else
{
    printf("Error initializing lut_0: %d\n", ret);
}
```

The driver does not include the load function. You have alternative ways to source the LUT entry data. The simplest is from a precompiled structure in the software source code. An example of using this method is:

```
int load_3d_lut(intel_vvp_3d_lut_instance* instance, uint8_t buffer)
{
    uint16_t r_idx = 0, g_idx = 0, b_idx = 0;
    uint32_t table_idx = 0;
    while (table_idx < ((sizeof(lut_table)/sizeof(uint16_t)) - 4))
    {
        int result = intel_vvp_3d_lut_load(instance, r_idx, g_idx, b_idx, buffer,
            lut_table[table_idx],
            lut_table[table_idx + 1],
            lut_table[table_idx + 2],
            lut_table[table_idx + 3]);
        if (result != 0)
        {
            return result;
        }
        table_idx += 4;
        if (++r_idx == LUT_TABLE_DIMENSION)
        {
            r_idx = 0;
            if (++g_idx == LUT_TABLE_DIMENSION)
            {
                g_idx = 0;
                if (++b_idx == LUT_TABLE_DIMENSION)
                {
                    break;
                }
            }
        }
    }
}
```

```

    }
  }
  return 0;
}

```

In this example, the data is a flat structure containing four elements per LUT entry ($4 * \text{lut_dimension}^3$).

The definition for the dimension and data table is:

```

#define LUT_TABLE_DIMENSION 17
const uint16_t lut_table[] = {
0x0000, 0x0000, 0x0000, 0x0000,
0x0000, 0x0000, 0x0000, 0x0000,
0x0000, 0x0000, 0x0000, 0x0000,
0x0000, 0x0000, 0x0000, 0x0000,
0x0000, 0x0000, 0x0000, 0x0000,
0x0000, 0x0000, 0x0000, 0x0000,
0x0000, 0x0000, 0x0000, 0x0000,
0x0000, 0x0000, 0x0000, 0x0000,
0x0000, 0x0000, 0x0000, 0x0000,
0x0000, 0x0000, 0x0000, 0x0000,
0x0000, 0x0000, 0x0000, 0x0000,
0x0000, 0x0000, 0x0000, 0x0000,
0x0000, 0x0000, 0x0000, 0x0000,
0x0000, 0x0000, 0x0000, 0x0000,
0x0050, 0x0050, 0x0050, 0x0000,
0x0079, 0x0079, 0x0079, 0x0000,
0x0092, 0x0092, 0x0092, 0x0000,
0x00A6, 0x00A6, 0x00A6, 0x0000,
0x00B9, 0x00B9, 0x00B9, 0x0000,
/* continues... 4913 lines total */

```

Table 61. 3D LUT IP API reference

The software driver for 3D LUT IP provides the following API functions.

Name	Description
intel_vvp_3d_lut_init	Initialize the LUT instance
intel_vvp_3d_lut_enable	Enable LUT processing
intel_vvp_3d_lut_buffer_select	Select between LUT buffers
intel_vvp_3d_lut_load	Load a LUT entry
intel_vvp_3d_lut_get_double_buffered	Get double buffered configuration parameter
intel_vvp_3d_lut_get_input_depth	Get bit resolution of input streams
intel_vvp_3d_lut_get_lut_alpha_channel	Get alpha channel support configuration parameter"
intel_vvp_3d_lut_get_lut_depth	Get bit resolution of LUT streams
intel_vvp_3d_lut_get_dimension	Get size of LUT
intel_vvp_3d_lut_get_output_depth	Get bit resolution of output streams
intel_vvp_3d_lut_get_pixels_per_clock	Get number of pixels processed per clock cycle

intel_vvp_3d_lut_init

```

void
intel_vvp_3d_lut_init( intel_vvp_3d_lut_instance*
instance, intel_vvp_core_base base);

```

<i>Description</i>	Initialize a 3D LUT instance
<i>Arguments</i>	instance – pointer to the 3D LUT software driver instance structure base – pointer to base address of 3D LUT IP
<i>Return Value</i>	Zero on success, negative integer otherwise

intel_vvp_3d_lut_enable

```
void  
intel_vvp_3d_lut_enable( intel_vvp_3d_lut_instance*  
instance, int enable);
```

<i>Description</i>	Enable LUT processing
<i>Arguments</i>	instance – pointer to the 3D LUT software driver instance structure enable – enable LUT operation: • 0 – Passthrough input stream unchanged • 1 – Enable LUT processing
<i>Return Value</i>	None

intel_vvp_3d_lut_buffer_select

```
int  
intel_vvp_3d_lut_buffer_select( intel_vvp_3d_lut_instance*  
instance, uint8_t buffer);
```

<i>Description</i>	Select between LUT procession buffers (double buffering must be enabled)
<i>Arguments</i>	instance – pointer to the 3D LUT software driver instance structure buffer – buffer to be selected (0 or 1)
<i>Return Value</i>	• 0 – operation is successful • -1 – buffer parameter is out of range or double buffering is not configured

intel_vvp_3d_lut_load

```
int  
intel_vvp_3d_lut_load( intel_vvp_3d_lut_instance*  
instance, uint16_t r_idx, uint16_t g_idx, uint16_t  
b_idx, uint8_t buffer, uint16_t r_val, uint16_t  
g_val, uint16_t b_val, uint16_t a_val);
```

<i>Description</i>	Load an entry into the LUT table. Parameters specify the indices for the table, and the R/G/B/A value for the table entry.
<i>Arguments</i>	instance – pointer to the 3D LUT software driver instance structure r_idx – red index. Range 0 to (LUT dimension - 1) g_idx – green index. Range 0 to (LUT dimension - 1) b_idx – blue index. Range 0 to (LUT dimension - 1) buffer – range 0 to 1 (for double buffered configuration) r_val – red value. Range 0 to ($2^{\text{lut_depth}} - 1$) g_val – green value. Range 0 to ($2^{\text{lut_depth}} - 1$) b_val – blue value. Range 0 to ($2^{\text{lut_depth}} - 1$) a_val – alpha value. LUT alpha must be enabled. If not, value must be set to 0.
<i>Return Value</i>	• 0 - successful • -1 if r_idx/g_idx/b_idx is out of range • -2 if buffer parameter is out of range, or double buffering is not configured • -3 if alpha value is set and not supported • -4 if r_val/g_val/b_val is out of range

intel_vvp_3d_lut_get_double_buffered

```
uint8_t
intel_vvp_3d_lut_get_double_buffered( intel_vvp_3d_lut_instance*
instance);
```

<i>Description</i>	Get double-buffered IP configuration
<i>Arguments</i>	instance – pointer to the 3D LUT software driver instance structure
<i>Return Value</i>	• 0 if double buffer option is not configured • 1 if double buffer option is configured

intel_vvp_3d_lut_get_input_depth

```
uint8_t
intel_vvp_3d_lut_get_input_depth( intel_vvp_3d_lut_instance*
instance);
```

<i>Description</i>	Get bit resolution of input streams
--------------------	-------------------------------------

Arguments instance – pointer to the 3D LUT software driver instance structure

Return Value Range is 8 to 16. Value is number of bits per input color plane

intel_vvp_3d_lut_get_lut_alpha_channel

```
uint8_t  
intel_vvp_3d_lut_get_lut_alpha_channel( intel_vvp_3d_lut_instance*  
instance);
```

Description Get alpha channel support configuration parameter

Arguments instance – pointer to the 3D LUT software driver instance structure

Return Value

- 0 if alpha channel is not supported
- 1 if alpha channel is supported

intel_vvp_3d_lut_get_lut_depth

```
uint8_t  
intel_vvp_3d_lut_get_lut_depth( intel_vvp_3d_lut_instance*  
instance);
```

Description Get configured bit resolution of LUT processing streams

Arguments instance – pointer to the 3D LUT software driver instance structure

Return Value Range is 8 to 16. Value is number of bits per LUT color plane

intel_vvp_3d_lut_get_dimension

```
uint8_t  
intel_vvp_3d_lut_get_dimension( intel_vvp_3d_lut_instance*  
instance);
```

Description Get configured LUT size. Value is single dimension size. A dimension size of A gives a LUT size of (A x A x A) entries

Arguments instance – pointer to the 3D LUT software driver instance structure

Return Value Valid values are {9, 17, 33, 65}

intel_vvp_3d_lut_get_output_depth

```
uint8_t
intel_vvp_3d_lut_get_output_depth( intel_vvp_3d_lut_instance*
instance);
```

Description Get bit resolution of LUT output streams

Arguments instance – pointer to the 3D LUT software driver instance structure

Return Value Range is 8 to 16. Value is number of bits per output color plane

intel_vvp_3d_lut_get_pixels_per_clock

```
uint8_t
intel_vvp_3d_lut_get_pixels_per_clock( intel_vvp_3d_lut_instance*
instance);
```

Description Number of input pixels processed for each video clock cycle

Arguments instance – pointer to the 3D LUT software driver instance structure

Return Value Number of pixels. Range is 1 to 8.

10. Adaptive Noise Reduction IP

10.1. About the Adaptive Noise Reduction IP

The Adaptive Noise Reduction IP is an edge preserving smoothing filter for mainly reducing independent pixel noise of a video stream. For one color plane operation the IP can process grayscale or RAW video streams with arbitrary 2x2 color filter array configuration. The IP does not support multiple color plane video streams like RGB.

The IP only supports lite variants. For more information about full and lite variants, refer to the *Altera Streaming Video Protocol Specification*

Related Information

[Altera Streaming Video Protocol Specification](#)

10.1.1. Adaptive Noise Reduction IP Performance and Resources

Altera provides resource and utilization data for guidance only. The IP resources depends on the device family, compilation tool version, compilation seed randomization, and other parameters.

Table 62. Adaptive Noise Reduction IP Performance and Resources

Device	Pixels in Parallel	Enable CFA	Bits per Color Sample	Max Resolution	ALM	M20k	DSP	f _{MAX} (MHz)
Agilex 5 speed grade 6 (A5ED065BB32AE6SR0)	1	1	10	2k	1,826	23	26	463 ⁽¹¹⁾
	1	0	16	2k	3,159	36	50	420 ⁽¹¹⁾
	2	1	10	4k	3,152	46	52	416 ⁽¹¹⁾
	2	0	16	4k	5,599	71	100	406 ⁽¹¹⁾
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	1	1	10	2k	1,979	23	26	755
	1	0	16	2k	3,073	36	50	809
	2	1	10	4k	3,183	46	52	809
	2	0	16	4k	5,615	71	100	762
	4	1	10	8k	5,974	92	104	749
	4	0	16	8k	10,059	142	200	746
Arria 10 speed grade 1 (10AS066H1F34E1HG)	1	1	10	2k	1,334	34	42	455
	1	0	16	2k	2,147	59	82	453

continued...

⁽¹¹⁾ Restricted f_{MAX} 321 MHz rest

Device	Pixels in Parallel	Enable CFA	Bits per Color Sample	Max Resolution	ALM	M20k	DSP	f _{MAX} (MHz)
	2	1	10	4k	2,167	68	84	453
	2	0	16	4k	3,880	119	164	444
Cyclone 10 GX speed grade 5 (10CX220YF672E5G)	1	1	10	2k	1,326	34	42	372
	1	0	16	2k	2,134	59	82	371
Stratix 10 speed grade 1 (1SX280LN2F43E1VG)	1	1	10	2k	2,113	23	42	498
	1	0	16	2k	3,887	36	82	497
	2	1	10	4k	3,593	46	84	498
	2	0	16	4k	6,201	71	164	497

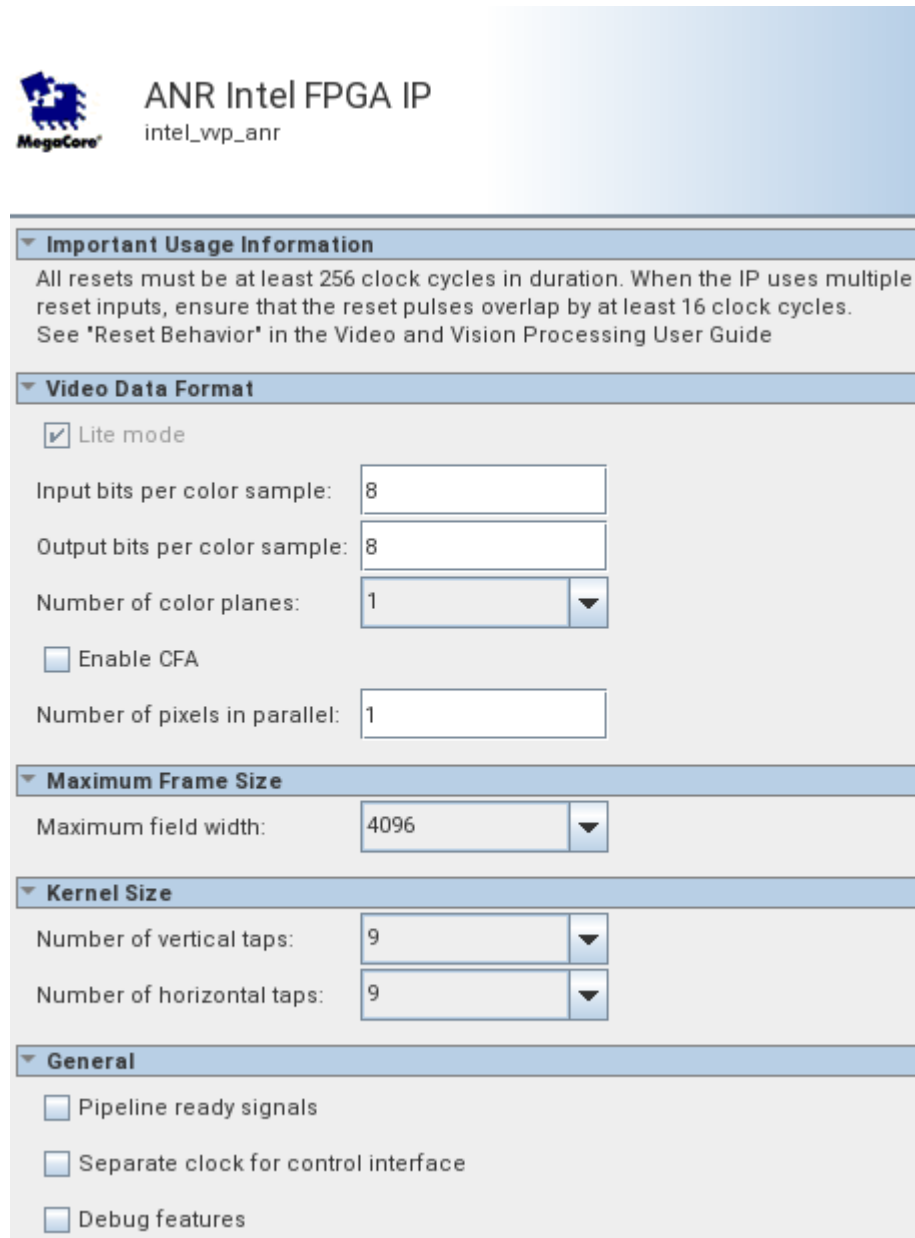
10.2. Adaptive Noise Reduction IP Parameters

The IP offers run-time and compile-time parameters.

Table 63. Adaptive Noise Reduction IP Parameters

Parameter	Values	Description
Video Data Format		
Lite mode	On	The IP only supports lite mode
Input bits per color sample	8 to 16	Select the number of input bits per color sample.
Output bits per color sample	8 to 16	Select the number of output bits per color sample.
Number of color planes	1 to 4	Number of color planes processed per pixel.
Enable CFA	On or off	Turn on 2x2 color filter array pattern. Only applies when the number of color planes is 1.
Number of pixels in parallel	1 to 8	Select the number of pixels in parallel.
Maximum Frame Size		
Maximum field width	2k, 4k, 8k, 16k, 32k, 64k	Specify the maximum width of incoming frames
Kernel Size		
Number of vertical taps	1, 3, 5, 7, 9, 11, 13, 15, 17	Specify the vertical size of the kernel. When enable CFA is on only values 1, 5, 9, 13 and 17 are legal.
Number of horizontal taps	1, 3, 5, 7, 9, 11, 13, 15, 17	Specify the horizontal size of the kernel. When enable CFA is on only values 1, 5, 9, 13 and 17 are legal.
General		
Pipeline ready signal	On or off	Turn on to add extra pipeline registers to the AXI4-S <code>tready</code> signals
Separate clock for control interface	On or off	Turn on for a separate clock for the control agent interface
Debug features	On or off	Turn on for read back of frame info registers and debugging information registers via the control agent interface.

Figure 18. Adaptive Noise Reduction IP GUI



ANR Intel FPGA IP
intel_vwp_anr

Important Usage Information
All resets must be at least 256 clock cycles in duration. When the IP uses multiple reset inputs, ensure that the reset pulses overlap by at least 16 clock cycles. See "Reset Behavior" in the Video and Vision Processing User Guide

Video Data Format

☒ Lite mode

Input bits per color sample: 8

Output bits per color sample: 8

Number of color planes: 1

☐ Enable CFA

Number of pixels in parallel: 1

Maximum Frame Size

Maximum field width: 4096

Kernel Size

Number of vertical taps: 9

Number of horizontal taps: 9

General

☐ Pipeline ready signals

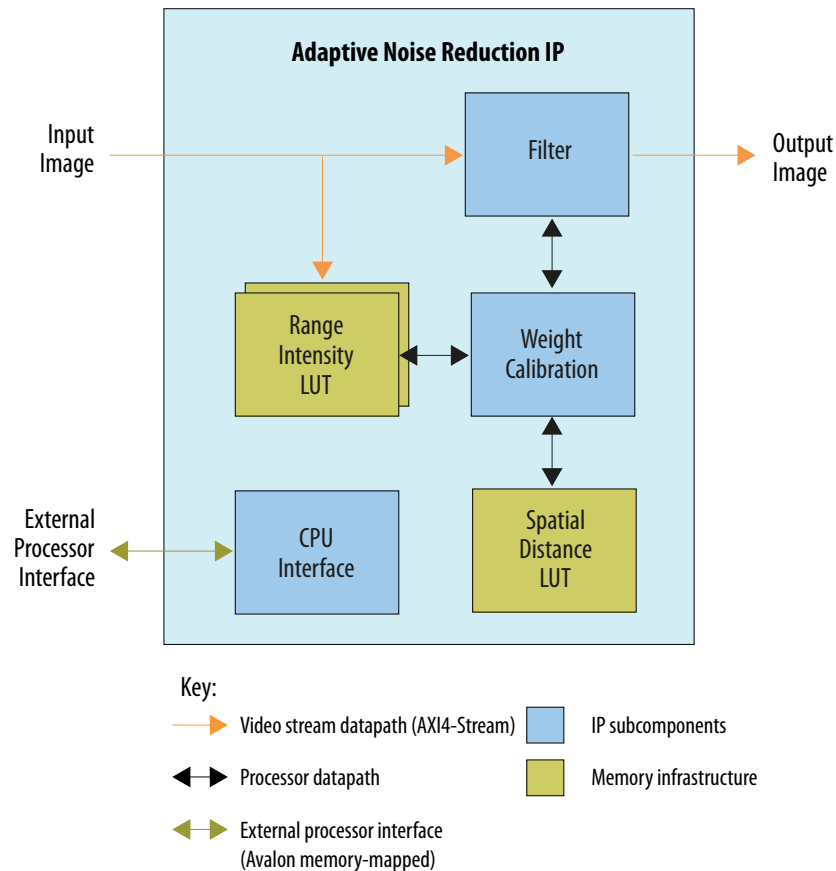
☐ Separate clock for control interface

☐ Debug features

10.3. Adaptive Noise Reduction IP Functional Description

The Adaptive Noise Reduction IP is a spatial weighted averaging filter that analyzes the scene and correlates similar pixels dynamically while simultaneously generating the weights. The IP uses two look-up tables when correlating the pixels, one for correlating pixel intensities, and the other correlating the spatial distance between the pixels.

Figure 19. Adaptive Noise Reduction IP block diagram



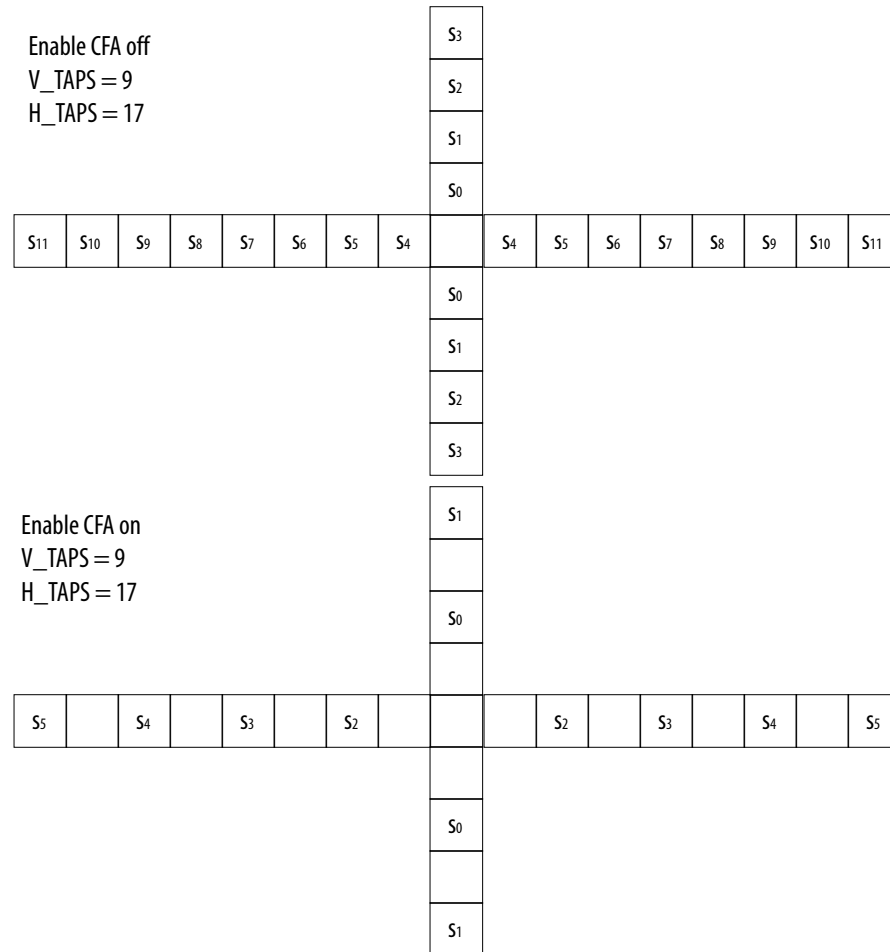
You configure the number of vertical and horizontal taps of the filter independently. Configuring a shorter height reduces the number M20K resources for buffering lines while sacrificing denoising strength.

You calibrate the intensity range lookup table offline by the difference between two video frames with identical content but different temporal noise. You isolate the noise level from the difference and calculate the intensity range look-up table entries. This look-up table allows you to program different denoising strengths across the pixel intensities. For example, you may opt for denoising dark content stronger to reduce shadow noise more aggressively while preserving the details on the mid tones and highlights.

The second look-up table is the spatial distance look-up table that makes the IP more versatile. You program this look-up table to create a weight distribution from the center pixel to the neighboring pixels. You may program a traditional distribution like a Hamming or Hanning window to reduce ringing artifacts. You may also prefer assigning the same value to all entries for a rectangular distribution to maximize denoising capability.

The IP supports 2x2 color filter array images where every other pixel belongs to a different color channel. This parameter changes the spatial distance look up table coefficient locations corresponding to the taps.

Figure 20. Spatial distance LUT entries overlayed on the spatial distance kernel



10.3.1. Adaptive Noise Reduction IP Interfaces

Table 64. Adaptive Noise Reduction IP Clock and Reset Interfaces

Name	Direction	Width	Description
main_clock_clk	In	1	AXI4-S processing clock.
main_reset_rst	In	1	AXI4-S processing reset.
agent_clock_clk	In	1	Clock for the memory-mapped control agent interface. Only if you select separate clock for control interface .
agent_reset_reset	In	1	Reset for the memory-mapped control agent interface. Only if you select separate clock for control interface .

Table 65. Adaptive Noise Reduction IP Control Interfaces

Name	Direction	Width	Description
av_mm_control_agent_address	In	11	Avalon memory-mapped agent address
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write.
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data.
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable.
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read.
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data.
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read.
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request.

Table 66. Adaptive Noise Reduction IP Altera Streaming Video Interfaces

Name	Direction	Width	Description
Input			
axi4s_vid_in_tdata	In	(12)	AXI4-S data in.
axi4s_vid_in_tvalid	In	1	AXI4-S data valid.
axi4s_vid_in_tuser	In	(13)	[0] AXI4-S start of video frame. [N-1:1] Unused.
axi4s_vid_in_tlast	In	1	AXI4-S end of packet.
axi4s_vid_in_tready	Out	1	AXI4-S data ready.
Output			
axi4s_vid_out_tdata	Out	(12)	AXI4-S data out.
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid.
axi4s_vid_out_tuser	Out	(13)	[0] AXI4-S start of video frame. [N-1:1] Unused.
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet.
axi4s_vid_out_tready	In	1	AXI4-S data ready

Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

10.4. Adaptive Noise Reduction IP Registers

Each register is either read-only (RO), read-write (RW) or write-only (WO).

(12) This equation gives all `tdata` widths sizes in these interfaces: pixels in parallel × max (round (bits per color sample × number of color planes, 8), 16)

(13) This equation gives all `tuser` widths sizes in these interfaces: $N = \text{ceil}(\text{tdata width} / 8)$

Table 67. Adaptive Noise Reduction IP Registers

In the software API the register names appear with a prefix of INTEL_VVP, INTEL_VVP_CORE or INTEL_VVP_ANR as appropriate and with an optional REG suffix.

Address	Register	Access		Description
		Lite (14)	Full	
Parameterization registers				
0x0000	VID_PID	RO	N/A	Read this register to retrieve the ID of the IP. This register always returns 0x6FA7_0176.
0x0004	VERSION	RO	N/A	Read this register to retrieve the version information for the IP.
0x0008	LITE_MODE	RO	N/A	Read this register to determine if lite mode is on. This register always returns 1.
0x000C	DEBUG_ENABLED	RO	N/A	Read this register to determine if debug features are on. This register returns 0 for off and 1 for on.
0x0010	BPS_IN	RO	N/A	Read this register to determine the bits per symbol for the input data.
0x0014	BPS_OUT	RO	N/A	Read this register to determine the bits per symbol for the output data.
0x0018	NUM_COLOR	RO	N/A	Read this register to determine the number of color planes.
0x001C	CFA_ENABLED	RO	N/A	Read this register to determine if enable CFA is on.
0x0020	PIP	RO	N/A	Read this register to determine the number of pixels in parallel.
0x0024	MAX_WIDTH	RO	N/A	Read this register to determine the maximum supported input field width.
0x0028	H_TAPS	RO	N/A	Read this register to determine the number of horizontal taps
0x002C	V_TAPS	RO	N/A	Read this register to determine the number of vertical taps
0x0030 to 0x011F	–	-	-	Reserved
Control, debug and statistics registers				
0x0120	IMG_INFO_WIDTH	RW	N/A	The expected width of the incoming video fields.
0x0124	IMG_INFO_HEIGHT	RW	N/A	The expected height of the incoming video fields.
0x0128 to 0x013F	–	-	-	Reserved
0x0140	STATUS	RO	N/A	Read this register for information about the Adaptive Noise Reduction IP status. [0]: Running [31:1]: Reserved
continued...				

⁽¹⁴⁾ Registers are RW only if you also turn on **Debug features**, otherwise they are WO.

Address	Register	Access		Description
		Lite ⁽¹⁴⁾	Full	
0x0144	FRAME_STATS	RO	N/A	Read this register for some frame statistics. [7:0]: A simple checksum of the frame [31:8]: Reserved
0x0148	COMMIT	RW	N/A	Write any value to this register to submit changes to the control register, and intensity range, and spatial distance lookup tables.
0x014c	CONTROL	RW	N/A	Control bits and fields of Adaptive Noise Reduction IP [0]: Bypass bit. When set Adaptive Noise Reduction IP passes input pixel values untouched. [31:1]: Reserved
0x0150 to 0x01FF	–	–	–	Reserved
0x0200 to 0x11FF	INTENSITY_LUT[k] ⁽¹⁵⁾	WO	N/A	Intensity range lookup table entries. [19:0]: Lookup value [31:20]: Reserved
0x1200 to 0x1200 + 4 x SPATIAL_LUT_DEPTH ⁽¹⁶⁾ - 1	SPATIAL_LUT[i] ⁽¹⁷⁾	RW	N/A	Spatial distance lookup table entries. [19:0]: Lookup value [31:20]: Reserved
0x1200 + 4 x SPATIAL_LUT_DEPTH to 0x1FFF	–	–	–	Not implemented

Register Bit Descriptions

Table 68. STATUS

Name	Bits	Description
Reserved	31:1	Reserved.
Running	0	When 1, the IP processes data.

Table 69. FRAME_STATS

Name	Bits	Description
Reserved	31:8	Reserved.
Checksum	7:0	A simple checksum of the frame.

⁽¹⁴⁾ Registers are RW only if you also turn on **Debug features**, otherwise they are WO.

⁽¹⁵⁾ Index k from 0 to 1023

⁽¹⁶⁾ If **Enable CFA** is off $SPATIAL_LUT_DEPTH = (H_TAPS - 1) / 2 + (V_TAPS - 1) / 2$
if **Enable CFA** is on $SPATIAL_LUT_DEPTH = (H_TAPS - 1) / 4 + (V_TAPS - 1) / 4$

⁽¹⁷⁾ Index i from 0 to $SPATIAL_LUT_DEPTH - 1$

Table 70. CONTROL

Name	Bits	Description
Reserved	31:1	Reserved
Bypass	0	Set to bypass adaptive noise reduction. When set, adaptive noise reduction passes pixel values unprocessed.

10.5. Adaptive Noise Reduction IP Software API

Register definition header file: `intel_vvp_anr_regs.h`

Include file: `intel_vvp_anr.h`

Table 71. ANR IP Software API

Name	Description
<code>intel_vvp_anr_init</code>	Initialize an IP instance
<code>intel_vvp_core_*</code>	Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i>
<code>intel_vvp_anr_get_lite_mode</code>	Returns if Lite mode is on
<code>intel_vvp_anr_get_debug_enabled</code>	Returns if Debug features is on
<code>intel_vvp_anr_get_bits_per_sample_in</code>	Returns the number of bits per color sample for the streaming input interface
<code>intel_vvp_anr_get_bits_per_sample_out</code>	Returns the number of bits per color sample value for the streaming output interface
<code>intel_vvp_anr_get_num_color_planes</code>	Returns the number of color planes of the streaming input and output interfaces
<code>intel_vvp_anr_get_cfa_enabled</code>	Returns if Enable CFA is on
<code>intel_vvp_anr_get_pixels_in_parallel</code>	Returns the pixels in parallel of the streaming input and output interfaces
<code>intel_vvp_anr_get_max_width</code>	Returns the maximum number of pixels that the IP supports on the horizontal dimension of an image or video frame
<code>intel_vvp_anr_get_num_h_taps</code>	Returns the number of horizontal filter taps
<code>intel_vvp_anr_get_num_v_taps</code>	Returns the number of vertical filter taps
<code>intel_vvp_anr_is_running</code>	Reads if the IP is running
<code>intel_vvp_anr_commit_is_pending</code>	Reads if the IP has uncommitted writes
<code>intel_vvp_anr_get_status</code>	Reads the status register
<code>intel_vvp_anr_get_frame_stats</code>	Reads the frame statistics register
<code>intel_vvp_anr_commit</code>	Commit pending writes
<code>intel_vvp_anr_get_bypass</code>	Reads the bypass bit from the status register
<code>intel_vvp_anr_set_bypass</code>	Writes the bypass bit of the status register
<i>continued...</i>	

Name	Description
intel_vvp_anr_write_intensity_lut_entry	Writes a single value to the intensity range lookup table
intel_vvp_anr_write_intensity_lut_array	Writes all entries of the intensity range lookup table from an array of values
intel_vvp_anr_write_spatial_lut_entry	Writes a single value to the spatial distance lookup table
intel_vvp_anr_write_spatial_lut_array	Writes all entries of the spatial distance lookup table from an array of values

intel_vvp_anr_init

Prototype

```
int
intel_vvp_anr_init(intel_vvp_anr_instance* instance,
intel_vvp_core_base base);
```

Description Initialize an ip instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure

base - base address of the register map

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7)

kIntelVvpCorePidErr if the product_id does not match with the expected product ID (0x0176)

kIntelVvpAnrRegMapVersionErr if the register map is not supported

intel_vvp_anr_get_lite_mode

Prototype

```
bool
intel_vvp_anr_get_lite_mode(intel_vvp_anr_instance* instance);
```

Description Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_anr_instance fully initialized.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure

Returns true if the IP is in lite mode

intel_vvp_anr_get_debug_enabled

Prototype

```
bool  
intel_vvp_anr_get_debug_enabled(intel_vvp_anr_instance*  
instance);
```

Description Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_anr_instance fully initialized.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure

Returns true if the IP has debug features on

intel_vvp_anr_get_bits_per_sample_in

Prototype

```
uint8_t  
intel_vvp_anr_get_bits_per_sample_in(intel_vvp_anr_instance*  
instance);
```

Description Returns the value of the BPS_IN register. The instance must be a valid intel_vvp_anr_instance fully initialized.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure

Returns the input bits per color sample parameter to generate the IP

intel_vvp_anr_get_bits_per_sample_out

Prototype

```
uint8_t  
intel_vvp_anr_get_bits_per_sample_out(intel_vvp_anr_instance*  
instance);
```

Description Returns the value of the BPS_OUT register. The instance must be a valid intel_vvp_anr_instance fully initialized.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure

Returns the output bits per color sample parameter to generate the IP

intel_vvp_anr_get_num_color_planes

Prototype

```
uint8_t
intel_vvp_anr_get_num_color_planes(intel_vvp_anr_instance*
instance);
```

Description Returns the value of the NUM_COLOR register. The instance must be a valid intel_vvp_anr_instance fully initialized.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure

Returns the number of color planes parameter to generate the IP

intel_vvp_anr_get_cfa_enabled

Prototype

```
bool
intel_vvp_anr_get_cfa_enabled(intel_vvp_anr_instance*
instance);
```

Description Returns the value of the CFA_ENABLED register. The instance must be a valid intel_vvp_anr_instance fully initialized.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure

Returns the color filter array parameter to generate the IP

intel_vvp_anr_get_pixels_in_parallel

Prototype

```
uint8_t
intel_vvp_anr_get_pixels_in_parallel(intel_vvp_anr_instance*
instance);
```

Description Returns the value of the PIXELS_IN_PARALLEL register. The instance must be a valid intel_vvp_anr_instance fully initialized.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure

Returns the pixels in parallel parameter to generate the IP

intel_vvp_anr_get_max_width

Prototype

```
uint32_t  
intel_vvp_anr_get_max_width(intel_vvp_anr_instance* instance);
```

Description Returns the value of the MAX_WIDTH register. The instance must be a valid intel_vvp_anr_instance fully initialized.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure

Returns the maximum field width parameter to generate the IP

intel_vvp_anr_get_num_h_taps

Prototype

```
uint32_t  
intel_vvp_anr_get_num_h_taps(intel_vvp_anr_instance* instance);
```

Description Returns the value of the H_TAPS register. The instance must be a valid intel_vvp_anr_instance fully initialized.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure

Returns the number of horizontal taps parameter to generate the IP

intel_vvp_anr_get_num_v_taps

Prototype

```
uint32_t  
intel_vvp_anr_get_num_v_taps(intel_vvp_anr_instance* instance);
```

Description Returns the value of the H_TAPS register. The instance must be a valid intel_vvp_anr_instance fully initialized.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure

Returns the number of vertical taps parameter to generate the IP

intel_vvp_anr_is_running

Prototype

```
bool  
intel_vvp_anr_is_running(intel_vvp_anr_instance* instance);
```

Description Returns the value of the IP Running bit from the STATUS register. The instance must be a valid intel_vvp_anr_instance fully initialized.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure

Returns the running bit from the status register

intel_vvp_anr_commit_is_pending

Prototype

```
bool
intel_vvp_anr_commit_is_pending(intel_vvp_anr_instance*
instance);
```

Description Returns the value of the commit pending bit from the STATUS register. The instance must be a valid intel_vvp_anr_instance fully initialized.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure

Returns true if the agent interface has pending writes from a previous commit request

intel_vvp_anr_get_status

Prototype

```
uint32_t
intel_vvp_anr_get_status(intel_vvp_anr_instance* instance);
```

Description Returns the value of the STATUS register. The instance must be a valid intel_vvp_anr_instance fully initialized.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure

Returns the value of the status register

intel_vvp_anr_get_frame_stats

Prototype

```
int
intel_vvp_anr_get_frame_stats(intel_vvp_anr_instance*
instance, uint32_t* stats_out);
```

<i>Description</i>	Reads the value of the FRAME_STATS register. The instance must be a valid intel_vvp_anr_instance fully initialized.
<i>Arguments</i>	instance - pointer to the intel_vvp_anr_instance software driver instance structure stats_out - pointer of a variable to return the statistics register value
<i>Returns</i>	kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error kIntelVvpCoreInstanceErr if the instance is a null pointer kIntelVvpAnrPointerErr if stats_out is a null pointer

intel_vvp_anr_commit

Prototype

```
int
intel_vvp_anr_commit(intel_vvp_anr_instance* instance);
```

<i>Description</i>	Commits all pending writes before processing the next field. The instance must be a valid intel_vvp_anr_instance fully initialized.
<i>Arguments</i>	instance - pointer to the intel_vvp_anr_instance software driver instance structure
<i>Returns</i>	kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error kIntelVvpCoreInstanceErr if the instance is a null pointer

intel_vvp_anr_get_bypass

Prototype

```
bool
intel_vvp_anr_get_bypass(intel_vvp_anr_instance* instance);
```

<i>Description</i>	Returns the value of the bypass bit from the CONTROL register. The instance must be a valid intel_vvp_anr_instance fully initialized.
<i>Arguments</i>	instance - pointer to the intel_vvp_anr_instance software driver instance structure
<i>Returns</i>	the value of the bypass bit form the control register

intel_vvp_anr_set_bypass

Prototype

```
int
intel_vvp_anr_set_bypass(intel_vvp_anr_instance* instance,
    bool bypass);
```

Description Writes to the bypass bit of the CONTROL register. The instance must be a valid intel_vvp_anr_instance fully initialized.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure

bypass - new value of the bypass bit

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpAnrCommitPendingErr if a previous commit request is pending

intel_vvp_anr_write_intensity_lut_entry

Prototype

```
int
intel_vvp_anr_write_intensity_lut_entry(intel_vvp_anr_instance*
    instance, uint32_t data,
    uint32_t addr);
```

Description Write a single value to the spatial distance lookup table. The instance must be a valid intel_vvp_anr_instance fully initialized.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure

data - value of the lookup table entry

addr - address of the lookup table entry

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpAnrPointerErr if data_array is a null pointer

kIntelVvpAnrOutOfBoundsErr if the address is out of range

intel_vvp_anr_write_intensity_lut_array

Prototype

```
int
intel_vvp_anr_write_intensity_lut_array(intel_vvp_anr_instance*
instance,
const uint32_t* data_array);
```

Description Writes all entries of the spatial distance lookup table from an array of values. The instance must be a valid intel_vvp_anr_instance fully initialized.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure
data_array - pointer of the array containing all lookup table entries

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
kIntelVvpCoreInstanceErr if the instance is a null pointer
kIntelVvpAnrPointerErr if data_array is a null pointer

intel_vvp_anr_write_spatial_lut_entry

Prototype

```
int
intel_vvp_anr_write_spatial_lut_entry(intel_vvp_anr_instance*
instance, uint32_t data,
uint32_t addr);
```

Description Write a single value to the spatial distance lookup table. The instance must be a valid intel_vvp_anr_instance fully initialized.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure
data - value of the lookup table entry
addr - address of the lookup table entry

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
kIntelVvpCoreInstanceErr if the instance is a null pointer
kIntelVvpAnrPointerErr if data_array is a null pointer
kIntelVvpAnrOutOfBoundsErr if the address is out of range

intel_vvp_anr_write_spatial_lut_array

Prototype

```
int
intel_vvp_anr_write_spatial_lut_array(intel_vvp_anr_instance*
instance,
const uint32_t* data_array);
```

Description Writes all entries of the spatial distance lookup table from an array of values. The instance must be a valid intel_vvp_anr_instance fully initialized.

Arguments instance - pointer to the intel_vvp_anr_instance software driver instance structure
data_array - pointer of the array containing all lookup table entries

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
kIntelVvpCoreInstanceErr if the instance is a null pointer
kIntelVvpAnrPointerErr if data_array is a null pointer

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

11. Advanced Test Pattern Generator IP

11.1. About the Advanced Test Pattern Generator IP

The IP produces an Altera Streaming Video compliant lite or full variant output stream. Each frame or field contains between two and eight test patterns with varying dimensions, offsets, and transparencies.

You can adjust at run time the dimensions, offsets, and transparency of test patterns, output video resolution, and individual test pattern configuration, if you turn on **Memory-mapped control interface**.

The IP:

- Produces progressive or interlaced video frames or fields with resolutions up to 16384 by 16384 pixels, with 1-8 pixels in parallel in most cases.
- Supports outputs with either RGB, YCbCr or monochrome color spaces, with 4:4:4, 4:2:2 or 4:2:0 subsampling.
- Produces output on request, allowing for debugging of a video processing system.

This IP is the equivalent of multiple Test Pattern Generator IP instances feeding into the Mixer IP, with unified parameters and run-time control settings.

To blend multiple test patterns on the same output, instantiate this IP not the Test Pattern Generator IP. For only a single test pattern at any one time, use the Test Pattern Generator IP.

11.1.1. Advanced Test Pattern Generator IP Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family, number of supported bits per sample, pixels in parallel, number of test patterns.

Table 72. Advanced Test Pattern Generator IP Performance and Resources

The table shows ALM block usage, DSP usage and f_{MAX} for a design with 10 bits per color sample, 2 pixels in parallel, 4 test patterns enabled (bars pattern, constant color pattern, zone plate pattern, digital clock pattern), with a memory-mapped control interface.

Target Device	DSP Blocks	ALMs	f_{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	22	2,863	723
Arria 10 speed grade 1 (10AS016C3U19E2LG)	22	2,634	556
Cyclone 10 GX speed grade 5 (10CX220YF672E5G)	22	2,643	475
Stratix 10 GX speed grade 1 (1SX280LN2F43E1VG)	22	3,046	560

11.2. Advanced Test Pattern Generator IP Parameters

The IP offers run time and compile time parameters.

Table 73. Advanced Test Pattern Generator IP Parameters

Parameter	Values	Description
General		
Interface Configuration		
Lite mode	On or Off	Turn on to use the Lite variant of the Altera Streaming Video Protocol.
Bits per color sample	8 to 16	Selects the number of bits per color sample
Number of pixels in parallel	1 to 8	Selects the number of pixels transmitted per clock cycle. If either YCbCr422 or YCbCr420 is selected as the Color space , then this must be 1 or even.
Color space	RGB444, YCbCr444, YCbCr422, YCbCr420, Monochrome	Selects the output color space. RGB444, YCbCr444 and YCbCr420 have 3 color channels. YCbCr422 has 2 color channels. Monochrome has 1 color channel.
General Settings		
Memory mapped control interface	On or Off	Turn on for the Avalon memory-mapped control agent interface and allow run-time configuration via the register map.
Debug features	On or Off	Turn on to read back from any writable registers and read-only registers. If off, you cannot read back from any writable registers. However you can still read any read-only registers.
Separate clock for control interface	On or Off	Enable a separate clock for the Memory mapped control agent interface
Pipeline ready signals	On or Off	Enable to add extra pipeline registers to AXI4-S <code>ready</code> signals.
Number of test patterns	2 to 8	Select the number of test patterns.
Video Configuration		
Fixed field width	1 to 16384	Only if Memory-mapped control interface is off. Select the width of the frames and fields produced at the output. If the selected Color space is YCbCr 4:2:2 or YCbCr 4:2:0, the frame width must be a multiple of 2.
Fixed field height	1 to 16384	Only if Memory-mapped control interface is off. Select the height of fields produced at the output. If the selected Color space is YCbCr 4:2:0, the field height must be a multiple of 2.
Fixed interlace	0 to 15	Only if Memory-mapped control interface is off. If Lite mode is off, this value populates the interlace nibble field in the output image info packets. - Values 0 to 7 give progressive output. - Values 8 to 11 give interlaced output with F0 sent first. - Values 12 to 15 give interlaced output with F1 sent first.
Pattern X Pattern X is repeated for all patterns 0 to 7, where X is the pattern number.		
<i>continued...</i>		

Parameter	Values	Description
Pattern X test pattern	Bars Constant color SDI pathological Zone plate Digital clock	Sets the test pattern for pattern X.
Pattern X Dimensions, Offsets, and Alpha These parameters are only available if Memory mapped control interface is off. Otherwise, refer to the register map for control of these settings. Pattern 0 is the base layer and uses Fixed frame width, Fixed frame height, Fixed frame interlacing instead of these parameters listed here.		
Pattern width	1 to 16384	Sets the width of pattern X.
Pattern height	1 to 16384	Sets the height of pattern X.
Alpha value	0 to (2 [^] Bits per color sample) – 1	Sets the alpha value (transparency) used to blend pattern X with
Horizontal offset	0 to 16383	Sets the horizontal offset for pattern X.
Vertical offset	0 to 16383	Sets the vertical offset for pattern X.
Pattern X Pattern Configuration		
Constant color R/Cr	0 – (2 [^] Bits per color sample) – 1	Sets the value that the IP uses for the R color plane if output is RGB, or Cr color plane if output is YCbCr. (18)
Constant color G/Y	0 – (2 [^] Bits per color sample) – 1	Sets the value that the IP uses for the G color plane if output is RGB, or Y color plane if output is YCbCr. (18)
Constant color B/Cb	0 – (2 [^] Bits per color sample) – 1	Sets the value that the IP uses for the B color plane if output is RGB, or Cb color plane if output is YCbCr. (18)
Bars mode	Color Greyscale Black and White Mixed	Sets which variant of the bars pattern to use. Only if pattern X is bars test pattern.
Zone plate Origin x-coordinate	0 to 16383	Set to configure the x-coordinate of the zone plate origin. ⁽¹⁹⁾
Zone plate Origin y-coordinate	0 to 16383	Set to configure the y-coordinate of the zone plate origin. ⁽¹⁹⁾
Zone plate Coarse scaling factor	1 to 31	Set to configure the coarse scaling coefficient of the zone plate size. Increase this value to increase the size of the zone plate. The value is arbitrary, but recommended starting values are 17 for HD and 19 for 4k resolutions. (19)
Zone plate Fine-tune scaling factor	1 to 65535	The fine tune scaling coefficient of the zone plate size. This 16-bit fixed point number has 8 bits fractional part (hence 256 is a value of 1.0). Increase this value to decrease the size of the zone plate. (19)
<i>continued...</i>		

(18) Only pattern X is **constant color** test pattern.

(19) Only if pattern X is **zone plate** test pattern.

Parameter	Values	Description
Digital clock Background color (B/Cb)	0 to (2 [^] Bits per color sample) – 1	Set to configure the B component of the digital clock display background color if the output color space is RGB, or Cb component if YCbCr. (20)
Digital clock Background color (G/Y)	0 to (2 [^] Bits per color sample) – 1	Set to configure the G component of the digital clock display background color if the output color space is RGB, or Y component if YCbCr. (20)
Fixed digital clock background color (R/Cr)	0 to (2 [^] Bits per color sample) – 1	Set to configure the R component of the digital clock display background color if the output color space is RGB, or Cr component if YCbCr. (20)
Digital clock font color (B/Cb)	0 to (2 [^] Bits per color sample) – 1	Set to configure the B component of the digital clock display font color if the output color space is RGB, or the Cb component if YCbCr. (20)
Digital clock font color (G/Y)	0 – (2 [^] Bits per color sample) – 1	Set to configure the G component of the digital clock display font color if the output color space is RGB, or the Y component if YCbCr. (20)
Digital clock font color (R/Cr)	0 – (2 [^] Bits per color sample) – 1	Set to configure the R component of the digital clock display font color if the output color space is RGB, or the Cr component if YCbCr. (20)
Digital clock location X coordinate	0 to 16383	Set to configure the x-coordinate of the location of the top-left pixel of the digital clock display. (20)
Digital clock location Y coordinate	0 to 16383	Set to configure the y-coordinate of the location of the top-left pixel of the digital clock display. (20)
Digital clock scale factor	1 to 2048	Set to configure the scale factor of the digital clock display in both vertical and horizontal dimensions. If scale factor is 1, each character is 8x8 pixels, and a total of 88x8 for the display. (20)
Digital clock FPS value	1 to 60	Set to configure the FPS used to determine after how many frames a second occurs. Does not effect true outgoing video refresh rate. (20)

11.3. Advanced Test Pattern Generator IP Functional Description

The IP expands on the functionality of the Test Pattern Generator IP, allowing you to combine between 2 to 8 test patterns on the same video output with varying dimensions, offsets, and alpha value per pattern. You can set the configurations of these patterns either at compile time, or at run time via the memory-mapped control interface.

(20) Only if pattern X is **digital clock** test pattern.

11.3.1. Test Patterns

The IP offers the following patterns:

- Bars pattern
 - ☐ Color bars
 - ☐ Grayscale bars
 - ☐ Black and white bars
 - ☐ Mixed bars
- Constant color
- SDI pathological
- Zone plate
- Digital clock

Bars pattern

If you select Bars Pattern, the IP produces a test pattern with 8 vertical bars, each covering approximately 1/8th of each output video line. The bars contain either a sequence of changing colors (color bars), a sequence of greyscale tones with decreasing brightness (greyscale bars), or a sequence of alternating black and white bars. Mixed bars change the pattern in the vertical direction, with the first 1/8th of the lines showing the black and white bars, the next 1/8th showing the greyscale bars, and the final $\frac{3}{4}$ showing the color bars. If you select bars pattern (and turn on the Avalon memory-mapped control agent interface), a value in the register map selects at run time which variant of the bars pattern to display. If you do not turn on **Avalon memory-mapped control agent**, the **fixed bars mode** parameter controls which variant of the bars pattern to display, and you cannot change it at run time.

For all test patterns, each bar is approximately 1/8th of the width of the output frame, but this width is not exact. The actual width of each color bar is affected by the field width and the horizontal subsampling.

When the width of the field is not divisible by eight, the remainder pixels from the division are spread as evenly as possible across the bars. The first bar, on the left-hand side of the field, always has $\text{floor}(\text{field_width}/8)$ pixels. A single remainder pixel is added to the width of each subsequent bar until all the remainder pixels are exhausted. The width of each bar cannot be less than the number of pixels transmitted per beat at the output interface (*pixels in parallel*). If the field width is set to less than $8 \times \text{pixels in parallel}$, as many bars as possible display with *pixels in parallel* pixels per bar, but fewer than 8 bars display. When the output is horizontally subsampled (4:2:2 or 4:2:0), the pixel-width of each color bar is a multiple of two and we alter the methods described above to ensure this is always the case.

The tables define the values of the color components in each bar for each of the three basic modes. The values are the actual output values if you set bits per color plane to 8 bits. If bits per color is greater than 8 bits, the internal logic upshifts the value by the required number of bits, with zeros added in the LSBs.

Table 74. Output values for each color plane in both the RGB and YCbCr color spaces for the color bars test pattern

Color	RGB	YCbCr
White (left)	(180, 180, 180)	(180, 128, 128)
Yellow	(180, 180, 16)	(162, 44, 142)
Cyan	(16, 180, 180)	(131, 156, 44)
Green	(16, 180, 16)	(112, 72, 58)
Magenta	(180, 16, 180)	(84, 184, 198)
Red	(180, 16, 16)	(65, 100, 212)
Blue	(16, 16, 180)	(35, 212, 114)
Black (right)	(16, 16, 16)	(16, 128, 128)

Table 75. Output values for each color plane in both the RGB and YCbCr color spaces for the greyscale bars test pattern

Color	RGB	YCbCr
0 (left)	(180, 180, 180)	(180, 128, 128)
1	(162, 162, 162)	(162, 128, 128)
2	(131, 131, 131)	(131, 128, 128)
3	(112, 112, 112)	(112, 128, 128)
4	(84, 84, 84)	(84, 128, 128)
5	(65, 65, 65)	(65, 128, 128)
6	(35, 35, 35)	(35, 128, 128)
7 (right)	(16, 16, 16)	(16, 128, 128)

Table 76. Output values for each color plane in both the RGB and YCbCr color spaces for the black and white bars test pattern

Color	RGB	YCbCr
0 (left)	(180, 180, 180)	(180, 128, 128)
1	(16, 16, 16)	(16, 128, 128)
2	(180, 180, 180)	(180, 128, 128)
3	(16, 16, 16)	(16, 128, 128)
4	(180, 180, 180)	(180, 128, 128)
5	(16, 16, 16)	(16, 128, 128)
6	(180, 180, 180)	(180, 128, 128)
7 (right)	(16, 16, 16)	(16, 128, 128)

Constant color

The constant color test pattern is a complete field or frame of a constant color. This test pattern has limited value for testing, but you can use the pattern to form a background layer for a mixer. Set the RGB or YCbCr values for the desired color as either fixed, compile-time-set parameters or run-time controlled values through the Avalon memory-mapped control agent interface.

SDI Pathological

The SDI pathological test pattern is specifically designed to stress test the SDI equalizer and PLL performance. The test pattern consists of a static test image with the top half of the lines filled with a shade of magenta, and the bottom half of the lines filled with a shade of gray.

Zone plate

The zone plate test pattern is an arbitrarily sized, arbitrarily placed set of concentric rings that get wider and narrower the further from the **origin**. If **Avalon memory-mapped control agent** is off, the origin is always at the center of the display. If **Avalon memory-mapped control agent** is on, you can configure the origin by writing to the `ZONE_X_ORIGIN` and `ZONE_Y_ORIGIN` register map values.

Set the `Coarse scaling coefficient` bits in the `ZONE_POWER_FACTOR` register to set the initial size of the zone plate so that it is visible with the current resolution. Set the `fine tune scaling coefficient` bit in the `ZONE_FINE_FACTOR` register for incremental changes to the size that smoothly resize the plate. If **Avalon memory-mapped control agent** is on, you can configure these values by writing to the respective registers, otherwise these values are fixed at compile time.

The following equation shows how the `Coarse scaling coefficient`, c and the `fine tune scaling coefficient`, k , relate to the zone plate, where r is the radius away from the origin.

Equation 1. Coarse and Fine scaling Equation

$$\text{pixel} = \begin{cases} \text{black} & \cos((r^2 \gg c) \times k \times \pi) \geq 0 \\ \text{white} & \text{else} \end{cases}$$

Because of the position of k in the equation, increasing k causes a decrease in the size of the zone plate (decreased zoom). Decreasing k causes an increase in the size of the zone plate (increased zoom).

Figure 21. Example of zone plate on a 1920x1080 screen

With origin set to (960, 540), $c = 14$, $k = 1$ (register value 0b0000_0001_0000_0000)

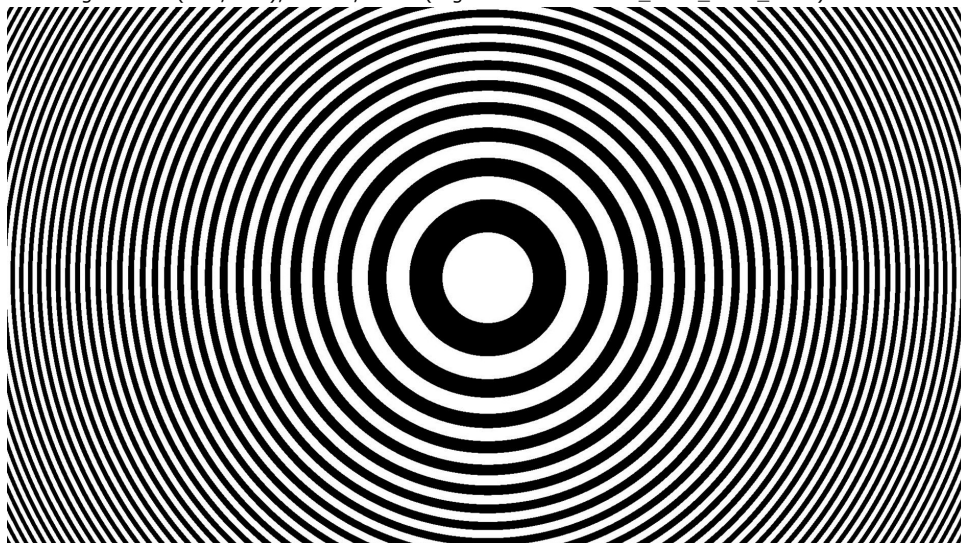


Figure 22. Example of zone plate on a 1920x1080 screen

With origin set to (500, 300), $c = 14$, $k = 10$ (register value 0b0000_0100_0110_0000)

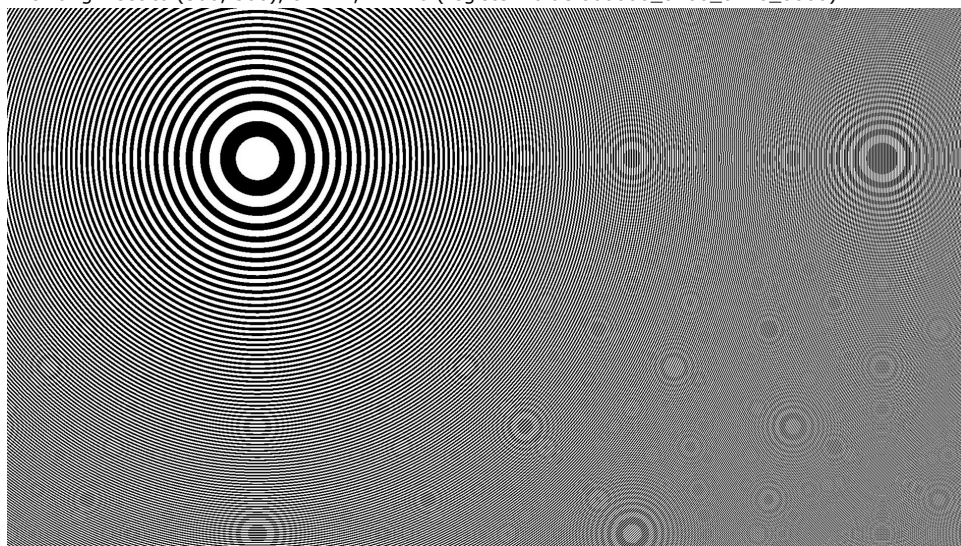
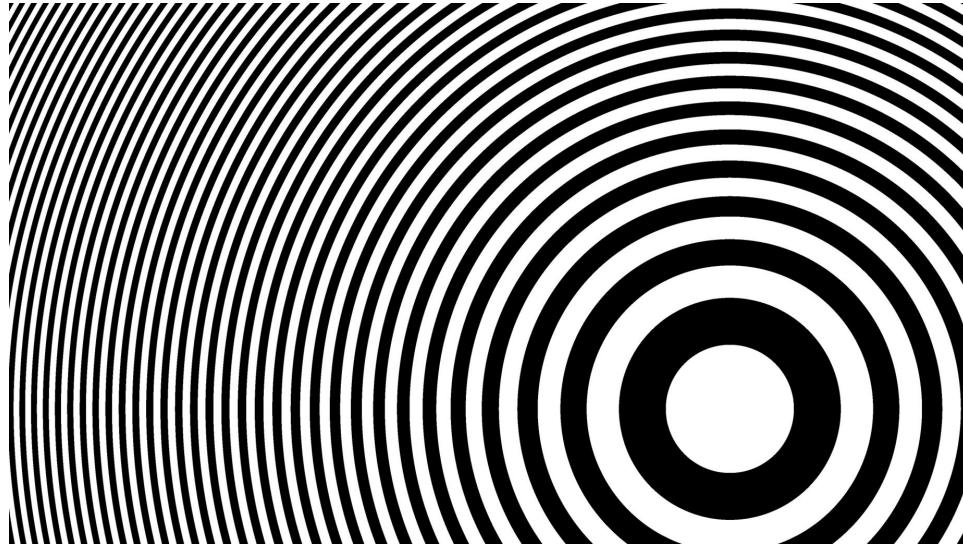


Figure 23. Example of zone plate on a 3840x2160 screen

With origin set to (2880, 1620), $c = 17$, $k = 1$ (register value 0b0000_0100_0110_0000)



Digital clock

The digital clock pattern consists of a digital display with format HH:MM:SS,FC, where FC is field count. The time is not precise but is a function of frames per second and field count.

You can customize the size of the display. The original dimensions of the display are 88x8 pixels. If **Avalon memory-mapped control agent** is on, the value of **Fixed clock scale factor** is a scalar multiple to both dimensions. If **Avalon memory-mapped control agent** is off, the value of the `Scale factor` bit in the `CLOCK_SCALE_FACTOR` register is a scalar multiple to both dimensions.

Set the background or font color and location at either compile time if **Avalon memory-mapped control agent** is off, or at run time via the register map.

11.3.2. Dimensions, Offsets, and Alpha Settings

These Advanced Test Pattern Generator IP settings control the dimensions, offsets, and alpha blending of patterns (layers) 1-7. These settings do not apply to pattern 0, which acts as a canvas and is always fully opaque with no offsets.

If the configured settings for a given layer break the boundaries of the canvas, the corresponding pattern does not show.

- Width
- Height
- Alpha (transparency)
- Horizontal offset
- Vertical offset

You can change these settings during runtime if you turn on **Memory-mapped control interface**. If off, you must parameterize fixed values for each enabled layer.

This alpha blending setting affects the opacity per layer, given by an alpha value between 0 and 1 where 0 represents full transparency and 1 represents fully opaque.

The IP recursively applies alpha blending. Start with blending pattern 1 on top of pattern 0 to create an intermediate image, then blending pattern 2 on top of the intermediate image produced, until the IP blends all patterns.

Equation: Given pattern N represented as I_N

$$O_1 = \alpha \times I_1 + (1 - \alpha) \times I_0$$

$$O_N = \alpha \times I_N + (1 - \alpha) \times I_{(N-2)}$$

11.3.3. Run-time Settings

If the Advanced Test Pattern Generator IP **Memory-mapped control interface** is on, you can adjust at run time the resolution, interlace settings, pattern configuration and mixer configuration via the register map. Changing the settings at runtime differs depending on whether the IP is parameterized to output the full or lite variant of the Altera Streaming Video Protocol.

If off, the IP only uses parameters prefixed with `Fixed` to configure resolution, interlace, mixer and pattern configuration.

For resolution, interlace and pattern dimensions, offsets, alpha settings, write to the `COMMIT` register (address 0x14C) to apply the settings at the next frame boundary. If the settings you provide cause a pattern to clip partially or wholly off-screen, the IP does not show it.

For pattern configuration settings, write to `PATTERN_X_SELECT` (0x1E8) the pattern number you want to apply the settings to at the next frame boundary. Only the pattern configuration relevant to the pattern selected has an effect. You may write to `PATTERN_X_SELECT` multiple times a frame, to apply different settings to each of the patterns enabled.

Unlike other video and vision IP registers, you do not need to write to the `COMMIT` register after writing to any of the pattern registers.

The full sequence to update the registers for resolution, interlace, and pattern dimensions, offsets, alpha settings is:

1. Make required edits to anywhere within the subset of the resolution, interlace and mixer registers (addresses 0x150 to 0x1E4).
2. Write any value to the `COMMIT` register (0x14C) to commit the changes as a coherent set.
3. Do not make any further edits to the settings until the IP deasserts the pending bit of the `STATUS` register (0x140); this occurs at the next field boundary after the write to the `COMMIT` register at which points the IP applies the settings.
4. When the IP deasserts bit 1 of the `STATUS` register, you can make further edits..

The full sequence to update the register for pattern configuration is:

1. Make all edits to the subset of pattern configuration registers (0x1E8 onwards).
You must write to all registers related to the type of pattern you wish to configure, otherwise there may be data left from a previous write.
2. Write a number X to `PATTERN_X_SELECT` to apply the changes to Pattern X.
 - a. Ensure that the changes you make apply for the type of Pattern X. This can be checked by reading `PATTERN_X_TYPE` (0x24 to 0x40).
 - b. Writing to `PATTERN_X_SELECT` automatically commit the registers such that all changes apply to the next field. You do not need to write to the `COMMIT` register afterwards for these specific registers.
3. Repeat steps 1-2 for every pattern you want to change.

11.3.4. Advanced Test Pattern Generator IP Interfaces

Name	Direction	Width	Description
Clocks and resets			
<code>main_clock_clk</code>	In	1	AXI4-S processing clock.
<code>main_reset_rst</code>	In	1	AXI4-S processing reset.
<code>agent_clock_clk</code>	In	1	Clock for the Avalon memory-mapped control agent interface. Only if you select separate clock for control interface .
<code>agent_reset_rst</code>	In	1	Reset for the Avalon memory-mapped control agent interface. Only if you select separate clock for control interface .
Control interfaces			
<code>av_mm_control_agent_address</code>	In	7	Avalon memory-mapped agent address.
<code>av_mm_control_agent_write</code>	In	1	Avalon memory-mapped agent write.
<code>av_mm_control_agent_writedata</code>	In	32	Avalon memory-mapped agent write data.
<code>av_mm_control_agent_byteenable</code>	In	4	Avalon memory-mapped agent byte enable.
<code>av_mm_control_agent_read</code>	In	1	Avalon memory-mapped agent read.
<code>av_mm_control_agent_readdata</code>	Out	32	Avalon memory-mapped agent read data.
<code>av_mm_control_agent_readdatavalid</code>	Out	1	Avalon memory-mapped agent read.
<code>av_mm_control_agent_waitrequest</code>	Out	1	Avalon memory-mapped agent wait request.
Altera Streaming Video interfaces			
<code>axi4s_vid_out_tdata</code>	Out	(21)	AXI4-S data out.
<code>axi4s_vid_out_tvalid</code>	Out	1	AXI4-S data valid.
<i>continued...</i>			

(21) The equation gives all `tdata` widths in these interfaces: $\max(2, \text{ceil}((\text{Bits per color sample} \times \text{Number of color planes})/8)) \times \text{Pixels in parallel} \times 8$

Name	Direction	Width	Description
axi4s_vid_out_tuser	Out	(22)	AXI4-S tuser. ⁽²³⁾
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet.
axi4s_vid_out_tready	In	1	AXI4-S data ready.

Related Information

Video and Vision Processing IP Interfaces on page 30

11.4. Advanced Test Pattern Generator IP Registers

Each register is either read-only (RO) or read-write (RW). If **Debug features** is off, all RW registers are write-only (WO).

Table 77. Advanced Test Pattern Generator IP General Registers

Address	Register	Access	Description
Parameterization registers			
0x0000	VID_PID	RO	Read this register to retrieve the Advanced test pattern generator product ID. This register always returns 0x6AF7_0252.
0x0004	VERSION	RO	Read this register to retrieve the version information for the Quartus release that Altera uses to build the IP.
0x0008	LITE_MODE	RO	Read this register to determine if lite mode is on. This register returns 0 for Full mode and 1 for Lite mode .
0x000C	DEBUG_ENABLED	RO	Read this register to determine if Debug features is on. This register returns 1 if reads to other registers designated as RW return the last value you write to the register, or an undefined value.
0x0010	NUM_PATTERNS	RO	Read this register to determine the number of test pattern configurations
0x0014	BPS	RO	Read this register to determine the number of bits per color plane
0x0018	PIXELS_IN_PARALLEL	RO	Read this register to determine the number of pixels transmitted per clock cycle at the streaming output
0x001C	COLOR_SPACE	RO	Read this register to determine the color space and chroma sampling. This register returns: 0 for RGB 1 for YCbCr 4:4:4 2 for YCbCr 4:2:2 3 for YCbCr 4:2:0 4 for monochrome
0x00210	PATTERN_0_TYPE	RO	Read this register to determine the pattern type of test pattern configuration 0. This register returns:
continued...			

⁽²²⁾ The equation gives all `tuser` widths in these interfaces: $N = \text{ceil}(tdata \text{ width} / 8)$

⁽²³⁾ Refer to Altera Streaming Video Protocol for full information about `TUSER`

Address	Register	Access	Description
			• 0 for the bars pattern • 1 for constant color • 2 for SDI pathological • 3 for zone plate • 4 for digital clock
0x0024	PATTERN_1_TYPE	RO	Read this register to determine the pattern type of test pattern configuration 1. This register returns: • 0 for the bars pattern • 1 for constant color • 2 for SDI pathological • 3 for zone plate • 4 for digital clock
0x002C	PATTERN_2_TYPE	RO	Read this register to determine the pattern type of test pattern configuration 2. This register returns: • 0 for the bars pattern • 1 for constant color • 2 for SDI pathological • 3 for zone plate • 4 for digital clock
0x002C	PATTERN_3_TYPE	RO	Read this register to determine the pattern type of test pattern configuration 3. This register returns: • 0 for the bars pattern • 1 for constant color • 2 for SDI pathological • 3 for zone plate • 4 for digital clock
0x0030	PATTERN_4_TYPE	RO	Read this register to determine the pattern type of test pattern configuration 4. This register returns: • 0 for the bars pattern • 1 for constant color • 2 for SDI pathological • 3 for zone plate • 4 for digital clock
0x0034	PATTERN_5_TYPE	RO	Read this register to determine the pattern type of test pattern configuration 5. This register returns: • 0 for the bars pattern • 1 for constant color • 2 for SDI pathological • 3 for zone plate • 4 for digital clock
0x0038	PATTERN_6_TYPE	RO	Read this register to determine the pattern type of test pattern configuration 6. This register returns: • 0 for the bars pattern • 1 for constant color • 2 for SDI pathological • 3 for zone plate • 4 for digital clock
0x003C	PATTERN_7_TYPE	RO	Read this register to determine the pattern type of test pattern configuration 7. This register returns:
continued...			

Address	Register	Access	Description
			0 for the bars pattern 1 for constant color 2 for SDI pathological 3 for zone plate 4 for digital clock
<i>0x0040 to 0x013C unused</i>			
0x0140	STATUS	RO	Bit 0: Status bit. 1 = advanced test pattern generator is processing a video field, 0 otherwise. Bit 1 : Pending register updates bit Lite mode is off only. Any writes to registers excluding pattern X registers cause the IP to raise pending register updates bit, to indicate outstanding changes yet to be committed. Refer to register Pattern X Select for more information. The IP lowers this bit at the next field boundary after a write to the COMMIT register.
0x0144	FIELD_COUNT	RO	Read this register to determine the output field count. The value in this register is reset each time the IP stops.
0x0148	CONTROL	RW	Bit 0: Go bit. Write 0 to this register to stop the test pattern generator at the end of the current frame. Write 1 to this register to start or restart output generation. This register initializes to 0 at start-up and no output fields are generated until after a write of 1 to this register. Writes to this register apply immediately, without needing to further write to COMMIT.
0x014C	COMMIT	RW	The IP holds any changes pending to registers (excluding those prefixed with PATTERN_X_) until you send a write to this register. The value you write is unimportant.
0x0150	FIELD_WIDTH	RW	Write to this register to set the width of outgoing video fields. If the output chroma sampling is 4:2:2 or 4:2:0, the width should be a multiple of 2. The width of the canvas that applies to pattern 0. If the widths and offsets of other patterns (set via LAYER_N registers) cause that pattern to go out of the canvas boundaries, the IP does not show that pattern.
0x0154	FIELD_HEIGHT	RW	Use this register to set the height for outgoing video fields. If the output chroma subsampling is 4:2:0, the height should be a multiple of 2. The height of the canvas that applies to pattern 0. If the height and offsets of other patterns (set via LAYER_N registers) cause that pattern to go out of the canvas boundaries, the IP does not show that pattern.
0x0158	FIELD_INTERLACE	RW	Use this register to set the interlace output behavior for the first outgoing field. The value you write corresponds to the interlace identifier nibble of outgoing image information packets in full mode. Values 0 to 7 produce progressive outputs. Values 8 to 1 cause the field sequence to restart with F0. Values 12 to 15 cause the field sequence to restart with F1.
0x015C + (n-1) * 0x14	LAYER_N_WIDTH	RW	Use this register to set the width of layer or pattern <i>n</i> (excluding pattern 0). (24)
<i>continued...</i>			

(24) If the dimensions and offsets of layer *n* cause pattern *n* to cross the canvas boundaries, the IP does not show the pattern.

Address	Register	Access	Description
0x0160 + (n-1) *0x14	LAYER_N_HEIGHT	RW	Use this register to set the height of layer or pattern <i>n</i> (excluding pattern 0). (24)
0x0164 + (n-1) *0x14	LAYER_N_ALPHA	RW	Use this register to set the alpha value of layer or pattern <i>n</i> (excluding pattern 0). .
0x0168 + (n-1) *0x14	LAYER_N_H_OFFSET	RW	Use this register to set the horizontal offset of layer or pattern <i>n</i> (excluding pattern 0). (24)
0x016C + (n-1) *0x14	LAYER_N_V_OFFSET	RW	Use this register to set the vertical offset of layer/pattern <i>n</i> (excluding pattern 0). (24)
0x01E8	PATTERN_X_SELECT	RW	Write to this register with values 0 to 7 to apply all pattern X registers to the given pattern number. Only the registers that apply to the selected pattern have an effect.

Pattern configuration registers

The following sets of registers have different semantics according to the different types of test patterns, and they may share addresses. When you write a pattern number "X" to PATTERN_X_SELECT, only the registers that correspond to the test pattern type of pattern X have an effect.

Ensure you write to all registers specific for a type of pattern before committing via PATTERN_X_SELECT; values from previous writes may persist in registers otherwise, leading to unwanted behavior.

Table 78. Advanced Test Pattern Generator IP Bars Pattern Registers

Address	Register	Access	Description ⁽²⁵⁾
0x01EC	PATTERN_X_BARS_SELECT	WO	Write to this register to set the variant of the bars pattern to use. Write: . 0 for color bars 1 for greyscale bars 2 for black and white bars 3 for mixed bars

Table 79. Advanced Test Pattern Generator IP Constant color pattern registers

Address	Register	Access	Description ⁽²⁶⁾
0x01EC	PATTERN_X_CONST_C0	WO	Write to this register to set the value for color plane 0 (B or Cb) by the constant color test pattern.
0x01F0	PATTERN_X_CONST_C1	WO	Write to this register to set the value for color plane 1 (G or Y) by the constant color test pattern.
0x01F4	PATTERN_X_CONST_C2	WO	Write to this register to set the value for color plane 2 (R or Cr) by the constant color test pattern.

(25) The IP ignores the value in this register for pattern X if it does not use the **bars** pattern.

(26) The IP ignores the value in this register for pattern X if it does not use the **constant color** test pattern.

Table 80. Advanced Test Pattern Generator IP Zone plate pattern registers

Address	Register	Access	Description ⁽²⁷⁾
0x01EC	PATTERN_X_ZONE_X_ORIGIN	WO	Write to this register to set the X coordinate of the zone plate center (origin). For example, write 960 to horizontally set the centre of the zone plate to the middle of a 1920x1080 display.
0x01F0	PATTERN_X_ZONE_Y_ORIGIN	WO	Write to this register to set the Y coordinate of the zone plate center (origin). For example, write 540 to vertically set the centre of the zone plate to the middle of a 1920x1080 display.
0x01F4	PATTERN_X_ZONE_COARSE_FACTOR	WO	This value is a rough scale up or down of the zone plate . The bigger the value, the more zoomed in the zone plate is. This value is arbitrary, but start with a value of 14 on 1920x1080, and 20 on a 4k display.
0x01F8	PATTERN_X_ZONE_FINE_FACTOR	WO	The value of this register acts as a fine tune scaling factor of the zone plate . It is a 16-bit unsigned fixed-point number, with 8 bits fractional part. The IP implements this value as a constant k in f(kx) (applied after ZONE_POWER_FACTOR). The bigger the value, the more zoomed out the zone plate is.

Table 81. Advanced Test Pattern Generator IP Digital clock pattern registers

Address	Register	Access	Description ⁽²⁸⁾
0x01EC	PATTERN_X_DIGITAL_CONFIGURATION	WO	Bit 0: write 1 to pause the digital clock, 0 to resume. Bit 1: write 1 to reset the clock to 00:00:00,00; write 0 to resume.
0x01F0	PATTERN_X_DIGITAL_BACKGROUND_B	WO	Write to this register to set the value for color plane 0 (B or Cb), which is the background color for the digital clock pattern.
0x01F4	PATTERN_X_DIGITAL_BACKGROUND_G	WO	Write to this register to set the value for color plane 1 (G or Y), which is the background color for the digital clock pattern.
0x01F8	PATTERN_X_DIGITAL_BACKGROUND_R	WO	Write to this register to set the value for color plane 2 (B or Y), which is the background color for the digital clock pattern.
0x01FC	PATTERN_X_DIGITAL_FONT_B	WO	Write to this register to set the value for color plane 0 (B or Cb), which is the font color for the digital clock pattern.
0x0200	PATTERN_X_DIGITAL_FONT_G	WO	Write to this register to set the value for color plane 1 (G or Y), which is the font color for the digital clock pattern.
0x0204	PATTERN_X_DIGITAL_FONT_R	WO	Write to this register to set the value for color plane 2 (B or Y) used as the font color for the digital clock pattern.
0x0208	PATTERN_X_DIGITAL_LOCATION_X	WO	Write to this register to set the X coordinate of the top-left pixel of the digital clock test pattern.
continued...			

⁽²⁷⁾ The IP ignores the value in this register for pattern X if it does not use the **zone plate** pattern.

⁽²⁸⁾ The IP ignores the value in this register for pattern X if it does not use the digital clock pattern.

Address	Register	Access	Description ⁽²⁸⁾
0x020C	PATTERN_X_DIGITAL_LOCATION_Y	WO	Write to this register to set the Y coordinate of the top-left pixel of the digital clock test pattern.
0x0210	PATTERN_X_DIGITAL_SCALE_FACTOR	WO	Write to this register to set the scaling factor of the digital clock. When 1, each character of the display is 8x8 pixels; in total 88x8 pixels.
0x0214	PATTERN_X_DIGITAL_FPS	WO	Write to this register to set the FPS that the digital clock; uses. Determines how many frames before a second passes. Does not effect on actual video refresh rate.

Register Bit Description

Table 82. VID_PID

Name	Bits	Description
Advanced test pattern generator version ID and product ID	31:0	This register always returns 0x6AF7_0252. 15:0 is the product ID and always returns 0x0252 31:16 is the vendor ID and always returns 0x6AF7

Table 83. VERSION

Name	Bits	Description
Register map version	7:0	Register map version. Returns 0x01.
QPDS patch revision	15:8	This returns 0x00
QPDS update revision	23:16	Updated for each release. For 24.2 this returns 0x02.
QPDS major revision	31:24	Updated for each release. For 24.2 this returns 0x18.

Table 84. LITE_MODE

Name	Bits	Description
Lite mode parameterization bit	31:0	Returns 1 if Lite mode is on and 0 otherwise

Table 85. DEBUG_ENABLED

Name	Bits	Description
Lite mode parameterization bit	31:0	Returns 1 if Debug features is on and 0 otherwise

Table 86. NUM_PATTERNS

Name	Bits	Description
Number of enabled patterns	31:0	Returns the number of enabled test patterns

Table 87. BPS

Name	Bits	Description
Bits per color sample	31:0	Returns the number of bits per color sample

⁽²⁸⁾ The IP ignores the value in this register for pattern X if it does not use the digital clock pattern.

Table 88. PIXELS_IN_PARALLEL

Name	Bits	Description
Lite mode parameterization bit	31:0	Returns the number of pixels processed per clock cycle

Table 89. COLOR_SPACE

Name	Bits	Description
Color space and subsampling	31:0	Returns: 0 for RGB 1 for YCbCr 4:4:4 2 for YCbCr 4:2:2 3 for YCbCr 4:2:0 4 for monochrome

Table 90. PATTERN_X_TYPE

Name	Bits	Description
Pattern type for pattern number X	31:0	Returns the pattern type of pattern number X: 0 for bars pattern 1 for constant color pattern 2 for SDI pathological pattern 3 for zone plate pattern 4 for digital clock pattern

Table 91. STATUS

Name	Bits	Description
Processing video status bit	0	Returns 1 if the IP is currently processing a video field; 0 otherwise
Pending register updates bit	1	Returns 1 if the IP has pending updates to any writable registers excluding pattern configuration registers (prefixed with <code>PATTERN_X_</code>)
Unused	31:2	Unused

Table 92. FIELD_COUNT

Name	Bits	Description
Field count	15:0	Returns the number of fields or frames the IP generates since it last started.
Unused	31:16	Unused

Table 93. CONTROL

Name	Bits	Description
Go bit	0	Write 1 to start the IP. Write 0 to stop the IP as soon as it has finishes generating the current frame.
Unused	31:1	Unused

Table 94. COMMIT

Name	Bits	Description
Unused	31:0	Unused. (The value you write value is unimportant).

Table 95. FIELD_WIDTH

Name	Bits	Description
Field/frame width	16:0	Set the width (in pixels) of outgoing fields.
Unused	31:17	Unused.

Table 96. FIELD_HEIGHT

Name	Bits	Description
Field/frame height	16:0	Set the height (in pixels) of outgoing fields.
Unused	31:17	Unused.

Table 97. FIELD_INTERLACE

Name	Bits	Description
Field/frame width	3:0	Set the interlace configuration for outgoing fields.
Unused	31:4	Unused.

Table 98. LAYER_N_WIDTH

Name	Bits	Description
Layer n field width	16:0	Field width for layer n. LAYER_N_WIDTH where n >= NUM_PATTERNS has no effect.
Unused	31:17	Unused

Table 99. LAYER_N_HEIGHT

Name	Bits	Description
Layer n field height	16:0	Field height for layer n. LAYER_N_WIDTH where n >= NUM_PATTERNS has no effect.
Unused	31:17	Unused

Table 100. LAYER_N_ALPHA

Name	Bits	Description
Layer n alpha	(Bits per color sample - 1):0	Alpha value for layer n. LAYER_N_WIDTH where n >= NUM_PATTERNS has no effect.
Unused	31:(Bits per color sample)	Unused

Table 101. LAYER_N_H_OFFSET

Name	Bits	Description
Layer n horizontal offset	16:0	Horizontal offset for layer n. LAYER_N_WIDTH where n >= NUM_PATTERNS has no effect.
Unused	31:17	Unused

Table 102. LAYER_N_V_OFFSET

Name	Bits	Description
Layer n vertical offset	16:0	Vertical offset for layer n. LAYER_N_WIDTH where n >= NUM_PATTERNS has no effect.
Unused	31:17	Unused

Table 103. PATTERN_X_SELECT

Name	Bits	Description
Pattern number	2:0	Write the pattern number (0-7) you want to apply all PATTERN_X settings to. Any write to PATTERN_X registers are pending until you write to this register. Only the relevant registers to a given pattern applies, i.e. PATTERN_X_ZONE_FINE_FACTOR only applies to pattern numbers corresponding to a zone plate test pattern and ignored for all others.
Unused	31:3	Unused

Table 104. PATTERN_X_CONST_C0/1/2

Name	Bits	Description
Color value	(BPS – 1):0	Constant color value for color plane 0/1/2 for pattern number X.
Unused	31:BPS	Unused

Table 105. PATTERN_X_BARS_SELECT

Name	Bits	Description
Bars mode select	1:0	0 = color bars, 1 = greyscale bars, 2 = black and white bars, 3 = mixed bars for pattern number X.
Unused	31:2	Unused

Table 106. PATTERN_X_ZONE_X/Y_ORIGIN

Name	Bits	Description
Origin X/Y	15:0	Sets x/y part of center coordinates of zone plate for pattern number X.
Unused	31:16	Unused

Table 107. PATTERN_X_ZONE_COARSE_FACTOR

Name	Bits	Description
Coarse scaling factor	4:0	Arbitrary sizing factor (same as scaling factor above). Increase factor to increase zone plate size coarsely (downshifts k in f(kx) so increased size). This value applies to pattern number X.
Unused	31:5	Unused

Table 108. PATTERN_X_ZONE_FINE_FACTOR

Name	Bits	Description
Fine tune scaling factor	15:0	Fixed point operand, with 8 bits fractional part. Increase factor to decrease zone plate size smoothly (increase k in f(kx) hence decreased size). This value applies to pattern number X.
Unused	31:16	Unused

Table 109. PATTERN_X_DIGITAL_CONFIGURATION

Name	Bits	Description
Pause	0	Write 1 to pause the digital clock display at its currently displayed time for pattern number X. Write 0 to resume the digital clock display for pattern number X.
Soft reset	1	Write 1 to reset and hold the digital clock display at 00:00:00,00. Write 0 to resume the digital clock display for pattern number X. This value takes precedence over pause, so even if pause is set to 0 the display remains locked until you write 0 to this register. Similarly, if the display is paused, writing 1 to this register resets it to 00:00:00,00.
Unused	31:2	Unused

Table 110. PATTERN_X_DIGITAL_BACKGROUND_B/G/R

Name	Bits	Description
Background color value	15:0	Sets B/G/R or Cb/Y/Cr component of the background color of the digital clock for pattern number X.
Unused	31:16	Unused

Table 111. PATTERN_X_DIGITAL_FONT_B/G/R

Name	Bits	Description
Font color value	15:0	Sets B/G/R or Cb/Y/Cr component of the font color of the digital clock for pattern number X.
Unused	31:16	Unused

Table 112. PATTERN_X_DIGITAL_LOCATION_X/Y

Name	Bits	Description
Location X/Y	15:0	Sets the X/Y coordinate of the digital clock for Pattern number X
Unused	31:16	Unused

Table 113. PATTERN_X_DIGITAL_SCALE_FACTOR

Name	Bits	Description
Scale factor	10:0	Sets the scaling multiplier of the digital clock display.
Unused	31:11	Unused

Table 114. PATTERN_X_DIGITAL_FPS

Name	Bits	Description
Scale factor	5:0	Sets the FPS that determines after how many frames a second passes for the digital clock display.
Unused	31:16	Unused

11.5. Advanced Test Pattern Generator IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios V HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in Video and Vision Processing IPs Software API are applicable.

Register definition file: intel_vvp_adv_tpg_regs.h

Include file: intel_vvp_adv_tpg.h

Table 115. Advanced Test Pattern Generator IP Software API

Name	Description
intel_vvp_adv_tpg_init	Initialize the Advanced Test Pattern Generator instance
intel_vvp_core_*	Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i> . Writable when Lite is on. Readable when Lite is off and Debug features is on.
intel_vvp_adv_tpg_get_lite_mode	Returns if Lite mode is on
intel_vvp_adv_tpg_get_debug_enabled	Returns if Debug enabled is on
intel_vvp_adv_tpg_get_bits_per_sample	Returns the number of bits per color sample
intel_vvp_adv_tpg_get_pixels_in_parallel	Returns the number of pixels processed in parallel per clock cycle
intel_vvp_adv_tpg_get_color_space	Returns the color space and subsampling used across all patterns
intel_vvp_adv_tpg_get_pattern_type	Returns the output pattern type of a given pattern number
intel_vvp_adv_tpg_get_num_patterns	Returns the number of patterns enabled
intel_vvp_adv_tpg_commit_writes	Commits all outstanding writes excluding pattern configuration registers
intel_vvp_adv_tpg_get_status	Returns the contents of the status register
intel_vvp_adv_tpg_is_running	Returns whether the IP is currently generating video data
intel_vvp_adv_tpg_get_commit_status	Returns true if the IP has any outstanding writes, excluding writes to pattern configuration registers
intel_vvp_adv_tpg_get_fields_since_last_start	Returns the number of fields the IP generates since it was last started
intel_vvp_adv_tpg_enable	Start/stop the IP
intel_vvp_adv_tpg_is_enabled	Returns whether the IP is currently enabled
intel_vvp_adv_tpg_start	Stop the IP; alias for intel_vvp_adv_tpg_enable
intel_vvp_adv_tpg_stop	Start the IP; alias for intel_vvp_adv_tpg_enable
intel_vvp_adv_tpg_get_field_width	Returns the width of outgoing video fields.
intel_vvp_adv_tpg_set_field_width	Set the width of outgoing video fields
intel_vvp_adv_tpg_get_field_height	Returns the height of outgoing video fields.
intel_vvp_adv_tpg_set_field_height	Set the height of outgoing video fields
intel_vvp_adv_tpg_get_field_interlace	Returns the current interlace configuration/nibble
intel_vvp_adv_tpg_set_field_interlace	Set the interlace configuration/nibble of outgoing video fields
intel_vvp_adv_tpg_get_pattern_width	Returns the configured width of test patterns
intel_vvp_adv_tpg_set_pattern_width	Set the width of test patterns
intel_vvp_adv_tpg_get_pattern_height	Returns the configured height of test patterns
continued...	

intel_vvp_adv_tpg_set_pattern_height	Sets the height of test patterns
intel_vvp_adv_tpg_get_pattern_alpha	Returns the opaqueness of test patterns
intel_vvp_adv_tpg_set_pattern_alpha	Set the opaqueness of test patterns
intel_vvp_adv_tpg_get_pattern_horiz_offset	Returns the configured horizontal offset of test patterns
intel_vvp_adv_tpg_set_pattern_horiz_offset	Sets the horizontal offset from the left of test patterns
intel_vvp_adv_tpg_get_pattern_vert_offset	Return the configured vertical offset of test patterns
intel_vvp_adv_tpg_set_pattern_vert_offset	Set the vertical offset from the left of test patterns

Table 116. Advanced Test Pattern Generator IP Software API

These API functions do not need a call to `intel_vvp_adv_tpg_commit_writes` to take effect; writing to the `PATTERN_X_SELECT` register is selfcontained.

Name	Description
intel_vvp_adv_tpg_setBarsType	Select a type of color bars for the bars pattern
intel_vvp_adv_tpg_setConstantColors	Select the color for the constant color pattern
intel_vvp_adv_tpg_setZonePlateSettings	Sets the zone plate settings
intel_vvp_adv_tpg_setZonePlateSettings_s	Sets the zone plate settings using a helper structure
intel_vvp_adv_tpg_setDigitalClockSettings	Sets the digital clock settings
intel_vvp_adv_tpg_setDigitalClockSettings_s	Sets the digital clock settings using a helper structure

enum eIntelAdvTpgPatternType

Members

- kIntelVvpAdvTpgBarsPattern - Bars pattern
- kIntelVvpAdvTpgConstantColorPattern - Uniform pattern
- kIntelVvpAdvTpgPathologicalPattern - Pattern that stresses SDI receiving equipment
- kIntelVvpAdvTpgZonePlatePattern - Zone plate pattern
- kIntelVvpAdvTpgDigitalClockPattern - Digital clock pattern
- kIntelVvpAdvTpgInvalidPattern - Invalid pattern (indicates errors)

Description Enumeration for the supported pattern types

enum eIntelAdvTpgColorSpace

Members

- kIntelVvpAdvTpgRgb - RGB 4:4:4
- kIntelVvpAdvTpgYcc444 - YcbCr/YUV 4:4:4
- kIntelVvpAdvTpgYcc422 - YcbCr/YUV 4:2:2
- kIntelVvpAdvTpgYcc420 - YcbCr/YUV 4:2:0

kIntelVvpAdvTpgMono - Monochrome

kIntelVvpAdvTpgInvalidColor - Invalid color space/subsampling (indicates errors)

Description Enumeration for the supported color spaces and subsampling

enum eIntelAdvTpgBarsType

Members Members kIntelVvpAdvTpgColorBars - Standard color bars

kIntelVvpAdvTpgGreyBars - Grey bars

kIntelVvpAdvTpgBlackWhiteBars - Alternating black and white bars

kIntelVvpAdvTpgMixedBars - Mixed type bars

kIntelVvpAdvTpgInvalidBars - Invalid bars pattern (indicates errors)

Description Enumeration for the supported color bar patterns

struct intel_vvp_adv_tpg_zone_plate_settings

Members uint16_t origin_x - The desired zone plate origin x-coordinate

uint16_t origin_y - The desired zone plate origin y-coordinate

uint16_t coarse_factor - The coarse scaling coefficient of the zone plate

uint16_t fine_factor - The fine-tune scaling coefficient of the zone plate

Description Structure to contain all necessary information about a single instance of the zone plate pattern.

struct intel_vvp_adv_digital_clock_settings

Members uint8_t config_bits - Control bits to pause or reset the display of the digital clock

uint16_t background_b - The desired digital clock background color (B/Cb component)

uint16_t background_g - The desired digital clock background color (G/Y component)

uint16_t background_r - The desired digital clock background color (R/Cr component) uint16_t font_b - The desired digital clock font color (B/Cb component)

uint16_t font_g - The desired digital clock font color (G/Y component)

uint16_t font_r - The desired digital clock font color (R/Cr component)

uint16_t location_x - The desired digital clock position x-coordinate

uint16_t location_y - The desired digital clock position y-coordinate

uint16_t scale_factor - The digital clock scaling multiplier

uint16_t fps - The value for the digital clock to determine after how many fields one second passes

Description Structure to contain all necessary information about a single instance of the digital clock pattern.

intel_vvp_adv_tpg_init

Prototype

```
int intel_vvp_adv_tpg_init(intel_vvp_adv_tpg_instance
*instance, intel_vvp_core_base base);
```

Description Initializes an Advanced Test Pattern Generator instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store IP compile-time parameterization. The instance does not fully initialize and should not be used by the application if returning a non-zero error code.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure
base - base address of the register map

Returns kIntelVvpCoreOk (0) in case of success; a negative error code in case of error
kIntelVvpCoreVidErr if the vendor ID of the IP is not the IntelFPGA vendor ID (0x6AF7)
kIntelVvpCorePidErr if the product_id does not match with the Advanced Test Pattern Generator product ID (0x0252)
kIntelVvpCoreInstanceErr if the instance is a null pointer
kIntelVvpAdvTpgRegMapVerisonErr if the register map is not supported

intel_vvp_adv_tpg_get_lite_mode

Prototype

```
bool
intel_vvp_adv_tpg_get_lite_mode(intel_vvp_adv_tpg_instance
*instance);
```

Description Returns the value of the LITE_MODE register. The instance must be a fully initialized valid intel_vvp_adv_tpg_instance

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

Returns true if the IP is in lite mode; false otherwise

intel_vvp_adv_tpg_get_debug_enabled

Prototype

```
bool
intel_vvp_adv_tpg_get_debug_enabled(intel_vvp_adv_tpg_instance
*instance);
```

Description Returns the value of the DEBUG_ENABLED register. The instance must be a fully initialized valid intel_vvp_adv_tpg_instance

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

Returns true if debug features is on; false otherwise

intel_vvp_adv_tpg_get_bits_per_sample

Prototype

```
uint8_t intel_vvp_adv_tpg_get_bps(intel_vvp_adv_tpg_instance
*instance);
```

Description Returns the value of the BPS register.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

Returns The number of bits per color sample set at compile time

intel_vvp_adv_tpg_get_pixels_in_parallel

Prototype

```
uint8_t
intel_vvp_adv_tpg_get_pixels_in_parallel(intel_vvp_adv_tpg_instance
*instance);
```

Description Returns the value of the PIXELS_IN_PARALLEL register. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

Returns The number of pixels generated in parallel per clock cycle

intel_vvp_adv_tpg_get_color_space

Prototype

```
eIntelVvpAdvTpgColorSpace
intel_vvp_adv_tpg_get_color_space(intel_vvp_adv_tpg_instance
*instance);
```

Description Returns the color space that the IP has been configured with. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

Returns eIntelVvpAdvTpgColorSpace corresponding to the color space and subsampling used across all patterns.

intel_vvp_adv_tpg_get_pattern_type

Prototype

```
eIntelVvpAdvTpgPatternType
intel_vvp_adv_tpg_get_pattern_type(intel_vvp_adv_tpg_instance
*instance, int pattern_num);
```

Description

Arguments

Returns

Prototype

Description Returns the pattern type of a given pattern number. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

Returns pattern_num - The pattern number to query.
eIntelVvpAdvTpgPatternType corresponding to the queried pattern number
kIntelVvpAdvTpgInvalidPattern if pattern_num is greater than or equal to the number of patterns.

intel_vvp_adv_tpg_get_num_patterns

Prototype

```
int
intel_vvp_adv_tpg_get_num_patterns(intel_vvp_adv_tpg_instance
*instance);
```

Description Returns the value of the NUM_PATTERNS register, that is the number of patterns enabled at compile-time. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

Returns The number of patterns configured at compile time

intel_vvp_adv_tpg_commit_writes

Prototype

```
int
intel_vvp_adv_tpg_commit_writes(intel_vvp_adv_tpg_instance*
instance);
```

Description Commit all outstanding writes before starting the next field. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

Returns Returns kIntelVvpCoreOk (0) in case of success.
kIntelVvpCoreInstanceErr if the instance is null.

intel_vvp_adv_tpg_get_status

Prototype

```
int intel_vvp_adv_tpg_get_status(intel_vvp_adv_tpg_instance
*instance);
```

Description Returns the value of the STATUS register. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

Returns The value read from the STATUS register

intel_vvp_adv_tpg_is_running

Prototype

```
bool intel_vvp_adv_tpg_is_running(intel_vvp_adv_tpg_instance*
instance);
```

Description Returns the running bit of the STATUS register, where true indicates that the IP is running. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

Returns True if the IP is currently generating an output field, false if between fields or the IP stops.

intel_vvp_adv_tpg_get_commit_status

Prototype

```
bool  
intel_vvp_adv_tpg_get_commit_status(intel_vvp_adv_tpg_instance*  
instance);
```

Description Returns the commit pending bit of the STATUS register, where true indicates that there are outstanding register writes. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

Returns True if you have outstanding writes to the IP, false once you commit all write.

intel_vvp_adv_tpg_enable

Prototype

```
int intel_vvp_adv_tpg_enable(intel_vvp_adv_tpg_instance*  
instance, bool enabled);
```

Description Start or stop the IP after the current field. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

enabled - True to start the IP and generate fields, false to stop the IP after the current field.

Returns Returns kIntelVvpCoreOk (0) in case of success.

kIntelVvpCoreInstanceErr if the instance is null.

intel_vvp_adv_tpg_is_enabled

Prototype

```
bool intel_vvp_adv_tpg_is_enabled(intel_vvp_adv_tpg_instance*  
instance);
```

Description Returns whether the IP is enabled. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance, and debug features is on.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

Returns True if the IP is enabled (started), false if the IP is stopped or instance is invalid or debug features is off.

intel_vvp_adv_tpg_start

Prototype

```
int intel_vvp_adv_tpg_start(intel_vvp_adv_tpg_instance* instance);
```

Description Start the IP; alias for intel_vvp_adv_tpg_enable(instance, true). The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

Returns kIntelVvpCoreOk (0) in case of success.
kIntelVvpCoreInstanceErr if the instance is null.

intel_vvp_adv_tpg_stop

Prototype

```
int intel_vvp_adv_tpg_stop(intel_vvp_adv_tpg_instance* instance);
```

Description Stop the IP after the current field; alias for intel_vvp_adv_tpg_enable(instance, false). The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

Returns kIntelVvpCoreOk (0) in case of success.
kIntelVvpCoreInstanceErr if the instance is null.

intel_vvp_adv_tpg_get_fields_since_last_start

Prototype

```
uint32_t intel_vvp_adv_tpg_get_fields_since_last_start(intel_vvp_adv_tpg_instance* instance);
```

<i>Description</i>	Returns the number of fields the IP generates since it last started. The instance must be a valid and initialized instance of <code>intel_vvp_adv_tpg_instance</code> .
<i>Arguments</i>	<code>instance</code> - pointer to the <code>intel_vvp_adv_tpg_instance</code> software driver instance structure
<i>Returns</i>	The number of fields generated since the IP was last started (or 0xFFFFFFFF if instance is null).

`intel_vvp_adv_tpg_get_field_width`

Prototype

```
uint32_t  
intel_vvp_adv_tpg_get_field_width(intel_vvp_adv_tpg_instance*  
instance);
```

<i>Description</i>	Returns the configured width of generated fields. The instance must be a valid and initialized instance of <code>intel_vvp_adv_tpg_instance</code> , and debug features must be enabled.
<i>Arguments</i>	<code>instance</code> - pointer to the <code>intel_vvp_adv_tpg_instance</code> software driver instance structure
<i>Returns</i>	The field width (or 0xFFFFFFFF if instance is null and debug features is off).

`intel_vvp_adv_tpg_set_field_width`

Prototype

```
int  
intel_vvp_adv_tpg_set_field_width(intel_vvp_adv_tpg_instance*  
instance, uint32_t width);
```

<i>Description</i>	Returns the configured width of generated fields. The instance must be a valid and initialized instance of <code>intel_vvp_adv_tpg_instance</code> .
<i>Arguments</i>	<code>instance</code> - pointer to the <code>intel_vvp_adv_tpg_instance</code> software driver instance structure
<i>Returns</i>	<code>kIntelVvpCoreOk</code> (0) in case of success. <code>kIntelVvpCoreInstanceErr</code> if the instance is null or debug features is off.

`intel_vvp_adv_tpg_get_field_height`

Prototype

```
uint32_t  
intel_vvp_adv_tpg_get_field_height(intel_vvp_adv_tpg_instance*  
instance);
```

Description Returns the configured height of generated fields. The instance must be a valid and initialized instance of `intel_vvp_adv_tpg_instance`, and debug features must be on.

Arguments instance - pointer to the `intel_vvp_adv_tpg_instance` software driver instance structure

Returns The field height (or 0xFFFFFFFF if instance is null and debug features is off).

`intel_vvp_adv_tpg_set_field_height`

Prototype

```
int
intel_vvp_adv_tpg_set_field_height(intel_vvp_adv_tpg_instance*
instance, int32_t height);
```

Description Returns the configured width of generated fields. The instance must be a valid and initialized instance of `intel_vvp_adv_tpg_instance`.

Arguments instance - pointer to the `intel_vvp_adv_tpg_instance` software driver instance structure

height - The desired field height

Returns `kIntelVvpCoreOk` (0) in case of success.

`kIntelVvpCoreInstanceErr` if the instance is null or debug features is off.

`intel_vvp_adv_tpg_get_field_interlace`

Prototype

```
uint8_t
intel_vvp_adv_tpg_get_field_interlace(intel_vvp_adv_tpg_instance*
instance);
```

Description Returns the set interlace configuration of generated fields. The instance must be a valid and initialized instance of `intel_vvp_adv_tpg_instance`, and debug features must be on.

Arguments instance - pointer to the `intel_vvp_adv_tpg_instance` software driver instance structure

Returns The field width (or 0xFF if instance is null and debug features is off).

`intel_vvp_adv_tpg_set_field_interlace`

Prototype

```
int
intel_vvp_adv_tpg_set_field_interlace(intel_vvp_adv_tpg_instance*
instance, uint8_t interlace);
```


<i>Description</i>	Set the interlace configuration of generated fields and the value to populate the interlace nibble in image information metapackets. Write 0-7 for progressive output, 8-11 for F0 first interlaced output, and 12-15 for F1 first interlaced output. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.
<i>Arguments</i>	instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure interlace - The desired interlace configuration/nibble
<i>Returns</i>	kIntelVvpCoreOk (0) in case of success. kIntelVvpCoreInstanceErr if the instance is null or debug features is off.

intel_vvp_adv_tpg_get_pattern_width

<i>Prototype</i>	<pre>uint32_t intel_vvp_adv_tpg_get_pattern_width(intel_vvp_adv_tpg_instance* instance, uint8_t pattern_num);</pre>
<i>Description</i>	Returns the configured width of a given pattern excluding pattern 0. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance, and debug features must be on.
<i>Arguments</i>	instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure pattern_num - The pattern number to query
<i>Returns</i>	The configured pattern width (or 0xFFFFFFFF if instance is null/ or debug features is off or pattern_num is 0).

intel_vvp_adv_tpg_set_pattern_width

<i>Prototype</i>	<pre>int intel_vvp_adv_tpg_set_pattern_width(intel_vvp_adv_tpg_instance* instance, uint8_t pattern_num, uint32_t width);</pre>
<i>Description</i>	Set the width of a given pattern excluding pattern 0. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.
<i>Arguments</i>	instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure pattern_num - The pattern number corresponding to the desired pattern. width - The desired width to configure a pattern with

Returns

- kIntelVvpCoreOk (0) in case of success.
- kIntelVvpCoreInstanceErr if the instance is null.
- kIntelVvpAdvTpgPatternZeroErr if pattern_num is 0
- kIntelVvpAdvTpgInvalidPatternErr if pattern_num is greater or equal to the number of patterns.

intel_vvp_adv_tpg_get_pattern_height

Prototype

```
uint32_t
intel_vvp_adv_tpg_get_pattern_height(intel_vvp_adv_tpg_instance*
instance, uint8_t pattern_num);
```

Description Returns the configured height of a given pattern excluding pattern 0. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance, and debug features must be on.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

pattern_num - The pattern number to query

Returns The configured pattern height (or 0xFFFFFFFF if instance is null or debug features is off or pattern_num is 0).

intel_vvp_adv_tpg_set_pattern_height

Prototype

```
int
intel_vvp_adv_tpg_set_pattern_height(intel_vvp_adv_tpg_instance*
instance, uint8_t pattern_num, uint32_t height);
```

Description Set the height of a given pattern excluding pattern 0. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

pattern_num - The pattern number corresponding to the desired pattern.

height - The desired height to configure a pattern with

Returns

- kIntelVvpCoreOk (0) in case of success.
- kIntelVvpCoreInstanceErr if the instance is null.
- kIntelVvpAdvTpgPatternZeroErr if pattern_num is 0
- kIntelVvpAdvTpgInvalidPatternErr if pattern_num is greater or equal to the number of patterns.

intel_vvp_adv_tpg_get_pattern_alpha

Prototype

```
uint16_t  
intel_vvp_adv_tpg_get_pattern_alpha(intel_vvp_adv_tpg_instance*  
instance, uint8_t pattern_num);
```

Description

Returns the configured alpha value (opacity) of a given pattern excluding pattern 0. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance, and debug features must be on.

Arguments

instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

pattern_num - The pattern number to query

Returns

The configured pattern alpha (or 0xFFFF if instance is null or debug features is off or pattern_num is 0).

intel_vvp_adv_tpg_set_pattern_alpha

Prototype

```
int  
intel_vvp_adv_tpg_set_pattern_alpha(intel_vvp_adv_tpg_instance*  
instance, uint8_t pattern_num, uint16_t alpha);
```

Description

Set the alpha value (opacity) of a given pattern excluding pattern 0. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.

Arguments

instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

pattern_num - The pattern number corresponding to the desired pattern.

alpha - The desired alpha to configure a pattern with

Returns

kIntelVvpCoreOk (0) in case of success.

kIntelVvpCoreInstanceErr if the instance is null.

kIntelVvpAdvTpgColorErr if alpha is greater than the maximum allowed by the bits per color sample

kIntelVvpAdvTpgPatternZeroErr if pattern_num is 0

kIntelVvpAdvTpgInvalidPatternErr if pattern_num is greater or equal to the number of patterns

intel_vvp_adv_tpg_get_pattern_horiz_offset

Prototype

```
uint32_t
intel_vvp_adv_tpg_get_pattern_horiz_offset(intel_vvp_adv_tpg_instance*
instance, uint8_t pattern_num);
```

Description Returns the configured horizontal offset (from the left) of a given pattern excluding pattern 0. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance, and debug features must be on.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

pattern_num - The pattern number to query

Returns The configured pattern horizontal offset (or 0xFFFFFFFF if instance is null or debug features is off or pattern_num is 0).

intel_vvp_adv_tpg_set_pattern_horiz_offset

Prototype

```
int
intel_vvp_adv_tpg_set_pattern_horiz_offset(intel_vvp_adv_tpg_instance*
instance, uint8_t pattern_num, uint32_t h_offset);
```

Description Set the horizontal offset (from the left) of a given pattern excluding pattern 0. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

pattern_num - The pattern number corresponding to the desired pattern.

h_offset - The desired horizontal offset to configure a pattern with

Returns kIntelVvpCoreOk (0) in case of success.

kIntelVvpCoreInstanceErr if the instance is null.

kIntelVvpAdvTpgPatternZeroErr if pattern_num is 0

kIntelVvpAdvTpgInvalidPatternErr if pattern_num is greater or equal to the number of patterns.

intel_vvp_adv_tpg_get_pattern_vert_offset

Prototype

```
uint32_t
intel_vvp_adv_tpg_get_pattern_vert_offset(intel_vvp_adv_tpg_instance*
instance, uint8_t pattern_num);
```

Description Returns the configured vertical offset (from the top) of a given pattern excluding pattern 0. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance, and debug features must be on.

- Arguments** instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure
- pattern_num - The pattern number to query
- Returns** The configured pattern vertical offset (or 0xFFFFFFFF if instance is null and debug features is off and pattern_num is 0).

intel_vvp_adv_tpg_set_pattern_vert_offset

Prototype

```
int  
intel_vvp_adv_tpg_set_pattern_vert_offset(intel_vvp_adv_tpg_instance*  
instance, uint8_t pattern_num, uint32_t v_offset);
```

- Description** Set the vertical offset (from the top) of a given pattern excluding pattern 0. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.

- Arguments** instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure
- pattern_num - The pattern number corresponding to the desired pattern.
- v_offset - The desired vertical offset to configure a pattern with

- Returns** kIntelVvpCoreOk (0) in case of success.
- kIntelVvpCoreInstanceErr if the instance is null.
- kIntelVvpAdvTpgPatternZeroErr if pattern_num is 0
- kIntelVvpAdvTpgInvalidPatternErr if pattern_num is greater or equal to the number of patterns.

intel_vvp_tpg_setBars_type

Prototype

```
int  
intel_vvp_adv_tpg_setBars_type(intel_vvp_adv_tpg_instance*  
instance, uint8_t pattern_num, eIntelVvpAdvTpgBarsType  
bar_type);
```

- Description** Selects the type of color bars (black/white, grey, color, mixed) of a given pattern of type kIntelVvpAdvTpgBarsPattern. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.

- Arguments** instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure
- pattern_num - The pattern number corresponding to the desired pattern
- bar_type - The type of bars pattern

Returns

- kIntelVvpCoreOk (0) in case of success.
- kIntelVvpCoreInstanceErr if the instance is null.
- kIntelVvpAdvTpgPatternZeroErr if pattern_num is 0
- kIntelVvpAdvTpgInvalidPatternErr if pattern_num is greater or equal to the number of patterns, or if the pattern_num does not correspond to a pattern of type
- kIntelVvpAdvTpgBarsPattern

intel_vvp_tpg_set_constant_colors

Prototype

```
int
intel_vvp_adv_tpg_set_constant_colors(intel_vvp_adv_tpg_instance*
instance, uint8_t pattern_num, uint16_t color1, uint16_t color2,
uint16_t color3);
```

Description Set the color of a given pattern of type kIntelVvpAdvTpgConstantColorPattern. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.

Arguments instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

pattern_num - The pattern number corresponding to the desired pattern

color1, color2, color3 - The new color sample values

Returns

- kIntelVvpCoreOk (0) in case of success.
- kIntelVvpCoreInstanceErr if the instance is null.
- kIntelVvpAdvTpgPatternZeroErr if pattern_num is 0
- kIntelVvpAdvTpgInvalidPatternErr if pattern_num is greater or equal to the number of patterns, or if the pattern_num does not correspond to a pattern of type
- kIntelVvpAdvTpgConstantColorPattern

kIntelVvpAdvTpgColorErr if any of the provided color sample values are greater than the maximum allowed by the bits per color sample

intel_vvp_tpg_set_zone_plate_settings

Prototype

```
int
intel_vvp_adv_tpg_set_zone_plate_settings(intel_vvp_adv_tpg_instance*
instance, uint8_t pattern_num, uint16_t x_coord, uint16_t y_coord,
uint16_t coarse_factor, uint16_t fine_tune_factor);
```

<i>Description</i>	Configure the scaling and origin coordinates of a given pattern of type <code>kIntelVvpAdvTpgZonePlatePattern</code> . The instance must be a valid and initialized instance of <code>intel_vvp_adv_tpg_instance</code> .
<i>Arguments</i>	<code>instance</code> - pointer to the <code>intel_vvp_adv_tpg_instance</code> software driver instance structure <code>pattern_num</code> - The pattern number corresponding to the desired pattern <code>x_coord</code> , <code>y_coord</code> - The coordinates of the zone plate origin <code>coarse_factor</code> - The coarse scaling coefficient of the zone plate <code>fine_tune_factor</code> - The fine-tune scaling coefficient of the zone plate; this is a 16-bit fixed point number with 8 fractional bits.
<i>Returns</i>	<code>kIntelVvpCoreOk</code> (0) in case of success. <code>kIntelVvpCoreInstanceErr</code> if the instance is null. <code>kIntelVvpAdvTpgPatternZeroErr</code> if <code>pattern_num</code> is 0 <code>kIntelVvpAdvTpgInvalidPatternErr</code> if <code>pattern_num</code> is greater or equal to the number of patterns, or if the <code>pattern_num</code> does not correspond to a pattern of type <code>kIntelVvpAdvTpgZonePlatePattern</code>

`intel_vvp_tpg_set_zone_plate_settings_s`

Prototype

```
int  
intel_vvp_adv_tpg_set_zone_plate_settings_s(intel_vvp_adv_tpg_instance*  
instance, uint8_t pattern_num, intel_vvp_adv_tpg_zone_plate_settings  
*settings_struct);
```

<i>Description</i>	Configure the scaling and origin coordinates of a given pattern of type <code>kIntelVvpAdvTpgZonePlatePattern</code> . The instance must be a valid and initialized instance of <code>intel_vvp_adv_tpg_instance</code> .
<i>Arguments</i>	<code>instance</code> - pointer to the <code>intel_vvp_adv_tpg_instance</code> software driver instance structure <code>pattern_num</code> - The pattern number corresponding to the desired pattern <code>settings_struct</code> - Pointer to the struct containing zone plate configuration
<i>Returns</i>	<code>kIntelVvpCoreOk</code> (0) in case of success. <code>kIntelVvpCoreInstanceErr</code> if the instance is null. <code>kIntelVvpAdvTpgPatternZeroErr</code> if <code>pattern_num</code> is 0 <code>kIntelVvpAdvTpgInvalidPatternErr</code> if <code>pattern_num</code> is greater or equal to the number of patterns, or if the <code>pattern_num</code> does not correspond to a pattern of type

kIntelVvpAdvTpgZonePlatePattern

intel_vvp_tpg_set_digital_clock_settings

Prototype

```
int
intel_vvp_adv_tpg_set_digital_clock_settings(intel_vvp_adv_tpg_instance*
instance, uint8_t pattern_num, uint8_t config_bits, uint16_t
background_r, uint16_t background_g, uint16_t background_b, uint16_t
font_r, uint16_t font_g, uint16_t font_b, uint16_t location_x, uint16_t
location_y, uint16_t scale_factor, uint8_t fps);
```

Description

Configure colors, location, size and control values of a given pattern of type kIntelVvpAdvTpgDigitalClockPattern using the helper struct. The instance must be a valid and initialized instance of intel_vvp_adv_tpg_instance.

Arguments

instance - pointer to the intel_vvp_adv_tpg_instance software driver instance structure

pattern_num - The pattern number corresponding to the desired pattern

config_bits - Set bit 0 to pause the digital display, set bit 1 to reset the digital display

background_b, background_g, background_r - The background color samples

font_b, font_g, font_r - The font color samples

location_x, location_y - The location for the digital clock display

scale_factor - The digital display sizing coefficient. If 1, the display is 88x88 pixels.

fps - The number of frames the IP generates for one second to have elapsed.

Returns

kIntelVvpCoreOk (0) in case of success

kIntelVvpCoreInstanceErr if the instance is null.

kIntelVvpAdvTpgPatternZeroErr if pattern_num is 0

kIntelVvpAdvTpgInvalidPatternErr if pattern_num is greater or equal to the number of patterns, or if the pattern_num does not correspond to a pattern of type

kIntelVvpAdvTpgDigitalClockPattern

kIntelVvpAdvTpgColorErr if any of the background or font color samples are greater than the maximum allowed by the bits per color sample

intel_vvp_tpg_set_digital_clock_settings_s

Prototype

```
int
intel_vvp_adv_tpg_set_digital_clock_settings_s(intel_vvp_adv_tpg_instance*
instance, uint8_t pattern_num, intel_vvp_adv_tpg_digital_clock_settings
*settings_struct);
```


<i>Description</i>	Configure colors, location, size and control values of a given pattern of type <code>kIntelVvpAdvTpgDigitalClockPattern</code> using the helper struct. The instance must be a valid and initialized instance of <code>intel_vvp_adv_tpg_instance</code> .
<i>Arguments</i>	<code>instance</code> - pointer to the <code>intel_vvp_adv_tpg_instance</code> software driver instance structure <code>pattern_num</code> - The pattern number corresponding to the desired pattern <code>settings_struct</code> - Pointer to the struct containing the digital clock configuration
<i>Returns</i>	<code>kIntelVvpCoreOk</code> (0) in case of success. <code>kIntelVvpCoreInstanceErr</code> if the instance is null. <code>kIntelVvpAdvTpgPatternZeroErr</code> if <code>pattern_num</code> is 0 <code>kIntelVvpAdvTpgInvalidPatternErr</code> if <code>pattern_num</code> is greater or equal to the number of patterns, or if the <code>pattern_num</code> does not correspond to a pattern of type <code>kIntelVvpAdvTpgZonePlatePattern</code> <code>kIntelVvpAdvTpgColorErr</code> if any of the background or font color samples are greater than the maximum allowed by the bits per color sample

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38



12. AXI-Stream Broadcaster IP

12.1. About the AXI-Stream Broadcaster IP

The IP broadcasts an input video streaming to multiple output video streaming interfaces. Both video input and output interfaces work on the same clock domain. The IP comprises the main AXI4-Stream broadcast logic and synchronous streaming FIFO buffers.

The broadcast logic replicates the video streaming input bus to N outputs. You specify N at build time. The IP offers full, lite, and full-raster variants of Altera Streaming Video. For more information, refer to the Altera Streaming Video Protocol Specification.

To process `TREADY` backpressure on the output interfaces, each output has either a FIFO buffer of configurable depth, or a shim, which is equivalent to a 1-depth FIFO buffer. If an output deasserts its `TREADY`, its FIFO buffer continues to accept input until it is full. If any one of the output FIFO buffers is full and you turn on **Global stall**, the broadcaster stalls the input by deasserting its `TREADY`. The IP stops new input from filling any of the FIFO buffers until all the FIFO buffers are ready to accept new data. If you turn off **Global stall** option, the broadcaster input never stalls but the FIFO buffers drop new input when they are full. In either case, each output always asserts `TVALID` when data is present in its FIFO buffer.

Typically, you use full-raster variants for real-time video. The input and output `TREADY` signals are entirely optional. Disabling `TREADY` on any interface removes the signal and stops any backpressure on that interface. For outputs, this action replaces the FIFO buffer with a simple register stage. If you do not turn on `TREADY` on any of the outputs, global stall has no effect and the IP removes the option.

With these different backpressure options available, you consider the system design and determine what amount of backpressure is acceptable on each interface. For example, a processing pipeline operating on non-real-time video streams might want the global stall option. However, in a system where the broadcaster is dealing with real-time video, stalling the input might be unacceptable. Output FIFO buffers must be large enough so that no pushback reaches the broadcaster.

Related Information

[Altera Streaming Video Protocol Specification](#)

12.1.1. AXI-Stream Broadcaster IP Features

- Altera Streaming Video compliant input interface
- Altera Streaming Video compliant output interface
- Optional `TREADY` signals for full-raster variants

- Optional global stall
- Optional output FIFO buffers up to 32K depth
- 8-bit to 16-bit per color component
- 1 to 4 color planes per pixel
- 1 to 8 pixels in parallel
- Small FPGA resource footprint

12.1.2. AXI-Stream Broadcaster IP Performance and Resources

The IP resources depend on the device family, number of supported output ports, bits per sample, pixels in parallel, number of color planes, and maximum depth of video line buffer.

Table 117. AXI-Stream Broadcaster IP Performance and Resources

The table shows ALM usage and f_{MAX} for a design with 2 output video interfaces, 2 pixels in parallel, 10 bits per color sample and 3 color planes.

Target Device	ALMs	M20Ks	f_{MAX} MHz
Agilex 7 (AGFA012R24A2E2V)	279	0	981
Arria 10 (10AS066H1F34E1HG)	222	0	645
Cyclone 10 GX (10CX220YF672E5G)	222	0	645
Stratix 10 (1SX280LN2F43E1VG)	272	0	848

12.2. AXI-Stream Broadcaster IP Parameters

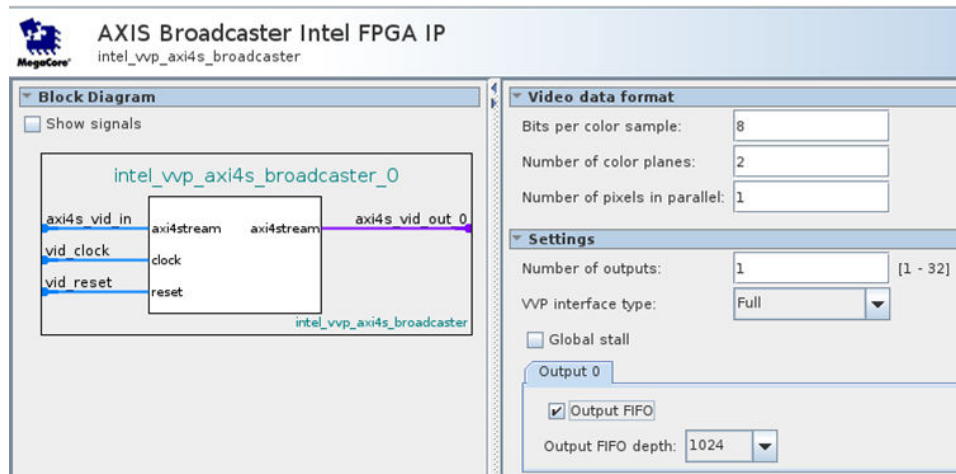
The IP offers compile-time parameters.

Table 118. AXI-Stream Broadcaster IP Parameters

Parameter	Values	Description
Video Data Format		
Bits per color sample	8 to 16	Select the number of pixels in parallel.
Number of color planes	1 to 4	Select the number of color planes per pixel.
Number of pixels in parallel	1 to 8	Select the number of bits per color sample.
Settings (Global)		
Number of outputs	1 to 32	The number of broadcaster outputs
VVP interface type	Full, Lite or Full Raster (FR)	The interface type of the AXI4-S input and outputs
Input TREADY	On or off	The input has TREADY
Global stall	On or off	If at least one output's TREADY signal is low, deassert the input TREADY and stall all output FIFO buffers.
Settings (Per Output Interface)		
<i>continued...</i>		

Parameter	Values	Description
Output TREADY	On or off	The output has TREADY
Output FIFO	On or off	Add a FIFO buffer on the output interface to handle TREADY backpressure
Output FIFO depth	16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768	The maximum depth of the output FIFO buffer for handling TREADY backpressure

Figure 24. AXI-Stream Broadcaster IP Parameter GUI



12.3. AXI-Stream Broadcaster IP Interfaces

Table 119. AXI-Stream Broadcaster IP Interfaces

Name	Direction	Width	Description
Clocks and Resets			
vid_clock	In	1	AXI4-S processing clock
vid_reset	In	1	AXI4-S processing reset
Altera Streaming Video interfaces			
axi4s_vid_in_tdata	In	(29)	AXI4-S data in
axi4s_vid_in_tvalid	In	1	AXI4-S data valid
axi4s_vid_in_tuser[0]	In	1	AXI4-S start of video frame
continued...			

(29) The equation gives the TDATA width for these interfaces for full or lite variants:
 $\max(\text{floor}(((\text{bits per color sample} \times \text{number of color planes} \times \text{pixels in parallel}) + 7) / 8) \times 8, 16)$
 The equation gives the TDATA width for these interfaces for full-raster variants:
 $\max(\text{floor}(((\text{bits per color sample} \times (\text{number of color planes} + 1) \times \text{pixels in parallel}) + 7) / 8) \times 8, 16)$

Name	Direction	Width	Description
axi4s_vid_in_tuser[1]	In	1	- Control or data packet (full variant) - Field Flag for interlaced formats (lite variant) - Not used (full-raster variant)
axi4s_vid_in_tuser[N-1:2]	In	(30)	Unused
axi4s_vid_in_tlast	In	1	AXI4-S end of packet
axi4s_vid_in_tready	Out	1	AXI4-S data ready
axi4s_vid_out_x_tdata	Out	(29)	AXI4-S data out
axi4s_vid_out_x_tvalid	Out	1	AXI4-S data valid
axi4s_vid_out_x_tuser[0]	Out	1	AXI4-S start of video frame
axi4s_vid_out_x_tuser[1]	Out	1	- Control or data packet (full variant) - Field flag for interlaced formats (lite variant) - Not used (full-raster variant)
axi4s_vid_out_x_tuser[N-1:2]	Out	(30)	Unused
axi4s_vid_out_x_tlast	Out	1	AXI4-S end of packet
axi4s_vid_out_tready	In	1	AXI4-S data ready

Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

(30) This equation gives the TUSER width N for these interfaces: $\text{ceil}(\text{tdata width} / 8)$

13. Bits per Color Sample Adapter IP

13.1. About the Bits per Color Sample Adapter IP

The Altera Streaming Video Protocol requires you to configure each interface for a maximum number of bits to represent each color sample. This setting determines the width of the `tdata` bus. The IP allows you to connect two interfaces with different settings for bits per color sample. The IP offers dithering that adds noise to the signal when connecting two interfaces with different settings for bits per color sample.

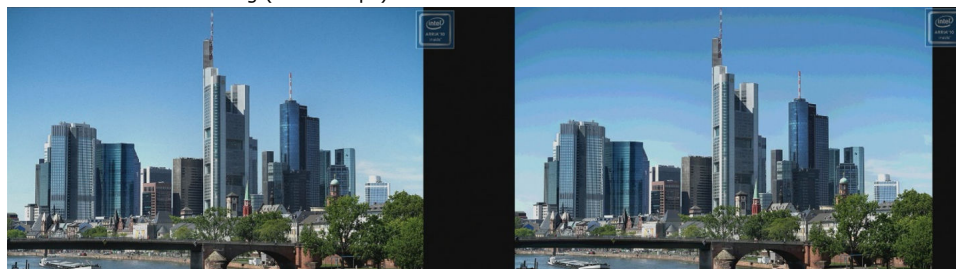
When the input bits per color sample is less than the output bits per color sample, the IP adds the required number of bits at the LSB end of the `tdata` bus. The IP fixes these additional bits to 0. When the input bits per color sample is greater than the output bits per color sample, the IP clips the required number of bits from the LSB end of each color sample.

When you configure the IP for use with the lite variant of the *Altera Streaming Video interface protocol*, it consumes no FPGA resources as the clip and pad operations do not require any. However, when you configure the IP for use with the full variant of the protocol, a small amount of additional logic is required to ensure that the clip and pad operations do not affect the data in the nonvideo packets. The full variant of the protocol includes image information packets, and these packets contain a field indicating how many of the bits in each color plane are active for the current video stream.

When using the full variant of the protocol, you can include an Avalon Memory-mapped agent control interface to control the output bits per color field value at runtime via the register map. If you do not turn on **Memory mapped control interface**, the IP uses the default value for the bits per color field. When converting from a lower input bits per color sample value at the input to a higher value at the output, the IP does not modify this field as the pixel data does not change. However, when converting from a higher input bits per color sample value to a lower output bits per color sample value, you must modify the active bits per color sample field if it indicates a value greater than the maximum bits per color supported at the output.

Figure 25. Dithering

With and without dithering (4 to 10 bps)



13.1.1. Bits per Color Sample Adapter IP Performance and Resources

Altera provides resource and utilization data for guidance.

Table 120. Bits per Color Sample Adapter IP Performance and Resources

For an example parameterization of 16 bits per color sample in, 10 bits per color sample out, 3 color planes per pixel, 8 pixels in parallel for the full variant.

Target device	ALMs	f _{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	900	892
Arria 10 speed grade 1 (10AS066H1F34E1HG)	714	645
Cyclone 10 GX speed grade 5 (10CX220YF78015G)	714	645
Stratix 10 GX speed grade 1 (1SX280LN2F43E1VG)	901	668

Table 121. Bits per Color Sample Adapter IP Performance and Resources

For an example parameterization of 8 bits per color sample in, 12 bits per color sample out, 4 color planes per pixel, 2 pixels in parallel for the full variant (with run-time control), with dithering.

Target device	ALM	f _{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	770	771
Agilex 5 speed grade 6 (A5ED065BB32AE6SR0)	781	487
Arria 10 speed grade 1 (10AS066H1F34E1HG)	694	751
Stratix 10 GX speed grade 1 (1SX280LN2F43E1VG)	806	738
Cyclone 10 GX speed grade 5 (10CX220YF672E5G)	691	604

For an example parameterization of 8 bits color sample in, 16 bits per color sample out, 4 color planes per pixel, 8 pixels in parallel for the full variant (with run-time control), with dithering

Target device	ALM	f _{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	2117	784
Agilex 5 speed grade 6 (A5ED065BB32AE6SR0)	2119	476
Arria 10 speed grade 1 (10AS066H1F34E1HG)	1983	622
Stratix 10 GX speed grade 1 (1SX280LN2F43E1VG)	2231	626
Cyclone 10 GX speed grade 5 (10CX220YF672E5G)	1982	559

13.2. Bits per Color Sample Adapter IP Parameters

The IP offers compile-time parameters.

Table 122. Bits per Color Sample Adapter IP Parameters - BPS Settings

Parameter	Allowed range	Description
Video data format		
Lite mode	On or off	Turn on to use the lite variant of the Altera Streaming Video Protocol.
Input bits per color sample	8 to 16	Select the number of bits per color sample at the input.
Output bits per color sample	8 to 16	Select the number of bits per color sample at the output.
<i>continued...</i>		

Parameter	Allowed range	Description
Number of color planes	1 to 4	Select the number of color planes per pixel
Number of pixels in parallel	1 to 8	Select the number of pixels in parallel.
Control Settings		
Memory mapped control interface	On or off	Turn on for the Avalon memory-mapped agent control interface and to allow runtime configuration via the register map.
General		
Pipeline ready signals	On or off	Turn on to add extra pipeline registers to the AXI4-S <code>tready</code> signals.
Separate clock for control	On or off	Turn on for a separate clock for the control agent interface.
Debug features	On or off	Turn on to read back writeable registers via the control agent interface

Table 123. Bits per Color Sample Adapter IP Parameters - Dithering Settings

Parameter	Value	Description
Dither Configuration		
Enable dithering	On or off	Turn on for dithering.
Dither Operation	Combined, Subtractive, Additive	Select the type of dither operation
Fixed LFSR seed	1 to ($2^{30} - 1$)	Select the 30 MSBs of the 33 bits of the random number generator seed value
Dither Mask		
Overwrite noise mask bits	On or off	-
Blue (Cb) Dither bits	0 to BPS out - BPS in	Select the number of bits of noise to add to dither for the blue channel (channel 0)
Green (Y) Dither bits	0 to BPS out - BPS in	Select the number of bits of noise to add to dither for the green channel (channel 1)
Red (Cr) Dither bits	0 to BPS out - BPS in	Select the number of bits of noise to add to dither for the red channel (channel 2)
Alpha (A) Dither bits	0 to BPS out - BPS in	Select the number of bits of noise to add to dither for the alpha channel (channel 3)

Figure 26. Dithering Parameters

Bits Per Color Sample Adapter Intel FPGA IP
intel_vvp_pixel_adapter

Important Usage Information
All resets must be at least 256 clock cycles in duration. When the IP uses multiple reset inputs, ensure to see "Reset Behaviour" in the Video and Vision Processing User Guide

BPS Adapter **Dithering Settings**

Dither Configuration

☒ Enable Dithering

Dither Operation: Combined

Fixed LFSR Seed: 1

Dither Mask

☐ Overwrite Noise Mask Bits

Blue (Cb) Dither Noise Pad Bits: 2

Green (Y) Dither Noise Pad Bits: 2

Red (Cr) Dither Noise Pad Bits: 0

Alpha (A) Dither Noise Pad Bits: 0

Parameterization Messages

Type	
?	
!	Dithering Enabled
!	2 Color planes selected, disabling colorplane 4, Alpha Channel & 3, Red(Cr) Channel

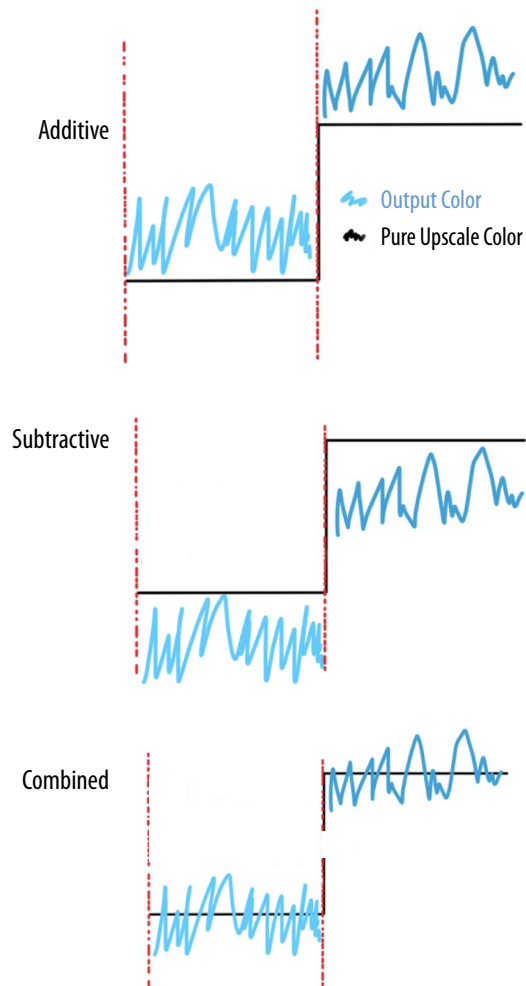
13.3. Bits per Color Sample Adapter IP Dithering Functional Description

Dithering operates differently for dithering upwards (input BPS > output BPS) and dithering downwards (input BPS < output BPS). The IP offers three types of dither: additive, subtractive, or combined.

Dithering Upwards

The IP processes the dithering upwards after shifting the value to the desired output BPS.

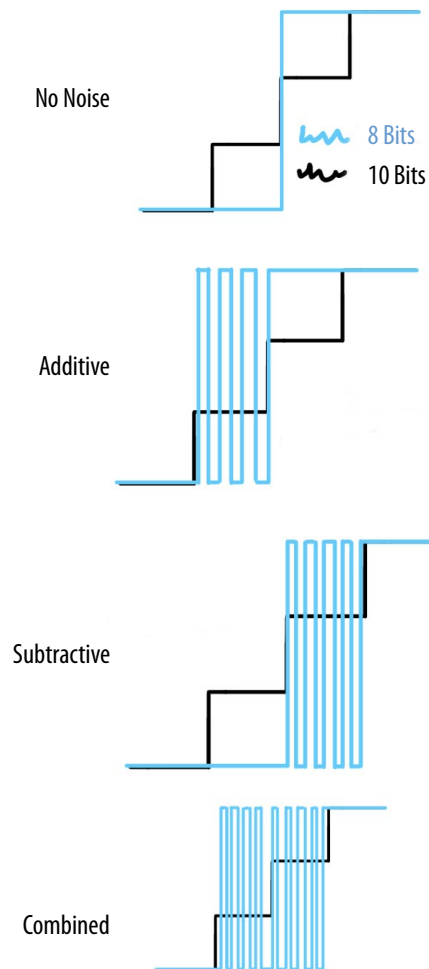
Figure 27. Types of Dithering (Upwards)



Dithering Downwards

The IP processes the dithering upwards before shifting the value to the desired output BPS.

Figure 28. Types of Dithering (Downwards)

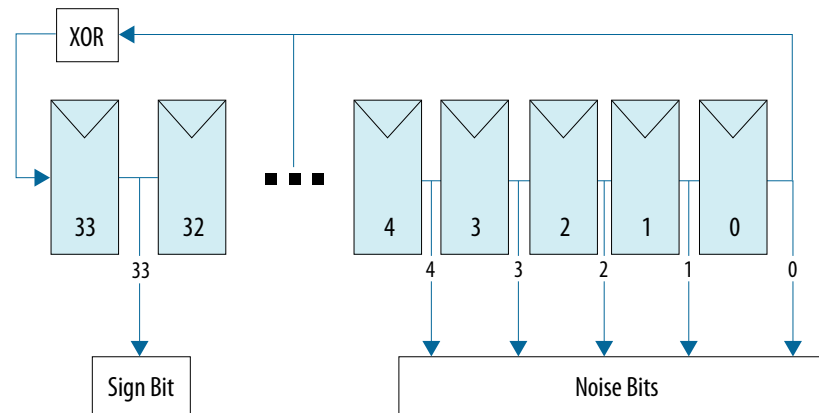


Fixed LSFR Seed Value

Linear feedback shift registers (LSFR) generate pseudo-random numbers for the noise. For the largest screen without creating a repeating pattern, a minimum size of 32 shift registers is required. The IP implements a size of 33 shift registers to reduce number of taps required and increases the maximum frequency of the LFSR. The Fibonacci LFSR has tap points at bit 0 and bit 13.

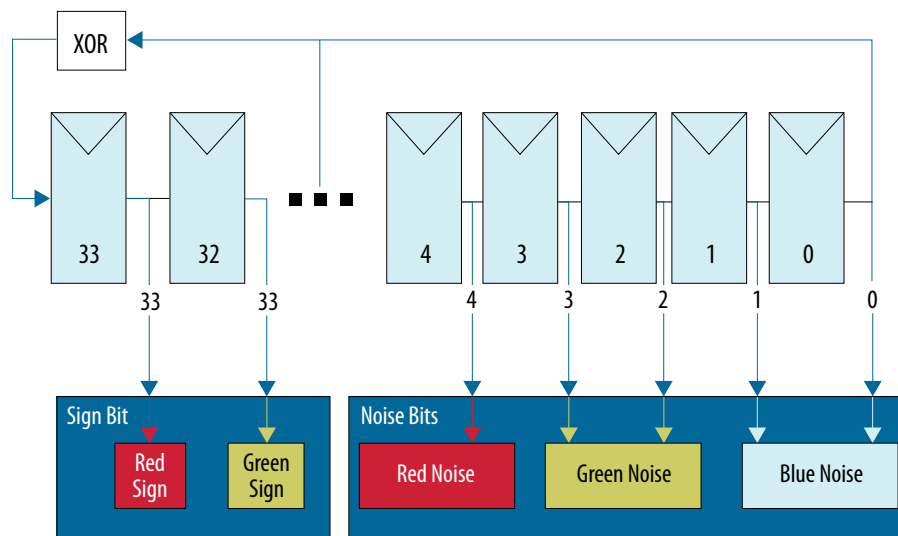
The IP gets the noise bits by masking the random number generated such that only the number of bits required shows. The MSB of the number generated then determines the operation (addition or subtraction).

Figure 29. Noise Selection



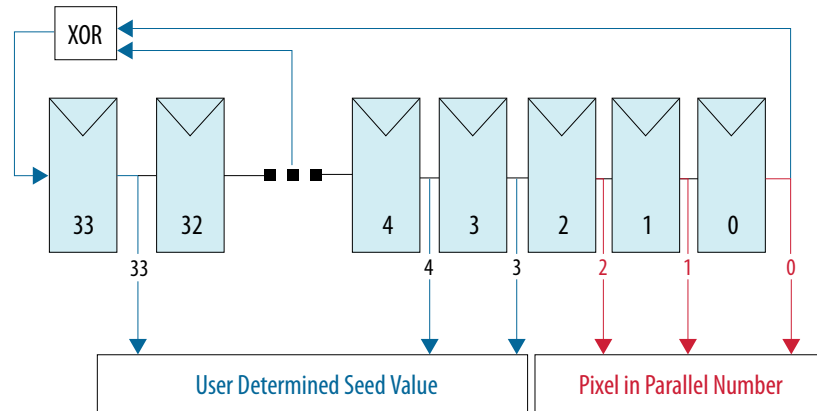
The number of masking bits is the sum of mask for each individual color plane. The IP packs it in the order of the color plane number (i.e. first few bits is for color plane 0 and color plane 1, etc.). The IP uses this packing order for the sign bits. The IP interprets the sign bits as if you assert it as additive, otherwise it is subtractive. The LFSR value is reset to the value in the **Fixed LSFR Seed**.

Figure 30. Noise Masking



The **Fixed LSFR Seed** value that you select is the most significant 30-bits. The least significant 3-bit remainder is determined by the pixel-in-parallel number. The pixel in parallel calculation is then more efficient as it does not require and adder to calculate the PIP seed.

Figure 31. Fixed LSFR Seed Value



13.3.1. Bits per Color Sample Adapter IP Interfaces

Table 124. Bits per Color Sample Adapter IP Interfaces

Signal name	Direction	Width	Description
Clocks and resets			
main_clock_clk	In	1	AXI4-S processing clock.
main_reset_rst	In	1	AXI4-S processing reset.
Control interfaces			
av_mm_control_agent_address	In	7	Avalon memory-mapped agent address.
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write.
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data.
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable.
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read.
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data.
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read data valid
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request.
Altera Streaming Video interfaces			
axi4s_vid_in_tdata	In	(31)	AXI4-S data in
axi4s_vid_in_tvalid	In	1	AXI4-S data valid
axi4s_vid_in_tuser	In	(32)	AXI4-S tuser

continued...

continued...

- (31) The equation gives all tdata widths in these interfaces:

$$\max(\text{ceil}((\text{bits per color sample} \times \text{number of color planes}) / 8) \times \text{pixels in parallel} \times 8, 16)$$
- (32) The equation gives all tuser widths in these interfaces

$$N = \text{ceil}(\text{tdata_width} / 8)$$

Signal name	Direction	Width	Description
			tuser[0] indicates start of video frame when asserted tuser[1] indicates the start of a nonvideo packet when asserted
axi4s_vid_in_tlast	In	1	AXI4-S end of packet
axi4s_vid_in_tready	Out	1	AXI4-S data ready
axi4s_vid_out_tdata	Out		AXI4-S data in
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid
axi4s_vid_out_tuser	Out	(32)	AXI4-S tuser tuser[0] indicates start of video frame when asserted tuser[1] indicates the start of a non-video packet when asserted
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet
axi4s_vid_out_tready	In	1	AXI4-S data ready

Related Information

Video and Vision Processing IP Interfaces on page 30

13.4. Bits per Color Sample Adapter IP Registers

Each register is either read-only (RO) or read-write (RW).

In the software API the register names appear with a prefix of INTEL_VVP,INTEL_VVP_CORE or INTEL_VVP_PIXEL_ADAPTER as appropriate and with an optional REGsuffix.

Table 125. Parameterization Registers

Address	Register	Access	Description
0x0000	VID_PID	RO	Read this register for the Bits per Color Sample Adapter product ID. This register always returns 6AF7_024C
0x0004	VERSION	RO	Read this register for the IP version information.
0x0008	LITE_MODE	RO	Read this register to determine if lite mode is on. This register returns 0 when you turn off lite mode and 1 when you turn on lite mode .
0x000C	DEBUG_ENABLED	RO	Read this register to determine if Debug features is on. This register returns 1 if reads to other registers designated as RW return the last value the IP writes to the register, or an undefined value.
0x0010	BPS_IN	RO	Read this register to determine the number of bits that represent each color plane on the input tdata bus
0x0014	BPS_OUT	RO	Read this register to determine the number of bits that represent each color plane on the output tdata bus
0x0018	ENABLE_DITHER	RO	Read this register to determine if the core has dither enable
0x001C	NOISE_OPERATION	RO	Read this register to determine the noise operation. ⁽³³⁾
continued...			

⁽³³⁾ This register is only defined when dither is on, otherwise it undefined.

Address	Register	Access	Description
0x0020	R_DITHER_BITS	RO	Read this register to determine the noise pad bits for color channel 2 (R/Cr). ⁽³³⁾
0x0024	G_DITHER_BITS	RO	Read this register to determine the noise pad bits for color channel 1 (G/Y). ⁽³³⁾
0x0028	B_DITHER_BITS	RO	Read this register to determine the noise pad bits for color channel 0 (B/Cb). ⁽³³⁾
0x002C	A_DITHER_BITS	RO	Read this register to determine the noise pad bits for color channel 3 (A). ⁽³³⁾
0x0030	FIXED_RAND_SEED	RO	Read this register to determine the fixed rand seed value. ⁽³³⁾
0x0034 to 0x011F	-	-	Unused.

Table 126. Control and Debug Registers

For more information, refer to *Control Packets*. You must turn on **debug features** to read the values stored in these registers. If you turn off **debug features**, reads to these registers return undefined data. The only exception is the **STATUS** register, the value of which you can always read

Address	Register	Access	Description
0x0120	IMG_INFO_WIDTH	RO	When you turn on Debug features , this register returns the width that the Bits per Color Sample Adapter derives from information in the image information packet.
0x0124	IMG_INFO_HEIGHT	RO	When you turn on Debug features , this register returns the height that the Bits per Color Sample Adapter derives from information in the image information packet.
0x0128	IMG_INFO_INTERLACE	RO	When you turn on Debug features , this register returns the interlace nibble that the Bits per Color Sample Adapter derives from information in the image information packet.
0x012C	Reserved	-	Reserved.
0x0130	IMG_INFO_COLORSPACE	RO	When you turn on Debug features , this register returns the color space that the Bits per Color Sample Adapter derives from information in the image information packet.
0x0134	IMG_INFO_SUBSAMPLING	RO	When you turn on Debug features , this register returns the subsampling that the Bits per Color Sample Adapter derives from information in the image information packet.
0x0138	IMG_INFO_COSITING	RO	When you turn on Debug features , this register returns the cositing that the Bits per Color Sample Adapter derives from information in the image information packet.
0x013C	IMG_INFO_FIELD_COUNT	RO	When you turn on Debug features , this register returns the field count that the Bits per Color Sample Adapter derives from information in the image information packet.
0x0140	STATUS	RO	Bit0: Status bit. 1 means Bits per Color Sample Adapter is processing a video field, 0 otherwise. Bit1: Pendingregister updates bit. Any writes to the output sampling register (0x0148) cause the IP to raise the pending register updates bit, to indicate outstanding changes to the resampling settings. The IP lowers this bit at the next field boundary after a write to the COMMIT register.
continued...			

Address	Register	Access	Description
0x0144	COMMIT	RW	The IP holds any changes to output bits per color value via the register map until you send a write to this register. The value you write is unimportant.
0x0148	IMG_INFO_BPS_OUT	RW	Write the value for the bits per color field for outgoing image information packets to this register. You write the actual bits per color value. The value for the image information packet is bits per color minus one, but the IP performs the subtraction.
0x014C	CONFIGURATION_REG	RW	Dithering only
0x0150	RAND_SEED_REG	RW	Write to this register to set the seed value. Dithering only

Register Bit Descriptions

Table 127. VID_PID

Name	Bits	Description
Bits per Color Sample Adapter vendor ID and product ID	31:0	This register always returns 0x6AF7_024C. 15:0 is the product ID and always returns 0x024C 31:16 is the vendor ID and always returns 0x6AF7

Table 128. VERSION

Name	Bits	Description
Register map version	7:0	Register map version.
IP patch revision	15:8	-
IP update revision	23:16	Updated when the IP version changes.
IP major revision	31:24	Updated when the IP version changes.

Table 129. LITE_MODE

Name	Bits	Description
Lite mode parameterization bit	31:0	The IP only uses this register when Lite mode is off. Reads to this register always return 0

Table 130. DEBUG_ENABLED

Name	Bits	Description
Debug features parameterization bit	31:0	Returns 1 if you turn on Debug features and 0 otherwise.

Table 131. BPS_IN

Name	Bits	Description
Bits per color in	31:0	Returns the number of bits that represent each color plane on the input tdata bus

Table 132. BPS_OUT

Name	Bits	Description
Bits per color out	31:0	Returns the number of bits that represent each color plane on the output tdata bus

Table 133. ENABLE_DITHER

Name	Bits	Description
Dither enabled	0	Returns if Enable dithering is on.
Unused	31:1	Unused

Table 134. NOISE_OPERATION

Name	Bits	Description
Dither Noise Operation	2:0	Returns the noise operation if Enable dithering is on, otherwise undefined
Unused	31:3	Unused

Table 135. R_DITHER_BITS

Name	Bits	Description
Dither Red (Cr) Dither bits	3:0	Return the R(Cr) dither bits if Enable dithering is on, otherwise undefined
Unused	31:4	Unused

Table 136. G_DITHER_BITS

Name	Bits	Description
Dither Green (Y) Dither bits	3:0	Return the G(Y) dither bits if Enable dithering is on, otherwise undefined
Unused	31:4	Unused

Table 137. B_DITHER_BITS

Name	Bits	Description
Dither Blue (Cb) Dither bits	3:0	Returns the B(Cb) dither bits if Enable dithering is on, otherwise undefined
Unused	31:4	Unused

Table 138. A_DITHER_BITS

Name	Bits	Description
Dither Alpha Dither bits	3:0	Returns the alpha dither bits if Enable dithering is on, otherwise undefined
Unused	31:4	Unused

Table 139. FIXED_RAND_SEED

Name	Bits	Description
Fixed Random Seed	31:0	Returns the fixed random seed if Enable dithering is on, otherwise undefined

Table 140. IMG_INFO_WIDTH

Name	Bits	Description
Width bits	15:0	When you turn on Debug features , this register reads the <code>width-1</code> field from the most recently received image information packet and adds 1 to return a value for <code>width</code> .
unused	31:16	Unused.

Table 141. IMG_INFO_HEIGHT

Name	Bits	Description
Height bits	15:0	When you turn on Debug features , this register reads the <code>height-1</code> field from the most recently received image information packet and adds 1 to return a value for <code>height</code> .
unused	31:16	Unused.

Table 142. IMG_INFO_INTERLACE

Name	Bits	Description
IntlaceNibble bits	3:0	When you turn on Debug features , this register returns the <code>intlaceNibble</code> field from the most recently received image information packet.
Unused	31:4	Unused.

Table 143. IMG_INFO_COLORSPACE

Name	Bits	Description
CSP code bits	6:0	When you turn on Debug features , this register returns the 7 bit <code>CSP</code> field from the most recently received image information packet.
unused	31:7	Unused.

Table 144. IMG_INFO_SUBSAMPLING

Name	Bits	Description
SubSa code bits	1:0	When you turn on Debug features , this register returns the <code>SUBSA</code> field from the most recently received image information packet.
Unused	31:2	Unused.

Table 145. IMG_INFO_COSITING

Name	Bits	Description
Cosite code bits	1:0	When you turn on Debug features , this register returns the <code>COSITE</code> field from the most recently received image information packet.
unused	31:2	Unused.

Table 146. IMG_INFO_FIELD_COUNT

Name	Bits	Description
Count bits	6:0	When you turn on Debug features , this register returns the 7 bit <code>FIELD_COUNT</code> field from the most recently received image information packet.
unused	31:7	Unused.

Table 147. STATUS

Name	Bits	Description
Status bit	0	1 means Bits per Color Sample Adapter is processing a video field, 0 otherwise.
Pending register updates bit	1	1 means Bits per Color Sample Adapter has pending updates, 0 otherwise
Unused	31:2	Unused.

Table 148. COMMIT

Name	Bits	Description
Unused	31:0	Unused.

Table 149. IMG_INFO_BPS_OUT

Name	Bits	Description
Image info bps out	4:0	Value the IP uses for the bits per color sample field in outgoing image information packers. You should write the true value for the number of bits per color and the IP subtracts 1 from this value to create the correct value for output.
Unused	31:5	Unused.

Table 150. CONFIGURATION_REG

Name	Bits	Description
Passthrough	0	Write 1 to turn off dithering
Unused	31:1	Unused

Table 151. RAND_SEED_REG

Name	Bits	Description
Random Seed Value	29:0	Write 30 MSBs of the seed value
Unused	31:30	Unused

13.5. Bits per Color Sample Adapter Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_pixel_adapter_regs.h`

Include file: `intel_vvp_pixel_adapter.h`

Table 152. Bits per Color Sample Adapter IP Software API

Name	Description
<code>intel_vvp_pixel_adapter_init</code>	Initialize the Bits per Color Sample Adapter instance
<code>intel_vvp_core_*</code>	Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i> . Writable when Lite is on. Readable when Lite is off and Debug features is on.
<code>intel_vvp_pixel_adapter_get_lite_mode</code>	Returns if Lite mode is on
<code>intel_vvp_pixel_adapter_get_debug_enabled</code>	Returns if Debug features is on
<code>intel_vvp_pixel_adapter_get_bps_in</code>	Returns the number of bits per color sample for the streaming input interface
<code>intel_vvp_pixel_adapter_get_bps_out</code>	Returns the number of bits per color sample value for the streaming output interface
<i>continued...</i>	

Name	Description
intel_vvp_pixel_adapter_is_running	Returns if the IP is currently processing video data
intel_vvp_pixel_adapter_get_commit_status	Returns if the IP has uncommitted writes
intel_vvp_pixel_adapter_get_status	Reads the status register
intel_vvp_pixel_adapter_get_dither_enable	Returns if Enable dithering is on
intel_vvp_pixel_adapter_get_noise_operation	Returns the noise operation if Enable dithering is on
intel_vvp_pixel_adapter_get_dither_bits	Returns the noise dither bits if Enable dithering is on
intel_vvp_pixel_adapter_set_rand_seed_value	Sets the random number generator seed value, if Enable dithering is on
intel_vvp_pixel_adapter_get_rand_seed_value	Returns the random number generator seed value, if Enable dithering is on
intel_vvp_pixel_adapter_set_configuration	Sets the configuration mode, if Enable dithering is on
intel_vvp_pixel_adapter_get_configuration	Returns the configuration mode, if Enable dithering is on
intel_vvp_pixel_adapter_set_output_img_info_bps	Sets the number of bits per color sample value required in output image info packet
intel_vvp_pixel_adapter_get_output_img_info_bps	Returns the number of bits per color sample value used in output image info packet
intel_vvp_pixel_adapter_commit_writes	Commit all outstanding writes

intel_vvp_pixel_adapter_init

Prototype

```
int
intel_vvp_pixel_adapter_init(intel_vvp_pixel_adapter_instance
*instance, intel_vvp_core_base base);
```

Description

Initializes a Bits per Color Sample Adapter instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

Arguments

instance - pointer to the intel_vvp_pixel_adapter_instance software driver instance structure

base - base address of the register map

Returns

kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product_id does not match with the Pixel Adapter product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpPixel AdapterRegMapVersionErr if the register map is not supported

intel_vvp_pixel_adapter_get_lite_mode

Prototype

```
bool  
intel_vvp_pixel_adapter_get_lite_mode(intel_vvp_pixel_adapter_instance  
*instance);
```

Description Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_pixel_adapter_instance fully initialized

Arguments instance - pointer to the intel_vvp_pixel_adapter_instance software driver instance structure

Returns true if the IP has lite mode on

intel_vvp_pixel_adapter_get_debug_enabled

Prototype

```
bool  
intel_vvp_pixel_adapter_get_debug_enabled(intel_vvp_pixel_adapter_instance  
*instance);
```

Description Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_pixel_adapter_instance fully initialized.

Arguments instance - pointer to the intel_vvp_pixel_adapter_instance software driver instance structure

Returns true if the IP has debug features on

intel_vvp_pixel_adapter_get_bps_in

Prototype

```
uint8_t  
intel_vvp_pixel_adapter_get_bps_in(intel_vvp_pixel_adapter_instance  
*instance);
```

Description Returns the value of the BPS_IN register. The instance must be a valid intel_vvp_pixel_adapter_instance fully initialized.

Arguments instance - pointer to the intel_vvp_pixel_adapter_instance software driver instance structure

Returns Returns the value of the BPS_IN parameter for the IP. The instance must be a valid intel_vvp_pixel_adapter_instance fully initialized

intel_vvp_pixel_adapter_get_bps_out

Prototype

```
uint8_t
intel_vvp_pixel_adapter_get_bps_out(intel_vvp_pixel_adapter_instance
*instance);
```

Description Returns the value of the BPS_OUT parameter for the IP. The instance must be a valid intel_vvp_pixel_adapter_instance fully initialized.

Arguments instance - pointer to the intel_vvp_pixel_adapter_instance software driver instance structure

Returns Returns the value of the BPS_IN parameter for the IP. The instance must be a valid intel_vvp_pixel_adapter_instance fully initialized

intel_vvp_pixel_adapter_is_running

Prototype

```
bool
intel_vvp_pixel_adapter_is_running(intel_vvp_pixel_adapter_instance*
instance);
```

Description Reads and returns the running bit of the STATUS register. The instance must be a valid intel_vvp_pixel_adapter_instance fully initialized.

Arguments instance - pointer to the intel_vvp_pixel_adapter_instance software driver instance structure

Returns True is the IP is currently receiving or storing an input field, false when between two input fields

intel_vvp_pixel_adapter_get_commit_status

Prototype

```
bool
intel_vvp_pixel_adapter_get_commit_status(intel_vvp_clipper_instance*
instance);
```

Description Reads and returns the commit pending bit of the STATUS register. The instance must be a valid intel_vvp_pixel_adapter_instance fully initialized and is in full mode.

Arguments instance - pointer to the intel_vvp_pixel_adapter_instance software driver instance structure

Returns True is the IP is in full mode and if the agent interface contains uncommitted writes.

intel_vvp_pixel_adapter_get_status

Prototype

```
uint32_t  
intel_vvp_pixel_adapter_get_status(intel_vvp_pixel_adapter_instance*  
instance);
```

Description Reads the STATUS register. The instance must be a valid intel_vvp_pixel_adapter_instance fully initialized.

Arguments instance - pointer to the intel_vvp_pixel_adapter_instance software driver instance structure

Returns The content of the STATUS register

intel_vvp_pixel_adapter_get_dither_enable

Prototype

```
bool  
intel_vvp_pixel_adapter_get_dither_enable(intel_vvp_pixel_adapter_instance*);
```

Description Return the value of the DITHER_ENABLE parameter for the IP. The instance must be a valid intel_vvp_pixel_adapter_instance fully initialized.

Arguments Instance - pointer to the intel_vvp_pixel_adapter_instance software driver instance structure

Returns True if the IP has dither mode on

intel_vvp_pixel_adapter_get_noise_operation

Prototype

```
eIntelVvpPixelAdapterNoiseOperation  
intel_vvp_pixel_adapter_get_noise_operation(intel_vvp_pixel_adapter_instance*  
instance);
```

Description Returns the value of the NOISE_OPERATION parameter for the IP. The instance must be a valid intel_vvp_pixel_adapter_instance fully initialized parameterized with dither enable on

Arguments Instance - pointer to the intel_vvp_pixel_adapter_instance software driver instance structure

Returns The value of the NOISE_OPERATION parameter for the IP. The instance must be a valid intel_vvp_pixel_adapter_instance fully initialized parameterized with dither mode on

intel_vvp_pixel_adapter_get_dither_bits

Prototype

```
int
intel_vvp_pixel_adapter_get_dither_bits(intel_vvp_pixel_adapter_instance*
instance, uint8_t* b_dither_bits, uint8_t* g_dither_bits, uint8_t*
r_dither_bits, uint8_t* a_dither_bits);
```

Description Returns the value of the R_DITHER_BITS, G_DITHER_BITS, B_DITHER_BITS, A_DITHER_BITS parameters for the IP. The instance must be a valid intel_vvp_pixel_adapter_instance fully initialized parameterized with dither enable on

Arguments instance - pointer to the intel_vvp_pixel_adapter_instance software driver instance structure

b_dither_bits - pointer to store the B_DITHER_BITS parameter

g_dither_bits - pointer to store the G_DITHER_BITS parameter

r_dither_bits - pointer to store the R_DITHER_BITS parameter

a_dither_bits - pointer to store the A_DITHER_BITS parameter

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpPixelAdapterNoDitherInstanceErr if the instance is parameterized with dither enable off

intel_vvp_pixel_adapter_set_configuration_mode

Prototype

```
int
intel_vvp_pixel_adapter_set_configuration_mode(intel_vvp_pixel_adapter_instance*
instance, bool passthrough);
```

Description Set the passthrough mode of the CONFIGURATION_REG. The instance must be a valid intel_vvp_pixel_adapter_instance fully initialized and parameterized with dither on

Arguments Instance - pointer to the intel_vvp_pixel_adapter_instance software driver instance structure

Passthrough - the value of the passthrough

Returns kIntelVvpCoreOk (0) for success, negative error code in case of failures

kIntelVvpInstanceErr if the instance is invalid

kIntelVvpPixelFormatNoDitherInstanceErr if the instance

intel_vvp_pixel_adapter_get_configuration_mode

Prototype

bool

```
intel_vvp_pixel_adapter_get_configuration_mode(intel_vvp_pixel_adapter_instance* instance);
```

Description

Reads and return the passthrough mode of the CONFIGURATION_REG. The instance must be a valid intel_vvp_pixel_adapter_instance fully initialized and parameterized with dither enable on

Arguments

Instance - pointer to the intel_vvp_pixel_adapter_instance software driver instance structure

Returns True if passthrough is enable else otherwise (0 if instance is parameterized with dither off)

intel_vvp_pixel_adapter_set_rand_seed_value

Prototype

int

```
intel_vvp_pixel_adapter_set_rand_seed_value(intel_vvp_pixel_adapter_instance* instance, uint32_t rand_seed_value);
```

Description

Set the 30 MSB of the RAND_SEED_REG. The instance must be a valid intel_vvp_pixel_adapter_instance fully initialized and parameterized with dither enable on

Arguments

Argument instance - pointer to the intel_vvp_pixel_adapter_instance software driver instance structure

rand_seed_value - random number generator seed value

Returns kIntelVvpCoreOk (0) for success, negative error code in case of failures

kIntelVvpInstanceErr if the instance is invalid

kIntelVvpPixelFormatNoDitherInstanceErr if the instance is parameterized with dither enable off

kIntelVvpPixelFormatRandSeedOutOfRangeErr if the rand seed value is out of range (rand_seed_value == 0 or rand_seed_value $\geq 2^{30}$)

intel_vvp_pixel_adapter_get_rand_seed_value

Prototype

```
int
intel_vvp_pixel_adapter_get_rand_seed_value(intel_vvp_pixel_adapter_instance*
instance);
```

Description Read and return the 30 MSB of the random number generator seed. The instance must be a valid `intel_vvp_pixel_adapter_instance` parameterized with dither enable on

Arguments Instance - pointer to the `intel_vvp_pixel_adapter_instance` software driver instance structure

Returns The value of the random number generator seed, negative error code in case of failure

`kIntelVvpInstanceErr` if the instance is invalid

`kIntelVvpPixelAdapterNoDitherInstanceErr` if the instance is parameterized with dither off

intel_vvp_pixel_adapter_set_output_img_info_bps

Prototype

```
int
intel_vvp_pixel_adapter_set_output_img_info_bps(intel_vvp_pixel_adapter_instance
*instance, uint8_t out_bps);
```

Description Sets the bits per sample value the IP uses in output image info packet. The instance must be a valid `intel_vvp_pixel_adapter_instance` fully initialized.

Arguments instance - pointer to the `intel_vvp_pixel_adapter_instance` software driver instance structure

out_bps - output bps value

Returns `kIntelVvpCoreOk` for success, negative error code in case of failures

`kIntelVvpCoreInstanceErr` if the instance is invalid

intel_vvp_pixel_adapter_get_output_img_info_bps

Prototype

```
uint8_t
intel_vvp_pixel_adapter_get_output_img_info_bps(intel_vvp_pixel_adapter_instance
*instance);
```

Description Returns the bits per sample value used in output image info packet. The instance must be a valid `intel_vvp_pixel_adapter_instance` fully initialized and parameterized with debug features on.

Arguments instance - pointer to the `intel_vvp_pixel_adapter_instance` software driver instance structure

Returns `kIntelVvpCoreOk` for success, negative error code in case of failures
`kIntelVvpCoreInstanceErr` if the instance is invalid

`intel_vvp_pixel_adapter_commit_writes`

Prototype

```
int  
intel_vvp_pixel_adapter_commit_writes(intel_vvp_pixel_adapter_instance*  
instance)
```

Description Commits all pending writes before processing the next field. The instance must be valid and in full mode.

Arguments instance - pointer to the `intel_vvp_pixel_adapter_instance` software driver instance structure

Returns `kIntelVvpCoreOk` (0) for success, negative error code otherwise

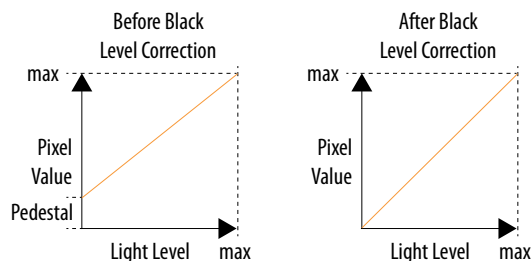
14. Black Level Correction IP

14.1. About the Black Level Correction IP

The Black Level Correction IP adjusts the brightness level of the image, to ensure the minimum pixel intensity represents a true black value. The IP operates only on 2x2 Bayer color filter array images. Imaging sensors add a pedestal value to the captured image, which you need to remove after reducing the image noise.

The IP subtracts the pedestal value from the input video stream and resizes the result back to the full dynamic range.

Figure 32. Black Level Correction IP Operation



The IP only supports lite variants. For more information about full and lite variants, refer to the *Altera Streaming Video Protocol Specification*

Related Information

[Altera Streaming Video Protocol Specification](#)

14.1.1. Black Level Correction IP Performance and Resources

Altera provides resource and utilization data for guidance only. The IP resource utilization depends on the device family, compilation tool version, compilation seed randomization, and other parameters.

Table 153. Black Level Correction IP Performance and Resources

Device	Pixel in Parallel	Bits per Color Sample	Max Resolution	ALM	M20k	DSP	f _{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	1	10	2k	475	0	1	845
	1	16	2k	573	0	1	810
	4	10	8k	733	0	4	739
	4	16	8k	889	0	4	755
continued...							

Device	Pixel in Parallel	Bits per Color Sample	Max Resolution	ALM	M20k	DSP	f _{MAX} (MHz)
Arria 10 speed grade 1 (10AS066H1F34E1HG)	1	10	2k	397	0	1	591
	1	16	2k	445	0	1	577
	2	10	4k	468	0	2	555
	2	16	4k	542	0	2	583
Cyclone 10 GX speed grade 5 (10CX220YF672E5G)	1	10	2k	396	0	1	513
	1	16	2k	444	0	1	476
Stratix 10 speed grade 1 (1SX280LN2F43E1VG)	1	10	2k	514	0	1	712
	1	16	2k	562	0	1	698
	2	10	4k	620	0	2	674
	2	16	4k	710	0	2	700

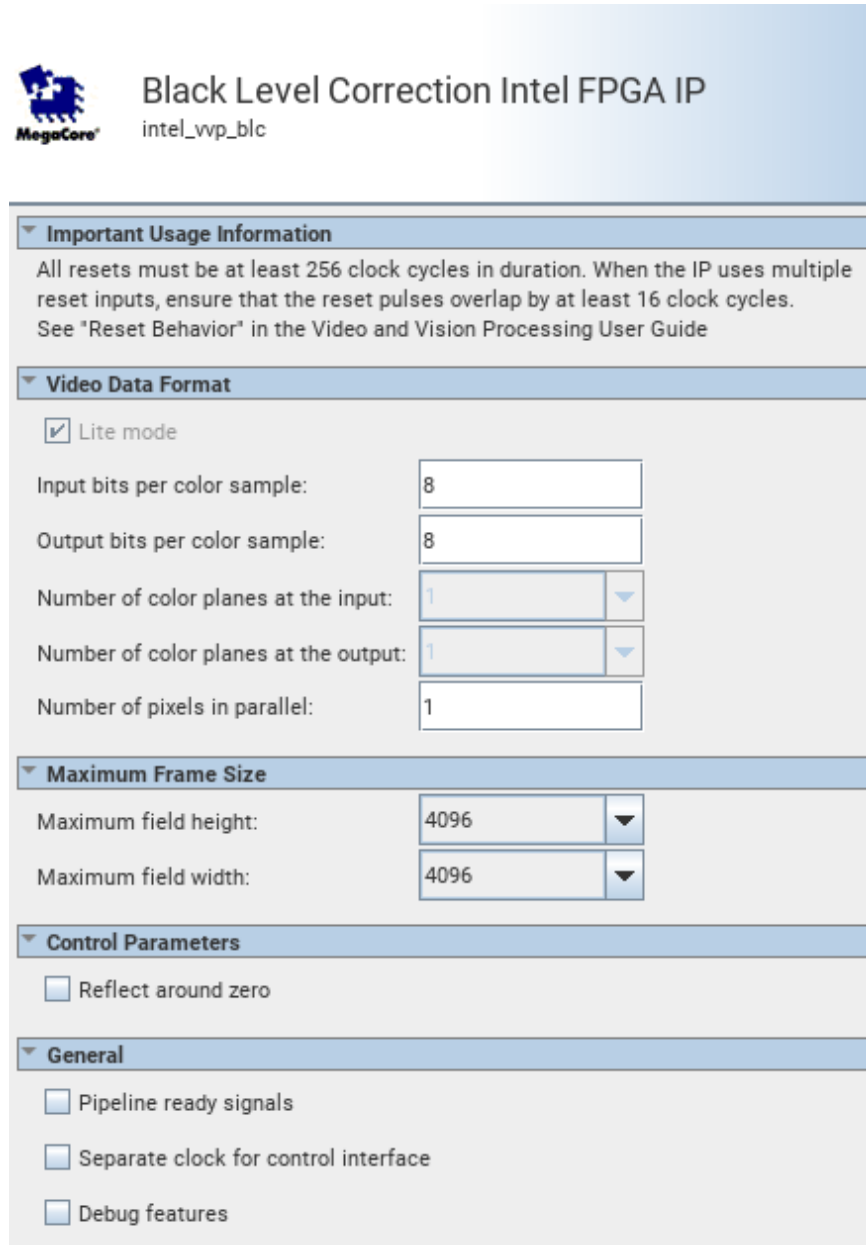
14.2. Black Level Correction IP Parameters

The IP offers run-time and compile-time parameters.

Table 154. Black Level Correction IP Parameters

Parameter	Values	Description
Video Data Format		
Lite mode	On	The IP only supports lite mode
Input bits per color sample	8 to 16	Select the number of input bits per color sample.
Output bits per color sample	8 to 16	Select the number of output bits per color sample.
Number of color planes at the input	1	The IP only supports 1 color plane at its input
Number of color planes at the output	1	The IP only supports 1 color plane at its output
Number of pixels in parallel	1 to 8	Select the number of pixels in parallel.
Maximum Frame Size		
Maximum field height	1k, 2k, 4k, 8k, 16k	Specify the maximum height of incoming frames
Maximum field width	2k, 4k, 8k, 16k	Specify the maximum width of incoming frames
Control Parameters		
Reflect around zero	On or Off	Turn off for the IP to clip the input pixel values below black level to 0. Turn on for the IP to reflect the values below black level around 0 and avoid clipping.
General		
Pipeline ready signal	On or off	Turn on to add extra pipeline registers to the AXI4-S <code>tready</code> signals
Separate clock for control interface	On or off	Turn on for a separate clock for the control agent interface
Debug features	On or off	Turn on to read back frame information registers and debugging information registers via the control agent interface.

Figure 33. Black Level Correction IP GUI



Black Level Correction Intel FPGA IP
intel_vvp_blc

Important Usage Information
All resets must be at least 256 clock cycles in duration. When the IP uses multiple reset inputs, ensure that the reset pulses overlap by at least 16 clock cycles. See "Reset Behavior" in the Video and Vision Processing User Guide

Video Data Format

☒ Lite mode

Input bits per color sample: 8

Output bits per color sample: 8

Number of color planes at the input: 1

Number of color planes at the output: 1

Number of pixels in parallel: 1

Maximum Frame Size

Maximum field height: 4096

Maximum field width: 4096

Control Parameters

☐ Reflect around zero

General

☐ Pipeline ready signals

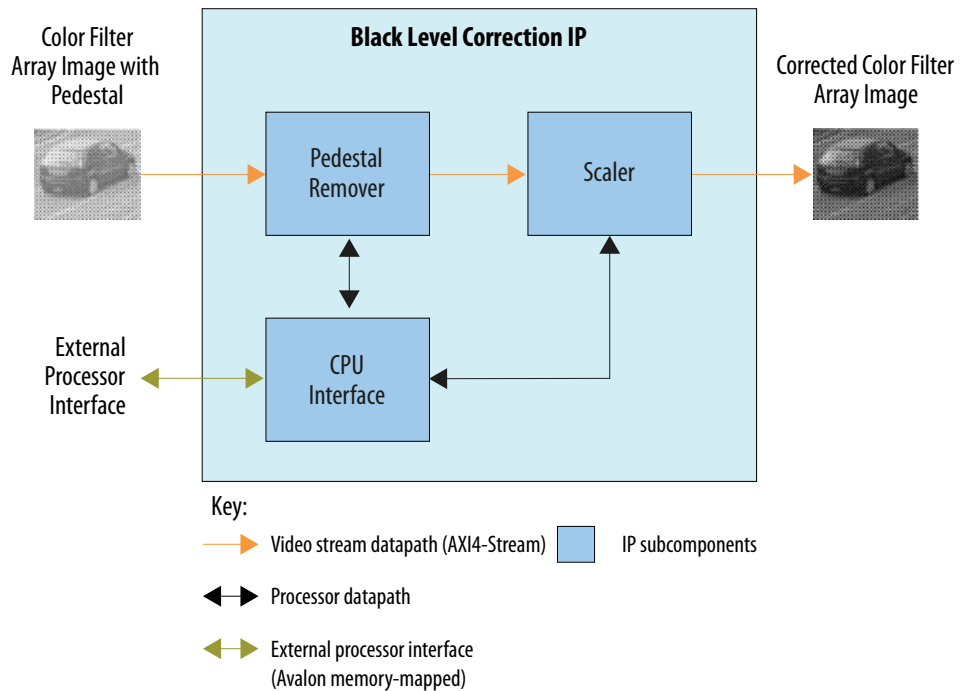
☐ Separate clock for control interface

☐ Debug features

14.3. Black Level Correction IP Functional Description

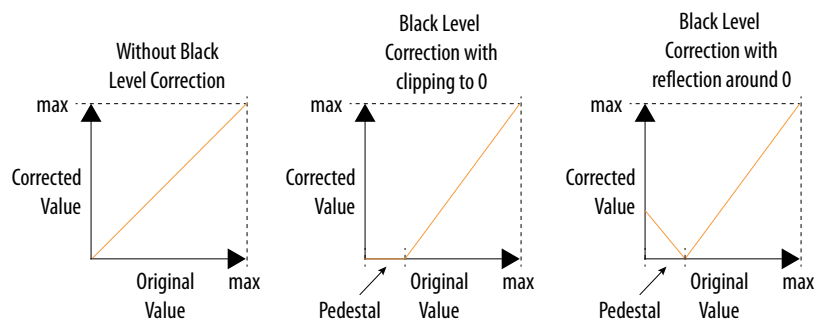
The Black Level Correction IP removes the pedestal offset value from the input video stream and resizes the result back to the full dynamic range of the video stream. You write pedestal and scaler values for each of the 4 color channels of the 2x2 color filter array over the external processor interface. The IP uses these values to calculate the output video stream. The IP supports arbitrary 2x2 color filter arrays.

Figure 34. Black Level Correction IP Block Diagram



The pedestal remover subtracts the offset values from the input pixel values. The IP clips negative values to zero if you turn off **Reflect around zero**, or if you turn on **Reflect around zero** but set the run-time `CONTROL` register `Clip Zero En` bit to turn on clipping to zero. The IP reflects the negative values around zero if you turn on **Reflect around zero** and clear the run-time `CONTROL` register `Clip Zero En` bit to turn off clipping at zero.

Figure 35. The effect of reflection around zero

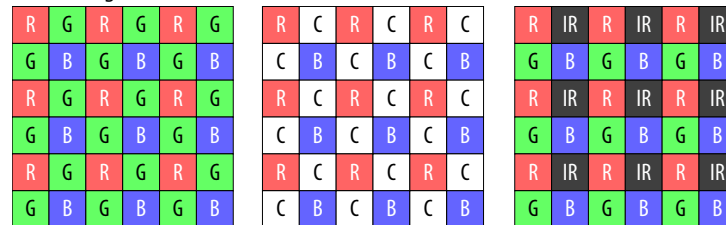


The scaler part of the IP multiplies the intermediate result after subtraction with an arbitrary factor that you program over the external processor interface. You must calculate the correct multiplication factor from the dynamic range of the video stream and the pedestal value. The IP clips the overflowed values to the maximum value of the output video stream.

You must set the color filter array phase to its correct value, which depends on the framing, rotation, and flipping options you apply on the original image that an imaging sensor captures. If you do not set the correct color filter array phase, the IP may mix color channels and produce incorrect outputs.

Figure 36. Examples of 2x2 Color Filter Arrays

For a 6x6 section of an image



14.3.1. Black Level Correction IP Interfaces

Table 155. Black Level Correction Clock and Reset Interfaces

Name	Direction	Width	Description
main_clock_clk	In	1	AXI4-S processing clock.
main_reset_rst	In	1	AXI4-S processing reset.
agent_clock_clk	In	1	Clock for the memory-mapped control agent interface. Only if you turn on Separate clock for control interface .
agent_reset_reset	In	1	Reset for the memory-mapped control agent interface. Only if you turn on Separate clock for control interface .

Table 156. Black Level Correction Control Interfaces

Name	Direction	Width	Description
av_mm_control_agent_address	In	7	Avalon memory-mapped agent address
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write.
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data.
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable.
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read.
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data.
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read.
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request.

Table 157. Black Level Correction Altera Streaming Video Interfaces

Name	Direction	Width	Description
Input			
axi4s_vid_in_tdata	In	(34)	AXI4-S data in.
axi4s_vid_in_tvalid	In	1	AXI4-S data valid.

continued...

Name	Direction	Width	Description
axi4s_vid_in_tuser	In	(35)	[0] AXI4-S start of video frame. [N-1:1] Unused.
axi4s_vid_in_tlast	In	1	AXI4-S end of packet.
axi4s_vid_in_tready	Out	1	AXI4-S data ready.
Output			
axi4s_vid_out_tdata	Out	(34)	AXI4-S data out.
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid.
axi4s_vid_out_tuser	Out	(35)	[0] AXI4-S start of video frame. [N-1:1] Unused.
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet.
axi4s_vid_out_tready	In	1	AXI4-S data ready

Related Information

Video and Vision Processing IP Interfaces on page 30

14.4. Black Level Correction IP Registers

Each register is either read-only (RO) or read-write (RW).

Table 158. Black Level Correction Registers

In the software API the register names appear with a prefix of INTEL_VVP, INTEL_VVP_CORE or INTEL_VVP_BLC as appropriate and with an optional REG suffix

Address	Register	Access		Description
		Lite (36)	Full	
Parameterization registers				
0x0000	VID_PID	RO	N/A	Read this register to retrieve the ID of the IP. This register always returns 0x6FA7_0177.
0x0004	VERSION	RO	N/A	Read this register to retrieve the version information for the IP.
0x0008	LITE_MODE	RO	N/A	Read this register to determine if Lite mode is on. This register always returns 1.
0x000C	DEBUG_ENABLED	RO	N/A	Read this register to determine if Debug features are on. This register returns 0 for off and 1 for on.
continued...				

(34) The equation gives all tdata widths sizes in these interfaces: pixels in parallel × max (round (bits per color sample × number of color planes, 8), 16)

(35) This equation gives all tuserwidths sizes in these interfaces: N = ceil (tdata width / 8)

(36) Registers are RW only if you also turn on **Debug features**, otherwise they are WO.

Address	Register	Access		Description
		Lite ⁽³⁶⁾	Full	
0x0010	BPS_IN	RO	N/A	Read this register to determine the Input bits per color symbol .
0x0014	BPS_OUT	RO	N/A	Read this register to determine the Output bits per color symbol .
0x0018	NUM_COLOR_IN	RO	N/A	Read this register to determine the Number of color planes at the input . This register always returns 1.
0x001C	NUM_COLOR_OUT	RO	N/A	Read this register to determine the Number of color planes at the output . This register always returns 1.
0x0020	PIP	RO	N/A	Read this register to determine the Number of pixels in parallel .
0x0024	MAX_WIDTH	RO	N/A	Read this register to determine the Maximum field width .
0x0028	MAX_HEIGHT	RO	N/A	Read this register to determine the Maximum field height .
0x002C	REFLECT_AROUND_ZERO	RO	N/A	Read this register to determine Reflect around zero .
0x0030 to 0x011F	-	-	-	Reserved
Control, debugging, and statistics registers				
0x0120	IMG_INFO_WIDTH	RW	N/A	The expected width of the incoming video fields.
0x0124	IMG_INFO_HEIGHT	RW	N/A	The expected height of the incoming video fields.
0x0028 to 0x013F	-	-	-	Reserved
0x0140	STATUS	RO	N/A	Read this register for information about the IP status.
0x0144	FRAME_STATS	RO	N/A	Read this register for some frame statistics.
0x0148	COMMIT	RW	N/A	Write any value to this register to submit changes to the control, black pedestal, and color scaler registers.
0x014C	CONTROL	RW	N/A	Control bits and fields of the IP
0x0150	CFA_00_BLACK_PEDESTAL	RW	N/A	Black Pedestal value for color channel 0.
0x0154	CFA_00_COLOR_SCALER	RW	N/A	Color scaler value for color channel 0.
0x0158	CFA_01_BLACK_PEDESTAL	RW	N/A	Black Pedestal value for color channel 1.
0x015C	CFA_01_COLOR_SCALER	RW	N/A	Color scaler value for color channel 1.
0x0160	CFA_10_BLACK_PEDESTAL	RW	N/A	Black Pedestal value for color channel 2.
0x0164	CFA_10_COLOR_SCALER	RW	N/A	Color scaler value for color channel 2.
continued...				

⁽³⁶⁾ Registers are RW only if you also turn on **Debug features**, otherwise they are WO.

Address	Register	Access		Description
		Lite ⁽³⁶⁾	Full	
0x0168	CFA_11_BLACK_PEDESTAL	RW	N/A	Black Pedestal value for color channel 3.
0x016C	CFA_11_COLOR_SCALER	RW	N/A	Color scaler value for color channel 3.
0x0170 to 0x01FF	-	-	-	Reserved

Register Bit Descriptions

Table 159. STATUS

Name	Bits	Description
Reserved	31:2	Reserved.
Commit	1	Pending commit
Running	0	When 1, the IP is processing data.

Table 160. FRAME_STATS

Name	Bits	Description
Reserved	31:8	Reserved.
Checksum	7:0	A simple checksum of the frame

Table 161. CONTROL

Name	Bits	Description
Reserved	31:4	Reserved. Write 0.
Clip Zero En	3	You may set this bit to override reflection if Reflect around zero is on. The IP clips values below black level to 0 instead of reflecting them around 0.
Color filter array phase	2:1	Specifies 2x2 color filter order starting from the top left corner of the image. 00 01 10 11 C₀C₁ C₁C₀ C₂C₃ C₃C₂ C₂C₃ C₃C₂ C₀C₁ C₁C₀
Bypass	0	Set to bypass Black Level Correction IP. When set the IP passes the input image unmodified.

Table 162. CFA_00_BLACK_PEDESTAL

Name	Bits	Description
Reserved	31:Input bits per color sample	Reserved. Write 0.
Black pedestal	Input bits per color sample - 1:0	Black pedestal value for color channel 0 (C0).

⁽³⁶⁾ Registers are RW only if you also turn on **Debug features**, otherwise they are WO.

Table 163. CFA_00_COLOR_SCALER

Name	Bits	Description
Reserved	31:18	Reserved. Write 0.
Color scaler	17:0	Unsigned 0.18 fixed-point color scaler for color channel 0 (C0). MSBs to LSBs correspond to the binary digits 2^{-1} to 2^{-18} .

Table 164. CFA_01_BLACK_PEDESTAL

Name	Bits	Description
Reserved	31:Input bits per color sample	Reserved. Write 0.
Black pedestal	Input bits per color sample - 1:0	Black pedestal value for color channel 1 (C1).

Table 165. CFA_01_COLOR_SCALER

Name	Bits	Description
Reserved	31:18	Reserved. Write 0.
Color scaler	17:0	Unsigned 0.18 fixed-point color scaler for color channel 1 (C1). MSBs to LSBs correspond to the binary digits 2^{-1} to 2^{-18} .

Table 166. CFA_10_BLACK_PEDESTAL

Name	Bits	Description
Reserved	31:Input bits per color sample	Reserved. Write 0.
Black pedestal	Input bits per color sample - 1:0	Black pedestal value for color channel 2 (C2).

Table 167. CFA_10_COLOR_SCALER

Name	Bits	Description
Reserved	31:18	Reserved. Write 0.
Color scaler	17:0	Unsigned 0.18 fixed-point color scaler for color channel 2 (C2). MSBs to LSBs correspond to the binary digits 2^{-1} to 2^{-18} .

Table 168. CFA_11_BLACK_PEDESTAL

Name	Bits	Description
Reserved	31:Input bits per color sample	Reserved. Write 0.
Black pedestal	Input bits per color sample - 1:0	Black pedestal value for color channel 3 (C3).

Table 169. CFA_11_COLOR_SCALER

Name	Bits	Description
Reserved	31:18	Reserved. Write 0.
Color scaler	17:0	Unsigned 0.18 fixed-point color scaler for color channel 3 (C3). MSBs to LSBs correspond to the binary digits 2^{-1} to 2^{-18} .

14.5. Black Level Correction IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_blc_regs.h`

Include file: `intel_vvp_blc.h`

Table 170. Black Level Correction IP Software API

Name	Description
<code>intel_vvp_blc_init</code>	Initialize an IP instance
<code>intel_vvp_core_*</code>	Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i>
<code>intel_vvp_blc_get_lite_mode</code>	Returns if Lite mode is on
<code>intel_vvp_blc_get_debug_enabled</code>	Returns if Debug features is on
<code>intel_vvp_blc_get_bits_per_sample_in</code>	Returns the number of bits per color sample for the streaming input interface
<code>intel_vvp_blc_get_bits_per_sample_out</code>	Returns the number of bits per color sample value for the streaming output interface
<code>intel_vvp_blc_get_num_color_planes_in</code>	Returns the number of color planes of the streaming input interface
<code>intel_vvp_blc_get_num_color_planes_out</code>	Returns the number of color planes of the streaming output interface
<code>intel_vvp_blc_get_pixels_in_parallel</code>	Returns the pixels in parallel of the streaming input and output interfaces
<code>intel_vvp_blc_get_max_width</code>	Returns the maximum number of pixels that the IP supports on the horizontal dimension of an image or video frame
<code>intel_vvp_blc_get_max_height</code>	Returns the maximum number of pixels that the IP supports on the vertical dimension of an image or video frame
<code>intel_vvp_blc_get_reflect_around_zero</code>	Returns if reflect around zero mode is enabled
<code>intel_vvp_blc_is_running</code>	Reads if the IP is running
<code>intel_vvp_blc_commit_is_pending</code>	Reads if the IP has uncommitted writes
<code>intel_vvp_blc_get_status</code>	Reads the status register
<code>intel_vvp_blc_get_frame_stats</code>	Reads the frame statistics register
<code>intel_vvp_blc_commit</code>	Commit pending writes
<code>intel_vvp_blc_get_bypass</code>	Reads the bypass bit from the status register
<code>intel_vvp_blc_set_bypass</code>	Writes the bypass bit of the status register
<code>intel_vvp_blc_get_cfa_phase</code>	Reads the color filter array phase field from the status register
<code>intel_vvp_blc_set_cfa_phase</code>	Writes the color filter array phase field of the status register
<code>intel_vvp_blc_get_clip_zero_en</code>	Reads the clip zero enable bit from the status register
<i>continued...</i>	

Name	Description
intel_vvp_blc_set_clip_zero_en	Writes the clip zero enable bit of the status register
intel_vvp_blc_get_cfa_00_black_pedestal	Reads the black level pedestal register associated with the color filter phase 00
intel_vvp_blc_set_cfa_00_black_pedestal	Writes the black level pedestal register associated with the color filter phase 00
intel_vvp_blc_get_cfa_00_color_scaler	Reads the color scaler register associated with the color filter phase 00
intel_vvp_blc_set_cfa_00_color_scaler	Writes the color scaler register associated with the color filter phase 00
intel_vvp_blc_get_cfa_01_black_pedestal	Reads the black level pedestal register associated with the color filter phase 01
intel_vvp_blc_set_cfa_01_black_pedestal	Writes the black level pedestal register associated with the color filter phase 01
intel_vvp_blc_get_cfa_01_color_scaler	Reads the color scaler register associated with the color filter phase 01
intel_vvp_blc_set_cfa_01_color_scaler	Writes the color scaler register associated with the color filter phase 01
intel_vvp_blc_get_cfa_10_black_pedestal	Reads the black level pedestal register associated with the color filter phase 10
intel_vvp_blc_set_cfa_10_black_pedestal	Writes the black level pedestal register associated with the color filter phase 10
intel_vvp_blc_get_cfa_10_color_scaler	Reads the color scaler register associated with the color filter phase 10
intel_vvp_blc_set_cfa_10_color_scaler	Writes the color scaler register associated with the color filter phase 10
intel_vvp_blc_get_cfa_11_black_pedestal	Reads the black level pedestal register associated with the color filter phase 11
intel_vvp_blc_set_cfa_11_black_pedestal	Writes the black level pedestal register associated with the color filter phase 11
intel_vvp_blc_get_cfa_11_color_scaler	Reads the color scaler register associated with the color filter phase 11
intel_vvp_blc_set_cfa_11_color_scaler	Writes the color scaler register associated with the color filter phase 11

intel_vvp_blc_init

Prototype

```
int
intel_vvp_blc_init(intel_vvp_blc_instance* instance,
intel_vvp_core_base base);
```

Description

Initialize an IP instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

<i>Arguments</i>	instance - pointer to the intel_vvp_blc_instance software driver instance structure
	base - base address of the register map
<i>Returns</i>	kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
	kIntelVvpCoreInstanceErr if the instance is a null pointer
	kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7)
	kIntelVvpCorePidErr if the product_id does not match with the expected product ID (0x0177)
	kIntelVvpBlcRegMapVersionErr if the register map is not supported

intel_vvp_blc_get_lite_mode

Prototype

```
bool
intel_vvp_blc_get_lite_mode(intel_vvp_blc_instance* instance);
```

Description Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns true if the IP is parameterized in lite mode

intel_vvp_blc_get_debug_enabled

Prototype

```
bool
intel_vvp_blc_get_debug_enabled(intel_vvp_blc_instance* instance);
```

Description Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns true if the IP is parameterized with debug features enabled

intel_vvp_blc_get_bits_per_sample_in

Prototype

```
uint8_t
intel_vvp_blc_get_bits_per_sample_in(intel_vvp_blc_instance*
instance);
```

Description Returns the value of the BPS_IN register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the input bits per color sample parameter used to generate the IP

intel_vvp_blc_get_bits_per_sample_out

Prototype

```
uint8_t
intel_vvp_blc_get_bits_per_sample_out(intel_vvp_blc_instance*
instance);
```

Description Returns the value of the BPS_OUT register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the output bits per color sample parameter used to generate the IP

intel_vvp_blc_get_num_color_planes_in

Prototype

```
uint8_t
intel_vvp_blc_get_num_color_planes_in(intel_vvp_blc_instance*
instance);
```

Description Returns the value of the NUM_COLOR_IN register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the input number of color planes parameter used to generate the IP

intel_vvp_blc_get_num_color_planes_out

Prototype

```
uint8_t  
intel_vvp_blc_get_num_color_planes_out(intel_vvp_blc_instance*  
instance);
```

Description Returns the value of the NUM_COLOR_OUT register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the output number of color planes parameter used to generate the IP

intel_vvp_blc_get_pixels_in_parallel

Prototype

```
uint8_t  
intel_vvp_blc_get_pixels_in_parallel(intel_vvp_blc_instance*  
instance);
```

Description Returns the value of the PIXELS_IN_PARALLEL register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the pixels in parallel parameter used to generate the IP

intel_vvp_blc_get_max_width

Prototype

```
uint32_t  
intel_vvp_blc_get_max_width(intel_vvp_blc_instance* instance);
```

Description Returns the value of the MAX_WIDTH register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the maximum field width parameter used to generate the IP

intel_vvp_blc_get_max_height

Prototype

```
uint32_t
intel_vvp_blc_get_max_height(intel_vvp_blc_instance* instance);
```

Description Returns the value of the MAX_HEIGHT register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the maximum field height parameter used to generate the IP

intel_vvp_blc_get_reflect_around_zero

Prototype

```
bool
intel_vvp_blc_get_reflect_around_zero(intel_vvp_blc_instance*
instance);
```

Description Returns the value of the REFLECT_AROUND_ZERO register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the reflect around zero parameter used to generate the IP

intel_vvp_blc_is_running

Prototype

```
bool
intel_vvp_blc_is_running(intel_vvp_blc_instance* instance);
```

Description Returns the value of the IP Running bit from the STATUS register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the running bit from the status register

intel_vvp_blc_commit_is_pending

Prototype

```
bool  
intel_vvp_blc_commit_is_pending(intel_vvp_blc_instance*  
instance);
```

Description Returns the value of the commit pending bit from the STATUS register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns true if the agent interface has pending writes from a previous commit request

intel_vvp_blc_get_status

Prototype

```
uint32_t  
intel_vvp_blc_get_status(intel_vvp_blc_instance* instance);
```

Description Returns the value of the STATUS register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the value of the status register

intel_vvp_blc_get_frame_stats

Prototype

```
int  
intel_vvp_blc_get_frame_stats(intel_vvp_blc_instance*  
instance, uint32_t* stats_out);
```

Description Reads the value of the FRAME_STATS register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

stats_out - pointer of a variable to return the statistics register value

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpBlcPointerErr if stats_out is a null pointer

intel_vvp_blc_commit

Prototype

```
int
intel_vvp_blc_commit(intel_vvp_blc_instance* instance);
```

Description Commits all pending writes before processing the next field. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

intel_vvp_blc_get_bypass

Prototype

```
bool
intel_vvp_blc_get_bypass(intel_vvp_blc_instance* instance);
```

Description Returns the value of the bypass bit from the CONTROL register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the value of the bypass bit form the control register

intel_vvp_blc_set_bypass

Prototype

```
int
intel_vvp_blc_set_bypass(intel_vvp_blc_instance* instance,
bool bypass);
```

Description Writes to the bypass bit of the CONTROL register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

bypass - new value of the bypass bit

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpBlcCommitPendingErr if a previous commit request is pending

intel_vvp_blc_get_cfa_phase

Prototype

```
uint8_t  
intel_vvp_blc_get_cfa_phase(intel_vvp_blc_instance* instance);
```

Description Returns the value of the color filter array phase field from the CONTROL register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the value of the color filter array phase field form the control register

intel_vvp_blc_set_cfa_phase

Prototype

```
int  
intel_vvp_blc_set_cfa_phase(intel_vvp_blc_instance* instance,  
uint8_t cfa_phase);
```

Description Writes to the color filter array phase field of the CONTROL register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

cfa_phase - new value of the color filter array phase field

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpBlcValueErr if the value of cfa_phase is out of range

kIntelVvpBlcCommitPendingErr if a previous commit request is pending

intel_vvp_blc_get_clip_zero_en

Prototype

```
bool
intel_vvp_blc_get_clip_zero_en(intel_vvp_blc_instance*
instance);
```

Description Returns the value of the clip zero enable bit from the CONTROL register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the value of the clip zero enable bit form the control register

intel_vvp_blc_set_clip_zero_en

Prototype

```
int
intel_vvp_blc_set_clip_zero_en(intel_vvp_blc_instance*
instance, bool clip_zero_en);
```

Description Writes to the clip zero enable bit of the CONTROL register. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

clip_zero_en - new value of the clip zero enable bit

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpBlcCommitPendingErr if a previous commit request is pending

intel_vvp_blc_get_cfa_00_black_pedestal

Prototype

```
uint32_t
intel_vvp_blc_get_cfa_00_black_pedestal(intel_vvp_blc_instance*
instance);
```

Description Returns the black level pedestal register associated with the color filter phase 00. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the value of the black level pedestal register associated with the color filter phase 00

intel_vvp_blc_set_cfa_00_black_pedestal

Prototype

```
int
intel_vvp_blc_set_cfa_00_black_pedestal(intel_vvp_blc_instance*
instance, uint32_t pedestal);
```

Description Writes to the black level pedestal register associated with the color filter phase 00. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

pedestal - new value of the black level pedestal register associated with the color filter phase 00

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpBlcValueErr if the value of pedestal is out of range

kIntelVvpBlcCommitPendingErr if a previous commit request is pending

intel_vvp_blc_get_cfa_00_color_scaler

Prototype

```
uint32_t
intel_vvp_blc_get_cfa_00_color_scaler(intel_vvp_blc_instance
*instance);
```

Description Returns the color scaler register associated with the color filter phase 00. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the value of the color scaler register associated with the color filter phase 00

intel_vvp_blc_set_cfa_00_color_scaler

Prototype

```
int
intel_vvp_blc_set_cfa_00_color_scaler(intel_vvp_blc_instance*
instance, uint32_t scaler);
```

Description Writes to the color scaler register associated with the color filter phase 00. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

scaler - new value of the color scaler register associated with the color filter phase 00

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpBlcValueErr if the value of scaler is out of range

kIntelVvpBlcCommitPendingErr if a previous commit request is pending

intel_vvp_blc_get_cfa_01_black_pedestal

Prototype

```
uint32_t
intel_vvp_blc_get_cfa_01_black_pedestal(intel_vvp_blc_instance
*instance);
```

Description Returns the black level pedestal register associated with the color filter phase 01. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the value of the black level pedestal register associated with the color filter phase 01

intel_vvp_blc_set_cfa_01_black_pedestal

Prototype

```
int
intel_vvp_blc_set_cfa_01_black_pedestal(intel_vvp_blc_instance*
instance, uint32_t pedestal);
```


<i>Description</i>	Writes to the black level pedestal register associated with the color filter phase 01. The instance must be a valid intel_vvp_blc_instance fully initialized.
<i>Arguments</i>	instance - pointer to the intel_vvp_blc_instance software driver instance structure pedestal - new value of the black level pedestal register associated with the color filter phase 01
<i>Returns</i>	kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error kIntelVvpCoreInstanceErr if the instance is a null pointer kIntelVvpBlcValueErr if the value of pedestal is out of range kIntelVvpBlcCommitPendingErr if a previous commit request is pending

intel_vvp_blc_get_cfa_01_color_scaler

Prototype

```
uint32_t
intel_vvp_blc_get_cfa_01_color_scaler(intel_vvp_blc_instance
*instance);
```

<i>Description</i>	Returns the color scaler register associated with the color filter phase 01. The instance must be a valid intel_vvp_blc_instance fully initialized.
<i>Arguments</i>	instance - pointer to the intel_vvp_blc_instance software driver instance structure
<i>Returns</i>	the value of the color scaler register associated with the color filter phase 01

intel_vvp_blc_set_cfa_01_color_scaler

Prototype

```
int
intel_vvp_blc_set_cfa_01_color_scaler(intel_vvp_blc_instance*
instance, uint32_t scaler);
```

<i>Description</i>	Writes to the color scaler register associated with the color filter phase 01. The instance must be a valid intel_vvp_blc_instance fully initialized.
<i>Arguments</i>	instance - pointer to the intel_vvp_blc_instance software driver instance structure

scaler - new value of the color scaler register associated with the color filter phase 01

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpBlcValueErr if the value of scaler is out of range

kIntelVvpBlcCommitPendingErr if a previous commit request is pending

intel_vvp_blc_get_cfa_10_black_pedestal

Prototype

```
uint32_t
intel_vvp_blc_get_cfa_10_black_pedestal(intel_vvp_blc_instance*instance);
```

Description Returns the black level pedestal register associated with the color filter phase 10. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the value of the black level pedestal register associated with the color filter phase 10

intel_vvp_blc_set_cfa_10_black_pedestal

Prototype

```
int
intel_vvp_blc_set_cfa_10_black_pedestal(intel_vvp_blc_instance*instance, uint32_t pedestal);
```

Description Writes to the black level pedestal register associated with the color filter phase 10. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

pedestal - new value of the black level pedestal register associated with the color filter phase 10

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpBlcValueErr if the value of pedestal is out of range

kIntelVvpBlcCommitPendingErr if a previous commit request is pending

intel_vvp_blc_get_cfa_10_color_scaler

Prototype

```
uint32_t  
intel_vvp_blc_get_cfa_10_color_scaler(intel_vvp_blc_instance  
*instance);
```

Description Returns the color scaler register associated with the color filter phase 10. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the value of the color scaler register associated with the color filter phase 10

intel_vvp_blc_set_cfa_10_color_scaler

Prototype

```
int  
intel_vvp_blc_set_cfa_10_color_scaler(intel_vvp_blc_instance*  
instance, uint32_t scaler);
```

Description Writes to the color scaler register associated with the color filter phase 10. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

scaler - new value of the color scaler register associated with the color filter phase 10

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpBlcValueErr if the value of scaler is out of range

kIntelVvpBlcCommitPendingErr if a previous commit request is pending

intel_vvp_blc_get_cfa_11_black_pedestal

Prototype

```
uint32_t
intel_vvp_blc_get_cfa_11_black_pedestal(intel_vvp_blc_instance
*instance);
```

Description Returns the black level pedestal register associated with the color filter phase 11. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

Returns the value of the black level pedestal register associated with the color filter phase 11

intel_vvp_blc_set_cfa_11_black_pedestal

Prototype

```
int
intel_vvp_blc_set_cfa_11_black_pedestal(intel_vvp_blc_instance*
instance, uint32_t pedestal);
```

Description Writes to the black level pedestal register associated with the color filter phase 11. The instance must be a valid intel_vvp_blc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_blc_instance software driver instance structure

pedestal - new value of the black level pedestal register associated with the color filter phase 11

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpBlcValueErr if the value of pedestal is out of range

kIntelVvpBlcCommitPendingErr if a previous commit request is pending

intel_vvp_blc_get_cfa_11_color_scaler

Prototype

```
uint32_t
intel_vvp_blc_get_cfa_11_color_scaler(intel_vvp_blc_instance
*instance);
```

<i>Description</i>	Returns the color scaler register associated with the color filter phase 11. The instance must be a valid intel_vvp_blc_instance fully initialized.
<i>Arguments</i>	instance - pointer to the intel_vvp_blc_instance software driver instance structure
<i>Returns</i>	the value of the color scaler register associated with the color filter phase 11

intel_vvp_blc_set_cfa_11_color_scaler

Prototype

```
int  
intel_vvp_blc_set_cfa_11_color_scaler(intel_vvp_blc_instance*  
instance, uint32_t scaler);
```

<i>Description</i>	Writes to the color scaler register associated with the color filter phase 11. The instance must be a valid intel_vvp_blc_instance fully initialized.
<i>Arguments</i>	instance - pointer to the intel_vvp_blc_instance software driver instance structure scaler - new value of the color scaler register associated with the color filter phase 11
<i>Returns</i>	kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error kIntelVvpCoreInstanceErr if the instance is a null pointer kIntelVvpBlcValueErr if the value of scaler is out of range kIntelVvpBlcCommitPendingErr if a previous commit request is pending

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

15. Black Level Statistics IP

15.1. About the Black Level Statistics IP

The Black Level Statistics IP accumulates pixel values in a region of interest. This region may be a shielded area of an imaging sensor (an optical black region) or the entire sensor area, when the lens cap is on. You can use these statistics to track the accuracy of a sensor in keeping its black level value constant and compensate for deviations if necessary.

The IP only supports lite variants. For more information about full and lite variants, refer to the *Altera Streaming Video Protocol Specification*

Related Information

[Altera Streaming Video Protocol Specification](#)

15.1.1. Black Level Statistics IP Performance and Resources

Altera provides resource and utilization data for guidance only. The IP resource utilization depends on the device family, compilation tool version, compilation seed randomization, and other parameters.

Table 171. Black Level Statistics IP Performance and Resources

Device	Pixels in Parallel	Bit per Color Sample	Max Resolution	ALM	M20k	DSP	f _{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	1	10	2k	688	0	0	777
	1	16	2k	709	0	0	775
	4	10	8k	1,347	0	0	773
	4	16	8k	1,314	0	0	816
Arria 10 speed grade 1 (10AS066H1F34E1HG)	1	10	2k	599	0	0	618
	1	16	2k	613	0	0	601
	2	10	4k	817	0	0	630
	2	16	4k	847	0	0	572
Cyclone 10 GX speed grade 5 (10CX220YF672E5G)	1	10	2k	600	0	0	543
	1	16	2k	613	0	0	504
Stratix 10 speed grade 1 (1SX280LN2F43E1VG)	1	10	2k	697	0	0	703
	1	16	2k	697	0	0	685
	2	10	4k	908	0	0	652
	2	16	4k	869	0	0	663

15.2. Black Level Statistics IP Parameters

The IP offers run-time and compile-time parameters.

Table 172. Black Level Statistics IP Parameters

Parameter	Values	Description
Video Data Format		
Lite mode	On	The IP only supports lite mode
Bits per color sample	8 to 16	Select the number of bits per color sample.
Number of color planes	1	The IP only supports 1 color plane
Number of pixels in parallel	1 to 8	Select the number of pixels in parallel.
Maximum Frame Size		
Maximum field height	1k, 2k, 4k, 8k, 16k	Specify the maximum height of incoming frames
Maximum field width	2k, 4k, 8k, 16k	Specify the maximum width of incoming frames
General		
Pipeline ready signal	On or off	Turn on to add extra pipeline registers to the AXI4-S <code>trready</code> signals
Separate clock for control interface	On or off	Turn on for a separate clock for the control agent interface
Debug features	On or off	Turn on to read back frame information registers and debugging information registers via the control agent interface.

Figure 37. Black Level Statistics IP GUI

Black Level Statistics Intel FPGA IP
intel_vvp_bls

Important Usage Information
All resets must be at least 256 clock cycles in duration. When the IP uses multiple reset inputs, ensure that the reset pulses overlap by at least 16 clock cycles. See "Reset Behavior" in the Video and Vision Processing User Guide

Video Data Format

☒ Lite mode

Bits per color sample: 8

Number of color planes: 1

Number of pixels in parallel: 1

Maximum Frame Size

Maximum field height: 4096

Maximum field width: 4096

General

☐ Pipeline ready signals

☐ Separate clock for control interface

☐ Debug features

15.3. Black Level Statistics IP Interfaces

Table 173. Black Level Statistics Clock and Reset Interfaces

Name	Direction	Width	Description
main_clock_clk	In	1	AXI4-S processing clock.
main_reset_rst	In	1	AXI4-S processing reset.
agent_clock_clk	In	1	Clock for the memory-mapped control agent interface. Only if you turn on Separate clock for control interface .
agent_reset_reset	In	1	Reset for the memory-mapped control agent interface. Only if you turn on Separate clock for control interface .

Table 174. Black Level Statistics Control Interfaces

Name	Direction	Width	Description
av_mm_control_agent_address	In	7	Avalon memory-mapped agent address
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write.
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data.
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable.
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read.
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data.
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read.
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request.

Table 175. Black Level Statistics Altera Streaming Video Interfaces

Name	Direction	Width	Description
Input			
axi4s_vid_in_tdata	In	(37)	AXI4-S data in.
axi4s_vid_in_tvalid	In	1	AXI4-S data valid.
axi4s_vid_in_tuser	In	(38)	[0] AXI4-S start of video frame. [N-1:1] Unused.
axi4s_vid_in_tlast	In	1	AXI4-S end of packet.
axi4s_vid_in_tready	Out	1	AXI4-S data ready.
Output			
axi4s_vid_out_tdata	Out	(37)	AXI4-S data out.
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid.
axi4s_vid_out_tuser	Out	(38)	[0] AXI4-S start of video frame. [N-1:1] Unused.
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet.
axi4s_vid_out_tready	In	1	AXI4-S data ready.

Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

15.4. Black Level Statistics IP Registers

Each register is either read-only (RO) or read-write (RW).

(37) The equation gives all tdata widths sizes in these interfaces: pixels in parallel × max (round (bits per color sample × number of color planes, 8), 16)

(38) This equation gives all tuserwidths sizes in these interfaces: $N = \text{ceil}(\text{tdata width} / 8)$

Table 176. Black Level Statistics Registers

In the software API the register names appear with a prefix of INTEL_VVP, INTEL_VVP_CORE, or INTEL_VVP_BLS as appropriate and with an optional REG suffix

Address	Register	Access		Description
		Lite ⁽³⁹⁾	Full	
Parameterization registers				
0x0000	VID_PID	RO	N/A	Read this register to retrieve the ID of the IP. This register always returns 0x6FA7_0174.
0x0004	VERSION	RO	N/A	Read this register to retrieve the version information for the IP.
0x0008	LITE_MODE	RO	N/A	Read this register to determine if Lite mode is on. This register always returns 1.
0x000C	DEBUG_ENABLED	RO	N/A	Read this register to determine if Debug features are on. This register returns 0 for off and 1 for on.
0x0010	BPS_IN	RO	N/A	Read this register to determine the Bits per color symbol for the input streaming video interface.
0x0014	BPS_OUT	RO	N/A	Read this register to determine the Bits per color symbol for the output streaming video interface. The value of this register reads the same for its input counterpart.
0x0018	NUM_COLOR_IN	RO	N/A	Read this register to determine the Number of color planes for the input streaming video interface.
0x001C	NUM_COLOR_OUT	RO	N/A	Read this register to determine the Number of color planes for the output streaming video interface. The value of this register reads the same for its input counterpart.
0x0020	PIP	RO	N/A	Read this register to determine the Number of pixels in parallel .
0x0024	MAX_WIDTH	RO	N/A	Read this register to determine the Maximum field width .
0x0028	MAX_HEIGHT	RO	N/A	Read this register to determine the Maximum field height .
0x002C to 0x011F	-	-	-	Reserved
Control, debug and statistics registers				
0x0120	IMG_INFO_WIDTH	RW	N/A	The expected width of the incoming video fields.
0x0124	IMG_INFO_HEIGHT	RW	N/A	The expected height of the incoming video fields.
0x0128 to 0x013F	-	-	-	Reserved
0x0140	STATUS	RO	N/A	Read this register for information about the IP status.
0x0144	FRAME_STATS	RO	N/A	Read this register for some frame statistics.
0x0148	CFA_00_SUM	RO	N/A	Sum of pixel values in the optical black region for color channel 0.
continued...				

⁽³⁹⁾ Registers are RW only if you also turn on **Debug features**, otherwise they are WO.

Address	Register	Access		Description
		Lite ⁽³⁹⁾	Full	
0x014c	CFA_01_SUM	RO	N/A	Sum of pixel values in the optical black region for color channel 1.
0x0150	CFA_10_SUM	RO	N/A	Sum of pixel values in the optical black region for color channel 2.
0x0154	CFA_11_SUM	RO	N/A	Sum of pixel values in the optical black region for color channel 3.
0x0158	COMMIT	RW	N/A	Write any value to this register to submit changes to the control and region of interest registers. A pending commit request does not block <i>Freeze Statistics Request</i> of the <code>CONTROL</code> register, which the IP samples continuously.
0x015c	CONTROL	RW	N/A	Control bits and fields of Black Level Statistics IP
0x0160	H_START	RW	N/A	Region of interest horizontal start value.
0x0164	V_START	RW	N/A	Region of interest vertical start value.
0x0168	H_END	RW	N/A	Region of interest horizontal end value.
0x016c	V_END	RW	N/A	Region of interest vertical end value.

Register Bit Descriptions

Table 177. STATUS

Name	Bits	Description
Reserved	31:3	Reserved.
Stats are Frozen	2	The IP sets this bit to indicate it stopped updating the statistics.
Commit	1	Pending commit
Running	0	When 1, the IP is processing data.

Table 178. FRAME_STATS

Name	Bits	Description
Reserved	31:8	Reserved.
Checksum	7:0	A simple checksum of the frame.

Table 179. CONTROL

Name	Bits	Description
Freeze Statistics Request	31	Set this bit to 1 for the IP to start sampling a new set of black level and frame statistics. The IP lowers the <code>Stats are Frozen</code> bit of the <code>STATUS</code> register as a response, and you need to poll <code>Stats are Frozen</code> bit until it is 1 before reading the statistics. The IP expects you to keep

continued...

⁽³⁹⁾ Registers are RW only if you also turn on **Debug features**, otherwise they are WO.

Name	Bits	Description
		Freeze Statistics Request 1 until it raises Stats are Frozen. Set Freeze Statistics Request to 0 before starting over with a new sampling request. Deviating from this flow might result in unexpected IP behavior.
Reserved	30:3	Reserved. Write 0.
Color filter array phase	2:1	Specifies 2x2 color filter order starting from the top left corner of the image. 00 01 10 11 C0C1 C1C0 C2C3 C3C2 C2C3 C3C2 C0C1 C1C0
Reserved.	0	Reserved. Write 0

Table 180. H_START

Name	Bits	Description
Reserved	31:16	Reserved, Write 0.
Position	15:0	Horizontal position of the first pixel in the region of interest. The IP starts counting the pixels from index 0.

Table 181. V_START

Name	Bits	Description
Reserved	31:16	Reserved, Write 0
Position	15:0	Vertical position of the first line in the region of interest decremented by 1. The IP starts counts the lines from index 0. Set the value of this register to the real starting line index minus 1. The line counter wraps around. Write 65535 to set vertical starting position to line 0.

Table 182. H_END

Name	Bits	Description
Reserved	31:16	Reserved. Write 0.
Position	15:0	Horizontal position of the last pixel in the region of interest incremented by 1. Set this register to point to the first pixel index following the end of the optical black region.

Table 183. V_END

Name	Bits	Description
Reserved	31:16	Reserved, write 0.
Position	15:0	Vertical position of the last line in the region of interest. Set this register to point to the last line index of the optical black region.

15.5. Black Level Statistics IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_bls_regs.h`

Include file: intel_vvp_bls.h

Table 184. Black Level Statistics IP Software API

Name	Description
intel_vvp_bls_init	Initialize an IP instance
intel_vvp_core_*	Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i>
intel_vvp_bls_get_lite_mode	Returns if Lite mode is on
intel_vvp_bls_get_debug_enabled	Returns if Debug features is on
intel_vvp_bls_get_bits_per_sample_in	Returns the number of bits per color sample for the streaming input interface
intel_vvp_bls_get_bits_per_sample_out	Returns the number of bits per color sample value for the streaming output interface
intel_vvp_bls_get_num_color_planes_in	Returns the number of color planes of the streaming input interface
intel_vvp_bls_get_num_color_planes_out	Returns the number of color planes of the streaming output interface
intel_vvp_bls_get_pixels_in_parallel	Returns the pixels in parallel of the streaming input and output interfaces
intel_vvp_bls_get_max_width	Returns the maximum number of pixels that the IP supports on the horizontal dimension of an image or video frame
intel_vvp_bls_get_max_height	Returns the maximum number of pixels that the IP supports on the vertical dimension of an image or video frame
intel_vvp_bls_is_running	Reads if the IP is running
intel_vvp_bls_commit_is_pending	Reads if the IP has uncommitted writes
intel_vvp_bls_stats_are_frozen	Reads if the IP finished collecting the statistics
intel_vvp_bls_get_status	Reads the status register
intel_vvp_bls_get_frame_stats	Reads the frame statistics register
intel_vvp_bls_get_cfa_00_sum	Reads the black level sum register associated with the color filter phase 00
intel_vvp_bls_get_cfa_01_sum	Reads the black level sum register associated with the color filter phase 01
intel_vvp_bls_get_cfa_10_sum	Reads the black level sum register associated with the color filter phase 10
intel_vvp_bls_get_cfa_11_sum	Reads the black level sum register associated with the color filter phase 11
intel_vvp_bls_commit	Commit pending writes
intel_vvp_bls_get_cfa_phase	Reads the color filter array phase field from the status register
intel_vvp_bls_set_cfa_phase	Writes the color filter array phase field of the status register
intel_vvp_bls_get_freeze_stats_request	Reads the freeze statistics request bit from the status register
intel_vvp_bls_set_freeze_stats_request	Writes the freeze statistics request bit of the status register
intel_vvp_bls_get_h_start	Reads the horizontal starting coordinate of the region of interest
intel_vvp_bls_set_h_start	Writes the horizontal starting coordinate of the region of interest
continued...	

Name	Description
intel_vvp_bls_get_v_start	Reads the vertical starting coordinate of the region of interest
intel_vvp_bls_set_v_start	Writes the vertical starting coordinate of the region of interest
intel_vvp_bls_get_h_end	Reads the horizontal ending coordinate of the region of interest
intel_vvp_bls_set_h_end	Writes the horizontal ending coordinate of the region of interest
intel_vvp_bls_get_v_end	Reads the vertical ending coordinate of the region of interest
intel_vvp_bls_set_v_end	Writes the vertical ending coordinate of the region of interest

intel_vvp_bls_init

Prototype

```
int
intel_vvp_bls_init(intel_vvp_bls_instance* instance,
intel_vvp_core_base base);
```

Description Initialize an IP instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

base - base address of the register map

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7)

kIntelVvpCorePidErr if the product_id does not match with the expected product ID (0x0174)

kIntelVvpBlsRegMapVersionErr if the register map is not supported

intel_vvp_bls_get_lite_mode

Prototype

```
bool
intel_vvp_bls_get_lite_mode(intel_vvp_bls_instance* instance);
```

Description Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns true if the IP is parameterized in lite mode

intel_vvp_bls_get_debug_enabled

Prototype

```
bool  
intel_vvp_bls_get_debug_enabled(intel_vvp_bls_instance*  
instance);
```

Description Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns true if the IP is parameterized with debug features enabled

intel_vvp_bls_get_bits_per_sample_in

Prototype

```
uint8_t  
intel_vvp_bls_get_bits_per_sample_in(intel_vvp_bls_instance*  
instance);
```

Description Returns the value of the BPS_IN register. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns the input bits per color sample parameter used to generate the IP

intel_vvp_bls_get_bits_per_sample_out

Prototype

```
uint8_t  
intel_vvp_bls_get_bits_per_sample_out(intel_vvp_bls_instance*  
instance);
```

Description Returns the value of the BPS_OUT register. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns the output bits per color sample parameter used to generate the IP

intel_vvp_bls_get_num_color_planes_in

Prototype

```
uint8_t
intel_vvp_bls_get_num_color_planes_in(intel_vvp_bls_instance*
instance);
```

Description Returns the value of the NUM_COLOR_IN register. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns the input number of color planes parameter used to generate the IP

intel_vvp_bls_get_num_color_planes_out

Prototype

```
uint8_t
intel_vvp_bls_get_num_color_planes_out(intel_vvp_bls_instance*
instance);
```

Description Returns the value of the NUM_COLOR_OUT register. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns the output number of color planes parameter used to generate the IP

intel_vvp_bls_get_pixels_in_parallel

Prototype

```
uint8_t
intel_vvp_bls_get_pixels_in_parallel(intel_vvp_bls_instance*
instance);
```

Description Returns the value of the PIXELS_IN_PARALLEL register. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns the pixels in parallel parameter used to generate the IP

intel_vvp_bls_get_max_width

Prototype

```
uint32_t  
intel_vvp_bls_get_max_width(intel_vvp_bls_instance* instance);
```

Description Returns the value of the MAX_WIDTH register. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns the maximum field width parameter used to generate the IP

intel_vvp_bls_get_max_height

Prototype

```
uint32_t  
intel_vvp_bls_get_max_height(intel_vvp_bls_instance* instance);
```

Description Returns the value of the MAX_HEIGHT register. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns the maximum field height parameter used to generate the IP

intel_vvp_bls_is_running

Prototype

```
bool  
intel_vvp_bls_is_running(intel_vvp_bls_instance* instance);
```

Description Returns the value of the IP Running bit from the STATUS register. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns the running bit from the status register

intel_vvp_bls_commit_is_pending

Prototype

```
bool  
intel_vvp_bls_commit_is_pending(intel_vvp_bls_instance* instance);
```

Description Returns the value of the commit pending bit from the STATUS register. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns true if the agent interface has pending writes from a previous commit request

intel_vvp_bls_stats_are_frozen

Prototype

```
bool
intel_vvp_bls_stats_are_frozen(intel_vvp_bls_instance*
instance);
```

Description Returns the value of the statistics are frozen bit from the STATUS register. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns the statistics are frozen bit from the status register

intel_vvp_bls_get_status

Prototype

```
uint32_t
intel_vvp_bls_get_status(intel_vvp_bls_instance* instance);
```

Description Returns the value of the STATUS register. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns the value of the status register

intel_vvp_bls_get_frame_stats

Prototype

```
int
intel_vvp_bls_get_frame_stats(intel_vvp_bls_instance*
instance, uint32_t* stats_out);
```

<i>Description</i>	Reads the value of the FRAME_STATS register. The instance must be a valid intel_vvp_bls_instance fully initialized.
<i>Arguments</i>	instance - pointer to the intel_vvp_bls_instance software driver instance structure stats_out - pointer of a variable to return the statistics register value
<i>Returns</i>	kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error kIntelVvpCoreInstanceErr if the instance is a null pointer kIntelVvpBlsPointerErr if stats_out is a null pointer kIntelVvpBlsFreezePendingErr if the last freeze request is pending

intel_vvp_bls_get_cfa_00_sum

Prototype

```
int
intel_vvp_bls_get_cfa_00_sum(intel_vvp_bls_instance* instance,
uint32_t* stats_out);
```

<i>Description</i>	Reads the value of the CFA_00_SUM register. The instance must be a valid intel_vvp_bls_instance fully initialized.
<i>Arguments</i>	instance - pointer to the intel_vvp_bls_instance software driver instance structure stats_out - pointer of a variable to return the statistics register value
<i>Returns</i>	kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error kIntelVvpCoreInstanceErr if the instance is a null pointer kIntelVvpBlsPointerErr if stats_out is a null pointer kIntelVvpBlsFreezePendingErr if the last freeze request is pending

intel_vvp_bls_get_cfa_01_sum

Prototype

```
int
intel_vvp_bls_get_cfa_01_sum(intel_vvp_bls_instance* instance,
uint32_t* stats_out);
```

<i>Description</i>	Reads the value of the CFA_01_SUM register. The instance must be a valid intel_vvp_bls_instance fully initialized.
--------------------	--

- Arguments**
- instance - pointer to the intel_vvp_bls_instance software driver instance structure
 - stats_out - pointer of a variable to return the statistics register value
- Returns**
- kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
 - kIntelVvpCoreInstanceErr if the instance is a null pointer
 - kIntelVvpBlsPointerErr if stats_out is a null pointer
 - kIntelVvpBlsFreezePendingErr if the last freeze request is pending

intel_vvp_bls_get_cfa_10_sum

Prototype

```
int
intel_vvp_bls_get_cfa_10_sum(intel_vvp_bls_instance* instance,
uint32_t* stats_out);
```

- Description** Reads the value of the CFA_10_SUM register. The instance must be a valid intel_vvp_bls_instance fully initialized.
- Arguments**
- instance - pointer to the intel_vvp_bls_instance software driver instance structure
 - stats_out - pointer of a variable to return the statistics register value
- Returns**
- kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
 - kIntelVvpCoreInstanceErr if the instance is a null pointer
 - kIntelVvpBlsPointerErr if stats_out is a null pointer
 - kIntelVvpBlsFreezePendingErr if the last freeze request is pending

intel_vvp_bls_get_cfa_11_sum

Prototype

```
int
intel_vvp_bls_get_cfa_11_sum(intel_vvp_bls_instance* instance,
uint32_t* stats_out);
```

- Description** Reads the value of the CFA_11_SUM register. The instance must be a valid intel_vvp_bls_instance fully initialized.
- Arguments**
- instance - pointer to the intel_vvp_bls_instance software driver instance structure

stats_out - pointer of a variable to return the statistics register value

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpBlsPointerErr if stats_out is a null pointer

kIntelVvpBlsFreezePendingErr if the last freeze request is pending

intel_vvp_bls_commit

Prototype

```
int
intel_vvp_bls_commit(intel_vvp_bls_instance* instance);
```

Description Commits all pending writes before processing the next field. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

intel_vvp_bls_get_cfa_phase

Prototype

```
uint8_t
intel_vvp_bls_get_cfa_phase(intel_vvp_bls_instance* instance);
```

Description Returns the value of the color filter array phase field from the CONTROL register. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns the value of the color filter array phase field form the control register

intel_vvp_bls_set_cfa_phase

Prototype

```
int
intel_vvp_bls_set_cfa_phase(intel_vvp_bls_instance* instance,
uint8_t cfa_phase);
```

Description Writes to the color filter array phase field of the CONTROL register. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

cfa_phase - new value of the color filter array phase field

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpBlsValueErr if the value of cfa_phase is out of range

kIntelVvpBlsCommitPendingErr if a previous commit request is pending

intel_vvp_bls_get_freeze_stats_request

Prototype

```
bool
intel_vvp_bls_get_freeze_stats_request(intel_vvp_bls_instance*
instance);
```

Description Returns the value of the freeze statistics request bit from the CONTROL register. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns the value of the freeze statistics request bit form the control register

intel_vvp_bls_set_freeze_stats_request

Prototype

```
int
intel_vvp_bls_set_freeze_stats_request(intel_vvp_bls_instance*
instance, bool freeze_stats);
```

Description Writes to the freeze statistics request bit of the CONTROL register. The instance must be a valid intel_vvp_bls_instance fully initialized.

- Arguments**
- instance - pointer to the intel_vvp_bls_instance software driver instance structure
 - freeze_stats - new value of the freeze statistics request bit
- Returns**
- kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
 - kIntelVvpCoreInstanceErr if the instance is a null pointer

intel_vvp_bls_get_h_start

Prototype

```
uint16_t
intel_vvp_bls_get_h_start(intel_vvp_bls_instance* instance);
```

- Description**
- Returns the horizontal starting coordinate of the region of interest. The instance must be a valid intel_vvp_bls_instance fully initialized.
- Arguments**
- instance - pointer to the intel_vvp_bls_instance software driver instance structure
- Returns**
- the value of the horizontal starting coordinate of the region of interest

intel_vvp_bls_set_h_start

Prototype

```
int
intel_vvp_bls_set_h_start(intel_vvp_bls_instance* instance,
uint16_t h_start);
```

- Description**
- Writes the horizontal starting coordinate of the region of interest. The instance must be a valid intel_vvp_bls_instance fully initialized.
- Arguments**
- instance - pointer to the intel_vvp_bls_instance software driver instance structure
 - h_start - new value of the horizontal starting coordinate of the region of interest
- Returns**
- kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
 - kIntelVvpCoreInstanceErr if the instance is a null pointer
 - kIntelVvpBlsValueErr if the value of h_start is out of range
 - kIntelVvpBlsCommitPendingErr if a previous commit request is pending

intel_vvp_bls_get_v_start

Prototype

```
uint16_t
intel_vvp_bls_get_v_start(intel_vvp_bls_instance* instance);
```

Description Returns the vertical starting coordinate of the region of interest. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns the value of the vertical starting coordinate of the region of interest

intel_vvp_bls_set_v_start

Prototype

```
int
intel_vvp_bls_set_v_start(intel_vvp_bls_instance* instance,
uint16_t v_start);
```

Description Writes the vertical starting coordinate of the region of interest. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

v_start - new value of the vertical starting coordinate of the region of interest

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpBlsValueErr if the value of v_start is out of range

kIntelVvpBlsCommitPendingErr if a previous commit request is pending

intel_vvp_bls_get_h_end

Prototype

```
uint16_t
intel_vvp_bls_get_h_end(intel_vvp_bls_instance* instance);
```

Description Returns the horizontal ending coordinate of the region of interest. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns the value of the horizontal ending coordinate of the region of interest

intel_vvp_bls_set_h_end

Prototype

```
int
intel_vvp_bls_set_h_end(intel_vvp_bls_instance* instance,
uint16_t h_end);
```

Description Writes the horizontal ending coordinate of the region of interest. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

h_end - new value of the horizontal ending coordinate of the region of interest

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpBlsValueErr if the value of h_end is out of range

kIntelVvpBlsCommitPendingErr if a previous commit request is pending

intel_vvp_bls_get_v_end

Prototype

```
uint16_t
intel_vvp_bls_get_v_end(intel_vvp_bls_instance* instance);
```

Description Returns the vertical ending coordinate of the region of interest. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

Returns the value of the vertical ending coordinate of the region of interest

intel_vvp_bls_set_v_end

Prototype

```
int
intel_vvp_bls_set_v_end(intel_vvp_bls_instance* instance,
uint16_t v_end);
```

Description Writes the vertical ending coordinate of the region of interest. The instance must be a valid intel_vvp_bls_instance fully initialized.

Arguments instance - pointer to the intel_vvp_bls_instance software driver instance structure

v_end - new value of the vertical ending coordinate of the region of interest

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpBlsValueErr if the value of v_end is out of range

kIntelVvpBlsCommitPendingErr if a previous commit request is pending

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

16. Chroma Key IP

16.1. About the Chroma Key IP

The Chroma Key IP appends an additional alpha plane to each incoming pixel of video data.

The IP appends either:

- A constant alpha value to each pixel.
- A conditional alpha value with conditional pixel replacement.

When you turn on **Constant Alpha mode**, the IP appends a single alpha value to each pixel. The constant alpha value is set at compile time using a parameter.

When you turn off **Constant Alpha mode**, the IP selects the alpha value to append to each pixel based on whether the pixel matches a set of conditions described in the control registers. You can also turn on pixel replacement, which allows you to set the pixel's color plane values. These values depend on if the pixel matches the user-defined condition you define using the memory-mapped control interface.

The IP is available as full or lite variants. For more information on full and lite, refer to the *Altera Streaming Video Protocol Specification*. The Chroma Key IP decodes input resolutions and field type by reading image information packets for full variants or using the register interface for lite variants.

Related Information

[Altera Streaming Video Protocol Specification](#)

16.1.1. Chroma Key IP Performance and Resources

The IP resources depend on the device family, bits per sample, pixels in parallel, number of color planes.

Table 185. Chroma Key IP Performance and Resources

The table shows ALM usage and f_{MAX} for an example full parameterization with 3 pixels in parallel, 10 bits per color sample, 3 color planes.

Target Device	ALMs	f_{MAX} MHz
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	780	781
Arria 10 speed grade 1 (10AS066H1F34E1HG)	664	460
Cyclone 10 GX speed grade 5 (10CX220YF672E5G)	666	435
Stratix 10 speed grade 1 (1SX280LN2F43E1VG)	776	873

16.2. Chroma Key IP Parameters

The IP offers compile-time parameters.

Table 186. Chroma Key IP Parameters

Parameter	Values	Description
Video Data Format		
Number of pixels in parallel	1 to 8	Select the number of pixels in parallel.
Number of color planes	1 or 3	Select the number of input color planes per pixel. (Output color planes are $N+1$ for carriage of alpha channel)
Bits per color sample	8 to 16	Select the number of bits per color sample.
Lite mode	On or off	Turn on to use the lite variant of the Altera Streaming Video Protocol.
Control		
Memory-mapped control interface	On or off	Turn on to specify the pixel matching ranges and alpha values for pixel tagging. The memory-mapped control interface is mandatory in lite mode.
Separate clock for control interface	On or off	Turn on to enable a separate clock for the control interface.
Chroma Key Behavior		
Constant Alpha Mode	On or off	Turn on for constant alpha mode.
Constant Alpha Tag	0 to 65535	If Constant Alpha Mode is on, the IP appends the Constant Alpha Tag value to each pixel.
General		
Debug features	On or off	Turn on for debug features (not applicable for lite mode).
Pipeline Ready	On or off	Turn on to pipeline the <code>Tready</code> signal.

Figure 38. Chroma Key IP GUI

The screenshot displays the Chroma Key IP GUI with the following settings:

- Video Data Format:**
 - Number of pixels in parallel: 1
 - Number of color planes: 3
 - Bits per color sample: 8
 - ☐ Lite mode
- Control Settings:**
 - ☒ Memory mapped control interface
 - ☐ Separate clock for control interface
- General Settings:**
 - ☐ Pipeline ready signals
 - ☐ Debug features
- Alpha Mode Settings:**
 - ☐ Constant Alpha mode
 - Constant alpha tag: 0

16.3. Chroma Key IP Interfaces

Table 187. Chroma Key IP Interfaces

Name	Direction	Width	Description
Clocks and resets			
main_clock_clk	Input	1	AXI4-S processing clock.
main_reset_reset	Input	1	AXI4-S processing reset.
agent_clock_clk	Input	1	Optional control agent interface clock.
agent_reset_reset	Input	1	Optional control agent interface reset.
Control interfaces			
av_mm_control_agent_address	Input	7	Avalon memory-mapped agent address.
av_mm_control_agent_write	Input	1	Avalon memory-mapped agent write.
av_mm_control_agent_writedata	Input	32	Avalon memory-mapped agent write data.
av_mm_control_agent_byteenable	Input	4	Avalon memory-mapped agent byte enable.
av_mm_control_agent_read	Input	1	Avalon memory-mapped agent read.
av_mm_control_agent_readdata	Output	32	Avalon memory-mapped agent read data.
continued...			

Name	Direction	Width	Description
av_mm_control_agent_readdatavalid	Output	1	Avalon memory-mapped agent read.
av_mm_control_agent_waitrequest	Output	1	Avalon memory-mapped agent wait request.
Altera Streaming Video interfaces			
axi4s_vid_in_tdata	Input	(40)	AXI4-S data in.
axi4s_vid_in_tvalid	Input	1	AXI4-S data valid.
axi4s_vid_in_tuser[0]	Input	1	AXI4-S start of video frame.
axi4s_vid_in_tuser[1]	Input	1	AXI4-S control or data packet.
axi4s_vid_in_tuser[N-1:2]	Input	(41)	Unused.
axi4s_vid_in_tlast	Input	1	AXI4-S end of packet.
axi4s_vid_in_tready	Output	1	AXI4-S data ready.
axi4s_vid_out_tdata	Output	(40)	AXI4-S data in.
axi4s_vid_out_tvalid	Output	1	AXI4-S data valid.
axi4s_vid_out_tuser[0]	Output	1	AXI4-S start of video frame.
axi4s_vid_out_tuser[1]	Output	1	AXI4-S control or data packet.
axi4s_vid_out_tuser[N-1:2]	Output	(41)	Unused.
axi4s_vid_out_tlast	Output	1	AXI4-S end of packet.
axi4s_vid_out_tready	Input	1	AXI4-S data ready

Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

16.4. Chroma Key Replacement

Color plane replacement only occurs if the value is set by the control register. You must specify the upper and lower bounds for pixel matching.

If you turn off each plane comparison, it is equivalent to using a constant alpha specified by the matched alpha register as all pixels match. If a pixel falls in the specified range of all color planes, the IP applies the matched alpha to the most significant plane and it implements any enabled replacement values. The figure shows each red, green and blue planes fall within their respective ranges. The IP applies the matched alpha value (550) and replaces the red value with the specified replacement value (1023).

⁽⁴⁰⁾ The equation gives all tdata widths sizes in these interfaces:

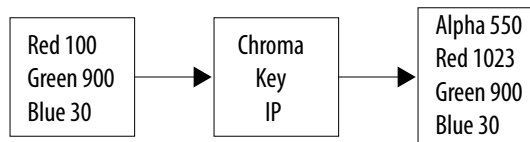
$$\max(2, \text{ceil}((\text{bits per color sample} \times \text{Number of color planes})/8)) \times \text{pixels in parallel} \times 8$$

⁽⁴¹⁾ This equation gives all tuser widths sizes in these interfaces $N = \text{ceil}(\text{tdata width} / 8)$

Table 188. Chroma Key Replacement

Setting	Value
Red range	0 to 200
Blue range	0 to 200
Green range	550 to 1023
Matched alpha	550
Unmatched alpha	1023
Red replace	1023
Green replace	Off
Blue replace	Off

Figure 39. Chroma Key Replacement



16.5. Chroma Key IP Registers

Each register is either read-only (RO) or read-write (RW).

In the software API the register names appear with a prefix of `INTEL_VVP`, `INTEL_VVP_CORE`, or `INTEL_VVP_CHROMA_KEY` as appropriate and with an optional `REG` suffix.

Table 189. Control Registers

Address	Register	Access		Description
		Lite (42)	Full	
Parameterization registers				
0x0000	PROD_ID	RO	RO	Read this register for the Chroma Key product ID. This register always returns 0x6AF7_023F.
0x0004	VER	RO	RO	Read this register to retrieve the version information for the Chroma Key.
0x0008	LITE_MODE	RO	RO	Read this register to determine if Lite mode is on or off. This register returns 0 when Lite mode is off and 1 when on.
0x000C	DEBUG_ENABLED	RO	RO	Read this register to determine if Debug features are on. This register returns 1 if reads to other registers designated as RW return the last value the IP writes to the register.
0x0010	CONSTANT_ALPHA	RO	RO	Read this register to determine if the chroma key is operating in the constant alpha mode.
continued...				

continued...

(42) When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

Address	Register	Access		Description
		Lite ⁽⁴²⁾	Full	
0x0014	CONSTANT_ALPHA_TAG	RO	RO	Read this register for the value of alpha appended to pixels when operating in the constant alpha mode.
0x0018	BITS_PER_SYMBOL	RO	RO	Read this register for the number of bits that represent each color plane.
0x001C	NUM_COLOR_PLANES	RO	RO	Read this register for the number of color planes.
0x0020	PIXELS_IN_PARALLEL	RO	RO	Read this register for the number of pixels processed each clock cycle.
Control and debug registers				
For more details about these registers, refer to <i>Control Packets</i>				
0x0120	IMG_INFO_WIDTH	RW	RO	For lite variants, use this register to set the expected width of the incoming video fields. For full variants, read this register for the received width the IP derives from the image information packets.
0x0124	IMG_INFO_HEIGHT	RW	RO	For lite variants, use this register to set the expected height of the incoming video fields. For full variants, read this register for the received height the IP derives from the image information packets.
0x0128	IMG_INFO_INTERLACE	RW	RO	For lite variants, use this register to set the expected interlace information of the incoming video fields. For full variants, read this register for the received interlace information the IP derives from the image information packets.
0x012C	RESERVED	RW	RO	Unused
0x0130	IMG_INFO_COLORSPACE	RW	RO	For lite variants, use this register to set the expected color space of the incoming video fields. For full variants, read this register for the received color space the IP derives from the image information packets.
0x0134	IMG_INFO_SUBSAMPLING	RW	RO	For lite variants, use this register to set the expected chroma subsampling of the incoming video fields. For full variants, read this register for the received chroma subsampling the IP derives from the image information packets.
0x0138	IMG_INFO_COSITING	RW	RO	For lite variants, use this register to set the expected chroma co-siting of the incoming video fields. For full variants, read this register for the received chroma co-siting the IP derives from the image information packets.
0x013C	IMG_INFO_FIELD_COUNT	-	RO	The received field count field in image information packets.
0x0140	INPUT_STATUS	RO	RO	Bit 0: Status bit. 1 means Chroma Key is processing a video field, 0 otherwise. When you turn off Lite mode: Bit 1: Pending register updates bit. Any writes to the output sampling register (0x0148) cause the IP to raise the pending register updates bit, to indicate outstanding changes to the chroma key settings. The IP lowers this bit at the next field boundary after a write to the COMMIT register.
continued...				

⁽⁴²⁾ When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

Address	Register	Access		Description
		Lite ⁽⁴²⁾	Full	
0x0144	Commit	-	RW	Only when you turn off lite mode. The IP holds any changes to the chroma key settings via the register map until you issue a write to this register. The value you write is unimportant.
0x0148	Matched_Alpha	RW	RW	The alpha value appended to the pixels which meet the conditions specified using the upper and lower bounds registers.
0x014C	Unmatched_Alpha	RW	RW	The alpha value appended to the pixels which do not meet the conditions specified using the upper and lower bounds registers.
0x0150	C1_Upper_Bound	RW	RW	The upper bound of the condition for the most significant color plane.
0x0154	C2_Upper_Bound	RW	RW	The upper bound of the condition for the center color plane.
0x0158	C3_Upper_Bound	RW	RW	The upper bound of the condition for the least significant color plane. This bound is for the condition for single color plane configurations.
0x015C	C1_Lower_Bound	RW	RW	The lower bound of the condition for the most significant color plane.
0x0160	C2_Lower_Bound	RW	RW	The lower bound of the condition for the center color plane.
0x0164	C3_Lower_Bound	RW	RW	The lower bound of the condition for the least significant color plane. This bound is for the condition for single color plane configurations.
0x0168	C1_Replace	RW	RW	The replacement value for the most significant color plane if the pixel's condition is met.
0x016C	C2_Replace	RW	RW	The replacement value for the center color plane if the pixel's condition is met.
0x0170	C3_Replace	RW	RW	The replacement value for the least significant color plane if the pixel's condition is met. This bound is for the replacement of single color plane configurations.
0x0174	Compare_Enable	RW	RW	The control bits to enable or disable comparison of the color components when operating with conditional alpha and pixel replacement. Bit 0: enables comparison of the most significant component (C1). It is also the comparison bit for single color plane configurations Bit 1: enables comparison of the middle component (C2). Bit 2: enables comparison of the least significant component (C3).
0x0178	Replace_Enable	RW	RW	The control bits to enable or disable replacement of the color components when operating with conditional alpha and pixel replacement. Bit 0: enables replacement of the most significant component (C1). It is also the replacement field for single color plane configurations. Bit 1: enables replacement of the middle component (C2). Bit 2: enables replacement of the least significant component (C3).

⁽⁴²⁾ When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

Register Bit Descriptions

Table 190. PROD_ID

Name	Bits	Description
Chroma Key product ID	31:0	This register always returns 0x6AF7_023F.

Table 191. VER

Name	Bits	Description
Register map version	7:0	Register map version. Returns 0x01.
Unused	15:8	Unused. Returns 0x00
QPDS minor revision	23:16	Updated for each release. For 21.4, returns 0x04
QPDS major revision	31:24	Updated for each release. For 21.4, returns 0x15.

Table 192. LITE_MODE

Name	Bits	Description
Lite mode parameterization bit	0	Returns 1 if you turn on lite mode .
Unused	31:1	Unused.

Table 193. DEBUG_ENABLED

Name	Bits	Description
Debug features parameterization bit	0	Returns 1 if you turn on Debug features .
Unused	31:1	Unused.

Table 194. CONSTANT_ALPHA

Name	Bits	Description
Constant Alpha parameterization bit	0	Returns 1 if you turn on Constant Alpha .
Unused	31:1	Unused.

Table 195. CONSTANT_ALPHA_TAG

Name	Bits	Description
Constant Alpha Tag Value	15:0	This register returns the constant value being appended to each pixel when Constant Alpha mode is on.
Unused	31:16	Unused.

Table 196. BITS_PER_SYMBOL

Name	Bits	Description
BPS Value	31:0	This register returns the number of bits per color plane (symbol).

Table 197. NUM_COLOR_PLANES

Name	Bits	Description
Number of Color Planes	31:0	This register returns the number of color planes you parameterize the IP for.

Table 198. PIXELS_IN_PARALLEL

Name	Bits	Description
Pixels in Parallel	31:0	This register returns the number of pixels processed each clock cycle.

Table 199. IMG_INFO_WIDTH

Name	Bits	Description
Width bits	15:0	When you turn on lite mode , write to this register to set the expected width of the incoming video fields. When you turn off lite mode and turn on Debug features , this register returns the <code>width-1</code> field from the most recently received image information packet and adds 1 to return a value for <code>width</code> .
unused	31:16	Unused.

Table 200. IMG_INFO_HEIGHT

Name	Bits	Description
Height bits	15:0	When you turn on lite mode , write to this register to set the expected height of the incoming video fields. When you turn off lite mode and turn on Debug features , this register returns the <code>height-1</code> field from the most recently received image information packet and adds 1 to return a value for <code>height</code> .
unused	31:16	Unused.

Table 201. IMG_INFO_INTERLACE

Name	Bits	Description
InterlaceNibble bits	3:0	When you turn on lite mode , write to this register to set the expected interlacing of the incoming video fields. When you turn off lite mode and turn on Debug features , this register returns the <code>intlaceNibble</code> field from the most recently received image information packet.
unused	31:4	Unused.

Table 202. IMG_INFO_COLORSPACE

Name	Bits	Description
CSPcode bits	6:0	When you turn on lite mode , write to this register to set the expected color space of the incoming video fields. When you turn off lite mode and turn on Debug features , this register returns the 7 bit <code>CSP</code> field from the most recently received image information packet.
unused	31:7	Unused.

Table 203. IMG_INFO_SUBSAMPLING

Name	Bits	Description
CSPSubScode bits	1:0	When you turn on lite mode , write to this register to set the expected chroma subsampling of the incoming video fields. When you turn off lite mode and turn on Debug features , this register returns the <code>SUBSA</code> field from the most recently received image information packet.
unused	31:2	Unused.

Table 204. IMG_INFO_COSITING

Name	Bits	Description
Cositecode bits	1:0	When you turn on lite mode , write to this register to set the expected chroma co-siting of the incoming video fields. When you turn off lite mode and turn on Debug features , this register returns the COSITE field from the most recently received image information packet.
Unused	31:2	Unused

Table 205. IMG_INFO_FIELD_COUNT

Name	Bits	Description
Countbits	6:0	When you turn on lite mode , this register has no function. When you turn off lite mode and turn on Debug features , this register returns the 7 bit FIELD_COUNT field from the most recently received image information packet.
Unused	31:7	Unused

Table 206. INPUT_STATUS

Name	Bits	Description
Status bit	0	1 means the chroma key is receiving and processing a video field, 0 otherwise.
Pending register updates bit	1	1 means the configurable chroma key registers have pending updates, 0 otherwise.
Unused	31:2	Unused

Table 207. COMMIT

Name	Bits	Description
Commit	31:0	Write to any bits to trigger a commit.

Table 208. MATCHED_ALPHA

Name	Bits	Description
Alpha Value	15:0	The alpha value appended to the pixels that meets the conditions specified using the upper and lower bounds registers.
Unused	31:16	Unused.

Table 209. UNMATCHED_ALPHA

Name	Bits	Description
Alpha Value	15:0	The alpha value appended to the pixels that does not meet the conditions specified using the upper and lower bounds registers.
Unused	31:16	Unused.

Table 210. C1_UPPER_BOUND

Name	Bits	Description
Upper Bound	15:0	The upper bound of the comparison range of the most significant color plane used in alpha matching and value replacement, if enabled.
Unused	31:16	Unused.

Table 211. C2_UPPER_BOUND

Name	Bits	Description
Upper Bound	15:0	The upper bound of the comparison range of the center color plane used in alpha matching and value replacement, if enabled.
Unused	31:16	Unused.

Table 212. C3_UPPER_BOUND

Name	Bits	Description
Upper Bound	15:0	The upper bound of the comparison range of the least significant color plane used in alpha matching and value replacement, if enabled.
Unused	31:16	Unused.

Table 213. C1_LOWER_BOUND

Name	Bits	Description
Lower Bound	15:0	The lower bound of the comparison range of the most significant color plane used in alpha matching and value replacement, if enabled.
Unused	31:16	Unused.

Table 214. C2_LOWER_BOUND

Name	Bits	Description
Lower Bound	15:0	The lower bound of the comparison range of the center color plane used in alpha matching and value replacement, if enabled.
Unused	31:16	Unused.

Table 215. C3_LOWER_BOUND

Name	Bits	Description
Lower Bound	15:0	The lower bound of the comparison range of the least significant color plane used in alpha matching and value replacement, if enabled.
Unused	31:16	Unused.

Table 216. C1_REPLACEMENT

Name	Bits	Description
Replacement	15:0	The replacement value of the most significant color plane if enabled and the pixel lays within the range.
Unused	31:16	Unused.

Table 217. C2_REPLACEMENT

Name	Bits	Description
Replacement	15:0	The replacement value of the center color plane if enabled and the pixel lays within the range.
Unused	31:16	Unused.

Table 218. C3_REPLACEMENT

Name	Bits	Description
Replacement	15:0	The replacement value of the least significant color plane if enabled and the pixel lays within the range.
Unused	31:16	Unused.

Table 219. COMPARE_ENABLE

Name	Bits	Description
C1 Compare Enable	0	1= Enabled in pixel comparison, 0 = Disabled in pixel comparison.
C2 Compare Enable	1	1= Enabled in pixel comparison, 0 = Disabled in pixel comparison.
C3 Compare Enable	2	1= Enabled in pixel comparison, 0 = Disabled in pixel comparison.
Unused	31:3	Unused.

Table 220. REPLACE_ENABLE

Name	Bits	Description
C1 Replace Enable	0	1= Enabled in color plane replacement, 0 = Disabled in color plane replacement.
C2 Replace Enable	1	1= Enabled in color plane replacement, 0 = Disabled in color plane replacement.
C3 Replace Enable	2	1= Enabled in color plane replacement, 0 = Disabled in color plane replacement.
Unused	31:3	Unused.

16.6. Chroma Key IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_chroma_key_regs.h`

Include file: `intel_vvp_chroma_key.h`

Table 221. Chroma Key IP Software API

Name	Description
<code>intel_vvp_chroma_key_init</code>	Initialize the Chroma Key instance
<code>intel_vvp_core_*</code>	Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i> . Writable when Lite mode is on. Readable when Lite mode is off and Debug features is on.
<code>intel_vvp_chroma_key_get_lite_mode</code>	Returns if Lite mode is on
<code>intel_vvp_chroma_key_get_debug_enabled</code>	Returns if Debug features is on
<code>intel_vvp_chroma_key_get_constant_alpha_mode</code>	Returns if Constant alpha mode is on
<code>intel_vvp_chroma_key_get_constant_alpha_value</code>	Returns the constant alpha value if Constant alpha mode is on
continued...	

Name	Description
intel_vvp_chroma_key_get_bits_per_sample	Returns the bits per color sample value
intel_vvp_chroma_key_get_num_planes	Returns the number of color planes
intel_vvp_chroma_key_is_running	Returns if the IP is currently processing video data
intel_vvp_chroma_key_get_commit_status	Returns if IP has uncommitted writes
intel_vvp_chroma_key_get_status	Reads the status register
intel_vvp_chroma_key_set_matched_alpha	Sets the alpha value for matched pixels
intel_vvp_chroma_key_set_unmatched_alpha	Sets the alpha value for unmatched pixels
intel_vvp_chroma_key_enable_c1_matching	Enables the first color plane when checking for matches
intel_vvp_chroma_key_disable_c1_matching	Disables the first color plane when checking for matches
intel_vvp_chroma_key_enable_c2_matching	Enables the second color plane when checking for matches
intel_vvp_chroma_key_disable_c2_matching	Disables the second color plane when checking for matches
intel_vvp_chroma_key_enable_c3_matching	Enables the third color plane when checking for matches
intel_vvp_chroma_key_disable_c3_matching	Disables the third color plane when checking for matches
intel_vvp_chroma_key_enable_c1_replacement	Enables the swapping of the first color plane sample in case of a match
intel_vvp_chroma_key_disable_c1_replacement	Disables the swapping of the first color plane sample in case of a match
intel_vvp_chroma_key_enable_c2_replacement	Enables the swapping of the second color plane sample in case of a match
intel_vvp_chroma_key_disable_c2_replacement	Disables the swapping of the second color plane sample in case of a match
intel_vvp_chroma_key_enable_c3_replacement	Enables the swapping of the third color plane sample in case of a match
intel_vvp_chroma_key_disable_c3_replacement	Disables the swapping of the third color plane sample in case of a match
intel_vvp_chroma_key_commit_writes	Commit all outstanding writes

intel_vvp_chroma_key_init

Prototype

```
int intel_vvp_chroma_key_init(intel_vvp_chroma_key_instance
*instance, intel_vvp_core_base base);
```

Description

Initializes an Chroma Key instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

Arguments

instance - pointer to the intel_vvp_chroma_key_instance software driver instance structure

base - base address of the register map

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product_id does not match with the Chroma Key product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpChroma KeyRegMapVersionErr if the register map is not supported

intel_vvp_chroma_key_get_lite_mode

Prototype

```
bool
intel_vvp_chroma_key_get_lite_mode(intel_vvp_chroma_key_instance
*instance);
```

Description Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_chroma_key_instance fully initialized

Arguments instance - pointer to the intel_vvp_chroma_key_instance software driver instance structure

Returns true if the IP is parameterized in lite mode

intel_vvp_chroma_key_get_debug_enabled

Prototype

```
bool
intel_vvp_chroma_key_get_debug_enabled(intel_vvp_chroma_key_instance
*instance);
```

Description Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_chroma_key_instance fully initialized.

Arguments instance - pointer to the intel_vvp_chroma_key_instance software driver instance structure

Returns true if the IP is parameterized with debug features enabled

intel_vvp_chroma_key_get_constant_alpha_mode

Prototype

```
bool
intel_vvp_chroma_key_get_constant_alpha_mode(intel_vvp_chroma_key_instance
*instance);
```


<i>Description</i>	Returns the value of the <code>CONSTANT_ALPHA_MODE</code> register. The instance must be a valid <code>intel_vvp_chroma_key_instance</code> fully initialized.
<i>Arguments</i>	instance - pointer to the <code>intel_vvp_chroma_key_instance</code> software driver instance structure
<i>Returns</i>	true if the IP is parameterized to add a constant alpha value to the video stream

`intel_vvp_chroma_key_get_constant_alpha_value`

<i>Prototype</i>	<pre>uint16_t intel_vvp_chroma_key_get_constant_alpha_value(intel_vvp_chroma_key_instance *instance);</pre>
<i>Description</i>	Returns the value of the <code>CONSTANT_ALPHA_VALUE</code> register. The instance must be a valid <code>intel_vvp_chroma_key_instance</code> fully initialized and parameterized for constant alpha mode
<i>Arguments</i>	instance - pointer to the <code>intel_vvp_chroma_key_instance</code> software driver instance structure
<i>Returns</i>	The constant alpha value if the IP is parameterized with constant alpha mode

`intel_vvp_chroma_key_get_bits_per_sample`

<i>Prototype</i>	<pre>uint8_t intel_vvp_chroma_key_get_bits_per_sample(intel_vvp_chroma_key_instance *instance);</pre>
<i>Description</i>	Returns the value of the <code>BITS_PER_SAMPLES</code> parameter used to configure the IP. The instance must be a valid <code>intel_vvp_chroma_key_instance</code> fully initialized.
<i>Arguments</i>	instance - pointer to the <code>intel_vvp_chroma_key_instance</code> software driver instance structure
<i>Returns</i>	the number of bits per color sample

`intel_vvp_chroma_key_get_num_planes`

<i>Prototype</i>	<pre>uint8_t intel_vvp_chroma_key_get_num_planes(intel_vvp_chroma_key_instance *instance);</pre>
<i>Description</i>	Returns the value of the <code>NUMBER_OF_COLOR_PLANES</code> parameter used to configure the IP. The instance must be a valid <code>intel_vvp_chroma_key_instance</code> fully initialized.

Arguments instance - pointer to the `intel_vvp_chroma_key_instance` software driver instance structure

Returns the number of color planes per pixel

`intel_vvp_chroma_key_is_running`

Prototype

```
bool
intel_vvp_chroma_key_is_running(intel_vvp_chroma_key_instance*
instance);
```

Description Reads and returns the running bit of the `STATUS` register. The instance must be a valid `intel_vvp_chroma_key_instance` fully initialized.

Arguments instance - pointer to the `intel_vvp_chroma_key_instance` software driver instance structure

Returns True is the IP is currently receiving/storing an input field, false when between two input fields

`intel_vvp_chroma_key_get_commit_status`

Prototype

```
bool
intel_vvp_chroma_key_get_commit_status(intel_vvp_chroma_key_instance*
instance);
```

Description Reads and returns the commit pending bit of the `STATUS` register. The instance must be a valid `intel_vvp_chroma_key_instance` fully initialized and configured in full mode.

Arguments instance - pointer to the `intel_vvp_chroma_key_instance` software driver instance structure

Returns True is the IP is configured in full mode and if the agent interface contains uncommitted writes.

`intel_vvp_chroma_key_get_status`

Prototype

```
uint32_t
intel_vvp_chroma_key_get_status(intel_vvp_chroma_key_instance*
instance);
```

Description Reads the `STATUS` register. The instance must be a valid `intel_vvp_chroma_key_instance` fully initialized.

Arguments instance - pointer to the `intel_vvp_chroma_key_instance` software driver instance structure

Returns The content of the STATUS register

`intel_vvp_chroma_key_set_matched_alpha`

Prototype

```
int  
intel_vvp_chroma_key_set_matched_alpha(intel_vvp_chroma_key_instance  
*instance, uint16_t matched_alpha);
```

Description Sets the alpha value for matched pixels. The instance must be a valid `intel_vvp_chroma_key_instance` fully initialized and not parameterized for constant alpha mode.

Arguments instance - pointer to the `intel_vvp_chroma_key_instance` software driver instance structure

matched_alpha - the alpha value for matched pixels in the range [0..bits_per_sample-1]

Returns kIntelVvpCoreOk for success, negative error code in case of failures
kIntelVvpCoreInstanceErr if the instance is invalid or parameterized for constant alpha only
kIntelVvpChromaKeyValueErr if the matched alpha is invalid

`intel_vvp_chroma_key_set_unmatched_alpha`

Prototype

```
int  
intel_vvp_chroma_key_set_unmatched_alpha(intel_vvp_chroma_key_instance  
*instance, uint16_t unmatched_alpha);
```

Description Sets the alpha value for unmatched pixels. The instance must be a valid `intel_vvp_chroma_key_instance` fully initialized and not parameterized for constant alpha mode.

Arguments instance - pointer to the `intel_vvp_chroma_key_instance` software driver instance structure

unmatched_alpha - the alpha value for unmatched pixels in the range [0..bits_per_sample-1]

Returns kIntelVvpCoreOk for success, negative error code in case of failures
kIntelVvpCoreInstanceErr if the instance is invalid
kIntelVvpChromaKeyValueErr if the unmatched alpha is invalid

intel_vvp_chroma_key_enable_c1_matching

Prototype

```
int
intel_vvp_chroma_key_enable_c1_matching(intel_vvp_chroma_key_instance
*instance, uint16_t c1_lower_bound, uint16_t c1_upper_bound);
```

Description Enables pixel matching with the first color sample value

Arguments instance - pointer to the intel_vvp_chroma_key_instance software driver instance structure

c1_lower_bound - the lowest sample value, in the range [0..bits_per_sample-1], to match a color sample

c1_upper_bound - the highest sample value, in the range [0..bits_per_sample-1], to match a color sample

Returns kIntelVvpCoreOk for success, negative error code in case of failures

kIntelVvpCoreInstanceErr if the instance is invalid or parameterized for constant alpha output

kIntelVvpChromaKeyValueErr if the lower bound or upper bound are invalid

intel_vvp_chroma_key_disable_c1_matching

Prototype

```
int
intel_vvp_chroma_key_disable_c1_matching(intel_vvp_chroma_key_instance*
instance);
```

Description Disables pixel matching with the first color sample value

Arguments instance - pointer to the intel_vvp_chroma_key_instance software driver instance structure

Returns kIntelVvpCoreOk for success, negative error code in case of failures

kIntelVvpCoreInstanceErr if the instance is invalid or parameterized for constant alpha output

intel_vvp_chroma_key_enable_c2_matching

Prototype

```
int
intel_vvp_chroma_key_enable_c1_matching(intel_vvp_chroma_key_instance
*instance, uint16_t c2_lower_bound, uint16_t c2_upper_bound);
```

Description Enables pixel matching with the second color sample value

Arguments instance - pointer to the intel_vvp_chroma_key_instance software driver instance structure

c2_lower_bound - the lowest sample value, in the range [0..bits_per_sample-1], to match a color sample

c2_upper_bound - the highest sample value, in the range [0..bits_per_sample-1], to match a color sample

Returns kIntelVvpCoreOk for success, negative error code in case of failures

kIntelVvpCoreInstanceErr if the instance is invalid or parameterized for constant alpha output or with less than two color planes

kIntelVvpChromaKeyValueErr if the lower bound or upper bound are invalid

intel_vvp_chroma_key_disable_c2_matching

Prototype

```
int  
intel_vvp_chroma_key_disable_c2_matching(intel_vvp_chroma_key_instance*  
instance);
```

Description Disables pixel matching with the second color sample value

Arguments instance - pointer to the intel_vvp_chroma_key_instance software driver instance structure

Returns kIntelVvpCoreOk for success, negative error code in case of failures

kIntelVvpCoreInstanceErr if the instance is invalid or parameterized for constant alpha output or with less than two color planes

intel_vvp_chroma_key_enable_c3_matching

Prototype

```
int  
intel_vvp_chroma_key_enable_c3_matching(intel_vvp_chroma_key_instance  
*instance, uint16_t c3_lower_bound, uint16_t c3_upper_bound);
```

Description Enables pixel matching with the third color sample value

Arguments instance - pointer to the intel_vvp_chroma_key_instance software driver instance structure

c3_lower_bound - the lowest sample value, in the range [0..bits_per_sample-1], to match a color sample

c3_upper_bound - the highest sample value, in the range [0..bits_per_sample-1], to match a color sample

Returns kIntelVvpCoreOk for success, negative error code in case of failures

kIntelVvpCoreInstanceErr if the instance is invalid or parameterized for constant alpha output or with less than three color planes

kIntelVvpChromaKeyValueErr if the lower bound or upper bound are invalid

intel_vvp_chroma_key_disable_c3_matching

Prototype

```
int
intel_vvp_chroma_key_disable_c3_matching(intel_vvp_chroma_key_instance*
instance);
```

Description Disables pixel matching with the third color sample value

Arguments instance - pointer to the intel_vvp_chroma_key_instance software driver instance structure

Returns kIntelVvpCoreOk for success, negative error code in case of failures
kIntelVvpCoreInstanceErr if the instance is invalid or parameterized for constant alpha output or with less than three color planes

intel_vvp_chroma_key_enable_c1_replacement

Prototype

```
int
intel_vvp_chroma_key_enable_c1_replacement(intel_vvp_chroma_key_instance
*instance, uint16_t c1_replacement)
```

Description Swaps the first color plane sample value when a pixel is matched

Arguments instance - pointer to the intel_vvp_chroma_key_instance software driver instance structure

c1_replacement - the replacement sample value, in the range [0..bits_per_sample-1], when a pixel was matched

Returns kIntelVvpCoreOk for success, negative error code in case of failures
kIntelVvpCoreInstanceErr if the instance is invalid or parameterized for constant alpha output
kIntelVvpChromaKeyValueErr if the replacement value is invalid

intel_vvp_chroma_key_disable_c1_replacement

Prototype

```
int
intel_vvp_chroma_key_disable_c1_replacement(intel_vvp_chroma_key_instance
*instance)
```

Description Cancels the swap of the first color plane sample value when a pixel is matched

Arguments instance - pointer to the intel_vvp_chroma_key_instance software driver instance structure

Returns kIntelVvpCoreOk for success, negative error code in case of failures
kIntelVvpCoreInstanceErr if the instance is invalid or parameterized for constant alpha output

intel_vvp_chroma_key_enable_c2_replacement

Prototype

```
int
intel_vvp_chroma_key_enable_c2_replacement(intel_vvp_chroma_key_instance
*instance, uint16_t c2_replacement)
```

Description Swaps the second color plane sample value when a pixel is matched

Arguments instance - pointer to the intel_vvp_chroma_key_instance software driver instance structure

c2_replacement - the replacement sample value, in the range [0..bits_per_sample-1], when a pixel was matched

Returns kIntelVvpCoreOk for success, negative error code in case of failures
kIntelVvpCoreInstanceErr if the instance is invalid or parameterized for constant alpha output or with less than two color planes
kIntelVvpChromaKeyValueChanged if the replacement value is invalid

intel_vvp_chroma_key_disable_c2_replacement

Prototype

```
int
intel_vvp_chroma_key_disable_c2_replacement(intel_vvp_chroma_key_instance
*instance)
```

Description Cancels the swap of the second color plane sample value when a pixel is matched

Arguments instance - pointer to the intel_vvp_chroma_key_instance software driver instance structure

Returns kIntelVvpCoreOk for success, negative error code in case of failures
kIntelVvpCoreInstanceErr if the instance is invalid or parameterized for constant alpha output or with less than two color planes

intel_vvp_chroma_key_enable_c3_replacement

Prototype

```
int
intel_vvp_chroma_key_enable_c3_replacement(intel_vvp_chroma_key_instance
*instance, uint16_t c3_replacement)
```

Description Swaps the third color plane sample value when a pixel is matched

Arguments instance - pointer to the `intel_vvp_chroma_key_instance` software driver instance structure

c3_replacement - the replacement sample value, in the range [0..bits_per_sample-1], when a pixel was matched

Returns kIntelVvpCoreOk for success, negative error code in case of failures

kIntelVvpCoreInstanceErr if the instance is invalid or parameterized for constant alpha output or with less than three color planes

kIntelVvpChromaKeyValueErr if the replacement value is invalid

`intel_vvp_chroma_key_disable_c3_replacement`

Prototype

```
int
intel_vvp_chroma_key_disable_c3_replacement(intel_vvp_chroma_key_instance*instance)
```

Description Cancels the swap of the third color plane sample value when a pixel is matched

Arguments instance - pointer to the `intel_vvp_chroma_key_instance` software driver instance structure

Returns kIntelVvpCoreOk for success, negative error code in case of failures

kIntelVvpCoreInstanceErr if the instance is invalid or parameterized for constant alpha output or with less than three color planes

`intel_vvp_chroma_key_commit_writes`

Prototype

```
int
intel_vvp_chroma_key_commit_writes(intel_vvp_chroma_key_instance*instance)
```

Description Commits all pending writes before processing the next field. The instance must be valid and parameterized in full mode.

Arguments instance - pointer to the `intel_vvp_chroma_key_instance` software driver instance structure

Returns kIntelVvpCoreOk (0) for success, negative error code otherwise

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

17. Chroma Resampler IP

17.1. About the Chroma Resampler IP

The chroma resampler IP converts between the different chroma sampling formats available in the YCbCr color space. The IP supports 4:4:4, 4:2:2 and 4:2:0 sampled formats at both the input and output streaming interfaces, and the user may enable support for the conversions required for all combinations of input and output sampling.

The human eye has higher visual acuity (resolution) for brightness than for tone (color). Color spaces that separate the luma (brightness) information from the chroma (color), such as YCbCr, can reduce the resolution at which the chroma data samples. These color spaces can reduce the bandwidth to transmit and store the data and the FPGA resources to process it.

4:4:4 sampling has the same number of samples on the luma (Y) and 2 chroma planes (Cb and Cr). For each Y sample there is one Cb sample and one Cr sample.

4:2:2 sampling reduces the Cb and Cr sampling by a factor of 2 in the horizontal direction. For each pair of Y samples there is now a single Cb sample and a single Cr sample.

4:2:0 sampling reduces the Cb and Cr sampling by a factor of 2 in both the horizontal and vertical directions. For each 2x2 group of 4 Y samples there is now a single Cb sample and a single Cr sample.

17.1.1. Chroma Resampler Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family and the number of supported bits per color sample and pixels in parallel.

Table 222. Chroma Resampler Performance and Resources

The table shows ALM usage and f_{MAX} for a design with 10 bits per color sample, 6 pixels in parallel, bilinear algorithm and all sampling conversions on.

Target device	ALM	M20K	f_{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	5,350	132	741
Arria 10 speed grade 1 (10AS066H1F34E1HG)	4,940	132	461
Cyclone 10 GX speed grade 5 (10CX220YF78015G)	5,217	132	459
Stratix 10 GX speed grade 1 (1SX280LN2F43E1VG)	5,756	132	625

17.2. Chroma Resampler IP Parameters

The IP offers compile- and run-time parameters.

Parameter	Value	Description
Video data format		
Lite mode	On or off	Turn on to use the Lite variant of the Altera Streaming Video Protocol.
Bits per color sample	8 to 16	Select the number of bits per color sample.
Input pixels in parallel	1, 2, 4, 6, 8	Select the number of pixels transmitted every clock cycle at the input interface.
Output pixels in parallel	1, 2, 4, 6, 8	Select the number of pixels transmitted every clock cycle at the output interface.
Disable flush/fill between frames	On or off	Turn on to turn off the flush and refill of the line buffer between frames.
Maximum field width	1 to 16384	Set the maximum supported field width.
Chroma sampling support		
4:2:0 passthrough	On or off	Turn on for passthrough of 4:2:0 chroma sampled fields.
4:2:0 to 4:2:2 conversion	On or off	Turn on for conversion from 4:2:0 chroma sampling at the input to 4:2:2 chroma sampling at the output.
4:2:0 to 4:4:4 conversion	On or off	Turn on for conversion from 4:2:0 chroma sampling at the input to 4:4:4 chroma sampling at the output.
4:2:2 passthrough	On or off	Turn on for passthrough of 4:2:2 chroma sampled fields.
4:2:2 to 4:2:0 conversion	On or off	Turn on for conversion from 4:2:2 chroma sampling at the input to 4:2:0 chroma sampling at the output.
4:2:2 to 4:4:4 conversion	On or off	Turn on for conversion from 4:2:2 chroma sampling at the input to 4:4:4 chroma sampling at the output.
4:4:4 passthrough	On or off	Turn on for passthrough of 4:4:4 chroma sampled fields.
4:4:4 to 4:2:0 conversion	On or off	Turn on for conversion from 4:4:4 chroma sampling at the input to 4:2:0 chroma sampling at the output.
4:4:4 to 4:2:2 conversion	On or off	Turn on for conversion from 4:4:4 chroma sampling at the input to 4:2:2 chroma sampling at the output.
Horizontal resampling settings		
Horizontal resampling algorithm	Nearest Neighbor, Bilinear, Filtered	Select which resampling algorithm to use for all horizontal chroma resampling operations.
Horizontal chroma siting	Left, Center	Select the expected horizontal chroma siting for fields received at the input interface.
Horizontal luma adaptive resampling	On or off	Turn on for the luma adaptive algorithm for horizontal resampling conversions. Only if you select Filtered .
Vertical resampling settings		
Vertical resampling algorithm	Nearest Neighbor, Bilinear, Filtered	Select which resampling algorithm to use for all vertical chroma resampling operations.
Vertical chroma siting	Top, Center	Select the expected vertical chroma siting for fields received at the input interface
Vertical luma adaptive resampling	On or off	Turn on for the luma adaptive algorithm for vertical resampling conversions. Only if you select Filtered .
Control settings		
<i>continued...</i>		

Parameter	Value	Description
Memory-mapped control interface	On or off	Turn on to for the Avalon memory-mapped control agent interface and allow runtime configuration via the register map. You must have the Avalon memory-mapped control agent interface if you turn on Lite mode .
General		
Pipeline ready signals	On or off	Turn on to add extra pipeline registers to the AXI4-S tready signals.
Separate clock for control interface	On or off	Turn on for a separate clock for the control agent interface.
Debug features	On or off	Turn on to read back writeable registers via the control agent interface.

17.3. Chroma Resampler IP Functional Description

Chroma resampler algorithms

The chroma resampler implements four basic resampling operations:

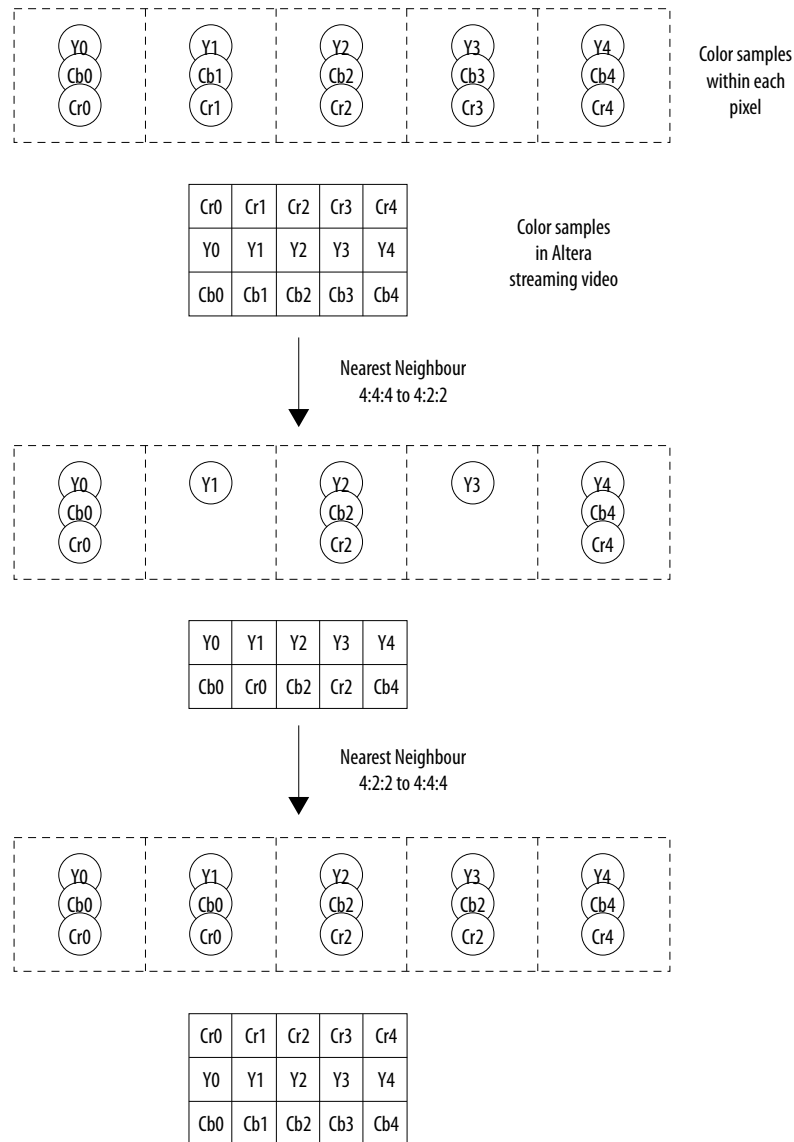
- Horizontal upsampling: 4:2:2 -> 4:4:4
- Horizontal downsampling: 4:4:4 -> 4:2:2
- Vertical upsampling: 4:2:0 -> 4:2:2
- Vertical downsampling: 4:2:2 -> 4:2:0

For conversions between 4:4:4 and 4:2:0 the relevant horizontal and vertical sampling operations chain automatically to achieve the correct end result. The chroma resampler supports three different algorithms for each resampling operation; nearest neighbor, bilinear and filtered. The three algorithms vary in the level of visual quality provided, the chroma siting, and the resources for implementation.

Nearest Neighbor

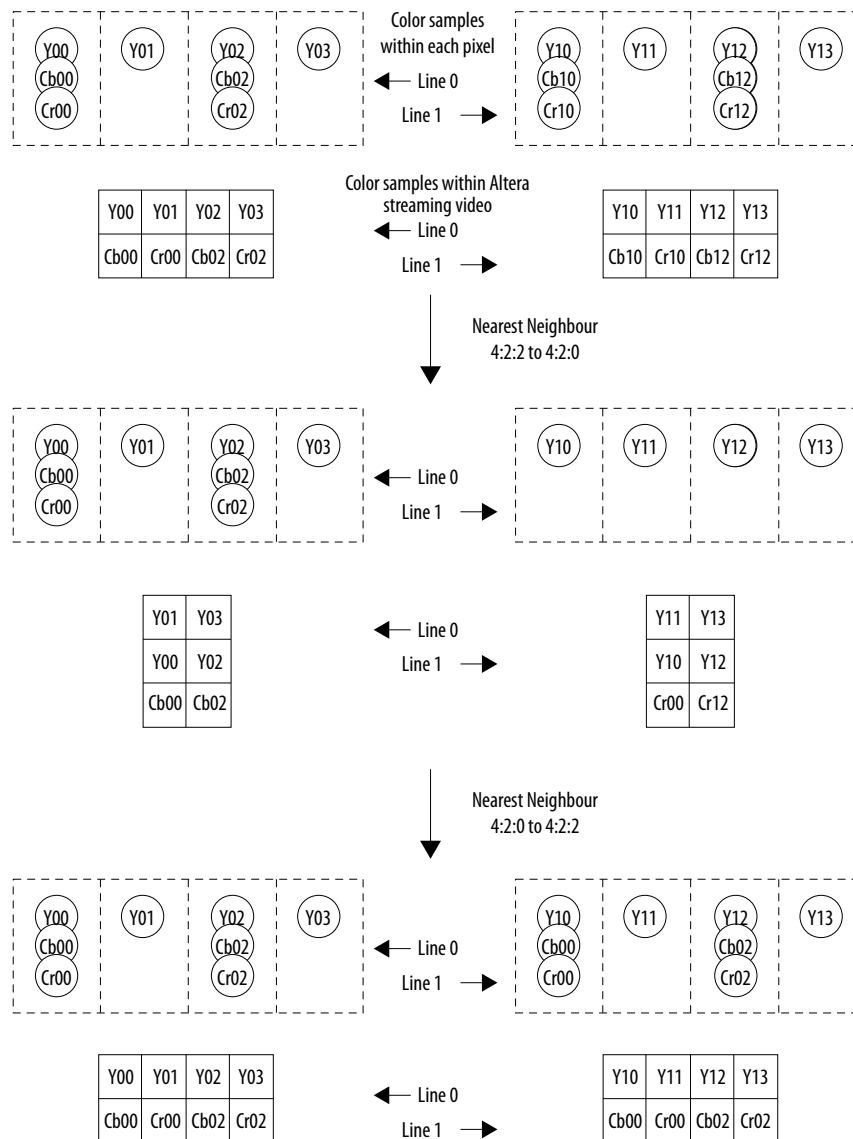
Nearest neighbor is the lowest quality resampling algorithm, with the lowest device resource usage. For horizontal downsampling (4:4:4 -> 4:2:2) it drops every other Cb and Cr sample within the video line. For horizontal upsampling (4:2:2 -> 4:4:4) it repeats each Cb and Cr sample. The nearest neighbor algorithm uses left siting (co-siting) for the 4:2:2 chroma samples – both the Cb and Cr samples from the even indexed Y samples are retained during downsampling.

Figure 40. Nearest Neighbor



The nearest neighbor algorithm works in the same way for vertical resampling. For vertical downsampling (4:2:2 -> 4:2:0) the IP discards the chroma data from every other video line. For upsampling (4:2:0 -> 4:2:2) the IP repeats the chroma data for two lines of luma data. The nearest neighbor algorithm uses top siting (cositing) for both the Cb and Cr planes. The IP preserves chroma data from lines 0, 2, 4, etc in downsampling. The IP discards the data from lines 1, 3, 5, etc.

Figure 41. Nearest Neighbor



Bilinear

The bilinear algorithm offers a midpoint between visual image quality and device resource cost. You can parameterize the bilinear algorithm to use either left or center horizontal siting of the chroma data, and top or centre vertical siting of the chroma data.

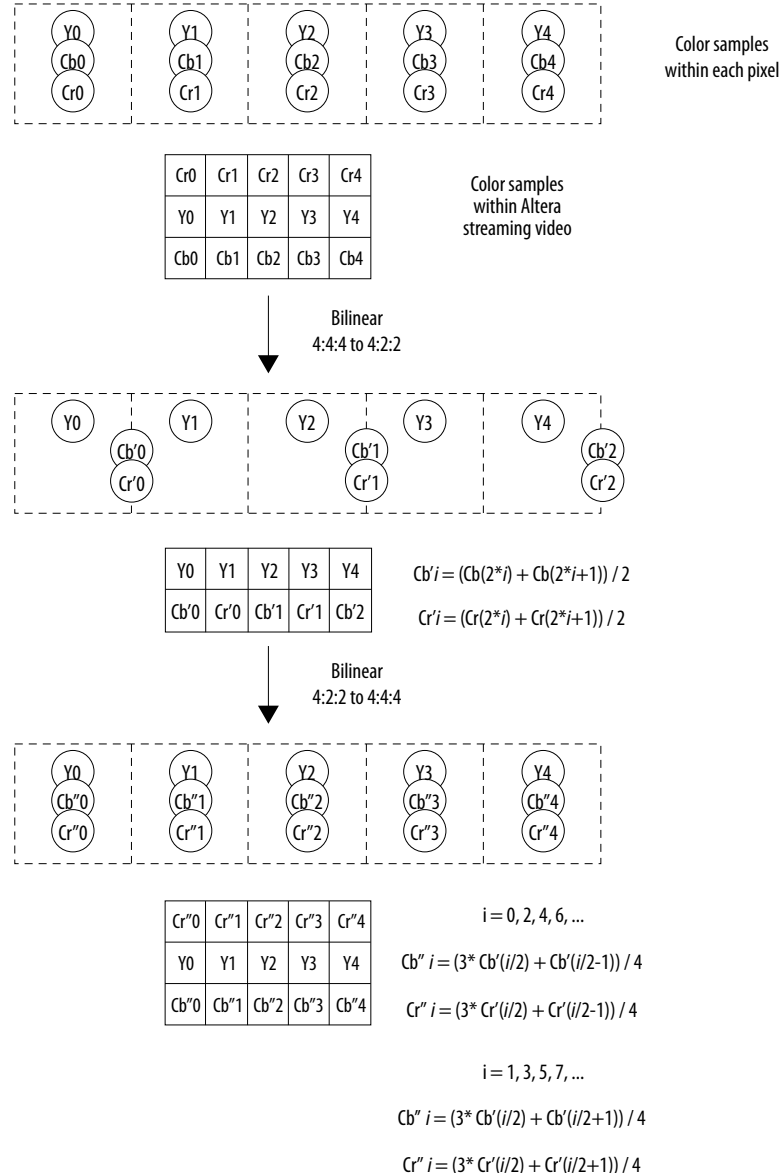
Top or left chroma siting is the simpler of the two algorithm options. Top or left downsampling using the bilinear algorithm is exactly equivalent to nearest neighbor downsampling. The IP discards the chroma data from the odd indexed pixels. Top or left upsampling recreates the chroma data for odd indexed pixels by taking the average of the chroma samples for the two neighboring even indexed pixels. The IP takes the chroma values for the even indexed pixels directly from the input values. Both the nearest neighbor algorithm and the bilinear algorithm with top or left chroma

siting are fully reversible. The upsampling algorithm creates new data without altering the values of the any the original input data. The downsampling algorithm simply discards the values that the upsampling algorithm creates.

Centre chroma sampling for the bilinear algorithm is more complex and is not reversible because the IP alters all of the chroma values during both the upsample and downsample processes.

Figure 42. Bilinear

The figure and equations show how the IP calculates the bilinear resampled chroma for horizontal upsampling (4:2:4 -> 4:4:4) and downsampling (4:4:4 -> 4:2:2) using center chroma siting.



The bilinear algorithm using center siting works in the same way for vertical upsampling (4:2:0 -> 4:2:2) and downsampling (4:2:2 -> 4:2:0).

Filtered

The filtered algorithm is the most computationally expensive and device resource intense algorithm, but it offers increased visual quality. You can parameterize the filtered algorithm to use either left or center horizontal siting of the chroma data, and top or center vertical siting of the chroma data.

For downsampling conversions (4:2:2 -> 4:2:0 and 4:4:4 -> 4:2:2) the filtered algorithm applies an 8-tap Lanczos2 resampling filter to generate the downsampled data. The IP applies different phase shifts to the Lanczos2 function when generating the coefficients for each chroma siting option. For top and left chroma siting, the IP applies a phase shift of 0 pixels to the Lanczos2 coefficients. For center chroma siting the phase shift is +0.5 pixels.

For upsampling conversions (4:2:0 -> 4:2:2 and 4:2:2 -> 4:4:4) the filtered algorithm applies a 4-tap Lanczos2 resampling filter to generate the upsampled data. For top/left chroma siting, phase shifts of 0 and 0.5 pixels are applied to the Lanczos2 coefficients for the even and odd indexed chroma samples respectively. For center chroma siting the phase shifts are -0.25 and +0.25 pixels.

You can also turn on **luma adaption** for upsampling conversions. This feature further increases device resource usage but may reduce color bleed around edges when compared to the default filtered algorithm. The feature is the only chroma resampler to implement some logic in DSP blocks. When you turn on **luma adaption**, the IP compares differences between successive luma samples to an edge threshold to detect significant edges. In areas where the IP detects edges, you can shift the phase of the Lanczos-2 filter by up to 0.25 pixels to the left or right. This shift weights the resulting chroma samples more intensely towards the more appropriate side of the edge.

Color planes per pixel

Most Video and Vision Processing IPs allow you to select the number of color planes per pixel, but the chroma resampler does not have that parameter and sets the value automatically. The number of color planes per pixel is a function of the chroma resampling conversions that you select, which determine the chroma sampling formats that are supported at the input and output interfaces. 4:4:4 and 4:2:0 chroma samplings both require three color panes per pixel. 4:2:2 only requires 2 color planes. 4:2:2 can be transmitted through a link with 3 color planes, with the planes in the most significant bits of each pixel unused.

If you select any of the following chroma sampling conversions, the input interface uses three color planes per pixel, otherwise it uses two color planes:

- 4:2:0 passthrough
- 4:2:0 to 4:4:4
- 4:2:0 to 4:2:2
- 4:4:4 passthrough
- 4:4:4 to 4:2:2
- 4:4:4 to 4:2:0

If you select any of the following chroma sampling conversions, the output interface uses three color planes per pixel, otherwise it uses two color planes:

- 4:2:0 passthrough
- 4:2:0 to 4:4:4
- 4:4:4 passthrough
- 4:4:4 to 4:2:0
- 4:2:2 to 4:4:4
- 4:2:2 to 4:2:0

17.3.1. Chroma Resampler IP Interfaces

Table 223. Chroma Resampler IP Interfaces

Name	Direction	Width	Description
Clocks and resets			
main_clock_clk	In	1	AXI4-S processing clock.
main_reset_rst	In	1	AXI4-S processing reset.
agent_clock_clk	In	1	Clock for the Avalon memory-mapped control agent interface. Only if you turn on Separate clock for control interface .
agent_reset_rst	In	1	Reset for the Avalon memory-mapped control agent interface. Only if you turn on Separate clock for control interface .
Control interfaces			
av_mm_control_agent_address	In	7	Avalon memory-mapped agent address.
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write.
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data.
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable.
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read.
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data valid.
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request .
Altera Streaming Video interfaces			
axi4s_vid_in_tdata	In		AXI4-S data in.
axi4s_vid_in_tvalid	In	1	AXI4-S data valid.
<i>continued...</i>			

(43) the equation gives all tdata widths in these interfaces:
 $\max(\text{ceil}((\text{bits per color sample} \times \text{number of color planes} \times \text{pixels in parallel}) / 8) \times 8, 16)$

Name	Direction	Width	Description
axi4s_vid_in_tuser	In		AXI4-S tuser tuser[0] indicates start of video frame when asserted. tuser[1] indicates the start of a non-video packet when asserted.
axi4s_vid_in_tlast	In	1	AXI4-S end of packet .
axi4s_vid_in_tready	Out	1	AXI4-S data ready.
axi4s_vid_out_tdata	Out	(43)	AXI4-S data in.
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid.
axi4s_vid_in_tuser	Out	(44)	AXI4-S tuser tuser[0] indicates start of video frame when asserted tuser[1] indicates the start of a non-video packet when asserted.
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet .
axi4s_vid_out_tready	In	1	AXI4-S data ready.

Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

17.4. Chroma Resampler IP Registers

Each register is either read-only (RO) or read-write (RW).

Table 224. Parameterization registers

In the software API the register names appear with a prefix of INTEL_VVP, INTEL_VVP_CORE or INTEL_VVP_CRS as appropriate and with an optional REG suffix

Address	Register	Access	Description
0x0000	VID_PID	RO	Read this register to retrieve chroma resampler product ID. This register always returns 6AF7_022E
0x0004	VERSION	RO	Read this register for the IP version information.
0x0008	LITE_MODE	RO	Read this register to determine if lite mode is on. This register returns 0 when you turn off lite and 1 when you turn on lite .
0x000C	DEBUG_ENABLED	RO	Read this register to determine if debug features is on. This register returns 1 if reads to other registers designated as RW return the last value the IP writes to the register, or an undefined value.
0x0010	ENABLE_444	RO	Read this register to determine if 4:4:4 chroma sampling is supported at the output.
<i>continued...</i>			

(44) The equation gives all tuser widths in these interfaces

$$N = \text{ceil}(\text{tdata width} / 8)$$

Address	Register	Access	Description
0x0014	ENABLE_422	RO	Read this register to determine if 4:2:2 chroma sampling is supported at the output.
0x0018	ENABLE_420	RO	Read this register to determine if 4:2:0 chroma sampling is supported at the output.
0x0060 to 0x011F	-	-	Unused.

Table 225. Control and debug registers

For more information, refer to *Control Packets*. You must turn on debug features to read the values stored in these registers. If you turn off debug features, reads to these registers return undefined data. The only exception is the STATUS register, the value of which you can always read

Address	Register	Access		Description
		Lite	Full	
0x0120	IMG_INFO_WIDTH	RW	RO	When you turn on lite , use this register to set the expected width of incoming video fields. When you turn off lite and turn on Debug features , this register returns the width that the chroma resampler derives from information in the image information packet.
0x0124	IMG_INFO_HEIGHT	RW	RO	When you turn on lite , use this register to set the expected height of incoming video fields. When you turn off lite and turn on Debug features , this register returns the height that the chroma resampler derives from information in the image information packet.
0x0128	IMG_INFO_INTERLACE	-	RO	When you turn off lite and turn on Debug features , this register returns the interlace nibble that the chroma resampler derives from information in the image information packet.
0x012C	Reserved	-	-	Reserved
0x0130	IMG_INFO_COLORSPACE	-	RO	When you turn off lite and turn on Debug features , this register returns the color space that the chroma resampler derives from information in the image information packet.
0x0134	IMG_INFO_SUBSAMPLING	RW	RO	When you turn on lite , use this register to set the expected chroma sampling of incoming video fields. When you turn off lite and turn on Debug features , this register returns the subsampling that the chroma resampler derives from information in the image information packet.
0x0138	IMG_INFO_COSITING	-	RO	When you turn off lite and turn on Debug features , this register returns the cositing that the chroma resampler derives from information in the image information packet.
0x013C	IMG_INFO_FIELD_COUNT	-	RO	When you turn off lite and turn on Debug features , this register returns the field count that the chroma resampler derives from information in the image information packet.
0x012C to 0x013C	-	-	-	Unused.
0x0140	STATUS	RO		Bit 0 : Status bit. 1 means chroma resampler is processing a video field, 0 otherwise. When you turn off Lite mode : Bit 1 : Pending register updates bit. Any writes to the output sampling register (0x0148) cause the IP to raise the pending register updates bit, to indicate outstanding changes to the resampling settings.

continued...

Address	Register	Access		Description
		Lite	Full	
				The IP lowers this bit at the next field boundary after a write to the COMMIT register.
0x0144	COMMIT	RW		Only when you turn off lite mode . The IP holds any changes to the scaling settings (not coefficients) via the register map until you issue a write to this register. The value you write is unimportant.
0x0148	OUTPUT_MODE	RW		Write to this register to set the output chroma sampling. Write 0 for 4:2:0, 2 for 4:2:2 and 3 for 4:4:4

Register Bit Descriptions

Table 226. VID_PID

Name	Bits	Description
Chroma resampler vendor ID and product ID	31:0	This register always returns 0x6AF7_022E. 15:0 is the product ID and always returns 0x022E 31:16 is the vendor ID and always returns 0x6AF7

Table 227. VERSION

Name	Bits	Description
Register map version	7:0	Register map version.
IP patch revision	15:8	-
IP update revision	23:16	Updated when the IP version changes.
IP major revision	31:24	Updated when the IP version changes.

Table 228. LITE_MODE

Name	Bits	Description
Lite mode parameterization bit	31:0	Returns 1 if you turn on lite mode and 0 otherwise.

Table 229. DEBUG_ENABLED

Name	Bits	Description
Debug features parameterization bit	31:0	Returns 1 if you turn on debug features and 0 otherwise.

Table 230. ENABLE_444

Name	Bits	Description
Enable_444_out	31:0	Returns 1 if 4:4:4 chroma sampling is supported at the output.

Table 231. ENABLE_422

Name	Bits	Description
Enable_422_out	31:0	Returns 1 if 4:2:2 chroma sampling is supported at the output.

Table 232. ENABLE_420

Name	Bits	Description
Enable_420_out	31:0	Returns 1 if 4:2:0 chroma sampling is supported at the output.

Table 233. IMG_INFO_WIDTH

Name	Bits	Description
Width bits	15:0	When you turn on lite , write to this register to set the expected width of the incoming video fields. When you turn off lite and turn on Debug features , this register reads the <code>width-1</code> field from the most recently received image information packet and adds 1 to return a value for <code>width</code> .
unused	31:16	Unused.

Table 234. IMG_INFO_HEIGHT

Name	Bits	Description
Height bits	15:0	When you turn on lite , write to this register to set the expected height of the incoming video fields. When you turn off lite and turn on Debug features , this register reads the <code>height-1</code> field from the most recently received image information packet and adds 1 to return a value for <code>height</code> .
unused	31:16	Unused.

Table 235. IMG_INFO_INTERLACE

Name	Bits	Description
IntlaceNibble bits	3:0	When you turn on lite , this register has no function. When you turn off lite and turn on Debug features , this register returns the <code>intlaceNibble</code> field from the most recently received image information packet.
unused	31:4	Unused.

Table 236. IMG_INFO_COLORSPACE

Name	Bits	Description
CSP code bits	6:0	When you turn on lite , this register has no function. When you turn off lite and turn on Debug features , this register returns the 7 bit <code>CSP</code> field from the most recently received image information packet.
unused	31:7	Unused.

Table 237. IMG_INFO_SUBSAMPLING

Name	Bits	Description
SubSa code bits	1:0	When you turn on lite , write to this register to set the expected chroma sampling of the incoming video fields. When you turn off lite and turn on Debug features , this register returns the <code>SUBSA</code> field from the most recently received image information packet.
unused	31:2	Unused.

Table 238. IMG_INFO_COSITING

Name	Bits	Description
Cosite code bits	1:0	When you turn on lite , this register has no function.
continued...		

		When you turn off lite and turn on Debug features , this register returns the <code>COSITE</code> field from the most recently received image information packet.
unused	31:2	Unused.

Table 239. IMG_INFO_FIELD_COUNT

Name	Bits	Description
Count bits	6:0	When you turn on lite , this register has no function. When you turn off lite and turn on Debug features , this register returns the 7 bit <code>FIELD_COUNT</code> field from the most recently received image information packet.
unused	31:7	Unused.

Table 240. STATUS

Name	Bits	Description
Status bit	0	1 means chroma resampler is processing a video field, 0 otherwise.
Pending register updates bit	1	1 means chroma resampler has pending updates, 0 otherwise
unused	31:2	Unused.

Table 241. COMMIT

Name	Bits	Description
unused	31:0	Unused.

Table 242. OUTPUT_MODE

Name	Bits	Description
Output subsampling	1:0	Output chroma sampling.
unused	31:2	Unused.

17.5. Chroma Resampler IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_crs_regs.h`

Include file: `intel_vvp_crs.h`

Table 243. Chroma Resampler IP API reference

Name	Description
<code>intel_vvp_crs_init</code>	Initialize the Chroma Resampler instance
<code>intel_vvp_core_*</code>	Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i> . Writable when Lite is on. Readable when Lite is off and Debug features is on.
<code>intel_vvp_crs_get_lite_mode</code>	Returns if Lite Mode is on.
continued...	

Name	Description
intel_vvp_crs_get_debug_enabled	Returns if Debug features is on.
intel_vvp_crs_is_conversion_supported	Returns if the IP supports a specified chroma resampling.
intel_vvp_crs_is_running	Returns if the IP is currently processing video data.
intel_vvp_crs_get_commit_status	Returns uncommitted writes.
intel_vvp_crs_get_status	Reads the status register.
intel_vvp_crs_set_output_subsampling	Selects the current output subsampling.
intel_vvp_crs_get_output_subsampling	Returns the current selected output subsampling.
intel_vvp_crs_commit_writes	Commit all outstanding writes.

enum eIntelCrsSubsampling

- Members**
- kIntelVvpCrsSubsampling420 (0) – 4:2:0 subsampling, chroma planes resolution is halved vertically and horizontally
 - kIntelVvpCrsSubsampling422 (2) – 4:2:2 subsampling, chroma planes resolution is halved horizontally
 - kIntelVvpCrsSubsampling444 (3) – 4:4:4 subsampling, chroma planes are fully sampled
 - kIntelVvpCrsSubsamplingInvalid (1) – Invalid subsampling (used to indicate errors)

Description Enumeration for the supported chroma subsamplings

intel_vvp_crs_init

Prototype

```
int intel_vvp_crs_init(intel_vvp_crs_instance *instance,
intel_vvp_core_base base);
```

Description Initializes a Chroma Resampler instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and should not be used further by the application if returning a non-zero error code.

Arguments

- instance – pointer to the intel_vvp_crs_instance software driver instance structure

- base – base address of the register map

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product_id does not match with the Chroma Resampler product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpCrsRegMapVersionErr if the register map is not supported

intel_vvp_crs_get_lite_mode

Prototype

```
bool intel_vvp_crs_get_lite_mode(intel_vvp_crs_instance
*instance);
```

Description

Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_crs_instance fully initialized

Arguments

instance – pointer to the intel_vvp_crs_instance software driver instance structure

Returns

true if the IP is parameterized in lite mode

intel_vvp_crs_get_debug_enabled

Prototype

```
bool intel_vvp_crs_get_debug_enabled(intel_vvp_crs_instance
*instance);
```

Description

Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_crs_instance fully initialized.

Arguments

instance – pointer to the intel_vvp_crs_instance software driver instance structure

Returns

true if the IP is parameterized with debug features enabled

intel_vvp_crs_is_conversion_supported

Prototype

```
bool intel_vvp_crs_is_conversion_supported
(intel_vvp_crs_instance *instance, eIntelCrsSubsampling
input_subsampling, eIntelCrsSubsampling output_subsampling);
```

Description

Returns the value of the MAX_WIDTH register. The instance must be a valid intel_vvp_crs_instance fully initialized.

Arguments

instance – pointer to the intel_vvp_crs_instance software driver instance structure

input_subsampling – the input subsampling

output_subsampling – the output subsampling

Returns if chroma resampling, or pass-through, from the input subsampling to the output subsampling is supported

intel_vvp_crs_is_running

Prototype

```
bool intel_vvp_crs_is_running(intel_vvp_crs_instance* instance);
```

Description Reads and returns the running bit of the STATUS register. The instance must be a valid intel_vvp_crs_instance fully initialized.

Arguments instance – pointer to the intel_vvp_crs_instance software driver instance structure

Returns True is the IP is currently receiving/storing an input field, false when between two input fields

intel_vvp_crs_get_commit_status

Prototype

```
bool intel_vvp_crs_get_commit_status(intel_vvp_crs_instance* instance);
```

Description Reads and returns the commit pending bit of the STATUS register. The instance must be a valid intel_vvp_crs_instance fully initialized and configured in full mode.

Arguments instance – pointer to the intel_vvp_crs_instance software driver instance structure

Returns True is the IP is configured in full mode and if the agent interface contains uncommitted writes.

intel_vvp_crs_get_status

Prototype

```
uint32_t intel_vvp_crs_get_status(intel_vvp_crs_instance* instance);
```

Description Reads the STATUS register. The instance must be a valid intel_vvp_crs_instance fully initialized.

Arguments instance – pointer to the intel_vvp_crs_instance software driver instance structure

Returns The content of the STATUS register

intel_vvp_crs_set_output_subsampling

Prototype

```
int
intel_vvp_crs_set_output_subsampling(intel_vvp_crs_instance*
instance, eIntelCrsSubsampling output_subsampling);
```

Description Selects the required output subsampling. Use intel_vvp_core_set_img_info_subsampling to set the input subsampling in lite mode. The instance must be a valid intel_vvp_crs_instance fully initialized. The call does not check if the IP supports the requested chroma resampling conversion and a default conversion is used if the input/output selections are unsupported.

Arguments instance – pointer to the intel_vvp_crs_instance software driver instance structure

output_subsampling – requested output subsampling

Returns kIntelVvpCoreOk or a negative error code in case of error
kIntelVvpCoreInstanceErr, if the instance is the NULL pointer
kIntelVvpCrsParamErr, if the output subsampling is invalid

intel_vvp_crs_get_output_subsampling

Prototype

```
eIntelCrsSubsampling
intel_vvp_crs_get_output_subsampling(intel_vvp_crs_instance*
instance);
```

Description Reads the OUTPUT_SUBSAMPLING register. The instance must be a valid intel_vvp_crs_instance fully initialized and configured in debug mode.

Arguments instance – pointer to the intel_vvp_crs_instance software driver instance structure

Returns the last value written to the OUTPUT_SUBSAMPLING register or kIntelVvpCrsSubsamplingInvalid in case of errors

intel_vvp_crs_commit_writes

Prototype

```
int intel_vvp_crs_commit_writes(intel_vvp_crs_instance*
instance)
```

Description Commits all pending writes before processing the next field. The instance must be valid and parameterized in full mode.

Arguments instance – pointer to the intel_vvp_crs_instance software driver instance structure

Returns kIntelVvpCoreOk (0) for success, negative error code otherwise

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

18. Clipper IP

18.1. About the Clipper IP

The IP crops an active area from a video stream and discards the remainder. To specify the active region, you provide offsets from each edge of the image, or a point to be the top-left corner of the active region, and the region's width and height.

The IP is available as full or lite variants. For more information on full and lite, refer to the *Altera Streaming Video Protocol Specification*. The clipper IP changes input resolutions by reading video and vision processing image information packets, or using the register interface for lite variants.

An Avalon memory-mapped interface allows you to change the clipping settings at run time. This interface is mandatory for lite variants.

Related Information

[Altera Streaming Video Protocol Specification](#)

18.1.1. Clipper IP Performance and Resources

Table 244. Performance and Resources

For a design with 2 pixels in parallel, 8 bits per color sample, 2 color planes, and clipping with offsets

Target Device	ALMs	f _{MAX} MHz
Agilex 7 AGFA012R24A2E2V	721	781
Arria 10 10AS066H1F34E1HG	627	482
Cyclone 10 GX 10CX220YF672E5G	625	425
Stratix 10 1SX280LN2F43E1VG	771	669

18.2. Clipper IP Parameters

Table 245. Clipper IP Parameters

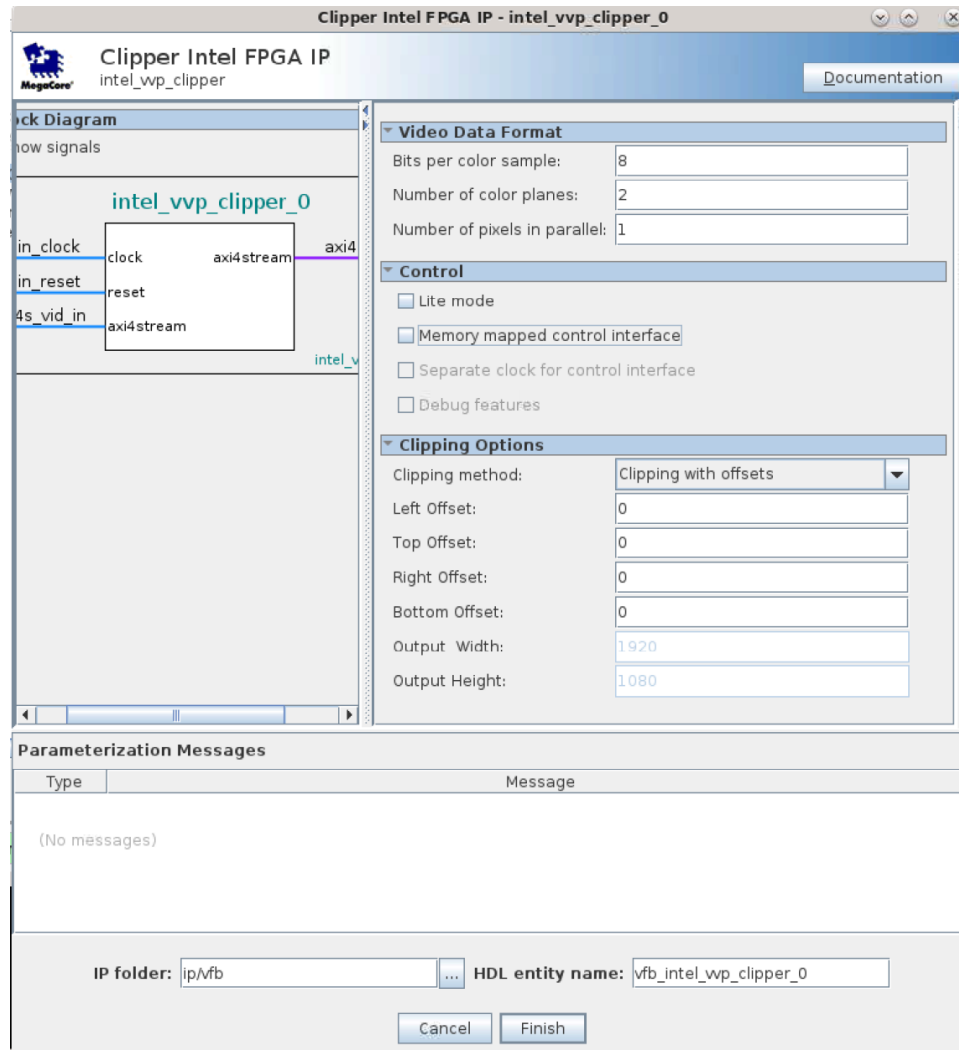
Parameter	Values	Description
Video Data Format		
Bits per color sample	8 to 16	Select the number of bits per color sample.
Number of color planes	1 to 4	Select the number of color planes per pixel.
Number of pixels in parallel	1 to 8	Select the number of pixels in parallel.
<i>continued...</i>		

Parameter	Values	Description
Control		
Lite mode	On or off	Turn on to use the lite variant of the Altera Streaming Video Protocol.
Memory-mapped control interface	On or off	Turn on to specify clipping offsets using the Avalon memory-mapped interface. You must have the Avalon memory-mapped control agent interface if you turn on Lite mode .
Separate clock for control interface	On or off	Turn on for a separate clock for the control interface.
Debug features	On or off	Turn on for debugging features (not applicable when you turn on Lite mode).
Clipping options		
Clipping method	Clipping with offsets or Clipping with output dimensions	Specify the clipping area as offsets from the edge of the input area or as a fixed rectangle.
Left offset	0 to 16383	For clips of fixed dimensions, specify the x coordinate for the left edge of the clipping rectangle. 0 is the left edge of the input area. The left and right offset values must be less than or equal to the input image width.
Top offset (45)	0 to 16383	For clips of fixed dimensions, specify the x coordinate for the top edge of the clipping rectangle. 0 is the top edge of the input area. The top and bottom offset values must be less than or equal to the input image height.
Right offset (45)	0 to 16383	For clips of fixed dimensions, use this parameter to specify the x coordinate for the right edge of the clipping rectangle. 0 is the right edge of the input area. The left and right offset values must be less than or equal to the input image width.
Bottom offset (45)	0 to 16383	For clips of fixed dimensions, specify the x coordinate for the bottom edge of the clipping rectangle. 0 is the bottom edge of the input area. The top and bottom offset values must be less than or equal to the input image height.
Output width	1 to 16384	For clips of fixed dimensions, specify the width of the clipping rectangle.
Output height (46)	1 to 16384	For clips of fixed dimensions, specify the height of the clipping rectangle.

(45) Not applicable when you turn on the memory-mapped control interface or when you select **Clipping with output dimensions**

(46) Not applicable when you turn on the memory-mapped control interface or when you select **Clipping with offsets**.

Figure 43. Clipper IP GUI



18.3. Clipper IP Interfaces

Table 246. Clipper IP Interfaces

Name	Direction	Width	Description
Clocks and resets			
main_clock_clk	In	1	AXI4-S processing clock.
main_reset_rst	In	1	AXI4-S processing reset.
agent_clock_clk	In	1	Optional agent interface clock.
agent_reset_rst	In	1	Optional agent interface reset.
Control interfaces			
<i>continued...</i>			

Name	Direction	Width	Description
av_mm_control_agent_address	In	7	Avalon memory-mapped agent address
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write.
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data.
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable.
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read.
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data.
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read.
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request.
Altera Streaming Video interfaces			
axi4s_vid_in_tdata	In		AXI4-S data in.
axi4s_vid_in_tvalid	In	1	AXI4-S data valid.
axi4s_vid_in_tuser[0]	In	1	AXI4-S start of video frame.
axi4s_vid_in_tuser[1]	In	1	AXI4-S control or data packet.
axi4s_vid_in_tuser[N-1:2]	In		Unused.
axi4s_vid_in_tlast	In	1	AXI4-S end of packet.
axi4s_vid_in_tready	Out	1	AXI4-S data ready.
axi4s_vid_out_tdata	Out	(47)	AXI4-S data out.
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid.
axi4s_vid_out_tuser[0]	Out	1	AXI4-S start of video frame.
axi4s_vid_out_tuser[1]	Out	1	AXI4-S control or data packet.
axi4s_vid_out_tuser[N-1:2]	Out	(48)	Unused.
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet.
axi4s_vid_out_tready	In	1	AXI4-S data ready.

Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

-
- (47) The equation gives all `tdata` widths sizes in these interfaces:

$$\max(2, \text{ceil}((\text{bits per color sample} \times \text{number of color planes})/8)) \times \text{pixels in parallel} \times 8$$
- (48) This equation gives all `tuser` widths sizes in these interfaces $N = \text{ceil}(\text{tdata width} / 8)$

18.4. Clipper IP Registers

Table 247. Control Registers

In the software API the register names appear with a prefix of INTEL_VVP, INTEL_VVP_CORE or INTEL_VVP_CLIPPER as appropriate and with an optional REG suffix

Address	Register	Access		Description
		Lite (49)	Full	
Parameterization registers				
0x0000	VID_PID	RO	RO	Read this register for the clipper product ID. This register always returns 0x6AF7_022D.
0x0004	VERSION	RO	RO	Read this register for the IP version information.
0x0008	LITE_MODE	RO	RO	Read this register to determine if Lite mode is on or off. This register returns 0 when Lite mode is off and 1 when on.
0x000C	DEBUG_ENABLED	RO	RO	Read this register to determine if Debug features is on.
0x0010	CLIPPING_MODE	RO	RO	Read this register to determine the Clipping method
0x0014 to 0x011F				Unused.
Control and debug registers ⁽⁵⁰⁾				
0x0120	IMG_INFO_WIDTH	RW	RO	For lite designs, the expected width of the incoming video fields. For full, the received width in the IP derives from the image information packets.
0x0124	IMG_INFO_HEIGHT	RW	RO	For lite designs, the expected height of the incoming video fields. For full the received height in the IP derives from the image information packets.
0x0128	IMG_INFO_INTERLACE	RW	RO	For lite designs, the expected interlace information of the incoming video fields. For full, the received interlace information in image information packets.
0x012C	Reserved	-	-	Reserved.
0x0130	IMG_INFO_COLORSPACE	RW	RO	For lite designs, the expected color space of the incoming video fields. For full, the received color space in image information packets.
0x0134	IMG_INFO_SUBSAMPLING	RW	RO	For lite designs, the expected chroma subsampling of the incoming video fields. For full, the received chroma subsampling in image information packets.
0x0138	IMG_INFO_COSITING	RW	RO	For lite designs, the expected chroma co-siting of the incoming video fields. For full, the received chroma co-siting in image information packets.
0x013C	IMG_INFO_FIELD_COUNT	-	RO	The received field count field in image information packets.
0x0140	STATUS	RO	RO	Bit 0: status bit.
continued...				

(49) When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

(50) For more details of these registers, refer to *Control Packets*

Address	Register	Access		Description
		Lite (49)	Full	
				1 = clipper is processing a video field, 0 otherwise. Full designs only: Bit 1: Pending register updates bit. Any writes to the clipping specification registers (0x0148 - 0x0154) cause the pending register updates bit to be raised, to indicate outstanding changes to the clipping settings. The IP lowers this bit at the start of the first packet for a new frame (first packet after the EOP packet) after a write to the <code>COMMIT</code> register.
0x0144	COMMIT	RW	RW	The IP uses this register with the <code>STATUS</code> register pending register updates bit. Write any value to this register after writing new values to any of the clipping specification registers (0x0148 - 0x0154). The clipper switches to the new clipping specification values at the start of the next video frame after you write to this register. The commit register ensures atomic register updates, avoiding the IP only applying some of the updated register values for the next incoming frame.
Clipping specification registers				
0x0148	LEFT_OFFSET	RW	RW	If you select Clipping with offsets, write to this register to set the required offset from the left edge of the video. If you select Clipping with output dimensions, this offset defines the offset from the left edge of the video for top left corner of the clipped output.
0x014C	TOP_OFFSET	RW	RW	If you select Clipping with offsets, write to this register to set the required offset from the top edge of the video. If you select Clipping with output dimensions, this offset defines the offset from the top edge of the video for the top left corner of the clipped output.
0x0150	RIGHT_OFFSET/CLIP_WIDTH	RW	RW	If you select Clipping with offsets, write to this register to set the required offset from the right edge of the video. If you select Clipping with output dimensions, write to this register to set the required width of the output video.
0x0154	BOTTOM_OFFSET/CLIP_HEIGHT	RW	RW	If you select Clipping with offsets, write to this register to set the required offset from the bottom edge of the video. If you select Clipping with output dimensions, write to this register to set the required height of the output video.

Register Bit Descriptions

Table 248. VID_PID

Name	Bits	Description
Clipper version ID and product ID	31:0	This register always returns 0x6AF7_022D. 15:0 is the product ID and always returns 0x022D 31:16 is the vendor ID and always returns 0x6AF7

⁽⁴⁹⁾ When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

Table 249. VERSION

Name	Bits	Description
Register map version	7:0	Register map version.
IP patch revision	15:8	-
IP update revision	23:16	Updated when the IP version changes.
IP major revision	31:24	Updated when the IP version changes.

Table 250. LITE_MODE

Name	Bits	Description
Lite mode parameterization bit	0	Returns 1 if you turn on lite mode .
Unused	31:1	Unused.

Table 251. DEBUG_ENABLED

Name	Bits	Description
Debug features parameterization bit	0	Returns 1 if you turn on Debug features .
Unused	31:1	Unused.

Table 252. CLIPPING_MODE

Name	Bits	Description
Clipping mode bit	0	This bit is 0 if you select Clipping with offsets and 1 if you select Clipping with output dimensions .
Unused	31:1	Unused.

Table 253. IMG_INFO_WIDTH

Name	Bits	Description
Width bits	15:0	When you turn on Lite mode , write to this register to set the expected width of the incoming video fields. When you turn off Lite mode and turn on Debug features , this register returns the <code>width-1</code> field from the most recently received image information packet and adds 1 to return a value for <code>width</code> .
unused	31:16	Unused.

Table 254. IMG_INFO_HEIGHT

Name	Bits	Description
Height bits	15:0	When you turn on Lite mode , write to this register to set the expected height of the incoming video fields. When you turn off Lite mode and turn on Debug features , this register returns the <code>height-1</code> field from the most recently received image information packet and adds 1 to return a value for <code>height</code> .
unused	31:16	Unused.

Table 255. IMG_INFO_INTERLACE

Name	Bits	Description
IntlaceNibble bits	3:0	When you turn on Lite mode , write to this register to set the expected interlacing of the incoming video fields. When you turn off Lite mode and turn on Debug features , this register returns the <code>intlaceNibble</code> field from the most recently received image information packet.
unused	31:4	Unused.

Table 256. IMG_INFO_COLORSPACE

Name	Bits	Description
CSP code bits	6:0	When you turn on Lite mode , write to this register to set the expected color space of the incoming video fields. When you turn off Lite mode and turn on Debug features , this register returns the 7 bit CSP field from the most recently received image information packet.
unused	31:7	Unused.

Table 257. IMG_INFO_SUBSAMPLING

Name	Bits	Description
CSPSubSa code bits	1:0	When you turn on Lite mode , write to this register to set the expected chroma subsampling of the incoming video fields. When you turn off Lite mode and turn on Debug features , this register returns the <code>SUBSA</code> field from the most recently received image information packet.
unused	31:2	Unused.

Table 258. IMG_INFO_COSITING

Name	Bits	Description
Cosite code bits	1:0	When you turn on Lite mode , write to this register to set the expected chroma co-siting of the incoming video fields. When you turn off Lite mode and turn on Debug features , this register returns the <code>COSITE</code> field from the most recently received image information packet.
unused	31:2	Unused.

Table 259. IMG_INFO_FIELD_COUNT

Name	Bits	Description
Count bits	6:0	When you turn on Lite mode , this register has no function. When you turn off Lite mode and turn on Debug features , this register returns the 7 bit <code>FIELD_COUNT</code> field from the most recently received image information packet.
unused	31:7	Unused.

Table 260. STATUS

Name	Bits	Description
Status bit	0	1 = clipper is processing a video field, 0 otherwise.
Pending register updates bit	1	1 = clipper has pending updates, 0 otherwise

Table 261. COMMIT

Name	Bits	Description
Commit bits	31:0	Write to any bits to trigger a commit.

Table 262. LEFT_OFFSET

Name	Bits	Description
Left offset	15:0	If you select Clipping with offsets in the GUI, write to this register to set the required offset from the left edge of the video.

Table 263. TOP_OFFSET

Name	Bits	Description
Top offset	15:0	If you select Clipping with offsets in the GUI, write to this register to set the required offset from the top edge of the video.

Table 264. RIGHT_OFFSET_REG/CLIP_WIDTH

Name	Bits	Description
Offset or width	15:0	If you select Clipping with offsets in the GUI, write to this register to set the required offset from the right edge of the video. If you select Clipping with output dimensions in the GUI, write to this register to set the required width of the output video.

Table 265. BOTTOM_OFFSET/CLIP_HEIGHT

Name	Bits	Description
Offset or height	15:0	If you select Clipping with offsets in the GUI, write to this register to set the required offset from the bottom edge of the video. If you select Clipping with output dimensions in the GUI, write to this register to set the required height of the output video.

18.5. Clipper IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_clipper_regs.h`

Include file: `intel_vvp_clipper.h`

Table 266. Clipper API Reference

Name	Description
<code>intel_vvp_clipper_init</code>	Initialize the Clipper instance
<code>Intel_vvp_core_*</code>	.Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i> . Writable when Lite is on. Readable when Lite is off and Debug features is on.
<i>continued...</i>	

Name	Description
intel_vvp_clipper_get_lite_mode	Returns if Lite mode is on
intel_vvp_clipper_get_debug_enabled	Returns if Debug features is on
Intel_vvp_clipper_get_clipping_mode	Returns the run-time clipping method
intel_vvp_clipper_is_running	Returns if the IP is processing data
Intel_vvp_clipper_get_commit_status	Returns if the IP has uncommitted writes
intel_vvp_clipper_get_status	Reads the status register
intel_vvp_clipper_set_left_offset	Writes new value to the left offset register
intel_vvp_clipper_set_top_offset	Writes new value to the top offset register
intel_vvp_clipper_set_right_offset	Writes new value to the right offset register
intel_vvp_clipper_set_bottom_offset	Writes new value to the bottom offset register
intel_vvp_clipper_set_clip_offsets	Writes new values to the four offset registers
intel_vvp_clipper_set_clip_width	Writes new value to the clipping width register
intel_vvp_clipper_set_clip_height	Writes new value to the clipping height register
intel_vvp_clipper_set_clip_area	Writes new values to the offset/clip registers
intel_vvp_clipper_get_left_offset	Reads the left offset register
intel_vvp_clipper_get_top_offset	Reads the top offset register
intel_vvp_clipper_get_right_offset	Reads the right offset register
intel_vvp_clipper_get_bottom_offset	Reads the bottom offset register
intel_vvp_clipper_get_clip_offsets	Reads the four offset registers
intel_vvp_clipper_get_clip_width	Reads the clipping width register
intel_vvp_clipper_get_clip_height	Reads the clipping height register
intel_vvp_clipper_get_clip_area	Reads the offset and clip registers
intel_vvp_clipper_commit_writes	Commit all outstanding writes

enum eIntelVvpClippingMode

- Members**
- kIntelVvpOffsetClipping – Clipping with four offsets from the input borders
 - kIntelVvpRectangleClipping – Clipping with a top-left offset and the dimensions of the clipped area
 - kIntelVvpInvalidClipping – Invalid clipping (used to indicate errors)

Description Enumeration for the supported clipping methods

intel_vvp_clipper_init

Prototype	<pre>int intel_vvp_clipper_init(intel_vvp_clipper_instance *instance, intel_vvp_core_base base);</pre>
Description	Initializes a clipper instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance does not fully initialize and should not be used further by the application if returning a non-zero error code.
Arguments	instance – pointer to the intel_vvp_clipper_instance software driver instance structure base – base address of the register map
Returns	kIntelVvpCoreOk (0) in case of success, a negative error code in case of error kIntelVvpCoreVidErr if the vendor id of the IP is not the Altera vendor ID (0x6AF7). kIntelVvpCorePidErr if the product_id does not match with the clipper product ID kIntelVvpCoreInstanceErr if the instance is a null pointer kIntelVvpClipperRegMapVersionErr if the register map is not supported

intel_vvp_clipper_get_lite_mode

Prototype	<pre>bool intel_vvp_clipper_get_lite_mode(intel_vvp_clipper_instance* instance)</pre>
Description	Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_clipper_instance fully initialized
Arguments	instance – pointer to the intel_vvp_clipper_instance software driver instance structure
Returns	true if the IP is parameterized in lite mode

intel_vvp_clipper_get_debug_enabled

Prototype `bool
intel_vvp_clipper_get_debug_enabled(intel_vvp_clipper_instance
*instance);`

Description Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_clipper_instance fully initialized.

Arguments instance – pointer to the intel_vvp_clipper_instance software driver instance structure

Returns true if the IP is parameterized with debug features enabled

intel_vvp_clipper_get_clipping_mode

Prototype `eIntelVvpClippingMode
intel_vvp_clipper_get_clipping_mode(intel_vvp_clipper_instance
*instance);`

Description Returns the value of the CLIPPING_MODE register. The instance must be a valid intel_vvp_clipper_instance fully initialized.

Arguments instance – pointer to the intel_vvp_clipper_instance software driver instance structure

Returns the clipping mode or kIntelVvpInvalidClipping in case of error

intel_vvp_clipper_is_running

Prototype `bool intel_vvp_clipper
_is_running(intel_vvp_clipper_instance* instance);`

Description Reads and returns the running bit of the STATUS register. The instance must be a valid intel_vvp_clipper_instance fully initialized.

Arguments instance – pointer to the intel_vvp_clipper_instance software driver instance structure

Returns True is the IP is currently generating an output field, false when between two fields

intel_vvp_clipper_get_commit_status

Prototype `bool
intel_vvp_clipper_get_commit_status(intel_vvp_clipper_instance*
instance);`

<i>Description</i>	Reads and returns the commit pending bit of the STATUS register. The instance must be a valid intel_vvp_clipper_instance fully initialized and configured in full mode.
<i>Arguments</i>	instance – pointer to the intel_vvp_clipper_instance software driver instance structure
<i>Returns</i>	True is the IP is configured in full mode and if the agent interface contains uncommitted writes.

intel_vvp_clipper_get_status

Prototype `uint8_t
intel_vvp_clipper_get_status(intel_vvp_clipper_instance*
instance);`

Description Reads and returns the STATUS register. The instance must be a valid intel_vvp_clipper_instance fully initialized.

Arguments instance – pointer to the intel_vvp_clipper_instance software driver instance structure

Returns The value read from the status register

intel_vvp_clipper_set_left_offset

Prototype `int
intel_vvp_clipper_set_left_offset(intel_vvp_clipper_instance*
instance , uint16_t left_offset);`

Description Writes a new offset value to the LEFT_OFFSET register. The instance must be a valid intel_vvp_clipper_instance fully initialized.

Arguments instance – pointer to the intel_vvp_clipper_instance software driver instance structure

left_offset – new left offset value

Returns kIntelVvpCoreOk in case of success, negative value in case of errors
kIntelVvpCoreInstanceErr if the instance is invalid

intel_vvp_clipper_set_top_offset

Prototype `int
intel_vvp_clipper_set_top_offset(intel_vvp_clipper_instance*
instance , uint16_t top_offset);`

Description Writes a new offset value to the TOP_OFFSET register. The instance must be a valid intel_vvp_clipper_instance fully initialized.

Arguments instance – pointer to the intel_vvp_clipper_instance software driver instance structure

top_offset – new top offset value

Returns kIntelVvpCoreOk in case of success, negative value in case of errors

kIntelVvpCoreInstanceErr if the instance is invalid

intel_vvp_clipper_set_right_offset

Prototype int
intel_vvp_clipper_set_right_offset(intel_vvp_clipper_instance* instance , uint16_t right_offset);

Description Writes a new offset value to the RIGHT_OFFSET register. The instance must be a valid intel_vvp_clipper_instance fully initialized and configured for clipping with four offsets

Arguments instance – pointer to the intel_vvp_clipper_instance software driver instance structure

right_offset – new right offset value

Returns kIntelVvpCoreOk in case of success, negative value in case of errors

kIntelVvpCoreInstanceErr if the instance is invalid

intel_vvp_clipper_set_bottom_offset

Prototype int
intel_vvp_clipper_set_bottom_offset(intel_vvp_clipper_instance* instance , uint16_t bottom_offset);

Description Writes a new offset value to the BOTTOM_OFFSET register. The instance must be a valid intel_vvp_clipper_instance fully initialized and configured for clipping with four offsets

Arguments instance – pointer to the intel_vvp_clipper_instance software driver instance structure

bottom_offset – new bottom offset value

Returns kIntelVvpCoreOk in case of success, negative value in case of errors

kIntelVvpCoreInstanceErr if the instance is invalid

`intel_vvp_clipper_set_clip_offsets`

Prototype `int`
 `intel_vvp_clipper_set_clip_offsets(intel_vvp_clipper_instance*`
 `instance , uint16_t left_offset, uint16_t top_offset, uint16_t`
 `right_offset, uint16_t bottom_offset);`

Description Writes new offsets value to the four offset registers. The instance must be a valid `intel_vvp_clipper_instance` fully initialized and configured for clipping with four offsets

Arguments `instance` – pointer to the `intel_vvp_clipper_instance` software driver instance structure

`left_offset` – new left offset value

`top_offset` – new top offset value

`right_offset` – new right offset value

`bottom_offset` – new bottom offset value

Returns `kIntelVvpCoreOk` in case of success, negative value in case of errors

`kIntelVvpCoreInstanceErr` if the instance is invalid

`intel_vvp_clipper_set_clip_width`

Prototype `int`
 `intel_vvp_clipper_set_clip_width(intel_vvp_clipper_instance`
 `*instance, uint32_t clip_width)`

Description Writes a new clipping width value to the `CLIP_WIDTH` register. The instance must be a valid `intel_vvp_clipper_instance` fully initialized and configured for clipping with two offsets and dimensions of the clipped area

Arguments `instance` – pointer to the `intel_vvp_clipper_instance` software driver instance structure

`clip_width` – new clipped area width value

Returns `kIntelVvpCoreOk` in case of success, negative value in case of errors

`kIntelVvpCoreInstanceErr` if the instance is invalid

`intel_vvp_clipper_set_clip_height`

Prototype `int`
 `intel_vvp_clipper_set_clip_height(intel_vvp_clipper_instance`
 `*instance, uint32_t clip_height)`

Description Writes a new clipping height value to the CLIP_HEIGHT register. The instance must be a valid intel_vvp_clipper_instance fully initialized and configured for clipping with two offsets and dimensions of the clipped area

Arguments instance – pointer to the intel_vvp_clipper_instance software driver instance structure

clip_height – new clipped area height value

Returns kIntelVvpCoreOk in case of success, negative value in case of errors

kIntelVvpCoreInstanceErr if the instance is invalid

intel_vvp_clipper_set_clip_area

Prototype int
intel_vvp_clipper_set_clip_area(intel_vvp_clipper_instance
*instance, uint16_t left_offset , uint16_t top_offset ,
uint32_t clip_width, uint32_t clip_height)

Description Writes new offsets and new dimensions to the offset and clipping registers. The instance must be a valid intel_vvp_clipper_instance fully initialized and configured for clipping with two offsets and dimensions of the clipped area

Arguments instance – pointer to the intel_vvp_clipper_instance software driver instance structure

left_offset – new left offset value

top_offset – new top offset value

clip_width – new clipped area width value

clip_height – new clipped area height value

Returns kIntelVvpCoreOk in case of success, negative value in case of errors

kIntelVvpCoreInstanceErr if the instance is invalid

intel_vvp_clipper_get_left_offset

Prototype uint16_t
intel_vvp_clipper_get_left_offset(intel_vvp_clipper_instance*
instance);

Description Reads the LEFT_OFFSET register. The instance must be a valid intel_vvp_clipper_instance fully initialized and parameterized with debug features enabled

Arguments instance – pointer to the intel_vvp_clipper_instance software driver instance structure

Returns The left offset value

`intel_vvp_clipper_get_top_offset`

Prototype `uint16_t
intel_vvp_clipper_get_top_offset(intel_vvp_clipper_instance*
instance);`

Description Reads the TOP_OFFSET register. The instance must be a valid
intel_vvp_clipper_instance fully initialized and parameterized with debug
features enabled

Arguments instance – pointer to the intel_vvp_clipper_instance software driver instance
structure

Returns The top offset value

`intel_vvp_clipper_get_right_offset`

Prototype `uint16_t
intel_vvp_clipper_get_right_offset(intel_vvp_clipper_instance*
instance);`

Description Reads the RIGHT_OFFSET register. The instance must be a valid
intel_vvp_clipper_instance fully initialized, configured for clipping with four
offsets, and parameterized with debug features enabled

Arguments instance – pointer to the intel_vvp_clipper_instance software driver instance
structure

Returns The right offset value

`intel_vvp_clipper_get_bottom_offset`

Prototype `uint16_t
intel_vvp_clipper_get_bottom_offset(intel_vvp_clipper_instance*
instance);`

Description Reads the BOTTOM_OFFSET register. The instance must be a valid
intel_vvp_clipper_instance fully initialized, configured for clipping with four offsets,
and parameterized with debug features enabled

Arguments instance – pointer to the intel_vvp_clipper_instance software driver instance
structure

Returns The bottom offset value

intel_vvp_clipper_get_clip_offsets

Prototype `int
intel_vvp_clipper_get_clip_offsets(intel_vvp_clipper_instance*
instance , uint16_t *left_offset, uint16_t *top_offset,
uint16_t *right_offset, uint16_t *bottom_offset);`

Description Read current offsets value from the four offset registers. The instance must be a valid `intel_vvp_clipper_instance` fully initialized, configured for clipping with four offsets, and parameterized with debug features enabled

Arguments `instance` – pointer to the `intel_vvp_clipper_instance` software driver instance structure

`left_offset` – pointer to a `uint16_t` value to store the value read from the LEFT_OFFSET register

`top_offset` – pointer to a `uint16_t` value to store the value read from the TOP_OFFSET register

`right_offset` – pointer to a `uint16_t` value to store the value read from the RIGHT_OFFSET register

`bottom_offset` – pointer to a `uint16_t` value to store the value read from the BOTTOM_OFFSET register

Returns `kIntelVvpCoreOk` in case of success, negative value in case of errors

`kIntelVvpCoreInstanceErr` if the instance is invalid

`kIntelVvpCoreNullPtrErr` if one of the pointers to store an output value is the null pointer

intel_vvp_clipper_get_clip_width

Prototype `uint32_t
intel_vvp_clipper_get_clip_width(intel_vvp_clipper_instance
*instance)`

Description Reads the CLIP_WIDTH register. The instance must be a valid `intel_vvp_clipper_instance` fully initialized, configured for clipping with top-left offsets and dimensions of the clipped area, and parameterized with debug features enabled

Arguments `instance` – pointer to the `intel_vvp_clipper_instance` software driver instance structure

Returns Current clipping with value

`intel_vvp_clipper_get_clip_height`

Prototype	<code>uint32_t intel_vvp_clipper_get_clip_height(intel_vvp_clipper_instance *instance)</code>
Description	Reads the CLIP_HEIGHT register. The instance must be a valid <code>intel_vvp_clipper_instance</code> fully initialized, configured for clipping with top-left offsets and dimensions of the clipped area, and parameterized with debug features enabled
Arguments	<code>instance</code> – pointer to the <code>intel_vvp_clipper_instance</code> software driver instance structure
Returns	Current clipping height value

`intel_vvp_clipper_get_clip_area`

Prototype	<code>int intel_vvp_clipper_get_clip_area(intel_vvp_clipper_instance *instance, uint16_t *left_offset , uint16_t *top_offset , uint32_t *clip_width, uint32_t *clip_height)</code>
Description	Reads current top-left offset value from the two offset registers and dimensions of the clipped area from the clipping dimensions registers. The instance must be a valid <code>intel_vvp_clipper_instance</code> fully initialized, configured for clipping with top-left offsets and dimensions of the clipped area, and parameterized with debug features enabled
Arguments	<code>instance</code> – pointer to the <code>intel_vvp_clipper_instance</code> software driver instance structure <code>left_offset</code> – pointer to a <code>uint16_t</code> value to store the value read from the LEFT_OFFSET register <code>top_offset</code> – pointer to a <code>uint16_t</code> value to store the value read from the TOP_OFFSET register <code>right_offset</code> – pointer to a <code>uint32_t</code> value to store the value read from the CLIP_WIDTH register <code>bottom_offset</code> – pointer to a <code>uint32_t</code> value to store the value read from the CLIP_HEIGHT register
Returns	<code>kIntelVvpCoreOk</code> in case of success, negative value in case of errors <code>kIntelVvpCoreInstanceErr</code> if the instance is invalid <code>kIntelVvpCoreNullPtrErr</code> if one of the pointers to store an output value is the null pointer

`intel_vvp_clipper_commit_writes`

Prototype `int
intel_vvp_clipper_commit_writes(intel_vvp_clipper_instance*
instance)`

Description Commits all pending writes before processing the next field. The instance must be valid and parameterized in full mode.

Arguments instance – pointer to the intel_vvp_clipper_instance software driver instance structure

Returns kIntelVvpCoreOk (0) for success, negative error code otherwise

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

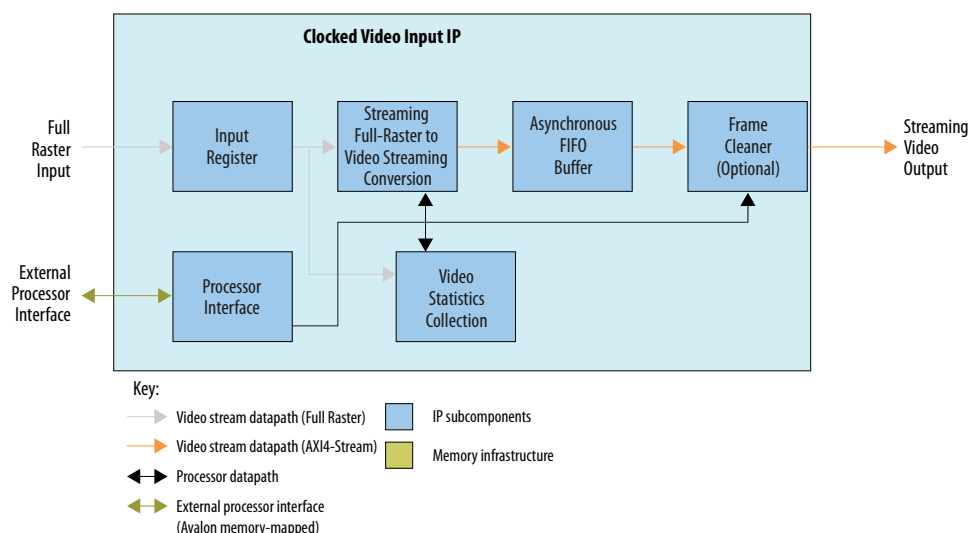
19. Clocked Video Input IP

19.1. About the Clocked Video Input IP

The Clocked Video Input IP allows transferring video data between a timing-aware clocked video input interface and a timing-agnostic video streaming output interface. The IP provides a seamless conversion by removing video timing data from a full-raster bus and leaves just the pixel data in Altera Streaming Video format. Typically, the IP is a bridge to transfer data from video connectivity to video processing IP.

The IP accepts full-raster video input format as a full-raster interface of pixel data and timing markers embedded in the `TDATA` bus of an AXI4-S interface. Also, the IP filters out the timing markers from the `TDATA` bus and provides a video-active only data format on the output interfaces that conforms to the Altera Streaming Video Protocol Specification.

Figure 44. Clocked Video Input IP Block Diagram



The IP provides a wide variety of video statistic collection, e.g. frame counter, frame per second, active and blanking video frame information. All this information is accessible directly from a processor register interface.

Figure 45. Video Statistics Information

The figure provides an example of the video statistics information that the IP can provide for a 1080p60 video resolution.

```
=====
CVI Full Raster Video Info
=====
Input Pixel Clock Rate (MHz)      : 148.498
FPS (V Freq in Hz)               : 60.00
Total Full Raster Pixels per Frame : 2475000
Total Full Raster Pixels per Line  : 2200
Total Full Raster Blanking Pixels per Line : 280
Total Full Raster F0 Lines         : 1125
Total Full Raster Blanking Lines   : 45
=====

CVI Active Video Info
=====
Total Active Pixels per Frame      : 2073600
Total Active Pixels per Line       : 1920
Total Active F0 Lines              : 1080
=====

Hsync and Vsync Auto-detect Polarity Info
=====
Hsync Polarity                    : 1
Vsync Polarity                    : 1
=====

Hsync Info
=====
HFP                               : 88
HSync/HBlank                      : 44
HBP                               : 148
=====

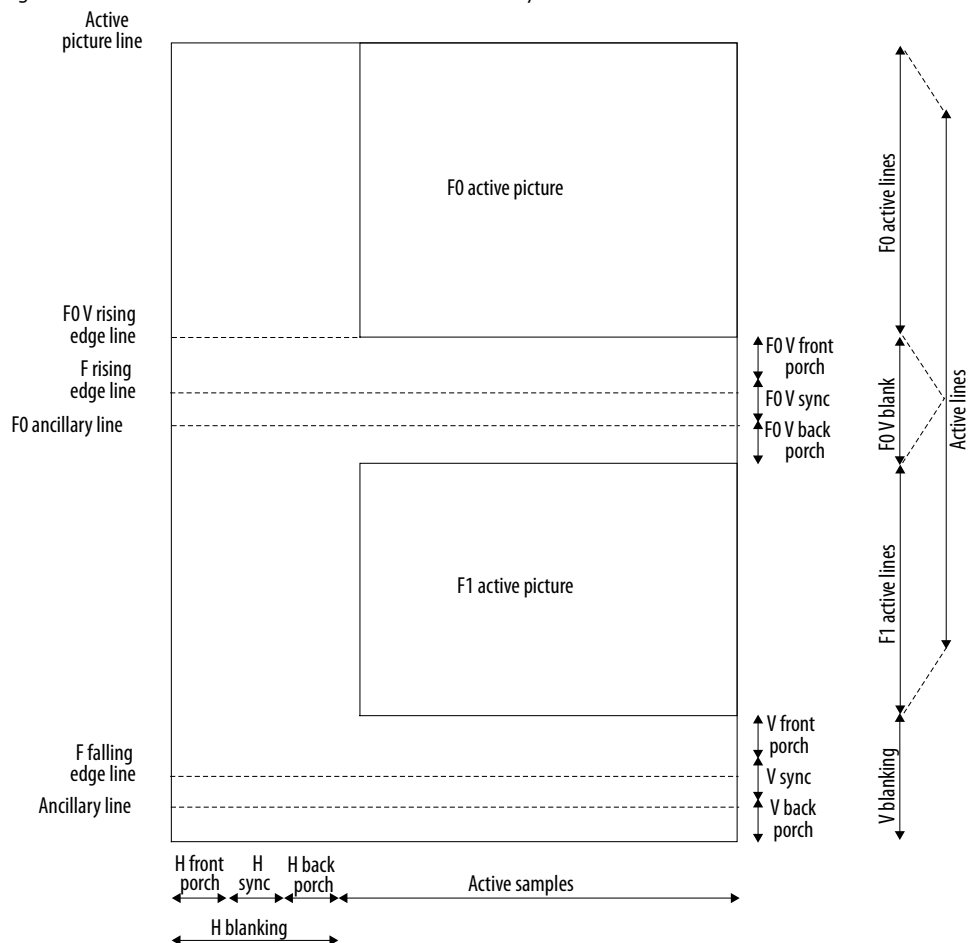
Vsync F0 Info
=====
F0 VFP                            : 4
F0 VSync/VBlank                   : 5
F0 VBP                            : 36
=====
*****
*****
=====
```

By default, the IP only provides full-raster and video active high and width information as part of the video statistic collection. However, if you select the video telemetrics information in the GUI, the IP provides a more comprehensive set of video timing parameters. An optional frame per second counter provides you with an accurate frame rate estimation and its corresponding full-raster video clock.

The IP provides autopolarity detection for the horizontal and vertical timing markers, Vsynch and Hsynch, and allows automatic detection of interlaced video formats. You can configure automatic detection using the processor control interface.

Figure 46. Full-Raster Timing

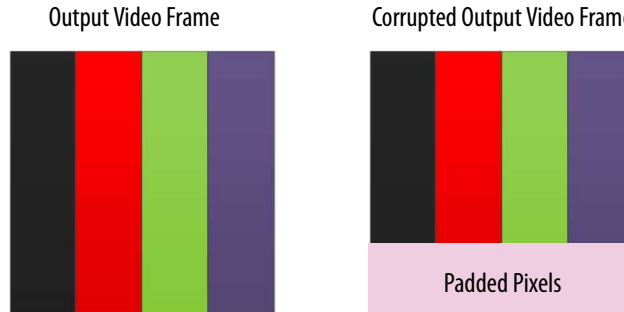
The figure shows how the video statistics values collected by the IP correlate with a full-raster video frame.



The IP also provides an asynchronous FIFO buffer to handle the clock domain jump from input to output video domain. An optional frame cleaner is available at the back end of the IP. The frame cleaner allows you to automatically pad an incomplete frame (for example, because of an unplugged cable) by inserting a user configurable color pattern. You can turn on or off the frame cleaner at run time using the processor control interface.

Figure 47. Corrupted Frame

The figure shows an example where an output frame is corrupted, and the embedded frame cleaner adds padding (pink pixels) from the Clocked Video Input IP to complete the video output raster.



The IP only supports lite variants. For more information about full and lite variants, refer to the *Altera Streaming Video Protocol Specification*

Related Information

[Altera Streaming Video Protocol Specification](#)

19.1.1. Clocked Video Input IP Features

- An Avalon memory-mapped control interface
- User configurable 1 to 32 clocked video input interfaces
- Full-raster compliant video input interface
- Altera Streaming Video compliant output interface (lite variant support only)
- Optional TREADY conduit on the full-raster interface
- Clock domain crossing between video pixel clock and video processing clock
- 1K to 32K depth video line buffer
- 8-bit to 16-bit per color component
- 1, 2, 3, and 4 color planes per pixel
- 1, 2, 4, and 8 pixels in parallel
- Small FPGA resource footprint

19.1.2. Clocked Video Input IP Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource usage depends on the device family, number of supported bits per sample, pixels in parallel, number of color planes and maximum depth of video line buffer.

Table 267. Clocked Video Input IP Performance and Resources

The table shows ALM usage and f_{MAX} for a design with 1 pixel in parallel, 8 bits per color sample, 3 color planes, and 1,024 output FIFO depth.

Target Device	ALMs	M20Ks	Input Clock f_{MAX} MHz	Output Clock f_{MAX} MHz
Agilex 7 (AGFA012R24A2E2V)	971	2	781	781
continued...				

Target Device	ALMs	M20Ks	Input Clock f_{MAX} MHz	Output Clock f_{MAX} MHz
Arria 10 (10AS066H1F34E1HG)	692	2	609	533
Cyclone 10 GX (10CX220YF672E5G)	693	2	506	511
Stratix 10 (1SX280LN2F43E1VG)	883	2	747	660

19.2. Initializing the Clocked Video Input IP

The IP provides an Avalon memory-mapped interface, which you can use as a control interface to configure the IP. Initially, the IP is disabled and does not transmit any data or video. However, the Clocked Video Input IP still detects the format of the clocked video input and accepts data on the input video interface.

To start the output of the IP:

1. Write a 1 to `control` register bit 0 to enable the clocked video input block
2. Write a 1 to `control` register bit 4 to enable Vsync and Hsync autopolarity detection.
3. Optionally, write a 1 to `control` register bit 3 to enable the frame cleaner logic.
4. Optionally, write the minimum expected number of frames for F0 and F1 to `control` register bits 23:16 to enable autodetect interlaced video format mode. If you write zeros to this set of bits, the IP does not automatically auto-detect video interlaced formats.
5. Write the expected output video height and width values to `ref_lock_cfg1` register. The IP only starts transmitting video on the output interface when the values on `ref_lock_cfg1` matches the values on registers `active_line_count` and `total_line_count`.
6. Write the expected number of frames and output video lines values to `ref_lock_cfg2` register. The IP only starts transmitting video on the output interface when the values on `ref_lock_cfg2` are matched.
7. Alternatively, if you write zeros to `ref_lock_cfg1` and `ref_lock_cfg2`, the IP does not try to match any specific output video resolution values and immediately produces video.
8. Optionally, set the values for each of the color planes that the frame cleaner use to do the padding on the output video frame in case a cable is pulled.
9. Read `status` register bit 4. When this bit is 1, the IP starts transmitting video. The transmission starts on the next start of frame boundary.

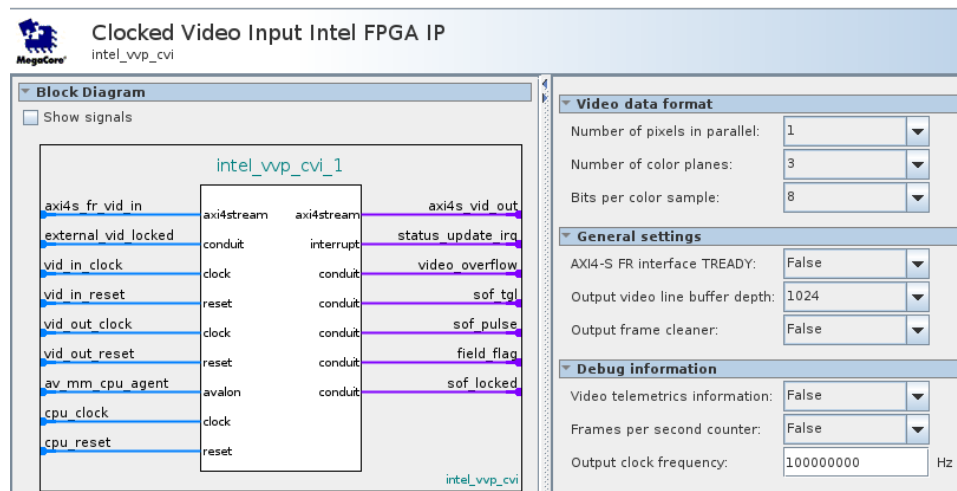
19.3. Clocked Video Input IP Parameters

The IP offers compile-time parameters.

Table 268. Clocked Video Input IP Parameters

Parameter	Values	Description
Video Format		
Bits per color sample	8 to 16	Select the number of bits per color sample at the input
Number of pixels in parallel	1 to 8	Select the number of pixels transmitted every clock cycle.
Number of color planes	1 to 4	Select the number of color planes per pixel
General Settings		
AXI4-S FR interface TREADY	True or false	Enable the TREADY signal as part of the full-raster interface
Output video line buffer depth	1024, 2048, 4096, 8192, 16384, 32768	Select the maximum depth of the output video line buffer
Output frame cleaner	True or false	Enable the output frame cleaner logic
Debug Information		
Video telemetrics information	True or false	Enable the video telemetrics information logic
Frames per second counter	True or false	Enable the frames per second counter logic
Output clock frequency	1 to 1000000000	Select the output clock frequency in Hz

Figure 48. Clocked Video Input IP GUI



19.4. Clocked Video Input IP Interfaces

Table 269. Clocked Video Input IP Interfaces

Name	Direction	Width	Description
Clocks and Resets			
vid_in_clk	In	1	Input video clock
vid_in_reset	In	1	Input video reset
continued...			

Name	Direction	Width	Description
vid_out_clk	In	1	Output video clock
vid_out_reset	In	1	Output video reset
cpu_clock	In	1	Control interface clock
cpu_reset	In	1	Control interface reset
Control Interfaces			
av_mm_control_agent_address	In	7	Avalon memory-mapped agent address
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request
Altera Streaming Video interfaces			
axi4s_fr_vid_in_tdata	In		AXI4-S data in
axi4s_fr_vid_in_tvalid	In	1	AXI4-S data valid
axi4s_fr_vid_in_tuser[0]	In	1	AXI4-S start of video frame
axi4s_fr_vid_in_tuser[N-1:1]	In		Unused
axi4s_fr_vid_in_tlast	In	1	AXI4-S end of packet
axi4s_fr_vid_in_tready	Out	1	AXI4-S data ready
axi4s_vid_out_tdata	Out		AXI4-S data out
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid
axi4s_vid_out_tuser[0]	Out	1	AXI4-S start of video frame
axi4s_vid_out_tuser[1]	Out	1	Field flag for interlaced formats
axi4s_vid_out_tuser[N-1:2]	Out	(52)	Unused
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet
axi4s_vid_out_tready	In	1	AXI4-S data ready
continued...			

- (51) The equation gives the TDATA width for these interfaces for full-raster variants:

$$\max(\text{floor}(((\text{bits per color sample} \times (\text{number of color planes} + 1) \times \text{pixels in parallel}) + 7) / 8) \times 8, 16)$$
- (52) This equation gives the TUSER width N for these interfaces: $\text{ceil}(\text{tdata width} / 8)$
- (53) The equation gives the TDATA width for these interfaces for full or lite variants:

$$\max(\text{floor}(((\text{bits per color sample} \times \text{number of color planes} \times \text{pixels in parallel}) + 7) / 8) \times 8, 16)$$

Name	Direction	Width	Description
External Conduits			
Sof_locked	Out	1	Start of frame locked signal. When asserted, the start of frame signal is valid and you can use it.
Field_flag	Out	1	Field flag for interlaced formats
Sof_pulse	Out	1	A single pulse indicating the start of a new frame
Sof_tgl	Out	1	A signal that toggles at the start of a new frame
Video_overflow	Out	1	A signal to indicate output FIFO buffer overflow
External_video_locked	In	1	Assert this signal when a stable video stream is available on the input. Deassert this signal when the video stream is removed.
Status_update_interrupt	Out	1	Interrupt signal

19.5. Clocked Video Input IP Registers

The IP allows runtime configuration of parameters via Avalon memory-mapped processor register interface.

Table 270. Clocked Video Input IP Registers

Offset	Register	Access	Description
Parameterization registers			
0x000	VID_PID	RO	Read this register to retrieve clocked video input product ID. This register always returns 0x6FA7_0170.
0x004	VERSION_NUMBER	RO	Read this register to retrieve the version information for this IP.
0x008–0x014	RESERVED	RO	
0x018	NUMBER OF PIXELS IN PARALLEL	RO	Read this register to determine the number of pixels processed per clock cycle.
0x01C	NUMBER OF COLOR PLANES	RO	Read this register to determine the number of color planes.
0x020	NUMBER OF BITS PER COLOR SAMPLE	RO	Read this register to determine the number of bits per color sample
0x024	AXI4-S FR INTERFACE TREADY	RO	Read this register to determine if the full-raster interface includes the TREADY conduit
0x028	OUTPUT VIDEO LINE BUFFER DEPTH	RO	Read this register to determine the depth of the output line buffer
0x02C	VIDEO TELEMETRICS INFORMATION	RO	Read this register to determine if the optional logic to retrieve video timing information was included for this IP
0x030	OUTPUT CLOCK FREQUENCY	RO	Read this register to determine the output clock frequency
Core Specific Registers			
0x148	Control	RW	Sets the IP control mode of operation
<i>continued...</i>			

Offset	Register	Access	Description
0x14C	Status	RW	Returns the status of the IP
0x150	Interrupt_Alarm	RW	Interrupt status and clear bits
0x154	Horizontal_sample_count	RO	Returns the detected width count per line of the video streams.
0x158	Active_line_count	RO	Returns the detected height count per frame of the video streams excluding blanking.
0x15C	Total_line_count	RO	Returns the detected height count per frame of the video streams including blanking.
0x160	Ref_lock_cfg1	RW	The IP reference width and height values to assert the video locked signal
0x164	Ref_lock_cfg2	RW	The IP reference number of frames and lines values to assert the video locked signal
0x168	Clk_in_freq_count	RO	Returns the detected frequency value of the input clock
0x16C	Tpg_cfg1	RW	Configure the value of the color plane 4 and color plane 3
0x170	Tpg_cfg2	RW	Configure the value of the color plane 2 and color plane 1
0x174-0x184	Reserved	-	
0x188	Total_fr_pixel_count	RO	Returns the detected total number of pixels on the input full-raster frame including blanking
0x18C	Total_active_pixel_count	RO	Return the detected total number of pixels on the input full-raster frame excluding blanking
0x190	Fr_pixel_line_count	RO	Returns the detected total number of pixels per line including and excluding blanking.
0x194	Hpulse_count	RO	Returns the Hsync width value if the video format uses Sync timing mode, otherwise it returns the total horizontal blanking period.
0x198	Hbp_hfp_count	RO	Returns the horizontal back porch width value if the video format uses Sync timing mode, otherwise it returns 0.
0x19C	f0f1_vfp_count	RO	Returns the vertical front porch value for field 0 and field 1.
0x1A0	f0f1_vpulse_count_reg	RO	Returns the Vsync value for field 0 and field 1.
0x1A4	f0f1_vbp_count_reg	RO	Returns the vertical back porch value for field 0 and field 1.
0x1A8	Hpulse_ref	RW	Reference value for the expected Hsync parameter. ⁽⁵⁴⁾
0x1AC	Hbp_hfp_ref	RW	Reference value for the expected horizontal front and back porch parameters. ⁽⁵⁴⁾
0x1B0	f0f1_vfp_ref	RW	Reference value for the expected vertical front porch parameter. ⁽⁵⁴⁾
continued...			

(54) If the corresponding alarm is on, and the detected value do not match this register, the IP asserts a sticky bit to indicate a mismatch.

Offset	Register	Access	Description
0x1B4	f0f1_vpulse_ref	RW	Reference value for the expected Vsync parameter. ⁽⁵⁴⁾
0x1B8	f0f1_vbp_ref	RW	Reference value for the expected vertical back porch parameter. ⁽⁵⁴⁾
0x1BC	f0f1_fps_cnt	RO	Returns the number of frames per second value for field 0 and field 1.

Register Bit Descriptions

Table 271. Vid_pid

Bits	Description
31:0	Product Identification Number

Table 272. Version_number_pid

Bits	Description
31:0	IP Version Number

Table 273. NUMBER OF PIXELS IN PARALLEL

Bits	Description
31:0	The number of pixels transmitted every clock cycle.

Table 274. NUMBER OF COLOR PLANES

Bits	Description
31:0	The number of color planes per sample at the input

Table 275. NUMBER OF BITS PER COLOR SAMPLE

Bits	Description
31:0	The number of bits per color sample

Table 276. AXI4-S FR INTERFACE TREADY

Bits	Description
31:0	Enable the TREADY signal as part of the full-raster interface

Table 277. OUTPUT VIDEO LINE BUFFER DEPTH

Bits	Description
31:0	The maximum depth of the output video line buffer

Table 278. VIDEO TELEMETRICS INFORMATION

Bits	Description
31:0	Enable the video telemetrics information logic

Table 279. OUTPUT CLOCK FREQUENCY

Bits	Description
31:0	The output clock frequency in Hz

Table 280. Control

Bits	Name	Description
23:16	f0f1_fps_ref_cnt	Setting these bits with the minimum expected number of frames per second values for F0 and F1 enables the automatic detection of interlaced video formats
7	ext_locked_ena	Setting this bit enables an external video locked signal to indicate when a stable video stream is present on the input.
6	hsync_pol	If auto polarity detection is off, set this bit to override the polarity for the hsync/hpulse signal
5	vsync_pol	If auto polarity detection is off, set this bit to override the polarity for the vsync/vpulse signal
4	auto_pol_dect	Set this bit to turn on the video sync autopolarity detection.
3	mux_out_sel	Set this bit to turn on the output frame cleaner.
2	irq2_ena	Set this bit to turn on the end of frame video interrupt.
1	irq1_ena	Set this bit to turn on the status update interrupt
0	Go bit	Set this bit to 1 to start IP data output on the next video frame boundary.

Table 281. Status

Bits	Name	Description
11	vid_in_clk_stopped	This bit indicates if the input video clock has stopped working
10	fr_interlaced_detected	This bit indicates if the input video format has been autodetected as interlaced video format
9	hsync_pol_detc	This bit indicates the detected polarity for the horizontal signal
8	vsync_pol_detc	This bit indicates the detected polarity for the vertical signal
7	drop_vid	This bit indicates that a frame was not completed
6	-	Reserved
5	external_vid_locked	This bit indicates if the external video locked has been asserted
4	internal_vid_locked	This bit indicates if the external video locked has been asserted
3	overflow_sticky	This bit indicates if a FIFO overflow occurs. Writing 1 to this bit clears it
2	vid_sync_mode	This bit indicates if the input video frame is using Blank timing or Sync timing
1	interlaced_detected	This bit indicates if an interlaced format has been detected
0	Status	This bit indicates if the input resolution has changed.

Table 282. Interrupt_Alarms

Bits	Name	Description
14	hfp_alarm_sticky	This bit indicates that the reference hfp counter is not matching the actual hfp counter. Set this bit to 1 to clear it.
13	hpulse_alarm_sticky	This bit indicates that the reference hpulse counter is not matching the actual hpulse counter. Set this bit to 1 to clear it.
continued...		

12	hbp_alarm_sticky	. This bit indicates that the reference hbp counter is not matching the actual hbp counter. Set this bit to 1 to clear it.
11	f0_vfp_alarm_sticky	This bit indicates that the reference vfp counter is not matching the actual vfp counter. Set this bit to 1 to clear it.
10	f0_vpulse_alarm_sticky	This bit indicates that the reference vpulse counter is not matching the actual vpulse counter. Set this bit to 1 to clear it.
9	f0_vbp_alarm_sticky	. This bit indicates that the reference vbp counter is not matching the actual vbp counter. Set this bit to 1 to clear it.
8	f1_vfp_alarm_sticky	This bit indicates that the reference vfp counter is not matching the actual vfp counter. Setting this bit to 1 to clear it.
7	f1_vpulse_alarm_sticky	This bit indicates that the reference vpulse counter is not matching the actual vpulse counter. Set this bit to 1 to clear it.
6	f1_vbp_alarm_sticky	. This bit indicates that the reference vbp counter is not matching the actual vbp counter. Set this bit to 1 to clear it.
5	video_width_alarm_sticky	This bit indicates that the reference width counter is not matching the actual width counter. Set this bit to 1 to clear it.
4	video_height_alarm_sticky	This bit indicates that the reference height counter is not matching the actual height counter. Set this bit to 1 to clear it.
3	video_frames_alarm_sticky	This bit indicates that the reference frame counter is not matching the actual frame counter. Set this bit to 1 to clear it.
2	video_lines_alarm_sticky	This bit indicates that the reference line counter is not matching the actual line counter. Set this bit to 1 to clear it.
1	eof_sticky	This bit indicates the end of frame. Set this bit to 1 to clear it.
0	irq_status_sticky	This bit indicates a change of input video resolution. Set this bit to 1 to clear it.

Table 283. Horizontal_sample_count

Bits	Description
31:16	The total number of pixels per line
15:0	The active number of pixels per line

Table 284. Active_line_count

Bits	Description
31:16	The active number of lines for field 0
15:0	The active number of lines for field 1

Table 285. Total_line_count

Bits	Description
31:16	The total (blanking + active) number of lines for field 0
15:0	The total (blanking + active) number of lines for field 1

Table 286. Ref_lock_cfg1

Bits	Description
31:16	The expected output video height to assert video locked signal
15:0	The expected output video width to assert video locked signal

Table 287. Ref_lock_cfg2

Bits	Description
31:16	The number of consecutive valid frames before asserting video locked signal
15:0	The number of consecutive valid lines before asserting video locked signal

Table 288. Clk_in_freq_count

Bits	Description
31:0	The value of input clock frequency in Hz

Table 289. Tpg_cfg1_reg

Bits	Description
31:16	Pixel value for the color plane # 4
15:0	Pixel value for the color plane # 3

Table 290. Tpg_cfg2

Bits	Description
31:16	Pixel value for the color plane # 2
15:0	Pixel value for the color plane # 1

Table 291. Total_fr_pixel_count

Bits	Description
31:0	Total number of full-raster pixels per frame

Table 292. Total_active_pixel_count

Bits	Description
31:0	Total number of actives pixels per frame

Table 293. Fr_pixel_line_count

Bits	Description
31:16	Full-raster blanking pixels per line
15:0	Full-raster blanking lines per frame

Table 294. Hpulse_count

Bits	Description
31:16	Total number of active lines per frame
15:0	Horizontal (H) counter

Table 295. Hbp_hfp_count

Bits	Description
31:16	Horizontal front porch counter
15:0	Horizontal back porch counter

Table 296. F0f1_vfp_count

Bits	Description
31:16	Vertical front porch counter for field 0
15:0	Vertical front porch counter for field 1

Table 297. F0f1_vpulse_count

Bits	Description
31:16	Vertical reference for field 0
15:0	Vertical counter for field 1

Table 298. F0f1_vbp_count

Bits	Description
31:16	Vertical back porch counter for field 0
15:0	Vertical back porch counter for field 1

Table 299. Hpulse_ref

Bits	Description
15:0	Horizontal (H) reference counter

Table 300. Hbp_hfp_ref

Bits	Description
31:16	Horizontal front porch reference counter
15:0	Horizontal back porch reference counter

Table 301. F0f1_vfp_ref

Bits	Description
31:16	Vertical front porch reference counter for field 0
15:0	Vertical front porch reference counter for field 1

Table 302. F0f1_vpulse_ref

Bits	Description
31:16	Vertical (V) reference counter for field 0
15:0	Vertical (V) reference counter for field 1

Table 303. F0f1_vbp_ref

Bits	Description
31:16	Vertical back porch reference counter for field 0
15:0	Vertical back porch reference counter for field 1

Table 304. f0f1_fps_cnt

Bits	Description
31:16	Number of frames per second detected for field 0
15:0	Number of frames per second detected for field 1

19.6. Clocked Video Input IP Software API

The IP has a software driver for software control of the IP at run time. The IP does not fit any of the generic device models provided by the Nios II HAL. It exposes a set of dedicated accessors to the control and status registers. The driver structure inherits the base driver structure so all common methods described in *Video and Vision Processing IPs Software API* are applicable.

intel_vvp_cvi_init

prototype:

```
int intel_vvp_cvi_init(intel_vvp_cvi_instance* instance,
intel_vvp_core_base base);
```

description:

Initialization function for a VVP CVI instance.

Attempts to initialize the fields of the CVI and its base core

argument:

instance, pointer to the intel_vvp_cvi_instance to initialize

base, the accessor for the core (on Nios this is a pointer to the base address of the core)

return value:

kIntelVvpCoreOk success

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID.

kIntelVvpCorePidErr if the product id of the core is not the cvi product id

kIntelVvpCoreInstanceErr if the instance parameter is zero (null pointer)

intel_vvp_cvi_get_param_vid_pid

prototype:

```
int intel_vvp_cvi_get_param_vid_pid(intel_vvp_cvi_instance*
instance);
```

argument:

instance, pointer to the intel_vvp_cvi_instance

return value:

The product ID number

intel_vvp_cvi_get_param_version_number

prototype:

```
int
intel_vvp_cvi_get_param_version_number(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: The IP version number

intel_vvp_cvi_get_param_pip

prototype:

```
int intel_vvp_cvi_get_param_pip(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: The number of pixels in parallel

intel_vvp_cvi_get_param_color_planes

prototype:

```
int
intel_vvp_cvi_get_param_color_planes(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: The number of color planes

intel_vvp_cvi_get_param_bps

prototype:

```
int intel_vvp_cvi_get_param_bps(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: The number of bits per sample

intel_vvp_cvi_get_param_tready_present

prototype:

```
int
intel_vvp_cvi_get_param_tready_present(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: 1 if the TREADY signal is present on the input interface, otherwise the conduit is not present

`intel_vvp_cvi_get_param_line_buffer_depth`

prototype:

```
int  
intel_vvp_cvi_get_param_line_buffer_depth(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: The maximum depth of the output video line buffer

`intel_vvp_cvi_get_param_extended_video_metrics_enabled`

prototype:

```
int  
intel_vvp_cvi_get_param_extended_video_metrics_enabled(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: 1 if the extended video metric logic was included during build-time

`intel_vvp_cvi_get_param_clk_freq_out`

prototype:

```
int  
intel_vvp_cvi_get_param_clk_freq_out(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: The output video frequency in Hz

`intel_vvp_cvi_enable_go`

prototype:

```
int intel_vvp_cvi_enable_go(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_disable_go

prototype:

```
int intel_vvp_cvi_disable_go(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_get_go

prototype:

```
uint16_t intel_vvp_cvi_get_go(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the go bit

intel_vvp_cvi_enable_status_update_interrupt

prototype:

```
int intel_vvp_cvi_enable_status_update_interrupt(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_disable_status_update_interrupt

prototype:

```
int intel_vvp_cvi_disable_status_update_interrupt(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_get_disable_status_update_interrupt

prototype:

```
uint16_t intel_vvp_cvi_get_status_update_interrupt(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the status_update_interrupt bit

`intel_vvp_cvi_enable_end_of_field_frame_interrupt`

prototype:

```
int
intel_vvp_cvi_enable_end_of_field_frame_interrupt(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

`intel_vvp_cvi_disable_end_of_field_frame_interrupt`

prototype:

```
int
intel_vvp_cvi_disable_end_of_field_frame_interrupt(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

`intel_vvp_cvi_get_end_of_field_frame_interrupt`

prototype:

```
uint16_t
intel_vvp_cvi_get_end_of_field_frame_interrupt(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the end_of_field_frame_interrupt bit

`intel_vvp_cvi_enable_frame_cleaner`

prototype:

```
int
intel_vvp_cvi_enable_frame_cleaner(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_disable_frame_cleaner

prototype:

```
int
intel_vvp_cvi_disable_frame_cleaner(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_get_frame_cleaner

prototype:

```
uint16_t
intel_vvp_cvi_get_frame_cleaner(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the frame_cleaner bit

intel_vvp_cvi_enable_auto_polarity_detection

prototype:

```
int
intel_vvp_cvi_enable_auto_polarity_detection(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_disable_auto_polarity_detection

prototype:

```
int
intel_vvp_cvi_disable_auto_polarity_detection(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_get_auto_polarity_detection

prototype:

```
uint16_t
intel_vvp_cvi_get_auto_polarity_detection(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the auto_polarity_detection bit

intel_vvp_cvi_set_manual_vsync_polarity_high

prototype:

```
int
intel_vvp_cvi_set_manual_vsync_polarity_high(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_set_manual_vsync_polarity_low

prototype:

```
int
intel_vvp_cvi_set_manual_vsync_polarity_low(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_get_manual_vsync_polarity

prototype:

```
uint16_t
intel_vvp_cvi_get_manual_vsync_polarity(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the manual_vsync_polarity bit

intel_vvp_cvi_set_manual_hsync_polarity_high

prototype:

```
int
intel_vvp_cvi_set_manual_hsync_polarity_high(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_set_manual_hsync_polarity_low

prototype:

```
int
intel_vvp_cvi_set_manual_hsync_polarity_low(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_get_manual_hsync_polarity

prototype:

```
uint16_t
intel_vvp_cvi_get_manual_hsync_polarity(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the manual_hsync_polarity bit

intel_vvp_cvi_is_producing_data

prototype:

```
bool intel_vvp_cvi_is_producing_data(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: 1 if the bit is asserted, otherwise 0

intel_vvp_cvi_is_interlaced

prototype:

```
bool intel_vvp_cvi_is_interlaced(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: 1 if the bit is asserted, otherwise 0

intel_vvp_cvi_is_blank_timing

prototype:

```
bool intel_vvp_cvi_is_blank_timing(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: 1 if the bit is asserted, otherwise 0

intel_vvp_cvi_is_overflowed

prototype:

```
bool intel_vvp_cvi_is_overflowed(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: 1 if the bit is asserted, otherwise 0

intel_vvp_cvi_is_internal_locked_high

prototype:

```
bool  
intel_vvp_cvi_is_internal_locked_high(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: 1 if the bit is asserted, otherwise 0

intel_vvp_cvi_is_external_locked_high

prototype:

```
bool  
intel_vvp_cvi_is_external_locked_high(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: 1 if the bit is asserted, otherwise 0

intel_vvp_cvi_is_picture_drop

prototype:

```
bool intel_vvp_cvi_is_picture_drop(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: 1 if the bit is asserted, otherwise 0

intel_vvp_cvi_is_vsync_high

prototype:

```
bool intel_vvp_cvi_is_vsync_high(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: 1 if the bit is asserted, otherwise 0

intel_vvp_cvi_is_hsync_high

prototype:

```
bool intel_vvp_cvi_is_hsync_high(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: 1 if the bit is asserted, otherwise 0

intel_vvp_cvi_clear_status_update_interrupt

prototype:

```
int intel_vvp_cvi_clear_status_update_interrupt(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_get_status_update_interrupt

prototype:

```
int intel_vvp_cvi_get_status_update_interrupt(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the status_update_interrupt bit

intel_vvp_cvi_clear_end_of_field_interrupt

prototype:

```
int intel_vvp_cvi_clear_end_of_field_interrupt(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_get_end_of_field_interrupt

prototype:

```
uint16_t intel_vvp_cvi_get_end_of_field_interrupt(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the end_of_field_interrupt bit

intel_vvp_cvi_get_total_sample_count

prototype:

```
uint16_t  
intel_vvp_cvi_get_total_sample_count(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the total_sample_count register field

intel_vvp_cvi_get_active_sample_count

prototype:

```
uint16_t  
intel_vvp_cvi_get_active_sample_count(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the active_sample_count register field

intel_vvp_cvi_get_active_line_count_f0

prototype:

```
uint16_t  
intel_vvp_cvi_get_active_line_count_f0(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the active_line_count_f0 register field

intel_vvp_cvi_get_active_line_count_f1

prototype:

```
uint16_t  
intel_vvp_cvi_get_active_line_count_f1(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the active_line_count_f1 register field

intel_vvp_cvi_get_total_line_count_f0

prototype:

```
uint16_t
intel_vvp_cvi_get_total_line_count_f0(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the total_line_count_f0 register field

intel_vvp_cvi_get_total_line_count_f1

prototype:

```
uint16_t
intel_vvp_cvi_get_total_line_count_f1(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the total_line_count_f1 register field

intel_vvp_cvi_get_reference_height_count

prototype:

```
uint16_t
intel_vvp_cvi_get_reference_height_count(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the reference_height_count register field

intel_vvp_cvi_get_reference_width_count

prototype:

```
uint16_t
intel_vvp_cvi_get_reference_width_count(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the reference_width_count register field

intel_vvp_cvi_get_reference_frames_count

prototype:

```
uint16_t
intel_vvp_cvi_get_reference_frames_count(intel_vvp_cvi_instance*
instance);
```


argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the reference_frame_count register field

intel_vvp_cvi_get_reference_lines_count

prototype:

```
uint16_t  
intel_vvp_cvi_get_reference_lines_count(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the reference_lines_count register field

intel_vvp_cvi_get_clk_in_freq_count

prototype:

```
uint16_t  
intel_vvp_cvi_get_clk_in_freq_count(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the clk_in_freq_count register field

intel_vvp_cvi_get_tpg_comp4

prototype:

```
uint16_t intel_vvp_cvi_get_tpg_comp4(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the tpg_comp4 register field

intel_vvp_cvi_get_tpg_comp3

prototype:

```
uint16_t intel_vvp_cvi_get_tpg_comp3(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the tpg_comp3 register field

intel_vvp_cvi_get_tpg_comp2

prototype:

```
uint16_t intel_vvp_cvi_get_tpg_comp2(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the tpg_comp2 register field

intel_vvp_cvi_get_tpg_comp1

prototype:

```
uint16_t intel_vvp_cvi_get_tpg_comp1(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the tpg_comp1 register field

intel_vvp_cvi_get_total_fr_pixels_per_frames_count

prototype:

```
uint16_t intel_vvp_cvi_get_total_fr_pixels_per_frames_count(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the total_fr_pixels_per_frames_count register field

intel_vvp_cvi_get_total_active_pixels_per_frames_count

prototype:

```
uint16_t intel_vvp_cvi_get_total_active_pixels_per_frames_count(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the total_active_pixels_per_frames_count register field

intel_vvp_cvi_get_fr_blanking_pixels_count

prototype:

```
uint16_t intel_vvp_cvi_get_fr_blanking_pixels_count(intel_vvp_cvi_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the fr_blanking_pixels_count register field

intel_vvp_cvi_get_fr_blanking_lines_count

prototype:

```
uint16_t  
intel_vvp_cvi_get_fr_blanking_lines_count(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the fr_blanking_lines_count register field

intel_vvp_cvi_get_hpulse_count

prototype:

```
uint16_t  
intel_vvp_cvi_get_hpulse_count(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the hpulse_count register field

intel_vvp_cvi_get_hfp_count

prototype:

```
uint16_t intel_vvp_cvi_get_hfp_count(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the hfp_count register field

intel_vvp_cvi_get_hbp_count

prototype:

```
uint16_t intel_vvp_cvi_get_hbp_count(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the hbp_count register field

intel_vvp_cvi_get_f0_vfp_count

prototype:

```
uint16_t  
intel_vvp_cvi_get_f0_vfp_count(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the f0_vfp_count register field

intel_vvp_cvi_get_f1_vfp_count

prototype:

```
uint16_t
intel_vvp_cvi_get_f1_vfp_count(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the f1_vfp_count register field

intel_vvp_cvi_get_f0_vpulse_count

prototype:

```
uint16_t
intel_vvp_cvi_get_f0_vpulse_count(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the f0_vpulse_count register field

intel_vvp_cvi_get_f1_vpulse_count

prototype:

```
uint16_t
intel_vvp_cvi_get_f1_vpulse_count(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the f1_vpulse_count register field

intel_vvp_cvi_get_f0_vbp_count

prototype:

```
uint16_t
intel_vvp_cvi_get_f0_vbp_count(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the f0_vbp_count register field

intel_vvp_cvi_get_f1_vbp_count

prototype:

```
uint16_t  
intel_vvp_cvi_get_f1_vbp_count(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the f1_vbp_count register field

intel_vvp_cvi_get_hpulse_ref_count

prototype:

```
uint16_t  
intel_vvp_cvi_get_hpulse_count(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the hpulse_count register field

intel_vvp_cvi_get_hfp_ref_count

prototype:

```
uint16_t  
intel_vvp_cvi_get_hfp_ref_count(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the hfp_ref_count register field

intel_vvp_cvi_get_hbp_ref_count

prototype:

```
uint16_t  
intel_vvp_cvi_get_hbp_ref_count(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the hbp_ref_count register field

intel_vvp_cvi_get_f0_vfp_ref_count

prototype:

```
uint16_t  
intel_vvp_cvi_get_f0_vfp_ref_count(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the f0_vfp_ref_count register field

intel_vvp_cvi_get_f1_vfp_ref_count

prototype:

```
uint16_t
intel_vvp_cvi_get_f1_vfp_ref_count(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the f1_vfp_ref_count register field

intel_vvp_cvi_get_f0_vpulse_ref_count

prototype:

```
uint16_t
intel_vvp_cvi_get_f0_vpulse_ref_count(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the f0_vpulse_ref_count register field

intel_vvp_cvi_get_f1_vpulse_ref_count

prototype:

```
uint16_t
intel_vvp_cvi_get_f1_vpulse_ref_count(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the f1_vpulse_ref_count register field

intel_vvp_cvi_get_f0_vbp_ref_count

prototype:

```
uint16_t
intel_vvp_cvi_get_f0_vbp_ref_count(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the f0_vbp_ref_count register field

intel_vvp_cvi_get_f1_vbp_ref_count

prototype:

```
uint16_t  
intel_vvp_cvi_get_f1_vbp_ref_count(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the f1_vbp_ref_count register field

intel_vvp_cvi_get_input_freq_mhz

prototype:

```
float  
intel_vvp_cvi_get_input_freq_mhz(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of the clock input frequency value

intel_vvp_cvi_get_fps

prototype:

```
float intel_vvp_cvi_get_fps(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: the current value of number of frames per seconds

intel_vvp_cvi_set_reference_height_count

prototype:

```
int  
intel_vvp_cvi_set_reference_height_count(intel_vvp_cvi_instance*  
instance, uint16_t in_val);
```

argument: instance, pointer to the intel_vvp_cvi_instance
in_val, the reference value for the output height counter

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_set_reference_width_count

prototype:

```
int  
intel_vvp_cvi_set_reference_width_count(intel_vvp_cvi_instance*  
instance, uint16_t in_val);
```

argument: instance, pointer to the intel_vvp_cvi_instance
in_val, the reference value for the output width counter

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_set_reference_frames_count

prototype:

```
int
intel_vvp_cvi_set_reference_frames_count(intel_vvp_cvi_instance*
instance, uint16_t in_val);
```

argument: instance, pointer to the intel_vvp_cvi_instance
in_val, the reference value for the output frame counter

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_set_reference_lines_count

prototype:

```
int
intel_vvp_cvi_set_reference_lines_count(intel_vvp_cvi_instance*
instance, uint16_t in_val);
```

argument: instance, pointer to the intel_vvp_cvi_instance
in_val, the reference value for the output line counter

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_set_tpg_comp4

prototype:

```
int intel_vvp_cvi_set_tpg_comp4(intel_vvp_cvi_instance*
instance, uint16_t in_val);
```

argument: instance, pointer to the intel_vvp_cvi_instance
in_val, the pixel value for color plane # 4

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_set_tpg_comp3

prototype:

```
int intel_vvp_cvi_set_tpg_comp3(intel_vvp_cvi_instance*
instance, uint16_t in_val);
```

argument: instance, pointer to the intel_vvp_cvi_instance

in_val, the pixel value for color plane # 3

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_set_tpg_comp2

prototype:

```
int intel_vvp_cvi_set_tpg_comp2(intel_vvp_cvi_instance*  
instance, uint16_t in_val);
```

argument:

instance, pointer to the intel_vvp_cvi_instance

in_val, the pixel value for color plane # 2

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_set_tpg_comp1

prototype:

```
int intel_vvp_cvi_set_tpg_comp1(intel_vvp_cvi_instance*  
instance, uint16_t in_val);
```

argument:

instance, pointer to the intel_vvp_cvi_instance

in_val, the pixel value for color plane # 1

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_set_hpulse_ref_count

prototype:

```
intel_vvp_cvi_set_hpulse_ref_count(intel_vvp_cvi_instance*  
instance, uint16_t in_val);
```

argument:

instance, pointer to the intel_vvp_cvi_instance

in_val, the reference value for the output hpulse counter

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_set_hfp_ref_count

prototype:

```
intel_vvp_cvi_set_hfp_ref_count(intel_vvp_cvi_instance*  
instance, uint16_t in_val);
```

argument:

instance, pointer to the intel_vvp_cvi_instance

in_val, the reference value for the output hfp counter

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_set_hbp_ref_count

prototype:

```
intel_vvp_cvi_set_hbp_ref_count(intel_vvp_cvi_instance*
instance, uint16_t in_val);
```

argument: instance, pointer to the intel_vvp_cvi_instance
in_val, the reference value for the output hbp counter

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_set_f0_vfp_ref_count

prototype:

```
intel_vvp_cvi_set_f0_vfp_ref_count(intel_vvp_cvi_instance*
instance, uint16_t in_val);
```

argument: instance, pointer to the intel_vvp_cvi_instance
in_val, the reference value for the output f0_vfp counter

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_set_f1_vfp_ref_count

prototype:

```
intel_vvp_cvi_set_f1_vfp_ref_count(intel_vvp_cvi_instance*
instance, uint16_t in_val);
```

argument: instance, pointer to the intel_vvp_cvi_instance
in_val, the reference value for the output f1_vfp counter

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_set_f0_vpulse_ref_count

prototype:

```
intel_vvp_cvi_set_f0_vpulse_ref_count(intel_vvp_cvi_instance*
instance, uint16_t in_val);
```

argument: instance, pointer to the intel_vvp_cvi_instance
in_val, the reference value for the output f0_vpulse counter

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_set_f1_vpulse_ref_count

prototype:

```
intel_vvp_cvi_set_f1_vpulse_ref_count(intel_vvp_cvi_instance*  
instance, uint16_t in_val);
```

argument: instance, pointer to the intel_vvp_cvi_instance
in_val, the reference value for the output f1_vpulse counter

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_set_f0_vbp_ref_count

prototype:

```
intel_vvp_cvi_set_f0_vbp_ref_count(intel_vvp_cvi_instance*  
instance, uint16_t in_val);
```

argument: instance, pointer to the intel_vvp_cvi_instance
in_val, the reference value for the output f0_vbp counter

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

intel_vvp_cvi_set_f1_vbp_ref_count

prototype:

```
intel_vvp_cvi_set_f1_vbp_ref_count(intel_vvp_cvi_instance*  
instance, uint16_t in_val);
```

argument: instance, pointer to the intel_vvp_cvi_instance
in_val, the reference value for the output f1_vbp counter

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

int intel_vvp_cvi_set_field_ref_cnt

prototype:

```
int intel_vvp_cvi_set_field_ref_cnt(intel_vvp_cvi_instance*  
instance, uint16_t in_val);
```

argument: instance, pointer to the intel_vvp_cvi_instance
in_val, the reference value for the output f1_vbp counter

return value: if success kIntelVvpCviOk, otherwise kIntelVvpCviInstanceErr

`uint16_t intel_vvp_cvi_get_field_ref_cnt`

prototype:

```
uint16_t
intel_vvp_cvi_get_field_ref_cnt(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: The current value of the field reference counter

`bool intel_vvp_cvi_is_vid_in_clk_stopped`

prototype:

```
bool
intel_vvp_cvi_is_vid_in_clk_stopped(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: 1 if the bit is asserted, otherwise 0

`bool intel_vvp_cvi_is_fr_field_high`

prototype:

```
bool intel_vvp_cvi_is_fr_field_high(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: 1 if the bit is asserted, otherwise 0

`uint16_t intel_vvp_cvi_get_active_total_line_count`

prototype:

```
uint16_t
intel_vvp_cvi_get_active_total_line_count(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: The current value of the number of active lines per frame

`uint16_t intel_vvp_cvi_get_fl_fps_count`

prototype:

```
uint16_t
intel_vvp_cvi_get_fl_fps_count(intel_vvp_cvi_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: The current value of the frames per second for field 0

`uint16_t intel_vvp_cvi_get_f1_fps_count`

prototype:

```
uint16_t  
intel_vvp_cvi_get_f1_fps_count(intel_vvp_cvi_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvi_instance

return value: The current value of the frames per second for field 0

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38



20. Clocked Video to Full-Raster Converter IP

20.1. About the Clocked Video to Full-Raster Converter IP

The IP converts between pixel data in the clocked video format (HSync, Hblank, VSync, Vblank, field, data enable, and video data) to pixel data in the Altera streaming full-raster protocol. The IP connects between IPs that support only streaming full-raster and IPs that only support clocked video interfaces.

20.1.1. Clocked Video to Full-Raster Converter Features

- Clocked video data input interface for video IOs
- Full-raster data output interface for video IOs
- CPU control interface
- Video standard agnostic
- Support for Altera clocked video input and clocked video output legacy clocked video interfaces
- 6 to 16 bits per color component
- 1, 2, 4, or 8 pixels in parallel per clock
- 1, 2, 3, or 4 color planes per pixel
- Latency of 3 clock cycles
- Small FPGA resource footprint

20.1.2. Clocked Video to Full-Raster Converter Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family and the number of supported bits per color sample, pixels in parallel, and number of color planes.

Table 305. Clocked Video Converter to Full-Raster Performance and Resource Utilization

The table shows ALM usage and f_{MAX} for a design with 2 pixels in parallel, 10 bits per color sample and 3 color planes.

Target Device	ALMs	Video clock f_{MAX} (MHz)	CPU clock f_{MAX} (MHz)
Agilex 7 AGFA012R24A2E2V	435	618	870
Arria 10 10AS066H1F34E1HG	350	388	645
Cyclone 10 GX 10CX220YF672E5G	350	345	645
Stratix 10 1SX280LN2F43E1VG	430	563	805

20.2. Clocked Video to Full-Raster Converter Parameters

Table 306. Main Parameters

Parameter	Values	Description
Video Data Format		
CV Bus Style	Lite , CVI or CVO	Select which sideband signals the IP drives, and which signals are available in Platform Designer. A second tab allows you to parameterize the CV bus style .
Bits per color sample	6 to 16	Number of bits per color sample.
Number of color planes	1 to 4	Number of color planes in a pixel.
Number of pixels in parallel	1, 2, 4 or 8	Number of pixels transmitted every clock.
AXI4S FR Bus has tReady connection	True or False	Select true so the streaming full-raster interface contains the AXI4-S tReady signal. Select false to remove, the tReady signal.
Control Settings		
Memory-mapped control interface	True or False	Select True to turn on the CPU interface and associated signals. Select False to remove the CPU interface and for all the CPU registers to use default values.
Separate clock for control interface	True or False	Select True to include the signal cpu_clock in Platform Designer. It is asynchronous to the video domain. Select False so the CPU interface uses the signal vid_clock.
Dimensions		
Max full-raster width	1 to 65535	The maximum width of raster that this IP passes.
Max full-raster height	1 to 65535	The maximum height of raster this IP passes.
Default full-raster width	1 to 65535	The width of the raster. The CPU can override this value.
Default full-raster height	1 to 65535	The height of the raster. The CPU can override this value.
Default Vsync H	1 to 65535	The default pixel position of the rising edge of the vertical timing signal (vsync or vblank) The CPU can override this value.
Default Vsync V	1 to 65535	The default line where the rising edge of the vertical timing signal (vsync or vblank) occurs. The CPU can override this value.

Figure 49. Main Parameters

The screenshot shows the configuration window for the 'Clocked Video to VVP-FR Converter Intel FPGA IP' (intel_vvp_cv2fr). The 'Main' tab is selected, and the 'CV Lite Parameters' sub-tab is active. The configuration is organized into three sections:

- Video data format:**
 - CV Bus Style: Lite (dropdown)
 - Bits per color sample: 8 (text input)
 - Number of color planes: 2 (text input)
 - Number of pixels in parallel: 1 (dropdown)
 - AXI4-S FR Bus has tReady connection: True (dropdown)
- Control settings:**
 - Memory mapped control interface: False (dropdown)
- Dimensions:**
 - Max full raster width: 2200 (text input)
 - Max full raster height: 1125 (text input)
 - Default full raster width: 2200 (text input)
 - Default full raster height: 1125 (text input)
 - Default Vsync H: 0 (text input)
 - Default Vsync V: 0 (text input)

Table 307. CV Lite Parameters

Parameter	Values	Description
Parameters		
Valid signal	On or off	When you select Lite the IP includes an optional data valid input signal, <code>cv_vid_in_valid</code> . Turn on to turn on this input in the IP. Platform Designer includes or removes the signal from the <code>cv_vid_in</code> conduit as appropriate.
Ready signal	On or off	When you select Lite the IP includes an optional data ready output signal, <code>cv_vid_in_ready</code> . Turn on to turn on this output in the IP. Platform Designer includes or removes the signal from the <code>cv_vid_in</code> conduit as appropriate.
Timing Signals	sync, blank, or both	Select which timing are available. Platform Designer includes or removes the signals from the <code>cv_vid_in</code> conduit as appropriate

Figure 50. CV Lite Parameters

Table 308. CVI Parameters

Parameters	Values	Description
CVI Core Parameters		
Source for total resolution	Use Extra Logic or Use External Signals	The IP requires the height and width of the raster. Determine this information using additional logic within the IP, or get the information from optional input signals. This parameter selects the source for the raster dimensions. Platform Designer includes or removes the signals from the <code>cv_vid_in</code> conduit as appropriate
CVI Legacy Tie Offs		
Include the vid_hd_sdn signal	True or False	Select True to add this signal to the <code>cv_vid_in</code> conduit. The signal has no function within the IP and is included for connectivity within Platform Designer.
Include the vid_std signal	True or False	Select True to add this signal to the <code>cv_vid_in</code> conduit. The signal has no function within the IP and is included for connectivity within Platform Designer.
Width of vid_std	1 to 16	The width of the <code>vid_std</code> signal.
Include the hdmi_duplication signal	True or False	Select True to add this signal to the <code>cv_vid_in</code> conduit. The signal has no function within the IP and is included for connectivity within Platform Designer.

Figure 51. CVI Parameters

The screenshot shows the configuration window for the 'Clocked Video to VVP-FR Converter Intel FPGA IP' (intel_wp_cv2fr). The 'CVI Parameters' tab is selected. Under 'CVI Core Parameters', the 'Source for total resolution' is set to 'Use Extra Logic'. Under 'CVI Legacy Tie Offs', the following settings are shown: 'Include the vid_hd_sdn signal' is False, 'Include the vid_std signal' is False, 'Width of vid_std' is 1, and 'Include the hdmi_duplication signal' is False.

Table 309. CVO Parameters

Parameters	Values	Description
CVO Legacy Tie Offs		
Drive CV Output Clock	True or False	Select True to add this signal to the <code>cv_vid_in</code> conduit. The signal has no function within the IP and is included for connectivity within Platform Designer.
Include the underflow signal	True or False	Select True to add this signal to the <code>cv_vid_in</code> conduit. The signal has no function within the IP and is included for connectivity within Platform Designer.
Include the vid_mode_change signal	True or False	Select True to add this signal to the <code>cv_vid_in</code> conduit. The signal has no function within the IP and is included for connectivity within Platform Designer.
Include the frame lock signals	True or False	Select True to add this signal to the <code>cv_vid_in</code> conduit. The signal has no function within the IP and is included for connectivity within Platform Designer.
Include the vid_std signal	True or False	Select True to add this signal to the <code>cv_vid_in</code> conduit. The signal has no function within the IP and is included for connectivity within Platform Designer.
Width of vid_std	1 to 16	Select the width of the <code>vid_std</code> signal.

Figure 52. CVO Parameters

The screenshot shows the configuration window for the 'Clocked Video to VVP-FR Converter Intel FPGA IP' (intel_wp_cv2fr). The 'CVO Parameters' tab is selected. Under the 'CVO Legacy Tie Offs' section, the following parameters are configured:

Parameter	Value
Drive CV output clock:	True
Include the underflow signal:	True
Include the vid_mode_change signal:	False
Include frame lock signals:	False
Include the vid_std signal:	False
Width of vid_std:	1

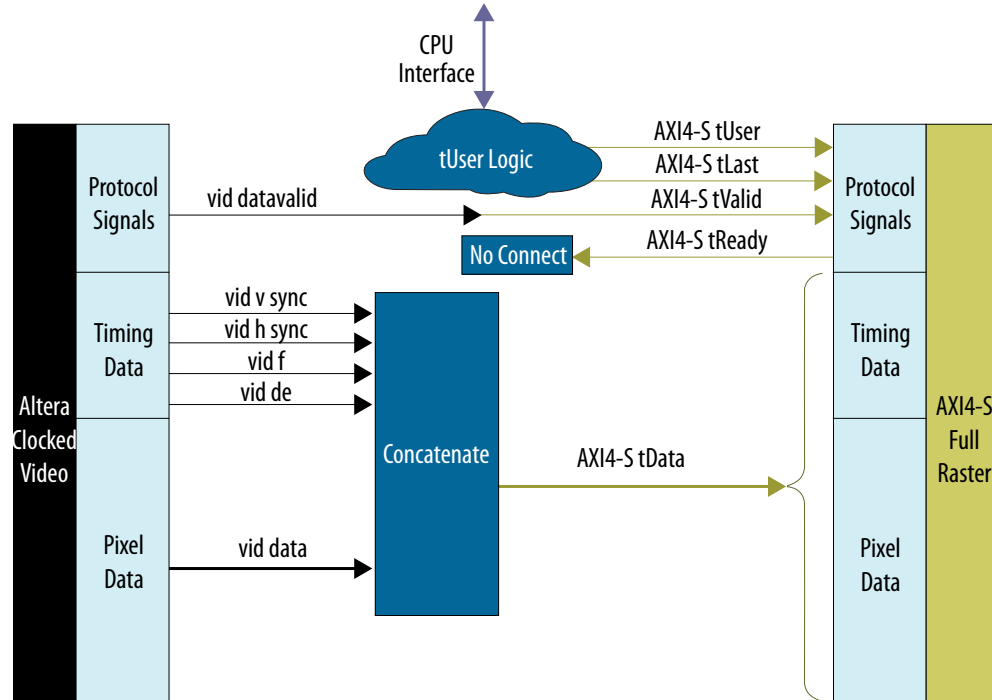
20.3. Clocked Video to Full-Raster Converter Block Description

The IP passes the pixel and timing data through unmodified. The AXI4-S based streaming full-raster bus encapsulates all the pixel and timing data on a single bus, `tData`. The clocked video bus is a bundle of multiple single wires for the individual video timing strobes, and a data bus for the pixel data.

The clocked video bus can contain additional sideband signals, such as discrete 16-bit signals for the width and height of the raster. The IP ignores these sideband signals and copies some signals CPU registers. The sideband signals provide backward IO interface compatibility between the IP and legacy Altera clocked video input and clocked video output interfaces.

Figure 53. High-level mapping from the clocked video to full-raster

The figure shows how mapping is a case of concatenating the discrete signals used by the clocked video interface into a single AXI4-S streaming full-raster tData bus.



The AXI4-S tUser signal cannot be generated automatically from the clocked video timing signals. The tUser is asserted for true pixels (0,0) in the full video raster, but the location of (0,0) relative to the timing strobes varies by video standard. Therefore, you need either a CPU interface to instruct the tUser logic where to place in the raster the tUser, or the IP is restricted to a single video standard.

20.3.1. Clocked Video to Full-Raster Converter Interfaces

The Clocked Video to Full-Raster Converter IP has three functional interfaces

- Clocked video data input interface for video IOs.
- Full-raster data output interface for video IOs
- Avalon memory-mapped CPU interface

Table 310. Clocks and Resets

Name	Direction	Width	Description
vid_clock	Input	1	When you select Lite and CVO for CV Bus style , vid_clock is the video clock for the lite and clocked video output inputs, and the streaming full-raster output. When you select CVI for CV Bus style , vid_clock is a dummy signal retained for Platform Designer connectivity. The IP uses the video clock included in the cv_vid_in conduit.
vid_reset	Input	1	Reset for vid_clock domain.

continued...

Name	Direction	Width	Description
cpu_clock	Input	1	Optional control interface clock.
cpu_reset	Input	1	Optional control interface reset.
cv_clk_out	Output	1	A copy of the video clock the IP uses.

Table 311. Control Interface

This interface is only available if you select **True** for **Memory-mapped control interface**.

Name	Direction	Width	Description
av_mm_cpu_agent_address	Input	7	Control agent port Avalon memory-mapped address bus. Specifies a word offset into the slave address space.
av_mm_cpu_agent_read	Input	1	Control agent port Avalon memory-mapped read signal. When you assert this signal, the control port drives new data onto the read data bus.
av_mm_cpu_agent_readdata	Output	32	Control agent port Avalon memory-mapped read data bus. These output lines are used for read transfers
av_mm_cpu_agent_waitrequest	Output	1	Control agent port Avalon memory-mapped wait request bus. This signal indicates that the slave is stalling the master transaction.
av_mm_cpu_agent_write	Input	1	Control agent port Avalon memory-mapped write signal. When you assert this signal, the control port accepts new data from the write data bus.
av_mm_cpu_agent_writedata	Input	32	Control agent port Avalon memory-mapped write data bus. These input lines are used for write transfers.
av_mm_cpu_agent_byteenable	Input	4	Control agent port Avalon memory-mapped byte enable bus. These lines indicate which bytes are selected for write and read transactions.

Table 312. Streaming full-raster video interface

Name	Direction	Width	Description
axi4s_fr_vid_out_tvalid	Output	1	AXI4-S full-raster data valid.
axi4s_fr_vid_out_tready	Input	1	Optional AXI4-S full-raster data ready.
axi4s_fr_vid_out_tdata	Output	⁽⁵⁵⁾	AXI4-S full-raster data in.
axi4s_fr_vid_out_tlast	Output	1	AXI4-S end of full-raster packet .
axi4s_fr_vid_out_tuser[0]	Output	⁽⁵⁶⁾	AXI4-S start of full-raster video frame.

⁽⁵⁵⁾ The equation gives all tdata widths in these interfaces:

$$\max(\text{floor}(((\text{bits per color sample} \times \text{number of color planes} + 1) \times \text{pixels in parallel}) + 7) / 8 \times 8, 16)$$

⁽⁵⁶⁾ The equation gives all tuser widths in these interfaces:

$$N = \text{ceil}(\text{tdata width} / 8)$$

Table 313. CV-Lite Streaming Video Interface

This interface is only available if you select **Lite** for **CV Bus Style**.

Name	Direction	Width	Description
cv_vid_in_h	Input	Pixels in parallel	When 1, the video is in a horizontal blanking.
cv_vid_in_v	Input	Pixels in parallel	When 1, the video is in a vertical blanking.
cv_vid_in_h_sync	Input	Pixels in parallel	When 1, the video is in a horizontal synchronization period.
cv_vid_in_v_sync	Input	Pixels in parallel	When 1, the video is in a vertical synchronization period.
cv_vid_in_f	Input	Pixels in parallel	When 1, the video is interlaced and in field 1. When 0, the video is either progressive or interlaced and in field 0.
cv_vid_in_active	Input	Pixels in parallel	When asserted, the video is in an active picture period (not horizontal or vertical blanking). Drive this signal for the correct operation of the IPs.
cv_vid_in_data	Input	(57)	Pixel data.
cv_vid_in_valid	Input	1	When 1, the input is valid.
cv_vid_in_ready	Output	1	When 1, the IP can accept new data. When 0, no new data can be accepted.

Table 314. Clocked Video Input Streaming Video Interface

This interface is only available if you select **CVI** for **CV Bus Style**.

Port Name	Direction	Width	Description
cv_vid_in_vid_clk	Input	1	The pixel synchronous clock
cv_vid_in_vid_h_sync	Input	Pixels in parallel	When 1, the video is in a horizontal blanking or synchronization period.
cv_vid_in_vid_v_sync	Input	Pixels in parallel	When 1, the video is in a vertical blanking or synchronization period.
cv_vid_in_vid_f	Input	Pixels in parallel	When 1, the video is interlaced and in field 1. When 0, the video is either progressive or interlaced and in field 0.
cv_vid_in_vid_data	Input	(58)	Pixel Data
cv_vid_in_vid_de	Input	Pixels in parallel	When asserted, the video is in an active picture period (not horizontal or vertical blanking). This signal must be driven for correct operation of the IP cores.
cv_vid_in_vid_datavalid	Input	1	When 1, the input is valid
cv_vid_in_vid_locked	Input	1	Unused legacy signal.
cv_vid_in_vid_hd_sdn	Input	1	Unused legacy signal.
cv_vid_in_vid_std	Input	User Specified	Unused legacy signal.
continued...			

(57) The equation gives the data width:
width = (bits per color sample X number of color planes X pixels in parallel)

(58) The equation gives the data width:
width = (bits per color sample X number of color planes X pixels in parallel)

cv_vid_in_vid_color_encoding	Input	8	Unused legacy signal.
cv_vid_in_vid_bit_width	Input	8	Unused legacy signal.
cv_vid_in_vid_total_sample_width	Input	16	Indicates the total (active + blanking) width of the raster
cv_vid_in_vid_total_line_count	Input	16	Indicates the total (active + blanking) height of the raster
cv_vid_in_vid_hdmi_duplication	Input	4	Unused legacy signal.
cv_vid_in_sof	Output	1	Unused legacy signal.
cv_vid_in_sof_locked	Output	1	Unused legacy signal.
cv_vid_in_refclk_div	Output	1	Unused legacy signal.
cv_vid_in_clipping	Output	1	Unused legacy signal.
cv_vid_in_padding	Output	1	Unused legacy signal.
cv_vid_in_overflow	Output	1	Unused legacy signal.

Table 315. Clocked Video Output Streaming Video Interface

This interface is only available if you select **CVO** for **CV Bus Style**.

Port Name	Direction	Width	Description
cv_vid_in_vid_clk	Output	1	Pixel synchronous clock.
cv_vid_in_vid_h	Input	Pixels in parallel	When 1, the video is in a horizontal blanking.
cv_vid_in_vid_v	Input	Pixels in parallel	When 1, the video is in a vertical blanking.
cv_vid_in_vid_h_sync	Input	Pixels in parallel	When 1, the video is in a horizontal synchronization period.
cv_vid_in_vid_v_sync	Input	Pixels in parallel	When 1, the video is in a vertical synchronization period.
cv_vid_in_vid_f	Input	Pixels in parallel	When 1, the video is interlaced and in field 1. When 0, the video is either progressive or interlaced and in field 0.
cv_vid_in_vid_data	Input	(59)	Pixel data.
cv_vid_in_vid_datavalid	Input	Pixels in parallel	When 1, the input is valid.
cv_vid_in_vid_underflow	Input	1	Unused legacy signal.
cv_vid_in_vid_mode_change	Input	1	Unused legacy signal.
cv_vid_in_vid_vcoclck_div	Input	1	Unused legacy signal.
cv_vid_in_vid_sof_locked	Input	1	Unused legacy signal.
cv_vid_in_vid_sof	Input	1	Unused legacy signal.
cv_vid_in_vid_std	Input	6	Unused legacy signal.

Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

- (59) The equation gives the data width:
width = (bits per color sample X number of color planes X pixels in parallel)

20.4. Clocked Video Converter to Full-Raster IP Registers

The IP allows runtime configuration of parameters using the Avalon memory-mapped processor register interface. Unless stated, all registers are 32-bit wide.

Table 316. Processor Register Description

Common registers			
Register	Offset	Access	Description
Reg_HV_Pos	0x140	RW	Specify the H and V position of the rising edge of the vsync pulse occurs.
Reg_Total_HV	0x144	RO	Shows the IP's total width and total height, including active pixels and blanking.
Clocked video input specific registers			
Reg_CVI_Legacy_0	0x148	RW	Drives legacy clocked video input conduit output signals and returns the current values.
Reg_CVI_Legacy_1	0x14C	RO	Drives legacy clocked video input conduit output signals and returns the current values.
Clocked video output specific registers			
Reg_CVO_Legacy_0	0x150	RO	The current value of the clocked video output conduit sideband signals vid_sof.

Table 317. Reg_HV_Pos

Name	Bits	Attribute	Description
H Position	15:0	RW	Specify the video pixel on which the rising edge of the vsync pulse occurs
V Position	31:16	RW	Specify the video line on which the rising edge of the vsync pulse occurs is the reset bit

Table 318. Reg_Total_HV

Name	Bits	Attribute	Description
Total Pixels	15:0	RO	Shows the IP's width of the video line, including active pixels and horizontal blanking.
Total Lines	31:16	RO	Shows the IP's height of the video, including active lines and vertical blanking.

Table 319. Reg_CVI_Legacy_0

Name	Bits	Attribute	Description
CVI SOF	0	RW	Drives legacy clocked video input conduit signal sof.
CVI SOF Locked	1	RW	Drives legacy clocked video input conduit signal sof_locked.
CVI Overflow	2	RW	Drives legacy clocked video input conduit signal overflow.
CVI Clipping	3	RW	Drives legacy clocked video input conduit signal clipping.
CVI Padding	4	RW	Drives legacy clocked video input conduit signal padding.
CVI refclk_div	5	RW	Drives legacy clocked video input conduit signal refclk_div.
Reserved	7:6	-	Reserved.
CVI video locked	8	RO	The current value of the clocked video input legacy signal vid_locked.
continued...			

Name	Bits	Attribute	Description
Reserved	15:9	-	Reserved.
CVI color encoding	23:16	RO	The current value of the clocked video input legacy signal <code>vid_color_encoding</code> .
CVI bit width	31:24	RO	The current value of the clocked video input legacy signal <code>vid_bit_width</code> .

Table 320. Reg_clocked video input_Legacy_1

Name	Bits	Attribute	Description
CVI vid std	Width of <code>vid_std</code> -1:0	RO	The current value of the clocked video input legacy signal <code>vid_std</code> .
CVI HDMI duplication	19:16	RO	The current value of the clocked video input signal <code>vid_hdmi_duplication</code> .
Reserved	23:20	-	Reserved.
CVI HD not SD	24	RO	The current value of the clocked video input signal <code>vid_hd_sdn</code> .
Reserved	31:25	-	Reserved.

Table 321. Reg_CVO_Legacy_0

Name	Bits	Attribute	Description
CVO SOF	0	RO	The value of the input legacy clocked video output conduit signal <code>vid_sof</code> .
CVO SOF Locked	1	RO	The value of the input legacy clocked video output conduit signal <code>vid_sof_locked</code> .
CVO Underflow	2	RO	The value of the input legacy clocked video output conduit signal <code>underflow</code> .
CVO vco clock divide	3	RO	The value of the input legacy clocked video output conduit signal <code>vid_vcoclck_div</code> .
CVO mode change	4	RO	The value of the input legacy clocked video output conduit signal <code>vid_mode_change</code> .
Reserved	15:5	-	Reserved.
CVO video standard	Width of <code>vid_std</code> +15:16	RO	The value of the input legacy clocked video output conduit signal <code>vid_std</code> .



21. Clocked Video Output IP

21.1. About the Clocked Video Output IP

The Clocked Video Output IP merges the pixel data from an a lite or full variant of the Altera Streaming Video Protocol with the real-time video signals from a reference full-raster stream. The output is a full-raster bus that you can connect directly to an Altera Protocol IP.

Another IP can provide the reference full-raster stream or an optional internal Video Timing Generator IP can generate the reference stream.

You can select the optional internal video timing generator from the IP GUI. It is a version of the video timing generator IP that provides the same features and functionalities (refer to *Video Timing Generator IP*).

Second Input Fall Back

The IP provides an optional second video streaming input. If the primary input stream fails, the IP automatically changes to use the second input. If the second input also fails, or is not used, the IP produces black. You can configure the exact pixel value for black at build time and via the processor interface at runtime.

By default, when the primary video input relocks to the timing reference the IP automatically changes back to the primary video input at the start of the next frame. However, you can change the default behavior via the processor interface. Using the processor interface, you can manually control when the IP changes from the test pattern input back to the video input. You can force the IP to select either the main video input, the test pattern input, or black.

Pixels in Parallel Support

The IP supports any number of pixels in parallel from 1 to 8. The raster dimensions have no restrictions on the number of pixels in parallel. The raster width does not have to be an integer multiple of pixels in parallel.

The pixel input stage contains a barrel shift to align the lite or full streams with the full-raster stream.

Related Information

- [Video Timing Generator IP](#) on page 970
- [Altera Protocol IPs](#)

21.1.1. Clocked Video Output IP Performance and Resources

Altera provides resource and utilization data for guidance. The Clocked Video Output IP resource usage depends on the device family, number of supported bits per color sample, pixels in parallel, number of color planes.

Table 322. Clocked Video Output IP Performance and Resources

For an example parameterization of 10 bps, 3 color planes, 2 pixels in parallel and non-aligned timing data.

Target Device	ALMs	M20Ks	f _{MAX} (MHz)
Agilex 7 (AGFA012R24A2E2V)	1,019	14	738
Arria 10 (10AS066H1F34E1HG)	968	14	609
Cyclone 10 GX (10CX220YF672E5G)	963	14	560
Stratix 10 (1SX280LN2F43E1VG)	1,197	14	579

21.1.2. Clocked Video Output IP Features

- Altera Streaming Video full-raster compliant video output interface
- Optional TREADY conduit on the streaming full-raster interface
- Optional processor interface for control and diagnostics.
- 8 to 16 bits per color component
- 1, 2, 3 or 4 color planes per pixel
- 1 to 8 pixels in parallel

21.2. Clocked Video Output IP Parameters

The IP offers compile-time parameters.

Table 323. CVO Configuration Parameters

These parameters configure the behavior of the IP at build time. You can only change them by recompiling the IP.

Parameter	Allowed Range	Description
Video Settings		
Number of pixels in parallel	1 to 8	Select the number of pixels in parallel
Number of color planes	1 to 4	Select the number of color planes per pixel
Bits per color sample	8 to 16	Select the number of bits per color sample
Default black for merge, color plane 0	0 to 65535	The initial value of black for this color plane
Default black for merge, color plane 1	0 to 65535	The initial value of black for this color plane
Default black for merge, color plane 2	0 to 65535	The initial value of black for this color plane
Default black for merge, color plane 3	0 to 65535	The initial value of black for this color plane
<i>continued...</i>		

Parameter	Allowed Range	Description
AXI4-S FR interface TREADY	True or False	Select True to include the tReady signal in the full-raster interface Select False to remove the tReady signal
Source Settings		
Internal Timing Generator	True or False	If True , the IP includes the Video Timing Generator IP. If False , the IP provides a full-raster stream input bus for an external video timing source.
Async Video Input	True or False	If True , the IP sees the video input as asynchronous to the full-raster stream output. If False , the video input and full-raster stream output must use the same clock.
Depth of Video Input FIFOs	0, 512, 1024, 2048, 4096	The approximate size of the FIFO buffer to use for the video input path. If set to 0, the IP registers the path internally to improve timing performance.
Test Pattern Input	True or False	If True , an additional lite or full stream input bus is generated. If the primary input bus fails for any reason, this additional input is used. If False , the IP generates no additional lite or full stream input buses. If the primary input bus fails for any reason, the output goes to black.
Async Test Pattern Input	True or False	If True , the TPG input is asynchronous to the full raster stream output. If False , the test pattern generator input and full-raster stream output must use the same clock.
Depth of TPG Input FIFOs	0, 512, 1024, 2048, 4096	The approximate size of the FIFO buffer to use for the test pattern generator input path. If set to 0, the IP registers the path internally to improve timing performance.
Control Settings		
Memory-Mapped Control Interface	True or False	Select True to turn on the processor interface and associated signals. When False , the IP removes the processor interface. All processor registers use default values
Frequency of CPU Clock⁽⁶⁰⁾	1 to 1000000000	The frequency, in Hz, of the processor clock.
Debug Settings		
Timing Dimensions	True or False	Select True to turn on the timing dimension processor registers. Additional logic is generated to measure the active and total widths and heights of the timing input, the full raster clock frequency, and the full raster frame period. Select False , and the IP generates no additional logic. The associated processor registers return 0x1234abcd.
Video Dimensions	True or False	Select True to turn on the video dimension processor registers. The IP generates additional logic to measure the active width and height of the video input, the video clock frequency, and the video input frame period. Select False , and the IP generates no additional logic. The associated processor registers return 0x1234abcd.
Video Merge Status	True or False	Select True to turn on diagnostic counters that report the status of the video merge process.
<i>continued...</i>		

⁽⁶⁰⁾ This parameter is only available when **Memory-Mapped Control Interface** is **True**

Parameter	Allowed Range	Description
		Select False , and the IP generates no additional logic. The associated processor registers return 0x1234abcd.
Test Pattern Dimensions	True or False	Select True to turn on the test pattern generator dimension processor registers. The IP generates additional logic to measure the active width and height of the test pattern generator input, the test pattern generator clock frequency, and the test pattern generator input frame period Select False , and the IP generates no additional logic. The associated processor registers return 0x1234abcd.
Test Pattern Merge Status	True or False	Select True to turn on diagnostic counters that report the status of the test pattern generator merge process. Select False , and the IP generates no additional logic. The associated processor registers return 0x1234abcd.

Figure 54. Clocked video Output IP Configuration Parameters GUI

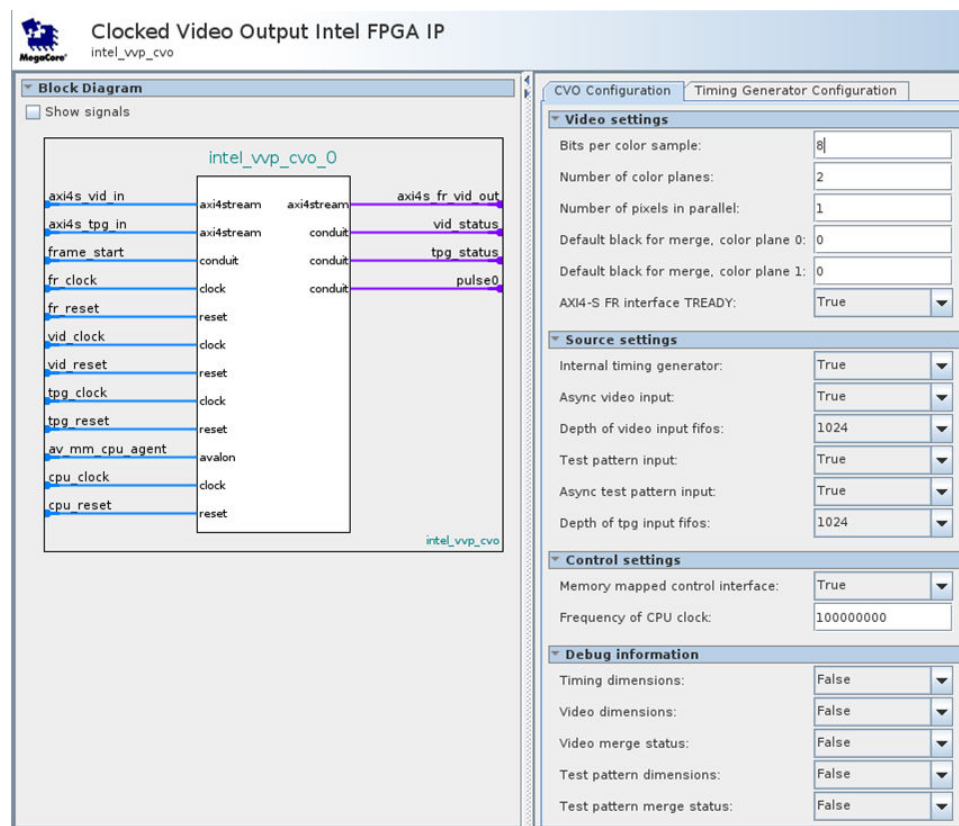


Table 324. Internal Timing Generator Build Parameters

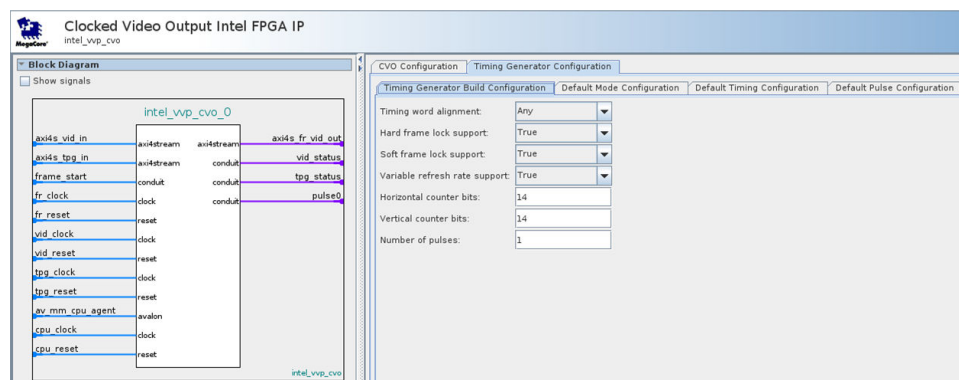
The Timing Generator Configuration GUI has four tabs. The **Build Parameters** tab contains all the build-time options appropriate to the Timing Generator when built as part of the IP. You cannot change these parameters once you build the IP. Any change requires a full rebuild of the IP.

The **Default Mode Configuration**, **Default Timing Configuration**, and **Default Pulse Configuration** parameters are part of the Video Timing Generator IP. Refer to the Video Timing Generator IP Parameters.

The parameters on the default tabs apply to the IP when the IP is reset and you can change them during run time of the IP via the processor interface, without recompiling the IP.

Parameters	Allowed Range	Description
Timing Word Alignment	Any or PIP-Aligned Only	Select Any for no restrictions on raster dimensions versus the number of pixels in parallel. When PIP-Aligned Only , all timing parameters must be integer multiples of the pixels In parallel value.
Hard frame lock support	True or False	Select True to turn on the hard frame lock support. When False , the IP removes all frame lock support.
Soft frame lock support	True or False	Select True to enable soft frame lock support. (61) When False , the IP removes soft frame lock support.
Variable refresh rate support	True or False	Select True to turn on variable refresh rate support When False , the IP removes support for variable refresh rate
Horizontal counter bits	4-16	The number of binary bits required to represent the maximum width of raster. For example, for a 4096 wide raster, set to 13.
Vertical counter bits	4-16	The number of binary bits required to represent the maximum height of raster. For example, for a 2048 high raster, set to 12.
Number of pulses	0-8	The number of additional general-purpose pulses that the IP can produce. Each additional pulse increases the gate count of this IP. If set to 0, the Default Pulse Configuration GUI turns off.

Figure 55. Clocked Video Output IP Timing Parameters GUI



(61) Hard Frame Lock must also be **True** for correct operation of soft frame lock

21.3. Clocked Video Output IP Functional Description

The IP includes a merge block, which monitors the timing bus. It does not push back on the timing bus. The merge block stalls the video pixel data bus and test pattern generator pixel data bus when it sees the start of frame on the pixel data bus. The IP releases the pixel data buses when the timing bus contains the start of active pixel data.

Diagnostics report the status of the merge block via discreet output pins and the processor Interface.

Figure 56. External Timing Reference Bus

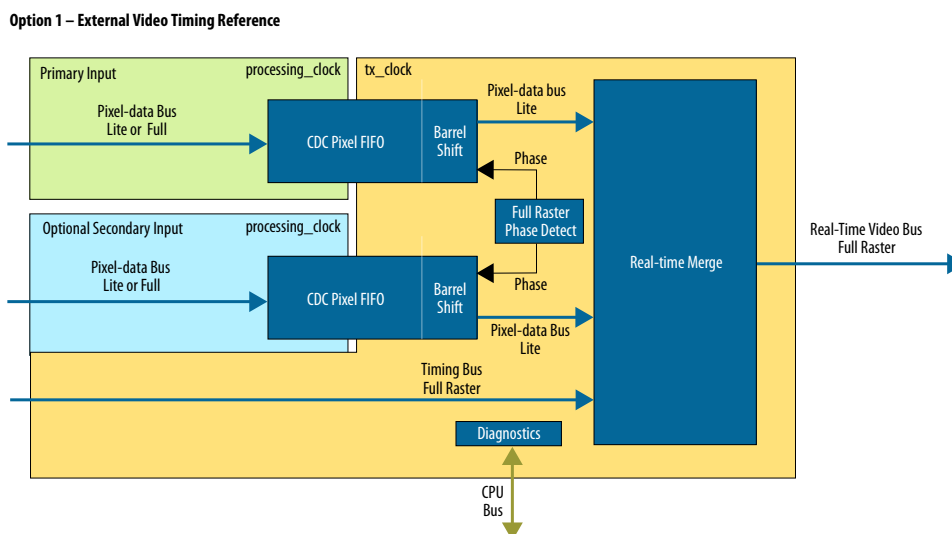
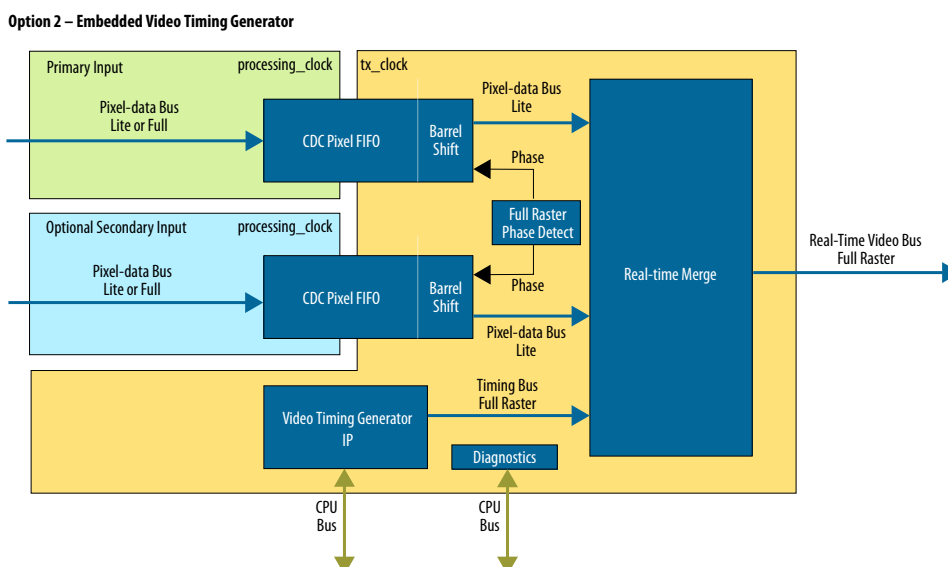


Figure 57. Embedded Video Timing Generator



The IP has a build-time option to include or exclude the `tReady` signal for the full-raster interface. The IP does not use this signal, but it allows you to connect to a full-raster bus that includes this signal. If the `tReady` signal is deasserted, the Clocked Video Output IP continues to output data.

The Clocked Video Output IP produces output on the transmit clock for the connectivity IP.

The processor interface operates on the processor clock domain. Altera recommends that you drive the processor Interface from a known stable clock, such as a dedicated processor clock. Do not drive with the transmit clock for the connectivity IP, which can be unstable. For example when standards change, the transmit clock can be unstable. If you use the transmit clock to clock the processor interface it can potentially corrupt the processor interface.

21.4. Clocked Video Output IP Interfaces

The IP has up to three video input interfaces:

- An AXI4-S video input for the primary pixel data input, `axi4s_vid_in`
- An optional AXI4-S video Input for the test pattern generator pixel data input, `axi4s_tpg_in`
- An optional AXI4-S full-raster input for the real-time video raster data, `axi4s_fr_timing_in`

The IP has one video output interface, an AXI4-S full-raster bus, `axi4s_fr_vid_out`

The IP has one status video interface, `vid_status`.

The IP has one optional CPU interface, `av_mm_cpu_agent`.

The IP has one optional TPG status interface, `tpg_status`.

The IP has nine optional internal video timing generators, `frame_start` and `pulse_0` to `pulse_7`.

The CPU interface is asynchronous to the video output interface. Assume the video clock can be unstable when you select a new standard, which can cause unreliable behavior if used for the CPU interface.

The proprietary Altera streaming full-raster protocol is compatible with AMBA AXI4-stream interfaces to connect components that exchange video data. The protocols allow interfaces to Altera video IPs or other AXI4-Stream compliant third-party video IPs. Table 4 provides a description for each of the conduits on the output and input interfaces.

The video clock and CPU clock are assumed to be asynchronous to each other. Internally, the Clocked Video Output IP includes clock domain crossing (CDC) circuits for both single bit and data bus signal cases, which allows safe data exchange between the two asynchronous clock domains. The Clocked Video Output IP also

includes an embedded entity `.sdc` file, which provides all the necessary information to the Timing Analyzer. For system integration, when you instantiate the Clocked Video Output IP in a design, the only constraints required are:

- Clock frequency constraints for the timing reference input clock (`fr_clock_clk`)
- Clock frequency constraints for the primary video clock (`vid_clock_clk`)
- Clock frequency constraints for the test pattern input video clock (`tpg_clock_clk`)
- Clock frequency constraints for the CPU clock (`cpu_clock_clk`)

Table 325. Clocked Video Output IP Interfaces

The table shows a description for each of the conduits on the output and input interfaces

Signal name	Direction	Width	Description
Clocks and resets			
<code>vid_clock_clk</code>	Input	1	Input AXI4-S video main video input processing clock
<code>vid_reset_reset</code>	Input	1	Input AXI4-S video main video processing reset
<code>tpg_clock_clk</code>	Input	1	Input AXI4-S video test pattern input processing clock
<code>tpg_reset_reset</code>	Input	1	Input AXI4-S video test pattern input processing reset
<code>fr_clock_clk</code>	Input	1	Processing clock for input and output AXI4-S full-raster interfaces.
<code>fr_reset_reset</code>	Input	1	Reset for input and output AXI4-S full-raster interfaces.
<code>cpu_clock_clk</code>	Input	1	Processor interface processing clock
<code>cpu_reset_reset</code>	Input	1	ProcessorInterface processing reset
Control Interface			
This interface is only available if you turn on CPU support.			
<code>av_mm_cpu_agent_address</code>	Input	7	Control agent port Avalon memory-mapped address bus. Specifies a word offset into the agent address space.
<code>av_mm_cpu_agent_read</code>	Input	1	Control agent port Avalon memory-mapped read signal. When you assert this signal, the control port drives new data onto the read data bus.
<code>av_mm_cpu_agent_read_data_valid</code>	Output	1	Control agent port Avalon memory-mapped read data valid signal. The IP asserts this signal on the same clock cycle when the read data is valid.
<code>av_mm_cpu_agent_readdata</code>	Output	32	Control agent port Avalon memory-mapped read data bus. These output lines are for read transfers
<code>av_mm_cpu_agent_waitrequest</code>	Output	1	Control agent port Avalon memory-mapped wait request bus. This signal indicates that the agent is stalling the master transaction.
<code>av_mm_cpu_agent_write</code>	Input	1	Control agent port Avalon memory-mapped write signal. When you assert this signal, the control port accepts new data from the write data bus.
<code>av_mm_cpu_agent_writedata</code>	Input	32	Control agent port Avalon memory-mapped write data bus. These input lines are for writing transfers.
<i>continued...</i>			

Signal name	Direction	Width	Description
av_mm_cpu_agent_byteenable	Input	4	Control agent port Avalon memory-mapped byte enable bus. These lines indicate which bytes are selected for write and read transactions.
Altera Streaming Video interfaces			
axi4s_fr_vid_out_tdata	Output	(62)(63)(64)	AXI4-S full-raster data out
axi4s_fr_vid_out_tvalid	Output	1	AXI4-S data valid
axi4s_fr_vid_out_tuser[0]	Output	1	AXI4-S start of video frame
axi4s_fr_vid_out_tlast	Output	1	AXI4-S end of packet
axi4s_fr_vid_out_tready	Input	1	AXI4-S data ready
axi4s_fr_timing_in_tdata	Input	(62)(63)(64)	AXI4-S full-raster timing data in
axi4s_fr_timing_in_tvalid	Input	1	AXI4-S data valid
axi4s_fr_timing_in_tuser[0]	Input	1	AXI4-S start of video frame
axi4s_fr_timing_in_tlast	Input	1	AXI4-S end of packet
axi4s_fr_timing_in_tready	Output	1	AXI4-S data ready
axi4s_vid_in_tdata	Output	(62)(63)(64)	AXI4-S data in
axi4s_vid_in_tvalid	Output	1	AXI4-S data valid
axi4s_vid_in_tuser[0]	Output	1	AXI4-S start of video frame
axi4s_vid_in_tlast	Output	1	AXI4-S end of packet
axi4s_vid_in_tready	Input	1	AXI4-S data ready
axi4s_tpg_in_tdata	Output	(62)(63)(64)	AXI4-S data in
axi4s_tpg_in_tvalid	Output	1	AXI4-S data valid
axi4s_tpg_in_tuser[0]	Output	1	AXI4-S start of video frame
axi4s_tpg_in_tlast	Output	1	AXI4-S end of packet
axi4s_tpg_in_tready	Input	1	AXI4-S data ready
Video Status			
vid_locked	Output	1	When '1', the IP locks the video input with the timing reference.
continued...			

(62) The equation gives all full-raster tdata widths:

$\max(\text{floor}(((\text{bits per color sample} \times (\text{number of color planes} + 1) \times \text{pixels in parallel}) + 7) / 8) \times 8, 16)$

(63) The equation gives all tdata video active only sizes:

$\max(\text{floor}(((\text{bits per color sample} \times \text{number of color planes} \times \text{pixels in parallel}) + 7) / 8) \times 8, 16)$

(64) The equation gives all tuser widths in these interfaces

$N = \text{ceil}(\text{tdata width} / 8)$

Signal name	Direction	Width	Description
			When '0', the IP waits to lock the video input and the timing reference.
vid_is_full	Output	1	When '1', the IP detects the video input is the full variant of the Altera Streaming Video Protocol. When '0', the IP detects the video input is the lite variant of the Altera Streaming Video Protocol.
vid_stall_error	Output	1	When '1', the video input stalls (drops its <code>tValid</code>) when the timing reference requires active pixels. When '0', the video input supplies pixels at the required pixel rate.
vid_size_mismatch	Output	1	When '1', the active dimensions of the video input do not match the active dimensions of the timing reference. When '0', the active dimensions of the video input and the timing reference match.
TPG status This interface is only available if you select True for Internal Timing Generator .			
tpg_locked	Output	1	When '1', the test pattern input locks with the timing reference. When '0', the IP is waits to lock the test pattern input and the timing reference.
tpg_is_full	Output	1	When '1', the IP detects the test pattern input is the full variant of the Altera Streaming Video Protocol. When '0', the IP has detects the test pattern input is the lite variant of the Altera Streaming Video Protocol..
tpg_stall_error	Output	1	When '1', the test pattern input stalls (drops its <code>TValid</code>) when the timing reference requires active pixels. When '0', the test pattern input supplies pixels at the required pixel rate.
tpg_size_mismatch	Output	1	When '1', the active dimensions of the test pattern input do not match the active dimensions of the timing reference. When '0', the active dimensions of the test pattern input and the timing reference match.
<i>continued...</i>			

Signal name	Direction	Width	Description
Internal Timing Generator Conduits This interface is only available if you select True for Internal Timing Generator .			
frame_start	Input	1	The internal video timing generator uses this signal to indicate the start of frame. Depending on the generator Frame lock mode , the IP ignores the frame_start signal i.e., freerunning (default configuration) If you select Pulse for Frame start signal type , the IP processes the frame start input signal as a pulse. The IP uses the rising edge of the signal to indicate the start of a frame. If you select Toggle for Frame start signal type , the IP processes the frame start input signal as a toggle and the IP uses both edges of the signal to indicate the start of a frame.
Pulse	Output	1	The IP can produce up to 8 additional outputs that each provide a pulse or a toggle once per frame. You can program these general-purpose signals to occur at any fixed point in the raster. Separate parameters specify the first and last pixel of the pulse. If the start and end pixels are the same, the IP generates a single clock pulse. If the end pixel is outside the defined raster, the signal becomes a once-per-frame toggle.

21.5. Clocked Video Output IP Registers

The Clocked Video Output IP allows runtime configuration of parameters via Avalon memory-mapped processor register interface. The processor interface is optional. You turn it on at build time in the **CVO Configuration** GUI. Unless stated, all registers are 32-bit wide.

The address space is split into two regions. Addresses 0x140-0x194 are reserved for the core Clocked Video Output IP registers. Addresses 0x240-0x2FC are reserved for the optional internal Video Timing Generator. If you use the external AXI4-S full-raster input, registers 0x240-0x2FC are read only and return 0x1234abcd.

Table 326. Parameterization Registers

Register	Offset	Access	Description
Clocked Video Output Parameterization Registers			
VID_PID	0x000	RO	Read this register to retrieve clocked video output IP product ID
VERSION	0x004	RO	Read this register to retrieve the version information for the Clocked Video Output IP
VID_FIFO_DEPTH	0x008	RO	The depth of the video input cdc fifo.
TPG_FIFO_DEPTH	0x00c	RO	The depth of the test pattern input cdc fifo.
BPS	0x010	RO	The bits per video sample
INTERNAL_VTIMING	0x014	RO	If '1', the IP includes an internal video timing generator. If '0', and external video timing generator is required.
FALL_BACK_INPUT_EN	0x018	RO	If '1', the second video input has been enabled.
PIXELS_IN_PARALLEL	0x01c	RO	The number of pixels per clock transferred on the AXI4-S busses.
<i>continued...</i>			

Register	Offset	Access	Description
NUMBER_OF_COLOR_PLANES	0x020	RO	The number of color planes per pixel
SEPARATE_SLAVE_CLOCK	0x024	RO	This returns '1' because the processor interface always uses a separate clock.
CPU_CLK_FREQ_HZ	0x028	RO	The frequency, in Hz, of the processor interface clock.
TIM_DIMENSIONS	0x02c	RO	If set to '1', the processor Interface diagnostics include the dimensions of the input timing reference. If 0, the dimensions of the timing reference are not available to the processor interface.
VID_IS_ASYNC	0x030	RO	If 1, the video input is treated as asynchronous to the timing reference. If 0, the video input must use the AXI4-S full-raster video clock
VID_DIMENSIONS	0x034	RO	If 1, the processor interface diagnostics include the dimensions of the video input. If 0, the dimensions of the video input are not available to the processor interface.
VID_DEBUG	0x038	RO	If 1, the processor Interface diagnostics include the <code>locked</code> , <code>size err</code> , and <code>stall</code> counters for the video input. If 0, the counters are not available to the processor Interface
TPG_IS_ASYNC	0x03c	RO	If 1, the test pattern input is treated as asynchronous to the timing reference. If 0, the test pattern input must use the AXI4-S full-raster video clock
TPG_DIMENSIONS	0x040	RO	If 1, the processor interface diagnostics include the dimensions of the test pattern input. If 0, the dimensions of the test pattern input are not available to the processor interface.
TPG_DEBUG	0x044	RO	If 1, the processor Interface diagnostics include the <code>locked</code> , <code>size err</code> , and <code>stall</code> counters for the test pattern input. If 0, the counters are not available to the processor Interface
Video Timing Generator Parameterization Registers These registers only exist if register 0x014 "INTERNAL_VTIMING" returns True .			
NUM_PULSES	0x048	RO	The number of additional output pulses generated by the internal Video Timing Generator.
HSIZE	0x04c	RO	The number of bits used to build the horizontal counters and comparators inside the Video Timing Generator. This determines the maximum width of raster that can be generated.
VSIZE	0x050	RO	The number of bits used to build the vertical counters and comparators inside the Video Timing Generator. This determines the maximum height of raster that can be generated.
BUILD_HARD_FRAME_LOCK	0x054	RO	If true, the internal Video Timing Generator includes the Hard Frame Lock function. If false, the internal Video Timing Generator cannot "hard lock" to an external reference.
BUILD_SOFT_FRAME_LOCK	0x058	RO	If true, the internal Video Timing Generator includes the Soft Frame Lock function. If false, the internal Video Timing Generator cannot "soft lock" to an external reference.
BUILD_VRR	0x05c	RO	If true, the internal Video Timing Generator includes the Variable Refresh Rate. If false, the internal Video Timing Generator cannot generate a variable refresh rate.

Table 327. Clocked Video Output Core Registers

The core registers control the operation of the Clocked Video Output IP. They include optional diagnostic registers that assist with system development.

Register	Offset	Access	Description
REG_STATUS	0x140	RO	Returns the status of the merge process.
REG_BLACK_0	0x144	RW	The initial value of "black" for this color plane
REG_BLACK_1	0x148	RW ⁽⁶⁵⁾	The initial value of "black" for this color plane
REG_BLACK_2	0x14c	RW ⁽⁶⁵⁾	The initial value of "black" for this color plane
REG_BLACK_3	0x150	RW ⁽⁶⁵⁾	The initial value of "black" for this color plane
REG_FALLBACK	0x154	RW	Defines the behavior of the merge block if the video input fails
REG_FR_H_DIMS	0x158	RO ⁽⁶⁶⁾	The active and total widths of the AXI4-S full-raster Timing Input
REG_FR_V_DIMS	0x15c	RO ⁽⁶⁶⁾	The active and total heights of the AXI4-S full-raster Timing Input
REG_FR_FULL_RASTREQ_DIMS	0x160	RO ⁽⁶⁶⁾	The frequency, in Hz, of the AXI4-S full-raster clock
REG_FR_CLKS_DIMS	0x164	RO ⁽⁶⁶⁾	The number of AXI4-S full-raster clocks per input frame.
REG_VID_DIMS	0x168	RO ⁽⁶⁶⁾	The active height and width of the video input.
REG_VID_FREQ_DIMS	0x16c	RO ⁽⁶⁶⁾	The frequency, in Hz, of the video input clock
REG_VID_CLKS_DIMS	0x170	RO ⁽⁶⁶⁾	The number of video input clocks per input frame.
REG_TPG_DIMS	0x174	RO ⁽⁶⁶⁾	The active height and width of the test pattern input.
REG_TPG_FREQ_DIMS	0x178	RO ⁽⁶⁶⁾	The frequency, in Hz, of the test pattern input clock
REG_TPG_CLKS_DIMS	0x17c	RO ⁽⁶⁶⁾	The number of test pattern input clocks per input frame.
REG_VID_LOCKED_COUNT	0x180	RO ⁽⁶⁶⁾	Diagnostic counter that increments once at the start of each locked video frame.
REG_VID_SIZE_ERR_COUNT	0x184	RO ⁽⁶⁶⁾	Diagnostic counter that increments every time a mismatch between the video input and timing input is detected.
REG_VID_STALL_COUNT	0x188	RO ⁽⁶⁶⁾	Diagnostic counter that increments every time the video input stalls and is dropped by the merge block.
REG_TPG_LOCKED_COUNT	0x18c	RO ⁽⁶⁶⁾	Diagnostic counter that increments once at the start of each locked test pattern frame.
REG_TPG_SIZE_ERR_COUNT	0x190	RO ⁽⁶⁶⁾	Diagnostic counter that increments every time a mismatch between the test pattern input and timing input is detected.
REG_TPG_STALL_COUNT	0x194	RO ⁽⁶⁶⁾	Diagnostic counter that increments every time the test pattern input stalls and is dropped by the merge block.

⁽⁶⁵⁾ This register is RO and returns 0x1234abcd if the color plane does not exist

⁽⁶⁶⁾ This register is optional. If turned off on the CVO Configuration GUI, this register returns 0x1234abcd

Register Bit Descriptions

Table 328. REG_STATUS

Name	Bits	Attribute	Description
VID_LOCKED	0	RO	When '1', the video input is locked with the timing reference When '0', the IP is waiting to achieve "lock" between the video input and the timing reference.
VID_AXI4-S_FULL	1	RO	When '1', the IP has detected the video input is an AXI4-S Full stream When '0', the IP has detected the video input is an AXI4-S Lite stream
VID_STALL_ERR	2	RO	When '1', the video input has stalled (dropped it's tValid) when the timing reference has required active pixels. When '0', the video input is supplying pixels at the required pixel rate.
VID_SIZE_ERR	3	RO	When '1', the active dimensions of the video input do not match the active dimensions of the timing reference. When '0', the active dimensions of the video input and the timing reference match.
TPG_LOCKED	4	RO	When '1', the test pattern input is locked with the timing reference When '0', the IP is waiting to achieve "lock" between the test pattern input and the timing reference.
TPG_AXI4-S_FULL	5	RO	When '1', the IP has detected the test pattern input is an AXI4-S Full stream When '0', the IP has detected the test pattern input is an AXI4-S Lite stream
TPG_STALL_ERR	6	RO	When '1', the test pattern input has stalled (dropped it's tValid) when the timing reference has required active pixels. When '0', the test pattern input is supplying pixels at the required pixel rate.
TPG_SIZE_ERR	7	RO	When '1', the active dimensions of the test pattern input do not match the active dimensions of the timing reference. When '0', the active dimensions of the test pattern input and the timing reference match.
TIMING_PHASE	11:8	RO	If there are 2 or more Pixels in Parallel, this indicates where the first active pixel occurred for the current line.
Reserved	31:12	Rsvd	Reserved

Table 329. REG_BLACK_0

Name	Bits	Attribute	Description
BLACK_0	BIT_DEPTH-1:0	RW	The initial value of "black" for this color plane

Table 330. REG_BLACK_1

Name	Bits	Attribute	Description
BLACK_1	BIT_DEPTH-1:0	RW	The initial value of "black" for this color plane This register is RO and returns 0x1234abcd if the color plane does not exist

Table 331. REG_BLACK_2

Name	Bits	Attribute	Description
BLACK_2	BIT_DEPTH-1:0	RW	The initial value of "black" for this color plane This register is RO and returns 0x1234abcd if the color plane does not exist

Table 332. REG_BLACK_3

Name	Bits	Attribute	Description
BLACK_3	BIT_DEPTH-1:0	RW	The initial value of "black" for this color plane This register is RO and returns 0x1234abcd if the color plane does not exist

Table 333. REG_FALLBACK

Name	Bits	Attribute	Description
Auto Recover	0	RW	If the input video is dropped due to an error, then when this bit is '1' the IP automatically returns to the video input once it is relocked If '0', the IP does not automatically return to the video input following an error, and you must manually switch using the "recover now" register bit.
Force Black	1	RW	If '1', the IP is forced to output the Clocked Video Output IP Black values.
Force TPG	2	RW	If '1', the IP is forced to output the Test Pattern video input.
Force Video	3	RW	If '1', the IP is forced to output the video input.
Reserved	30:4	Rsvd	Reserved
Recover Now	31	W1R0	This register bit self-clears. Write a '1' to this bit to force the IP to return to the video input if the video input is valid. Writing a '0' has no effect.

Table 334. REG_FR_H_DIMS

Name	Bits	Attribute	Description
hTotal	15:0	RO	Returns the detected total width of the timing reference.
hActive	31:16	RO	Returns the detected active width of the timing reference.

Table 335. REG_FR_V_DIMS

Name	Bits	Attribute	Description
vTotal	15:0	RO	Returns the detected total height of the timing reference.
vActive	31:16	RO	Returns the detected active height of the timing reference.

Table 336. REG_FR_FREQ_DIMS

Name	Bits	Attribute	Description
Clock Frequency	31:0	RO	The frequency, in Hz, of the AXI4-S full-raster clock

Table 337. REG_FR_CLKS_DIMS

Name	Bits	Attribute	Description
Frame Period	31:0	RO	The number of AXI4-S full-raster clocks per input frame.

Table 338. REG_VID_DIMS

Name	Bits	Attribute	Description
hActive	15:0	RO	The detected active width of the video input.
vActive	31:16	RO	The detected active height of the video input

Table 339. REG_VID_FREQ_DIMS

Name	Bits	Attribute	Description
Clock Frequency	31:0	RO	The frequency, in Hz, of the video input clock

Table 340. REG_VID_CLKS_DIMS

Name	Bits	Attribute	Description
Frame Period	31:0	RO	The number of video input clocks per input frame.

Table 341. REG_TPG_DIMS

Name	Bits	Attribute	Description
hActive	15:0	RO	The detected active width of the test pattern input.
vActive	31:16	RO	The detected active height of the test pattern input

Table 342. REG_TPG_FREQ_DIMS

Name	Bits	Attribute	Description
Clock Frequency	31:0	RO	The frequency, in Hz, of the test pattern input clock

Table 343. REG_TPG_CLKS_DIMS

Name	Bits	Attribute	Description
Frame Period	31:0	RO	The number of test pattern input clocks per input frame.

Table 344. REG_VID_LOCKED_COUNT

Name	Bits	Attribute	Description
Locked Count	31:0	RO	Diagnostic counter that increments once at the start of each locked video frame.

Table 345. REG_VID_SIZE_ERR_COUNT

Name	Bits	Attribute	Description
Size Error Count	31:0	RO	Diagnostic counter that increments every time a mismatch between the video input and timing input is detected.

Table 346. REG_VID_STALL_COUNT

Name	Bits	Attribute	Description
Stall Count	31:0	RO	Diagnostic counter that increments every time the video input stalls and is dropped by the merge block.

Table 347. REG_TPG_LOCKED_COUNT

Name	Bits	Attribute	Description
Locked Count	31:0	RO	Diagnostic counter that increments once at the start of each locked test pattern input frame.

Table 348. REG_TPG_SIZE_ERR_COUNT

Name	Bits	Attribute	Description
Size Error Count	31:0	RO	Diagnostic counter that increments every time a mismatch between the test pattern input and timing input is detected.

Table 349. REG_TPG_STALL_COUNT

Name	Bits	Attribute	Description
Stall Count	31:0	RO	Diagnostic counter that increments every time the test pattern input stalls and is dropped by the merge block.

Video Timing Generator Registers

The Clocked Video Output IP includes an optional internal Video Timing Generator IP. The Video Timing Generator IP processor registers start at address offset 0x240.

The offset to the Video Timing Generator IP registers is different when you use them in the Clocked Video Output IP, compared to standalone.

Table 350. Video Timing Generator Registers

Register	Offset	Access	Description
Raster Timing Registers			
REG_STATUS	0x240	RO	Reserved
REG_COMMIT	0x244	RW	Update internal parameters with new video standard.
REG_MODE	0x248	RW	Control mode of operation.
REG_RESET_POS	0x24c	RW	Expected position of the start of frame input signal, relative to the raster
REG_TOTALS	0x250	RW	Total height and width of the raster
REG_HB_END	0x254	RW	First active pixel of a line
REG_V1B_POS	0x258	RW	Start and end of vertical blanking for field 1
REG_V2B_POS	0x25c	RW	Start and end of vertical blanking for field 2
REG_FIELD_STARTS	0x260	RW	First lines of field 1 and 2
REG_HS_POS	0x264	RW	The start and end of horizontal sync
REG_V1S_START	0x268	RW	The horizontal and vertical position of the start of the vertical sync for field 1
REG_V1S_END	0x26c	RW	The horizontal and vertical position of the end of the vertical sync for field 1
REG_V2S_START	0x270	RW	The horizontal and vertical position of the start of the vertical sync for field 2
REG_V2S_END	0x274	RW	The horizontal and vertical position of the end of the vertical sync for field 2
REG_JITTER_CONT	0x278	RW	Timing parameters for the hard and soft frame lock
REG_BLACK_0	0x27c	RW	The initial value of "black" for this color plane
REG_BLACK_1	0x280	RW ⁽⁶⁵⁾	The initial value of "black" for this color plane
REG_BLACK_2	0x284	RW ⁽⁶⁵⁾	The initial value of "black" for this color plane
REG_BLACK_3	0x288	RW ⁽⁶⁵⁾	The initial value of "black" for this color plane
<i>continued...</i>			

Register	Offset	Access	Description
REG_FRAME_COUNTS	0x28c	RO	Returns the total number of frames output, and the number of external frame starts received.
REG_FRAME_LENGTH	0x290	RO	The number of video clocks between consecutive frame start input signals
REG_VTOTAL_ADJ	0x294	RO ⁽⁶⁷⁾	The total height of the raster after adjustment for soft Lock.
REG_VID_FREQ	0x298	RO	The frequency of the video clock, in Hz.
REG_GENLOCK_STATS0	0x29c	RO	Diagnostics for hard and soft frame lock
REG_GENLOCK_STATS1	0x2a0	RO	Diagnostics for Soft frame lock.
Pulse and Toggle Timing Registers			
REG_PULSE0_START	0x2c0	RW ⁽⁶⁸⁾	The horizontal and vertical position of the start of the pulse
REG_PULSE0_END	0x2c4	RW ⁽⁶⁸⁾	The horizontal and vertical position of the end of the pulse
REG_PULSE1_START	0x2c8	RW ⁽⁶⁸⁾	The horizontal and vertical position of the start of the pulse
REG_PULSE1_END	0x2cc	RW ⁽⁶⁸⁾	The horizontal and vertical position of the end of the pulse
REG_PULSE2_START	0x2d0	RW ⁽⁶⁸⁾	The horizontal and vertical position of the start of the pulse
REG_PULSE2_END	0x2d4	RW ⁽⁶⁸⁾	The horizontal and vertical position of the end of the pulse
REG_PULSE3_START	0x2d8	RW ⁽⁶⁸⁾	The horizontal and vertical position of the start of the pulse
REG_PULSE3_END	0x2dc	RW ⁽⁶⁸⁾	The horizontal and vertical position of the end of the pulse
REG_PULSE4_START	0x2e0	RW ⁽⁶⁸⁾	The horizontal and vertical position of the start of the pulse
REG_PULSE4_END	0x2e4	RW ⁽⁶⁸⁾	The horizontal and vertical position of the end of the pulse
REG_PULSE5_START	0x2e8	RW ⁽⁶⁸⁾	The horizontal and vertical position of the start of the pulse
REG_PULSE5_END	0x2ec	RW ⁽⁶⁸⁾	The horizontal and vertical position of the end of the pulse
REG_PULSE6_START	0x2f0	RW ⁽⁶⁸⁾	The horizontal and vertical position of the start of the pulse
REG_PULSE6_END	0x2f4	RW ⁽⁶⁸⁾	The horizontal and vertical position of the end of the pulse
REG_PULSE7_START	0x2f8	RW ⁽⁶⁸⁾	The horizontal and vertical position of the start of the pulse
REG_PULSE7_END	0x2fc	RW ⁽⁶⁸⁾	The horizontal and vertical position of the end of the pulse

21.6. Clocked Video Output IP Software API

The IP has a software driver for software control of the IP at run time. The IP does not fit any of the generic device models provided by the Nios II HAL. It exposes a set of dedicated accessors to the control and status registers. The driver structure inherits the base driver structure so all common methods described in *Video and Vision Processing IPs Software API* are applicable.

⁽⁶⁷⁾ If **Soft Mode** is turned off, this register is undefined.

⁽⁶⁸⁾ This register is RO and returns 0x1234abcd if the pulse does not exist. Refer to the **Number of pulses** parameter.

intel_vvp_cvo_get_param_vid_pid

prototype:

```
int intel_vvp_cvo_get_param_vid_pid(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The product ID number

intel_vvp_cvo_get_param_version_number

prototype:

```
int intel_vvp_cvo_get_param_version_number(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The IP version number

intel_vvp_cvo_get_param_vid_fifo_depth

prototype:

```
int intel_vvp_cvo_get_param_vid_fifo_depth(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The VID_FIFO_DEPTH parameter

intel_vvp_cvo_get_param_tpg_fifo_depth

prototype:

```
int intel_vvp_cvo_get_param_tpg_fifo_depth(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The TPG_FIFO_DEPTH parameter

intel_vvp_cvo_get_param_bps

prototype:

```
int intel_vvp_cvo_get_param_bps(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The BPS parameter

intel_vvp_cvo_get_param_internal_vtiming

prototype:

```
prototype: int  
intel_vvp_cvo_get_param_internal_vtiming(intel_vvp_cvo_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The INTERNAL_VTIMING parameter

intel_vvp_cvo_get_param_fall_back_input_en

prototype:

```
int  
intel_vvp_cvo_get_param_fall_back_input_en(intel_vvp_cvo_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The FALL_BACK_INPUT_EN parameter

intel_vvp_cvo_get_param_pip

prototype:

```
int intel_vvp_cvo_get_param_pip(intel_vvp_cvo_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The PIXELS_IN_PARALLEL parameter

intel_vvp_cvo_get_param_color_planes

prototype:

```
int  
intel_vvp_cvo_get_param_color_planes(intel_vvp_cvo_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The INTERNAL_VTIMING parameter

intel_vvp_cvo_get_separate_cpu_clock

prototype:

```
int  
intel_vvp_cvo_get_param_color_planes(intel_vvp_cvo_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The NUMBER_OF_COLOR_PLANES parameter

`intel_vvp_cvo_get_separate_cpu_freq`

prototype:

```
int
intel_vvp_cvo_get_separate_cpu_clock(intel_vvp_cvo_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The SEPARATE_SLAVE_CLOCK parameter

`intel_vvp_cvo_get_tim_dimensions`

prototype:

```
int intel_vvp_cvo_get_tim_dimensions(intel_vvp_cvo_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The TIM_DIMENSIONS parameter

`intel_vvp_cvo_get_vid_is_async`

prototype:

```
int intel_vvp_cvo_get_vid_is_async(intel_vvp_cvo_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The VID_IS_ASYNC parameter

`intel_vvp_cvo_get_vid_dimensions`

prototype:

```
int intel_vvp_cvo_get_vid_dimensions(intel_vvp_cvo_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The VID_DIMENSIONS parameter

`intel_vvp_cvo_get_vid_debug`

prototype:

```
int intel_vvp_cvo_get_vid_debug(intel_vvp_cvo_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The VID_DEBUG parameter

`intel_vvp_cvo_get_tpg_is_async`

prototype:

```
int intel_vvp_cvo_get_tpg_is_async(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The TPG_IS_ASYNC parameter

`intel_vvp_cvo_get_tpg_dimensions`

prototype:

```
int intel_vvp_cvo_get_tpg_dimensions(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The TPG_DIMENSIONS parameter

`intel_vvp_cvo_get_tpg_debug`

prototype:

```
int intel_vvp_cvo_get_tpg_debug(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The TPG_DEBUG parameter

`intel_vvp_cvo_vtiming_get_param_num_pulses`

prototype:

```
int intel_vvp_cvo_vtiming_get_param_num_pulses(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The NUM_PULSES parameter

`intel_vvp_cvo_vtiming_get_param_hsize`

prototype:

```
int intel_vvp_cvo_vtiming_get_param_hsize(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The HSIZE parameter

`intel_vvp_cvo_vtiming_get_param_vsize`

prototype:

```
int
intel_vvp_cvo_vtiming_get_param_vsize(intel_vvp_cvo_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The VSIZE parameter

`intel_vvp_cvo_vtiming_get_param_build_hard_frame_lock`

prototype:

```
int
intel_vvp_cvo_vtiming_get_param_build_hard_frame_lock(intel_vvp_cvo_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The BUILD_HARD_FRAME_LOCK parameter

`intel_vvp_cvo_vtiming_get_param_build_soft_frame_lock`

prototype:

```
int
intel_vvp_cvo_vtiming_get_param_build_soft_frame_lock(intel_vvp_cvo_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The BUILD_SOFT_FRAME_LOCK parameter

`intel_vvp_cvo_vtiming_get_param_build_vrr`

prototype:

```
int intel_vvp_cvo_vtiming_get_param_build_vrr
(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The BUILD_VRR parameter

`intel_vvp_cvo_get_status_vid_locked`

prototype:

```
int  
intel_vvp_cvo_get_status_vid_locked(intel_vvp_cvo_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The VID_LOCKED field from the REG_STATUS register

`intel_vvp_cvo_get_status_vid_is_full`

prototype:

```
int intel_vvp_cvo_get_status_vid_is_full  
(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The VID_AXI4-S_FULL field from the REG_STATUS register

`intel_vvp_cvo_get_status_vid_stall_error`

prototype:

```
int  
intel_vvp_cvo_get_status_vid_stall_error(intel_vvp_cvo_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The VID_STALL_ERR field from the REG_STATUS register

`intel_vvp_cvo_get_status_vid_size_mismatch`

prototype:

```
int  
intel_vvp_cvo_get_status_vid_size_mismatch(intel_vvp_cvo_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The VID_SIZE_ERR field from the REG_STATUS register

`intel_vvp_cvo_get_status_tpg_locked`

prototype:

```
int  
intel_vvp_cvo_get_status_tpg_locked(intel_vvp_cvo_instance*  
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The TPG_LOCKED field from the REG_STATUS register

intel_vvp_cvo_get_status_tpg_is_full

prototype:

```
int intel_vvp_cvo_get_status_tpg_is_full
(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The TPG_AXI4-S_FULL field from the REG_STATUS register

intel_vvp_cvo_get_status_tpg_stall_error

prototype:

```
int
intel_vvp_cvo_get_status_tpg_stall_error(intel_vvp_cvo_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The TPG_STALL_ERR field from the REG_STATUS register

intel_vvp_cvo_get_status_tpg_size_mismatch

prototype:

```
int
intel_vvp_cvo_get_status_tpg_size_mismatch(intel_vvp_cvo_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The TPG_SIZE_ERR field from the REG_STATUS register

intel_vvp_cvo_get_status_phase

prototype:

```
int intel_vvp_cvo_get_status_phase(intel_vvp_cvo_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The TIMING_PHASE field from the REG_STATUS register

intel_vvp_cvo_get_black_0

prototype:

```
int intel_vvp_cvo_get_black_0(intel_vvp_cvo_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The BLACK_0 register

intel_vvp_cvo_get_black_1

prototype:

```
int intel_vvp_cvo_get_black_1(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The BLACK_1 register

intel_vvp_cvo_get_black_2

prototype:

```
int intel_vvp_cvo_get_black_2(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The BLACK_2 register

intel_vvp_cvo_get_black_3

prototype:

```
int intel_vvp_cvo_get_black_3(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The BLACK_3 register

intel_vvp_cvo_set_black_0

prototype:

```
int intel_vvp_cvo_set_black_0(intel_vvp_cvo_instance* instance, uint32_t in_val);
```

argument: instance, pointer to the intel_vvp_cvo_instance

in_val, value for this color plane

return value If kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

intel_vvp_cvo_set_black_1

prototype:

```
int intel_vvp_cvo_set_black_1(intel_vvp_cvo_instance* instance, uint32_t in_val);
```

argument: instance, pointer to the intel_vvp_cvo_instance

in_val, value for this color plane

return value If kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

intel_vvp_cvo_set_black_2

prototype:

```
int intel_vvp_cvo_set_black_2(intel_vvp_cvo_instance*
instance, uint32_t in_val);
```

argument:

instance, pointer to the intel_vvp_cvo_instance

in_val, value for this color plane

return value If kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

intel_vvp_cvo_set_black_3

prototype:

```
int intel_vvp_cvo_set_black_3(intel_vvp_cvo_instance*
instance, uint32_t in_val);
```

argument:

instance, pointer to the intel_vvp_cvo_instance

in_val, value for this color plane

return value If kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

intel_vvp_cvo_get_fallback_auto_recover

prototype:

```
int intel_vvp_cvo_get_fallback_auto_recover
(intel_vvp_cvo_instance* instance);
```

argument:

instance, pointer to the intel_vvp_cvo_instance

return value The AUTO RECOVER field from the REG_FALLBACK register

intel_vvp_cvo_get_fallback_force_black

prototype:

```
int intel_vvp_cvo_get_fallback_force_black
(intel_vvp_cvo_instance* instance);
```

argument:

instance, pointer to the intel_vvp_cvo_instance

return value The FORCE BLACK field from the REG_FALLBACK register

intel_vvp_cvo_get_fallback_force_tpg

prototype:

```
int intel_vvp_cvo_get_fallback_force_tpg
(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The FORCE TPG field from the REG_FALLBACK register

intel_vvp_cvo_get_fallback_force_vid

prototype:

```
int intel_vvp_cvo_get_fallback_force_vid  
(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The FORCE VID field from the REG_FALLBACK register

intel_vvp_cvo_set_fallback_auto_recover

prototype:

```
int  
intel_vvp_cvo_set_fallback_auto_recover(intel_vvp_cvo_instance*  
instance, uint32_t in_val);
```

argument: instance, pointer to the intel_vvp_cvo_instance

in_val, set this value to enable/disable the auto recover mode for this IP

return value If kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

intel_vvp_cvo_set_fallback_force_black

prototype:

```
int  
intel_vvp_cvo_set_fallback_force_black(intel_vvp_cvo_instance*  
instance, uint32_t in_val);
```

argument: instance, pointer to the intel_vvp_cvo_instance

in_val, setting this value forces the IP to output the color planes values previously set in registers BLACK_0, BLACK_1, BLACK_2 and BLACK_3

return value If kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

intel_vvp_cvo_set_fallback_force_tpg

prototype:

```
int  
intel_vvp_cvo_set_fallback_force_tpg(intel_vvp_cvo_instance*  
instance, uint32_t in_val);
```

argument: instance, pointer to the intel_vvp_cvo_instance

in_val, setting this value forces the IP to output the test pattern video input

return value If kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

`intel_vvp_cvo_set_fallback_force_vid`

prototype:

```
int
intel_vvp_cvo_set_fallback_force_vid(intel_vvp_cvo_instance*
instance, uint32_t in_val);
```

argument: instance, pointer to the intel_vvp_cvo_instance

in_val, setting this value forces the IP to output the video input

return value If kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

`intel_vvp_cvo_set_fallback_do_recover`

prototype:

```
int
intel_vvp_cvo_set_fallback_force_vid(intel_vvp_cvo_instance*
instance, uint32_t in_val);
```

argument: instance, pointer to the intel_vvp_cvo_instance

in_val, setting this value forces the IP to return to the video input if the video input is valid.

return value If kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

`intel_vvp_cvo_get_fr_totalh`

prototype:

```
int intel_vvp_cvo_get_fr_totalh(intel_vvp_cvo_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_FR_TOTALH register

`intel_vvp_cvo_get_fr_activeh`

prototype:

```
int intel_vvp_cvo_get_fr_activeh(intel_vvp_cvo_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_FR_ACTIVEH register

intel_vvp_cvo_get_fr_totalv

prototype:

```
int intel_vvp_cvo_get_fr_totalv(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_FR_TOTALV register

intel_vvp_cvo_get_fr_activev

prototype:

```
int intel_vvp_cvo_get_fr_activev(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_FR_ACTIVEV register

intel_vvp_cvo_get_fr_freq_dims

prototype:

```
int intel_vvp_cvo_get_fr_freq_dims(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_FR_FREQ_DIMS register

intel_vvp_cvo_get_fr_clk_dims

prototype:

```
int intel_vvp_cvo_get_fr_clk_dims(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_FR_CLKS_DIMS register

intel_vvp_cvo_get_vid_activeh

prototype:

```
int intel_vvp_cvo_get_vid_activeh(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_VID_ACTIVEH register

intel_vvp_cvo_get_vid_activev

prototype:

```
int intel_vvp_cvo_get_vid_activev(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_VID_ACTIVEV register

intel_vvp_cvo_get_vid_freq_dims

prototype:

```
int intel_vvp_cvo_get_vid_freq_dims(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_VID_FREQ_DIMS register

intel_vvp_cvo_get_vid_clk_dims

prototype:

```
int intel_vvp_cvo_get_vid_clk_dims(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_VID_CLKS_DIMS register

intel_vvp_cvo_get_tpg_activeh

prototype:

```
int intel_vvp_cvo_get_tpg_activeh(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_TPG_ACTIVEH register

intel_vvp_cvo_get_tpg_activev

prototype:

```
int intel_vvp_cvo_get_tpg_activev(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_TPG_ACTIVEV register

intel_vvp_cvo_get_tpg_freq_dims

prototype:

```
int intel_vvp_cvo_get_tpg_freq_dims(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_TPG_FREQ_DIMS register

intel_vvp_cvo_get_tpg_clk_dims

prototype:

```
int intel_vvp_cvo_get_tpg_clk_dims(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_TPG_CLKS_DIMS register

intel_vvp_cvo_get_vid_locked_count

prototype:

```
int intel_vvp_cvo_get_vid_locked_count(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_VID_LOCKED_COUNT register

intel_vvp_cvo_get_vid_size_err_count

prototype:

```
int intel_vvp_cvo_get_vid_size_err_count(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_VID_SIZE_ERR_COUNT register

intel_vvp_cvo_get_vid_stall_count

prototype:

```
int intel_vvp_cvo_get_vid_stall_count(intel_vvp_cvo_instance* instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_VID_STALL_COUNT register

intel_vvp_cvo_get_tpg_locked_count

prototype:

```
int
intel_vvp_cvo_get_tpg_locked_count(intel_vvp_cvo_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_TPG_LOCKED_COUNT register

intel_vvp_cvo_get_tpg_size_err_count

prototype:

```
int
intel_vvp_cvo_get_tpg_size_err_count(intel_vvp_cvo_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_TPG_ERR_COUNT register

intel_vvp_cvo_get_tpg_stall_count

prototype:

```
int
intel_vvp_cvo_get_tpg_stall_count(intel_vvp_cvo_instance*
instance);
```

argument: instance, pointer to the intel_vvp_cvo_instance

return value The REG_TPG_STALL_COUNT register

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

22. Color Plane Manager IP

22.1. About the Color Plane Manager IP

The IP receives frames or fields of RGB video from one or two Altera Streaming Video inputs. It transmits them over one or two Altera Streaming Video outputs according to your requirements. The IP can rearrange, merge, or split color planes, which allows you to separate an alpha channel from a four-channel RGBA input or reorder or duplicate color planes. When you select rearrange, you select the color plane order in the GUI. You cannot change the color plane order at run time.

The color plane manager IP supports:

- Maximum frame resolutions of 16384 by 16384 pixels with 1 to 8 pixels in parallel in any color space.
- Reading of progressive frames or interlaced fields.
- Optional run-time control of pixel padding values.
- Configurable processing of user metapackets.

The IP is available as full or lite variants. For more information on full and lite, refer to the *Altera Streaming Video Protocol Specification*.

An Avalon memory-mapped interface allows you to change color plane padding values at run time.

For details about latency and reset behavior for the color plane rearranger, refer to *Video and Vision IPs Functional Description*.

Related Information

[Altera Streaming Video Protocol Specification](#)

22.1.1. Color Plane Manager IP Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family and the number of supported bits per color sample, colors per pixel, and pixels in parallel.

Table 351. Color Plane Manager Performance and Resources – rearrange mode

The table shows ALM usage and f_{MAX} for a design with a color plane manager IP in rearrange mode with 2 pixels in parallel, 10 bits per color sample with 3 color planes input and output with memory-mapped control.

Target Device	Maximum Frame Size	ALMs	f_{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	4K	344	981
Cyclone 10 GX speed grade 5 (10CX220YF672E5G)	1080p	272	645

Table 352. Color Plane Manager Performance and Resources – merge mode

The table shows ALM usage and f_{MAX} for a design with a color plane manager in merge mode with 2 pixels in parallel, 10 bits per color sample with 3 color planes input and output, and input 1 aux/user packets removed.

Target Device	Maximum frame Size	ALMs	f_{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	4K	448	862
Cyclone 10 GX speed grade 5 (10CX220YF672E5G)	1080p	360	645

Table 353. Color Plane Manager Performance and Resources – split mode

The table shows ALM usage and f_{MAX} for a design with a color plane manager in split mode with 2 pixels in parallel, 10 bits per color sample with 3 color planes input and output, and output 1 aux/user packets removed.

Target Device	Maximum Frame Size	ALMs	f_{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	4K	555	975
Cyclone 10 GX speed grade 5 (10CX220YF672E5G)	1080p	494	645

22.2. Color Plane Manager IP Parameters

The IP offers compile-time parameters. Set these from the GUI in Platform Designer.

Parameter	Values	Description
Color plane manager mode		
Color plane manager mode	Merge, rearrange, split	Select what you want the IP to do with the color planes. When you select merge , the IP uses input 0 and 1 and output 0. When you select rearrange , the IP uses input 0 and output 0. When you select split , the IP uses input 0 and outputs 0 and 1.
Color planes configuration		
Number of color planes per pixel for input 0	1 - 4	Select the number of color planes for input 0.
Number of color planes per pixel for input 1	1 - 4	Select the number of color planes for input 1.
Number of color planes per pixel for output 0	1 - 4	Select the number of color planes for output 0.
Number of color planes per pixel for output 1	1 - 4	Select the number of color planes for output 1.
Video data precision		
Bits per color plane	8 to 16	Select the number of bits per color plane.
Pixels in parallel		
Number of pixels in parallel	1 - 8	Select the number of pixels in parallel.
Rearrange mode configuration		
Mapping for output color plane 0	0,1,2,3 or padding	Select the required input color plane for output color plane 0. Select padding if you want the IP to pad this color plane with a static value. Change the padding value at run time by turning on Memory-mapped control interface , otherwise specify a static value.
continued...		

Parameter	Values	Description
Static padding value for output color plane 0	0 to 2[bits per color sample] -1	Set this value if you want to pad this color plane and Memory-mapped control interface is off.
Mapping for output color plane 1	0,1,2,3 or padding	Select the required input color plane for output color plane 1. Select padding if you want the IP to pad this color plane with a static value. Change the padding value at run time by turning on Memory-mapped control interface , otherwise specify a static value.
Static padding value for output color plane 1	0 to 2[bits per color sample] -1	Set the padding if you want to pad this color plane and Memory-mapped control interface is off.
Mapping for output color plane 2	0,1,2,3 or padding	Select the required input color plane for output color plane 2. Select padding if you want the IP to pad this color plane with a static value. Change the padding value at run time by turning on Memory-mapped control interface , otherwise specify a static value.
Static padding value for output color plane 2	0 to 2[bits per color sample] -1	Set the value if you want to pad the color plane and Memory-mapped control interface is off..
Mapping for output color plane 3	0,1,2,3 or padding	Select the required input color plane for output color plane 3. Select padding if you want the IP to pad this color plane with a static value. Change the padding value at run-time by turning on Memory-mapped control interface , otherwise specify a static value.
Static padding value for output color plane 3	0 to 2[bits per color sample] -1	Set the padding if you want to pad this color plane and Memory-mapped control interface is off.
Split mode configuration for output 0		
Keep color plane 0	On or off	Select to keep color plane 0 for output 0. The IP concatenates color planes for the final output.
Keep color plane 1	On or off	Turn on to keep color plane 1 for output 0. The IP concatenates color planes for the final output.
Keep color plane 2	On or off	Turn on to keep color plane 2 for output 0. The IP concatenates color planes for the final output.
Keep color plane 3	On or off	Turn on to keep color plane 3 for output 0 The IP concatenates color planes for the final output.
Split mode configuration for output 1		
Keep color plane 0	On or off	Turn on to keep color plane 0 for output 1. The IP concatenates color planes for the final output.
Keep color plane 1	On or off	Turn on to keep color plane 1 for output 1. The IP concatenates color planes for the final output.
Keep color plane 2	On or off	Turn on to keep color plane 2 for output 1. The IP concatenates color planes for the final output.
Keep color plane 3	On or off	Turn on to keep color plane 3 for output 1. The IP concatenates color planes for the final output.
Merge mode configuration for input 0		
Keep color plane 0	On or off	Turn on to keep color plane 0 from input 0. The IP concatenates color planes for the final output.
Keep color plane 1	On or off	Turn on to keep color plane 1 from input 0. The IP concatenates color planes for the final output.
Keep color plane 2	On or off	Turn on to keep color plane 2 from input 0. The IP concatenates color planes for the final output.
continued...		

Parameter	Values	Description
Keep color plane 3	On or off	Turn on to keep color plane 3 from input 0. The IP concatenates color planes for the final output.
Merge mode configuration for input 1		
Keep color plane 0	On or off	Turn on to keep color plane 0 from input 1. The IP concatenates color planes for the final output.
Keep color plane 1	On or off	Turn on to keep color plane 1 from input 1. The IP concatenates color planes for the final output.
Keep color plane 2	On or off	Turn on to keep color plane 2 from input 1. The IP concatenates color planes for the final output.
Keep color plane 3	On or off	Turn on to keep color plane 3 from input 1. The IP concatenates color planes for the final output.
Control		
Keep input 1 aux/user metapackets in merge mode	On or off	When you select merge , turn on to keep all metapackets (including any user or auxiliary packets) from input 1 and from input 0.
Keep output 1 aux/user metapackets in split mode	On or off	When you select split , turn on to route all metapackets (including any user or auxiliary packets) to output 1 and to output 0.
Lite mode	On or off	Turn on for a lite variant of the color plane manager IP.
Memory mapped control interface	On or off	Turn on if you select rearrange and you need run-time control of the padding values or if you need access to debugging features.
Separate clock for control interface	On or off	Turn on if Memory mapped control interface is on and your control system is on a different clock than the streaming video interfaces clock.
Debug features	On or off	Turn on if Memory mapped control interface is on to enable debugging features.

Figure 58. Color Plane Manager IP GUI - Merge Mode

Color plane manager Intel FPGA IP

intel_vvp_cpm

Details

Important Usage Information

All resets must be at least 256 clock cycles in duration. When the IP uses multiple reset inputs, ensure that the reset pulses overlap by at least 16 clock cycles. See "Reset Behaviour" in the Video and Vision Processing User Guide

Capinfo

Color plane manager mode

Mode: Merge

Color planes configuration

Number of color planes per pixel for input 0: 4

Number of color planes per pixel for input 1: 3

Number of color planes per pixel for output 0: 3

Number of color planes per pixel for output 1: 2

Video data precision

Bits per color plane: 10

Pixels in parallel

Number of pixels in parallel: 2

Rearrange mode configuration

Split mode configuration for output 0

Split mode configuration for output 1

Merge mode configuration for input 0

☒ Keep color plane 0
 ☐ Keep color plane 1
 ☐ Keep color plane 2
 ☐ Keep color plane 3

Merge mode configuration for input 1

☐ Keep color plane 0
 ☒ Keep color plane 1
 ☒ Keep color plane 2
 ☐ Keep color plane 3

Control

☐ Keep IP1 aux/user metapackets packets in merge mode
 ☐ Keep OP1 aux/user metapackets packets in split mode
 ☒ Lite mode
 ☐ Memory mapped control interface
 ☐ Separate clock for control interface
 ☐ Debug features

Figure 59. Color Plane Manager IP GUI - Rearrange Mode

Color plane manager Intel FPGA IP

intel_vwp_cpm

Details

Important Usage Information

All resets must be at least 256 clock cycles in duration. When the IP uses multiple reset inputs, ensure that the reset pulses overlap by at least 16 clock cycles. See "Reset Behaviour" in the Video and Vision Processing User Guide

CapInfo

Color plane manager mode

Mode: Rearrange

Color planes configuration

Number of color planes per pixel for input 0: 4

Number of color planes per pixel for input 1: 3

Number of color planes per pixel for output 0: 4

Number of color planes per pixel for output 1: 2

Video data precision

Bits per color plane: 16

Pixels in parallel

Number of pixels in parallel: 8

Rearrange mode configuration

Mapping for output color plane 0: 2

Static padding value for output color plane 0: 1

Mapping for output color plane 1: 3

Static padding value for output color plane 1: 1

Mapping for output color plane 2: 0

Static padding value for output color plane 2: 1

Mapping for output color plane 3: Padding

Static padding value for output color plane 3: 77

Split mode configuration for output 0

Split mode configuration for output 1

Merge mode configuration for input 0

Merge mode configuration for input 1

Control

☐ Keep IP1 aux/user metapackets packets in merge mode
 ☐ Keep OP1 aux/user metapackets packets in split mode
 ☐ Lite mode
 ☐ Memory mapped control interface
 ☐ Separate clock for control interface
 ☐ Debug features

Figure 60. Color Plane Manager IP GUI - Split Mode

Color plane manager Intel FPGA IP
intel_vvp_cpm

Important Usage Information
All resets must be at least 256 clock cycles in duration. When the IP uses multiple reset inputs, ensure that the reset pulses overlap by at least 16 clock cycles. See "Reset Behaviour" in the Video and Vision Processing User Guide

Capinfo

Color plane manager mode
Mode: Split

Color planes configuration
Number of color planes per pixel for input 0: 4
Number of color planes per pixel for input 1: 1
Number of color planes per pixel for output 0: 2
Number of color planes per pixel for output 1: 2

Video data precision
Bits per color plane: 8

Pixels in parallel
Number of pixels in parallel: 1

Rearrange mode configuration

Split mode configuration for output 0
☒ Keep color plane 0
☐ Keep color plane 1
☒ Keep color plane 2
☐ Keep color plane 3

Split mode configuration for output 1
☐ Keep color plane 0
☒ Keep color plane 1
☐ Keep color plane 2
☒ Keep color plane 3

Merge mode configuration for input 0

Merge mode configuration for input 1

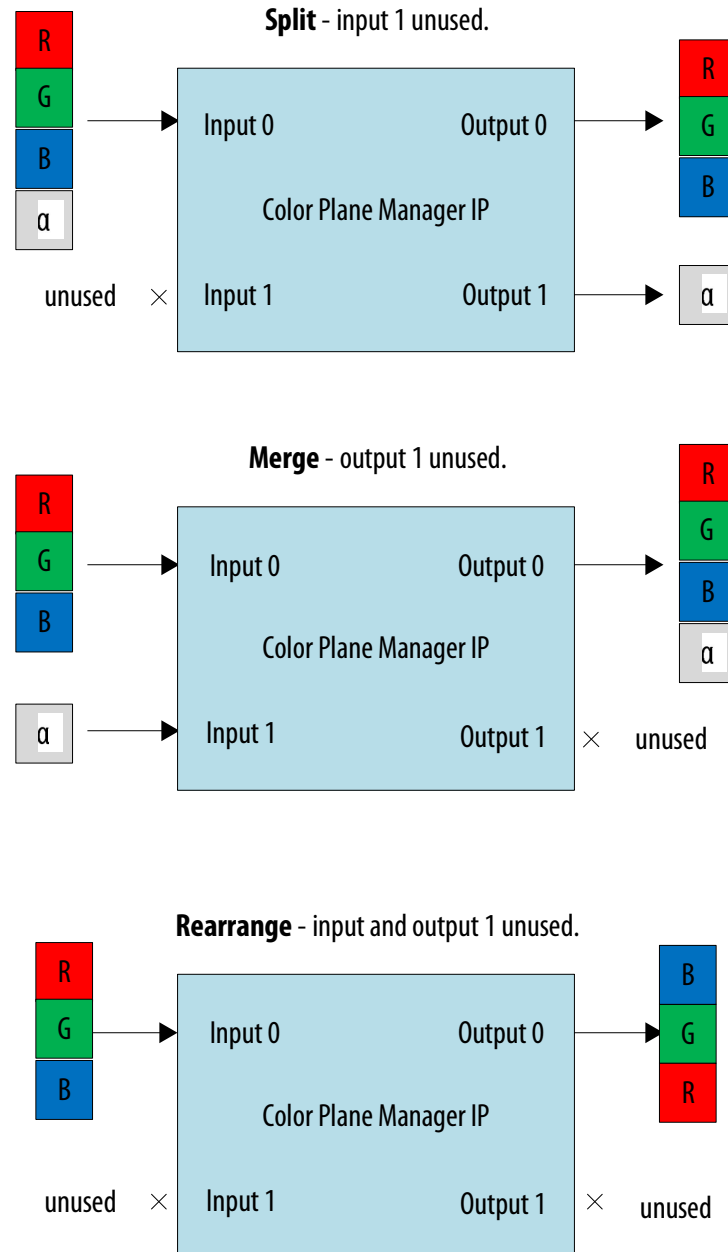
Control
☐ Keep IP1 aux/user metapackets packets in merge mode
☐ Keep OP1 aux/user metapackets packets in split mode
☐ Lite mode
☐ Memory mapped control interface
☐ Separate clock for control interface
☐ Debug features

22.3. Color Plane Manager IP Functional Description

You can select **rearrange**, **split**, or **merge** for the **Color Plane Manager Mode**.

Figure 61. Color Plane Manager Input and Outputs

The figure shows input 0 and output 0 are always active and they receive and transmit Altera Streaming Video. The IP uses input 1 and output 1 depending on the **Color Plane Manager Mode** that you select.



When you select **rearrange**, you can set static padding values for any of the output 0 color planes. You can set these values statically at compile time (*Color Plane Manager IP GUI - Rearrange Mode* figure in *Color Plane Manager Parameters* shows the output color plane 3 has a static value of 77 decimal). Alternatively, you can turn on **Memory mapped control interface** in the GUI to change padding values on a per-frame basis at run time.

The IP passes Altera Streaming Video Protocol metapackets from inputs to outputs with the video data. When you select **Merge**, if you want to propagate auxiliary and user metapackets from input 1 and input 0, turn on **Keep input 1 aux/user metapackets in merge mode**. When you select **split**, if you want to propagate auxiliary and user metapackets to output 1, turn on **Keep input 1 aux/user metapackets in split mode**.

Color Plane Examples with GUI Settings

Note: The figure show the IP starts numbering input and output color planes from 0, so an input or output with 4 color planes has planes 0,1,2 and 3.

Figure 62. Split Mode Example - extracting the luminance from a 2 color plane YCbCr input.

If your system does not use output 1, it can use any output 1 settings as the IP synthesizes away any output 1 logic.

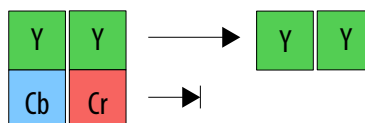


Table 354. Parameters for Split Mode Example - extracting the luminance from a 2 color plane YCbCr input

Parameter	Value
Number of color planes per pixel for input 0	2
Number of color plane per pixel for output 0	1
Split mode for output 0	Keep color plane 0

Figure 63. Split Mode Example – arbitrary selection of input color planes to outputs.

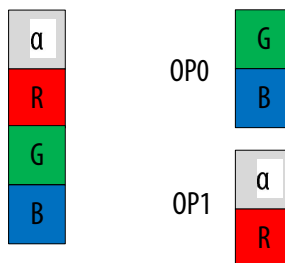


Table 355. Parameters for Split Mode Example - arbitrary selection of input color planes to outputs

Parameter	Value
Number of color planes per pixel for input 0	4
Number of color plane per pixel for output 0	2
Number of color plane per pixel for output 1	2
Split mode configuration for output 0	Keep color plane 1 Keep color plane 2
Split mode configuration for output 1	Keep color plane 0 Keep color plane 3

Figure 64. Split mode example – arbitrary selection of input color planes to outputs

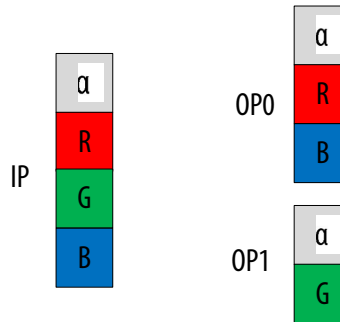


Table 356. Parameters for Split Mode Example - arbitrary selection of input color planes to outputs

Parameter	Value
Number of color planes per pixel for input 0	4
Number of color plane per pixel for output 0	3
Number of color plane per pixel for output 1	2
Split mode configuration for output 0	Keep color plane 0 Keep color plane 2 Keep color plane 3
Split mode configurationfor output 1	Keep color plane 1 Keep color plane 3

Figure 65. Rearrange Mode Example: adding a configurable transparency alpha plane to a 3 color plane input to make a 4 color plane output

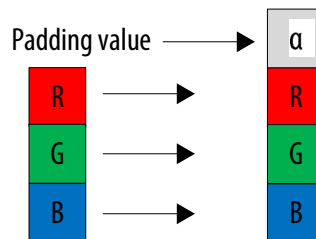


Table 357. Parameters Rearrange Mode Example: adding a configurable transparency alpha plane to a 3 color plane input to make a 4 color plane output

Parameter	Value
Number of color planes per pixel for input 0	3
Number of color plane per pixel for output 0	4
Mapping for output color plane 0	0
Mapping for output color plane 1	1
continued...	

Parameter	Value
Mapping for output color plane 2	2
Mapping for output color plane 3	Padding
Static padding value for color plane 3	Required value or turn on Memory mapped control interface to set the alpha value at run time.

Figure 66. Rearrange Mode Example: mapping a 3 color plane RGB input to a 3 color plane BGR output

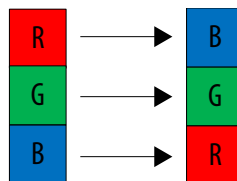


Table 358. Parameters Rearrange Mode Example: mapping a 3 color plane RGB input to a 3 color plane BGR output

Parameter	Value
Number of color planes per pixel for input 0	3
Number of color plane per pixel for output 0	3
Mapping for output color plane 0	2
Mapping for output color plane 1	1
Mapping for output color plane 2	0

Figure 67. Rearrange Mode Example: Extracting an alpha plane from a 4 color plane input to the low symbol of a 4 color plane output, with padding on upper 3 symbols

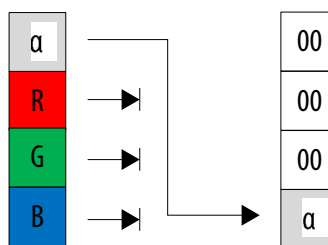


Table 359. Parameters Rearrange Mode Example: extracting an alpha plane from a 4 color plane input to the low symbol of a 4 color plane output, with padding on upper 3 symbols

Changes to the padding values made via the run-time control register take effect on the next video line.

Parameter	Value
Number of color planes per pixel for input 0	4
Number of color plane per pixel for output 0	4
Mapping for output color plane 0	3
<i>continued...</i>	

Parameter	Value
Mapping for output color plane 1	Padding
Mapping for output color plane 2	Padding
Mapping for output color plane 3	Padding

Merge Description

When you select **merge** for **Color Plane Manager Mode**, the IP discards image information and end of field packets from input 1. The IP propagates auxiliary and user metapackets from input 1 if you turn on **Keep IP1 aux/user metapackets in merge mode**, otherwise the IP consumes them. The IP always propagates all metapackets from input 0. This behavior does not apply if you turn on **Lite mode**.

If video lines on the two inputs are of unequal length, the merged output is the shorter length and the remaining pixels of the longer line are consumed. Merging resumes with the next packet.

If video frames on the two inputs are of unequal height, the merged output has the same number of lines as the shorter frame and the IP consumes the extra lines on the other input. Merging resumes with the next start of frame.

If `tvalid` is low on one the inputs, the color plane manager IP lowers `tready` on its other input until both `tvalid` inputs are high. If `tready` is low on the color plane manager IP output, the color plane manager IP also lowers `tready` on both inputs until `tready` on the output is high.

Split Description

When you select **split** for **Color Plane Manager Mode**, the IP duplicates image information and end of field packets from input 0 to both outputs with video data packets. The IP always propagates auxiliary and user metapackets to output 0 and to output 1 if you turn on **Keep OP1 aux/user metapackets in split mode**. This behavior does not apply if you turn on **Lite mode**.

If one of the outputs experiences backpressure via `tready`, the color plane manager IP also lowers `tready` on its input until the `tready` signals on both outputs is high.

Rearrange Description

When you select **rearrange** for **Color Plane Manager Mode**, the IP transfers all metapackets and data packets to the output.

Register Behavior

Bit [0] of the `csr_status` register goes high when the IP starts producing the first frame. It goes low after the IP finishes producing the last line of the frame. It returns high when the IP starts producing the next frame.

Applications that need to know when the IP produces frames can poll this status register.

Latency

Table 360. Latency

The table shows the IP latency under ideal conditions.

Color Plane Manager Mode	Latency (clock cycles)
Merge	15
Split	9
Rearrange	8

This latency increases if the streaming video outputs experience any backpressure via `tready`. Backpressure increases the latency by the same amount of cycles.

The padding values when you select **merge** do not change the latency.

22.3.1. Color Plane Manager IP Interfaces

Table 361. Color Plane Manager IP Interfaces

Name	Direction	Width	Description
Clocks and resets			
<code>main_clock_clk</code>	In	1	AXI4-S processing clock.
<code>main_reset_rst</code>	In	1	AXI4-S processing reset.
<code>control_clock_clk</code>	In	1	Optional control agent interface clock.
<code>control_reset_reset</code>	In	1	Optional control agent interface reset.
Control interfaces			
<code>av_mm_control_agent_address</code>	In	8 ⁽⁶⁹⁾	Avalon memory-mapped agent address
<code>av_mm_control_agent_write</code>	In	1	Avalon memory-mapped agent write.
<code>av_mm_control_agent_writedata</code>	In	32	Avalon memory-mapped agent write data.
<code>av_mm_control_agent_byteenable</code>	In	4	Avalon memory-mapped agent byte enable.
<code>av_mm_control_agent_read</code>	In	1	Avalon memory-mapped agent read.
<code>av_mm_control_agent_readdata</code>	Out	32	Avalon memory-mapped agent read data.
<code>av_mm_control_agent_readdatavalid</code>	Out	1	Avalon memory-mapped agent read.
<code>av_mm_control_agent_waitrequest</code>	Out	1	Avalon memory-mapped agent wait request.
Altera Streaming Video interfaces			
<code>axi4s_vid_in_0_tdata</code>	In	(69)	Input 0 AXI4-S data in.
<code>axi4s_vid_in_0_tvalid</code>	In	1	Input 0 AXI4-S data valid.
<code>axi4s_vid_in_0_tuser[0]</code>	In	1	Input 0 AXI4-S start of video frame.
<i>continued...</i>			

⁽⁶⁹⁾ The equation gives all `axi4s_vid_out_tdata` widths in these interfaces:
 $\max(2, \text{ceil}((\text{bits per color sample} \times \text{number of color planes})/8)) \times \text{pixels in parallel} \times 8$

Name	Direction	Width	Description
axi4s_vid_in_0_tuser[1]	In	1	Input 0 AXI4-S control or data packet for full variants. Field indicator for lite variants.
axi4s_vid_in_0_tuser[N-1:2]	In	(70)	Unused.
axi4s_vid_in_0_tlast	In	1	Input 0 AXI4-S end of packet .
axi4s_vid_in_0_tready	Out	1	Input 0 AXI4-S data ready
axi4s_vid_in_1_tdata	In	(69)	Input 1 AXI4-S data in.
axi4s_vid_in_1_tvalid	In	1	Input 1 AXI4-S data valid.
axi4s_vid_in_1_tuser[0]	In	1	Input 1 AXI4-S start of video frame.
axi4s_vid_in_1_tuser[1]	In	1	Input 1 AXI4-S control or data packet for full variants. Field indicator for lite variants.
axi4s_vid_in_1_tuser[N-1:2]	In	(70)	Unused.
axi4s_vid_in_1_tlast	In	1	Input 1 AXI4-S end of packet.
axi4s_vid_in_1_tready	Out	1	Input 1 AXI4-S data ready
axi4s_vid_out_0_tdata	Out	(69)	Output 0 AXI4-S data out.
axi4s_vid_out_0_tvalid	Out	1	Output 0 AXI4-S data valid.
axi4s_vid_out_0_tuser[0]	Out	1	Output 0 AXI4-S start of video frame.
axi4s_vid_out_0_tuser[1]	Out	1	Output 0 AXI4-S control or data packet for full variants. Field indicator for lite variants.
axi4s_vid_out_0_tuser[N-1:2]	Out	(70)	Unused.
axi4s_vid_out_0_tlast	Out	1	Output 0 AXI4-S end of packet.
axi4s_vid_out_0_tready	In	1	Output 0 AXI4-S data ready
axi4s_vid_out_1_tdata	Out	(69)	Output 1 AXI4-S data out.
axi4s_vid_out_1_tvalid	Out	1	Output 1 AXI4-S data valid.
axi4s_vid_out_1_tuser[0]	Out	1	Output 1 AXI4-S start of video frame.
axi4s_vid_out_1_tuser[1]	Out	1	Output 1 AXI4-S control or data packet for full variants. Field indicator for lite variants.
axi4s_vid_out_1_tuser[N-1:2]	Out	(70)	Unused.
axi4s_vid_out_1_tlast	Out	1	Output 1 AXI4-S end of packet.
axi4s_vid_out_1_tready	In	1	Output 1 AXI4-S data ready

Related Information

[Video and Vision Processing IP Interfaces on page 30](#)

22.4. Color Plane Manager IP Registers

Each register is either read-only (RO) or read-write (RW).

(70) This equation gives all axi4s_vid_out_tuser widths in these interfaces $N = \text{ceil}(\text{tdata width} / 8)$

Note: The register map is only available when you select **rearrange** for **Color plane manager mode**. When you select **split** or **merge**, the IP has no memory mapped control interface.

Table 362. Color Plane Manager IP Registers

In the software API the register names appear with a prefix of `INTEL_VVP`, `INTEL_VVP_CORE`, or `INTEL_VVP_COLOR_PLANE_MANAGER` as appropriate and with an optional `REG` suffix

Address	Register	Access	Description
Parameterization registers			
0x0000	VID_PID	RO	Read this register for the color plane manager product ID. This register always returns 0x6AF7_023B.
0x0004	VERSION	RO	Read this register for the IP version information
0x0008	LITE_MODE	RO	This register returns 1 if you select Lite Mode for the IP.
0x000C	DEBUG_ENABLED	RO	Read this register to determine if debug features are on.
0x0010	BITS_PER_SYMBOL	RO	Read this register for bits per symbol configuration information.
0x0014	PIXELS_IN_PARALLEL	RO	Read this register for pixels in parallel configuration information.
0x0018	NUMBER_OF_INPUT_COLOR_PLANES	RO	Read this register for input color plane configuration information.
0x001C	NUMBER_OF_OUTPUT_COLOR_PLANES	RO	Read this register for output color plane configuration information.
0x0020	MAPPING_FOR_OUTPUT_COLOR_PLANE0	RO	Read this register for color plane 0 mapping information.
0x0024	MAPPING_FOR_OUTPUT_COLOR_PLANE1	RO	Read this register for color plane 1 mapping information.
0x0028	MAPPING_FOR_OUTPUT_COLOR_PLANE2	RO	Read this register for color plane 2 mapping information.
0x002C	MAPPING_FOR_OUTPUT_COLOR_PLANE3	RO	Read this register for color plane 3 mapping information.
0x0030 to 0x011F	Reserved		
Control and Debug registers			
0x0120	IMG_INFO_WIDTH	RO	The expected width of the incoming video fields. For full designs, the received width in the IP derives from the image information packets.
0x0124	IMG_INFO_HEIGHT	RO	The expected height of the incoming video fields. For full designs, the received height in the IP derives from the image information packets.
0x0128	IMG_INFO_INTERLACE	RO	The expected interlace information of the incoming video fields. For full designs, the received interlace information in image information packets.
0x012C	RESERVED	RO	Unused.
0x0130	IMG_INFO_COLORSPACE	RO	The expected color space of the incoming video fields. For full designs, the received color space in image information packets.
continued..			

Address	Register	Access	Description
0x0134	IMG_INFO_SUBSAMPLING	RO	The expected chroma subsampling of the incoming video fields. For full designs, the received chroma subsampling in image information packets.
0x0138	IMG_INFO_COSITING	RO	The expected chroma co-siting of the incoming video fields. For full designs, the received chroma co-siting in image information packets.
0x013C	IMG_INFO_FIELD_COUNT	RO	The received field count field in image information packets.
0x0140	STATUS	RO	Bit 0: status bit. Bit 1: Pending writes bit.
0x0144	COMMIT	RW	Write 1 to bit 0 to commit the color plane padding values.
0x0148	COLOR_PLANE_0_PAD	RW	Color plane 0 dynamic padding value.
0x014C	COLOR_PLANE_1_PAD	RW	Color plane 1 dynamic padding value.
0x0150	COLOR_PLANE_2_PAD	RW	Color plane 2 dynamic padding value.
0x0154	COLOR_PLANE_3_PAD	RW	Color plane 3 dynamic padding value.

Register Bit Descriptions

Note: The register map is only available when you select **rearrange** for color plane manager. When you select **split** or **merge**, the IP has no memory mapped control interface.

Table 363. VID_PID

Name	Bits	Description
Color plane manager version ID and product ID	31:0	This register always returns 0x6AF7_023B 15:0 is the product ID and always returns 0x023B 31:16 is the vendor ID and always returns 0x6AF7

Table 364. VERSION

Name	Bits	Description
Register map version	7:0	Register map version.
IP patch revision	15:8	-
IP update revision	23:16	Updated when the IP version changes.
IP major revision	31:24	Updated when the IP version changes.

Table 365. LITE_MODE

Name	Bits	Description
Lite mode parameterization bit	0	Returns 1 if you turn on Lite mode .
Unused	31:1	Unused.

Table 366. DEBUG_ENABLED

Name	Bits	Description
Debug features parameterization bit	0	Returns 1 if you turn on Debug features .
Unused	31:1	Unused.

Table 367. BITS_PER_SYMBOL

Name	Bits	Description
Bits per symbol	31:0	Read this register for bits per symbol configuration information.

Table 368. PIXELS_IN_PARALLEL

Name	Bits	Description
Number of pixels in parallel	31:0	Read this register for pixels in parallel configuration information.

Table 369. NUMBER_OF_INPUT_COLOR_PLANES

Name	Bits	Description
Number of input color planes	31:0	Read this register for number of input color planes configuration information.

Table 370. NUMBER_OF_OUTPUT_COLOR_PLANES

Name	Bits	Description
Number of output color planes	31:0	Read this register for number of output color planes configuration information.

Table 371. MAPPING_FOR_OUTPUT_COLOR_PLANE0

Name	Bits	Description
Mapping for output color plane 0	31:0	Read this register for the output color plane 0 mapping information. 0 = Input color plane 0 1 = Input color plane 1 2 = Input color plane 2 3 = Input color plane 3 4 = Padding

Table 372. MAPPING_FOR_OUTPUT_COLOR_PLANE1

Name	Bits	Description
Mapping for output color plane 1	31:0	Read this register for the output color plane 1 mapping information. 0 = Input color plane 0 1 = Input color plane 1 2 = Input color plane 2 3 = Input color plane 3 4 = Padding

Table 373. MAPPING_FOR_OUTPUT_COLOR_PLANE2

Name	Bits	Description
Mapping for output color plane 2	31:0	Read this register for the output color plane 2 mapping information. 0 = Input color plane 0 1 = Input color plane 1 2 = Input color plane 2 3 = Input color plane 3 4 = Padding

Table 374. MAPPING_FOR_OUTPUT_COLOR_PLANE3

Name	Bits	Description
Mapping for output color plane 3	31:0	Read this register for the output color plane 3 mapping information. 0 = Input color plane 0 1 = Input color plane 1 2 = Input color plane 2 3 = Input color plane 3 4 = Padding

Table 375. IMG_INFO_WIDTH

Name	Bits	Description
Width bits	15:0	When you turn off lite mode and turn on Debug features , this register returns the <code>width-1</code> field from the most recently received image information packet and adds 1 to return a value for width.
unused	31:16	Unused.

Table 376. IMG_INFO_HEIGHT

Name	Bits	Description
Height bits	15:0	When you turn off lite mode and turn on Debug features , this register returns the <code>height-1</code> field from the most recently received image information packet and adds 1 to return a value for height.
unused	31:16	Unused.

Table 377. IMG_INFO_INTERLACE

Name	Bits	Description
InterlaceNibble bits	3:0	When you turn off lite mode and turn on Debug features , this register returns the <code>intlaceNibble</code> field from the most recently received image information packet.
unused	31:4	Unused.

Table 378. IMG_INFO_COLORSPACE

Name	Bits	Description
CSPcode bits	6:0	When you turn off lite mode and turn on Debug features , this register returns color space information from the most recently received image information packet.
unused	31:7	Unused.

Table 379. IMG_INFO_SUBSAMPLING

Name	Bits	Description
CSPSubScode bits	1:0	When you turn off lite mode and turn on Debug features , this register returns the SUBSA field from the most recently received image information packet.
unused	31:2	Unused.

Table 380. IMG_INFO_COSITING

Name	Bits	Description
Cositecode bits	1:0	When you turn off lite mode and turn on Debug features , this register returns the COSITE field from the most recently received image information packet.
unused	31:2	Unused.

Table 381. IMG_INFO_FIELD_COUNT

Name	Bits	Description
Countbits	6:0	When you turn on lite mode , this register has no function. When you turn off lite mode and turn on Debug features , this register returns the 7 bit FIELD_COUNT field from the most recently received image information packet.
unused	31:7	Unused.

Table 382. STATUS

Name	Bit	Description
Status	0	The status bit is set if the IP is producing a frame. It returns to 0 in between frames.
Pending writes	1	The pending writes bit is set after a write to: - COLOR_PLANE0_PAD - COLOR_PLANE1_PAD - COLOR_PLANE2_PAD - COLOR_PLANE3_PAD

Table 383. COMMIT

Name	Bit	Description
Commit	0	Write this register to commit new padding values. The values take effect at the start of frame.

Table 384. COLOR_PLANE0_PAD

Name	Bits	Description
Dynamic padding value for output color plane 0	31:0	Write this register to set the output color plane 0 dynamic padding information. If MAPPING_FOR_OUTPUT_COLOR_PLANE0 is not set to 4 (padding), this register has no effect. Write to the COMMIT register for the new padding values to take effect at the next frame.

Table 385. COLOR_PLANE1_PAD

Name	Bits	Description
Dynamic padding value for output color plane 1	31:0	Write this register to set the output color plane 1 dynamic padding information. If MAPPING_FOR_OUTPUT_COLOR_PLANE1 is not set to 4 (padding), this register has no effect. Write to the COMMIT register for the new padding values to take effect at the next frame.

Table 386. COLOR_PLANE2_PAD

Name	Bits	Description
Dynamic padding value for output color plane 2	31:0	Write this register to set the output color plane 2 dynamic padding information. If <code>MAPPING_FOR_OUTPUT_COLOR_PLANE2</code> is not set to 4 (padding), this register has no effect. Write to the <code>COMMIT</code> register for the new padding values to take effect at the next frame.

Table 387. COLOR_PLANE3_PAD

Name	Bits	Description
Dynamic padding value for output color plane 3	31:0	Write this register to set the output color plane 3 dynamic padding information. If <code>MAPPING_FOR_OUTPUT_COLOR_PLANE3</code> is not set to 4 (padding), this register has no effect. Write to the <code>COMMIT</code> register for the new padding values to take effect at the next frame.

22.5. Color Plane Manager Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Note: The software API is only available when you select **rearrange** for **Color plane manager mode**. When you select **split** or **merge**, the IP has no software API.

Register definition header file: `intel_vvp_cpm_regs.h`

Include file: `intel_vvp_cpm.h`

Table 388. Color Plane Manager IP Software API

Name	Description
<code>intel_vvp_cpm_init</code>	Initialize the Color Plane Manager IP instance
<code>intel_vvp_core_*</code>	Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i> . Writable when Lite is on. Readable when Lite is off and Debug features is on.
<code>intel_vvp_cpm_get_lite_mode</code>	Returns if Lite mode is on
<code>intel_vvp_cpm_get_debug_enabled</code>	Returns if Debug features is on
<code>intel_vvp_cpm_get_bits_per_sample</code>	Returns the number of bits per color sample for the streaming interfaces
<code>intel_vvp_cpm_get_pixels_in_parallel</code>	Returns the number of pixels per beat for the streaming interfaces
<code>intel_vvp_cpm_get_num_planes_in</code>	Returns the number of color planes per pixel for the input streaming interfaces
<code>intel_vvp_cpm_get_num_planes_out</code>	Returns the number of color planes per pixel for the output streaming interfaces
<code>intel_vvp_cpm_is_running</code>	Returns if the IP is currently processing video data
<code>intel_vvp_cpm_get_commit_status</code>	Returns if there are uncommitted writes
<code>intel_vvp_cpm_get_status</code>	Reads the status register
<code>intel_vvp_cpm_set_plane_padding</code>	Sets the padding value for a selected output color plane
continued...	

Name	Description
intel_vvp_cpm_set_padding	Sets the padding values for output color planes
intel_vvp_cpm_get_plane_padding	Returns the padding value for a selected output color plane
intel_vvp_cpm_get_padding	Returns the padding values for output color planes
intel_vvp_cpm_commit_writes	Commit all outstanding writes

intel_vvp_cpm_init

Prototype

```
int intel_vvp_cpm_init(intel_vvp_cpm_instance *instance,
intel_vvp_core_base base);
```

Description

Initializes a Color Plane Manager IP instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

Arguments

instance - pointer to the intel_vvp_cpm_instance software driver instance structure

base - base address of the register map

Returns

kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product_id does not match with the Color Plane Manager product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpColorPlaneManagerRegMapVersionErr if the register map is not supported

intel_vvp_cpm_get_lite_mode

Prototype

```
bool intel_vvp_cpm_get_lite_mode(intel_vvp_cpm_instance
*instance);
```

Description

Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_cpm_instance fully initialized

Arguments

instance - pointer to the intel_vvp_cpm_instance software driver instance structure

Returns

true if the IP is parameterized in lite mode

intel_vvp_cpm_get_debug_enabled

Prototype

```
bool intel_vvp_cpm_get_debug_enabled(intel_vvp_cpm_instance
*instance);
```

Description Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_cpm_instance fully initialized.

Arguments instance - pointer to the intel_vvp_cpm_instance software driver instance structure

Returns true if the IP is parameterized with debug features enabled

intel_vvp_cpm_get_bits_per_sample

Prototype

```
uint8_t
intel_vvp_cpm_get_bits_per_sample(intel_vvp_cpm_instance
*instance);
```

Description Returns the value of the BPS register. The instance must be a valid intel_vvp_cpm_instance fully initialized.

Arguments instance - pointer to the intel_vvp_cpm_instance software driver instance structure

Returns Returns the value of the BPS parameter used to configure the IP. The instance must be a valid intel_vvp_cpm_instance fully initialized

intel_vvp_cpm_get_pixels_in_parallel

Prototype

```
uint8_t
intel_vvp_cpm_get_pixels_in_parallel(intel_vvp_cpm_instance
*instance);
```

Description Returns the value of the PIXELS_IN_PARALLEL parameter used to configure the IP. The instance must be a valid intel_vvp_cpm_instance fully initialized.

Arguments instance - pointer to the intel_vvp_cpm_instance software driver instance structure

Returns Returns the value of the PIXELS_IN_PARALLEL parameter used to configure the IP. The instance must be a valid intel_vvp_cpm_instance fully initialized

intel_vvp_cpm_get_num_planes_in

Prototype

```
uint8_t  
intel_vvp_cpm_get_num_planes_in(intel_vvp_cpm_instance  
*instance);
```

Description Returns the value of the NUMBER_OF_COLOR_PLANES_IN0 parameter used to configure the IP. The instance must be a valid intel_vvp_cpm_instance fully initialized.

Arguments instance - pointer to the intel_vvp_cpm_instance software driver instance structure

Returns Returns the value of the NUMBER_OF_COLOR_PLANES_IN0 parameter used to configure the IP. The instance must be a valid intel_vvp_cpm_instance fully initialized

intel_vvp_cpm_get_num_planes_out

Prototype

```
uint8_t  
intel_vvp_cpm_get_num_planes_out(intel_vvp_cpm_instance  
*instance);
```

Description Returns the value of the NUMBER_OF_COLOR_PLANES_OUT0 parameter used to configure the IP. The instance must be a valid intel_vvp_cpm_instance fully initialized.

Arguments instance - pointer to the intel_vvp_cpm_instance software driver instance structure

Returns Returns the value of the NUMBER_OF_COLOR_PLANES_OUT0 parameter used to configure the IP. The instance must be a valid intel_vvp_cpm_instance fully initialized

intel_vvp_cpm_is_running

Prototype

```
bool intel_vvp_cpm_is_running(intel_vvp_cpm_instance*  
instance);
```

Description Reads and returns the running bit of the STATUS register. The instance must be a valid intel_vvp_cpm_instance fully initialized.

Arguments instance - pointer to the intel_vvp_cpm_instance software driver instance structure

Returns True is the IP is currently receiving/storing an input field, false when between two input fields

intel_vvp_cpm_get_commit_status

Prototype

```
bool
intel_vvp_cpm_get_commit_status(intel_vvp_clipper_instance*
instance);
```

Description Reads and returns the commit pending bit of the STATUS register. The instance must be a valid intel_vvp_cpm_instance fully initialized and configured in full mode.

Arguments instance - pointer to the intel_vvp_cpm_instance software driver instance structure

Returns True is the IP is configured in full mode and if the agent interface contains uncommitted writes.

intel_vvp_cpm_get_status

Prototype

```
uint32_t intel_vvp_cpm_get_status(intel_vvp_cpm_instance*
instance);
```

Description Reads the STATUS register. The instance must be a valid intel_vvp_cpm_instance fully initialized.

Arguments instance - pointer to the intel_vvp_cpm_instance software driver instance structure

Returns The content of the STATUS register

intel_vvp_cpm_set_plane_padding

Prototype

```
int intel_vvp_cpm_set_plane_padding(intel_vvp_cpm_instance
*instance, uint8_t plane, uint16_t padding_value);
```

Description Sets the padding value used for the selected output color plane. The instance must be a valid intel_vvp_cpm_instance fully initialized, the output plane selected must be valid.

Arguments instance - pointer to the intel_vvp_cpm_instance software driver instance structure

plane - output color plane

padding_value - padding value for the selected output color plane

Returns kIntelVvpCoreOk for success, negative error code in case of failures
kIntelVvpCoreInstanceErr if the instance is invalid
kIntelVvpColorPlaneManagerParameterErr if the output color plane is invalid

intel_vvp_cpm_set_padding

Prototype

```
int intel_vvp_cpm_set_padding(intel_vvp_cpm_instance  
*instance, const intel_vvp_cpm_padding padding);
```

Description

Sets the padding values used for output color planes. The instance must be a valid intel_vvp_cpm_instance fully initialized.

Arguments

instance - pointer to the intel_vvp_cpm_instance software driver instance structure

padding - an array of uint16_t values containing the new padding values

Returns

kIntelVvpCoreOk for success, negative error code in case of failures

kIntelVvpCoreInstanceErr if the instance is invalid

kIntelVvpCoreNullPtrErr if the padding is the null pointer

intel_vvp_cpm_get_plane_padding

Prototype

```
uint16_t  
intel_vvp_cpm_get_plane_padding(intel_vvp_cpm_instance  
*instance, uint8_t plane);
```

Description

Returns the padding value used for the selected output color plane. The instance must be a valid intel_vvp_cpm_instance fully initialized and parameterized with debug capability enabled, the output plane selected must be valid.

Arguments

instance - pointer to the intel_vvp_cpm_instance software driver instance structure

plane - selected output color plane

Returns

The requested padding value or 0 in case of failure.

intel_vvp_cpm_get_padding

Prototype

```
int intel_vvp_cpm_get_padding(intel_vvp_cpm_instance  
*instance, intel_vvp_cpm_padding padding);
```

Description

Reads the padding values used for output color planes. The instance must be a valid intel_vvp_cpm_instance fully initialized and parameterized with debug capability enabled.

Arguments

instance - pointer to the intel_vvp_cpm_instance software driver instance structure

padding - a pre-allocated array of uint16_t values to store the padding values

Returns kIntelVvpCoreOk for success, negative error code in case of failures
kIntelVvpCoreInstanceErr if the instance is invalid or if debug capability is disabled
kIntelVvpCoreNullPtrErr if padding is the null pointer

intel_vvp_cpm_commit_writes

Prototype

```
int intel_vvp_cpm_commit_writes(intel_vvp_cpm_instance* instance)
```

Description Commits all pending writes before processing the next field. The instance must be valid and parameterized in full mode.

Arguments instance - pointer to the intel_vvp_cpm_instance software driver instance structure

Returns kIntelVvpCoreOk (0) for success, negative error code otherwise

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38



23. Color Space Converter IP

23.1. About the Color Space Converter IP

The IP transforms video data between color spaces. The color spaces allow you to specify colors using three coordinate values.

You can configure this IP to change conversion values at run time using an Avalon memory-mapped interface.

A color space precisely specifies the display of color using a three-dimensional coordinate system. Different color spaces are best for different devices, such as R'G'B' (red-green-blue) for computer monitors or Y'CbCr (luminance chrominance) for digital television.

Color space conversion is often necessary when transferring data between devices that use different color space models. For example, to transfer a television image to a computer monitor, you must convert the image from the Y'CbCr color space to the R'G'B' color space. Conversely, transferring an image from a computer display to a television may require a transformation from the R'G'B' color space to Y'CbCr.

Different conversions may be required for standard definition television (SDTV) and high-definition television (HDTV). You may also want to convert to or from the Y'IQ (luminance-color) color model for National Television System Committee (NTSC) systems or the Y'UV (luminance-bandwidth-chrominance) color model for Phase Alternation Line (PAL) systems.

23.1.1. Color Space Converter IP Features

- Flexibly and efficiently converts image data from one color space to another.
- Supports a number of predefined conversions between standard color spaces.
- Allows custom coefficients to translate between any two three-valued color spaces.
- Supports up to 8 pixels per transmission.

23.1.2. Color Space Converter IP Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family and the number of supported bits per color sample, pixels in parallel, and number of color planes.

Table 389. Color Space Converter IP Performance and Resources

The table shows ALM usage and f_{MAX} for a design with 6 pixels in parallel, 16 bits per color sample, 3 color planes.

Target device	ALMs	f_{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	3,518	666
Cyclone 10 GX speed grade 5 (10CX220YF78015G)	3,811	397

23.2. Color Space Converter IP Parameters

Table 390. Color Space Converter IP Parameters - General Tab

Parameter	Value	Description
Control Settings		
Memory-mapped control interface	On or off	Turn on to specify Color Space Converter register values using the Avalon memory-mapped interface.
Separate clock for control interface	On or off	Turn on for a separate clock for the control interface.
Debug features	On or off	Turn on for debugging features (not applicable for lite mode).
Pipeline ready signals	On or off	Turn on to pipeline ready signals, which helps with improving F_{max} .
Video Data Format		
Lite mode	On or off	Turn on to operate the Color Space Converter in lite mode.
Input bits per color sample	8 to 16	Select the number of input bits per color sample.
Output bits per color sample	8 to 16	Select the number of output bits per color sample.
Number of pixels in parallel	1 to 8	Select the number of pixels in parallel.
Output Color space	RGB, YCbCr, or MONO	Only applies if Memory-mapped control interface and Lite mode are off. Use this to set the color space field value of outgoing image information packets. It does not affect the algorithmic values, only the metadata.

Table 391. Color Space Converter IP Parameters - Operands Tab

Parameter	Allowed range	Description
Properties		
Coefficient and summand fractional bits	0 to 24	Specify the number of fraction bits for the fixed-point type to store the coefficients and summands.
Coefficient precision: Signed	On or Off	Turn on to set the fixed-point type to store the constant coefficients as having a sign bit.
Coefficient precision: Integer bits	0 to 16	Specifies the number of integer bits for the fixed-point type to store the constant coefficients.
Summand precision: Signed	On or Off	Turn on to set the fixed-point type to store the constant summands as having a sign bit.
Summand precision: Integer bits	0 to 20	Specify the number of integer bits for the fixed-point type used to store the constant summands.
<i>continued...</i>		

Parameter	Allowed range	Description
Result to output data type conversion		
Move binary point right	-16 to +16	Specify the number of places by which to move the binary point to the right. Negative numbers indicate moving the binary point to the left.
Remove fraction bits by	- Round values - Half up - Round values - Half even - Truncate values to integer	Select the method of discarding fraction bits resulting from the calculation.
Coefficients and Summand Table A0, B0, C0, S0 A1, B1, C1, S1 A2, B2, C2, S2	12 fixed-point values	Only when you turn off Memory-mapped control interface . Each coefficient or summand is represented by a white cell with a gray cell underneath. The value in the white cell is the desired value and is editable. The value in the gray cell is the actual value, determined by the fixed-point type specified. The gray cells are not editable. You can create a custom coefficient and summand set by specifying one fixed-point value for each entry.

Figure 68. Color Space Converter General Tab

The figure also shows the presets

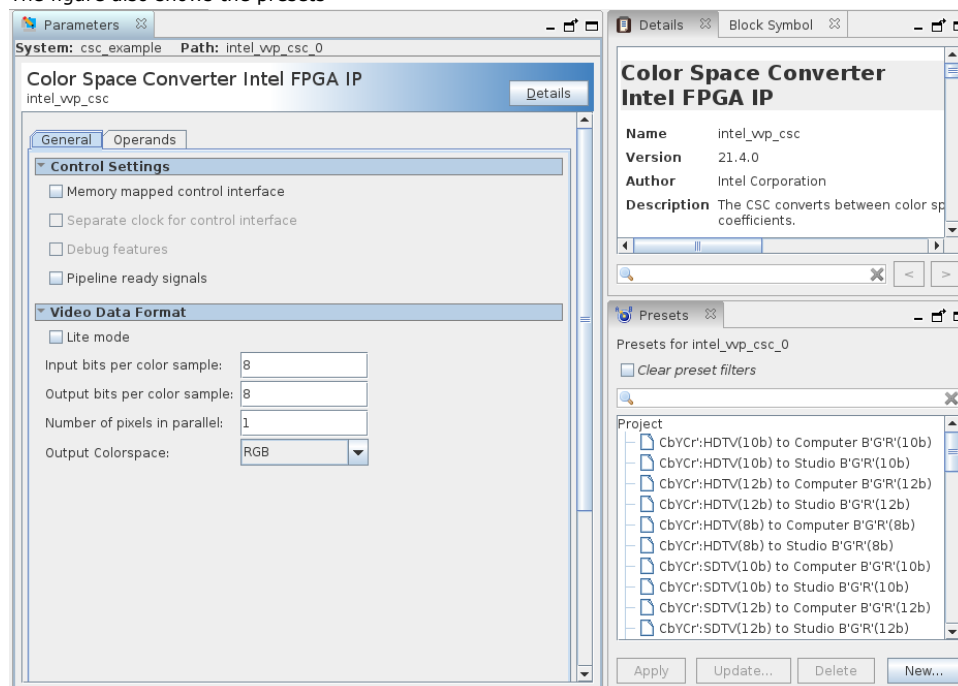


Figure 69. Color Space Converter GUI Operands Tab

Parameters ⓘ

System: csc_example Path: intel_wp_csc_0

Color Space Converter Intel FPGA IP
intel_wp_csc Details

General Operands

Properties

Coefficient and summand fractional bits: 8

Coefficients Precision

☒ Signed

Integer bits: 1

The total coefficient length (sign bit included) is 10 bits.

Summand Precision

☐ Signed

Integer bits: 10

The total summand length (sign bit included) is 18 bits.

Result to output data type conversion procedure

The results have 8 fractional bits.

Move binary point right: 0

The scaled results have 8 fractional bits.

Remove fraction bit by: Round values - Half up

Coefficients and Summand Table

din and dout refer to the input and output channels respectively.

$$\text{dout}_0 = (A0 * \text{din}_0) + (B0 * \text{din}_1) + (C0 * \text{din}_2) + S0$$

$$\text{dout}_1 = (A1 * \text{din}_0) + (B1 * \text{din}_1) + (C1 * \text{din}_2) + S1$$

$$\text{dout}_2 = (A2 * \text{din}_0) + (B2 * \text{din}_1) + (C2 * \text{din}_2) + S2$$

(din_# and dout_# are sent in order #=0,1,2.)

	A	B	C	S
0	1.0	0.0	0.0	0.0
	1.0	0.0	0.0	0.0
1	0.0	1.0	0.0	0.0
	0.0	1.0	0.0	0.0
2	0.0	0.0	1.0	0.0
	0.0	0.0	1.0	0.0

23.3. Color Space Converter IP Functional Description

The inputs and outputs of the IP support 8 to 16 bits per pixel per color plane.

Input and Output Data Types

If the conversion requires you to specify minimum or maximum values, which the input or output cannot lie outside of, use the Guard Bands IP. The Guard Bands IP can be instantiated before, after, or before and after, depending on the requirements of the design. You can use the Guard Bands IP if you use the Color Space Converter in a pipeline with signed data.

Color Space Conversion

You convert between color spaces by providing an array of nine coefficients and three summands that relate color spaces.

You can set these coefficients and summands at compile time, or you can enable the Avalon memory-mapped interface to change them dynamically at run-time.

Given a set of nine coefficients [A0, A1, A2, B0, B1, B2, C0, C1, C2] and a set of three summands [S0, S1, S2], the Color Space Converter calculates the output values for color planes 0, 1, and 2 (denoted dout_0, dout_1, and dout_2):

- $dout_0 = (A0 \times din_0) + (B0 \times din_1) + (C0 \times din_2) + S0$
- $dout_1 = (A1 \times din_0) + (B1 \times din_1) + (C1 \times din_2) + S1$
- $dout_2 = (A2 \times din_0) + (B2 \times din_1) + (C2 \times din_2) + S2$

din_0, din_1, and din_2 are inputs read from color planes 0, 1, and 2.

The Color Space Converter IP supports the following predefined conversions that are available through the Platform Designer presets:

- Computer B'G'R' to CbY'Cr: SDTV
- CbY'Cr: SDTV to Computer B'G'R'
- Computer B'G'R' to CbY'Cr: HDTV
- CbY'Cr: HDTV to Computer B'G'R'
- Studio B'G'R' to CbY'Cr: SDTV
- CbY'Cr: SDTV to Studio B'G'R'
- Studio B'G'R' to CbY'Cr: HDTV
- CbY'Cr: HDTV to Studio B'G'R'

The IP assigns the values in the order indicated by the conversion name. For example, if you select Computer B'G'R' to CbY'Cr: SDTV, din_0 = B', din_1 = G', din_2 = R', dout_0 = Cb, dout_1 = Y', and dout_2 = Cr. As the Altera Streaming Video Protocol states, din_0 occupies the LSBs of the word, din_1 the middle bits, and din_2 the MSBs. For example, for 8 bits per sample and one of the predefined conversions inputs B'G'R', din_0 carries B' in bits 0–7, din_1 carries G' in bits 8–15, and din_2 carries R' in bits 16–23.

Predefined Conversions

When using a predefined conversion, the precision of the coefficients and summands must still be defined. The predefined coefficients also only are available if you turn off Memory-Mapped control. Otherwise, the coefficients must be set at runtime using the Avalon memory-mapped interface.

Predefined conversions are only defined for input and output bits per pixel per color plane equal to 8, 10, and 12. You must manually scale the summands accordingly when using different bits per color plane values. If you use different input and output bits per pixel per color plane, you must also shift the results by the correct number of binary places to compensate. For example, to convert from 10-bit CbYCr' to 8-bit Computer B'G'R', select the conversion preset for 10-bit CbYCr' to 10-bit computer B'G'R'. The summands are already scaled for a 10-bit input, so they remain unchanged. Change the output bits per color plane value from 10 to 8 on the parameter editor and follow the instructions of the warning message to shift the results by the correct number of binary places (2 places to the left).

Always check the matrix of coefficients after applying a predefined conversion or after custom modifications. If the differences between the desired floating-point coefficient values and their actual fixed-point quantized values indicate an unacceptable loss of precision, you must increase the number of integer and or fractional bits to fix the problem.

Output Data Type Conversion

After the calculation, the fixed-point type of the results the IP must convert to the integer data type of the output.

The IP performs this conversion in four stages, in the following order:

1. Scales result. You can choose to scale up the results, increasing their range, which is useful to quickly increase the color depth of the output. The available options are a shift of the binary point right -16 to +16 places. The IP implements the scale as a simple shift operation so it does not require multipliers
2. Removes fractional bits. If any fractional bits exist, you can choose to remove them:
 - Truncate to integer. Fractional bits are removed from the data. This is equivalent to rounding towards negative infinity.
 - Round-half up. Round up to the nearest integer. If the fractional bits equal 0.5, rounding is towards positive infinity.
 - Round-half even. Round to the nearest integer. If the fractional bits equal 0.5, rounding is towards the nearest even integer.
3. Converts from signed to unsigned. If any negative numbers can exist in the results and the output type is unsigned, they are saturated to the minimum output value.
4. Constrain to range. Logic that saturates the results to the minimum and maximum output values:
 - If any of the results are not within the minimum and maximum values allowed by the output bits per pixel, the values are automatically constrained to this range.
 - If greater control is required over the lower and upper values, consider instantiating a Guard Bands IP after the Color Space Converter

23.3.1. Color Space Converter IP Interfaces

Table 392. Color Space Converter IP Interfaces

Name	Direction	Width	Description
Clocks and resets			
main_clock_clk	In	1	AXI4-S processing clock.
main_reset_rst	In	1	AXI4-S processing reset.
agent_clock_clk	In	1	Optional agent interface clock.
agent_reset_rst	In	1	Optional agent interface reset.
Control interfaces			
av_mm_control_agent_address	In	7	Avalon memory-mapped agent address.
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write.
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data.
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable.
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read.
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data.
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read.
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request.
Altera Streaming Video interfaces			
axi4s_vid_in_tdata	In	(71)	AXI4-S data in.
axi4s_vid_in_tvalid	In	1	AXI4-S data valid.
axi4s_vid_in_tuser[0]	In	1	AXI4-S start of video frame.
axi4s_vid_in_tuser[1]	In	1	AXI4-S control or data packet.
axi4s_vid_in_tuser[N-1:2]	In	(72)	Unused.
axi4s_vid_in_tlast	In	1	AXI4-S end of packet .
axi4s_vid_in_tready	Out	1	AXI4-S data ready.
axi4s_vid_out_tdata	Out		AXI4-S data out.
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid.
axi4s_vid_out_tuser[0]	Out	1	AXI4-S start of video frame.
axi4s_vid_out_tuser[1]	Out	1	AXI4-S control or data packet.
<i>continued...</i>			

(71) The equation gives all tdata widths in these interfaces :
 $\max(2, \text{ceil}((\text{bits per color sample} \times 3)/8)) \times \text{pixels in parallel} \times 8$

(72) The equation gives all tuser widths in these interfaces
 $N = \text{ceil}(\text{tdata width} / 8)$

Name	Direction	Width	Description
axi4s_vid_out_tuser[N-1:2]	Out		Unused.
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet
axi4s_vid_out_tready	In	1	AXI4-S data ready.

Related Information

Video and Vision Processing IP Interfaces on page 30

23.4. Color Space Converter IP Registers

Each register is either read-only (RO) or read-write (RW).

In the software API the register names appear with a prefix of `INTEL_VVP`, `INTEL_VVP_CORE` or `INTEL_VVP_CSC` as appropriate and with an optional `REG` suffix

Address	Register	Access		Description
		Lite (73)	Full	
Parameterization registers				
0x0000	VID_PID	RO	RO	Read this register to retrieve the Color Space Converter product ID. This register always returns 0x6AF7_022F.
0x0004	VERSION	RO	RO	Read this register to retrieve the IP version information.
0x0008	LITE_MODE	RO	RO	Read this register to determine if Lite mode is on. This register returns 0 when Lite mode is off and 1 when Lite mode is on.
0x000C	DEBUG_ENABLED	RO	RO	Read this register to determine if Debug features is on. This register returns 0 for off and 1 for on.
0x0010	BPS_IN	RO	RO	Read this register to determine the bits per symbol configured for the input data. Range [8 : 16].
0x0014	BPS_OUT	RO	RO	Read this register to determine the bits per symbol configured for the output data. Range [8 : 16].
0x0018	FRAC_BITS	RO	RO	Read this register to determine the number of fractional bits used by the fixed-point data type to store the coefficients and summands Range [0 : 31].
0x001C	COEFFS_SIGNED	RO	RO	Read this register to determine if the coefficient fixed-point type has a signed bit. (0 – unsigned, 1 - signed).
0x0020	COEFFS_INT_BITS	RO	RO	Read this register to determine the number of integer bits used by fixed-point data type to store coefficients. Range [0 : 16].
continued...				

⁽⁷³⁾ When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

Address	Register	Access		Description
		Lite (73)	Full	
0x0024	SUMMANDS_SIGNED	RO	RO	Read this register to determine if the summand fixed-point type has a signed bit. (0 – unsigned, 1 - signed).
0x0028	SUMMANDS_INT_BITS	RO	RO	Read this register to determine the number of integer bits used by fixed-point data type to store summands. Range [0 : 20].
0x002C	BINARY_POINT_RIGHT_MOVE	RO	RO	Read this register to determine the number of places by which the binary point moves to the right. Use to scale the result of the calculation. Range [-16 : 16].
0x0030	ROUND_METHOD	RO	RO	Read this register to determine the method to remove the fractional bits when converting output result back to integer format. 1: Round half up 2: Round half even 3: Truncate to integer
0x0034 to 0x011F	-	-	-	Unused.
Control and debug registers For more information, refer to <i>Control Packets</i>				
0x0120	IMG_INFO_WIDTH	RW	RO	When Lite mode is on, the expected width of the incoming video fields. When Lite mode is off, the received width in image information packets.
0x0124	IMG_INFO_HEIGHT	RW	RO	When Lite mode is on, the expected height of the incoming video fields. When Lite mode is off, the received height in image information packets.
0x0128	IMG_INFO_INTERLACE	RW	RO	When Lite mode is on, the expected interlace information of the incoming video fields. When Lite mode is off, the received interlace information in image information packets.
0x012C	RESERVED	RW	RO	Unused.
0x0130	IMG_INFO_COLORSPACE	RW	RO	When Lite mode is on, the expected color space of the incoming video fields. When Lite mode is off, the received color space in image information packets.
0x0134	IMG_INFO_SUBSAMPLING	RW	RO	When Lite mode is on, the expected chroma sub-sampling of the incoming video fields. When Lite mode is off, the received chroma sub-sampling in image information packets.
0x0138	IMG_INFO_COSITING	RW	RO	When Lite mode is on, the expected chroma co-siting of the incoming video fields. When Lite mode is off, the received chroma co-siting in image information packets.
0x013C	IMG_INFO_FIELD_COUNT	-	RO	The received field count field in image information packets.
0x0140	STATUS	RO	RO	Bit 0: Status bit. 1 means Color Space Converter is processing a video field, 0 otherwise. When Lite mode is off: Bit 1 : Pending register updates bit.
continued...				

(73) When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

Address	Register	Access		Description
		Lite (73)	Full	
				Any writes to the color space converter specification registers (0x0148 - 0x0178) cause the IP to raise the pending register updates bit, to indicate outstanding changes to the color space converter settings. The IP lowers this bit at the start of the first packet for a new frame (first packet after the end of packet) after a write to the COMMIT register.
0x0144	COMMIT	RW	RW	The IP uses this register with the STATUS register pending register updates bit. Write any value to this register after writing new values to any of the color space converter specification registers (0x0148 - 0x0178). The color space converter switches to the new color space converter specification values at the start of the next video frame after you write to this register. The commit register ensures atomic register updates, avoiding the IP only applying some of the updated register values before the next incoming frame
Specification registers				
0x0148	Coefficient A0	RW	RW	Write to these registers to set the coefficient and summand values. The coefficients and summands are stored in the format indicated by the IP's parameters. The coefficient and summand registers use integer numbers, with specified number of integer and fractional bits. If signed representation is on, the IP uses signed 2's complement.
0x014C	Coefficient B0			
0x0150	Coefficient C0			
0x0154	Coefficient A1			
0x0158	Coefficient B1			
0x015C	Coefficient C1			
0x0160	Coefficient A2			
0x0164	Coefficient B2			
0x0168	Coefficient C2			
0x016C	Summand S0			
0x0170	Summand S1			
0x0174	Summand S2			
0x0178	Output Color Space	RW	RW	Only applicable when Lite mode is off. Write to this register to set the value of color space field in outgoing image information packets from the Color Space Converter. Refer to the Altera Streaming Video Protocol Specification for more information about color space encoding and image information packets.

Register Bit Descriptions

Table 393. VID_PID

Name	Bits	Description
Color Space Converter version ID and product ID	31:0	This register always returns 0x6AF7_022F.
continued...		

⁽⁷³⁾ When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

- 15:0 is the product ID and always returns 0x022F
- 31:16 is the vendor ID and always returns 0x6AF7

Table 394. VERSION

Name	Bits	Description
Register map version	7:0	Register map version.
IP patch revision	15:8	-
IP update revision	23:16	Updated when the IP version changes.
IP major revision	31:24	Updated when the IP version changes.

Table 395. LITE_MODE

Name	Bits	Description
Lite mode parameterization bit	0	Returns 1 if lite mode is on.
Unused	31:1	Unused.

Table 396. DEBUG_ENABLED

Name	Bits	Description
Debug features parameterization bit	0	Returns 1 if debug features is on.
Unused	31:1	Unused.

Table 397. BPS_IN

Name	Bits	Description
BPS_IN	15:0	Returns the value of the BPS_IN parameter, indicating the bit per symbol the input interface is configured for.
Unused	31:16	Unused.

Table 398. BPS_OUT

Name	Bits	Description
BPS_OUT	15:0	Returns the value of the BPS_OUT parameter, indicating the bit per symbol the output interface is configured for.
Unused	31:16	Unused.

Table 399. FRAC_BITS

Name	Bits	Description
Coefficient and summand fractional bits	15:0	Returns the value of the coefficient and summand fractional bits parameter, indicating the number of fractional bits used by the fixed-point data type to store the coefficients and summands.
Unused	31:16	Unused.

Table 400. COEFFS_SIGNED

Name	Bits	Description
Coefficients signed	15:0	Returns the value of the coefficient signed parameter indicating coefficients are stored in a signed format.
Unused	31:16	Unused.

Table 401. COEFFS_INT_BITS

Name	Bits	Description
Coefficient integer bits	15:0	Returns the value of the coefficient integer bits parameter, indicating number of integer bits used by fixed-point format to represent coefficients.
Unused	31:16	Unused.

Table 402. SUMMANDS_SIGNED

Name	Bits	Description
Summands signed	15:0	Returns the value of the summands signed parameter indicating summands are stored in a signed format.
Unused	31:16	Unused.

Table 403. SUMMANDS_INT_BITS

Name	Bits	Description
Summand integer bits	15:0	Returns the value of the summand integer bits parameter, indicating number of integer bits used by fixed-point format to represent summands.
Unused	31:16	Unused.

Table 404. BINARY_POINT_RIGHT_MOVE

Name	Bits	Description
Binary point right move	15:0	Returns the value of the move binary point right parameter, indicating the number of places the binary point is shifted by when scaling the result.
Unused	31:16	Unused.

Table 405. ROUND_METHOD

Name	Bits	Description
Round method	15:0	Returns the value of the round method parameter, indicating method used to round excess fractional bits from the result. 1: Round half up 2: Round half even 3: Truncate to integer
Unused	31:16	Unused.

Table 406. IMG_INFO_WIDTH

Name	Bits	Description
Width bits	15:0	When Lite mode is on, write to this register to set the expected width of the incoming video fields.
<i>continued...</i>		

		When Lite mode is off with Debug features on, this register returns the width field from the most recently received.
unused	31:16	Unused.

Table 407. IMG_INFO_HEIGHT

Name	Bits	Description
Height bits	15:0	When Lite mode is on, write to this register to set the expected height of the incoming video fields. When Lite mode is off with Debug features on, this register returns the height field from the most recently received. image information packet.
unused	31:16	Unused.

Table 408. IMG_INFO_INTERLACE

Name	Bits	Description
interlaced bits	3:0	When Lite mode is on, write to this register to set the expected interlacing of the incoming video fields. When Lite mode is off with Debug features on, this register returns the intlaceNibble field from the most recently received image information packet.
unused	31:4	Unused.

Table 409. IMG_INFO_COLORSPACE

Name	Bits	Description
Colorspace code bits	6:0	When Lite mode is on, write to this register to set the expected color space of the incoming video fields. When Lite mode is off with Debug features on, this register returns the CSP field from the most recently received image information packet.
unused	31:7	Unused.

Table 410. IMG_INFO_SUBSAMPLING

Name	Bits	Description
Subsampling code bits	1:0	When Lite mode is on, write to this register to set the expected chroma sub-sampling of the incoming video fields. When Lite mode is off with Debug features on, this register returns the SUBSA field from the most recently received image information packet.
unused	31:2	Unused.

Table 411. IMG_INFO_COSITING

Name	Bits	Description
Cosite code bits	1:0	When Lite mode is on, write to this register to set the expected chroma co-siting of the incoming video fields. When Lite mode is off with Debug features on, this register returns the COSITE field from the most recently received image information packet.
unused	31:2	Unused.

Table 412. STATUS

Name	Bits	Description
Status bit	0	1 means the Color Space Converter is processing a video field, 0 otherwise.
Pending register updates bit	1	1 means the Color Space Converter has pending updates, 0 otherwise.

Table 413. COMMIT

Name	Bits	Description
Commit bits	31:0	Write to any bits to trigger a commit.

Table 414. COEFFICIENT_VALUE

Name	Bits	Description
Coefficient_value	(Coeff_signed + coeff_int_bits + Coeff_sum_frac_bits):0	This block of registers allows the setting of coefficient. The coefficients are represented by fixed-point format, and are determined by the IP's parameterization.

Table 415. SUMMAND_VALUES

Name	Bits	Description
Summand_value	(Summand_signed + summand_int_bits + Coeff_sum_frac_bits):0	This block of registers allows the setting of summand. The summands are represented by fixed-point format, and are determined by the IP's parameterization.

Table 416. OUTPUT_COLORSPACE

Name	Bits	Description
Output Colorspace	6:0	Write to this register to set the color space code for outgoing image information packets when you turn off Lite mode . Refer to Altera Streaming Video Protocol Specification for more details on color space field of image information packets.

23.5. Color Space Converter IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_csc_regs.h`

Include file: `intel_vvp_csc.h`

Table 417. Color Space Converter IP Software API

Name	Description
<code>intel_vvp_csc_init</code>	Initialize the Color Space Converter instance
<code>Intel_vvp_core_*</code>	Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i> . Writable when Lite is on. Readable when Lite is off and Debug features is on.
<code>intel_vvp_csc_get_lite_mode</code>	Returns if Lite mode is on
continued...	

Name	Description
intel_vvp_csc_get_debug_enabled	Returns if Debug features is on
intel_vvp_csc_get_bits_per_sample_in	Returns the number of input bits per color sample
intel_vvp_csc_get_bits_per_sample_out	Returns the number of output bits per color sample
intel_vvp_csc_are_coeffs_signed	Returns if the nine coefficients have a sign bit and can be negative
intel_vvp_csc_get_coeffs_int_bits	Returns the number of integer bits (excluding sign bit) available to quantize coefficients
intel_vvp_csc_are_summands_signed	Returns if the three summands have a sign bit and can be negative
intel_vvp_csc_get_summands_int_bits	Returns the number of integer bits (excluding sign bit) available to quantize summands
intel_vvp_csc_get_coeffs_frac_bits	Returns the number of fractional bits available to quantize both coefficients and summands
intel_vvp_csc_get_binary_point_right_move	Returns the fixed scaling factor applied to values at the output of the IP
intel_vvp_csc_get_rounding_method	Returns the method used to convert floating-point values back to integer values at the output of the IP
intel_vvp_csc_is_running	Returns if the IP is currently processing video data
intel_vvp_csc_get_commit_status	Returns if there are uncommitted writes
intel_vvp_csc_get_status	Reads the status register
intel_vvp_csc_set_quantized_coeff_data	Program the IP with a new set of quantized coefficients/summands
intel_vvp_csc_get_quantized_coeff_data	Retrieve the set of quantized coefficients/summands programmed into the IP
intel_vvp_csc_set_coeff_data	Program the IP with a new set of floating-point coefficients/summands
intel_vvp_csc_get_coeff_data	Retrieve the set of floating-point coefficients/summands programmed into the IP
intel_vvp_csc_set_output_color_space	Selects the current output color space (full mode)
intel_vvp_csc_get_output_color_space	Returns the current output color space (full mode)
intel_vvp_csc_commit_writes	Commit all outstanding writes

enum eIntelVvpCscRounding

- Members**
- kIntelVvpCscRoundUp (1) – Rounding up (half up towards positive infinity)
 - kIntelVvpCscRoundHalfEven (2) – Rounding half to even (bankers' rounding)
 - kIntelVvpCscRoundTruncate (3) – Truncation (no rounding)
 - kIntelVvpCscRoundInvalid (-1) – Invalid rounding method (used to indicate errors)

Description Enumeration for the rounding method used by the IP

enum eIntelVvpCscColorSpace

- Members**
- kIntelVvpCsRgb (0) – Rounding up (half up towards positive infinity)
 - kIntelVvpCsYcc (1) – Rounding half to even (bankers' rounding)
 - kIntelVvpCsMono (2) – Truncation (no rounding)
 - kIntelVvpCscRoundInvalid (-1) – Invalid rounding method (used to indicate errors)

Description Enumeration for the rounding method used by the IP

struct intel_vvp_coefficients

- Members**
- struct { float c1; float c2; float c3; } coeffs[3]; – Matrix of 9 coefficients
 - float s[3] – Vector of 3 summands

Description The coefficients and summands as floating-point values

struct intel_vvp_quantized_coefficients

- Members**
- struct { int32_t c1; int32_t c2; int32_t c3; } coeffs[3]; – Matrix of 9 coefficients
 - int32_t s[3] – Vector of 3 summands

Description The coefficients and summands as quantized integer values

intel_vvp_csc_init

Prototype

```
int intel_vvp_csc_init(intel_vvp_csc_instance *instance,
intel_vvp_core_base base);
```

Description Initializes a Color Space Converter instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance does not fully initialize and should not be used further by the application if returning a non-zero error code.

Arguments

- instance – pointer to the intel_vvp_csc_instance software driver instance structure
- base – base address of the register map

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product_id does not match with the Color Space Converter product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpCscRegMapVersionErr if the register map is not supported

intel_vvp_csc_get_lite_mode

Prototype

```
bool intel_vvp_csc_get_lite_mode(intel_vvp_csc_instance
*instance);
```

Description

Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_csc_instance fully initialized

Arguments

instance – pointer to the intel_vvp_csc_instance software driver instance structure

Returns

true if the IP is parameterized in lite mode

intel_vvp_csc_get_debug_enabled

Prototype

```
bool intel_vvp_csc_get_debug_enabled(intel_vvp_csc_instance
*instance);
```

Description

Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_csc_instance fully initialized.

Arguments

instance – pointer to the intel_vvp_csc_instance software driver instance structure

Returns

true if the IP is parameterized with debug features enabled

intel_vvp_csc_get_bits_per_sample_in

Prototype

```
uint8_t
intel_vvp_csc_get_bits_per_sample_in(intel_vvp_csc_instance
*instance);
```

Description

Returns the value of the BPS_IN register. The instance must be a valid intel_vvp_csc_instance fully initialized.

Arguments

instance – pointer to the intel_vvp_csc_instance software driver instance structure

Returns

the input bits per sample parameter used to generate the IP.

intel_vvp_csc_get_bits_per_sample_out

Prototype

```
uint8_t
intel_vvp_csc_get_bits_per_sample_out(intel_vvp_csc_instance
*instance);
```

Description Returns the value of the BPS_OUT register. The instance must be a valid intel_vvp_csc_instance fully initialized.

Arguments instance – pointer to the intel_vvp_csc_instance software driver instance structure

Returns the output bits per sample parameter used to generate the IP.

intel_vvp_csc_are_coeffs_signed

Prototype

```
bool intel_vvp_csc_are_coeffs_signed(intel_vvp_csc_instance*
instance);
```

Description Returns the value of the COEFFS_SIGNED register. The instance must be a valid intel_vvp_csc_instance fully initialized.

Arguments instance – pointer to the intel_vvp_csc_instance software driver instance structure

Returns If signed coefficients have been allowed at IP generation time.

intel_vvp_csc_get_coeffs_int_bits

Prototype

```
uint8_t intel_vvp_csc_get_coeffs_int_bits
(intel_vvp_csc_instance *instance);
```

Description Returns the value of the COEFFS_INT_BITS register. The instance must be a valid intel_vvp_csc_instance fully initialized.

Arguments instance – pointer to the intel_vvp_csc_instance software driver instance structure

Returns the number of integer bits set at IP generation time to parameterize the quantization of the coefficients

intel_vvp_csc_are_summands_signed

Prototype

```
bool
intel_vvp_csc_are_summands_signed(intel_vvp_csc_instance*
instance);
```

Description Returns the value of the SUMMANDS_SIGNED register. The instance must be a valid intel_vvp_csc_instance fully initialized.

Arguments instance – pointer to the intel_vvp_csc_instance software driver instance structure

Returns If signed summands have been allowed at IP generation time.

intel_vvp_csc_get_summands_int_bits

Prototype

```
uint8_t intel_vvp_csc_get_summands_int_bits
(intel_vvp_csc_instance *instance);
```

Description Returns the value of the SUMMANDS_INT_BITS register. The instance must be a valid intel_vvp_csc_instance fully initialized.

Arguments instance – pointer to the intel_vvp_csc_instance software driver instance structure

Returns the number of integer bits set at IP generation time to parameterize the quantization of the summands

intel_vvp_csc_get_coeffs_frac_bits

Prototype

```
uint8_t intel_vvp_csc_get_coeffs_frac_bits
(intel_vvp_csc_instance *instance);
```

Description Returns the value of the FRAC_BITS register. The instance must be a valid intel_vvp_csc_instance fully initialized.

Arguments instance – pointer to the intel_vvp_csc_instance software driver instance structure

Returns the number of fractional bits set at IP generation time to parameterize the quantization of the coefficients and summands

intel_vvp_csc_get_binary_point_right_move

Prototype

```
int8_t
intel_vvp_csc_get_binary_point_right_move(intel_vvp_csc_instance
*instance);
```

Description Returns the value of the BINARY_POINT_RIGHT_MOVE register. The fixed scaling factor applied to the IP output samples and selected at IP generation time. Output values are scaled by $2^{(\text{binary_point_right_move})}$. This is usually done to account for differences between bps_in and bps_out. The instance must be a valid intel_vvp_csc_instance fully initialized.

Arguments instance – pointer to the intel_vvp_csc_instance software driver instance structure

Returns The binary_point_right_move value selected at generation time

intel_vvp_csc_get_rounding_method

Prototype

```
eIntelVvpCscRounding
intel_vvp_csc_get_rounding_method(intel_vvp_csc_instance
*instance);
```

Description Returns the value of the ROUND_METHOD register. The instance must be a valid intel_vvp_csc_instance fully initialized. This is the method to convert fixed-point values to integer values at the end of the CSC processing.

Arguments instance – pointer to the intel_vvp_csc_instance software driver instance structure

Returns The IP rounding method.

intel_vvp_csc_is_running

Prototype

```
bool intel_vvp_csc_is_running(intel_vvp_csc_instance*
instance);
```

Description Reads and returns the running bit of the STATUS register. The instance must be a valid intel_vvp_csc_instance fully initialized.

Arguments instance – pointer to the intel_vvp_csc_instance software driver instance structure

Returns True is the IP is currently receiving/storing an input field, false when between two input fields

intel_vvp_csc_get_commit_status

Prototype

```
bool intel_vvp_csc_get_commit_status(intel_vvp_csc_instance*
instance);
```

Description Reads and returns the commit pending bit of the STATUS register. The instance must be a valid intel_vvp_csc_instance fully initialized and configured in full mode.

Arguments instance – pointer to the intel_vvp_csc_instance software driver instance structure

Returns True is the IP is configured in full mode and if the agent interface contains uncommitted writes.

intel_vvp_csc_get_status

Prototype

```
uint32_t intel_vvp_csc_get_status(intel_vvp_csc_instance*
instance);
```


- Description* Reads the STATUS register. The instance must be a valid intel_vvp_csc_instance fully initialized.
- Arguments* instance – pointer to the intel_vvp_csc_instance software driver instance structure
- Returns* The content of the STATUS register

intel_vvp_csc_set_quantized_coeff_data

Prototype

```
int
intel_vvp_csc_set_quantized_coeff_data(intel_vvp_csc_instance*
instance, const intel_vvp_quantized_coefficients*
quantized_coeffs);
```

- Description* Programs the IP with the set of coefficients and summands. The call checks for correctness of the quantized values and returns an error if invalid bits are set. Sign extension to the full 32 bit range is accepted. Quantization errors do not prevent the coefficients from being updated
- Arguments* instance – pointer to the intel_vvp_csc_instance software driver instance structure
- quantized_coeffs – pointer to the structure containing the 9 quantized coefficients and summands
- Returns* kIntelVvpCoreOk in case of success, negative error code otherwise
- kIntelVvpCoreInstanceErr if the instance is invalid
- kIntelVvpCoreNullPtrErr if quantized_coeffs is a NULL pointer
- kIntelVvpCscQuantizationErr if the quantized coefficients/summands are out-of-range

intel_vvp_csc_get_quantized_coeff_data

Prototype

```
int
intel_vvp_csc_get_quantized_coeff_data(intel_vvp_csc_instance*
instance, intel_vvp_quantized_coefficients* quantized_coeffs)
```

- Description* Reads and returns the current set of coefficients programmed in the IP. The instance must be a valid intel_vvp_csc_instance fully initialized and configured with debug enabled.
- Arguments* instance – pointer to the intel_vvp_csc_instance software driver instance structure
- quantized_coeffs – storage space to return the quantized coefficients

Returns

- kIntelVvpCoreOk or a negative error code in case of error
- kIntelVvpCoreInstanceErr, if the instance is invalid
- kIntelVvpCoreNullPtrErr, if quantized_coeffs is the NULL pointer

intel_vvp_csc_set_coeff_data

Prototype

```
int intel_vvp_csc_set_coeff_data(intel_vvp_csc_instance*
instance, const intel_vvp_coefficients* coeffs, int
summand_rescale);
```

Description Programs the IP with the set of coefficients and summands. The call quantizes the user-defined floating-point coefficients and returns an error if values are out-of-range. Quantization errors do not prevent the coefficients from being updated

Arguments

- instance – pointer to the intel_vvp_csc_instance software driver instance structure
- coeffs – pointer to the structure containing the 9 floating-point coefficients and 3 summands
- summand_rescale – scaling factor ($2^{\text{summand_rescale}}$) applied to the input summand values. Non-zero values are typically used when using a ready-made conversion table designed for a number of bits per color sample that does not match with the input bits per sample value used to parameterize the IP

Returns

- kIntelVvpCoreOk in case of success, negative error code otherwise
- kIntelVvpCoreInstanceErr if the instance is invalid
- kIntelVvpCoreNullPtrErr if coeffs is a NULL pointer
- kIntelVvpCscQuantizationErr if the coefficients/summands are out-of-range and cannot be quantized accurately

intel_vvp_csc_get_coeff_data

Prototype

```
int intel_vvp_csc_get_coeff_data(intel_vvp_csc_instance*
instance, intel_vvp_coefficients* coeffs)
```

Description Reads and returns the current set of coefficients programmed in the IP. The instance must be a valid intel_vvp_csc_instance fully initialized and configured with debug enabled. Coefficients are converted back to floating-point values before being returned.

Arguments

- instance – pointer to the intel_vvp_csc_instance software driver instance structure
- coeffs – storage space to return the floating-point coefficients

Returns kIntelVvpCoreOk or a negative error code in case of error
kIntelVvpCoreInstanceErr, if the instance is invalid
kIntelVvpCoreNullPtrErr, if coeffs is the NULL pointer

intel_vvp_csc_set_output_colorspace

Prototype

```
int  
intel_vvp_csc_set_output_colorspace(intel_vvp_csc_instance*  
instance, int8_t output_color_space);
```

Description Writes the required output colorspace to the OUTPUT_CS register. The instance must be a valid intel_vvp_csc_instance fully initialized and configured in full mode.

Arguments instance – pointer to the intel_vvp_csc_instance software driver instance structure
output_color_space – a colorspace code

Returns kIntelVvpCoreOk or a negative error code in case of error
kIntelVvpCoreInstanceErr, if the instance is invalid
kIntelVvpCscParamErr, if the output colorspace is invalid (<0)

intel_vvp_csc_get_output_colorspace

Prototype

```
int8_t  
intel_vvp_csc_get_output_colorspace(intel_vvp_csc_instance*  
instance);
```

Description Reads the OUTPUT_CS register. The instance must be a valid intel_vvp_csc_instance fully initialized and configured in full mode with debug enabled.

Arguments instance – pointer to the intel_vvp_csc_instance software driver instance structure

Returns the last value written to the OUTPUT_CS register, negative value in case of errors

intel_vvp_csc_commit_writes

Prototype

```
int intel_vvp_csc_commit_writes(intel_vvp_csc_instance*  
instance)
```

Description Commits all pending writes before processing the next field. The instance must be valid and parameterized in full mode.

Arguments instance – pointer to the intel_vvp_csc_instance software driver instance structure

Returns kIntelVvpCoreOk (0) for success, negative error code otherwise

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

24. Defective Pixel Correction IP

24.1. About the Defective Pixel Correction IP

The Defective Pixel Correction IP removes impulse noise associated with defective pixels in image sensors. Hardware corruption in image sensor circuitry can cause impulse noise, which in certain pixels gives a constant random value, regardless of the scene. The IP operates only on 2x2 Bayer color filter array images. The IP identifies defective pixels using a configurable nonlinear filter and corrects them via selective filtering.

The IP only supports a lite variant only. For more information about full and lite variants, refer to the *Altera Streaming Video Protocol Specification*.

Related Information

[Altera Streaming Video Protocol Specification](#)

24.1.1. Defective Pixel Correction IP Performance and Resources

Altera provides resource and utilization data for guidance only. The IP resource utilization depends on the device family, compilation tool version, compilation seed randomization, and other parameters.

Table 418. Defective Pixel Correction IP Performance and Resources

Device	Pixel in Parallel	Bits per Color Sample	Max Resolution	ALM	M20k	f _{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	1	10	2,048	1,141	5	790
	1	16	2,048	1,483	9	731
	4	10	8,192	2,956	20	759
	4	16	8,192	4,237	33	708
Arria 10 speed grade 1 (10AS066H1F34E1HG)	1	10	2,048	1,025	5	480
	1	16	2,048	1,376	9	479
	2	10	4,096	1,667	10	491
	2	16	4,096	2,360	17	461
Cyclone 10 GX speed grade 5 (10CX220YF672E5G)	1	10	2,048	1,027	5	420
	1	16	2,048	1,370	9	446
Stratix 10 speed grade 1 (1SX280LN2F43E1VG)	1	10	2,048	1,144	5	665
continued...						

Device	Pixel in Parallel	Bits per Color Sample	Max Resolution	ALM	M20k	f _{MAX} (MHz)
	1	16	2,048	1,491	9	626
	2	10	4,096	1,770	10	670
	2	16	4,096	2,392	17	588

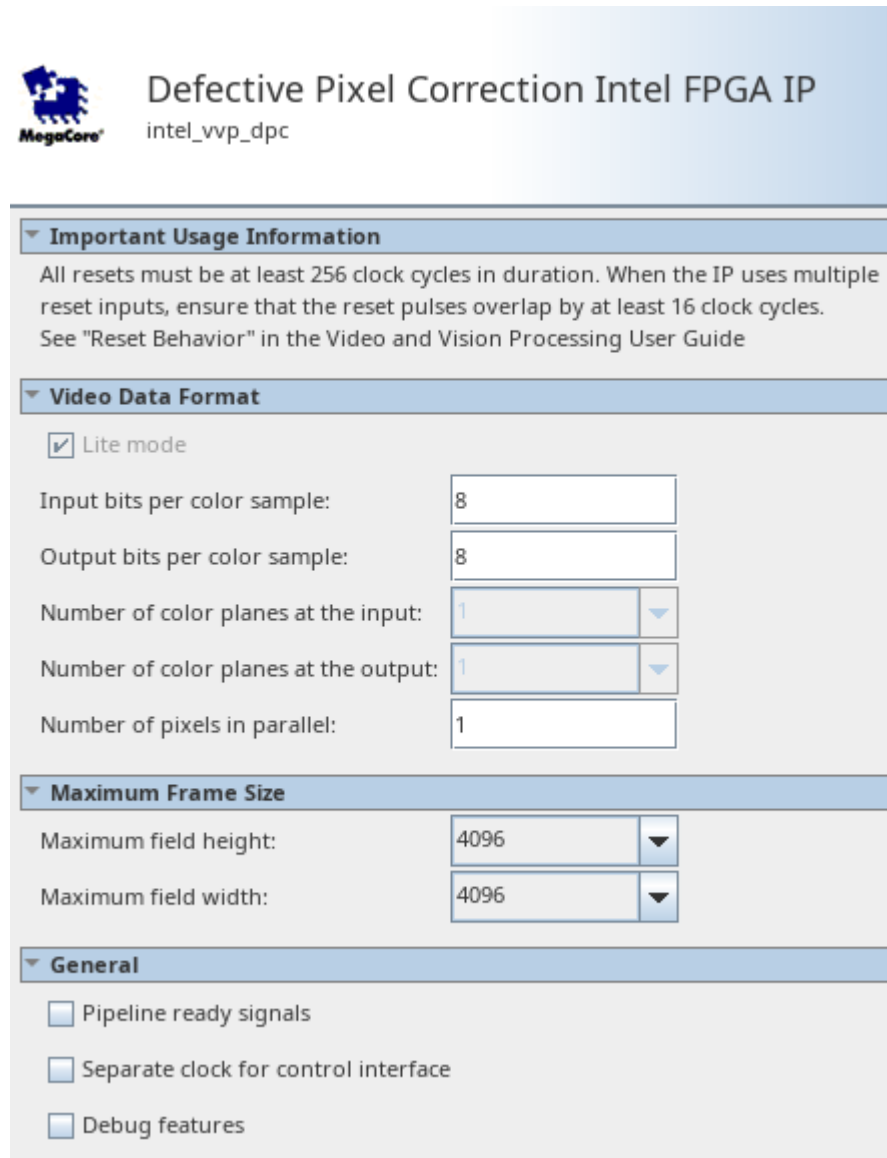
24.2. Defective Pixel Correction IP Parameters

The IP offers run- and compile-time parameters.

Table 419. Defective Pixel Correction IP Parameters

Parameter	Values	Description
Video Data Format		
Lite mode	On	The IP only supports lite mode
Input bits per color sample	8 to 16	Select the number of input bits per color sample.
Output bits per color sample	8 to 16	Select the number of output bits per color sample.
Number of color planes at the input	1	The IP only supports 1 color plane at its input
Number of color planes at the output	1	The IP only supports 1 color planes at its output
Number of pixels in parallel	1 to 8	Select the number of pixels in parallel.
Maximum Frame Size		
Maximum field height	1024, 2048, 4096, 8192, 16384	Specify the maximum height of incoming frames
Maximum field width	2048, 4096, 8192, 16384	Specify the maximum width of incoming frames
General		
Pipeline ready signal	On or off	Turn on to add extra pipeline registers to the AXI4-S tready signals
Separate clock for control interface	On or off	Turn on for a separate clock for the control agent interface
Debug features	On or off	Turn on to read back frame information registers and debugging information registers via the control agent interface.

Figure 70. Defective Pixel Correction IP GUI



Defective Pixel Correction Intel FPGA IP
intel_vvp_dpc

Important Usage Information
All resets must be at least 256 clock cycles in duration. When the IP uses multiple reset inputs, ensure that the reset pulses overlap by at least 16 clock cycles. See "Reset Behavior" in the Video and Vision Processing User Guide

Video Data Format

☒ Lite mode

Input bits per color sample: 8

Output bits per color sample: 8

Number of color planes at the input: 1

Number of color planes at the output: 1

Number of pixels in parallel: 1

Maximum Frame Size

Maximum field height: 4096

Maximum field width: 4096

General

☐ Pipeline ready signals

☐ Separate clock for control interface

☐ Debug features

24.3. Defective Pixel Correction IP Functional Description

The Defective Pixel Correction IP identifies defective pixels on 2x2 Bayer images and replaces them with the corrected pixel values.

The IP works on a 5x5 pixel neighborhood.

Figure 71. A Bayer Color Filter Array

For an 6x6 section of an image and example pixel neighborhood for green, red, and blue pixels.

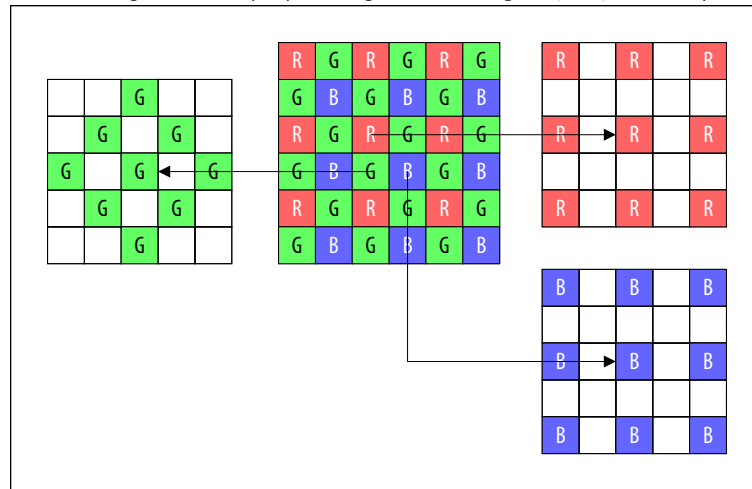
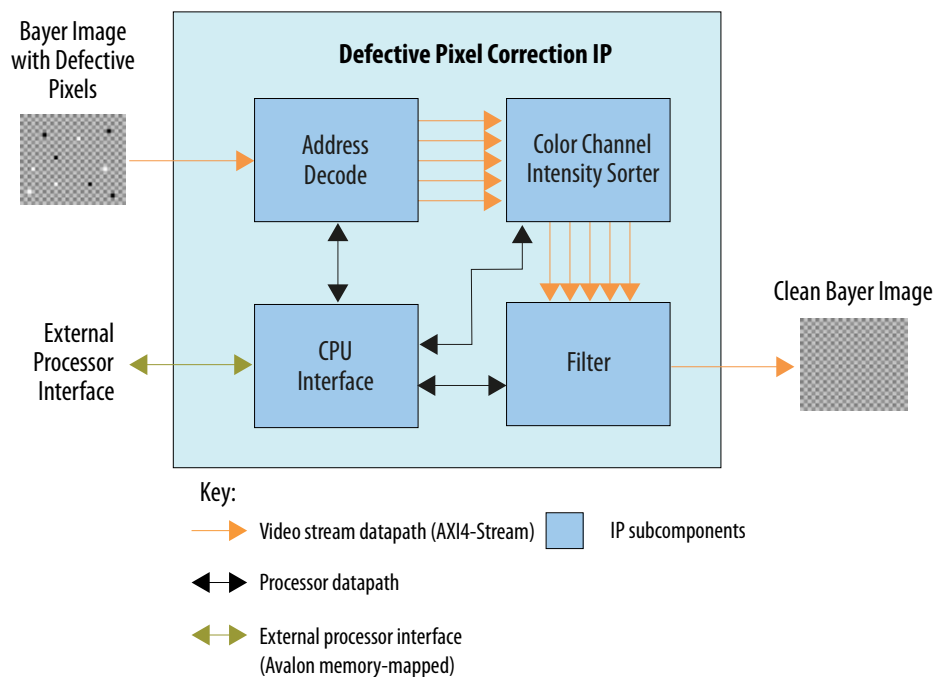


Figure 72. Defective Pixel Correction IP Block Diagram



The color channel intensity sorter selects pixels closest to the center pixel and sorts them in order of their intensities. A filter calculates a corrected pixel value from the sorted group of pixels depending on the sensitivity level that you set.

The IP identifies and filters defect pixels dynamically and does not support static defect pixel correction. The sensitivity level determines the filtering strength. When you set the sensitivity level to the weakest value, the IP alters the pixel value only if

its original value falls outside the value range of its 5x5 neighborhood. As the sensitivity increases, the range reduces closer to the median value. The IP passes the image unmodified if you set the `Bypass` bit in the `CONTROL` register.

You must set the color filter array phase to its correct value, which depends on the framing, rotation, and flipping options you apply to the original image that an imaging sensor captures. If you do not set the correct color filter array phase, the IP may mix color channels and produce incorrect outputs.

Information registers indicate how many pixels the IP modifies on any given frame by the IP. This information can indicate if the sensitivity setting is too high or whether the number of defective pixels is stable, or fluctuates depending on the conditions. These registers provide information about how many pixels are lower (`PIX_LO_COUNTER`) than the specified sensitivity setting or higher (`PIX_HI_COUNTER`) (refer to *Defective Pixel Correction IP Registers*). The IP instantiates these pair of registers for each **Number of pixels in parallel**. The IP shows the counter values of the instance that you select via `PIP Clip Counter Selection` value in the `CONTROL` register. You start updating the counter values by setting the `Freeze Stats Request` value in the `CONTROL` register. The IP counts the number of pixels it filters for exactly one frame. The IP sets `Stats are Frozen` bit of the `STATUS` register to indicate you may read the counter values. You should clear a previous freeze request and select the counter pair you want the IP to capture before starting a new freeze request.

24.3.1. Defective Pixel Correction IP Interfaces

Table 420. Defective Pixel Correction Clock and Reset Interfaces

Name	Direction	Width	Description
agent_reset_reset	In	1	Reset for the memory-mapped control agent interface. Only if you turn on Separate clock for control interface .
main_clock_clk	In	1	AXI4-S processing clock.
main_reset_rst	In	1	AXI4-S processing reset.
agent_clock_clk	In	1	Clock for the memory-mapped control agent interface. Only if you turn on Separate clock for control interface .

Table 421. Defective Pixel Correction Avalon memory-mapped host Interfaces

Name	Direction	Width	Description
av_mm_control_agent_address	In	7	Avalon memory-mapped agent address
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write.
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data.
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable.
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read.
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data.
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read.
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request.

Table 422. Defective Pixel Correction Altera Streaming Video Interfaces

Name	Direction	Width	Description
Input			
axi4s_vid_in_tdata	In	(74)	AXI4-S data in.
axi4s_vid_in_tvalid	In	1	AXI4-S data valid.
axi4s_vid_in_tuser	In	(75)	[0] AXI4-S start of video frame. [N-1:1] Unused.
axi4s_vid_in_tlast	In	1	AXI4-S end of packet.
axi4s_vid_in_tready	Out	1	AXI4-S data ready.
Output			
axi4s_vid_out_tdata	Out	(74)	AXI4-S data out.
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid.
axi4s_vid_out_tuser	Out	(75)	[0] AXI4-S start of video frame. [N-1:1] Unused.
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet.
axi4s_vid_out_tready	In	1	AXI4-S data ready

Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

24.4. Defective Pixel Correction IP Registers

Each register is either read-only (RO) or read-write (RW).

In the software API the register names appear with a prefix of `INTEL_VVP`, `INTEL_VVP_CORE` or `INTEL_VVP_DPC` as appropriate and with an optional `REG` suffix

Table 423. Defective Pixel Correction Registers

Address	Register	Access		Description
		Lite ⁽⁷⁶⁾	Full	
Parameterization registers				
0x0000	VID_PID	RO	N/A	Read this register to retrieve the ID of the Defective Pixel Correction IP. This register always returns 0x6FA7_0175.
0x0004	VERSION	RO	N/A	Read this register for the IP version information.
continued...				

(74) The equation gives all tdata widths sizes in these interfaces: pixels in parallel × max (round (bits per color sample × number of color planes, 8), 16)

(75) This equation gives all tuserwidths sizes in these interfaces: N = ceil (tdata width / 8)

(76) Registers are RW only if you also turn on **Debug features**, otherwise they are WO.

Address	Register	Access		Description
		Lite ⁽⁷⁶⁾	Full	
0x0008	LITE_MODE	RO	N/A	Read this register to determine if Lite mode is on. This register always returns 1.
0x000c	DEBUG_ENABLED	RO	N/A	Read this register to determine if Debug features are on. This register returns 0 for off and 1 for on.
0x0010	BPS_IN	RO	N/A	Read this register to determine the bits per symbol for the input data.
0x0014	BPS_OUT	RO	N/A	Read this register to determine the bits per symbol for the output data.
0x0018	NUM_COLOR_IN	RO	N/A	Read this register to determine the number of color planes for the input data. This register always returns 1.
0x001c	NUM_COLOR_OUT	RO	N/A	Read this register to determine the number of color planes for the output data. This register always returns 1.
0x0020	PIP	RO	N/A	Read this register to determine the number of pixels in parallel.
0x0024	MAX_WIDTH	RO	N/A	Read this register to determine the maximum supported input field width.
0x0028	MAX_HEIGHT	RO	N/A	Read this register to determine the maximum supported input field height.
0x002c to 0x011F	-	-	-	Reserved
Control, debug, and statistics registers				
0x0120	IMG_INFO_WIDTH	RW	N/A	The expected width of the incoming video fields.
0x0124	IMG_INFO_HEIGHT	RW	N/A	The expected height of the incoming video fields.
0x0128 to 0x013F	-	-	-	Reserved
0x0140	STATUS	RO	N/A	Read this register for information about the IP status.
0x0144	FRAME_STATS	RO	N/A	Read this register for some frame statistics.
0x0148	PIX_LO_COUNTER	RO	N/A	For the selected parallel pixel pipeline, read this register for the current count of pixels that have been clipped to the low value set via the sensitivity setting.
0x014c	PIX_HI_COUNTER	RO	N/A	For the selected parallel pixel pipeline, read this register for the current count of pixels that have been clipped to the high value set via the sensitivity setting.
0x0150	-	-	-	Reserved
0x0154	CONTROL	RW	N/A	Control bits and fields of Defective Pixel Correction IP
0x0158 to 0x01FF	-	-	-	Reserved

⁽⁷⁶⁾ Registers are RW only if you also turn on **Debug features**, otherwise they are WO.

Register Bit Descriptions

Table 424. STATUS

Name	Bits	Description
Reserved	31:3	Reserved.
Stats are Frozen	2	IP sets this bit to indicate it stopped updating the frame statistics and low and high pixel counters.
Reserved	1	Reserved.
Running	0	When 1, the IP is processing data.

Table 425. FRAME_STATS

Name	Bits	Description
Reserved	31:8	Reserved..
Checksum	7:0	A simple checksum of the frame.

Table 426. CONTROL

Name	Bits	Description
Freeze Stats Request	31	Allows atomic access across all statistics registers. When set, the IP updates the frame statistics and low and high pixel counters at the next frame boundary. Raise stats are frozen flag in the status register. The statistics remain unchanged until this bit is cleared.
Reserved	30:9	Write 0.
PIP clip counter selection	8:6	Specifies which pixels in parallel pipelines the IP maps to the low and high pixel counters.
Clip counter reset	5	Set to reset high and low clip counters across all pixels in parallel pipelines. Clear to enable counters.
Sensitivity level	4:3	Specifies how strong the correction factor is: 00: Very weak 01: Weak 10: Strong 11: Very strong
Color filter array phase.	2:1	Specifies 2x2 color filter order starting from the top left corner of the image. 00 01 10 11 RG GR GB BG GB BG RG GR
Bypass	0	Set to bypass the Defective Pixel Correction IP. When set, the IP passes the input image unmodified.

24.5. Defective Pixel Correction IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_dpc_regs.h`

Include file: intel_vvp_dpc.h

Table 427. Defect Pixel Correction IP Software API

Name	Description
intel_vvp_dpc_init	Initialize an IP instance
intel_vvp_core_*	Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i>
intel_vvp_dpc_get_lite_mode	Returns if Lite mode is on
intel_vvp_dpc_get_debug_enabled	Returns if Debug features is on
intel_vvp_dpc_get_bits_per_sample_in	Returns the number of bits per color sample for the streaming input interface
intel_vvp_dpc_get_bits_per_sample_out	Returns the number of bits per color sample value for the streaming output interface
intel_vvp_dpc_get_num_color_planes_in	Returns the number of color planes of the streaming input interface
intel_vvp_dpc_get_num_color_planes_out	Returns the number of color planes of the streaming output interface
intel_vvp_dpc_get_pixels_in_parallel	Returns the pixels in parallel of the streaming input and output interfaces
intel_vvp_dpc_get_max_width	Returns the maximum number of pixels that the IP supports on the horizontal dimension of an image or video frame
intel_vvp_dpc_get_max_height	Returns the maximum number of pixels that the IP supports on the vertical dimension of an image or video frame
intel_vvp_dpc_is_running	Reads if the IP is running
intel_vvp_dpc_stats_are_frozen	Reads if the IP finished collecting the statistics
intel_vvp_dpc_get_status	Reads the status register
intel_vvp_dpc_get_frame_stats	Reads the frame statistics register
intel_vvp_dpc_get_pix_lo_clip_cnt	Reads the low pixel clip counter register
intel_vvp_dpc_get_pix_hi_clip_cnt	Reads the high pixel clip counter register
intel_vvp_dpc_get_bypass	Reads the bypass bit from the status register
intel_vvp_dpc_set_bypass	Writes the bypass bit of the status register
intel_vvp_dpc_get_cfa_phase	Reads the color filter array phase field from the status register
intel_vvp_dpc_set_cfa_phase	Writes the color filter array phase field of the status register
intel_vvp_dpc_get_sense_level	Reads the sensitivity level field from the status register
intel_vvp_dpc_set_sense_level	Writes the sensitivity level field of the status register
intel_vvp_dpc_get_pix_clip_cnt_rst	Reads the pixel clip counter reset bit from the status register
intel_vvp_dpc_set_pix_clip_cnt_rst	Writes the pixel clip counter reset bit of the status register
intel_vvp_dpc_get_pix_clip_cnt_sel	Reads the pixel clip counter select field from the status register
intel_vvp_dpc_set_pix_clip_cnt_sel	Writes the pixel clip counter select field of the status register
intel_vvp_dpc_get_freeze_stats_request	Reads the freeze statistics request bit from the status register
intel_vvp_dpc_set_freeze_stats_request	Writes the freeze statistics request bit of the status register

intel_vvp_dpc_init

Prototype

```
int
intel_vvp_dpc_init(intel_vvp_dpc_instance* instance,
intel_vvp_core_base base);
```

Description Initialize an IP instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

Arguments instance - pointer to the intel_vvp_dpc_instance software driver instance structure

base - base address of the register map

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7)

kIntelVvpCorePidErr if the product_id does not match with the expected product ID (0x0175)

kIntelVvpDpcRegMapVersionErr if the register map is not supported

intel_vvp_dpc_get_lite_mode

Prototype

```
bool
intel_vvp_dpc_get_lite_mode(intel_vvp_dpc_instance* instance);
```

Description Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_dpc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_dpc_instance software driver instance structure

Returns true if the IP is parameterized in lite mode

intel_vvp_dpc_get_debug_enabled

Prototype

```
bool  
intel_vvp_dpc_get_debug_enabled(intel_vvp_dpc_instance*  
instance);
```

Description Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_dpc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_dpc_instance software driver instance structure

Returns true if the IP is parameterized with debug features enabled

intel_vvp_dpc_get_bits_per_sample_in

Prototype

```
uint8_t  
intel_vvp_dpc_get_bits_per_sample_in(intel_vvp_dpc_instance*  
instance);
```

Description Returns the value of the BPS_IN register. The instance must be a valid intel_vvp_dpc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_dpc_instance software driver instance structure

Returns the input bits per color sample parameter used to generate the IP

intel_vvp_dpc_get_bits_per_sample_out

Prototype

```
uint8_t  
intel_vvp_dpc_get_bits_per_sample_out(intel_vvp_dpc_instance*  
instance);
```

Description Returns the value of the BPS_OUT register. The instance must be a valid intel_vvp_dpc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_dpc_instance software driver instance structure

Returns the output bits per color sample parameter used to generate the IP

intel_vvp_dpc_get_num_color_planes_in

Prototype

```
uint8_t
intel_vvp_dpc_get_num_color_planes_in(intel_vvp_dpc_instance*
instance);
```

- Description** Returns the value of the NUM_COLOR_IN register. The instance must be a valid intel_vvp_dpc_instance fully initialized.
- Arguments** instance - pointer to the intel_vvp_dpc_instance software driver instance structure
- Returns** the input number of color planes parameter used to generate the IP

intel_vvp_dpc_get_num_color_planes_out

Prototype

```
uint8_t
intel_vvp_dpc_get_num_color_planes_out(intel_vvp_dpc_instance*
instance);
```

- Description** Returns the value of the NUM_COLOR_OUT register. The instance must be a valid intel_vvp_dpc_instance fully initialized.
- Arguments** instance - pointer to the intel_vvp_dpc_instance software driver instance structure
- Returns** the output number of color planes parameter used to generate the IP

intel_vvp_dpc_get_pixels_in_parallel

Prototype

```
uint8_t
intel_vvp_dpc_get_pixels_in_parallel(intel_vvp_dpc_instance*
instance);
```

- Description** Returns the value of the PIXELS_IN_PARALLEL register. The instance must be a valid intel_vvp_dpc_instance fully initialized.
- Arguments** instance - pointer to the intel_vvp_dpc_instance software driver instance structure
- Returns** the pixels in parallel parameter used to generate the IP

intel_vvp_dpc_get_max_width

Prototype

```
uint32_t  
intel_vvp_dpc_get_max_width(intel_vvp_dpc_instance* instance);
```

Description Returns the value of the MAX_WIDTH register. The instance must be a valid intel_vvp_dpc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_dpc_instance software driver instance structure

Returns the maximum field width parameter used to generate the IP

intel_vvp_dpc_get_max_height

Prototype

```
uint32_t  
intel_vvp_dpc_get_max_height(intel_vvp_dpc_instance* instance);
```

Description Returns the value of the MAX_HEIGHT register. The instance must be a valid intel_vvp_dpc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_dpc_instance software driver instance structure

Returns the maximum field height parameter used to generate the IP

intel_vvp_dpc_is_running

Prototype

```
bool  
intel_vvp_dpc_is_running(intel_vvp_dpc_instance* instance);
```

Description Returns the value of the IP Running bit from the STATUS register. The instance must be a valid intel_vvp_dpc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_dpc_instance software driver instance structure

Returns the running bit from the status register

intel_vvp_dpc_stats_are_frozen

Prototype

```
bool
intel_vvp_dpc_stats_are_frozen(intel_vvp_dpc_instance*
instance);
```

Description Returns the value of the statistics are frozen bit from the STATUS register. The instance must be a valid intel_vvp_dpc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_dpc_instance software driver instance structure

Returns the statistics are frozen bit from the status register

intel_vvp_dpc_get_status

Prototype

```
uint32_t
intel_vvp_dpc_get_status(intel_vvp_dpc_instance* instance);
```

Description Returns the value of the STATUS register. The instance must be a valid intel_vvp_dpc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_dpc_instance software driver instance structure

Returns the value of the status register

intel_vvp_dpc_get_frame_stats

Prototype

```
int
intel_vvp_dpc_get_frame_stats(intel_vvp_dpc_instance*
instance, uint32_t* stats_out);
```

Description Reads the value of the FRAME_STATS register. The instance must be a valid intel_vvp_dpc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_dpc_instance software driver instance structure

stats_out - pointer of a variable to return the statistics register value

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpDpcFreezePendingErr if the last freeze request is pending
kIntelVvpDpcPointerErr if stats_out is a null pointer

intel_vvp_dpc_get_pix_lo_clip_cnt

Prototype

```
int  
intel_vvp_dpc_get_pix_lo_clip_cnt(intel_vvp_dpc_instance*  
instance, uint32_t* stats_out);
```

Description Reads the value of the PIX_LO_COUNTER register. The instance must be a valid intel_vvp_dpc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_dpc_instance software driver instance structure

stats_out - pointer of a variable to return the statistics register value

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpDpcFreezePendingErr if the last freeze request is pending

kIntelVvpDpcPointerErr if stats_out is a null pointer

intel_vvp_dpc_get_pix_hi_clip_cnt

Prototype

```
int  
intel_vvp_dpc_get_pix_hi_clip_cnt(intel_vvp_dpc_instance*  
instance, uint32_t* stats_out);
```

Description Reads the value of the PIX_HI_COUNTER register. The instance must be a valid intel_vvp_dpc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_dpc_instance software driver instance structure

stats_out - pointer of a variable to return the statistics register value

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpDpcFreezePendingErr if the last freeze request is pending

kIntelVvpDpcPointerErr if stats_out is a null pointer

intel_vvp_dpc_get_bypass

Prototype

```
bool
intel_vvp_dpc_get_bypass(intel_vvp_dpc_instance* instance);
```

Description Returns the value of the bypass bit from the CONTROL register. The instance must be a valid intel_vvp_dpc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_dpc_instance software driver instance structure

Returns the value of the bypass bit form the control register

intel_vvp_dpc_set_bypass

Prototype

```
int
intel_vvp_dpc_set_bypass(intel_vvp_dpc_instance* instance,
bool bypass);
```

Description Writes to the bypass bit of the CONTROL register. The instance must be a valid intel_vvp_dpc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_dpc_instance software driver instance structure

bypass - new value of the bypass bit

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

intel_vvp_dpc_get_cfa_phase

Prototype

```
uint8_t
intel_vvp_dpc_get_cfa_phase(intel_vvp_dpc_instance* instance);
```

Description Returns the value of the color filter array phase field from the CONTROL register. The instance must be a valid intel_vvp_dpc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_dpc_instance software driver instance structure

Returns the value of the color filter array phase field from the control register

`intel_vvp_dpc_set_cfa_phase`

Prototype

```
int
intel_vvp_dpc_set_cfa_phase(intel_vvp_dpc_instance* instance,
uint8_t cfa_phase);
```

Description Writes to the color filter array phase field of the CONTROL register. The instance must be a valid `intel_vvp_dpc_instance` fully initialized.

Arguments instance - pointer to the `intel_vvp_dpc_instance` software driver instance structure

cfa_phase - new value of the color filter array phase field

Returns `kIntelVvpCoreOk` (0) in case of success, a negative error code in case of an error

`kIntelVvpCoreInstanceErr` if the instance is a null pointer

`kIntelVvpDpcValueErr` if the value of `cfa_phase` is out of range

`intel_vvp_dpc_get_sense_level`

Prototype

```
uint8_t
intel_vvp_dpc_get_sense_level(intel_vvp_dpc_instance*
instance);
```

Description Returns the value of the sensitivity level field from the CONTROL register. The instance must be a valid `intel_vvp_dpc_instance` fully initialized.

Arguments instance - pointer to the `intel_vvp_dpc_instance` software driver instance structure

Returns the value of the sensitivity level field from the control register

`intel_vvp_dpc_set_sense_level`

Prototype

```
int
intel_vvp_dpc_set_sense_level(intel_vvp_dpc_instance*
instance, uint8_t sense_level);
```

- Description** Writes to the sensitivity level field of the CONTROL register. The instance must be a valid intel_vvp_dpc_instance fully initialized.
- Arguments** instance - pointer to the intel_vvp_dpc_instance software driver instance structure
- sense_level - new value of the sensitivity level field
- Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
- kIntelVvpCoreInstanceErr if the instance is a null pointer
- kIntelVvpDpcValueErr if the value of sense_level is out of range

intel_vvp_dpc_get_pix_clip_cnt_rst

Prototype

```
bool
intel_vvp_dpc_get_pix_clip_cnt_rst(intel_vvp_dpc_instance*
instance);
```

- Description** Returns the value of the pixel clip counter reset bit from the CONTROL register. The instance must be a valid intel_vvp_dpc_instance fully initialized.
- Arguments** instance - pointer to the intel_vvp_dpc_instance software driver instance structure
- Returns** the value of the pixel clip counter reset bit form the control register

intel_vvp_dpc_set_pix_clip_cnt_rst

Prototype

```
int
intel_vvp_dpc_set_pix_clip_cnt_rst(intel_vvp_dpc_instance*
instance, bool reset);
```

- Description** Writes to the pixel clip counter reset bit of the CONTROL register. The instance must be a valid intel_vvp_dpc_instance fully initialized.
- Arguments** instance - pointer to the intel_vvp_dpc_instance software driver instance structure
- reset - new value of the pixel clip counter reset bit
- Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
- kIntelVvpCoreInstanceErr if the instance is a null pointer

intel_vvp_dpc_get_pix_clip_cnt_sel

Prototype

```
uint8_t  
intel_vvp_dpc_get_pix_clip_cnt_sel(intel_vvp_dpc_instance*  
instance);
```

Description Returns the value of the pixel clip counter select field from the CONTROL register. The instance must be a valid intel_vvp_dpc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_dpc_instance software driver instance structure

Returns the value of the pixel clip counter select field from the control register

intel_vvp_dpc_set_pix_clip_cnt_sel

Prototype

```
int  
intel_vvp_dpc_set_pix_clip_cnt_sel(intel_vvp_dpc_instance*  
instance, uint8_t pix_clip_cnt_sel);
```

Description Writes to the pixel clip counter select field of the CONTROL register. The instance must be a valid intel_vvp_dpc_instance fully initialized.

Arguments instance - pointer to the intel_vvp_dpc_instance software driver instance structure

pix_clip_cnt_sel - new value of the pixel clip counter select field

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpDpcValueErr if the value of pix_clip_cnt_sel is out of range

intel_vvp_dpc_get_freeze_stats_request

Prototype

```
bool  
intel_vvp_dpc_get_freeze_stats_request(intel_vvp_dpc_instance*  
instance);
```

- Description* Returns the value of the freeze statistics request bit from the CONTROL register. The instance must be a valid intel_vvp_dpc_instance fully initialized.
- Arguments* instance - pointer to the intel_vvp_dpc_instance software driver instance structure
- Returns* the value of the freeze statistics request bit form the control register

intel_vvp_dpc_set_freeze_stats_request

Prototype

```
int
intel_vvp_dpc_set_freeze_stats_request(intel_vvp_dpc_instance*
instance, bool freeze_stats);
```

- Description* Writes to the freeze statistics request bit of the CONTROL register. The instance must be a valid intel_vvp_dpc_instance fully initialized.
- Arguments* instance - pointer to the intel_vvp_dpc_instance software driver instance structure
- freeze_stats - new value of the freeze statistics request bit
- Returns* kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
- kIntelVvpCoreInstanceErr if the instance is a null pointer

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38



25. Deinterlacer IP

25.1. About the Deinterlacer IP

The IP takes a stream of interlaced video fields and outputs progressive frames generated using a bob, weave, or motion adaptive algorithm. The IP passes progressive video frames through unchanged.

The bob algorithm produces output frames by filling in the missing lines from the current field with the linear interpolation of the lines above and below. The deinterlacing algorithm can be setup for:

- Deinterlacing F0 fields while dropping F1 fields
- Deinterlacing F1 fields while dropping F0 fields
- Deinterlacing both F0 and F1 fields

For the bob deinterlacer, with a regular input sequence of alternating field types, the frame rate on the output matches the input field rate when deinterlacing both F0 and F1 fields. Otherwise the IP halves the field rate.

The weave algorithm creates an output frame by filling all the missing lines in the current field with lines from the previous field. With a regular input sequence of alternating field types, the IP halves the field rate. The weave deinterlacer includes an optional FIFO buffer with fixed depth 1024.

The motion adaptive algorithm avoids the weaknesses of bob and weave algorithms by using bob deinterlacing for moving areas of the image and weave deinterlacing for still areas. The frame rate on the output matches the input field rate when deinterlacing both F0 and F1 fields.

The IP is available as full or lite variants. For more information on full and lite, refer to the *Altera Streaming Video Protocol Specification*. For full variants, the deinterlacer IP decodes input resolutions and field type by reading image information packets. Lite variants use the register interface to determine input resolutions and whether the video is progressive or interlaced. Lite variants then decode the `axi4s_vid_in_tuser[1]` signal to determine the incoming field's interlaced type. If you require high quality motion adaptive deinterlacing, use protocol converters and the video and image processing suite deinterlacer II IP.

Related Information

[Altera Streaming Video Protocol Specification](#)

25.1.1. Deinterlacer Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family and the number of supported bits per color sample, pixels in parallel, and number of color planes.

Table 428. Bob Deinterlacer Performance and Resources

The table shows ALM usage and f_{MAX} for a design with 2 pixels in parallel, 10 bits per color sample, and 3 color planes.

Target Device	ALMs	f_{MAX} MHz
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	562	764
Arria 10 speed grade 1 (10AS066H1F34E1HG)	466	593
Cyclone 10 GX speed grade 5 (10CX220YF672E5G)	470	530
Stratix 10 speed grade 1 (1SX280LN2F43E1VG)	607	623

Table 429. Weave Deinterlacer Performance and Resources

The table shows ALM usage and f_{MAX} for a design with 2 pixels in parallel, 10 bits per color sample, and 3 color planes, **Use FIFO** off and separate control clock.

Target Device	ALMs	f_{MAX} MHz
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	2,316	757
Arria 10 speed grade 1 (10AS066H1F34E1HG)	1,500	542
Cyclone 10 GX speed grade 5 (10CX220YF672E5G)	1,498	535
Stratix 10 speed grade 1 (1SX280LN2F43E1VG)	1,809	671

Table 430. Motion Adaptive Deinterlacer Performance and Resources

The table shows ALM usage and f_{MAX} for a design with 2 pixels in parallel, 10 bits per color sample, and 3 color planes, and separate control clock.

Target Device	ALMs	f_{MAX} MHz
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	4,471	697
Arria 10 speed grade 1 (10AS066H1F34E1HG)	4,097	437
Cyclone 10 GX speed grade 5 (10CX220YF672E5G)	4,089	360
Stratix 10 speed grade 1 (1SX280LN2F43E1VG)	4,725	553

25.2. Deinterlacer Parameters

The IP offers run- and compile-time parameters.

Table 431. Deinterlacer Parameters

Parameter	Values	Description
Parameters		
Deinterlacing mode	Bob Weave Motion Adaptive	Select bob, weave, or motion adaptive deinterlacer.
Video Data Format		
Bits per color sample	8 to 16	Select the number of bits per color sample.
Number of color planes	1 to 4	Select the number of color planes per pixel.
<i>continued...</i>		

Parameter	Values	Description
Number of pixels in parallel	1 to 8	Select the number of pixels in parallel.
Memory ⁽⁷⁷⁾		
Avalon memory-mapped host(s) local ports width	16, 32, 64, 128, 256, 512, 1024	Select in bits the width of the Avalon memory-mapped host read and write ports.
Avalon memory-mapped host(s) local ports address width	8 to 32	Select in bits the width of the Avalon memory-mapped host read and write ports. It must be sufficient to fully address the last buffer.
The depth of the write FIFO	32, 64, 128, 256, 512, 1024, 2048	Select the depth of the write FIFO buffer. Each FIFO buffer entry holds one word the width of the specified Avalon memory-mapped local port width. You must specify a FIFO depth of at least twice the specified burst target so that at least 2 bursts can be held at any one time. Increase the FIFO depth to improve resilience to latency on the Avalon memory-mapped bus.
Avalon memory-mapped write burst target	2,4,8,16,32,64	Select the burst target for writes. Longer bursts provide more efficiency on the bus but require more local storage in the write FIFO buffer.
The depth of the read FIFO	32,64,128,256,512,1024,2048	Specify the depth of the read FIFO buffer. Each FIFO buffer entry holds one word the width of the specified Avalon memory-mapped local port width. You must specify a FIFO depth of at least twice the specified burst target so that at least 2 bursts can be held at any one time. Increase the FIFO depth to improve resilience to latency on the Avalon memory-mapped bus.
Avalon memory-mapped read burst target	2,4,8,16,32,64	Select the burst target for reads. Longer bursts provide more efficiency on the bus but require more local storage in the read FIFO buffer.
Field memory base address	0x0000_0000 – 0x7FFF_FFFF	Select the base address for the field buffer to store field 0 or field 1.
Interline stride	Depends on video frame format and maximum frame width.	Set a stride (in bytes) large enough to separate lines of the specified maximum width. The GUI gives an error if the stride is too low.
Packing method	Perfect, color or pixel	Perfect packing minimizes memory footprint of stored fields but increases complexity and therefore size of the field buffer slightly. Color packing leaves spaces in memory between colors if colors do not pack into memory words exactly. Pixel packing leaves spaces in memory between pixels if pixels do not pack into memory words exactly.
Separate clock for the Avalon memory-mapped host interface(s)	On or off	A separate clock allows the control and data portions of the field buffer to run at their maximum clock frequencies. Turn on separate clocks for highest performance.
Control		
Lite mode	On or off	Turn on to operate the deinterlacer in lite mode.
continued...		

⁽⁷⁷⁾ Only for weave and motion adaptive deinterlacer

Parameter	Values	Description
Memory-mapped control interface	On or off	Turn on to read frame statistics and turn the deinterlacer on and off using an Avalon memory-mapped interface. The memory-mapped control interface is mandatory in lite mode.
Separate clock for control interface	On or off	Turn on for a separate clock for the control interface.
Deinterlacer Behavior ⁽⁷⁸⁾		
Bob deinterlacing mode	DEINTERLACE_F0_ONLY, DEINTERLACE_F1_ONLY, DEINTERLACE_F0_AND_F1	Select to drop off F0 or F1 input fields.
Maximum Frame Size		
Maximum field width	32 to 16384	Select the maximum field width to determine the size of the line buffer. Set this parameter to the line length of the widest fields you want to deinterlace. Progressive frames of any size pass through unchanged.
General		
Debug features	On or off	Turn on to enable debug features (not applicable for lite mode).
Pipeline ready signals	On or off	Turn on to add extra pipeline registers to the AXI4-S <code>trready</code> signals
Use FIFO ⁽⁷⁹⁾	On or off	Add optional FIFO buffer.

⁽⁷⁸⁾ Bob deinterlacer only

⁽⁷⁹⁾ Only for weave deinterlacer

Figure 73. Deinterlacer GUI

Deinterlacer Intel FPGA IP
intel_vvp_dil

Parameters
Deinterlacing Type: Weave

Video Data Format
Bits per color sample: 8
Number of color planes: 1
Number of pixels in parallel: 2

Memory
Avalon memory-mapped local ports width: 16
Avalon memory-mapped host(s) local ports address width: 32
The depth of the write FIFO.: 32
Avalon memory-mapped write burst target: 2
The depth of the read FIFO.: 1024
Avalon memory-mapped read burst target: 8
Frame buffer memory base address: 0x00000100
Inter-line stride: 0x00008000
Packing method: PERFECT
☒ Separate clock for the Avalon memory-mapped host interface(s)

Control
☐ Lite mode
☐ Memory mapped control interface
☐ Separate clock for control interface

Deinterlacer Behaviour
Bob deinterlacing Type: Bob F0 fields

Maximum Frame Size
Maximum interlaced field width: 2048

General
☐ Debug features
☐ Pipeline ready signals

25.3. Deinterlacer IP Functional Description

The IP offers bob, weave, or motion adaptive deinterleavers.

Figure 74. Bob Deinterlacing

The figure shows the bob deinterlacing where the IP drops or deinterlaces interlaced fields. The IP passes all progressive frames through.

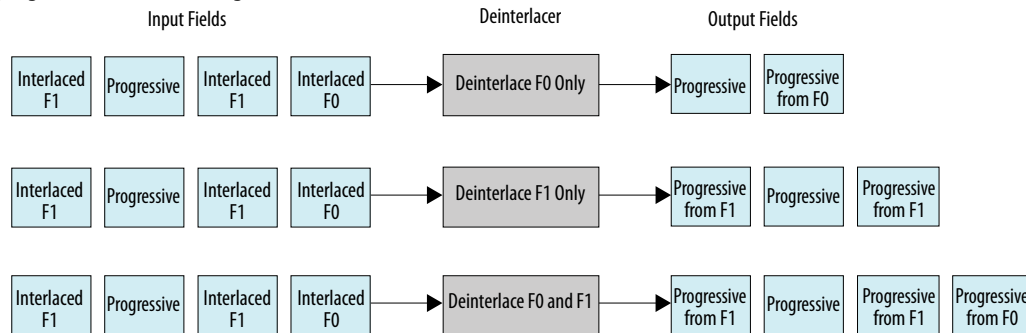
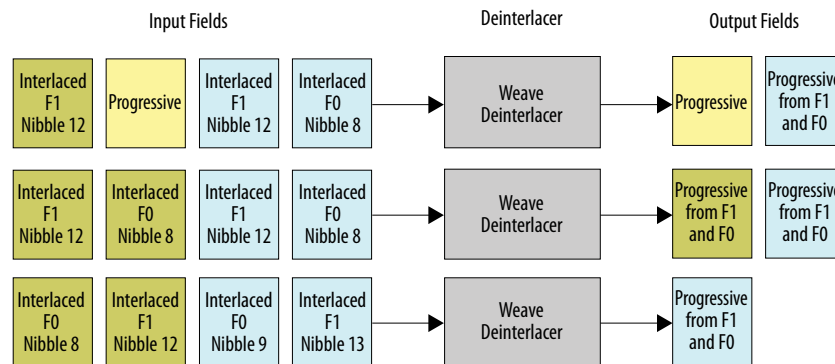


Figure 75. Weave Interlacing

The figure shows the weave deinterlacing, where the IP drops or deinterlaces interlaced fields. The IP passes all progressive frames through. For incoming F1 and F0, the weave deinterlacer deinterlaces or drops the fields based on the nibble values. For more information on nibble, refer to the *Altera Streaming Video Protocol Specification*.



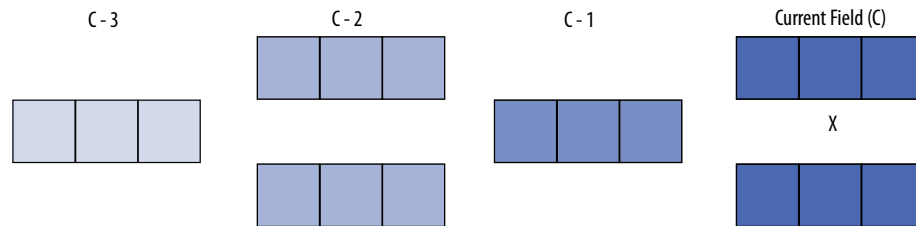
For motion adaptive deinterlacer, the IP calculates the motion coefficient for each pixel and calculates the output pixel depending on motion coefficient. If the motion calculated from the current and the previous pixels is higher than the stored motion value, the stored motion value is irrelevant. The IP uses the calculated motion in the blending algorithm, which then becomes the next stored motion value. However, if the computed motion value is lower than the stored motion value, the following actions occur:

- The IP determines the next stored motion value by calculating the sum of three-fourth of the computed motion and one-fourths of the previously stored motion.
- The blending algorithm uses the next stored motion value.

This computed motion means that the motion that the blending algorithm uses climbs up immediately, but takes a few frames to stabilize. The motion-adaptive algorithm fills in the rows that are missing in the current field by calculating a function of other pixels in the current field and the three preceding fields as in the following sequence:

1. The IP collects pixels from the current field and the three preceding it (the X denotes the location of the desired output pixel).

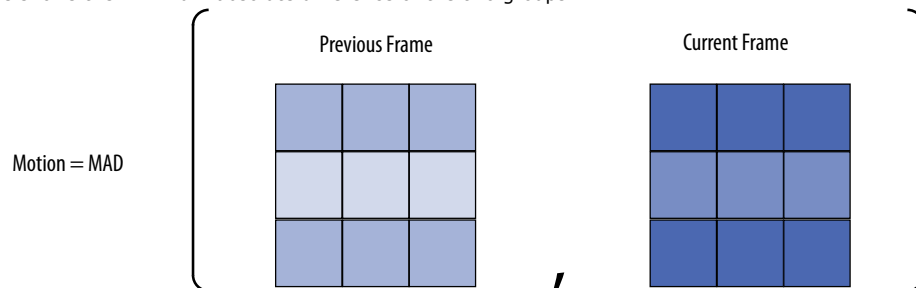
Figure 76. Pixel Collection for the Motion-Adaptive Algorithm



- The IP assembles these pixels into two 3×3 groups of pixels.

Figure 77. Pixel Assembly for the Motion-Adaptive Algorithm

The figure shows the minimum absolute difference of the two groups.



- The IP normalizes the minimum absolute difference value into the same range as the input pixel data. The IP compares the motion value with a recorded motion value for the same location in the previous frame. If it is greater, the IP keeps the new value. If the new value is less than the stored value, the IP uses the motion value that is the mean of the two values. This action reduces unpleasant flickering artifacts.
- The IP uses a weighted mean of the interpolation pixels to calculate the output pixel and the equivalent to the output pixel in the previous field with the following equation:

$$\text{Output Pixel} = M \times \frac{\text{Upper Pixel} + \text{Lower Pixel}}{2} + (1 - M) \times \text{Still Pixel}$$

25.3.1. Deinterlacer Interfaces

Name	Direction	Width	Description
Clocks and resets			
main_clock_clk	In	1	AXI4-S processing clock.
main_reset_rst	In	1	AXI4-S processing reset.
control_clock_clk	In	1	Optional control agent interface clock.
control_reset_reset	In	1	Optional control agent interface reset.
Control interfaces			
av_mm_control_agent_address	In	7	Avalon memory-mapped agent address.
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write.
<i>continued...</i>			

Name	Direction	Width	Description
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data.
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable.
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read.
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data.
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read.
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request.
Altera Streaming Video interfaces			
axi4s_vid_in_tdata	In	(80)	AXI4-S data in.
axi4s_vid_in_tvalid	In	1	AXI4-S data valid.
axi4s_vid_in_tuser[0]	In	1	AXI4-S start of video frame.
axi4s_vid_in_tuser[1]	In	1	AXI4-S control or data packet.
axi4s_vid_in_tuser[N-1:2]	In	(81)	Unused.
axi4s_vid_in_tlast	In	1	AXI4-S end of packet.
axi4s_vid_in_tready	Out	1	AXI4-S data ready.
axi4s_vid_out_tdata	Out	(80)	AXI4-S data in.
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid.
axi4s_vid_out_tuser[0]	Out	1	AXI4-S start of video frame.
axi4s_vid_out_tuser[1]	Out	1	AXI4-S control or data packet.
axi4s_vid_out_tuser[N-1:2]	Out	(81)	Unused.
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet.
axi4s_vid_out_tready	In	1	AXI4-S data ready.
Avalon memory-mapped host interfaces (only for weave and motion adaptive deinterlacer) Widths for address, data, and burst count buses are as you specify in the GUI. The table shows example widths.			
mem_clock_clk	In	1	Optional host interface clock.
mem_reset_reset	In	1	Optional host interface reset.
av_mm_mem_write_host_address	Out	32	Avalon memory-mapped host address
av_mm_mem_write_host_write	Out	1	Avalon memory-mapped host write.
av_mm_mem_write_host_burstcount	Out	5	Avalon memory-mapped host write burst count.
av_mm_mem_write_host_writedata	Out	64	Avalon memory-mapped host write data.
av_mm_mem_write_host_waitrequest	In	1	Avalon memory-mapped host wait request.
continued...			

(80) The equation gives all tdata widths sizes in these interfaces:
 $\max(2, \text{ceil}((\text{bits per color sample} \times \text{number of color planes})/8)) \times \text{pixels in parallel} \times 8$

(81) This equation gives all tuser widths sizes in these interfaces: $N = \text{ceil}(\text{tdata width} / 8)$

Name	Direction	Width	Description
av_mm_mem_read_host_address	Out	32	Avalon memory-mapped host address
av_mm_mem_read_host_read	Out	1	Avalon memory-mapped host read.
av_mm_mem_read_host_burstcount	Out	5	Avalon memory-mapped host read burst count.
av_mm_mem_read_host_readdata	In	64	Avalon memory-mapped host read data.
av_mm_mem_read_host_readdatavalid	In	1	Avalon memory-mapped host read data valid.
av_mm_mem_read_host_waitrequest	In	1	Avalon memory-mapped host wait request.

Related Information

Video and Vision Processing IP Interfaces on page 30

25.4. Deinterlacer IP Registers

Each register is either read-only (RO) or read-write (RW).

In the software API the register names appear with a prefix of `INTEL_VVP`, `INTEL_VVP_CORE`, or `INTEL_VVP_DEINTERLACER` as appropriate and with an optional `REG` suffix.

Table 432. Control Registers

Address	Register	Access		Description
		Lite ⁽⁸²⁾	Full	
Parameterization registers				
0x0000	PROD_ID	RO	RO	Read this register for the Deinterlacer product ID. This register always returns 0x6AF7_023A.
0x0004	VER	RO	RO	Read this register for the IP version information.
0x0008	LITE_MODE	RO	RO	Read this register to determine if Lite mode is on or off. This register returns 0 when Lite mode is off and 1 when on.
0x000C	DEBUG_ENABLED	RO	RO	Read this register to determine if Debug features is on.
0x0010	MAX_WIDTH	RO	RO	Read this register to determine the maximum supported frame width (bob deinterlacer only).
0x0014	DIL_MODE	RO	RO	Read this register to determine the deinterlacing algorithm.
0x0018	BOB_DIL_MODE	RO	RO	Read this register to determine the Bob deinterlacing mode .
0x001C	MEM_BASE_ADDR	RO	RO	Read this register for the base address of stored field in memory. ⁽⁸³⁾
0x0020	MEM_LINE_STRIDE	RO	RO	Read this register for the line stride in memory. ⁽⁸³⁾
continued...				

⁽⁸²⁾ When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

⁽⁸³⁾ The content of this register is undefined if you select **Bob** for **Deinterlacing mode**.

Address	Register	Access		Description
		Lite ⁽⁸²⁾	Full	
0x0024	BPS	RO	RO	Read this register for the number of bits per symbol configured. ⁽⁸³⁾
0x0028	NUMBER_OF_COLOR_PLANES	RO	RO	Read this register for the number of color planes. ⁽⁸³⁾
0x002C	PIXELS_IN_PARALLEL	RO	RO	Read this register for the number of pixels in parallel. ⁽⁸³⁾
0x0030	PACKING	RO	RO	Read this register for the pixel packing scheme. ⁽⁸³⁾
0x0034 to 0x011F	-	-	-	Unused.
Control and debug registers For more information, refer to <i>Control Packets</i>				
0x0120	IMG_INFO_WIDTH	RW	RO	For lite designs, the expected width of the incoming video fields. For full, the received width in the IP derives from the image information packets.
0x0124	IMG_INFO_HEIGHT	RW	RO	For lite designs, the expected height of the incoming video fields. For full the received height in the IP derives from the image information packets.
0x0128	IMG_INFO_INTERLACE	RW	RO	For lite designs, the expected interlace information of the incoming video fields. For full, the received interlace information in image information packets.
0x012C	RESERVED	RW	RO	Unused.
0x0130	IMG_INFO_COLORSPACE	RW	RO	For lite designs, the expected color space of the incoming video fields. For full, the received color space in image information packets.
0x0134	IMG_INFO_SUBSAMPLING	RW	RO	For lite designs, the expected chroma subsampling of the incoming video fields. For full, the received chroma subsampling in image information packets.
0x0138	IMG_INFO_COSITING	RW	RO	For lite designs, the expected chroma co-siting of the incoming video fields. For full, the received chroma co-siting in image information packets.
0x013C	IMG_INFO_FIELD_COUNT	-	RO	The received field count field in image information packets.
0x0140	INPUT_STATUS	RO	RO	Bit 0: input status bit. 1 = Deinterlacer is receiving and processing a video field, 0 otherwise.
0x0144	Commit	-	RW	Only when you turn off lite mode . The IP holds any changes to the deinterlacer settings via the register map until you write to this register. The value you write is unimportant.
0x0148	Adjust	RW	RW	The adjustment parameter.
0x014C	Ctrl	RW	RW	The control register

⁽⁸²⁾ When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

Register Bit Descriptions

Table 433. PROD_ID

Name	Bits	Description
Deinterlacer product ID	31:0	This register always returns 0x6AF7_023A.

Table 434. VERSION

Name	Bits	Description
Register map version	7:0	Register map version.
IP patch revision	15:8	-
IP update revision	23:16	Updated when the IP version changes.
IP major revision	31:24	Updated when the IP version changes.

Table 435. LITE_MODE

Name	Bits	Description
Lite mode parameterization bit	0	Returns 1 if you turn on lite mode .
Unused	31:1	Unused.

Table 436. DEBUG_ENABLED

Name	Bits	Description
Debug features parameterization bit	0	Returns 1 if you turn on Debug features .
Unused	31:1	Unused.

Table 437. DIL_MODE

Name	Bits	Description
Deinterlacer mode	31:0	Returns 0 if the IP is using a bob algorithm. Returns 1 if the IP is using a weave algorithm. Returns 2 if the IP is using a motion adaptive algorithm.

Table 438. MAX_WIDTH

Bob deinterlacer only

Name	Bits	Description
Max width	31:0	This register returns the maximum supported frame width.

Table 439. BOB_DIL_MODE

Bob deinterlacer only

Name	Bits	Description
Bob deinterlacer mode	31:0	Returns the bob deinterlacing mode. Returns 0 if deinterlacing F0 fields only, returns 1 if deinterlacing F1 fields only, otherwise returns 2. The content of this register is undefined if you select Weave or Motion Adaptive for Deinterlacing mode .

Table 440. MEM_BASE_ADDR

Weave and motion adaptive deinterlacer only

Name	Bits	Description
Membase address	31:0	Returns the base address of stored field in memory.

Table 441. MEM_LINE_STRIDE

Weave and motion adaptive deinterlacer only

Name	Bits	Description
Memline stride	31:0	Returns the line stride in memory.

Table 442. PACKING

Weave and motion adaptive deinterlacer only

Name	Bits	Description
Packing	31:0	Returns the packing scheme: 0= Perfect packing 1= Color packing 2= Pixel packing

Table 443. IMG_INFO_WIDTH

Name	Bits	Description
Width bits	15:0	When you turn on lite mode , write to this register to set the expected width of the incoming video fields. When you turn off lite mode and turn on Debug features , this register returns the <code>width-1</code> field from the most recently received image information packet and adds 1 to return a value for <code>width</code> .
unused	31:16	Unused.

Table 444. IMG_INFO_HEIGHT

Name	Bits	Description
Height bits	15:0	When you turn on lite mode , write to this register to set the expected height of the incoming video fields. When you turn off lite mode and turn on Debug features , this register returns the <code>height-1</code> field from the most recently received image information packet and adds 1 to return a value for <code>height</code> .
unused	31:16	Unused.

Table 445. IMG_INFO_INTERLACE

Name	Bits	Description
InterlaceNibble bits	3:0	When you turn on lite mode , write to this register to set the expected interlacing of the incoming video fields: - Set bit 3 of this register high to indicate interlaced video - Set bit 3 low to indicate progressive video. - The IP does not use bits [2:0]. When you turn off lite mode and turn on Debug features , this register returns the <code>intlaceNibble</code> field from the most recently received image information packet.
unused	31:4	Unused.

Table 446. IMG_INFO_COLORSPACE

Name	Bits	Description
CSPcode bits	6:0	When you turn on lite mode , write to this register to set the expected color space of the incoming video fields. When you turn off lite mode and turn on Debug features , this register returns the 7 bit <code>CSP</code> field from the most recently received image information packet.
unused	31:7	Unused.

Table 447. IMG_INFO_SUBSAMPLING

Name	Bits	Description
CSPSubSa code bits	1:0	When you turn on lite mode , write to this register to set the expected chroma subsampling of the incoming video fields. When you turn off lite mode and turn on Debug features , this register returns the <code>SUBSA</code> field from the most recently received image information packet.
unused	31:2	Unused.

Table 448. IMG_INFO_COSITING

Name	Bits	Description
Cosite code bits	1:0	When you turn on lite mode , write to this register to set the expected chroma co-siting of the incoming video fields. When you turn off lite mode and turn on Debug features , this register returns the <code>COSITE</code> field from the most recently received image information packet.
unused	31:2	Unused.

Table 449. IMG_INFO_FIELD_COUNT

Name	Bits	Description
Count bits	6:0	When you turn on lite mode , this register has no function. When you turn off lite mode and turn on Debug features , this register returns the 7 bit <code>FIELD_COUNT</code> field from the most recently received image information packet.
unused	31:7	Unused.

Table 450. INPUT_STATUS

Name	Bits	Description
Status bit	0	1= Deinterlacer is receiving and processing a video field, 0 otherwise.

Table 451. COMMIT

Motion adaptive deinterlacer only

Name	Bits	Description
Commit	31:0	Write to any bits to trigger a commit of new motion adaptive deinterlacer settings.

Table 452. ADJUST

Motion adaptive deinterlacer only

Name	Bits	Description
Adjust Value	15:0	Applies the adjustment parameter multiplied by the calculated motion coefficient to either amplify or reduce the motion value. The adjustment parameter is represented by a 16-bit value. However, when the calculation completes, the resultant value shifts right by 12 bits. To preserve the initially calculated motion coefficient unchanged, set the adjustment parameter to 0x1000. The default setting for this parameter is 0x2000. To achieve optimal performance, tune this parameter according to the specific requirements of your application. Begin with an initial value of 0x6000.
Unused	31:16	Unused.

Table 453. CTRL

Motion adaptive deinterlacer only

Name	Bits	Description
Control Value	1:0	The control register has two bits to configure the operational mode of the motion adaptive deinterlacer. Setting bit 0 to 1 activates the deinterlacer to perform either bob or weave calculations. Within this context, if bit 0 is set to 1, the state of bit 1 determines the specific operation. A value of 0 for bit 1 initiates a weave operation. A value of 1 for bit 1 triggers a bob operation. If bit 0 is not set to 1, the value of bit 1 is disregarded.
Unused	31:2	Unused.

25.5. Deinterlacer IP Software API

The IP has a software driver for software control of the IP at run time. The IP does not fit any of the generic device models provided by the Nios II HAL. It exposes a set of dedicated accessors to the control and status registers. The driver structure inherits the base driver structure so all common methods described in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_dil_regs.h`

Include file: `intel_vvp_dil.h`

Table 454. Deinterlacer IP Software API

Name	Description
<code>intel_vvp_dil_init</code>	Initialize the Deinterlacer instance
<code>Intel_vvp_core_*</code>	Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i>
<code>intel_vvp_dil_get_lite_mode</code>	Returns lite mode is on
<code>intel_vvp_dil_get_debug_enabled</code>	Returns debug features is on.
<code>Intel_vvp_dil_get_max_width</code>	Returns the maximum supported width
<code>intel_vvp_dil_get_dil_algorithm</code>	Returns the deinterlacing algorithm
<code>intel_vvp_dil_get_field_sync</code>	Returns the field synchronization method
<code>intel_vvp_dil_get_mem_base_address</code>	Returns the base address (weave and motion adaptive deinterlacing)
<code>intel_vvp_dil_get_mem_buff_line_stride</code>	Returns the line stride (weave and motion adaptive deinterlacing)
continued...	

Name	Description
intel_vvp_dil_get_bps	Returns the Bits per sample value (weave and motion adaptive deinterlacing)
intel_vvp_dil_get_num_colors	Returns the Number of color planes value (weave and motion adaptive deinterlacing)
intel_vvp_dil_get_pixels_in_parallel	Returns the Pixels in parallel value (weave and motion adaptive deinterlacing)
intel_vvp_dil_get_mem_packing	Returns the Memory packing method (weave and motion adaptive deinterlacing)
intel_vvp_dil_is_running	Returns if the IP is processing data
intel_vvp_dil_get_status	Reads the status register
intel_vvp_dil_get_mem_buff_line_stride	Returns the line stride (weave and motion adaptive deinterlacing)
intel_vvp_dil_get_bps	Returns the Bits per sample value (weave and motion adaptive deinterlacing)
intel_vvp_dil_get_num_colors	Returns the Number of color planes value (weave and motion adaptive deinterlacing)
intel_vvp_dil_get_pixels_in_parallel	Returns the Pixels in parallel value (weave and motion adaptive deinterlacing)
intel_vvp_dil_get_mem_packing	Returns the Memory packing method (weave and motion adaptive deinterlacing)
intel_vvp_dil_set_motion_adaptive_adjust	Adjusts motion sensitivity (motion adaptive deinterlacing)
intel_vvp_dil_get_motion_adaptive_adjust	Returns current motion sensitivity (motion adaptive deinterlacing)
intel_vvp_dil_set_motion_adaptive_ctrl	Override deinterlacing algorithm at run-time to perform bob/weave deinterlacing (motion adaptive deinterlacing)
intel_vvp_dil_get_motion_adaptive_ctrl	Returns current run-time deinterlacing algorithm (motion adaptive deinterlacing)
intel_vvp_dil_commit_writes	Commit all outstanding writes
intel_vvp_dil_get_mem_base_address	Returns the base address (weave and motion adaptive deinterlacing)

enum eIntelVvpDilAlgorithms

Members

- kIntelVvpDilBobAlgo (0) - Bob deinterlacing
- kIntelVvpDilWeaveAlgo (1) - Weave deinterlacing
- kIntelVvpDilAlgoErr (-1) - Invalid deinterlacing algorithm (used to indicate errors)

Description Enumeration for the configurable deinterlacing algorithms

enum eIntelVvpDilMemPacking

Members

- kIntelVvpDilPerfectPacking (0) - Perfect packing
- kIntelVvpDilColorPacking (1) - Color packing

kIntelVvpDilPixelPacking (2) - Pixel packing

kIntelVvpDilPackingErr (-1) - Invalid value (used to indicate errors)

Description Enumeration for the configurable memory packing algorithm

enum eIntelVvpDilFieldSyncs

Members kIntelVvpDilDeinterlaceSyncF0 - Synchronization on F0 (input frame rate = output frame rate)

kIntelVvpDilDeinterlaceSyncF1 - Synchronization on F1 (input frame rate = output frame rate)

kIntelVvpDilDeinterlaceSyncBoth - Synchronization on F0 and F1 (input field rate = output frame rate)

Description Enumeration for the three supported field-to-frame rate conversions

intel_vvp_dil_init

Prototype

```
int intel_vvp_dil_init(intel_vvp_dil_instance *instance,
intel_vvp_core_base base);
```

Description Initializes a deinterlacer instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

Arguments instance - pointer to the intel_vvp_dil_instance software driver instance structure

base - base address of the register map

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product_id does not match with the Deinterlacer product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpDilRegMapVersionErr if the register map is not supported

intel_vvp_dil_get_lite_mode

Prototype

```
bool intel_vvp_dil_get_lite_mode(intel_vvp_dil_instance  
*instance);
```

Description

Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_dil_instance fully initialized

Arguments

instance – pointer to the intel_vvp_dil_instance software driver instance structure

Returns

true if the IP is parameterized in lite mode

intel_vvp_dil_get_debug_enabled

Prototype

```
bool intel_vvp_dil_get_debug_enabled(intel_vvp_dil_instance  
*instance);
```

Description

Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_dil_instance fully initialized.

Arguments

instance – pointer to the intel_vvp_dil_instance software driver instance structure

Returns

true if the IP is parameterized with debug features enabled

intel_vvp_dil_get_max_width

Prototype

```
uint32_t intel_vvp_dil_get_max_width(intel_vvp_dil_instance  
*instance);
```

Description

Returns the value of the MAX_WIDTH register. The instance must be a valid intel_vvp_dil_instance fully initialized.

Arguments

instance – pointer to the intel_vvp_dil_instance software driver instance structure

Returns

Maximum supported width

intel_vvp_dil_get_dil_algorithm

Prototype

```
eIntelVvpDilAlgorithms_t  
intel_vvp_dil_get_dil_algorithm(intel_vvp_dil_instance  
*instance);
```

Description

Returns the value of the deinterlacing algorithm register. The instance must be a valid intel_vvp_dil_instance fully initialized.

Arguments

instance – pointer to the intel_vvp_dil_instance software driver instance structure

Returns Deinterlacing algorithm selected at compile-time

intel_vvp_dil_get_field_sync

Prototype

```
eIntelVvpDilFieldSyncs
intel_vvp_dil_get_field_sync(intel_vvp_dil_instance *instance);
```

Description Returns the value of the field synchronization register. The instance must be a valid intel_vvp_dil_instance fully initialized.

Arguments instance – pointer to the intel_vvp_dil_instance software driver instance structure

Returns Field synchronization algorithm selected at compile-time

intel_vvp_dil_get_mem_base_address

Prototype

```
uint32_t
intel_vvp_dil_get_mem_base_address(intel_vvp_dil_instance
*instance);
```

Description Returns the value of the MEM_BASE_ADDR register. The instance must be a valid intel_vvp_dil_instance fully initialized and configured for weave deinterlacing.

Arguments instance - pointer to the intel_vvp_dil_instance software driver instance structure

Returns the storage buffer base address in memory

intel_vvp_dil_get_mem_buff_line_stride

Prototype

```
uint32_t
intel_vvp_dil_get_mem_buff_line_stride(intel_vvp_dil_instance
*instance);
```

Description Returns the value of the MEM_BUFF_LINE_STRIDE register. The instance must be a valid intel_vvp_dil_instance fully initialized and configured for weave deinterlacing.

Arguments instance - pointer to the intel_vvp_dil_instance software driver instance structure

Returns the line stride used in the storage buffer

intel_vvp_dil_get_bps

Prototype

```
uint8_t intel_vvp_dil_get_bps(intel_vvp_dil_instance
*instance);
```

Description Returns the value of the BITS_PER_SAMPLE register. The instance must be a valid intel_vvp_dil_instance fully initialized and configured for weave deinterlacing.

Arguments instance - pointer to the intel_vvp_dil_instance software driver instance structure

Returns the number of bits per color sample

intel_vvp_dil_get_num_colors

Prototype

```
uint8_t intel_vvp_dil_get_num_colors(intel_vvp_dil_instance  
*instance);
```

Description Returns the value of the NUM_COLOR_PLANES register. The instance must be a valid intel_vvp_dil_instance fully initialized and configured for weave deinterlacing.

Arguments instance - pointer to the intel_vvp_dil_instance software driver instance structure

Returns the number of color planes per pixel

intel_vvp_dil_get_pixels_in_parallel

Prototype

```
uint8_t  
intel_vvp_dil_get_pixels_in_parallel(intel_vvp_dil_instance  
*instance);
```

Description Returns the value of the PIXELS_IN_PARALLEL register. The instance must be a valid intel_vvp_dil_instance fully initialized and configured for weave deinterlacing.

Arguments instance - pointer to the intel_vvp_dil_instance software driver instance structure

Returns the number of pixels streamed in parallel per beat

intel_vvp_dil_get_mem_packing

Prototype

```
eIntelVvpDilMemPacking  
intel_vvp_dil_get_mem_packing(intel_vvp_dil_instance  
*instance);
```

Description Returns the value of the MEM_PACKING register. The instance must be a valid intel_vvp_dil_instance fully initialized and configured for weave deinterlacing.

Arguments instance - pointer to the intel_vvp_dil_instance software driver instance structure

Returns the memory packing method used to store fields in memory

intel_vvp_dil_is_running

Prototype

```
bool intel_vvp_dil_is_running(intel_vvp_dil_instance* instance);
```

Description Reads and returns the running bit of the STATUS register. The instance must be a valid intel_vvp_dil_instance fully initialized.

Arguments instance – pointer to the intel_vvp_dil_instance software driver instance structure

Returns True is the IP is currently generating an output field, false when between two fields

intel_vvp_dil_get_status

Prototype

```
uint8_t intel_vvp_dil_get_status(intel_vvp_dil_instance* instance);
```

Description Reads and returns the STATUS register. The instance must be a valid intel_vvp_dil_instance fully initialized.

Arguments instance – pointer to the intel_vvp_dil_instance software driver instance structure

Returns The value read from the status register

Register definition header file: intel_vvp_dil_regs.h

Include file: intel_vvp_dil.h

enum eIntelVvpDilAlgorithms

Members

- kIntelVvpDilBobAlgo (0) - Bob deinterlacing
- kIntelVvpDilWeaveAlgo (1) - Weave deinterlacing
- kIntelVvpDilMotionAdaptiveAlgo (1) - Motion adaptive deinterlacing
- kIntelVvpDilAlgoErr (-1) - Invalid deinterlacing algorithm (used to indicate errors)

Description Enumeration for the configurable deinterlacing algorithms

intel_vvp_dil_[existing_function]

Prototype

```
uint32_t intel_vvp_dil_[existing_function](intel_vvp_dil_instance *instance);
```

Description Returns the value of the X register. The instance must be a valid intel_vvp_dil_instance fully initialized and configured for weave or motion_adaptive deinterlacing.

Arguments instance - pointer to the intel_vvp_dil_instance software driver instance structure

Returns -

intel_vvp_dil_set_motion_adaptive_adjust

Prototype

```
int  
intel_vvp_dil_set_motion_adaptive_adjust(intel_vvp_dil_instance*  
instance, uint16_t ma_adjust)
```

Description Changes the motion sensitivity. The instance must be a valid intel_vvp_dil_instance fully initialized and configured for motion_adaptive deinterlacing.

Arguments instance - pointer to the intel_vvp_dil_instance software driver instance structure

ma_adjust - motion sensitivity

Returns kIntelVvpCoreOk (0) for success, negative error code in case of error

intel_vvp_dil_get_motion_adaptive_adjust

Prototype

```
uint16_t  
intel_vvp_dil_get_motion_adaptive_adjust(intel_vvp_dil_instance*  
instance)
```

Description Returns the current run-time motion sensitivity parameter. The instance must be valid and configured for motion_adaptive deinterlacing with debug features enabled.

Arguments instance - pointer to the intel_vvp_dil_instance software driver instance structure

Returns the current motion sensitivity or 0 if the call is invalid

intel_vvp_dil_set_motion_adaptive_control

Prototype

```
int  
intel_vvp_dil_set_motion_adaptive_control(intel_vvp_dil_instance*  
instance, eIntelVvpDilAlgorithms override_algo)
```

Description Override the run-time behavior of the motion adaptive deinterlacer to switch to either full bob or full weave. The instance must be valid and configured for motion_adaptive deinterlacing.

Arguments instance - pointer to the intel_vvp_dil_instance software driver instance structure

override_algo - kIntelVvpDilBobAlgo or kIntelVvpDilWeaveAlgo to override the default behavior, kIntelVvpDilMotionAdaptiveAlgo to restore it

Returns kIntelVvpCoreOk (0) for success, negative error code in case of error

intel_vvp_dil_get_motion_adaptive_control

Prototype

```
eIntelVvpDilAlgorithms
intel_vvp_dil_get_motion_adaptive_control(intel_vvp_dil_instance*
instance)
```

Description Returns the current run-time behavior of the motion adaptive deinterlacer. The instance must be valid and configured for motion_adaptive deinterlacing with debug features enabled.

Arguments instance - pointer to the intel_vvp_dil_instance software driver instance structure

Returns kIntelVvpDilMotionAdaptiveAlgo by default if the default motion adaptive algorithm is enabled, kIntelVvpDilBobAlgo or kIntelVvpDilWeaveAlgo if the default algorithm is overridden, kIntelVvpDilAlgoErr if the call is invalid

intel_vvp_dil_commit_writes

Prototype

```
int intel_vvp_dil_commit_writes(intel_vvp_dil_instance*
instance)
```

Description Commits all pending writes before processing the next field. The instance must be valid and configured for full variant motion_adaptive deinterlacing.

Arguments instance - pointer to the intel_vvp_dil_instance software driver instance structure

Returns kIntelVvpCoreOk (0) for success, kIntelVvpCoreInstanceErr (-1) if the instance is invalid

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

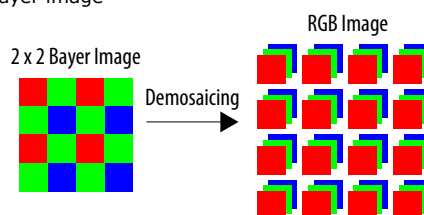
26. Demosaic IP

26.1. About the Demosaic IP

The Demosaic IP converts 2x2 Bayer filter array video stream to an RGB video stream. The IP interpolates missing colors for each pixel, based on its neighboring pixels.

Figure 78. Demosaicing

RGB color reconstruction from Bayer image



The IP only supports the lite variant of the Altera Streaming Video Protocol. For more information about full and lite variants, refer to the *Altera Streaming Video Protocol Specification*

Related Information

[Altera Streaming Video Protocol Specification](#)

26.1.1. Demosaic IP Performance and Resources

Altera provides resource and utilization data for guidance only. The IP resource utilization depends on the device family, compilation tool version, compilation seed randomization, and other parameters.

Table 455. Demosaic IP Performance and Resources

Device	Pixel in Parallel	Bits per Color Sample	Max Resolution	ALM	M20k	f _{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	1	10	2k	853	5	785
	1	16	2k	1,098	9	781
	4	10	8k	2,077	20	824
	4	16	8k	2,911	33	804
Arria 10 speed grade 1 (10AS066H1F34E1HG)	1	10	2k	750	5	616
	1	16	2k	942	9	602
	2	10	4k	1,069	10	646
continued...						

Device	Pixel in Parallel	Bits per Color Sample	Max Resolution	ALM	M20k	f _{MAX} (MHz)
	2	16	4k	1,458	17	589
Cyclone 10 GX speed grade 5 (10CX220YF672E5G)	1	10	2k	746	5	554
	1	16	2k	942	9	557
Stratix 10 speed grade 1 (1SX280LN2F43E1VG)	1	10	2k	894	5	701
	1	16	2k	1,133	9	723
	2	10	4k	1,333	10	701
	2	16	4k	1,804	17	660

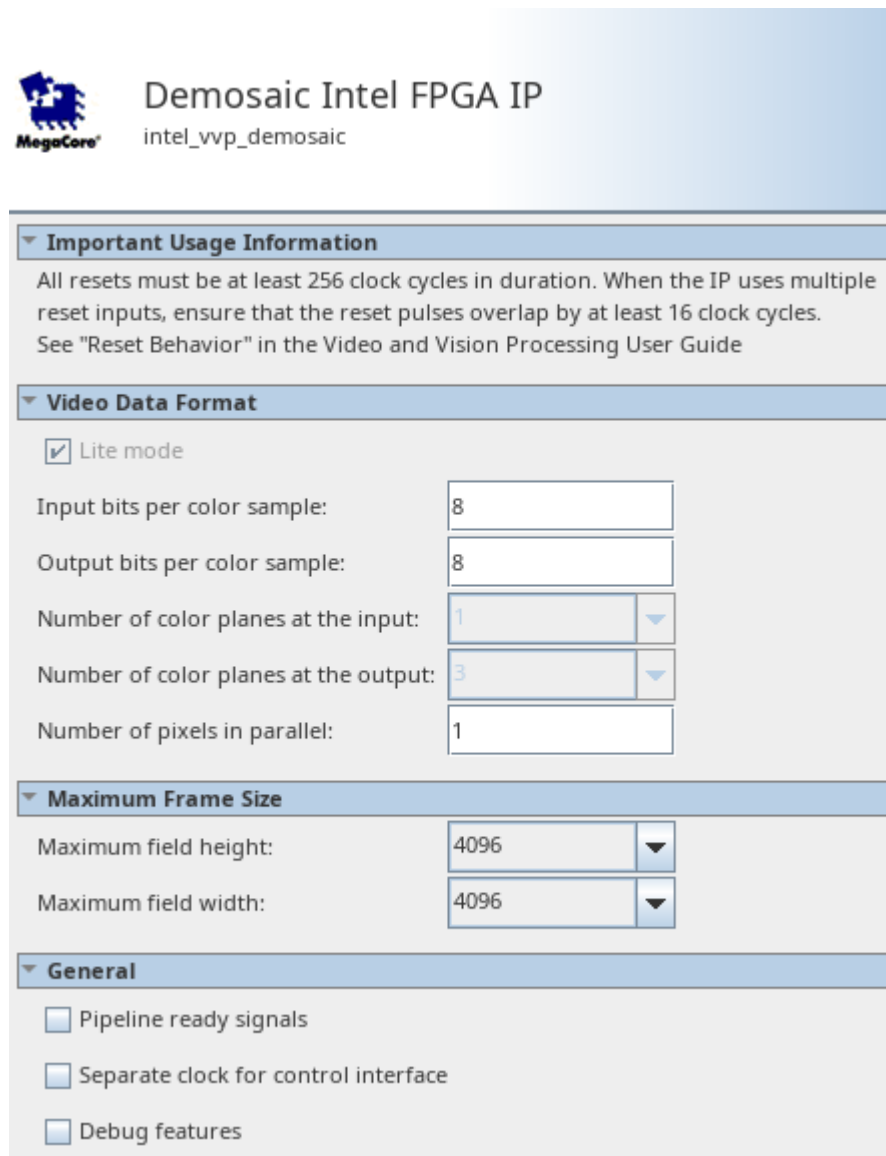
26.2. Demosaic IP Parameters

The IP offers run- and compile-time parameters.

Table 456. Demosaic IP Parameters

Parameter	Values	Description
Video Data Format		
Lite mode	On	The IP only supports lite mode
Input bits per color sample	8 to 16	Select the number of input bits per color sample.
Output bits per color sample	8 to 16	Select the number of output bits per color sample.
Number of color planes at the input	1	The IP only supports 1 color plane at its input
Number of color planes at the output	3	The IP only supports 3 color planes at its output
Number of pixels in parallel	1 to 8	Select the number of pixels in parallel.
Maximum Frame Size		
Maximum field height	1k, 2k, 4k, 8k, 16k	Specify the maximum height of incoming frames
Maximum field width	2k, 4k, 8k, 16k	Specify the maximum width of incoming frames
General		
Pipeline ready signal	On or off	Turn on to add extra pipeline registers to the AXI4-S <code>tready</code> signals
Separate clock for control interface	On or off	Turn on for a separate clock for the control agent interface
Debug features	On or off	Turn on to read back frame information registers and debugging information registers via the control agent interface.

Figure 79. Demosaic IP GUI



Demosaic Intel FPGA IP
intel_vvp_demosaic

Important Usage Information
All resets must be at least 256 clock cycles in duration. When the IP uses multiple reset inputs, ensure that the reset pulses overlap by at least 16 clock cycles. See "Reset Behavior" in the Video and Vision Processing User Guide

Video Data Format

☒ Lite mode

Input bits per color sample: 8

Output bits per color sample: 8

Number of color planes at the input: 1

Number of color planes at the output: 3

Number of pixels in parallel: 1

Maximum Frame Size

Maximum field height: 4096

Maximum field width: 4096

General

☐ Pipeline ready signals

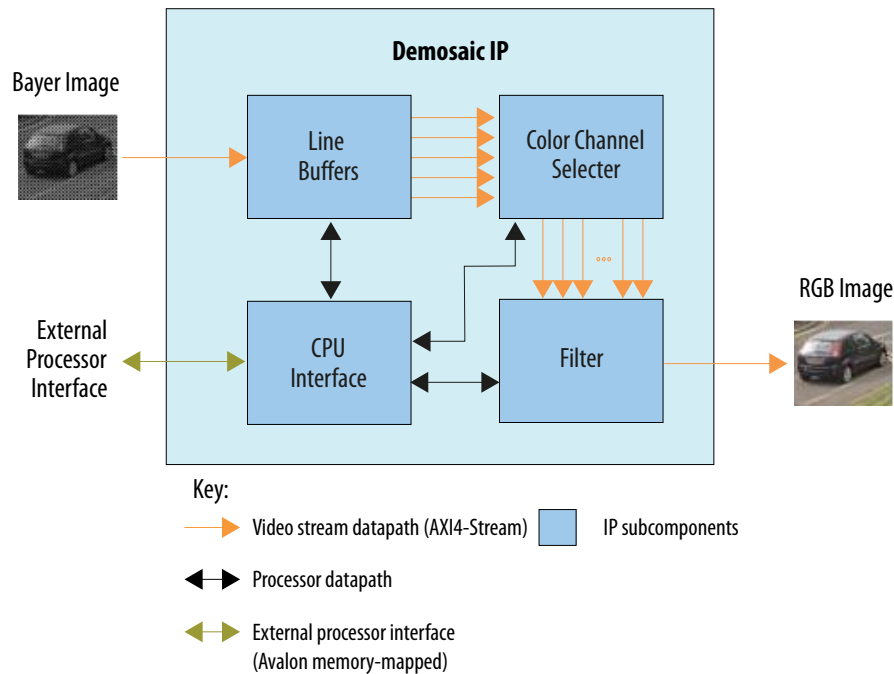
☐ Separate clock for control interface

☐ Debug features

26.3. Demosaic IP Functional Description

For every pixel the Demosaic IP analyzes its neighborhood and interpolates missing colors. The IP converts 2x2 input Bayer color filter array to an RGB output. The IP creates buffers of multiple lines of the input video stream to construct pixel neighborhood information for subsequent interpolation.

Figure 80. Demosaic IP Block Diagram

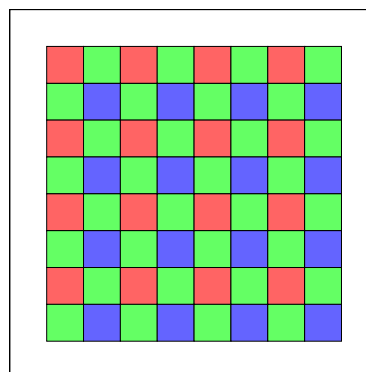


The color channel selector creates a map of the pixels in the neighborhood depending on the position on the 2x2 color filter. It sends them to a filter, which calculates an RGB output by interpolating missing colors.

You must set the color filter array phase to its correct value, which depends on the framing, rotation, and flipping options you apply on the original image that an imaging sensor captures. If you do not set the correct color filter array phase, the IP may mix color channels and produce incorrect outputs.

Figure 81. A Bayer Color Filter Array

For an 8x8 section of an image.



26.3.1. Demosaic IP Interfaces

Table 457. Demosaic Clock and Reset Interfaces

Name	Direction	Width	Description
main_clock_clk	In	1	AXI4-S processing clock.
main_reset_reset	In	1	AXI4-S processing reset.
agent_clock_clk	In	1	Clock for the memory-mapped control agent interface. Only if you select Separate clock for control interface .
agent_reset_reset	In	1	Reset for the memory-mapped control agent interface. Only if you select Separate clock for control interface .

Table 458. Demosaic Avalon memory-mapped host interfaces

Name	Direction	Width	Description
av_mm_control_agent_address	In	7	Avalon memory-mapped agent address
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write.
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data.
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable.
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read.
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data.
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read.
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request.

Table 459. Demosaic Altera Streaming Video Interfaces

Name	Direction	Width	Description
Input			
axi4s_vid_in_tdata	In	(84)	AXI4-S data in.
axi4s_vid_in_tvalid	In	1	AXI4-S data valid.
axi4s_vid_in_tuser	In	(85)	[0] AXI4-S start of video frame. [N-1:1] Unused.
axi4s_vid_in_tlast	In	1	AXI4-S end of packet.
axi4s_vid_in_tready	Out	1	AXI4-S data ready.
Output			
axi4s_vid_out_tdata	Out	(84)	AXI4-S data out.
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid.
<i>continued...</i>			

(84) The equation gives all tdata widths sizes in these interfaces: pixels in parallel × max (round (bits per color sample × number of color planes, 8), 16)

(85) The equation gives all tuser widths sizes in these interfaces: N = ceil (tdata width / 8)

Name	Direction	Width	Description
axi4s_vid_out_tuser	Out	(85)	[0] AXI4-S start of video frame. [N-1:1] Unused.
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet.
axi4s_vid_out_tready	In	1	AXI4-S data ready.

Related Information

Video and Vision Processing IP Interfaces on page 30

26.4. Demosaic IP Registers

Each register is either read-only (RO) or read-write (RW).

Table 460. Demosaic IP Registers

In the software API the register names appear with a prefix of INTEL_VVP, INTEL_VVP_CORE, or INTEL_VVP_DEMOSAIC as appropriate and with an optional **REG** suffix

Address	Register	Access		Description
		Lite ⁽⁸⁶⁾	Full	
Parameterization registers				
0x0000	VID_PID	RO	N/A	Read this register to retrieve the ID of the Demosaic IP. This register always returns 0x6FA7_0246.
0x0004	VERSION	RO	N/A	Read this register for the IP version information.
0x0008	LITE_MODE	RO	N/A	Read this register to determine if Lite mode is on. This register always returns 1.
0x000c	DEBUG_ENABLED	RO	N/A	Read this register to determine if Debug features is on. This register returns 0 for off and 1 for on.
0x0010	BPS_IN	RO	N/A	Read this register to determine the bits per symbol for the input data.
0x0014	BPS_OUT	RO	N/A	Read this register to determine the bits per symbol for the output data.
0x0018	NUM_COLOR_IN	RO	N/A	Read this register to determine the number of color planes for the input data. This register always returns 1.
0x001c	NUM_COLOR_OUT	RO	N/A	Read this register to determine the number of color planes for the output data. This register always returns 3.
0x0020	PIP	RO	N/A	Read this register to determine the number of pixels in parallel.
0x0024	MAX_WIDTH	RO	N/A	Read this register to determine the maximum supported input field width.
0x0028	MAX_HEIGHT	RO	N/A	Read this register to determine the maximum supported input field height.
continued...				

(86) Registers are RW only if you turn on **Debug features**, otherwise they are WO.

Address	Register	Access		Description
		Lite ⁽⁸⁶⁾	Full	
0x002c to 0x011F	–	–	–	Reserved
Control, debug and statistics registers				
0x0120	IMG_INFO_WIDTH	RW	N/A	The expected width of the incoming video fields.
0x0124	IMG_INFO_HEIGHT	RW	N/A	The expected height of the incoming video fields.
0x0128 to 0x013F	–	–	–	Reserved
0x0140	STATUS	RO	N/A	Read this register for information about the Demosaic IP status.
0x0144	FRAME_STATS	RO	N/A	Read this register for some frame statistics.
0x0148	–	–	–	Reserved
0x014C	CONTROL	RW	N/A	Control bits and fields of Demosaic IP
0x0150 to 0x01FF	–	–	–	Reserved

Register Bit Descriptions

Table 461. STATUS

Name	Bits	Description
Reserved	31:1	Reserved.
Running	0	When 1, the IP is processing data.

Table 462. FRAME_STATS

Name	Bits	Description
Reserved	31:8	Reserved.
Checksum	7:0	A simple checksum of the frame.

Table 463. CONTROL

Name	Bits	Description
Reserved	31:3	Write 0.
Color filter array phase.	2:1	Specifies 2x2 color filter order starting from the top left corner of the image. 00 01 10 11 RG GR GB BG GB BG RG GR
Bypass	0	Set to bypass the Demosaic IP. When set, the IP passes the input image unmodified.

⁽⁸⁶⁾ Registers are RW only if you turn on **Debug features**, otherwise they are WO.

26.5. Demosaic IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_demosaic_regs.h`

Include file: `intel_vvp_demosaic.h`

Table 464. Demosaic IP Software API

Name	Description
<code>intel_vvp_demosaic_init</code>	Initialize an IP instance
<code>intel_vvp_core_*</code>	Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i>
<code>intel_vvp_demosaic_get_lite_mode</code>	Returns if Lite mode is on
<code>intel_vvp_demosaic_get_debug_enabled</code>	Returns if Debug features is on
<code>intel_vvp_demosaic_get_bits_per_sample_in</code>	Returns the number of bits per color sample for the streaming input interface
<code>intel_vvp_demosaic_get_bits_per_sample_out</code>	Returns the number of bits per color sample value for the streaming output interface
<code>intel_vvp_demosaic_get_num_color_planes_in</code>	Returns the number of color planes of the streaming input interface
<code>intel_vvp_demosaic_get_num_color_planes_out</code>	Returns the number of color planes of the streaming output interface
<code>intel_vvp_demosaic_get_pixels_in_parallel</code>	Returns the pixels in parallel of the streaming input and output interfaces
<code>intel_vvp_demosaic_get_max_width</code>	Returns the maximum number of pixels that the IP supports on the horizontal dimension of an image or video frame
<code>intel_vvp_demosaic_get_max_height</code>	Returns the maximum number of pixels that the IP supports on the vertical dimension of an image or video frame
<code>intel_vvp_demosaic_is_running</code>	Reads if the IP is running
<code>intel_vvp_demosaic_get_status</code>	Reads the status register
<code>intel_vvp_demosaic_get_frame_stats</code>	Reads the frame statistics register
<code>intel_vvp_demosaic_get_bypass</code>	Reads the bypass bit from the status register
<code>intel_vvp_demosaic_set_bypass</code>	Writes the bypass bit of the status register
<code>intel_vvp_demosaic_get_cfa_phase</code>	Reads the color filter array phase field from the status register
<code>intel_vvp_demosaic_set_cfa_phase</code>	Writes the color filter array phase field of the status register

intel_vvp_demosaic_init

Prototype

```
int  
intel_vvp_demosaic_init(intel_vvp_demosaic_instance* instance,  
intel_vvp_core_base base);
```

Description

Initialize an IP instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

Arguments

instance - pointer to the intel_vvp_demosaic_instance software driver instance structure

base - base address of the register map

Returns

kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7)

kIntelVvpCorePidErr if the product_id does not match with the expected product ID (0x0246)

kIntelVvpDemosaicRegMapVersionErr if the register map is not supported

intel_vvp_demosaic_get_lite_mode

Prototype

```
bool  
intel_vvp_demosaic_get_lite_mode(intel_vvp_demosaic_instance*  
instance);
```

Description

Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_demosaic_instance fully initialized.

Arguments

instance - pointer to the intel_vvp_demosaic_instance software driver instance structure

Returns

true if the IP is parameterized in lite mode

intel_vvp_demosaic_get_debug_enabled

Prototype

```
bool
intel_vvp_demosaic_get_debug_enabled(intel_vvp_demosaic_instance*
instance);
```

Description Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_demosaic_instance fully initialized.

Arguments instance - pointer to the intel_vvp_demosaic_instance software driver instance structure

Returns true if the IP is parameterized with debug features enabled

intel_vvp_demosaic_get_bits_per_sample_in

Prototype

```
uint8_t
intel_vvp_demosaic_get_bits_per_sample_in(intel_vvp_demosaic_instance*
instance);
```

Description Returns the value of the BPS_IN register. The instance must be a valid intel_vvp_demosaic_instance fully initialized.

Arguments instance - pointer to the intel_vvp_demosaic_instance software driver instance structure

Returns the input bits per color sample parameter used to generate the IP

intel_vvp_demosaic_get_bits_per_sample_out

Prototype

```
uint8_t
intel_vvp_demosaic_get_bits_per_sample_out(intel_vvp_demosaic_instance*
instance);
```

Description Returns the value of the BPS_OUT register. The instance must be a valid intel_vvp_demosaic_instance fully initialized.

Arguments instance - pointer to the intel_vvp_demosaic_instance software driver instance structure

Returns the output bits per color sample parameter used to generate the IP

intel_vvp_demosaic_get_num_color_planes_in

Prototype

```
uint8_t  
intel_vvp_demosaic_get_num_color_planes_in(intel_vvp_demosaic_instance*  
instance);
```

Description Returns the value of the NUM_COLOR_IN register. The instance must be a valid intel_vvp_demosaic_instance fully initialized.

Arguments instance - pointer to the intel_vvp_demosaic_instance software driver instance structure

Returns the input number of color planes parameter used to generate the IP

intel_vvp_demosaic_get_num_color_planes_out

Prototype

```
uint8_t  
intel_vvp_demosaic_get_num_color_planes_out(intel_vvp_demosaic_instance*  
instance);
```

Description Returns the value of the NUM_COLOR_OUT register. The instance must be a valid intel_vvp_demosaic_instance fully initialized.

Arguments instance - pointer to the intel_vvp_demosaic_instance software driver instance structure

Returns the output number of color planes parameter used to generate the IP

intel_vvp_demosaic_get_pixels_in_parallel

Prototype

```
uint8_t  
intel_vvp_demosaic_get_pixels_in_parallel(intel_vvp_demosaic_instance*  
instance);
```

Description Returns the value of the PIXELS_IN_PARALLEL register. The instance must be a valid intel_vvp_demosaic_instance fully initialized.

Arguments instance - pointer to the intel_vvp_demosaic_instance software driver instance structure

Returns the pixels in parallel parameter used to generate the IP

intel_vvp_demosaic_get_max_width

Prototype

```
uint32_t
intel_vvp_demosaic_get_max_width(intel_vvp_demosaic_instance*
instance);
```

Description Returns the value of the MAX_WIDTH register. The instance must be a valid intel_vvp_demosaic_instance fully initialized.

Arguments instance - pointer to the intel_vvp_demosaic_instance software driver instance structure

Returns the maximum field width parameter used to generate the IP

intel_vvp_demosaic_get_max_height

Prototype

```
uint32_t
intel_vvp_demosaic_get_max_height(intel_vvp_demosaic_instance*
instance);
```

Description Returns the value of the MAX_HEIGHT register. The instance must be a valid intel_vvp_demosaic_instance fully initialized.

Arguments instance - pointer to the intel_vvp_demosaic_instance software driver instance structure

Returns the maximum field height parameter used to generate the IP

intel_vvp_demosaic_is_running

Prototype

```
bool
intel_vvp_demosaic_is_running(intel_vvp_demosaic_instance*
instance);
```

Description Returns the value of the IP Running bit from the STATUS register. The instance must be a valid intel_vvp_demosaic_instance fully initialized.

Arguments instance - pointer to the intel_vvp_demosaic_instance software driver instance structure

Returns the running bit from the status register

intel_vvp_demosaic_get_status

Prototype

```
uint32_t  
intel_vvp_demosaic_get_status(intel_vvp_demosaic_instance*  
instance);
```

Description Returns the value of the STATUS register. The instance must be a valid intel_vvp_demosaic_instance fully initialized.

Arguments instance - pointer to the intel_vvp_demosaic_instance software driver instance structure

Returns the value of the status register

intel_vvp_demosaic_get_frame_stats

Prototype

```
int  
intel_vvp_demosaic_get_frame_stats(intel_vvp_demosaic_instance*  
instance, uint32_t* stats_out);
```

Description Reads the value of the FRAME_STATS register. The instance must be a valid intel_vvp_demosaic_instance fully initialized.

Arguments instance - pointer to the intel_vvp_demosaic_instance software driver instance structure

stats_out - pointer of a variable to return the statistics register value

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpDemosaicPointerErr if stats_out is a null pointer

intel_vvp_demosaic_get_bypass

Prototype

```
bool  
intel_vvp_demosaic_get_bypass(intel_vvp_demosaic_instance*  
instance);
```

Description Returns the value of the bypass bit from the CONTROL register. The instance must be a valid intel_vvp_demosaic_instance fully initialized.

Arguments instance - pointer to the intel_vvp_demosaic_instance software driver instance structure

Returns the value of the bypass bit form the control register

intel_vvp_demosaic_set_bypass

Prototype

```
int
intel_vvp_demosaic_set_bypass(intel_vvp_demosaic_instance*
instance, bool bypass);
```

Description Writes to the bypass bit of the CONTROL register. The instance must be a valid intel_vvp_demosaic_instance fully initialized.

Arguments instance - pointer to the intel_vvp_demosaic_instance software driver instance structure

bypass - new value of the bypass bit

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

intel_vvp_demosaic_get_cfa_phase

Prototype

```
uint8_t
intel_vvp_demosaic_get_cfa_phase(intel_vvp_demosaic_instance*
instance);
```

Description Returns the value of the color filter array phase field from the CONTROL register. The instance must be a valid intel_vvp_demosaic_instance fully initialized.

Arguments instance - pointer to the intel_vvp_demosaic_instance software driver instance structure

Returns the value of the color filter array phase field form the control register

intel_vvp_demosaic_set_cfa_phase

Prototype

```
int
intel_vvp_demosaic_set_cfa_phase(intel_vvp_demosaic_instance*
instance, uint8_t cfa_phase);
```

<i>Description</i>	Writes to the color filter array phase field of the CONTROL register. The instance must be a valid intel_vvp_demosaic_instance fully initialized.
<i>Arguments</i>	instance - pointer to the intel_vvp_demosaic_instance software driver instance structure cfa_phase - new value of the color filter array phase field
<i>Returns</i>	kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error kIntelVvpCoreInstanceErr if the instance is a null pointer kIntelVvpDemosaicValueErr if the value of cfa_phase is out of range

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

27. FIR Filter IP

27.1. About the FIR Filter

The FIR Filter IP performs 2D convolutions using matrices of specific coefficients.

The IP is available as full or lite variants. For more information on full and lite, refer to the *Altera Streaming Video Protocol Specification*. The FIR filter IP takes input resolution information from image information packets or using the register interface for lite variants.

An Avalon memory-mapped interface allows you to read the running status of the IP and optionally update the filter coefficients at run time. This interface is mandatory for lite variants.

Related Information

[Altera Streaming Video Protocol Specification](#)

27.1.1. FIR Filter IP Performance and Resources

Altera provides resource and utilization data for guidance.

Table 465.

The table shows ALM usage, M20k usage and f_{MAX} for a design with 4 pixels in parallel, 10 bits per color sample in and out, 3 color planes, max frame width and height of 8192, 5x5 kernel size. Run-time control is on and Lite mode is off.

Target device	ALM	M20K	f_{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	7,974	50	781
Arria 10 speed grade 1 (10AS066H1F34E1HG)	6,590	50	372
Cyclone 10 GX speed grade 5 (10CX220YF78015G)	8,342	50	320
Stratix 10 GX speed grade 1 (1SX280LN2F43E1VG)	8,241	132	390

27.2. FIR Filter Parameters

The IP offers run- and compile-time parameters.

Table 466. FIR Filter Parameters

Parameter	Values	Description
Video Data Format		
Input bits per color sample	8 to 16	Select the number of input bits per color sample.
Output bits per color sample	8 to 16	Select the number of output bits per color sample.
<i>continued...</i>		

Parameter	Values	Description
Number of color planes	1 to 4	Select the number of color planes per pixel.
Number of pixels in parallel	1 to 8	Select the number of pixels in parallel.
Video Blanking	On or off	Turn on when the filtered video has blanking
Maximum Frame Size		
Maximum field height	2048-16384	Specify the maximum height of incoming frames
Maximum field width	2048-16384	Specify the maximum width of incoming frames
Filter Parameters		
Vertical filter taps	3-16	Height of the FIR filter
Horizontal filter taps	3-16	Width of the FIR filter
Vertical kernel mirroring	On or Off	Turn on enable vertical symmetry
Horizontal kernel mirroring	On or Off	Turn on enable horizontal symmetry
Diagonal kernel mirroring	On or Off	Turn on diagonal symmetry
Rounding method	Truncate, round half up or round half even	Select the rounding method of choice for the output of the filter
Signed coefficients	On or Off	Turn on signed coefficients
Coefficient integer bits	0-18	Select the number of the coefficient's integer bits. The sum of integer bits + fraction bits cannot exceed 18
Coefficient fraction bits	0-18	Select the number of the coefficient's fraction bits. The sum of integer bits + fraction bits cannot exceed 18
Control Parameters		
Memory-mapped control interface	On or off	Turn on for the Avalon memory-mapped control interface. You must have the Avalon memory-mapped control interface if you turn on Lite mode
Update Coefficients at runtime	On or off	Turn on run-time coefficient loading via the Avalon memory-mapped interface
Use memory initialization coefficients file	On or off	Turn on for a fixed coefficient file that the IP loads at compile time to assign coefficient values on reset.
Coefficients initialization file	<file path>	Specify the coefficients initialization file path

Figure 82. FIR Filter Parameters

The screenshot shows the 'FIR Filter Intel FPGA IP' configuration window for the 'intel_vvp_fir' component. The window has a 'Details' button in the top right corner. The configuration is organized into several expandable sections:

- Video Data Format:**
 - Input data bit width: 8
 - Output data bit width: 8
 - Number of color planes: 3
 - Number of pixels in parallel: 2
 - ☒ Video blanking
- Maximum Frame Size:**
 - Maximum frame height: 2048
 - Maximum frame width: 2048
- Filter Parameters:**
 - Number of vertical taps: 5
 - Number of horizontal taps: 5
 - ☐ Turns vertical kernel mirroring on
 - ☐ Turns horizontal kernel mirroring on
 - ☐ Turns diagonal kernel mirroring on
 - Rounding method: TRUNCATE
 - ☒ Use signed coefficients
 - Coefficient integer bits: 2
 - Coefficient fraction bits: 8
 - Binary Point Shift: 0
- Control Parameters:**
 - ☒ Memory mapped control interface
 - ☒ Update coefficients at runtime
 - ☐ Use memory initialization coefficients file
 - Coefficient initialization file: <enter file name (including full path)>
- General:**
 - ☒ Pipeline ready signals
 - ☐ Debug features
 - ☒ Separate clock for control interface
 - ☐ Lite mode

27.3. FIR Filter IP Operation

27.3.1. FIR Filter Processing

The IP calculates the output pixel values in 3 stages.

1. Creates kernel

The IP creates an $N \times M$ array of input pixels around the input pixel at the same position in the input image as the position of the output pixel in the output image. This center pixel has $(N-1)/2$ pixels to its left and $N/2$ pixels to its right in the array, and $(M-1)/2$ lines above it and $M/2$ lines below it.

When the pixels to the left, right, above, or below the center pixel in the kernel extend beyond the edges of the image, the filter uses either replication of the edge pixel or full data mirroring, according to the value of a compile time parameter.

2. Convolutes

The IP multiplies each pixel in the $N \times M$ input array by the corresponding coefficient in the $N \times M$ coefficient array. The IP sums the results to produce the filtered value.

The IP retains full precision throughout the calculation of each filtered value, with all rounding and saturation to the required output precision applied as a final stage.

3. Rounds and saturates.

The IP rounds and saturates the resulting full precision filtered value according to the output precision specification

27.3.2. FIR Filter Precision

The FIR IP does not lose calculation precision during the FIR calculation.

- You may parameterize the input data to between 8 to 16 bits per color per pixel. The IP treats this data as unsigned integer data. You may enable optional guard bands at the input to keep the data inside a reduced range of values.
- You may parameterize the coefficient data up to a total width of 18 bits per coefficient. The coefficients may be signed or unsigned and contain up to 18 fractional bits.
- You may parameterize the output data to between 8 to 16 bits per color per pixel, and the selected output data width may be different from the input data width.

To convert from the full precision result of the filtering to the selected output precision, the IP first rounds up the value to remove the required number of fraction bits. Then the IP saturates the value. You may select how many fraction bits should be preserved in the final output using the IP parameter editor. As with the input data, the output data is treated as unsigned, so the IP clips any negative values that result from the filtering to 0. Any values greater than the maximum value that can be represented in the selected number of bits per color per pixel are clipped to this maximum value.

27.3.3. FIR Coefficient Specification

You can specify the FIR filter IP filtering operation coefficients as fixed values that the IP applies on reset if **Use memory initialization coefficients file** is on. Also, you can specify them at run time when you turn on **Memory-mapped control interface** and **Update coefficients at run time**.

The IP requires you to define a fixed-point type for the coefficients. The user-entered coefficients (shown as white boxes in the parameter editor) are rounded to fit in the chosen coefficient fixed-point type (shown as purple boxes in the parameter editor).

- For run-time editable coefficients, you must enter the desired coefficient values through an Avalon memory-mapped agent control interface at run time, and you can update the coefficient values as often as once per frame.
Note: Without enabling the coefficient initialization file, the coefficient values all revert to undefined after every reset, so you must initialize coefficients at least once on start-up.
- To keep the register map as small as possible and to reduce complexity in the hardware, the IP reduces the number of coefficients that it edits at run time when you turn on any of the symmetric modes.
- For T unique coefficient values after symmetry, the register map contains T addresses into which you should write coefficients, starting at address 7 and finishing at T+ 6.

Fixed Coefficient

For fixed coefficients, you specify the values for the coefficients with a comma-separated .csv text file. The selected coefficient values take effect immediate effect at reset.

Regardless of the symmetry mode, the text file must contain a full listing of all the coefficients in the $N \times M$ array i.e. the file must always contain $N \times M$ comma-separated values. When the .csv file is parsed in Platform Designer to create the list of compile time coefficients, the IP checks the values entered against the selected symmetry mode and provides warnings if the coefficients are not symmetric across the selected axes. The values specified in the .csv file must be in their unquantized format. For example, if you want a value of 1.7 for a given coefficient, the value in the file should be 1.75. When the file is parsed in Platform Designer, the coefficients automatically quantize according to the precision you specify.

Note: The quantization process selects the closest value available in the given precision format. If you select the coefficients arbitrarily without reference to the available precision, the quantized value may differ from the desired value.

Run-time Editable Coefficients

To keep the register map as small as possible and to reduce complexity in the hardware, the number of coefficients that are edited at run time is reduced when you turn on any of the symmetric modes.

If the IP has T unique coefficient values after symmetry, the register map contains T addresses into which you should write coefficients, starting at address 0x200 and finishing at $0x200 + (4 \times (T-1))$.

Write coefficient index 0 (as described in the symmetry section) to address 0x200. Then write each successively indexed coefficient at each following address. The updated coefficient set takes effect immediately. To avoid the IP updating coefficients in the middle of a frame, Altera recommends reading bit 3 of the status register to check that the algorithmic core is idle before writing new coefficients.

Note: The coefficient values you write to the register map must be in prequantized format as the hardware cost to implement quantization on floating point values is prohibitive.

27.3.4. FIR Filter Symmetry

The FIR Filter IP supports symmetry modes for coefficient data.

The IP provides five symmetry modes for you to select:

- No symmetry
- Horizontal symmetry
- Vertical symmetry
- Horizontal and vertical symmetry
- Diagonal symmetry

27.3.4.1. No Symmetry

The FIR Filter IP has no axes of symmetry in the 2D coefficient array.

The number of horizontal taps (N) and the number of vertical taps (M) may both be even or odd numbers.

With run-time control of the coefficient data, the register map includes $N \times M$ addresses to allow the value of each coefficient to be updated individually. The IP indexes the coefficients within the register map in raster scan order.

Figure 83. No Symmetry

No Symmetry

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19
20	21	22	23	24

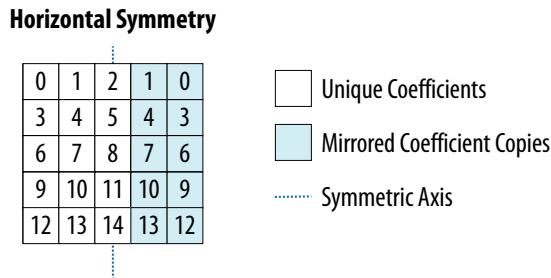
27.3.4.2. Horizontal Symmetry

The FIR Filter IP has 1 axis of symmetry across a vertical line through the center tap in the 2D coefficient array.

The number of vertical taps (M) may be even or odd, but the number of horizontal taps (N) must be odd. Horizontal symmetry offers only $((N+1)/2) \times M$ unique coefficient values. The remaining values in the array are mirrored duplicates.

With run-time control of the coefficients, the register map only includes addresses to update the $((N+1)/2) \times M$ unique coefficient values, indexed for an example 5×5 array as shown in the figure below.

Figure 84. Horizontal Symmetry



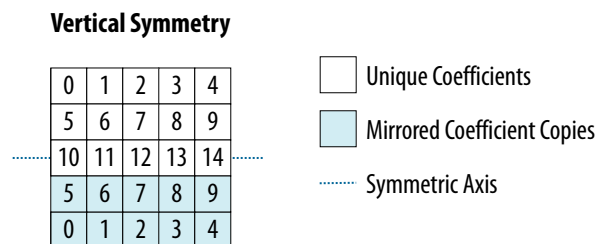
27.3.4.3. Vertical Symmetry

The FIR Filter IP has 1 axis of symmetry across a horizontal line through the center tap in the 2D coefficient array.

In this case, the number of horizontal taps (N) may be even or odd, but the number of vertical taps (M) must be odd. Vertical symmetry offers only $N \times ((M+1)/2)$ unique coefficient values, with the remaining values in the array being mirrored duplicates.

With run-time control of the coefficients, the register map only includes addresses to update the $N \times ((M+1)/2)$ unique coefficient values, indexed for an example 5×5 array.

Figure 85. Vertical Symmetry



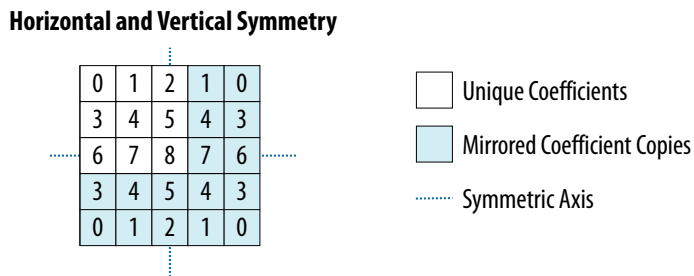
27.3.4.4. Horizontal and Vertical Symmetry

The FIR Filter IP has 2 axes of symmetry across a horizontal line and a vertical line through the center tap in the 2D coefficient array.

In this case, the number of horizontal taps (N) and the number of vertical taps (M) must be even. Horizontal and vertical symmetry offers only $((N+1)/2) \times ((M+1)/2)$ unique coefficient values. The remaining values in the array are mirrored duplicates.

With run-time control of the coefficients, the register map only includes addresses to update the $((N+1)/2) \times ((M+1)/2)$ unique coefficient values, indexed for an example 5×5 array.

Figure 86. Horizontal and Vertical Symmetry



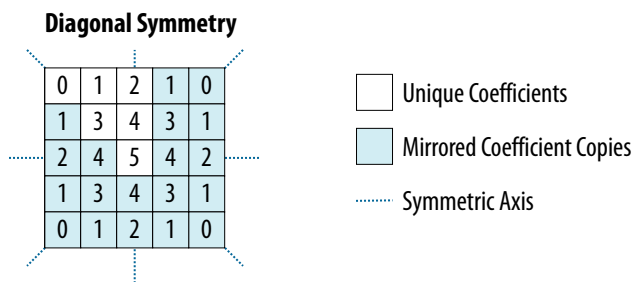
27.3.4.5. Diagonal Symmetry

The FIR Filter IP has 4 axes of symmetry in the 2D coefficient array across a horizontal line, a vertical line, and 2 diagonal lines through the center tap.

In this case, the number of horizontal taps (N) and the number of vertical taps (M) must be odd, and they must have same value ($N = M$). Diagonal symmetry offers only $T_u = ((N+1)/2)$ unique coefficient values in either the horizontal or vertical directions, and a total of $(T_u * (T_u + 1))/2$ unique coefficient values.

With run-time control of the coefficients, the register map only includes addresses to update the $(T_u * (T_u + 1))/2$ unique coefficient values, indexed for an example 5x5 array.

Figure 87. Diagonal Symmetry



27.3.5. Result to Output Data Type Conversion

After calculation, the FIR Filter IP converts the fixed-point type of the results to the integer data type of the output.

1. Scales result. Scaling quickly increases the color depth of the output. You can shift the binary point right -16 to +16 places. The IP implements scaling as a simple shift operation so it does not require multipliers.
2. Removes fractional bits. If any fractional bits exist, you can choose to remove them through these methods:

- Truncate to integer. The IP removes fractional bits from the data; equivalent to rounding towards negative infinity.
 - Round half up. The IP rounds up to the nearest integer. If the fractional bits equal 0.5, rounding is towards positive infinity.
 - Round half even. The IP rounds to the nearest integer. If the fractional bits equal 0.5, rounding is towards the nearest even integer.
3. Convert from signed to unsigned. If any negative numbers exist in the results and the output type is unsigned, you can convert to unsigned through these methods:
 - Saturate to the minimum output value (constraining to range).
 - Replace negative numbers with their absolute positive value.
 4. Constrain to range. If any of the results are beyond a specific range, the IP automatically adds logic to saturate the results to the minimum and maximum output values. The specific range is the specified range of the output guard bands, or if unspecified, the minimum and maximum values allowed by the output bits per pixel.

27.4. FIR Filter Interfaces

Table 467. FIR Filter Clock and Reset Interfaces

Name	Direction	Width	Description
main_clock_clk	In	1	AXI4-S processing clock.
main_reset_rst	In	1	AXI4-S processing reset.
control_clock_clk	In	1	Optional control agent interface clock.
control_reset_reset	In	1	Optional control agent interface reset.

Table 468. FIR Filter Control Interfaces

Name	Direction	Width	Description
av_mm_control_agent_address	In	7	Avalon memory-mapped agent address
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write.
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data.
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable.
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read.
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data.
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read.
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request.

Table 469. FIR Filter Altera Streaming Video Interfaces

Name	Direction	Width	Description
axi4s_vid_in_tdata	In		AXI4-S data in.
axi4s_vid_in_tvalid	In	1	AXI4-S data valid.
<i>continued...</i>			

Name	Direction	Width	Description
axi4s_vid_in_tuser[0]	In	1	AXI4-S start of video frame.
axi4s_vid_in_tuser[1]	In	1	AXI4-S control or data packet.
axi4s_vid_in_tuser[N-1:2]	In		Unused.
axi4s_vid_in_tlast	In	1	AXI4-S end of packet .
axi4s_vid_in_tready	Out	1	AXI4-S data ready.
axi4s_vid_out_tdata	Out	(⁽⁸⁷⁾)	AXI4-S data in.
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid.
axi4s_vid_out_tuser[0]	Out	1	AXI4-S start of video frame.
axi4s_vid_out_tuser[1]	Out	1	AXI4-S control or data packet.
axi4s_vid_out_tuser[N-1:2]	Out	(⁽⁸⁸⁾)	Unused.
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet .
axi4s_vid_out_tready	In	1	AXI4-S data ready

Table 470. FIR Filter Avalon memory-mapped host interfaces

Specify widths for address, data and burst count buses in the GUI. The table shows example widths.

Name	Direction	Width	Description
mem_clock_clk	In	1	Optional host interface clock.
mem_reset_reset	In	1	Optional host interface reset.
av_mm_mem_write_host_address	Out	32	Avalon memory-mapped host address
av_mm_mem_write_host_write	Out	1	Avalon memory-mapped host write.
av_mm_mem_write_host_burstcount	Out	5	Avalon memory-mapped host write burst count.
av_mm_mem_write_host_writedata	Out	64	Avalon memory-mapped host write data.
av_mm_mem_write_host_waitrequest	In	1	Avalon memory-mapped host wait request.
av_mm_mem_read_host_address	Out	32	Avalon memory-mapped host address
av_mm_mem_read_host_read	Out	1	Avalon memory-mapped host read.
av_mm_mem_read_host_burstcount	Out	5	Avalon memory-mapped host read burst count.
av_mm_mem_read_host_readdata	In	64	Avalon memory-mapped host read data.
av_mm_mem_read_host_readdatavalid	In	1	Avalon memory-mapped host read data valid.
av_mm_mem_read_host_waitrequest	In	1	Avalon memory-mapped host wait request.

Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

(87) The equation gives all tdata widths sizes in these interfaces: pixels in parallel × max (round (bits per color sample × number of color planes, 8), 16)

(88) This equation gives all tuserwidths sizes in these interfaces: N = ceil (tdata width / 8)

27.5. FIR Filter Registers

Each register is either read-only (RO) or read-write (RW).

Address	Register	Access		Description
		Lite (89)	Full	
Parameterization registers				
0x0000	PROD_ID	RO		Read this register to retrieve the FIR Filter product ID. This register always returns 0x6FA7_022C.
0x0004	VER	RO		Read this register for the IP version information.
0x0008	LITE_MODE	RO		Read this register to determine if Lite mode is on. This register returns 0 for full mode and 1 for lite mode.
0x000C	DEBUG_ENABLED	RO		Read this register to determine if Debug features is on. This register returns 0 for off and 1 for on.
0x0010	MAX_FRAME_WIDTH	RO		Read this register to determine the maximum supported input field width.
0x0014	MAX_FRAME_HEIGHT	RO		Read this register to determine the maximum supported input field height.
0x0018	BPS_IN	RO		Read this register to determine the bits per symbol for the input data.
0x001C	BPS_OUT	RO		Read this register to determine the bits per symbol for the output data.
0x0020	ROUNDING_METHOD	RO		Read this register to determine how the IP removes the fractional bits when converting output result back to integer format. • 1: Round half up • 2: Round half even • 3: Truncate to integer
0x0024	BINARY_POINT_SHIFT	RO		Read this register to determine the number of places by which the binary point moves to the right. Use to scale the result of the calculation
0x0028	USE_FIXED_COEFF	RO		Read this register to determine if the IP is set to use a fixed coefficient file. This register returns 0 for false and 1 for true.
0x002C	H_TAPS	RO		Read this register to determine the number of horizontal filter taps.
0x0030	V_TAPS	RO		Read this register to determine the number of vertical filter taps.
0x0034	HORIZONTAL_SYM_COEFFS	RO		Read this register to determine if horizontal kernel mirroring is on. This register returns 0 for false and 1 for true.
continued...				

⁽⁸⁹⁾ When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

Address	Register	Access		Description
		Lite (89)	Full	
0x0038	VERTICAL_SYM_COEFFS	RO		Read this register to determine if vertical kernel mirroring is on. This register returns 0 for false and 1 for true.
0x003C	DIAGONAL_SYM_COEFFS	RO		Read this register to determine if diagonal kernel mirroring is on. This register returns 0 for false and 1 for true
0x0040	SIGNED_COEFFS	RO		Read this register to determine whether the filter uses signed coefficients. This register returns 0 for unsigned and 1 for signed
0x0044	COEFF_INT_BITS	RO		Read this register to determine the number of integer bits that the fixed-point data type uses to store coefficients
0x0048	COEFF_FRAC_BITS	RO		Read this register to determine the number of fractional bits that the fixed-point data type uses to store coefficients
0x004C	COEFFS_RUNTIME_LOAD	RO		Read this register to determine if updates to the coefficients at run time via the Avalon memory-mapped agent interface are supported. Returns 1 if updates are supported and 0 otherwise.
0x0050 to 0x011F	-	-		Unused.
Control and debug registers For more information, refer to <i>Control Packets</i>				
0x0120	IMG_INFO_WIDTH	RW	RO	When you turn on lite mode , use this register to set the expected width of incoming video fields. When you turn off Lite mode and turn on Debug features , this register returns the width that the FIR Filter derives from information in the image information packet
0x0124	IMG_INFO_HEIGHT	RW	RO	When you turn on lite mode , use this register to set the expected height of incoming video fields. When you turn off Lite mode and turn on Debug features , this register returns the height that the FIR Filter derives from information in the image information packet.
0x0128	IMG_INFO_INTERLACE	RO	RO	When you turn on Lite Mode , set the expected interlaced properties of incoming video fields in this register. Set bit 3 of the register to indicate interlaced video and to enable propagation of the F0/F1 tuser bit. When you turn off Lite mode and turn on Debug features , this register returns the interlace nibble that the FIR Filter derives from information in the image information packet.
0x012C	RESERVED	-	-	Unused.
0x0130	IMG_INFO_COLORSPACE	-	RO	When you turn off Lite mode and turn on Debug features , this register returns the color space that the FIR Filter derives from information in the image information packet. Unused in lite mode.
continued...				

(89) When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

Address	Register	Access		Description
		Lite (89)	Full	
0x0134	IMG_INFO_SUBSAMPLING	-	RO	When you turn off Lite mode and turn on Debug features , this register returns the subsampling that the FIR Filter derives from information in the image information packet. Unused in lite mode.
0x0138	IMG_INFO_COSITING	-	RO	When you turn off Lite mode and turn on Debug features , this register returns the cositing that the FIR Filter derives from information in the image information packet. Unused in lite mode.
0x013C	IMG_INFO_FIELD_COUNT	-	RO	When you turn off Lite mode and turn on Debug features , this register returns the field count that the FIR Filter derives from information in the image information packet. Unused in lite mode.
0x0140	STATUS	RO	RO	- Bit 0: Status bit. 1 means FIR Filter is processing a video field, 0 otherwise. - Bit 2: Algo core Idle. 1 indicates algorithmic core is finished processing all lines and it is safe to update coefficient values.
0x0144 to 0x0200	-	-	-	Unused
0x0200 to 0x0200 + 4*(Number of coefficients-1)	COEFFICIENTS	WO	WO	Up to [V_TAPS * H_TAPS] number of registers that set the new coefficient values. The coefficients apply to the filter kernel in raster scan order. Only available if COEFFS_RUNTIME_LOAD is on.

Register Bit Descriptions

Table 471. VID_PID

Name	Bits	Description
FIR Filter vendor ID and product ID	31:0	This register always returns 0x6AF7_022C. 15:0 is the product ID and always returns 0x022C 31:16 is the vendor ID and always returns 0x6AF7

Table 472. VERSION

Name	Bits	Description
Register map version	7:0	Register map version.
IP patch revision	15:8	-
IP update revision	23:16	Updated when the IP version changes.
IP major revision	31:24	Updated when the IP version changes.

Table 473. LITE_MODE

Name	Bits	Description
Lite mode parameterization bit	0	Returns 1 if you turn on lite mode .
Unused	31:1	Unused.

⁽⁸⁹⁾ When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

Table 474. DEBUG_ENABLED

Name	Bits	Description
Debug features parameterization bit	0	Returns 1 if you turn on Debug features .
Unused	31:1	Unused.

Table 475. MAX_FRAME_WIDTH

Name	Bits	Description
Maximum input width	31:0	Maximum supported input field width.

Table 476. MAX_FRAME_HEIGHT

Name	Bits	Description
Maximum input height	31:0	Maximum supported input field height.

Table 477. BPS_IN

Name	Bits	Description
BPS_IN	31:0	Returns the value of the BPS_IN parameter, indicating the bit per symbol the input interface is configured for.

Table 478. BPS_OUT

Name	Bits	Description
BPS_OUT	31:0	Returns the value of the BPS_OUT parameter, indicating the bit per symbol the output interface is configured for.

Table 479. ROUNDING_METHOD

Name	Bits	Description
Round method	31:0	Returns the value of the round method parameter, indicating method used to round excess fractional bits from the result. 1: Round half up 2: Round half even 3: Truncate to integer

Table 480. BINARY_POINT_SHIFT

Name	Bits	Description
Binary point shift	31:0	Returns the value of the move binary point parameter, indicating the number of places to the right the binary point is shifted by when scaling the result. A negative value indicates a shift to the left.

Table 481. USE_FIXED_COEFF

Name	Bits	Description
Using a fixed coeff file.	31:0	Returns 1 if Coefficient initialization file is on, indicating the core uses a hex file to assign coefficient values on reset. 0 otherwise.

Table 482. H_TAPS

Name	Bits	Description
Horizontal filter taps	31:0	Returns the number of horizontal filter taps.

Table 483. V_TAPS

Name	Bits	Description
Vertical filter taps	31:0	Returns the number of vertical filter taps.

Table 484. HORIZONTAL_SYM_COEFFS

Name	Bits	Description
Horizontal symmetry coefficients	31:0	Returns 1 if Horizontal kernel mirroring is on and 0 otherwise.

Table 485. VERTICAL_SYM_COEFFS

Name	Bits	Description
Vertical symmetry coefficients	31:0	Returns 1 if Vertical kernel mirroring is on and 0 otherwise.

Table 486. DIAGONAL_SYM_COEFFS

Name	Bits	Description
Diagonal symmetry coefficients	31:0	Returns 1 if Diagonal kernel mirroring is on and 0 otherwise.

Table 487. SIGNED_COEFFS

Name	Bits	Description
Signed coefficients	31:0	Returns 1 if filter coefficients are signed values and 0 otherwise.

Table 488. COEFF_INT_BITS

Name	Bits	Description
Coefficient integer bits	31:0	Returns the number of integer bits that represent the filter coefficients.

Table 489. COEFF_FRAC_BITS

Name	Bits	Description
Coefficient fractional bits	31:0	Returns the number of fractional bits that represent the filter coefficients.

Table 490. COEFFS_RUNTIME_LOAD

Name	Bits	Description
Coefficient runtime load	31:0	Returns 1 if Update coefficients at runtime is on and 0 otherwise.

Table 491. IMG_INFO_WIDTH

Name	Bits	Description
Width bits	15:0	When Lite mode is on, write to this register to set the expected width of the incoming video fields.
<i>continued...</i>		

		When Lite mode is off and Debug features is on, this register returns the <code>width-1</code> field from the most recently received image information packet and adds 1 to return a value for <code>width</code> .
unused	31:16	Unused.

Table 492. IMG_INFO_HEIGHT

Name	Bits	Description
Heightbits	15:0	When lite mode is on, write to this register to set the expected height of the incoming video fields. When Lite mode is off and Debug features is on, this register reads the <code>height-1</code> field from the most recently received image information packet and adds 1 to return a value for <code>height</code> .
unused	31:16	Unused.

Table 493. IMG_INFO_INTERLACE

Name	Bits	Description
IntlaceNibble bits	3:0	When Lite mode is on, write to this register to set expected interlace properties of incoming video field. Set bit 3 of the register to indicate interlaced video and to enable propagation of the F0/F1 tuser bit. When Lite mode is off with Debug features on, this register returns the <code>intlaceNibble</code> field from the most recently received image information packet.
unused	31:4	Unused.

Table 494. IMG_INFO_COLORSPACE

Name	Bits	Description
CSP code bits	6:0	When Lite mode is on, this register has no function. When you turn off Lite mode and turn on Debug features , this register returns the 7 bit CSP field from the most recently received image information packet.
unused	31:7	Unused.

Table 495. IMG_INFO_SUBSAMPLING

Name	Bits	Description
SubSa code bits	1:0	When Lite mode is on, this register has no function. When Lite mode is off with Debug features on, this register returns the <code>SUBSA</code> field from the most recently received image information packet.
unused	31:2	Unused.

Table 496. IMG_INFO_COSITING

Name	Bits	Description
Cosite code bits	1:0	When you turn on Lite mode , write to this register to set the expected chroma co-siting of the incoming video fields. When you turn off Lite mode and turn on Debug features , this register returns the COSITE field from the most recently received image information packet.
unused	31:2	Unused.

Table 497. IMG_INFO_FIELD_COUNT

Name	Bits	Description
Count bits	6:0	When you turn on Lite mode , this register has no function. When you turn off Lite mode and turn on Debug features , this register returns the 7 bit FIELD_COUNT field from the most recently received image information packet.
unused	31:7	Unused.

Table 498. STATUS

Name	Bits	Description
Status bit	0	1 indicates FIR filter is processing a video field, 0 otherwise.
Reserved	1	Reserved
Algo core idle	2	1 indicates algorithmic core is finished processing all lines and it is safe to update coefficient values. 0 otherwise.
Unused	31:3	Unused

Table 499. COEFFICIENTS_X

Name	Bits	Description
Coefficient value	Coefficient_width-1:0	Coefficient value for filter tap X
Unused	31:coefficient_width	

27.6. FIR Filter IP Software API

The IP has a software driver for software control of the IP at run time. The IP does not fit any of the generic device models provided by the Nios II HAL. It exposes a set of dedicated accessors to the control and status registers. The driver structure inherits the base driver structure so all common methods described in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_fir_regs.h`

Include file: `intel_vvp_fir.h`

Table 500. FIR Filter IP Software API

Name	Description
<code>intel_vvp_fir_init</code>	Initialize the FIR Filter instance
<code>Intel_vvp_core_*</code>	Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i>
<code>intel_vvp_fir_get_lite_mode</code>	Returns if lite mode is on.
<code>intel_vvp_fir_get_debug_enabled</code>	Returns if debug features is on.
<code>intel_vvp_fir_uses_fixed_coeffs_file</code>	Returns whether the IP uses a set of compile-time coefficients to initialize the filter at start
<code>intel_vvp_fir_get_max_width</code>	Returns the maximum supported width
<code>intel_vvp_fir_get_max_height</code>	Returns the maximum supported height
<i>continued...</i>	

Name	Description
intel_vvp_fir_get_bits_per_sample_in	Returns the number of bits per color sample for the input interface
intel_vvp_fir_get_bits_per_sample_out	Returns the number of bits per color sample for the output interface
intel_vvp_fir_get_num_h_taps	Returns the width of the FIR kernel
intel_vvp_fir_get_num_v_taps	Returns the height of the FIR kernel
intel_vvp_fir_has_horizontal_symmetry	Returns if the FIR coefficients are symmetrical in the horizontal dimension
intel_vvp_fir_has_vertical_symmetry	Returns if the FIR coefficients are symmetrical in the vertical dimension
intel_vvp_fir_has_diagonal_symmetry	Returns if the FIR coefficients are diagonally symmetric
intel_vvp_fir_get_num_coeffs	Returns the number of unique FIR coefficients once symmetries are considered
intel_vvp_fir_are_coeffs_signed	Returns the signed/unsigned status of coefficients
intel_vvp_fir_get_coeffs_int_bits	Returns the number of integer bits used for coefficients (sign bit excluded)
intel_vvp_fir_get_coeffs_frac_bits	Returns the number of fractional bits used for filter coefficients
intel_vvp_fir_get_binary_point_right_move	Returns the left/right binary point move applied to the computation
intel_vvp_fir_get_rounding_method	Returns the rounding algorithm used for the final floating-point to integer conversion
intel_vvp_fir_runtime_coeffs_load	Returns if you can update the coefficients at run time
intel_vvp_fir_is_running	Returns if the IP is currently processing video data
intel_vvp_fir_get_status	Reads the status register
intel_vvp_fir_set_quantized_coeffs	Programs the FIR Filter with a set of quantized coefficients
intel_vvp_fir_set_coeffs	Quantizes a set of floating-point coefficients and programs the FIR Filter.

enum eIntelVvpFirRounding

- Members**
- kIntelVvpFirRoundUp (1) – Rounding up (half up towards positive infinity)
 - kIntelVvpFirRoundHalfEven (2) – Rounding half to even (bankers' rounding)
 - kIntelVvpFirRoundTruncate (3) – Truncation (no rounding)
 - kIntelVvpFirRoundInvalid (-1) – Invalid rounding method (used to indicate errors)

Description Enumeration for the rounding method used by the IP

intel_vvp_fir_init

Prototype

```
int intel_vvp_fir_init(intel_vvp_fir_instance *instance,
intel_vvp_core_base base);
```

Description

Initializes a FIR Filter instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

Argument

instance – pointer to the intel_vvp_fir_instance software driver instance structure

base – base address of the register map

Returns

kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product_id does not match with the FIR Filter product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpFirRegMapVersionErr if the register map is not supported

intel_vvp_fir_get_lite_mode

Prototype

```
bool intel_vvp_fir_get_lite_mode(intel_vvp_fir_instance
*instance);
```

Description

Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_fir_instance fully initialized

Argument

instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns

true if the IP is parameterized in lite mode

intel_vvp_fir_get_debug_enabled

Prototype

```
bool intel_vvp_fir_get_debug_enabled(intel_vvp_fir_instance
*instance);
```

Description

Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_fir_instance fully initialized.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns true if the IP is parameterized with debug features enabled

intel_vvp_fir_uses_fixed_coeffs_file

Prototype

```
bool
intel_vvp_fir_uses_fixed_coeffs_file(intel_vvp_fir_instance
*instance);
```

Description Returns the value of the USES_FIXED_COEFFS_FILE register. The instance must be a valid intel_vvp_fir_instance fully initialized.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns True if the FIR Filter is configured with static compile-time coefficients, false if coefficients are uninitialized at startup

intel_vvp_fir_get_max_width

Prototype

```
uint32_t intel_vvp_fir_get_max_width(intel_vvp_fir_instance
*instance);
```

Description Returns the value of the MAX_WIDTH register. The instance must be a valid intel_vvp_fir_instance fully initialized.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns Maximum supported width

intel_vvp_fir_get_max_height

Prototype

```
uint32_t intel_vvp_fir_get_max_height(intel_vvp_fir_instance
*instance);
```

Description Returns the value of the MAX_HEIGHT register. The instance must be a valid intel_vvp_fir_instance fully initialized.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns Maximum supported height

intel_vvp_fir_get_bits_per_sample_in

Prototype

```
uint8_t intel_vvp_fir_get_bits_per_sample_in
(intel_vvp_fir_instance *instance);
```

Description Returns the value of the BPS_IN register. The instance must be a valid intel_vvp_fir_instance fully initialized.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns Number of bits per color samples for the input interface

intel_vvp_fir_get_bits_per_sample_out

Prototype

```
uint8_t
intel_vvp_fir_get_bits_per_sample_out(intel_vvp_fir_instance
*instance);
```

Description Returns the value of the BPS_OUT register. The instance must be a valid intel_vvp_fir_instance fully initialized.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns Number of bits per color samples for the output interface

intel_vvp_fir_get_num_h_taps

Prototype

```
uint8_t intel_vvp_fir_get_num_h_taps (intel_vvp_fir_instance
*instance);
```

Description Returns the value of the NUM_H_TAPS register. The instance must be a valid intel_vvp_fir_instance fully initialized.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns The width of the 2D FIR kernel

intel_vvp_fir_get_num_v_taps

Prototype

```
uint8_t intel_vvp_fir_get_num_v_taps (intel_vvp_fir_instance
*instance);
```

Description Returns the value of the NUM_V_TAPS register. The instance must be a valid intel_vvp_fir_instance fully initialized.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns The height of the 2D FIR kernel

`intel_vvp_fir_has_horizontal_symmetry`

Prototype

```
bool  
intel_vvp_fir_has_horizontal_symmetry(intel_vvp_fir_instance  
*instance);
```

Description Returns the value of the HORIZONTAL_SYM register. The instance must be a valid intel_vvp_fir_instance fully initialized.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns Whether the 2D FIR kernel is symmetrical horizontally

`intel_vvp_fir_has_vertical_symmetry`

Prototype

```
bool  
intel_vvp_fir_has_vertical_symmetry(intel_vvp_fir_instance  
*instance);
```

Description Returns the value of the VERTICAL_SYM register. The instance must be a valid intel_vvp_fir_instance fully initialized.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns Whether the 2D FIR kernel is symmetrical vertically

`intel_vvp_fir_has_diagonal_symmetry`

Prototype

```
bool  
intel_vvp_fir_has_diagonal_symmetry(intel_vvp_fir_instance  
*instance);
```

Description Returns the value of the DIAGONAL_SYM register. The instance must be a valid intel_vvp_fir_instance fully initialized.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns Whether the 2D FIR kernel is symmetrical diagonally

`intel_vvp_fir_get_num_coeffs`

Prototype

```
uint8_t intel_vvp_fir_get_num_coeffs(intel_vvp_fir_instance*  
instance);
```

Description Deduce the number of unique coefficients used by the FIR kernel from the kernel dimensions and the symmetries.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns The number of unique coefficients used to program the FIR

intel_vvp_fir_are_coeffs_signed

Prototype

```
bool intel_vvp_fir_are_coeffs_signed(intel_vvp_fir_instance* instance);
```

Description Returns the value of the COEFFS_SIGNED register. The instance must be a valid intel_vvp_fir_instance fully initialized.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns Whether the 2D FIR coefficients have a sign bit

intel_vvp_fir_get_coeffs_int_bits

Prototype

```
bool intel_vvp_fir_get_coeffs_int_bits(intel_vvp_fir_instance* instance);
```

Description Returns the value of the COEFFS_INT_BITS register. The instance must be a valid intel_vvp_fir_instance fully initialized.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns The number of integer bits used by fixed-point coefficients

intel_vvp_fir_get_coeffs_frac_bits

Prototype

```
bool intel_vvp_fir_get_coeffs_frac_bits(intel_vvp_fir_instance* instance);
```

Description Returns the value of the COEFFS_FRAC_BITS register. The instance must be a valid intel_vvp_fir_instance fully initialized.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns The number of fractional bits used by fixed-point coefficients

intel_vvp_fir_get_binary_point_right_move

Prototype

```
int8_t  
intel_vvp_fir_get_binary_point_right_move(intel_vvp_fir_instance*  
instance);
```

Description Returns the value of the BINARY_POINT_RIGHT_MOVE register. The instance must be a valid intel_vvp_fir_instance fully initialized.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns The number of bits to the right used to move the binary point

intel_vvp_fir_get_rounding_method

Prototype

```
eIntelVvpFirRounding  
intel_vvp_fir_get_rounding_method(intel_vvp_fir_instance*  
instance);
```

Description Returns the value of the ROUND_METHOD register. The instance must be a valid intel_vvp_fir_instance fully initialized.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns The rounding algorithm that converts fixed-point values back to integer values at the output of the FIR.

intel_vvp_fir_runtime_coeffs_load

Prototype

```
bool intel_vvp_fir_runtime_coeffs_load(intel_vvp_fir_instance*  
instance);
```

Description Returns the value of the RUNTIME_COEFFS_LOAD register. The instance must be a valid intel_vvp_fir_instance fully initialized

Argument instance - pointer to the intel_vvp_fir_instance software driver instance structure

Returns True if the IP supports run-time loading of coefficients

intel_vvp_fir_is_running

Prototype

```
bool intel_vvp_fir_is_running(intel_vvp_fir_instance*  
instance);
```

Description Reads and returns the running bit of the STATUS register. The instance must be a valid intel_vvp_fir_instance fully initialized.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns True is the IP is currently generating an output field, false when between two fields

intel_vvp_fir_get_status

Prototype

```
uint8_t intel_vvp_fir_get_status(intel_vvp_fir_instance* instance);
```

Description Reads and returns the STATUS register. The instance must be a valid intel_vvp_fir_instance fully initialized.

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

Returns The value read from the status register

intel_vvp_fir_set_quantized_coeffs

Prototype

```
int intel_vvp_fir_set_quantized_coeffs(intel_vvp_fir_instance* instance, const int32_t* quantized_coeffs, uint8_t num_coeffs);
```

Description Programs the IP with a new set of coefficients for the FIR kernel. An error is returned if the IP is not properly configured to use these coefficients. The num_coeffs parameter indicates the size of the quantized_coeffs array and must match with the number of unique coefficients in the 2D kernel once symmetries are considered. Also, the call checks for correctness of the quantized values and returns an error if invalid bits are set. Sign extension to the full 32 bit range is accepted. Invalid quantized values do not prevent the coefficients from being updated

Argument instance – pointer to the intel_vvp_fir_instance software driver instance structure

quantized_coeffs – pointer to the array containing the set of quantized coefficients

num_coeffs – the number of coefficients in the array. This must match with the number of expected coefficients:
intel_vvp_fir_get_num_coeffs(instance)

Returns kIntelVvpCoreOk in case of success, negative error code otherwise

kIntelVvpCoreInstanceErr if the instance is invalid, or not configured for run-time loading of coefficients

kIntelVvpFirParameterErr if the number of coefficients does not match with the IP configuration

kIntelVvpCoreNullPtrErr if quantized_coefs is a NULL pointer

kIntelVvpFirQuantizationErr if the quantized coefficients are out-of-range (this does not prevent the coefficients update)

intel_vvp_fir_set_coefs

Prototype

```
int intel_vvp_fir_set_coefs(intel_vvp_fir_instance* instance,  
const float* coefs, uint8_t num_coefs);
```

Description

Programs the IP with a new set of coefficients for the FIR kernel. Coefficients are quantized using the fixed-point configuration reported by the IP. An error is returned if the IP is not properly configured to use these coefficients. The num_coefs parameter indicates the size of the coefs array and must match with the number of unique coefficients in the 2D kernel once symmetries are considered. An error is returned if the quantization could not be achieved successfully.

Argument

instance – pointer to the intel_vvp_fir_instance software driver instance structure

coefs – pointer to the array containing the set of floating-point coefficients

num_coefs – the number of coefficients in the array. This must match with the number of expected coefficients:
intel_vvp_fir_get_num_coefs(instance)

Returns

kIntelVvpCoreOk in case of success, negative error code otherwise

kIntelVvpCoreInstanceErr if the instance is invalid, or not configured for run-time loading of coefficients

kIntelVvpFirParameterErr if the number of coefficients does not match with the IP configuration

kIntelVvpCoreNullPtrErr if coefs is a NULL pointer

kIntelVvpFirQuantizationErr if the quantized coefficients are out-of-range (this does not prevent the coefficients update)

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

28. Frame Cleaner IP

28.1. About the Frame Cleaner IP

The Altera Streaming Video Protocol allows for the transmission of video fields that do not conform to expected width and height values (field size mismatch errors). The IP removes field size mismatch errors by ensuring that every field matches the expected width and height. The IP either crops or pads errant fields to match the values specified. The IP can use both the full and lite variants of the Altera Streaming Video Protocol.

Every IP in the Video and Vision Processing Suite is tolerant of field size mismatch errors, so including the Frame Cleaner IP in a pipeline built entirely from Altera IPs is not necessary. It can help in the debug processes when building and testing a video pipeline.

If you choose to write your own Altera Streaming Video compliant IPs, you may add the Frame Cleaner IP to the pipeline at the input to your IPs. The IP allows you to write the code for your IPs without considering their behavior in all the potential error cases.

28.1.1. Frame Cleaner IP Performance and Resources

Altera provides resource and utilization data for guidance.

Table 501. Frame Cleaner IP Performance and Resources

For an example parameterization of 8 bits per color sample, 3 color planes, 1 pixel in parallel

Target device	ALMs	f _{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	282	868
Arria 10 speed grade 1 (10AS066H1F34E1HG)	256	645
Cyclone 10 GX speed grade 5 (10CX220YF78015G)	255	638
Stratix 10 GX speed grade 1 (1SX280LN2F43E1VG)	277	819

28.2. Frame Cleaner IP Parameters

The IP offers run-time and compile-time parameters.

Parameters	Allowed range	Description
Video data format		
Lite mode	On or off	Turn on to use the lite variant of the Altera Streaming Video Protocol.
Bits per color sample	8 to 16	Select the number of bits per color sample.
<i>continued...</i>		

Parameters	Allowed range	Description
Number of color planes	1 to 4	Select the number of color planes per pixel.
Number of pixels in parallel	1 to 8	Select the number of pixels in parallel at the input and output interfaces.
Protocol settings		
Discard auxiliary control packets	On or off	Turn on to discard all auxiliary control packets (packet types > 1). Not applicable if Lite mode is on.
Custom resolution limits	On or off	Turn on to apply user-set limits to the minimum and maximum field width and height reported in the outgoing image information packets. Not applicable if Lite mode is on.
Minimum field width	1 to 65536	Set the minimum output field width.
Maximum field width	1 to 65536	Set the maximum output field width.
Minimum field height	1 to 65536	Set the minimum output field height.
Maximum field height	1 to 65536	Set the maximum output field height.
Control settings		
Debug features	On or off	Turn on to read back frame information registers (full variant only) and debug information registers via the control agent interface.
Separate clock for control interface	On or off	Turn on for a separate clock for the control agent interface.
Pipeline optimization		
Pipeline ready signals	On or off	Turn on to add extra pipeline registers to the AXI4-S <code>tready</code> signals.

28.3. Frame Cleaner IP Functional Description

When using the full variant of the protocol, the expected field width and height are in the image information packets that precede each field. When using the lite variant, you specify the field width and height to the Frame Cleaner IP via the register map.

In both variants of the protocol, the video fields may have too few or too many pixels in each line packet compared to the specified field width. Each line packet can even contain a different number of pixels, which does not violate the protocol specification. For line packets with too many pixels, the IP clips the surplus beats of data. For line packets with too few pixels, extra beats of data are appended to the packet to match the expected number of pixels.

Video fields may also have too few or too many line packets compared to the specified field height. For fields with too many line packets, the extra packets are discarded. For fields with too few packets, additional packets are added after the last input packet to match the expected length.

When the Frame Cleaner IP creates extra pixel data the value used is always the same. The most significant bit of each color plane is set to '1' and all other bits are set to '0'. The IP uses this value as it displays as a similar light grey color in the RGB, YCbCr and monochrome color spaces.

Backpressure and Overflow

When the Frame Cleaner IP receives fields that have fewer pixels per line than expected it must add additional pixels at the end of the line packet. When it receives fields with fewer line packets than expected it must insert additional line packets into the stream. In both cases, to allow the extra pixels to be inserted into the stream, the IP must lower the `tready` signal on the input interface and create backpressure. If the disparity between the expected and received width or height is small, this additional backpressure may be absorbed by the blanking in the incoming video (if any exists). However, if the disparity is larger (or there is no blanking in the incoming video), the large periods of backpressure can force video sources that you can't stall (such as HDMI) to drop data (overflow).

The loss of data at the input is an undesirable (if unavoidable) consequence of padding fields to match and expected size. However, the additional backpressure might create a positive feedback loop that leads to continuous overflow. In this case, a shorter than expected input field results in padding and backpressure. The input continues to receive the next field, but the backpressure causes overflow, which typically causes the input IP to terminate the stream for the input field earlier than expected. The Frame Cleaner IP is then forced to apply padding to this field, which creates more backpressure and the cycle repeats.

To avoid hitting this overflow feedback condition, only use the Frame Cleaner IP with video sources that can either accept backpressure (and reduce the video rate), or that can detect (and signal) an overflow condition that the system controller can pause.

Maximum and minimum field size restrictions

When you write IPs for video processing you may want to restrict the maximum and or minimum field sizes that they can process to save resources and improve clock speed. For example, the IPs in the Altera Video and Vision Processing Suite can process fields as small as one pixel wide and or high. However, while the Altera Streaming Video Protocol allows for fields with widths and or heights up to 65536 pixels, the majority of the Altera Video and Vision Processing Suite IPs only support widths and heights up to 16384 pixels. The IPs do not hang if they receive fields that are too large, but the output field is not processed in the manner expected.

When using the Frame Cleaner IP with the Lite variant of the Altera Streaming Video Protocol, the field width and height are configured via the register map. As you control the values you write to the register map, you can enforce any minimum or maximum field size restrictions in software by limiting the values you write. The Frame Cleaner IP accepts the values in the register map and the output fields are always bound by the maximum and minimum values.

When using the Frame Cleaner with the Full variant of the Altera Streaming Video, the field dimensions are specified by the incoming image information packets. Typically, you may not have full control over the width and height values sent by upstream IPs, the image information packets in the stream may not adhere to any minimum or maximum frame size requirements for downstream IPs. For this reason, with full variant Frame Cleaner IPs, you can enforce user-set maximum and minimum width and height values. When you turn on **Custom resolution limits**, the IP edits any incoming image information packet with a height or width value that falls outside the specified range to the appropriate maximum or minimum value. The IP pads or crops accompanying field as appropriate to match the values in the outgoing image information packet.

Control Agent Interface

For lite variant Frame Cleaner IPs, you must turn on the Avalon memory-mapped control agent interface so you can set the expected field height and width values. For full variants of the Frame Cleaner, the IP takes these values from the incoming image information packets. The Frame Cleaner IP has no other runtime editable settings, so the control agent interface is not required with full variants (but may still be on if you turn on **Debug features** during parametrization).

The Frame Cleaner IP can count the number of error conditions encountered and report these values via the register map to assist with debugging. This feature is only on if you turn on **Debug features**.

28.4. Frame Cleaner IP Interfaces

Table 502. Frame Cleaner IP Interfaces

Signal name	Direction	Width	Description
Clocks and resets			
main_clock_clk	in	1	AXI4-S processing clock.
main_reset_rst	in	1	AXI4-S processing reset.
agent_clock_clk	in	1	Clock for the Avalon memory-mapped control agent interface. Only if you select Separate clock for control interface
agent_reset_rst	in	1	Reset for the Avalon memory-mapped control agent interface. Only if you select Separate clock for control interface
Control interfaces			
av_mm_control_agent_address	in	7	Avalon memory-mapped agent address
av_mm_control_agent_write	in	1	Avalon memory-mapped agent write
av_mm_control_agent_writedata	in	32	Avalon memory-mapped agent write data
av_mm_control_agent_byteenable	in	4	Avalon memory-mapped agent byte enable
av_mm_control_agent_read	in	1	Avalon memory-mapped agent read
av_mm_control_agent_readdata	out	32	Avalon memory-mapped agent read data
av_mm_control_agent_readdatavalid	out	1	Avalon memory-mapped agent read
av_mm_control_agent_waitrequest	out	1	Avalon memory-mapped agent wait request
Altera Streaming Video interfaces			
axi4s_vid_in_tdata	in	(90)	AXI4-S data in
axi4s_vid_in_tvalid	in	1	AXI4-S data valid
axi4s_vid_in_tuser	in		AXI4-S tuser tuser[0] indicates start of video frame when asserted
<i>continued...</i>			

(90) The equation gives all tdata widths in these interfaces:

$$\max(\text{ceil}((\text{bits per color sample} \times \text{number of color planes}) / 8) \times \text{pixels in parallel} \times 8, 16)$$

Signal name	Direction	Width	Description
			tuser[1] indicates the start of a non-video packet when asserted
axi4s_vid_in_tlast	in	1	AXI4-S end of packet
axi4s_vid_in_tready	out	1	AXI4-S data ready
axi4s_vid_out_tdata	out		AXI4-S data in
axi4s_vid_out_tvalid	out	1	AXI4-S data valid
axi4s_vid_out_tuser	out	⁽⁹¹⁾	AXI4-S tuser tuser[0] indicates start of video frame when asserted tuser[1] indicates the start of a non-video packet when asserted
axi4s_vid_out_tlast	out	1	AXI4-S end of packet
axi4s_vid_out_tready	in	1	AXI4-S data ready

Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

28.5. Frame Cleaner IP Registers

Each register is either read-only (RO) or read-write (RW).

Table 503. Frame Cleaner IP Control Registers

In the software API these register names appear with a prefix of INTEL_VVP, INTEL_VVP_CORE or INTEL_VVP_CLEANER as appropriate and with an optional REG suffix

Address	Register	Access	Description
Parameterization Registers			
0x0000	PROD_ID	RO	Read this register to retrieve the frame cleaner product ID. This register always returns 0x6AF7_023E.
0x0004	VERSION	RO	Read this register to retrieve the IP version information.
0x0008	LITE_MODE	RO	Read this register to determine if lite mode is on. This register returns 1 if lite mode is on and 0 if lite mode is off
0x000C	DEBUG_ENABLED	RO	Read this register to determine if you turn on Debug features . This register returns 1 if you turn on Debug features and 0 otherwise
0x0010	USE_CUSTOM_MIN_MAX	RO	Read this register to determine if the custom resolution guard bands are on. This register returns 1 if custom resolution guard bands are on, and 0 otherwise
0x0014	MIN_WIDTH	RO	Read this register to determine the minimum supported field width
0x0018	MAX_WIDTH	RO	Read this register to determine the maximum supported field width
<i>continued...</i>			

⁽⁹¹⁾ The equation gives all tuser widths in these interfaces

$$N = \text{ceil}(\text{tdata width} / 8)$$

Address	Register	Access	Description
0x001C	MIN_HEIGHT	RO	Read this register to determine the minimum supported field height
0x0020	MAX_HEIGHT	RO	Read this register to determine the maximum supported field height
0x0024 to 0x011F unused			

Table 504. Control and debugging registers

Refer to *Image Information Control Packets* in the *Altera Streaming Video Protocol Specification*, for more details of these registers

Address	Register	Access ⁽⁹²⁾		Description
		Lite	Full	
0x0120	IMG_INFO_WIDTH	RW	RO	When you turn on lite , use this register to set the expected width of incoming video fields. When you turn off lite and turn on Debug features , this register returns the width that the frame cleaner derives from information in the image information control packet.
0x0124	IMG_INFO_HEIGHT	RW	RO	When you turn on lite , use this register to set the expected height of incoming video fields. When you turn off lite and turn on Debug features , this register returns the height that the frame cleaner derives from information in the image information control packet.
0x0128	IMG_INFO_INTERLACE	-	RO	When you turn off lite and turn on Debug features , this register returns the interlace nibble that the frame cleaner derives from information in the image information control packet. Unused in lite mode.
0x012C	Reserved	-	-	Reserved
continued...				

⁽⁹²⁾ You must turn on **Debug features** to read the values stored in these registers. If **Debug features** is off, reads to these registers return undefined data. The only exception is the STATUS register, the value of which you can always read.

Address	Register	Access ⁽⁹²⁾		Description
		Lite	Full	
0x0130	IMG_INFO_COLORSPACE	-	RO	When you turn off lite and turn on Debug features , this register returns the color space that the frame cleaner derives from information in the image information control packet. Unused in lite mode.
0x0134	IMG_INFO_SUBSAMPLING	-	RO	When you turn off lite and turn on Debug features , this register returns the subsampling that the frame cleaner derives from information in the image information control packet. Unused in lite mode.
0x0138	IMG_INFO_COSITING	-	RO	When you turn off lite and turn on Debug features , this register returns the cositing that the frame cleaner derives from information in the image information control packet. Unused in lite mode.
0x013C	IMG_INFO_FIELD_COUNT	-	RO	When you turn off lite and turn on Debug features , this register returns the field count that the frame cleaner derives from information in the image information control packet. Unused in lite mode.
0x0138 to 0x013C unused				
0x0140	STATUS	RO		Bit 0 : Status bit. 1 = frame cleaner is processing a video field, 0 otherwise. For other bits, see Table 21
0x0144	FIELD_NUM	RO		When you turn off lite and turn on Debug features , reads to this register return the field number value

continued...

⁽⁹²⁾ You must turn on **Debug features** to read the values stored in these registers. If **Debug features** is off, reads to these registers return undefined data. The only exception is the STATUS register, the value of which you can always read.

Address	Register	Access ⁽⁹²⁾		Description
		Lite	Full	
				from the most recently received end of field control packet.
0x0148	FIELD_COUNT	RO		When you turn on Debug features , reads to this register return the total number of fields received since the debug counts were last reset.
0x014C	CLIPPED_FIELD_COUNT	RO		When you turn on Debug features , reads to this register return the number of fields that have been clipped to match their expected size since the debug counts were last reset.
0x0150	PADDED_FIELD_COUNT	RO		When you turn on Debug features , reads to this register return the number of fields that have been padded to match their expected size since the debug counts were last reset.
0x0154	PREV_FIELD_LINE_COUNT	RO		When you turn on Debug features , reads to this register return the number of lines in the most recently completed field.
0x0158	PREV_FIELD_LINE_WIDTH	RO		When you turn on Debug features , reads to this register return the number of pixels in the final line of the most recently completed field.
0x015C	DEBUG_RESET	RW		Write any value to this register to reset all debug counts to 0.

Register Bit Descriptions

Table 505. PROD_ID

Name	Bits	Description
Frame cleaner product ID	31:0	This register always returns 0x6AF7_023E.

⁽⁹²⁾ You must turn on **Debug features** to read the values stored in these registers. If **Debug features** is off, reads to these registers return undefined data. The only exception is the STATUS register, the value of which you can always read.

Table 506. VERSION

Name	Bits	Description
Register map version	7:0	Register map version.
IP patch revision	15:8	-
IP update revision	23:16	Updated when the IP version changes.
IP major revision	31:24	Updated when the IP version changes.

Table 507. LITE_MODE

Name	Bits	Description
Lite mode parameterization bit	31:0	Returns 1 if lite mode is on and 0 otherwise

Table 508. DEBUG_ENABLED

Name	Bits	Description
Debug features parameterization bit	31:0	Returns 1 if debug features is on and 0 otherwise

Table 509. USE_CUSTOM_MIN_MAX

Name	Bits	Description
Use custom min max	31:0	Returns 1 if custom minimum and maximum resolution checks are enabled

Table 510. MIN_WIDTH

Name	Bits	Description
Min width	31:0	Minimum output field width

Table 511. MAX_WIDTH

Name	Bits	Description
Max width	31:0	Maximum output field width

Table 512. MIN_HEIGHT

Name	Bits	Description
Min height	31:0	Minimum output field height

Table 513. MAX_HEIGHT

Name	Bits	Description
Max height	31:0	Maximum output field height

Table 514. IMG_INFO_WIDTH

Name	Bits	Description
Width bits	15:0	For lite parameterizations, write to this register to set the expected width of the incoming video fields. For full parameterizations with debug features enabled, this register returns the <code>width-1</code> field from the most recently received <code>image information packet</code> and adds 1 to return a value for <code>width</code> .
unused	31:16	Unused.

Table 515. IMG_INFO_HEIGHT

Name	Bits	Description
Height bits	15:0	For lite parameterizations, write to this register to set the expected height of the incoming video fields. For full parameterizations with debug features enabled, this register reads the <code>height-1</code> field from the most recently received <code>image information packet</code> and adds 1 to return a value for <code>height</code> .
unused	31:16	Unused.

Table 516. IMG_INFO_INTERLACE

Name	Bits	Description
IntlaceNibble bits	3:0	For lite parameterizations, this register has no function. For full parameterizations with debug features enabled, this register returns the <code>intlaceNibble</code> field from the most recently received <code>image information packet</code> .
unused	31:4	Unused.

Table 517. IMG_INFO_COLORSPACE

Name	Bits	Description
CSP code bits	6:0	For lite parameterizations, this register has no function. For full parameterizations with debug features enabled, this register returns the 7 bit <code>CSP</code> field from the most recently received <code>image information packet</code> .
unused	31:7	Unused.

Table 518. IMG_INFO_SUBSAMPLING

Name	Bits	Description
SubSa code bits	1:0	For lite parameterizations this register has no function. For full parameterizations with debug features enabled, this register returns the <code>SUBSA</code> field from the most recently received <code>image information packet</code> .
unused	31:2	Unused.

Table 519. IMG_INFO_COSITING

Name	Bits	Description
Cosite code bits	1:0	For lite parameterizations, this register has no function. For full parameterizations with debug features enabled, this register returns the <code>COSITE</code> field from the most recently received <code>image information packet</code> .
unused	31:2	Unused.

Table 520. IMG_INFO_FIELD_COUNT

Name	Bits	Description
Count bits	6:0	For lite parameterizations, this register has no function. For full parameterizations with debug features enabled, this register returns the 7 bit <code>FIELD_COUNT</code> field from the most recently received <code>image information packet</code> .
unused	31:7	Unused.

Table 521. STATUS

Name	Bits	Description
Status bit	0	1 = frame cleaner is processing a video field, 0 otherwise.
Field clipping bit	1	1 = current field is being clipped, 0 otherwise
Field padding bit	2	1 = current field is being padded, 0 otherwise
Previous field clipped	3	1 = previous field was clipped, 0 otherwise
Previous field padded	4	2 = previous field was padded, 0 otherwise
unused	31:5	Unused.

Table 522. FIELD_NUM

Name	Bits	Description
Field num	31:0	Field number value taken from the most recently received eof control packet

Table 523. FIELD_COUNT

Name	Bits	Description
Field count	31:0	Number of fields received since last debug reset

Table 524. CLIPPED_FIELD_COUNT

Name	Bits	Description
Clipped field count	31:0	Number of fields clipped since last debug reset

Table 525. PADDED_FIELD_COUNT

Name	Bits	Description
Padded field count	31:0	Number of fields padded since last debug reset

Table 526. PREV_FIELD_LINE_COUNT

Name	Bits	Description
Previous field line count	31:0	Number of lines in last completed field

Table 527. PREV_FIELD_LINE_WIDTH

Name	Bits	Description
Previous field line width	31:0	Number of pixels in the final line of the previous field

Table 528. DEBUG_RESET

Name	Bits	Description
unused	31:0	Unused.

28.6. Frame Cleaner IP Software API

The IP has a software driver for software control of the IP at run time. The IP does not fit any of the generic device models provided by the Nios II HAL. It exposes a set of dedicated accessors to the control and status registers. The driver structure inherits the base driver structure so all common methods described in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_cleaner_regs.h`

Include file: `intel_vvp_cleaner.h`

Table 529. Frame Cleaner IP Software API

Name	Description
<code>intel_vvp_cleaner_init</code>	Initialize the Frame Cleaner instance
<code>Intel_vvp_core_*</code>	Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i>
<code>intel_vvp_cleaner_get_lite_mode</code>	Returns if lite mode is on.
<code>intel_vvp_cleaner_get_debug_enabled</code>	Returns if debug features is on.
<code>intel_vvp_cleaner_uses_custom_min_max</code>	Returns whether the IP sets its own limit on the dimensions of field.
<code>intel_vvp_cleaner_get_min_width</code>	Returns the minimum width
<code>intel_vvp_cleaner_get_max_width</code>	Returns the maximum width
<code>intel_vvp_cleaner_get_min_height</code>	Returns the minimum height
<code>intel_vvp_cleaner_get_max_height</code>	Returns the maximum height
<code>intel_vvp_cleaner_is_running</code>	Returns if the IP is processing data
<code>intel_vvp_cleaner_is_clipping</code>	Returns if the IP is clipping lines
<code>intel_vvp_cleaner_is_padding</code>	Returns if the IP is padding lines
<code>intel_vvp_cleaner_previous_field_clipped</code>	Returns if the previous field is clipped
<code>intel_vvp_cleaner_previous_field_padded</code>	Returns is the previous field was padded
<code>intel_vvp_cleaner_get_status</code>	Reads the status register
<code>intel_vvp_cleaner_get_field_num</code>	Returns the last image info field number in Full mode
<code>intel_vvp_cleaner_get_field_count</code>	Returns the current field counter value
<code>intel_vvp_cleaner_get_clipped_field_count</code>	Returns the number of fields clipped
<code>intel_vvp_cleaner_get_padded_field_count</code>	Returns the number of fields padded
<code>intel_vvp_cleaner_get_previous_field_line_count</code>	Returns the line counter value
<code>intel_vvp_cleaner_get_line_width</code>	Returns the width counter value
<code>intel_vvp_cleaner_reset_debug_counters</code>	Resets all debug counters to zero

intel_vvp_cleaner_init

Prototype

```
int intel_vvp_cleaner_init(intel_vvp_cleaner_instance
*instance, intel_vvp_core_base base);
```

Description

Initializes a frame cleaner instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

Arguments

instance – pointer to the intel_vvp_cleaner_instance software driver instance structure

base – base address of the register map

Returns

kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product_id does not match with the Frame Cleaner product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpCleanerRegMapVersionErr if the register map is not supported

intel_vvp_cleaner_get_lite_mode

Prototype

```
bool
intel_vvp_cleaner_get_lite_mode(intel_vvp_cleaner_instance
*instance);
```

Description

Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_cleaner_instance fully initialized

Arguments

instance – pointer to the intel_vvp_cleaner_instance software driver instance structure

Returns

true if the IP is parameterized in lite mode

intel_vvp_cleaner_get_debug_enabled

Prototype

```
bool
intel_vvp_cleaner_get_debug_enabled(intel_vvp_cleaner_instance
*instance);
```

Description Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_cleaner_instance fully initialized.

Arguments instance – pointer to the intel_vvp_cleaner_instance software driver instance structure

Returns true if the IP is parameterized with debug features enabled

Intel_vvp_cleaner_uses_custom_min_max

Prototype

```
bool Intel_vvp_
cleaner_uses_custom_min_max(intel_vvp_cleaner_instance
*instance);
```

Description Returns the value of the CUSTOM_MIN_MAX register. The instance must be a valid intel_vvp_cleaner_instance fully initialized.

Arguments instance – pointer to the intel_vvp_cleaner_instance software driver instance structure

Returns True if the frame cleaner is configured with user-specified minimum and maximum dimension values

intel_vvp_cleaner_get_min_width

Prototype

```
uint32_t
intel_vvp_cleaner_get_min_width(intel_vvp_cleaner_instance
*instance);
```

Description Returns the value of the MIN_WIDTH register. The instance must be a valid intel_vvp_cleaner_instance fully initialized.

Arguments instance – pointer to the intel_vvp_cleaner_instance software driver instance structure

Returns Minimum allowed width before lines are padded with extra pixels

intel_vvp_cleaner_get_max_width

Prototype

```
uint32_t
intel_vvp_cleaner_get_max_width(intel_vvp_cleaner_instance
*instance);
```

Description Returns the value of the MAX_WIDTH register. The instance must be a valid intel_vvp_cleaner_instance fully initialized.

Arguments instance – pointer to the intel_vvp_cleaner_instance software driver instance structure

Returns Maximum allowed width before extra pixels in a line are truncated

intel_vvp_cleaner_get_min_height

Prototype

```
uint32_t
intel_vvp_cleaner_get_min_height(intel_vvp_cleaner_instance
*instance);
```

Description Returns the value of the MIN_HEIGHT register. The instance must be a valid intel_vvp_cleaner_instance fully initialized.

Arguments instance – pointer to the intel_vvp_cleaner_instance software driver instance structure

Returns Minimum supported height before frames are padded with extra lines

intel_vvp_cleaner_get_max_height

Prototype

```
uint32_t
intel_vvp_cleaner_get_max_height(intel_vvp_cleaner_instance
*instance);
```

Description Returns the value of the MAX_HEIGHT register. The instance must be a valid intel_vvp_cleaner_instance fully initialized.

Arguments instance – pointer to the intel_vvp_cleaner_instance software driver instance structure

Returns Maximum supported height before extra lines are discarded

intel_vvp_cleaner_is_running

Prototype

```
bool intel_vvp_cleaner_is_running(intel_vvp_cleaner_instance*
instance);
```

Description Reads and returns the running bit of the STATUS register. The instance must be a valid intel_vvp_cleaner_instance fully initialized.

Arguments instance – pointer to the intel_vvp_cleaner_instance software driver instance structure

Returns True is the IP is currently generating an output field, false when between two fields

intel_vvp_cleaner_is_clipping

Prototype

```
bool intel_vvp_cleaner_is_clipping(intel_vvp_cleaner_instance*
instance);
```

Description Reads and returns the clipping bit of the STATUS register. The instance must be a valid intel_vvp_cleaner_instance fully initialized.

- Arguments* instance – pointer to the intel_vvp_cleaner_instance software driver instance structure
- Returns* True is the IP is currently discarding extra lines from the current field to keep at the required height

intel_vvp_cleaner_is_padding

Prototype

```
bool intel_vvp_cleaner_is_padding(intel_vvp_cleaner_instance* instance);
```

Description Reads and returns the padding bit of the STATUS register. The instance must be a valid intel_vvp_cleaner_instance fully initialized.

Arguments instance – pointer to the intel_vvp_cleaner_instance software driver instance structure

Returns True is the IP is currently adding extra lines to the current field to meet the required height

intel_vvp_cleaner_previous_field_clipped

Prototype

```
bool intel_vvp_cleaner_is_previous_field_clipped(intel_vvp_cleaner_instance* instance);
```

Description Reads and returns the previous field clipped bit of the STATUS register. The instance must be a valid intel_vvp_cleaner_instance fully initialized.

Arguments instance – pointer to the intel_vvp_cleaner_instance software driver instance structure

Returns True is the IP discarded extra lines from the previous field

intel_vvp_cleaner_previous_field_padded

Prototype

```
bool intel_vvp_cleaner_previous_field_padded(intel_vvp_cleaner_instance* instance);
```

Description Reads and returns the previous field padded bit of the STATUS register. The instance must be a valid intel_vvp_cleaner_instance fully initialized.

Arguments instance – pointer to the intel_vvp_cleaner_instance software driver instance structure

Returns True is the IP added extra lines to the previous field

intel_vvp_cleaner_get_status

Prototype

```
uint8_t
intel_vvp_cleaner_get_status(intel_vvp_cleaner_instance*
instance);
```

Description Reads and returns the STATUS register. The instance must be a valid intel_vvp_cleaner_instance fully initialized.

Arguments instance – pointer to the intel_vvp_cleaner_instance software driver instance structure

Returns The value read from the status register

intel_vvp_cleaner_get_field_num

Prototype

```
uint16_t
intel_vvp_cleaner_get_field_num(intel_vvp_cleaner_instance
*instance)
```

Description Reads and returns the FIELD_NUM register. The instance must be a valid intel_vvp_cleaner_instance fully initialized and parameterized in Full mode with debug enabled.

Arguments instance – pointer to the intel_vvp_cleaner_instance software driver instance structure

Returns The field_num value read from the previous image info packet

intel_vvp_cleaner_get_field_count

Prototype

```
uint32_t
intel_vvp_cleaner_get_field_count(intel_vvp_cleaner_instance
*instance)
```

Description Reads and returns the FIELD_COUNT register. The instance must be a valid intel_vvp_cleaner_instance fully initialized and parameterized with debug enabled.

Arguments instance – pointer to the intel_vvp_cleaner_instance software driver instance structure

Returns The current value of the field counter

intel_vvp_cleaner_get_clipped_field_count

Prototype

```
uint32_t
intel_vvp_cleaner_get_clipped_field_count(intel_vvp_cleaner_instance
*instance)
```


Description Reads and returns the CLIPPED_FIELD_COUNT register. The instance must be a valid intel_vvp_cleaner_instance fully initialized and parameterized with debug enabled.

Arguments instance – pointer to the intel_vvp_cleaner_instance software driver instance structure

Returns The current value of the clipped field counter

intel_vvp_cleaner_get_padded_field_count

Prototype

```
uint32_t  
intel_vvp_cleaner_get_padded_field_count(intel_vvp_cleaner_instance  
*instance)
```

Description Reads and returns the PADDED_FIELD_COUNT register. The instance must be a valid intel_vvp_cleaner_instance fully initialized and parameterized with debug enabled.

Arguments instance – pointer to the intel_vvp_cleaner_instance software driver instance structure

Returns The current value of the padded field counter

intel_vvp_cleaner_get_previous_field_line_count

Prototype

```
uint32_t  
intel_vvp_cleaner_get_previous_field_line_count(intel_vvp_cleaner_instance  
*instance)
```

Description Reads and returns the PREV_FIELD_LINE_COUNT register. The instance must be a valid intel_vvp_cleaner_instance fully initialized and parameterized with debug enabled.

Arguments instance – pointer to the intel_vvp_cleaner_instance software driver instance structure

Returns The number of lines in the previous field

intel_vvp_cleaner_get_line_width

Prototype

```
uint32_t  
intel_vvp_cleaner_get_line_width(intel_vvp_cleaner_instance  
*instance)
```

Description Reads and returns the LINE_WIDTH register. The instance must be a valid intel_vvp_cleaner_instance fully initialized and parameterized with debug enabled.

- Arguments* instance – pointer to the intel_vvp_cleaner_instance software driver instance structure
- Returns* The width of the last invalid line, either shorter or longer than expected, in the previous field. If all lines were the correct length, the register returns the actual field width.

intel_vvp_cleaner_reset_debug_counters

Prototype

```
uint32_t
intel_vvp_cleaner_reset_debug_counters(intel_vvp_cleaner_instance
*instance)
```

- Description* Resets all debug counters to zero. The instance must be a valid intel_vvp_cleaner_instance fully initialized and parameterized with debug enabled.
- Arguments* instance – pointer to the intel_vvp_cleaner_instance software driver instance structure
- Returns* kIntelVvpCoreOk in case of success.
kIntelVvpCoreInstanceErr if the call is not appropriate for the given instance

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38



29. Full-Raster to Clocked Video Converter IP

29.1. About the Full-Raster to Clocked Video Converter

The IP remaps pixel data and video timing information from streaming full-raster video to clocked video format. The remapping does not change the pixel data and video timing information.

The IP converts pixel data in the full-raster video format to pixel data in the clocked video format. The pixel data is HSync, Hblank, VSync, Vblank, field, data enable, and video data.

The IP connects between IPs that support only full-raster interfaces and IPs that only support clocked video interfaces.

29.1.1. Full-Raster to Clocked Video Converter Features

- Streaming full-raster data input interface for video IOs
- Clocked video data output interface for video IOs
- CPU control interface
- Video standard agnostic
- Support for Altera legacy clocked video input and clocked video output legacy interfaces
- 6 to 16 bits per color component
- 1, 2, 4, or 8 pixels in parallel per clock
- 1, 2, 3, or 4 color planes per pixel
- Latency of 2 clock cycles
- Small FPGA resource footprint

29.1.2. Full-Raster to Clocked Video Converter Performance and Resource Utilization

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family and the number of supported bits per color sample, pixels in parallel, and number of color planes.

Table 530. Full-Raster to Clocked Video Converter Performance and Resource Utilization

The table shows ALM usage and f_{MAX} for a design with 2 pixels in parallel, 10 bits per color sample and 3 color planes.

Target Device	ALMs	Video clock f_{MAX} (MHz)
Agilex 7 AGFA012R24A2E2V	160	1000
Arria 10 10AS066H1F34E1HG	126	645
Cyclone 10 GX 10CX220YF672E5G	126	645
Stratix 10 1SX280LN2F43E1VG	160	815

29.2. Full Raster to Clocked Video Converter Parameters

The IP offers compile-time parameters.

Table 531. Main Parameters

Parameter	Values	Description
Video Data Format		
CV Bus Style	Lite, CVI or CVO	Select which sideband signals the IP drives, and which signals are available in Platform Designer. A second tab allows configuration of the CV bus style .
Bits per color sample	6 to 16	Number of bits per color sample
Number of color planes	1 to 4	Number of color planes in a pixel
Number of pixels in parallel	1, 2, 4 or 8	Number of pixels transmitted every clock
AXI4S FR Bus has tReady connection	True or false	Select True so the full raster interface contains the AXI4-S tReady signal. Select False to remove the tReady signal.
Control Settings⁽⁹³⁾		
Memory-mapped control interface	True or false	Select True , to enable the CPU interface and associated signals. When False , the IP removes the CPU interface, and all CPU registers use default values.
Separate clock for control interface	True or false	Select True to include the signal <code>cpu_clock</code> to Platform Designer. You can assume it to be asynchronous to the video domain. When False , the CPU interface uses the signal <code>vid_clock</code> .

⁽⁹³⁾ These parameters are only available when you select **CVI** or **CVO** for **CV Bus Style**

Figure 88. Main Parameters

The screenshot shows the configuration window for the VVP-FR to Clocked Video Converter Intel FPGA IP. The title bar reads "VVP-FR to Clocked Video Converter Intel FPGA IP" and "intel_wp_fr2cv". There is a "Details" button in the top right. The "Main" tab is selected, and the "CVO Parameters" sub-tab is active. Under the "Video data format" section, the following parameters are set: CV Bus Style to "CVO", Bits per color sample to "8", Number of color planes to "2", Number of pixels in parallel to "1", and AXI4-S FR Bus has tReady connection to "True". Under the "Control settings" section, Memory mapped control interface is set to "False" and Separate clock for control interface is set to "False".

Table 532. Lite Parameters

Parameters	Values	Description
Parameters		
Valid signal	On or off	When you select Lite the IP includes an optional data valid output signal, <code>cv_vid_out_valid</code> . Turn on to turn on this output in the IP. The Platform Designer GUI includes or removes the signal from the <code>cv_vid_out</code> conduit as appropriate.
Ready signal	On or off	When you select Lite , the IP includes an optional data ready input signal, <code>cv_vid_out_ready</code> . Turn on to turn on this input in the IP. Platform Designer includes or removes the signal from the <code>cv_vid_out</code> conduit as appropriate.
Timing signals	sync, blank, or both	Select which timing signals (blank timing or sync timing) are available. Platform Designer includes or removes the signals from the <code>cv_vid_out</code> conduit as appropriate

Figure 89. Lite Parameters

The screenshot shows the configuration window for the VVP-FR to Clocked Video Converter Intel FPGA IP, specifically the "CV Lite Parameters" tab. The title bar reads "VVP-FR to Clocked Video Converter Intel FPGA IP" and "intel_wp_fr2cv". There is a "Details" button in the top right. The "Main" tab is selected, and the "CV Lite Parameters" sub-tab is active. Under the "Parameters" section, the following parameters are set: Valid signal to "Off", Ready signal to "Off", and Timing Signals to "sync".

Table 533. CVI Parameters

Parameter	Allowed Range	Description
CVI Core Parameters		
Export the total resolution	No Export or Add signals for export	The dimensions of the full raster (active and blanking) can be output from the IP as 16-bit signals. Platform Designer includes or removes the signals from the <code>cv_vid_out</code> conduit as appropriate
CVI Legacy Tie Offs		
Include the vid_hd_sdn signal	True or False	Select True to add this signal to the <code>cv_vid_out</code> conduit. The signal has no function within the IP and is included for connectivity within Platform Designer.
Include the vid_std signal	True or False	Select True to add this signal to the <code>cv_vid_out</code> conduit. The signal has no function within the IP and is included for connectivity within Platform Designer.
Width of vid_std	1 to 16	The width of the <code>vid_std</code> signal.
Include the hdmi_duplication signal	True or False	Select True to add this signal to the <code>cv_vid_out</code> conduit. The signal has no function within the IP and is included for connectivity within Platform Designer.

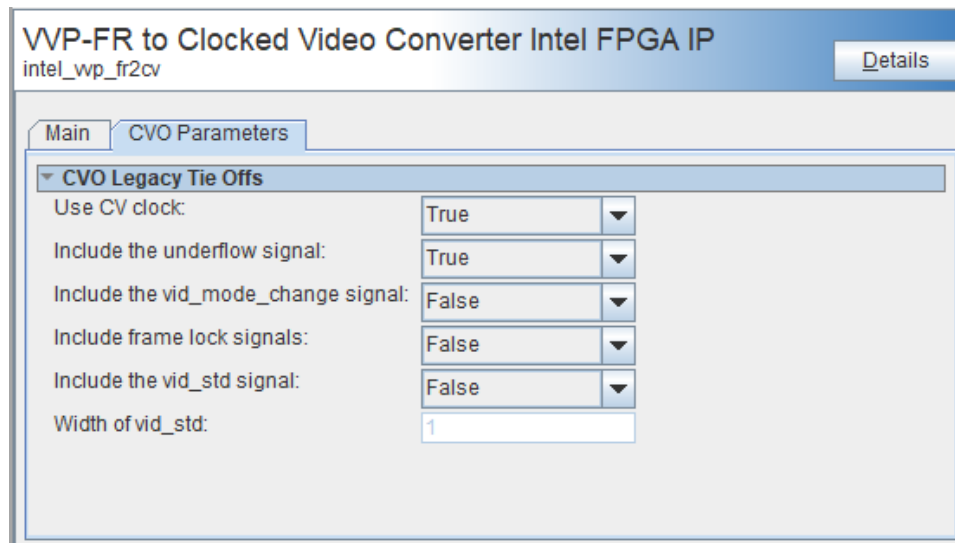
Figure 90. CVI Parameters

Table 534. CVO Parameters

Parameters	Allowed Range	Description
CVO Legacy Tie Offs		
Use CV Clock	True or False	Select True to add this signal to the <code>cv_vid_out</code> conduit. The clock signal is an input on the <code>cv_vid_out</code> conduit. If you select True , the IP uses it as the internal video clock.
Include the underflow signal	True or False	Select True to add this signal to the <code>cv_vid_out</code> conduit. The signal has no function within the IP and is included for connectivity within Platform Designer.
<i>continued...</i>		

Parameters	Allowed Range	Description
Include the <code>vid_mode_change</code> signal	True or False	Select True to add this signal to the <code>cv_vid_out</code> conduit. The signal has no function within the IP and is included for connectivity within Platform Designer.
Include the frame lock signals	True or False	Select True to add this signal to the <code>cv_vid_out</code> conduit. The signal has no function within the IP and is included for connectivity within Platform Designer.
Include the <code>vid_std</code> signal	True or False	Select True to add this signal to the <code>cv_vid_out</code> conduit. The signal has no function within the IP and is included for connectivity within Platform Designer.
Width of <code>vid_std</code>	1 to 16	The width of the <code>vid_std</code> signal.

Figure 91. CVO Parameters



VWP-FR to Clocked Video Converter Intel FPGA IP
intel_wp_fr2cv

Details

Main CVO Parameters

▼ CVO Legacy Tie Offs

Use CV clock: True

Include the underflow signal: True

Include the `vid_mode_change` signal: False

Include frame lock signals: False

Include the `vid_std` signal: False

Width of `vid_std`: 1

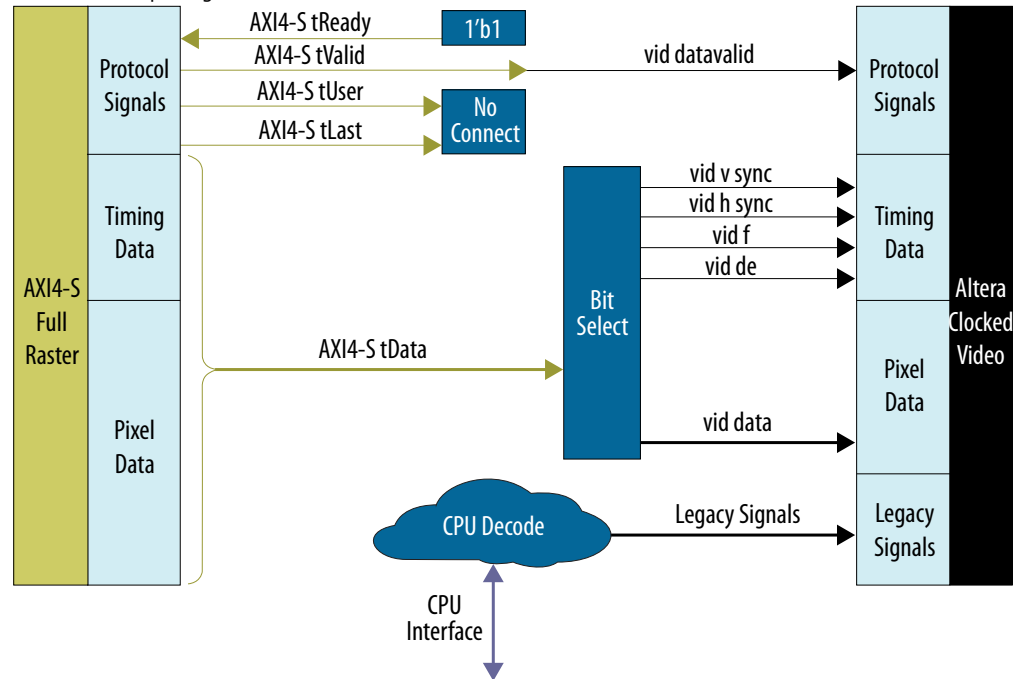
29.3. Full-Raster to Clocked Video Converter Block Description

The IP passes the pixel and timing data through unmodified. The streaming full-raster bus encapsulates all the pixel and timing data on a single bus, `tData`. The clocked video bus is a bundle of multiple single wires for the individual video timing strobes, and a data bus for the pixel data.

The clocked video bus can contain additional side-band signals, such as discrete 16-bit signals for the width and height of the raster. The IP ignores these side-band signals. The IP copies some signals from CPU registers. The sideband signals provide backward IO interface compatibility between this IP and legacy Altera clocked video input and clocked video output interfaces.

Figure 92. High-level mapping from streaming full-raster to clocked video format

The figure shows how mapping separates the discrete signals used by the clocked video protocol from the single AXI4-S tData bus. You need a processor interface to construct the legacy Altera clocked video input and clocked video output signals.



29.3.1. Full Raster to Clocked Video Converter Interfaces

The IP has three functional interfaces:

- Full raster data input interface for video IOs
- Altera clocked video data output interface for video IOs
- Avalon memory-mapped CPU interface

Table 535. Clocks and Resets

Name	Direction	Width	Description
vid_clock	Input	1	When you select Lite and CVI , this is the video clock for the lite and clocked video outputs, and the streaming full raster input. When you select CVO , the clocked video outputs bus can contain an optional input clock. If you turn on the clocked video output input clock, this signal is a dummy signal retained for Platform Designer connectivity. The IP uses the video clock included in the cv_vid_out conduit.
vid_reset	Input	1	Reset for vid_clock domain.
cpu_clock	Input	1	Optional control interface clock.
cpu_reset	Input	1	Optional control interface reset.
cv_clk_out	Output	1	A copy of the video clock that the IP uses.

Table 536. Control Interface

This interface is only available if you select **True** for **Memory-mapped control interface**.

Name	Direction	Width	Description
av_mm_cpu_agent_address	Input	7	Control agent port Avalon memory-mapped address bus. Specifies a word offset into the slave address space.
av_mm_cpu_agent_read	Input	1	Control agent port Avalon memory-mapped read signal. When you assert this signal, the control port drives new data onto the read data bus.
av_mm_cpu_agent_readdata	Output	32	Control agent port Avalon memory-mapped read data bus. These output lines are used for read transfers.
av_mm_cpu_agent_waitrequest	Output	1	Control agent port Avalon memory-mapped wait request bus. This signal indicates that the slave is stalling the host transaction.
av_mm_cpu_agent_write	Input	1	Control agent port Avalon memory-mapped write signal. When you assert this signal, the control port accepts new data from the write data bus.
av_mm_cpu_agent_writedata	Input	32	Control agent port Avalon memory-mapped write data bus. These input lines are used for writing transfers.
av_mm_cpu_agent_byteenable	Input	4	Control agent port Avalon memory-mapped byte enable bus. These lines indicate which bytes are selected for write and read transactions.

Table 537. Altera streaming full raster video interface

Name	Direction	Width	Description
axi4s_fr_vid_in_tvalid	Input	1	AXI4-S full raster data valid.
axi4s_fr_vid_in_tready	Output	1	Optional AXI4-S full raster data ready.
axi4s_fr_vid_in_tdata	Input	(94)	AXI4-S full raster data in.
axi4s_fr_vid_in_tlast	Input	1	AXI4-S end of full raster packet.
axi4s_fr_vid_in_tuser[0]	Input	(95)	AXI4-S start of full raster video frame.

Table 538. CV Lite Streaming Video Interface

This interface is only available if you select **Lite** for **CV Bus Style**.

Name	Direction	Width	Description
cv_vid_out_h	Output	Pixels in parallel	When asserted, the video is in a horizontal blanking.
cv_vid_out_v	Output	Pixels in parallel	When asserted, the video is in a vertical blanking.
cv_vid_out_h_sync	Output	Pixels in parallel	When asserted, the video is in a horizontal synchronization period.
cv_vid_out_v_sync	Output	Pixels in parallel	When asserted, the video is in a vertical synchronization period.
<i>continued...</i>			

(94) The equation gives the sizes of all tdata widths in these interfaces;

$$\max (\text{floor}(((\text{bits per color sample} \times (\text{number of color planes} + 1) \times \text{pixels in parallel}) + 7) / 8) \times 8, 16)$$

(95) The equation gives the sizes of all tuser widths in these interfaces

$$N = \text{ceil} (\text{tdata width} / 8)$$

Name	Direction	Width	Description
cv_vid_out_f	Output	Pixels in parallel	When asserted, the video is interlaced and in field 1. When deasserted, the video is either progressive or interlaced and in field 0.
cv_vid_out_active	Output	Pixels in parallel	When asserted, the video is in an active picture period (not horizontal or vertical blanking). You must drive this signal for correct operation of the IPs.
cv_vid_out_data	Output	(96)	Pixel data.
cv_vid_out_valid	Output	1	When asserted, the output is valid.
cv_vid_out_ready	Input	1	When asserted, the IP can drive new data. When deasserted, the IP can drive no new data.

Table 539. Clocked Video Input Streaming Video Interface

This interface is only available if you select **CVI** for **CV Bus Style**.

Name	Direction	Width	Description
cv_vid_out_vid_clk	Input	1	The pixel synchronous clock.
cv_vid_out_vid_h_sync	Output	Pixels in parallel	When asserted, the video is in a horizontal blanking or synchronization period.
cv_vid_out_vid_v_sync	Output	Pixels in parallel	When asserted, the video is in a vertical blanking or synchronization period.
cv_vid_out_vid_f	Output	Pixels in parallel	When asserted, the video is interlaced and in field 1. When deasserted, the video is either progressive or interlaced and in field 0.
cv_vid_out_vid_data	Output	(97)	Pixel data.
cv_vid_out_vid_de	Output	Pixels in parallel	When asserted, the video is in an active picture period (not horizontal or vertical blanking).
cv_vid_out_vid_datavalid	Output	1	When asserted, the output is valid.
cv_vid_out_vid_locked	Output	1	Unused legacy signal.
cv_vid_out_vid_hd_sdn	Output	1	Unused legacy signal.
cv_vid_out_vid_std	Output	User Specified	Unused legacy signal.
cv_vid_out_vid_color_encoding	Output	8	Unused legacy signal.
cv_vid_out_vid_bit_width	Output	8	Unused legacy signal.
cv_vid_out_vid_total_sample_width	Output	16	Indicates the total (active + blanking) width of the raster.
cv_vid_out_vid_total_line_count	Output	16	Indicates the total (active + blanking) height of the raster.
cv_vid_out_vid_hdmi_duplication	Output	4	Unused legacy signal.
continued...			

(96) The equation gives the data width size:
width = (bits per color sample X number of color planes X pixels in parallel)

(97) The equation gives the data width size:
width = (bits per color sample X number of color planes X pixels in parallel)

Name	Direction	Width	Description
cv_vid_out_sof	Input	1	Unused legacy signal.
cv_vid_out_sof_locked	Input	1	Unused legacy signal.
cv_vid_out_refclk_div	Input	1	Unused legacy signal.
cv_vid_out_clipping	Input	1	Unused legacy signal.
cv_vid_out_padding	Input	1	Unused legacy signal.
cv_vid_out_overflow	Input	1	Unused legacy signal.

Table 540. Clocked Video Output Streaming Video Interface

This interface is only available if you select **CVO** for **CV Bus Style**.

Name	Direction	Width	Description
cv_vid_out_vid_clk	Input	1	Pixel synchronous clock.
cv_vid_out_vid_h	Output	Pixels in parallel	When asserted, the video is in a horizontal blanking.
cv_vid_out_vid_v	Output	Pixels in parallel	When asserted, the video is in a vertical blanking.
cv_vid_out_vid_h_sync	Output	Pixels in parallel	When asserted, the video is in a horizontal synchronization period.
cv_vid_out_vid_v_sync	Output	Pixels in parallel	When asserted, the video is in a vertical synchronization period.
cv_vid_out_vid_f	Output	Pixels in parallel	When asserted, the video is interlaced and in field 1. When deasserted, the video is either progressive or interlaced and in field 0.
cv_vid_out_vid_data	Output	(98)	Pixel data.
cv_vid_out_vid_datavalid	Output	Pixels in parallel	When asserted, the output is valid.
cv_vid_out_underflow	Output	1	Unused legacy signal.
cv_vid_out_vid_mode_change	Output	1	Unused legacy signal.
cv_vid_out_vid_vcclk_div	Output	1	Unused legacy signal.
cv_vid_out_vid_sof_locked	Output	1	Unused legacy signal.
cv_vid_out_vid_sof	Output	1	Unused legacy signal.
cv_vid_out_vid_std	Output	6	Unused legacy signal.

Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

29.4. Full-Raster to Clocked Video Converter Registers

The IP allows runtime configuration of parameters using the Avalon memory-mapped CPU register interface. Unless stated, all registers are 32-bit wide.

(98) The equation gives the data width sizes:
width = (bits per color sample X number of color planes X pixels in parallel)

Table 541. Processor Register Description

Register	Offset	Access	Description
CVI specific registers			
Reg_CVI_Legacy_0	0x140	RW	Drives legacy CVI conduit output signals and returns the current values.
Reg_CVI_Legacy_1	0x144	RW	Drives legacy CVI conduit output signals and returns the current values.
Reg_CVI_Legacy_2	0x148	RW	Drives legacy CVI conduit output signals and returns the current values.
CVO specific registers			
Reg_CVO_Legacy_0	0x14C	RW	The current value of the CVO conduit side-band signals vid_sof.

Table 542. Reg_CVI_Legacy_0

Name	Bits	Attribute	Description
CVI SOF	0	RO	The current value of the CVI legacy signal sof.
CVI SOF Locked	1	RO	The current value of the CVI legacy signal sof_locked.
CVI Overflow	2	RO	The current value of the CVI legacy signal overflow.
CVI Clipping	3	RO	The current value of the CVI legacy signal clipping.
CVI Padding	4	RO	The current value of the CVI legacy signal padding.
CVI refclk_div	5	RO	The current value of the CVI legacy signal refclk_div.
Reserved	7:6	-	Reserved.
CVI video locked	8	RW	Drives legacy CVI conduit signal vid_locked.
Reserved	15:9	-	Reserved.
CVI color encoding	23:16	RW	Drives legacy CVI conduit signal vid_color_encoding.
CVI bit width	31:24	RW	Drives legacy CVI conduit signal vid_bit_width.

Table 543. Reg_CVI_Legacy_1

Name	Bits	Attribute	Description
CVI vid std	Width of vid_std -1:0	RW	Drives legacy CVI conduit signal vid_std.
CVI HDMI duplication	19:16	RW	Drives legacy CVI conduit signal vid_hdmi_duplication.
Reserved	23:20	-	Reserved.
CVI HD not SD	24	RW	Drives legacy CVI conduit signal vid_hd_sdn.
Reserved	31:25	-	Reserved.

Table 544. Reg_CVI_Legacy_2

Name	Bits	Attribute	Description
Total Pixels	15:0	RW	Drives legacy CVI conduit signal total_sample_count.
Total Lines	31:16	RW	Drives legacy CVI conduit signal total_line_count.

Table 545. Reg_CVO_Legacy_0

Name	Bits	Attribute	Description
CVO SOF	0	RW	Drives legacy clocked video output conduit signal <code>vid_sof</code> .
CVO SOF Locked	1	RW	Drives legacy clocked video output conduit signal <code>vid_sof_locked</code> .
CVO Underflow	2	RW	Drives legacy clocked video output conduit signal <code>underflow</code> .
CVO vco clock divide	3	RW	Drives legacy clocked video output conduit signal <code>vid_vcoclck_div</code> .
CVO mode change	4	RW	Drives legacy clocked video output conduit signal <code>vid_mode_change</code> .
Reserved	15:5	-	Reserved.
CVO video standard	Width of vid_std +15:16	RW	Drives legacy clocked video output conduit signal <code>vid_std</code> .



30. Full-Raster to Streaming Converter IP

30.1. About the Full-Raster to Streaming Converter IP

The IP provides a seamless conversion between streaming full-raster and lite video formats. The IP removes video timing data from a full-raster bus and leaves just the pixel data in Altera Streaming Video format. Typically, the IP is a bridge to transfer data from video connectivity to video processing IP.

The IP only supports lite variants. For more information about full and lite variants, refer to the *Altera Streaming Video Protocol Specification*

Related Information

[Altera Streaming Video Protocol Specification](#)

30.1.1. Full-Raster to Streaming Converter Features

- Full raster compliant video input interface
- Altera Streaming Video compliant output interface (lite variant support only)
- Optional TREADY conduit on the full raster interface
- Clock domain crossing between video pixel clock and video processing clock
- 1K to 32K depth video line buffer
- Small FPGA resource footprint
- 8-bit to 16-bit per color component
- 1, 2, 3, and 4 color planes per pixel
- 1, 2, 4, and 8 pixels in parallel

30.1.2. Full-Raster to Streaming Converter Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family, number of supported bits per sample, pixels in parallel, number of color planes and maximum depth of video line buffer.

Table 546. Full-Raster to Streaming Converter Performance and Resource Utilization

The table shows ALM usage and f_{MAX} for a design with 2 pixels in parallel, 10 bits per color sample and 3 color planes, and 1,024 output FIFO depth.

Target Device	ALMs	M20Ks	f_{MAX} MHz
Agilex 7 (AGFA012R24A2E2V)	412	4	781.86
Arria 10 (10AS066H1F34E1HG)	363	4	645.16
Cyclone 10 GX (10CX220YF672E5G)	362	4	599.88
Stratix 10 (1SX280LN2F43E1VG)	425	4	787.40

30.2. Full-Raster to Streaming Converter Parameters

The IP offers compile-time parameters.

Table 547. Full Raster to Streaming Converter Parameters

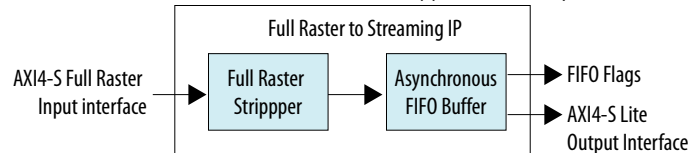
Parameter	Values	Description
Video Data Format		
Number of pixels in parallel	1, 2, 4, 8	Select the number of pixels in parallel.
Number of color planes	1 to 4	Select the number of color planes per pixel.
Bits per color sample	8 to 16	Select the number of bits per color sample.
Settings		
AXI4-S FR Bus has tReady connection	0 to 1	The switch to enable or disable tReady conduit as part of the FR interface.
Output FIFO Depth	1024, 2048, 4096, 8192, 16384, 32768	Select the maximum depth on the output FIFO buffer.

30.3. Full-Raster to Streaming Converter Block Description

The IP accepts full-raster video input format as a full-raster interface of pixel data and timing markers embedded in the `TDATA` bus of an AXI4-S interface. Also, the IP filters out the timing markers from the `TDATA` bus and provides a video-active only data format on the output interfaces that conforms to the Altera Streaming Video Protocol Specification.

Figure 93. Full-Raster to Streaming Converter high-level block diagram.

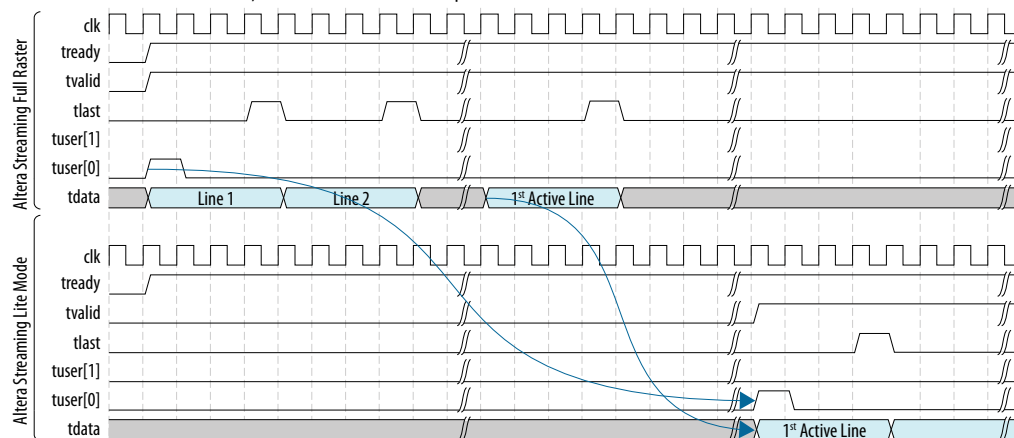
The figure shows the IP consists of two blocks: a full-raster stripper and an asynchronous FIFO buffer.



The full-raster stripper strips the full-raster timing information from the AXI4-S full-raster stream, leaving active-video data on the AXI4-S lite bus. The IP strips the timing information by setting the AXI4-S `TVALID` signal low during the blanking interval. When the input AXI4-S full-raster bus sets its `TUSER` signal high, the IP is in blanking. The stripper holds the `TUSER` high until the IP transmits the first active pixel.

Figure 94. Full-Raster to Streaming Converter Timing Diagram

An example of how the output `TUSER` signal is generated by detecting and holding the input `TUSER` signal on the full-raster video domain, until the first active pixel is detected on the `TDATA` bus.



The full-raster stripper has a pixel formatter logic, which is mainly based on a barrel shifter logic. It allows the IP to detect the correct position of the start of field for a pixel in parallel scenarios. It aligns the `tdata` packet content on the output video stream to ensure that the start of field always coincides with pixel 0.

The IP assumes that the transmitter side generates video data with a slower or faster pixel clock rate compared to the receiver video interface. Consequently, moving video data between two different clock domains requires clock domain crossings.

Additionally, the TVALID and TREADY flow-controlled AXI4-S video interface may cause the following control-flow problems on the video datapath:

- Clock-skew matching, if video receiver and transmitter and video processing clocks do not match.
- In-rush of data if one of the modules in the video pipeline cannot offer the necessary throughput to move the data at the expected rate.

This IP includes an asynchronous output FIFO buffer that handles clock crossing domains between the full-raster and video processing clock domains. Additionally, it temporally accepts data to accommodate a small amount of data-skew mismatching or in-rush data when transferring data between full-raster and active-video only video interfaces.

30.3.1. Full-Raster to Streaming Converter Interfaces

The IP has two functional video interfaces, two clock domains, and two resets. The Altera Streaming Video Protocol and the full-raster variant are standard interfaces to connect components that exchange video data.

All two input clocks are asynchronous from each other. Internally, the IP includes clock domain crossing circuits for both single bit and data bus signal cases, which safely allows data exchange between any of the two asynchronous clock domains. The IP also includes an embedded entity .sdc file, which provides all the necessary information to the Timing Analyzer. For system integration, when you instantiate the IP in a design, the only constraints required are:

- Clock frequency constraints for the input video clock (vid_in_clock_clk)
- Clock frequency constraints for the output video clock (vid_out_clock_clk)

Table 548. Full-Raster to Streaming Converter input and output video interfaces

Name	Direction	Width	Description
Clocks and resets			
vid_in_clock_clk	In	1	Input AXI4-S full-raster processing clock.
vid_in_reset_reset	In	1	Input AXI4-S full-raster processing reset.
vid_out_clock_clk	In	1	Output AXI4-S active-video processing clock.
vid_out_reset_reset	In	1	Output AXI4-S processing reset.
Altera Streaming Video interfaces			
axi4s_fr_vid_in_tdata	in	(99)(100)	AXI4-S data in.
<i>continued...</i>			

(99) The equation gives all full-raster tdata width sizes in these interfaces:

$$\max (\text{floor}(((\text{bits per color sample} \times (\text{number of color planes} + 1) \times \text{pixels in parallel}) + 7) / 8) \times 8, 16)$$

Name	Direction	Width	Description
axi4s_fr_vid_in_tvalid	in	1	AXI4-S data valid.
axi4s_fr_vid_in_tuser[pixels in parallel-1:0]	in	1	AXI4-S start of video frame.
axi4s_fr_vid_in_tuser[N-1:pixels in parallel]	in	(101)	Unused.
axi4s_fr_vid_in_tlast	in	1	AXI4-S end of packet .
axi4s_fr_vid_in_tready	out	1	Optional AXI4-S data ready.
axi4s_vid_out_tdata	out	(102)(103)	AXI4-S data in.
axi4s_vid_out_tvalid	out	1	AXI4-S data valid.
axi4s_vid_out_tuser[0]	out	1	AXI4-S start of video frame.
axi4s_vid_out_tuser[N-1:1]	out	(104)	Unused.
axi4s_vid_out_tlast	out	1	AXI4-S end of packet.
axi4s_vid_out_tready	in	1	AXI4-S data ready.

Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

-
- (100) The equation gives all tdata width sizes in these interfaces:

$$\max(\text{floor}(((\text{bits per color sample} \times \text{number of color planes} \times \text{pixels in parallel}) + 7) / 8) \times 8, 16)$$
- (101) The equation gives all tuser width sizes in these interfaces:

$$N = \text{ceil}(tdata\ width / 8)$$
- (102) The equation gives all full-raster tdata width sizes in these interfaces:

$$\max(\text{floor}(((\text{bits per color sample} \times (\text{number of color planes} + 1) \times \text{pixels in parallel}) + 7) / 8) \times 8, 16)$$
- (103) The equation gives all tdata width sizes in these interfaces:

$$\max(\text{floor}(((\text{bits per color sample} \times \text{number of color planes} \times \text{pixels in parallel}) + 7) / 8) \times 8, 16)$$
- (104) The equation gives all tuser width sizes in these interfaces:

$$N = \text{ceil}(tdata\ width / 8)$$

31. Genlock Controller IP

31.1. About the Genlock Controller IP

The Genlock Controller IP is a control loop system that matches the frequency of the voltage-controlled crystal oscillator (VCXO) clock to the selected reference clock.

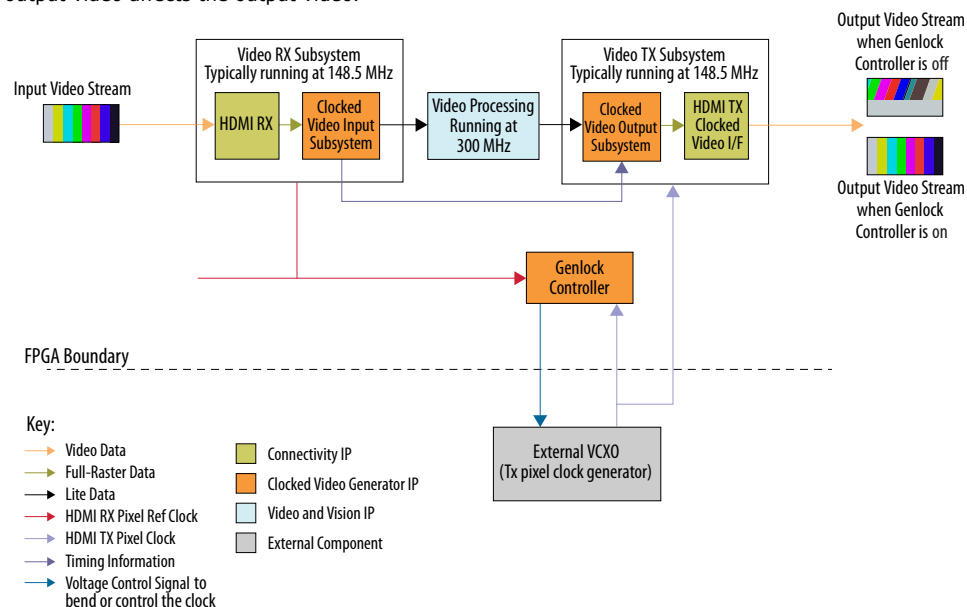
You can use the IP in an FPGA to support external voltage-controlled crystal oscillator (VCXO) clock tracking to a reference clock. The IP is highly parameterizable and programmable for various scenarios.

Typically, you generate video receiver and transmitter pixel clocks and video processing clock from three different clock generators. Hence, they are asynchronous. If the video receiver and transmitter clocks are asynchronous, the output video stream drifts over time relative to the input video stream.

The Genlock Controller IP allows locking receiver and transmitter pixel clocks, to avoid any drifting or rolling effect on the output video stream. The video drifting is visually noticeable when the processing video pipeline does not include a frame buffer in it.

Figure 95. Video Processing Pipeline

The figure shows a video processing pipeline without frame buffer. It shows how adding a genlock to the input and output video affects the output video.



31.1.1. Genlock Controller IP Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource usage depends on the device family, number of input reference clocks, and IP parameters.

Table 549. Genlock Controller IP Performance and Resources

The table shows ALM usage and f_{MAX} for a design with 1 input reference clock, 27 bits for the error counter, 20 bits for the DAC output value, and 6 bits for the confidence lock counter.

Target Device	ALMs	DSP Blocks	VCXO Clock f_{MAX} MHz	Ref0 Clock f_{MAX} MHz
Agilex 7 (AGFA012R24A2E2V)	1,553	2	710	882
Arria 10 (10AS066H1F34E1HG)	1,221	2	517	549
Cyclone 10 GX (10CX220YF672E5G)	1,231	2	458	458
Stratix 10 (1SX280LN2F43E1VG)	1,516	2	548	777

31.2. Genlock Controller IP Parameters

The IP offers compile- and run-time parameters

Table 550. Genlock Controller IP Parameters

Parameter	Values	Description
Configuration		
Number of reference clock	1 to 4	The number of reference clocks as source for VCXO tracking.
PFD Parameters		
Differential value size	12 to 27	The size of the PFD clock counters and output difference value.
Differential value size	8 to 24	The number of bits for the error counter to calculate the difference between two samples of output difference values.
Sample period counter size	4 to 32	Select how many least significant bits (LSB) the IP uses for the sample period counter.
LPF Parameters		
Filter Mode	Frequency Mode Phase Mode CPU selectable	Select the filter mode
P Gain Mode	Positive Only Negative Only CPU selectable	Select the proportional gain modes
I Gain Mode	Positive Only Negative Only CPU selectable	Select the integral gain modes
D Term enable	On or off	Turn on the derivative term logic
D Gain Mode	Positive Only Negative Only CPU selectable	Select the derivative gain modes.
LPF to DAC LSB Position	0 to 3	Select the LPF output to DAC least significant bit position. Allows the IP to ignore error wobble by moving the LSB.
<i>continued...</i>		

Parameter	Values	Description
VCXO Lock Confidence Counter Size	6 to 32	The lock confidence count size in bits. The number of successive samples with no error before the IP indicates lock.
DAC Parameters		
Resolution	8 to 24	PWM DAC value output size. Defines the precision of the value to drive the DAC. Higher number of bits indicates more resolution, and longer time to lock. Fewer bits could decrease the locking time but increase the VCXO jitter.
PMW Output Clock Divider	3 to 4096	The IP divides the VCXO clock by this value (and then by an additional 2). The IP uses the value to drive the DAC output pin. The value should be low enough to provide a fast response, but not too low to increase VCXO jitter.
Debug		
Enable Debug	0 to 2	Extra CPU debugging registers: 0 = Debug off 1 = VCXO and reference clock measurement 2 = PFD status value
Enable Genlock Profiler	Profiler enabled Profiler disabled	The profiler allows you to measure the difference between receive and transmit video pixel clocks. The profiler also measures the latency between receive and transmit start-of-frame (SOF) toggle signals.
CPU Clock Frequency	1 MHz to 1 GHz	CPU clock frequency for measuring VCXO and reference clock frequencies for debugging,

Figure 96. Genlock Controller IP Parameters

The screenshot displays the Genlock Controller IP configuration interface. On the left, the **Block Diagram** tab shows the internal components and their interconnections. On the right, the **Parameters** tab is active, showing various configuration settings categorized into Configuration, PFD Parameters, LPF Parameters, DAC Parameters, and Debug.

Block Diagram: The diagram shows the `intel_vvp_genlock_controller_0` block. Inputs include `vid_rx_ref_tim`, `vid_tx_ref_tim`, `vcxo_clock`, `vcxo_reset`, `ref0_clock`, `ref0_reset`, `av_mm_cpu_agent`, `cpu_clock`, and `cpu_reset`. Outputs include `vcxo_pwm`, `locked`, and `gpo_bus`.

Parameters:

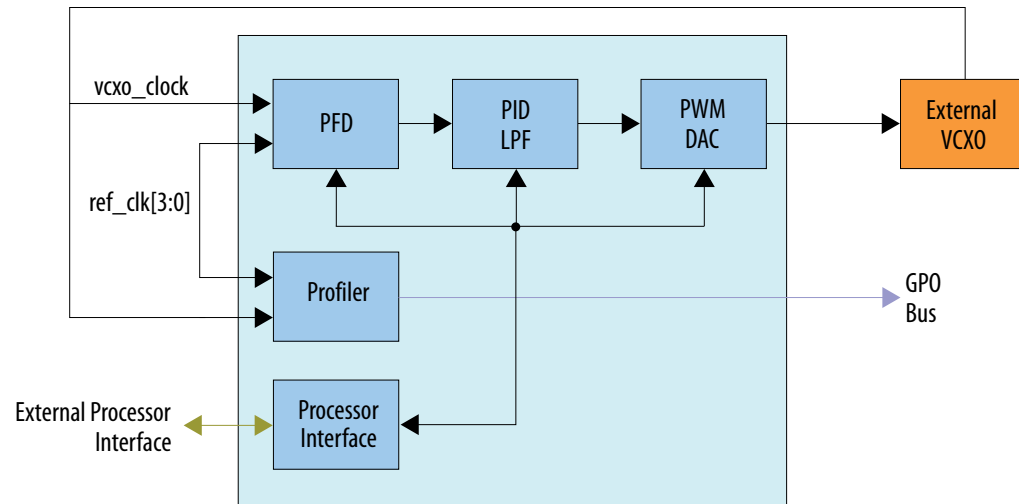
- Configuration:**
 - Number of Ref Clocks: 1 [1 - 4]
- PFD Parameters:**
 - e(t) Diff Value Size: 27 [12 - 27] bits
 - e(t) Diff Value Delta Size: 8 [8 - 24] bits
 - e(t) Sample Period Counter Size: 11 [4 - 32] bits
- LPF Parameters:**
 - Filter Mode: CPU selectable
 - P Gain Mode: CPU selectable
 - I Gain Mode: CPU selectable
 - ☒ D Term Enable
 - D Gain Mode: CPU selectable
 - LPF to DAC LSB Position: 0 data bit position
 - VCXO Lock Confidence Counter Size: 6 [6 - 32] bits
- DAC Parameters:**
 - Resolution: 20 [8 - 24] bits
 - PWM Output Clock Divider: 1000
- Debug:**
 - Enable Debug: 2 level [0 - 2]
 - CPU Clock Frequency: 100000000 Hz
 - Enable Genlock Profiler: Profiler enabled

31.3. Genlock Controller IP Functional Description

The Genlock Controller IP is a feedback control loop system consisting of a phase frequency detector (PFD), a proportional, integral, and derivative (PID) low pass filter (LPF), a pulse width modulation (PWM) digital-to-analog converter (DAC), a profiler module, and a CPU Interface.

Figure 97. Genlock Controller IP Block Diagram

The figure shows the design's submodules



Key:



IP subcomponents



Processor datapath



External processor interface
(Avalon memory-mapped)

The PFD measures differences between the VCXO input clock (`vcxo_clk`) and selected reference clock (`ref_clk`) to produce an error value $e(t)$.

The PID LPF receives the error and accumulates it to produce an output control value $c(t)$ that is proportional to the error. This output control value passes into the PWM DAC that converts it to a series of pulses that drives the external VCXO via a simple external resistor-capacitor (RC) network. The rate of pulses determines a voltage level on the VCXO, which changes the VCXO frequency. Therefore, the PFD output error changes as it is based on the new VCXO clock frequency (hence the feedback). The algorithm continues until the IP minimizes the error value.

The IP aims to reduce the error between the VCXO clock and the reference clock to zero indicating that the clocks must be at the same frequency.

The IP includes a profiler module, which allows measuring receive and transmit pixel clock frequency values. The profiler calculates the latency between receive and transmit SOF toggle pulses. It provides a general purpose output (GPO) bus, which you can map to LEDs to visualize the status of the genlock system.

Generally, the control loop operates in **Phase mode** or **Frequency mode**.

In **Phase mode**, the control loop aims to reduce the delta between the VCXO and selected reference clock measurements to remain in sync. In **Frequency mode**, the control loop aims to reduce the rate of change between the VCXO and selected reference clock measurements.

The choice of mode depends on requiring hard and soft genlock and the input to output video latencies.

The following factors contribute to the configuration parameters you select for your design:

- VCXO performance (base frequency, sweep range, reaction time)
- RC performance
- Nominal differences between VCXO and reference clocks, including ppm tolerances
- Rate of tracking required
- Stability (jitter) of reference clocks
- Additional error, jitter, and frequency changes that PLLs introduce between the VCXO and FPGA

Tuning the control loop must ensure the loop is stable and lock reasonably quickly. You should test your design using CPU programmable features (such as PID gains) to find the best solution.

PID Controller

The PID controller uses three control terms (proportional, integral, and derivative) to achieve an optimal response to a measured input. The IP reads the difference between the reference and VCXO clock frequencies. It generates a corrective output to modulate the VCXO frequency to match that of the reference clock.

The PID controller continuously calculates an error value $e(t)$ as the difference between two free running counters where one runs off the reference clock and the other the VCXO clock. Calculate an output from $e(t)$ using a control function $u(t)$ to minimise $e(t)$ over time. The control function uses PID terms and is:

Equation 2. Control Function

$$u(t) = K_p e(t) + \int_0^t K_i e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

Where:

- K_p is proportional term which includes a gain factor K . Changes in $e(t)$ cause the proportional effort to react accordingly. No error equates to no proportional effort.
- K_i denotes the integral term that also includes a gain factor K . The IP integrates $e(t)$ values (post gain) over time. They represent the past historic values of the control effort. The integral stops growing when you eliminate $e(t)$.
- K_d is the derivative term that can estimate the future trend based on the current rate of change in $e(t)$. Again, it includes a gain factor K .

You must tune the gain values to balance the terms to produce the required step response to the error input. The step response needs to be accurate but also respond timely to achieve lock.

For the genlock controller IP, $e(t)$ is:

$$e(t) = \text{reference clock counter} - \text{vcxo clock counter}$$

It can be positive or negative depending on which clock is faster. It uses 2's complement maths.

Depending on the ppm differences, $e(t)$ should be of a sufficient value to allow the controller to work using a feedback loop. Consequently, the IP samples $e(t)$ periodically instead of continuously. The period depends on many factors like ppm differences, VCXO reaction time, and lock time.

For the proportional gain, high values produce a large change on the output for a given change to $e(t)$. These high values can cause large over and under shoots of the ideal output value and the control loop can become unstable. However, if the gain is too small, the controller can be less responsive and take a long time to achieve lock.

The integral term is the sum of the sampled $e(t)$ (post gain) over time. The IP uses this term to accelerate towards the ideal output value. As with the proportional term, too high gain can cause over and under shoots and too small causes a slow response time.

In this IP, $e(t)$ experiences jitter from both the recovering input clocks and over time because of temperature. Therefore, it never stays at zero and likely drifts or wobbles around. The IP ensures that at this time small changes in $e(t)$ produce small enough changes to the output to ensure accuracy and achieve genlock. But it must not introduce excessive output jitter that may break any standards specifications (such as SDI).

31.3.1. Genlock Controller IP Interfaces

Table 551. Genlock Controller IP Interfaces

Name	Direction	Width	Description
Clocks and Resets			
vcxo_clock	Input	1	Input VCXO clock
vcxo_reset	Input	1	Input VCXO reset
ref0_clock	Input	1	Input reference # 0 clock
ref0_reset	Input	1	Input reference # 0 reset
<i>continued...</i>			

Name	Direction	Width	Description
ref1_clock	Input	1	Input reference # 1 clock
ref1_reset	Input	1	Input reference # 1 reset
ref2_clock	Input	1	Input reference # 2 clock
ref2_reset	Input	1	Input reference # 2 reset
ref3_clock	Input	1	Input reference # 3 clock
ref3_reset	Input	1	Input reference # 3 reset
cpu_clock	Input	1	Control interface clock
cpu_reset	Input	1	Control interface reset
Control Interfaces			
av_mm_control_agent_address	Input	7	Avalon memory-mapped agent address
av_mm_control_agent_write	Input	1	Avalon memory-mapped agent write
av_mm_control_agent_writedata	Input	32	Avalon memory-mapped agent write data
av_mm_control_agent_byteenable	Input	4	Avalon memory-mapped agent byte enable
av_mm_control_agent_read	Input	1	Avalon memory-mapped agent read request
av_mm_control_agent_readdata	Output	32	Avalon memory-mapped agent read data
av_mm_control_agent_readdatavalid	Output	1	Avalon memory-mapped agent read valid
av_mm_control_agent_waitrequest	Output	1	Avalon memory-mapped agent wait request
External Output Conduits			
vcxo_pwm	Output	1	A three-state PWM output to control an external VCXO
locked	Output	1	A level signal indicating if the IP achieves the genlock between the reference clock and VCXO.
Genlock Profiler Input Conduits			
vid_rx_ref_tim	Input	1	Receive toggle SOF signal
vid_tx_ref_tim	Input	1	Transmit toggle SOF signal
Genlock Profiler Output Conduits			
GPO_bus	Output	8	Genlock error mapping general purpose output (GPO) bus

Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

31.4. Using Genlock Controller IP Software

31.4.1. Achieving Genlock Controller Free Running (for Initialization or from Lock to Reference Clock N)

The IP drives the Genlock controller DAC PWM output pin to high impedance if you do not drive it. Therefore, the external Genlock controller floats to either high or low, which pushes the VCXO to one of its limits. However, this floating isn't always stable

especially because of changes in temperature. Altera recommends you always drive the Genlock controller DAC PWM output pin with a known value. At this stage you can drive that pin and no contention exists on the PCB.

If the Genlock controller is free running, at some point you may need to lock to a reference clock. As you don't know the reference clock frequency, lock from the VCXO mid value.

1. Enable DAC driver from High Z to mid value:
 - Write DAC control register = 0x1
2. If the IP is already locked to reference clock, return the DAC to its mid value. Leave both the PFD and LPF as they have no impact on the DAC output.

31.4.2. Locking to Reference Clock N (from Genlock Controller IP free running)

Locking to an external reference clock involves various bits within the same register. You require multiple writes to control the behavior in the correct sequence and to also ensure that any transition pulses are wide enough.

1. Always disable the PFD (in case the IP previously uses them):
 - a. Disable LPF
 - Write LPF Control 1 Register = 0x0
 - Write LPF Control 3 Register = 0x0
 - b. Disable PFD
 - Write PFD Control Register = 0x0
 - Write TxRx VCXO Clock Ratio Register = 0x0
 - Write TxRx Reference Clock Ratio Registers = 0x0
2. Setup and enable the LPF, which you can do as the PFD is still disabled that feeds the LPF:
 - a. Initialize LPF. In this **Phase Mode** example, the Integrator reset value = 0, P and I gain are 3.0 and 1.0 respectively. D is not used, nor is negative gain or I fraction gain mode. Lock status is **Phase mode** and shift the LSB position by 1 (for checking lock). Also reset DAC Saturation bit, in case it is set (bit 29, which you need to clear).
 - Write LPF Control 2 Register = 0x0
 - Write LPF Control 1 Register = 0x600d0000
 - Write LPF Control 3 Register = 0x000301
3. Perform the enable as an extra step (maintain all the other bits, except bit 29, which you should return to '0'):
 - a. Enable the LPF
 - Read modify write bit 0 LPF control 1 register = 0x400d0001
4. Setup and enable the PFD by setting the output update period MSBs, the choice of reference clock, its rate, and place the reference clock and VCXO clock in reset in one operation:
 - a. Initialize the PFD. In this example, set 10 bits. Set the 10 LSBs to '1' and the upper 6 MSBs to '0'. The 16 MSBs multiply the build time value by 1024.

- Write bits 31:16 PFD Control Register = 0x03ff
- b. In this example choose Ref0 and VCXO clock ratios to be 1:1. They both have theoretically the same frequency value:
 - Write TxRx VCXO clock ratio register = 0x04000000
 - Write TxRx reference clock ratio registers = 0x04000000
- c. Initialize the PFD (reset reference and VCXO clock counters)
 - Write bits 7:4 for the selected clock counter (bit 4 in our example) and bit 12 for VCXO clock counter PFD control register (Reg 0) = 1
- 5. Take the internal clock counters out of reset:
 - a. Read modify write bits 7:4 for the selected clock counter (bit 4 in the example) and bit 12 for VCXO clock counter PFD control register (Reg 0) = 0.
- 6. Enable the PFD:
Read modify write bit 0 PFD control register (Reg 0) = 0x1
- 7. Write the LPF value to drive DAC:
 - Write DAC control register (Reg 5) = 0x3

31.4.3. Setting the VCXO hold over

You can set the Genlock Controller IP to hold the last LPF value. For example, when the reference clock stops and maybe you require temporarily a test pattern to display. Holding the value does not make sense with any other DAC value mode (like mid value for instance).

1. Drive DAC
2. Write to DAC control register (Reg 5) = 0x13

31.4.4. Restarting the Genlock Controller IP

1. Depending on the configuration mode for this IP, restart the configuration algorithm:
 - Genlock controller free run (from lock to reference clock N) - where N is the old reference clock
 - Lock to reference clock N (from VCXO free run) - where N is the new reference clock

31.4.5. Locking to Reference Clock N New (from Locking to Reference Clock N Old)

Restart the Genlock Controller IP

31.4.6. Changing to Reference Clock or VCXO Base Frequencies (switch between p50 and p59.94 video formats and vice-versa)

Restart the Genlock Controller IP

31.4.7. Disturbing a Reference Clock (a cable pull)

The Genlock Controller IP PFD reacts to a stopped clock within 64 clock cycles. The PFD stops producing updates and the DAC value freezes. If the input changes over to a test pattern, it continues to display error free. However, when you reinsert a cable, Altera recommends that you restart the controller, even though it may appear to carry on where it left off. You must restart the genlock controller.

31.5. Genlock Controller IP Registers

The IP allows runtime configuration of parameters via Avalon memory-mapped processor register interface.

Table 552. Genlock Controller IP Registers

Offset	Register	Access	Description
Parameterization Registers			
0x000	VID PID	RO	Read this register to retrieve clocked video input product ID. This register always returns 0x6FA7_0170.
0x004	Version number	RO	Read this register to retrieve the version information for this IP.
0x008 to 0x00C	Reserved	-	Reserved register area.
0x010	CPU clock frequency	RO	Read this register to retrieve the value for the CPU clock frequency in Hz.
0x014	Number of reference clock	RO	Read this register to retrieve the value for the number of input reference clocks used by this IP.
0x018	Difference value delta size	RO	Read this value to retrieve the value for the number of bits this IP uses to calculate the phase or frequency error between two samples.
0x01C	Difference value size	RO	Read this value to retrieve the value for the number of bits this IP uses to calculate the phase or frequency error
0x020	Sample period counter size	RO	Read this register to retrieve the number of bits this IP uses to generate a sample period counter
0x024	LPF to DAC LSB position	RO	Read this register to retrieve the position of the LSBs that the IP ignores to calculate the error.
0x028	DAC resolution	RO	Read this register to retrieve the number of bits this IP uses to set the DAC resolution logic
0x02C	PWM Output Clock Divider Value	RO	Read this register to retrieve the exact value used to divide the clock generating the PWM output pulse
0x030	Derivative enable	RO	Read this register to check if the derivate logic for this IP is on
0x034	LPF Mode	RO	Read this register to retrieve the exact mode of operation for the low pass filter
0x038	Proportional gain mode	RO	Read this register to retrieve the exact mode of operation for the proportional gain
0x03C	Integral gain mode	RO	Read this register to retrieve the exact mode of operation for the integral gain
0x040	Derivative gain mode	RO	Read this register to retrieve the exact mode of operation for the derivative gain
<i>continued...</i>			

Offset	Register	Access	Description
0x044	VCXO Lock Confidence Counter Size	RO	Read this register to retrieve the number of bits this IP uses to generate a confidence lock counter
0x048	Enable Debug	RO	Read this register to check if the debugging logic for this IP is on
Core Specific Registers			
0x148	PFD Control	RW	This register configures the phase and frequency detector logic
0x14C	PFD Status	RO	This register provides debug information about the phase and frequency detector logic
0x150	LPF Control 1	RW	This register configures the LPF logic
0x154	LPF Control 2	RW	This register configures the LPF logic
0x158	LPF Status	RO	This register provides debug information about the LPF logic
0x15C	DAC Control	RW	This register configures the DAC control logic
0x160	DAC Status	RO	This register provides debug information about the DAC control logic
0x164	LPF Control 3	RW	This register configures the LPF logic
0x168	Clock Debug Status 1	RO	This register provides debug information about the VCXO clock frequency
0x16C	Clock Debug Status 2	RO	This register provides debug information about the Ref0 clock frequency
0x170	Clock Debug Status 3	RO	This register provides debug information about the Ref1 clock frequency
0x174	Clock Debug Status 4	RO	This register provides debug information about the Ref2 clock frequency
0x178	Clock Debug Status 5	RO	This register provides debug information about the Ref3 clock frequency
0x17C	PFD Debug Status	RO	This register provides debug information about the PFD accumulated error
0x180	Profiler Control	RW	This register configures the genlock profiler
0x184	Profiler Error Mapping 1	RW	This register allows define a genlock profile error region
0x188	Profiler Error Mapping 2	RW	This register allows define a genlock profile error region
0x18C	Profiler Pixel Counter	RO	This register provides pixel count information gathered by the genlock profiler module
0x190	Profiler Jitter Sample	RO	This register provides jitter information gathered by the genlock profiler module
0x194	Profiler Statistics	RO	This register provides statistics information gathered by the genlock profiler module
0x198 to 0x1A0	Reserved	-	Reserved register area
0x1A4	Tx Rx Ref0 Clock Ratio	RW	If Ref0 clock frequency is greater than VCXO clock frequency, this register sets the ratio between the two clocks. Otherwise set this register to 1.0
0x1A8	Tx Rx Ref1 Clock Ratio	RW	If Ref1 clock frequency is greater than VCXO clock frequency, this register sets the ratio between the two clocks. Otherwise set this register to 1.0
continued...			

Offset	Register	Access	Description
0x1AC	Tx Rx Ref2 Clock Ratio	RW	If Ref2 clock frequency is greater than VCXO clock frequency, this register sets the ratio between the two clocks. Otherwise set this register to 1.0
0x1B0	Tx Rx Ref3 Clock Ratio	RW	If Ref3 clock frequency is greater than VCXO clock frequency, this register sets the ratio between the two clocks. Otherwise set this register to 1.0
0x1B4	Tx Rx VCXO Clock Ratio	RW	If VCXO clock frequency is greater than reference clock frequency, this register sets the ratio between the two clocks. Otherwise set this register to 1.0

Register Bit Descriptions

Table 553. PFD Control

Bits	Name	Description
0	Enable	'1' enables the PFD error value e(t) outputs that drive the LPF Inputs. '0' produces no output but the internal counters still run and hence an algorithm restart sequence is required before use. <i>Enable</i> should be the last bit to transition in this register.
3:2	Ref Clock Switch Selection	Determines the reference clock for VCXO tracking: 00=ref0_clk (default) 01=ref1_clk 10=ref2_clk 11=ref3_clk Clock selection should only select clocks that exist in the design and ideally are running.
7:4	Ref Clock Reset	A 4-bit wide vector where each bit corresponds to an input reference clock (bit 4 = ref0_clk and bit 7 = ref3_clk). A 1 puts that reference places clock counter into reset and out of reset when you write, with bit 12 before you enable the PFD.
12	VCXO Clock Reset	A '1' puts the VCXO clock counter into reset and out of reset when you write '0'. Set this bit when you reset the corresponding Ref Clock Reset required for tracking against.
31:16	Output Update Period MSBS	Defines the top 16 MSB of the output update period counter. The LSB are defined by the C_ERR_UPDATE_BITS at build and is the duration of error updates from the PFD to the LPF (and subsequently the DAC). Set up this value before you use the <i>Enable</i> bit.

Table 554. PFD Status

Bits	Name	Description
3:0	Ref Clock Running	A 4 bits vector where each bit corresponds to an input reference clock (bit 0 = ref0_clk and bit 3 = ref3_clk) to indicate that the reference clock appears to be running. Running is based on samples the IP takes against the VCXO clock. If C_ENABLE_DEBUG>0, further clock measure registers are available.
7:4	Ref Clock Stopped	A 4 bits vector where each bit corresponds to an input reference clock (bit 0 = ref0_clk and bit 3 = ref3_clk) to indicate that the reference clock stops changing over the sampling period. These bits are sticky and only reset following an algorithm restart sequence. If the clock that dies is the current selected reference clock, the PFD does not update any further error changes to the LPF. The IP requires a restart sequence using either another reference clock or the same reference clock if it returns. The output disabled status is indicated in bit 9.
8	Overflow	Indicates an overflow in the difference between the internal tracking counters. An overflow is a serious error and the VCXO can be tracking anywhere. This bit only resets after a restart.
continued...		

Bits	Name	Description
9	Output Disabled	Indicates that the output is currently disabled because of the reference clock dying. The IP makes no further updates to the LPF. This bit only clears following an algorithm restart sequence.
10	diff_diff_value Output Overflow	This output value is smaller than the internal value you use. Indicates an overflow if the smaller value saturates. Consider increasing C_ERR_VAL_BITS. This bit only resets after an algorithm restart.
11	(diff_diff_value - last diff_diff_value) Output Overflow	This output value is smaller than the internal value. Indicates an overflow if the smaller value saturates. Consider increasing C_ERR_VAL_BITS. This bit only resets after a restart.

Table 555. Profiler Control

Bits	Name	Description
15:8	Reference Jitter Value	This value defines a genlock region equal to +/- number of pixels, where a SOF signal is allowed to jitter before the frame is outside the genlock region
6:4	GPO Clock Divider	This register defines the sample rate at which the IP updates the GPO with a new value. '4' = every ¼ of a second '2' = every ½ a second - Other values = every second
0	Reset Frame Counter	Reset the frame counter register.

Table 556. Profiler GPO Error Mapping 1

Bits	Name	Description
31:16	GPO Error Region 2	This value defines the upper bound for the first GPO error mapping region. This value also defines the lower bound for the second GPO error mapping region.
15:0	GPO Error Region 1	This value defines the lower bound for the first GPO error mapping region. This value defines the upper bound for the GPO genlock mapping region.

Table 557. Profiler Error Mapping 2

Bits	Name	Description
31:16	GPO Error Region 4	This value defines the upper bound for the third GPO error mapping region. This value defines the lower bound for the fourth GPO error mapping region.
15:0	GPO Error Region 3	This value defines the lower bound for the third GPO error mapping region. This value defines the upper bound for the second GPO error mapping region

Table 558. Profiler Pixels Counter

Bits	Name	Description
31:0	Total Pixels per Frame	Count of number of pixels per input frame

Table 559. Profiler Jitter Sample

Bits	Name	Description
31:16	SOF Delay	Count of number of pixels between receive and transmit SOF signals
15:0	Jitter Counter	Count of number of pixels between two consecutives SOF pulses

Table 560. Profiler Statistics

Bits	Name	Description
31:0	Bad Frames Counter	Count of frames that are outside the predefined genlock region set using the Reference Jitter Value

Table 561. LPF Control 1

Bits	Name	Description
0	Enable	1 enables the LPF. 0 keeps it in reset. You can enable the LPF anytime if the PFD is not enabled as no input values enter the LPF.
1	Integral Accumulator Reset Disable	If the DAC output reaches its maximum value (in either positive or negative direction), it automatically resets the Integral to the reset value as defined in LPF Control 2 register. Writing 1 to this bit stops the automatic reset, which may be useful for debugging.
2	Lock Loss Confidence Count Enable	This bit turns on the confidence counter for detection loss of lock. The IP gains lock when the lock confidence counter reaches its maximum value (C_LOCK_CNT_SIZE) and the IP sees successive no errors. However, a single error causes loss of lock immediately. Enabling this bit allows you to use the same confidence counter for indicating loss of lock. The IP indicates only if it reaches its minimum value of zero loss of. Hence lock status changes only when the extremes of the confidence counter are hit.
7:3	Lock Status LSB Position	By default, the IP allows 1 LSB bit change that does not change the lock status. You can increase the number of LSB bits by up to 31 (to 32 LSB) using these register bits. If the LSB position exceeds the size of the error value (C_CLK_DIV_BITS for diff_val in Phase mode , or C_ERR_VAL_BITS for diff_diff_val in Frequency mode), no amount of error causes the IP to lose lock.
17:16	Locked Status Control	Controls how the IP derives the Locked Status in the LPF Status Register. 00 = Frequency mode . Diff_Diff_Val Input is very small. The IP allows 1 LSB change and still indicates lock. You can increase the LSB position by up to a further 3 positions (to 4) using bits 31:30. Increase when you see larger error values because of larger sampling windows or jittery input. 01 = Phase mode . Diff_Val Input is very small. The IP allows 1 LSB change and still indicates lock. You can increase the LSB position by up to a further 3 positions (to 4) using bits 31:30. 10 = Reserved. 11 = Reserved.
18	I Mode	Defines the integral mode. '0' for Frequency mode where the integral part of the filter performs $I = \text{Sum of } (I \text{ Gain} * \text{Diff_Diff_Val})$ '1' for Phase mode where the integral part of the filter performs $I = \text{Sum of } (I \text{ Gain} * \text{Diff_Val})$
19	P Mode	Defines the proportional mode '0' for Frequency mode where the proportional part of the filter performs $P = P \text{ Gain} * \text{Diff_Diff_Val}$ '1' for Phase mode where the proportional part of the filter performs $P = P \text{ Gain} * \text{Diff_Val}$
24	D Mode	Defines the differential mode. '0' for Frequency mode where the differential part of the filter performs $D = D \text{ Gain} * (\text{Diff_Diff_Val} - \text{last Diff_Diff_Val})$ '1' for Phase mode where the differential part of the filter performs $D = D \text{ Gain} * \text{Diff_Diff_Val}$
25	Negative I Gain	Inverts the sign of the 2's complement input error. A positive input turns negative and vice-versa for the integral gain. Select this value after selecting I Mode .
continued...		

Bits	Name	Description
26	Negative P Gain	Inverts the sign of the 2's complement input error. A positive input turns negative and vice-versa for the proportional gain. Select this value after selecting P Mode .
27	Negative D Gain	Inverts the sign of the 2's complement input error. A positive input turns negative and vice-versa for the differential gain. Select this value after selecting D Mode .
28	I Gain Fraction	This bit reverses the direction of the I Gain powers of 2 shift, producing fractions of gain, instead of whole number gain.
29	Reset DAC Saturation Status	Setting high clears any latched DAC Saturation status as indicated in the LPF Status register. You must return this bit low.

Table 562. LPF Control 2

Bits	Name	Description
31:0	Integral Accumulator Reset Value	Value the IP loads into the 32 LSBs of the integral accumulator at reset or if it is reset by saturation (if on). If the internal integral size is greater than 31 bits, the IP uses the MSB of this register as a sign extend.

Table 563. LPF Status

Bits	Name	Description
0	Locked	Indicates when the error from the PFD reduces to a very small amount indicating locked status.
1	Integral Overflow	Indicates that an overflow in the integral accumulator is occurring and is reaching its maximum positive or negative value. It remains at that value until reset as part of a restart (refer to <i>Software Usage</i>). This bit indicates a serious error and the VCXO can be tracking anywhere.
2	DAC Saturated	Indicates that the DAC value output from the filter reaches its maximum positive or negative value. It automatically resets the Integral accumulator to its reset value unless you disable it using the LPF Control 1 register bit 1. This bit remains set until you clear it using bit 29 in the LPF Control 1 register.

Table 564. DAC Control

Bits	Name	Description
0	Output Enable	'0' Outputs High Impedance on the VCXO Driver Pin. '1' enables the DAC to drive the pin. Avoid contention before enabling the DAC and any HW I2C writes in advance.
3:1	DAC Output Data Select	Controls the PWM output via binary offset input value: 000 = sets mid value and VCXO sits at mid frequency 001 = uses value input from LPF Output (converted from 2's complement, which is just inverted MSB). 010 = sets largest negative value and slowest VCXO frequency 011 = sets largest positive value and fastest VCXO frequency 100 = Use CPU value bits 31:8 101 – 111 = Reserved
4	Hold	Hold value and do not update anymore. Only valid for DAC Output Data Select = 001.
5	Output Force Low	When set, the IP forces the PWM output low and ignores the output mode. Output Force High has a higher priority.
continued...		

Bits	Name	Description
6	Output Force High	When set, the IP forces the PWM output high and ignores the output mode. This bit takes priority over Output Force Low.
7	Output Clock Divide Disable	By default, the IP sets the output clock rate of the DAC control pin using the build time parameter C_DAC_CLK_DIV. If this bit is 1, the build time divide is disabled, and the output clock rate runs at full speed (which is half the VCXO frequency).
31:8	CPU DAC Output Data Value	Only available if DAC Output Data Select = 100. Value must be offset binary range to the size of the DAC output as in the DAC Status Register. 2's complement to offset binary conversion is invert MSB. Examples (assuming 24bit DAC size): 0x000000 sets largest negative value and slowest VCXO frequency 0xFFFFF sets largest positive value and fastest VCXO frequency 0x800000 sets the mid value and the VCXO sets at its mid frequency

Table 565. DAC Status

Bits	Name	Description
23:0	Current DAC Output Data Value	Current offset binary range number that the DAC produces.

Table 566. LPF Control 3

Bits	Name	Description
7:0	I Gain	Integral Gain. Provides powers of 2 shifts to the input error value e(t) (2's complement positive or negative number). The IP adds or subtracts this value into the Integrator before adding it to the final output sum. Valid values are 0 to 15 where 0 means that the IP does not use the integral in the control value to the DAC. Example: Gain set to 7, gives $2^{(7-1)} = 64$ i.e. the input e(t) is multiplied by 64. The input error value the IP uses depends on the I Mode bit and is either Diff_Val for Phase Mode or Diff_Diff_Val for Frequency Mode . You can invert the gain such that in the example above the input can be multiplied by -64. You invert the gain with the Negative I Gain Mode bit. Modes may not be available because of build parameters even though the bits remain persistent in this control register. Identify build time parameters using the LPF Status register.
15:8	P Gain	Proportional gain. Provides powers of 2 shifts to the input error value e(t) (2's complement positive or negative number). The IP adds or subtracts this value to the final output sum. Valid values 0 to 15 where 0 means that IP does not use the in the control value to the DAC. Example: Gain set to 7, gives $2^{(7-1)} = 64$ i.e. the input e(t) is multiplied by 64. The Input Error Value depends on the P Mode bit and is either Diff_Val for Phase Mode or Diff_Diff_Val for Frequency Mode . You can invert the gain such that in the example above the input can be multiplied by -64. Set the gain using the Negative P Gain Mode bit. Additionally, changes the gain shift (left or right) using the I Gain Fraction bit. Right Shifts produce fractional power of 2, which are useful after locking to reduce jitter caused by changes in the input error. Modes may not be available because of build parameters even though the bits remain persistent in this control register. Use the LPF Status register to identify build time parameters.
23:16	D Gain	Derivative gain. Provides powers of 2 shifts to the input error value e(t) (2's complement positive or negative number). The IP adds or subtracts this value to the final output sum. Valid values 0 to 15 where 0 means that the derivative is not used in the control value to the DAC. Example: Gain set to 7, gives $2^{(7-1)} = 64$ i.e. the input e(t) is multiplied by 64. The input error value depends on the D Mode bit and is either Diff_Diff_Val for Phase Mode or (Diff_Diff_Val - last Diff_Diff_Val) for Frequency Mode . You can invert the gain such that in the example above the input can be multiplied by -64. Set this gain using the Negative D Gain Mode bit. Modes may not be available because of build parameters even though the bits remain persistent in this control register. Use the LPF Status register to identify build time parameters.

Table 567. Clock Debug Status 1

Bits	Name	Description
31:0	VCXO Clock Measure	Count of clock ticks over 1 second. Zero indicates clock not running. Spurious counts indicate unstable clock. Stable clocks may deviate by a couple of counts because of frequency of CPU clock used to measure against and metastability.

Table 568. Clock Debug Status 2 to 5

Bits	Name	Description
31:0	Ref 0-3 Clock Measure	Count of clock ticks over 1 second. Zero indicates clock not running. Spurious counts could indicate unstable clock. Stable clocks may deviate by a couple of counts because of frequency of CPU clock the IP uses to measure against and metastability.

Table 569. PFD Debug Status

Bits	Name	Description
31:0	Current e(t)	This value indicates the current internal e(t) value (PFD counter difference value <i>Diff_Val</i>) that represents the continuous error difference value. It is a 2's complement value representing positive and negative numbers indicating which counter (reference clock or VCXO clock) is ahead. For Frequency Mode , the error value should trend to a value and remain there. Although it may oscillate around the trend value and drift slowly over time. For Phase Mode , the error value should reduce to zero and remain there. Although it may oscillate +/- 1. It should not drift over time.

Table 570. TxRx VCXO Clock Ratio

Bits	Name	Description
26:22	VCXO Integer number part	Sets the integer part of the clock ratio value. For example, if the clock ratio value is 8.750, this field needs to be set to 8.
21:0	VCXO Fractional number part	Sets the fractional part of the clock ratio value. For example, if the clock ratio value is 8.750, this field needs to be set to $(2^{22}) * (0.750)$. The 2^{22} multiplying factor is because of this IP using 22 bits to represent the fractional part in a fixed-point format.

Table 571. TxRx Ref 0-3 Clock Ratio

Bits	Name	Description
26:22	Ref 0-3 Integer number part	Sets the integer part of the clock ratio value. For example, if the clock ratio value is 8.750, this field needs to be set to 8.
21:0	Ref 0-3 Fractional number part	Sets the fractional part of the clock ratio value. For example, if the clock ratio value is 8.750, this field needs to be set to $(2^{22}) * (0.750)$. The 2^{22} multiplying factor is because of this IP using 22 bits to represent the fractional part in a fixed-point format.

31.6. Genlock Controller IP Software API

The IP has a software driver for software control of the IP at run time. The IP does not fit any of the generic device models provided by the Nios II HAL. It exposes a set of dedicated accessors to the control and status registers. The driver structure inherits the base driver structure, so all common methods in *Video and Vision Processing IPs Software API* are applicable.

intel_vvp_genlock_controller_init

Prototype uint32_t intel_vvp_genlock_controller_init
 (intel_vvp_genlock_controller_instance* instance,
 intel_vvp_core_base base);

Description Initialization function for a VVP Genlock Controller instance.

Attempts to initialize the fields of the Genlock Controller and its base core

Arguments instance, pointer to the intel_vvp_genlock_controller_instance to
 initialize

 base, the accessor for the core (on Nios this is a pointer to the base
 address of the core)

Returns kIntelVvpCoreOk success

 kIntelVvpCoreVidErr if the v kIntelVvpCoreVidErr if the vendor id of
 the core is not the IntelFPGA vendor ID.

 kIntelVvpCorePidErr if the product id of the core is not the
 genlock_controller product id

 kIntelVvpCoreInstanceErr if the instance parameter is zero (null
 pointer)

intel_vvp_genlock_controller_get_param_vid_pid

Prototype uint32_t intel_vvp_genlock_controller_get_param_vid_pid
 (intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The product ID number

intel_vvp_genlock_controller_get_param_version_number

Prototype uint32_t intel_vvp_genlock_controller_get_param_version_number
 (intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The IP version number value

intel_intel_vvp_genlock_controller_get_param_cpu_clk_freq_hz

Prototype uint32_t intel_vvp_genlock_controller_get_param_cpu_clk_freq_hz
 (intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The CPU frequency value in Hertz

`intel_vvp_genlock_controller_get_param_num_ref_clks`

Prototype uint32_t intel_vvp_genlock_controller_get_param_num_ref_clks
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The maximum number of reference clock for VCXO tracking

`intel_vvp_genlock_controller_get_param_err_val_bits`

Prototype uint32_t intel_vvp_genlock_controller_get_param_err_val_bits
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The number of bits to calculate an error value during a sampling period

`intel_vvp_genlock_controller_get_param_clk_div_bits`

Prototype uint32_t intel_vvp_genlock_controller_get_param_clk_div_bits
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The number of bits to generate the PDF clock counter

`intel_vvp_genlock_controller_get_param_err_update_bits`

Prototype uint32_t intel_vvp_genlock_controller_get_param_err_update_bits
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The number of bit to generate a PDF sampling window

`intel_vvp_genlock_controller_get_param_dac_filter_lsb`

Prototype uint32_t intel_vvp_genlock_controller_get_param_dac_filter_lsb
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The number of LSBs to discard during the error calculation

`intel_vvp_genlock_controller_get_param_dac_val_bits`

Prototype `uint32_t intel_vvp_genlock_controller_get_param_dac_val_bits
(intel_vvp_genlock_controller_instance* instance);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

Returns The number of bit to generate the DAC data output word.

`intel_vvp_genlock_controller_get_param_dac_clk_div`

Prototype `uint32_t intel_vvp_genlock_controller_get_param_dac_clk_div
(intel_vvp_genlock_controller_instance* instance);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

Returns The clock divider factor to generate the PWM output pulse

`intel_vvp_genlock_controller_get_param_derivative_en`

Prototype `uint32_t intel_vvp_genlock_controller_get_param_derivative_en
(intel_vvp_genlock_controller_instance* instance);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

Returns The status for the PID derivative term enable flag

`intel_vvp_genlock_controller_get_param_lpf_mode`

Prototype `uint32_t intel_vvp_genlock_controller_get_param_lpf_mode
(intel_vvp_genlock_controller_instance* instance);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

Returns The LPF mode value

`intel_vvp_genlock_controller_get_param_i_gain_mode`

Prototype `uint32_t intel_vvp_genlock_controller_get_param_i_gain_mode
(intel_vvp_genlock_controller_instance* instance);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

Returns The integral gain mode value

`intel_vvp_genlock_controller_get_param_p_gain_mode`

Prototype `uint32_t intel_vvp_genlock_controller_get_param_p_gain_mode
(intel_vvp_genlock_controller_instance* instance);`

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The proportional gain mode value

intel_vvp_genlock_controller_get_param_d_gain_mode

Prototype uint32_t intel_vvp_genlock_controller_get_param_d_gain_mode
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The derivative gain mode value

intel_vvp_genlock_controller_get_param_lock_cnt_size

Prototype uint32_t intel_vvp_genlock_controller_get_param_lock_cnt_size
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The number of bit to generate the lock status counter

intel_vvp_genlock_controller_get_param_enable_debug

Prototype uint32_t intel_vvp_genlock_controller_get_param_enable_debug
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The debug mode value

intel_vvp_genlock_controller_is_rxtx_locked

Prototype uint32_t intel_vvp_genlock_controller_is_rxtx_locked
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns 1 if the bit is asserted, 0 otherwise

intel_vvp_genlock_controller_is_int_overflow

Prototype uint32_t intel_vvp_genlock_controller_is_int_overflow
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns 1 if the bit is asserted, 0 otherwise

`intel_vvp_genlock_controller_is_dac_sat`

Prototype uint32_t intel_vvp_genlock_controller_is_dac_sat
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns 1 if the bit is asserted, 0 otherwise

`intel_vvp_genlock_controller_get_lpf_control_1_enable`

Prototype uint32_t intel_vvp_genlock_controller_get_lpf_control_1_enable
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns the status of the LPF enable flag.

`intel_vvp_genlock_controller_get_lpf_control_1_int_rst_dis`

Prototype uint32_t
intel_vvp_genlock_controller_get_lpf_control_1_int_rst_dis
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns the status of the LPF automatic reset flag.

`intel_vvp_genlock_controller_get_lpf_control_1_lock_loss_conf_en`

Prototype uint32_t
intel_vvp_genlock_controller_get_lpf_control_1_lock_loss_conf_en
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns the status of the LPF lock loss confidence counter enable flag.

`intel_vvp_genlock_controller_get_lpf_control_1_freq_lock_lsb`

Prototype uint32_t
intel_vvp_genlock_controller_get_lpf_control_1_freq_lock_lsb
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns the number of LSBs to ignore during the error calculation.

`intel_vvp_genlock_controller_get_lpf_control_1_lock_status_mode`

Prototype uint32_t
intel_vvp_genlock_controller_get_lpf_control_1_lock_status_mode
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns the LPF lock operation mode.

`intel_vvp_genlock_controller_get_lpf_control_1_i_mode`

Prototype uint32_t intel_vvp_genlock_controller_get_lpf_control_1_i_mode
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns the LPF integral term operation mode.

`intel_vvp_genlock_controller_get_lpf_control_1_p_mode`

Prototype uint32_t intel_vvp_genlock_controller_get_lpf_control_1_p_mode
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns the LPF proportional term operation mode.

`intel_vvp_genlock_controller_get_lpf_control_1_d_mode`

Prototype uint32_t intel_vvp_genlock_controller_get_lpf_control_1_d_mode
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns the LPF derivative term operation mode.

`intel_vvp_genlock_controller_get_lpf_control_1_i_gain_sign_inv`

Prototype uint32_t
intel_vvp_genlock_controller_get_lpf_control_1_i_gain_sign_inv

```
(intel_vvp_genlock_controller_instance* instance);
```

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns the status of the LPF integral term sign flag.

`intel_vvp_genlock_controller_get_lpf_control_1_p_gain_sign_inv`

Prototype uint32_t
intel_vvp_genlock_controller_get_lpf_control_1_p_gain_sign_inv
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns the status of the LPF proportional term sign flag.

`intel_vvp_genlock_controller_get_lpf_control_1_d_gain_sign_inv`

Prototype uint32_t
intel_vvp_genlock_controller_get_lpf_control_1_d_gain_sign_inv
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns the status of the LPF derivative term sign flag.

`intel_vvp_genlock_controller_get_lpf_control_1_i_gain_frac`

Prototype uint32_t
intel_vvp_genlock_controller_get_lpf_control_1_i_gain_frac
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns the status of the DAC enable flag.

`intel_vvp_genlock_controller_get_dac_control_op_en`

Prototype uint32_t intel_vvp_genlock_controller_get_dac_control_op_en
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns the status of the DAC enable flag.

`intel_vvp_genlock_controller_get_dac_control_data_sel`

Prototype `uint32_t intel_vvp_genlock_controller_get_dac_control_data_sel`
 `(intel_vvp_genlock_controller_instance* instance);`

Arguments instance, pointer to the `intel_vvp_genlock_controller_instance`

Returns the status of the DAC operation mode.

`intel_vvp_genlock_controller_get_dac_control_hold`

Prototype `uint32_t intel_vvp_genlock_controller_get_dac_control_hold`
 `(intel_vvp_genlock_controller_instance* instance);`

Arguments instance, pointer to the `intel_vvp_genlock_controller_instance`

Returns the status of the DAC hold flag.

`intel_vvp_genlock_controller_get_dac_control_op_force_low`

Prototype `uint32_t`
 `intel_vvp_genlock_controller_get_dac_control_op_force_low`
 `(intel_vvp_genlock_controller_instance* instance);`

Arguments instance, pointer to the `intel_vvp_genlock_controller_instance`

Returns the status of the DAC force-low flag.

`intel_vvp_genlock_controller_get_dac_control_op_force_high`

Prototype `uint32_t`
 `intel_vvp_genlock_controller_get_dac_control_op_force_high`
 `(intel_vvp_genlock_controller_instance* instance);`

Arguments instance, pointer to the `intel_vvp_genlock_controller_instance`

Returns the status of the DAC force-high flag.

`intel_vvp_genlock_controller_get_dac_control_clk_div_disable`

Prototype `uint32_t`
 `intel_vvp_genlock_controller_get_dac_control_clk_div_disable`
 `(intel_vvp_genlock_controller_instance* instance);`

Arguments instance, pointer to the `intel_vvp_genlock_controller_instance`

Returns the status of the DAC clock divider flag.

`intel_vvp_genlock_controller_get_dac_control_ob_cpu_value`

Prototype `uint32_t`
 `intel_vvp_genlock_controller_get_dac_control_ob_cpu_value`
 `(intel_vvp_genlock_controller_instance* instance);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

Returns the DAC offset value.

`intel_vvp_genlock_controller_get_pfd_control_enable`

Prototype `uint32_t intel_vvp_genlock_controller_get_pfd_control_enable`
 `(intel_vvp_genlock_controller_instance* instance);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

Returns the status of the PFD enable flag.

`intel_vvp_genlock_controller_get_pfd_control_ref_clk_sw_sel`

Prototype `uint32_t`
 `intel_vvp_genlock_controller_get_pfd_control_ref_clk_sw_sel`
 `(intel_vvp_genlock_controller_instance* instance);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

Returns the value of the reference clock for VCXO tracking.

`intel_vvp_genlock_controller_get_pfd_control_ref_clk_sw_rst`

Prototype `uint32_t`
 `intel_vvp_genlock_controller_get_pfd_control_ref_clk_sw_rst`
 `(intel_vvp_genlock_controller_instance* instance);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

Returns the status of the reset associated to each of the reference clocks.

`intel_vvp_genlock_controller_get_pfd_control_vcxo_clk_sw_rst`

Prototype `uint32_t`
 `intel_vvp_genlock_controller_get_pfd_control_vcxo_clk_sw_rst`

```
(intel_vvp_genlock_controller_instance* instance);
```

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns the status of the reset associated to the VCXO clock.

`intel_vvp_genlock_controller_get_pfd_control_update_cnt_msbs`

Prototype uint32_t
intel_vvp_genlock_controller_get_pfd_control_update_cnt_msbs
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns the value to update the update period counter.

`intel_vvp_genlock_controller_get_stat_clk_run`

Prototype uint32_t intel_vvp_genlock_controller_get_stat_clk_run
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns If you assert a bit that indicates the reference clock associated to it is running.

`intel_vvp_genlock_controller_get_stat_clk_stop`

Prototype uint32_t intel_vvp_genlock_controller_get_stat_clk_stop
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns If you assert a bit that indicates the reference clock associated to it is not running.

`intel_vvp_genlock_controller_get_ref_clk_stop`

Prototype uint32_t intel_vvp_genlock_controller_get_ref_clk_stop
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns the status of the PID output block. If you assert this bit indicates that the PID is not updating this output value because of a reference clock not running.

`intel_vvp_genlock_controller_get_pfd_overflow_val`

Prototype `uint32_t intel_vvp_genlock_controller_get_pfd_overflow_val
 (intel_vvp_genlock_controller_instance* instance);`

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns the status of the error overflow flag.

`intel_vvp_genlock_controller_get_lpf_control_2_int_rst_val`

Prototype `uint32_t
 intel_vvp_genlock_controller_get_lpf_control_2_int_rst_val
 (intel_vvp_genlock_controller_instance* instance);`

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The integral accumulator value at reset

`intel_vvp_genlock_controller_get_lpf_control_3_i_gain`

Prototype `uint32_t intel_vvp_genlock_controller_get_lpf_control_3_i_gain
 (intel_vvp_genlock_controller_instance* instance);`

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The gain to set the integral term for the PID controller

`intel_vvp_genlock_controller_get_lpf_control_3_p_gain`

Prototype `uint32_t intel_vvp_genlock_controller_get_lpf_control_3_p_gain
 (intel_vvp_genlock_controller_instance* instance);`

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The gain to set the proportional term for the PID controller

`intel_vvp_genlock_controller_get_lpf_control_3_d_gain`

Prototype `uint32_t intel_vvp_genlock_controller_get_lpf_control_3_d_gain
 (intel_vvp_genlock_controller_instance* instance);`

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns

`intel_vvp_genlock_controller_get_dac_status_val`

Prototype `uint32_t intel_vvp_genlock_controller_get_dac_status_val
 (intel_vvp_genlock_controller_instance* instance);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

Returns The offset value output to the external VCXO

`intel_vvp_genlock_controller_get_clk_dbg_status_1_val`

Prototype `uint32_t intel_vvp_genlock_controller_get_clk_dbg_status_1_val
 (intel_vvp_genlock_controller_instance* instance);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

Returns The frequency value for VCXO clock

`intel_vvp_genlock_controller_get_clk_dbg_status_2_val`

Prototype `uint32_t intel_vvp_genlock_controller_get_clk_dbg_status_2_val
 (intel_vvp_genlock_controller_instance* instance);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

Returns The frequency value for reference clock 0

`intel_vvp_genlock_controller_get_clk_dbg_status_3_val`

Prototype `uint32_t intel_vvp_genlock_controller_get_clk_dbg_status_3_val
 (intel_vvp_genlock_controller_instance* instance);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

Returns The frequency value for reference clock 1

`intel_vvp_genlock_controller_get_clk_dbg_status_4_val`

Prototype `uint32_t intel_vvp_genlock_controller_get_clk_dbg_status_4_val
 (intel_vvp_genlock_controller_instance* instance);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

Returns The frequency value for reference clock 2

`intel_vvp_genlock_controller_get_clk_dbg_status_5_val`

Prototype `uint32_t intel_vvp_genlock_controller_get_clk_dbg_status_5_val
 (intel_vvp_genlock_controller_instance* instance);`

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The frequency value for reference clock 3

intel_vvp_genlock_controller_get_pfd_dbg_status_val

Prototype uint32_t intel_vvp_genlock_controller_get_pfd_dbg_status_val
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The PID error

intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_ref0_int_val

Prototype uint32_t
intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_ref0_int_val
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The integer part of the TxRx reference # 0 clock ratio

intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_ref0_frc_val

Prototype uint32_t
intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_ref0_frc_val
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The fractional part of the TxRx reference # 0 clock ratio

intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_ref1_int_val

Prototype uint32_t
intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_ref1_int_val
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The integer part of the TxRx reference # 1 clock ratio

intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_ref1_frc_val

Prototype uint32_t
intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_ref1_frc_val
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The fractional part of the TxRx reference # 1 clock ratio

[intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_ref2_int_val](#)

Prototype uint32_t
intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_ref2_int_val
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The integer part of the TxRx reference # 2 clock ratio

[intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_ref2_frc_val](#)

Prototype uint32_t
intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_ref2_frc_val
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The fractional part of the TxRx reference # 2 clock ratio

[intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_ref3_int_val](#)

Prototype uint32_t
intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_ref3_int_val
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The integer part of the TxRx reference # 3 clock ratio

[intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_ref3_frc_val](#)

Prototype uint32_t
intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_ref3_frc_val
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The fractional part of the TxRx reference # 3 clock ratio

[intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_vcxo_int_val](#)

Prototype uint32_t
intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_vcxo_int_val
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The integer part of the TxRx VCXO clock ratio

intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_vcxo_frc_val

Prototype uint32_t
intel_vvp_genlock_controller_get_pfd_tx_rx_ratio_vcxo_frc_val
(intel_vvp_genlock_controller_instance* instance);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance

Returns The fractional part of the TxRx VCXO clock ratio

intel_vvp_genlock_controller_set_pfd_control_enable

Prototype int intel_vvp_genlock_controller_set_pfd_control_enable
(intel_vvp_genlock_controller_instance* instance, uint32_t in_val);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance
in_val, the input reference value

Returns kIntelVvpGenlocControllerOk, otherwise
kIntelVvpGenlockControllerInstanceErr

intel_vvp_genlock_controller_set_pfd_control_ref_clk_sw_sel

Prototype int intel_vvp_genlock_controller_set_pfd_control_ref_clk_sw_sel
(intel_vvp_genlock_controller_instance* instance, uint32_t in_val);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance
in_val, the input reference value

Returns kIntelVvpGenlocControllerOk, otherwise
kIntelVvpGenlockControllerInstanceErr

intel_vvp_genlock_controller_set_pfd_control_ref_clk_sw_rst

Prototype int intel_vvp_genlock_controller_set_pfd_control_ref_clk_sw_rst
(intel_vvp_genlock_controller_instance* instance, uint32_t in_val);

Arguments instance, pointer to the intel_vvp_genlock_controller_instance
in_val, the input reference value

Returns kIntelVvpGenlocControllerOk, otherwise
kIntelVvpGenlockControllerInstanceErr

`intel_vvp_genlock_controller_set_pfd_control_vcxo_clk_sw_rst`

Prototype `int intel_vvp_genlock_controller_set_pfd_control_vcxo_clk_sw_rst
 (intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_pfd_control_update_cnt_msbs`

Prototype `int intel_vvp_genlock_controller_set_pfd_control_update_cnt_msbs
 (intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_lpf_control_2_int_rst_val`

Prototype `int intel_vvp_genlock_controller_set_lpf_control_2_int_rst_val
 (intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_lpf_control_3_i_gain`

Prototype `int intel_vvp_genlock_controller_set_lpf_control_3_i_gain
 (intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_lpf_control_3_p_gain`

Prototype `int intel_vvp_genlock_controller_set_lpf_control_3_p_gain
(intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
`in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
`kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_lpf_control_3_d_gain`

Prototype `int intel_vvp_genlock_controller_set_lpf_control_3_d_gain
(intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
`in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
`kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_dac_control_op_en`

Prototype `int intel_vvp_genlock_controller_set_dac_control_op_en
(intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
`in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
`kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_dac_control_data_sel`

Prototype `int intel_vvp_genlock_controller_set_dac_control_data_sel
(intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
`in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
`kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_dac_control_hold`

Prototype `int intel_vvp_genlock_controller_set_dac_control_hold`
 `(intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_dac_control_op_force_low`

Prototype `int intel_vvp_genlock_controller_set_dac_control_op_force_low`
 `(intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_dac_control_op_force_high`

Prototype `int intel_vvp_genlock_controller_set_dac_control_op_force_high`
 `(intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_dac_control_clk_div_disable`

Prototype `int intel_vvp_genlock_controller_set_dac_control_clk_div_disable`
 `(intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_dac_control_ob_cpu_value`

Prototype `int intel_vvp_genlock_controller_set_dac_control_ob_cpu_value
(intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
`in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
`kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_lpf_control_1_enable`

Prototype `prototype: int
intel_vvp_genlock_controller_set_lpf_control_1_enable
(intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
`in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
`kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_lpf_control_1_int_rst_dis`

Prototype `int intel_vvp_genlock_controller_set_lpf_control_1_int_rst_dis
(intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
`in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
`kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_lpf_control_1_lock_loss_conf_en`

Prototype `int
intel_vvp_genlock_controller_set_lpf_control_1_lock_loss_conf_en
(intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments *argument:* `instance`, pointer to the
`intel_vvp_genlock_controller_instance`
`in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
`kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_lpf_control_1_freq_lock_lsb`

Prototype `int intel_vvp_genlock_controller_set_lpf_control_1_freq_lock_lsb
 (intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_lpf_control_1_lock_status_mode`

Prototype `int
 intel_vvp_genlock_controller_set_lpf_control_1_lock_status_mode
 (intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_lpf_control_1_i_mode`

Prototype `int intel_vvp_genlock_controller_set_lpf_control_1_i_mode
 (intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_lpf_control_1_p_mode`

Prototype `int intel_vvp_genlock_controller_set_lpf_control_1_p_mode
 (intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_lpf_control_1_d_mode`

Prototype `int intel_vvp_genlock_controller_set_lpf_control_1_d_mode
 (intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
`in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_lpf_control_1_i_gain_sign_inv`

Prototype `int intel_vvp_genlock_controller_set_lpf_control_1_i_gain_sign_inv
 (intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
`in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_lpf_control_1_p_gain_sign_inv`

Prototype `int intel_vvp_genlock_controller_set_lpf_control_1_p_gain_sign_inv
 (intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
`in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_lpf_control_1_d_gain_sign_inv`

Prototype `int intel_vvp_genlock_controller_set_lpf_control_1_d_gain_sign_inv
 (intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
`in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_lpf_control_1_i_gain_frac`

Prototype `int intel_vvp_genlock_controller_set_lpf_control_1_i_gain_frac
 (intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_lpf_control_1_rst_dac_sat_latch`

Prototype `int
 intel_vvp_genlock_controller_set_lpf_control_1_rst_dac_sat_latch
 (intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_ref0_frc_val`

Prototype `int intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_ref0_frc_val
 (intel_vvp_genlock_controller_instance* instance, float in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_ref0_int_val`

Prototype `int intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_ref0_int_val
 (intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_ref1_frc_val`

Prototype `int intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_ref1_frc_val
 (intel_vvp_genlock_controller_instance* instance, float in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

`in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_ref1_int_val`

Prototype `int intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_ref1_int_val
 (intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

`in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_ref2_frc_val`

Prototype `int intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_ref2_frc_val
 (intel_vvp_genlock_controller_instance* instance, float in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

`in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_ref2_int_val`

Prototype `int intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_ref2_int_val
 (intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`

`in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_ref3_frc_val`

Prototype `int intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_ref3_frc_val`
 `(intel_vvp_genlock_controller_instance* instance, float in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_ref3_int_val`

Prototype `int intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_ref3_int_val`
 `(intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_vcxo_frc_val`

Prototype `int intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_vcxo_frc_val`
 `(intel_vvp_genlock_controller_instance* instance, float in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

`intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_vcxo_int_val`

Prototype `int intel_vvp_genlock_controller_set_pfd_tx_rx_ratio_vcxo_int_val`
 `(intel_vvp_genlock_controller_instance* instance, uint32_t in_val);`

Arguments `instance`, pointer to the `intel_vvp_genlock_controller_instance`
 `in_val`, the input reference value

Returns `kIntelVvpGenlocControllerOk`, otherwise
 `kIntelVvpGenlockControllerInstanceErr`

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

32. Generic Crosspoint IP

Related Information

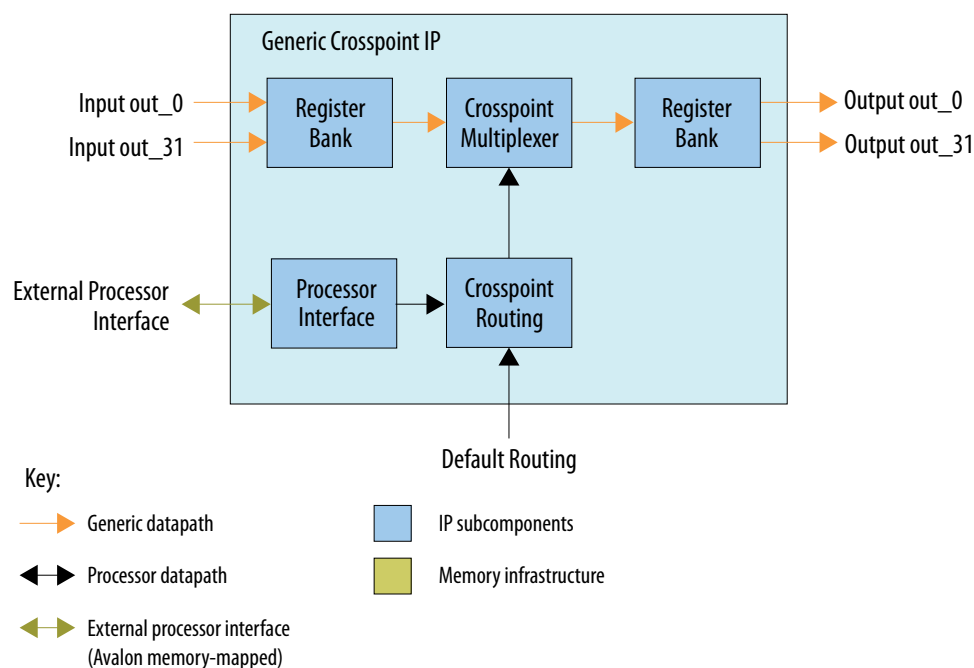
Video and Vision Processing IP Interfaces on page 30

32.1. About the Generic Crosspoint IP

The IP is a $M \times N$ generic data crosspoint where M and N signify the number of input and output ports, respectively. This IP can route discrete signals around an FPGA design under software control. Both input and output ports work on the same clock domain.

Data is input to and output from the Generic Crosspoint IP via a selectable number of ports. The size of the input and output ports is a global parameter configurable from the GUI. The number of input and output ports is in the range of 1 to 32.

Figure 98. Generic Crosspoint Block Diagram



The front-end and back-end of this IP include a bank of registers, and the crosspoint multiplexer and routing logic, which can process run-time and build-time configurable routing between input and output ports.

You can control the input-to-output routing dynamically at run-time via the CPU interface. You can also assign a default routing at build-time via the Platform Designer IP GUI. The crosspoint routing reverts to the default routing on reset. If you turn off the CPU interface, the crosspoint is statically fixed at the default routing, which you can use if the routing does not need to change at run-time.

32.1.1. Generic Crosspoint IP Features

- Parameterizable 1 to 32 discrete input and output conduits
- Optional Avalon memory-mapped processor control interface to program the crosspoint
- Run-time and build-time crosspoint routing of inputs to outputs
- Synchronous I/O interfaces only
- 1 to 1024 bits port width configuration
- Small FPGA resource footprint

32.1.2. Generic Crosspoint Performance and Resources

The IP resources depend on the device family, number of supported inputs and output ports, and port data width.

Table 572. Generic Crosspoint Performance and Resources

The table shows ALM usage and f_{MAX} for a design with 2 input interfaces, 2 output interfaces, and 8-bit port data width.

Target Device	ALMs	M20Ks	f_{MAX} MHz
Agilex 7 (AGFA012R24A2E2V)	25	0	1000
Arria 10 (10AS066H1F34E1HG)	17	0	645
Cyclone 10 GX (10CX220YF672E5G)	17	0	645
Stratix 10 (1SX280LN2F43E1VG)	25	0	1000

32.2. Generic Crosspoint IP Parameters

The IP offers compile-time parameters.

Table 573. Generic Crosspoint IP Parameters

Parameter	Values	Description
Control Settings		
Memory-mapped control interface	True or false	Turn on for the memory-mapped control interface
Separate clock for control interface	True or false	Turn on to run the run-time control interface on a different clock domain.
Crosspoint Settings		
Crosspoint port width	1 to 1024	Width of crosspoint input and output ports in bits
Input (Per Input Interface)		
<i>continued...</i>		

Parameter	Values	Description
Number of inputs	1 to 32	Number of crosspoint input ports
Conduit Signal type	User-defined string	Set the signal type of the input to match the type of the conduit it connects to.
Conduit associated clock	True or false	The conduit that the IP connects to may or may not have an associated clock set. This parameter sets the associated clock parameter of the input to <code>main_clock</code> .
Output (Per Output Interface)		
Number of outputs	1 to 32	Number of crosspoint output ports
Default input	0 to 31	The output port selects this input by default out of reset. If run-time configuration is disabled, this setting is fixed.
Conduit Signal type	User-defined string	Set the signal type of the output to match the type of the conduit it connects to.
Conduit associated clock	True or false	The conduit that the IP connects to may or may not have an associated clock. This parameter sets the associated clock parameter of the output to <code>main_clock</code> .

32.3. Generic Crosspoint IP Interfaces

Table 574. Generic Crosspoint IP Interfaces

Name	Direction	Width	Description
Clocks and Resets			
<code>main_clock</code>	In	1	Input and output crosspoint clock
<code>main_reset</code>	In	1	Input and output crosspoint reset
<code>cpu_clock</code>	In	1	Control interface clock
<code>cpu_reset</code>	In	1	Control interface reset
Crosspoint interfaces			
<code>in_N</code>	In	1 to 1024	Crosspoint input ports, where N goes from 0 to 31
<code>out_N</code>	Out	1 to 1024	Crosspoint output ports, where N goes from 0 to 31
Control Interfaces			
<code>av_mm_control_agent_address</code>	In	7	Avalon memory-mapped agent address
<code>av_mm_control_agent_write</code>	In	1	Avalon memory-mapped agent write
<code>av_mm_control_agent_writedata</code>	In	32	Avalon memory-mapped agent write data
<code>av_mm_control_agent_byteenable</code>	In	4	Avalon memory-mapped agent byte enable
<code>av_mm_control_agent_read</code>	In	1	Avalon memory-mapped agent read
<code>av_mm_control_agent_readdata</code>	Out	32	Avalon memory-mapped agent read data
<code>av_mm_control_agent_readdatavalid</code>	Out	1	Avalon memory-mapped agent read
<code>av_mm_control_agent_waitrequest</code>	Out	1	Avalon memory-mapped agent wait request

Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

32.4. Generic Crosspoint IP Registers

Each register is either read-only (RO) or read-write (RW).

Table 575. Generic Crosspoint IP Registers

Offset	Register	Access
Parameterization Registers		
0x000	VID_PID	RO
0x004	VERSION_NUMBER	RO
0x008	NUM_INPUTS	RO
0x00C	NUM_OUTPUTS	RO
Core Specific Registers		
0x140	REG_OUTPUT_0	RW
0x144	REG_OUTPUT_1	RW
0x148	REG_OUTPUT_2	RW
0x14C	REG_OUTPUT_3	RW
0x150	REG_OUTPUT_4	RW
0x154	REG_OUTPUT_5	RW
0x158	REG_OUTPUT_6	RW
0x15C	REG_OUTPUT_7	RW
0x160	REG_OUTPUT_8	RW
0x164	REG_OUTPUT_9	RW
0x168	REG_OUTPUT_10	RW
0x16C	REG_OUTPUT_11	RW
0x170	REG_OUTPUT_12	RW
0x174	REG_OUTPUT_13	RW
0x178	REG_OUTPUT_14	RW
0x17C	REG_OUTPUT_15	RW
0x180	REG_OUTPUT_16	RW
0x184	REG_OUTPUT_17	RW
0x188	REG_OUTPUT_18	RW
0x18C	REG_OUTPUT_19	RW
0x190	REG_OUTPUT_20	RW
0x194	REG_OUTPUT_21	RW
continued...		

Offset	Register	Access
0x198	REG_OUTPUT_22	RW
0x19C	REG_OUTPUT_23	RW
0x1A0	REG_OUTPUT_24	RW
0x1A4	REG_OUTPUT_25	RW
0x1A8	REG_OUTPUT_26	RW
0x1AC	REG_OUTPUT_27	RW
0x1B0	REG_OUTPUT_28	RW
0x1B4	REG_OUTPUT_29	RW
0x1B8	REG_OUTPUT_30	RW
0x1BC	REG_OUTPUT_31	RW

Register Bit Descriptions

Table 576. Vid_pid

Bits	Description
31:0	Product Identification Number

Table 577. Version_number_pid

Bits	Description
31:0	IP Version Number

Table 578. Reg_output_N

where N goes from 0 to 31

Bits	Description
31:0	Number of the input port to select

32.5. Generic Crosspoint IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Table 579. Generic Crosspoint IP Software API

Name	Description
intel_vvp_generic_crosspoint_init	Initialize a Generic Crosspoint instance
intel_vvp_generic_crosspoint_get_output	Get an output's selected input port
intel_vvp_generic_crosspoint_set_output	Set an output's selected input port

intel_vvp_generic_crosspoint_init

prototype:

```
int
intel_vvp_generic_crosspoint_init(intel_vvp_generic_crosspoint_instance*
instance, intel_vvp_core_base base);
```

description:

Initialization function for a VVP Generic Crosspoint instance.

Attempts to initialize the fields of the Generic Crosspoint and its base core

argument:

instance, pointer to the intel_vvp_generic_crosspoint_instance to initialize
base, the accessor for the core (on Nios this is a pointer to the base address
of the core)

*return
value:*

kIntelVvpCoreOk success

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor
ID.

kIntelVvpCorePidErr if the product id of the core is not the Generic Crosspoint
product id

kIntelVvpCoreInstanceErr if the instance parameter is zero (null pointer)

intel_vvp_generic_crosspoint_get_output

prototype:

```
uint8_t
intel_vvp_generic_crosspoint_get_output(intel_vvp_generic_crosspoint_instance*
instance, uint8_t output);
```

description:

Get an output's selected input port

argument:

instance, pointer to the intel_vvp_generic_crosspoint_instance
output, number of the output port

*return
value:*

Number of the input port selected by the output port

intel_vvp_generic_crosspoint_set_output

prototype:

```
int
intel_vvp_generic_crosspoint_set_output(intel_vvp_generic_crosspoint_instance*
instance, uint8_t output, uint8_t input);
```

description:

argument:

instance, pointer to the intel_vvp_generic_crosspoint_instance
output, number of the output port
input, number of the input port to select

return value:

- kIntelVvpGenericCrosspointOk success
- kIntelVvpGenericCrosspointInstanceErr if the instance number is zero (null pointer)
- kIntelVvpGenericCrosspointInputErr if the selected input doesn't exist
- kIntelVvpGenericCrosspointOutputErr if the selected output doesn't exist

Related Information

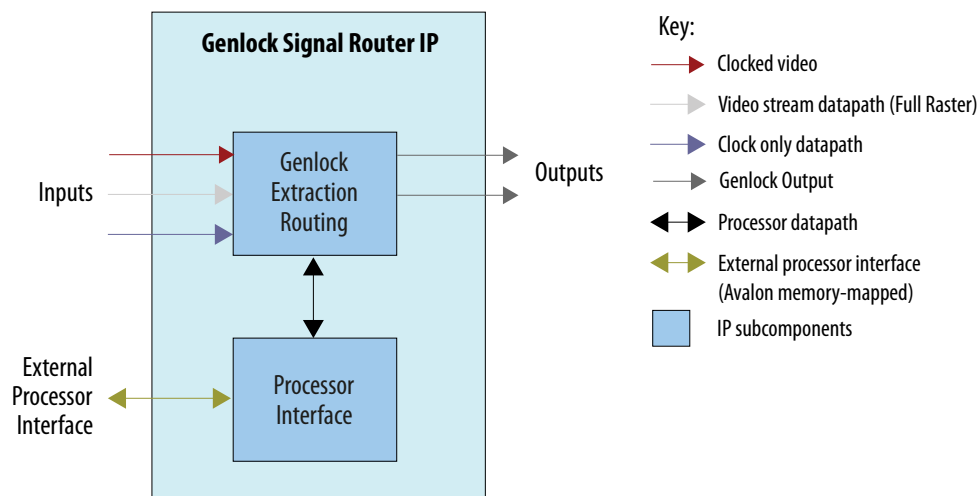
[Video and Vision Processing IPs Software Programming Model](#) on page 38

33. Genlock Signal Router IP

33.1. About the Genlock Signal Router IP

The IP is a multichannel genlock strobe extractor and router. This IP passes genlock timing signals to internal or external FPGA multirate video clock generators. The IP helps video input and output clock genlock and frame synchronization, based on video timing markers derived from video connectivity IPs.

Figure 99. Genlock Signal Router IP Block Diagram



Data is input to and output from the Genlock Signal Router IP via a selectable number of ports. The final number of input and output ports is in the range 1 to 32 inclusive, which you can configure in the Genlock Signal Router IP GUI.

At build time, you can configure the input port interfaces in three different modes: Altera video streaming full-raster, Altera clocked video, and clock only mode. The IP automatically extracts timing information from the input signals, and then routes the signals between the input and output ports. You can control the input-to-output routing dynamically at run-time using the processor interface.

The output interface for this IP provides two optional interfaces:

- An interface with a set of discrete timing signals, such as field flag, horizontal, and vertical synchronization timing markers.
- An interface that provides clocks only ports

The timing markers can then pass to internal or external FPGA multirate video clock generators.

33.1.1. Genlock Signal Router IP Features

- An Avalon memory-mapped control interface
 - 1 to 32 clocked video input interfaces
 - Three input clocked video interfaces:
 - ☐ Altera clocked video
 - ☐ full-raster (refer to *Altera Streaming Video Protocol Specification*)
 - ☐ Clock only
- Synchronous and asynchronous clocked video input signals
- Five user configurable timing control extract modes:
 - ☐ Altera clocked video blank or sync timing
 - ☐ full-raster blank or sync timing
 - ☐ Genlock discrete field, vertical, and horizontal timing signals, and clocks only output interface
- Three user configurable frame sync extract modes:
 - ☐ full-raster T_{USER} start of frame timing
 - ☐ Field toggle
 - ☐ Field pulse
- Processor configurable genlock router switching between input and output ports

Related Information

[Altera Streaming Video Protocol Specification](#)

33.1.2. Genlock Signal Router IP Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource usage depends on the device family, number of supported inputs and output ports, and input interface type.

Table 580. Genlock Signal Router IP Performance and Resources

The table shows ALM usage and f_{MAX} for a design with 4 input interfaces and 4 output interfaces. All the interfaces are Altera video streaming full-raster interfaces with 10-bit per sample, 3 color planes and 2 pixels in parallel.

Target Device	ALMs	M20Ks	f_{MAX} MHz
Agilex 7 (AGFA012R24A2E2V)	429	0	1000
Arria 10 (10AS066H1F34E1HG)	405	0	645
Cyclone 10 GX (10CX220YF672E5G)	402	0	645
Stratix 10 (1SX280LN2F43E1VG)	428	0	994

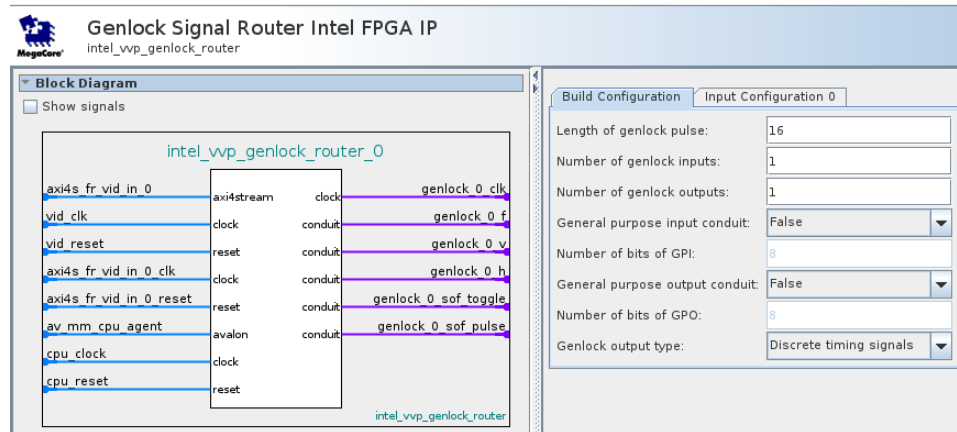
33.2. Genlock Signal Router IP Parameters

The IP offers compile-time parameters.

Table 581. Genlock Signal Router IP Parameters

Parameter	Values	Description
Build configuration		
Length of clock pulse	1 to 32	The number of clocks for the output genlock pulse
Number of genlock inputs	1 to 32	The number of input ports
Number of genlock outputs	1 to 32	The number of output ports
General-purpose input conduit	True or false	Turn on a general-purpose input port for this IP
Number of bits of GPI	1 to 32	The number of bits for the general-purpose input interface
General-purpose output conduit	True or false	Turn on a general-purpose output port for this IP
Number of bits of GPO	1 to 32	The number of bits for the general-purpose output interface
Genlock output type	Discrete timing signals, Clocks only	Select the type for all available outputs
Genlock Input Type: AXI-S FR (Per Input Interface)		
Number of bits per color plane	8 to 16	The number of bits per color sample at the input
Number of pixels in parallel	1 to 8	The number of pixels transmitted every clock cycle.
Number of color planes	1 to 4	The number of color planes per pixel
AXI4-S FR interface TREADY	True or false	Enable the TREADY signal as part of the full-raster interface
Genlock Input Type: Discrete timing Clocked Video signals (Per Input Interface)		
Clock	0 to 1	Discrete input interface has Input clock signal
F	0 to 1	Discrete input interface has Input field signal
V	0 to 1	Discrete input interface has Input vertical blanking signal
H	0 to 1	Discrete input interface has Input horizontal blanking signal
V sync	0 to 1	Discrete input interface has Input vertical sync signal
H sync	0 to 1	Discrete input interface has Input horizontal sync signal
Toggle	0 to 1	Discrete input interface has Input field pulse signal
Pulse	0 to 1	Discrete input interface has Input field toggle signal

Figure 100. Genlock Signal Router IP GUI



33.3. Genlock Signal Router IP Interfaces

Table 582. Genlock Signal Router IP Interfaces

Name	Direction	Width	Description
Clocks and Resets			
vid_clk	In	1	Reserved
vid_reset	In	1	Reserved
axi4s_fr_vid_in_clk	In	1	Optional Input clock associated with each of the full-raster interfaces
axi4s_fr_vid_in_reset	In	1	Optional Input reset associated with each of the full-raster interfaces
async_clk	In	1	Optional Input clock associated with each of the discrete and clocks only interfaces
cpu_clock	In	1	Control interface clock
cpu_reset	In	1	Control interface reset
Control Interfaces			
av_mm_control_agent_address	In	7	Avalon memory-mapped agent address
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request
Altera Streaming Video interfaces (Optional per input interface)			
axi4s_fr_vid_in_tdata	In	(105)	AXI4-S data in
continued...			

Name	Direction	Width	Description
axi4s_fr_vid_in_tvalid	In	1	AXI4-S data valid
axi4s_fr_vid_in_tuser[0]	In	1	AXI4-S start of video frame
axi4s_fr_vid_in_tuser[N-1:1]	In	(106)	Unused
axi4s_fr_vid_in_tlast	In	1	AXI4-S end of packet
axi4s_fr_vid_in_tready	Out	1	AXI4-S data ready
Altera Discrete Timing Signals (Optional per input interface)			
async_clock	In	1	Input clock
async_f	In	1	Field
async_v	In	1	Vertical blanking
async_h	In	1	Horizontal blanking
async_v_sync	In	1	Vertical sync
async_h_sync	In	1	Horizontal sync
async_toggle	In	1	Start of frame toggle
axi4s_pulse	In	1	Start of frame pulse
Altera Clocks only signal (Optional per output interface)			
genlock_clock	In	1	Output Clock
Altera Discrete Timing Signals (Optional per input interface)			
genlock_clock	In	1	Input Clock
genlock_f	In	1	Field
genlock_v	In	1	Vertical blanking
genlock_h	In	1	Horizontal blanking
genlock_sof_toggle	In	1	Start of frame toggle
genlock_sof_pulse	In	1	Start of frame pulse

Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

33.4. Genlock Signal Router IP Registers

The IP allows runtime configuration of parameters via Avalon memory-mapped processor register interface.

⁽¹⁰⁵⁾ The equation gives the TDATA width for interfaces for full-raster variants:

$$\max(\text{floor}(((\text{bits per color sample} \times (\text{number of color planes} + 1) \times \text{pixels in parallel}) + 7) / 8) \times 8, 16)$$

⁽¹⁰⁶⁾ This equation gives the TUSER width N for these interfaces: $\text{ceil}(\text{tdata width} / 8)$

Table 583. Genlock Signal Router IP Registers

Offset	Register	Access	Description
Parameterization registers			
0x000	VID_PID	RO	Read this register to retrieve genlock signal router product ID. This register always returns 0x6FA7_0172.
0x004	VERSION_NUMBER	RO	Read this register to retrieve the version information for this IP.
0x008	PULSE_LENGTH	RO	Read this register to retrieve the number of clock cycles the start of frame pulse expects to be asserted high
0x00C	Reserved	RO	
0x010	GENLOCK_INPUTS	RO	Read this register to retrieve the number of input ports
0x014	GENLOCK_OUTPUTS	RO	Read this register to retrieve the number of output ports
0x018	GENLOCK_OUTPUT_TYPE	RO	Read this register to retrieve the output type interface
0x01C - 0x098	INPUT_TYPE_{0 to 31}	RO	Read this register to retrieve the input type interface
Core Specific Registers			
0x100	GPIO_INPUT	RO	Read this register to retrieve the data loaded in the general-purpose input register
0x104	GPIO_OUTPUT	RW	Read this register to retrieve the data loaded in the general-purpose output register. Alternatively, set this register to load a value into it.
0x180	OUTPUT_PORT_0	RW	Sets the configuration parameters for output each of the output ports
0x184	OUTPUT_PORT_1	RW	
0x188	OUTPUT_PORT_2	RW	
0x18C	OUTPUT_PORT_3	RW	
0x190	OUTPUT_PORT_4	RW	
0x194	OUTPUT_PORT_5	RW	
0x198	OUTPUT_PORT_6	RW	
0x19C	OUTPUT_PORT_7	RW	
0x1A0	OUTPUT_PORT_8	RW	
0x1A4	OUTPUT_PORT_9	RW	
0x1A8	OUTPUT_PORT_10	RW	
0x1AC	OUTPUT_PORT_11	RW	
0x1B0	OUTPUT_PORT_12	RW	
0x1B4	OUTPUT_PORT_13	RW	
0x1B8	OUTPUT_PORT_14	RW	
0x1BC	OUTPUT_PORT_15	RW	
0x1C0	OUTPUT_PORT_16	RW	
0x1C4	OUTPUT_PORT_17	RW	
continued...			

0x1C8	OUTPUT_PORT_18	RW
0x1CC	OUTPUT_PORT_19	RW
0x1D0	OUTPUT_PORT_20	RW
0x1D4	OUTPUT_PORT_21	RW
0x1D8	OUTPUT_PORT_22	RW
0x1DC	OUTPUT_PORT_23	RW
0x1E0	OUTPUT_PORT_24	RW
0x1E4	OUTPUT_PORT_25	RW
0x1E8	OUTPUT_PORT_26	RW
0x1EC	OUTPUT_PORT_27	RW
0x1F0	OUTPUT_PORT_28	RW
0x1F4	OUTPUT_PORT_29	RW
0x1F8	OUTPUT_PORT_30	RW
0x1FC	OUTPUT_PORT_31	RW

Table 584. Vid_pid

Bits	Description
31:0	Product Identification Number

Table 585. Version_number_pid

Bits	Description
31:0	IP Version Number

Table 586. Pulse_length

Bits	Description
31:0	The number of clocks for the output genlock pulse

Table 587. Genlock inputs

Bits	Description
31:0	The number of input ports

Table 588. Genlock output

Bits	Description
31:0	The number of output ports

Table 589. Genlock output type

Bits	Description
31:0	Type of output interface

Table 590. Input Type N

where N goes from 0 to 31

Bits	Description
31:0	Type of input interface

Table 591. GPIO Input

Bits	Description
31:0	General-purpose input register

Table 592. GPIO output

Bits	Description
31:0	General-purpose output register

Table 593. Output Port N

where N goes from 0 to 31

Bits	Description
31	This field enables the output port
4:0	This field selects the input port to route to the output

33.5. Genlock Signal Router IP Software API

intel_vvp_genlock_router_init

prototype:

```
int
intel_vvp_genlock_router_init(intel_vvp_genlock_router_instance* instance, intel_vvp_core_base base);
```

Description:

Initialization function for a VVP Genlock Router instance.

Attempts to initialize the fields of the Genlock Router and its base core

argument:

instance, pointer to the intel_vvp_genlock_router_instance to initialize

base, the accessor for the core (on Nios this is a pointer to the base address of the core)

return value:

kIntelVvpCoreOk success

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID.

kIntelVvpCorePidErr if the product id of the core is not the genlock_router product id

kIntelVvpCoreInstanceErr if the instance parameter is zero (null pointer)

intel_vvp_genlock_router_get_param_vid_pid

prototype:

```
int intel_vvp_genlock_router_get_param_vid_pid(intel_vvp_genlock_router_instance* instance);
```

argument: instance, pointer to the intel_vvp_genlock_router_instance

return value: The product ID number

intel_vvp_genlock_router_get_param_version_number

prototype:

```
int intel_vvp_genlock_router_get_param_version_number(intel_vvp_genlock_router_instance* instance);
```

argument: instance, pointer to the intel_vvp_genlock_router_instance

return value: The IP version number

intel_vvp_genlock_router_get_param_pulse_length

prototype:

```
int intel_vvp_genlock_router_get_param_pulse_length(intel_vvp_genlock_router_instance* instance);
```

argument: instance, pointer to the intel_vvp_genlock_router_instance

return value: The number of clock cycles the start of frame output pulse remains active-high

intel_vvp_genlock_router_get_param_number_of_genlock_inputs

prototype:

```
int intel_vvp_genlock_router_get_param_number_of_genlock_inputs(intel_vvp_genlock_router_instance* instance);
```

argument: instance, pointer to the intel_vvp_genlock_router_instance

return value: The number of input ports that were selected during build time configuration

intel_vvp_genlock_router_get_param_number_of_genlock_outputs

prototype:

```
int
intel_vvp_genlock_router_get_param_number_of_genlock_outputs(
    intel_vvp_genlock_router_instance*instance);
```

argument: instance, pointer to the intel_vvp_genlock_router_instance

return value: The number of output ports that were selected during build time configuration

intel_vvp_genlock_router_get_param_genlock_output_type

prototype:

```
int
intel_vvp_genlock_router_get_param_genlock_output_type(
    intel_vvp_genlock_router_instance*instance);
```

argument: instance, pointer to the intel_vvp_genlock_router_instance

return value: The type of output interface that was selected during build time configuration

intel_vvp_genlock_router_get_param_genlock_input_type

prototype:

```
int
intel_vvp_genlock_router_get_param_genlock_input_type(
    intel_vvp_genlock_router_instance*instance, uint16_t in_val);
```

argument: instance, pointer to the intel_vvp_genlock_router_instance
in_val, the index of the input port

return value: The type of input interface that was selected during build time configuration for a particular input port

intel_vvp_genlock_router_get_gpio_input

prototype:

```
int
intel_vvp_genlock_router_get_gpio_input(
    intel_vvp_genlock_router_instance* instance);
```

argument: instance, pointer to the intel_vvp_genlock_router_instance

return value: The data on the general-purpose input register

intel_vvp_genlock_router_get_gpio_output

prototype:

```
int
intel_vvp_genlock_router_get_gpio_output(
    intel_vvp_genlock_router_instance* instance);
```

argument: instance, pointer to the intel_vvp_genlock_router_instance

return value: The data on the general-purpose output register

intel_vvp_genlock_router_set_gpio_output

prototype:

```
int  
intel_vvp_genlock_router_set_gpio_output(intel_vvp_genlock_  
router_instance* instance, uint32_t out_val);
```

argument: instance, pointer to the intel_vvp_genlock_router_instance

out_val, the value that is stored in the general-purpose output register.

return value: if successful kIntelVvpGenlockRouterOk, otherwise
kIntelVvpGenlockRouterInstanceErr

intel_vvp_genlock_router_enable_output_intf

prototype:

```
int  
intel_vvp_genlock_router_enable_output_intf(intel_vvp_genlo  
ck_router_instance* instance, uint16_t out_port);
```

argument: instance, pointer to the intel_vvp_genlock_router_instance

out_port, the index value associated with the output port.

return value: if successful kIntelVvpGenlockRouterOk, otherwise
kIntelVvpGenlockRouterInstanceErr

intel_vvp_genlock_router_disable_output_intf

prototype:

```
int  
intel_vvp_genlock_router_disable_output_intf(intel_vvp_genl  
ock_router_instance* instance, uint16_t out_port);
```

argument: instance, pointer to the intel_vvp_genlock_router_instance

out_port, the index value associated with the output port.

return value: if successful kIntelVvpGenlockRouterOk, otherwise
kIntelVvpGenlockRouterInstanceErr

intel_vvp_genlock_router_sel_inout_pair

prototype:

```
intel_vvp_genlock_router_sel_inout_pair(intel_vvp_genlock_r  
outer_instance* instance, uint16_t in_port, uint16_t  
out_port);
```

argument: instance, pointer to the intel_vvp_genlock_router_instance
in_port, the index value associated with the input port.
out_port, the index value associated with the output port.

return value: if successful kIntelVvpGenlockRouterOk, otherwise
kIntelVvpGenlockRouterInstanceErr

intel_vvp_genlock_router_get_out_cfg

prototype:

```
int intel_vvp_genlock_router_get_out_cfg
(intel_vvp_genlock_router_instance* instance, uint16_t
out_port);
```

argument: instance, pointer to the intel_vvp_genlock_router_instance
out_port, the index value associated with the output port.

return value: The configuration value stored on a particular output register

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

34. Guard Bands IP

34.1. About the Guard Bands IP

The IP compares each color plane in the input video to upper and lower guard band values. If the value in any color plane exceeds the upper guard band, the IP replaces the value with the upper guard band. Similarly, if the value in any color plane falls below the lower guard band, the IP replaces the value with the lower guard band.

You can specify different guard band values for each color plane. You can alter these values at run-time through the Avalon memory-mapped interface. Otherwise, the guard band values are fixed at compile-time.

The input can be unsigned data or signed 2's complement data for which the IP converts the data to an unsigned format (by adding half the maximum range) before applying guard bands. The guard bands are specified as unsigned values.

The IP can drive the output data as signed data but, as with signed input data, the IP applies guard bands on the unsigned data before it converts it to signed output. The IP converts the output data to a signed format by subtracting half the maximum range after applying guard bands.

You cannot select both signed input and signed output data. The IP prepares data for other video and vision processing IPs, which primarily operate on unsigned data. You can place this IP before and after another video and vision IP, configuring the first for signed input and the second to use signed output data.

34.1.1. Guard Bands IP Performance and Resources

Altera provides resource and utilization data for guidance.

Table 594. Guard Bands IP Performance and Resources

The table shows ALM usage and f_{MAX} for a design with 4 pixels in parallel, 12 bits per color sample, and 3 color planes

Target device	ALMs	f_{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	1,271	679
Cyclone 10 GX speed grade 5 (10CX220YF78015G)	1,090	414

34.2. Guard Bands IP Parameters

Parameter	Values	Description
Video Data Format		
Bits per color sample	8 to 16	Select the number of bits per color sample.
Number of color planes	1 to 4	Select the number of color planes per pixel.
Number of pixels in parallel	1 to 8	Select the number of pixels in parallel.
4:2:2 data	On or off	Turn on to indicate that the input data is 4:2:2 sampled.
Signed input data	On or off	Turn on to indicate that the input data is signed 2's complement numbers.
Signed output data	On or off	Turn on to indicate that the output data is signed 2's complement numbers.
Control		
Lite mode	On or off	Turn on to use the lite variant of the Altera Streaming Video Protocol.
Memory-mapped control interface	On or off	Turn on to specify guard bands values at run-time using the Avalon memory-mapped interface.
Separate clock for control interface	On or off	Turn on to enable a separate clock for the control interface.
Debug features	On or off	Turn on for debugging features (not applicable when you turn on Lite mode).
Guard Bands		
Lower/Upper guard band for color <0-3>	0 to (Bits per color sample)-1	These parameters define the guard bands for each color plane (up to 4 colors per pixel). Color 0 is in the LSBs of the AXI-streaming video data bus. If you turn on Memory-mapped control interface , the IP does not use these values and you must write their value using Avalon memory-mapped interface.

34.3. Guard Bands IP Interfaces

Table 595. Configurable Guard Bands IP Input and Output Interfaces

Name	Direction	Width	Description
Clocks and resets			
main_clock_clk	In	1	AXI4-S processing clock.
main_reset_rst	In	1	AXI4-S processing reset.
agent_clock_clk	In	1	Optional agent interface clock.
agent_reset_rst	In	1	Optional agent interface reset.
Control interfaces			
av_mm_control_agent_address	In	7	Avalon memory-mapped agent address.
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write.
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data.
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable.
<i>continued...</i>			

Name	Direction	Width	Description
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read.
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data.
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read.
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request.
Altera Streaming Video interfaces			
axi4s_vid_in_tdata	In		AXI4-S data in.
axi4s_vid_in_tvalid	In	1	AXI4-S data valid.
axi4s_vid_in_tuser[0]	In	1	AXI4-S start of video frame.
axi4s_vid_in_tuser[1]	In	1	AXI4-S control or data packet.
axi4s_vid_in_tuser[N-1:2]	In		Unused.
axi4s_vid_in_tlast	In	1	AXI4-S end of packet.
axi4s_vid_in_tready	Out	1	AXI4-S data ready.
axi4s_vid_out_tdata	Out	(107)	AXI4-S data out.
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid.
axi4s_vid_out_tuser[0]	Out	1	AXI4-S start of video frame.
axi4s_vid_out_tuser[1]	Out	1	AXI4-S control or data packet.
axi4s_vid_out_tuser[N-1:2]	Out	(108)	Unused.
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet.
axi4s_vid_out_tready	In	1	AXI4-S data ready.

Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

34.4. Guard Bands IP Registers

Each register is either read-only (RO) or read-write (RW).

(107) The equation gives all tdata widths in these interfaces:

$$\max(2, \text{ceil}((\text{bits per color sample} \times \text{number of color planes})/8)) \times \text{pixels in parallel} \times 8$$

(108) The equation gives all tuser widths in these interfaces:

$$N = \text{ceil}(\text{tdata width} / 8)$$

Table 596. Guard Bands IP Registers

In the software API the register names appear with a prefix of INTEL_VVP, INTEL_VVP_CORE or INTEL_VVP_GUARD_BANDS as appropriate and with an optional REG suffix

Address	Register	Access		Description
		Lite ⁽¹⁰⁹⁾	Full	
Parameterization registers				
0x0000	VID_PID	RO	RO	Read this register to retrieve the Guard Bands IP product ID. This register always returns 0x6AF7_0231.
0x0004	VERSION	RO	RO	Read this register for the IP version information.
0x0008	LITE_MODE	RO	RO	Read this register to determine if Lite mode is on or off. This register returns 0 when you turn off Lite mode and 1 for when you turn on Lite mode .
0x000C	DEBUG_ENABLED	RO	RO	Read this register to determine if Debug features is on or off.
0x0010	NUM_GB_VALUES	RO	RO	Read this register to determine the number of guard bands that you can set. Equivalent to number of color planes or (number of color planes + 1) if you turn on 4:2:2 support .
0x0014 to 0x011F	-	-	-	Unused.
Control and debug registers				
For more information, refer to <i>Control Packets</i>				
0x0120	IMG_INFO_WIDTH	RW	RO	When you turn on Lite , the expected width of the incoming video fields. When you turn off Lite , the received width in image information packets.
0x0124	IMG_INFO_HEIGHT	RW	RO	When you turn on Lite , the expected height of the incoming video fields. When you turn off Lite , the received height in image information packets.
0x0128	IMG_INFO_INTERLACE	RW	RO	When you turn on Lite , the expected interlace information of the incoming video fields. When you turn off Lite , the received interlace information in image information packets.
0x012C	Reserved	RW	RO	Unused.
0x0130	IMG_INFO_COLORSPACE	RW	RO	When you turn on Lite , the expected color space of the incoming video fields. When you turn off Lite , the received color space in image information packets.
0x0134	IMG_INFO_SUBSAMPLING	RW	RO	When you turn on Lite , the expected chroma sub-sampling of the incoming video fields. When you turn off Lite , the received chroma sub-sampling in image information packets.
0x0138	IMG_INFO_COSITING	RW	RO	When you turn on Lite , the expected chroma co-siting of the incoming video fields. When you turn off Lite , the received chroma co-siting in image information packets.
0x013C	IMG_INFO_FIELD_COUNT	RO	RO	The received field count field in image information packets.
0x0140	STATUS	RO	RO	Bit 0: Status bit. 1 means the IP is processing a video field, 0 otherwise. When you turn off Lite : Bit 1: Pending register updates bit.
continued...				

⁽¹⁰⁹⁾ When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

Address	Register	Access		Description
		Lite ⁽¹⁰⁹⁾	Full	
				Any writes to the specification registers (0x0148 - 0x0154) cause the IP to raise the <code>pending register updates</code> bit, to indicate outstanding changes to the guard band settings. The IP lowers this bit at the start of the first packet for a new frame (first packet after the EOP) after a write to the <code>COMMIT</code> register.
0x0144	COMMIT	RW	RW	The IP uses this register with the <code>STATUS</code> register <code>Pending register updates</code> bit. Write any value to this register after writing new values to any of the specification registers (0x0148 - 0x0154). The configurable guard bands change to the new guard band specification values at the start of the next video frame after the IP writes to this register. The <code>COMMIT</code> register ensures atomic register updates, avoiding the IP only applying some of the updated register values for the next incoming frame.
Specification registers				
0x0148	GB_0_VALUES	RW	RW	Write to this register to set the lower and upper guard band values for color plane 0.
0x014C	GB_1_VALUES	RW	RW	Write to this register to set the lower and upper guard band values for color plane 1. Unused if <code>NUM_GB_VALUES</code> < 2.
0x0150	GB_2_VALUES	RW	RW	Write to this register to set the lower and upper guard band values for color plane 2. Unused if <code>NUM_GB_VALUES</code> < 3.
0x0154	GB_3_VALUES	RW	RW	Write to this register to set the lower and upper guard band values for color plane 3. Unused if <code>NUM_GB_VALUES</code> < 4.

Register Bit Descriptions

Table 597. VID_PID

Name	Bits	Description
Guard Bands IP vendor ID and product ID	31:0	This register always returns 0x6AF7_0231. 15:0 is the product ID and always returns 0x0231 31:16 is the vendor ID and always returns 0x6AF7

Table 598. VERSION

Name	Bits	Description
Register map version	7:0	Register map version.
IP patch revision	15:8	-
IP update revision	23:16	Updated when the IP version changes.
IP major revision	31:24	Updated when the IP version changes.

⁽¹⁰⁹⁾ When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

Table 599. LITE_MODE

Name	Bits	Description
Lite mode parameterization bit	0	Returns 1 if you turn on lite mode .
Unused	31:1	Unused.

Table 600. DEBUG_ENABLED

Name	Bits	Description
Debug features parameterization bit	0	Returns 1 if you turn on debug features .
Unused	31:1	Unused.

Table 601. NUM_GB_VALUES

Name	Bits	Description
Number of GB values used	3:0	This field indicates number of guard bands that are active. Equivalent to number of color planes or (number of color planes + 1) if you turn on 4:2:2 support .
Unused	31:4	Unused.

Table 602. IMG_INFO_WIDTH

Name	Bits	Description
Width bits	15:0	When you turn on Lite , write to this register to set the expected width of the incoming video fields. When you turn off Lite and turn on Debug features , this register returns the <code>width</code> field from the most recently received.
Unused	31:16	Unused.

Table 603. IMG_INFO_HEIGHT

Name	Bits	Description
Height bits	15:0	When you turn on Lite , write to this register to set the expected height of the incoming video fields. When you turn off Lite and turn on Debug features , this register returns the <code>height</code> field from the most recently received.
Unused	31:16	Unused.

Table 604. IMG_INFO_INTERLACE

Name	Bits	Description
interlaced bits	3:0	When you turn on Lite , write to this register to set the expected interlacing of the incoming video fields. When you turn off Lite and turn on Debug features , this register returns the <code>intlaceNibble</code> field from the most recently received <code>image information</code> packet.
Unused	31:4	Unused.

Table 605. IMG_INFO_COLORSPACE

Name	Bits	Description
Color space code bits	6:0	When you turn on Lite , write to this register to set the expected color space of the incoming video fields.
continued...		

Name	Bits	Description
		When you turn off Lite and turn on Debug features , this register returns the <code>CSP</code> field from the most recently received image information packet.
Unused	31:7	Unused.

Table 606. IMG_INFO_SUBSAMPLING

Name	Bits	Description
Subsampling code bits	1:0	When you turn on Lite , write to this register to set the expected chroma sub-sampling of the incoming video fields. When you turn off Lite and turn on Debug features , this register returns the <code>SUBSA</code> field from the most recently received image information packet.
Unused	31:2	Unused.

Table 607. IMG_INFO_COSITING

Name	Bits	Description
Cosite code bits	1:0	When you turn on Lite , write to this register to set the expected chroma co-siting of the incoming video fields. When you turn off Lite and turn on Debug features , this register returns the <code>COSITE</code> field from the most recently received image information packet.
Unused	31:2	Unused.

Table 608. STATUS

Name	Bits	Description
Status bit	0	1 means configurable guard bands are processing a video field, 0 otherwise.
Pending register updates bit	1	1 means configurable guard bands have pending updates, 0 otherwise.

Table 609. COMMIT

Name	Bits	Description
Commit bits	31:0	Write to any bits to trigger a commit.

Table 610. GB_0_VALUES

Name	Bits	Description
GB_0_LOWER_VALUE	15:0	Write to this register to set the lower guard band value for color plane 0.
GB_0_UPPER_VALUE	31:16	Write to this register to set the upper guard band value for color plane 0.

Table 611. GB_1_VALUES

Name	Bits	Description
GB_1_LOWER_VALUE	15:0	Write to this register to set the lower guard band value for color plane 1. Unused if <code>NUM_GB_VALUES < 2</code> .
GB_1_UPPER_VALUE	31:16	Write to this register to set the upper guard band value for color plane 1. Unused if <code>NUM_GB_VALUES < 2</code> .

Table 612. GB_2_VALUES

Name	Bits	Description
GB_2_LOWER_VALUE	15:0	Write to this register to set the lower guard band value for color plane 2. Unused if NUM_GB_VALUES < 3.
GB_2_UPPER_VALUE	31:16	Write to this register to set the upper guard band value for color plane 2. Unused if NUM_GB_VALUES < 3.

Table 613. GB_3_VALUES

Name	Bits	Description
GB_3_LOWER_VALUE	15:0	Write to this register to set the lower guard band value for color plane 3. Unused if NUM_GB_VALUES < 4.
GB_3_UPPER_VALUE	31:16	Write to this register to set the upper guard band value for color plane 3. Unused if NUM_GB_VALUES < 4.

34.5. Guard Bands IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_guard_bands_regs.h`

Include file: `intel_vvp_guard_bands.h`

Table 614. Guard Bands IP API reference

Name	Description
<code>intel_vvp_guard_bands_init</code>	Initialize the Guard Bands instance
<code>Intel_vvp_core_*</code>	Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i> . Writable when Lite is on. Readable when Lite is off and Debug features is on.
<code>intel_vvp_guard_bands_get_lite_mode</code>	Returns if lite mode is on
<code>intel_vvp_guard_bands_get_debug_enabled</code>	Returns if debug features is on
<code>intel_vvp_guard_bands_get_num_planes</code>	Returns the number of color planes
<code>intel_vvp_guard_bands_is_running</code>	Returns if the IP is producing data
<code>intel_vvp_guard_bands_get_commit_status</code>	Returns if the IP has uncommitted writes
<code>intel_vvp_guard_bands_get_status</code>	Reads the status register
<code>intel_vvp_guard_bands_set_color_min_max</code>	Selects the clipping values for a color plane
<code>intel_vvp_guard_bands_set_min_max</code>	Sets clipping values for all color planes
<code>intel_vvp_guard_bands_get_color_min_max</code>	Retrieves the clipping values for a color plane
<code>intel_vvp_guard_bands_get_min_max</code>	Retrieves clipping values for all color planes
<code>intel_vvp_guard_bands_commit_writes</code>	Commit all outstanding writes

struct intel_vvp_min_max

Members

```
struct {uint16_t min; uint16_t max;} planes[4];
```

Description

Clipping boundary values for each color plane

intel_vvp_guard_bands_init

Prototype

```
int intel_vvp_guard_bands_init(intel_vvp_guard_bands_instance
*instance, intel_vvp_core_base base);
```

Description

Initializes a Guard Bands instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance does not fully initialize and should not be used further by the application if returning a non-zero error code.

Arguments

instance – pointer to the intel_vvp_guard_bands_instance software driver instance structure

base – base address of the register map

Returns

kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product_id does not match with the Guard Bands product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpGuardBandsRegMapVersionErr if the register map is not supported

intel_vvp_guard_bands_get_lite_mode

Prototype

```
bool
intel_vvp_guard_bands_get_lite_mode(intel_vvp_guard_bands_instance
*instance);
```

Description

Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_guard_bands_instance fully initialized

Arguments

instance – pointer to the intel_vvp_guard_bands_instance software driver instance structure

Returns

true if the IP is parameterized in lite mode

intel_vvp_guard_bands_get_debug_enabled

Prototype

```
int
intel_vvp_guard_bands_get_debug_enabled(intel_vvp_guard_bands_instance*instance);
```

Description Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_guard_bands_instance fully initialized.

Arguments instance – pointer to the intel_vvp_guard_bands_instance software driver instance structure

Returns true if the IP is parameterized with debug features enabled

intel_vvp_guard_bands_get_num_planes

Prototype

```
uint8_t
intel_vvp_guard_bands_get_num_planes(intel_vvp_guard_bands_instance*instance);
```

Description Returns the value of the NUM_GB register. The instance must be a valid intel_vvp_guard_bands_instance fully initialized.

Arguments instance – pointer to the intel_vvp_guard_bands_instance software driver instance structure

Returns The number of color planes in the video stream (4:2:2 is three color planes)

intel_vvp_guard_bands_is_running

Prototype

```
bool
intel_vvp_guard_bands_is_running(intel_vvp_guard_bands_instance*instance);
```

Description Reads and returns the running bit of the STATUS register. The instance must be a valid intel_vvp_guard_bands_instance fully initialized.

Arguments instance – pointer to the intel_vvp_guard_bands_instance software driver instance structure

Returns True is the Guard Bands is currently generating an output field, false when between two fields

intel_vvp_guard_bands_get_commit_status

Prototype

```
bool
intel_vvp_guard_bands_get_commit_status(intel_vvp_guard_bands_instance*instance);
```


<i>Description</i>	Reads and returns the commit pending bit of the STATUS register. The instance must be a valid intel_vvp_guard_bands_instance fully initialized and configured in full mode.
<i>Arguments</i>	instance – pointer to the intel_vvp_guard_bands_instance software driver instance structure
<i>Returns</i>	True is the Guard Bands is configured in full mode and if the agent interface contains uncommitted writes.

intel_vvp_guard_bands_get_status

Prototype

```
uint8_t
intel_vvp_guard_bands_get_status(intel_vvp_guard_bands_instance*
instance);
```

<i>Description</i>	Reads and returns the STATUS register. The instance must be a valid intel_vvp_guard_bands_instance fully initialized.
<i>Arguments</i>	instance – pointer to the intel_vvp_guard_bands_instance software driver instance structure
<i>Returns</i>	The value read from the status register

intel_vvp_guard_bands_set_color_min_max

Prototype

```
int intel_vvp_guard_bands_set_color_min_max
(intel_vvp_guard_bands_instance* instance, uint8_t color,
uint16_t min, uint16_t max);
```

<i>Description</i>	Sets the minimum and maximum allowed values, before clipping, for the chosen color plane. The instance must be a valid intel_vvp_guard_bands_instance fully initialized and the color plane must be valid
<i>Arguments</i>	instance – pointer to the intel_vvp_guard_bands_instance software driver instance structure color – the identifier for the color plane needing new clipping boundaries. The valid range is [0..num_color_planes-1] where num_color_planes is returned by intel_vvp_guard_bands_get_num_planes(...) min, max – new clipping boundaries
<i>Returns</i>	kIntelVvpCoreOk (0) for success, negative error code otherwise kIntelVvpCoreInstanceErr if the instance is invalid kIntelVvpGuardBandsParameterErr if the color plane is invalid

intel_vvp_guard_bands_set_min_max

Prototype

```
int intel_vvp_guard_bands_set_min_max
(intel_vvp_guard_bands_instance* instance, const
intel_vvp_min_max* min_max);
```

Description Sets the minimum and maximum allowed values, before clipping, for all color planes. The instance must be a valid intel_vvp_guard_bands_instance fully initialized.

Arguments instance – pointer to the intel_vvp_guard_bands_instance software driver instance structure
min_max – new clipping boundaries

Returns kIntelVvpCoreOk (0) for success, negative error code otherwise
kIntelVvpCoreInstanceErr if the instance is invalid
kIntelVvpCoreNullPtrErr if min_max is the NULL pointer

intel_vvp_guard_bands_get_color_min_max

Prototype

```
int intel_vvp_guard_bands_get_color_min_max
(intel_vvp_guard_bands_instance* instance, uint8_t color,
uint16_t *min, uint16_t *max);
```

Description Retrieves the minimum and maximum allowed values, before clipping, for the chosen color plane. The instance must be a valid intel_vvp_guard_bands_instance, fully initialized, configured with debug enabled, and the color plane must be valid

Arguments instance – pointer to the intel_vvp_guard_bands_instance software driver instance structure
color – the identifier for the color plane. The valid range is [0..num_color_planes-1] where num_color_planes is returned by intel_vvp_guard_bands_get_num_planes(...)
min, max – Valid pointers to uint16_t variables to store the results

Returns kIntelVvpCoreOk (0) for success, negative error code otherwise
kIntelVvpCoreNullPtrErr if min or max are the NULL pointer
kIntelVvpCoreInstanceErr if the instance is invalid
kIntelVvpGuardBandsParameterErr if the color plane is invalid

intel_vvp_guard_bands_get_min_max

Prototype

```
int intel_vvp_guard_bands_get_min_max  
(intel_vvp_guard_bands_instance* instance, intel_vvp_min_max*  
min_max);
```

Description

Retrieves the minimum and maximum allowed values, before clipping, for all color planes. The instance must be a valid intel_vvp_guard_bands_instance fully initialized and configured with debug enabled

Arguments

instance – pointer to the intel_vvp_guard_bands_instance software driver instance structure

min_max – returned clipping boundaries

Returns

kIntelVvpCoreOk (0) for success, negative error code otherwise

kIntelVvpCoreInstanceErr if the instance is invalid

kIntelVvpCoreNullPtrErr if min_max is the NULL pointer

intel_vvp_guard_bands_commit_writes

Prototype

```
int  
intel_vvp_guard_bands_commit_writes(intel_vvp_guard_bands_instance*  
instance)
```

Description

Commits all pending writes before starting the next output field. The instance must be valid and parameterized in full mode.

Arguments

instance – pointer to the intel_vvp_guard_bands_instance software driver instance structure

Returns

kIntelVvpCoreOk (0) for success, negative error code otherwise

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

35. Histogram Statistics IP

35.1. About the Histogram Statistics IP

The Histogram Statistics IP operates on RGB images, analyzes the pixel values for each frame, and collects data to form a luma histogram. You can use this IP to collect statistics of luminance distribution and adjust sensor exposure settings accordingly.

The IP only supports lite variants. For more information about full and lite variants, refer to the *Altera Streaming Video Protocol Specification*

Related Information

[Altera Streaming Video Protocol Specification](#)

35.1.1. Histogram Statistics IP Performance and Resources

Altera provides resource and utilization data for guidance only. The IP resource utilization depends on the device family, compilation tool version, compilation seed randomization and other parameters.

Device	Pixel in Parallel	Bits per Color Sample	Max Resolution	ALM	M20k	DSP	f _{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	1	10	2k	682	3	2	679
	1	16	2k	769	3	2	690
	4	10	8k	1416	17	6	683
	4	16	8k	1789	17	6	689
Arria 10 speed grade 1 (10AS066H1F34E1HG)	1	10	2k	600	3	2	411
	1	16	2k	666	3	2	429
	2	10	4k	820	9	3	414
	2	16	4k	964	9	3	410
Cyclone 10 GX speed grade 5 (10CX220YF672E5G)	1	10	2k	601	3	2	378
	1	16	2k	668	3	2	370
Stratix 10 speed grade 1 (1SX280LN2F43E1VG)	1	10	2k	724	3	2	673
	1	16	2k	812	3	2	669
	2	10	4k	1020	9	3	678
	2	16	4k	1224	9	3	665

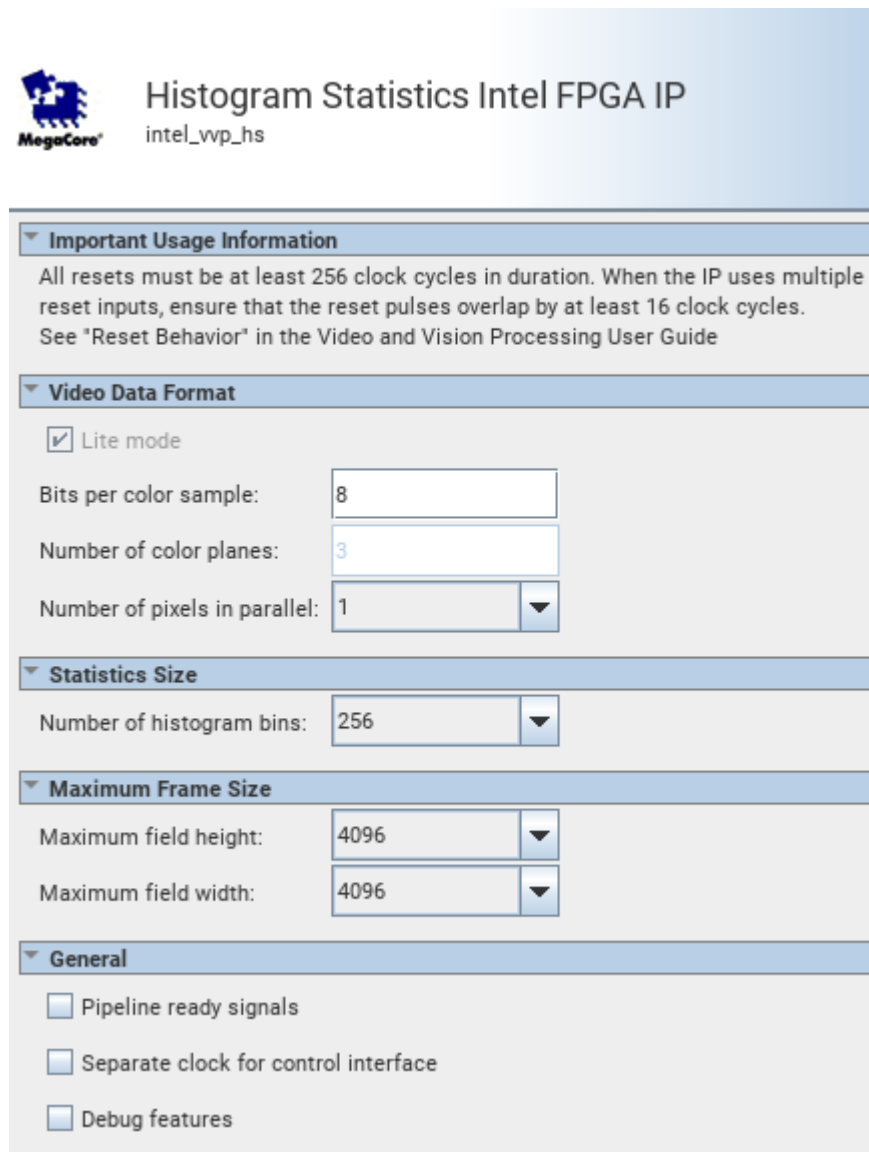
35.2. Histogram Statistics IP Parameters

The IP offers run-time and compile-time parameters.

Table 615. Histogram Statistics IP Parameters

Parameter	Values	Description
Video Data Format		
Lite mode	On	The IP only supports lite mode
Bits per color sample	8 to 16	Select the number of bits per color sample.
Number of color planes	3	The IP only supports 3 color planes
Number of pixels in parallel	1, 2, 4 or 8	Select the number of pixels in parallel.
Maximum Frame Size		
Maximum field height	1k, 2k, 4k, 8k, 16k	Specify the maximum height of incoming frames
Maximum field width	2k, 4k, 8k, 16k	Specify the maximum width of incoming frames
Statistics size		
Number of histogram bins	32, 64, 128, 256	Specify the number of histogram bins of each histogram statistics.
General		
Pipeline ready signal	On or off	Turn on to add extra pipeline registers to the AXI4-S <code>tready</code> signals
Separate clock for control interface	On or off	Turn on for a separate clock for the control agent interface
Debug features	On or off	Turn on to read back frame information registers and debugging information registers via the control agent interface.

Figure 101. Histogram Statistics IP GUI



Histogram Statistics Intel FPGA IP
intel_vvp_hs

Important Usage Information
All resets must be at least 256 clock cycles in duration. When the IP uses multiple reset inputs, ensure that the reset pulses overlap by at least 16 clock cycles. See "Reset Behavior" in the Video and Vision Processing User Guide

Video Data Format

☒ Lite mode

Bits per color sample: 8

Number of color planes: 3

Number of pixels in parallel: 1

Statistics Size

Number of histogram bins: 256

Maximum Frame Size

Maximum field height: 4096

Maximum field width: 4096

General

☐ Pipeline ready signals

☐ Separate clock for control interface

☐ Debug features

35.3. Histogram Statistics IP Interfaces

Table 616. Histogram Statistics Clock and Reset Interfaces

Name	Direction	Width	Description
main_clock_clk	In	1	AXI4-S processing clock.
main_reset_rst	In	1	AXI4-S processing reset.
agent_clock_clk	In	1	Clock for the memory-mapped control agent interface. Only if you turn on Separate clock for control interface .
agent_reset_reset	In	1	Reset for the memory-mapped control agent interface. Only if you turn on Separate clock for control interface .

Table 617. Histogram Statistics Control Interfaces

Name	Direction	Width	Description
av_mm_control_agent_address	In	(110)	Avalon memory-mapped agent address
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write.
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data.
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable.
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read.
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data.
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read.
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request.

Table 618. Histogram Statistics Altera Streaming Video Interfaces

Name	Direction	Width	Description
Input			
axi4s_vid_in_tdata	In	(111)	AXI4-S data in.
axi4s_vid_in_tvalid	In	1	AXI4-S data valid.
axi4s_vid_in_tuser	In	(112)	[0] AXI4-S start of video frame. [N-1:1] Unused.
axi4s_vid_in_tlast	In	1	AXI4-S end of packet.
axi4s_vid_in_tready	Out	1	AXI4-S data ready.
Output			
axi4s_vid_out_tdata	Out	(111)	AXI4-S data out.
<i>continued...</i>			

(110) The width is 8, 8, 9 and 10 bits for **Number of histogram bins** of 32, 64, 128 and 256, respectively.

(111) The equation gives all tdata widths sizes in these interfaces: pixels in parallel × max (round (bits per color sample × number of color planes, 8), 16)

(112) This equation gives all tuser widths sizes in these interfaces: N = ceil (tdata width / 8)

Name	Direction	Width	Description
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid.
axi4s_vid_out_tuser	Out	(112)	[0] AXI4-S start of video frame. [N-1:1] Unused.
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet.
axi4s_vid_out_tready	In	1	AXI4-S data ready

Related Information

Video and Vision Processing IP Interfaces on page 30

35.4. Histogram Statistics IP Registers

Each register is either read-only (RO) or read-write (RW).

Table 619. Histogram Statistics Registers

In the software API the register names appear with a prefix of `INTEL_VVP`, `INTEL_VVP_CORE`, or `INTEL_VVP_HS` as appropriate and with an optional `REG` suffix.

Address	Register	Access		Description
		Lite ⁽¹¹³⁾	Full	
Parameterization registers				
0x0000	VID_PID	RO	N/A	Read this register to retrieve the ID of the IP. This register always returns 0x6FA7_017B.
0x0004	VERSION	RO	N/A	Read this register for the IP version information.
0x0008	LITE_MODE	RO	N/A	Read this register to determine if Lite mode is on. This register always returns 1.
0x000C	DEBUG_ENABLED	RO	N/A	Read this register to determine if Debug features are on. This register returns 0 for off and 1 for on.
0x0010	BPS_IN	RO	N/A	Read this register to determine the Bits per color symbol used for the input streaming video interface.
0x0014	BPS_OUT	RO	N/A	Read this register to determine the Bits per color symbol used for the output streaming video interface. The value of this register reads the same for its input counterpart.
0x0018	NUM_COLOR_IN	RO	N/A	Read this register to determine the Number of color planes for the input streaming video interface.
0x001C	NUM_COLOR_OUT	RO	N/A	Read this register to determine the Number of color planes for the output streaming video interface. The value of this register reads the same for its input counterpart.
continued...				

⁽¹¹³⁾ Registers are RW only if you also turn on **Debug features**, otherwise they are WO.

Address	Register	Access		Description
		Lite ⁽¹¹³⁾	Full	
0x0020	PIP	RO	N/A	Read this register to determine the Number of pixels in parallel .
0x0024	MAX_WIDTH	RO	N/A	Read this register to determine the Maximum field width .
0x0028	MAX_HEIGHT	RO	N/A	Read this register to determine the Maximum field height .
0x002C	NUM_HIST_BINS	RO	N/A	Read this register to determine the Number of histogram bins .
0x0030 to 0x011F	-	-	-	Reserved
Control, debugging, and statistics registers				
0x0120	IMG_INFO_WIDTH	RW	N/A	The expected width of the incoming video fields.
0x0124	IMG_INFO_HEIGHT	RW	N/A	The expected height of the incoming video fields.
0x0128 to 0x013F	-	-	-	Reserved
0x0140	STATUS	RO	N/A	Read this register for information about the IP status.
0x0144	FRAME_STATS	RO	N/A	Read this register for some frame statistics.
0x0148	COMMIT	RW	N/A	Write any value to this register to submit changes to the control and region of interest registers. A pending commit request does not block Freeze Statistics Request of the CONTROL register, which the IP samples continuously.
0x014C	CONTROL	RW	N/A	Control bits and fields of the IP.
0x0150	H_START	RW	N/A	Region of interest horizontal start value.
0x0154	V_START	RW	N/A	Region of interest vertical start value.
0x0158	H_END	RW	N/A	Region of interest horizontal end value.
0x015C	V_END	RW	N/A	Region of interest vertical end value.
0x0160 to 0x01FF	-	-	-	Reserved.
0x0200 to 0x0200 + 4 x NUM_HIST_BINS - 1	FRAME_STATISTICS	RO	N/A	Histogram statistics calculated across the whole frame. The IP stores Number of histogram bins linearly starting from bin index 0 across the range of the registers.
0x0200 + 4 x NUM_HIST_BINS to 0x0200 + 8 x NUM_HIST_BINS - 1	ROI_STATISTICS	RO	N/A	Histogram statistics calculated inside the region of interest. The IP stores Number of histogram bins linearly starting from bin index 0 across the range of the registers.
0x0200 + 8 x NUM_HIST_BINS to end of memory	-	-	-	Reserved or not implemented. Reserved if the address is within the range of the memory-mapped control interface, otherwise not implemented

(113) Registers are RW only if you also turn on **Debug features**, otherwise they are WO.

Register Bit Descriptions

Table 620. STATUS

Name	Bits	Description
Reserved	31:3	Reserved.
Stats are Frozen.	2	The IP sets this bit to indicate it stopped updating the statistics.
Commit	1	Pending commit
Running	0	When 1, the IP is processing data..

Table 621. FRAME_STATS

Name	Bits	Description
Reserved	31:8	Reserved.
Checksum	7:0	A simple checksum of the frame.

Table 622. CONTROL

Name	Bits	Description
Freeze Statistics Request	31	Set this bit to 1 for the IP to start sampling a new set of histogram and frame statistics. The IP lowers the Stats are Frozen bit of the STATUS register as a response, and you need to poll Stats are Frozen bit until it is 1 before reding the statistics. The IP expects you to keep Freeze Statistics Request 1 until it raises Stats are Frozen. Set Freeze Statistics Request to 0 before starting over with a new sampling request. Deviation from this flow might result in unexpected IP behavior.
Reserved	30:0	Reserved. Write 0.

Table 623. H_START

Name	Bits	Description
Reserved	31:16	Reserved.
Position	15:0	Horizontal position of the first pixel in the region of interest. The IP starts counts the pixels from index 0.

Table 624. V_START

Name	Bits	Description
Reserved	31:16	Reserved.
Position	15:0	Vertical position of the first line in the region of interest. The IP starts counts the pixels from index 0.

Table 625. H_END

Name	Bits	Description
Reserved	31:16	Reserved.
Position	15:0	Horizontal position of the last pixel in the region of interest. The IP starts counts the pixels from index 0.

Table 626. V_END

Name	Bits	Description
Reserved	31:16	Reserved.
Position	15:0	Vertical position of the last line in the region of interest. The IP starts counts the pixels from index 0.

35.5. Histogram Statistics IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_hs_regs.h`

Include file: `intel_vvp_hs.h`

Table 627. Histogram Statistics IP Software API

Name	Description
<code>intel_vvp_hs_init</code>	Initialize an IP instance
<code>intel_vvp_core_*</code>	Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i>
<code>intel_vvp_hs_get_lite_mode</code>	Returns if Lite mode is on
<code>intel_vvp_hs_get_debug_enabled</code>	Returns if Debug features is on
<code>intel_vvp_hs_get_bits_per_sample_in</code>	Returns the number of bits per color sample for the streaming input interface
<code>intel_vvp_hs_get_bits_per_sample_out</code>	Returns the number of bits per color sample value for the streaming output interface
<code>intel_vvp_hs_get_num_color_planes_in</code>	Returns the number of color planes of the streaming input interface
<code>intel_vvp_hs_get_num_color_planes_out</code>	Returns the number of color planes of the streaming output interface
<code>intel_vvp_hs_get_pixels_in_parallel</code>	Returns the pixels in parallel of the streaming input and output interfaces
<code>intel_vvp_hs_get_max_width</code>	Returns the maximum number of pixels that the IP supports on the horizontal dimension of an image or video frame
<code>intel_vvp_hs_get_max_height</code>	Returns the maximum number of pixels that the IP supports on the vertical dimension of an image or video frame
<code>intel_vvp_hs_get_num_hist_bins</code>	Returns the number of histogram bins
<code>intel_vvp_hs_is_running</code>	Reads if the IP is running
<code>intel_vvp_hs_commit_is_pending</code>	Reads if the IP has uncommitted writes
<code>intel_vvp_hs_stats_are_frozen</code>	Reads if the IP finished collecting the statistics
<code>intel_vvp_hs_get_status</code>	Reads the status register
<code>intel_vvp_hs_get_frame_stats</code>	Reads the frame statistics register
<i>continued...</i>	

Name	Description
intel_vvp_hs_commit	Commit pending writes
intel_vvp_hs_get_freeze_stats_request	Reads the freeze statistics request bit from the status register
intel_vvp_hs_set_freeze_stats_request	Writes the freeze statistics request bit of the status register
intel_vvp_hs_get_h_start	Reads the horizontal starting coordinate of the region of interest
intel_vvp_hs_set_h_start	Writes the horizontal starting coordinate of the region of interest
intel_vvp_hs_get_v_start	Reads the vertical starting coordinate of the region of interest
intel_vvp_hs_set_v_start	Writes the vertical starting coordinate of the region of interest
intel_vvp_hs_get_h_end	Reads the horizontal ending coordinate of the region of interest
intel_vvp_hs_set_h_end	Writes the horizontal ending coordinate of the region of interest
intel_vvp_hs_get_v_end	Reads the vertical ending coordinate of the region of interest
intel_vvp_hs_set_v_end	Writes the vertical ending coordinate of the region of interest
intel_vvp_hs_read_frame_hist_bin	Reads a single bin from the histogram statistics of the whole frame
intel_vvp_hs_read_roi_hist_bin	Reads a single bin from the histogram statistics of the region of interest
intel_vvp_hs_read_frame_hist_array	Reads all histogram statistics of the whole frame
intel_vvp_hs_read_roi_hist_array	Reads all histogram statistics of the region of interest

intel_vvp_hs_init

Prototype

```
int
intel_vvp_hs_init(intel_vvp_hs_instance* instance,
intel_vvp_core_base base);
```

Description Initialize an IP instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

base - base address of the register map

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7)

kIntelVvpCorePidErr if the product_id does not match with the expected product ID (0x017B)

kIntelVvpHsRegMapVersionErr if the register map is not supported

intel_vvp_hs_get_lite_mode

Prototype

```
bool  
intel_vvp_hs_get_lite_mode(intel_vvp_hs_instance* instance);
```

Description Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns true if the IP is parameterized in lite mode

intel_vvp_hs_get_debug_enabled

Prototype

```
bool  
intel_vvp_hs_get_debug_enabled(intel_vvp_hs_instance* instance);
```

Description Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns true if the IP is parameterized with debug features enabled

intel_vvp_hs_get_bits_per_sample_in

Prototype

```
uint8_t  
intel_vvp_hs_get_bits_per_sample_in(intel_vvp_hs_instance* instance);
```

Description Returns the value of the BPS_IN register. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns the input bits per color sample parameter used to generate the IP

intel_vvp_hs_get_bits_per_sample_out

Prototype

```
uint8_t
intel_vvp_hs_get_bits_per_sample_out(intel_vvp_hs_instance*
instance);
```

Description Returns the value of the BPS_OUT register. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns the output bits per color sample parameter used to generate the IP

intel_vvp_hs_get_num_color_planes_in

Prototype

```
uint8_t
intel_vvp_hs_get_num_color_planes_in(intel_vvp_hs_instance*
instance);
```

Description Returns the value of the NUM_COLOR_IN register. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns the input number of color planes parameter used to generate the IP

intel_vvp_hs_get_num_color_planes_out

Prototype

```
uint8_t
intel_vvp_hs_get_num_color_planes_out(intel_vvp_hs_instance*
instance);
```

Description Returns the value of the NUM_COLOR_OUT register. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns the output number of color planes parameter used to generate the IP

intel_vvp_hs_get_pixels_in_parallel

Prototype

```
uint8_t  
intel_vvp_hs_get_pixels_in_parallel(intel_vvp_hs_instance*  
instance);
```

Description Returns the value of the PIXELS_IN_PARALLEL register. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns the pixels in parallel parameter used to generate the IP

intel_vvp_hs_get_max_width

Prototype

```
uint32_t  
intel_vvp_hs_get_max_width(intel_vvp_hs_instance* instance);
```

Description Returns the value of the MAX_WIDTH register. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns the maximum field width parameter used to generate the IP

intel_vvp_hs_get_max_height

Prototype

```
uint32_t  
intel_vvp_hs_get_max_height(intel_vvp_hs_instance* instance);
```

Description Returns the value of the MAX_HEIGHT register. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns the maximum field height parameter used to generate the IP

intel_vvp_hs_get_num_hist_bins

Prototype

```
uint16_t
intel_vvp_hs_get_num_hist_bins(intel_vvp_hs_instance*
instance);
```

Description Returns the value of the NUM_HIST_BINS register. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns the number of histogram bins parameter used to generate the IP

intel_vvp_hs_is_running

Prototype

```
bool
intel_vvp_hs_is_running(intel_vvp_hs_instance* instance);
```

Description Returns the value of the IP Running bit from the STATUS register. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns the running bit from the status register

intel_vvp_hs_commit_is_pending

Prototype

```
bool
intel_vvp_hs_commit_is_pending(intel_vvp_hs_instance*
instance);
```

Description Returns the value of the commit pending bit from the STATUS register. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns true if the agent interface has pending writes from a previous commit request

intel_vvp_hs_stats_are_frozen

Prototype

```
bool  
intel_vvp_hs_stats_are_frozen(intel_vvp_hs_instance* instance);
```

Description Returns the value of the statistics are frozen bit from the STATUS register. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns the statistics are frozen bit from the status register

intel_vvp_hs_get_status

Prototype

```
uint32_t  
intel_vvp_hs_get_status(intel_vvp_hs_instance* instance);
```

Description Returns the value of the STATUS register. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns the value of the status register

intel_vvp_hs_get_frame_stats

Prototype

```
int  
intel_vvp_hs_get_frame_stats(intel_vvp_hs_instance* instance,  
uint32_t* stats_out);
```

Description Reads the value of the FRAME_STATS register. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

stats_out - pointer of a variable to return the statistics register value

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpHsFreezePendingErr if the last freeze request is pending
kIntelVvpHsPointerErr if stats_out is a null pointer

intel_vvp_hs_commit

Prototype

```
int
intel_vvp_hs_commit(intel_vvp_hs_instance* instance);
```

Description Commits all pending writes before processing the next field. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

intel_vvp_hs_get_freeze_stats_request

Prototype

```
bool
intel_vvp_hs_get_freeze_stats_request(intel_vvp_hs_instance*
instance);
```

Description Returns the value of the freeze statistics request bit from the CONTROL register. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns the value of the freeze statistics request bit form the control register

intel_vvp_hs_set_freeze_stats_request

Prototype

```
int
intel_vvp_hs_set_freeze_stats_request(intel_vvp_hs_instance*
instance, bool freeze_stats);
```

Description Writes to the freeze statistics request bit of the CONTROL register. The instance must be a valid intel_vvp_hs_instance fully initialized.

- Arguments**
- instance - pointer to the intel_vvp_hs_instance software driver instance structure
 - freeze_stats - new value of the freeze statistics request bit
- Returns**
- kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
 - kIntelVvpCoreInstanceErr if the instance is a null pointer

intel_vvp_hs_get_h_start

Prototype

```
uint16_t
intel_vvp_hs_get_h_start(intel_vvp_hs_instance* instance);
```

- Description**
- Returns the horizontal starting coordinate of the region of interest. The instance must be a valid intel_vvp_hs_instance fully initialized.
- Arguments**
- instance - pointer to the intel_vvp_hs_instance software driver instance structure
- Returns**
- the value of the horizontal starting coordinate of the region of interest

intel_vvp_hs_set_h_start

Prototype

```
int
intel_vvp_hs_set_h_start(intel_vvp_hs_instance* instance,
uint16_t h_start);
```

- Description**
- Writes the horizontal starting coordinate of the region of interest. The instance must be a valid intel_vvp_hs_instance fully initialized.
- Arguments**
- instance - pointer to the intel_vvp_hs_instance software driver instance structure
 - h_start - new value of the horizontal starting coordinate of the region of interest
- Returns**
- kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
 - kIntelVvpCoreInstanceErr if the instance is a null pointer
 - kIntelVvpHsValueErr if the value of h_start is out of range
 - kIntelVvpHsCommitPendingErr if a previous commit request is pending

intel_vvp_hs_get_v_start

Prototype

```
uint16_t
intel_vvp_hs_get_v_start(intel_vvp_hs_instance* instance);
```

Description Returns the vertical starting coordinate of the region of interest. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns the value of the vertical starting coordinate of the region of interest

intel_vvp_hs_set_v_start

Prototype

```
int
intel_vvp_hs_set_v_start(intel_vvp_hs_instance* instance,
uint16_t v_start);
```

Description Writes the vertical starting coordinate of the region of interest. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

v_start - new value of the vertical starting coordinate of the region of interest

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpHsValueErr if the value of v_start is out of range

kIntelVvpHsCommitPendingErr if a previous commit request is pending

intel_vvp_hs_get_h_end

Prototype

```
uint16_t
intel_vvp_hs_get_h_end(intel_vvp_hs_instance* instance);
```

Description Returns the horizontal ending coordinate of the region of interest. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns the value of the horizontal ending coordinate of the region of interest

intel_vvp_hs_set_h_end

Prototype

```
int
intel_vvp_hs_set_h_end(intel_vvp_hs_instance* instance,
uint16_t h_end);
```

Description Writes the horizontal ending coordinate of the region of interest. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

h_end - new value of the horizontal ending coordinate of the region of interest

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpHsValueErr if the value of h_end is out of range

kIntelVvpHsCommitPendingErr if a previous commit request is pending

intel_vvp_hs_get_v_end

Prototype

```
uint16_t
intel_vvp_hs_get_v_end(intel_vvp_hs_instance* instance);
```

Description Returns the vertical ending coordinate of the region of interest. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

Returns the value of the vertical ending coordinate of the region of interest

intel_vvp_hs_set_v_end

Prototype

```
int
intel_vvp_hs_set_v_end(intel_vvp_hs_instance* instance,
uint16_t v_end);
```

Description Writes the vertical ending coordinate of the region of interest. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

v_end - new value of the vertical ending coordinate of the region of interest

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpHsValueErr if the value of v_end is out of range

kIntelVvpHsCommitPendingErr if a previous commit request is pending

intel_vvp_hs_read_frame_hist_bin

Prototype

```
int
intel_vvp_hs_read_frame_hist_bin(intel_vvp_hs_instance*
instance, uint32_t* data,
uint16_t addr);
```

Description Reads the value of a single bin from the histogram statistics memory of the whole frame. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments instance - pointer to the intel_vvp_hs_instance software driver instance structure

data - pointer of a variable to return the value of the histogram bin

addr - address of the histogram bin

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpHsPointerErr if data is a null pointer

kIntelVvpHsFreezePendingErr if the last freeze request is pending

kIntelVvpHsOutOfBoundsErr if the address is out of range

intel_vvp_hs_read_roi_hist_bin

Prototype

```
int
intel_vvp_hs_read_roi_hist_bin(intel_vvp_hs_instance*
instance, uint32_t* data,
uint16_t addr);
```

Description Reads the value of a single bin from the histogram statistics memory of the region of interest. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments

instance - pointer to the intel_vvp_hs_instance software driver instance structure

data - pointer of a variable to return the value of the histogram bin

addr - address of the histogram bin

Returns

kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpHsPointerErr if data is a null pointer

kIntelVvpHsFreezePendingErr if the last freeze request is pending

kIntelVvpHsOutOfBoundsErr if the address is out of range

intel_vvp_hs_read_frame_hist_array

Prototype

```
int
intel_vvp_hs_read_frame_hist_array(intel_vvp_hs_instance*
instance, uint32_t* data_array);
```

Description Reads the histogram statistics memory of the whole frame into an array. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments

instance - pointer to the intel_vvp_hs_instance software driver instance structure

data_array - pointer of an array to return the value of all histogram bins

Returns

kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpHsPointerErr if data_array is a null pointer

kIntelVvpHsFreezePendingErr if the last freeze request is pending

intel_vvp_hs_read_roi_hist_array

Prototype

```
int
intel_vvp_hs_read_roi_hist_array(intel_vvp_hs_instance*
instance, uint32_t* data_array);
```

Description Reads the histogram statistics memory of the region of interest into an array. The instance must be a valid intel_vvp_hs_instance fully initialized.

Arguments

- instance - pointer to the intel_vvp_hs_instance software driver instance structure
- data_array - pointer of an array to return the value of all histogram bins

Returns

- kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
- kIntelVvpCoreInstanceErr if the instance is a null pointer
- kIntelVvpHsPointerErr if data_array is a null pointer
- kIntelVvpHsFreezePendingErr if the last freeze request is pending

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

36. Interlacer IP

36.1. About the Interlacer IP

The Interlacer IP converts streams of progressive frames into streams of alternating F0 and F1 fields by discarding either the odd lines (F0) or the even lines (F1). The output field rate is consequently equal to the input frame rate. The Interlacer IP is compatible with both the lite and full variants of the Altera Streaming Video interface protocol.

You can turn on an Avalon memory-mapped control agent interface to control the behavior of the Interlacer IP at run time. You can edit settings in the register map to turn interlacing on and off (progressive passthrough), change the output field order (F0 or F1 first), and reset the interlacing sequence.

36.1.1. Interlacer IP Performance and Resources

The IP resources depend on the device family, bits per sample, pixels in parallel, number of color planes.

Table 628. Interlacer IP Performance and Resources

The table shows ALM usage and f_{MAX} for an example parameterization of 10 bits per color sample, 3 color planes, 2 pixels in parallel

Target device	ALMs	f_{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	519	890
Arria 10 speed grade 1 (10AS066H1F34E1HG)	450	645
Cyclone 10 GX speed grade 5 (10CX220YF78015G)	451	645
Stratix 10 GX speed grade 1 (1SX280LN2F43E1VG)	526	730

36.2. Interlacer IP Parameters

The IP offers compile-time parameters.

Parameter	Values	Description
Video data format		
Lite mode	On or off	Turn on for the lite variant of the Altera Streaming Video Protocol
Bits per color sample	8 to 16	Select the number of bits per color sample
Number of color planes	1 to 4	Select the number of color planes per pixel
Number of pixels in parallel	1 to 8	Select the number of pixels in parallel at the input and output interfaces
<i>continued...</i>		

Parameter	Values	Description
Interlace settings		
Send F1 first	On or off	Turn on to begin output with an F1 field after any reset to the interlace sequence. If you turn on Memory mapped control interface , you set this behavior via the register map and the parameter is not used.
Override of interlace sequence from image information packet	On or off	Turn on to allow override of the default interlace sequence if the interlace nibble in the image information packet indicates that the incoming frame was created by deinterlacing original interlaced content (full protocol variant only). If you select Memory mapped control interface , you set this behavior via the register map and the IP does not use this parameter.
Control settings		
Memory mapped control interface	On or off	Turn on for the Avalon memory-mapped control agent interface and to allow run-time configuration via the register map. The Avalon memory-mapped control agent interface is mandatory in lite mode
General		
Pipeline ready signals	On or off	Turn on to add extra pipeline registers to the AXI4-S <i>Tready</i> signals
Debug features	On or off	Turn on to read back frame information registers (full variants only) and writeable registers via the control agent interface
Separate clock for control interface	On or off	Turn on to add a separate clock for the control agent interface

36.3. Interlacer IP Functional Description

The IP converts input sequences of progressive frames into sequences of alternating F0 and F1 fields. F0 fields contain the even indexed lines from the progressive frame and F1 fields contain the odd indexed lines. Each progressive frame is converted into a single output field, either F0 or F1. The IP has no option to buffer the input frame and output both fields. If the Interlacer IP receives content that is already interlaced, the incoming interlaced fields propagate to the output unaltered.

It may not be appropriate for the interlacer to produce an interlaced sequence according to the original interlaced history for the progressive frames. For example, if frame rate conversion occurs after deinterlacing and before the interlacer, the rate conversion may destroy the original sequence of alternating F0 and F1 fields. You can set a parameter for this feature if you do not turn on **Memory-mapped control interface** via the register map, or via a register map setting if **Memory-mapped control interface** is on. This feature is only available when using the full variant of the Altera Streaming Video Protocol as image information packets make it work. Image information packets are not available with the lite variant.

If you do not turn on run-time control of the IP through the register map, the IP converts all progressive input frames to interlaced fields. After any change to the height, width, or interlace identifier specified in the incoming image information packets, the IP resets the sequence of F0 and F1 fields at the output. The **Send F1 first** parameter sets whether F0 or F1 is sent first after any reset. If you turn on run-time control via the register map, you can select whether the output sequence restarts with an F0 or an F1 field at run time, and the parameter is unused. Run-time control also allows you to turn off interlacing and pass through the progressive frames unaltered. If you turn on Lite mode, you must set the image information values (input frame width, input frame height and input interlace nibble) via the register map. Any

edits to these registers cause a reset of the output interlacing sequence. Similarly, changes to these values in the incoming image information packets cause a reset with the full variant of the protocol.

Override of interlace sequence from image information packet

The interlaced nibble field in the Altera Streaming Video image information packets indicates the progressive or interlaced format for each video field.

Table 629. Interfaced Nibble Frame Format

The table shows the format specified for each value of this field.

Interlaced nibble	Frame format
0	Progressive frame, deinterlaced from an F0 field
1	Progressive frame, deinterlaced from an F1 field
2	Progressive frame
3	Progressive frame
4	Progressive frame, deinterlaced from an F0 field
5	Progressive frame, deinterlaced from an F1 field
6	Progressive frame
7	Progressive frame
8	Interlaced F0 field, paired with the F1 field preceding it
9	Interlaced F0 field, paired with the F1 field following it
10	Interlaced F0 field, pairing unknown
11	Interlaced F0 field, pairing none
12	Interlaced F1 field, paired with the F0 field following it
13	Interlaced F1 field, paired with the F0 field preceding it
14	Interlaced F1 field, pairing unknown
15	Interlaced F1 field, pairing none

By default, the IP passes through fields preceded by image information packets that specify interlaced unaltered. The IP converts progressive frames to an alternating sequence of F0 and F1 fields. However, if a progressive frame has been created as a result of deinterlacing original interlaced content, you may want the interlacer to restore this original interlaced content. Interlace nibble values 0, 1, 4 and 5 allow a system to communicate the original interlaced format of a progressive frame to the interlacer so that this process can be reliably implemented.

If you do not want the interlacer to process any original interlaced history for the progressive frames, the IP does not preserve the original sequence of alternating F0 and F1. For example, if the applies frame fate conversion to progressive frames. You can set a parameter for this feature if you do not turn on **Memory-mapped control interface** via the register map, or via a register map setting if **Memory-mapped control interface** is on. This feature is only available when using the full variant of the Altera Streaming Video Protocol as image information packets make it work. Image information packets are not available with the lite variant.

36.3.1. Interlacer IP Interfaces

Table 630. Interlacer IP Interfaces

Name	Direction	Width	Description
Clocks and resets			
main_clock_clk	Input	1	AXI4-S processing clock.
main_reset_rst	Input	1	AXI4-S processing reset.
agent_clock_clk	Input	1	Clock for the Avalon memory-mapped control agent interface. Only if you turn on Separate clock for control interface .
agent_reset_rst	Input	1	Reset for the Avalon memory-mapped control agent interface. Only if you turn on Separate clock for control interface .
Control interfaces			
av_mm_control_agent_address	Input	7	Avalon memory-mapped agent address
av_mm_control_agent_write	Input	1	Avalon memory-mapped agent write
av_mm_control_agent_writedata	Input	32	Avalon memory-mapped agent write data
av_mm_control_agent_byteenable	Input	4	Avalon memory-mapped agent byte enable
av_mm_control_agent_read	Input	1	Avalon memory-mapped agent read
av_mm_control_agent_readdata	Output	32	Avalon memory-mapped agent read data
av_mm_control_agent_readdatavalid	Output	1	Avalon memory-mapped agent read
av_mm_control_agent_waitrequest	Output	1	Avalon memory-mapped agent wait request
Altera Streaming Video interfaces			
axi4s_vid_in_tdata	Input	(114)	AXI4-S data in
axi4s_vid_in_tvalid	Input	1	AXI4-S data valid
axi4s_vid_in_tuser	Input	(115)	AXI4-S tuser - tuser[0] indicates start of video frame when asserted - tuser[1] indicates the start of a nonvideo packet when asserted
axi4s_vid_in_tlast	Output	1	AXI4-S end of packet
axi4s_vid_in_tready	Output	1	AXI4-S data ready
axi4s_vid_out_tdata	Output	(114)	AXI4-S data in
axi4s_vid_out_tvalid	Output	1	AXI4-S data valid
<i>continued...</i>			

(114) The equation shows the sizes of all `tdata` widths in these interfaces:

$$\max(\text{ceil}((\text{bits per color sample} \times \text{number of color planes}) / 8) \times \text{pixels in parallel} \times 8, 16)$$

(115) The equation shows the sizes of all `tuser` widths in these interfaces:

$$N = \text{ceil}(\text{tdata width} / 8)$$

Name	Direction	Width	Description
axi4s_vid_out_tuser	Output	(115)	AXI4-S tuser - tuser[0] indicates start of video frame when asserted - tuser[1] indicates the start of a nonvideo packet when asserted
axi4s_vid_out_tlast	Output	1	AXI4-S end of packet
axi4s_vid_out_tready	Input	1	AXI4-S data ready

Related Information

Video and Vision Processing IP Interfaces on page 30

36.4. Interlacer IP Registers

Each register is either read-only (RO) or read-write (RW).

In the software API these register names appear with a prefix of INTEL_VVP, INTEL_VVP_CORE, or INTEL_VVP_INTERLACER as appropriate and with an optional REG suffix

Address	Register	Access		Description
		Lite	Full	
Parameterization registers				
0x0000	PROD_ID	RO		Read this register to retrieve the interlacer product ID. This register always returns 0x6AF7_0232.
0x0004	VER	RO		Read this register for the IP version information.
0x0008	LITE_MODE	RO		Read this register to determine if Lite mode is on. This register returns 0 when you turn off Lite mode and 1 when you turn on Lite mode .
0x000C	DEBUG_ENABLED	RO		Read this register to determine if Debug features are on. This register returns 1 if Debug features are on and 0 otherwise
0x0010 to 0x011F unused				
Control and debug registers				
For more details of these registers, refer to <i>Image information control packets</i> in the <i>Altera Streaming Video Protocol Specification</i> . You must turn on Debug features to read the values in these registers. If Debug features are off, reads to these registers return undefined data. The only exception is the STATUS register, whose value you can always read.				
0x0120	IMG_INFO_WIDTH	RW	RO	When you turn on Lite mode , use this register to set the expected width of incoming video fields. When you turn off Lite mode and turn on Debug features , this register returns the width that the interlacer derives from information in the image information control packet.
0x0124	IMG_INFO_HEIGHT	RW	RO	When you turn on Lite mode , use this register to set the expected height of incoming video fields. When you turn off Lite mode and turn on Debug features , this register returns the height that the interlacer derives from information in the image information control packet.
continued...				

Address	Register	Access		Description
		Lite	Full	
0x0128	IMG_INFO_INTERLACE	RW	RO	When you turn on Lite mode , use this register to set the expected interlace format (standard image information format interlace nibble) of incoming video fields. When you turn off Lite mode and turn on Debug features , this register returns the interlace nibble that the interlacer receives in the image information control packet.
0x012C	Reserved	-	-	Reserved
0x0130	IMG_INFO_COLORSPACE	-	RO	When you turn off Lite mode and turn on Debug features , this register returns the color space that the interlacer derives from information in the image information control packet. Unused when you turn on Lite mode .
0x0134	IMG_INFO_SUBSAMPLING	-	RO	When you turn off Lite mode and turn on Debug features , this register returns the subsampling that the interlacer derives from information in the image information control packet. Unused when you turn on Lite mode .
0x0138	IMG_INFO_CHROMA_SITING	-	RO	When you turn off Lite mode and turn on Debug features , this register returns the chroma siting that the interlacer derives from information in the image information control packet. Unused when you turn on Lite mode .
0x013C	IMG_INFO_FIELD_COUNT	-	RO	When you turn off Lite mode and turn on Debug features , this register returns the field count that the interlacer derives from information in the image information control packet. Unused when you turn on Lite mode .
0x0138 to 0x013C unused				
0x0140	STATUS	RO		Bit 0: Status bit. 1 = interlacer is processing a video field, 0 otherwise. Lite mode off only: Bit 1: Pending register updates bit. Any writes to the enable, ctrl_override or send_fl_first registers (0x0148 – 0x0150) cause the IP to raise the pending register updates bit, to indicate outstanding changes to the interlacer settings. The IP lowers this bit at the next field boundary after a write to the COMMIT register.
0x0144	COMMIT	RW		Only used when you turn off Lite mode . the IP holds any changes to the interlacer settings via the register map until a write is issued to this register. The value you write is unimportant.
continued...				

Address	Register	Access		Description
		Lite	Full	
0x0148	ENABLE	RW		Bit 0 of this register controls the creation of interlaced fields from progressive input. Set bit 0 to 0 to turn off interlacing and propagate progressive frames unaltered. Set bit 0 to 1 to turn on interlacing of progressive frames.
0x014C	CTRL_OVERRIDE	RW		Bit 0 of this register controls override of the interlacing sequence by the interlaced nibble in the incoming image information packets (Lite mode off only). Set bit 1 of this register to 0 to turn off interlaced nibble override. Set bit 0 to 1 to turn on interlaced nibble override.
0x0150	SEND_F1_FIRST	RW		Bit 0 of this register controls whether an F0 or an F1 field is created first after any reset to the interlacing sequence. Set bit 0 to 0 to generate F0 first. Set bit 0 to 1 to generate F1 first.

Register Bit Descriptions

Table 631. PROD_ID

Name	Bits	Description
Interlacer product ID	31:0	This register always returns 0x6AF7_0232.

Table 632. VERSION

Name	Bits	Description
Register map version	7:0	Register map version.
IP patch revision	15:8	-
IP update revision	23:16	Updated when the IP version changes.
IP major revision	31:24	Updated when the IP version changes.

Table 633. LITE_MODE

Name	Bits	Description
Lite mode parameterization bit	31:0	Returns 1 if you turn on lite mode and 0 otherwise

Table 634. DEBUG_ENABLED

Name	Bits	Description
Debug features parameterization bit	31:0	Returns 1 you turn on Debug features and 0 otherwise

Table 635. IMG_INFO_WIDTH

Name	Bits	Description
Width bits	15:0	If Lite mode is on, write to this register to set the expected width of the incoming video fields. If Lite mode is off and Debug features are on, this register returns the width-1 field from the most recently received image information packet and adds 1 to return a value for width.
unused	31:16	unused

Table 636. IMG_INFO_HEIGHT

Name	Bits	Description
Height bits	15:0	If Lite mode is on, write to this register to set the expected height of the incoming video fields. If Lite mode is off and Debug features are on, this register reads the <code>height-1</code> field from the most recently received image information packet and adds 1 to return a value for <code>height</code> .
unused	31:16	unused

Table 637. IMG_INFO_INTERLACE

Name	Bits	Description
IntlaceNibble bits	3:0	If Lite mode is on, write to this register to set the expected interlace format of the incoming video fields (interlace nibble). If Lite mode is off and Debug features are on, this register returns the <code>intlaceNibble</code> field from the most recently received image information packet.
unused	31:4	unused

Table 638. IMG_INFO_COLORSPACE

Name	Bits	Description
CSP code bits	6:0	If Lite mode is on, this register has no function. If Lite mode is off and Debug features are on, this register returns the 7 bit <code>CSP</code> field from the most recently received image information packet.
unused	31:7	unused

Table 639. IMG_INFO_SUBSAMPLING

Name	Bits	Description
SubSa code bits	1:0	If Lite mode is on, this register has no function. If Lite mode is off and Debug features are on, this register returns the <code>SUBSA</code> field from the most recently received image information packet.
unused	31:2	unused

Table 640. IMG_INFO_COSITING

Name	Bits	Description
Cosite code bits	1:0	If Lite mode is on, this register has no function. If Lite mode is off and Debug features are on, this register returns the <code>COSITE</code> field from the most recently received image information packet.
Unused	31:2	Unused

Table 641. IMG_INFO_FIELD_COUNT

Name	Bits	Description
Count bits	6:0	If Lite mode is on, this register has no function. If Lite mode is off and Debug features are on, this register returns the 7 bit <code>FIELD_COUNT</code> field from the most recently received image information packet.
Unused	31:7	Unused

Table 642. STATUS

Name	Bits	Description
Status bit	0	1 = interlacer is processing a video field, 0 otherwise.
Pending register updates bit	1	1 = interlacer has pending updates, 0 otherwise
Unused	31:2	Unused

Table 643. COMMIT

Name	Bits	Description
Unused	31:0	Unused

Table 644. ENABLE

Name	Bits	Description
Interlacing enable	0	Set to 1 to turn on interlacing, set to 0 to turn off interlacing
Unused	31:1	Unused

Table 645. CTRL_OVERRIDE

Name	Bits	Description
Interlace nibble override	0	Set to 1 to turn on interlace nibble override, set to 0 to turn off interlace nibble override
Unused	31:1	Unused

Table 646. SEND_F1_FIRST

Name	Bits	Description
Send F1 first	0	Set to 1 to send F1 first after a reset to the interlacing sequence, set to 0 to send F0 first
Unused	31:1	Unused

36.5. Interlacer IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_interlacer_regs.h`

Include file: `intel_vvp_interlacer.h`

Table 647. Interlacer IP Software API

Name	Description
<code>intel_vvp_interlacer_init</code>	Initialize the Interlacer instance
<code>intel_vvp_core_*</code>	Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i> . Writable when Lite mode is on. Readable when Lite mode is off and Debug features is on.
<i>continued...</i>	

Name	Description
intel_vvp_interlacer_get_lite_mode	Returns if Lite mode is on
intel_vvp_interlacer_get_debug_enabled	Returns if Debug features is on
intel_vvp_interlacer_is_running	Returns if the IP is currently processing video data
intel_vvp_interlacer_get_commit_status	Returns if the IP has uncommitted writes
intel_vvp_interlacer_get_status	Reads the status register
intel_vvp_interlacer_set_enable	Enables or disables interlacing
intel_vvp_interlacer_enable	Enables interlacing
intel_vvp_interlacer_disable	Disables interlacing
intel_vvp_interlacer_set_ctrl_override	Enables or disables using image information packets to select the lines to keep in full variants.t
intel_vvp_interlacer_set_fl_first	Selects if the output interlaced stream starts with a F0 or F1
intel_vvp_interlacer_is_enabled	Returns the interlacer enabled or disabled status
intel_vvp_interlacer_get_ctrl_override	Returns if image information packets select the lines to discard
intel_vvp_interlacer_get_fl_first	Returns if the output interlaced stream starts with a F0 or F1
intel_vvp_interlacer_commit_writes	Commit all outstanding writes

intel_vvp_interlacer_init

Prototype

```
int intel_vvp_interlacer_init(intel_vvp_interlacer_instance
*instance, intel_vvp_core_base base);
```

Description Initializes an Interlacer instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

Arguments instance - pointer to the intel_vvp_interlacer_instance software driver instance structure

base - base address of the register map

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product_id does not match with the Interlacer product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpInterlacerRegMapVersionErr if the register map is not supported

intel_vvp_interlacer_get_lite_mode

Prototype

```
bool
intel_vvp_interlacer_get_lite_mode(intel_vvp_interlacer_instance
*instance);
```

Description Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_interlacer_instance fully initialized

Arguments instance - pointer to the intel_vvp_interlacer_instance software driver instance structure

Returns true if the IP is parameterized in lite mode

intel_vvp_interlacer_get_debug_enabled

Prototype

```
bool
intel_vvp_interlacer_get_debug_enabled(intel_vvp_interlacer_instance
*instance);
```

Description Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_interlacer_instance fully initialized.

Arguments instance - pointer to the intel_vvp_interlacer_instance software driver instance structure

Returns true if the IP is parameterized with debug features on

intel_vvp_interlacer_is_running

Prototype

```
bool
intel_vvp_interlacer_is_running(intel_vvp_interlacer_instance*
instance);
```

Description Reads and returns the running bit of the STATUS register. The instance must be a valid intel_vvp_interlacer_instance fully initialized.

Arguments instance - pointer to the intel_vvp_interlacer_instance software driver instance structure

Returns True is the IP is currently receiving or storing an input field, false when between two input fields

intel_vvp_interlacer_get_commit_status

Prototype

```
bool
intel_vvp_interlacer_get_commit_status(intel_vvp_interlacer_instance*
instance);
```

Description Reads and returns the commit pending bit of the STATUS register. The instance must be a valid intel_vvp_interlacer_instance fully initialized and configured in full mode.

Arguments instance - pointer to the intel_vvp_interlacer_instance software driver instance structure

Returns True is the IP is configured in full mode and if the agent interface contains uncommitted writes.

intel_vvp_interlacer_get_status

Prototype

```
uint32_t
intel_vvp_interlacer_get_status(intel_vvp_interlacer_instance*
instance);
```

Description Reads the STATUS register. The instance must be a valid intel_vvp_interlacer_instance fully initialized.

Arguments instance - pointer to the intel_vvp_interlacer_instance software driver instance structure

Returns The content of the STATUS register

intel_vvp_interlacer_set_enable

Prototype

```
int
intel_vvp_interlacer_set_enable(intel_vvp_interlacer_instance
*instance, bool enable);
```

Description Writes to the ENABLE register to enable or disable the interlacing. The instance must be a valid intel_vvp_interlacer_instance fully initialized.

Arguments instance - pointer to the intel_vvp_interlacer_instance software driver instance structure

enable - true to enable interlacing, false for input passthrough

Returns kIntelVvpCoreOk for success, negative error code in case of failures
kIntelVvpCoreInstanceErr if the instance is invalid

intel_vvp_interlacer_enable

Prototype

```
int intel_vvp_interlacer_enable(intel_vvp_interlacer_instance
*instance);
```

Description

Enables interlacing. The instance must be a valid `intel_vvp_interlacer_instance` fully initialized.

Arguments

instance - pointer to the `intel_vvp_interlacer_instance` software driver instance structure

Returns

kIntelVvpCoreOk for success, negative error code in case of failures
kIntelVvpCoreInstanceErr if the instance is invalid

intel_vvp_interlacer_disable

Prototype

```
int
intel_vvp_interlacer_disable(intel_vvp_interlacer_instance
*instance);
```

Description

Disables interlacing and enables input passthrough

Arguments

instance - pointer to the `intel_vvp_interlacer_instance` software driver instance structure

Returns

kIntelVvpCoreOk for success, negative error code in case of failures
kIntelVvpCoreInstanceErr if the instance is invalid

intel_vvp_interlacer_set_ctrl_override

Prototype

```
int
intel_vvp_interlacer_set_ctrl_override(intel_vvp_interlacer_instance*
instance, bool ctrl_override);
```

Description

Writes to the `CTRL_OVERRIDE` register to enable/disable the reading and use of the interlace nibble on image info packet to select the field to keep during interlacing. The instance must be an initialized, valid, full variant, `intel_vvp_interlacer_instance`.

Arguments

instance - pointer to the `intel_vvp_interlacer_instance` software driver instance structure

ctrl_override - true to enable the use of interlace nibbles in image info packet, false otherwise

Returns

kIntelVvpCoreOk for success, negative error code in case of failures
kIntelVvpCoreInstanceErr if the instance is invalid or lite variant

intel_vvp_interlacer_set_f1_first

Prototype

```
int
intel_vvp_interlacer_set_f1_first(intel_vvp_interlacer_instance
*instance, bool f1_first)
```

Description Writes to the F1_FIRST register to select which interlaced field is produced first after a setup change

Arguments instance - pointer to the intel_vvp_interlacer_instance software driver instance structure

f1_first - true to output a F1 field first whenever the IP detects a change of input or configuration, false to start with a F0

Returns kIntelVvpCoreOk for success, negative error code in case of failures
kIntelVvpCoreInstanceErr if the instance is invalid

intel_vvp_interlacer_is_enabled

Prototype

```
bool
intel_vvp_interlacer_is_enabled(intel_vvp_interlacer_instance
*instance)
```

Description Returns the current interlacer mode of operation. The instance must be a valid intel_vvp_interlacer_instance fully initialized with debug on

Arguments instance - pointer to the intel_vvp_interlacer_instance software driver instance structure

Returns The current interlacer configuration if debug is on. True if the interlacer is active, false for progressive passthrough or if debug is off

intel_vvp_interlacer_get_ctrl_override

Prototype

```
bool
intel_vvp_interlacer_get_ctrl_override(intel_vvp_interlacer_instance
*instance)
```

Description Returns the current interlacer ctrl_override setting. The instance must be a valid, full variant, intel_vvp_interlacer_instance fully initialized with debug on.

Arguments instance - pointer to the intel_vvp_interlacer_instance software driver instance structure

Returns True if the interlacer uses the input image info packet interlace nibble to determine which interlaced field is kept on a progressive frame, false otherwise or if debug is off

`intel_vvp_interlacer_get_f1_first`

Prototype

```
bool  
intel_vvp_interlacer_get_f1_first(intel_vvp_interlacer_instance  
*instance)
```

Description Returns the current interlacer f1 first setting. The instance must be a valid `intel_vvp_interlacer_instance` fully initialized and with debug on.

Arguments instance - pointer to the `intel_vvp_interlacer_instance` software driver instance structure

Returns The current `f1_first` setting or false if debug is off

`intel_vvp_interlacer_commit_writes`

Prototype

```
int  
intel_vvp_interlacer_commit_writes(intel_vvp_interlacer_instance*  
instance)
```

Description Commits all pending writes before processing the next field. The instance must be valid and parameterized in full mode.

Arguments instance - pointer to the `intel_vvp_interlacer_instance` software driver instance structure

Returns `kIntelVvpCoreOk` (0) for success, negative error code otherwise

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

37. Mixer IP

37.1. About the Mixer IP

The mixer IP allows you to overlay video fields from multiple inputs on each other, either with or without alpha blending (transparency).

The mixer supports up to eight inputs (or layers). Layer 0 is the base layer, onto which the IP overlays every other layer (overlay layers). The resolution of the base layer determines the output resolution. Overlay layers can be offset horizontally and vertically within the base layer so that their first pixel overlays at any position within the base layer. Offsets must be such that the full overlay layer image remains inside the confines of the base layer.

The mixer supports both full and lite variants of the Altera Streaming Video Protocol.

37.1.1. Mixer IP Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family and the number of supported bits per color sample, colors per pixel, and pixels in parallel.

Table 648. Mixer IP Performance and Resources

The table shows ALM usage and f_{MAX} for a design with 10 bits per color sample, 2 colors per pixel, 2 pixels in parallel, 3 layers with static alpha blending.

Target device	ALM	DSP	f_{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	976	8	759
Arria 10 speed grade 1 (10AS066H1F34E1HG)	805	8	527
Cyclone 10 GX speed grade 5 (10CX220YF78015G)	889	8	458
Stratix 10 GX speed grade 1 (1SX280LN2F43E1VG)	1001	8	564

37.2. Mixer IP Parameters

The Mixer IP offers run-time and compile-time parameters.

Table 649. Mixer IP Parameters

Parameter	Value	Description
Video data format		
Lite mode	On or off	Turn on to use the lite variant of the Altera Streaming Video Protocol.
Bits per color sample	8 to 16	Select the number of bits per color sample.
<i>continued...</i>		

Parameter	Value	Description
Number of color planes	1 to 4	Select the number of pixels transmitted every clock cycle at the input interface.
Number of pixels in parallel	1 to 8	Select the number of pixels transmitted every clock cycle at the output interface.
Control settings		
Memory-mapped control interface	On or off	Turn on to enable the Avalon memory-mapped control agent interface and allow runtime configuration via the register map. You must have the Avalon memory-mapped control agent interface if you turn on Lite mode .
Algorithm settings		
Number of layers	2 to 8	Total number of mixers layers, including the base layer (layer 0).
Do rounding	On or off	Turn on to apply round-half-up logic at the end of the alpha blending mathematics for each layer. If this feature is not enabled then all fraction bits are cropped without rounding.
Layer 1: Use restricted offsets	On or off	Turn on for the restricted offsets for mixer layer 1.
Layer 1: Alpha blending mode	No blending, Alpha from command, Alpha from data, Alpha from command or data	Set the alpha blending mode for mixer layer 1.
Layer 2: Use restricted offsets	On or off	Turn on for the restricted offsets for mixer layer 2.
Layer 2: Alpha blending mode	No blending, Alpha from command, Alpha from data, Alpha from command or data	Set the alpha blending mode for mixer layer 2.
Layer 3: Use restricted offsets	On or off	Turn on for the restricted offsets for mixer layer 3.
Layer 3: Alpha blending mode	No blending, Alpha from command, Alpha from data, Alpha from command or data	Set the alpha blending mode for mixer layer 3.
Layer 4: Use restricted offsets	On or off	Turn on for the restricted offsets for mixer layer 4.
Layer 4: Alpha blending mode	No blending, Alpha from command, Alpha from data, Alpha from command or data	Set the alpha blending mode for mixer layer 4.
Layer 5: Use restricted offsets	On or off	Turn on for the restricted offsets for mixer layer 5.
Layer 5: Alpha blending mode	No blending, Alpha from command, Alpha from data, Alpha from command or data	Set the alpha blending mode for mixer layer 5.
Layer 6: Use restricted offsets	On or off	Turn on for the restricted offsets for mixer layer 6.
Layer 6: Alpha blending mode	No blending, Alpha from command, Alpha from data, Alpha from command or data	Set the alpha blending mode for mixer layer 6.
Layer 7: Use restricted offsets	On or off	Turn on for the restricted offsets for mixer layer 7.
<i>continued...</i>		

Parameter	Value	Description
Layer 7: Alpha blending mode	No blending, Alpha from command, Alpha from data, Alpha from command or data	Set the alpha blending mode for mixer layer 7.
General		
Extra pipeline level	0 to 3	Set the number of additional register stages to add the pipeline.
Debug features	On or off	Turn on to read back writeable registers via the control agent interface.
Separate clock for control interface	On or off	Turn on for a separate clock for the control agent interface.

37.3. Mixer IP Functional Description

The mixer includes alpha blending for each overlay layer. Alpha blending allows you to vary the transparency of the overlay layer as it is mixed onto the existing image.

Alpha blending

The value to determine the level of transparency is the alpha value. The alpha value varies between 0 and 1. 0 represents full transparency (the overlay layer data does not appear at all). 1 represents fully opaque (the overlay layer completely obscures the existing image).

The IP recursively applies alpha blending, with the image from layer 1 (I_1) blended onto the image from layer 0 (I_0) to create the intermediate image (O_1). The IP then blends the image from layer 2 (I_2) onto O_1 to create O_2 , and so on for each of the layers in turn. For a mixer with N layers, O_{N-1} is the final output image. The equations show the alpha blending:

Equation 3. Alpha Blending

$$O_1 = \alpha \times I_1 + (1 - \alpha) \times I_0 O_{N-1} = \alpha \times I_{N-1} + (1 - \alpha) \times O_{N-2}$$

The IP applies the alpha blending process independently to each color plane in the image and only in the areas where the offset overlay image overlaps the intermediate image.

Alpha blending mode

You can parameterize each layer of the mixer to support different options for how the IP superimposes the foreground layer data onto the existing image. The **Alpha blending mode** parameter has four options:

- **No blending.** The mixer does not include the logic required to perform the alpha blending process for the given layer. The IP overlays the layer image onto the existing image without transparency. It completely replaces the pixel value for the existing image with that of the layer image in the appropriate regions of the output image (as defined by the layer offsets and layer dimensions). You may still select to turn off the overlay at run time via the register map. **No blending** does not require any DSP blocks.
- **Alpha from command.** The IP includes the logic for alpha blending. The IP takes the alpha value from the register map and applies it as a constant (static alpha) value to every pixel in the layer image. You can still turn off the overlay at run time via the register map or select to ignore the alpha value entirely and revert to no blending.
- **Alpha from data.** The IP includes logic for alpha blending. The incoming video data includes an additional color plane that contains a per-pixel alpha value to determine the transparency. The alpha value is always in the highest index color plane, in the MSBs of each pixel. You can still turn off the overlay at run time via the register map or select to ignore the alpha value entirely and revert to no blending.
- **Alpha from command or data.** You have full flexibility to select at run time between disabling the overlay, overlay with no blending, overlay with static alpha, or overlay with per pixel alpha.

Register map settings

You can adjust the settings for each overlay layer in the mixer at run time by making edits to the mixer's register map through the Avalon memory-mapped control agent interface. When you turn on **Lite mode**, the properties of the base layer are also set via the register map.

Each overlay layer has five run time controllable settings when you turn off **Lite mode**:

- Layer enable (turns the layer on and off)
- Layer blend mode
- Layer static alpha
- Layer horizontal offset
- Layer vertical offset

When you turn on **Lite mode**, two additional registers per overlay layer set the expected resolution for that layer. All these registers are double buffered and edits their values only take effect at the next frame boundary. When you turn off **Lite mode**, a write is required the `COMMIT` register for any of the edits to take effect at the next frame boundary.

Overlay layer enables

Each overlay layer n ($1 \leq n \leq 7$) has a register in the Avalon memory-mapped control agent register map to define if the IP turns on or off the layer (`LAYER $n$ _MODE`). The three LSBs of the register value determine how the IP processes the layer.

- Bit 0. The main enable bit for the given layer. If you set this bit to 0, the IP holds the layer at a field boundary with the `tready` signal set to 0. When you set this bit to 1, the IP accepts the input packets for the given layer, with one field's worth of packets accepted for every field on the base layer. The IP propagates the control packets to the output, and either displays the field data as part of the output frame or not, depending on the value in bit 1 of the register.
- Bit 1. This bit determines if the IP displays the layer or not. If you write 0 to this register, the IP mixes the layer's field data into the output field (display mode) (according to the settings in the `LAYER $n$ _BLEND_MODE` register). If you write 1 to this register, the IP does not include the layer's field data (consume mode). In consume mode, the mixer accepts 1 field of layer data for each field of the base layer. The control packets in the layer stream are still propagated to the output. In display mode, the mixer accepts the layer field data at the input only at times when it must make up part of the output image – according to the vertical and horizontal offsets settings for this layer. In consume mode, the mixer accepts the layer field data as quickly as possible, with no reference to the offsets and no attempt to accept one line of layer field data for each line of base field data.
- Bit 2. This bit determines how the mixer behaves for the first field after any change to the value in the `LAYER $n$ _MODE` register. If you set this bit to 0 (and bit 0 is set to 1), the mixer waits for data to be available on layer n before starting the output field. If no data is available, the mixer pauses and waits (hard start condition). If you set this bit to 1, the mixer can progress with the output frame, even if a new frame is not ready at the layer n input. At each field boundary the mixer again checks to see if data becomes available. When data is available at layer n , normal mixing for that layer resumes, and the mixer waits for data on subsequent fields if it is not immediately available (soft-start condition).

Carefully consider the difference between the hard- and soft-start conditions when deciding which is more appropriate for your system.

- If the layer input comes directly from a source that you cannot stall, such as a clocked video input from HDMI or DisplayPort for example, do not use soft start. In this case the IP forces the output to match the start-of-frame timing of the layer input, as anything else forces a stall on the input. Using soft start allows the output to run in sync with the start-of-frame timing of the base layer (or another overlay layer). If the start-of-frame timing of the overlay layer does not exactly align to that of the base layer, the overlay layer misses the output start of frame and holds off for some amount of time, up to a full frame period in the worst case. This behavior causes the input to overflow.
- Unless multiple input layers are being externally controlled to force an alignment of their start-of-frame and frame rates, only one layer may come directly from a nonstable source. If the layers are not synchronized at the input, there is no way that each of them can have a hard start without at least one being stalled and overflowing. This typically means that, at most, only one layer should use the hard start condition.
- Typically, a source that you can stall drives most layers – such as a frame buffer with drop or repeat of frames. You can stall a frame buffer without overflowing. A soft start condition is more appropriate, as it allows any existing output from the mixer to retain its current start-of-frame timing. The overlay layer from the frame buffer synchronizes to the existing timing. Because the output timing stays the same, no glitch in any display connected to it occurs.

The mixer does not commit to processing a field until the data for all layers is pending at each input. Until the IP reaches this condition, the mixer remains in a state where you can still edit the register map via the Avalon memory-mapped control agent interface. The changes take effect immediately because the mixer is between fields. If the system experiences a loss of video on any input, the system controller can turn off the given layer and avoid lock-up in the system.

Restricted offsets

You supply a horizontal offset for each overlay layer. This determines the pixel index within the line at which that layer begins to be mixed onto the background. If the number of pixels in parallel is 1, it is trivial to begin mixing at any pixel offset. If the number of pixels in parallel is greater than 1, any offset that is not an integer multiple of pixels in parallel gives additional challenges.

The mixer must include logic to shift pixels within and across data words by any number of pixels between 0 and the number of pixels in parallel minus 1. If the number of pixels in parallel is small, the ALM cost of this extra hardware is relatively small, but at 4 to 8 pixels in parallel the resource usage can be a significant proportion of the overall mixer ALM count.

If you limit the horizontal offset for any layer such that it is always an integer multiple of the pixels in parallel, you can turn on **use restricted offsets** for that layer. Then the mixer omits the extra pixel shifting logic for that layer and reduces the ALM cost. If you turn on **use restricted offsets** for any layer and you try to set a horizontal offset for that layer to a noninteger multiple of pixels in parallel, the internal logic moves the offset back to the nearest integer position.

Subsampling support

The mixer supports 4:4:4, 4:2:2 and 4:2:0 chroma sampled video data and can switch dynamically between chroma samplings at run time. Full variants drive the current chroma sampling by the image information packet that accompanies each field. Lite variants let you set the chroma sampling via the register map. The following restrictions regarding chroma sampling apply:

- All layers must be the same chroma sampling as the base layer (layer 0).
- For 4:2:2 chroma sampling, the field width for all layers (including the base layer) must be an integer multiple of 2.
- For 4:2:2 chroma sampling, the horizontal offset for each active overlay layer must be an integer multiple of 2.
- For 4:2:0 chroma sampling, both the field height and field width for all layers (including the base layer) must be integer multiples of 2.
- For 4:2:0 chroma sampling, both the horizontal and vertical offsets for each active layer must be integer multiples of 2.

Interlaced support

The mixer is mainly for progressive video but does have limited support for interlaced video for full variants. You should apply vertical offsets within each field and not within the progressive frame. Each layer should supply either an F0 or F1 field in sync with the base layer (layer 0). If any layer that you turn on for display provides an F1 field when the base layer provides F0, or vice versa, the IP asserts the broken-field flag in the output end of frame packet.

Overlay error conditions

Each overlay layer has an expected field width and height, derived either from the image information packets for full variants, or the register map for lite variants. Each layer also has a horizontal and vertical offset within the base layer which you specify through the register map. If the sum of the overlay field width and the horizontal offset for any layer exceed the field width of the base layer, the IP does not display layer (the IP forces the layer into consume mode). The IP asserts the broken-field flag in the output end of frame packet

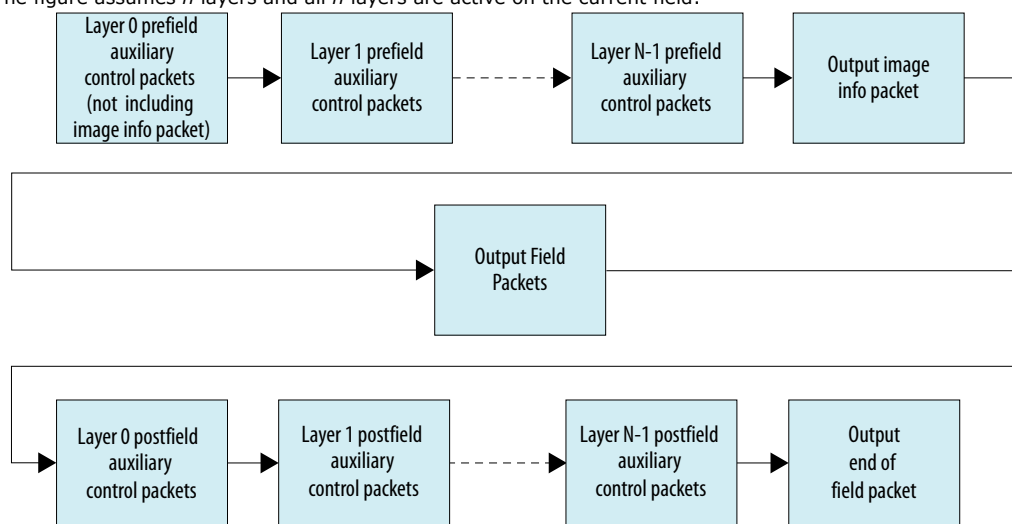
Packet propagation

In the full variant of Altera Streaming Video, the input stream on each layer may contain image information, end of field, and auxiliary control packets with the video line packets. The mixer adheres to the following rules when handling these packets from each layer:

- Image information and end of frame packets. The image information and end of frame packet from the base layer (layer 0) propagate to the output. The image information packet is always unaltered. The end of frame packet propagates with the broken-field flag potentially edited, but otherwise unaltered. If the IP asserts the broken-field flag in the base layer or any displayed overlay layer. Then the IP asserts the output broken field flag. The broken-field flag in the end of frame packet may also be asserted if the mixer experiences an error during processing, such as an interlaced field mismatch.
- Auxiliary control packets. When any layer is active, either for display or consume, its auxiliary control packets propagate to the output with the video packets from the base layer. The IP takes the auxiliary control packets from each layer in sequence, both before and after the field line packets.

Figure 102. Control packet propagation

The figure assumes n layers and all n layers are active on the current field:



37.3.1. Mixer IP Interfaces

Table 650. Mixer IP Interfaces

Name	Direction	Width	Description
Clocks and resets			
main_clock_clk	In	1	AXI4-S processing clock.
main_reset_rst	In	1	AXI4-S processing reset.
agent_clock_clk	In	1	Clock for the Avalon memory-mapped control agent interface. Only if you turn on Separate clock for control interface .
<i>continued...</i>			

Name	Direction	Width	Description
agent_reset_rst	In	1	Reset for the Avalon memory-mapped control agent interface. Only if you turn on Separate clock for control interface .
Control interfaces			
av_mm_control_agent_address	In	7	Avalon memory-mapped agent address.
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write.
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data.
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable.
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read.
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data.
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read.
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request.
Altera Streaming Video interfaces The IP has are up to eight video streaming input interfaces, one for each enabled mixer layer. In the signal names, x represents the layer number. 0 is the base layer. (116)			
axi4s_vid_x_in_tdata	In		AXI4-S data in.
axi4s_vid_x_in_tvalid	In	1	AXI4-S data valid.
axi4s_vid_x_in_tuser	In		AXI4-S tuser <i>tuser[0]</i> indicates start of video frame when asserted <i>tuser[1]</i> indicates the start of a non-video packet when asserted.
axi4s_vid_x_in_tlast	In	1	AXI4-S end of packet.
axi4s_vid_x_in_treadyout	1	1	AXI4-S data ready.
axi4s_vid_out_tdata	Out	(117)	AXI4-S data in.
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid.
axi4s_vid_in_tuser	Out	(118)	AXI4-S tuser <i>tuser[0]</i> indicates start of video frame when asserted <i>tuser[1]</i> indicates the start of a non-video packet when asserted
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet.
axi4s_vid_out_tready	In	1	AXI4-S data ready.

(116) The equation gives all tuser widths in these interfaces:

$$N = \text{ceil}(\text{tdata width} / 8)$$

(117) The equation gives all tdata widths in these interfaces:

$$\max(\text{ceil}((\text{bits per color sample} \times \text{number of color planes} \times \text{pixels in parallel}) / 8) \times 8, 16)$$

(118) The equation gives all tuser widths in these interfaces:

$$N = \text{ceil}(\text{tdata width} / 8)$$

Related Information

Video and Vision Processing IP Interfaces on page 30

37.4. Mixer IP Registers

Each register is either read-only (RO) or read-write (RW).

Table 651. Mixer IP Parameterization Registers

In the software API the register names appear with a prefix of INTEL_VVP, INTEL_VVP_CORE or INTEL_VVP_MIXER as appropriate and with an optional REG suffix

Address	Register	Access	Description
0x0000	VID_PID	RO	Read this register to retrieve mixer product ID. This register always returns 0x6FA7_0233
0x0004	VERSION	RO	Read this register to retrieve the IP version information.
0x0008	LITE_MODE	RO	Read this register to determine if you turn on lite mode . This register returns 0 when you turn off lite and 1 when you turn on lite .
0x000C	DEBUG_ENABLED	RO	Read this register to determine if Debug features is on. This register returns 1 if reads to other registers designated as RW return the last value you write to the register, or an undefined value.
0x0010	PIXELS_IN_PARALLEL	RO	Read this register to determine if the IP supports 4:4:4 chroma sampling at the output
0x0014	NUM_LAYERS	RO	Read this register to determine if the IP supports 4:2:2 chroma sampling at the output
0x0018 + (n-1)*8	LAYER_n_SUPPORTED_BLEND_MODES	RO	Read return values
0x001C+ (n-1)*8	LAYER_n_RESTRICTED_OFFSETS	RO	Read this register to determine if restricted offsets is on for layer <i>n</i> (1 ≤ X ≤ 7). Returns 1 if restricted offsets is on and 0 otherwise
0x0060 to 0x011F	-	-	Unused.

Table 652. Mixer IP Control and Debug Registers

For more information, refer to *Control Packets*. You must turn on **Debug features** to read the values stored in these registers. If you turn off **Debug features**, reads to these registers return undefined data. The only exception is the STATUS register, whose value can always be read

Address	Register	Access		Description
		Lite	Full	
0x0120	IMG_INFO_WIDTH	RW	RO	When you turn on lite , use this register to set the expected width of incoming video fields. When you turn off lite and turn on Debug features , this register returns the width that the mixer derives from information in the image information packet.
0x0124	IMG_INFO_HEIGHT	RW	RO	When you turn on lite , use this register to set the expected height of incoming video fields. When you turn off lite and turn on Debug features , this register returns the height that the mixer derives from information in the image information packet.

continued...

Address	Register	Access		Description
		Lite	Full	
0x0128	IMG_INFO_INTERLACE	-	RO	When you turn off lite and turn on Debug features , this register returns the interlace nibble that the mixer derives from information in the image information packet. Not in lite mode.
0x012C	Reserved	-	-	Reserved.
0x0130	IMG_INFO_COLORSPACE	-	RO	When you turn off lite and turn on Debug features , this register returns the color space that the mixer derives from information in the image information packet. Not in lite mode.
0x0134	IMG_INFO_SUBSAMPLING	RW	RO	When you turn on lite , use this register to set the expected chroma sampling of incoming video fields. When you turn off lite and turn on Debug features , this register returns the subsampling that the mixer derives from information in the image information packet. Not in lite mode.
0x0138	IMG_INFO_COSITING	-	RO	When you turn off lite and turn on Debug features , this register returns the cositing that the mixer derives from information in the image information packet. Not in lite mode.
0x013C	IMG_INFO_FIELD_COUNT	-	RO	When you turn off lite and turn on Debug features , this register returns the field count that the mixer derives from information in the image information packet. Not in lite mode.
0x012C to 0x013C	-	-	-	Unused.
0x0140	STATUS	RO		Bit 0: Status bit. 1 = mixer is processing a video field, 0 otherwise. When you turn off Lite mode : Bit 1: Pending register updates bit. Any writes to the output sampling register (0x0148) causes the IP to raise the pending register updates bit, to indicate outstanding changes to the resampling settings. The IP lowers this bit at the next field boundary after a write to the COMMIT register. Bits [14:8]: Layer running bits. 1 bit per overlay layer (layers 1 and up). Bit $n+7$ returns 1 if the IP displays or consumes layer n for the current base layer frame.
0x0144	COMMIT	RW		Only when you turn off lite mode . Any changes to the mixer settings via the register map are held pending until a write is issued to this register. The value you write is unimportant.
0x0148 + $(n-1) * 0x1C$	LAYER n _MODE	RW		Write to this register to set the layer enable mode for layer n .
0x014C + $(n-1) * 0x1C$	LAYER n _BLEND_MODE	RW		Write to this register to set the blend mode for layer n .
0x0150 + $(n-1) * 0x1C$	LAYER n _STATIC_ALPHA	RW		Write to this register to set the static alpha value for layer n .
0x0154 + $(n-1) * 0x1C$	LAYER n _H_OFFSET	RW		Write to this register to set the horizontal offset for layer n .
continued...				

Address	Register	Access		Description
		Lite	Full	
0x0158 + (n-1)*0x1C	LAYER _n _V_OFFSET	RW		Write to this register to set the vertical offset for layer <i>n</i> .
0x015C + (n-1)*0x1C	LAYER _n _WIDTH	RW		Write to this register to set the field width for layer <i>n</i> . Only when you turn on lite mode .
0x0160 + (n-1)*0x1C	LAYER _n _HEIGHT	RW		Write to this register to set the field height for layer <i>n</i> . Only when you turn on lite mode .

Register Bit Descriptions

Table 653. VID_PID

Name	Bits	Description
Mixer version ID and product ID	31:0	This register always returns 0x6AF7_0233. 15:0 is the product ID and always returns 0x0233 31:16 is the vendor ID and always returns 0x6AF7

Table 654. VERSION

Name	Bits	Description
Register map version	7:0	Register map version.
IP patch revision	15:8	-
IP update revision	23:16	Updated when the IP version changes.
IP major revision	31:24	Updated when the IP version changes.

Table 655. LITE_MODE

Name	Bits	Description
Lite mode parameterization bit	31:0	Returns 1 if you turn on lite mode and 0 otherwise.

Table 656. DEBUG_ENABLED

Name	Bits	Description
Debug features parameterization bit	31:0	Returns 1 if you turn on debug features and 0 otherwise.

Table 657. PIXELS_IN_PARALLEL

Name	Bits	Description
Pixels in parallel	31:0	Returns the number of pixels processed per clock cycle.

Table 658. NUM_LAYERS

Name	Bits	Description
Number of layers	31:0	Returns the number of mixer layers (including the base layer).

Table 659. LAYER_n_SUPPORTED_BLEND_MODES

Name	Bits	Description
Layer n supported blend modes	31:0	Returns 0 for no blending (opaque overlay only), 1 for blending with static alpha, 2 for per-pixel blending using in-stream alpha values, and 3 for runtime switching between static alpha and per-pixel alpha.

Table 660. LAYER_n_RESTRICTED_OFFSETS

Name	Bits	Description
Layer n restricted offsets	31:0	Returns 1 if horizontal offsets for layer n should be limited to integer multiples of pixels in parallel and 0 otherwise.

Table 661. IMG_INFO_WIDTH

Name	Bits	Description
Width bits	15:0	If you turn on lite mode, write to this register to set the expected width of the incoming video fields. If you turn off lite mode and turn on debug features, this register returns the <code>width-1</code> field from the most recently received <code>image information packet</code> and adds 1 to return a value for width.
unused	31:16	unused

Table 662. IMG_INFO_HEIGHT

Name	Bits	Description
Height bits	15:0	If you turn on lite mode, write to this register to set the expected height of the incoming video fields. If you turn off lite mode and turn on debug features, this register reads the <code>height-1</code> field from the most recently received <code>image information packet</code> and adds 1 to return a value for height.
unused	31:16	unused

Table 663. IMG_INFO_INTERLACE

Name	Bits	Description
IntlaceNibble bits	3:0	If you turn on lite mode, this register has no function. If you turn off lite mode and turn on debug features, this register returns the <code>intlaceNibble</code> field from the most recently received <code>image information packet</code> .
unused	31:4	unused

Table 664. IMG_INFO_COLORSPACE

Name	Bits	Description
CSP code bits	6:0	If you turn on lite mode, this register has no function. If you turn off lite mode and turn on debug features, this register returns the 7 bit <code>CSP</code> field from the most recently received <code>image information packet</code> .
unused	31:7	Unused.

Table 665. IMG_INFO_SUBSAMPLING

Name	Bits	Description
SubSa code bits	1:0	If you turn on lite mode, write to this register to set the expected chroma sampling of the incoming video fields.
<i>continued...</i>		

		If you turn off lite mode and turn on debug features, this register returns the SUBSA field from the most recently received image information packet.
unused	31:2	Unused.

Table 666. IMG_INFO_COSITING

Name	Bits	Description
Cosite code bits	1:0	If you turn on lite mode, this register has no function. If you turn off lite mode and turn on debug features, this register returns the COSITE field from the most recently received image information packet.
unused	31:2	Unused.

Table 667. IMG_INFO_FIELD_COUNT

Name	Bits	Description
Count bits	6:0	If you turn on lite mode, this register has no function. If you turn off lite mode and turn on debug features, this register returns the 7 bit FIELD_COUNT field from the most recently received image information packet.
unused	31:7	Unused.

Table 668. STATUS

Name	Bits	Description
Status bit	0	1 means mixer is processing a video field, 0 otherwise.
Pending register updates bit	1	1 means mixer has pending updates, 0 otherwise (full mode only).
unused	7:2	Unused.
Layer running bits	14:8	Bit 7+n = 1 if you turn on layer n for display or consume for this field of the base layer.
unused	31:15	Unused.

Table 669. COMMIT

Name	Bits	Description
unused	31:0	Unused.

Table 670. LAYERn_MODE

Name	Bits	Description
Layer n enable bit	0	Host enable bit for layer n (1 ≤ n ≤ 7). Set to 0 to disable input for layer n. The IP holds TREADY signal for layer n low and accepts no data. Set to 1 to enable input for layer n, subject to the consume and soft start settings in bits 1 and 2.
Layer n consume bit	1	Set to 0 if input from layer n should display in the overlaid output video. Set to 1 to consume the input video from layer n without displaying it. The IP only considers the value of this bit if bit 0 is set to 1.
Layer n soft start bit	2	Set to 0 if the internal mixer logic should wait for valid data on layer n before starting the current frame. Set to 1 if the internal mixer logic is free to disregard layer n if no video data is pending at the start of each output field. When the IP accepts one field from layer n since the last update to this register, it ignores the soft start bit. The internal mixer logic always waits for data to be available on layer n before proceeding with each output field. The IP only considers this value if bit 0 is set to 1.
unused	31:3	Unused.

Table 671. LAYERn_BLEND_MODE

Name	Bits	Description
Layer n blend mode	1:0	Blend mode for layer n. Write 0 to not display layer n, 1 for overlay without blending, 2 for blending using static alpha from register map and 3 for per-pixel blending using in-band alpha values. Blend modes must be enabled in the hardware parameter set to be selectable at runtime.
unused	31:2	Unused.

Table 672. LAYERn_STATIC_ALPHA

Name	Bits	Description
Layer n static alpha	Bits per color-1:0	Static alpha value for layer n.
unused	31:bits per color	Unused.

Table 673. LAYERn_H_OFFSET

Name	Bits	Description
Layer n horizontal offset	15:0	Horizontal offset for layer n
unused	31:16	Unused.

Table 674. LAYERn_V_OFFSET

Name	Bits	Description
Layer n vertical offset	15:0	Vertical offset for layer n.
unused	31:16	Unused.

Table 675. LAYERn_WIDTH

Name	Bits	Description
Layer n field width	15:0	Field width for layer n. Only in lite mode
unused	31:16	Unused.

Table 676. LAYERn_HEIGHT

Name	Bits	Description
Layer n field width	15:0	Field height for layer n. Only in lite mode.
unused	31:16	Unused.

37.5. Mixer IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_mixer_regs.h`

Include file: `intel_vvp_mixer.h`

Table 677. Mixer IP Software API

Name	Description
intel_vvp_mixer_init	Initialize the Mixer Protocol Converter instance
Intel_vvp_core_*	.Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i> . Writable when Lite is on. Readable when Lite is off and Debug features is on.
intel_vvp_mixer_get_lite_mode	Returns if lite mode is on
intel_vvp_mixer_get_debug_enabled	Returns if debug features is on
intel_vvp_mixer_get_num_layers	Returns the number of layers
intel_vvp_mixer_get_pixels_in_parallel	Returns the number of pixels in parallel
intel_vvp_mixer_get_supported_blend_mode	Returns the supported mixing configurations
intel_vvp_mixer_requires_restricted_offsets	Returns if there are restrictions on mixing offsets
intel_vvp_mixer_is_running	Returns if the IP is processing data
intel_vvp_mixer_get_commit_status	Returns uncommitted writes
intel_vvp_mixer_is_layer_active	Returns if a layer is active
intel_vvp_mixer_get_status	Reads the status register
intel_vvp_mixer_set_input_mode	Setup a layer
intel_vvp_mixer_get_input_mode	Returns layer setup
intel_vvp_mixer_set_blend_mode	Configure alpha-blending for a layer
intel_vvp_mixer_get_blend_mode	Returns alpha-blending configuration
intel_vvp_mixer_set_static_alpha	Sets a layer static alpha blend value
intel_vvp_mixer_get_static_alpha	Retrieves a layer static alpha blend value
intel_vvp_mixer_set_horiz_offset	Sets the horizontal offset for a layer
intel_vvp_mixer_set_vert_offset	Sets the vertical offset for a layer
intel_vvp_mixer_get_horiz_offset	Retrieves the horizontal offset for a layer
intel_vvp_mixer_get_vert_offset	Retrieves the vertical offset for a layer
intel_vvp_mixer_set_width	Sets a layer width
intel_vvp_mixer_set_height	Sets a layer height
intel_vvp_mixer_get_width	Retrieves a layer width
intel_vvp_mixer_get_height	Retrieves a layer height
intel_vvp_mixer_commit_writes	Commits pending writes

enum eIntelVvpBlendModeConfig

Members

kIntelVvpMixerBlendConfigNone (0) – Alpha-blending not supported

kIntelVvpMixerBlendConfigStaticAlpha (1) – Static alpha-blending supported

kIntelVvpMixerBlendConfigInputAlpha (2) – Alpha-blending with in-band alpha values supported

kIntelVvpMixerBlendConfigStaticOrInputAlpha (3) – Alpha-blending with static alpha value or in-band alpha values supported

Description Compile-time configuration for a layer blending methods support

enum eIntelVvpBlendMode

Members kIntelVvpMixerBlendTransparent (0) – Input not displayed

kIntelVvpMixerBlendOpaque (1) – Input completely hides lower layers

kIntelVvpMixerBlendStaticAlpha (2) – Alpha-blending with a unique static alpha value

kIntelVvpMixerBlendInputAlpha (3) – Alpha-blending with in-band alpha values supported

Description Run-time selection for the blending mode

intel_vvp_mixer_init

Prototype

```
int intel_vvp_mixer_init(intel_vvp_mixer_instance *instance,
                        intel_vvp_core_base base);
```

Description Initializes a Protocol Converter instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and should not be used further by the application if returning a non-zero error code. Beware that the IP is left in a stopped state after successful initialization.

Arguments instance – pointer to the intel_vvp_mixer_instance software driver instance structure

base – base address of the register map

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product_id does not match with the Mixer product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpMixerRegMapVersionErr if the register map is not supported

intel_vvp_mixer_get_lite_mode

Prototype

```
bool intel_vvp_mixer_get_lite_mode (intel_vvp_mixer_instance
*instance);
```

Description

Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_mixer_instance fully initialized

Arguments

instance – pointer to the intel_vvp_mixer_instance software driver instance structure

Returns

true if the IP is parameterized in lite mode

intel_vvp_mixer_get_debug_enabled

Prototype

```
bool
intel_vvp_mixer_get_debug_enabled(intel_vvp_mixer_instance
*instance);
```

Description

Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_mixer_instance fully initialized.

Arguments

instance – pointer to the intel_vvp_mixer_instance software driver instance structure

Returns

true if the IP is parameterized with debug features enabled

intel_vvp_mixer_get_num_layers

Prototype

```
uint8_t
intel_vvp_mixer_get_num_layers(intel_vvp_mixer_instance
*instance);
```

Description

Returns the configured number of layers (background layer included)

Arguments

instance – pointer to the intel_vvp_mixer_instance software driver instance structure

Returns

The number of layers

intel_vvp_mixer_get_pixels_in_parallel

Prototype

```
uint8_t
intel_vvp_mixer_get_pixels_in_parallel(intel_vvp_mixer_instance
*instance);
```

Description Returns the configured number of pixels transmitted in parallel

Arguments instance – pointer to the intel_vvp_mixer_instance software driver instance structure

Returns The number of layers

intel_vvp_mixer_get_supported_blend_mode

Prototype

```
uint8_t
intel_vvp_mixer_get_supported_blend_mode(intel_vvp_mixer_instance
*instance, uint8_t layer);
```

Description Returns the blending algorithms supported for a specific layer. The instance must be a valid, initialized, intel_vvp_mixer_instance and the layer must be valid

Arguments instance – pointer to the intel_vvp_mixer_instance software driver instance structure

layer – valid foreground layer in the range [1 .. get_num_layers()-1]

Returns Blend modes supported by the layer (static alpha and/or in-band alpha)

intel_vvp_mixer_requires_restricted_offsets

Prototype

```
bool
intel_vvp_mixer_requires_restricted_offsets(intel_vvp_mixer_instance
*instance, uint8_t layer);
```

Description Returns if there is a restriction on the horizontal offset for the specified layer

Arguments instance – pointer to the intel_vvp_mixer_instance software driver instance structure

layer – valid foreground layer in the range [1 .. get_num_layers()-1]

Returns True if the layer does not support an offset that is not a multiple of get_num_pixels_in_parallel()

intel_vvp_mixer_is_running

Prototype

```
bool
intel_vvp_mixer_is_running(intel_vvp_mixer_conv_instance*
instance);
```

Description Reads and returns the running bit of the STATUS register. The instance must be a valid intel_vvp_mixer_instance fully initialized.

Arguments instance – pointer to the intel_vvp_mixer_instance software driver instance structure

Returns True is the IP is currently generating an output field, false when disabled and/or between two fields

intel_vvp_mixer_get_commit_status

Prototype

```
bool
intel_vvp_mixer_get_commit_status(intel_vvp_mixer_instance*
instance);
```

Description Reads and returns the commit pending bit of the STATUS register. The instance must be a valid intel_vvp_mixer_instance fully initialized and configured in full mode.

Arguments instance – pointer to the intel_vvp_mixer_instance software driver instance structure

Returns True is the Mixer is configured in full mode and if the agent interface contains uncommitted writes.

intel_vvp_mixer_is_layer_active

Prototype

```
bool intel_vvp_mixer_is_layer_active
(intel_vvp_mixer_instance* instance , uint8_t layer);
```

Description Reads and returns the relevant layer_active bit in the STATUS register. The instance must be a valid intel_vvp_mixer_instance fully initialized and the layer should be a valid layer.

Arguments instance – pointer to the intel_vvp_mixer_instance software driver instance structure

layer – valid foreground layer in the range [1 .. get_num_layers()-1]

Returns The layer active bit value read from the status register

intel_vvp_mixer_get_status

Prototype

```
uint8_t intel_vvp_mixer_get_status(intel_vvp_mixer_instance*
instance);
```

Description Reads and returns the STATUS register. The instance must be a valid intel_vvp_mixer_instance fully initialized.

Arguments instance – pointer to the intel_vvp_mixer_instance software driver instance structure

Returns The value read from the status register

intel_vvp_mixer_set_input_mode

Prototype

```
int intel_vvp_mixer_set_input_mode(intel_vvp_mixer_instance*
instance, uint8_t layer, bool enable, bool consume, bool
soft_start);
```

Description Setup the run-time configuration for a given layer. The instance must be valid and initialized. The layer must be a valid foreground layer. The kIntelVvpMixerLayerErr error code is returned if some input parameters were incompatible but the input mode is still overwritten in these cases. Specifically, enabled is forced to true if consumed is true and soft_start is forced to false if enabled is false

Arguments instance – pointer to the intel_vvp_mixer_instance software driver instance structure

layer – valid foreground layer in the range [1 .. get_num_layers()-1]

enabled – enabled status of the layer (returned value)

consumed – consumed status of the layer (returned value)

soft_start – soft-start status of the layer (returned value)

Returns kIntelVvpCoreOk if the input mode was changed, negative error code otherwise

kIntelVvpCoreInstanceErr if the instance is the null pointer

kIntelVvpMixerParameterErr if some parameters have been ignored/overwritten

kIntelVvpMixerLayerErr if the layer is not in the valid range

intel_vvp_mixer_get_input_mode

Prototype

```
int intel_vvp_mixer_get_input_mode(intel_vvp_mixer_instance*
instance, uint8_t layer, bool *enabled, bool *consumed, bool
*soft_start);
```

Description Returns the current run-time configuration for a given layer. The instance must be a valid and initialized instance with debug enabled. The layer must be a valid foreground layer.

Arguments instance – pointer to the intel_vvp_mixer_instance software driver instance structure

layer – valid foreground layer in the range [1 .. get_num_layers()-1]

enabled – enabled status of the layer (returned value)

consumed – consumed status of the layer (returned value)

soft_start – soft-start status of the layer (returned value)

Returns

kIntelVvpCoreOk if the input mode was retrieved, negative error code otherwise

kIntelVvpCoreInstanceErr if the instance is the null pointer or debug is not enabled

kIntelVvpCoreNullPtrErr if one of the pointers to store return values is the null pointer

kIntelVvpMixerLayerErr if the layer is not in the valid range

intel_vvp_mixer_set_blend_mode

Prototype

```
int intel_vvp_mixer_set_blend_mode(intel_vvp_mixer_instance* instance, uint8_t layer, eIntelVvpBlendMode blend_mode)
```

Description

Select the blending for a given layer. The instance must be valid and initialized. The layer must be a valid foreground layer that supports the chosen blending algorithm

Arguments

instance – pointer to the intel_vvp_mixer_instance software driver instance structure

layer – valid foreground layer in the range [1 .. get_num_layers()-1]

blend_mode – selected blending

Returns

kIntelVvpCoreOk if the blending was changed, negative error code otherwise

kIntelVvpCoreInstanceErr if the instance is the null pointer

kIntelVvpMixerParameterErr if the blending is not supported

kIntelVvpMixerLayerErr if the layer is not in the valid range

Prototype

```
eIntelVvpBlendMode intel_vvp_mixer_get_blend_mode(intel_vvp_mixer_instance* instance, uint8_t layer)
```

Description

Returns the current run-time blending for a given layer. The instance must be valid and initialized with debug enabled. The layer must be a valid foreground layer.

Arguments

instance – pointer to the intel_vvp_mixer_instance software driver instance structure

layer – valid foreground layer in the range [1 ..
get_num_layers()-1]

Returns the current blending algorithm for the layer

intel_vvp_mixer_set_static_alpha

Prototype

```
int intel_vvp_mixer_set_static_alpha(intel_vvp_mixer_instance*
instance, uint8_t layer, uint32_t static_alpha)
```

Description Select the static alpha value for a given layer. The instance must be valid and initialized. The layer must be a valid foreground layer that supports static alpha blending

Arguments instance – pointer to the intel_vvp_mixer_instance software driver instance structure

layer – valid foreground layer in the range [1 ..
get_num_layers()-1]

blend_mode – selected blending

Returns kIntelVvpCoreOk if the static alpha value was changed, negative error code otherwise

kIntelVvpCoreInstanceErr if the instance is the null pointer

kIntelVvpMixerLayerErr if the layer is not in the valid range

intel_vvp_mixer_get_static_alpha

Prototype

```
uint32_t
intel_vvp_mixer_get_static_alpha(intel_vvp_mixer_instance*
instance, uint8_t layer)
```

Description Returns the current static alpha value for a given layer. The instance must be valid and initialized with debug enabled. The layer must be a valid foreground layer supporting alpha blending with a static value.

Arguments instance – pointer to the intel_vvp_mixer_instance software driver instance structure

layer – valid foreground layer in the range [1 ..
get_num_layers()-1]

Returns the static alpha value for the layer

intel_vvp_mixer_set_horiz_offset

Prototype

```
int intel_vvp_mixer_set_horiz_offset(intel_vvp_mixer_instance*  
instance, uint8_t layer, uint32_t offset)
```

Description

Select the horizontal offset for a given layer. The instance must be valid and initialized. The layer must be a valid foreground layer. The kIntelVvpMixerParameterErr error code is returned when trying to set an unsupported horizontal offset but the configuration is still changed in this case.

Arguments

instance – pointer to the intel_vvp_mixer_instance software driver instance structure

layer – valid foreground layer in the range [1 .. get_num_layers()-1]

offset – new horizontal offset

Returns

kIntelVvpCoreOk in case of success, negative error code otherwise

kIntelVvpCoreInstanceErr if the instance is the null pointer

kIntelVvpMixerLayerErr if the layer is not in the valid range

kIntelVvpMixerParameterErr if offset is not a multiple of the number of pixels in parallel and this is not supported by the layer

intel_vvp_mixer_set_vert_offset

Prototype

```
int intel_vvp_mixer_set_vert_offset(intel_vvp_mixer_instance*  
instance, uint8_t layer, uint32_t offset)
```

Description

Select the vertical offset for a given layer. The instance must be valid and initialized. The layer must be a valid foreground layer.

Arguments

instance – pointer to the intel_vvp_mixer_instance software driver instance structure

layer – valid foreground layer in the range [1 .. get_num_layers()-1]

offset – new vertical offset

Returns

kIntelVvpCoreOk in case of success, negative error code otherwise

kIntelVvpCoreInstanceErr if the instance is the null pointer

kIntelVvpMixerLayerErr if the layer is not in the valid range

intel_vvp_mixer_get_horiz_offset

Prototype

```
uint32_t
intel_vvp_mixer_get_horiz_offset(intel_vvp_mixer_instance*
instance, uint8_t layer)
```

Description Returns the current horizontal offset for a given layer. The instance must be valid and initialized with debug enabled. The layer must be a valid foreground layer.

Arguments instance – pointer to the intel_vvp_mixer_instance software driver instance structure

layer – valid foreground layer in the range [1 .. get_num_layers()-1]

Returns the horizontal offset for the layer

intel_vvp_mixer_get_vert_offset

Prototype

```
uint32_t
intel_vvp_mixer_get_vert_offset(intel_vvp_mixer_instance*
instance, uint8_t layer)
```

Description Returns the current vertical offset for a given layer. The instance must be valid and initialized with debug enabled. The layer must be a valid foreground layer.

Arguments instance – pointer to the intel_vvp_mixer_instance software driver instance structure

layer – valid foreground layer in the range [1 .. get_num_layers()-1]

Returns the vertical offset for the layer

intel_vvp_mixer_set_width

Prototype

```
int intel_vvp_mixer_set_width(intel_vvp_mixer_instance*
instance, uint8_t layer, uint32_t width)
```

Description Set the width of a foreground layer. The instance must be valid and initialized configured with the lite variant. The layer must be a valid foreground layer.

Arguments instance – pointer to the intel_vvp_mixer_instance software driver instance structure

layer – valid foreground layer in the range [1 .. get_num_layers()-1]

width – new width

Returns kIntelVvpCoreOk in case of success, negative error code otherwise

 kIntelVvpCoreInstanceErr if the instance is the null pointer or not the lite variant

 kIntelVvpMixerLayerErr if the layer is not in the valid range

intel_vvp_mixer_set_height

Prototype

```
int intel_vvp_mixer_set_height(intel_vvp_mixer_instance* instance, uint8_t layer, uint32_t height)
```

Description Set the height of a foreground layer. The instance must be valid and initialized configured with the lite variant. The layer must be a valid foreground layer.

Arguments instance – pointer to the intel_vvp_mixer_instance software driver instance structure

 layer – valid foreground layer in the range [1 .. get_num_layers()-1]

 height – new height

Returns kIntelVvpCoreOk in case of success, negative error code otherwise

 kIntelVvpCoreInstanceErr if the instance is the null pointer or not the lite variant

 kIntelVvpMixerLayerErr if the layer is not in the valid range

intel_vvp_mixer_get_width

Prototype

```
uint32_t intel_vvp_mixer_get_width(intel_vvp_mixer_instance* instance, uint8_t layer)
```

Description Returns the width selected for a given layer. The instance must be valid and initialized with debug enabled and in lite variant. The layer must be a valid foreground layer.

Arguments instance – pointer to the intel_vvp_mixer_instance software driver instance structure

 layer – valid foreground layer in the range [1 .. get_num_layers()-1]

Returns the width of the layer

intel_vvp_mixer_get_height

Prototype

```
uint32_t intel_vvp_mixer_get_height(intel_vvp_mixer_instance*
instance, uint8_t layer)
```

Description

Returns the height selected for a given layer. The instance must be valid and initialized with debug enabled and in lite variant. The layer must be a valid foreground layer.

Arguments

instance – pointer to the intel_vvp_mixer_instance software driver instance structure

layer – valid foreground layer in the range [1 .. get_num_layers()-1]

Returns

the height of the layer

intel_vvp_mixer_commit_writes

Prototype

```
int intel_vvp_mixer_commit_writes(intel_vvp_mixer_instance*
instance)
```

Description

Commits all pending writes before starting the next output field. The instance must be valid and parameterized in full mode.

Arguments

instance – pointer to the intel_vvp_mixer_instance software driver instance structure

Returns

kIntelVvpCoreOk (0) for success, negative error code otherwise

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

38. Pixels in Parallel Converter IP

38.1. About the Pixels in Parallel Converter IP

The Altera Streaming Video Protocol allows multiple pixels to be transmitted in a single clock cycle (beat). The number of pixels the interface transmits per beat (pixels in parallel) is a fixed property of the interface. The IP converts from one value of pixels in parallel at the input interface to a higher or lower number of pixels in parallel at the output interface. The IP supports any number of pixels in parallel between 1 and 8 for both the input and output interface and supports all possible conversions.

To help manage data rates, the IP includes the option for a FIFO buffer on the datapath. For conversions that lower the pixels in parallel, the IP locates the FIFO buffer at the input interface, before the conversion logic. For conversions that increase the pixels in parallel, the IP locates the FIFO buffer at the output interface, after the conversion logic. A parameter selects either single clock or dual clock mode for the FIFO buffer. If you select dual clock mode, the input and output interfaces can run on different clock domains.

To correctly implement the pixels in parallel conversion in all cases, the IP must know how many pixels are in each video line. Without this information the IP does not know how many of the pixels in parallel are valid on the final beat of each video line packet. If you configure the IP for use with the full variant of the Altera Streaming Video Protocol, you can obtain this information directly from the image information packets contained within the video stream. The IP has no requirement for a register map or for the control agent interface to access it. If you configure for the lite variant of the protocol, the video stream has no image information so you must supply the line length through the register map, via the control agent interface. Selecting to use the lite variant of the protocol automatically enables the control agent interface.

38.1.1. Pixels in Parallel Converter IP Performance and Resources

Table 678. Pixels in Parallel Converter IP Performance and Resources

Target device	ALMs	f _{MAX} (MHz)
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	80	1,000
Arria 10 speed grade 1 (10AS066H1F34E1HG)	60	645
Cyclone 10 GX speed grade 5 (10CX220YF78015G)	61	645
Stratix 10 GX speed grade 1 (1SX280LN2F43E1VG)	81	746

38.2. Pixels in Parallel Converter IP Parameters

Table 679. Pixels in Parallel Converter IP Parameters

Parameter	Allowed range	Description
Video data format		
Lite mode	On or off	Turn on to use the lite variant of the Altera Streaming Video Protocol.
Bits per color sample	8 to 16	Select the number of bits per color sample.
Number of color planes	1 to 4	Select the number of color planes per pixel.
Input pixels in parallel	1 to 8	Select the number of pixels in parallel at the input interface.
Output pixels in parallel	1 to 8	Select the number of pixels in parallel at the output interface.
Datapath settings		
Dual clock	On or off	Turn on to operate dual clocks.
FIFO depth	0, 32, 64, 128, 256, 1024, 2048	Specify the depth of the FIFO in words (beats of data). Select 0 to omit the FIFO entirely. The FIFO depth must be > 0 if you turn on Dual clock .
Control settings		
Separate clock for control interface	On or off	Turn on to enable a separate clock for the control agent interface.
Debug features	On or off	Turn on to read back frame information registers via the control agent interface.
Pipeline optimization		
Pipeline ready signals	On or off	Turn on to add extra pipeline registers to the AXI4-S <code>tready</code> signals.

38.3. Pixels in Parallel Converter Interfaces

Table 680. Pixels in Parallel Converter Interfaces

Signal name	Direction	Width	Description
Clocks and resets			
<code>main_clock_clk</code>	In	1	AXI4-S processing clock. Only available when you turn off Dual clock .
<code>main_reset_rst</code>	In	1	AXI4-S processing reset. Only available when you turn off Dual clock .
<code>in_clock_clk</code>	In	1	AXI4-S processing clock for the input interface domain. Only available when you turn on Dual clock .
<code>in_reset_rst</code>	In	1	AXI4-S processing reset for the input interface domain. Only available when you turn on Dual clock .
<code>out_clock_clk</code>	In	1	AXI4-S processing clock for the output interface domain. Only available when you turn on Dual clock .
<code>out_reset_rst</code>	In	1	AXI4-S processing reset for the output interface domain. Only available when you turn on Dual clock .
<i>continued...</i>			

Signal name	Direction	Width	Description
agent_clock_clk	In	1	Clock for the Avalon memory-mapped control agent interface. Only available if you select Separate clock for control interface .
agent_reset_rst	In	1	Reset for the Avalon memory-mapped control agent interface. Only available if you select Separate clock for control interface .
Control interfaces			
av_mm_control_agent_address	In	7	Avalon memory-mapped agent address
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request
Altera Streaming Video interfaces			
axi4s_vid_in_tdata	In		AXI4-S data in.
axi4s_vid_in_tvalid	In	1	AXI4-S data valid.
axi4s_vid_in_tuser	In		AXI4-S tuser tuser[0] indicates start of video frame when asserted tuser[1] indicates the start of an image information, end-of-field, or auxiliary control packet when asserted.
axi4s_vid_in_tlast	In	1	AXI4-S end of packet.
axi4s_vid_in_tready	Out	1	AXI4-S data ready.
axi4s_vid_out_tdata	Out	(119)	AXI4-S data in,
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid,
axi4s_vid_in_tuser	Out	(120)	AXI4-S tuser tuser[0] indicates start of video frame when asserted tuser[1] indicates the start of an image information, end-of-field, or auxiliary control packet when asserted.
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet.
axi4s_vid_out_tready	In	1	AXI4-S data ready.

Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

(119) The equation gives all tdata sizes in these interfaces:

$$\max(\text{ceil}((\text{bits per color sample} \times \text{number of color planes} \times \text{pixels in parallel}) / 8) \times 8, 16)$$

(120) The equation gives all tuser widths in these interfaces:

$$N = \text{ceil}(\text{tdata width} / 8)$$

38.4. Pixels in Parallel Converter Registers

Each register is either read-only (RO) or read-write (RW). Writes to RO registers complete but the IP ignores them. A read to any RW register also completes, but returns undefined data by default. These registers are RW to save FPGA resources. To read back the values you write to these registers, you must turn on **Debug features** when you configure the IP.

Table 681. Pixels in Parallel IP Registers

In the software API the register names appear with a prefix of `INTEL_VVP`, `INTEL_VVP_CORE` or `INTEL_VVP_CLIPPER` as appropriate and with an optional `REG` suffix

Address	Register	Access	Description
Parameterization registers			
0x0000	VID_PID	RO	Read this register to retrieve the pixels in parallel converter product ID. This register always returns 0x6AF7_0239.
0x0004	VERSION	RO	Read this register for the IP version information.
0x0008	LITE_MODE	RO	Read this register to determine if Lite mode is on. This register always returns 1 as the Avalon memory-mapped control interface is only available when you turn on Lite mode .
0x000C	DEBUG_ENABLED	RO	Read this register to determine if Debug features is on. This register returns 1 if reads to other registers designated as RW return the last value the IP writes to the register, or an undefined value.
0x0010 to 0x011F	-	-	Unused
Control and debug registers			
For more information, refer to <i>Control Packets</i>			
0x0120	IMG_INFO_WIDTH	RW	Set the expected width of incoming video fields.
0x0124 to 0x0130	-	-	Unused.
0x0134	IMG_INFO_SUBSAMPLING	RW	The expected chroma subsampling of the incoming video fields.
0x0138 to 0x013C	-	-	Unused.
0x0140	STATUS	RO	Bit 0: Status bit. 1 means the pixels in parallel converter is processing a video field, 0 otherwise.

Register Bit Descriptions

Table 682. VID_PID

Name	Bits	Description
Pixels in parallel converter version ID and product ID	31:0	This register always returns 0x6AF7_0239. 15:0 is the product ID and always returns 0x0239 31:16 is the vendor ID and always returns 0x6AF7

Table 683. VERSION

Name	Bits	Description
Register map version	7:0	Register map version.
continued...		

IP patch revision	15:8	-
IP update revision	23:16	Updated when the IP version changes.
IP major revision	31:24	Updated when the IP version changes.

Table 684. LITE_MODE

Name	Bits	Description
Lite mode parameterization bit	0	Returns 1 if you turn on Lite mode .
Unused	31:1	Unused.

Table 685. DEBUG_ENABLED

Name	Bits	Description
Debug features parameterization bit	0	Returns 1 if you turn on Debug features .
Unused	31:1	Unused.

Table 686. IMG_INFO_WIDTH

Name	Bits	Description
Width bits	15:0	Write to this register to set the expected width of the incoming video fields.
unused	31:16	Unused.

Table 687. IMG_INFO_SUBSAMPLING

Name	Bits	Description
SubSa code bits	1:0	Write to this register to set the expected chroma sub-sampling of the incoming video fields.
unused	31:2	Unused.

Table 688. STATUS

Name	Bits	Description
Status bit	0	1 means the pixels in parallel converter is processing a video field, 0 otherwise.
unused	31:1	Unused.

38.5. Pixels in Parallel Converter IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_pip_conv_regs.h`

Include file: `intel_vvp_pip_conv.h`

Table 689. Pixels in Parallel IP API Reference

Name	Description
intel_vvp_pip_conv_init	Initialize the pixels in parallel instance
Intel_vvp_core_*	.Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i> . Writable when Lite is on. Readable when Lite is off and Debug features is on.
intel_vvp_pip_conv_get_lite_mode	Returns if Lite mode is on
intel_vvp_pip_conv_get_debug_enabled	Returns if Lite mode is on
intel_vvp_pip_conv_is_running	Returns if the IP is processing data
intel_vvp_pip_conv_get_status	Reads the status register

intel_vvp_pip_conv_init

Prototype

```
int intel_vvp_pip_conv_init(intel_vvp_pip_conv_instance
*instance, intel_vvp_core_base base);
```

Description

Initializes a pip_conv instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance does not fully initialize and should not be used further by the application if returning a non-zero error code.

Arguments

instance – pointer to the intel_vvp_pip_conv_instance software driver instance structure

base – base address of the register map

Returns

kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the IP is not the Altera vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product_id does not match with the Pixels in Parallel product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpPipConvRegMapVersionErr if the register map is not supported

intel_vvp_pip_conv_get_lite_mode

Prototype

```
bool
intel_vvp_pip_conv_get_lite_mode(intel_vvp_pip_conv_instance
*instance);
```


Description Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_pip_conv_instance fully initialized

Arguments instance – pointer to the intel_vvp_pip_conv_instance software driver instance structure

Returns true in case of success because the IP does not have an agent interface when parameterized in full mode

intel_vvp_pip_conv_get_debug_enabled

Prototype

```
bool  
intel_vvp_pip_conv_get_debug_enabled(intel_vvp_pip_conv_instance  
*instance);
```

Description Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_pip_conv_instance fully initialized.

Arguments instance – pointer to the intel_vvp_pip_conv_instance software driver instance structure

Returns true if the IP is parameterized with debug features enabled

intel_vvp_pip_conv_is_running

Prototype

```
bool intel_vvp_pip_conv_is_running(intel_vvp_  
pip_conv_instance* instance);
```

Description Reads and returns the running bit of the STATUS register. The instance must be a valid intel_vvp_pip_conv_instance fully initialized.

Arguments instance – pointer to the intel_vvp_pip_conv_instance software driver instance structure

Returns True is the IP is currently generating an output field, false when disabled and/or between two fields

intel_vvp_pip_conv_get_status

Prototype

```
uint8_t  
intel_vvp_pip_conv_get_status(intel_vvp_pip_conv_instance*  
instance);
```

Description Reads and returns the STATUS register. The instance must be a valid intel_vvp_pip_conv_instance fully initialized.

Arguments instance – pointer to the intel_vvp_pip_conv_instance software driver instance structure

Returns The value read from the status register

Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

39. Scaler IP

39.1. About the Scaler

The Scaler IP resizes the fields in an Altera Streaming Video compliant input to produce output fields of a different height and or width. The IP supports both full and lite variants of interface protocol. The scaler offers three algorithms for resizing images: nearest neighbor, bilinear and polyphase. The three algorithms offer three cost versus quality options. Nearest neighbor offers the lowest FPGA resource cost and the lowest output image quality and polyphase offers the highest quality at the highest cost.

The scaler can resize an incoming video field to produce an output field of any size, restricted only by defined minimum and maximum widths and heights. You set the desired output field width and height at runtime via the register map (if the Avalon memory-mapped control agent interface is enabled), or via fixed parameters (if the Avalon memory-mapped control agent interface is not enabled). Full variants define the input field width and height by the image info packets received at the input interface. Lite variants define the input field width and height via the register map. With defined input and output widths and heights, the scaler applies the correct horizontal and vertical scaling ratios (which may be different).

The following bounds apply to the input and output field widths.

- Input field width must be less than or equal to the value set in the maximum input field width parameter.
- Output field width must be less than or equal to the value set in the maximum output field width parameter
- Output field width must be greater than or equal to the value set in the number of pixels in parallel parameter

You may also choose to turn off scaling in the horizontal or vertical directions if either is not required. In this case, the output width or height is unaltered, regardless of the settings in the register map. Also, the scaler only supports scaling for video with 4:4:4 chroma siting. Full variants assert the error flag for any of these restrictions.

39.1.1. Scaler IP Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family and the number of supported bits per color sample, pixels in parallel, and number of color planes.

Table 690. Scaler IP Performance and Resources

The table shows ALM usage and f_{MAX} for a design with 10 bits per color sample, 3 color planes, 4 pixels in parallel, 4 tap, 16 phase polyphase scaling, 444 chroma sampling only.

Target device	ALM	M20K	DSP	f_{MAX}
Agilex 7 speed grade 2 (AGFA012R24A2E2V)	6,097	28	48	758
Arria 10 speed grade 1 (10AS066H1F34E1HG)	5,413	28	48	511
Cyclone 10 GX speed grade 5 (10CX220YF78015G)	5,917	28	48	449
Stratix 10 GX speed grade 1 (1SX280LN2F43E1VG)	6,414	28	48	636

39.2. Scaler IP Parameters

Table 691. Scaler IP Parameters

Parameter	Allowed range	Description
Video data format		
Lite mode	On or off	Turn on to use the lite variant of the Altera Streaming Video Protocol.
Bits per color sample	8 to 16	Select the number of bits per color sample.
Number of color planes	1 to 4	Select the number of color planes per pixel.
Number of pixels in parallel	1 to 8	Select the number of pixels transmitted every clock cycle.
444 chroma sampling	On or off	Turn on to support scaling of 444 chroma sampled video data.
422 chroma sampling	On or off	Turn on to support scaling of 422 chroma sampled video data.
420 chroma sampling	On or off	Turn on to support scaling of 420 chroma sampled video data.
Disable flush/fill between frames	On or off	Turn on to turn off the flush and refill of the line buffer between frames.
Maximum input field width	1 to 65536	Set the maximum supported input field width. If you turn on horizontal partial scaling , the value in this parameter specifies the size of the internal line buffer. The value you specify should be the maximum number of pixels expected per line in the tile of the overall image that this scaler processes, including any overscan data.
Maximum output field width	1 to 65536	Set the maximum supported output field width. If Memory-mapped control interface is off, this value sets the fixed output field width. If you turn on horizontal partial scaling , this parameter sets the precision of the internal fixed-point arithmetic to calculate the pixel drop and repeat patterns. Set this value to the size of the maximum expected input or output image before tiling, which is the maximum overall input or output image width.
Output Picture Height	1 to 65536	Set the height for the output fields. Only used if Memory-mapped control interface is off.
Control settings		
Memory-mapped control interface	On or off	Turn on for the Avalon memory-mapped control agent interface and to allow run-time configuration via the register map. You must have the Avalon memory-mapped control agent interface if you turn on Lite mode .
Scaling		
continued...		

Parameter	Allowed range	Description
Scaling algorithm	Nearest Neighbor, Bilinear, Polyphase	Select the algorithm to resize the video fields
Edge behavior	Replicate edge pixel or Mirror edge pixels	Only with Polyphase . Controls which pixel values fill filter taps that otherwise are empty (as they are off-screen) when scaling around the edges of the image. Refer to <i>Filter Behavior at Edge Boundaries</i>
Runtime coefficient updates	On or off	Only with Polyphase . Turn on for updates to the scaling coefficients at run time via the Avalon memory-mapped control agent interface.
Initialize coefficients at startup	On or off	Only with Polyphase . Turn on to initialize the scaling coefficients to preset values at startup. Select the coefficient values in the Vertical coefficient function and Horizontal coefficient function parameters
Vertical scaling		
Vertical scaling	On or off	Turn on for image resizing in the vertical direction
Vertical partial image scaling	On or off	Turn on for partial image scaling in the vertical direction
Mirror 420 chroma data	On or off	Only with 420 chroma sampling on. Turn on to reduce the memory resources required to implement the line buffer for scaling 420 sampled data (refer to <i>422 and 420 Chroma Sampled Data Scaling</i>)
Number of vertical taps	1 to 64	Polyphase only. Set the number of vertical scaling filter taps
Number of vertical phases	2 to 256	Polyphase only. Set the number of vertical scaling filter phases
Number of vertical banks	1 to 16	Polyphase only. Set the number of vertical scaling filter coefficient banks
Use signed vertical coefficients	On or off	Polyphase only. Turn on to use signed coefficients for the vertical scaling filter
Vertical coefficient integer bits	0 to 17	Polyphase only. Set the number of integer bits used to represent the vertical scaling filter coefficients
Vertical coefficient fraction bits	1 to 18	Bilinear and polyphase only. Set the number of fraction bits to represent the vertical scaling filter coefficients
Fraction bits preserved between vertical and horizontal scaling	0 to vertical coefficient fraction bits	Bilinear and polyphase only. Set the number of fraction bits to preserve in the intermediate result between the vertical and horizontal scaling pipelines
Vertical coefficient function	Bicubic, Lanczos 1, Lanczos 2, Lanczos 3 or Lanczos 4	If you turn on Initialize coefficients at startup , select the function that calculates the initial vertical scaling coefficient values. If you also turn on Runtime coefficient updates , you can overwrite these values at run time.
Horizontal scaling		
Horizontal scaling	On or off	Turn on for image resizing in the horizontal direction
Horizontal partial image scaling	On or off	Turn on for partial image scaling in the horizontal direction (see <i>Partial Frame Scaling</i>)
Half rate 420	On or off	Turn on to specify that you supply only 420 chroma sampled data at data rates up to 50% of the maximum supported by the interface. Turning on reduces the hardware resources that the design requires (refer to <i>422 and 420 Chroma Sampled Data Scaling</i>)
continued...		

Parameter	Allowed range	Description
Number of horizontal taps	1 to 64	Polyphase only. Set the number of horizontal scaling filter taps
Number of horizontal phases	2 to 256	Polyphase only. Set the number of horizontal scaling filter phases
Number of horizontal banks	1 to 16	Polyphase only. Set the number of horizontal scaling filter coefficient banks
Use signed horizontal coefficients	On or off	Polyphase only. Turn on to use signed coefficients for the horizontal scaling filter
Horizontal coefficient integer bits	0 to 17	Polyphase only. Set the number of integer bits used to represent the horizontal scaling filter coefficients
Horizontal coefficient fraction bits	1 to 8	Bilinear and polyphase only. Set the number of fraction bits used to represent the horizontal scaling filter coefficients
Horizontal coefficient function	Bicubic, Lanczos 1, Lanczos 2, Lanczos 3 or Lanczos 4	If you turn on Initialize coefficients at startup , select the function that calculates the initial vertical scaling coefficient values. If you also turn on Runtime coefficient updates , you can overwrite these values at run time.
General		
Separate clock for control interface	On or off	Turn on for a separate clock for the control agent interface
Debug features	On or off	Turn on to read back writeable registers via the control agent interface
Pipeline ready signals	On or off	Turn on to add extra pipeline registers to the AXI4-S <code>tready</code> signals

Related Information

[Filter Behavior at Edge Boundaries](#) on page 696

39.2.1. Scaler Maximum Resolution

The scaler supports input and output field widths and heights up to 65336 pixels. You must ensure that the **Number of pixels in parallel** combined with the clock rate gives sufficient bandwidth to process the incoming and outgoing fields at the required rate.

39.3. Scaler IP Functional Description

The scaler supports three options for the algorithm that resizes the video fields: nearest neighbor, bilinear and polyphase.

Nearest neighbor

Nearest neighbor is the lowest cost and lowest quality algorithm. Each pixel in the output field is a direct copy of a pixel from the input field, with no filtering or interpolation between pixels. The algorithm repeats or drops input pixels according to the required scaling ratio. If $out[i,j]$ is the pixel value at horizontal position i and vertical position j in the output field. $in[x,y]$ is the pixel value at horizontal position x and vertical position y in the input field. The equation defines the pixel values in the output field:

Equation 4. Nearest Neighbor Equations

$$x = \left\lfloor \frac{i \times width_{in}}{width_{out}} \right\rfloor, y = \left\lfloor \frac{j \times height_{in}}{height_{out}} \right\rfloor, out[i, j] = in[x, y]$$

The nearest neighbor algorithm requires a line buffer with storage for a single line of the input field to allow for this line to repeat multiple times at the output during upscales. Because nearest neighbor scaling does not involve any interpolation or filtering, the resulting image can have a blocky appearance.

Bilinear

Bilinear scaling offers improved quality compared to nearest neighbor by interpolating between neighboring pixels to remove the blocky look of nearest neighbor scaling. Bilinear scaling is better suited to upscaling (increasing image size) than downscaling (reducing image size). The cut-off frequency of the basic bilinear filter is generally too high to remove all the aliasing artifacts that downscaling can introduce. However, even for upscales the results can look somewhat blurred, with the edges in the image softened.

The bilinear algorithm selects the same input pixel to create each output pixel as the nearest neighbor algorithm. It builds a 2x2 pixel window around the target input pixel, with the target pixel in the top left corner of the window. The *floor* function calculates the input pixel position (the values of *x* and *y*) to give integer indices, as pixels only exist at integer locations. But the integer indices have some error compared to the ideal location that preserves all the fractional position information. For example, if the ideal value before applying the *floor* function is 1.5, the integer value after applying the *floor* function is 1, and the error is 0.5. The bilinear algorithm uses the horizontal and vertical position error values, *err_h* and *err_v* respectively, to create coefficients that, when applied to the 2x2 pixel window created around the integer pixel location, produce a resulting pixel that is effectively located at the desired fractional position. The equations show how the values of the coefficients are created and applied to the input window of pixels to create the output pixel.

Equation 5. Bilinear Equations

$$\begin{aligned} coeff_0 &= ((1 \ll frac_h) - err_h) \times ((1 \ll frac_v) - err_v) \\ coeff_1 &= err_h \times ((1 \ll frac_v) - err_v) \\ coeff_2 &= ((1 \ll frac_h) - err_h) \times err_v \\ coeff_3 &= err_h \times err_v \\ out[i, j] &= in[x, y] \times coeff_0 + in[x + 1, y] \times coeff_1 + in[x, y + 1] \times coeff_2 + in[x + 1, y + 1] \times coeff_3 \end{aligned}$$

To calculate the values of *err_h* and *err_v* with exact precision for all possible scaling ratios requires an infinite number of fractional bits in the hardware implementing the mathematics. You must specify via parameters how many fraction bits you want to include for the calculations in the horizontal and vertical directions, *frac_h* and *frac_v* respectively. The IP takes the value for *frac_h* from the **Horizontal coefficient fraction bits** parameter, and the value for *frac_v* from the **Vertical coefficient fraction bits** parameter. With the desired level of precision set, the equation shows the values of and

$$err_h = \frac{(i \times width_{in} \% width_{out}) \ll frac_h}{width_{out}}, err_v = \frac{(j \times height_{in} \% height_{out}) \ll frac_v}{height_{out}}$$

To create the 2x2 pixel window required for the bilinear filter, the bilinear algorithm requires a line buffer with storage for two lines of input video.

Polyphase

The polyphase algorithm requires the most resources, but it produces the highest quality results. It uses interpolation filters that are larger than the 2x2 tap filter used for bilinear scaling. Depending on the coefficients you select, these filters can provide improved frequency response, resulting in less blurring on the edges during upscales, and less aliasing artifacts during downscales. The increased size of the filters requires an increase in the number of input video lines that must be stored to create the vertical window, and increased DSP block (multiplier) usage to implement the filter mathematics.

The polyphase algorithm uses the same initial integer pixel position as nearest neighbor scaling and calculates positional error values in the same way as the bilinear algorithm. However, instead of using these error values directly to calculate the filter coefficients, the polyphase algorithm uses the error values as addresses into horizontal and vertical filter coefficients memories. Each address in the coefficient memory is referred to as a phase (for reasons that are explained in the coefficient selection section) and you define the number of horizontal and vertical phases, num_phase_h and num_phase_v respectively, via parameters. The **Number of horizontal phases** parameter sets the value for num_phase_h and the **Number of vertical phases** parameter sets the value for num_phase_v . The equation shows the horizontal phase, $phase_h$, and the vertical phase, $phase_v$, for each output pixel:

Equation 6. Phase Equations

$$phase_h = \frac{(i \times width_{in} \% width_{out}) \times num_phases_h}{width_{out}} \quad phase_v = \frac{(i \times height_{in} \% height_{out}) \times num_phases_v}{height_{out}}$$

You define the number of taps used in the horizontal and vertical scaling filters (num_taps_h and num_taps_v respectively). The number of taps can be any value between 4 and 64. A higher number of taps can allow for a more precise filter transfer function but comes at the cost of extra DSP block utilization and, in the case of the vertical filter, increased block memory utilization in the line buffer required to create the vertical sample window. Each vertical or horizontal phase in the coefficient memory contains one coefficient for each tap of the vertical or horizontal filter.

The scaler implements the vertical scaling function first (if selected), followed by the horizontal scaling function (if selected). The result of the vertical scaling is an intermediate image with the desired output height but retaining the original input width. If $inter[x,j]$ is the pixel value in the intermediate image at horizontal position x and vertical position j , and $coeff_f_v[n]$ as the vertical scaling filter coefficient for tap N (selected from $phase_v$), the equation shows how the IP calculates the intermediate image. The filter taps are indexed with 0 the 'oldest' data (closest to the top edge of the image) and $num_taps_v - 1$ the newest data (closest to the bottom edge of the image).

Equation 7. Intermediate Image Equation

$$inter[x,j] = \sum_{n=0}^{num_taps_v} in \left[x, \left(y - \frac{(num_taps_v - 1)}{2} + n \right) \right] \times coeff_f_v[n]$$

If $coeff_f_h[n]$ is the horizontal scaling filter coefficient for tap N (selected from $phase_v$), the equation shows how the IP calculates the final output image from the intermediate image. The filter taps are again indexed with 0 the oldest data (closest to the left edge of the image) and $num_taps_h - 1$ the newest data (closest to the right edge of the image)

Equation 8. Final Output Image Equation

$$out[i,j] = \sum_{n=0}^{num_taps_h} inter \left[\left(x - \frac{(num_taps_h - 1)}{2} + n \right), j \right] \times coef_h[n]$$

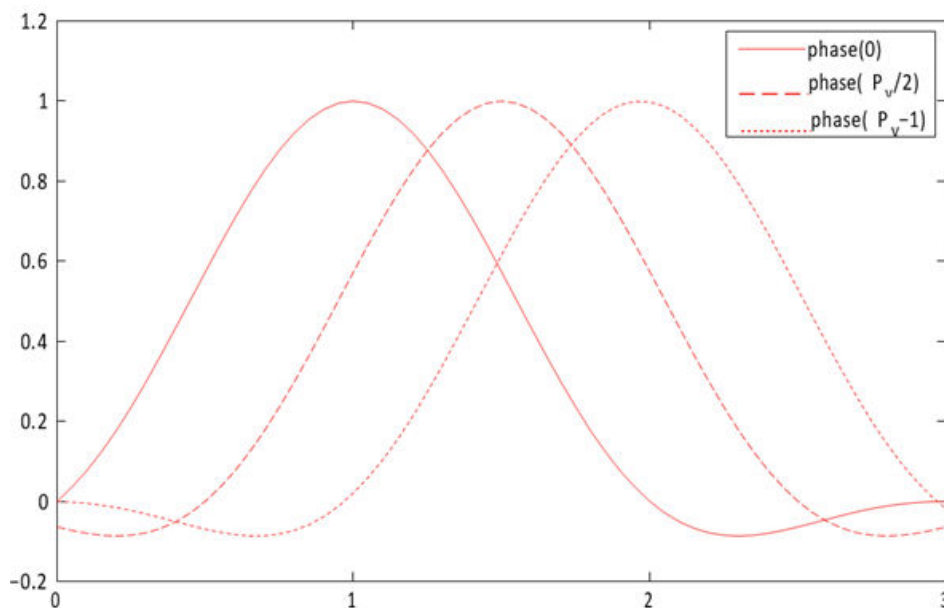
39.3.1. Coefficient Selection

If you select polyphase scaling for the Scaler IP, the coefficients that the scaling filters use are read from a memory. You must define the contents of this memory. Either specify fixed horizontal and vertical coefficient sets in the **Horizontal coefficient function** and **Vertical coefficient function** parameters or turn on **Update coefficients at runtime**. If you turn on **Update coefficients at runtime**, you can write whatever values you want to the coefficient memory. If you already have well defined coefficient sets that you use, the flexibility in the scaler to select the desired number of filter taps and phases should allow you to continue with these. If you are new to scaling, read the following guidance on coefficient selection.

Generally, the set of coefficients that the IP writes to filter phase 0 yield a low-pass filter, with most weight in generating the output pixel value given to the pixel value in tap $\frac{num_taps - 1}{2}$. The function is centered at tap $\frac{num_taps - 1}{2}$. The coefficients the IP writes to the other phases are then just phase-shifted versions of this function (hence the name phase for each coefficient address), with the function centered at a point that is shifted by $1/ num_of_pixels$ of a pixel with every subsequent phase.

Figure 103. 2-lobe Lanczos function at 3 different phases

This figure shows an example of how a function is progressively phase-shifted to create the coefficients for each scaling filter phase. The figure shows a 4 tap filter, with the taps shown on the x-axis



The Lanczos function is a common function that defines scaling coefficients. The Lanczos function is a base sinc function, with the primary lobe of a sinc function used as a window function to preserve a given number of lobes of the base sinc. The number of lobes is generally appended to Lanczos when referring to a specific variant

of the function, so a Lanczos function where two lobes are preserved is referred to as Lanczos2. In the case where N lobes are preserved, the Lanczos N function is defined as:

Equation 9. Coefficient Selection Equation

$$\text{Lanczos}N(x) = \begin{cases} 1 & \text{if } x = 0 \\ \frac{\sin(\pi x)}{\pi x} \times \frac{\sin(\pi x/N)}{\pi x/N} & \text{if } x \neq 0 \text{ and } |x| < N \\ 0 & \text{if } |x| \geq N \end{cases}$$

When using Lanczos N coefficients, Altera recommends configuring the scaler filters with the following numbers of taps for the upscale and downscale cases:

- Upscale: $2 \times N$
- Downscale: $\begin{cases} 2 \times N \times \frac{\text{width}_{in}}{\text{width}_{out}} \text{ for horizontal filters} \\ 2 \times N \times \frac{\text{height}_{in}}{\text{height}_{out}} \text{ for vertical filters} \end{cases}$

The number of lobes in the Lanczos function affects the frequency response of the filter and, as a result, the quality of the image produced. Generally, Lanczos functions with lower numbers of lobes give a softer frequency response and a resulting image with more blur on the edges, but with less risk of ringing artifacts in the areas immediately around the edges. Conversely, Lanczos functions with higher numbers of lobes give sharper edges but introduce more ringing artifacts. Lanczos2 is a good compromise between minimizing blur and minimizing ringing, but you can experiment with Lanczos3 or Lanczos4 to make your own judgment. The higher the number of lobes, the more filter taps (and therefore FPGA device resources) the design requires to implement the filter correctly. You should experiment with Lanczos1 coefficients for large downscales.

39.3.1.1. Updating Scaler IP Runtime Coefficients

If you select **polyphase** and turn on **runtime coefficient updates**, you can edit the values in the horizontal and vertical scaling coefficient memories at any time via the Avalon memory-mapped agent interface.

To minimize the implementation cost of the coefficient memories, you cannot directly access the value of each coefficient through a single read or write to the register map. You must write the values for the coefficients in a given phase (vertical or horizontal) to a set of addresses in the register map. Then declare through separate addresses which phase and which bank the IP writes the coefficients to.

1. Write the values of the coefficients for one phase to the RT_COEFF_LOAD_TAP_Xregisters ($0 \leq X < 64$, addresses 0x020C to 0x0308). The IP has 64 addresses into which you may write new coefficients. If the coefficient phase you are updating only has coefficients for N taps (where $N < 64$), you need only write values to the first N addresses in this range.
2. Write the index of the bank to which the IP should write this phase of coefficients to the RT_COEFF_LOAD_BANK_SELECT register (address 0x0200).
3. Write the index of the phase to which the IP should write the coefficients to the RT_COEFF_LOAD_PHASE_SELECT register (address 0x0204).
4. Write to the RT_COEFF_LOAD_COMMIT register (address 0x208) to commit the new coefficients to the specified phase in the specified bank. Write 1 to this register if it is a horizontal scaling filter coefficient phase, and 0 if it is a vertical scaling filter coefficient phase.

You must repeat this process for every phase in every bank that you want to update. Unlike updates to settings registers (such as the output field width), coefficient updates take immediate effect. The IP does not hold them until a frame boundary. To avoid the IP updating coefficients in the middle of a frame, Altera recommends using at least two banks of coefficients. This way the software controlling the system can make updates to the coefficients in one bank, while the scaler reads coefficients for processing from the other bank. When the coefficient updates are complete, the software can swap the coefficient read bank to the newly updated bank by updating the value in the H_BANK or V_BANK address in the register map. These registers are standard settings registers, so the update to the coefficient read bank only takes effect at a frame boundary.

39.3.2. Coefficient Quantization

The scaler IP implements the filters when you select **Polyphase** algorithm using fixed-point logic, so you must supply the filter coefficients in a fixed-point format. You define the format for the coefficients with parameters that select whether they are signed or unsigned, the number of integer bits, and the number of fraction bits.

- Signed or unsigned: if you want to represent negative coefficients, turn on **Use signed vertical coefficients**. If all coefficients are positive values, reduce logic and turn off **Use signed vertical coefficients**.
- Integer bits: the number of integer bits defines the maximum value that can be represented.
- Fraction bits: the number of fraction bits defines the precision with which the IP can convert floating-point coefficients into the fixed-point format.

The overall bit width of each coefficient is the sum of the integer and fraction bits, plus one extra bit for signed coefficients. When using Lanczos coefficients, Altera recommends the following settings:

- Turn on **Use signed vertical coefficients** as the Lanczos function for any number of lobes greater than 1 requires negative values, so the coefficients must be signed.
- Use 1 integer bit as the maximum value required for any Lanczos coefficient is 1.0
- Use between 6 and 8 fraction bits.

Typically, the filter coefficients produce noninteger floating-point values. To convert each floating-point coefficient into its closest quantized representation in the selected fixed-point format:

- Multiply each coefficient by 2^{frac} , where *frac* is the number of fraction bits you select
- Apply float to integer conversion to each coefficient

However, small errors in the coefficient values introduced by the quantization process can accumulate so that the coefficients in each phase no longer sum to their intended value. Generally, the coefficients in any phase should sum to exactly 1.0. Any value greater than 1.0 increases the overall brightness of the resulting image. Any value less than 1.0 reduces the brightness. The coefficients can sum to more or less than 1.0 if you want a brighter or darker image. You should still ensure your coefficients sum to your original, intended value post quantization. To restore the coefficients to values that sum to the intended value:

Figure 104. Restore Coefficient Values

```
float quantization_error = 0.0;
for (int j = 0; j < taps; j++) {
    quantization_error += original_float_coeff[j] - ((float)quant_coeff[j]);
    if (quantization_error < -0.5) {
        quant_coeff[j]--;
        quantization_error += 1.0;
    } else {
        if (quantization_error > 0.5) {
            quant_coeff[j]++;
            quantization_error -= 1.0;
        }
    }
}
```

39.3.3. Filter Behavior at Edge Boundaries

For the Scaler IP, when you select **Polyphase** and select *N* filter taps, $(N-1)/2$ of these taps are above (vertical scaling) or to the left of (horizontal scaling) the center tap of the filter. $N/2$ of the taps are below (vertical scaling) or to the right of (horizontal scaling) the center pixel.

When scaling pixels around the top, bottom, left or right edges (boundaries) of the image, the filter taps above, below, to the left of or to the right of the center pixel naturally attempt to sample pixels that do not exist as they are in off-screen locations. To avoid using undefined data in these taps, the scaler detects when any tap is off-screen and replaces the data in that tap with data from an on-screen pixel. The **Edge behavior** parameter allows you to choose between two methods for filling the off-screen taps:

- **Replicate edge pixels.** The IP replicates the pixel closest to the edge to fill all off-screen taps
- **Mirror edge pixels.** The pixels leading up to the image edge reflect into the off-screen area, just as if a mirror is at the edge of the image. The reflected pixel values fill any off-screen taps.

Edge mirroring for a filter with *N* taps requires a minimum input image size of $N/2$ pixels to successfully fill all off-screen taps. Input images smaller than this minimum create undefined outputs. Edge replication works for all image sizes, down to 1x1 pixel. You cannot swap between edge methods at runtime.

You can only select this parameter if you select **Polyphase** scaling. **Nearest neighbor** scaling has no filter and no off-screen taps. **Bilinear** scaling requires a 2 tap filter, but edge replication and edge mirroring give the same result when the number of taps is 2, so the parameter is not required.

39.3.4. Partial Frame Scaling

For large video wall applications, you may want to scale video frames using multiple scaler IPs. The output frames are split into tiles (either horizontally, vertically or horizontally and vertically), and each scaler IP processes the data for a single tile.

Typically, the goal is for the combined output image to look identical to how it looks when processed by a single scaler, without tiling. This result is not achieved automatically when using multiple scalers, and the IP requires additional information about the horizontal and vertical offset position of the scaler inside the combined image. If the scaling algorithms are not offset correctly for the chosen tiling, visible seams may appear in the combined output image. You can configure the scaler to support partial frame scaling, with separate parameters to enable tiling in the horizontal and vertical directions (**Horizontal partial image scaling** and **Vertical partial image scaling** respectively).

If you turn on partial image scaling in either the horizontal or vertical direction, you must provide additional offset information at run time via the register map. You must enable the Avalon memory-mapped control agent interface if you turn on partial scaling. The additional control values specify the fractional pixel offset for the first input pixel, an initial phase offset, and a fractional phase offset. You must also supply the resolutions of the untiled input and output images so that the IP can calculate the overall scaling ratio. For further details of the register map and how control values should be set for partial scaling, see *Scaler Registers*.

When using partial frames, you must calculate which portion of the overall input image is required to create the given output tile. Only send the required tile in the input image to the scaler IP. The following example demonstrates the calculation of the horizontal start index and end index of the required area within each line. The same calculations determine the vertical window, with *width* replaced by *height*. Where i_s is the start index for the required tile in each input line and i_e is the end index for the required tile in each input line. o_s and o_e are the indices of the first and last pixels of the output tile respectively in the overall, combined scaled output frame.

$$i_s = \frac{o_s * total_width_{in}}{total_width_{out}} \quad i_e = \frac{o_e * total_width_{in}}{total_width_{out}}$$

The indices i_s and i_e mark the edges of the minimum input tile required to generate the given output tile. However, to achieve a seamless join between tiles, extra overscan data is required on the edges of the tile to populate all the taps of the scaling filter. A filter with N taps has $(N-1)/2$ taps to the left (above for vertical) and $N/2$ taps to the right (below for vertical) of the center pixel in the filter. On the true left or right edge of the overall frame, you cannot fill these taps with any real data, so the IP replicates the edge pixel to populate them. However, for an output tile whose left and right edge does not sit at the edge of the overall frame, the IP must populate these taps with pixels from the input frame. The first and last indices of the input tile including overscan, i_{so} and i_{eo} respectively, are then defined as follows:

$$i_{so} = i_s - \frac{N-1}{2} \quad i_{eo} = i_e + \frac{N}{2}$$

The horizontal dimension the scaler requires that pixel index i_s is transmitted in pixel 0 within each group of pixels transmitted in parallel (as set by the **Number of pixels in parallel** parameter). Ensure you round the number of overscan pixels to the left of i_s (nominally $(N-1)/2$) to the nearest multiple of pixels in parallel (pip). The equation defines the final horizontal input tile left index (h_{iso}) when overscan is on:

$$h_{iso} = i_s - pip * \left(\frac{\left(\frac{N-1}{2} + pip - 1 \right)}{pip} \right)$$

You can choose not to supply the additional overscan data if your system does not allow it or the cost is too high, but expect to see noticeable seams at the tile edges. Registers in the register map allow you to turn on and off the left and top edge overscan for horizontal and vertical scaling respectively at run time. This register merely informs the scaler whether it should expect the overscan data or not. The scaler responds automatically if it receives right or bottom edge overscan data, with no register map setting required. Controlling the overscan behavior of the scaler at runtime allows you to change the position of the tile that each scaler IP processes dynamically.

39.3.5. 422 and 420 Chroma Sampled Data Scaling

The Altera Streaming Video Protocol that the scaler IP uses to transmit and receive video fields allows for the chroma sampling of the video to vary from frame to frame. For most of the IPs in the Video and Vision Processing suite, support for variable chroma sampling adds little or no extra resource cost. Variable chroma sampling of the video is always on and not gated by any parameters.

However, support for scaling 444, 422 and 420 chroma sampling on a field-variable basis adds significant ALM and memory cost to the resource footprint of the scaler. The scaler includes three parameters (**444 chroma sampling**, **422 chroma sampling** and **420 chroma sampling**) to allow you to select which chroma sampling formats the IP should support. Altera recommends that you only turn on chroma samplings formats that your scaler expects to receive in your system.

When the scaler processes 444 sampled data, each color plane is scaled independently, and each color plane has the same input and output dimensions. When the scaler processes 422 sampled data, each color plane is still processed independently (using the same filters as the 444 case), but the width of the chroma planes at the input and output is half that of the luma plane. Likewise, for 420 sampled data, each color plane is scaled independently, and the width and height of the chroma planes is half that of the luma plane. In all cases, the scaling ratio applied to each color plane must be the same. The scaler requires that the input and output field width are even for 422 sampled data, and the input and output width and height are even for 420 sampled data.

For 420 sampled data, each video line contains data from either the blue chroma (Cb) or red chroma (Cr) color plane. To supply both N lines of Cb and Cr data to an N tap vertical scaling filter, $2N$ lines of video data must be stored in line buffer, which is twice the storage required to process 444 or 422 sampled data. To reduce the on-chip memory footprint of the scaler with 420 chroma sampling enabled, you can turn on **Mirror 420 chroma data**. When on, the size of the line buffer is reduced back the N lines (for an N tap filter) and the IP mirrors $N/2$ lines of Cb and Cr data available in the buffer to fill the available taps. This parameter might give a small reduction in the quality of the scaled results for 420 chroma sampled data. You must assess any reduction in memory usage against the potential reduction in quality.

For 420 sampled data, the IP packs two luma samples into the three color planes that make up a standard pixel for either 444 or 422 chroma sampling. The scaler applies the horizontal scaling algorithm to the luma plane independently of the chroma planes. To allow for the double packing of the luma samples the scaler must, by default, implement the luma scaling with processing for twice the number of pixels in parallel you specify. However, you might not require the full processing bandwidth of the interface using 420 sampling. For example, if you have an HDMI 2.0 interface that supports up to 4k60 processing in both 444 and 420 sampling modes, the 420 mode only uses half the bandwidth allowed by the interface. In this example, the scaler need not use double the pixels in parallel to process the luma data and you should turn on the **Half rate 420** parameter. This parameter specifies that the IP does not require internal doubling of pixels in parallel for processing the luma plane.

39.3.6. Scaler IP Interfaces

Table 692. Scaler IP Interfaces

Name	Direction	Width	Description
Clocks and resets			
main_clock_clk	In	1	AXI4-S processing clock.
main_reset_rst	In	1	AXI4-S processing reset.
agent_clock_clk	In	1	Clock for the Avalon memory-mapped control agent interface. Only if you select Separate clock for control interface .
agent_reset_rst	In	1	Reset for the Avalon memory-mapped control agent interface. Only if you select Separate clock for control interface .
Control interfaces			
av_mm_control_agent_address	In	7	Avalon memory-mapped agent address.
av_mm_control_agent_write	In	1	Avalon memory-mapped agent write.
av_mm_control_agent_writedata	In	32	Avalon memory-mapped agent write data.
av_mm_control_agent_byteenable	In	4	Avalon memory-mapped agent byte enable.
av_mm_control_agent_read	In	1	Avalon memory-mapped agent read.
av_mm_control_agent_readdata	Out	32	Avalon memory-mapped agent read data.
av_mm_control_agent_readdatavalid	Out	1	Avalon memory-mapped agent read.
av_mm_control_agent_waitrequest	Out	1	Avalon memory-mapped agent wait request.
Altera Streaming Video interfaces			
axi4s_vid_in_tdata	In		AXI4-S data in.
axi4s_vid_in_tvalid	In	1	AXI4-S data valid.
<i>continued...</i>			

(121) The equation shows all tdata widths in these interfaces:

$$\max \left(\text{ceil} \left(\frac{\text{bits per color sample} \times \text{number of color planes} \times \text{pixels in parallel}}{8} \right) \times 8, 16 \right)$$

(121)

Name	Direction	Width	Description
axi4s_vid_in_tuser	In		AXI4-S tuser tuser[0] indicates start of video frame when asserted. tuser[1] indicates the start of a non-video packet when asserted.
axi4s_vid_in_tlast	In	1	AXI4-S end of packet.
axi4s_vid_in_tready	Out	1	AXI4-S data ready.
axi4s_vid_out_tdata	Out	(121)	AXI4-S data in.
axi4s_vid_out_tvalid	Out	1	AXI4-S data valid.
axi4s_vid_in_tuser	Out	(122)	AXI4-S tuser. tuser[0] indicates start of video frame when asserted tuser[1] indicates the start of a non-video packet when asserted
axi4s_vid_out_tlast	Out	1	AXI4-S end of packet.
axi4s_vid_out_tready	In	1	AXI4-S data ready.

Related Information

Video and Vision Processing IP Interfaces on page 30

39.4. Scaler IP Registers

Each register is either read-only (RO) or read-write (RW).

Table 693. Scaler IP Registers

In the software API the register names appear with a prefix of INTEL_VVP, INTEL_VVP_CORE or INTEL_VVP_SCALER as appropriate and with an optional REG suffix

Address	Register	Access	Description
Parameterization registers			
0x0000	VID_PID	RO	Read this register to retrieve scaler product ID. This register always returns 0x6AF7_0234.
0x0004	VERSION	RO	Read this register for the IP version information.
0x0008	LITE_MODE	RO	Read this register to determine if Lite mode is on or off. This register returns 0 when you turn off Lite mode and 1 when you turn on Lite mode .
0x000C	DEBUG_ENABLED	RO	Read this register to determine if Debug features is on or off. This register returns 1 if reads to other registers designated as RW return the last value the IP writes to the register, or an undefined value.
0x0010	PIXELS_IN_PARALLEL	RO	Read this register to determine the number of pixels processed per clock cycle.
<i>continued...</i>			

(122) The equation shows all tuser widths in these interfaces:

$$N = \text{ceil}(\text{tdata width} / 8)$$

(122)

Address	Register	Access	Description
0x0014	MAX_INPUT_WIDTH	RO	Read this register to determine the maximum supported input field width.
0x0018	MAX_OUTPUT_WIDTH	RO	Read this register to determine the maximum supported output field width.
0x001C	ALGORITHM	RO	Read this register to determine which algorithm the scaler is configured to use. Returns 0 for Nearest neighbor , 1 for Bilinear and 2 for Polyphase .
0x0020	COEFFS_RUNTIME_LOAD	RO	Read this register to determine if updates to the coefficients at runtime via the Avalon memory-mapped agent interface are supported. Returns 1 if updates are supported and 0 otherwise.
0x0024	COEFF_MEM_INIT	RO	Read this register to determine if the scaler is configured with pre-initialized coefficients at reset. Returns 1 if the coefficients are preinitialized and 0 otherwise.
0x0028	V_SCALING_ENABLED	RO	Read this register to determine if the scaler is configured to support vertical scaling. Returns 1 if vertical scaling is supported and 0 otherwise.
0x002C	V_NUM_TAPS	RO	Read this register to determine the number of vertical scaling filter taps.
0x0030	V_NUM_PHASES	RO	Read this register to determine the number of vertical scaling filter phases.
0x0034	V_NUM_BANKS	RO	Read this register to determine the number of vertical scaling filter coefficient banks.
0x0038	V_COEFFS_SIGNED	RO	Read this register to determine if the vertical scaling filter uses signed coefficients. Returns 1 if the coefficients are signed and 0 otherwise.
0x003C	V_COEFFS_INT_BITS	RO	Read this register to determine the number of integer bits used to represent the vertical scaling filter coefficients.
0x0040	V_COEFFS_FRAC_BITS	RO	Read this register to determine the number of fraction bits that represent the vertical scaling filter coefficients.
0x0044	H_SCALING_ENABLED	RO	Read this register to determine if the scaler supports horizontal scaling. Returns 1 if horizontal scaling is supported and 0 otherwise.
0x0048	H_NUM_TAPS	RO	Read this register to determine the number of horizontal scaling filter taps.
0x004C	H_NUM_PHASES	RO	Read this register to determine the number of horizontal scaling filter phases.
0x0050	H_NUM_BANKS	RO	Read this register to determine the number of horizontal scaling filter coefficient banks.
0x0054	H_COEFFS_SIGNED	RO	Read this register to determine if the horizontal scaling filter uses signed coefficients. Returns 1 if the coefficients are signed and 0 otherwise.
0x0058	H_COEFFS_INT_BITS	RO	Read this register to determine the number of integer bits to represent the horizontal scaling filter coefficients.
0x005C	H_COEFFS_FRAC_BITS	RO	Read this register to determine the number of fraction bits to represent the horizontal scaling filter coefficients.
0x0060 to 0x0067	-	-	Reserved.
continued...			

Address	Register	Access	Description
0x0068	V_PARTIAL_SCALING_ENABLED	RO	Read this register to determine if vertical partial scaling is on
0c006C	H_PARTIAL_SCALING_ENABLED	RO	Read this register to determine if horizontal partial scaling is on
0x0070to 0x011F	-	-	Unused.

Table 694. Scaler IP Control and Debug Registers

For more information, refer to *Control Packets*. In the software API the register names appear with a prefix of INTEL_VVP, INTEL_VVP_CORE or INTEL_VVP_SCALER as appropriate and with an optional REG suffix

Address	Register	Access		Description
		Lite ⁽¹²³⁾	Full	
0x0120	IMG_INFO_WIDTH	RW	RO	When you turn on Lite mode , use this register to set the expected width of incoming video fields. When you turn off Lite mode and turn on Debug features , this register returns the width that the scaler derives from information in the image information packet. When you turn on Horizontal partial scaling , the value in this register specifies the number of pixels that this scaler receives in each incoming video line, including any overscan data.
0x0124	IMG_INFO_HEIGHT	RW	RO	When you turn on Lite mode , use this register to set the expected height of incoming video fields. When you turn off Lite mode and turn on Debug features , this register returns the height that the scaler derives from information in the image information packet. When you turn on Vertical partial scaling , the value in this register specifies the number of lines that this scaler receives in each incoming frame, including any overscan data.
0x0128	IMG_INFO_INTERLACE	-	RO	When you turn off Lite mode and turn on Debug features , this register returns the interlace nibble that the scaler derives from information in the image information packet. Unused in lite mode.
0x012C	Reserved	-	-	Reserved
0x0130	IMG_INFO_COLORSPACE	-	RO	When you turn off Lite mode and turn on Debug features , this register returns the color space that the scaler derives from information in the image information packet. Unused in lite mode.
0x0134	IMG_INFO_SUBSAMPLING	-	RO	When you turn off Lite mode and turn on Debug features , this register returns the subsampling that the scaler derives from information in the image information packet. Unused in lite mode.
0x0138	IMG_INFO_COSITING	-	RO	When you turn off Lite mode and turn on Debug features , this register returns the cositing that the scaler derives from information in the image information packet. Unused in lite mode.
0x013C	IMG_INFO_FIELD_COUNT	-	RO	When you turn off Lite mode and turn on Debug features , this register returns the field count that the scaler derives from information in the image information packet. Unused in lite mode.
continued...				

⁽¹²³⁾ When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

Address	Register	Access		Description
		Lite ⁽¹²³⁾	Full	
0x012C to 0x013C	-	-	-	Unused.
0x0140	STATUS	RO		Bit 0 : Status bit. 1 means scaler is processing a video field, 0 otherwise. When you turn off Lite mode : Bit 1 : Pending register updates bit. Any writes to the settings register (0x0148 - 0x0154) cause the IP to raise pending register updates bit, to indicate outstanding changes to the scaling settings. The IP lowers this bit at the next field boundary after a write to the COMMIT register.
0x0144	COMMIT	RW		Only use when you turn off Lite mode . The IP holds any changes to the scaling settings (not coefficients) via the register map until you issue a write to this register. The value you write is unimportant.
0x0148	OUTPUT_WIDTH	RW		Write to this register to set the output field width. Unused if horizontal scaling is not enabled. If you turn on Horizontal partial scaling , the value in this register specifies the number of output pixels that this scaler produces for each line.
0x014C	OUTPUT_HEIGHT	RW		Write to this register to set the output field height. Unused if vertical scaling is not enabled. If you turn on Vertical partial scaling , the value in this register specifies the number of output lines that this scaler produces for each frame.
0x0150	H_BANK	RW		Write to this register to set the read bank for the horizontal scaling filter coefficients.
0x0154	V_BANK	RW		Write to this register to set the read bank for the vertical scaling filter coefficients
0x0158	TOTAL_IN_WIDTH	RW		Only if you turn on Horizontal partial image scaling . Write to this register to set the overall width of the tiled input frame.
0x015C	TOTAL_OUT_WIDTH	RW		Only if you turn on Horizontal partial image scaling . Write to this register to set the overall width of the tiled output frame.
0x0160	H_POS_ERROR	RW		Only if you turn on Horizontal partial image scaling . Write to this register to set the horizontal sub-pixel offset. If o_s is the index of the first output pixel within the complete output frame that this scaler should produce, the horizontal sub-pixel offset is calculated as: $(o_s * total_in_width) \% total_out_width$
0x0164	H_PHASE_OFFSET	RW		Only if you turn on Horizontal partial image scaling . Write to this register to set the horizontal phase offset. If o_s is the index of the first output pixel within the complete output frame that this scaler should produce, the horizontal phase offset is calculated as: $\frac{(o_s * total_in_width) \% total_out_width * num_phases_h}{total_out_width}$
continued...				

(123) When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

Address	Register	Access		Description
		Lite ⁽¹²³⁾	Full	
0x0168	H_PHASE_ERROR	RW		Only if you turn on Horizontal partial image scaling . Write to this register to set the horizontal sub-phase offset. If o_s is the index of the first output pixel within the complete output frame that this scaler should produce, the horizontal sub-phase offset is calculated as: $(((o_s * total_in_width) \% total_out_width) * num_phases_h) \% total_out_width$
0x016C	H_EXTENDED_FILL	RW		Only if you turn on Horizontal partial image scaling . Write to this register to enable the left edge overscan feature. Write 1 to bit 0 to enable overscan, or 0 to bit 0 to disable overscan
0x0170	TOTAL_IN_HEIGHT	RW		Only if you turn on Vertical partial image scaling . Write to this register to set the overall height of the tiled input frame ().
0x0174	TOTAL_OUT_HEIGHT	RW		Only if you turn on Vertical partial image scaling . Write to this register to set the overall height of the tiled output frame ().
0x0178	V_POS_ERROR	RW		Only if you turn on Vertical partial image scaling . Write to this register to set the vertical sub-pixel offset. If o_s is the index of the first output line within the complete output frame that this scaler should produce, the vertical sub-pixel offset is calculated as: $(o_s * total_in_height) \% total_out_height$
0x017C	V_PHASE_OFFSET	RW		Only if you turn on Vertical partial image scaling . Write to this register to set the vertical phase offset. If o_s is the index of the first output line within the complete output frame that this scaler should produce, the vertical phase offset is calculated as: $\frac{((o_s * total_in_height) \% total_out_height) * num_phases_v}{total_out_height}$
0x0180	V_PHASE_ERROR	RW		Only if you turn on Vertical partial image scaling . Write to this register to set the vertical sub-phase offset. If o_s is the index of the first output line within the complete output frame that this scaler should produce, then the vertical sub-phase offset is calculated as: $(((o_s * total_in_height) \% total_out_height) * num_phases_v) \% total_out_height$
0x0184	V_EXTENDED_FILL	RW		Only if you turn on Vertical partial image scaling . Write to this register to enable the top edge overscan feature. Write 1 to bit 0 to enable overscan, or 0 to bit 0 to disable overscan
0x0188 to 0x01FC	-	-		Unused.
0x0200	RT_COEFF_LOAD_BANK_SELECT	WO		Write to this register to set the index of the scaling filter coefficient bank that is being updated
0x0204	RT_COEFF_LOAD_PHASE_SELECT	WO		Write to this register to set the index of the scaling filter phase that you are updating
0x0208	RT_COEFF_LOAD_COMMIT	WO		Write 1 to this register to commit the coefficient information in the RT_COEFF_LOAD_TAP_X registers to the horizontal scaling filter coefficient bank and phase specified in RT_COEFF_LOAD_BANK_SELECT and RT_COEFF_LOAD_PHASE_SELECT respectively. Write 0 to this register to commit the coefficient information in the

continued...

(123) When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

Address	Register	Access		Description
		Lite ⁽¹²³⁾	Full	
				RT_COEFF_LOAD_TAP_X registers to the vertical scaling filter coefficient bank and phase specified in RT_COEFF_LOAD_BANK_SELECT and RT_COEFF_LOAD_PHASE_SELECT respectively.
0x020C-0x0308	RT_COEFF_LOAD_TAP_0 - RT_COEFF_LOAD_TAP_63	WO		Up to 64 registers to set the new coefficient values for a vertical or horizontal scaling filter phase. If the vertical scaling filter has <i>N</i> taps, only the first <i>N</i> registers need to update before writing to the RT_COEFF_LOAD_COMMIT register. If the horizontal scaling filter has <i>M</i> taps, only the first <i>M</i> registers need to update before writing to the RT_COEFF_LOAD_COMMIT register.

Register Bit Descriptions

Table 695. VID_PID

Name	Bits	Description
Scaler version ID and product ID	31:0	This register always returns 0x6AF7_0234. 15:0 is the product ID and always returns 0x0234 31:16 is the vendor ID and always returns 0x6AF7

Table 696. VERSION

Name	Bits	Description
Register map version	7:0	Register map version.
IP patch revision	15:8	-
IP update revision	23:16	Updated when the IP version changes.
IP major revision	31:24	Updated when the IP version changes.

Table 697. LITE_MODE

Name	Bits	Description
Lite mode parameterization bit	31:0	Returns 1 if Lite mode is on and 0 otherwise.

Table 698. DEBUG_ENABLED

Name	Bits	Description
Debug features parameterization bit	31:0	Returns 1 if Debug features is on and 0 otherwise.

Table 699. PIXELS_IN_PARALLEL

Name	Bits	Description
Pixels in parallel	31:0	Number of pixels processed per clock cycle.

⁽¹²³⁾ When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

Table 700. MAX_INPUT_WIDTH

Name	Bits	Description
Maximum input width	31:0	Maximum supported input field width.

Table 701. MAX_OUTPUT_WIDTH

Name	Bits	Description
Maximum output width	31:0	Maximum supported output field width.

Table 702. ALGORITHM

Name	Bits	Description
Algorithm	31:0	Returns 0 for Nearest neighbor , 1 for Bilinear and 2 for Polyphase .

Table 703. COEFFS_RUNTIME_LOAD

Name	Bits	Description
Runtime load	31:0	Returns 1 if Runtime coefficient updates is on and 0 otherwise. Only valid for Polyphase .

Table 704. COEFF_MEM_INIT

Name	Bits	Description
Coefficient memory initialized	31:0	Returns 1 if the scaling filter coefficient memory is preinitialized at reset and 0 otherwise. Only valid for Polyphase .

Table 705. V_SCALING_ENABLED

Name	Bits	Description
Vertical scaling enabled	31:0	Returns 1 if Enable vertical scaling is on and 0 otherwise

Table 706. V_NUM_TAPS

Name	Bits	Description
Vertical scaling filter taps	31:0	Returns the number of vertical scaling filter taps.

Table 707. V_NUM_PHASES

Name	Bits	Description
Vertical scaling filter phases	31:0	Returns the number of vertical scaling filter phases. Only valid for Polyphase .

Table 708. V_NUM_BANKS

Name	Bits	Description
Vertical scaling filter coefficient banks	31:0	Returns the number of vertical scaling filter coefficient banks. Only valid if polyphase scaling is selected.

Table 709. V_COEFFS_SIGNED

Name	Bits	Description
Vertical scaling filter coefficients signed	31:0	Returns 1 if the vertical scaling filter coefficients are signed values and 0 otherwise. Only valid for Polyphase .

Table 710. V_COEFFS_INT_BITS

Name	Bits	Description
Vertical scaling filter coefficients integer bits	31:0	Returns the number of integer bits used to represent the vertical scaling filter coefficients. Only valid for Polyphase .

Table 711. V_COEFFS_FRAC_BITS

Name	Bits	Description
Vertical scaling filter coefficients fraction bits	31:0	Returns the number of fraction bits to represent the vertical scaling filter coefficients. Only valid for Polyphase .

Table 712. H_SCALING_ENABLED

Name	Bits	Description
Horizontal scaling enabled	31:0	Returns 1 if horizontal scaling is enabled and 0 otherwise.

Table 713. H_NUM_TAPS

Name	Bits	Description
Horizontal scaling filter taps	31:0	Returns the number of horizontal scaling filter taps.

Table 714. H_NUM_PHASES

Name	Bits	Description
Horizontal scaling filter phases	31:0	Returns the number of horizontal scaling filter phases. Only valid for Polyphase .

Table 715. H_NUM_BANKS

Name	Bits	Description
Horizontal scaling filter coefficient banks	31:0	Returns the number of horizontal scaling filter coefficient banks. Only valid for Polyphase .

Table 716. H_COEFFS_SIGNED

Name	Bits	Description
Horizontal scaling filter coefficients signed	31:0	Returns 1 if the horizontal scaling filter coefficients are signed values and 0 otherwise. Only valid for Polyphase .

Table 717. H_COEFFS_INT_BITS

Name	Bits	Description
Horizontal scaling filter coefficients integer bits	31:0	Returns the number of integer bits to represent the horizontal scaling filter coefficients. Only valid for Polyphase .

Table 718. H_COEFFS_FRAC_BITS

Name	Bits	Description
Horizontal scaling filter coefficients fraction bits	31:0	Returns the number of fraction bits to represent the horizontal scaling filter coefficients. Only valid for Polyphase .

Table 719. V_PARTIAL_SCALING_ENABLED

Name	Bits	Description
Vertical partial image scaling enabled	31:0	Returns 1 if you turn on vertical partial image scaling and 0 otherwise

Table 720. H_PARTIAL_SCALING_ENABLED

Name	Bits	Description
Horizontal partial image scaling enabled	31:0	Returns 1 if you turn on horizontal partial image scaling and 0 otherwise

Table 721. IMG_INFO_WIDTH

Name	Bits	Description
Width bits	15:0	When Lite mode is on, write to this register to set the expected width of the incoming video fields. For full parameterizations with debug features enabled, this register returns the <code>width-1</code> field from the most recently received image information packet and adds 1 to return a value for <code>width</code> .
unused	31:16	Unused.

Table 722. IMG_INFO_HEIGHT

Name	Bits	Description
Height bits	15:0	When Lite mode is on, write to this register to set the expected height of the incoming video fields. For full parameterizations with debug features enabled, this register reads the <code>height-1</code> field from the most recently received image information packet and adds 1 to return a value for <code>height</code> .
unused	31:16	Unused.

Table 723. IMG_INFO_INTERLACE

Name	Bits	Description
IntlaceNibble bits	3:0	When Lite mode is on, this register has no function. When Lite mode is off with Debug features on, this register returns the <code>intlaceNibble</code> field from the most recently received image information packet.
unused	31:4	Unused.

Table 724. IMG_INFO_COLORSPACE

Name	Bits	Description
CSP code bits	6:0	When Lite mode is on, this register has no function. When Lite mode is off with Debug features on, this register returns the 7 bit CSP field from the most recently received image information packet.
unused	31:7	Unused.

Table 725. IMG_INFO_SUBSAMPLING

Name	Bits	Description
SubSa code bits	1:0	When Lite mode is on, this register has no function. When Lite mode is off with Debug features on, this register returns the <code>SUBSA</code> field from the most recently received image information packet.
unused	31:2	Unused.

Table 726. IMG_INFO_COSITING

Name	Bits	Description
Cosite code bits	1:0	When Lite mode is on, this register has no function. When Lite mode is off with Debug features on,, this register returns the COSITE field from the most recently received image information packet.
unused	31:2	Unused.

Table 727. IMG_INFO_FIELD_COUNT

Name	Bits	Description
Count bits	6:0	When Lite mode is on, this register has no function. When Lite mode is off with Debug features on,, this register returns the 7 bit FIELD_COUNT field from the most recently received image information packet.
unused	31:7	Unused.

Table 728. STATUS

Name	Bits	Description
Status bit	0	1 means scaler is processing a video field, 0 otherwise.
Pending register updates bit	1	1 means scaler has pending updates, 0 otherwise
unused	31:2	Unused.

Table 729. COMMIT

Name	Bits	Description
unused	31:0	Unused.

Table 730. OUTPUT_WIDTH

Name	Bits	Description
Output width	15:0	Output field width
unused	31:16	Unused.

Table 731. OUTPUT_HEIGHT

Name	Bits	Description
Output height	15:0	Output field height
unused	31:16	Unused.

Table 732. H_BANK

Name	Bits	Description
Horizontal scaling filter coefficient read bank	3:0	Horizontal scaling filter coefficient read bank.
unused	31:4	Unused.

Table 733. V_BANK

Name	Bits	Description
Vertical scaling filter coefficient read bank	3:0	Vertical scaling filter coefficient read bank.
unused	31:4	Unused.

Table 734. TOTAL_IN_WIDTH

Name	Bits	Description
Total input frame width	15:0	Width of the complete, tiled input frame (only used if Horizontal partial image scaling is turned on)
Unused	31:4	Unused.

Table 735. TOTAL_OUT_WIDTH

Name	Bits	Description
Total output frame width	15:0	Width of the complete, tiled output frame (only used if Horizontal partial image scaling is turned on)
Unused	31:4	Unused.

Table 736. H_POS_ERROR

Name	Bits	Description
Horizontal sub-pixel offset	15:0	Horizontal subpixel offset (only used if Horizontal partial image scaling is turned on)
Unused	31:4	Unused.

Table 737. H_PHASE_OFFSET

Name	Bits	Description
Horizontal phase offset	15:0	Horizontal phase offset (only used if Horizontal partial image scaling is turned on)
Unused	31:4	Unused.

Table 738. H_PHASE_ERROR

Name	Bits	Description
Horizontal sub-phase offset	15:0	Horizontal subphase offset (only used if Horizontal partial image scaling is turned on)
Unused	31:4	Unused.

Table 739. H_EXTENDED_FILL

Name	Bits	Description
Enable horizontal left edge overscan	0	Enable horizontal left edge overscan (only used if Horizontal partial image scaling is turned on)
Unused	31:1	Unused.

Table 740. TOTAL_IN_HEIGHT

Name	Bits	Description
Total input frame height	15:0	Height of the complete, tiled input frame (only used if Vertical partial image scaling is turned on)
Unused	31:4	Unused.

Table 741. TOTAL_OUT_HEIGHT

Name	Bits	Description
Total output frame height	15:0	Height of the complete, tiled output frame (only used if Vertical partial image scaling is turned on)
Unused	31:16	Unused.

Table 742. V_POS_ERROR

Name	Bits	Description
Vertical sub-pixel offset	15:0	Vertical subpixel offset (only used if Vertical partial image scaling is turned on)
Unused	31:16	Unused.

Table 743. V_PHASE_OFFSET

Name	Bits	Description
Vertical phase offset	15:0	Vertical phase offset (only used if Vertical partial image scaling is turned on)
Unused	31:16	Unused.

Table 744. V_PHASE_ERROR

Name	Bits	Description
Vertical sub-phase offset	15:0	Vertical subphase offset (only used if Vertical partial image scaling is turned on)
Unused	31:16	Unused.

Table 745. V_EXTENDED_FILL

Name	Bits	Description
Enable vertical top edge overscan	0	Enable vertical top edge overscan (only used if Vertical partial image scaling is turned on)
Unused	31:1	Unused.

Table 746. RT_COEFF_LOAD_BANK_SELECT

Name	Bits	Description
Coefficient write bank	3:0	Index of coefficient bank to be updated
unused	31:4	Unused.

Table 747. RT_COEFF_LOAD_PHASE_SELECT

Name	Bits	Description
Coefficient write phase	7:0	Index of coefficient phase to be updated
unused	31:8	Unused.

Table 748. RT_COEFF_LOAD_COMMIT

Name	Bits	Description
unused	31:0	Unused.

Table 749. RT_COEFF_LOAD_TAP_X

Name	Bits	Description
Coefficient value	coeff_width-1:0	Coefficient value for filter tap X
unused	31:coeff_width	Unused.

39.5. Scaler IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_scaler_regs.h`

Include file: `intel_vvp_scaler.h`

Table 750. Scaler IP Software API

Name	Description
<code>intel_vvp_scaler_init</code>	Initialize the Scaler instance
<code>Intel_vvp_core_*</code>	.Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i> . Writable when Lite is on. Readable when Lite is off and Debug features is on.
<code>intel_vvp_scaler_get_lite_mode</code>	Returns if Lite mode is on
<code>intel_vvp_scaler_get_debug_enabled</code>	Returns if Debug features is on
<code>intel_vvp_scaler_get_pixels_in_parallel</code>	Returns the number of pixels transmitted in parallel (per valid beat)
<code>intel_vvp_scaler_get_max_input_width</code>	Returns the maximum input width supported
<code>intel_vvp_scaler_get_max_output_width</code>	Returns the maximum output width supported
<code>intel_vvp_scaler_get_scaling_algorithm</code>	Returns the scaling algorithm
<code>intel_vvp_scaler_is_coeffs_loading_enabled</code>	Returns if the scaling coefficients can be loaded/updated at run-time
<code>intel_vvp_scaler_are_coeffs_preinitialized</code>	Returns if the scaling coefficients are automatically initialized with default values out of reset
<code>intel_vvp_scaler_is_vertical_scaling_enabled</code>	Returns if the IP can do vertical scaling
<code>Intel_vvp_scaler_is_partial_vertical_scaling_enabled</code>	Returns if the IP can do split vertical scaling
<code>intel_vvp_scaler_get_num_vertical_taps</code>	Returns the number of vertical taps
<code>intel_vvp_scaler_get_num_vertical_phases</code>	Returns the number of vertical phases
<i>continued...</i>	

Name	Description
intel_vvp_scaler_get_num_vertical_banks	Returns the number of vertical banks
intel_vvp_scaler_are_vertical_coeffs_signed	Returns if the vertical scaling coefficients have a sign bit and can be negative
intel_vvp_scaler_get_vertical_coeffs_int_bits	Returns the number of integer bits (excluding sign bit) available to quantize the vertical scaling coefficients
intel_vvp_scaler_get_vertical_coeffs_frac_bits	Returns the number of fractional bits available to quantize the vertical scaling coefficients
intel_vvp_scaler_is_horizontal_scaling_enabled	Returns if the IP can do horizontal scaling
intel_vvp_scaler_is_partial_horizontal_scaling_enabled	Returns if the IP can do split horizontal scaling
intel_vvp_scaler_get_num_horizontal_taps	Returns the number of horizontal taps
intel_vvp_scaler_get_num_horizontal_phases	Returns the number of horizontal phases
intel_vvp_scaler_get_num_horizontal_banks	Returns the number of horizontal banks
intel_vvp_scaler_are_horizontal_coeffs_signed	Returns if the horizontal scaling coefficients have a sign bit and can be negative
intel_vvp_scaler_get_horizontal_coeffs_int_bits	Returns the number of integer bits (excluding sign bit) available to quantize the horizontal scaling coefficients
intel_vvp_scaler_get_horizontal_coeffs_frac_bits	Returns the number of fractional bits available to quantize the horizontal scaling coefficients
intel_vvp_scaler_get_rounding_method	Returns the method used to convert floating-point values back to integer values at the output of the IP
intel_vvp_scaler_is_running	Returns if the IP is currently processing video data
intel_vvp_scaler_get_commit_status	Returns if there are uncommitted writes
intel_vvp_scaler_get_status	Reads the status register
intel_vvp_scaler_set_output_width	Selects the output width
intel_vvp_scaler_set_total_input_width	Selects the full image input width (split scaling only)
intel_vvp_scaler_set_total_output_width	Selects the full image output width (split scaling only)
intel_vvp_scaler_set_output_height	Selects the output height
intel_vvp_scaler_set_total_input_height	Selects the full image input height (split scaling only)
intel_vvp_scaler_set_total_output_height	Selects the full image output height (split scaling only)
intel_vvp_scaler_set_h_pos_error	Selects the horizontal offset error (split scaling only)
intel_vvp_scaler_set_v_pos_error	Selects the vertical offset error (split scaling only)
intel_vvp_scaler_set_h_extended_fill	Enables horizontal split extension (split scaling only)
continued...	

Name	Description
intel_vvp_scaler_set_v_extended_fill	Enables vertical split extension (split scaling only)
intel_vvp_scaler_set_h_phase_offset_error	Selects the horizontal phase offset (split scaling only)
intel_vvp_scaler_set_v_phase_offset_error	Selects the vertical phase offset (split scaling only)
intel_vvp_scaler_set_split_h_scaling	Setup the scaler for a horizontal split scaling
intel_vvp_scaler_set_split_v_scaling	Setup the scaler for a vertical split scaling
intel_vvp_scaler_set_v_bank	Selects the vertical coefficients bank for the next commit
intel_vvp_scaler_set_h_bank	Selects the horizontal coefficients bank for the next commit
intel_vvp_scaler_get_output_width	Returns the last selected output width
intel_vvp_scaler_get_total_input_width	Returns the last selected full image input width (split scaling)
intel_vvp_scaler_get_total_output_width	Returns the last selected full image output width (split scaling)
intel_vvp_scaler_get_output_height	Returns the last selected output height
intel_vvp_scaler_get_total_input_height	Returns the last selected full image input height (split scaling)
intel_vvp_scaler_get_total_output_height	Returns the last selected full image output height (split scaling)
intel_vvp_scaler_get_h_pos_error	Returns the last selected horizontal offset error (split scaling)
intel_vvp_scaler_get_v_pos_error	Returns the last selected vertical offset error (split scaling)
intel_vvp_scaler_get_h_extended_fill	Returns if horizontal split extension is enabled (split scaling only)
intel_vvp_scaler_get_v_extended_fill	Returns if vertical split extension is enabled (split scaling only)
intel_vvp_scaler_get_h_phase_offset_error	Returns the last selected horizontal phase offset and error (split scaling)
intel_vvp_scaler_get_v_phase_offset_error	Returns the last selected vertical phase offset and error (split scaling)
intel_vvp_scaler_get_v_bank	Returns the last selected vertical coefficients bank
intel_vvp_scaler_get_h_bank	Returns the last selected horizontal coefficients bank
intel_vvp_scaler_set_quantized_coeffs	Program the IP with a new set of quantized coefficients for a specified phase
intel_vvp_scaler_set_coeffs	Program the IP with a new set of floating-point coefficients for a specified phase
intel_vvp_scaler_commit_writes	Commit all outstanding writes

enum eIntelVvpScalerAlgorithms

- Members**
- kIntelVvpScalerNearestNeighborScaling (0) – Nearest neighbor scaling
 - kIntelVvpScalerBilinearScaling (1) – Bilinear scaling
 - kIntelVvpScalerPolyphaseScaling (2) – Polyphase scaling
 - kIntelVvpScalerInvalidScalingAlgo (-1) – Invalid scaling algorithm (used to indicate errors)

Description Enumeration for the configurable scaling algorithms

intel_vvp_scaler_init

Prototype

```
int intel_vvp_scaler_init(intel_vvp_scaler_instance *instance,
intel_vvp_core_base base);
```

Description Initializes a Scaler instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

base – base address of the register map

Returns kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product_id does not match with the Scaler product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpScalerMapVersionErr if the register map is not supported

intel_vvp_scaler_get_lite_mode

Prototype

```
bool intel_vvp_scaler_get_lite_mode(intel_vvp_scaler_instance
*instance);
```

Description Returns the value of the LITE_MODE register. The instance must be a valid intel_vvp_scaler_instance fully initialized

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns true if the IP is parameterized in lite mode

intel_vvp_scaler_get_debug_enabled

Prototype

```
bool
intel_vvp_scaler_get_debug_enabled(intel_vvp_scaler_instance
*instance);
```

Description Returns the value of the DEBUG_ENABLED register. The instance must be a valid intel_vvp_scaler_instance fully initialized.

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns true if the IP is parameterized with debug features enabled

intel_vvp_scaler_get_pixels_in_parallel

Prototype

```
uint8_t
intel_vvp_scaler_get_pixels_in_parallel(intel_vvp_scaler_instance
*instance);
```

Description Returns the value of the PIXELS_IN_PARALLEL register. The instance must be a valid intel_vvp_scaler_instance fully initialized.

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns the pixel in parallel parameter (pixels transmitted per beat/clock cycle) used to generate the IP.

intel_vvp_scaler_get_max_input_width

Prototype

```
uint32_t
intel_vvp_scaler_get_max_input_width(intel_vvp_scaler_instance
*instance);
```

Description Returns the value of the MAX_INPUT_WIDTH register. The instance must be a valid intel_vvp_scaler_instance fully initialized.

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns the maximum input width supported by the IP

intel_vvp_scaler_get_max_output_width

Prototype

```
uint32_t  
intel_vvp_scaler_get_max_output_width(intel_vvp_scaler_instance  
*instance);
```

Description Returns the value of the MAX_OUTPUT_WIDTH register. The instance must be a valid intel_vvp_scaler_instance fully initialized.

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns the maximum output width supported by the IP

intel_vvp_scaler_get_scaling_algorithm

Prototype

```
eIntelVvpScalerAlgorithms  
intel_vvp_scaler_get_scaling_algorithm(intel_vvp_scaler_instance*  
instance);
```

Description Returns the value of the SCALER_ALGORITHM register. The instance must be a valid intel_vvp_scaler_instance fully initialized.

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns The scaling algorithm the IP is configured with

intel_vvp_scaler_is_coeffs_loading_enabled

Prototype

```
bool  
intel_vvp_scaler_is_coeffs_loading_enabled(intel_vvp_scaler_instance*  
instance);
```

Description Returns the value of the COEFFS_RUNTIME_LOAD register. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for polyphase scaling

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns If the IP is configured to support run-time reloading of coefficients

intel_vvp_scaler_are_coeffs_preinitialized

Prototype

```
bool  
intel_vvp_scaler_are_coeffs_preinitialized(intel_vvp_scaler_instance*  
instance);
```

Description Returns the value of the COEFFS_MEM_INIT register. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for polyphase scaling

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns If the IP is configured to start with initialized coefficients

intel_vvp_scaler_is_vertical_scaling_enabled

Prototype

```
bool intel_vvp_scaler_is_vertical_scaling_enabled
(intel_vvp_scaler_instance* instance);
```

Description Returns the value of the V_SCALING_ENABLED register. The instance must be a valid intel_vvp_scaler_instance fully initialized

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns If the IP is configured to perform vertical scaling

intel_vvp_scaler_is_partial_vertical_scaling_enabled

Prototype

```
bool intel_vvp_scaler_is_partial_vertical_scaling_enabled
(intel_vvp_scaler_instance* instance);
```

Description Returns the value of the V_PARTIAL_SCALING_ENABLED register. The instance must be a valid intel_vvp_scaler_instance fully initialized

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns If the IP is configured to perform split vertical scaling

intel_vvp_scaler_get_num_vertical_taps

Prototype

```
uint8_t
intel_vvp_scaler_get_num_vertical_taps(intel_vvp_scaler_instance*
instance);
```

Description Returns the value of the V_NUM_TAPS register. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for vertical scaling

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns The number of taps used for vertical scaling

intel_vvp_scaler_get_num_vertical_phases

Prototype

```
uint16_t  
intel_vvp_scaler_get_num_vertical_phases(intel_vvp_scaler_instance*  
instance);
```

Description Returns the value of the V_NUM_PHASES register. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for vertical scaling

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns The number of taps used for vertical scaling

intel_vvp_scaler_get_num_vertical_banks

Prototype

```
uint8_t  
intel_vvp_scaler_get_num_vertical_banks(intel_vvp_scaler_instance*  
instance);
```

Description Returns the value of the V_NUM_BANKS register. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for vertical polyphase scaling

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns The number of coefficient banks available for vertical scaling

intel_vvp_scaler_are_vertical_coeffs_signed

Prototype

```
bool  
intel_vvp_scaler_are_coeffs_signed(intel_vvp_scaler_instance*  
instance);
```

Description Returns the value of the V_COEFFS_SIGNED register. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for vertical polyphase scaling

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns If signed coefficients have been enabled for vertical scaling at IP generation time

intel_vvp_scaler_get_vertical_coeffs_int_bits

Prototype

```
uint8_t intel_vvp_scaler_get_coeffs_int_bits  
(intel_vvp_scaler_instance *instance);
```

Description Returns the value of the V_COEFFS_INT_BITS register. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for vertical polyphase scaling

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns the number of integer bits, excluding sign bit, set at IP generation time to parameterize the quantization of the vertical scaling coefficients

intel_vvp_scaler_get_vertical_coeffs_frac_bits

Prototype

```
uint8_t intel_vvp_scaler_get_vertical_coeffs_frac_bits
(intel_vvp_scaler_instance *instance);
```

Description Returns the value of the V_COEFFS_FRAC_BITS register. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for vertical polyphase scaling

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns the number of fractional bits set at IP generation time to parameterize the quantization of the vertical scaling coefficients

intel_vvp_scaler_is_horizontal_scaling_enabled

Prototype

```
bool intel_vvp_scaler_is_horizontal_scaling_enabled
(intel_vvp_scaler_instance* instance);
```

Description Returns the value of the H_SCALING_ENABLED register. The instance must be a valid intel_vvp_scaler_instance fully initialized

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns If the IP is configured to perform horizontal scaling

intel_vvp_scaler_is_partial_horizontal_scaling_enabled

Prototype

```
bool intel_vvp_scaler_is_partial_horizontal_scaling_enabled
(intel_vvp_scaler_instance* instance);
```

Description Returns the value of the H_PARTIAL_SCALING_ENABLED register. The instance must be a valid intel_vvp_scaler_instance fully initialized

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns If the IP is configured to perform split horizontal scaling

`intel_vvp_scaler_get_num_horizontal_taps`

Prototype

```
uint8_t  
intel_vvp_scaler_get_num_horizontal_taps(intel_vvp_scaler_instance*  
instance);
```

Description Returns the value of the H_NUM_TAPS register. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for horizontal scaling

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns The number of taps used for horizontal scaling

`intel_vvp_scaler_get_num_horizontal_phases`

Prototype

```
uint16_t  
intel_vvp_scaler_get_num_horizontal_phases(intel_vvp_scaler_instance*  
instance);
```

Description Returns the value of the H_NUM_PHASES register. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for horizontal scaling

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns The number of taps used for horizontal scaling

`intel_vvp_scaler_get_num_horizontal_banks`

Prototype

```
uint8_t  
intel_vvp_scaler_get_num_horizontal_banks(intel_vvp_scaler_instance*  
instance);
```

Description Returns the value of the H_NUM_BANKS register. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for horizontal polyphase scaling

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns The number of coefficient banks available for horizontal scaling

intel_vvp_scaler_are_horizontal_coeffs_signed

Prototype

```
bool
intel_vvp_scaler_are_coeffs_signed(intel_vvp_scaler_instance*
instance);
```

Description Returns the value of the H_COEFFS_SIGNED register. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for horizontal polyphase scaling

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns If signed coefficients have been enabled for horizontal scaling at IP generation time

intel_vvp_scaler_get_horizontal_coeffs_int_bits

Prototype

```
uint8_t intel_vvp_scaler_get_coeffs_int_bits
(intel_vvp_scaler_instance *instance);
```

Description Returns the value of the H_COEFFS_INT_BITS register. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for horizontal polyphase scaling

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns the number of integer bits, excluding sign bit, set at IP generation time to parameterize the quantization of the horizontal scaling coefficients

intel_vvp_scaler_get_horizontal_coeffs_frac_bits

Prototype

```
uint8_t intel_vvp_scaler_get_horizontal_coeffs_frac_bits
(intel_vvp_scaler_instance *instance);
```

Description Returns the value of the H_COEFFS_FRAC_BITS register. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for horizontal polyphase scaling

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns the number of fractional bits set at IP generation time to parameterize the quantization of the horizontal scaling coefficients

intel_vvp_scaler_is_running

Prototype

```
bool intel_vvp_scaler_is_running(intel_vvp_scaler_instance* instance);
```

Description

Reads and returns the running bit of the STATUS register. The instance must be a valid intel_vvp_scaler_instance fully initialized.

Arguments

instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns

True is the IP is currently receiving/storing an input field, false when between two input fields

intel_vvp_scaler_get_commit_status

Prototype

```
bool intel_vvp_scaler_get_commit_status(intel_vvp_scaler_instance* instance);
```

Description

Reads and returns the commit pending bit of the STATUS register. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured in full mode.

Arguments

instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns

True is the IP is configured in full mode and if the agent interface contains uncommitted writes.

intel_vvp_scaler_get_status

Prototype

```
uint32_t intel_vvp_scaler_get_status(intel_vvp_scaler_instance* instance);
```

Description

Reads the STATUS register. The instance must be a valid intel_vvp_scaler_instance fully initialized.

Arguments

instance – pointer to the intel_vvp_scaler_instance software driver instance structure

Returns

The content of the STATUS register

intel_vvp_scaler_set_output_width

Prototype

```
int intel_vvp_scaler_set_output_width(intel_vvp_scaler_instance* instance, uint32_t output_width)
```

- Description** Selects the new output width. The instance must be a valid `intel_vvp_scaler_instance` fully initialized and configured for horizontal scaling. Also, the output width must be within the allowed range.
- If split horizontal scaling is enabled, consider using `intel_vvp_scaler_set_split_h_scaling` to set up the scaler for split horizontal scaling rather than modifying individual registers manually.
- Arguments** `instance` – pointer to the `intel_vvp_scaler_instance` software driver instance structure
- `output_width` – new output width in the range `[1..max_h_width-1]` where `max_h_width` is the maximum configured `output_width` returned by `intel_vvp_scaler_get_max_output_width`
- Returns** `kIntelVvpCoreOk` for success, negative error code in case of failures
- `kIntelVvpCoreInstanceErr` if the instance is invalid or not configured for horizontal scaling
- `kIntelVvpScalerParameterErr` if the new output width is out-of-range

`intel_vvp_scaler_set_total_input_width`

Prototype

```
int
intel_vvp_scaler_set_total_input_width(intel_vvp_scaler_instance
*instance, uint32_t total_input_width)
```

- Description** Selects the new total input width. The instance must be a valid `intel_vvp_scaler_instance` fully initialized and configured for split horizontal scaling. Consider using `intel_vvp_scaler_set_split_h_scaling` to set up the scaler for split horizontal scaling rather than modifying individual registers manually.
- Arguments** `instance` – pointer to the `intel_vvp_scaler_instance` software driver instance structure
- `total_input_width` – new width of the full input image
- Returns** `kIntelVvpCoreOk` for success, negative error code in case of failures
- `kIntelVvpCoreInstanceErr` if the instance is invalid or not configured for split horizontal scaling
- `kIntelVvpScalerParameterErr` if the new input width is invalid

intel_vvp_scaler_set_total_output_width

Prototype

```
int  
intel_vvp_scaler_set_total_output_width(intel_vvp_scaler_instance  
*instance, uint32_t total_output_width)
```

Description Selects the new total output width. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for split horizontal scaling. Consider using intel_vvp_scaler_set_split_h_scaling to set up the scaler for split horizontal scaling rather than modifying individual registers manually.

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

total_output_width – new width of the full output image

Returns kIntelVvpCoreOk for success, negative error code in case of failures
kIntelVvpCoreInstanceErr if the instance is invalid or not configured for split horizontal scaling
kIntelVvpScalerParameterErr if the new output width is invalid

intel_vvp_scaler_set_output_height

Prototype

```
int  
intel_vvp_scaler_set_output_height(intel_vvp_scaler_instance  
*instance, uint32_t output_height)
```

Description Selects the new output height. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for vertical scaling. If split vertical scaling is enabled, consider using intel_vvp_scaler_set_split_v_scaling to set up the scaler for split vertical scaling rather than modifying individual registers manually.

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

output_height – new output height

Returns kIntelVvpCoreOk for success, negative error code in case of failures
kIntelVvpCoreInstanceErr if the instance is invalid or not configured for vertical scaling
kIntelVvpScalerParameterErr if the new output height is invalid

intel_vvp_scaler_set_total_input_height

Prototype

```
int  
intel_vvp_scaler_set_total_input_height(intel_vvp_scaler_instance  
*instance, uint32_t total_input_height)
```

Description Selects the new total input height. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for split vertical scaling. Consider using intel_vvp_scaler_set_split_v_scaling to set up the scaler for split vertical scaling rather than modifying individual registers manually.

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

total_input_height – new height of the full input image

Returns kIntelVvpCoreOk for success, negative error code in case of failures
kIntelVvpCoreInstanceErr if the instance is invalid or not configured for split vertical scaling
kIntelVvpScalerParameterErr if the new input height is invalid

intel_vvp_scaler_set_total_output_height

Prototype

```
int
intel_vvp_scaler_set_total_output_height(intel_vvp_scaler_instance
*instance, uint32_t total_output_height)
```

Description Selects the new total output height. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for split vertical scaling. Consider using intel_vvp_scaler_set_split_v_scaling to set up the scaler for split vertical scaling rather than modifying individual registers manually.

Arguments instance – pointer to the intel_vvp_scaler_instance software driver instance structure

total_output_height – new height of the full output image

Returns kIntelVvpCoreOk for success, negative error code in case of failures
kIntelVvpCoreInstanceErr if the instance is invalid or not configured for split vertical scaling
kIntelVvpScalerParameterErr if the new output height is invalid

intel_vvp_scaler_set_h_pos_error

Prototype

```
int intel_vvp_scaler_set_h_pos_error(intel_vvp_scaler_instance
*instance, uint32_t h_pos_error)
```

Description Selects the new horizontal position error. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for split horizontal scaling. Consider using

intel_vvp_scaler_set_split_h_scaling to set up the scaler for split horizontal scaling rather than modifying individual registers manually.

- Arguments** instance – pointer to the intel_vvp_scaler_instance software driver instance structure
- h_pos_error – new horizontal position error
- Returns** kIntelVvpCoreOk for success, negative error code in case of failures
- kIntelVvpCoreInstanceErr if the instance is invalid or not configured for split horizontal scaling

intel_vvp_scaler_set_v_pos_error

- Prototype** int intel_vvp_scaler_set_v_pos_error(intel_vvp_scaler_instance *instance, uint32_t v_pos_error)
- Description** Selects the new vertical position error. The instance must be a valid intel_vvp_scaler_instance fully initialized and configured for split vertical scaling. Consider using intel_vvp_scaler_set_split_v_scaling to set up the scaler for split vertical scaling rather than modifying individual registers manually.
- Arguments** instance – pointer to the intel_vvp_scaler_instance software driver instance structure
- v_pos_error – new vertical position error
- Returns** kIntelVvpCoreOk for success, negative error code in case of failures
- kIntelVvpCoreInstanceErr if the instance is invalid or not configured for split vertical scaling

intel_vvp_scaler_set_h_extended_fill

- Prototype**
- ```
int
intel_vvp_scaler_set_h_extended_fill(intel_vvp_scaler_instance
*instance, bool extended_fill)
```
- Description** Enables horizontal extension for split scaling. The instance must be a valid intel\_vvp\_scaler\_instance fully initialized and configured for split horizontal scaling. Consider using intel\_vvp\_scaler\_set\_split\_h\_scaling to set up the scaler for split horizontal scaling rather than modifying individual registers manually.
- Arguments** instance – pointer to the intel\_vvp\_scaler\_instance software driver instance structure

extended\_fill – if horizontal split extension is enabled

**Returns** kIntelVvpCoreOk for success, negative error code in case of failures  
kIntelVvpCoreInstanceErr if the instance is invalid or not configured for split horizontal scaling

#### intel\_vvp\_scaler\_set\_v\_extended\_fill

##### Prototype

```
int
intel_vvp_scaler_set_v_extended_fill(intel_vvp_scaler_instance
*instance, bool extended_fill)
```

**Description** Enables vertical extension for split scaling. The instance must be a valid intel\_vvp\_scaler\_instance fully initialized and configured for split vertical scaling. Consider using intel\_vvp\_scaler\_set\_split\_v\_scaling to set up the scaler for split vertical scaling rather than modifying individual registers manually.

**Arguments** instance – pointer to the intel\_vvp\_scaler\_instance software driver instance structure

extended\_fill – if vertical split extension is enabled

**Returns** kIntelVvpCoreOk for success, negative error code in case of failures  
kIntelVvpCoreInstanceErr if the instance is invalid or not configured for split vertical scaling

#### intel\_vvp\_scaler\_set\_h\_phase\_offset\_error

##### Prototype

```
int
intel_vvp_scaler_set_h_phase_offset_error(intel_vvp_scaler_instance
*instance, uint32_t phase_offset, uint32_t phase_error);
```

**Description** Sets the phase offset and phase offset error for split horizontal scaling. The instance must be a valid intel\_vvp\_scaler\_instance fully initialized and configured for split horizontal scaling. Consider using intel\_vvp\_scaler\_set\_split\_h\_scaling to set up the scaler for split horizontal scaling rather than modifying individual registers manually.

**Arguments** instance – pointer to the intel\_vvp\_scaler\_instance software driver instance structure

phase\_offset – new phase offset

phase\_error – new phase offset error

**Returns** kIntelVvpCoreOk for success, negative error code in case of failures  
kIntelVvpCoreInstanceErr if the instance is invalid or not configured for split horizontal scaling

### intel\_vvp\_scaler\_set\_v\_phase\_offset\_error

#### Prototype

```
int
intel_vvp_scaler_set_v_phase_offset_error(intel_vvp_scaler_instance
*instance, uint32_t phase_offset, uint32_t phase_error);
```

#### Description

Sets the phase offset and phase offset error for split vertical scaling. The instance must be a valid intel\_vvp\_scaler\_instance fully initialized and configured for split vertical scaling. Consider using intel\_vvp\_scaler\_set\_split\_v\_scaling to set up the scaler for split vertical scaling rather than modifying individual registers manually.

#### Arguments

instance – pointer to the intel\_vvp\_scaler\_instance software driver instance structure

phase\_offset – new phase offset

phase\_error – new phase offset error

#### Returns

kIntelVvpCoreOk for success, negative error code in case of failures

kIntelVvpCoreInstanceErr if the instance is invalid or not configured for split vertical scaling

### intel\_vvp\_scaler\_set\_split\_h\_scaling

#### Prototype

```
int
intel_vvp_scaler_set_split_h_scaling(intel_vvp_scaler_instance
*instance, uint32_t total_input_width, uint32_t
total_output_width, uint32_t split_output_width, uint32_t
split_output_left_offset, bool extended_fill, uint32_t
*split_input_width, uint32_t *split_input_left_offset);
```

#### Description

This function sets the scaler for split horizontal scaling. Specify the full width of the input image, the full width of the desired output image and the required offset/width of the output split to set up the scaler to generate the given output band. Optionally, the function returns the offset and width of the corresponding band in the input image. This should be used to configure the upstream video processing pipeline and stream the correct video portion into the scaler. This call does not update the input width image info register in Lite mode. intel\_vvp\_core\_set\_img\_info\_width should still be called, presumably using the split\_input\_width value returned here as a parameter.

#### Arguments

instance – pointer to the intel\_vvp\_scaler\_instance software driver instance structure

total\_input\_width – new width of the full input image

total\_output\_width – new width of the full output image

split\_output\_width – width of the scaled split (at the output)

split\_output\_left\_offset – left offset of the scaled split (at the output)

extended\_fill – if the input split should contain extra columns of pixels on the left and/or right to ensure the output splits can be stitched with a joint that is not noticeable

split\_input\_width – the recommended width for the input split. The value is computed and returned by this call (optional, may be NULL)

split\_input\_left\_offset – the recommended left offset for the input split respective to the full input image. The value is computed and returned by this call (optional, may be NULL)

**Returns**

kIntelVvpCoreOk for success, negative error code in case of failures

kIntelVvpCoreInstanceErr if the instance is invalid or not configured for split horizontal scaling

kIntelVvpScalerParameterErr, if one of the dimensions is invalid or if the split output width is larger than allowed by the configuration.

kIntelVvpScalerParameterErr, if the width and left offset of the output split go out-of-range compared to the total output width

kIntelVvpScalerParameterErr, if the split\_input\_width value computed during the call is larger than allowed by the configuration. Note that this value is computed last and the scaler is still reprogrammed in this case.

### intel\_vvp\_scaler\_set\_split\_v\_scaling

#### Prototype

```
int
intel_vvp_scaler_set_split_v_scaling(intel_vvp_scaler_instance
*instance, uint32_t total_input_height, uint32_t
total_output_height, uint32_t split_output_height, uint32_t
split_output_top_offset, bool extended_fill, uint32_t
*split_input_height, uint32_t *split_input_top_offset);
```

**Description**

This function sets the scaler for split vertical scaling. Specify the full height of the input image, the full height of the desired output image and the required offset/height of the output split to set up the scaler to generate the given output band. Optionally, the function returns the offset and height of the corresponding band in the input image. This should be used to configure the upstream video processing pipeline and stream the correct video portion into the scaler. This call does not update the input height image info register in Lite mode. intel\_vvp\_core\_set\_img\_info\_height should still be called, presumably using the split\_input\_height value returned here as a parameter.

|                  |                                                                                                                                                                                     |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Arguments</i> | instance – pointer to the intel_vvp_scaler_instance software driver instance structure                                                                                              |
|                  | total_input_height – new height of the full input image                                                                                                                             |
|                  | total_output_height – new height of the full output image                                                                                                                           |
|                  | split_output_height – height of the scaled split (at the output)                                                                                                                    |
|                  | split_output_top_offset – top offset of the scaled split (at the output)                                                                                                            |
|                  | extended_fill – if the input split should contain extra lines on the top and/or bottom to ensure the output splits can be stitched with a joint that is not noticeable              |
|                  | split_input_height – the recommended height for the input split. The value is computed and returned by this call (optional, may be NULL)                                            |
|                  | split_input_top_offset – the recommended top offset for the input split respective to the full input image. The value is computed and returned by this call (optional, may be NULL) |
| <i>Returns</i>   | kIntelVvpCoreOk for success, negative error code in case of failures                                                                                                                |
|                  | kIntelVvpCoreInstanceErr if the instance is invalid or not configured for split vertical scaling                                                                                    |
|                  | kIntelVvpScalerParameterErr, if one of the dimensions is invalid                                                                                                                    |
|                  | kIntelVvpScalerParameterErr, if the height and top offset of the output split go out-of-range compared to the total output height                                                   |

#### intel\_vvp\_scaler\_set\_v\_bank

##### Prototype

```
int intel_vvp_scaler_set_v_bank(intel_vvp_scaler_instance
*instance, uint8_t bank);
```

|                    |                                                                                                                                                                                                                                          |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i> | Selects the bank of vertical scaler coefficient to use. The instance must be a valid intel_vvp_scaler_instance, fully initialized, and configured for vertical polyphase scaling. Also, the bank of vertical coefficient should be valid |
| <i>Arguments</i>   | instance – pointer to the intel_vvp_scaler_instance software driver instance structure                                                                                                                                                   |
|                    | bank – the identifier for the horizontal bank in the range [0..num_v_banks-1] where v_num_banks is the value returned by intel_vvp_scaler_get_num_vertical_banks                                                                         |
| <i>Returns</i>     | kIntelVvpCoreOk for success, negative error code in case of failures                                                                                                                                                                     |

kIntelVvpCoreInstanceErr if the instance is invalid or not configured for vertical polyphase scaling

kIntelVvpScalerParameterErr if the vertical bank is invalid

### intel\_vvp\_scaler\_set\_h\_bank

#### Prototype

```
int intel_vvp_scaler_set_h_bank(intel_vvp_scaler_instance
*instance, uint8_t bank);
```

**Description** Selects the bank of horizontal scaler coefficient to use. The instance must be a valid intel\_vvp\_scaler\_instance, fully initialized, and configured for horizontal polyphase scaling. Also, the bank of horizontal coefficient should be valid

**Arguments** instance – pointer to the intel\_vvp\_scaler\_instance software driver instance structure

bank – the identifier for the horizontal bank in the range [0..num\_h\_banks-1] where h\_num\_banks is the value returned by intel\_vvp\_scaler\_get\_num\_horizontal\_banks

**Returns** kIntelVvpCoreOk for success, negative error code in case of failures  
kIntelVvpCoreInstanceErr if the instance is invalid or not configured for horizontal polyphase scaling  
kIntelVvpScalerParameterErr if the horizontal bank is invalid

### intel\_vvp\_scaler\_get\_output\_width

#### Prototype

```
uint32_t
intel_vvp_scaler_get_output_width(intel_vvp_scaler_instance
*instance)
```

**Description** Returns the selected output width. The instance must be a valid intel\_vvp\_scaler\_instance fully initialized and configured for horizontal scaling. Also, debug should be enabled to read a sensible value back

**Arguments** instance – pointer to the intel\_vvp\_scaler\_instance software driver instance structure

**Returns** the selected output width

### intel\_vvp\_scaler\_get\_total\_input\_width

#### Prototype

```
uint32_t
intel_vvp_scaler_get_total_input_width(intel_vvp_scaler_instance
*instance)
```



**Description** Returns the selected total input width. The instance must be a valid `intel_vvp_scaler_instance` fully initialized and configured for split horizontal scaling. Also, debug should be enabled to read a sensible value back

**Arguments** instance – pointer to the `intel_vvp_scaler_instance` software driver instance structure

**Returns** the selected total input width for the full image

#### `intel_vvp_scaler_get_total_output_width`

**Prototype**

```
uint32_t
intel_vvp_scaler_get_total_output_width(intel_vvp_scaler_instance
*instance)
```

**Description** Returns the selected total output width. The instance must be a valid `intel_vvp_scaler_instance` fully initialized and configured for split horizontal scaling. Also, debug should be enabled to read a sensible value back

**Arguments** instance – pointer to the `intel_vvp_scaler_instance` software driver instance structure

**Returns** the selected total output width for the full image

#### `intel_vvp_scaler_get_output_height`

**Prototype**

```
uint32_t
intel_vvp_scaler_get_output_height(intel_vvp_scaler_instance
*instance)
```

**Description** Returns the selected output height. The instance must be a valid `intel_vvp_scaler_instance` fully initialized and configured for vertical scaling. Also, debug should be enabled to read a sensible value back

**Arguments** instance – pointer to the `intel_vvp_scaler_instance` software driver instance structure

**Returns** the selected output height

#### `intel_vvp_scaler_get_total_input_height`

**Prototype**

```
uint32_t
intel_vvp_scaler_get_total_input_height(intel_vvp_scaler_instance
*instance)
```

*Description* Returns the selected total input height. The instance must be a valid `intel_vvp_scaler_instance` fully initialized and configured for split vertical scaling. Also, debug should be enabled to read a sensible value back

*Arguments* instance – pointer to the `intel_vvp_scaler_instance` software driver instance structure

*Returns* the selected total input height for the full image

#### `intel_vvp_scaler_get_total_output_height`

*Prototype*

```
uint32_t intel_vvp_scaler_get_total_output_height
(intel_vvp_scaler_instance *instance)
```

*Description* Returns the selected total output height. The instance must be a valid `intel_vvp_scaler_instance` fully initialized and configured for split vertical scaling. Also, debug should be enabled to read a sensible value back

*Arguments* instance – pointer to the `intel_vvp_scaler_instance` software driver instance structure

*Returns* the selected total output height for the full image

#### `intel_vvp_scaler_get_h_pos_error`

*Prototype*

```
uint32_t
intel_vvp_scaler_get_h_pos_error(intel_vvp_scaler_instance
*instance)
```

*Description* Returns the selected horizontal position error. The instance must be a valid `intel_vvp_scaler_instance` fully initialized and configured for split horizontal scaling. Also, debug should be enabled to read a sensible value back

*Arguments* instance – pointer to the `intel_vvp_scaler_instance` software driver instance structure

*Returns* the selected horizontal position error for the full image

#### `intel_vvp_scaler_get_v_pos_error`

*Prototype*

```
uint32_t
intel_vvp_scaler_get_v_pos_error(intel_vvp_scaler_instance
*instance)
```

**Description** Returns the selected vertical position error. The instance must be a valid `intel_vvp_scaler_instance` fully initialized and configured for split vertical scaling. Also, debug should be enabled to read a sensible value back

**Arguments** instance – pointer to the `intel_vvp_scaler_instance` software driver instance structure

**Returns** the selected vertical position error for the full image

#### `intel_vvp_scaler_get_h_extended_fill`

**Prototype**

```
bool
intel_vvp_scaler_get_h_extended_fill(intel_vvp_scaler_instance
*instance)
```

**Description** Returns if horizontal extended fill is enabled. The instance must be a valid `intel_vvp_scaler_instance` fully initialized and configured for split horizontal scaling. Also, debug should be enabled to read a sensible value back

**Arguments** instance – pointer to the `intel_vvp_scaler_instance` software driver instance structure

**Returns** if horizontal extended fill is assumed

#### `intel_vvp_scaler_get_v_extended_fill`

**Prototype**

```
bool
intel_vvp_scaler_get_v_extended_fill(intel_vvp_scaler_instance
*instance)
```

**Description** Returns if vertical extended fill is enabled. The instance must be a valid `intel_vvp_scaler_instance` fully initialized and configured for split vertical scaling. Also, debug should be enabled to read a sensible value back

**Arguments** instance – pointer to the `intel_vvp_scaler_instance` software driver instance structure

**Returns** if vertical extended fill is assumed

#### `intel_vvp_scaler_get_h_phase_offset_error`

**Prototype**

```
int
intel_vvp_scaler_get_h_phase_offset_error(intel_vvp_scaler_instance
*instance, uint32_t *phase_offset, uint32_t *phase_error)
```

**Description** Returns the selected horizontal phase offset and phase offset error. The instance must be a valid `intel_vvp_scaler_instance` fully initialized and configured for split horizontal polyphase scaling. Also, debug should be enabled to read sensible values back

**Arguments** instance – pointer to the `intel_vvp_scaler_instance` software driver instance structure

**Returns** `kIntelVvpCoreOk` in case of success, negative error code otherwise  
  
`kIntelVvpCoreInstanceErr` if the instance is invalid, or not configured for horizontal split scaling with polyphase scaling algorithm  
  
`kIntelVvpCoreNullPtrErr` if `phase_offset` or `phase_error` are NULL pointers

#### `intel_vvp_scaler_get_v_phase_offset_error`

**Prototype**

```
int
intel_vvp_scaler_get_v_phase_offset_error(intel_vvp_scaler_instance
*instance, uint32_t *phase_offset, uint32_t *phase_error)
```

**Description** Returns the selected vertical phase offset and phase offset error. The instance must be a valid `intel_vvp_scaler_instance` fully initialized and configured for split horizontal polyphase scaling. Also, debug should be enabled to read a sensible value back

**Arguments** instance – pointer to the `intel_vvp_scaler_instance` software driver instance structure

**Returns** `kIntelVvpCoreOk` in case of success, negative error code otherwise  
  
`kIntelVvpCoreInstanceErr` if the instance is invalid, or not configured for vertical split scaling with polyphase scaling algorithm  
  
`kIntelVvpCoreNullPtrErr` if `phase_offset` or `phase_error` are NULL pointers

#### `intel_vvp_scaler_get_v_bank`

**Prototype**

```
uint8_t intel_vvp_scaler_get_v_bank(intel_vvp_scaler_instance
*instance);
```

**Description** Returns the selected bank of vertical scaling coefficient. The instance must be a valid `intel_vvp_scaler_instance`, fully initialized, and configured for vertical polyphase scaling. Also, debug must be enabled to read a sensible value back.

**Arguments** instance – pointer to the `intel_vvp_scaler_instance` software driver instance structure

**Returns** the selected vertical bank

### intel\_vvp\_scaler\_get\_h\_bank

#### Prototype

```
uint8_t intel_vvp_scaler_get_h_bank(intel_vvp_scaler_instance*instance);
```

#### Description

Returns the selected bank of horizontal scaling coefficient. The instance must be a valid intel\_vvp\_scaler\_instance, fully initialized, and configured for horizontal polyphase scaling. Also, debug must be enabled to read a sensible value back.

#### Arguments

instance – pointer to the intel\_vvp\_scaler\_instance software driver instance structure

#### Returns

the selected horizontal bank

### intel\_vvp\_scaler\_set\_quantized\_coeffs

#### Prototype

```
int
intel_vvp_scaler_set_quantized_coeffs(intel_vvp_scaler_instance*
instance, bool h_coeffs, const int32_t* quantized_coeffs,
uint8_t num_coeffs, uint8_t bank, uint8_t phase);
```

#### Description

Programs the IP with a new set of num\_coeffs coefficients for the specified horizontal or vertical phase in the specified bank of coefficients. An error is returned if the IP is not properly configured to use these coefficients. The num\_coeffs parameter indicates the guaranteed minimum size of the quantized\_coeffs array and must match with the number of horizontal or vertical taps as appropriate. Also, the call checks for correctness of the quantized values and returns an error if invalid bits are set. Sign extension to the full 32 bit range is accepted. Quantization errors do not prevent the coefficients from being updated

#### Arguments

instance – pointer to the intel\_vvp\_scaler\_instance software driver instance structure

h\_coeffs – true if committing a set of horizontal coefficients, false for a set of vertical coefficients

quantized\_coeffs – pointer to the structure containing the quantized coefficients

num\_coeffs – the number of horizontal taps if h\_coeffs is true, otherwise the number of vertical taps. The caller guarantees that the quantized\_coeffs array is larger than the specified number of values

bank – the bank of coefficients used to commit the new set

phase – the phase for the set of new coefficients

#### Returns

kIntelVvpCoreOk in case of success, negative error code otherwise

kIntelVvpCoreInstanceErr if the instance is invalid, or not configured for horizontal/vertical polyphase scaling and run-time loading of coefficients

kIntelVvpScalerParameterErr if the phase, bank or number of coefficients are invalid for the IP configuration

kIntelVvpCoreNullPtrErr if quantized\_coefs is a NULL pointer

kIntelVvpScalerQuantizationErr if the quantized coefficients are out-of-range (this does not prevent the coefficients update)

### intel\_vvp\_scaler\_set\_coefs

#### Prototype

```
int intel_vvp_scaler_set_coefs(intel_vvp_scaler_instance*
instance, bool h_coefs, const float* quantized_coefs,
uint8_t num_coefs, uint8_t bank, uint8_t phase);
```

#### Description

Programs the IP with a new set of num\_coefs coefficients for the specified horizontal or vertical phase in the specified bank of coefficients. Coefficients are quantized using the fixed-point configuration reported by the IP. An error is returned if the IP is not properly configured to use these coefficients. The num\_coefs parameter indicates the guaranteed minimum size of the coefs array and must match with the number of horizontal or vertical taps as appropriate. An error is returned if the quantization could not be achieved successfully.

#### Arguments

instance – pointer to the intel\_vvp\_scaler\_instance software driver instance structure

h\_coefs – true if committing a set of horizontal coefficients, false for a set of vertical coefficients

coefs – pointer to the structure containing the set of floating-point coefficients

num\_coefs – the number of horizontal taps if h\_coefs is true, otherwise the number of vertical taps. The caller guarantees that the quantized\_coefs array is larger than the specified number of values

bank – the bank of coefficients used to commit the new set

phase – the phase for the set of new coefficients

#### Returns

kIntelVvpCoreOk in case of success, negative error code otherwise

kIntelVvpCoreInstanceErr if the instance is invalid, or not configured for horizontal/vertical polyphase scaling and run-time loading of coefficients

kIntelVvpScalerParameterErr if the phase, bank or number of coefficients are invalid for the IP configuration

kIntelVvpCoreNullPtrErr if quantized\_coeffs is a NULL pointer

kIntelVvpScalerQuantizationErr if the quantized coefficients are out-of-range for the quantization constraints (which does not prevent the coefficients update)

#### intel\_vvp\_scaler\_commit\_writes

*Prototype*

```
int intel_vvp_scaler_commit_writes(intel_vvp_scaler_instance* instance)
```

*Description*

Commits all pending writes before processing the next field. The instance must be valid and parameterized in full mode.

*Arguments*

instance – pointer to the intel\_vvp\_scaler\_instance software driver instance structure

*Returns*

kIntelVvpCoreOk (0) for success, negative error code otherwise

#### Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

## 40. Stream Cleaner IP

### 40.1. About the Stream Cleaner IP

The Stream Cleaner IP fixes broken streams of video.

To comply with the Altera Streaming Video Protocol Specification, the Stream Cleaner:

- Fixes broken packets.
- Fixes the order of packets.

The IP has one Altera Streaming Video input and one Altera Streaming Video output. The IP is only available as a full variant. For more information on IP variants, refer to the *Altera Streaming Video Protocol Specification*.

#### Related Information

[Altera Streaming Video Protocol Specification](#)

#### 40.1.1. Stream Cleaner IP Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family and the number of supported bits per color sample, pixels in parallel, and the number of color planes.

**Table 751. Stream Cleaner IP Performance and Resources**

For an example parameterization of 2 pixels in parallel, 10 bits per color sample, 3 color planes and turning on the pipelined `ready` signal.

| Target Device                                 | ALMs | f <sub>MAX</sub> (MHz) |
|-----------------------------------------------|------|------------------------|
| Agilex 7 speed grade 2 (AGFA012R24A2E2V)      | 327  | 881.8                  |
| Arria 10 speed grade 1 (10AS066H1F34E1HG)     | 256  | 613.9                  |
| Cyclone 10 GX speed grade 5 (10CX220YF672E5G) | 258  | 546.2                  |
| Stratix 10 speed grade 1 (1SX280LN2F43E1VG)   | 326  | 554.3                  |

### 40.2. Stream Cleaner IP Parameters

The IP offers compile-time parameters

**Table 752. Stream Cleaner IP Parameters**

| Parameter                    | Values  | Description                                 |
|------------------------------|---------|---------------------------------------------|
| <b>Video Data Format</b>     |         |                                             |
| <b>Bits per color sample</b> | 8 to 16 | Select the number of bits per color sample. |
| <i>continued...</i>          |         |                                             |



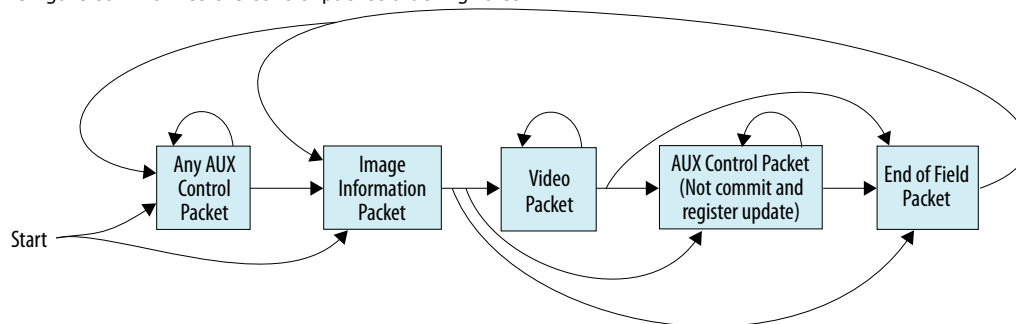
| Parameter                    | Values    | Description                                                      |
|------------------------------|-----------|------------------------------------------------------------------|
| Number of color planes       | 1 to 4    | Select the number of color planes per pixel.                     |
| Number of pixels in parallel | 1 to 8    | Select the number of pixels in parallel.                         |
| <b>General</b>               |           |                                                                  |
| Pipeline Ready               | On or off | Turn on for pipelining of the <code>tready</code> output signal. |

### 40.3. Stream Cleaner IP Functional Description

Altera Video and Vision IPs follow the Altera Video and Vision Protocol. In full mode, this protocol consists of metapackets of a specific length and data packets of variable length. For example, an image information packet is four beats in length; end-of-field packets and timestamp packets are two and three beats in length, respectively. For legal packet ordering, refer to the *Altera Video and Vision Protocol Specification*

**Figure 105. Packet ordering rules**

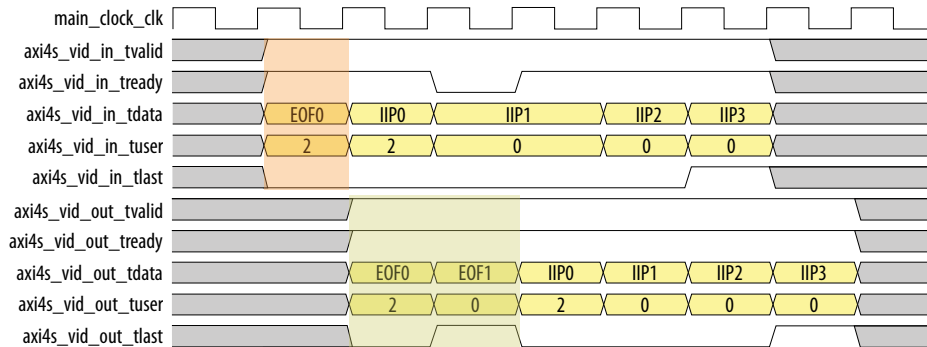
The figure summarizes the control packet ordering rules



A field must start with an image information packet, followed by 0 to  $n$  video data packets and then 0 to  $m$  auxiliary and timestamp packets. The field is then terminated by an end-of-field packet. After the end-of-field packet and before the image information packet, the protocol allows any number of auxiliary packets. If in any way this protocol is not adhered to, the video and vision IPs do not work correctly.

All IPs receive and transmit according to the protocol. It is difficult to produce a stream that does not comply with the protocol. In most cases, you do not require a Stream Cleaner IP in the video processing pipeline. However, in situations such as when switching may occur midpacket (crash switching), the output stream may produce broken control packets that do not match their required length, or packet orders which are not permitted by the protocol.

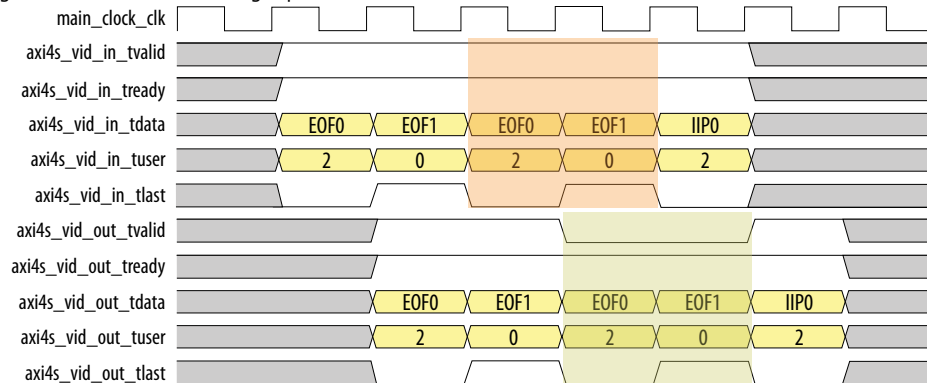
**Figure 106. Stream Cleaner fixing broken packet example**



In the figure, the Altera Streaming Video input receives the first beat of an end-of-field packet followed by the first beat of an image information packet. The end-of-field is therefore missing its second beat. Detecting this fault, the Stream Cleaner pauses the input stream by dropping `axi_vid_in_tready` low and inserts the second beat of the end-of-field before continuing the input stream by raising `axi_vid_in_tready`. The packet is now legal, although the data in the second beat is incorrect.

**Figure 107. An example of a packet order that violates the Altera Streaming Video Protocol Specification**

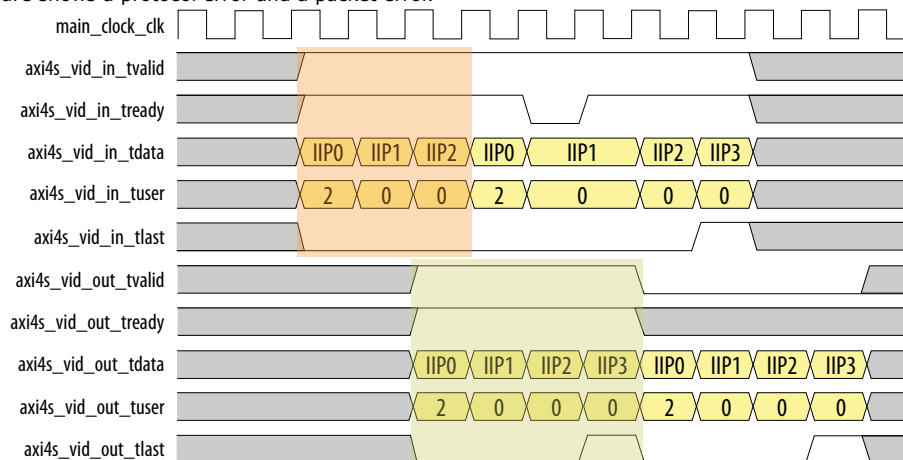
The figure shows the IP detecting a protocol error.



An end-of-field packet is followed by another end-of-field packet, which is an erroneous sequence. The *Altera Streaming Video Protocol Specification* states that an end-of-field packet must be followed by either an image information packet or an auxiliary control packet. The IP drops output `axi_vid_in_tvalid` low to prevent it from passing the data on to the next component. `axi_vid_in_tvalid` goes high again when a valid packet is on the output of the Stream Cleaner.

**Figure 108. An example of a broken packet and a packet order that violates the Altera Streaming Video Protocol Specification.**

The figure shows a protocol error and a packet error.



A broken image information packet (shaded red) is followed by a complete image information packet. The first image information packet is broken due to missing its final beat. The Stream Cleaner produces a fixed packet (shaded green) by recreating the missing beat using the data from the previous beat. It then ensures the `axi_vid_out_tlast` signal goes high.

The *Altera Streaming Video Protocol Specification* states that an image information packet is followed by

- A data packet (with the `tuser[0]` start-of-field signal high)
- A timestamp packet
- A custom auxiliary packet
- An end-of-field packet.

The Stream Cleaner discards the second image information packet by lowering the `axi_vid_out_tvalid` signal. The `axi_vid_out_tvalid` signal raises again when a packet adhering to the specification passes through.

The Stream Cleaner only supports user auxiliary Packets of ID range 16-31 and requires them to be 4 beats in length.

### 40.3.1. Stream Cleaner IP Interfaces

**Table 753. Stream Cleaner IP Interfaces**

| Name                                     | Direction | Width | Description              |
|------------------------------------------|-----------|-------|--------------------------|
| <b>Clocks and resets</b>                 |           |       |                          |
| <code>main_clock_clk</code>              | Input     | 1     | AXI4-S processing clock. |
| <code>main_reset_rst</code>              | Input     | 1     | AXI4-S processing reset. |
| <b>Altera Streaming Video interfaces</b> |           |       |                          |
| <i>continued...</i>                      |           |       |                          |

| Name                       | Direction | Width | Description                    |
|----------------------------|-----------|-------|--------------------------------|
| axi4s_vid_in_tdata         | Input     | (124) | AXI4-S data in.                |
| axi4s_vid_in_tvalid        | Input     | 1     | AXI4-S data valid.             |
| axi4s_vid_in_tuser[0]      | Input     | 1     | AXI4-S start of video frame.   |
| axi4s_vid_in_tuser[1]      | Input     | 1     | AXI4-S control or data packet. |
| axi4s_vid_in_tuser[N-1:2]  | Input     | (125) | Unused.                        |
| axi4s_vid_in_tlast         | Input     | 1     | AXI4-S end of packet.          |
| axi4s_vid_in_tready        | Output    | 1     | AXI4-S data ready.             |
| axi4s_vid_out_tdata        | Output    | (124) | AXI4-S data in.                |
| axi4s_vid_out_tvalid       | Output    | 1     | AXI4-S data valid.             |
| axi4s_vid_out_tuser[0]     | Output    | 1     | AXI4-S start of video frame.   |
| axi4s_vid_out_tuser[1]     | Output    | 1     | AXI4-S control or data packet. |
| axi4s_vid_out_tuser[N-1:2] | Output    | (125) | Unused.                        |
| axi4s_vid_out_tlast        | Output    | 1     | AXI4-S end of packet.          |
| axi4s_vid_out_tready       | Input     | 1     | AXI4-S data ready              |

### Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

---

(124) The equation gives all `tdata` widths sizes in these interfaces:  

$$\max(2, \text{ceil}((\text{bits per color sample} \times \text{number of color planes})/8)) \times \text{pixels in parallel} \times 8$$

(125) This equation gives all `tuser` widths sizes in these interfaces:  $N = \text{ceil}(\text{tdata width} / 8)$



## 41. Switch IP

---

### 41.1. About the Switch IP

The Switch IP allows connections between video inputs and outputs to provide cross-point switching, multipoint switching, and video broadcast functions.

The switch supports:

- Up to 8 independent video outputs.
- Up to 8 independent video inputs, configurable to block, consume or drive any number of the 1-8 video outputs.
- Clean or crash switching of video outputs.
- Lite, full, or full raster variants.
- Optional `tready` signals for full raster variants.
- Clean switching on field boundaries.
- Configurable line switching for lite or full raster variants.
- Propagation of auxiliary control packets with their associated field, for full variants.
- 1 to 8 pixels in parallel and any color space.
- Autoconsume inputs for full variants with clean switching

For more information on lite, full, and full raster variants refer to the *Altera Streaming Video Protocol Specification*. The switch IP takes input resolution information from image information packets or from the register interface for lite and full raster variants.

An Avalon memory-mapped interface allows the run-time configuration of the switch.

For information about the reset behavior for the switch, refer to *Reset Behavior* in *Video and Vision IPs Functional Description*.

#### Related Information

- [Video and Vision Processing IPs Functional Description](#) on page 21
- [Altera Streaming Video Protocol Specification](#)

#### 41.1.1. Switch IP Performance and Resources

The IP resources depend on the device family, bits per sample, pixels in parallel, number of color planes.

**Table 754. Switch IP Performance and Resources**

The table shows ALM usage and  $f_{MAX}$  for a 4 input, 4 output switch with 2 pixels in parallel, 10 bits per color sample, 3 color planes, and separate control clock.

| Target Device                                  | ALMs  | $f_{MAX}$ (MHz) |
|------------------------------------------------|-------|-----------------|
| Agilex 7 speed grade 2 (AGFA012R24A2E2V)       | 3,265 | 671             |
| Arria 10 speed grade 2 (10AS066H1F34E1HG )     | 3,034 | 449             |
| Cyclone 10 GX speed grade 5 (10CX220YF672E5G)  | 3,046 | 417             |
| Stratix 10 GX speed grade 2 (1SX280LN2F43E1VG) | 3,365 | 581             |

## 41.2. Switch IP Parameters

The IP offers compile- and run-time parameters.

**Table 755. Switch IP Parameters**

| Parameter                                                     | Values                         | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------------------------------------|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Video Data Format</b>                                      |                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Altera Streaming Video interface type</b>                  | <b>Full, Lite, Full raster</b> | Select the required streaming video protocol.                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Bits per color sample</b>                                  | 8 to 16                        | Select the number of bits per color sample.                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Number of color planes</b>                                 | 1 to 4                         | Select the number of color planes per pixel.                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Number of pixels in parallel</b>                           | 1 to 8                         | Select the number of color planes per pixel.                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Switch settings</b>                                        |                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Number of inputs</b>                                       | 1 to 8                         | Select the number of inputs required                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Number of outputs</b>                                      | 1 to 8                         | Select the number of outputs required                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Crash switching</b>                                        | On or off                      | Select the type of switching required. Crash switching may cut short packets.                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Autoconsume inputs</b>                                     | On or off                      | Turn on to allow inputs to consume automatically during switches if required.                                                                                                                                                                                                                                                                                                                                                                                |
| <b>All inputs are uninterrupted</b>                           | On or off                      | For lite variants, turn on only when you can ensure that the start of another field always follows the end of each field. If you turn on for lite variants, the switch occurs at the start-of-field, as indicated by <code>TUSER[0]</code> . If you turn off for lite variants, the IP switch occurs at the end of each line, as indicated by <code>TLAST</code> . Full variants do not use this parameter, as the switch always occurs at field boundaries. |
| <b>'tready' signal present on switch streaming interfaces</b> | On or off                      | For full raster variants only, select if <code>tready</code> signals are present on the switch inputs and output connections.                                                                                                                                                                                                                                                                                                                                |
| <b>Control settings</b>                                       |                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Debug features</b>                                         | On or off                      | No effect. The Switch IP has no debugging features.                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Separate clock for control interface</b>                   | On or off                      | Turn on for a separate clock for the control interface                                                                                                                                                                                                                                                                                                                                                                                                       |

**Figure 109. Switch IP GUI**

System: foo Path: intel\_vvp\_switch\_0

HDL entity name: foo\_intel\_vvp\_switch\_0 IP file: ip/foo/foo\_intel\_vvp\_switch\_0.ip

Any changes here will be written out to disk when the system is saved.

**Switch Intel FPGA IP**  
intel\_vvp\_switch

**Video Data Format**

Intel FPGA Streaming Video interface type: Full raster

Bits per color sample: 10

Number of color planes: 2

Number of pixels in parallel: 2

**Switch settings**

Number of inputs: 4

Number of outputs: 4

☒ Crash switching

☐ Auto-consume inputs

☐ All inputs are uninterrupted

☐ 'tready' signals are present on switch streaming interfaces

**Control**

☐ Debug features

☒ Separate clock for control interface

**Parameterization Messages**

| Type        | Message                                                                                               |
|-------------|-------------------------------------------------------------------------------------------------------|
| Information | 'Crash switching' is set. Switching will occur immediately without waiting for line or field endings. |

### 41.3. Switch IP Functional Description

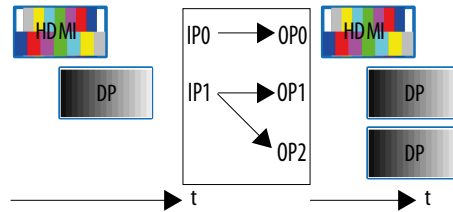
The IP receives incoming video fields from its Altera Streaming Video inputs and propagates them to its Altera Streaming Video outputs with control via an Avalon memory-mapped interface. The IP blocks, consumes, or routes input fields to outputs according to the control register settings. The IP does not perform a switch on interlaced fields in pairs. The switch occurs as soon as possible and so field pairs may separate during the switch. If crash switching is on, packets may be cut short during a switch. For full variants with crash switching, the IP may cut short output packets but it also makes them legal according to the Altera Streaming Video Protocol .

### Autoconsume Inputs

Autoconsume minimizes backpressure during switching. Turn on **Autoconsume inputs** if the switch inputs connect directly or indirectly to other IPs that are sensitive to backpressure, such as the video connectivity IPs. If the switch inputs connect directly or indirectly to sources such as a frame buffer, the frame buffer can tolerate backpressure and so you can turn off **Autoconsume inputs** to ensure that outputs switch promptly.

**Figure 110. Autoconsume Example: 2 input, 3 output switch initial connections**

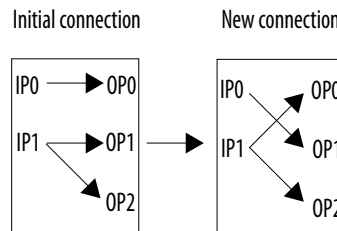
To explain autoconsume, consider a 2 input, 3 output switch with two unsynchronized inputs (one HDMI and one DisplayPort)



In this example, input 0 (IP0) drives output 0 (OP0) and input 1 (IP1) drives both outputs 1 and 2 (OP1, OP2). 4k60 video drives both inputs but with the HDMI start-of-fields occurring before the DisplayPort start-of-fields.

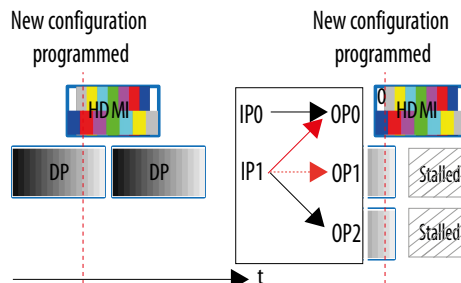
**Figure 111. Autoconsume Example: configuring new connections**

The figure shows a new configuration with IP0 driving OP1 and IP1 driving OP0, but keeping the OP2 from IP1 connection open. The Switch IP makes the new connections at the earliest opportunity after a write to the COMMIT register.



**Figure 112. Autoconsume Example: 2 input, 3 output switch with new connections**

The figure shows the IP receiving a write to the COMMIT register for the new configuration at a time marked by the dotted line.



OP0 is still completing the HDMI frame from IP0. OP0 must therefore create backpressure for this new connection request to IP1, which consequently creates backpressure to the DisplayPort input on completing its current field. This

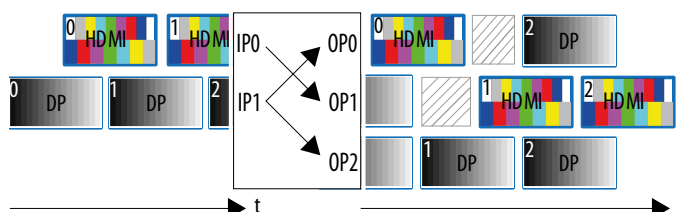


backpressure has the undesirable side effect of starving OP2 of video and stalling it. Incidentally, OP1 is changing its input source and so also stalls until the next HDMI field becomes available from IP0.

To prevent OP2 from starving, you must turn on **Autoconsume** in the GUI, so OP2 can continue uninterrupted while its current field completes.

Turn on **Autoconsume inputs** so that the IP defers the new connections until the end of the current HDMI field. Until that time OP0 and OP1 consume the IP1 DisplayPort input so the IP does not create backpressure for IP1 and new connections do not affect OP2.

**Figure 113. Switch IP with Autoconsume Inputs**



#### Broken field behavior

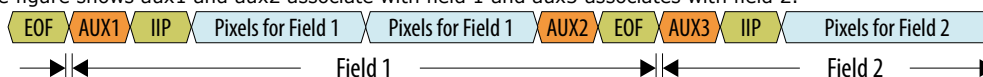
The switch has no sensitivity to broken fields. The IP performs switches on broken fields in the same way as unbroken fields.

#### Metapacket and auxiliary control packet support for full variants

For full variants, the switch supports all packet types and routes all the packets received on its inputs to the appropriate outputs. The IP defers the switch until it sees an end of field packet on an input, so that all packets between the image information packet and the end of field packet keep together. For more information, refer to the *Altera Streaming Video Protocol Specification*.

**Figure 114. Auxiliary Packet Rules**

The figure shows aux1 and aux2 associate with field 1 and aux3 associates with field 2.



Pairs of image information and end-of-field packets mark one field of video. Any auxiliary control packets before the current field's end-of-field packet are associated with that field. In addition, any auxiliary control packets that occur after the end-of-field packet of the preceding field are also associated with the current field.

### 41.3.1. Switch IP Latency

The Switch IP latency depends on clean or crash switches

#### Clean Switch Latency

When you turn off **Crash switching**, the IP switch occurs cleanly, with the last packet on each output completed legally, as denoted by `tlast`.

The switch IP latency for clean switches depends on the complexity of the switch made, the configuration of the switch, the timing of the switch command, the timing on the inputs, and any backpressure experienced on the outputs.

The minimum switch latency ( $L_{\text{clean\_switch}}$ ) is the number of clock cycles from the submitting of a new switch configuration via a write to the `COMMIT` register, to the start of the first image information packet (full variants) or first line (lite variants) produced at the configured outputs.

$$L_{\text{clean\_switch}} = T_{\text{remaining}} + 8 + (C \cdot 6 : 3) \cdot I + 8 \cdot O$$

where

- $T_{\text{remaining}}$  = the number of cycles from the write to `COMMIT` to the end-of-field packet of the current input field (for full variants) or to the `TLAST` of the current line (lite variants) or to the next `TUSER[0]` (lite variants with **All inputs are uninterrupted** on).
- $I$  = The number of inputs whose state is changing (either consume, enable, disable or destination)
- $O$  = The number of outputs whose state is changing (either enable, disable, or source)
- $C$  is 1 with **Autoconsume inputs** on.

This equation holds in the absence of backpressure and in a fully synchronized system with all switch inputs receiving fields of the same size at the same time, and common host and main clocks.

Latency in a real system is dominated by the timing of the input fields and  $L_{\text{switch}}$  usually only represents a very small percentage of overall switching time.

The fastest switching configurations are lite variants with **All inputs are uninterrupted** off, as changes occur at line endings, not field endings.

### Crash Switch Latency

When you turn on **Crash switching**, the IP switches occur faster than for clean switching but broken packets may occur at the switch outputs. Crash switch latency is given by  $L_{\text{crash\_switch}}$  and switch latency is unaffected by backpressure.

$$L_{\text{crash\_switch}} \leq (\text{Total number of outputs configured}) \cdot 4 + 4$$

## 41.3.2. Switch IP Interfaces

Table 756. Switch IP Interfaces

| Name                           | Direction | Width | Description                             |
|--------------------------------|-----------|-------|-----------------------------------------|
| <b>Clocks and resets</b>       |           |       |                                         |
| <code>main_clock_clk</code>    | Input     | 1     | AXI4-S processing clock.                |
| <code>main_reset_rst</code>    | Input     | 1     | AXI4-S processing reset.                |
| <code>agent_clock_clk</code>   | Input     | 1     | Optional control agent interface clock. |
| <code>agent_reset_reset</code> | Input     | 1     | Optional control agent interface reset. |
| continued...                   |           |       |                                         |

| Name                                          | Direction | Width | Description                              |
|-----------------------------------------------|-----------|-------|------------------------------------------|
| <b>Control interfaces</b>                     |           |       |                                          |
| av_mm_control_agent_address                   | Input     | 7     | Avalon memory-mapped agent address       |
| av_mm_control_agent_write                     | Input     | 1     | Avalon memory-mapped agent write.        |
| av_mm_control_agent_writedata                 | Input     | 32    | Avalon memory-mapped agent write data.   |
| av_mm_control_agent_byteenable                | Input     | 4     | Avalon memory-mapped agent byte enable.  |
| av_mm_control_agent_read                      | Input     | 1     | Avalon memory-mapped agent read.         |
| av_mm_control_agent_readdata                  | Output    | 32    | Avalon memory-mapped agent read data.    |
| av_mm_control_agent_readdatavalid             | Output    | 1     | Avalon memory-mapped agent read.         |
| av_mm_control_agent_waitrequest               | Output    | 1     | Avalon memory-mapped agent wait request. |
| <b>Altera Streaming Video interfaces</b>      |           |       |                                          |
| Input interface number $N$ ( $1 \leq N < 8$ ) |           |       |                                          |
| axi4s_vid_in_N_tdata                          | Input     | (126) | AXI4-S data in.                          |
| axi4s_vid_in_N_tvalid                         | Input     | 1     | AXI4-S data valid.                       |
| axi4s_vid_in_N_tuser[0]                       | Input     | 1     | AXI4-S start of video frame.             |
| axi4s_vid_in_N_tuser[1]                       | Input     | 1     | AXI4-S control or data packet.           |
| axi4s_vid_in_N_tuser[TUSERW-1:2]              | Input     | (127) | Unused.                                  |
| axi4s_vid_in_N_tlast                          | Input     | 1     | AXI4-S end of packet.                    |
| axi4s_vid_in_N_tready                         | Output    | 1     | AXI4-S data ready.                       |
| <b>Output interface number</b>                |           |       |                                          |
| $M$ ( $1 \leq M < 8$ )                        |           |       |                                          |
| axi4s_vid_out_M_tdata                         | Output    | (126) | AXI4-S data in.                          |
| axi4s_vid_out_M_tvalid                        | Output    | 1     | AXI4-S data valid.                       |
| axi4s_vid_out_M_tuser[0]                      | Output    | 1     | AXI4-S start of video frame.             |
| axi4s_vid_out_M_tuser[1]                      | Output    | 1     | AXI4-S control or data packet.           |
| axi4s_vid_out_M_tuser[TUSERW-1:2]             | Output    | (127) | Unused.                                  |
| axi4s_vid_out_M_tlast                         | Output    | 1     | AXI4-S end of packet.                    |
| axi4s_vid_out_M_tready                        | Input     | 1     | AXI4-S data ready.                       |

### Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

(126) The equation gives all tdata widths sizes in these interfaces:  
 $\max(2, \text{ceil}((\text{bits per color sample} \times \text{number of color planes})/8)) \times \text{pixels in parallel} \times 8$

(127) The equation gives all tuser widths sizes in these interfaces  $TUSERW = \text{ceil}(\text{tdata width} / 8)$   
 (127)

## 41.4. Switch IP Registers

Each register is either read-only (RO) or read-write (RW).

**Table 757. Switch IP Registers**

In the software API the register names appear with a prefix of `INTEL_VVP`, `INTEL_VVP_CORE`, or `INTEL_VVP_VIDEO_SWITCH` as appropriate and with an optional `REG` suffix.

| Address                                                              | Register             | Access | Description                                                                                                                                                                                                                                                                                           |
|----------------------------------------------------------------------|----------------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Parameterization registers</b>                                    |                      |        |                                                                                                                                                                                                                                                                                                       |
| 0x0000                                                               | PROD_ID              | RO     | Read this register for the Switch IP product ID. This register always returns 0x6AF7_0235                                                                                                                                                                                                             |
| 0x0004                                                               | VERSION              | RO     | Read this register for the IP version information.                                                                                                                                                                                                                                                    |
| 0x0008                                                               | INTF_TYPE            | RO     | Read this register to determine the interface type. This register returns: <ul style="list-style-type: none"> <li>0 when you select <b>Full</b> interfaces.</li> <li>1 when you select <b>Lite</b> interfaces.</li> <li>2 when you select <b>full raster</b> interfaces.</li> </ul>                   |
| 0x000C                                                               | DEBUG_ENABLED        | RO     | Read this register to determine if <b>Debug features</b> are on.                                                                                                                                                                                                                                      |
| 0x0010                                                               | UNINTERRUPTED_INPUTS | RO     | Read this register to determine if <b>All inputs are uninterrupted</b> is on.                                                                                                                                                                                                                         |
| 0x0014                                                               | AUTO_CONSUME         | RO     | Read this register to determine if <b>Autoconsume</b> is on.                                                                                                                                                                                                                                          |
| 0x0018                                                               | NUM_INPUTS           | RO     | Read this register to determine the number of configured inputs.                                                                                                                                                                                                                                      |
| 0x001C                                                               | NUM_OUTPUTS          | RO     | Read this register to determine the number of configured outputs.                                                                                                                                                                                                                                     |
| 0x0020                                                               | USE_TREADIES         | RO     | Read this register to determine if <b>'tready' signal present on switch streaming interfaces</b> is on.                                                                                                                                                                                               |
| 0x0024                                                               | CRASH_SWITCH         | RO     | Read this register to determine if <b>Crash switching</b> is on.                                                                                                                                                                                                                                      |
| 0x0028<br>to<br>0x011F                                               | RESERVED             | -      | Unused.                                                                                                                                                                                                                                                                                               |
| <b>Control and debug registers</b>                                   |                      |        |                                                                                                                                                                                                                                                                                                       |
| For more details of these registers, refer to <i>Control Packets</i> |                      |        |                                                                                                                                                                                                                                                                                                       |
| 0x0120<br>to 0x13F                                                   | RESERVED             | -      | Unused.                                                                                                                                                                                                                                                                                               |
| 0x0140                                                               | STATUS               | RO     | Bit 0: Goes low when a switch starts and returns high when switching completes<br>Bit 1: Pending run-time control bit. Goes high when a write occurs to one of the input or output control registers. Goes low after a write to the <code>COMMIT</code> register and the IP switch starts the switch. |
| 0x0144                                                               | COMMIT               | RW     | Write 1 to bit 0 to commit the control register settings and request a new switch. The new switch starts immediately if no switch is currently in progress, otherwise it starts when the current switch completes.                                                                                    |
| continued...                                                         |                      |        |                                                                                                                                                                                                                                                                                                       |

| Address          | Register         | Access              | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------|------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x0148           | INPUT_CONTROL_0  | RW <sup>(128)</sup> | Each input has a control register. The IP uses the two input LSBs for control, decoded as follows:<br><ul style="list-style-type: none"> <li>00: disable. Drive TREADY low and do not propagate data.</li> <li>01: enable. If one of the output registers is configured to receive or consume this input, data propagates, otherwise TREADY is low.</li> <li>10: disable. Drive TREADY low and do not propagate data.</li> <li>11: consume. Drive TREADY high but do not propagate data.</li> </ul> |
| 0x014C           | INPUT_CONTROL_1  | RW <sup>(128)</sup> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 0x0150           | INPUT_CONTROL_2  | RW <sup>(128)</sup> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 0x0154           | INPUT_CONTROL_3  | RW <sup>(128)</sup> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 0x0158           | INPUT_CONTROL_4  | RW <sup>(128)</sup> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 0x015C           | INPUT_CONTROL_5  | RW <sup>(128)</sup> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 0x0160           | INPUT_CONTROL_6  | RW <sup>(128)</sup> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 0x0164           | INPUT_CONTROL_7  | RW <sup>(128)</sup> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 0x0168 to 0x187  | RESERVED         | -                   | Reserved.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 0x0188           | OUTPUT_CONTROL_0 | RW <sup>(128)</sup> | Set bit[8] to enable an output.<br>Set bits [7:0] to give the integer value of the input to drive the output. An out-of-range input value has the effect of disabling the output.<br>Inputs and outputs start at zero.<br>For example, a write of 0x104 to an output control register enables the output and selects input number 4. If the two LSBs in INPUT_CONTROL_4 are 01, the IP enables the connection and input number 4 drives the output.                                                 |
| 0x018C           | OUTPUT_CONTROL_1 | RW <sup>(128)</sup> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 0x0190           | OUTPUT_CONTROL_2 | RW <sup>(128)</sup> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 0x0194           | OUTPUT_CONTROL_3 | RW <sup>(128)</sup> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 0x0198           | OUTPUT_CONTROL_4 | RW <sup>(128)</sup> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 0x019C           | OUTPUT_CONTROL_5 | RW <sup>(128)</sup> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 0x01A0           | OUTPUT_CONTROL_6 | RW <sup>(128)</sup> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 0x01A4           | OUTPUT_CONTROL_7 | RW <sup>(128)</sup> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 0x01A8 to 0x01C4 | RESERVED         | -                   | Reserved.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |

## Register Bit Descriptions

Table 758. PROD\_ID

| Name              | Bits | Description                              |
|-------------------|------|------------------------------------------|
| Switch product ID | 31:0 | This register always returns 0x6AF7_0235 |

Table 759. VERSION

| Name                 | Bits  | Description                          |
|----------------------|-------|--------------------------------------|
| Register map version | 7:0   | Register map version.                |
| IP patch revision    | 15:8  | -                                    |
| IP update revision   | 23:16 | Updated when the IP version changes. |
| IP major revision    | 31:24 | Updated when the IP version changes. |

<sup>(128)</sup> Read only if you turn on **Crash switching**.

**Table 760. INTF\_TYPE**

| Name                                 | Bits | Description                                                                                                                                                                                                                                                                     |
|--------------------------------------|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interface type parameterization bits | 1:0  | A 2-bit interface type register: <ul style="list-style-type: none"> <li>00 when you select <b>Full</b> interfaces.</li> <li>01 when you select <b>Lite</b> interfaces.</li> <li>10 when you select <b>Full raster</b> interfaces.</li> <li>11 is illegal and unused.</li> </ul> |
| Unused                               | 31:2 | Unused.                                                                                                                                                                                                                                                                         |

**Table 761. DEBUG\_ENABLED**

| Name           | Bits | Description |
|----------------|------|-------------|
| Debug features | 31:0 | Unused.     |

**Table 762. UNINTERRUPTED\_INPUTS**

| Name                                      | Bits | Description                                                    |
|-------------------------------------------|------|----------------------------------------------------------------|
| Uninterrupted inputs parameterization bit | 0    | Returns 1 if you turn on <b>All inputs are uninterrupted</b> . |
| Unused                                    | 31:1 | Unused.                                                        |

**Table 763. AUTO\_CONSUME**

| Name                                    | Bits | Description                                          |
|-----------------------------------------|------|------------------------------------------------------|
| Autoconsume inputs parameterization bit | 0    | Returns 1 if you turn on <b>Autoconsume inputs</b> . |
| Unused                                  | 31:1 | Unused.                                              |

**Table 764. NUM\_INPUTS**

| Name             | Bits | Description                              |
|------------------|------|------------------------------------------|
| Number of inputs | 31:0 | Returns the number of inputs configured. |

**Table 765. NUM\_OUTPUTS**

| Name              | Bits | Description                              |
|-------------------|------|------------------------------------------|
| Number of outputs | 31:0 | Returns the number of outputs configured |

**Table 766. USE\_TREADIES**

| Name                               | Bits | Description                                                                              |
|------------------------------------|------|------------------------------------------------------------------------------------------|
| use_ treadies parameterization bit | 0    | Returns 1 if you turn on <b>'tready' signal present on switch streaming interfaces</b> . |
| Unused                             | 31:1 | Unused.                                                                                  |

**Table 767. CRASH\_SWITCH**

| Name                                 | Bits | Description                                       |
|--------------------------------------|------|---------------------------------------------------|
| Crash switching parameterization bit | 0    | Returns 1 if you turn on <b>Crash switching</b> . |
| Unused                               | 31:1 | Unused.                                           |

**Table 768. STATUS**

| Name                         | Bits | Description                                                                                                                                                      |
|------------------------------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Status bit                   | 0    | Goes low when a switch starts and returns high when switching completes                                                                                          |
| Pending register updates bit | 1    | Goes high when a write occurs to one of the input or output control registers. Goes low after a write to the <code>COMMIT</code> register and the switch starts. |
| Unused                       | 31:2 | Unused.                                                                                                                                                          |

**Table 769. COMMIT**

| Name            | Bits | Description                                                                                                                                                                                               |
|-----------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commit register | 0    | Write 1 to commit the control register settings and request a new switch. The new switch starts immediately if no switch is currently in progress, otherwise it starts when the current switch completes. |
| Unused          | 31:1 | Unused.                                                                                                                                                                                                   |

**Table 770. INPUT N CONTROL (0 ≤ N ≤ 7)**

| Name        | Bits | Description                                                                       |
|-------------|------|-----------------------------------------------------------------------------------|
| Enable bit  | 0    | Enable this input                                                                 |
| Consume bit | 1    | Consume this input. You must enable the input for the consume bit to take effect. |
| Unused      | 31:2 | Unused.                                                                           |

**Table 771. OUTPUT N CONTROL (0 ≤ N ≤ 7)**

| Name       | Bits | Description                                                                                                      |
|------------|------|------------------------------------------------------------------------------------------------------------------|
| Source     | 7:0  | Integer value of the input source to drive this output. An out-of-range input source value turns off the output. |
| Enable bit | 8    | Turn on this output                                                                                              |
| Unused     | 31:9 | Unused.                                                                                                          |

## 41.5. Switch IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_switch_regs.h`

Include file: `intel_vvp_switch.h`

**Table 772. Switch IP Software API**

| Name                                            | Description                            |
|-------------------------------------------------|----------------------------------------|
| <code>intel_vvp_switch_init</code>              | Initialize the Switch instance         |
| <code>intel_vvp_switch_get_lite_mode</code>     | Returns if <b>Lite mode</b> is on      |
| <code>intel_vvp_switch_get_debug_enabled</code> | Returns if <b>Debug features</b> is on |
| <i>continued...</i>                             |                                        |

| Name                                      | Description                                                                          |
|-------------------------------------------|--------------------------------------------------------------------------------------|
| intel_vvp_switch_has_uninterrupted_inputs | Returns if the Switch input is uninterruptible and always provides video data        |
| intel_vvp_switch_has_auto_consume         | Returns if the Switch is using a two-phase rerouting to avoid backpressure at inputs |
| intel_vvp_switch_get_num_inputs           | Returns the number of inputs                                                         |
| intel_vvp_switch_get_num_outputs          | Returns the number of outputs                                                        |
| intel_vvp_switch_is_running               | Returns true if the IP is currently processing video data                            |
| intel_vvp_switch_get_commit_status        | Returns true if the IP has uncommitted writes                                        |
| intel_vvp_switch_get_status               | Reads the status register                                                            |
| intel_vvp_switch_set_input_config         | Configures an input behavior (enabled, disabled, consumed)                           |
| intel_vvp_switch_get_input_config         | Returns the selected input behavior                                                  |
| intel_vvp_switch_set_output_config        | Configures an output behavior                                                        |
| intel_vvp_switch_get_output_config        | Returns the selected output behavior                                                 |
| intel_vvp_switch_commit_writes            | Commit all outstanding writes                                                        |

#### enum eIntelVvpSwitchInputConfig

**Members**

- kIntelVvpSwitchInputDisabled (0) - input disabled
- kIntelVvpSwitchInputEnabled (1) - input enabled
- kIntelVvpSwitchInputConsumed (3) - input consumed
- kIntelVvpSwitchInputConfigInvalid (-1) - invalid input configuration (to indicate errors)

**Description** Enumeration for the configurable input behaviors

#### intel\_vvp\_switch\_init

**Prototype**

```
int intel_vvp_switch_init(intel_vvp_switch_instance *instance,
intel_vvp_core_base base);
```

**Description** Initializes a Switch instance. The initialization stops early if the vendor ID or product ID the IP reads at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

**Arguments**

- instance - pointer to the intel\_vvp\_switch\_instance software driver instance structure
- base - base address of the register map



**Returns**

- kIntelVvpCoreOk (0) in case of success, a negative error code in case of error
- kIntelVvpCoreVidErr if the vendor id of the core is not the Altera vendor ID (0x6AF7).
- kIntelVvpCorePidErr if the product\_id does not match with the Switch product ID
- kIntelVvpCoreInstanceErr if the instance is a null pointer
- kIntelVvpSwitchRegMapVersionErr if the register map is not supported

#### intel\_vvp\_switch\_get\_lite\_mode

**Prototype**

```
bool intel_vvp_switch_get_lite_mode(intel_vvp_switch_instance *instance);
```

**Description** Returns the value of the LITE\_MODE register. The instance must be a valid intel\_vvp\_switch\_instance fully initialized

**Arguments** instance - pointer to the intel\_vvp\_switch\_instance software driver instance structure

**Returns** true if the IP is in lite mode

#### intel\_vvp\_switch\_get\_debug\_enabled

**Prototype**

```
bool intel_vvp_switch_get_debug_enabled(intel_vvp_switch_instance *instance);
```

**Description** Returns the value of the DEBUG\_ENABLED register. The instance must be a valid intel\_vvp\_switch\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_switch\_instance software driver instance structure

**Returns** true if the IP is parameterized with debug features enabled

#### intel\_vvp\_switch\_has\_uninterrupted\_inputs

**Prototype**

```
bool intel_vvp_switch_has_uninterrupted_inputs(intel_vvp_switch_instance *instance);
```

**Description** Returns the value of the UNINTERRUPTED\_INPUTS register. The instance must be a valid intel\_vvp\_switch\_instance fully initialized.

**Arguments** instance - pointer to the `intel_vvp_switch_instance` software driver instance structure

**Returns** true if the IP was configured to assume uninterruptible inputs.

#### `intel_vvp_switch_has_auto_consume`

**Prototype**

```
bool
intel_vvp_switch_has_auto_consume(intel_vvp_switch_instance
*instance);
```

**Description** Returns the value of the `AUTO_CONSUME` register. The instance must be a valid `intel_vvp_switch_instance` fully initialized.

**Arguments** instance - pointer to the `intel_vvp_switch_instance` software driver instance structure

**Returns** true is the IP was configured to automatically consume waiting inputs during a switch

#### `intel_vvp_switch_get_num_inputs`

**Prototype**

```
uint8_t
intel_vvp_switch_get_num_inputs(intel_vvp_switch_instance
*instance);
```

**Description** Returns the value of the `NUM_INPUTS` register. The instance must be a valid `intel_vvp_switch_instance` fully initialized.

**Arguments** instance - pointer to the `intel_vvp_switch_instance` software driver instance structure

**Returns** The number of inputs

#### `intel_vvp_switch_get_num_outputs`

**Prototype**

```
uint8_t
intel_vvp_switch_get_num_outputs(intel_vvp_switch_instance*
instance);
```

**Description** Returns the value of the `NUM_OUTPUTS` register. The instance must be a valid `intel_vvp_switch_instance` fully initialized.

**Arguments** instance - pointer to the `intel_vvp_switch_instance` software driver instance structure

**Returns** The number of outputs

### intel\_vvp\_switch\_is\_running

**Prototype**

```
bool intel_vvp_switch_is_running(intel_vvp_switch_instance* instance);
```

**Description**

Reads and returns the running bit of the STATUS register. The instance must be a valid `intel_vvp_switch_instance` fully initialized.

**Arguments**

instance - pointer to the `intel_vvp_switch_instance` software driver instance structure

**Returns**

True if the IP is currently receiving or storing an input field, false when between two input fields

### intel\_vvp\_switch\_get\_commit\_status

**Prototype**

```
bool intel_vvp_switch_get_commit_status(intel_vvp_clipper_instance* instance);
```

**Description**

Reads and returns the commit pending bit of the STATUS register. The instance must be a valid `intel_vvp_switch_instance` fully initialized and configured in full mode.

**Arguments**

instance - pointer to the `intel_vvp_switch_instance` software driver instance structure

**Returns**

True is the IP is configured in full mode and if the agent interface contains uncommitted writes.

### intel\_vvp\_switch\_get\_status

**Prototype**

```
uint32_t intel_vvp_switch_get_status(intel_vvp_switch_instance* instance);
```

**Description**

Reads the STATUS register. The instance must be a valid `intel_vvp_switch_instance` fully initialized.

**Arguments**

instance - pointer to the `intel_vvp_switch_instance` software driver instance structure

**Returns**

The content of the STATUS register

### intel\_vvp\_switch\_set\_input\_config

#### Prototype

```
int
intel_vvp_switch_set_input_config(intel_vvp_switch_instance
*instance, uint8_t input, eIntelVvpSwitchInputConfig
input_config)
```

**Description** Changes the configuration for a chosen input. The instance must be a valid `intel_vvp_switch_instance` fully initialized and the input must be valid

**Arguments** instance - pointer to the `intel_vvp_switch_instance` software driver instance structure

input - selected input in the range `[0..num_inputs-1]` where `num_inputs` is the number of configured inputs

input\_config - new input configuration (enabled, disabled, consumed). An input that is enabled but not tied to any output is automatically consumed

**Returns** `kIntelVvpCoreOk` for success, negative error code in case of failures  
`kIntelVvpCoreInstanceErr` if the instance is invalid  
`kIntelVvpSwitchParameterErr` if the input is out-of-range or the `input_config` parameter is invalid

### intel\_vvp\_switch\_get\_input\_config

#### Prototype

```
eIntelVvpSwitchInputConfig
intel_vvp_switch_get_input_config(intel_vvp_switch_instance
*instance, uint8_t input)
```

**Description** Returns the current configuration for a requested input

**Arguments** instance - pointer to the `intel_vvp_switch_instance` software driver instance structure

input - chosen input in the range `[0..num_inputs-1]` where `num_inputs` is the number of configured inputs

**Returns** The configuration for the selected input,  
`kIntelVvpSwitchInputConfigInvalid` in case of error (invalid instance, debug not enabled, invalid input)

### intel\_vvp\_switch\_set\_output\_config

#### Prototype

```
int
intel_vvp_switch_set_output_config(intel_vvp_switch_instance
*instance, uint8_t output, bool enabled, uint8_t input)
```

|                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i> | Change the configuration of a selected output. The instance must be a valid <code>intel_vvp_switch_instance</code> fully initialized. The output, and the optional input, must be valid                                                                                                                                                                                                                                                                                            |
| <i>Arguments</i>   | <p>instance - pointer to the <code>intel_vvp_switch_instance</code> software driver instance structure</p> <p>output - chosen output in the range <code>[0..num_outputs-1]</code> where <code>num_outputs</code> is the number of configured outputs</p> <p>enabled - true to enable the output</p> <p>input - if the output is enabled, this is the chosen input in the range <code>[0..num_inputs-1]</code> where <code>num_inputs</code> is the number of configured inputs</p> |
| <i>Returns</i>     | <p><code>kIntelVvpCoreOk</code> for success, negative error code in case of failures</p> <p><code>kIntelVvpCoreInstanceErr</code> if the instance is invalid</p> <p><code>kIntelVvpSwitchParameterErr</code> if the output and or the input is invalid. Note that selecting an invalid input disables the output</p>                                                                                                                                                               |

#### `intel_vvp_switch_get_output_config`

|                    |                                                                                                                                                                                                                                                                                                        |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Prototype</i>   | <pre>int intel_vvp_switch_get_output_config(intel_vvp_switch_instance *instance, uint8_t output)</pre>                                                                                                                                                                                                 |
| <i>Description</i> | Returns the current configuration for a requested output. The instance must be a valid <code>intel_vvp_switch_instance</code> fully initialized and with debug enabled. The output must be valid                                                                                                       |
| <i>Arguments</i>   | <p>instance - pointer to the <code>intel_vvp_switch_instance</code> software driver instance structure</p> <p>output - chosen output in the range <code>[0..num_outputs-1]</code> where <code>num_outputs</code> is the number of configured outputs</p>                                               |
| <i>Returns</i>     | <p>The selected input for the given output, -1 if the output is disabled</p> <p>Other negative values are for errors:</p> <p><code>kIntelVvpCoreInstanceErr</code> if the instance is invalid or not configured for debug</p> <p><code>kIntelVvpSwitchParameterErr</code> if the output is invalid</p> |

#### `intel_vvp_switch_commit_writes`

|                  |                                                                                    |
|------------------|------------------------------------------------------------------------------------|
| <i>Prototype</i> | <pre>int intel_vvp_switch_commit_writes(intel_vvp_switch_instance* instance)</pre> |
|------------------|------------------------------------------------------------------------------------|

- Description*    Commits all pending writes before processing the next field. The instance must be valid and parameterized in full mode.
- Arguments*    instance - pointer to the `intel_vvp_switch_instance` software driver instance structure
- Returns*        kIntelVvpCoreOk (0) for success, negative error code otherwise

#### Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

## 42. Text Box IP

### 42.1. About the Text Box IP

The IP produces an Altera Streaming Video compliant output stream, with each frame containing a rendered string of text. The IP enables you to adjust the text font size, colors, transparency, interlace, and text content at run time. The IP supports both the full and lite variants of the Altera streaming protocol. Use the IP in combination with the Mixer IP to overlay debugging information or other annotations over another video stream.

#### 42.1.1. Text Box IP Performance and Resources

Altera provides resource and usage data for guidance. The IP resource usage depends on the device family, number of bits per color sample, pixels in parallel, color space, and text capacity parameters.

**Table 773. Text Box IP Performance and Resources**

| Target Device                                  | ALMs | Memory (M20K) | f <sub>MAX</sub> MHz |
|------------------------------------------------|------|---------------|----------------------|
| Agilex 7 speed grade 2 (AGFA012R24A2E2V)       | 956  | 5             | 766                  |
| Agilex 5 speed grade 6 (A5ED065BB32AE6SR0)     | 942  | 5             | 411                  |
| Arria 10 speed grade 1 (10AS066H1F34E1HG)      | 842  | 5             | 644                  |
| Stratix 10 GX speed grade 1 (1SX280LN2F43E1VG) | 960  | 5             | 660                  |
| Cyclone 10 GX speed grade 5 (10CX220YF672E5G)  | 841  | 5             | 600                  |

### 42.2. Text Box IP Parameters

The IP offers run time and compile time parameters

**Table 774. Text Box IP Parameters**

| Parameter                      | Values    | Description                                                            |
|--------------------------------|-----------|------------------------------------------------------------------------|
| <b>Interface Configuration</b> |           |                                                                        |
| <b>Lite mode</b>               | On or Off | Turn on to use the Lite variant of the Altera Streaming Video Protocol |
| <b>Bits per color sample</b>   | 8 to 16   | Select the number of bits per color sample                             |
| <i>continued...</i>            |           |                                                                        |

| Parameter                                   | Values                                                 | Description                                                                                                                                                                                                                                                |
|---------------------------------------------|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Number of pixels in parallel</b>         | 1 to 8                                                 | Select the number of pixels transmitted per clock cycle                                                                                                                                                                                                    |
| <b>Color space</b>                          | RGB, YCbCr 4:4:4, YCbCr 4:2:2, YCbCr 4:2:0, Monochrome | Select the output color space.<br>RGB, YCbCr 4:4:4 and YCbCr 4:2:0 have 3 color channels.<br>YCbCr 4:2:2 has 2 color channels.<br>Monochrome has 1 color channel.                                                                                          |
| <b>Enable alpha channel</b>                 | On or Off                                              | Turn on to add an extra alpha color plane containing transparency information                                                                                                                                                                              |
| <b>General Settings</b>                     |                                                        |                                                                                                                                                                                                                                                            |
| <b>Memory mapped control interface</b>      | On or Off                                              | Turn on for the <b>Memory-mapped control interface</b> and allow runtime configuration of the IP via the register map.                                                                                                                                     |
| <b>Debug features</b>                       | On or Off                                              | Turn on for read back of writable registers via the <b>Memory-mapped control interface</b> .<br>If off, you cannot read back from any writable registers. Read-only registers are always readable.                                                         |
| <b>Separate clock for control interface</b> | On or Off                                              | Turn on for a separate clock for the <b>Memory mapped control interface</b> .                                                                                                                                                                              |
| <b>Pipeline ready signals</b>               | On or Off                                              | Enable to add extra pipeline registers to AXI4-S <i>ready</i> signals                                                                                                                                                                                      |
| <b>Enable interrupt</b>                     | On or Off                                              | (129)<br>Add an interrupt sender signal that fires when the IP accepts configuration for the next frame, indicating when it is safe to start writing new settings.                                                                                         |
| <b>Text Box Configuration</b>               |                                                        |                                                                                                                                                                                                                                                            |
| <b>Text double buffering</b>                | On or Off                                              | (129)<br>Turn on to add a second text buffer, allowing you to control both which buffer you write text to, and which buffer the IP uses.                                                                                                                   |
| <b>Number of characters per line</b>        | 1, 2, 4, 8, 16, 32, 64, 128, 256                       | The number of characters you can write per line of text. The field/frame width is derived from this parameter and the <b>Maximum font size</b> .                                                                                                           |
| <b>Number of lines</b>                      | 1, 2, 4, 8, 16, 32, 64, 128, 256                       | The number of lines of text. The IP derives the field or frame height from this parameter and the <b>Maximum font size</b> .                                                                                                                               |
| <b>Maximum font size</b>                    | 3, 7, 15, 31, 63, 127, 255                             | (129)<br>Set the maximum font size available to be set at runtime. This parameter with <b>Number of characters per line</b> and <b>Number of lines</b> determine the maximum field dimensions.<br>With a font size of 1, each character is 8x8 pixels      |
| <b>Static Text Configuration</b>            |                                                        |                                                                                                                                                                                                                                                            |
| <b>Text file name</b>                       | A valid filepath                                       | (130)<br>Select the text file that contains the static text to display.<br>The text is truncated horizontally if a line is longer than <b>Number of characters per line</b> , and truncated vertically if the file is longer than <b>Number of lines</b> . |
| continued...                                |                                                        |                                                                                                                                                                                                                                                            |

(129) Only if **Memory-mapped control interface** is **on**.

(130) Only if **Memory-mapped control interface** is **off**.



| Parameter                            | Values                             | Description                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------------------|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Fixed interlace nibble</b>        | 0 to 15                            | (130)<br>Select the required initial interlace nibble configuration, as described by the Altera Streaming Video Protocol.<br><ul style="list-style-type: none"> <li>Values 0-7 give progressive output</li> <li>Values 8-11 give interlaced output with F0 first after reset</li> <li>Values 12-15 give interlaced output with F1 first after reset.</li> </ul> If <b>Lite mode</b> is off, this value populates the initial image information packet. |
| <b>Fixed font size</b>               | 3 to 255                           | (130)<br>Set the font size for the static text.<br>The fixed field width is ( <b>Number of characters per line * Fixed font size * 8</b> )<br>The fixed field height is ( <b>Number of lines * Fixed font size * 8</b> )                                                                                                                                                                                                                               |
| <b>Fixed font color (R/Cr)</b>       | 0 to (2 bits per color sample) - 1 | (130)<br>Set the static text's font color R component if color space is RGB, or Cr if color space is YCbCr.                                                                                                                                                                                                                                                                                                                                            |
| <b>Fixed font color (G/Y)</b>        | 0 to (2 bits per color sample) - 1 | (130)<br>Set the static text's font color G component if color space is RGB, or Y if color space is YCbCr.                                                                                                                                                                                                                                                                                                                                             |
| <b>Fixed font color (B/Cb)</b>       | 0 to (2 bits per color sample) - 1 | (130)<br>Set the static text's font color B component if color space is RGB, or Cb if color space is YCbCr, or the monochrome component.                                                                                                                                                                                                                                                                                                               |
| <b>Fixed font alpha</b>              | 0 to (2 bits per color sample) - 1 | Only if <b>Memory-mapped control interface</b> is <b>off</b> and <b>Enable alpha channel</b> is <b>on</b> . Set the static text's font alpha value.                                                                                                                                                                                                                                                                                                    |
| <b>Fixed background color (R/Cr)</b> | 0 to (2 bits per color sample) - 1 | (130)<br>Set the static text's background color R component if color space is RGB, or Cr if color space is YCbCr.                                                                                                                                                                                                                                                                                                                                      |
| <b>Fixed background color (G/Y)</b>  | 0 to (2 bits per color sample) - 1 | (130)<br>Set the static text's background color G component if color space is RGB, or Y if color space is YCbCr.                                                                                                                                                                                                                                                                                                                                       |
| <b>Fixed background color (B/Cb)</b> | 0 to (2 bits per color sample) - 1 | (130)<br>Set the static text's background color B component if color space is RGB, or Cb if color space is YCbCr, or the monochrome component.                                                                                                                                                                                                                                                                                                         |
| <b>Fixed background alpha</b>        | 0 to (2 bits per color sample) - 1 | Only if <b>Memory-mapped control interface</b> is <b>off</b> , and <b>Enable alpha channel</b> is <b>on</b> .<br>Set the static text's background alpha value.                                                                                                                                                                                                                                                                                         |

Figure 115. Text Box IP Parameters

The screenshot shows the 'Text Box Intel FPGA IP' configuration window. The title bar includes 'Details' and 'intel\_vvp\_text\_box'. The window is divided into several sections:

- Important Usage Information:** A note stating that all resets must be at least 256 clock cycles in duration and that reset pulses must overlap by at least 16 clock cycles when multiple reset inputs are used.
- Interface Configuration:**
  - ☐ Lite mode
  - Bits per color sample: 8
  - Number of pixels in parallel: 1
  - Color space: RGB (dropdown)
  - ☐ Enable alpha channel
- General Settings:**
  - ☐ Memory mapped control interface
  - ☐ Debug features
  - ☐ Separate clock for control interface
  - ☐ Pipeline ready signals
  - ☐ Enable interrupt
- Text Box Configuration:**
  - ☒ Text double buffering
  - Number of characters per line: 16 (dropdown)
  - Number of lines: 1 (dropdown)
  - Max font size: 15 (dropdown)
- Static Text Configuration:**
  - Text file name: (text input)
  - Fixed interlace nibble: 0
  - Fixed font size: 3
  - Fixed Font R/Cr: 0
  - Fixed Font G/Y: 0
  - Fixed Font B/Cb: 0
  - Fixed Font alpha: 0
  - Fixed Background R/Cr: 0
  - Fixed Background G/Y: 0
  - Fixed Background B/Cb: 0
  - Fixed Background alpha: 0

### 42.3. Text Box IP Functional Description

The Text Box IP allows you to generate an output of a string of text. You can update the contents of the text box, if you turn on **Memory-mapped control interface**, otherwise it is a static text box with contents taken from the file provided in the **Text file name** parameter.

The number of characters per line and number of lines are fixed at compile-time. However if **Memory-mapped control interface** is on you can update the font size, font, and background colors and interlace configuration at run time. The field size changes as font size changes, and the field height halves if **Fixed interlace nibble** is 8 to 15..

Figure 116. Example Text Box

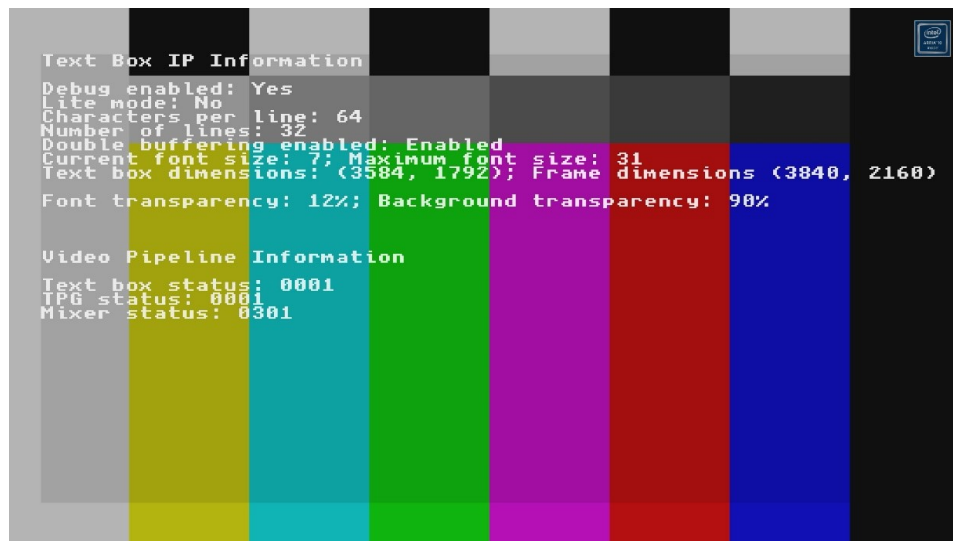


Table 775. Number of color planes for different parameters

| Color Space | Number of color planes | Number of color planes with Enable alpha channel |
|-------------|------------------------|--------------------------------------------------|
| RGB         | 3                      | 4                                                |
| YCbCr 4:4:4 | 3                      | 4                                                |
| YCbCr 4:2:2 | 2                      | 3                                                |
| YCbCr 4:2:0 | 3                      | 4                                                |
| Monochrome  | 1                      | 2                                                |

### Text Box Format

The number of characters per line is:  $n_{chars}$ , and number of lines named is  $n_{lines}$ . Character index is  $i$ .

The IP sequentially indexes each character starting from 0, with maximum index of:

$$(n_{chars} \times n_{lines}) - 1$$

that is:

$$0 \leq i \leq (n_{chars} * n_{lines}) - 1$$

The line or row number is:

$$i / n_{chars}$$

The column number is:

$$i \bmod n_{chars}$$

That is, the text wraps to the next line at index  $n_{chars}$ .

If **Memory-mapped control interface** is on, you can write a string by writing the character's ASCII code to the corresponding addresses. The control interface takes 4 bytes of data, where characters are ordered in little-endian (to write the string "0123" (ASCII values 0x30, 0x31, 0x32, 0x33) you write "0x33323130" to address 0x200). You may use the control interface's byte-enable signal to write to specific bytes only.

If **Memory-mapped control interface** is off, the IP shows static text with the contents of the file in the **Text file name** parameter. The file automatically is formatted into an initialization file and used as ROM.

### Field Dimensions

You select progressive or interlaced outputs with **Fixed interlace nibble** or in the `FIELD_INTERLACE` register.

The IP derives the field size from the font size, number of characters per line, and number of lines. Each character is 8x8 pixels with font size of 1. For progressive output:

- The width is  $(8 \times \text{font size} \times n\_chars)$
- The height is  $(8 \times \text{font size} \times n\_lines)$

For interlaced output, the IP halves the field height so that when deinterlacing F0 with F1 the output is identical to a progressive frame.

- The width is  $(8 \times \text{font size} \times n\_chars)$
- The height is  $(4 \times \text{font size} \times n\_lines)$

If **Memory-mapped control interface** is on, the field size updates whenever the font size or interlace settings change.

#### 42.3.1. Writing Text at Run Time with Double Buffering

If **Text double buffering** is on in the Text Box IP, you tell the IP which of two text buffers to query for the next field. The IP queries the buffer set by `RD_BANK`, while you write new data to the `WR_BANK` buffer.

1. Set `WR_BANK` to the opposite of `RD_BANK` (or 0 for first time), then write your string to the corresponding addresses.
  - Address 0x200 corresponds with character indices 0-3, 0x204 corresponds with character indices 4-7, etc, where in each word the characters are ordered in little-endian.
  - Although the word size of the control interface is 4 bytes wide (hence takes 4 characters), you may use the control interface's byte-enable signal to only write specific bytes.
  - Writes to `WR_BANK` and any text addresses do not need a further write to `COMMIT`.
2. When all writes are complete, set `RD_BANK` to the same as `WR_BANK`, and write any value to the `COMMIT` register. The IP uses the new bank at the next field boundary.
3. Wait for the interrupt that fires when the IP accepts the new `RD_BANK`.

- a. If not using the interrupt, poll for changes in the `FIELD_COUNT` register.
4. Repeat steps 1-3 iteratively, updating text as needed.

### 42.3.2. Writing Text at Run Time without Double Buffering

If **Text double buffering** is off, the Text Box IP uses only a single buffer. Both the control interface and the IP use the buffer simultaneously. To ensure that changes to the text are clean, you may either:

1. Make changes only to lines the IP is not using
  - a. Poll the `CURRENT_LINE` register to see which line of the string the IP is currently generating.  
The returned value may be ahead by 1. Avoid changes to both the returned line and line before.
  - b. Write to character indices that do not correspond to row number `CURRENT_LINE`; that is indices between  $(CURRENT\_LINE * NUM\_CHARS\_PER\_LINE)$  and  $((CURRENT\_LINE * (NUM\_CHARS\_PER\_LINE + 1)) - 1)$ .
2. Stop the IP whenever changes are to be made (this stops video output)
  - a. Write 0 to the `CONTROL` register, to stop the IP after the current field.
  - b. Poll bit 0 of the `STATUS` register, to wait for the IP to stop.
  - c. Make all edits to the character buffer as necessary, by writing to addresses 0x200 onwards.
  - d. Write 1 to the `CONTROL` register, to start the IP again.

### 42.3.3. Text Box IP Interfaces

**Table 776. Text Box IP Interfaces**

| Name                                        | Direction | Width            | Description                            |
|---------------------------------------------|-----------|------------------|----------------------------------------|
| <b>Clocks and Resets</b>                    |           |                  |                                        |
| <code>main_clock_clk</code>                 | In        | 1                | AXI4-S video processing clock          |
| <code>main_reset_rst</code>                 | In        | 1                | AXI4-S video processing reset          |
| <code>agent_clock_clk</code>                | In        | 1                | Control agent interface clock          |
| <code>agent_reset_rst</code>                | In        | 1                | Control agent interface reset          |
| <b>Control interfaces</b>                   |           |                  |                                        |
| <code>av_mm_control_agent_address</code>    | In        | <sup>(131)</sup> | Avalon memory-mapped agent address     |
| <code>av_mm_control_agent_write</code>      | In        | 1                | Avalon memory-mapped agent write       |
| <code>av_mm_control_agent_writedata</code>  | In        | 32               | Avalon memory-mapped agent write data  |
| <code>av_mm_control_agent_byteenable</code> | In        | 4                | Avalon memory-mapped agent byte enable |
| <code>av_mm_control_agent_read</code>       | In        | 1                | Avalon memory-mapped agent read        |
| <i>continued...</i>                         |           |                  |                                        |

<sup>(131)</sup>  $\log_2(128 + ((\text{number of characters per line} * \text{number of lines})/4))$

| Name                                     | Direction | Width | Description                                |
|------------------------------------------|-----------|-------|--------------------------------------------|
| av_mm_control_agent_readdata             | Out       | 32    | Avalon memory-mapped agent read data       |
| av_mm_control_agent_readdatavalid        | Out       | 1     | Avalon memory-mapped agent read data valid |
| av_mm_control_agent_waitrequest          | Out       | 1     | Avalon memory-mapped agent wait request    |
| irq                                      | Out       | 1     | Interrupt sender                           |
| <b>Altera Streaming Video interfaces</b> |           |       |                                            |
| axi4s_vid_out_tdata                      | Out       | (132) | AXI4-S data out                            |
| axi4s_vid_out_tvalid                     | Out       | 1     | AXI4-S data valid                          |
| axi4s_vid_out_tuser                      | Out       | (133) | AXI4-S user                                |
| axi4s_vid_out_tlast                      | Out       | 1     | AXI4-S end of packet                       |
| axi4s_vid_out_tready                     | In        | 1     | AXI4-S data ready                          |

## 42.4. Text Box IP Registers

Each register is either read-only (RO) or read-write (RW).

**Table 777. Text Box IP Registers**

| Address      | Register           | Access | Description                                                                                                                                                                           |
|--------------|--------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x0000       | VID_PID            | RO     | Read this register to retrieve the Text box product ID. This register always returns 0x6AF7_017E.                                                                                     |
| 0x0004       | VERSION            | RO     | Read this register for the IP version information.                                                                                                                                    |
| 0x0008       | LITE_MODE          | RO     | Read this register to determine if <b>Lite mode</b> is on or off. This register returns 0 when you turn off <b>Lite mode</b> and 1 when you turn on <b>Lite mode</b> .                |
| 0x000C       | DEBUG_ENABLED      | RO     | Read this register to determine if <b>Debug features</b> is on                                                                                                                        |
| 0x0010       | IRQ_ENABLED        | RO     | Read this register to determine if <b>Enable interrupts</b> is on                                                                                                                     |
| 0x0014       | BPS                | RO     | Read this register to determine the number of bits per color plane                                                                                                                    |
| 0x0018       | PIXELS_IN_PARALLEL | RO     | Read this register to determine the number of pixels transmitted per clock cycle at the streaming output                                                                              |
| 0x001C       | COLOR_SPACE        | RO     | Read this register to determine the color space and chroma sampling.<br>This register returns 0 for RGB, 1 for YCbCr 4:4:4, 2 for YCbCr 4:2:2, 3 for YCbCr 4:2:0 and 4 for monochrome |
| 0x0020       | ALPHA_ENABLED      | RO     | Read this register to determine if <b>Enable alpha channel</b> is on.                                                                                                                 |
| 0x0024       | NUM_CHARS_PER_LINE | RO     | Read this register to return the <b>Number of characters per line</b> .                                                                                                               |
| continued... |                    |        |                                                                                                                                                                                       |

(132)  $\max(2, \text{ceil}((\text{bits per color sample} \times \text{number of color planes})/8)) \times \text{pixels in parallel}$

(133)  $\text{ceil}(\text{tdata width} / 8)$

| Address                 | Register        | Access | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------------------------|-----------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x0028                  | NUM_LINES       | RO     | Read this register to return the <b>Number of lines</b> .                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 0x002C                  | DOUBLE_BUFFERED | RO     | Read this register to determine if <b>Text Double Buffering</b> is on                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 0x0030                  | MAX_FONT_SIZE   | RO     | Read this register to return the <b>Maximum font size</b> .                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 0x0044 to 0x013C unused |                 |        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 0x0100                  | IRQ_CONTROL     | RW     | IRQ control. Set this register to configure interrupts. <b>Bit 0</b> : Set to 1 to enable the interrupt in IRQ_STATUS, 0 otherwise.                                                                                                                                                                                                                                                                                                                                                    |
| 0x0104                  | IRQ_STATUS      | RW     | Read this status register to see if an interrupt fired.                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 0x0108 to 0x00FC unused |                 |        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 0x0140                  | STATUS          | RO     | The IP lowers this bit at the next field boundary (or if you stop the IP) after a write to the COMMIT register.                                                                                                                                                                                                                                                                                                                                                                        |
| 0x0144                  | FIELD_COUNT     | RO     | Read this register to determine the output field count. The value in this register is reset to 0 each time you stop the IP.                                                                                                                                                                                                                                                                                                                                                            |
| 0x0148                  | CURRENT_LINE    | RO     | Read this register to determine which line of text the IP is currently generating.                                                                                                                                                                                                                                                                                                                                                                                                     |
| 0x014C                  | CONTROL         | RW     | Control register. Write 1 to start the IP, 0 to stop after the current field..                                                                                                                                                                                                                                                                                                                                                                                                         |
| 0x0150                  | COMMIT          | RW     | The IP holds pending any changes to registers (except where specified) until you write to this register. The value you write is unimportant.                                                                                                                                                                                                                                                                                                                                           |
| 0x0154                  | WR_BANK         | WR     | Only if <b>text double buffering</b> is on. Write to this register to set which buffer any writes to the character buffer go. Writing to this register apply immediately, without needing to write to COMMIT.                                                                                                                                                                                                                                                                          |
| 0x0158                  | RD_BANK         | RW     | Only if <b>text double buffering</b> is on. Write to set which buffer the IP retrieves characters from.                                                                                                                                                                                                                                                                                                                                                                                |
| 0x015C                  | FIELD_INTERLACE | RW     | Use this register to set the interlace output behavior for outgoing fields. The value you write corresponds to the interlace identifier nibble of outgoing image information packets in full mode.<br>The IP toggles the f0 and f1 bit when running..<br>The changes apply at the next field boundary.<br>Values 0-7 produce progressive outputs.<br>Values 8-11 cause the field sequence to restart with F0 first.<br>Values 12-15 cause the field sequence to restart with F1 first. |
| 0x0160                  | FONT_SIZE       | RW     | Write to set the font size of the text.<br>This value changes the field dimensions.                                                                                                                                                                                                                                                                                                                                                                                                    |
| 0x0164                  | FONT_R_CR       | RW     | Write to set the font color R component if color space is RGB, or Cr component for YCbCr.                                                                                                                                                                                                                                                                                                                                                                                              |
| 0x0168                  | FONT_G_Y        | RW     | Write to set the font color G component if color space is RGB, or Y component for YCbCr.                                                                                                                                                                                                                                                                                                                                                                                               |
| 0x016C                  | FONT_B_CB       | RW     | Write to set the font color B component if color space is RGB, or Cb component for YCbCr, or the monochrome component.                                                                                                                                                                                                                                                                                                                                                                 |
| 0x0170                  | FONT_ALPHA      | RW     | Only if <b>enable alpha channel</b> is on. Write to set the font alpha (transparency) value.                                                                                                                                                                                                                                                                                                                                                                                           |
| 0x0174                  | BACKGROUND_R_CR | RW     | Write to set the background color R component if color space is RGB, or Cr component for YCbCr.                                                                                                                                                                                                                                                                                                                                                                                        |
| continued...            |                 |        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

| Address                                                  | Register                 | Access | Description                                                                                                                                                            |
|----------------------------------------------------------|--------------------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x0178                                                   | BACKGROUND_G_Y           | RW     | Write to set the background color G component if color space is RGB, or Y component for YCbCr.                                                                         |
| 0x017C                                                   | BACKGROUND_B_CB          | RW     | Write to set the background color B component if color space is RGB, or Cb component for YCbCr, or the monochrome component.                                           |
| 0x0180                                                   | BACKGROUND_ALPHA         | RW     | Only if <b>enable alpha channel</b> is on. Write to set the background alpha (transparency) value.                                                                     |
| 0x0178 to 0x1FC unused                                   |                          |        |                                                                                                                                                                        |
| 0x0200 + (i - 1)*4<br>to<br>0x0200 + (num_chars - 1) * 4 | CHARACTER_INDEX[i : i+3] | WO     | Character buffer entries. Each word corresponds to 4 extended ASCII characters. You may write to individual bytes via the byte-enable signal of the control interface. |

### Register Bit Descriptions

**Table 778. VID\_PID**

| Name                            | Bits | Description                              |
|---------------------------------|------|------------------------------------------|
| Text box version and product ID | 31:0 | This register always returns 0x6AF7_017E |

**Table 779. VERSION**

| Name                 | Bits  | Description                                  |
|----------------------|-------|----------------------------------------------|
| Register map version | 7:0   | Register map version. Returns 1              |
| QPDS patch revision  | 15:8  | Returns 0x00                                 |
| QPDS update revision | 23:16 | Updated each release. For 24.3, returns 0x03 |
| QPDS major revision  | 31:24 | Updated each release. For 24.3, returns 0x18 |

**Table 780. LITE\_MODE**

| Name                           | Bits | Description                          |
|--------------------------------|------|--------------------------------------|
| Lite mode parameterization bit | 0    | Returns 1 if <b>Lite mode</b> is on. |
| Unused                         | 31:1 | Unused                               |

**Table 781. DEBUG\_ENABLED**

| Name                                | Bits | Description                              |
|-------------------------------------|------|------------------------------------------|
| Debug features parameterization bit | 0    | Returns 1 if <b>Debug features</b> is on |
| Unused                              | 31:1 | Unused                                   |

**Table 782. IRQ\_ENABLED**

| Name                                   | Bits | Description                                 |
|----------------------------------------|------|---------------------------------------------|
| Interrupt enabled parameterization bit | 0    | Returns 1 if <b>Enable interrupt</b> is on. |
| Unused                                 | 31:1 | Unused                                      |



**Table 783. BPS**

| Name                  | Bits | Description                                    |
|-----------------------|------|------------------------------------------------|
| Bits per color sample | 31:0 | Returns the <b>Bits per color sample</b> value |

**Table 784. PIP**

| Name               | Bits | Description                                 |
|--------------------|------|---------------------------------------------|
| Pixels in parallel | 31:0 | Returns the <b>Pixels in parallel</b> value |

**Table 785. COLOR\_SPACE**

| Name        | Bits | Description                                                                                                                     |
|-------------|------|---------------------------------------------------------------------------------------------------------------------------------|
| Color space | 31:0 | Returns the <b>Color space</b> value with. 0 for RGB, 1 for YCbCr 4:4:4, 2 for YCbCr 4:2:2, 3 for YCbCr 4:2:0, 4 for Monochrome |

**Table 786. ALPHA\_ENABLED**

| Name                  | Bits | Description                                     |
|-----------------------|------|-------------------------------------------------|
| Alpha channel enabled | 0    | Returns 1 if <b>Enable alpha channel</b> is on. |
| Unused                | 31:1 | Unused                                          |

**Table 787. NUM\_CHARS\_PER\_LINE**

| Name                          | Bits | Description                                               |
|-------------------------------|------|-----------------------------------------------------------|
| Number of characters per line | 31:0 | Returns the <b>Number of characters per line</b> of text. |

**Table 788. NUM\_LINES**

| Name            | Bits | Description                                 |
|-----------------|------|---------------------------------------------|
| Number of lines | 31:0 | Returns the <b>Number of lines</b> of text. |

**Table 789. DOUBLE\_BUFFERED**

| Name                     | Bits | Description                                 |
|--------------------------|------|---------------------------------------------|
| Double buffering enabled | 0    | Returns 1 if <b>Double buffering</b> is on. |
| Unused                   | 31:1 | Unused                                      |

**Table 790. MAX\_FONT\_SIZE**

| Name              | Bits | Description                            |
|-------------------|------|----------------------------------------|
| Maximum font size | 31:0 | Returns the <b>Maximum font size</b> . |

**Table 791. IRQ\_CONTROL**

| Name             | Bits | Description                                                                                                  |
|------------------|------|--------------------------------------------------------------------------------------------------------------|
| Interrupt enable | 0    | Write 1 to enable the frame boundary interrupt. Set to 1 to enable the interrupt in IRQ_STATUS, 0 otherwise. |
| Unused           | 31:1 | Unused                                                                                                       |

**Table 792. IRQ\_STATUS**

| Name             | Bits | Description                                                                                                                                                                                              |
|------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interrupt enable | 0    | Field done. Asserts 1 to indicate when the IP accepts the RD_BANK for the next field. It is then safe to write new data to the opposing bank (buffer). Write 1 to this bit to acknowledge the interrupt. |
| Unused           | 31:1 | Unused                                                                                                                                                                                                   |

**Table 793. STATUS**

| Name                        | Bits | Description                                                                                                                                                                                          |
|-----------------------------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Running bit                 | 0    | Returns 1 if the IP is currently generating a field, 0 otherwise.                                                                                                                                    |
| Pending register update bit | 1    | Returns 1 if any outstanding register writes yet to be committed by a write to COMMIT exist. Any writes to registers (excluding WR_BANK, CONTROL, and addresses corresponding to character indices). |
| Unused                      | 31:2 | Unused                                                                                                                                                                                               |

**Table 794. FIELD\_COUNT**

| Name        | Bits  | Description                                                          |
|-------------|-------|----------------------------------------------------------------------|
| Running bit | 15:0  | Returns the number of fields generated since the IP was last started |
| Unused      | 31:16 | Unused                                                               |

**Table 795. CURRENT\_LINE**

| Name         | Bits  | Description                                                                  |
|--------------|-------|------------------------------------------------------------------------------|
| Current line | 15:0  | Returns the text row number that the IP is currently using to generate text. |
| Unused       | 31:16 | Unused                                                                       |

**Table 796. CONTROL**

| Name         | Bits | Description                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Current line | 0    | Write 1 to start the core, 0 to stop after the current field.<br>Write 0 to this register to stop the IP at the end of the current field. Write 1 to this register to start or restart output generation.<br>This register initializes to 0 at start-up and no output fields are generated until after a write of 1 to this register.<br>Writes to this register apply immediately, without needing to further write to COMMIT. |
| Unused       | 31:1 | Unused                                                                                                                                                                                                                                                                                                                                                                                                                          |

**Table 797. COMMIT**

| Name   | Bits | Description                                   |
|--------|------|-----------------------------------------------|
| Unused | 31:0 | Write to this register after updating RD_BANK |

**Table 798. WR\_BANK**

| Name              | Bits | Description                                                      |
|-------------------|------|------------------------------------------------------------------|
| Write bank number | 0    | Write 0 for characters writes to go to buffer 0, 1 for buffer 1. |
| Unused            | 31:1 | Unused                                                           |

**Table 799. RD\_BANK**

| Name             | Bits | Description                                                    |
|------------------|------|----------------------------------------------------------------|
| Read bank number | 0    | Write 0 for the IP to read text from buffer 0, 1 for buffer 1. |
| Unused           | 31:1 | Unused                                                         |

**Table 800. FIELD\_INTERLACE**

| Name             | Bits | Description                                                                                                                                                     |
|------------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interlace nibble | 3:0  | The interlace configuration of outgoing video fields. The nibble follows the standard from the image information packet of the Altera Video streaming protocol. |
| Unused           | 31:4 | Unused                                                                                                                                                          |

**Table 801. FONT\_SIZE**

| Name           | Bits                                         | Description                                                                                 |
|----------------|----------------------------------------------|---------------------------------------------------------------------------------------------|
| Text font size | $\text{clog2}(\text{MAX\_FONT\_SIZE}) - 1:0$ | The font size the IP uses to render the text. At font size 1, each character is 8x8 pixels. |
| Unused         | 31: $\text{clog2}(\text{MAX\_FONT\_SIZE})$   | Unused                                                                                      |

**Table 802. FONT/BACKGROUND\_X**

| Name              | Bits      | Description                      |
|-------------------|-----------|----------------------------------|
| Color component X | BPS – 1:0 | The value for color component X. |
| Unused            | 31:BPS    | Unused                           |

**Table 803. CHARACTER\_INDEX[i:i+3]**

| Name        | Bits  | Description              |
|-------------|-------|--------------------------|
| String[i]   | 7:0   | Character at index i     |
| String[i+1] | 15:8  | Character at index (i+1) |
| String[i+2] | 23:16 | Character at index (i+2) |
| String[i+3] | 31:24 | Character at index (i+3) |

## 42.5. Text Box IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios V HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure, so all common methods defined in Video and Vision Processing IPs Software API are applicable.

Register definition file: `intel_vvp_text_box_regs.h`

Include file: `intel_vvp_text_box.h`

**Table 804. Text Box IP Software API**

| Name                                          | Description                                                                                                                  |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| intel_vvp_text_box_init                       | Initialize the Text Box instance                                                                                             |
| intel_vvp_core_*                              | Accessors defined in Video and Vision Processing IP Software Programming Model. Image info accessors are unused for this IP. |
| intel_vvp_text_box_get_lite_mode              | Returns whether <b>Lite mode</b> is on                                                                                       |
| intel_vvp_text_box_get_debug_enabled          | Returns whether <b>Debug enabled</b> is on                                                                                   |
| intel_vvp_text_box_get_bits_per_sample        | Returns the <b>Bits per color sample</b>                                                                                     |
| intel_vvp_text_box_get_pixels_in_parallel     | Returns the <b>pixels in parallel</b>                                                                                        |
| intel_vvp_text_box_get_color_space            | Returns the <b>color space</b> of outgoing fields set at compile time                                                        |
| intel_vvp_text_box_get_alpha_enabled          | Returns if the IP has an <b>extra alpha channel</b>                                                                          |
| intel_vvp_text_box_get_num_chars_per_line     | Returns the <b>number of characters per line</b> of text                                                                     |
| intel_vvp_text_box_get_num_lines              | Returns the <b>number of lines</b> of text                                                                                   |
| intel_vvp_text_box_get_double_buffered        | Returns if the IP has <b>Double buffering</b>                                                                                |
| intel_vvp_text_box_get_max_font_size          | Returns the maximum supported <b>font size</b>                                                                               |
| intel_vvp_text_box_get_status                 | Reads and returns the contents of the status register                                                                        |
| intel_vvp_text_box_is_running                 | Returns if the IP is currently generating video fields                                                                       |
| intel_vvp_text_box_get_commit_status          | Returns if any register writes are yet to be committed to the IP, excluding explicitly stated registers                      |
| intel_vvp_text_box_get_field_since_last_start | Returns the number of fields the IP generated since it last started                                                          |
| intel_vvp_text_box_get_current_line           | Returns which line of text the IP is currently generating                                                                    |
| intel_vvp_text_box_enable                     | Start or stop the IP                                                                                                         |
| intel_vvp_text_box_start                      | Start the IP                                                                                                                 |
| intel_vvp_text_box_stop                       | Stop the IP after the current field                                                                                          |
| intel_vvp_text_box_is_enabled                 | Returns if the IP is enabled (reads the <b>CONTROL</b> register)                                                             |
| intel_vvp_text_box_set_write_bank             | Set which buffer the IP uses for writes to text registers                                                                    |
| intel_vvp_text_box_get_write_bank             | Returns which buffer the IP uses for writes to text registers                                                                |
| intel_vvp_text_box_set_read_bank              | Set which buffer the IP retrieves text from                                                                                  |
| intel_vvp_text_box_get_read_bank              | Returns which buffer the IP uses to retrieve text                                                                            |
| intel_vvp_text_box_set_field_interlace        | Set the initial interlace nibble/configuration of outgoing fields.                                                           |
| intel_vvp_text_box_get_field_interlace        | Returns the interlace nibble of the current field                                                                            |
| intel_vvp_text_box_set_font_size              | Set the font size for rendered text                                                                                          |
| intel_vvp_text_box_get_font_size              | Returns the current font size                                                                                                |
| intel_vvp_text_box_set_font_color             | Set the text's font color                                                                                                    |
| continued...                                  |                                                                                                                              |

| Name                                    | Description                                                             |
|-----------------------------------------|-------------------------------------------------------------------------|
| intel_vvp_text_box_get_font_color       | Return the text's current font color                                    |
| intel_vvp_text_box_set_background_color | Set the text's background color                                         |
| intel_vvp_text_box_get_background_color | Return the text's current background color                              |
| intel_vvp_text_box_write_word           | Write a word (uint32_t) to a text register                              |
| intel_vvp_text_box_write_string         | Helper method to write a string to the corresponding registers          |
| intel_vvp_text_box_get_field_width      | Returns the current field width                                         |
| intel_vvp_text_box_get_field_height     | Return the current field height                                         |
| intel_vvp_text_box_commit_writes        | Commit any outstanding writes from set methods (except where specified) |

### enum eIntelVvpTextBoxColorSpace

**Members**

- kIntelVvpTextBoxRgb - RGB
- kIntelVvpTextBoxYcc444 - YCbCr 4:4:4
- kIntelVvpTextBoxYcc422 - YCbCr 4:2:2
- kIntelVvpTextBoxYcc420 - YCbCr 4:2:0
- kIntelVvpTextBoxMono - Monochrome
- kIntelVvpTextBoxInvalidColorSpace - Invalid color space (used for errors)

**Description** Enumeration for the supported color spaces and subsampling

### intel\_vvp\_text\_box\_init

#### Prototype

```
int intel_vvp_text_box(intel_vvp_text_box_instance* instance,
intel_vvp_core_base base);
```

**Description** Initializes a Text Box instance. If the return code is non-zero, the structure was not successfully initialized and cannot be used further.

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure  
software driver instance structure

**Returns** kIntelVvpCoreOk in case of success  
kIntelVvpCoreInstanceErr if instance is a null pointer  
kIntelVvpCoreVidErr if the vendor id of the IP is not the IntelFPGA vendor ID (0x6AF7)

kIntelVvpCorePidErr if the product id of the IP is not the Text Box product ID (0x017E)

kIntelVvpTextBoxRegMapVersionErr if the register map version is not supported

### intel\_vvp\_text\_box\_get\_lite\_mode

#### Prototype

```
bool
intel_vvp_text_box_get_lite_mode(intel_vvp_text_box_instance*
instance);
```

**Description** Returns whether the IP is in Lite mode

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** False if not lite mode or if instance is invalid, true otherwise

### intel\_vvp\_text\_box\_get\_debug\_enabled

#### Prototype

```
bool
intel_vvp_text_box_get_debug_enabled(intel_vvp_text_box_instance*
instance);
```

**Description** Returns whether Debug enabled is on

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** False if debug is not available or if instance is invalid, true otherwise

### intel\_vvp\_text\_box\_get\_bits\_per\_sample

#### Prototype

```
uint8_t
intel_vvp_text_box_get_bits_per_sample(intel_vvp_text_box_instance*
instance);
```

**Description** Returns the Bits per color sample.

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** Returns the Bits per color sample

### intel\_vvp\_text\_box\_get\_pixels\_in\_parallel

#### Prototype

```
uint8_t
intel_vvp_text_box_get_pixels_in_parallel(intel_vvp_text_box_instance*
instance);
```

**Description** Returns the pixels in parallel

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** The number of pixels processed in parallel (1-8), or 0 in case of error

### intel\_vvp\_text\_box\_get\_color\_space

#### Prototype

```
eIntelVvpTextBoxColorSpace
intel_vvp_text_box_get_color_space(intel_vvp_text_box_instance*
instance);
```

**Description** Returns the color space of outgoing fields set at compile time if **Enable alpha channel** is on

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** The color space the IP was compiled in, or kIntelVvpTextBoxInvalidColorSpace in case of error

### intel\_vvp\_text\_box\_get\_alpha\_enabled

#### Prototype

```
bool
intel_vvp_text_box_get_alpha_enabled(intel_vvp_text_box_instance*
instance);
```

**Description** Returns the color space of outgoing fields set at compile time

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** True if alpha is enabled, false otherwise or in case of error

### intel\_vvp\_text\_box\_get\_num\_chars\_per\_line

#### Prototype

```
uint16_t
intel_vvp_text_box_get_num_chars_per_line(intel_vvp_text_box_instance*
instance);
```

**Description** Returns the number of characters per line of text

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** The number of characters per lines, or 0 in case of error

#### intel\_vvp\_text\_box\_get\_num\_lines

**Prototype**

```
uint16_t
intel_vvp_text_box_get_num_lines(intel_vvp_text_box_instance*
instance);
```

**Description** Returns the number of lines of text

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** The number of lines, or 0 in case of error

#### intel\_vvp\_text\_box\_get\_double\_buffered

**Prototype**

```
bool
intel_vvp_text_box_get_double_buffered(intel_vvp_text_box_instance*
instance);
```

**Description** Returns if double buffering is on

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** True if double buffering is on, false otherwise or in case of error

#### intel\_vvp\_text\_box\_get\_max\_font\_size

**Prototype**

```
uint16_t
intel_vvp_text_box_get_max_font_size(intel_vvp_text_box_instance*
instance);
```

**Description** Returns the maximum supported font size.

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** The maximum font size, or 0 in case of error



### intel\_vvp\_text\_box\_get\_status

#### Prototype

```
uint32_t
intel_vvp_text_box_get_status(intel_vvp_text_box_instance*
instance);
```

**Description** Reads and returns the contents of the status register

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** Contents of the status register, or 0xFFFFFFFF in case of error

### intel\_vvp\_text\_box\_is\_running

#### Prototype

```
bool
intel_vvp_text_box_is_running(intel_vvp_text_box_instance*
instance);
```

**Description** Returns if the IP is generating video fields

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** True if currently generating video, false if stopped or in case of error

### intel\_vvp\_text\_box\_get\_commit\_status

#### Prototype

```
bool
intel_vvp_text_box_get_commit_status(intel_vvp_text_box_instance*
instance);
```

**Description** Returns if there are register writes yet to be committed to the IP

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** True if there are outstanding writes, false otherwise or in case of error

### intel\_vvp\_text\_box\_get\_fields\_since\_last\_start

#### Prototype

```
uint32_t
intel_vvp_text_box_get_fields_since_last_start(intel_vvp_text_box_instance*
instance);
```

**Description** Returns the number of fields the IP has generated since it was last started

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** The number of fields generated, or 0xFFFFFFFF in case of error

### intel\_vvp\_text\_box\_get\_current\_line

**Prototype**

```
uint16_t
intel_vvp_text_box_get_current_line(intel_vvp_text_box_instance*
instance);
```

**Description** Returns which line of text the IP is currently generating

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** The current line number, or 0xFFFF in case of error

### intel\_vvp\_text\_box\_enable

**Prototype**

```
int intel_vvp_text_box_enable(intel_vvp_text_box_instance*
instance, bool enabled);
```

**Description** Start the IP

**Arguments** instance - Pointer to the intel\_vvp\_text\_box\_instance structure  
enabled - True to start the IP, false to stop

**Returns** kIntelVvpCoreOk in case of success  
kIntelVvpCoreInstanceErr if instance is NULL

### intel\_vvp\_text\_box\_start

**Prototype**

```
int intel_vvp_text_box_start(intel_vvp_text_box_instance*
instance);
```

**Description** Start the IP

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** kIntelVvpCoreOk in case of success  
kIntelVvpCoreInstanceErr if instance is NULL

### intel\_vvp\_text\_box\_stop

*Prototype*

```
int intel_vvp_text_box_stop(intel_vvp_text_box_instance* instance);
```

*Description* Stop the IP after generating the current field

*Arguments* instance - pointer to the intel\_vvp\_text\_box\_instance structure

*Returns* kIntelVvpCoreOk in case of success  
kIntelVvpCoreInstanceErr if instance is NULL

### intel\_vvp\_text\_box\_is\_enabled

*Prototype*

```
bool
intel_vvp_text_box_is_enabled(intel_vvp_text_box_instance* instance);
```

*Description* Returns if the IP is enabled (reads the CONTROL register)

*Arguments* instance - pointer to the intel\_vvp\_text\_box\_instance structure

*Returns* True if enabled, false disabled or if instance is NULL or debug features is off

### intel\_vvp\_text\_box\_set\_write\_bank

*Prototype*

```
int
intel_vvp_text_box_set_write_bank(intel_vvp_text_box_instance* instance, bool bank);
```

*Description* Set which buffer writes to text go to

*Arguments* instance - pointer to the intel\_vvp\_text\_box\_instance structure  
bank - False to write to the buffer 0, true to write to buffer 1

*Returns* kIntelVvpCoreOk in case of success  
kIntelVvpCoreInstanceErr if instance is NULL  
kIntelVvpCoreTextBoxBankErr if the double buffering is off

### intel\_vvp\_text\_box\_get\_write\_bank

**Prototype**

```
bool
intel_vvp_text_box_get_write_bank(intel_vvp_text_box_instance*
instance);
```

**Description** Return which buffer the IP is querying from

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** False for the first buffer or if error, true for the second buffer

### intel\_vvp\_text\_box\_set\_read\_bank

**Prototype**

```
int
intel_vvp_text_box_set_read_bank(intel_vvp_text_box_instance*
instance, bool bank);
```

**Description** Set which buffer the IP queries text from

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

bank - False to write to the first buffer, true to write to the second buffer

**Returns** kIntelVvpCoreOk in case of success

kIntelVvpCoreInstanceErr if instance is NULL

kIntelVvpCoreTextBoxBankErr if double buffering is off

### intel\_vvp\_text\_box\_get\_read\_bank

**Prototype**

```
bool
intel_vvp_text_box_get_read_bank(intel_vvp_text_box_instance*
instance);
```

**Description** Return which buffer the IP is querying from

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** False for the first buffer or if error, true for the second buffer

### intel\_vvp\_text\_box\_set\_field\_interlace

**Prototype**

```
uint8_t
intel_vvp_text_box_set_field_interlace(intel_vvp_text_box_instance*
instance);
```

**Description** Set the initial interlace configuration nibble of outgoing fields. This is applied at the field boundary, such that it is applied to the next field.

This value is used to initially populate the Image Information Packet if the IP was compiled in Full mode.

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

interlace - The interlace configuration nibble

**Returns** kIntelVvpCoreOk in case of success

kIntelVvpCoreInstanceErr if instance is NULL

#### intel\_vvp\_text\_box\_get\_field\_interlace

**Prototype**

```
uint8_t
intel_vvp_text_box_get_field_interlace(intel_vvp_text_box_instance*
instance);
```

**Description** Return the interlace configuration nibble of the current field

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** The current field's interlace configuration nibble, or 0xFF in case of error

#### intel\_vvp\_text\_box\_set\_font\_size

**Prototype**

```
int
intel_vvp_text_box_set_font_size(intel_vvp_text_box_instance*
instance, uint8_t font_size);
```

**Description** Set the font size to render text at. A change in font size will change the field dimensions.

If font size is 1, then each character is 8x8 pixels

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

font\_size - The required font size

**Returns** kIntelVvpCoreOk in case of success

kIntelVvpCoreInstanceErr if instance is NULL

kIntelVvpCoreTextBoxFontSizeErr if font size is greater than the compiled maximum font size

### intel\_vvp\_text\_box\_get\_font\_size

#### Prototype

```
uint8_t
intel_vvp_text_box_get_font_size(intel_vvp_text_box_instance*
instance);
```

**Description** Returns the current font size

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** The current font size, or 0 in case of error

### intel\_vvp\_text\_box\_set\_font\_color

#### Prototype

```
int
intel_vvp_text_box_set_font_color(intel_vvp_text_box_instance*
instance, uint16_t r_cr, uint16_t g_y, uint16_t b_cb, uint16_t
alpha);
```

**Description** Set the text's font color

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

r\_cr - the red component in RGB, or Cr in YCbCr

g\_y - the green component in RGB, or Y in YCbCr

b\_cb - the blue component in RGB, or Cb in YCbCr, or the monochrome component

alpha - the alpha component; this is ignored if alpha is not enabled

**Returns** kIntelVvpCoreOk in case of success

kIntelVvpCoreInstanceErr if instance is NULL

kIntelVvpCoreTextBoxColorErr if any color component is greater than the maximum allowed by the bits per color sample

### intel\_vvp\_text\_box\_get\_font\_color

#### Prototype

```
int
intel_vvp_text_box_get_font_color(intel_vvp_text_box_instance*
instance, uint16_t *r_cr, uint16_t *g_y, uint16_t *b_cb,
uint16_t *alpha);
```

**Description** Set the text's font color

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

r\_cr - pointer to store the red component in RGB, or Cr in YCbCr

g\_y - pointer to store the red component in RGB, or Y in YCbCr

b\_cb - pointer to store the blue component in RGB, Cb in YCbCr, or the monochrome component

alpha - pointer to store the alpha component. Set to 0 if **Enable alpha** is off

*Returns* kIntelVvpCoreOk in case of success

kIntelVvpCoreInstanceErr if instance is NULL or if **Debug features** is off

#### intel\_vvp\_text\_box\_set\_background\_color

##### Prototype

```
int
intel_vvp_text_box_set_background_color(intel_vvp_text_box_instance*
instance, uint16_t r_cr, uint16_t g_y, uint16_t b_cb, uint16_t
alpha);
```

*Description* Set the text's background color

*Arguments* instance - pointer to the intel\_vvp\_text\_box\_instance structure

r\_cr - the red component in RGB, or Cr in YCbCr

g\_y - the green component in RGB, or Y in YCbCr

b\_cb - the blue component in RGB, or Cb in YCbCr, or the monochrome component

alpha - the alpha component; this is ignored if alpha is not enabled

*Returns* kIntelVvpCoreOk in case of success

kIntelVvpCoreInstanceErr if instance is NULL

kIntelVvpCoreTextBoxColorErr if any color component is greater than the maximum allowed by the bits per color sample

#### intel\_vvp\_text\_box\_get\_background\_color

##### Prototype

```
int
intel_vvp_text_box_get_background_color(intel_vvp_text_box_instance*
instance, uint16_t *r_cr, uint16_t *g_y, uint16_t *b_cb, uint16_t
*alpha);
```

*Description* Set the text's background color

**Arguments**

- instance - pointer to the intel\_vvp\_text\_box\_instance structure
- r\_cr - pointer to store the red component in RGB, or Cr in YCbCr
- g\_y - pointer to store the red component in RGB, or Y in YCbCr
- b\_cb - pointer to store the blue component in RGB, Cb in YCbCr, or the monochrome component
- alpha - pointer to store the alpha component; this is set to 0 if alpha is not enabled

**Returns**

- kIntelVvpCoreOk in case of success
- kIntelVvpCoreInstanceErr if instance is NULL or if **Debug features** is off

### intel\_vvp\_text\_box\_write\_word

#### Prototype

```
int
intel_vvp_text_box_write_word(intel_vvp_text_box_instance*
instance, uint32_t data, uint32_t reg_num);
```

**Description** Write a word (uint32\_t) to a text register. This method can be used to implement a custom, optimised version of intel\_vvp\_text\_box\_write\_string.

Characters are stored in little-endian order, such that register 0 & 0xFF returns the first character.

**Arguments**

- instance - pointer to the intel\_vvp\_text\_box\_instance structure
- data - The word to write
- reg\_num - The text register number to write to

**Returns**

- kIntelVvpCoreOk in case of success
- kIntelVvpCoreInstanceErr if instance is NULL
- kIntelVvpCoreTextBoxOutOfBoundsErr if reg\_num is out of bounds

### intel\_vvp\_text\_box\_write\_string

#### Prototype

```
int
intel_vvp_text_box_write_string(intel_vvp_text_box_instance*
instance, const char str[]);
```

**Description** Helper method to write a string to the corresponding registers



This is an inefficient function; it will always write to every register. If \n is detected, then spaces are added until the end of the line. If the string is shorter than the maximum capacity, then spaces are also added.

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure  
str - String to write

**Returns** kIntelVvpCoreOk in case of success  
kIntelVvpCoreInstanceErr if instance is NULL

#### intel\_vvp\_text\_box\_get\_field\_width

**Prototype**

```
uint32_t
intel_vvp_text_box_get_field_width(intel_vvp_text_box_instance*
instance);
```

**Description** Returns the current field width

This is derived from the current font size and compiled number of characters per line

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** The field width, or 0 in case of error

#### intel\_vvp\_text\_box\_get\_field\_height

**Prototype**

```
uint32_t
intel_vvp_text_box_get_field_height(intel_vvp_text_box_instance*
instance);
```

**Description** Returns the current field height

This is derived from the current font size and compiled number of lines

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** The field height, or 0 in case of error

## intel\_vvp\_text\_box\_commit\_writes

### Prototype

```
int
intel_vvp_text_box_get_commit_writes(intel_vvp_text_box_instance*
instance);
```

**Description** Commit any outstanding register writes (except where specified)

**Arguments** instance - pointer to the intel\_vvp\_text\_box\_instance structure

**Returns** kIntelVvpCoreOk in case of success  
kIntelVvpCoreInstanceErr if instance is NULL



## 43. Tone Mapping Operator IP

---

### 43.1. About the Tone Mapping Operator IP

The tone mapping operator (TMO) IP dynamically adapts the processing of an image based on a regional (tile-based) approach. It improves the visibility of latent image detail and enhances the overall viewing experience.

You can configure the required number of bits per symbol and pixels in parallel. Typical applications include:

- Medical imaging
- Machine vision
- Video conferencing
- Surveillance
- Automotive imaging

#### Figure 117. Example of processing a real-life image using the TMO IP

The figure shows example results obtained after applying the TMO IP dataflow on a real-life image: left is the original image; right is the output image after TMO IP processing.



You provide and receive video data to the TMO IP in RGB format via the AXI4-Stream compatible Altera video streaming interfaces. The IP determines the size of the video busses from the number of pixels processed per clock cycle, the color bit depth, and the number of component streams parameters. The number of video component streams is fixed at 3. The IP supports:

- Component bit depths of 8, 10 and 12-bit.
- Pixels per clock of 1, 2 and 4.

You control the strength of the contrast enhancement for the output images provided by the TMO IP via an Avalon memory-mapped control interface. The data bus for the control interface is set to 32-bit to interface with an embedded CPU. During operation, you can configure the TMO IP using a software driver that controls all the IP parameters via a set of software APIs.

The TMO IP allows four modes of operations:

- Passthrough
- Contrast-enhancement
- Horizontal and vertical slider
- Region of interest

The *Example of processing a real-life image using the TMO IP* figure shows passthrough and contrast enhancement.

Region of interest allows you to define a specific window within the active picture, so that the contrast enhancement is only applied in that specific part of the output image.

**Figure 118. Examples of horizontal slider and region of interest**



The TMO IP supports RGB sampling. The sampling method at the output is always the same as the input. You must provide details of the current standard video resolution via the CPU control interface to ensure correct behavior. the IP only supports 4:4:4 progressive sampling. You should perform any deinterlacing and chroma up or down sampling externally to the TMO IP.

The IP only supports lite variants. For more information about full and lite variants, refer to the *Altera Streaming Video Protocol Specification*

### Related Information

[Altera Streaming Video Protocol Specification](#)

#### 43.1.1. TMO IP Features

- Altera video streaming data interfaces for video IOs
- Avalon memory-mapped interface for CPU control interfaces
- Volume controls to dial up or down contrast enhancement strength
- RGB 8-bit, 10bit, or 12-bit per color component
- 1, 2, or 4 parallel pixels per clock
- 16 tiles (arranged in a 4x4 grid) for local image statistics collection

- Video resolutions up to 8192x4320 at 60 fps or 8192x8192 at 30 fps
- Latency of less than 150 pixels
- FPGA footprint of approximately:
  - ☐ 7K ALMs
  - ☐ 56 DSP blocks
  - ☐ 60 M20Ks

### 43.1.2. TMO IP Performance and Resource Utilization

Altera provides resource and utilization data for guidance. TMO IP resource utilization depends on the device family and the number of supported bits per sample and pixels in parallel.

**Table 805. Resource Utilization for Agilex 7 Devices**

Targeting Agilex 7 AGIB027R29A1E1V device

| Parameters      |                    | Resource Utilization |       |            |
|-----------------|--------------------|----------------------|-------|------------|
| Bits per Sample | Pixels in Parallel | ALMs                 | M20Ks | DSP Blocks |
| 8               | 1                  | 7,796                | 53    | 56         |
| 8               | 2                  | 11,560               | 79    | 105        |
| 8               | 4                  | 18,141               | 130   | 203        |
| 10              | 1                  | 7,812                | 53    | 56         |
| 10              | 2                  | 11,718               | 79    | 105        |
| 10              | 4                  | 18,797               | 132   | 203        |
| 12              | 1                  | 8,034                | 56    | 56         |
| 12              | 2                  | 11,881               | 85    | 105        |
| 12              | 4                  | 19,998               | 144   | 203        |

**Table 806. Resource Utilization for Arria 10 Devices**

Targeting Arria 10 10AS066H1F34E1HG device.

| Parameters      |                    | Resource Utilization |       |            |
|-----------------|--------------------|----------------------|-------|------------|
| Bits per Sample | Pixels in Parallel | ALMs                 | M20Ks | DSP Blocks |
| 8               | 1                  | 6,396                | 64    | 56         |
| 8               | 2                  | 9,051                | 99    | 105        |
| 8               | 4                  | 14,330               | 164   | 203        |
| 10              | 1                  | 6,536                | 64    | 56         |
| 10              | 2                  | 9,287                | 99    | 105        |
| 10              | 4                  | 14,787               | 167   | 203        |
| 12              | 1                  | 6,628                | 67    | 56         |
| 12              | 2                  | 9,569                | 105   | 105        |
| 12              | 4                  | 15,249               | 181   | 203        |

**Table 807. Resource Utilization for Cyclone 10 GX Devices**

Targeting Cyclone 10 GX 10CX220YF672E5G device

| Parameters      |                    | Resource Utilization |       |            |
|-----------------|--------------------|----------------------|-------|------------|
| Bits per Sample | Pixels in Parallel | ALMs                 | M20Ks | DSP Blocks |
| 8               | 1                  | 6,437                | 64    | 56         |
| 8               | 2                  | 9,076                | 99    | 105        |
| 10              | 1                  | 6,539                | 64    | 56         |
| 10              | 2                  | 9,313                | 99    | 105        |
| 12              | 1                  | 6,662                | 67    | 56         |
| 12              | 2                  | 9,543                | 105   | 105        |

**Table 808. Resource Utilization for Stratix 10 Devices**

Targeting Stratix 10 1SX280LN2F43E1VG device

| Parameters      |                    | Resource Utilization |       |            |
|-----------------|--------------------|----------------------|-------|------------|
| Bits per Sample | Pixels in Parallel | ALMs                 | M20Ks | DSP Blocks |
| 8               | 1                  | 7,921                | 53    | 56         |
| 8               | 2                  | 11,466               | 79    | 105        |
| 8               | 4                  | 18,079               | 130   | 203        |
| 10              | 1                  | 8,087                | 53    | 56         |
| 10              | 2                  | 11,482               | 79    | 105        |
| 10              | 4                  | 18,663               | 132   | 203        |
| 12              | 1                  | 8,317                | 56    | 56         |
| 12              | 2                  | 12,349               | 85    | 105        |
| 12              | 4                  | 18,746               | 144   | 203        |

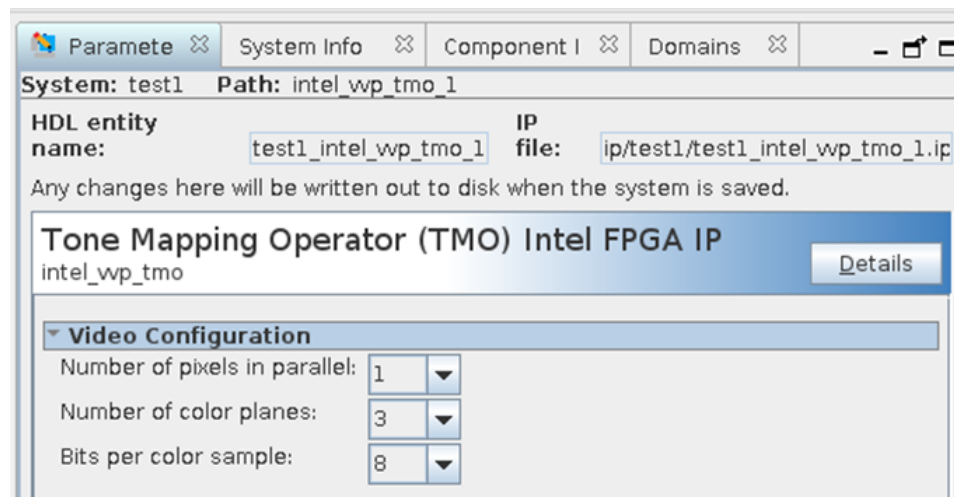
## 43.2. TMO IP Parameters

The IP offers compile-time and run-time parameters

**Table 809. Compile-time Parameters**

| Name                         | Values    | Description                              |
|------------------------------|-----------|------------------------------------------|
| Video Configuration          |           |                                          |
| Number of pixels in parallel | 1, 2, 4   | Number of pixels/samples in parallel     |
| Number of color planes       | 3         | Number of color planes (4:4:4 RGB video) |
| Bits per color sample        | 8, 10, 12 | Number of bits per pixels/samples        |

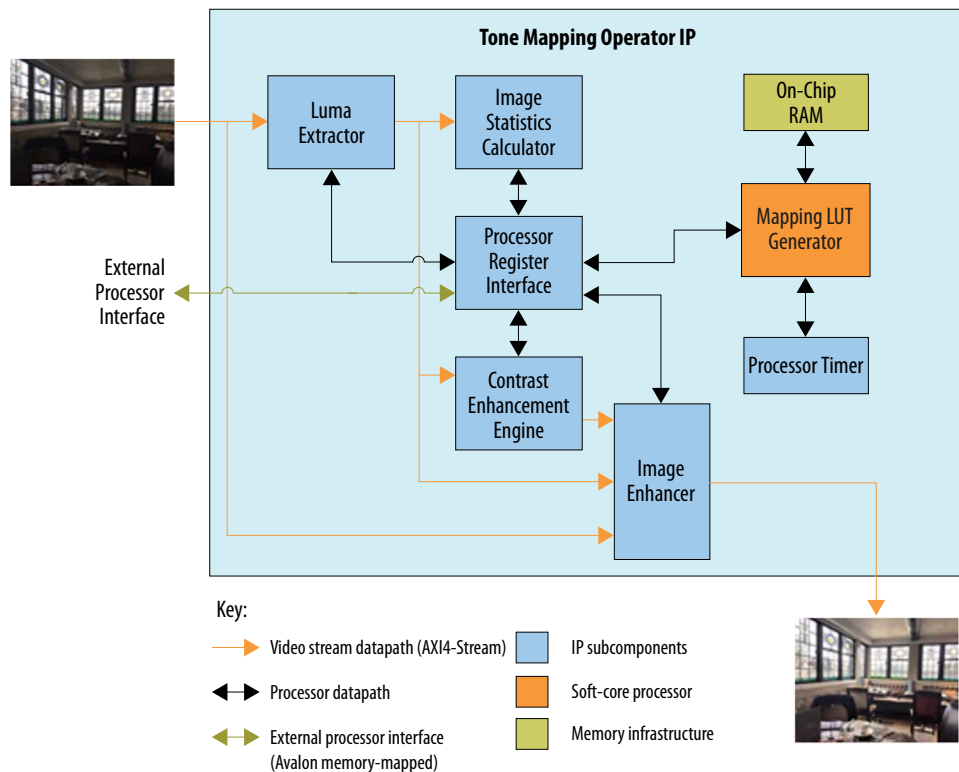
For more information on the run-time parameters refer to *TMO IP Registers*.

**Figure 119. TMO IP GUI**

### 43.3. TMO IP Block Description

The IP accepts RGB-format video input as an Altera video streaming interface, statistically analyses image content (locally and globally), and dynamically enhances the luma range to improve overall image contrast. This IP enhances input video frame imagery into a well-lit and detailed image.

Figure 120. TMO IP High-level block diagram.



The TMO IP consists of several blocks for video processing, memory, and control. The video datapath includes a luma extractor, image statistics calculator, a soft-processor-based mapping LUT generator, CPU register interface, a contrast enhancement engine, and an image enhancer.

The luma extractor takes an RGB input frame, analyzes it, and extracts luminance. The image statistics calculator takes luma information contained in a video frame and provides a set of global and local statistic parameters regarding the contrast information on the input video frame.

The IP collects local information about the input images in different regions on a video frame, providing the necessary granularity to properly enhance contrast in areas within the video frame that need to be adjusted.

The soft-processor-based mapping LUT generator takes the data gathered from the image statistics calculator block and generates a set of mapping transfer functions. The IP temporarily stores the mapping transfer functions in LUTs to reduce resource utilization footprint.

The contrast enhancement engine applies different amounts of mapping transfer functions in different regions of a video frame, providing the necessary granularity to properly enhance contrast in areas within the frame that you need to adjust. The TMO



IP does not use external video frame buffers. Consequently, the contrast enhancement process that the IP applies to the current frame uses statistical information it collects from the previous video frame.

The image enhancer takes the image statics information gathered from the input video frame and with the generated mapping transfer function, it enhances the luma range. The image enhancer calculates a set of weights that it applies to the input RGB data to generate contrast enhanced RGB output video streams.

The embedded Nios® V/g processor used as a mapping LUT generator, is packaged as part of the TMO IP, and customers do not have direct access to it. An external Avalon memory-mapped processor control interface allows you to interact and configure TMO IP, giving them access to the control registers. Because of a higher level of abstraction, a set of software APIs allow you to easily configure and interact with the IP.

You need to get the free license for the Nios V processor to compile the design in Quartus Prime software.

**Figure 121. Graphical description of a tile-based histogram generation**

The figure shows a graphical description of the tile-based approach, explicitly showing tiles boundaries. Tile boundaries are not visible when you operate the TMO IP. The figure shows them only to demonstrate the IP operation.



### Related Information

Nios V Quick Start Guide

## 43.3.1. TMO IP Interfaces

The IP has three functional interfaces, three clock domains, and three resets.

## Functional Interfaces

The TMO IP has three functional interfaces

- Altera video stream input interface (`axi4s_vid_in`)
- Altera video stream output interface (`axi4s_vid_out`)
- External Avalon memory-mapped compatible CPU interface (`av_mm_cpu_agent`)

The Altera video streaming protocol is a standard interface to connect components that exchange data.

## Avalon memory-mapped interface

The IP external CPU interface is an Avalon memory-mapped interface that accesses the control and status registers.

For CPU control interfaces, the TMO IP uses the Avalon memory-mapped protocol. AXI4 protocols are natively supported in Platform Designer. You can automatically adapt to and from Avalon memory-mapped interfaces.

The Avalon memory-mapped interface is an address-based read and write interface typical of host and agent connections. A host is the interface that initiates a transfer request, and an agent is the interface that receives the transfer request. The Avalon memory-mapped interface allows you to dynamically control parameters within the TMO IP by connecting the TMO IP Avalon memory-mapped interface (Avalon memory-mapped agent) to an embedded ARM processor or soft system processor such as a Nios V/g processor (Avalon memory-mapped host).

You can control the TMO IP through the Avalon memory-mapped interface with the IP drivers and API functions.

| Signal Name                             | Avalon Specification Name | Direction     | Width (Bits) | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----------------------------------------|---------------------------|---------------|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>av_mm_cpu_agent_address</code>    | address                   | Host to Agent | 7            | For a host, by default, the address signal represents a byte address. The value of the address must align to the data width. To write to specific bytes within a data word, the host must use the <code>byteenable</code> signal.<br><br>For an agent, by default, the interconnect translates the byte address into a word address in the agent's address space. From the perspective of the agent, each agent access is for a word of data.                                           |
| <code>av_mm_cpu_agent_byteenable</code> | byteenable                | Host to Agent | 4            | Enables one or more specific byte lanes during transfers on interfaces of width greater than 8 bits. Each bit in <code>byteenable</code> corresponds to a byte in <code>writedata</code> and <code>readdata</code> . The host bit <code>&lt;n&gt;</code> of <code>byteenable</code> indicates whether the IP is writing to byte <code>&lt;n&gt;</code> . During writes, <code>byteenable</code> specifies which bytes the IP is writing to. Other bytes should be ignored by the agent. |

*continued...*

| Signal Name                            | Avalon Specification Name | Direction     | Width (Bits) | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------------------------------|---------------------------|---------------|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                        |                           |               |              | During reads, <code>byteenable</code> indicates which bytes the host is reading. Agents that return read data with no side effects are free to ignore <code>byteenable</code> signals during reads. If an interface does not have a <code>byteenable</code> signal, the transfer proceeds as if all <code>byteenable</code> signals are asserted. When more than one bit of the <code>byteenable</code> signal is asserted, all asserted lanes are adjacent. TMO IP ignores the <code>byteenable</code> signal. |
| <code>av_mm_cpu_agent_write</code>     | <code>write</code>        | Host to Agent | 1            | Asserted to indicate a write transfer. If present, <code>writedata</code> is required. Required for interfaces that support writes.                                                                                                                                                                                                                                                                                                                                                                             |
| <code>av_mm_cpu_agent_writedata</code> | <code>writedata</code>    | Host to Agent | 32           | Data for write transfers. The width must be the same as the width of <code>readdata</code> if both are present. Required for interfaces that support writes.                                                                                                                                                                                                                                                                                                                                                    |
| <code>av_mm_cpu_agent_read</code>      | <code>read</code>         | Host to Agent | 1            | Asserted to indicate a read transfer. If present, <code>readdata</code> is required. Required for interfaces that support reads.                                                                                                                                                                                                                                                                                                                                                                                |
| <b>continued...</b>                    |                           |               |              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

| Signal Name                   | Avalon Specification Name | Direction     | Width (Bits) | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------------------------------|---------------------------|---------------|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| av_mm_cpu_agent_readdata      | readdata                  | Agent to Host | 32           | You drive the <code>readdata</code> from the agent to the host in response to a read transfer. Required for interfaces that support reads.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| av_mm_cpu_agent_readdatavalid | readdatavalid             | Agent to Host | 1            | Use for variable-latency, pipelined read transfers. When asserted, indicates that the <code>readdata</code> signal contains valid data. For a read burst with burstcount value $<n>$ , the <code>readdatavalid</code> signal must be asserted $<n>$ times, once for each read data item. Ensure at least one cycle of latency between acceptance of the read and assertion of <code>readdatavalid</code> . An agent may assert read data valid to transfer data to the host independently of whether the agent is stalling a new command with <code>waitrequest</code> . Required if the host supports pipelined reads. Bursting hosts with read functionality must include the <code>readdatavalid</code> signal.                                                                                                                                                                                                                                  |
| av_mm_cpu_agent_waitrequest   | waitrequest               | Agent to Host | 1            | An agent asserts <code>waitrequest</code> when unable to respond to a read or write request. Forces the host to wait until the interconnect is ready to proceed with the transfer. At the start of all transfers, a host initiates the transfer and waits until <code>waitrequest</code> is deasserted. A host must make no assumption about the assertion state of <code>waitrequest</code> when the host is idle: wait request may be high or low, depending on system properties. When wait request is asserted, host control signals to the agent must remain constant except for begin burst transfer.<br><br>An Avalon memory-mapped agent may assert <code>waitrequest</code> during idle cycles. An Avalon memory-mapped host may initiate a transaction when <code>waitrequest</code> is asserted and wait for that signal to be deasserted. To avoid system lockup, an agent device should assert <code>waitrequest</code> when in reset. |

## Clocks

**Table 810. TMO IP Clocks**

| Signal Name            | Direction | Width (Bits) | Associated Interface              | Description                                          |
|------------------------|-----------|--------------|-----------------------------------|------------------------------------------------------|
| internal_cpu_clock_clk | Input     | 1            | N.A                               | Input clock for the soft-processor-based mapping LUT |
| external_cpu_clock_clk | Input     | 1            | CPU control interface             | Input clock for the external CPU control interface   |
| video_clock_clk        | Input     | 1            | Video input and output interfaces | Input clock for the video and processing datapath    |

**Table 811. Video Clock Frequency Range Values**

| Device Family | Frequency range (MHz) |
|---------------|-----------------------|
| Cyclone 10 GX | 150 to 300            |
| Arria 10      | 150 to 300            |
| Stratix 10    | 150 to 400            |
| Agilex 7      | 150 to 600            |

Frequency depends on:

- Number of pixels in parallel
- Maximum video resolution
- Frame rate
- Video blanking area
- Device family

All three input clocks are asynchronous from each other. Internally, the TMO IP includes clock domain crossing (CDC) circuits for both single bit and data bus signal cases, which safely allows data exchange between any of the three asynchronous clock domains. The TMO IP also includes an embedded `.sdc` file, which provides all the necessary information to the Timing Analyzer. For system integration, when you instantiate the TMO IP in a design, the only constraints required are:

- Clock frequency constraints for the video clock (`video_clock_clk`)
- Processor clock (`external_cpu_clock_clk`)
- Soft-processor-based mapping LUT generator clock (`internal_cpu_clock_clk`)

### Resets

| Name                                  | Direction | Width (Bits) | Type        | Associated Interface   | Description                                                    |
|---------------------------------------|-----------|--------------|-------------|------------------------|----------------------------------------------------------------|
| <code>internal_cpu_reset_reset</code> | Input     | 1            | Active-high | N.A                    | Input reset for the soft-processor-based mapping LUT generator |
| <code>external_cpu_reset_reset</code> | Input     | 1            | Active-high | CPU control            | Input reset for the external CPU control interface             |
| <code>video_reset_reset</code>        | Input     | 1            | Active-high | Video input and output | Input reset for the video and processing datapath              |

Ensure before connecting the reset signals to the TMO IP, they are synchronized with their respective associated clock domain. Platform Designer provides a Reset Bridge IP for this task.

### Related Information

- [Video and Vision Processing IP Interfaces](#) on page 30
- [Platform Designer: Reset Bridge](#)  
The Reset Bridge allows you to use a reset signal in two or more subsystems of your Platform Designer system.

### 43.3.2. TMO IP Latency

The latency information can predict the approximate latency between the input and the output of your video processing pipeline.

**Table 812. TMO Latency**

The table shows latency as a number of valid clock cycles. Altera measures the latency assuming that other functions are not stalling the IP on the datapath (the output ready signal is high).

| Mode                 | Latency (cycles) |
|----------------------|------------------|
| Processing or bypass | 106              |

### 43.4. TMO IP Registers

The TMO IP allows run-time configuration parameters via AXI4-Lite CPU register interface.

The IP offers the following categories of run-time configuration parameters:

- Flow control parameters that allow you to put the TMO IP into either reset, bypass, or operational mode.
- Status and debug parameters that provide information about compile-time parameters and current status of the TMO IP
- Video configuration parameters that allow you to configure the input video frame geometry
- Image statistics collection parameters that allow you to configure the tile's dimension

**Table 813. Register Map**

| Register Name          | Byte Address Offset | Access Type |
|------------------------|---------------------|-------------|
| vid_pid                | 0x000               | RO          |
| version_number         | 0x004               | RO          |
| reserved_area          | 0x140 – 0x147       | Reserved    |
| ip_information_0       | 0x148               | RO          |
| ip_information_1       | 0x14C               | RO          |
| ip_information_2       | 0x150               | RO          |
| vid_flow_control       | 0x154               | RW          |
| actv_vid_size          | 0x158               | RW          |
| volume_control         | 0x15C               | RW          |
| tmo_derived_parameters | 0x160 – 0x1D0       | Reserved.   |
| roi_horizontal_pos     | 0x1D4               | RW          |
| roi_vertical_pos       | 0x1D8               | RW          |
| reserved_area          | 0x1D0 – 0x17F       | Reserved.   |

**Table 814.** vid\_pid

| Bits | Description                    |
|------|--------------------------------|
| 31:0 | Product Identification Number. |

**Table 815.** version\_number

| Bits | Description     |
|------|-----------------|
| 31:0 | Version Number. |

**Table 816.** reserved\_area

| Bits | Description            |
|------|------------------------|
| 31:0 | Reserved register area |

**Table 817.** ip\_information\_0

| Bits  | Description                       |
|-------|-----------------------------------|
| 27:24 | Number of tiles (C_TILE)          |
| 23:16 | AXI4-Stream data width            |
| 11:8  | Pixels in parallel (C_PIXELS)     |
| 7:4   | Components per sample (C_STREAMS) |
| 3:0   | Bits per component (C_DEPTH)      |

**Table 818.** ip\_information\_1

| Bits  | Description                                                            |
|-------|------------------------------------------------------------------------|
| 29:25 | Fractional precision for luminance weights (C_FRAC_PREC_MLUT)          |
| 24:20 | Fractional precision for TMO volume control (C_FRAC_PREC_VOLCNTR)      |
| 19:15 | Fractional precision for RGB to luma conversion (C_FRAC_PREC_RGB2LUMA) |
| 14:10 | Fractional precision for luma to RGB conversion (C_FRAC_PREC_LUMA2RGB) |
| 9:5   | Histogram address data width (C_HIST_ADDR_WIDTH)                       |
| 4:0   | Histogram data width (C_HIST_DATA_WIDTH)                               |

**Table 819.** ip\_information\_2

| Bits  | Description                                                           |
|-------|-----------------------------------------------------------------------|
| 21:17 | Fractional precision for interpolation (C_FRAC_PREC_INTP)             |
| 16:12 | Fractional precision for histogram equalization (C_FRAC_PREC_HEQ)     |
| 11:0  | Fractional precision for histogram normalization factor (C_NORM_FACT) |



**Table 820.** vid\_flow\_control

| Bits | Description                                                                                                                                                                                                                        |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31   | Soft-reset bit. When set to 1 the TMO IP is in reset.                                                                                                                                                                              |
| 2    | Swap region bit for region of interest mode.<br>0: Process inside of the selected region and bypass outside.<br>1: Process outside of the selected region and process outside.<br>The IP ignores this bit in bypass mode.          |
| 1    | Region of Interest bit. Set to 1 to put the TMO IP in region of interest mode. Swap region bit determines whether inside or outside of the region of interest is processed or bypassed.<br>The IP ignores this bit in bypass mode. |
| 0    | Bypass bit. Set to 1 to put the TMO IP into bypass mode.                                                                                                                                                                           |

**Table 821.** actv\_vid\_size

| Bits  | Description                                                                                                                                                                                                                     |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 29:16 | Total number of active pixels per video line (C_WIDTH).<br>Because the IP uses an internal grid for collecting local image statistics, the value must be an integer multiple of: (number of pixels in parallel multiplied by 8) |
| 13:0  | Total number of active lines per video frame (C_HEIGHT).<br>Because the IP uses an internal grid for collecting local image statistics, the value must be an integer multiple of 8                                              |

**Table 822.** roi\_horizontal\_pos

| Bits  | Description                                                              |
|-------|--------------------------------------------------------------------------|
| 29:16 | Horizontal pixel coordinate where rectangular region of interest starts. |
| 13:0  | Horizontal pixel coordinate where rectangular region of interest ends.   |

**Table 823.** roi\_vertical\_pos

| Bits  | Description                                                            |
|-------|------------------------------------------------------------------------|
| 29:16 | Vertical pixel coordinate where rectangular region of interest starts. |
| 13:0  | Vertical pixel coordinate where rectangular region of interest ends.   |

**Table 824.** volume\_control

| Bits  | Description                                                       |
|-------|-------------------------------------------------------------------|
| 22:16 | Fine-level TMO volume control. Valid range [0:100] decimal        |
| 13:0  | Coarse-level TMO strength threshold. Valid range [0:9000] decimal |

**Table 825.** tmo\_derived\_parameters

| Bits | Description                                                                              |
|------|------------------------------------------------------------------------------------------|
| 31:0 | This area is reserved for all derived parameter registers. Do not write or read from it. |

## 43.5. TMO IP Software API

The IP includes a software driver that configures and controls all the IP parameters.

## Software Driver Example

```
int main()
{
 int ret = -1;

 /* Initialize datapath Video Timing Generator instance
 /* This API function is not part of the TMO driver. */
 /* Hence, it is up to the user to implement it, to provide VIDEO_WIDTH,
 VIDEO_HEIGHT values */
 datapath_vtiming_config(VTIMING_TMO_BASE, 1);

 /* Initialize TMO IP instance*/
 intel_vvp_base_t base = INTEL_VVP_TMO_BASE;
 intel_vvp_tmo_instance_t intel_vvp_tmo_instance;

 /*Query TMO IP instance, to know whether is has been initialized correctly. */
 /* A value == '0' indicates TMO IP has been initialized correctly */
 ret = intel_vvp_tmo_init_instance(&intel_vvp_tmo_instance, base);

 if (ret == 0)
 {
 /* TMO IP Bypass control */
 uint32_t bypass = 0;

 intel_vvp_tmo_set_bypass(&intel_vvp_tmo_instance, bypass);
 printf("Intel VVP TMO Bypass %s\n\n", bypass ? "ENABLED" : "DISABLED");

 /* Initialize TMO IP to specific video resolution */
 intel_vvp_tmo_set_resolution(&intel_vvp_tmo_instance, VIDEO_WIDTH,
 VIDEO_HEIGHT);

 /* Update TMO IP volume control values */
 intel_vvp_tmo_set_volume(&intel_vvp_tmo_instance, VOLUME_CTL_USER);
 intel_vvp_tmo_set_threshold(&intel_vvp_tmo_instance, INT_THR_USER);

 printf("TMO initialization done\n");
 fcntl(STDIN_FILENO, F_SETFL, O_NONBLOCK);

 while (1)
 {
 /* This function checks if TMO IP bypass mode and/or volume control values
 need to be updated */
 /* but it is not part of the TMO IP SW driver package. Hence, customers
 are expected */
 /* to implement their own function according to their specific needs */
 process_user_input(&intel_vvp_tmo_instance);

 /* This part of the code is for debug purpose only */
 /* Two conditions are checked: */

 /* Condition #1: Missing data collection per tile */
 static const uint32_t num_rows = 4;
 static const uint32_t row_mask = ~(~0u << num_rows);

 /* Read TMO IP debug register to check for error conditions */
 uint32_t reg_val = intel_vvp_tmo_get_debug(&intel_vvp_tmo_instance);

 if (reg_val & row_mask)
 {
 printf ("Timeout rows: ");

 for (uint32_t i = 0; i < num_rows; ++ i)
 {
```

```

 if (reg_val & (0x1 << i))
 printf("%PRIu32" ", i);
 }

 printf("\n";
}

/* Condition #2: Missing entire video frame */
if (reg_val & 0x100)
{
 printf ("FSYNC toggle error\n";
}

/* Set TMO IP debug register to capture new error conditions */
intel_vvp_tmo_set_debug(&intel_vvp_tmo_instance, reg_val);
}
else
{
 printf("Error initializing intel_vvp_tmo instance: %d\n", ret);
}

return ret;
}

```

The TMO IP software driver does not include the `process_user_input(&intel_vvp_tmo_instance)` function, as its implementation depends on your system requirements and specifications. You can query and update TMO strength volume values in various ways. For example, you can implement the `process_user_input(&intel_vvp_tmo_instance)` in this way:

```

void process_user_input(intel_vvp_tmo_instance* instance)
{
 int c = getchar();
 static const int32_t VOL_STEP = 5;
 static const int32_t TS_STEP = 1000;
 int vol_delta = 0;
 int ts_delta = 0;

 if(c != EOF)
 {
 switch(c)
 {
 case 'w':
 case 'W':
 vol_delta = VOL_STEP;
 break;
 case 's':
 case 'S':
 vol_delta = -VOL_STEP;
 break;
 case 'd':
 case 'D':
 ts_delta = TS_STEP;
 break;
 case 'a':
 case 'A':
 ts_delta = - TS_STEP;
 break;
 case 'b':
 case 'B':
 {
 uint32_t bypass = intel_vvp_tmo_get_bypass(instance);
 bypass ^= 0x1;
 intel_vvp_tmo_set_bypass(instance, bypass);
 printf("TMO Bypass %s\n", bypass ? "ENABLED" : "DISABLED");
 }
 break;
 }
 }
}

```

```

 default:
 break;
 }

 if(vol_delta || ts_delta)
 {
 int32_t vol = (int32_t)(intel_vvp_tmo_get_volume(instance));
 int32_t ts = (int32_t)(intel_vvp_tmo_get_threshold(instance));

 vol += vol_delta;
 ts += ts_delta;

 if(vol > 100)
 vol = 100;

 if(vol < 0)
 vol = 0;

 if(ts > 9000)
 ts = 9000;

 if(ts < 0)
 ts = 0;

 printf("VOL: %"PRIi32"\n", vol);
 printf("TS: %"PRIi32"\n", ts);

 intel_vvp_tmo_set_volume(instance, (uint32_t)(vol));
 intel_vvp_tmo_set_threshold(instance, (uint32_t)(ts));
 }
}

```

All the API functions require a pointer to `intel_vvp_tmo_instance_t` structure as the first parameter. The structure represents an individual instance of TMO IP and defined as following:

```

typedef struct intel_vvp_tmo_instance {
 intel_vvp_tmo_base_t base;
} intel_vvp_tmo_instance_t;

```

Where:

`intel_vvp_tmo_base_t base` is a platform specific access handler that the driver uses to access configuration and control registers of the IP. Default definition for a bare metal environment is 32-bit unsigned integer representing base address of the TMO IP on the external CPU bus.

The internal driver uses the following macros to access individual registers of the IP:

- `tmoss_read_reg(x)` - read register
- `tmoss_write_reg(x, y)` - write register

Where:

`x` is the byte offset of the register from the IP core base address

`y` is the 32-bit register value to write.

Default bare metal implementation of the IP register access provided in the file `intel_vvp_tmo_io.h`. Provide alternative implementations through separate header files included conditionally from `intel_vvp_tmo_io.h`.

Byte offsets of all TMO IP registers are defined in the file `intel_vvp_tmo_regs.h`.

**Table 826. Software driver API reference**

The software driver for TMO IP provides various API functions

| Name                                               | Description                                                                                                        |
|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <code>intel_vvp_tmo_init_instance</code>           | Initialize TMO IP driver instance                                                                                  |
| <code>intel_vvp_tmo_set_bypass</code>              | Set TMO IP into bypass mode                                                                                        |
| <code>intel_vvp_tmo_get_bypass</code>              | Return current setting of the bypass mode                                                                          |
| <code>intel_vvp_tmo_set_resolution</code>          | Set input video resolution for TMO IP                                                                              |
| <code>intel_vvp_tmo_set_volume</code>              | Adjust fine-level TMO volume as a percentage, which allows more granularity when setting TMO strength value        |
| <code>intel_vvp_tmo_get_volume</code>              | Return current level of TMO strength                                                                               |
| <code>intel_vvp_tmo_set_threshold</code>           | Adjust coarse level TMO volume, which allows setting TMO strength value                                            |
| <code>intel_vvp_tmo_get_threshold</code>           | Return current value of fine-level TMO volume adjustment                                                           |
| <code>intel_vvp_tmo_set_roi_en</code>              | Enable or disable the region of interest mode                                                                      |
| <code>intel_vvp_tmo_set_roi_swap</code>            | Enable or disable swap region in region of interest mode                                                           |
| <code>intel_vvp_tmo_set_roi_horizontal_x0x1</code> | Set horizontal starting and ending positions of the region of interest, implicitly enables region of interest mode |
| <code>intel_vvp_tmo_set_roi_vertical_y0y1</code>   | Set vertical starting and ending positions of the region of interest, implicitly enables region of interest mode   |
| <code>intel_vvp_tmo_get_roi_en</code>              | Return current value of the region of interest enable bit                                                          |
| <code>intel_vvp_tmo_get_roi_swap</code>            | Return current value of the region of interest swap bit                                                            |
| <code>intel_vvp_tmo_get_roi_horizontal_x0</code>   | Return current value of the horizontal starting position of the region of interest                                 |
| <code>intel_vvp_tmo_get_roi_horizontal_x1</code>   | Return current value of the horizontal ending position of the region of interest                                   |
| <code>intel_vvp_tmo_get_roi_vertical_y0</code>     | Return current value of the vertical starting position of the region of interest                                   |
| <code>intel_vvp_tmo_get_roi_vertical_y1</code>     | Return current value of the vertical ending position of the region of interest                                     |
| <code>intel_vvp_tmo_get_pip</code>                 | Return current configuration value of the number of pixels-in-parallel                                             |
| <code>intel_vvp_tmo_get_resolution_width</code>    | Return width of the current video resolution in number of pixels                                                   |
| <code>intel_vvp_tmo_get_resolution_height</code>   | Return height of the current video resolution in number of lines                                                   |
| <code>intel_vvp_tmo_get_debug</code>               | Return current value of debug information register                                                                 |
| <code>intel_vvp_tmo_set_debug</code>               | Clear individual bits in debug information register                                                                |

#### `intel_vvp_tmo_init_instance`

```
int
intel_vvp_tmo_init_instance(intel_vvp_tmo_instance_t*
instance, intel_vvp_tmo_base_t base)
```

*Description* Initialize TMO software driver instance

*Arguments* instance – pointer to the TMO software driver instance structure

base – platform specific IP access handle. In a bare metal environment this is typically defined as 32-bit unsigned integer representing base address of the IP on the external CPU bus

*Return Value* Zero on success, negative integer otherwise

#### intel\_vvp\_tmo\_set\_bypass

```
void
intel_vvp_tmo_set_bypass(intel_vvp_tmo_instance_t*
instance, uint32_t bypass)
```

*Description* Enable/disable TMO bypass mode. With bypass mode the TMO IP does not process the incoming video stream and passes it as is.

*Arguments* instance – pointer to the TMO software driver instance structure

bypass - 1 – enable bypass mode; 0 – disable bypass mode

*Return Value* void

#### intel\_vvp\_tmo\_get\_bypass

```
uint32_t
intel_vvp_tmo_get_bypass(intel_vvp_tmo_instance_t*
instance)
```

*Description* Get the current setting of the bypass mode

*Arguments* instance – pointer to the TMO software driver instance structure

*Return Value* 1 – bypass mode is enabled; 0 – bypass mode is disabled

#### intel\_vvp\_tmo\_set\_resolution

```
void
intel_vvp_tmo_set_resolution(intel_vvp_tmo_instance_t*
instance, const uint32_t width, const uint32_t height)
```

*Description* Set up TMO IP for the required video resolution

*Arguments* instance – pointer to the TMO software driver instance structure

width - video width in pixels e.g. 1920  
height - video height in lines e.g. 1080

*Return Value*      void

#### `intel_vvp_tmo_set_volume`

```
void
intel_vvp_tmo_set_volume(intel_vvp_tmo_instance_t*
instance, const uint32_t volume)
```

*Description*      Set desired tone enhancement strength

*Arguments*      instance – pointer to the TMO software driver instance structure  
volume - tone enhancement strength in the range [0..100]

*Return Value*      void

#### `intel_vvp_tmo_get_volume`

```
uint32_t
intel_vvp_tmo_get_volume(intel_vvp_tmo_instance_t*
instance)
```

*Description*      Get currently configured tone enhancement strength

*Arguments*      instance – pointer to the TMO software driver instance structure

*Return Value*      Current tone enhancement strength in the range [0..100]

#### `intel_vvp_tmo_set_threshold`

```
void
intel_vvp_tmo_set_threshold(intel_vvp_tmo_instance_t*
instance, const uint32_t threshold)
```

*Description*      Set fine-level tone enhancement strength

*Arguments*      instance – pointer to the TMO software driver instance structure  
threshold – fine level tone enhancement strength in the range [0..10000]

*Return Value*      void

#### intel\_vvp\_tmo\_get\_threshold

```
uint32_t
intel_vvp_tmo_get_threshold(intel_vvp_tmo_instance_t*
instance)
```

**Description** Get currently configured fine-level tone enhancement strength

**Arguments** instance – pointer to the TMO software driver instance structure

**Return Value** Current fine level tone enhancement strength in the range [0..10000]

#### intel\_vvp\_tmo\_set\_roi\_en

```
void
intel_vvp_tmo_set_roi_en(intel_vvp_tmo_instance_t*
instance, uint32_t roi_en)
```

**Description** Enable or disable the region of interest mode

**Arguments** instance – pointer to the TMO software driver instance  
region of interest enable – 1 – enable region of interest mode, 0 – disable region of interest mode

**Return Value** void

#### intel\_vvp\_tmo\_set\_roi\_swap

```
void
intel_vvp_tmo_set_roi_swap(intel_vvp_tmo_instance_t*
instance, uint32_t roi_swap)
```

**Description** Enable or disable swap region in region of interest mode

**Arguments** instance – pointer to the TMO software driver instance  
region of interest swap – 1 – swap the region of interest, 0 – do not swap the region of interest

**Return Value** void



### intel\_vvp\_tmo\_set\_roi\_horizontal\_x0x1

```
void
intel_vvp_tmo_set_roi_horizontal_x0x1(intel_vvp_tmo_instance_t*
instance, const uint32_t
pos_x0, const uint32_t pos_x1)
```

- Description** Set horizontal starting and ending positions of the region of interest, implicitly enables region of interest mode
- Arguments** instance – pointer to the TMO software driver instance
- horizontal region of interest starting position – starting pixel position of the region of interest in the width dimension, divided by pixels-in-parallel
- horizontal region of interest ending position – ending pixel position of the region of interest in the width dimension, divided by pixels-in-parallel
- Return Value** void

### intel\_vvp\_tmo\_set\_roi\_vertical\_y0y1

```
void
intel_vvp_tmo_set_roi_vertical_y0y1(intel_vvp_tmo_instance_t*
instance, const uint32_t
pos_y0, const uint32_t pos_y1)
```

- Description** Set vertical starting and ending positions of the region of interest, implicitly enables region of interest mode
- Arguments** instance – pointer to the TMO software driver instance
- vertical region of interest starting position – starting line position of the region of interest in the height dimension
- vertical region of interest ending position – ending line position of the region of interest in the height dimension
- Return Value** void

### intel\_vvp\_tmo\_get\_roi\_en

```
uint32_t
intel_vvp_tmo_get_roi_en(intel_vvp_tmo_instance_t* instance)
```

- Description** Return current value of the region of interest enable bit
- Arguments** instance – pointer to the TMO software driver instance structure

**Return Value** 1 – region of interest mode is enabled; 0 – region of interest mode is disabled

#### intel\_vvp\_tmo\_get\_roi\_swap

```
uint32_t
intel_vvp_tmo_get_roi_swap(intel_vvp_tmo_instance_t*
instance)
```

**Description** Return current value of the region of interest swap bit

**Arguments** instance – pointer to the TMO software driver instance structure

**Return Value** 1 – region of interest is swapped; 0 – region of interest is not swapped

#### intel\_vvp\_tmo\_get\_roi\_horizontal\_x0

```
uint32_t
intel_vvp_tmo_get_roi_horizontal_x0(intel_vvp_tmo_instance_t*
instance)
```

**Description** Return current value of the horizontal starting position of the region of interest

**Arguments** instance – pointer to the TMO software driver instance structure

**Return Value** Current value of the horizontal starting pixel position of the region of interest, divided by pixels-in-parallel

#### intel\_vvp\_tmo\_get\_roi\_horizontal\_x1

```
uint32_t
intel_vvp_tmo_get_roi_horizontal_x1(intel_vvp_tmo_instance_t*
instance)
```

**Description** Return current value of the horizontal ending position of the region of interest

**Arguments** instance – pointer to the TMO software driver instance structure

**Return Value** Current value of the horizontal ending pixel position of the region of interest, divided by pixels-in-parallel

#### intel\_vvp\_tmo\_get\_roi\_vertical\_y0

```
uint32_t
intel_vvp_tmo_get_roi_vertical_y0(intel_vvp_tmo_instance_t*
instance)
```

|                     |                                                                                  |
|---------------------|----------------------------------------------------------------------------------|
| <i>Description</i>  | Return current value of the vertical starting position of the region of interest |
| <i>Arguments</i>    | instance – pointer to the TMO software driver instance structure                 |
| <i>Return Value</i> | Current value of the vertical starting line position of the region of interest   |

#### intel\_vvp\_tmo\_get\_roi\_vertical\_y1

```
uint32_t
intel_vvp_tmo_get_roi_vertical_y1(intel_vvp_tmo_instance_t*
instance)
```

|                     |                                                                                |
|---------------------|--------------------------------------------------------------------------------|
| <i>Description</i>  | Return current value of the vertical ending position of the region of interest |
| <i>Arguments</i>    | instance – pointer to the TMO software driver instance structure               |
| <i>Return Value</i> | Current value of the vertical ending line position of the region of interest   |

#### intel\_vvp\_tmo\_get\_pip

```
uint32_t
intel_vvp_tmo_get_pip(intel_vvp_tmo_instance_t*
instance)
```

|                     |                                                                        |
|---------------------|------------------------------------------------------------------------|
| <i>Description</i>  | Return current configuration value of the number of pixels-in-parallel |
| <i>Arguments</i>    | instance – pointer to the TMO software driver instance structure       |
| <i>Return Value</i> | Configuration value of the number of pixels-in-parallel                |

#### intel\_vvp\_tmo\_get\_resolution\_width

```
uint32_t
intel_vvp_tmo_get_resolution_width(intel_vvp_tmo_instance_t*
instance)
```

|                     |                                                                  |
|---------------------|------------------------------------------------------------------|
| <i>Description</i>  | Return width of the current video resolution in number of pixels |
| <i>Arguments</i>    | instance – pointer to the TMO software driver instance structure |
| <i>Return Value</i> | Current value of the video resolution width in number of pixels  |

### intel\_vvp\_tmo\_get\_resolution\_height

```
uint32_t
intel_vvp_tmo_get_resolution_height(intel_vvp_tmo_instance_t*
instance)
```

**Description** Return height of the current video resolution in number of lines

**Arguments** instance – pointer to the TMO software driver instance structure

**Return Value** Current value of the video resolution height in number of lines

### intel\_vvp\_tmo\_get\_debug

```
uint32_t
intel_vvp_tmo_get_debug(intel_vvp_tmo_instance_t*
instance)
```

**Description** Get current value of the Debug information register Ext\_Reg\_0xA0

**Arguments** instance – pointer to the TMO software driver instance structure

**Return Value** Current value of Ext\_Reg\_0xA0 as 32-bit unsigned integer

### intel\_vvp\_tmo\_set\_debug

```
void
intel_vvp_tmo_set_debug(intel_vvp_tmo_instance_t*
instance, uint32_t val)
```

**Description** Clear individual bits of the Debug information register Ext\_Reg\_0xA0

**Arguments** instance – pointer to the TMO software driver instance structure  
val - bit mask of the bits within the debug register to clear

**Return Value** void

### Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

## 44. Test Pattern Generator IP

### 44.1. About the Test Pattern Generator IP

The IP produces an Altera Streaming Video compliant output stream, with each field containing one of the fixed test images. The IP enables you to adjust the output resolution, color space, subsampling, and selected test pattern at run time.

The IP supports both the full and lite variants of the Altera Streaming Video Protocol. You can use IP during the design cycle to validate a video system without the possible throughput issues associated with a real video input.

The IP:

- Supports Altera Streaming Video Protocol, including variable color and chroma sampling interfaces.
- Produces data on request and consequently permits easier debugging of a video data path without the risks of overflow or misconfiguration associated with the use of real video input.

#### 44.1.1. Test Pattern Generator IP Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family, number of supported bits per sample, pixels in parallel, number of test patterns.

**Table 827. Test Pattern Generator IP Performance and Resources**

The table shows ALM usage and  $f_{MAX}$  for a design with 10 bits per color sample, 2 pixels in parallel, 3 test patterns (bars, constant color and SDI pathological).

| Target device                                  | ALMs | $f_{MAX}$ (MHz) |
|------------------------------------------------|------|-----------------|
| Agilex 7 speed grade 2 (AGFA012R24A2E2V)       | 725  | 908             |
| Arria 10 speed grade 1 (10AS066H1F34E1HG)      | 655  | 594             |
| Cyclone 10 GX speed grade 5 (10CX220YF672E5G)  | 656  | 513             |
| Stratix 10 GX speed grade 1 (1SX280LN2F43E1VG) | 735  | 749             |

**Table 828. Test Pattern Generator IP Performance and Resources - Digital Clock**

The table shows ALM usage and  $f_{MAX}$  for a design with 10 bits per color sample, 2 pixels in parallel, 1 test pattern (digital clock).

| Target device                                  | ALMs  | $f_{MAX}$ (MHz) |
|------------------------------------------------|-------|-----------------|
| Agilex 7 speed grade 2 (AGFA012R24A2E2V)       | 1,961 | 733             |
| Arria 10 speed grade 1 (10AS066H1F34E1HG)      | 1,721 | 572             |
| Cyclone 10 GX speed grade 5 (10CX220YF672E5G)  | 1,723 | 502             |
| Stratix 10 GX speed grade 1 (1SX280LN2F43E1VG) | 2,024 | 569             |

**Table 829. Test Pattern Generator IP Performance and Resources - Zone Plate**

The table shows ALM usage and  $f_{MAX}$  for a design with 10 bits per color sample, 2 pixels in parallel, 1 test pattern (zone plate).

| Target device                                  | DSP Blocks | ALMs | $f_{MAX}$ (MHz) |
|------------------------------------------------|------------|------|-----------------|
| Agilex 7 speed grade 2 (AGFA012R24A2E2V)       | 8          | 787  | 804             |
| Arria 10 speed grade 1 (10AS066H1F34E1HG)      | 8          | 649  | 523             |
| Cyclone 10 GX speed grade 5 (10CX220YF672E5G)  | 8          | 647  | 458             |
| Stratix 10 GX speed grade 1 (1SX280LN2F43E1VG) | 8          | 885  | 651             |

## 44.2. Test Pattern Generator IP Parameters

The IP offers run time and compile time parameters.

| Parameter                                   | Value                                              | Description                                                                                                                                                                                                           |
|---------------------------------------------|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Interface Configuration</b>              |                                                    |                                                                                                                                                                                                                       |
| <b>Lite mode</b>                            | On or off                                          | Turn on to use the Lite variant of the Altera Streaming Video Protocol.                                                                                                                                               |
| <b>Bits per color sample</b>                | 8 to 16                                            | Select the number of bits per color sample                                                                                                                                                                            |
| <b>Number of pixels in parallel</b>         | 1 to 8                                             | Select the number of pixels in parallel at the output interface.                                                                                                                                                      |
| <b>Output format</b>                        | <b>4:4:4, 4:2:2, 4:2:0. Variable or Monochrome</b> | Select the chroma sampling format for the output interface.                                                                                                                                                           |
| <b>General</b>                              |                                                    |                                                                                                                                                                                                                       |
| <b>Memory-mapped control interface</b>      | On or off                                          | Turn on for the Avalon memory-mapped control agent interface and allow runtime configuration via the register map. You must have the Avalon memory-mapped control agent interface when you turn on <b>Lite mode</b> . |
| <b>Separate clock for control interface</b> | On or off                                          | Turn on for a separate clock for the control agent interface.                                                                                                                                                         |
| <b>Debug features</b>                       | On or off                                          | Turn on for read back of writeable registers via the control agent interface.                                                                                                                                         |
| <b>Pipeline ready signals</b>               | On or off                                          | Turn on to add extra pipeline registers to the AXI4-S <i>ready</i> signals.                                                                                                                                           |
| <b>Shared configuration</b>                 |                                                    |                                                                                                                                                                                                                       |
| <i>continued...</i>                         |                                                    |                                                                                                                                                                                                                       |

| Parameter                                            | Value                                           | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------------------------------------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Fixed interlacing</b>                             | 0 to 15                                         | Only if <b>Memory-mapped control interface</b> is not on. Select if output stream is progressive, interlaced with F0 sent first after reset, or interlaced with F1 sent first after reset. If <b>Lite mode</b> is off, the selected value populates the interlaced identifier field in the output image info packets.<br>1. Values 0-7 give progressive output<br>2. Values 8-11 give interlaced output with F0 sent first<br>3. Values 12-15 give interlaced output with F1 sent first<br>. |
| <b>Fixed frame width</b>                             | 1 to 16384                                      | Only if <b>Memory-mapped control interface</b> is off. Select the width of the frames and fields produced at the output. If the selected output subsampling is 4:2:2 or 4:2:0, the frame width must be a multiple of 2.                                                                                                                                                                                                                                                                      |
| <b>Fixed frame height</b>                            | 1 to 16384                                      | Only if <b>Memory-mapped control interface</b> is off. Select the height of the frames and fields produced at the output. If the selected output subsampling is <b>4:2:0</b> , the field height must be a multiple of 2.                                                                                                                                                                                                                                                                     |
| <b>Pattern configuration</b>                         |                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Fixed bars mode</b>                               | <b>Color, Greyscale, Black and white, Mixed</b> | Only if <b>Memory-mapped control interface</b> is off and you select the <b>bars</b> test pattern. Sets which variant of the bars pattern to use.                                                                                                                                                                                                                                                                                                                                            |
| <b>Fixed constant color R/Cr</b>                     | 0 – (2 <sup>n</sup> Bits per color sample)-1    | Only if <b>Memory-mapped control interface</b> is off and you select the <b>constant color</b> test pattern. Sets the value the IP uses for the R color plane if the output is RGB, or the Cr color plane if the output is YCbCr.                                                                                                                                                                                                                                                            |
| <b>Fixed constant color G/Y</b>                      | 0 – (2 <sup>n</sup> Bits per color sample)-1    | Only if <b>Memory-mapped control interface</b> is off and you select the <b>constant color</b> test pattern. Sets the value the IP uses for the G color plane if the output is RGB, or the Y color plane if the output is YCbCr.                                                                                                                                                                                                                                                             |
| <b>Fixed constant color B/Cb</b>                     | 0 – (2 <sup>n</sup> Bits per color sample)-1    | Only if <b>Memory-mapped control interface</b> is off and you select the <b>constant color</b> test pattern. Sets the value the IP uses for the B color plane if the output is RGB, or the Cb color plane if the output is YCbCr.                                                                                                                                                                                                                                                            |
| <b>Fixed zone plate coarse scaling factor</b>        | 1 to 31                                         | Only if <b>Memory-mapped control interface</b> is off. Increase this value to increase the size of the zone plate. The value is arbitrary, but Altera recommends a starting value of 14 for HD and 20 for 4k resolutions.                                                                                                                                                                                                                                                                    |
| <b>Fixed zone plate fine tune scaling factor</b>     | 1 to 65535                                      | Only if <b>Memory-mapped control interface</b> is off. A 16-bit fixed-point value, with 8 bits fractional part. For example, 256 represents a value of 1.0. Increase this value to decrease the size of the zone plate.                                                                                                                                                                                                                                                                      |
| <b>Fixed clock background color (B/Cb component)</b> | 0– (2 <sup>n</sup> Bits per color sample)-1     | Only if <b>Memory-mapped control interface</b> is off. Select the value the IP uses for the B background color plane if the output is RGB, or the Cb color plane if the output is YCbCr.                                                                                                                                                                                                                                                                                                     |
| <b>Fixed clock background color (G/Y component)</b>  | 0– (2 <sup>n</sup> Bits per color sample)-1     | Only if <b>Memory-mapped control interface</b> is off. Sets the value the IP uses for the G background color plane if the output is RGB, or the Y color plane if the output is YCbCr.                                                                                                                                                                                                                                                                                                        |
| <b>Fixed clock background color (R/Cr component)</b> | 0– (2 <sup>n</sup> Bits per color sample)-1     | Only if <b>Memory-mapped control interface</b> is off. Select the value the IP uses for the R background color plane if the output is RGB, or the Cr color plane if the output is YCbCr.                                                                                                                                                                                                                                                                                                     |
| <b>Fixed clock font color (B/Cb component)</b>       | 0– (2 <sup>n</sup> Bits per color sample)-1     | Only if <b>Memory-mapped control interface</b> is off. Select the value the IP uses for the B font color plane if the output is RGB, or the Cb color plane if the output is YCbCr.                                                                                                                                                                                                                                                                                                           |
| <i>continued...</i>                                  |                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |

| Parameter                                                                                                                                                                                                                                    | Value                                                                    | Description                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Fixed clock font color (G/Y component)</b>                                                                                                                                                                                                | 0– (2 <sup>^</sup> Bits per color sample)-1                              | Only if <b>Memory-mapped control interface</b> is off. Select the value the IP uses for the G font color plane if the output is RGB, or the Y color plane if the output is YCbCr.        |
| <b>Fixed clock font color (R/Cr component)</b>                                                                                                                                                                                               | 0– (2 <sup>^</sup> Bits per color sample)-1                              | Only if <b>Memory-mapped control interface</b> is off. Select the value the IP uses for the R font color plane if the output is RGB, or the Cr color plane if the output is YcbCr.       |
| <b>Fixed clock location x-coordinate</b>                                                                                                                                                                                                     | 0 to 16383                                                               | Only if <b>Memory-mapped control interface</b> is off. Select the x-location of the top-left pixel of the digital clock.                                                                 |
| <b>Fixed clock location y-coordinate</b>                                                                                                                                                                                                     | 0 to 16383                                                               | Only if <b>Memory-mapped control interface</b> is off. Select the y-location of the top-left pixel of the digital clock.                                                                 |
| <b>Fixed clock scale factor</b>                                                                                                                                                                                                              | 1 to 65535                                                               | Only if <b>Memory-mapped control interface</b> is off and you select the <b>digital clock</b> test pattern. Select the multiplying coefficient to the original digital clock dimensions. |
| <b>Fixed clock FPS</b>                                                                                                                                                                                                                       | 1 to 60                                                                  | Only if <b>Memory-mapped control interface</b> is off. Select the number of frames until one second is reached. This parameter does not affect the output video refresh rate).           |
| <b>Number of test patterns</b>                                                                                                                                                                                                               | 1 to 8                                                                   | Set the number of test pattern configurations to turn on.                                                                                                                                |
| <b>Core 0 test pattern</b><br><b>Core 1 test pattern</b><br><b>Core 2 test pattern</b><br><b>Core 3 test pattern</b><br><b>Core 4 test pattern</b><br><b>Core 5 test pattern</b><br><b>Core 6 test pattern</b><br><b>Core 7 test pattern</b> | <b>Bars, Constant color, SDI pathological, Zone plate, Digital clock</b> | Selects the test pattern for each configuration.                                                                                                                                         |
| <b>Core 0 color space</b><br><b>Core 1 color space</b><br><b>Core 2 color space</b><br><b>Core 3 color space</b><br><b>Core 4 color space</b><br><b>Core 5 color space</b><br><b>Core 6 color space</b><br><b>Core 7 color space</b>         | <b>RGB, YCbCr 444, YCbCr 422, YCbCr 420, MONO</b>                        | Sets the output color space and chroma sampling for each test pattern configuration.                                                                                                     |

### 44.3. Test Pattern Generator IP Functional Description

The Test Pattern Generator IP enables you to choose between different test patterns, either at run time or compile time.



### 44.3.1. Test Patterns

The IP offers the following patterns:

- Bars pattern
  - ☐ Color bars
  - ☐ Grayscale bars
  - ☐ Black and white bars
  - ☐ Mixed bars
- Constant color
- SDI pathological
- Zone plate
- Digital clock

#### Bars pattern

If you select Bars Pattern, the IP produces a test pattern with 8 vertical bars, each covering approximately 1/8th of each output video line. The bars contain either a sequence of changing colors (color bars), a sequence of greyscale tones with decreasing brightness (greyscale bars), or a sequence of alternating black and white bars. Mixed bars change the pattern in the vertical direction, with the first 1/8th of the lines showing the black and white bars, the next 1/8th showing the greyscale bars, and the final  $\frac{3}{4}$  showing the color bars. If you select bars pattern (and turn on the Avalon memory-mapped control agent interface), a value in the register map selects at run time which variant of the bars pattern to display. If you do not turn on **Avalon memory-mapped control agent**, the **fixed bars mode** parameter controls which variant of the bars pattern to display, and you cannot change it at run time.

For all test patterns, each bar is approximately 1/8th of the width of the output frame, but this width is not exact. The actual width of each color bar is affected by the field width and the horizontal subsampling.

When the width of the field is not divisible by eight, the remainder pixels from the division are spread as evenly as possible across the bars. The first bar, on the left-hand side of the field, always has  $\text{floor}(\text{field\_width}/8)$  pixels. A single remainder pixel is added to the width of each subsequent bar until all the remainder pixels are exhausted. The width of each bar cannot be less than the number of pixels transmitted per beat at the output interface (*pixels in parallel*). If the field width is set to less than  $8 \times \text{pixels in parallel}$ , as many bars as possible display with *pixels in parallel* pixels per bar, but fewer than 8 bars display. When the output is horizontally subsampled (4:2:2 or 4:2:0), the pixel-width of each color bar is a multiple of two and we alter the methods described above to ensure this is always the case.

The tables define the values of the color components in each bar for each of the three basic modes. The values are the actual output values if you set bits per color plane to 8 bits. If bits per color is greater than 8 bits, the internal logic upshifts the value by the required number of bits, with zeros added in the LSBs.

**Table 830. Output values for each color plane in both the RGB and YCbCr color spaces for the color bars test pattern**

| Color         | RGB             | YCbCr           |
|---------------|-----------------|-----------------|
| White (left)  | (180, 180, 180) | (180, 128, 128) |
| Yellow        | (180, 180, 16)  | (162, 44, 142)  |
| Cyan          | (16, 180, 180)  | (131, 156, 44)  |
| Green         | (16, 180, 16)   | (112, 72, 58)   |
| Magenta       | (180, 16, 180)  | (84, 184, 198)  |
| Red           | (180, 16, 16)   | (65, 100, 212)  |
| Blue          | (16, 16, 180)   | (35, 212, 114)  |
| Black (right) | (16, 16, 16)    | (16, 128, 128)  |

**Table 831. Output values for each color plane in both the RGB and YCbCr color spaces for the greyscale bars test pattern**

| Color     | RGB             | YCbCr           |
|-----------|-----------------|-----------------|
| 0 (left)  | (180, 180, 180) | (180, 128, 128) |
| 1         | (162, 162, 162) | (162, 128, 128) |
| 2         | (131, 131, 131) | (131, 128, 128) |
| 3         | (112, 112, 112) | (112, 128, 128) |
| 4         | (84, 84, 84)    | (84, 128, 128)  |
| 5         | (65, 65, 65)    | (65, 128, 128)  |
| 6         | (35, 35, 35)    | (35, 128, 128)  |
| 7 (right) | (16, 16, 16)    | (16, 128, 128)  |

**Table 832. Output values for each color plane in both the RGB and YCbCr color spaces for the black and white bars test pattern**

| Color     | RGB             | YCbCr           |
|-----------|-----------------|-----------------|
| 0 (left)  | (180, 180, 180) | (180, 128, 128) |
| 1         | (16, 16, 16)    | (16, 128, 128)  |
| 2         | (180, 180, 180) | (180, 128, 128) |
| 3         | (16, 16, 16)    | (16, 128, 128)  |
| 4         | (180, 180, 180) | (180, 128, 128) |
| 5         | (16, 16, 16)    | (16, 128, 128)  |
| 6         | (180, 180, 180) | (180, 128, 128) |
| 7 (right) | (16, 16, 16)    | (16, 128, 128)  |

### Constant color

The constant color test pattern is a complete field or frame of a constant color. This test pattern has limited value for testing, but you can use the pattern to form a background layer for a mixer. Set the RGB or YCbCr values for the desired color as either fixed, compile-time-set parameters or run-time controlled values through the Avalon memory-mapped control agent interface.

### SDI Pathological

The SDI pathological test pattern is specifically designed to stress test the SDI equalizer and PLL performance. The test pattern consists of a static test image with the top half of the lines filled with a shade of magenta, and the bottom half of the lines filled with a shade of gray.

### Zone plate

The zone plate test pattern is an arbitrarily sized, arbitrarily placed set of concentric rings that get wider and narrower the further from the **origin**. If **Avalon memory-mapped control agent** is off, the origin is always at the center of the display. If **Avalon memory-mapped control agent** is on, you can configure the origin by writing to the `ZONE_X_ORIGIN` and `ZONE_Y_ORIGIN` register map values.

Set the `Coarse scaling coefficient` bits in the `ZONE_POWER_FACTOR` register to set the initial size of the zone plate so that it is visible with the current resolution. Set the `fine tune scaling coefficient` bit in the `ZONE_FINE_FACTOR` register for incremental changes to the size that smoothly resize the plate. If **Avalon memory-mapped control agent** is on, you can configure these values by writing to the respective registers, otherwise these values are fixed at compile time.

The following equation shows how the `Coarse scaling coefficient`,  $c$  and the `fine tune scaling coefficient`,  $k$ , relate to the zone plate, where  $r$  is the radius away from the origin.

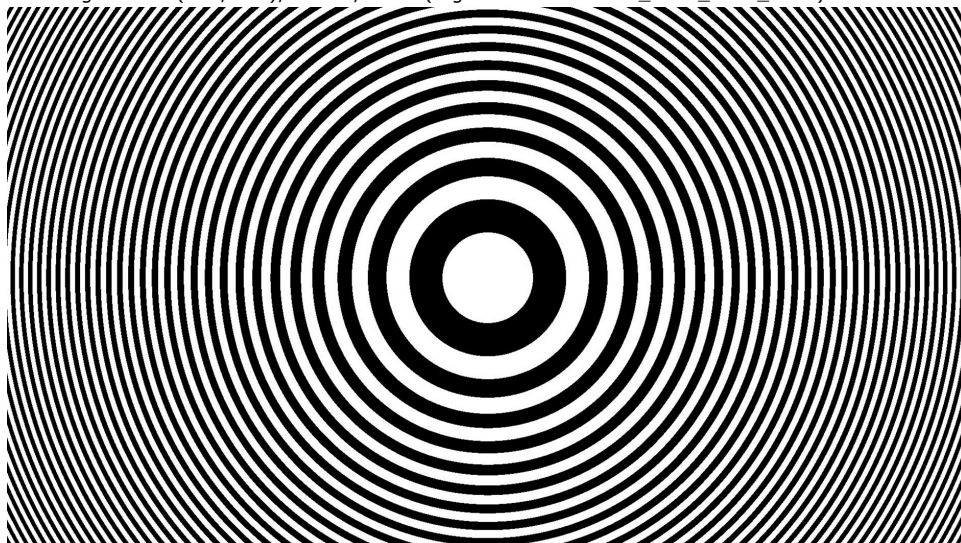
#### Equation 10. Coarse and Fine scaling Equation

$$\text{pixel} = \begin{cases} \text{black} & \cos((r^2 \gg c) \times k \times \pi) \geq 0 \\ \text{white} & \text{else} \end{cases}$$

Because of the position of  $k$  in the equation, increasing  $k$  causes a decrease in the size of the zone plate (decreased zoom). Decreasing  $k$  causes an increase in the size of the zone plate (increased zoom).

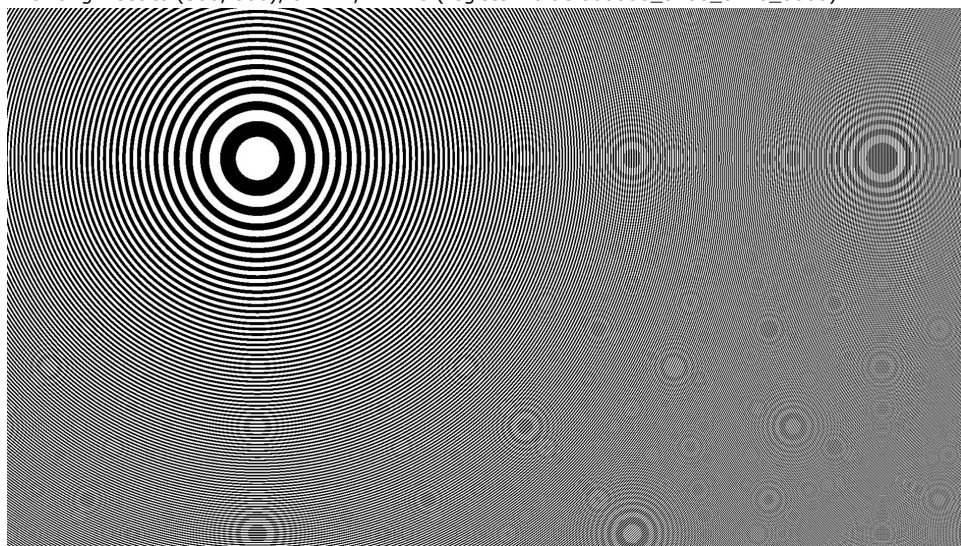
**Figure 122. Example of zone plate on a 1920x1080 screen**

With origin set to (960, 540),  $c = 14$ ,  $k = 1$  (register value 0b0000\_0001\_0000\_0000)



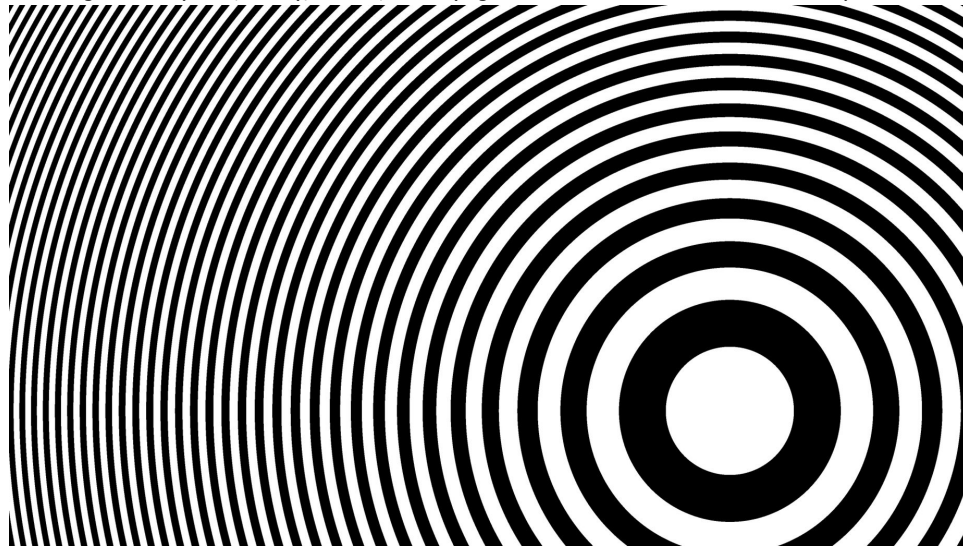
**Figure 123. Example of zone plate on a 1920x1080 screen**

With origin set to (500, 300),  $c = 14$ ,  $k = 10$  (register value 0b0000\_0100\_0110\_0000)



**Figure 124. Example of zone plate on a 3840x2160 screen**

With origin set to (2880, 1620),  $c = 17$ ,  $k = 1$  (register value 0b0000\_0100\_0110\_0000)

**Digital clock**

The digital clock pattern consists of a digital display with format HH:MM:SS,FC, where FC is field count. The time is not precise but is a function of frames per second and field count.

You can customize the size of the display. The original dimensions of the display are 88x8 pixels. If **Avalon memory-mapped control agent** is on, the value of **Fixed clock scale factor** is a scalar multiple to both dimensions. If **Avalon memory-mapped control agent** is off, the value of the `Scale factor` bit in the `CLOCK_SCALE_FACTOR` register is a scalar multiple to both dimensions.

Set the background or font color and location at either compile time if **Avalon memory-mapped control agent** is off, or at run time via the register map.

**44.3.2. Output Subsampling and Color Space**

You can configure the Test Pattern gNerator IP output subsampling and color space to either be fixed and set at compile time, or to be variable at run time using the Avalon memory-mapped control agent interface.

The **Output Format** parameter sets the subsampling option:

- 4:4:4. Output is fixed at full sampling for each color plane (can be RGB or YCbCr). Each pixel has 3 color planes.
- 4:2:2. Output is fixed at full sampling on the Y plane and horizontal subsampling on the Cb and Cr planes (RGB data is not supported in 4:2:2 mode). Each pixel has 2 color planes.
- 4:2:0. Output is fixed at full sampling on the Y plane and horizontal and vertical subsampling on the Cb and Cr planes (RGB data is not supported in 4:2:0 mode). Each pixel has 3 color planes.
- Monochrome. Output has only 1 color plane per pixel and represents only a fully sampled Y plane.
- Variable. Output can be configured at run time to be 4:4:4 RGB, 4:4:4 YCbCr, 4:2:2 YCbCr, or 4:2:0 YCbCr. Each pixel has 3 color planes.

The IP allows you to select up to 8 different test pattern configurations to switch between at run time:

- Each configuration is a combination of 1 of the 3 available patterns and a formatting option.
- Each test pattern configuration is fixed in one particular output format. However, by selecting between the patterns, you can vary the overall output format at run time.

For example, you may include 4 different configurations, all of which use the bars pattern, but with the first configuration set to 4:4:4 RGB, the second set to 4:4:4 YCbCr, the third set to 4:2:2 YCbCr, and the fourth set to 4:2:0 YCbCr. With this setting, you can validate all the possible configurations of an HDMI 2.0 output.

### 44.3.3. Run-time Settings

If you turn on the Avalon memory-mapped control agent interface, the resolution, interlace settings and test pattern configuration can be adjusted at run time via the register map. Changing settings at run time differs depending on whether the test pattern generator is parameterized to output the full or lite variant of the Altera Streaming Video Protocol.

#### Full Variants

When you turn off **Lite mode**, you can edit any of the test pattern generator settings at any time. The IP does not apply the settings until the start of the next frame after a write to the COMMIT register (address 0x014C). The IP can complete a change to the settings that requires writes to multiple registers without the risk of applying the earlier writes in the sequence one frame before the later writes in the sequence.



The full sequence for register updates for full variants is:

1. Make the required edits to any subset of the test pattern generator settings in the register map (addresses 0x120 to 0x0128 and addresses 0x0150 and 0x015C to 0x0168).
2. After the first write to update a setting, the IP asserts bit 1 of the STATUS register (address 0x0140) in the return data for any read to this address.
3. Write any value to the COMMIT register (address 0x014C) to commit the changes as a coherent set.
4. Do not make any further edits to the settings until the IP deasserts bit 1 of the STATUS register at the next field boundary after the write to the COMMIT register. If the IP has any updates to the test pattern settings (address 0x0120 to 0x0128, and 0x0150), at this point the field index is reset to 0. The field index is a count of the number of frames since the last update. The interlace sequence is reset to start at F0 or F1, as specified in the register map.
5. When the IP deasserts bit 1 of the STATUS register, you may make further settings updates.

The COMMIT mode register gates the application of updates to all registers except the CONTROL register (address 0x0148). The CONTROL register allows you to stop and start the IP output at frame boundaries. The IP applies updates to the CONTROL register at the next field boundary, regardless of whether you write to the COMMIT register.

#### Lite mode

If you turn on **Lite mode** the IP has no COMMIT register functionality. The recommended flow for updating the test pattern generator settings is:

1. Write 0 to the CONTROL register (address 0x0148) to stop the test pattern generator at the next field boundary.
2. Read the STATUS register (address 0x0140) until bit 0 is de-asserted, indicating that the test pattern generator is in an idle state between fields.
3. Make the required edits to any subset of the test pattern generator settings in the register map (addresses 0x120 to 0x0128 and addresses 0x0150 and 0x015C to 0x0168).
4. Write 1 to the CONTROL register to restart the test pattern generator with the new settings. If the IP has any updates to the test pattern settings (address 0x0120 to 0x0128, and 0x0150), at this point the field index is reset to 0. The interlace sequence is reset to start at F0 or F1, as specified in the register map.

### 44.3.4. Test Pattern Generator IP Interfaces

| Name                     | Direction | Width | Description                                                                                                                  |
|--------------------------|-----------|-------|------------------------------------------------------------------------------------------------------------------------------|
| <b>Clocks and resets</b> |           |       |                                                                                                                              |
| main_clock_clk           | In        | 1     | AXI4-S processing clock.                                                                                                     |
| main_reset_rst           | In        | 1     | AXI4-S processing reset.                                                                                                     |
| agent_clock_clk          | In        | 1     | Clock for the Avalon memory-mapped control agent interface. Only if you select <b>separate clock for control interface</b> . |
| <i>continued...</i>      |           |       |                                                                                                                              |

| Name                                     | Direction | Width | Description                                                                                                                                |
|------------------------------------------|-----------|-------|--------------------------------------------------------------------------------------------------------------------------------------------|
| agent_reset_rst                          | In        | 1     | Reset for the Avalon memory-mapped control agent interface. Only if you select <b>separate clock for control interface</b> .               |
| <b>Control interfaces</b>                |           |       |                                                                                                                                            |
| av_mm_control_agent_address              | In        | 7     | Avalon memory-mapped agent address .                                                                                                       |
| av_mm_control_agent_write                | In        | 1     | Avalon memory-mapped agent write.                                                                                                          |
| av_mm_control_agent_writedata            | In        | 32    | Avalon memory-mapped agent write data.                                                                                                     |
| av_mm_control_agent_byteenable           | In        | 4     | Avalon memory-mapped agent byte enable.                                                                                                    |
| av_mm_control_agent_read                 | In        | 1     | Avalon memory-mapped agent read.                                                                                                           |
| av_mm_control_agent_readdata             | Out       | 32    | Avalon memory-mapped agent read data.                                                                                                      |
| av_mm_control_agent_readdatavalid        | Out       | 1     | Avalon memory-mapped agent read.                                                                                                           |
| av_mm_control_agent_waitrequest          | Out       | 1     | Avalon memory-mapped agent wait request.                                                                                                   |
| <b>Altera Streaming Video interfaces</b> |           |       |                                                                                                                                            |
| axi4s_vid_out_tdata                      | Out       | (134) | AXI4-S data in.                                                                                                                            |
| axi4s_vid_out_tvalid                     | Out       | 1     | AXI4-S data valid.                                                                                                                         |
| axi4s_vid_in_tuser                       | Out       | (135) | AXI4-S tuser<br>tuser[0] indicates start of video frame when asserted<br>tuser[1] indicates the start of a non-video packet when asserted. |
| axi4s_vid_out_tlast                      | Out       | 1     | AXI4-S end of packet .                                                                                                                     |
| axi4s_vid_out_tready                     | In        | 1     | AXI4-S data ready.                                                                                                                         |

### Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

## 44.4. Test Pattern Generator IP Registers

Each register is either read-only (RO) or read-write (RW).

**Table 833. Test Pattern Generator IP Registers**

In the software API these register names appear with a prefix of INTEL\_VVP, INTEL\_VVP\_CORE or INTEL\_VVP\_TPG as appropriate and with an optional REG suffix

| Address                           | Register | Access | Description                                                           |
|-----------------------------------|----------|--------|-----------------------------------------------------------------------|
| <b>Parameterization registers</b> |          |        |                                                                       |
| 0x0000                            | VID_PID  | RO     | Read this register to retrieve the test pattern generator product ID. |
| <i>continued...</i>               |          |        |                                                                       |

(134) The equation gives all tdata widths in these interfaces:  

$$\max(\text{ceil}((\text{bits per color sample} \times \text{number of color planes} \times \text{pixels in parallel}) / 8) \times 8, 16)$$

(135) The equation gives all tuser widths in these interfaces  

$$N = \text{ceil}(\text{tdata width} / 8)$$



| Address      | Register        | Access | Description                                                                                                                                                                                                  |
|--------------|-----------------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|              |                 |        | This register always returns 0x6AF7_0236.                                                                                                                                                                    |
| 0x0004       | VERSION         | RO     | Read this register for the IP version informationP.                                                                                                                                                          |
| 0x0008       | LITE_MODE       | RO     | Read this register to determine if <b>lite mode</b> is on. This register returns 0 if <b>lite mode</b> is off and 1 if <b>Lite mode</b> is on.                                                               |
| 0x000C       | DEBUG_ENABLED   | RO     | Read this register to determine if <b>debug features</b> is on. This register returns 1 if reads to other registers designated as RW return the last value you write to the register, or an undefined value. |
| 0x0010       | NUM_PATTERNS    | RO     | Read this register to determine the number of test pattern configurations.                                                                                                                                   |
| 0x0014       | Reserved        | RO     | Reserved for future use.                                                                                                                                                                                     |
| 0x0018       | BPS             | RO     | Read this register for the number of bits per color plane.                                                                                                                                                   |
| 0x001C       | PIP             | RO     | Read this register for the number of pixels transmitted per clock cycle at the streaming output                                                                                                              |
| 0x0020       | PATTERN_0_TYPE  | RO     | Read this register for the pattern type of test pattern configuration 0. This register returns 0 for the <b>bars</b> pattern, 1 for <b>constant color</b> and 2 for <b>SDI pathological</b> .                |
| 0x0024       | PATTERN_0_COLOR | RO     | Read this register for the color space and chroma sampling for test pattern configuration 0. This register returns 0 for RGB, 1 for YCbCr 4:4:4, 2 for YCbCr 4:2:2, 3 for YCbCr 4:2:0 and 4 for monochrome.  |
| 0x0028       | PATTERN_1_TYPE  | RO     | Read this register for the pattern type of test pattern configuration 1. This register returns 0 for the <b>bars</b> pattern, 1 for <b>constant color</b> and 2 for <b>SDI pathological</b> .                |
| 0x002C       | PATTERN_1_COLOR | RO     | Read this register for the color space and chroma sampling for test pattern configuration 1. This register returns 0 for RGB, 1 for YCbCr 4:4:4, 2 for YCbCr 4:2:2, 3 for YCbCr 4:2:0 and 4 for monochrome.  |
| 0x0030       | PATTERN_2_TYPE  | RO     | Read this register for the pattern type of test pattern configuration 2. This register returns 0 for the <b>bars</b> pattern, 1 for <b>constant color</b> and 2 for <b>SDI pathological</b> .                |
| 0x0034       | PATTERN_2_COLOR | RO     | Read this register for the color space and chroma sampling for test pattern configuration 2. This register returns 0 for RGB, 1 for YCbCr 4:4:4, 2 for YCbCr 4:2:2, 3 for YCbCr 4:2:0 and 4 for monochrome.  |
| 0x0038       | PATTERN_3_TYPE  | RO     | Read this register for the pattern type of test pattern configuration 3. This register returns 0 for the <b>bars</b> pattern, 1 for <b>constant color</b> and 2 for <b>SDI pathological</b> .                |
| 0x003C       | PATTERN_3_COLOR | RO     | Read this register for the color space and chroma sampling for test pattern configuration 3. This register returns 0 for RGB, 1 for YCbCr 4:4:4, 2 for YCbCr 4:2:2, 3 for YCbCr 4:2:0 and 4 for monochrome.  |
| 0x0040       | PATTERN_4_TYPE  | RO     | Read this register for the pattern type of test pattern configuration 4. This register returns 0 for the <b>bars</b> pattern, 1 for <b>constant color</b> and 2 for <b>SDI pathological</b> .                |
| 0x0044       | PATTERN_4_COLOR | RO     | Read this register for the color space and chroma sampling for test pattern configuration 4. This register returns 0 for RGB, 1 for YCbCr 4:4:4, 2 for YCbCr 4:2:2, 3 for YCbCr 4:2:0 and 4 for monochrome.  |
| 0x0048       | PATTERN_5_TYPE  | RO     | Read this register for the pattern type of test pattern configuration 5. This register returns 0 for the <b>bars</b> pattern, 1 for <b>constant color</b> and 2 for <b>SDI pathological</b> .                |
| 0x004C       | PATTERN_5_COLOR | RO     | Read this register for the color space and chroma sampling for test pattern configuration 5. This register returns 0 for RGB, 1 for YCbCr 4:4:4, 2 for YCbCr 4:2:2, 3 for YCbCr 4:2:0 and 4 for monochrome.  |
| continued... |                 |        |                                                                                                                                                                                                              |

| Address                                                                                                | Register           | Access | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------------------------------------------------------------------------------------------|--------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x0050                                                                                                 | PATTERN_6_TYPE     | RO     | Read this register for the pattern type of test pattern configuration 6. This register returns 0 for the <b>bars</b> pattern, 1 for <b>constant color</b> and 2 for <b>SDI pathological</b> .                                                                                                                                                                                                                                                                                                                                                                 |
| 0x0054                                                                                                 | PATTERN_6_COLOR    | RO     | Read this register for the color space and chroma sampling for test pattern configuration 6. This register returns 0 for RGB, 1 for YCbCr 4:4:4, 2 for YCbCr 4:2:2, 3 for YCbCr 4:2:0 and 4 for monochrome.                                                                                                                                                                                                                                                                                                                                                   |
| 0x0058                                                                                                 | PATTERN_7_TYPE     | RO     | Read this register for the pattern type of test pattern configuration 7. This register returns 0 for the <b>bars</b> pattern, 1 for <b>constant color</b> and 2 for <b>SDI pathological</b> .                                                                                                                                                                                                                                                                                                                                                                 |
| 0x005C                                                                                                 | PATTERN_7_COLOR    | RO     | Read this register for the color space and chroma sampling for test pattern configuration 7. This register returns 0 for RGB, 1 for YCbCr 4:4:4, 2 for YCbCr 4:2:2, 3 for YCbCr 4:2:0 and 4 for monochrome.                                                                                                                                                                                                                                                                                                                                                   |
| 0x0060 to 0x011F                                                                                       | -                  | -      | Unused.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Control and debug registers</b><br>For more information, refer to <i>Control Packets</i> .<br>(136) |                    |        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 0x0120                                                                                                 | IMG_INFO_WIDTH     | RW     | Use this register to set the width of outgoing video fields. If the output chroma sampling is 4:2:2 or 4:2:0, the width should be a multiple of 2.                                                                                                                                                                                                                                                                                                                                                                                                            |
| 0x0124                                                                                                 | IMG_INFO_HEIGHT    | RW     | Use this register to set the height for outgoing video fields. If the output chroma sampling is 4:2:0, the height should be a multiple of 2.                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 0x0128                                                                                                 | IMG_INFO_INTERLACE | RW     | Use this register to set the interlace output behavior for the outgoing fields. The value you write to this register corresponds to the value that is output in the interlace identifier nibble of the outgoing image info packets (full variants only). Values 0-7 produce progressive output, values 8-15 produce interlaced output. The interlaced sequence is restarted at F0 or F1 a change to the test pattern settings (address 0x0120 to 0x0128, and 0x0150). Values 8-11 in this register restart the sequence with F0, values 12-15 restart with F1 |
| 0x012C to 0x013C                                                                                       | -                  | -      | Unused                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 0x0140                                                                                                 | STATUS             | RO     | Bit 0: Status bit.<br>1 = test pattern generator is processing a video field, 0 otherwise.<br>Lite mode off:<br>Bit 1: Pending register updates bit.<br>Any writes to the settings register (0x0148 - 0x0154) cause the IP to raise the pending register updates bit, to indicate outstanding changes to the clipping settings.<br>The IP lowers this bit at the next field boundary after a write to the COMMIT register.                                                                                                                                    |
| 0x0144                                                                                                 | FIELD_COUNT        | RO     | Read this register to determine the output field index. The field index is reset to 0 following a change to the test pattern settings (address 0x0120 to 0x0128, and 0x0150). The field index increments at the end of each output field.                                                                                                                                                                                                                                                                                                                     |
| 0x0148                                                                                                 | CONTROL            | RW     | Bit 0: Go bit.<br>Write 0 to this register to stop the test pattern generator at the end of the current frame. Write 1 to this register to start/restart output generation. This register initializes to 0 at start-up and no output fields are generated until after a write of 1 to this register.                                                                                                                                                                                                                                                          |
| continued...                                                                                           |                    |        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

(136) When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO.

| Address      | Register           | Access | Description                                                                                                                                                                                                                                                                                     |
|--------------|--------------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x014C       | COMMIT             | RW     | Only if Lite mode is off. The IP holds any changes to the test pattern settings via the register map pending until you issue a write to this register. The value you write is unimportant.                                                                                                      |
| 0x0150       | PATTERN_SELECT     | RW     | Write to this register to select the pattern configuration used to generate the output. The value you write is the test pattern configuration index.                                                                                                                                            |
| 0x0154       | Reserved           | RW     | Reserved.                                                                                                                                                                                                                                                                                       |
| 0x0158       | Reserved           | RW     | Reserved.                                                                                                                                                                                                                                                                                       |
| 0x015C       | C0                 | RW     | Write to this register to set the value used for color plane 0 (B or Cb) by the constant color test pattern. The value in this register is ignored if no test pattern configurations use the constant color pattern.                                                                            |
| 0x0160       | C1                 | RW     | Write to this register to set the value used for color plane 1 (G or Y) by the constant color test pattern. The value in this register is ignored if no test pattern configurations use the constant color pattern.                                                                             |
| 0x0164       | C2                 | RW     | Write to this register to set the value used for color plane 2 (R or Cr) by the constant color test pattern. The value in this register is ignored if no test pattern configurations use the constant color pattern.                                                                            |
| 0x0168       | BARS_SELECT        | RW     | Write to this register to set the variant of the bars pattern to use. Write 0 for color bars, 1 for greyscale bars, 2 for black and white bars and 3 for mixed bars. The value in this register is unused if no test pattern configurations use the bars pattern.                               |
| 0x016C       | ZONE_X_ORIGIN      | RW     | Write to this register to set the X coordinate of the zone plate center. For example, write 960 to horizontally set the center of the zone plate to the middle of a 1920x1080 display.                                                                                                          |
| 0x0170       | ZONE_Y_ORIGIN      | RW     | Write to this register to set the Y coordinate of the zone plate center. For example, write 540 to vertically set the center of the zone plate to the middle of a 1920x1080 display.                                                                                                            |
| 0x0174       | ZONE_COARSE_FACTOR | RW     | This value is a rough scale up or down of the zone plate. A bigger value gives more zoom in of the zone plate.<br>Altera suggests a value of 14 for 1920x1080, or 20 for a 4k display.                                                                                                          |
| 0x0178       | ZONE_FINE_FACTOR   | RW     | This value is a final fine tune scaling factor. It is a 16-bit unsigned fixed point number, with 8 bits fractional part. The IP implements this value as a constant $k$ in $f(kx)$ and applies it after <code>ZONE_POWER_FACTOR</code> , a bigger value, gives more zoom out of the zone plate. |
| 0x017C       | CLOCK_B_BACKGROUND | RW     | Write to this register to set the value for color plane 0 (B or Cb) by the digital clock pattern.                                                                                                                                                                                               |
| 0x0180       | CLOCK_G_BACKGROUND | RW     | Write to this register to set the value for color plane 1 (G or Y) by the digital clock pattern.                                                                                                                                                                                                |
| 0x0184       | CLOCK_R_BACKGROUND | RW     | Write to this register to set the value for color plane 2 (R or Cr) by the digital clock pattern.                                                                                                                                                                                               |
| 0x0188       | CLOCK_B_FONT       | RW     | Write to this register to set the value for color plane 0 (B or Cb) by the digital clock pattern.                                                                                                                                                                                               |
| 0x018C       | CLOCK_G_FONT       | RW     | Write to this register to set the value for color plane 1 (G or Y) by the digital clock pattern.                                                                                                                                                                                                |
| 0x0190       | CLOCK_R_FONT       | RW     | Write to this register to set the value for color plane 2 (R or Cr) by the clock test pattern as the font color.                                                                                                                                                                                |
| 0x0194       | CLOCK_LOCATION_X   | RW     | Write to this register to set the value for the x-coordinate of the top-left pixel of digital clock pattern.                                                                                                                                                                                    |
| continued... |                    |        |                                                                                                                                                                                                                                                                                                 |

| Address | Register           | Access | Description                                                                                                                                                   |
|---------|--------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0X0198  | CLOCK_LOCATION_Y   | RW     | Write to this register to set the value for the y-coordinate of the top-left pixel of digital clock pattern.                                                  |
| 0X019C  | CLOCK_SCALE_FACTOR | RW     | Write to this register to set the value for scale factor of the font by the digital clock test pattern.                                                       |
| 0X01A0  | CLOCK_FPS          | RW     | Write to this register to set frames per second - 1 for the clock counter by digital clock pattern. This value does not affect the output video refresh rate. |

### Register Bit Descriptions

**Table 834. VID\_PID**

| Name                                             | Bits | Description                                                                                                                                                                                            |
|--------------------------------------------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Test pattern generator version ID and product ID | 31:0 | This register always returns 0x6AF7_0236. <ul style="list-style-type: none"> <li>15:0 is the product ID and always returns 0x0236</li> <li>31:16 is the vendor ID and always returns 0x6AF7</li> </ul> |

**Table 835. VERSION**

| Name                 | Bits  | Description                          |
|----------------------|-------|--------------------------------------|
| Register map version | 7:0   | Register map version.                |
| IP patch revision    | 15:8  | -                                    |
| IP update revision   | 23:16 | Updated when the IP version changes. |
| IP major revision    | 31:24 | Updated when the IP version changes. |

**Table 836. LITE\_MODE**

| Name                           | Bits | Description                   |
|--------------------------------|------|-------------------------------|
| Lite mode parameterization bit | 0    | Returns 1 if lite mode is on. |
| Unused                         | 31:1 | Unused.                       |

**Table 837. DEBUG\_ENABLED**

| Name                                | Bits | Description                        |
|-------------------------------------|------|------------------------------------|
| Debug features parameterization bit | 0    | Returns 1 if debug features is on. |
| Unused                              | 31:1 | Unused.                            |

**Table 838. NUM\_PATTERNS**

| Name                                          | Bits | Description                                                |
|-----------------------------------------------|------|------------------------------------------------------------|
| Number of enabled test pattern configurations | 31:0 | Returns the number of enabled test pattern configurations. |

**Table 839. BPS**

| Name           | Bits | Description                                 |
|----------------|------|---------------------------------------------|
| Bits per color | 31:0 | Returns the number of bits per color plane. |

**Table 840. PIP**

| Name               | Bits | Description                                               |
|--------------------|------|-----------------------------------------------------------|
| Pixels in parallel | 31:0 | Returns the number of pixels transmitter per clock cycle. |

**Table 841. PATTERN\_X\_TYPE**

| Name                                   | Bits | Description                                                                                                 |
|----------------------------------------|------|-------------------------------------------------------------------------------------------------------------|
| Pattern type for configuration index X | 31:0 | Returns 0 for bars, 1 for constant color, 2 for SDI pathological, 3 for zone plate and 4 for digital clock. |

**Table 842. PATTERN\_X\_COLOR**

| Name                                  | Bits | Description                                                                                      |
|---------------------------------------|------|--------------------------------------------------------------------------------------------------|
| Color space for configuration index X | 31:0 | Returns 0 for RGB, 1 for YCbCr 4:4:4, and 2 YCbCr 4:2:2, 3 for YCbCr 4:2:0 and 4 for monochrome. |

**Table 843. IMG\_INFO\_WIDTH**

| Name       | Bits  | Description                           |
|------------|-------|---------------------------------------|
| Width bits | 15:0  | Width (in pixels) of outgoing fields. |
| unused     | 31:16 | Unused.                               |

**Table 844. IMG\_INFO\_HEIGHT**

| Name        | Bits  | Description                            |
|-------------|-------|----------------------------------------|
| Height bits | 15:0  | Height (in pixels) of outgoing fields. |
| unused      | 31:16 | Unused.                                |

**Table 845. IMG\_INFO\_INTERLACE**

| Name             | Bits | Description                                                         |
|------------------|------|---------------------------------------------------------------------|
| Interlace nibble | 3:0  | Standard Altera Streaming Video Image Info packet interlace nibble. |
| unused           | 31:4 | Unused.                                                             |

**Table 846. STATUS**

| Name                         | Bits | Description                                                          |
|------------------------------|------|----------------------------------------------------------------------|
| Status bit                   | 0    | 1 = test pattern generator is processing a video field, 0 otherwise. |
| Pending register updates bit | 1    | 1 = test pattern generator has pending updates, 0 otherwise.         |
| unused                       | 31:2 | Unused.                                                              |

**Table 847. FIELD\_COUNT**

| Name        | Bits  | Description               |
|-------------|-------|---------------------------|
| Field index | 15:0  | Current field index value |
| unused      | 31:16 | Unused.                   |

**Table 848. CONTROL**

| Name   | Bits | Description       |
|--------|------|-------------------|
| Go bit | 0    | 1 = go, 0 = stop. |
| unused | 31:1 | Unused.           |

**Table 849. COMMIT**

| Name   | Bits | Description |
|--------|------|-------------|
| unused | 31:0 | Unused.     |

**Table 850. PATTERN\_SELECT**

| Name           | Bits | Description                  |
|----------------|------|------------------------------|
| Pattern select | 2:0  | Pattern configuration index. |
| unused         | 31:3 | Unused.                      |

**Table 851. CX**

| Name        | Bits    | Description                             |
|-------------|---------|-----------------------------------------|
| Color value | BPS-1:0 | Constant color value for color plane X. |
| unused      | 31:BPS  | Unused.                                 |

**Table 852. BARS\_SELECT**

| Name            | Bits | Description                                                                   |
|-----------------|------|-------------------------------------------------------------------------------|
| Bar mode select | 1:0  | 0 = color bars, 1 = greyscale bars, 2 = black and white bars, 3 = mixed bars. |
| unused          | 31:2 | Unused.                                                                       |

**Table 853. ZONE\_X\_ORIGIN**

| Name     | Bits  | Description                                      |
|----------|-------|--------------------------------------------------|
| Origin X | 15:0  | Sets x part of center coordinates of zone plate. |
| Unused   | 31:16 | Unused                                           |

**Table 854. ZONE\_Y\_ORIGIN**

| Name     | Bits  | Description                                      |
|----------|-------|--------------------------------------------------|
| Origin Y | 15:0  | Sets y part of center coordinates of zone plate. |
| Unused   | 31:16 | Unused                                           |

**Table 855. ZONE\_POWER\_FACTOR**

| Name                       | Bits | Description                                                                                             |
|----------------------------|------|---------------------------------------------------------------------------------------------------------|
| Coarse scaling coefficient | 4:0  | Arbitrary sizing coefficient. Increase this value to increase the zone plate size coarsely (more zoom). |
| Unused                     | 31:5 | Unused                                                                                                  |

**Table 856. ZONE\_FINE\_FACTOR**

| Name                          | Bits  | Description                                                                                             |
|-------------------------------|-------|---------------------------------------------------------------------------------------------------------|
| Fine tune scaling coefficient | 15:0  | Fixed point operand, with 8 bits fractional part. Increase factor to decrease zone plate size smoothly. |
| Unused                        | 31:16 | Unused                                                                                                  |

**Table 857. CLOCK\_B\_BACKGROUND**

| Name                       | Bits  | Description                                                                                |
|----------------------------|-------|--------------------------------------------------------------------------------------------|
| Blue (Cb) background color | 15:0  | Sets the B color plane of the background color if color space is RGB, else is Cb in YCbCr. |
| Unused                     | 31:16 | Unused                                                                                     |

**Table 858. CLOCK\_G\_BACKGROUND**

| Name                       | Bits  | Description                                                                               |
|----------------------------|-------|-------------------------------------------------------------------------------------------|
| Green (G) background color | 15:0  | Sets the G color plane of the background color if color space is RGB, else is Y in YCbCr. |
| Unused                     | 31:16 | Unused                                                                                    |

**Table 859. CLOCK\_R\_BACKGROUND**

| Name                      | Bits  | Description                                                                                |
|---------------------------|-------|--------------------------------------------------------------------------------------------|
| Red (Cr) background color | 15:0  | Sets the R color plane of the background color if color space is RGB, else is Cr in YCbCr. |
| Unused                    | 31:16 | Unused                                                                                     |

**Table 860. CLOCK\_B\_FONT**

| Name                 | Bits  | Description                                                                          |
|----------------------|-------|--------------------------------------------------------------------------------------|
| Blue (Cb) font color | 15:0  | Sets the B color plane of the font color if color space is RGB, else is Cb in YCbCr. |
| Unused               | 31:16 | Unused                                                                               |

**Table 861. CLOCK\_G\_FONT**

| Name                 | Bits  | Description                                                                          |
|----------------------|-------|--------------------------------------------------------------------------------------|
| Blue (Cb) font color | 15:0  | Sets the G color plane of the font color if color space is RGB, else is Cb in YCbCr. |
| Unused               | 31:16 | Unused                                                                               |

**Table 862. CLOCK\_R\_FONT**

| Name                 | Bits  | Description                                                                          |
|----------------------|-------|--------------------------------------------------------------------------------------|
| Blue (Cb) font color | 15:0  | Sets the R color plane of the font color if color space is RGB, else is Cb in YCbCr. |
| Unused               | 31:16 | Unused                                                                               |

**Table 863. CLOCK\_LOCATION\_X**

| Name                  | Bits  | Description                                                           |
|-----------------------|-------|-----------------------------------------------------------------------|
| Location x-coordinate | 15:0  | Sets the the x-coordinate of the top-left pixel of the clock display. |
| Unused                | 31:16 | Unused                                                                |

**Table 864. CLOCK\_LOCATION\_Y**

| Name                  | Bits  | Description                                                            |
|-----------------------|-------|------------------------------------------------------------------------|
| Location y-coordinate | 15:0  | Sets the the y-coordinate of the top-left pixel of the clock display.. |
| Unused                | 31:16 | Unused                                                                 |

**Table 865. CLOCK\_SCALE\_FACTOR**

| Name         | Bits  | Description                                         |
|--------------|-------|-----------------------------------------------------|
| Scale factor | 15:0  | Sets the scale factor of the digital clock pattern. |
| Unused       | 31:16 | Unused                                              |

**Table 866. CLOCK\_FPS**

| Name              | Bits | Description                                                                                                                                                           |
|-------------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frames per second | 5:0  | Sets frames per second – 1, to determine after how many frames a second occur in the digital clock patterns. This value does not affect the output video refresh rate |
| Unused            | 31:6 | Unused                                                                                                                                                                |

## 44.5. Test Pattern Generator IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_tpg_regs.h`

Include file: `intel_vvp_tpg.h`

**Table 867. Test Pattern Generator IP Software API**

| Name                                           | Description                                                                                                                                                                                  |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>intel_vvp_tpg_init</code>                | Initialize the TPG instance                                                                                                                                                                  |
| <code>Intel_vvp_core_*</code>                  | .Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i> . Writable when <b>Lite</b> is on. Readable when <b>Lite</b> is off and <b>Debug features</b> is on. |
| <code>intel_vvp_tpg_get_lite_mode</code>       | Returns if <b>Lite mode</b> is on                                                                                                                                                            |
| <code>intel_vvp_tpg_get_debug_enabled</code>   | Returns if <b>Lite mode</b> is on                                                                                                                                                            |
| <code>intel_vvp_tpg_get_num_patterns</code>    | Returns the number of patterns available                                                                                                                                                     |
| <code>intel_vvp_tpg_get_bits_per_sample</code> | Returns the number of bits in a color sample                                                                                                                                                 |
| <i>continued...</i>                            |                                                                                                                                                                                              |



| Name                                             | Description                                                                                          |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------|
| intel_vvp_tpg_get_pixels_in_parallel             | Returns the number of pixels processed in parallel per clock cycle                                   |
| intel_vvp_tpg_get_pattern_data                   | Returns pattern type and subsampling                                                                 |
| intel_vvp_tpg_is_running                         | Returns if the IP is producing data                                                                  |
| intel_vvp_tpg_get_commit_status                  | Returns if there are uncommitted writes                                                              |
| intel_vvp_tpg_get_status                         | Reads the status register                                                                            |
| intel_vvp_tpg_enable                             | Starts/stops the IP                                                                                  |
| intel_vvp_tpg_start                              | Start the IP                                                                                         |
| intel_vvp_tpg_stop                               | Stops the IP                                                                                         |
| intel_vvp_tpg_is_enabled                         | Returns if the IP is currently enabled                                                               |
| intel_vvp_tpg_set_pattern                        | Select the current output pattern                                                                    |
| intel_vvp_tpg_get_pattern                        | Query the current output pattern                                                                     |
| intel_vvp_tpg_get_fields_since_sequence_reset    | Returns the number of output fields generated since the sequence started                             |
| intel_vvp_tpg_set_bars_type                      | Selects a type of color bars for the bar pattern                                                     |
| intel_vvp_tpg_get_bars_type                      | Returns the current type of color bars                                                               |
| intel_vvp_tpg_set_colors                         | Selects the color for the uniform pattern                                                            |
| intel_vvp_tpg_get_colors                         | Returns the current uniform color                                                                    |
| intel_vvp_tpg_set_zone_plate_origin              | Sets the zone plate origin to the coordinates provided                                               |
| intel_vvp_tpg_get_zone_plate_origin              | Returns the zone plate center coordinates                                                            |
| intel_vvp_tpg_set_zone_plate_factors             | Sets the coarse and fine tune scaling coefficients of the zone plate                                 |
| intel_vvp_tpg_get_zone_plate_factors             | Returns the coarse and fine tune coefficients of the zone plate                                      |
| intel_vvp_tpg_set_digital_clock_background_color | Sets the background color for the digital clock pattern.                                             |
| Intel_vvp_tpg_get_digital_clock_background_color | Returns the background color for the digital clock pattern.                                          |
| intel_vvp_tpg_set_digital_clock_font_color       | Sets the font color for the digital clock pattern.                                                   |
| intel_vvp_tpg_get_digital_clock_font_color       | Returns the font color for the digital clock pattern.                                                |
| intel_vvp_tpg_set_digital_clock_location         | Sets the location for the digital clock pattern.                                                     |
| intel_vvp_tpg_get_digital_clock_location         | Returns the location for the digital clock pattern.                                                  |
| intel_vvp_tpg_set_digital_clock_scale_factor     | Sets the scale factor for the digital clock pattern                                                  |
| intel_vvp_tpg_get_digital_clock_scale_factor     | Returns the scale factor for the digital clock pattern                                               |
| intel_vvp_tpg_set_digital_clock_fps              | Sets the FPS for the digital clock pattern. This value does not affect the output video refresh rate |
| intel_vvp_tpg_get_digital_clock_fps              | Returns the FPS for the digital clock pattern.                                                       |
| intel_vvp_tpg_commit_writes                      | Commit all outstanding writes                                                                        |

#### enum eIntelVvpTpgPatternType

- Members**
- kIntelVvpTpgBarsPattern – Bar pattern
  - kIntelVvpTpgUniformPattern – Uniform pattern
  - kIntelVvpTpgPathologicalPattern – Pattern that stresses SDI receiving equipment
  - kIntelVvpTpgZonePlatePattern – zone plate pattern
  - kIntelVvpTpgDigitalClockPattern – digital clock pattern
  - kIntelVvpTpgInvalidPattern – Invalid pattern (to indicate errors)

**Description** Enumeration for the supported pattern types

#### enum eIntelVvpTpgPatternColor

- Members**
- kIntelVvpTpgRgb – RGB 4:4:4
  - kIntelVvpTpgYcc444 – YCbCr/YUV 4:4:4
  - kIntelVvpTpgYcc422 – YCbCr/YUV 4:2:2
  - kIntelVvpTpgYcc420 – YCbCr/YUV 4:2:0
  - kIntelVvpTpgMono – Monochrome
  - kIntelVvpTpgInvalidColor – Invalid colorspace/subsampling (used to indicate errors)

**Description** Enumeration for the supported color spaces and subsampling

#### enum eIntelVvpTpgBarsType

- Members**
- kIntelVvpTpgColorBars – Standard color bars
  - kIntelVvpTpgGreyBars – Grey bars
  - kIntelVvpTpgBlackWhiteBars – Alternating black and white bars
  - kIntelVvpTpgMixedBars – Mixed bars types
  - kIntelVvpTpgInvalidBars – Invalid bar pattern (used to indicate errors)

**Description** Enumeration for the supported color bar patterns

#### struct intel\_vvp\_tpg\_pattern

- Members**
- eIntelVvpTpgPatternType type – the configured type for the pattern

eIntelVvpTpgPatternColor color – The configured color space for the pattern

**Description** Structure used to return the compile-time configuration for a given test pattern

### intel\_vvp\_tpg\_init

**Prototype**

```
int intel_vvp_tpg_init(intel_vvp_tpg_instance *instance,
intel_vvp_core_base base);
```

**Description** Initializes a TPG instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance does not fully initialize and should not be used further by the application if returning a non-zero error code.

**Arguments** instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

base – base address of the register map

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product\_id does not match with the TPG product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpTpgRegMapVersionErr if the register map is not supported

### intel\_vvp\_tpg\_get\_lite\_mode

**Prototype**

```
bool intel_vvp_tpg_get_lite_mode(intel_vvp_tpg_instance
*instance);
```

**Description** Returns the value of the LITE\_MODE register. The instance must be a valid intel\_vvp\_tpg\_instance fully initialized

**Arguments** instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

**Returns** true if the IP is parameterized in lite mode

### intel\_vvp\_tpg\_get\_debug\_enabled

**Prototype**

```
int intel_vvp_tpg_get_debug_enabled(intel_vvp_tpg_instance*instance);
```

**Description** Returns the value of the DEBUG\_ENABLED register. The instance must be a valid intel\_vvp\_tpg\_instance fully initialized.

**Arguments** instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

**Returns** true if the IP is parameterized with debug features enabled

### intel\_vvp\_tpg\_get\_num\_patterns

**Prototype**

```
uint8_t
intel_vvp_tpg_get_num_patterns(intel_vvp_tpg_instance*instance);
```

**Description** Returns the value of the NUM\_PATTERNS register. The instance must be a valid intel\_vvp\_tpg\_instance fully initialized.

**Arguments** instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

**Returns** The number of patterns available

### intel\_vvp\_tpg\_get\_bits\_per\_sample

**Prototype**

```
uint8_t
intel_vvp_tpg_get_bits_per_sample(intel_vvp_tpg_instance*instance);
```

**Description** Returns the value of the BPS register. The instance must be a valid intel\_vvp\_tpg\_instance fully initialized.

**Arguments** instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

**Returns** The compile-time bps parameter

### intel\_vvp\_tpg\_get\_pixels\_in\_parallel

**Prototype**

```
uint8_t
intel_vvp_tpg_get_pixels_in_parallel(intel_vvp_tpg_instance*instance);
```

**Description** Returns the value of the PIP register. The instance must be a valid intel\_vvp\_tpg\_instance fully initialized.

- Arguments* instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure
- Returns* The compile-time pixels in parallel parameter. This is the number of pixels generated per clock cycle.

#### intel\_vvp\_tpg\_get\_pattern\_data

*Prototype*

```
int intel_vvp_tpg_get_pattern_data(intel_vvp_tpg_instance*
instance, uint8_t pattern, intel_vvp_tpg_pattern*
pattern_data);
```

- Description* Returns the pattern configuration for a given pattern. The instance must be a valid intel\_vvp\_tpg\_instance fully initialized. The selected pattern must be valid and pattern\_data must point to a valid memory location to store the result.

- Arguments* instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure
- pattern – a valid pattern in the range [0..intel\_vvp\_get\_num\_patterns(instance)-1]
- pattern\_data – valid memory location for a intel\_vvp\_tpg\_pattern structure

- Returns* kIntelVvpCoreOk (0) in case of succes, a negative error code in case of error
- kIntelVvpCoreInstanceErr if the instance is null
- kIntelVvpTpgPatternNumErr if the pattern is out-of-range
- kIntelVvpCoreNullPtrErr if pattern\_data is null
- Upon successful return, pattern\_data is filled with the configuration for the selected pattern. This includes the pattern type (eIntelVvpTpgPatternType) and the colorspace/subsampling (eIntelVvpTpgPatternColor)

#### intel\_vvp\_tpg\_is\_running

*Prototype*

```
bool intel_vvp_tpg_is_running(intel_vvp_tpg_instance*
instance);
```

- Description* Reads and returns the running bit of the STATUS register. The instance must be a valid intel\_vvp\_tpg\_instance fully initialized.
- Arguments* instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

**Returns** True is the TPG is currently generating an output field, false when disabled and/or between two fields

### intel\_vvp\_tpg\_get\_commit\_status

**Prototype**

```
bool intel_vvp_tpg_get_commit_status(intel_vvp_tpg_instance* instance);
```

**Description** Reads and returns the commit pending bit of the STATUS register. The instance must be a valid intel\_vvp\_tpg\_instance fully initialized and configured in full mode.

**Arguments** instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

**Returns** True is the TPG is configured in full mode and if the agent interface contains uncommitted writes.

### intel\_vvp\_tpg\_get\_status

**Prototype**

```
uint8_t intel_vvp_tpg_get_status(intel_vvp_tpg_instance* instance);
```

**Description** Reads and returns the STATUS register. The instance must be a valid intel\_vvp\_tpg\_instance fully initialized.

**Arguments** instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

**Returns** The value read from the status register

### intel\_vvp\_tpg\_enable

**Prototype** int intel\_vvp\_tpg\_enable(intel\_vvp\_tpg\_instance\* instance, bool enable);

**Description** Starts the TPG IP. The instance must be a valid intel\_vvp\_tpg\_instance fully initialized.

**Arguments** instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

enable – true to start the IP and generate fields, false to stop/pause the IP after the current field

**Returns** kIntelVvpCoreOk (0) in case of success, otherwise negative error code. kIntelVvpCoreInstanceErr if the instance is null

### intel\_vvp\_tpg\_start

*Prototype*

```
int intel_vvp_tpg_start(intel_vvp_tpg_instance* instance);
```

*Description*

Starts the TPG IP. The instance must be a valid intel\_vvp\_tpg\_instance fully initialized. An alias for intel\_vvp\_tpg\_enable(instance, true)

*Arguments*

instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

*Returns*

kIntelVvpCoreOk (0) in case of success, negative error code otherwise. kIntelVvpCoreInstanceErr if the instance is null

### intel\_vvp\_tpg\_stop

*Prototype*

```
int intel_vvp_tpg_stop(intel_vvp_tpg_instance* instance);
```

*Description*

Stops the TPG IP. The instance must be a valid intel\_vvp\_tpg\_instance fully initialized. An alias for intel\_vvp\_tpg\_enable(instance, false)

*Arguments*

instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

*Returns*

kIntelVvpCoreOk (0) in case of success, negative error code otherwise. kIntelVvpCoreInstanceErr if the instance is null

### intel\_vvp\_tpg\_is\_enabled

*Prototype*

```
bool intel_vvp_tpg_is_enabled(intel_vvp_tpg_instance* instance);
```

*Description*

Returns if the TPG IP is started or paused. The instance must be a valid intel\_vvp\_tpg\_instance fully initialized. The state cannot be checked if debug is not enabled

*Arguments*

instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

*Returns*

true is the IP is enabled and in debug mode, false otherwise

### intel\_vvp\_tpg\_set\_pattern

*Prototype*

```
int intel_vvp_tpg_set_pattern(intel_vvp_tpg_instance* instance, uint8_t pattern);
```

*Description*

Select/change the output test pattern. intel\_vvp\_tpg\_instance must be valid and fully initialized. The pattern must be valid.

**Arguments** instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

pattern – A valid pattern configured at compile-time in the range [0..intel\_vvp\_get\_num\_patterns(instance)-1]

**Returns** kIntelVvpCoreOk (0) in case of success, negative error code otherwise. kIntelVvpCoreInstanceErr if the instance is null

kIntelVvpTpgPatternNumErr if the pattern is out of range

#### intel\_vvp\_tpg\_get\_pattern

**Prototype**

```
uint8_t intel_vvp_tpg_get_pattern(intel_vvp_tpg_instance* instance);
```

**Description** Returns the selected test pattern. The instance must be a valid intel\_vvp\_tpg\_instance fully initialized. The pattern cannot be retrieved if debug is not enabled

**Arguments** instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

**Returns** Currently selected pattern or error code 255 if the instance is invalid or debug disabled

#### intel\_vvp\_tpg\_get\_fields\_since\_sequence\_reset

**Prototype**

```
uint32_t intel_vvp_tpg_get_fields_since_sequence_reset(intel_vvp_tpg_instance* instance);
```

**Description** Read and returns the FIELD\_COUNT register. The field counter is automatically reset when the pattern changes or the image specifications change (after a write to the image info area). The value may remain undefined after a reset until the first field of the new sequence is sent.

**Arguments** instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

**Returns** the field counter value

#### intel\_vvp\_tpg\_setBars\_type

**Prototype**

```
int intel_vvp_tpg_setBars_type(intel_vvp_tpg_instance* instance, eIntelVvpTpgBarsType bar_type);
```

**Description** Selects the type of color bars (black/white, grey, color) for patterns of type kIntelVvpTpgBarsPattern



*Arguments* instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

bar\_type – the type of color bars

*Returns* kIntelVvpCoreOk (0) for success, negative error code otherwise.

#### intel\_vvp\_tpg\_get\_bars\_type

*Prototype* eIntelVvpTpgBarsType  
intel\_vvp\_tpg\_set\_bars\_type(intel\_vvp\_tpg\_instance\* instance)

*Description* Read and returns the selected bar type for patterns of type kIntelVvpTpgBarsPattern. Instance must be valid and debug enabled

*Arguments* instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

*Returns* the selected bar type, kIntelVvpTpgInvalidBars in case of error

#### intel\_vvp\_tpg\_set\_colors

*Prototype*

```
int intel_vvp_tpg_set_colors (intel_vvp_tpg_instance* instance, uint16_t color1, uint16_t color2, uint16_t color3);
```

*Description* Selects the uniform color for patterns of type kIntelVvpTpgUniformPattern

*Arguments* instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

color1, color2, color3 – the three new color sample values for the uniform pattern

The color samples must be in the range [0 .. (1 << intel\_vvp\_tpg\_get\_bits\_per\_sample(instance)) - 1]

*Returns* kIntelVvpCoreOk (0) for success, negative error code otherwise.

kIntelVvpCoreInstanceErr if the instance is invalid

kIntelVvpTpgC1Err/ kIntelVvpTpgC2Err/ kIntelVvpTpgC3Err if the color sample is out-of-range

#### intel\_vvp\_tpg\_get\_colors

*Prototype*

```
int intel_vvp_tpg_get_colors(intel_vvp_tpg_instance* instance, uint16_t* color1, uint16_t* color2, uint16_t* color3)
```

- Description** Reads and returns the selected color sample values for pattern of type `kIntelVvpTpgUniformPattern`. The instance must be a valid `intel_vvp_tpg_instance` with debug enabled. The three points that return values must point to valid memory locations containing `uint16_t` variables
- Arguments** `instance` – pointer to the `intel_vvp_tpg_instance` software driver instance structure
- `color1, color2, color3` – three memory locations to store the returned color sample values
- Returns** `kIntelVvpCoreOk` in case of success, negative error code otherwise.
- `kIntelVvpCoreInstanceErr`, invalid instance (null pointer or debug disabled)
- `kIntelVvpCoreNullPtrErr`, one of the `uint16_t` pointer is the null pointer

#### `intel_vvp_tpg_set_zone_plate_origin`

**Prototype**

```
int intel_vvp_tpg_set_zone_plate_origin(intel_vvp_tpg_instance* instance, uint16_t x_coord, uint16_t y_coord);
```

- Description** Set to change the coordinates of the origin of patterns of type `kIntelVvpTpgZonePlate`
- Arguments** `instance` – pointer to the `intel_vvp_tpg_instance` software driver instance structure
- `x_coord, y_coord` – the coordinates of the origin
- Returns** `kIntelVvpCoreOk` in case of success, negative error code otherwise.
- `kIntelVvpCoreInstanceErr` if the instance is invalid (null)

#### `intel_vvp_tpg_get_zone_plate_origin`

**Prototype**

```
int intel_vvp_tpg_get_zone_plate_origin(intel_vvp_tpg_instance* instance, uint16_t *x_coord, uint16_t *y_coord);
```

- Description** Returns the coordinates of the origin of patterns of type `kIntelVvpTpgZonePlate`
- Arguments** `instance` – pointer to the `intel_vvp_tpg_instance` software driver instance structure
- `x_coord, y_coord` – Pointers to `uint16_t` to store the x-coordinate and y-coordinate of the zone plate origin
- Returns** `kIntelVvpCoreOk` in case of success, negative error code otherwise.

kIntelVvpCoreInstanceErr,invalid instance (null pointer or debug disabled)

kIntelVvpCoreNullPtrErr if either x\_coord or y\_coord are null pointers

intel\_vvp\_tpg\_set\_zone\_plate\_factors

**Prototype**

```
int intel_vvp_tpg_set_zone_plate_factors(intel_vvp_tpg_instance* instance, uint16_t coarse_factor, uint16_t fine_factor);
```

**Description**

Set to change the coarse and fine tune scaling coefficients of patterns of type kIntelVvpTpgZonePlate

**Arguments**

instance– pointer to the intel\_vvp\_tpg\_instance software driver instance structure

coarse\_factor – The coarse scaling coefficient

fine\_factor – The fine tune scaling coefficient

**Returns**

kIntelVvpCoreOk in case of success, negative error code otherwise.

kIntelVvpCoreInstanceErr if the instance is invalid(null pointer or debug disabled)

intel\_vvp\_tpg\_get\_zone\_plate\_factors

**Prototype**

```
int intel_vvp_tpg_get_zone_plate_factors(intel_vvp_tpg_instance* instance, uint16_t coarse_factor, uint16_t fine_factor);
```

**Description**

Get to change the coordinates of the origin of patterns of type kIntelVvpTpgZonePlate

**Arguments**

instance– pointer to the intel\_vvp\_tpg\_instance software driver instance structure

coarse\_factor – Pointer to uint16\_t to store the coarse scaling coefficient of the zone plate

fine\_factor – Pointer to uint16\_t to store the fine tune scaling coefficient of the zone plate

**Returns**

kIntelVvpCoreOk in case of success, negative error code otherwise.

kIntelVvpCoreInstanceErr if the instance is invalid(null pointer or debug disabled)

kIntelVvpCoreNullPtrErr if either coarse\_factor or fine\_factor are null pointers

intel\_vvp\_tpg\_set\_digital\_clock\_background\_color

**Prototype**

```
int
intel_vvp_tpg_set_digital_clock_background_color(intel_vvp_tpg_instance*
instance, uint16_t background_r,
uint16_t background_g, uint16_t
background_b);
```

**Description** Selects the background color of types kIntelVvpTpgDigitalClockPattern

**Arguments** instance– pointer to the intel\_vvp\_tpg\_instance software driver instance structure

background\_r, background\_g, background\_b– the respective color components of the background color

The color samples must be in the range [0 .. (1 << intel\_vvp\_tpg\_get\_bits\_per\_sample(instance)) – 1]

**Returns** kIntelVvpCoreOk (0) for success, negative error code otherwise. kIntelVvpCoreInstanceErr if the instance is invalid

kIntelVvpTpgC1Err/kIntelVvpTpgC2Err/ kIntelVvpTpgC3Err if the corresponding color sample is out of range

intel\_vvp\_tpg\_get\_digital\_clock\_background\_color

**Prototype**

```
int
intel_vvp_tpg_get_digital_clock_background_color(intel_vvp_tpg_instance*
instance, uint16_t* background_r,
uint16_t* background_g, uint16_t*
background_b);
```

**Description** Reads and returns the selected color sample values for pattern of type kIntelVvpTpgDigitalClockPattern .

The instance must be a valid intel\_vvp\_tpg\_instance with debug enabled. The three points that return values must point to valid memory locations containing uint16\_t variables

**Arguments** instance– pointer to the intel\_vvp\_tpg\_instance software driver instance structure

background\_r, background\_g, background\_b– the respective color components of the background color

**Returns** kIntelVvpCoreOk in case of success, negative error code otherwise.

kIntelVvpCoreInstanceErr, invalid instance (null pointer or debug disabled)

kIntelVvpCoreNullPtrErr, one of the uint16\_t pointer is the null pointer

intel\_vvp\_tpg\_set\_digital\_clock\_font\_color

**Prototype**

```
int
intel_vvp_tpg_set_digital_clock_font_color(intel_vvp_tpg_instance*
instance, uint16_t font_r,
uint16_t font_g, uint16_t
font_b);
```

**Description**

Selects the font color of types kIntelVvpTpgDigitalClockPattern

font\_r, font\_g, font\_b– the respective color components of the background color

The color samples must be in the range [0 .. (1 << intel\_vvp\_tpg\_get\_bits\_per\_sample(instance)) – 1]

**Arguments**

instance– pointer to the intel\_vvp\_tpg\_instance software driver instance structure

**Returns**

kIntelVvpCoreOk (0) for success, negative error code otherwise. kIntelVvpCoreInstanceErr if the instance is invalid

kIntelVvpTpgC1Err/kIntelVvpTpgC2Err/ kIntelVvpTpgC3Err if the corresponding color sample is out of range

intel\_vvp\_tpg\_get\_digital\_clock\_font\_color

**Prototype**

```
int
intel_vvp_tpg_get_digital_clock_font_color(intel_vvp_tpg_instance*
instance, uint16_t* font_r,
uint16_t* font_g, uint16_t*
font_b);
```

**Description**

Reads and returns the selected color sample values for pattern of type kIntelVvpTpgDigitalClockPattern .

The instance must be a valid intel\_vvp\_tpg\_instance with debug enabled. The three points that return values must point to valid memory locations containing uint16\_t variables

**Arguments**

instance– pointer to the intel\_vvp\_tpg\_instance software driver instance structure

font\_r, font\_g, font\_b– the respective color components of the background color

**Returns**

kIntelVvpCoreOk in case of success, negative error code otherwise.

kIntelVvpCoreInstanceErr, invalid instance (null pointer or debug disabled)

kIntelVvpCoreNullPtrErr, one of the uint16\_t pointer is the null pointer

intel\_vvp\_tpg\_set\_digital\_clock\_location

*Prototype*

```
int
intel_vvp_tpg_set_digital_clock_location(intel_vvp_tpg_instance*
 instance, uint16_t location_x,
uint16_t
 location_y);
```

*Description* Selects the display location for patterns of type kIntelVvpTpgDigitalClockPattern

*Arguments* instance– pointer to the intel\_vvp\_tpg\_instance software driver instance structure

location\_x, location\_y– the respective location coordinate components

*Returns* kIntelVvpCoreOk (0) for success, negative error code otherwise. kIntelVvpCoreInstanceErr if the instance is invalid

intel\_vvp\_tpg\_get\_digital\_clock\_location

*Prototype*

```
int
intel_vvp_tpg_get_digital_clock_location(intel_vvp_tpg_instance*
 instance, uint16_t* location_x,
uint16_t*
 location_y);
```

*Description* Reads and returns the current display location for patterns of type kIntelVvpTpgDigitalClockPattern .

The instance must be a valid intel\_vvp\_tpg\_instancewith debug enabled. The two points that return values must point to valid memory locations containing uint16\_t variables

*Arguments* instance– pointer to the intel\_vvp\_tpg\_instance software driver instance structure

location\_x, location\_y– the respective location coordinate components

*Returns* kIntelVvpCoreOkin case of success, negative error code otherwise.

kIntelVvpCoreInstanceErr,invalid instance (null pointer or debug disabled)

kIntelVvpCoreNullPtrErr,one or more of the uint16\_t pointers are a null pointer

intel\_vvp\_tpg\_set\_digital\_clock\_scale\_factor

*Prototype*

```
int
intel_vvp_tpg_set_digital_clock_scale_factor(intel_vvp_tpg_instance*
 instance, uint16_t scale_factor);
```

*Arguments*

instance– pointer to the intel\_vvp\_tpg\_instance software driver instance structure

scale\_factor – The scaling coefficient to apply to the digital clock size

*Description*

Sets the scale factor of patterns of type kIntelVvpTpgDigitalClockPattern

*Returns*

kIntelVvpCoreOk (0) for success, negative error code otherwise.  
kIntelVvpCoreInstanceErr if the instance is invalid

kIntelVvpTpgScaleFactorErr if the corresponding scale factor sample is greater than 11 bits

intel\_vvp\_tpg\_get\_digital\_clock\_scale\_factor

*Prototype*

```
int
intel_vvp_tpg_get_digital_clock_scale_factor(intel_vvp_tpg_instance*
 instance, uint16_t* scale_factor);
```

*Description*

Reads and returns the FPS for patterns of type kIntelVvpTpgDigitalClockPattern

The instance must be a valid intel\_vvp\_tpg\_instance with debug enabled. The two pointers that return values must point to valid memory locations containing uint16\_t variables

*Arguments*

instance– pointer to the intel\_vvp\_tpg\_instance software driver instance structure

scale\_factor – pointer to store the scaling coefficient of the digital clock size

*Returns*

kIntelVvpCoreOk in case of success, negative error code otherwise.

kIntelVvpCoreInstanceErr, invalid instance (null pointer or debug disabled)

kIntelVvpCoreNullPtrErr, one or more of the uint16\_t pointers are a null pointer

intel\_vvp\_tpg\_set\_digital\_clock\_fps

*Prototype*

```
int intel_vvp_tpg_set_digital_clock_fps(intel_vvp_tpg_instance*
 instance, uint16_t fps);
```

**Description** Selects the FPS of patterns of type `kIntelVvpTpgDigitalClockPattern` . This value controls the increment of one second.

This value does not affect the output video refresh rate.

**Arguments** Arguments instance– pointer to the `intel_vvp_tpg_instance` software driver instance structure

fps –the frames per second to determine when a second has passed

**Returns** `kIntelVvpCoreOk` (0) for success, negative error code otherwise.  
`kIntelVvpCoreInstanceErr` if the instance is invalid

`kIntelVvpTpgFpsErr` if the FPS sample is out of range

`intel_vvp_tpg_get_digital_clock_fps`

**Prototype**

```
int intel_vvp_tpg_get_digital_clock_fps(intel_vvp_tpg_instance*
 instance, uint16_t* fps);
```

**Description** Reads and returns the FPS for patterns of type `kIntelVvpTpgDigitalClockPattern` .

The instance must be a valid `intel_vvp_tpg_instance` with debug enabled. The two points that return values must point to valid memory locations containing `uint16_t` variables

**Arguments** instance– pointer to the `intel_vvp_tpg_instance` software driver instance structure

fps– pointer to a `uint16_t` to store the FPS value

**Returns** `kIntelVvpCoreOk` in case of success, negative error code otherwise.

`kIntelVvpCoreInstanceErr`, invalid instance (null pointer or debug disabled)

`kIntelVvpCoreNullPtrErr`, one or more of the `uint16_t` pointers are a null pointer

### `intel_vvp_tpg_commit_writes`

**Prototype**

```
int intel_vvp_tpg_commit_writes(intel_vvp_tpg_instance*
 instance)
```

**Description** Commits all pending writes before starting the next output field. The instance must be valid and parameterized in full mode.



*Arguments* instance – pointer to the intel\_vvp\_tpg\_instance software driver instance structure

*Returns* kIntelVvpCoreOk (0) for success, negative error code otherwise.

#### **Related Information**

[Video and Vision Processing IPs Software Programming Model](#) on page 38

## 45. Unsharp Mask IP

### 45.1. About the Unsharp Mask IP

The IP improves the sharpness of a video by intensifying the high frequency components. The IP receives frames or fields of RGB video from an Altera Streaming Video input. It transmits them over an Altera Streaming Video output after performing an unsharp mask operation. The Unsharp Mask IP has an Avalon memory-mapped agent interface to allow run-time control.

The IP supports:

- Maximum frame resolutions of 16,384 by 16,384 pixels with 1,2,4 or 8 pixels in parallel in the RGB color space.
- Progressive frames or interlaced fields.
- Run-time control of strength setting. A setting of zero is bypass mode.

The IP only supports lite variants. For more information on full and lite, refer to the *Altera Streaming Video Protocol Specification*.

An Avalon memory-mapped interface allows you to change the strength at run time.

For details about latency and reset behavior for the Unsharp Mask IP, refer to *Video and Vision IPs Functional Description*.

#### Related Information

[Altera Streaming Video Protocol Specification](#)

#### 45.1.1. Unsharp Mask IP Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family and the number of supported bits per color sample, colors per pixel, and pixels in parallel.

**Table 868. Unsharp Mask IP Performance and Resources**

The table shows ALM usage and  $f_{MAX}$  for a design with an unsharp mask with 1 pixel in parallel, 10 bits per color sample and separate control clock.

| Target Device                                 | Maximum Frame Size | ALMs  | $f_{MAX}$ (MHz) |
|-----------------------------------------------|--------------------|-------|-----------------|
| Agilex 7 speed grade 2 (AGFA012R24A2E2V)      | 4K                 | 1,812 | 771             |
| Cyclone 10 GX speed grade 5 (10CX220YF672E5G) | 1080p              | 1,193 | 414             |

## 45.2. Unsharp Mask IP Parameters

The IP offers compile-time parameters. Set these from the GUI in Platform Designer. Set the maximum frame width and heights to the largest frame size you need the IP to process and no greater, to avoid consuming unnecessary FPGA resources.

**Table 869. Unsharp Mask IP Parameters**

| Parameter                                   | Values        | Description                                                                              |
|---------------------------------------------|---------------|------------------------------------------------------------------------------------------|
| <b>Video Data Format</b>                    |               |                                                                                          |
| <b>Bits per color sample</b>                | 8 to 10       | Select the number of bits per color sample.                                              |
| <b>Number of pixels in parallel</b>         | 1, 2, 4 and 8 | Select the number of pixels in parallel.                                                 |
| <b>Maximum Frame Size</b>                   |               |                                                                                          |
| <b>Maximum frame height</b>                 | 1024 to 16384 | Select the maximum height of frames. The IP crops taller input frames to this height.    |
| <b>Maximum frame width</b>                  | 1024 to 16384 | Select the maximum width of frames. The IP crops wider input frames wider to this width. |
| <b>Control</b>                              |               |                                                                                          |
| <b>Separate clock for control interface</b> | On or off     | Turn on for a separate clock for the control interface.                                  |
| <b>Debug features</b>                       | On or off     | Turn on for debugging features.                                                          |

Figure 125. Unsharp Mask IP GUI

The screenshot shows the 'Unsharp mask Intel FPGA IP' configuration window. The title bar reads 'Unsharp mask Intel FPGA IP - intel\_vvp\_usm\_0'. The window has a 'Documentation' button in the top right. The main content area is divided into three sections: 'Video Data Format', 'Maximum Frame Size', and 'General'. The 'Video Data Format' section has a checked 'Lite mode' checkbox, a 'Bits per color sample' input set to 8, and a 'Number of pixels in parallel' dropdown set to 1. The 'Maximum Frame Size' section has 'Maximum field height' and 'Maximum field width' dropdowns both set to 4096. The 'General' section has three unchecked checkboxes: 'Pipeline ready signals', 'Separate clock for control interface', and 'Debug features'. Below these sections is a 'Parameterization Messages' table with columns 'Type' and 'Message', showing '(No messages)'. At the bottom, there is an 'IP folder' field with 'ip/usm', an 'HDL entity' field with 'usm\_intel\_vvp\_usm\_0', and 'Cancel' and 'Finish' buttons.

### 45.3. Unsharp Mask IP Functional Description

The IP creates an unsharp mask by first calculating an approximation to the input image luminance using the input luminance equation:

#### Equation 11. Input Luminance

$$\text{Luma}_{in} = \left( \frac{R_{in} + G_{in} + B_{in}}{4} \right)$$

The IP then performs a low-pass Gaussian blur (low-pass filter) of  $Luma_{in}$ .

The IP subtracts this filtered luminance from the original luminance to get the high-frequency components of the luminance.

The IP multiplies this value by the strength to obtain the sharpened luminance. The following equation shows this process.

### Equation 12. Sharpened Luminance

The equation shows that a strength of 0 gives  $Luma_{OUT}$  equal to  $Luma_{IN}$ .  

$$Luma_{out} = Luma_{in} + (Luma_{in} - h_{blur} \times Luma_{in}) \cdot S$$

where:

$Luma_{in}$  = Input luminance

$Luma_{out}$  = Output luminance

$h_{blur}$  = Kernel for the Gaussian blur

$S$  = Strength to control the amount of sharpening

The IP creates the final output image by calculating a ratio of  $\frac{Luma_{out}}{Luma_{in}}$ , which scales the RGB components of the original image. A strength of 0 results in unity ratio.

### Equation 13. Final Output Image RGB Scaling

$$R_{out} = R_{in} \times \left( \frac{Luma_{out}}{Luma_{in}} \right)$$

$$G_{out} = G_{in} \times \left( \frac{Luma_{out}}{Luma_{in}} \right)$$

$$B_{out} = B_{in} \times \left( \frac{Luma_{out}}{Luma_{in}} \right)$$

The unsharp mask has an Avalon® memory-mapped agent interface to allow run-time control of the strength of the unsharp mask operation. Set the strength to zero for the IP to pass through the video unchanged.

You need not start the IP via the run-time interface as it starts processing video on its input as soon as it comes out of reset

Changes to the strength run-time control register take effect on the next video line.

### Register Behavior

Write values to the control and debugging registers to set video field dimensions. Turn on **Debug Features** in the GUI so you can read back these values.

### Latency

The IP latency under ideal conditions is three video lines plus an additional 34 clock cycles. This latency increases if the streaming video output experiences backpressure via its `axi4s_vid_out_tready` input. Backpressure increases the latency by the same amount of cycles.

The sharpening strength register setting does not change the latency.

### 45.3.1. Unsharp Mask IP Interfaces

**Table 870. Unsharp Mask IP Interfaces**

| Name                                     | Direction | Width | Description                              |
|------------------------------------------|-----------|-------|------------------------------------------|
| <b>Clocks and resets</b>                 |           |       |                                          |
| main_clock_clk                           | In        | 1     | AXI4-S processing clock.                 |
| main_reset_rst                           | In        | 1     | AXI4-S processing reset.                 |
| control_clock_clk                        | In        | 1     | Optional control agent interface clock.  |
| control_reset_reset                      | In        | 1     | Optional control agent interface reset.  |
| <b>Control interfaces</b>                |           |       |                                          |
| av_mm_control_agent_address              | In        | 8     | Avalon memory-mapped agent address       |
| av_mm_control_agent_write                | In        | 1     | Avalon memory-mapped agent write.        |
| av_mm_control_agent_writedata            | In        | 32    | Avalon memory-mapped agent write data.   |
| av_mm_control_agent_byteenable           | In        | 4     | Avalon memory-mapped agent byte enable.  |
| av_mm_control_agent_read                 | In        | 1     | Avalon memory-mapped agent read.         |
| av_mm_control_agent_readdata             | Out       | 32    | Avalon memory-mapped agent read data.    |
| av_mm_control_agent_readdatavalid        | Out       | 1     | Avalon memory-mapped agent read.         |
| av_mm_control_agent_waitrequest          | Out       | 1     | Avalon memory-mapped agent wait request. |
| <b>Altera Streaming Video interfaces</b> |           |       |                                          |
| axi4s_vid_in_tdata                       | In        | (137) | AXI4-S data in.                          |
| axi4s_vid_in_tvalid                      | In        | 1     | AXI4-S data valid.                       |
| axi4s_vid_in_tuser[0]                    | In        | 1     | F1 interlaced field flag.                |
| axi4s_vid_in_tuser[1]                    | In        | 1     | AXI4-S control or data packet.           |
| axi4s_vid_in_tuser[N-1:2]                | In        | (138) | Unused.                                  |
| axi4s_vid_in_tlast                       | In        | 1     | AXI4-S end of packet.                    |
| axi4s_vid_in_tready                      | Out       | 1     | AXI4-S data ready.                       |
| axi4s_vid_out_tdata                      | Out       | (137) | AXI4-S data out.                         |
| axi4s_vid_out_tvalid                     | Out       | 1     | AXI4-S data valid.                       |
| axi4s_vid_out_tuser[0]                   | Out       | 1     | AXI4-S start of video frame.             |
| axi4s_vid_out_tuser[1]                   | Out       | 1     | F1 interlaced field flag.                |
| <i>continued...</i>                      |           |       |                                          |

(137) The equation gives all axi4s\_vid\_out\_tdata widths in these interfaces:  
 $\max(2, \text{ceil}((\text{bits per color sample} \times \text{number of color planes})/8)) \times \text{pixels in parallel} \times 8$

(138) This equation gives all axi4s\_vid\_out\_tuser widths in these interfaces  $N = \text{ceil}(\text{tdata width} / 8)$

| Name                       | Direction | Width | Description           |
|----------------------------|-----------|-------|-----------------------|
| axi4s_vid_out_tuser[N-1:2] | Out       | (138) | Unused.               |
| axi4s_vid_out_tlast        | Out       | 1     | AXI4-S end of packet. |
| axi4s_vid_out_tready       | In        | 1     | AXI4-S data ready.    |

### Related Information

Video and Vision Processing IP Interfaces on page 30

## 45.4. Unsharp Mask IP Registers

Each register is either read-only (RO) or read-write (RW).

**Table 871. Unsharp Mask IP Registers**

In the software API the register names appear with a prefix of `INTEL_VVP`, `INTEL_VVP_CORE`, or `INTEL_VVP_UNSHARP_MASK` as appropriate and with an optional **REG** suffix

| Address                                | Register           | Access | Description                                                                                                                                                                                                                                                           |
|----------------------------------------|--------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Parameterization registers</b>      |                    |        |                                                                                                                                                                                                                                                                       |
| 0x0000                                 | VID_PID            | RO     | Read this register for the unsharp mask product ID. This register always returns 0x027C.                                                                                                                                                                              |
| 0x0004                                 | VERSION            | RO     | Read this register for the IP version information.                                                                                                                                                                                                                    |
| 0x0008                                 | LITE_MODE          | RO     | This register returns 1 because the IP operates in lite mode.                                                                                                                                                                                                         |
| 0x000C                                 | DEBUG_ENABLED      | RO     | Read this register to determine if <b>Debug features</b> are on.                                                                                                                                                                                                      |
| 0x0010                                 | BITS_PER_SYMBOL    | RO     | Read this register to determine the bits per symbol.                                                                                                                                                                                                                  |
| 0x0014                                 | PIXELS_IN_PARALLEL | RO     | Read this register to determine the pixels in parallel.                                                                                                                                                                                                               |
| 0x0018                                 | MAX_WIDTH          | RO     | Read this register to determine the maximum frame width.                                                                                                                                                                                                              |
| 0x001C                                 | MAX_HEIGHT         | RO     | Read this register to determine the maximum frame height.                                                                                                                                                                                                             |
| 0x001F<br>to<br>0x013F                 | Reserved           | -      | Reserved                                                                                                                                                                                                                                                              |
| <b>Control and debugging registers</b> |                    |        |                                                                                                                                                                                                                                                                       |
| 0x0120                                 | IMG_INFO_WIDTH     | RW     | Write the expected width of incoming video fields. Changes to this register take effect at the next start of line.<br>The IP removes the extra pixels from fields wider than this setting. Narrower fields can result in the IP producing pixels from previous lines. |
| 0x0124                                 | IMG_INFO_HEIGHT    | RW     | Write the expected height of incoming video fields. Changes to this register take effect at the next start of field.<br>The IP removes the extra lines from fields taller than this setting. Shorter fields can result in corrupted output pixels.                    |
| 0x0128<br>to<br>0x013F                 | Reserved           | -      | Reserved.                                                                                                                                                                                                                                                             |
| 0x0140                                 | STATUS             | RO     | Bit 0: status bit, which goes high when the IP starts producing the first frame. It goes low after the IP finishes producing the last line of the frame. It returns high when the IP starts producing the next frame.                                                 |

*continued...*

| Address | Register | Access | Description                                                                                                                                                                                                                                       |
|---------|----------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         |          |        | Applications that need to know when the IP produces frames can poll this status register.<br>1 = Unsharp mask is producing a video field, 0 otherwise.                                                                                            |
| 0x01443 | Reserved | -      | Reserved                                                                                                                                                                                                                                          |
| 0x0148  | STRENGTH | RW     | 14 bits.<br>2 integer bits and 12 fractional bits<br>Set STRENGTH to 0 for bypass mode.<br>Positive strengths sharpen the edges.<br>Negative strengths blur the edges.<br>Increase the magnitude of STRENGTH to increase the unsharp mask effect. |

## 45.5. Unsharp Mask IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_usm_regs.h`

Include file: `intel_vvp_usm.h`

**Table 872. Unsharp Mask IP Software API**

| Name                                               | Description                                                                                                      |
|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <code>intel_vvp_usm_init</code>                    | Initialize an IP instance                                                                                        |
| <code>intel_vvp_core_*</code>                      | Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i>                           |
| <code>intel_vvp_usm_get_lite_mode</code>           | Returns if <b>Lite mode</b> is on                                                                                |
| <code>intel_vvp_usm_get_debug_enabled</code>       | Returns if <b>Debug features</b> is on                                                                           |
| <code>intel_vvp_usm_get_bits_per_sample</code>     | Returns the number of bits per color sample                                                                      |
| <code>intel_vvp_usm_get_pixels_in_parallel</code>  | Returns the pixels in parallel of the streaming input and output interfaces                                      |
| <code>intel_vvp_usm_get_max_width</code>           | Returns the maximum number of pixels that the IP supports on the horizontal dimension of an image or video frame |
| <code>intel_vvp_usm_get_max_height</code>          | Returns the maximum number of pixels that the IP supports on the vertical dimension of an image or video frame   |
| <code>intel_vvp_usm_is_running</code>              | Reads if the IP is running                                                                                       |
| <code>intel_vvp_usm_get_status</code>              | Reads the status register                                                                                        |
| <code>intel_vvp_usm_get_sharpening_strength</code> | Reads the sharpening strength                                                                                    |
| <code>intel_vvp_usm_set_sharpening_strength</code> | Writes the sharpening strength                                                                                   |
| <code>intel_vvp_usm_set_output_height</code>       | Writes the output height                                                                                         |
| <code>intel_vvp_usm_set_output_width</code>        | Reads the output width                                                                                           |



### intel\_vvp\_usm\_init

#### Prototype

```
int
intel_vvp_usm_init(intel_vvp_usm_instance* instance,
intel_vvp_core_base base);
```

#### Description

Initialize an IP instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

#### Arguments

instance - pointer to the intel\_vvp\_usm\_instance software driver instance structure

base - base address of the register map

#### Returns

kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7)

kIntelVvpCorePidErr if the product\_id does not match with the expected product ID (0x0246)

kIntelVvpUsmRegMapVersionErr if the register map is not supported

### intel\_vvp\_usm\_get\_lite\_mode

#### Prototype

```
bool
intel_vvp_usm_get_lite_mode(intel_vvp_usm_instance* instance);
```

#### Description

Returns the value of the LITE\_MODE register. The instance must be a valid intel\_vvp\_usm\_instance fully initialized.

#### Arguments

instance - pointer to the intel\_vvp\_usm\_instance software driver instance structure

#### Returns

true if the IP is parameterized in lite mode

### intel\_vvp\_usm\_get\_debug\_enabled

#### Prototype

```
bool
intel_vvp_usm_get_debug_enabled(intel_vvp_usm_instance*
instance);
```

**Description** Returns the value of the DEBUG\_ENABLED register. The instance must be a valid intel\_vvp\_usm\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_usm\_instance software driver instance structure

**Returns** true if the IP is parameterized with debug features enabled

### intel\_vvp\_usm\_get\_bits\_per\_sample

#### Prototype

```
uint8_t
intel_vvp_usm_get_bits_per_sample(intel_vvp_usm_instance*
instance);
```

**Description** Returns the value of the BPS register. The instance must be a valid intel\_vvp\_usm\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_usm\_instance software driver instance structure

**Returns** the bits per color sample parameter used to generate the IP

### intel\_vvp\_usm\_get\_pixels\_in\_parallel

#### Prototype

```
uint8_t
intel_vvp_usm_get_pixels_in_parallel(intel_vvp_usm_instance*
instance);
```

**Description** Returns the value of the PIXELS\_IN\_PARALLEL register. The instance must be a valid intel\_vvp\_usm\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_usm\_instance software driver instance structure

**Returns** the pixels in parallel parameter used to generate the IP

### intel\_vvp\_usm\_get\_max\_width

*Prototype*

```
uint32_t
intel_vvp_usm_get_max_width(intel_vvp_usm_instance* instance);
```

*Description* Returns the value of the MAX\_WIDTH register. The instance must be a valid intel\_vvp\_usm\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_usm\_instance software driver instance structure

*Returns* the maximum field width parameter used to generate the IP

### intel\_vvp\_usm\_get\_max\_height

*Prototype*

```
uint32_t
intel_vvp_usm_get_max_height(intel_vvp_usm_instance* instance);
```

*Description* Returns the value of the MAX\_HEIGHT register. The instance must be a valid intel\_vvp\_usm\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_usm\_instance software driver instance structure

*Returns* the maximum field height parameter used to generate the IP

### intel\_vvp\_usm\_is\_running

*Prototype*

```
bool
intel_vvp_usm_is_running(intel_vvp_usm_instance* instance);
```

*Description* Returns the value of the IP Running bit from the STATUS register. The instance must be a valid intel\_vvp\_usm\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_usm\_instance software driver instance structure

*Returns* the running bit from the status register

### intel\_vvp\_usm\_get\_status

#### Prototype

```
uint32_t
intel_vvp_usm_get_status(intel_vvp_usm_instance* instance);
```

**Description** Returns the value of the STATUS register. The instance must be a valid intel\_vvp\_usm\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_usm\_instance software driver instance structure

**Returns** the value of the status register

### intel\_vvp\_usm\_get\_sharpening\_strength

#### Prototype

```
uint32_t
intel_vvp_usm_get_sharpening_strength(intel_vvp_usm_instance
*instance);
```

**Description** Returns the sharpening strength. The instance must be a valid intel\_vvp\_usm\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_usm\_instance software driver instance structure

**Returns** the sharpening strength

### intel\_vvp\_usm\_set\_sharpening\_strength

#### Prototype

```
int
intel_vvp_usm_set_sharpening_strength(intel_vvp_usm_instance*
instance, uint16_t strength);
```

**Description** Writes the sharpening strength. The instance must be a valid intel\_vvp\_usm\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_usm\_instance software driver instance structure

strength - new sharpening strength

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

### intel\_vvp\_usm\_set\_output\_height

*Prototype*

```
bool
intel_vvp_usm_set_output_height(intel_vvp_usm_instance*
instance, uint32_t new_height);
```

*Description* Writes the height of the field. The instance must be a valid intel\_vvp\_usm\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_usm\_instance software driver instance structure

new\_height - new height of the field

*Returns* true in case of success, false otherwise

### intel\_vvp\_usm\_set\_output\_width

*Prototype*

```
bool
intel_vvp_usm_set_output_width(intel_vvp_usm_instance*
instance, uint32_t new_width)
```

*Description* Writes the width of the field. The instance must be a valid intel\_vvp\_usm\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_usm\_instance software driver instance structure

new\_width - new width of the field

*Returns* true in case of success, false otherwise

### Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

## 46. Video and Vision Monitor IP

### 46.1. About the Video and Vision Monitor IP

The Video and Vision Monitor IP helps you debug systems that use the Altera Streaming Video interface protocol. The Video and Vision Monitor IP gathers information about the dimensions of incoming video frames and counts the number of frames that match or do not match their expected dimensions. The IP reports this information through its register map. The Video and Vision Monitor IP supports both the full and lite variants of the Altera Streaming Video Protocol.

#### 46.1.1. Video and Vision Monitor IP Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family.

**Table 873. Video and Vision Monitor IP Performance and Resources**

The table shows the Video and Vision Monitor IP ALM usage and  $f_{MAX}$  for a design with 8 bits per color sample, 2 color planes, and 1 pixel in parallel.

| Target device                                  | ALMs | $f_{MAX}$ |
|------------------------------------------------|------|-----------|
| Agilex 7 speed grade 2 (AGFA012R24A2E2V)       | 416  | 884       |
| Arria 10 speed grade 1 (10AS066H1F34E1HG)      | 379  | 536       |
| Cyclone 10 GX speed grade 5 (10CX220YF78015G)  | 381  | 459       |
| Stratix 10 GX speed grade 1 (1SX280LN2F43E1VG) | 429  | 599       |

### 46.2. Video and Vision Monitor IP Parameters

The IP offers compile-time parameters.

| Parameter                            | Allowed range | Description                                                                       |
|--------------------------------------|---------------|-----------------------------------------------------------------------------------|
| <b>Video data format</b>             |               |                                                                                   |
| Lite mode                            | On or off     | Turn on to use the lite variant of the Altera Streaming Video Protocol            |
| Bits per color sample                | 8 to 16       | Select the number of bits per color sample                                        |
| Number of color planes               | 1 to 4        | Select the number of color planes per pixel                                       |
| Number of pixels in parallel         | 1 to 8        | Select the number of pixels in parallel at the input and output interfaces        |
| <b>Control settings</b>              |               |                                                                                   |
| Separate clock for control interface | On or off     | Turn on to enable a separate clock for the control agent interface                |
| <b>Pipeline optimization</b>         |               |                                                                                   |
| Pipeline ready signals               | On or off     | Turn on to add extra pipeline registers to the AXI4-S <code>tready</code> signals |

### 46.3. Video and Vision Monitor IP Functional Description

When using the full variant of the Altera Streaming Video interface protocol, the Video and Vision Monitor IP searches the incoming video stream for image information packets. The IP decodes the image information packets and then produces the contents of the last image information packet that you receive via the register map. The Image Information packets also specify the expected resolution for each incoming video frame. The IP counts the number of pixels in each line and the number of lines in each frame. Based on the values of these counters and the expected frame size, the IP keeps a count of the number of frames that match and do not match their expected sizes. The IP also counts the number of frames for which the following end-of-field packets assert their broken frame flag. The IP reports all these values to you via the register map. You can also reset the frame counts at any time by writing to an address in the register map.

Lite variants of the Altera Streaming Video interface protocol have no image information packets in the incoming stream to provide details of the expected frame size. You must supply this information via the register map. Lite variants also have no end-of-field packets, so this counter is not when **Lite mode** is on. Other than these differences, the behavior of the IP is the same as with the full variant of the protocol.

When the IP and its interfaces are parameterized for a single pixel in parallel, the IP can count the exact number of pixels in each line. The Altera Streaming Video Protocol does not indicate that any of the pixels are unused on the final data beat of each packet when interfaces are parameterized for multiple pixels in parallel. Therefore, the IP cannot exactly count the number of pixels in each line when the number of pixels in parallel is greater than 1. It can count the number of beats of data in the packet and, based on this value, provide an upper and lower bound on the number of pixels.

$$pixels\_in\_parallel \times (data\_beats - 1) + 1 \leq number\_of\_pixels \leq pixels\_in\_parallel \times data\_beats$$

The IP reports two-line length values to the register map:

- The minimum number of pixels in any line packet for the most recent frame, taken from the lower bound count of the number of pixels for each line.
- The maximum number of pixels in any line packet for the most recent frame, taken from the upper bound count of the number of pixels for each line.

If the IP is parameterized for 1 pixel in parallel, for frames that match their specified dimensions both these reported width values match the specified width. If the IP is parameterized for multiple pixels in parallel, the maximum and minimum line lengths have different values but adhere to the following constraints for frames that match their sizes.

$$expected\_width - pixels\_in\_parallel < minimum\_width \leq expected\_width$$

$$expected\_width \leq maximum\_width < expected\_width + pixels\_in\_parallel$$

The IP checks these constraints when deciding if each frame has to match its expected dimensions.

### 46.3.1. Video and Vision Monitor IP Interfaces

| Name                                     | Direction | Width | Description                                                                                                                              |
|------------------------------------------|-----------|-------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Clocks and resets</b>                 |           |       |                                                                                                                                          |
| main_clock_clk                           | In        | 1     | AXI4-S processing clock.                                                                                                                 |
| main_reset_rst                           | In        | 1     | AXI4-S processing reset.                                                                                                                 |
| agent_clock_clk                          | In        | 1     | Clock for the Avalon memory-mapped control agent interface. Only used if you select <b>Separate clock for control interface</b>          |
| agent_reset_rst                          | In        | 1     | Reset for the Avalon memory-mapped control agent interface. Only used if you select <b>Separate clock for control interface</b>          |
| <b>Control interfaces</b>                |           |       |                                                                                                                                          |
| av_mm_control_agent_address              | In        | 7     | Avalon memory-mapped agent address                                                                                                       |
| av_mm_control_agent_write                | In        | 1     | Avalon memory-mapped agent write                                                                                                         |
| av_mm_control_agent_writedata            | In        | 32    | Avalon memory-mapped agent write data                                                                                                    |
| av_mm_control_agent_byteenable           | In        | 4     | Avalon memory-mapped agent byte enable                                                                                                   |
| av_mm_control_agent_read                 | In        | 1     | Avalon memory-mapped agent read                                                                                                          |
| av_mm_control_agent_readdata             | Out       | 32    | Avalon memory-mapped agent read data                                                                                                     |
| av_mm_control_agent_readdatavalid        | Out       | 1     | Avalon memory-mapped agent read                                                                                                          |
| av_mm_control_agent_waitrequest          | Out       | 1     | Avalon memory-mapped agent wait request                                                                                                  |
| <b>Altera Streaming Video interfaces</b> |           |       |                                                                                                                                          |
| axi4s_vid_in_tdata                       | In        | (139) | AXI4-S data in                                                                                                                           |
| axi4s_vid_in_tvalid                      | In        | 1     | AXI4-S data valid                                                                                                                        |
| axi4s_vid_in_tuser                       | In        | (140) | AXI4-S tuser<br>tuser[0] indicates start of video frame when asserted<br>tuser[1] indicates the start of a nonvideo packet when asserted |
| axi4s_vid_in_tlast                       | In        | 1     | AXI4-S end of packet                                                                                                                     |
| axi4s_vid_in_tready                      | Out       | 1     | AXI4-S data ready                                                                                                                        |
| axi4s_vid_out_tdata                      | Out       | (139) | AXI4-S data in                                                                                                                           |
| axi4s_vid_out_tvalid                     | Out       | 1     | AXI4-S data valid                                                                                                                        |
| axi4s_vid_out_tuser                      | Out       | (140) | AXI4-S tuser<br>tuser[0] indicates start of video frame when asserted                                                                    |
| <b>continued...</b>                      |           |       |                                                                                                                                          |

(139) The equation shows all tdata widths in these interfaces:  

$$\max(\text{ceil}((\text{bits per color sample} \times \text{number of color planes}) / 8) \times \text{pixels in parallel} \times 8, 16)$$

(140) The equation shows all tuser widths in these interfaces:  

$$N = \text{ceil}(\text{tdata width} / 8)$$



| Name                 | Direction | Width | Description                                                     |
|----------------------|-----------|-------|-----------------------------------------------------------------|
|                      |           |       | tuser[1] indicates the start of a nonvideo packet when asserted |
| axi4s_vid_out_tlast  | Out       | 1     | AXI4-S end of packet                                            |
| axi4s_vid_out_tready | In        | 1     | AXI4-S data ready                                               |

### Related Information

Video and Vision Processing IP Interfaces on page 30

## 46.4. Video and Vision Monitor IP Registers

Each register is either read-only (RO) or read-write (RW).

**Table 874. Video and Vision Monitor IP Registers**

In the software API these register names appear with a prefix of INTEL\_VVP, INTEL\_VVP\_CORE or INTEL\_VVP\_SNOOP as appropriate and with an optional REG suffix

| Address                                                                                                                                   | Register           | Access |      | Description                                                                                                                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------|--------------------|--------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                           |                    | Lite   | Full |                                                                                                                                                                                                                                                     |
| Parameterization registers                                                                                                                |                    |        |      |                                                                                                                                                                                                                                                     |
| 0x0000                                                                                                                                    | PROD_ID            | RO     |      | Read this register to retrieve the Video and Vision Monitor product ID. This register always returns 0x6AF7_0248.                                                                                                                                   |
| 0x0004                                                                                                                                    | VER                | RO     |      | Read this register to retrieve the IP version information.                                                                                                                                                                                          |
| 0x0008                                                                                                                                    | LITE_MODE          | RO     |      | Read this register to determine if <b>Lite mode</b> is on. This register returns 1 if <b>Lite mode</b> is on and 0 if <b>Lite mode</b> is off                                                                                                       |
| 0x000C                                                                                                                                    | DEBUG_ENABLED      | RO     |      | This register always returns 1 as this debugging IP always has debug features on                                                                                                                                                                    |
| 0x0010                                                                                                                                    | PIXELS_IN_PARALLEL | RO     |      | Read this register to determine the number of pixels processed per clock cycle.                                                                                                                                                                     |
| 0x0014 to 0x011F unused                                                                                                                   |                    |        |      |                                                                                                                                                                                                                                                     |
| Control and debug registers                                                                                                               |                    |        |      |                                                                                                                                                                                                                                                     |
| Refer to <i>Image Information Packets</i> in the <i>Altera Streaming Video Protocol Specification</i> for more details of these registers |                    |        |      |                                                                                                                                                                                                                                                     |
| 0x0120                                                                                                                                    | IMG_INFO_WIDTH     | RW     | RO   | When <b>lite mode</b> is on, use this register to set the expected width of incoming video fields.<br>When <b>lite mode</b> is off, this register returns the width that the IP derives from information in the image information control packet.   |
| 0x0124                                                                                                                                    | IMG_INFO_HEIGHT    | RW     | RO   | When <b>lite mode</b> is on, use this register to set the expected height of incoming video fields.<br>When <b>lite mode</b> is off, this register returns the height that the IP derives from information in the image information control packet. |
| 0x0128                                                                                                                                    | IMG_INFO_INTERLACE | -      | RO   | When <b>lite mode</b> is off, this register returns the interlace nibble that the IP derives from information in the image information control packet. Unused when lite mode is on.                                                                 |
| 0x012C                                                                                                                                    | Reserved           | -      | -    | Reserved                                                                                                                                                                                                                                            |
| continued...                                                                                                                              |                    |        |      |                                                                                                                                                                                                                                                     |

| Address                 | Register             | Access |      | Description                                                                                                                                                                                                                                                  |
|-------------------------|----------------------|--------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                         |                      | Lite   | Full |                                                                                                                                                                                                                                                              |
| 0x0130                  | IMG_INFO_COLORSPACE  | -      | RO   | When <b>lite mode</b> is off, this register returns the color space that the IP derives from information in the image information control packet. Unused when <b>lite mode</b> is on.                                                                        |
| 0x0134                  | IMG_INFO_SUBSAMPLING | RW     | RO   | When <b>lite mode</b> is on, use this register to set the expected subsampling of incoming video fields.<br>When <b>lite mode</b> is off, this register returns the subsampling that the IP derives from information in the image information control packet |
| 0x0138                  | IMG_INFO_COSITING    | -      | RO   | When <b>lite mode</b> is off, this register returns the cositing that the IP derives from information in the image information control packet.                                                                                                               |
| 0x013C                  | IMG_INFO_FIELD_COUNT | -      | RO   | When <b>lite mode</b> is off, this register returns the field count that the IP derives from information in the image information control packet.                                                                                                            |
| 0x0138 to 0x013C unused |                      |        |      |                                                                                                                                                                                                                                                              |
| 0x0140                  | STATUS               |        | RO   | <b>Bit 0 : Status bit.</b><br>1 = IP is processing a video field, 0 otherwise. For other bits, see Table 21                                                                                                                                                  |
| 0x0144                  | NUM_GOOD_FIELDS      |        | RO   | Reports the number of fields received which matched their expected size since the counters were last reset. When <b>lite mode</b> is on, this count excludes any fields for which the end-of-field control packet sets its broken field flag.                |
| 0x0148                  | NUM_BROKEN_FIELDS    |        | RO   | When <b>lite mode</b> is on, this register reports the number of fields received since the counters were last reset for which the end-of-field control packet set its broken field flag. Unused when Lite mode is on.                                        |
| 0x014C                  | NUM_MISMATCH_FIELDS  |        | RO   | Reports the number of fields received that do not match their expected size since the IP last reset the counters.                                                                                                                                            |
| 0x0150                  | LAST_NUM_LINES       |        | RO   | Reports the number of lines in the last field received.                                                                                                                                                                                                      |
| 0x0154                  | LAST_MIN_WIDTH       |        | RO   | Reports the minimum width of any line received as part of the last field.                                                                                                                                                                                    |
| 0x0158                  | LAST_MAX_WIDTH       |        | RO   | Reports the maximum width of any line received as part of the last field.                                                                                                                                                                                    |
| 0x015C                  | RESERVED             | -      |      | Reserved                                                                                                                                                                                                                                                     |
| 0x0160                  | RESET_COUNTERS       |        | RW   | Write any value to this register to reset all debug counts to 0.                                                                                                                                                                                             |

### Register Bit Descriptions

**Table 875. PROD\_ID**

| Name                                | Bits | Description                               |
|-------------------------------------|------|-------------------------------------------|
| Video and Vision Monitor product ID | 31:0 | This register always returns 0x6AF7_0248. |

**Table 876. VERSION**

| Name                 | Bits  | Description                          |
|----------------------|-------|--------------------------------------|
| Register map version | 7:0   | Register map version.                |
| IP patch revision    | 15:8  | -                                    |
| IP update revision   | 23:16 | Updated when the IP version changes. |
| IP major revision    | 31:24 | Updated when the IP version changes. |

**Table 877. LITE\_MODE**

| Name                       | Bits | Description                               |
|----------------------------|------|-------------------------------------------|
| Lite mode parameterization | 31:0 | Returns 1 if lite mode is on. 0 otherwise |

**Table 878. DEBUG\_ENABLED**

| Name                            | Bits | Description      |
|---------------------------------|------|------------------|
| Debug features parameterization | 31:0 | Always returns 1 |

**Table 879. PIXELS\_IN\_PARALLEL**

| Name               | Bits | Description                                |
|--------------------|------|--------------------------------------------|
| Pixels in parallel | 31:0 | Number of pixels processed per clock cycle |

**Table 880. IMG\_INFO\_WIDTH**

| Name       | Bits  | Description                                                                                                                                                                                                                                                                                                   |
|------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Width bits | 15:0  | When <b>lite mode</b> is on, write to this register to set the expected width of the incoming video fields. When <b>lite mode</b> is off, this register returns the <code>width-1</code> field from the most recently received image information packet and adds 1 to return a value for <code>width</code> . |
| Unused     | 31:16 | Unused                                                                                                                                                                                                                                                                                                        |

**Table 881. IMG\_INFO\_HEIGHT**

| Name        | Bits  | Description                                                                                                                                                                                                                                                                                                    |
|-------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Height bits | 15:0  | When <b>lite mode</b> is on, write to this register to set the expected height of the incoming video fields. When <b>lite mode</b> is off, this register reads the <code>height-1</code> field from the most recently received image information packet and adds 1 to return a value for <code>height</code> . |
| Unused      | 31:16 | Unused                                                                                                                                                                                                                                                                                                         |

**Table 882. IMG\_INFO\_INTERLACE**

| Name               | Bits | Description                                                                                                                                                                                                    |
|--------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IntlaceNibble bits | 3:0  | When <b>lite mode</b> is on, this register has no function. When <b>lite mode</b> is off, this register returns the <code>intlaceNibble</code> field from the most recently received image information packet. |
| Unused             | 31:4 | Unused                                                                                                                                                                                                         |

**Table 883. IMG\_INFO\_COLORSPACE**

| Name          | Bits | Description                                                                                                                                                                                    |
|---------------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CSP code bits | 6:0  | When <b>lite mode</b> is on, this register has no function. For full variants, this register returns the 7 bit <code>CSP</code> field from the most recently received image information packet |
| Unused        | 31:7 | Unused                                                                                                                                                                                         |

**Table 884. IMG\_INFO\_SUBSAMPLING**

| Name            | Bits | Description                                                                                                                                                                                                                                        |
|-----------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SubSa code bits | 1:0  | When <b>lite mode</b> is on, write to this register to set the expected subsampling of the incoming video fields.<br>When <b>lite mode</b> is off, this register returns the SUBSA field from the most recently received image information packet. |
| Unused          | 31:2 | Unused                                                                                                                                                                                                                                             |

**Table 885. IMG\_INFO\_COSITING**

| Name             | Bits | Description                                                                                                                                                                                   |
|------------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cosite code bits | 1:0  | When <b>lite mode</b> is on, this register has no function.<br>When <b>lite mode</b> is off, this register returns the COSITE field from the most recently received image information packet. |
| Unused           | 31:2 | Unused                                                                                                                                                                                        |

**Table 886. IMG\_INFO\_FIELD\_COUNT**

| Name       | Bits | Description                                                                                                                                                                                              |
|------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Count bits | 6:0  | When <b>lite mode</b> is on, this register has no function.<br>When <b>lite mode</b> is off, this register returns the 7 bit FIELD_COUNT field from the most recently received image information packet. |
| Unused     | 31:7 | Unused                                                                                                                                                                                                   |

**Table 887. STATUS**

| Name       | Bits | Description                                                            |
|------------|------|------------------------------------------------------------------------|
| Status bit | 0    | 1 = Video and Vision Monitor is processing a video field, 0 otherwise. |
| Unused     | 31:1 | Unused                                                                 |

**Table 888. NUM\_GOOD\_FIELDS**

| Name                  | Bits  | Description                                                                                                                                                                                             |
|-----------------------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Number of good fields | 15:0  | Number of fields received since last reset of counters that match their expected dimensions. When <b>lite mode</b> is off, do not have the broken field flag set in their end-of-field control packets. |
| Unused                | 31:16 | Unused                                                                                                                                                                                                  |

**Table 889. NUM\_BROKEN\_FIELDS**

| Name             | Bits  | Description                                                                                                                                           |
|------------------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Number of broken | 15:0  | Number of fields received since last reset of counters that have the broken field flag set in their end-of-field control packets. Lite mode off only. |
| Unused           | 31:16 | Unused                                                                                                                                                |

**Table 890. NUM\_MISMATCH\_FIELDS**

| Name                      | Bits  | Description                                                                                         |
|---------------------------|-------|-----------------------------------------------------------------------------------------------------|
| Number of mismatch fields | 15:0  | Number of fields received since last reset of counters that do not match their expected dimensions. |
| Unused                    | 31:16 | Unused                                                                                              |

**Table 891. LAST\_NUM\_LINES**

| Name            | Bits  | Description                                 |
|-----------------|-------|---------------------------------------------|
| Number of lines | 16:0  | Number of lines in the last field received. |
| Unused          | 31:17 | Unused                                      |

**Table 892. LAST\_MIN\_WIDTH**

| Name               | Bits  | Description                                                   |
|--------------------|-------|---------------------------------------------------------------|
| Minimum line width | 16:0  | Minimum number of pixels per line in the last field received. |
| Unused             | 31:17 | Unused                                                        |

**Table 893. LAST\_MAX\_WIDTH**

| Name               | Bits  | Description                                                  |
|--------------------|-------|--------------------------------------------------------------|
| Maximum line width | 16:0  | Maximum number of pixels per line in the last field received |
| Unused             | 31:17 | Unused                                                       |

**Table 894. RESET\_COUNTERS**

| Name          | Bits | Description                                                 |
|---------------|------|-------------------------------------------------------------|
| Reset counter | 31:0 | Write any value to this register to reset the frame counts. |

## 46.5. Video and Vision Monitor Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_snoop_regs.h`

Include file: `intel_vvp_snoop.h`

**Table 895. Video and Vision Monitor IP Software API**

| Name                                                | Description                                                                                                                                                                                 |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>intel_vvp_snoop_init</code>                   | Initialize the Video and Vision Monitor instance                                                                                                                                            |
| <code>intel_vvp_core_*</code>                       | Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i> . Writable when <b>Lite</b> is on. Readable when <b>Lite</b> is off and <b>Debug features</b> is on. |
| <code>intel_vvp_snoop_get_lite_mode</code>          | Returns if <b>Lite mode</b> is on                                                                                                                                                           |
| <code>intel_vvp_snoop_get_debug_enabled</code>      | Returns if <b>Debug features</b> is on                                                                                                                                                      |
| <code>intel_vvp_snoop_get_pixels_in_parallel</code> | Returns the number of pixels streamed in parallel                                                                                                                                           |
| <code>intel_vvp_snoop_is_running</code>             | Returns if the IP is currently processing video data                                                                                                                                        |
| <code>intel_vvp_snoop_get_status</code>             | Reads the status register                                                                                                                                                                   |
| <i>continued...</i>                                 |                                                                                                                                                                                             |

| Name                                    | Description                                                        |
|-----------------------------------------|--------------------------------------------------------------------|
| intel_vvp_snoop_get_num_good_fields     | Returns the numbers of valid fields processed                      |
| intel_vvp_snoop_get_num_broken_fields   | Returns the numbers of broken fields processed (full variant)      |
| intel_vvp_snoop_get_num_mismatch_fields | EReturns the numbers of invalid/mismatched fields processed        |
| intel_vvp_snoop_get_last_num_lines      | Returns the number of lines in the last processed field            |
| intel_vvp_snoop_get_last_min_width      | Returns the width of the smallest line in the last processed field |
| intel_vvp_snoop_get_last_max_width      | Returns the width of the largest line in the last processed field  |
| intel_vvp_snoop_reset_counters          | Resets all debug counters                                          |

### intel\_vvp\_snoop\_init

#### Prototype

```
int intel_vvp_snoop_init(intel_vvp_snoop_instance *instance,
intel_vvp_core_base base);
```

#### Description

Initializes a Video and Vision Monitor instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

#### Arguments

instance - pointer to the intel\_vvp\_snoop\_instance software driver instance structure

base - base address of the register map

#### Returns

kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product\_id does not match with the Video and Vision Monitor product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpSnoopRegMapVersionErr if the register map is not supported

### intel\_vvp\_snoop\_get\_lite\_mode

#### Prototype

```
bool intel_vvp_snoop_get_lite_mode(intel_vvp_snoop_instance
*instance);
```

#### Description

Returns the value of the LITE\_MODE register. The instance must be a valid intel\_vvp\_snoop\_instance fully initialized

*Arguments* instance - pointer to the intel\_vvp\_snoop\_instance software driver instance structure

*Returns* true if the IP is parameterized in lite mode

#### intel\_vvp\_snoop\_get\_debug\_enabled

*Prototype*

```
bool
intel_vvp_snoop_get_debug_enabled(intel_vvp_snoop_instance
*instance);
```

*Description* Returns the value of the DEBUG\_ENABLED register. The instance must be a valid intel\_vvp\_snoop\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_snoop\_instance software driver instance structure

*Returns* true if the IP is parameterized with debug features enabled

#### intel\_vvp\_snoop\_get\_pixels\_in\_parallel

*Prototype*

```
uint8_t
intel_vvp_snoop_get_pixels_in_parallel(intel_vvp_snoop_instance
*instance);
```

*Description* Returns the value of the PIXELS\_IN\_PARALLEL register. The instance must be a valid intel\_vvp\_snoop\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_snoop\_instance software driver instance structure

*Returns* the number of pixels streamed in parallel in the monitored Video and Vision stream interface

#### intel\_vvp\_snoop\_is\_running

*Prototype*

```
bool intel_vvp_snoop_is_running(intel_vvp_snoop_instance*
instance);
```

*Description* Reads and returns the running bit of the STATUS register. The instance must be a valid intel\_vvp\_snoop\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_snoop\_instance software driver instance structure

*Returns* True is the IP is currently receiving/storing an input field, false when between two input fields

### intel\_vvp\_snoop\_get\_status

**Prototype**

```
uint32_t intel_vvp_snoop_get_status(intel_vvp_snoop_instance* instance);
```

**Description**

Reads the STATUS register. The instance must be a valid intel\_vvp\_snoop\_instance fully initialized.

**Arguments**

instance - pointer to the intel\_vvp\_snoop\_instance software driver instance structure

**Returns**

The content of the STATUS register

### intel\_vvp\_snoop\_get\_num\_good\_fields

**Prototype**

```
uint16_t intel_vvp_snoop_get_num_good_fields(intel_vvp_snoop_instance* instance);
```

**Description**

Reads the NUM\_GOOD\_FIELDS register. The instance must be a valid intel\_vvp\_snoop\_instance fully initialized.

**Arguments**

instance - pointer to the intel\_vvp\_snoop\_instance software driver instance structure

**Returns**

the number of valid fields processed so far

### intel\_vvp\_snoop\_get\_num\_broken\_fields

**Prototype**

```
uint16_t intel_vvp_snoop_get_num_broken_fields(intel_vvp_snoop_instance* instance);
```

**Description**

Reads the NUM\_BROKEN\_FIELDS register. The instance must be a valid intel\_vvp\_snoop\_instance fully initialized and configured in lite variant.

**Arguments**

instance - pointer to the intel\_vvp\_snoop\_instance software driver instance structure

**Returns**

the number of broken fields processed so far (counted using the broken flag in EOF packets in full variant)

### intel\_vvp\_snoop\_get\_num\_mismatch\_fields

**Prototype**

```
uint16_t intel_vvp_snoop_get_num_mismatch_fields(intel_vvp_snoop_instance* instance);
```



|                    |                                                                                                                                                           |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i> | Reads the NUM_MISMATCH_FIELDS register. The instance must be a valid intel_vvp_snoop_instance fully initialized                                           |
| <i>Arguments</i>   | instance - pointer to the intel_vvp_snoop_instance software driver instance structure                                                                     |
| <i>Returns</i>     | the number of invalid fields processed so far, counted by looking for discrepancies between the image information and the number of pixels/lines received |

#### intel\_vvp\_snoop\_get\_last\_num\_lines

|                    |                                                                                                            |
|--------------------|------------------------------------------------------------------------------------------------------------|
| <i>Prototype</i>   | <pre>uint32_t<br/>intel_vvp_snoop_get_last_num_lines(intel_vvp_snoop_instance<br/>*instance)</pre>         |
| <i>Description</i> | Reads the LAST_NUM_LINES register. The instance must be a valid intel_vvp_snoop_instance fully initialized |
| <i>Arguments</i>   | instance - pointer to the intel_vvp_snoop_instance software driver instance structure                      |
| <i>Returns</i>     | the number of lines in the last processed field                                                            |

#### intel\_vvp\_snoop\_get\_last\_min\_width

|                    |                                                                                                            |
|--------------------|------------------------------------------------------------------------------------------------------------|
| <i>Prototype</i>   | <pre>uint32_t<br/>intel_vvp_snoop_get_last_min_width(intel_vvp_snoop_instance<br/>*instance)</pre>         |
| <i>Description</i> | Reads the LAST_MIN_WIDTH register. The instance must be a valid intel_vvp_snoop_instance fully initialized |
| <i>Arguments</i>   | instance - pointer to the intel_vvp_snoop_instance software driver instance structure                      |
| <i>Returns</i>     | the width of the smallest line in the last processed field                                                 |

#### intel\_vvp\_snoop\_get\_last\_max\_width

|                    |                                                                                                            |
|--------------------|------------------------------------------------------------------------------------------------------------|
| <i>Prototype</i>   | <pre>uint32_t<br/>intel_vvp_snoop_get_last_max_width(intel_vvp_snoop_instance<br/>*instance)</pre>         |
| <i>Description</i> | Reads the LAST_MAX_WIDTH register. The instance must be a valid intel_vvp_snoop_instance fully initialized |
| <i>Arguments</i>   | instance - pointer to the intel_vvp_snoop_instance software driver instance structure                      |

*Returns* the width of the largest line in the last processed field

### intel\_vvp\_snoop\_reset\_counters

*Prototype*

```
int intel_vvp_snoop_reset_counters(intel_vvp_snoop_instance
*instance)
```

*Description* Resets all debug counters to zero

*Arguments* instance - pointer to the intel\_vvp\_snoop\_instance software driver instance structure

*Returns* kIntelVvpCoreOk for success, negative error code in case of failures  
kIntelVvpCoreInstanceErr if the instance is invalid



## 47. Video Frame Buffer IP

---

### 47.1. About the Video Frame Buffer IP

The IP buffers frames of video and can perform frame dropping and repeating to double or triple buffer frames. The IP stores frames in external memory, so the frame buffer has Avalon memory-mapped interfaces to allow connection to an external memory interface.

The frame buffer supports:

- Triple or double buffering for progressive video frames
- Double buffering for interlaced video fields
- Auxiliary control packets. Local storage for up to 255. The IP can optionally drop and repeat auxiliary packets with their associated frame.
- Maximum frame resolutions of 16384 by 16384 pixels with 1 to 8 pixels in parallel and any color space
- Configurable memory packing scheme
- Optional dropping of broken frames
- Frame statistics counters

The IP is available as full or lite variants. For more information on full and lite, refer to the *Altera Streaming Video Protocol Specification*. The Video Frame Buffer IP takes input resolution information from image information packets or extracts it using the register interface for lite variants.

An Avalon memory-mapped interface allows you to read frame statistics and turn the frame buffer output off and on at run time. You must have this interface for lite variants.

For details about latency and reset behavior for the Video Frame Buffer, refer to *Video and Vision IPs Functional Description*.

#### Related Information

[Altera Streaming Video Protocol Specification](#)

#### 47.1.1. Video Frame Buffer Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family and the number of supported bits per color sample, colors per pixel, and pixels in parallel.

**Table 896. Video Frame Buffer Performance and Resources**

The table shows ALM usage and  $f_{MAX}$  for a design with a 4K frame buffer with 2 pixels in parallel, 10 bits per color sample, 2 color planes, and separate control clock.

| Target Device                                 | ALMs | $f_{MAX}$ (MHz) |
|-----------------------------------------------|------|-----------------|
| Agilex 7 speed grade 2 (AGFA012R24A2E2V)      | 2845 | 684             |
| Cyclone 10 GX speed grade 5 (10CX220YF78015G) | 2647 | 442             |

## 47.2. Video Frame Buffer IP Parameters

The IP offers compile-time parameters.

| Parameter                                                              | Values                     | Description                                                                                                                                                                                                                                                                                                                                                                                               |
|------------------------------------------------------------------------|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Video Data Format</b>                                               |                            |                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Bits per color sample</b>                                           | 8 to 16                    | Select the number of bits per color sample.                                                                                                                                                                                                                                                                                                                                                               |
| <b>Number of color planes</b>                                          | 1 to 4                     | Select the number of color planes per pixel.                                                                                                                                                                                                                                                                                                                                                              |
| <b>Number of pixels in parallel</b>                                    | 1 to 8                     | Select the number of pixels in parallel.                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Maximum Frame Size</b>                                              |                            |                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Maximum field height</b>                                            | 32 to 16384                | Select the maximum height of incoming frames.                                                                                                                                                                                                                                                                                                                                                             |
| <b>Maximum field width</b>                                             | 32 to 16384                | Select the maximum width of incoming frames. If you use the IP exclusively to buffer frames with 420 sub-sampling, optionally halve the maximum field width entry in the GUI. Halving the entry optimizes memory footprint because of the more efficient 420 pixel-packing.                                                                                                                               |
| <b>Buffer Behavior</b>                                                 |                            |                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Enable dropping of input frames</b>                                 | On or off                  | Turn on to drop input frames. Turn on for triple-buffering; turn off for double buffering.                                                                                                                                                                                                                                                                                                                |
| <b>Enable repeating of output frames</b>                               | On or off                  | Turn on to repeat input frames. Turn on for triple-buffering; turn off for double buffering.                                                                                                                                                                                                                                                                                                              |
| <b>Enable the dropping of broken frames at the input</b>               | On or off                  | Turn on to drop broken frames. When off, the IP stores any broken frames.                                                                                                                                                                                                                                                                                                                                 |
| <b>Drop/repeat aux control metapackets with their associated frame</b> | On or off                  | Turn on if you want the IP to drop and repeat any associated auxiliary control packets with their frame. When off, these packets still buffer even if the IP drops their associated frame.                                                                                                                                                                                                                |
| <b>Aux control metapacket local storage</b>                            | 0, 8, 16, 32, 64, 128, 256 | Determines the total amount of local buffering provided for timestamp and other auxiliary control packets. If the IP encounters more control packets, it deletes older packets. Selecting 8 allows the IP to store 8 4-beat control packets (regardless of which frame they arrive with) or 16 2-beat, or 32 1-beat. Set this parameter to 0 if you do not require support for auxiliary control packets. |
| <b>Control</b>                                                         |                            |                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Lite mode</b>                                                       | On or off                  | Turn on to operate the frame buffer in lite mode.                                                                                                                                                                                                                                                                                                                                                         |
| <b>continued...</b>                                                    |                            |                                                                                                                                                                                                                                                                                                                                                                                                           |

| Parameter                                                     | Values                                                 | Description                                                                                                                                                                                                                                                                                                                                                                     |
|---------------------------------------------------------------|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Memory-mapped control interface</b>                        | On or off                                              | Turn on to read frame statistics and turn the frame buffer on and off using an Avalon memory-mapped interface.<br>The memory-mapped control interface is mandatory in lite mode.                                                                                                                                                                                                |
| <b>Separate clock for control interface</b>                   | On or off                                              | Turn on for a separate clock for the control interface.                                                                                                                                                                                                                                                                                                                         |
| <b>Debug features</b>                                         | On or off                                              | Turn on for debug features (not applicable for <b>lite mode</b> ).                                                                                                                                                                                                                                                                                                              |
| <b>Memory</b>                                                 |                                                        |                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Avalon memory-mapped host(s) local ports width</b>         | 16, 32, 64, 128, 256, 512, 1024                        | Select in bits the width of the Avalon memory-mapped host read and write ports.                                                                                                                                                                                                                                                                                                 |
| <b>Avalon memory-mapped host(s) local ports address width</b> | 8 to 32                                                | Select in bits the width of the Avalon memory-mapped host read and write ports. It must be sufficient to fully address the last buffer.                                                                                                                                                                                                                                         |
| <b>The depth of the write FIFO</b>                            | 32, 64, 128, 256, 512, 1024, 2048                      | Specify the depth of the write FIFO buffer. Each FIFO buffer entry holds one word the width of the specified Avalon memory-mapped local port width. You must specify a FIFO depth of at least twice the specified burst target so that at least 2 bursts can be held at any one time. Increase the FIFO depth to improve resilience to latency on the Avalon memory-mapped bus. |
| <b>Avalon memory-mapped write burst target</b>                | 2,4,8,16,32,64                                         | Select the burst target for writes. Longer bursts provide more efficiency on the bus but require more local storage in the write FIFO buffer.                                                                                                                                                                                                                                   |
| <b>The depth of the read FIFO</b>                             | 32,64,128,256,512,1024,2048                            | Specify the depth of the read FIFO buffer. Each FIFO buffer entry holds one word the width of the specified Avalon memory-mapped local port width. You must specify a FIFO depth of at least twice the specified burst target so that at least 2 bursts can be held at any one time. Increase the FIFO depth to improve resilience to latency on the Avalon memory-mapped bus.  |
| <b>Avalon memory-mapped read burst target</b>                 | 2,4,8,16,32,64                                         | Select the burst target for reads. Longer bursts provide more efficiency on the bus but require more local storage in the read FIFO buffer.                                                                                                                                                                                                                                     |
| <b>Frame buffer memory base address</b>                       | 0x0000_0000 – 0x7FFF_FFFF                              | Select the base address for the frame buffer to store frames.                                                                                                                                                                                                                                                                                                                   |
| <b>Inter-buffer stride</b>                                    | Depends on video frame format and maximum frame size.  | Set a stride (in bytes) large enough to separate frames of the specified maximum height and width. The GUI gives an error if the stride is too low.                                                                                                                                                                                                                             |
| <b>Inter-line stride</b>                                      | Depends on video frame format and maximum frame width. | Set a stride (in bytes) large enough to separate lines of the specified maximum width. The GUI gives an error if the stride is too low.                                                                                                                                                                                                                                         |
| <b>Packing method</b>                                         | <b>Perfect, color</b> or <b>pixel</b>                  | Perfect packing minimizes memory footprint of stored frames but increases complexity and therefore size of the frame buffer slightly. Color packing leaves spaces in memory between colors if colors do not pack into                                                                                                                                                           |

*continued...*

| Parameter                                                            | Values    | Description                                                                                                                                                           |
|----------------------------------------------------------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                      |           | memory words exactly. Pixel packing leaves spaces in memory between pixels if pixels do not pack into memory words exactly.                                           |
| <b>Separate clock for the Avalon memory-mapped host interface(s)</b> | On or off | A separate clock allows the control and data portions of the frame buffer to run at their maximum clock frequencies. Turn on separate clocks for highest performance. |

Figure 126. Frame Buffer GUI

Parameters System Info Component Ins Domains

System: clipper Path: intel\_vfp\_vfb\_0

HDL entity name: clipper\_intel\_vfp\_vfb\_0 IP file: ip/clipper/clipper\_intel\_vfp\_vfb\_0.ip

Any changes here will be written out to disk when the system is saved.

### Video Frame Buffer Intel FPGA IP

intel\_vfp\_vfb [Details](#)

**Video Data Format**

Bits per color sample: 10

Number of color planes: 2

Number of pixels in parallel: 2

**Maximum Frame Size**

Maximum field height: 2160

Maximum field width: 4096

**Buffer Behaviour**

☒ Enable dropping of input frames

☒ Enable repeating of output frames

☒ Enable the dropping of broken frames at the input

☒ Drop/repeat control (aux and timestamp) packets with their associated frame

Control (including aux) packet local storage: 8

**Control**

☐ Lite mode

☒ Memory mapped control interface

☐ Separate clock for control interface

☐ Debug features

**Memory**

Avalon-MM host(s) local ports width: 64

Avalon-MM host(s) local ports address width: 32

The depth of the write FIFO.: 32

Av-MM write burst target: 16

The depth of the read FIFO.: 32

Av-MM read burst target: 16

Frame buffer memory base address: 0x00000000

Inter-buffer stride: 0x10000000

Inter-line stride: 0x00008000

Packing method: PERFECT

☒ Separate clock for the Avalon-MM host interface(s)

**Parameterization Messages**

| Type        | Message                                                                         |
|-------------|---------------------------------------------------------------------------------|
| Information | 3 Buffers will be employed for storage, total storage memory required is 786432 |

## 47.3. Video Frame Buffer IP Functional Description

The IP receives incoming video frames from its Altera Streaming Video input and writes them to external memory via an Avalon memory-mapped interface.

Received frames are available for output when the Avalon memory-mapped agent accepts the last beat of the last line.

The IP starts producing output frames when you set the `OUTPUT_CONTROL` bit. It selects which frame to output based on the buffer behavior parameters. You can set buffer behavior so that the IP drops incoming frames if no free buffers are available or repeats an output frame if no new frames exist to read.

### Register Behavior

Bit [0] of the `INPUT_STATUS` register goes high when the IP starts writing the first frame. It goes low after the IP finishes handling the last expected input line, or after receiving the last line of a short frame. It returns high when the IP starts writing the next frame.

Bit [0] of the `OUTPUT_STATUS` register goes high when the IP starts reading the first frame. It goes low in between output frames and remains low if it has no valid frames to output.

Applications which need to know when the IP receives or creates frames can poll these status registers.

### Triple-buffering Behavior

Select triple-buffering by turning on both **Enable dropping of input frames** and **Enable repeating of output frames** in the GUI.

For triple-buffering, the frame buffer uses three frame buffers, with base addresses calculated from the base address and interbuffer stride you select. The stride is the separation between frames in memory and is always fixed to the parameterized value regardless of the size of frames the IP receives. Similarly, lines are always spaced in memory according to the interline stride, regardless of the size of the line the IP receives. The IP has three buffers for triple buffering.

### Broken Frame Behavior

For full variants, (lite mode off), the IP processes and creates broken frames (frames with incorrect dimensions or with the broken bit in their associated image information packet set) in the same way as good frames. Alternatively, it discards them if you turn on **Enable the dropping of broken frames**.

For full variants, (lite mode off), the IP detects broken frames by checking the received frame dimensions against the expected dimensions. If the IP receives and measures a broken frame without the broken bit set, the IP sets the broken bit for that frame, if the IP produces it at the output.

For full variants, (lite mode off), broken frames may result in the IP creating frames of dimensions smaller or larger than the specified image information packet and may result in it displaying partial frames.

The IP clips frames with heights or widths greater than the maxima you specify in the GUI to those values.

### Double-buffering Behavior

Select double-buffering by turning off both **Enable dropping of input frames** and **Enable repeating of output frames** in the GUI.

Double buffering uses only two frame buffers, with base addresses calculated from the base address and interbuffer stride you specify.

If no free buffers are available and a new frame of video is pending at the input, frame writing stalls. The IP holds off the input by de-asserting its `TREADY` output until a free buffer becomes available.

If the IP finishes creating output from one buffer and the other buffer does not hold a complete, new frame, the IP stalls its output by lowering `TVALID`.

### Auxiliary control packet support

Turn on auxiliary packets by selecting some local storage in the GUI. You can allocate local storage slots up to a maximum of 256 4-beat auxiliary control packets (64 bits of data).

If you allocate no local storage, the IP discards any auxiliary control packets it receives.

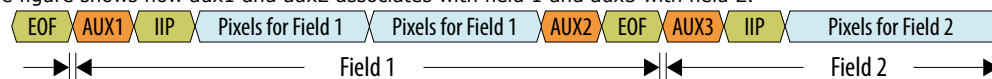
If you allocate local storage, every received frame consumes at least one of the 64 bit storage slots. The IP marks slots as empty if a frame has no associated auxiliary control packets.

The IP classes any auxiliary control packets the IP receives as owned by a parent frame according to the following rules in the Altera Streaming Video Protocol Specification, which are reproduced here :

Pairs of image information and end-of-field control packets surround one field of video. The IP associates any auxiliary control packets before the current field's end-of-field control packet with that field. In addition, the IP associates any auxiliary control packets that occur after the end-of-field control packet of the preceding field with the current field.

**Figure 127. Auxiliary control packets**

The figure shows how aux1 and aux2 associates with field 1 and aux3 with field 2.



The IP associates the frame with any auxiliary control packets, if you turn on **Drop/repeat auxiliary metapackets with their associated frame**. If frame repeating is on, the IP repeats auxiliary packets with their parent frame. If frame dropping is on, the IP drops any auxiliary packets associated with a dropped frame.

If you turn off **Drop/repeat auxiliary control metapackets with their associated frame**, the IP produces all auxiliary packets once, in the order in which it receives them, regardless of whether it drops or repeats their parent frame.



If the local storage becomes full, the IP raises the auxiliary FIFO overflow sticky bit in the `OUTPUT_STATUS` register (bit 1) and flushes the contents and buffering starts again. Ensure you have sufficient storage for your application.

### Frame statistics

The registers from `0x0144` to `0x0158` contain statistics on the numbers of frames the IP receives, drops, produces, and repeats. The `NUM_INVALID_FIELDS` register includes frames that the IP indicates as invalid via the broken bit and frames that the IP measures as invalid where their dimensions do not match the expected dimensions.

### Latency

The frame buffer latency depends on the availability of the external memory interfaces, which may backpressure the IP's read and write host interfaces via the `av_mm_mem_read_host_waitrequest` and `av_mm_mem_write_host_waitrequest` signals.

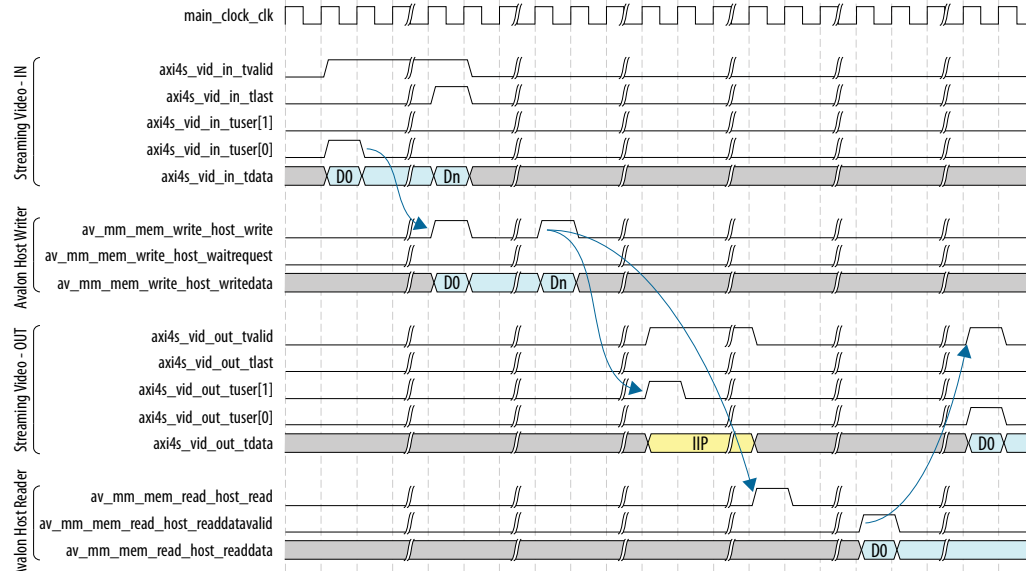
The streaming video output experiences backpressure via the `axi4s_vid_out_tready` input. The worst-case latency figures are when the frame buffer experiences no backpressure. Any backpressure increases these latencies by the same amount of cycles.

Worst case latency is on start-up, so these figures show where the frame buffer is writing and then reading the first frame after reset. The latency figures are for when the host interfaces do not have a separate clock, so all interfaces operate from the same clock. Latencies under steady-state conditions are less than or equal to these figures.

The latency figures are the same for both full and lite variants of the IP, as measured from the start of the first data packet of a new frame. The figures show the IP producing the image information packet. In lite variants of the IP, it does not produce this packet but the timing is otherwise identical.

**Figure 128. Video frame buffer latencies**

This figure only shows some signals. The figure shows four of the frame buffer interfaces: The Altera Streaming Video input, the Avalon memory-mapped host writer, the Altera Streaming Video output and the Avalon memory-mapped host reader. The figure shows the start of a new frame of video on the streaming video input.



**Table 897. Latencies**

The table shows latencies between the interfaces.

| Initiating event                                                              | Resultant event                                                                                     | Latency between events (measured in clock cycles) |
|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------|
| axi4s_vid_in_tuser[0] strobe to indicate start of frame.                      | First av_mm_mem_write_host_write strobe of a frame write.                                           | 44                                                |
| Last av_mm_mem_write_host_write strobe of a frame write.                      | First valid cycle of outgoing image information packet on the streaming output. Full variants only. | 43                                                |
| Last av_mm_mem_write_host_write strobe of a frame write.                      | First av_mm_mem_read_host_read strobe of a frame read.                                              | 74                                                |
| av_mm_mem_read_host_readdatavalid strobe high to return the first frame data. | axi4s_vid_out_tuser[0] strobe to indicate start of frame on streaming output.                       | 20                                                |

### 47.3.1. Video Frame Buffer IP Interfaces

**Table 898. Video Frame Buffer IP Interfaces**

| Name                     | Direction | Width | Description              |
|--------------------------|-----------|-------|--------------------------|
| <b>Clocks and resets</b> |           |       |                          |
| main_clock_clk           | In        | 1     | AXI4-S processing clock. |
| main_reset_rst           | In        | 1     | AXI4-S processing reset. |
| <i>continued...</i>      |           |       |                          |

| Name                                                                                                           | Direction | Width | Description                              |
|----------------------------------------------------------------------------------------------------------------|-----------|-------|------------------------------------------|
| control_clock_clk                                                                                              | In        | 1     | Optional control agent interface clock.  |
| control_reset_reset                                                                                            | In        | 1     | Optional control agent interface reset.  |
| <b>Control interfaces</b>                                                                                      |           |       |                                          |
| av_mm_control_agent_address                                                                                    | In        | 7     | Avalon memory-mapped agent address       |
| av_mm_control_agent_write                                                                                      | In        | 1     | Avalon memory-mapped agent write.        |
| av_mm_control_agent_writedata                                                                                  | In        | 32    | Avalon memory-mapped agent write data.   |
| av_mm_control_agent_byteenable                                                                                 | In        | 4     | Avalon memory-mapped agent byte enable.  |
| av_mm_control_agent_read                                                                                       | In        | 1     | Avalon memory-mapped agent read.         |
| av_mm_control_agent_readdata                                                                                   | Out       | 32    | Avalon memory-mapped agent read data.    |
| av_mm_control_agent_readdatavalid                                                                              | Out       | 1     | Avalon memory-mapped agent read.         |
| av_mm_control_agent_waitrequest                                                                                | Out       | 1     | Avalon memory-mapped agent wait request. |
| <b>Altera Streaming Video interfaces</b>                                                                       |           |       |                                          |
| axi4s_vid_in_tdata                                                                                             | In        | (141) | AXI4-S data in.                          |
| axi4s_vid_in_tvalid                                                                                            | In        | 1     | AXI4-S data valid.                       |
| axi4s_vid_in_tuser[0]                                                                                          | In        | 1     | AXI4-S start of video frame.             |
| axi4s_vid_in_tuser[1]                                                                                          | In        | 1     | AXI4-S control or data packet.           |
| axi4s_vid_in_tuser[N-1:2]                                                                                      | In        | (142) | Unused.                                  |
| axi4s_vid_in_tlast                                                                                             | In        | 1     | AXI4-S end of packet .                   |
| axi4s_vid_in_tready                                                                                            | Out       | 1     | AXI4-S data ready.                       |
| axi4s_vid_out_tdata                                                                                            | Out       | (141) | AXI4-S data out.                         |
| axi4s_vid_out_tvalid                                                                                           | Out       | 1     | AXI4-S data valid.                       |
| axi4s_vid_out_tuser[0]                                                                                         | Out       | 1     | AXI4-S start of video frame.             |
| axi4s_vid_out_tuser[1]                                                                                         | Out       | 1     | AXI4-S control or data packet.           |
| axi4s_vid_out_tuser[N-1:2]                                                                                     | Out       | (142) | Unused.                                  |
| axi4s_vid_out_tlast                                                                                            | Out       | 1     | AXI4-S end of packet .                   |
| axi4s_vid_out_tready                                                                                           | In        | 1     | AXI4-S data ready                        |
| <b>Avalon memory-mapped host interfaces</b>                                                                    |           |       |                                          |
| Widths for address, data, and burst count buses are as you specify in the GUI. The table shows example widths. |           |       |                                          |
| mem_clock_clk                                                                                                  | In        | 1     | Optional host interface clock.           |
| mem_reset_reset                                                                                                | In        | 1     | Optional host interface reset.           |
| continued...                                                                                                   |           |       |                                          |

(141) The equation gives all tdata widths sizes in these interfaces:  

$$\max(2, \text{ceil}((\text{bits per color sample} \times \text{number of color planes})/8)) \times \text{pixels in parallel} \times 8$$

(142) This equation gives all tuser widths sizes in these interfaces  $N = \text{ceil}(\text{tdata width} / 8)$

| Name                              | Direction | Width | Description                                  |
|-----------------------------------|-----------|-------|----------------------------------------------|
| av_mm_mem_write_host_address      | Out       | 32    | Avalon memory-mapped host address            |
| av_mm_mem_write_host_write        | Out       | 1     | Avalon memory-mapped host write.             |
| av_mm_mem_write_host_burstcount   | Out       | 5     | Avalon memory-mapped host write burst count. |
| av_mm_mem_write_host_writedata    | Out       | 64    | Avalon memory-mapped host write data.        |
| av_mm_mem_write_host_waitrequest  | In        | 1     | Avalon memory-mapped host wait request.      |
| av_mm_mem_read_host_address       | Out       | 32    | Avalon memory-mapped host address            |
| av_mm_mem_read_host_read          | Out       | 1     | Avalon memory-mapped host read.              |
| av_mm_mem_read_host_burstcount    | Out       | 5     | Avalon memory-mapped host read burst count.  |
| av_mm_mem_read_host_readdata      | In        | 64    | Avalon memory-mapped host read data.         |
| av_mm_mem_read_host_readdatavalid | In        | 1     | Avalon memory-mapped host read data valid.   |
| av_mm_mem_read_host_waitrequest   | In        | 1     | Avalon memory-mapped host wait request.      |

### Related Information

Video and Vision Processing IP Interfaces on page 30

## 47.4. Video Frame Buffer IP Registers

Each register is either read-only (RO) or read-write (RW).

**Table 899. Video Frame Buffer IP Registers**

In the software API the register names appear with a prefix of INTEL\_VVP, INTEL\_VVP\_CORE, or INTEL\_VVP\_VIDEO\_FRAME\_BUFFER as appropriate and with an optional REG suffix

| Address                    | Register      | Access     |      | Description                                                                                                                              |
|----------------------------|---------------|------------|------|------------------------------------------------------------------------------------------------------------------------------------------|
|                            |               | Lite (143) | Full |                                                                                                                                          |
| Parameterization registers |               |            |      |                                                                                                                                          |
| 0x0000                     | VID_PID       | RO         | RO   | Read this register for the Video Frame Buffer product ID. This register always returns 0x6AF7_0237.                                      |
| 0x0004                     | VERSION       | RO         | RO   | Read this register for the IP version information.                                                                                       |
| 0x0008                     | LITE_MODE     | RO         | RO   | Read this register to determine if <b>Lite mode</b> is on or off.<br>This register returns 0 when <b>Lite mode</b> is off and 1 when on. |
| 0x000C                     | DEBUG_ENABLED | RO         | RO   | Read this register to determine if <b>Debug features</b> is on.                                                                          |
| 0x0010                     | MAX_WIDTH     | RO         | RO   | Read this register to determine the maximum supported frame width.                                                                       |
| 0x0014                     | MAX_HEIGHT    | RO         | RO   | Read this register to determine the maximum supported frame height.                                                                      |
| continued...               |               |            |      |                                                                                                                                          |

(143) When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

| Address                                                                 | Register               | Access                |      | Description                                                                                                                                                                                                |
|-------------------------------------------------------------------------|------------------------|-----------------------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                         |                        | Lite <sup>(143)</sup> | Full |                                                                                                                                                                                                            |
| 0x0018                                                                  | DROP_ENABLED           | RO                    | RO   | Read this register to see if <b>Enable dropping of input frames</b> is on.                                                                                                                                 |
| 0x001C                                                                  | REPEAT_ENABLED         | RO                    | RO   | Read this register to see if <b>Enable repeating of output frames</b> is on.                                                                                                                               |
| 0x0020                                                                  | INVALID_FRAMES_DROPPED | RO                    | RO   | Read this register to see if <b>Enable the dropping of broken frames at the input</b> is on.                                                                                                               |
| 0x0024                                                                  | MEM_BASE_ADDR          | RO                    | RO   | Read this register for the base address of stored frames in memory.                                                                                                                                        |
| 0x0028                                                                  | MEM_BUFFER_STRIDE      | RO                    | RO   | Read this register for the buffer stride in memory.                                                                                                                                                        |
| 0x002C                                                                  | MEM_LINE_STRIDE        | RO                    | RO   | Read this register for the line stride in memory.                                                                                                                                                          |
| 0x0030                                                                  | BPS                    | RO                    | RO   | Read this register for the number of bits per symbol configured.                                                                                                                                           |
| 0x0034                                                                  | NUMBER_OF_COLOR_PLANES | RO                    | RO   | Read this register for the number of color planes.                                                                                                                                                         |
| 0x0038                                                                  | PIXELS_IN_PARALLEL     | RO                    | RO   | Read this register for the number of pixels in parallel.                                                                                                                                                   |
| 0x003C                                                                  | PACKING                | RO                    | RO   | Read this register for the pixel packing scheme.                                                                                                                                                           |
| 0x0040 to<br>0x011F                                                     |                        |                       |      | Unused.                                                                                                                                                                                                    |
| <b>Control and Debug registers</b>                                      |                        |                       |      |                                                                                                                                                                                                            |
| For more details about these registers, refer to <i>Control Packets</i> |                        |                       |      |                                                                                                                                                                                                            |
| 0x0120                                                                  | IMG_INFO_WIDTH         | RW                    | RO   | When you turn on <b>lite mode</b> , the expected width of the incoming video fields. When you turn off <b>lite mode</b> , the received width in the IP derives from the image information packets.         |
| 0x0124                                                                  | IMG_INFO_HEIGHT        | RW                    | RO   | When you turn on <b>lite mode</b> , the expected height of the incoming video fields. When you turn off <b>lite mode</b> , the received height in the IP derives from the image information packets.       |
| 0x0128                                                                  | IMG_INFO_INTERLACE     | RW                    | RO   | When you turn on <b>lite mode</b> , the expected interlace information of the incoming video fields. When you turn off <b>lite mode</b> , the received interlace information in image information packets. |
| 0x012C                                                                  | RESERVED               | RW                    | RO   | Unused.                                                                                                                                                                                                    |
| 0x0130                                                                  | IMG_INFO_COLORSPACE    | RW                    | RO   | When you turn on <b>lite mode</b> , the expected color space of the incoming video fields. When you turn off <b>lite mode</b> , the received color space in image information packets.                     |
| 0x0134                                                                  | IMG_INFO_SUBSAMPLING   | RW                    | RO   | When you turn on <b>lite mode</b> , the expected chroma subsampling of the incoming video fields. When you turn off <b>lite mode</b> , the received chroma subsampling in image information packets.       |
| 0x0138                                                                  | IMG_INFO_COSITING      | RW                    | RO   | When you turn on <b>lite mode</b> , the expected chroma co-siting of the incoming video fields. When you turn off <b>lite mode</b> , the received chroma co-siting in image information packets.           |
| <i>continued...</i>                                                     |                        |                       |      |                                                                                                                                                                                                            |

<sup>(143)</sup> When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

| Address | Register             | Access                |      | Description                                                                                                                                                                                                                      |
|---------|----------------------|-----------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         |                      | Lite <sup>(143)</sup> | Full |                                                                                                                                                                                                                                  |
| 0x013C  | IMG_INFO_FIELD_COUNT | -                     | RO   | The received field count field in image information packets.                                                                                                                                                                     |
| 0x0140  | INPUT_STATUS         | RO                    | RO   | Bit 0 input status bit.<br>1 = Video Frame Buffer is receiving and processing a video field, 0 otherwise.                                                                                                                        |
| 0x0144  | NUM_INPUT_FIELDS     | RO                    | RO   | Read this register for a 32 bit count of frames received. The counter still increments if the frame is subsequently dropped.                                                                                                     |
| 0x0148  | NUM_DROPPED_FIELDS   | RO                    | RO   | Read this register for a 32 bit count of frames dropped.                                                                                                                                                                         |
| 0x014C  | NUM_INVALID_FIELDS   | RO                    | RO   | Read this register for a 32 bit count of invalid frames received.                                                                                                                                                                |
| 0x0150  | OUTPUT_STATUS        | RO                    | RO   | Bit 0: output status bit.<br>1 = Video Frame Buffer is outputting a video field, 0 otherwise.<br>Bit 1: AUX FIFO overflow bit<br>1 = Auxiliary FIFO is overflowing, 0 otherwise. Reset the video frame buffer to reset this bit. |
| 0x0154  | NUM_OUTPUT_FIELDS    | RO                    | RO   | Read this register for a 32-bit count of frames output. The counter increments for frame repeats.                                                                                                                                |
| 0x0158  | NUM_REPEATED_FIELDS  | RO                    | RO   | Read this register for a 32-bit count of repeated frames.                                                                                                                                                                        |
| 0x015C  | OUTPUT_CONTROL       | RW                    | RW   | Write a 1 to this register for the frame buffer to start producing buffered frames.<br>Write a 0 to this register for the frame buffer to stop producing buffered frames.                                                        |

### Register Bit Descriptions

**Table 900. VID\_PID**

| Name                                   | Bits | Description                                                                                                                                                                                           |
|----------------------------------------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frame buffer version ID and product ID | 31:0 | This register always returns 0x6AF7_0237 <ul style="list-style-type: none"> <li>15:0 is the product ID and always returns 0x0237</li> <li>31:16 is the vendor ID and always returns 0x6AF7</li> </ul> |

**Table 901. VERSION**

| Name                 | Bits  | Description                          |
|----------------------|-------|--------------------------------------|
| Register map version | 7:0   | Register map version.                |
| IP patch revision    | 15:8  | -                                    |
| IP update revision   | 23:16 | Updated when the IP version changes. |
| IP major revision    | 31:24 | Updated when the IP version changes. |

<sup>(143)</sup> When you turn on **lite mode**, registers are RW only if you turn on **Debug features**, otherwise they are WO. For full, turn off **lite mode**.

**Table 902. LITE\_MODE**

| Name                           | Bits | Description                                 |
|--------------------------------|------|---------------------------------------------|
| Lite mode parameterization bit | 0    | Returns 1 if you turn on <b>lite mode</b> . |
| Unused                         | 31:1 | Unused.                                     |

**Table 903. DEBUG\_ENABLED**

| Name                                | Bits | Description                                      |
|-------------------------------------|------|--------------------------------------------------|
| Debug features parameterization bit | 0    | Returns 1 if you turn on <b>Debug features</b> . |
| Unused                              | 31:1 | Unused.                                          |

**Table 904. MAX\_WIDTH**

| Name      | Bits | Description                                              |
|-----------|------|----------------------------------------------------------|
| Max width | 31:0 | This register returns the maximum supported frame width. |

**Table 905. MAX\_HEIGHT**

| Name       | Bits | Description                                               |
|------------|------|-----------------------------------------------------------|
| Max height | 31:0 | This register returns the maximum supported frame height. |

**Table 906. DROP\_ENABLED**

| Name         | Bits | Description                                                                                 |
|--------------|------|---------------------------------------------------------------------------------------------|
| Drop enabled | 0    | This register returns 1 if <b>Enable dropping of input frames</b> is on and 0 if it is off. |

**Table 907. REPEAT\_ENABLED**

| Name           | Bits | Description                                                                                   |
|----------------|------|-----------------------------------------------------------------------------------------------|
| Repeat enabled | 0    | This register returns 1 if <b>Enable repeating of output frames</b> is on and 0 if it is off. |

**Table 908. INVALID\_FRAMES\_DROPPED**

| Name                   | Bits | Description                                                                                             |
|------------------------|------|---------------------------------------------------------------------------------------------------------|
| Invalid frames dropped | 0    | This register returns 1 if <b>Enable the dropping of broken frames at the input</b> and 0 if it is off. |

**Table 909. MEM\_BASE\_ADDR**

| Name             | Bits | Description                                                        |
|------------------|------|--------------------------------------------------------------------|
| Mem base address | 31:0 | This register returns the base address of stored frames in memory. |

**Table 910. MEM\_BUFFER\_STRIDE**

| Name              | Bits | Description                                        |
|-------------------|------|----------------------------------------------------|
| Mem buffer stride | 31:0 | This register returns the buffer stride in memory. |

**Table 911. MEM\_LINE\_STRIDE**

| Name            | Bits | Description                                      |
|-----------------|------|--------------------------------------------------|
| Mem line stride | 31:0 | This register returns the line stride in memory. |

**Table 912. BPS**

| Name | Bits | Description                                          |
|------|------|------------------------------------------------------|
| BPS  | 31:0 | This register returns the number of bits per symbol. |

**Table 913. NUMBER\_OF\_COLOR\_PLANES**

| Name                   | Bits | Description                                       |
|------------------------|------|---------------------------------------------------|
| Number of color planes | 31:0 | This register returns the number of color planes. |

**Table 914. PIXELS\_IN\_PARALLEL**

| Name               | Bits | Description                                             |
|--------------------|------|---------------------------------------------------------|
| Pixels in parallel | 31:0 | This register returns the number of pixels in parallel. |

**Table 915. PACKING**

| Name    | Bits | Description                                                                                                |
|---------|------|------------------------------------------------------------------------------------------------------------|
| Packing | 31:0 | This register returns the packing scheme:<br>0 = Perfect packing<br>1 = Color packing<br>2 = Pixel packing |

**Table 916. IMG\_INFO\_WIDTH**

| Name       | Bits  | Description                                                                                                                                                                                                                                                                                                                                     |
|------------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Width bits | 15:0  | When you turn on <b>lite mode</b> , write to this register to set the expected width of the incoming video fields.<br>When you turn off <b>lite mode</b> and turn on <b>Debug features</b> , this register uses the <code>width-1</code> field from the most recently received image information packet and adds 1 to return a value for width. |
| unused     | 31:16 | Unused.                                                                                                                                                                                                                                                                                                                                         |

**Table 917. IMG\_INFO\_HEIGHT**

| Name        | Bits  | Description                                                                                                                                                                                                                                                                                                                                         |
|-------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Height bits | 15:0  | When you turn on <b>lite mode</b> , write to this register to set the expected height of the incoming video fields.<br>When you turn off <b>lite mode</b> and turn on <b>Debug features</b> , this register reads the <code>height-1</code> field from the most recently received image information packet and adds 1 to return a value for height. |
| unused      | 31:16 | Unused.                                                                                                                                                                                                                                                                                                                                             |

**Table 918. IMG\_INFO\_INTERLACE**

| Name                  | Bits | Description                                                                                                                                                                                                                                                                                                             |
|-----------------------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interlace Nibble bits | 3:0  | When you turn on <b>lite mode</b> , write to this register to set the expected interlacing of the incoming video fields.<br>When you turn off <b>lite mode</b> and turn on <b>Debug features</b> , this register returns the <code>intlaceNibble</code> field from the most recently received image information packet. |
| unused                | 31:4 | Unused.                                                                                                                                                                                                                                                                                                                 |



**Table 919. IMG\_INFO\_COLORSPACE**

| Name          | Bits | Description                                                                                                                                                                                                                                                                                            |
|---------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CSP code bits | 6:0  | When you turn on <b>lite mode</b> , write to this register to set the expected color space of the incoming video fields.<br>When you turn off <b>lite mode</b> and turn on <b>Debug features</b> , this register returns the 7 bit CSP field from the most recently received image information packet. |
| unused        | 31:7 | Unused.                                                                                                                                                                                                                                                                                                |

**Table 920. IMG\_INFO\_SUBSAMPLING**

| Name               | Bits | Description                                                                                                                                                                                                                                                                                               |
|--------------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CSPSubSa code bits | 1:0  | When you turn on <b>lite mode</b> , write to this register to set the expected chroma subsampling of the incoming video fields.<br>When you turn off <b>lite mode</b> and turn on <b>Debug features</b> , this register returns the SUBSA field from the most recently received image information packet. |
| unused             | 31:2 | Unused.                                                                                                                                                                                                                                                                                                   |

**Table 921. IMG\_INFO\_COSITING**

| Name             | Bits | Description                                                                                                                                                                                                                                                                                              |
|------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cosite code bits | 1:0  | When you turn on <b>lite mode</b> , write to this register to set the expected chroma co-siting of the incoming video fields.<br>When you turn off <b>lite mode</b> and turn on <b>Debug features</b> , this register returns the COSITE field from the most recently received image information packet. |
| unused           | 31:2 | Unused.                                                                                                                                                                                                                                                                                                  |

**Table 922. IMG\_INFO\_FIELD\_COUNT**

| Name       | Bits | Description                                                                                                                                                                                                                                              |
|------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Count bits | 6:0  | When you turn on <b>lite mode</b> , this register has no function.<br>When you turn off <b>lite mode</b> and turn on <b>Debug features</b> , this register returns the 7 bit FIELD_COUNT field from the most recently received image information packet. |
| unused     | 31:7 | Unused.                                                                                                                                                                                                                                                  |

**Table 923. INPUT\_STATUS**

| Name       | Bits | Description                                                                    |
|------------|------|--------------------------------------------------------------------------------|
| Status bit | 0    | 1 = Video Frame Buffer is receiving and processing a video field, 0 otherwise. |

**Table 924. NUM\_INPUT\_FIELDS**

| Name             | Bits | Description                                                                                                                  |
|------------------|------|------------------------------------------------------------------------------------------------------------------------------|
| NUM_INPUT_FIELDS | 31:0 | Read this register for a 32 bit count of frames received. The counter still increments if the frame is subsequently dropped. |

**Table 925. NUM\_DROPPED\_FIELDS**

| Name               | Bits | Description                                   |
|--------------------|------|-----------------------------------------------|
| NUM_DROPPED_FIELDS | 15:0 | Read these bits for a count of frames dropped |

**Table 926. NUM\_INVALID\_FIELDS**

| Name               | Bits | Description                                            |
|--------------------|------|--------------------------------------------------------|
| NUM_INVALID_FIELDS | 31:0 | Read these bits for a count of invalid frames received |

**Table 927. OUTPUT\_STATUS**

| Name          | Bits | Description                                                                                                                                                                                                                |
|---------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OUTPUT_STATUS | 0    | Bit 0: output status bit.<br>1 = Video Frame Buffer is outputting a video field, 0 otherwise.<br>Bit 1: AUX FIFO overflow bit<br>1 = Aux FIFO is overflowing, 0 otherwise. Reset the video frame buffer to reset this bit. |

**Table 928. NUM\_OUTPUT\_FIELDS**

| Name              | Bits | Description                                                                                  |
|-------------------|------|----------------------------------------------------------------------------------------------|
| NUM_OUTPUT_FIELDS | 31:0 | Read these bits for a count of frames output. The counter also increments for frame repeats. |

**Table 929. NUM\_REPEATED\_FIELDS**

| Name                | Bits | Description                                                                                                                                        |
|---------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| NUM_REPEATED_FIELDS | 31:0 | Read these bits for a count of repeated frames. Subtract this value from NUM_OUTPUT_FIELDS to determine how many frames the IP produces once only. |

**Table 930. OUTPUT\_CONTROL**

| Name               | Bits | Description                                                                                                                                                                |
|--------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OUTPUT_CONTROL BIT | 0    | Write a 1 to this register for the frame buffer to start outputting buffered frames.<br>Write a 0 to this register for the frame buffer to stop producing buffered frames. |

## 47.5. Video Frame Buffer IP Software API

The IP has a software driver for software control of the IP at run time. The IP does not fit any of the generic device models provided by the Nios II HAL. It exposes a set of dedicated accessors to the control and status registers. The driver structure inherits the base driver structure so all common methods described in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_vfb_regs.h`

Include file: `intel_vvp_vfb.h`

**Table 931. Video Frame Buffer IP API Reference**

| Name                                     | Description                                                                                                                                                                                  |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>intel_vvp_vfb_init</code>          | Initialize the Frame Buffer instance.                                                                                                                                                        |
| <code>Intel_vvp_core_*</code>            | .Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i> . Writable when <b>Lite</b> is on. Readable when <b>Lite</b> is off and <b>Debug features</b> is on. |
| <code>intel_vvp_vfb_get_lite_mode</code> | Returns if <b>Lite mode</b> is on                                                                                                                                                            |
| continued...                             |                                                                                                                                                                                              |

| Name                                  | Description                                                                               |
|---------------------------------------|-------------------------------------------------------------------------------------------|
| intel_vvp_vfb_get_debug_enabled       | Returns if <b>Debug features</b> is on                                                    |
| intel_vvp_vfb_get_max_width           | Returns the maximum line width.                                                           |
| intel_vvp_vfb_get_max_height          | Returns the maximum frame height.                                                         |
| intel_vvp_vfb_is_frame_drop_enabled   | Returns if the IP can drop frame for frame rate conversion.                               |
| intel_vvp_vfb_is_frame_repeat_enabled | Returns if the IP can repeat frame for frame rate conversion.                             |
| intel_vvp_are_invalid_frames_dropped  | Returns if the Frame Buffer drops invalid frames.                                         |
| intel_vvp_vfb_get_mem_base_address    | Returns the memory base address.                                                          |
| intel_vvp_vfb_get_buffer_stride       | Returns the number of bytes allocated to each individual frame buffer in memory.          |
| intel_vvp_vfb_get_line_stride         | Returns the number of bytes allocated to each individual video line in memory.            |
| intel_vvp_vfb_get_bits_per_sample     | Returns the number of bits per color sample set at compile-time.                          |
| intel_vvp_vfb_get_num_color_planes    | Returns the number of color planes set at compile-time.                                   |
| intel_vvp_vfb_get_pixels_in_parallel  | Returns the number of pixels transmitted in parallel on the video path (per clock cycle). |
| intel_vvp_vfb_get_mem_word_packing    | Returns how tightly video data is packed in memory words.                                 |
| intel_vvp_vfb_is_input_running        | Returns if the IP is currently processing input data.                                     |
| intel_vvp_vfb_get_input_status        | Reads the input status register.                                                          |
| intel_vvp_vfb_is_output_running       | Returns if the IP is currently producing output data.                                     |
| intel_vvp_vfb_get_output_status       | Reads the output status register.                                                         |
| intel_vvp_vfb_get_num_input_fields    | Returns the number of fields received on the input.                                       |
| intel_vvp_vfb_get_num_dropped_fields  | Returns the number of input fields dropped.                                               |
| intel_vvp_vfb_get_num_invalid_fields  | Returns the number of invalid input fields.                                               |
| intel_vvp_vfb_get_num_output_fields   | Returns the number of fields sent.                                                        |
| intel_vvp_vfb_get_num_repeated_fields | Returns the number of repeated fields sent.                                               |
| intel_vvp_vfb_output_enable           | Enables the frame buffer output.                                                          |
| intel_vvp_vfb_is_output_enabled       | Returns if the output is on or off.                                                       |
| intel_vvp_vfb_start_output            | Enables the frame buffer output.                                                          |
| Intel_vvp_vfb_stop_output             | Disables the frame buffer output.                                                         |

#### enum eIntelVvpVfbPacking

- Members**
- kIntelVvpVfbPerfectPacking – Compact sample packing, memory words are fully utilized
  - kIntelVvpVfbColorPacking – Memory words are packed with an integer number of color samples
  - kIntelVvpVfbPixelPacking – Memory words are packed with an integer number of pixels

kIntelVvpInvalidClipping – Invalid packing (used to indicate errors)

*Description* Enumeration for the supported packing methods

### intel\_vvp\_vfb\_init

*Prototype*

```
int intel_vvp_vfb_init(intel_vvp_vfb_instance *instance,
intel_vvp_core_base base);
```

*Description* Initializes a Frame Buffer instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and you should not use it further by the application if returning a non-zero error code.

*Arguments* instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

base – base address of the register map

*Returns* kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the core is not the Altera vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product\_id does not match with the Frame Buffer product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpVfbRegMapVersionErr if the register map is not supported

### intel\_vvp\_vfb\_get\_lite\_mode

*Prototype*

```
bool intel_vvp_vfb_get_lite_mode(intel_vvp_vfb_instance
*instance);
```

*Description* Returns the value of the LITE\_MODE register. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized

*Arguments* instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

*Returns* true if the IP is parameterized in lite mode

### intel\_vvp\_vfb\_get\_debug\_enabled

**Prototype**

```
bool intel_vvp_vfb_get_debug_enabled(intel_vvp_vfb_instance
*instance);
```

**Description** Returns the value of the DEBUG\_ENABLED register. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized.

**Arguments** instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

**Returns** true if the IP is parameterized with debug features enabled

### intel\_vvp\_vfb\_get\_max\_width

**Prototype**

```
uint32_t intel_vvp_vfb_get_max_width(intel_vvp_vfb_instance
*instance);
```

**Description** Returns the value of the MAX\_WIDTH register. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized.

**Arguments** instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

**Returns** the maximum supported line width

### intel\_vvp\_vfb\_get\_max\_height

**Prototype**

```
uint32_t intel_vvp_vfb_get_max_height(intel_vvp_vfb_instance
*instance);
```

**Description** Returns the value of the MAX\_HEIGHT register. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized.

**Arguments** instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

**Returns** the maximum supported frame height

### intel\_vvp\_vfb\_is\_frame\_drop\_enabled

**Prototype**

```
bool
intel_vvp_vfb_is_frame_drop_enabled(intel_vvp_vfb_instance
*instance);
```

**Description** Returns the value of the DROP\_ENABLED register. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized.

**Arguments** instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

*Returns* If frames are dropped for the purpose of frame rate conversion

#### `intel_vvp_vfb_is_frame_repeat_enabled`

*Prototype*

```
bool
intel_vvp_vfb_is_frame_repeat_enabled(intel_vvp_vfb_instance
*instance);
```

*Description* Returns the value of the REPEAT\_ENABLED register. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized.

*Arguments* instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

*Returns* If frames are repeated for the purpose of frame rate conversion

#### `intel_vvp_are_invalid_frames_dropped`

*Prototype*

```
bool
intel_vvp_are_invalid_frames_dropped(intel_vvp_vfb_instance
*instance);
```

*Description* Returns the value of the INVALID\_FRAMES\_DROPPED register. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized.

*Arguments* instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

*Returns* If detected invalid frames are dropped

#### `intel_vvp_vfb_get_mem_base_address`

*Prototype*

```
uint32_t
intel_vvp_vfb_get_mem_base_address(intel_vvp_vfb_instance
*instance);
```

*Description* Reads and returns the value of the MEM\_BASE\_ADDR register. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized.

*Arguments* instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

*Returns* the base address of the frame buffers in memory

#### `intel_vvp_vfb_get_buffer_stride`

*Prototype*

```
uint32_t
intel_vvp_vfb_get_buffer_stride(intel_vvp_vfb_instance
*instance);
```

|                    |                                                                                                                                       |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i> | Reads and returns the value of the MEM_BUFFER_STRIDE register. The instance must be a valid intel_vvp_vfb_instance fully initialized. |
| <i>Arguments</i>   | instance – pointer to the intel_vvp_vfb_instance software driver instance structure                                                   |
| <i>Returns</i>     | the stride, in bytes, between two contiguous frame buffers in memory                                                                  |

#### intel\_vvp\_vfb\_get\_line\_stride

*Prototype*

```
uint32_t intel_vvp_vfb_get_line_stride(intel_vvp_vfb_instance *instance);
```

|                    |                                                                                                                                     |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i> | Reads and returns the value of the MEM_LINE_STRIDE register. The instance must be a valid intel_vvp_vfb_instance fully initialized. |
| <i>Arguments</i>   | instance – pointer to the intel_vvp_vfb_instance software driver instance structure                                                 |
| <i>Returns</i>     | the stride, in bytes, between two contiguous video lines in a frame buffer in memory                                                |

#### intel\_vvp\_vfb\_get\_bits\_per\_sample

*Prototype*

```
uint8_t intel_vvp_vfb_get_bits_per_sample(intel_vvp_vfb_instance *instance);
```

|                    |                                                                                                                         |
|--------------------|-------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i> | Reads and returns the value of the BPS register. The instance must be a valid intel_vvp_vfb_instance fully initialized. |
| <i>Arguments</i>   | instance – pointer to the intel_vvp_vfb_instance software driver instance structure                                     |
| <i>Returns</i>     | the compile-time bits per color sample value                                                                            |

#### intel\_vvp\_vfb\_get\_num\_color\_planes

*Prototype*

```
uint8_t intel_vvp_vfb_get_num_color_planes(intel_vvp_vfb_instance *instance);
```

|                    |                                                                                                                                            |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i> | Reads and returns the value of the NUMBER_OF_COLOR_PLANES register. The instance must be a valid intel_vvp_vfb_instance fully initialized. |
| <i>Arguments</i>   | instance – pointer to the intel_vvp_vfb_instance software driver instance structure                                                        |

*Returns* the number of color planes per pixel

### intel\_vvp\_vfb\_get\_pixels\_in\_parallel

*Prototype*

```
uint8_t
intel_vvp_vfb_get_pixels_in_parallel(intel_vvp_vfb_instance
*instance);
```

*Description* Reads and returns the value of the PIXELS\_IN\_PARALLEL register. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized.

*Arguments* instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

*Returns* the number of pixels transmitted in parallel per clock cycle

### intel\_vvp\_vfb\_get\_mem\_word\_packing

*Prototype*

```
eIntelVvpVfbPacking intel_vvp_vfb_get_mem_word_packing
(intel_vvp_vfb_instance *instance);
```

*Description* Reads and returns the value of the PACKING register. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized.

*Arguments* instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

*Returns* the method used to pack pixels into memory words

### intel\_vvp\_vfb\_is\_input\_running

*Prototype*

```
bool intel_vvp_vfb_is_input_running(intel_vvp_vfb_instance*
instance);
```

*Description* Reads and returns the running bit of the INPUT\_STATUS register. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized.

*Arguments* instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

*Returns* True is the IP is currently receiving/storing an input field, false when between two input fields



### intel\_vvp\_vfb\_get\_input\_status

*Prototype*

```
uint32_t
intel_vvp_vfb_get_input_status(intel_vvp_vfb_instance*
instance);
```

*Description* Reads the INPUT\_STATUS register. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized.

*Arguments* instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

*Returns* The content of the INPUT\_STATUS register

### intel\_vvp\_vfb\_is\_output\_running

*Prototype*

```
bool intel_vvp_vfb_is_output_running(intel_vvp_vfb_instance*
instance);
```

*Description* Reads and returns the running bit of the OUTPUT\_STATUS register. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized.

*Arguments* instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

*Returns* True is the IP is currently reading/producing an output field, false when between two output fields

### intel\_vvp\_vfb\_get\_output\_status

*Prototype*

```
uint32_t
intel_vvp_vfb_get_output_status(intel_vvp_vfb_instance*
instance);
```

*Description* Reads the OUTPUT\_STATUS register. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized.

*Arguments* instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

*Returns* The content of the OUTPUT\_STATUS register

### intel\_vvp\_vfb\_get\_num\_input\_fields

*Prototype*

```
uint32_t intel_vvp_vfb_get_num_input_fields
(intel_vvp_vfb_instance* instance);
```

**Description** Reads the NUM\_INPUT\_FIELDS register. This is the number of fields received and processed on the Frame Buffer input side. The counter is updated at the end of a field. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized.

**Arguments** instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

**Returns** The content of the NUM\_INPUT\_FIELDS register

#### intel\_vvp\_vfb\_get\_num\_dropped\_fields

**Prototype**

```
uint32_t intel_vvp_vfb_get_num_dropped_fields
(intel_vvp_vfb_instance* instance);
```

**Description** Reads the NUM\_DROPPED\_FIELDS register. This is the number of fields received and dropped on the Frame Buffer input side. The counter is updated when the decision to drop the field is made at the start of the next input field. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized.

**Arguments** instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

**Returns** The content of the NUM\_DROPPED\_FIELDS register

#### intel\_vvp\_vfb\_get\_num\_invalid\_fields

**Prototype**

```
uint32_t
intel_vvp_vfb_get_num_invalid_fields(intel_vvp_vfb_instance*
instance);
```

**Description** Reads the NUM\_INVALID\_FIELDS register. This is the number of fields detected as invalid on the Frame Buffer input side. The counter is updated at the end of a field. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized.

**Arguments** instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

**Returns** The content of the NUM\_INVALID\_FIELDS register

#### intel\_vvp\_vfb\_get\_num\_output\_fields

**Prototype**

```
uint32_t intel_vvp_vfb_get_num_output_fields
(intel_vvp_vfb_instance* instance);
```

**Description** Reads the NUM\_OUTPUT\_FIELDS register. This is the total number of fields sent on the Frame Buffer output side. The counter is updated at the end of each output field, including repeats. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized.

**Arguments** instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

**Returns** The content of the NUM\_OUTPUT\_FIELDS register

#### intel\_vvp\_vfb\_get\_num\_repeated\_fields

**Prototype**

```
uint32_t intel_vvp_vfb_get_num_repeated_fields
(intel_vvp_vfb_instance* instance);
```

**Description** Reads the NUM\_REPEATED\_FIELDS register. This is the number of repeats on the Frame Buffer output side. If a single field is sent five times, the counter for the number of output fields increases by five whereas the counter for the number of repeated fields increases by four. The counter increments at the end of each repeated fields. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized.

**Arguments** instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

**Returns** The content of the NUM\_REPEATED\_FIELDS register

#### intel\_vvp\_vfb\_output\_enable

**Prototype**

```
int intel_vvp_vfb_output_enable(intel_vvp_vfb_instance*
instance, bool enabled);
```

**Description** Starts/stops the output side of the Frame Buffer. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized. Note that, if dropping fields is not enabled, stopping the output causes backpressure at the input.

**Arguments** instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

enabled – true to start the output, false to stop it

**Returns** kIntelVvpCoreOk or a negative error code in case of error

#### intel\_vvp\_vfb\_is\_output\_enabled

**Prototype**

```
bool intel_vvp_vfb_is_output_enabled(intel_vvp_vfb_instance*
instance);
```

**Description** Starts and stops the output side of the Frame Buffer. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized and configured in debug mode.

**Arguments** instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

enabled – true to start the output, false to stop it

**Returns** kIntelVvpCoreOk (0) or a negative error code in case of error

### intel\_vvp\_vfb\_start\_output

**Prototype**

```
int intel_vvp_vfb_start_output(intel_vvp_vfb_instance* instance);
```

**Description** Starts the output side of the Frame Buffer. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized. Note that, if dropping fields is not enabled, stopping the output causes backpressure at the input. This is an alias for intel\_vvp\_vfb\_output\_enable(instance, true)

**Arguments** instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

**Returns** kIntelVvpCoreOk or a negative error code in case of error

### intel\_vvp\_vfb\_stop\_output

**Prototype**

```
int intel_vvp_vfb_stop_output(intel_vvp_vfb_instance* instance);
```

**Description** Starts the output side of the Frame Buffer. The instance must be a valid intel\_vvp\_vfb\_instance fully initialized. Note that, if dropping fields is not enabled, stopping the output causes backpressure at the input. This is an alias for intel\_vvp\_vfb\_output\_enable(instance, false)

**Arguments** instance – pointer to the intel\_vvp\_vfb\_instance software driver instance structure

**Returns** kIntelVvpCoreOk or a negative error code in case of error

### Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38



## 48. Video Frame Reader IP

---

### 48.1. About the Video Frame Reader IP

The IP reads frames or frames of video stored in external memory and writes them on an Altera Streaming Video output. The frame reader has a host Avalon memory-mapped interface to allow connection to an external memory interface and an agent Avalon memory-mapped interface to allow run-time control.

The frame reader supports:

- Maximum frame resolutions of 16,384 by 16,384 pixels with 1 to 8 pixels in parallel and any color space
- Reading of progressive frames or interlaced fields
- Configurable memory packing scheme
- Runtime configurable frame location in memory
- Configurable number of buffer sets
- Continuous, single-shot, and frame sync modes over a single buffer set or multiple buffer sets
- Configurable interrupt on read completion

The IP is available as full or lite variants. For more information on full and lite, refer to the *Altera Streaming Video Protocol Specification*. The Video Frame Reader IP takes resolution information from the configurable buffer set registers.

An Avalon memory-mapped interface allows you to configure buffer sets and change the operating mode at run time. You must have this interface for both full and lite variants.

For details about latency and reset behavior for the Video Frame Reader, refer to *Video and Vision IPs Functional Description*.

#### Related Information

- [Video and Vision Processing IPs Functional Description](#) on page 21
- [Altera Streaming Video Protocol Specification](#)

#### 48.1.1. Video Frame Reader IP Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family, the number of supported bits per color sample, colors per pixel, and pixels in parallel.

**Table 932. Video Frame Reader IP Performance and Resources**

The table shows ALM usage and  $f_{MAX}$  for a design with a 4K frame reader with 2 pixels in parallel, 10 bits per color sample, 3 color planes, one or two buffer sets (Agilex 7 design has two buffer sets) and separate control clock.

| Target Device                                 | ALMs  | $f_{MAX}$ (MHz) |
|-----------------------------------------------|-------|-----------------|
| Agilex 7 speed grade 2 (AGFA012R24A2E2V)      | 2,209 | 715             |
| Cyclone 10 GX speed grade 5 (10CX220YF78015G) | 1,595 | 409             |

## 48.2. Video Frame Reader IP Parameters

The IP offers compile-time parameters.

| Parameter                            | Values      | Description                                                                                                                                                                                                                                                                                                                                                            |
|--------------------------------------|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Video Data Format</b>             |             |                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Bits per color sample</b>         | 8 to 16     | Select the number of bits per color sample.                                                                                                                                                                                                                                                                                                                            |
| <b>Number of color planes</b>        | 1 to 4      | Select the number of color planes per pixel.                                                                                                                                                                                                                                                                                                                           |
| <b>Number of pixels in parallel</b>  | 1 to 8      | Select the number of pixels in parallel.                                                                                                                                                                                                                                                                                                                               |
| <b>Maximum Frame Size</b>            |             |                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Maximum frame height</b>          | 32 to 16384 | Select the maximum height of frames. If you attempt to read out fields or frames taller than this value, the IP crops them to this height.                                                                                                                                                                                                                             |
| <b>Maximum frame width</b>           | 32 to 16384 | Select the maximum width of frames. If you attempt to read out fields or frames wider than this value, the IP crops them to this width.<br>If you use the IP exclusively to read frames with 420 subsampling, optionally halve the maximum frame width entry in the GUI. Halving the entry optimizes memory footprint because of the more efficient 420 pixel packing. |
| <b>Maximum number of buffer sets</b> | 1 to 16     | Select the maximum number of buffer sets.<br>Each buffer set implements a set of 16 additional control registers. To save resources configure the maximum number of buffer sets required for your application and no more.<br>Most applications only require 1 or 2 buffer sets.                                                                                       |
| <b>Control</b>                       |             |                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Lite mode</b>                     | On or off   | Turn on to operate the frame reader in lite mode.                                                                                                                                                                                                                                                                                                                      |

Figure 129. Frame Reader IP GUI

The screenshot displays the configuration window for the 'Video Frame Reader Intel FPGA IP' (intel\_vvp\_vfr). The window has a blue header bar with the title and a 'Details' button. The configuration is organized into four expandable sections:

- Video Data Format:**
  - Bits per color sample: 10
  - Number of color planes: 3
  - Number of pixels in parallel: 4
- Reader parameters:**
  - Maximum field height: 8192
  - Maximum field width: 8192
  - Maximum number of buffer sets: 1
- Control:**
  - ☐ Lite mode
  - ☒ Separate clock for control interface
  - ☒ Debug features
- Memory:**
  - Avalon memory-mapped local ports width: 128
  - The depth of the read FIFO.: 64
  - Avalon memory-mapped read burst target: 32
  - Packing method: PIXEL
  - ☒ Separate clock for the Avalon memory-mapped host interface(s)

### 48.3. Video Frame Reader IP Functional Description

The IP reads video fields or frames from external memory via an Avalon memory-mapped interface and outputs them over its Altera Streaming Video output.

Configure the IP with the number of buffer sets you want to set up by programming the `csr_num_buffer_sets` register. For each set, program the number of buffers, the base address and buffer offsets, the buffer dimensions and colorspace information

into the buffer set registers. Set the `csr_buffer_mode` register to multiple buffer sets mode, where the IP automatically reads buffers from the next set when the IP produces the last buffer from the current set. Otherwise, the IP reads buffers from the current set only. Program the `csr_starting_buffer_set` register and set the LSB of the `csr_run` register and the IP starts producing output frames as soon as you write to the `csr_commit` register.

If bit [1] of `csr_run` is set, this configures single-shot mode. In this mode the IP halts once the last buffer from a set has been output.

If you want to use an external timing synchronizing signal to time the output of the reads, set the `csr_run` register to 2. In this mode, configure the IP as usual but provide a rising or falling edge on the frame sync conduit when you want the IP to start outputting each frame.

If you want the IP to produce frames on a rising edge only, set `csr_fsyc_pulse_mode`.

If a frame sync edge is detected while the IP is already outputting a frame, the IP starts outputting the next frame immediately following the current frame.

Set `csr_run` to 0 and write to the `csr_commit` register for the IP to stop at the end of the current frame.

### Register Behavior

Bit [0] of the `csr_status` register goes high when the IP starts producing the first frame. It goes low after the IP finishes outputting the last line of the frame. It returns high when the IP starts producing the next frame.

The `csr_last_buffer_read` register updates with the base address of a frame when the IP finishes producing it.

Applications that need to know when the IP produces frames can poll these status registers. Alternatively, use the `IRQ_CONTROL` register to set up an interrupt when the IP finishes producing a frame.

### Double-buffer applications

You can often use a double buffer for applications where a CPU or GPU (the host) is producing frames. The host writes to one buffer while the IP reads from the other buffer. When the host completes writing the new frame, the IP switches to read from the new frame. The host can then start creating the next frame in the buffer that the IP is no longer reading from.

To configure a double buffer using the IP, Altera recommends you configure two buffer sets, each with one buffer. Think of each buffer set as one buffer. The IP produces video first from one buffer, then the other, according to the `csr_starting_buffer_set` register.



Program the registers as follows:

- **CSR\_NUM\_BUFFER\_SETS** = 2 (2 sets, each with 1 buffer)
- **CSR\_BUFFER\_MODE** = 0 (to prevent the IP automatically advancing to the buffer in set 1 before the host writes the new frame)
- **CSR\_STARTING\_BUFFER\_SET** = 0 (the host creates the frame in set 0 first)
- **CSR\_RUN** = 1 (the IP continuously produces all frames from the current set, in this case it is just one frame)

The buffer set registers at **CSR\_BUFFER\_0\_BASE** and **CSR\_BUFFER\_1\_BASE** must be programmed with the correct base addresses and other specifics for both buffers (refer to *Buffer set register banks*). **BUFFER\_0\_NUM\_BUFFERS** and **BUFFER\_1\_NUM\_BUFFERS** must be set to 1, as each set only contains one buffer. With only one buffer in each set, the **BUFFER\_N\_INTER\_BUFFER\_OFFSET** register is unused.

When the host completes writing the first frame to **CSR\_BUFFER\_0\_BASE**, write **CSR\_COMMIT** = 1 and the IP starts producing this frame repeatedly.

The host can now start constructing a frame at **CSR\_BUFFER\_1\_BASE**. When the host finishes, program:

- **CSR\_STARTING\_BUFFER\_SET** = 1
- **CSR\_COMMIT** = 1

The IP now produces repeatedly the new frame at **CSR\_BUFFER\_1\_BASE**.

This process can now iterate, with the host writing at **CSR\_BUFFER\_0\_BASE** again and instructing the IP to change to that frame:

- **CSR\_STARTING\_BUFFER\_SET** = 0
- **CSR\_COMMIT** = 1

### Multivideo applications

You can establish multiple sets of buffers for multivideo applications. The host holds videos segments in memory and the IP can read any of the segments, with the option to advance automatically via the multiple buffer sets mode to the next segment.

For an example with five video segments, with segments available at base addresses **CSR\_BUFFER\_0\_BASE** to **CSR\_BUFFER\_4\_BASE**, for a display of all the sequences in turn, program the registers as follows:

- **CSR\_NUM\_BUFFER\_SETS** = 5
- **CSR\_BUFFER\_MODE** = 1 (So the IP automatically advances to the first buffer in the next set after outputting the last buffer from the current set)
- **CSR\_STARTING\_BUFFER\_SET** = 3 (Start the output from set 3 first)

Program the base addresses and other specifics for both buffers in register sets from **CSR\_BUFFER\_0\_BASE** to **CSR\_BUFFER\_4\_BASE** (refer to *Buffer set register banks*).

**CSR\_RUN** = 1 (The IP continuously produces all frames from each set, before the **buffer\_mode** setting moves it on to the next set)

`CSR_BUFFER_N_NUM_BUFFERS` is programmed with the correct number of frames for sequence N, for example 3600 for a one-minute video at 60 Hz.

Each set may have different resolutions and color spaces, but every buffer within each set must have the same resolution and color space.

Write `CSR_COMMIT = 1` to commence output of all five video segments in turn.

To implement this 5 video segment example the host should manually establish when a set produces its final frame and dynamically configure the next set. However, using the in-built buffer set registers is more convenient and ensures multivideo playback continues even if the host cannot program the next set in time, for example when using a debugger.

### Frame synced applications

Set `CSR_RUN` to 2 to enable frame synchronization. In this mode, set up the buffer sets as described earlier and the IP produces frames whenever it detects a frame sync edge on the `fsync_in_read` input.

### Latency

The frame reader latency depends on the availability of the external memory interfaces, which may create backpressure on the IP's read interface via the `av_mm_mem_read_host_waitrequest` signal.

The streaming video output experiences backpressure via the `axi4s_vid_out_tready` input. The worst-case latency figures are for when the frame reader experiences no backpressure. Any backpressure increases these latencies by the same amount of cycles.

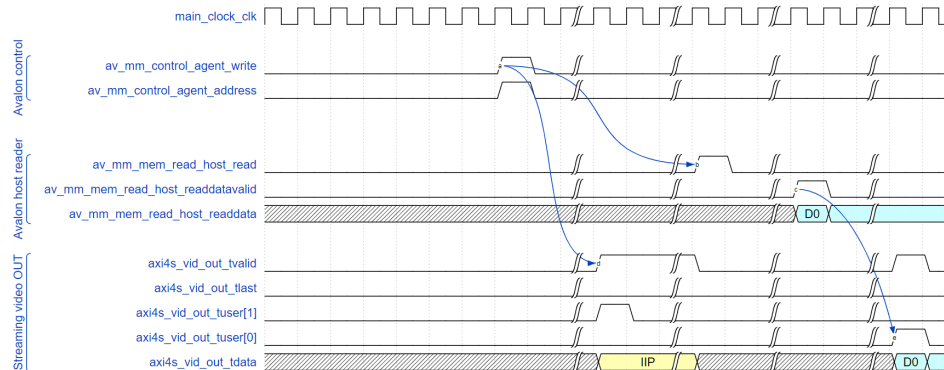
The latency figures are for when the host interfaces do not have a separate clock, so all interfaces operate from the same clock.

**Table 933. Latency**

The latency figures are the same for both full and lite variants of the IP.

| Initiating event                                                                          | Resultant event                                                                           | Latency (measured in clock cycles) | Labels in figure |
|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|------------------------------------|------------------|
| With <code>csr_run</code> register LSB previously set, a write to <code>csr_commit</code> | Image information packet (full mode only)                                                 | 10                                 | a->d             |
| With <code>csr_run</code> register LSB previously set, a write to <code>csr_commit</code> | Read initiated by <code>av_mm_mem_read_host_read</code>                                   | 14                                 | a->b             |
| <code>av_mm_mem_read_host_readdatavalid</code> strobe high to return the first frame data | <code>axi4s_vid_out_tuser[0]</code> strobe to indicate start of frame on streaming output | 20                                 | c->e             |

Figure 130. Latency



### 48.3.1. Video Frame Reader IP Interfaces

Table 934. Video Frame Reader IP Interfaces

| Name                                    | Direction | Width              | Description                              |
|-----------------------------------------|-----------|--------------------|------------------------------------------|
| <b>Clocks and resets</b>                |           |                    |                                          |
| main_clock_clk                          | In        | 1                  | AXI4-S processing clock.                 |
| main_reset_rst                          | In        | 1                  | AXI4-S processing reset.                 |
| control_clock_clk                       | In        | 1                  | Optional control agent interface clock.  |
| control_reset_reset                     | In        | 1                  | Optional control agent interface reset.  |
| <b>Control interfaces</b>               |           |                    |                                          |
| av_mm_control_agent_address             | In        | 8 <sup>(144)</sup> | Avalon memory-mapped agent address       |
| av_mm_control_agent_write               | In        | 1                  | Avalon memory-mapped agent write.        |
| av_mm_control_agent_writedata           | In        | 32                 | Avalon memory-mapped agent write data.   |
| av_mm_control_agent_byteenable          | In        | 4                  | Avalon memory-mapped agent byte enable.  |
| av_mm_control_agent_read                | In        | 1                  | Avalon memory-mapped agent read.         |
| av_mm_control_agent_readdata            | Out       | 32                 | Avalon memory-mapped agent read data.    |
| av_mm_control_agent_readdatavalid       | Out       | 1                  | Avalon memory-mapped agent read.         |
| av_mm_control_agent_waitrequest         | Out       | 1                  | Avalon memory-mapped agent wait request. |
| fsync_in_read                           | In        | 1                  | Conduit.                                 |
| frame_reader_int_irq                    | Out       | 1                  | Conduit.                                 |
| <b>Altera Streaming Video interface</b> |           |                    |                                          |
| axi4s_vid_out_tdata                     | Out       | <sup>(145)</sup>   | AXI4-S data out.                         |
| axi4s_vid_out_tvalid                    | Out       | 1                  | AXI4-S data valid.                       |
| <i>continued...</i>                     |           |                    |                                          |

<sup>(144)</sup> Address width depends on the setting of **Maximum number of buffer sets** compile-time parameter.

| Name                                                                                                                                                          | Direction | Width | Description                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------|---------------------------------------------|
| axi4s_vid_out_tuser[0]                                                                                                                                        | Out       | 1     | AXI4-S start of video frame.                |
| axi4s_vid_out_tuser[1]                                                                                                                                        | Out       | 1     | AXI4-S control or data packet.              |
| axi4s_vid_out_tuser[N-1:2]                                                                                                                                    | Out       | (146) | Unused.                                     |
| axi4s_vid_out_tlast                                                                                                                                           | Out       | 1     | AXI4-S end of packet.                       |
| axi4s_vid_out_tready                                                                                                                                          | In        | 1     | AXI4-S data ready.                          |
| <b>Avalon memory-mapped host interfaces</b><br>Widths for address, data, and burst count buses are as you specify in the GUI. The table shows example widths. |           |       |                                             |
| mem_clock_clk                                                                                                                                                 | In        | 1     | Optional host interface clock.              |
| mem_reset_reset                                                                                                                                               | In        | 1     | Optional host interface reset.              |
| av_mm_mem_read_host_address                                                                                                                                   | Out       | 32    | Avalon memory-mapped host address.          |
| av_mm_mem_read_host_read                                                                                                                                      | Out       | 1     | Avalon memory-mapped host read.             |
| av_mm_mem_read_host_burstcount                                                                                                                                | Out       | 5     | Avalon memory-mapped host read burst count. |
| av_mm_mem_read_host_readdata                                                                                                                                  | In        | 64    | Avalon memory-mapped host read data.        |
| av_mm_mem_read_host_readdatavalid                                                                                                                             | In        | 1     | Avalon memory-mapped host read data valid.  |
| av_mm_mem_read_host_waitrequest                                                                                                                               | In        | 1     | Avalon memory-mapped host wait request.     |

### Related Information

Video and Vision Processing IP Interfaces on page 30

## 48.4. Video Frame Reader IP Registers

Each register is either read-only (RO) or read-write (RW).

### Main register set

**Table 935. Video Frame Reader IP Registers**

In the software API, the register names appear with a prefix of INTEL\_VVP, INTEL\_VVP\_CORE, or INTEL\_VVP\_VIDEO\_FRAME\_READER as appropriate and with an optional REG suffix

| Address                           | Register  | Access | Description                                                                                         |
|-----------------------------------|-----------|--------|-----------------------------------------------------------------------------------------------------|
| <b>Parameterization registers</b> |           |        |                                                                                                     |
| 0x0000                            | VID_PID   | RO     | Read this register for the Video Frame Reader product ID. This register always returns 0x6AF7_024A. |
| 0x0004                            | VERSION   | RO     | Read this register for the IP version information.                                                  |
| 0x0008                            | LITE_MODE | RO     | Read this register to determine if <b>Lite mode</b> is on or off.                                   |
| <i>continued...</i>               |           |        |                                                                                                     |

- (145) The equation gives all axi4s\_vid\_out\_tdata widths in these interfaces:  
 $\max(2, \text{ceil}((\text{bits per color sample} \times \text{number of color planes})/8)) \times \text{pixels in parallel} \times 8$
- (146) This equation gives all axi4s\_vid\_out\_tuser widths in these interfaces  $N = \text{ceil}(\text{tdata width} / 8)$

| Address                     | Register               | Access | Description                                                                                                                                                                                                                                                                       |
|-----------------------------|------------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                             |                        |        | This register returns 0 when <b>Lite mode</b> is off and 1 when on.                                                                                                                                                                                                               |
| 0x000C                      | DEBUG_ENABLED          | RO     | Read this register to determine if <b>Debug features</b> is on.                                                                                                                                                                                                                   |
| 0x0010                      | MAX_BUFFER_SETS        | RO     | Read this register to determine the maximum supported number of buffer sets.                                                                                                                                                                                                      |
| 0x0014                      | MAX_HEIGHT             | RO     | Read this register to determine the maximum supported frame height.                                                                                                                                                                                                               |
| 0x0018                      | MAX_WIDTH              | RO     | Read this register to determine the maximum supported frame width.                                                                                                                                                                                                                |
| 0x001C                      | BITS_PER_SYMBOL        | RO     | Read this register for the number of bits per symbol configured.                                                                                                                                                                                                                  |
| 0x0020                      | NUMBER_OF_COLOR_PLANES | RO     | Read this register for the number of color planes.                                                                                                                                                                                                                                |
| 0x0024                      | PIXELS_IN_PARALLEL     | RO     | Read this register for the number of pixels in parallel.                                                                                                                                                                                                                          |
| 0x0028                      | PACKING                | RO     | Read this register for the pixel packing scheme.                                                                                                                                                                                                                                  |
| 0x002C to 0x00FF            | Reserved               |        |                                                                                                                                                                                                                                                                                   |
| Interrupt registers         |                        |        |                                                                                                                                                                                                                                                                                   |
| 0x0100                      | IRQ_CONTROL            | RW     | Set bit 0 to 1 to enable interrupts on completion of field reads                                                                                                                                                                                                                  |
| 0x0104                      | IRQ_STATUS             | RW     | Read the status register to see if the interrupt fired. If it did, the IP sets bit 0. To clear the interrupt, write a 1 to bit 0.                                                                                                                                                 |
| 0x0107 to 0x013F            | Reserved.              |        |                                                                                                                                                                                                                                                                                   |
| Control and Debug Registers |                        |        |                                                                                                                                                                                                                                                                                   |
| 0x0140                      | STATUS                 | RO     | Bit 0: status bit.<br>1 = Video Frame Reader is outputting a video field, 0 otherwise.<br>Bit 1: Pending RTC control bit. Goes high when a write occurs to one of the control registers. Goes low at the end of the current frame if the IP makes a write to the COMMIT register. |
| 0x0144                      | LAST_BUFFER_READ       | RO     | Returns the base address of the frame that the IP last produced. This register returns 0xFFFF_FFFF if the IP reads no buffers since reset.                                                                                                                                        |
| 0x0147-0x018F               | Reserved               |        |                                                                                                                                                                                                                                                                                   |
| 0x0190                      | COMMIT                 | RW     | Commit register.                                                                                                                                                                                                                                                                  |
| 0x0194                      | NUM_BUFFER_SETS        | RW     | Instructs the frame reader how many buffer sets to configure.                                                                                                                                                                                                                     |
| 0x0198                      | BUFFER_MODE            | RW     | Buffer mode.                                                                                                                                                                                                                                                                      |
| 0x019C                      | STARTING_BUFFER_SET    | RW     | Starting buffer set.                                                                                                                                                                                                                                                              |
| 0x01A0                      | RUN                    | RW     | Set the required operating mode:<br>0 = Frame reader is not running (stopped)<br>1 = Frame reader is free-running mode.<br>2 = Frame reader is in frame sync mode. In this mode the reader starts the output on receiving a frame sync and stops when the field is output.        |
| continued...                |                        |        |                                                                                                                                                                                                                                                                                   |

| Address | Register                      | Access | Description                                                                                                                                                                                                                                                                                                                                   |
|---------|-------------------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         |                               |        | 3 = Frame reader is in single-shot mode. In this mode, <code>BUFFER_N_NUM_BUFFERS</code> decides how many buffers are read from. After the IP reads all the buffers from the current set (or all sets if <code>BUFFER_MODE</code> is set to 1), output stops. To perform another single-shot read, write to the <code>COMMIT</code> register. |
| 0x01A4  | <code>FSYNC_PULSE_MODE</code> | RW     | Frame sync pulse mode.                                                                                                                                                                                                                                                                                                                        |

### Buffer set register banks

In addition to the main register set, the IP has one bank of registers per configured buffer set. Program the buffer set register banks with the relevant details for each set. The IP calculates the base address in hex for each register bank according to the buffer set number, which is an integer from 0 to `MAX_BUFFER_SETS`:

For example, buffer set 0 registers are at  $0x01B0 + 0 = 0x01B0$ .

For example, buffer set **1** registers are at  $0x01B0 + 1 \times 0x40 = 0x01F0$ .

For example, buffer set **3** registers are at  $0x01B0 + 3 \times 0x40 = 0x0270$ .

**Table 936. Buffer set register banks**

| Address                  | Register                                  | Access | Description                                                                                                                                                   |
|--------------------------|-------------------------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $0x01B0 + N \times 0x40$ | <code>BUFFER_N_BASE</code>                | RW     | Buffer base address                                                                                                                                           |
| $0x01B4 + N \times 0x40$ | <code>BUFFER_N_NUM_BUFFERS</code>         | RW     | Number of buffers in this buffer set. Often set to 1 for common double-buffering applications.                                                                |
| $0x01B8 + N \times 0x40$ | <code>BUFFER_N_INTER_BUFFER_OFFSET</code> | RW     | Interbuffer address increment.<br>If <code>BUFFER_N_NUM_BUFFERS</code> is set to 1, this register is unused.                                                  |
| $0x01BC + N \times 0x40$ | <code>BUFFER_N_INTER_LINE_OFFSET</code>   | RW     | Interline address increment.                                                                                                                                  |
| $0x01C0 + N \times 0x40$ | <code>BUFFER_N_WIDTH</code>               | RW     | For outgoing image information packets if <b>Lite mode</b> is off.                                                                                            |
| $0x01C4 + N \times 0x40$ | <code>BUFFER_N_HEIGHT</code>              | RW     | For outgoing image information packets if <b>Lite mode</b> is off and to determine how many lines to read from the <code>BASE</code> address.                 |
| $0x01C8 + N \times 0x40$ | <code>BUFFER_N_INTERLACE</code>           | RW     | For outgoing image information packets if <b>Lite mode</b> is off and to set <code>TUSER[1]</code> unless <code>BUFFER_0_PACKED_INTERLACE</code> is non-zero. |
| $0x01CC + N \times 0x40$ | <code>BUFFER_N_COLORSPACE</code>          | RW     | For outgoing image information packets if <b>Lite mode</b> is off.                                                                                            |
| $0x01D0 + N \times 0x40$ | <code>BUFFER_N_SUBSAMPLING</code>         | RW     | For outgoing image information packets if <b>Lite mode</b> is off.                                                                                            |
| continued...             |                                           |        |                                                                                                                                                               |

|                    |                      |    |                                                                                                                    |
|--------------------|----------------------|----|--------------------------------------------------------------------------------------------------------------------|
| 0x01D4 +<br>N*0x40 | BUFFER_N_COSITING    | RW | For outgoing image information packets if <b>Lite mode</b> is off.                                                 |
| 0x01D8 +<br>N*0x40 | BUFFER_N_BPS         | RW | For outgoing image information packets if <b>Lite mode</b> is off.                                                 |
| 0x01DC +<br>N*0x40 | BUFFER_N_FIELD_COUNT | RW | For outgoing image information (low 7 bits) and EOF packets for field count quantities if <b>Lite mode</b> is off. |

### Register Bit Descriptions

**Table 937. VID\_PID**

| Name                                   | Bits | Description                                                                                                                                                                                           |
|----------------------------------------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frame reader version ID and product ID | 31:0 | This register always returns 0x6AF7_024A <ul style="list-style-type: none"> <li>15:0 is the product ID and always returns 0x024A</li> <li>31:16 is the vendor ID and always returns 0x6AF7</li> </ul> |

**Table 938. VERSION**

| Name                 | Bits  | Description                          |
|----------------------|-------|--------------------------------------|
| Register map version | 7:0   | Register map version.                |
| IP patch revision    | 15:8  | -                                    |
| IP update revision   | 23:16 | Updated when the IP version changes. |
| IP major revision    | 31:24 | Updated when the IP version changes. |

**Table 939. LITE\_MODE**

| Name                           | Bits | Description                                 |
|--------------------------------|------|---------------------------------------------|
| Lite mode parameterization bit | 0    | Returns 1 if you turn on <b>lite mode</b> . |
| Unused                         | 31:1 | Unused.                                     |

**Table 940. DEBUG\_ENABLED**

| Name                                | Bits | Description                                      |
|-------------------------------------|------|--------------------------------------------------|
| Debug features parameterization bit | 0    | Returns 1 if you turn on <b>Debug features</b> . |
| Unused                              | 31:1 | Unused.                                          |

**Table 941. MAX\_BUFFER\_SETS**

| Name            | Bits | Description                                                        |
|-----------------|------|--------------------------------------------------------------------|
| Max buffer sets | 5:0  | This register returns the maximum supported number of buffer sets. |

**Table 942. MAX\_WIDTH**

| Name      | Bits | Description                                              |
|-----------|------|----------------------------------------------------------|
| Max width | 31:0 | This register returns the maximum supported frame width. |

**Table 943. MAX\_HEIGHT**

| Name       | Bits | Description                                               |
|------------|------|-----------------------------------------------------------|
| Max height | 31:0 | This register returns the maximum supported frame height. |

**Table 944. BITS\_PER\_SYMBOL**

| Name            | Bits | Description                                                                                                   |
|-----------------|------|---------------------------------------------------------------------------------------------------------------|
| Bits per symbol | 31:0 | This register returns the number of bits per symbol that the Altera Streaming Video output is configured for. |

**Table 945. NUMBER\_OF\_COLOR\_PLANES**

| Name                   | Bits | Description                                                                                                |
|------------------------|------|------------------------------------------------------------------------------------------------------------|
| Number of color planes | 31:0 | This register returns the number of color planes that the Altera Streaming Video output is configured for. |

**Table 946. PIXELS\_IN\_PARALLEL**

| Name                         | Bits | Description                                                                                                      |
|------------------------------|------|------------------------------------------------------------------------------------------------------------------|
| Number of pixels in parallel | 31:0 | This register returns the number of pixels in parallel that the Altera Streaming Video output is configured for. |

**Table 947. PACKING**

| Name    | Bits | Description                                                                                                                                                                            |
|---------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Packing | 31:0 | This register returns the packing scheme that the IP uses: <ul style="list-style-type: none"> <li>0 = Perfect packing</li> <li>1 = Color packing</li> <li>2 = Pixel packing</li> </ul> |

**Table 948. IRQ\_CONTROL**

| Name           | Bit | Description                                                                                                                                                                                  |
|----------------|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Field_read_irq | 0   | Set the <code>field_read_irq</code> bit of the <code>IRQ_CONTROL</code> register to enable interrupts on completing field reads.<br>Clear to disable interrupts on completion of field reads |

**Table 949. IRQ\_STATUS**

| Name           | Bit | Description                                                                                                                                                                                                                            |
|----------------|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Field_read_irq | 0   | The <code>field_read_irq</code> bit of the <code>IRQ_CONTROL</code> register is set if the interrupt has fired.<br>Write to the <code>field_read_irq</code> bit of the <code>IRQ_CONTROL</code> register to clear down any interrupts. |

**Table 950. STATUS**

| Name                     | Bit | Description                                                                                                                                                                                                                                                                                                                                                                                                |
|--------------------------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Status                   | 0   | Read this register to determine the status of the frame reader. The status bit is set if the reader is currently outputting a frame and returns to 0 in between frames.                                                                                                                                                                                                                                    |
| Pending run-time control | 1   | Read this register to determine whether any pending register writes exist that have yet to be committed. This bit goes high when a write occurs to one of the control registers and returns low at the end of the current frame when a write has been made to the <code>COMMIT</code> register. If the frame reader is idle, this bit goes low directly after a write to the <code>COMMIT</code> register. |

**Table 951. LAST\_BUFFER\_READ**

| Name             | Bit  | Description                                                                                                                                                              |
|------------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Last buffer read | 31:0 | Read this register to determine the base address of the most recent output buffer. After a reset, if no buffers appear, this register returns <code>0xFFFF_FFFF</code> . |



**Table 952. COMMIT**

| Name   | Bit | Description                                                                                                                                               |
|--------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commit | 0   | Write to this register to commit the current register settings. The IP starts operating according to the new settings when it produces the current frame. |

**Table 953. NUM\_BUFFER\_SETS**

| Name                  | Bits | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----------------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Number of buffer sets | 31:0 | <p>Set this register to the number of buffer sets that you want. The IP reads from buffers within each buffer set in turn. For a typical memory-mapped graphics double buffer application, you might set up two buffer sets, each containing just one buffer. Software tick-tocks between the buffers under API control using the <code>STARTING_BUFFER_SET</code> register.</p> <p>Legal values are between 1 and the maximum number of buffer sets you set in the GUI. To save area, the maximum number of buffer sets you set in the GUI restricts the number of buffer sets that the IP can use, as each set requires its own bank of registers. Setting this register to a value greater than <code>MAX_BUFFER_STRUCTURES</code> means the IP configures only one buffer set.</p> <p>This register is the number of buffer sets that the IP actively uses. Do not confuse it with the maximum number of buffer sets you set in the GUI.</p> |

**Table 954. BUFFER\_MODE**

| Name        | Bit | Description                                                                                                                                                                                                            |
|-------------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Buffer mode | 1   | <p>Set this register according to the behavior you want:</p> <p>0 = Read buffers in <code>STARTING_BUFFER_SET</code> only.</p> <p>1 = Reads all buffer sets in turn, starting at <code>STARTING_BUFFER_SET</code>.</p> |

**Table 955. STARTING\_BUFFER\_SET**

| Name                | Bits | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|---------------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Starting buffer set | 31:0 | <p>Set this register to a value from 0 to <code>NUM_BUFFER_SETS</code> to identify the starting buffer set.</p> <p>Set this register to indicate the buffer set that you want to read from. For example, to read the first buffer from the buffer set at <code>BUFFER_3_BASE</code>, set this register to 3.</p> <p>When starting reading from a new set, the first buffer in the set is output first.</p> <p>Read <code>LAST_BUFFER_READ(0x0144)</code> to determine which buffer from the set was last read and output.</p> <p>Reading starts from a new buffer set according to this register from <code>STARTING_BUFFER_SET</code> with the <code>COMMIT</code> register.</p> |

**Table 956. RUN**

| Name    | Bit | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Run LSB | 0   | <p>Set this bit to start the IP.</p> <p>Clear this bit and clear the MSB to halt the IP.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Run MSB | 1   | <p>If the LSB is set and this bit is clear, the IP operates in continuous (free-running) mode.</p> <p>If the LSB is set, set this bit to configure the IP in single-shot mode. In this mode, <code>BUFFER_N_NUM_BUFFERS</code> dictates how many buffers are read from the current set, <i>N</i>. After the IP reads all the buffers from the current set (if <code>BUFFER_MODE</code> is set to 1), output stops. To perform another single-shot read, write to the <code>COMMIT</code> register.</p> <p>If the LSB is clear, clear this bit to stop the IP at the end of the current frame read.</p> <p>If the LSB is clear, set this bit to configure frame sync mode. In this mode, the reader produces one frame per frame sync.</p> |

**Table 957. FSYNC\_PULSE\_MODE**

| Name             | Bit | Description                                                                                                                                                                                                                                                               |
|------------------|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FSYNC pulse mode | 0   | If set, when operating in frame sync mode, the frame reading is triggered by a rising edge on the frame sync conduit.<br>If clear, when operating in frame sync mode, the frame reading is triggered by a rising or falling edge on the frame sync conduit (toggle mode). |

**Table 958. BUFFER\_N\_BASE**

| Name                             | Bits | Description                                           |
|----------------------------------|------|-------------------------------------------------------|
| buffer set <i>N</i> base address | 31:0 | Set this to the base address of buffer set <i>N</i> . |

**Table 959. BUFFER\_N\_NUM\_BUFFERS**

| Name                                  | Bits | Description                                         |
|---------------------------------------|------|-----------------------------------------------------|
| buffer set <i>N</i> number of buffers | 31:0 | Set this to the number of buffers in set <i>N</i> . |

**Table 960. BUFFER\_N\_INTER\_BUFFER\_OFFSET**

| Name                                    | Bits | Description                                                                                                                                  |
|-----------------------------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------|
| buffer set <i>N</i> inter-buffer offset | 31:0 | Set this to the inter-buffer address increment for set <i>N</i> .<br>Usually this is equal or greater than bytes per line * BUFFER_N_HEIGHT. |

**Table 961. BUFFER\_N\_INTER\_LINE\_OFFSET**

| Name                                  | Bits | Description                                                                                                 |
|---------------------------------------|------|-------------------------------------------------------------------------------------------------------------|
| buffer set <i>N</i> inter-line offset | 31:0 | Set this to the interline address increment for set <i>N</i> .<br>Usually bytes per pixel * BUFFER_N_WIDTH. |

**Table 962. BUFFER\_N\_WIDTH**

| Name                             | Bits | Description                                                                                                                                              |
|----------------------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| buffer set <i>N</i> buffer width | 16:0 | Set this to the width in pixels of the buffers in set <i>N</i> . If <b>lite mode</b> is off, the IP uses it also for outgoing image information packets. |

**Table 963. BUFFER\_N\_HEIGHT**

| Name                              | Bits | Description                                                                                                                                               |
|-----------------------------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| buffer set <i>N</i> buffer height | 16:0 | Set this to the height in pixels of the buffers in set <i>N</i> . If <b>lite mode</b> is off, the IP uses it also for outgoing image information packets. |

**Table 964. BUFFER\_N\_INTERLACE**

| Name                                 | Bits | Description                                                                                                      |
|--------------------------------------|------|------------------------------------------------------------------------------------------------------------------|
| buffer set <i>N</i> interlace nibble | 3:0  | For outgoing image information packets if <b>Lite mode</b> is off and to set TUSER[1] if <b>Lite mode</b> is on. |

**Table 965. BUFFER\_N\_COLORSPACE**

| Name                                  | Bits | Description                                                                                           |
|---------------------------------------|------|-------------------------------------------------------------------------------------------------------|
| buffer set <i>N</i> buffer colorspace | 6:0  | If <b>Lite mode</b> is off, for outgoing image information packets. Unused if <b>Lite mode</b> is on. |

**Table 966. BUFFER\_N\_SUBSAMPLING**

| Name                                   | Bits | Description                                                                                           |
|----------------------------------------|------|-------------------------------------------------------------------------------------------------------|
| buffer set <i>N</i> buffer subsampling | 1:0  | If <b>Lite mode</b> is off, for outgoing image information packets. Unused if <b>Lite mode</b> is on. |

**Table 967. BUFFER\_N\_COSITING**

| Name                                | Bits | Description                                                                                           |
|-------------------------------------|------|-------------------------------------------------------------------------------------------------------|
| buffer set <i>N</i> buffer cositing | 1:0  | If <b>Lite mode</b> is off, for outgoing image information packets. Unused if <b>Lite mode</b> is on. |

**Table 968. BUFFER\_N\_BPS**

| Name                           | Bits | Description                                                                                           |
|--------------------------------|------|-------------------------------------------------------------------------------------------------------|
| buffer set <i>N</i> buffer BPS | 4:0  | If <b>Lite mode</b> is off, for outgoing image information packets. Unused if <b>Lite mode</b> is on. |

**Table 969. BUFFER\_N\_FIELD\_COUNT**

| Name                                     | Bits | Description                                                                                                                                                                                                                                                                |
|------------------------------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| buffer set <i>N</i> starting field count | 15:0 | If <b>Lite mode</b> is off, the starting field count for outgoing image information and EOF packets. The image information and EOF field counts increment for each frame output and reset to this value after a write to <b>COMMIT</b> . Unused if <b>Lite mode</b> is on. |

## 48.5. Video Frame Reader IP Software API

The IP has a software driver for software control of the IP at run time. The IP does not fit any of the generic device models provided by the Nios II HAL. It exposes a set of dedicated accessors to the control and status registers. The driver structure inherits the base driver structure so all common methods described in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_vfr_regs.h`

Include file: `intel_vvp_vfr.h`

**Table 970. Frame Reader IP Software API**

| Name                                           | Description                                                                                                                                                                                 |
|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>intel_vvp_vfr_init</code>                | Initialize the Frame Reader instance                                                                                                                                                        |
| <code>intel_vvp_core_*</code>                  | Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i> . Writable when <b>Lite</b> is on. Readable when <b>Lite</b> is off and <b>Debug features</b> is on. |
| <code>intel_vvp_vfr_get_lite_mode</code>       | Returns if <b>Lite mode</b> is on                                                                                                                                                           |
| <code>intel_vvp_vfr_get_debug_enabled</code>   | Returns if <b>Debug features</b> is on                                                                                                                                                      |
| <code>intel_vvp_vfr_get_max_height</code>      | Returns the maximum supported height                                                                                                                                                        |
| <code>intel_vvp_vfr_get_max_width</code>       | Returns the maximum supported width                                                                                                                                                         |
| <code>intel_vvp_vfr_get_max_buffer_sets</code> | Returns the maximum number of supported buffer sets                                                                                                                                         |
| <code>intel_vvp_vfr_get_bps</code>             | Returns the number of bits per color sample                                                                                                                                                 |
| <i>continued...</i>                            |                                                                                                                                                                                             |

| Name                                         | Description                                              |
|----------------------------------------------|----------------------------------------------------------|
| intel_vvp_vfr_get_number_of_color_planes     | Returns the number of color planes per pixel             |
| intel_vvp_vfr_get_pixels_in_parallel         | Returns the number of pixels streamed in parallel        |
| intel_vvp_vfr_get_mem_word_packing           | Returns the algorithm used for pixel packing in memory   |
| intel_vvp_vfr_is_running                     | Returns if the IP is currently processing video data     |
| intel_vvp_vfr_get_commit_status              | Returns if there are uncommitted writes                  |
| intel_vvp_vfr_get_status                     | Reads the status register                                |
| intel_vvp_vfr_get_last_buffer_read_addr      | Returns the memory address of the last buffer read       |
| intel_vvp_vfr_set_num_buffer_sets            | Changes the number of buffer sets in use                 |
| intel_vvp_vfr_get_num_buffer_sets            | Returns the number of buffer sets in use                 |
| intel_vvp_vfr_set_buffer_mode                | Changes the use mode of buffer sets                      |
| intel_vvp_vfr_get_buffer_mode                | Returns the use mode of buffer sets                      |
| intel_vvp_vfr_set_starting_buffer_set        | Changes the initial buffer set                           |
| intel_vvp_vfr_get_starting_buffer_set        | Returns the initial buffer set                           |
| intel_vvp_vfr_set_run_mode                   | Changes the run mode                                     |
| intel_vvp_vfr_get_run_mode                   | Returns the run mode                                     |
| intel_vvp_vfr_set_fsync_mode                 | Changes the field synchronization trigger                |
| intel_vvp_vfr_get_fsync_mode                 | Returns the field synchronization trigger                |
| intel_vvp_vfr_set_bufset_num_buffers         | Sets the number of buffers in a set                      |
| intel_vvp_vfr_get_bufset_num_buffers         | Returns the number of buffers in a set                   |
| intel_vvp_vfr_set_bufset_inter_buffer_offset | Sets the size of the buffers in a set                    |
| intel_vvp_vfr_get_bufset_inter_buffer_offset | Returns the size of the buffers in a set                 |
| intel_vvp_vfr_set_bufset_inter_line_offset   | Sets the size allocated for a field line in a set        |
| intel_vvp_vfr_get_bufset_inter_line_offset   | Returns the size allocated for a field line in a set     |
| intel_vvp_vfr_set_bufset_height              | Sets the height of fields in a set                       |
| intel_vvp_vfr_get_bufset_height              | Returns the height of fields in a set                    |
| intel_vvp_vfr_set_bufset_width               | Sets the width of fields in a set                        |
| intel_vvp_vfr_get_bufset_width               | Returns the width of fields in a set                     |
| intel_vvp_vfr_set_bufset_interlace           | Sets the interlace nibble of the first field in a set    |
| intel_vvp_vfr_get_bufset_interlace           | Returns the interlace nibble of the first field in a set |
| intel_vvp_vfr_set_bufset_colorspace          | Sets the colorspace of fields in a set                   |
| intel_vvp_vfr_get_bufset_colorspace          | Returns the colorspace of fields in a set                |
| intel_vvp_vfr_set_bufset_subsampling         | Sets the subsampling of fields in a set                  |
| intel_vvp_vfr_get_bufset_subsampling         | Returns the subsampling of fields in a set               |
| intel_vvp_vfr_set_bufset_cositing            | Sets the cositing of fields in a set                     |
| continued...                                 |                                                          |

| Name                                 | Description                                       |
|--------------------------------------|---------------------------------------------------|
| intel_vvp_vfr_get_bufset_cositing    | Returns the cositing of fields in a set           |
| intel_vvp_vfr_set_bufset_bps         | Sets the bps value for fields in a set            |
| intel_vvp_vfr_get_bufset_bps         | Returns the bps value for fields in a set         |
| intel_vvp_vfr_set_bufset_field_count | Sets the field count value for fields in a set    |
| intel_vvp_vfr_get_bufset_field_count | Returns the field count value for fields in a set |
| intel_vvp_vfr_commit_writes          | Commit all outstanding writes                     |

#### enum eIntelVvpVfrPacking

- Members**
- kIntelVvpVfrPerfectPacking (0) - Perfect sample packing (no bits wasted)
  - kIntelVvpVfrColorPacking (1) - Color sample packing (color samples are not split across memory words)
  - kIntelVvpVfrPixelPacking (2) - Pixel packing (pixels are not split across memory words)
  - kIntelVvpVfrInvalidPacking (-1) - Invalid packing algorithm (used to indicate errors)
- Description** Enumeration for the configurable sample packing in memory algorithms

#### enum eIntelVvpVfrRunMode

- Members**
- kIntelVvpVfrStop (0) - Frame Reader is stopped
  - kIntelVvpVfrFreeRunning (1) - Frame Reader proceeds in a round-robin scheme using all buffers made available
  - kIntelVvpVfrFsync (2) - Frame Reader proceeds as above but waits for fsync trigger to start reading a new buffer
  - kIntelVvpVfrSingleShot (3) - Frame Reader reads across the buffers once
  - kIntelVvpVfrInvalidRunMode (-1) - Invalid value (used to indicate errors)
- Description** Enumeration for the various read methods

#### enum eIntelVvpVfrBufferMode

- Members**
- kIntelVvpVfrSingleSet (0) - A single set of buffers is used
  - kIntelVvpVfrAllSets (1) - All sets configured are used

kIntelVvpVfrInvalidBufferMode (-1) – Invalid value (used to indicate errors)

*Description* Enumeration for the usage of the sets of buffer

#### **enum eIntelVvpVfrFsyncMode**

*Members*

kIntelVvpVfrFsyncBoth (0) - Triggers on both edges of the sync signal

kIntelVvpVfrFsyncRising (1) - Triggers on the rising edge of the sync signal

kIntelVvpVfrInvalidFsyncMode (-1) – Invalid value (used to indicate errors)

*Description* Enumeration for the two fsync trigger mechanisms

#### **intel\_vvp\_vfr\_init**

*Prototype*

```
int intel_vvp_vfr_init(intel_vvp_vfr_instance *instance,
intel_vvp_core_base base);
```

*Description* Initializes a Frame Reader instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

*Arguments*

instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

base - base address of the register map

*Returns*

kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product\_id does not match with the Frame Reader product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpFrame ReaderRegMapVersionErr if the register map is not supported

#### intel\_vvp\_vfr\_get\_lite\_mode

**Prototype**

```
bool intel_vvp_vfr_get_lite_mode(intel_vvp_vfr_instance
*instance);
```

**Description**

Returns the value of the LITE\_MODE register. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized

**Arguments**

instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

**Returns**

true if the IP is parameterized in lite mode

#### intel\_vvp\_vfr\_get\_debug\_enabled

**Prototype**

```
bool intel_vvp_vfr_get_debug_enabled(intel_vvp_vfr_instance
*instance);
```

**Description**

Returns the value of the DEBUG\_ENABLED register. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized.

**Arguments**

instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

**Returns**

true if the IP is parameterized with debug features enabled

#### intel\_vvp\_vfr\_get\_max\_height

**Prototype**

```
uint32_t intel_vvp_vfr_get_max_height(intel_vvp_vfr_instance
*instance);
```

**Description**

Returns the value of the MAX\_HEIGHT register. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized.

**Arguments**

instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

**Returns**

The maximum supported height set at configuration

#### intel\_vvp\_vfr\_get\_max\_width

**Prototype**

```
uint32_t intel_vvp_vfr_get_max_width(intel_vvp_vfr_instance
*instance);
```

**Description**

Returns the value of the MAX\_WIDTH register. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized.

**Arguments**

instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

**Returns** The maximum supported width set at configuration

### `intel_vvp_vfr_get_max_buffer_sets`

**Prototype**

```
eIntelVvpDilAlgorithms
intel_vvp_vfr_get_max_buffer_sets(intel_vvp_vfr_instance
*instance);
```

**Description** Returns the value of the MAX\_BUFFER\_SETS register. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

**Returns** The maximum number of sets of buffers configurable

### `intel_vvp_vfr_get_bps`

**Prototype**

```
uint8_t intel_vvp_vfr_get_bps(intel_vvp_vfr_instance
*instance);
```

**Description** Returns the value of the BPS register. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

**Returns** the number of bits per color sample selected at configuration

### `intel_vvp_vfr_get_number_of_color_planes`

**Prototype**

```
uint8_t
intel_vvp_vfr_get_number_of_color_planes(intel_vvp_vfr_instance
*instance);
```

**Description** Returns the value of the NUMBER\_OF\_COLOR\_PLANES register. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

**Returns** the number of color planes per pixel selected at configuration

### `intel_vvp_vfr_get_pixels_in_parallel`

**Prototype**

```
uint8_t
intel_vvp_vfr_get_pixels_in_parallel(intel_vvp_vfr_instance
*instance);
```



- Description* Returns the value of the PIXELS\_IN\_PARALLEL register. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized.
- Arguments* instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure
- Returns* the number of pixels streamed in parallel selected at configuration

#### intel\_vvp\_vfr\_get\_mem\_word\_packing

*Prototype*

```
eIntelVvpVfrPacking
intel_vvp_vfr_get_mem_word_packing(intel_vvp_vfr_instance
*instance);
```

- Description* Returns the value of the WORD\_PACKING register. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized.
- Arguments* instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure
- Returns* the algorithm used to pack/unpack pixels and color samples of a field in memory

#### intel\_vvp\_vfr\_is\_running

*Prototype*

```
bool intel_vvp_vfr_is_running(intel_vvp_vfr_instance*
instance);
```

- Description* Reads and returns the running bit of the STATUS register. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized.
- Arguments* instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure
- Returns* True is the IP is currently reading a field, false when between two output fields

#### intel\_vvp\_vfr\_get\_commit\_status

*Prototype*

```
bool intel_vvp_vfr_get_commit_status(intel_vvp_vfr_instance*
instance);
```

- Description* Reads and returns the commit status bit of the STATUS register. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized.
- Arguments* instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure
- Returns* True is there are uncommitted writes

### intel\_vvp\_vfr\_get\_status

**Prototype**

```
uint32_t intel_vvp_vfr_get_status(intel_vvp_vfr_instance* instance);
```

**Description**

Reads the STATUS register. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized.

**Arguments**

instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

**Returns**

The content of the STATUS register

### intel\_vvp\_vfr\_get\_last\_buffer\_read\_addr

**Prototype**

```
uint32_t intel_vvp_vfr_get_last_buffer_read_addr(intel_vvp_vfr_instance* instance);
```

**Description**

Reads the LAST\_BUFFER\_READ register. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized.

**Arguments**

instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

**Returns**

The memory address of the last buffer read

### intel\_vvp\_vfr\_set\_num\_buffer\_sets

**Prototype**

```
int intel_vvp_vfr_set_num_buffer_sets(intel_vvp_vfr_instance* instance, uint8_t num_sets);
```

**Description**

Writes a new value to the NUM\_BUFFER\_SETS register and changes the number of active buffer sets

**Arguments**

instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

num\_sets - new number of buffer sets in the range [1..MAX\_NUM\_BUFFER\_SETS]

**Returns**

kIntelVvpCoreOk for success, negative error code in case of failures

kIntelVvpCoreInstanceErr if the instance is invalid or the new number of buffer sets is invalid/unsupported

### intel\_vvp\_vfr\_get\_num\_buffer\_sets

**Prototype**

```
uint8_t
intel_vvp_vfr_get_num_buffer_sets(intel_vvp_vfr_instance*
instance);
```

**Description** Reads and returns the number of buffer sets in use. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized and configured to enable debug.

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

**Returns** Last value written to the NUM\_BUFFER\_SETS register if the IP is configured to enable register read-back

### intel\_vvp\_vfr\_set\_buffer\_mode

**Prototype**

```
int intel_vvp_vfr_set_buffer_mode(intel_vvp_vfr_instance
*instance, eIntelVvpVfrBufferMode buffer_mode);
```

**Description** Writes a new value to the BUFFER\_MODE register and selects if the IP uses a single set or all configured sets

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

buffer\_mode - either kIntelVvpVfrSingleSet or kIntelVvpVfrAllSets

**Returns** kIntelVvpCoreOk for success, negative error code in case of failures  
kIntelVvpCoreInstanceErr if the instance is invalid or the buffer\_mode argument is invalid

### intel\_vvp\_vfr\_get\_buffer\_mode

**Prototype**

```
eIntelVvpVfrBufferMode
intel_vvp_vfr_get_buffer_mode(intel_vvp_vfr_instance*
instance);
```

**Description** Reads and returns the current value of the BUFFER\_MODE register. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized and configured to enable debug.

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

**Returns** Last value written to the BUFFER\_MODE register if the IP is configured to enable register read-back

### intel\_vvp\_vfr\_set\_starting\_buffer\_set

#### Prototype

```
int
intel_vvp_vfr_set_starting_buffer_set(intel_vvp_vfr_instance
*instance, uint8_t starting_buffer_set);
```

**Description** Writes a new value to the STARTING\_BUFFER\_SET register and selects the first active buffer set when a read starts

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

starting\_buffer\_set - the new active set if buffer\_mode is kIntelVvpVfrSingleSet or the starting buffer set if buffer\_mode is kIntelVvpVfrAllSets, this should be in the range [0..num\_buffer\_sets - 1]

**Returns** kIntelVvpCoreOk for success, negative error code in case of failures

kIntelVvpCoreInstanceErr if the instance is invalid

### intel\_vvp\_vfr\_get\_starting\_buffer\_set

#### Prototype

```
uint8_t
intel_vvp_vfr_get_starting_buffer_set(intel_vvp_vfr_instance*
instance);
```

**Description** Reads and returns the current value of the STARTING\_BUFFER\_SET register. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized and configured to enable debug.

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

**Returns** Last value written to the STARTING\_BUFFER\_SET register if the IP is configured to enable register read-back

### intel\_vvp\_vfr\_set\_run\_mode

#### Prototype

```
int intel_vvp_vfr_set_run_mode(intel_vvp_vfr_instance
*instance, eIntelVvpVfrRunMode run_mode);
```

**Description** Writes a new value to the RUN\_MODE register and chooses the read selection process used while outputting buffers. Note that the chosen STARTING\_BUFFER\_SET and BUFFER\_MODE also have an impact on the run-time read behavior. Writes to the RUN\_MODE register take effect after commit.

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

run\_mode - the new run\_mode

kIntelVvpVfrStop to stop the frame reader until another run-mode is selected

kIntelVvpVfrFreeRunning to process through the buffer sets and the buffers within the buffer sets in a round-robin fashion

kIntelVvpVfrFsync to process through the buffers in the same order as in free-running mode but using the fsync signal to trigger each buffer read

kIntelVvpVfrSingleShot to process through the buffers in the same order as in free-running mode but stopping once all buffers have been read and output once

**Returns** kIntelVvpCoreOk for success, negative error code in case of failures  
kIntelVvpCoreInstanceErr if the instance is invalid

#### intel\_vvp\_vfr\_get\_run\_mode

##### Prototype

```
eIntelVvpVfrRunMode
intel_vvp_vfr_get_run_mode(intel_vvp_vfr_instance* instance);
```

**Description** Reads and returns the current value of the RUN\_MODE register. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized and configured to enable debug.

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

**Returns** Last value written to the RUN\_MODE register if the IP is configured to enable register read-back

#### intel\_vvp\_vfr\_set\_fsync\_mode

##### Prototype

```
int intel_vvp_vfr_set_fsync_mode(intel_vvp_vfr_instance
*instance, eIntelVvpVfrFsyncMode sync_mode);
```

**Description** Writes a new value to the FSYNC\_PULSE\_MODE register and selects the first active buffer set when a read starts

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

sync\_mode - the new fsync trigger mechanism,  
kIntelVvpVfrFsyncRising to trigger the next field read on a rising fsync edge, kIntelVvpVfrFsyncBoth to trigger on either edge

**Returns** kIntelVvpCoreOk for success, negative error code in case of failures  
kIntelVvpCoreInstanceErr if the instance is invalid

### intel\_vvp\_vfr\_get\_fsync\_mode

**Prototype**

```
eIntelVvpVfrFsyncMode
intel_vvp_vfr_get_fsync_mode(intel_vvp_vfr_instance* instance);
```

**Description** Reads and returns the current value of the FSYNC\_PULSE\_MODE register. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized and configured to enable debug.

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

**Returns** Last value written to the FSYNC\_PULSE\_MODE register if the IP is configured to enable register read-back

### intel\_vvp\_vfr\_set\_bufset\_base\_addr

**Prototype**

```
int intel_vvp_vfr_set_bufset_base_addr(intel_vvp_vfr_instance
*instance, uint8_t bufset, uint32_t base_addr);
```

**Description** Writes a new value to the BUFSET\_BASE\_ADDR register of the chosen buffer set and selects a new base address for the first buffer of the set

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]

base\_addr - the base address for the first buffer of the set

**Returns** kIntelVvpCoreOk for success, negative error code in case of failures  
kIntelVvpCoreInstanceErr if the instance is invalid

### intel\_vvp\_vfr\_get\_bufset\_base\_addr

**Prototype**

```
uint32_t
intel_vvp_vfr_get_bufset_base_addr(intel_vvp_vfr_instance*
instance, uint8_t bufset);
```

**Description** Reads and returns the current value of the BUFSET\_BASE\_ADDR register for the selected buffer set. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized and configured to enable debug.

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]

**Returns** Last value written to the BUFSET\_BASE\_ADDR register for the selected buffer set if the IP is configured to enable register read-back

#### `intel_vvp_vfr_set_bufset_num_buffers`

**Prototype**

```
int
intel_vvp_vfr_set_bufset_num_buffers(intel_vvp_vfr_instance
*instance, uint8_t bufset, uint32_t num_buffers);
```

**Description** Writes a new value to the BUFSET\_NUM\_BUFFERS register of the chosen buffer set and selects a new number of buffers in the set

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]

num\_buffers - new number of buffers

**Returns** kIntelVvpCoreOk for success, negative error code in case of failures  
kIntelVvpCoreInstanceErr if the instance is invalid

#### `intel_vvp_vfr_get_bufset_num_buffers`

**Prototype**

```
uint32_t
intel_vvp_vfr_get_bufset_num_buffers(intel_vvp_vfr_instance*
instance, uint8_t bufset);
```

**Description** Reads and returns the current value of the BUFSET\_NUM\_BUFFERS register for the selected buffer set. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized and configured to enable debug.

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]

**Returns** Last value written to the BUFSET\_NUM\_BUFFERS register for the selected buffer set if the IP is configured to enable register read-back

#### `intel_vvp_vfr_set_bufset_inter_buffer_offset`

**Prototype**

```
int
intel_vvp_vfr_set_bufset_inter_buffer_offset(intel_vvp_vfr_instance
*instance, uint8_t bufset, uint32_t inter_buffer_offset);
```

- Description** Writes a new value to the BUFSET\_INTER\_BUFFER\_OFFSET register of the chosen buffer set and selects a new inter-buffer offset for the buffers of the set
- Arguments**
- instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure
  - bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]
  - inter\_buffer\_offset - the offset, in bytes, between two buffers in the set, or, in other words, the number of bytes allocated to each buffer in the set
- Returns**
- kIntelVvpCoreOk for success, negative error code in case of failures
  - kIntelVvpCoreInstanceErr if the instance is invalid

#### intel\_vvp\_vfr\_get\_bufset\_inter\_buffer\_offset

##### Prototype

```
uint32_t
intel_vvp_vfr_get_bufset_inter_buffer_offset(intel_vvp_vfr_instance*
instance, uint8_t bufset);
```

- Description** Reads and returns the current value of the BUFSET\_INTER\_BUFFER\_OFFSET register for the selected buffer set. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized and configured to enable debug.
- Arguments**
- instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure
  - bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]
- Returns** Last value written to the BUFSET\_INTER\_BUFFER\_OFFSET register for the selected buffer set if the IP is configured to enable register read-back

#### intel\_vvp\_vfr\_set\_bufset\_inter\_line\_offset

##### Prototype

```
int
intel_vvp_vfr_set_bufset_inter_line_offset(intel_vvp_vfr_instance
*instance, uint8_t bufset, uint32_t inter_line_offset);
```

- Description** Writes a new value to the INTER\_LINE\_OFFSET register of the chosen buffer set and selects a new inter-line offset for the buffers of the set
- Arguments**
- instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure
  - bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]
  - inter\_line\_offset - the offset, in bytes, between consecutive lines



**Returns** kIntelVvpCoreOk for success, negative error code in case of failures  
kIntelVvpCoreInstanceErr if the instance is invalid

### intel\_vvp\_vfr\_get\_bufset\_inter\_line\_offset

**Prototype**

```
uint32_t
intel_vvp_vfr_get_bufset_inter_line_offset(intel_vvp_vfr_instance*
instance, uint8_t bufset);
```

**Description** Reads and returns the current value of the INTER\_LINE\_OFFSET register for the selected buffer set. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized and configured to enable debug.

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]

**Returns** Last value written to the INTER\_LINE\_OFFSET register for the selected buffer set if the IP is configured to enable register read-back

### intel\_vvp\_vfr\_set\_bufset\_height

**Prototype**

```
int intel_vvp_vfr_set_bufset_height(intel_vvp_vfr_instance
*instance, uint8_t bufset, uint32_t height);
```

**Description** Writes a new value to the BUFSET\_HEIGHT register of the chosen buffer set and selects a new field height for the buffers of the set

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]

height - the new field height for the buffers of the set

**Returns** kIntelVvpCoreOk for success, negative error code in case of failures  
kIntelVvpCoreInstanceErr if the instance is invalid

### intel\_vvp\_vfr\_get\_bufset\_height

**Prototype**

```
uint32_t
intel_vvp_vfr_get_bufset_height(intel_vvp_vfr_instance*
instance, uint8_t bufset);
```

**Description** Reads and returns the current value of the BUFSET\_HEIGHT register for the selected buffer set. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized and configured to enable debug.

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]

**Returns** Last value written to the BUFSET\_HEIGHT register for the selected buffer set if the IP is configured to enable register read-back

#### intel\_vvp\_vfr\_set\_bufset\_width

**Prototype**

```
int intel_vvp_vfr_set_bufset_width(intel_vvp_vfr_instance
*instance, uint8_t bufset, uint32_t width);
```

**Description** Writes a new value to the BUFSET\_WIDTH register of the chosen buffer set and selects a new width for the buffers of the set

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]

width - the new width for the buffers of the set

**Returns** kIntelVvpCoreOk for success, negative error code in case of failures

kIntelVvpCoreInstanceErr if the instance is invalid

#### intel\_vvp\_vfr\_get\_bufset\_width

**Prototype**

```
uint32_t
intel_vvp_vfr_get_bufset_width(intel_vvp_vfr_instance*
instance, uint8_t bufset);
```

**Description** Reads and returns the current value of the BUFSET\_WIDTH register for the selected buffer set. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized and configured to enable debug.

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]

**Returns** Last value written to the BUFSET\_WIDTH register for the selected buffer set if the IP is configured to enable register read-back

#### intel\_vvp\_vfr\_set\_bufset\_interlace

**Prototype**

```
int intel_vvp_vfr_set_bufset_interlace(intel_vvp_vfr_instance
*instance, uint8_t bufset, uint32_t interlace);
```

- Description** Writes a new value to the BUFSET\_INTERLAC register of the chosen buffer set and selects a new interlace nibble for the first buffer of the set
- Arguments**
- instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure
  - bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]
  - interlace - the interlace nibble for the first field in the set
- Returns**
- kIntelVvpCoreOk for success, negative error code in case of failures
  - kIntelVvpCoreInstanceErr if the instance is invalid

#### intel\_vvp\_vfr\_get\_bufset\_interlace

**Prototype**

```
uint32_t
intel_vvp_vfr_get_bufset_interlace(intel_vvp_vfr_instance*
instance, uint8_t bufset);
```

- Description** Reads and returns the current value of the BUFSET\_INTERLAC register for the selected buffer set. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized and configured to enable debug.
- Arguments**
- instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure
  - bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]
- Returns** Last value written to the BUFSET\_INTERLAC register for the selected buffer set if the IP is configured to enable register read-back

#### intel\_vvp\_vfr\_set\_bufset\_colorspace

**Prototype**

```
int
intel_vvp_vfr_set_bufset_colorspace(intel_vvp_vfr_instance
*instance, uint8_t bufset, uint8_t colorspace);
```

- Description** Writes a new value to the BUFSET\_COLORSPACE register of the chosen buffer set and selects a new colorspace for the set
- Arguments**
- instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure
  - bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]
  - colorspace - the new colorspace code for the set
- Returns** kIntelVvpCoreOk for success, negative error code in case of failures

kIntelVvpCoreInstanceErr if the instance is invalid

### intel\_vvp\_vfr\_get\_bufset\_colorspace

#### Prototype

```
uint8_t
intel_vvp_vfr_get_bufset_colorspace(intel_vvp_vfr_instance*
instance, uint8_t bufset);
```

**Description** Reads and returns the current value of the BUFSET\_COLORSPACE register for the selected buffer set. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized and configured to enable debug.

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]

**Returns** Last value written to the BUFSET\_COLORSPACE register for the selected buffer set if the IP is configured to enable register read-back

### intel\_vvp\_vfr\_set\_bufset\_subsampling

#### Prototype

```
int
intel_vvp_vfr_set_bufset_subsampling(intel_vvp_vfr_instance
*instance, uint8_t bufset, uint8_t subsampling);
```

**Description** Writes a new value to the BUFSET\_SUBSAMPLING register of the chosen buffer set and selects a new subsampling for the set

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]

subsampling - the new subsampling code for the set

**Returns** kIntelVvpCoreOk for success, negative error code in case of failures  
kIntelVvpCoreInstanceErr if the instance is invalid

### intel\_vvp\_vfr\_get\_bufset\_subsampling

#### Prototype

```
uint8_t
intel_vvp_vfr_get_bufset_subsampling(intel_vvp_vfr_instance*
instance, uint8_t bufset);
```

|                    |                                                                                                                                                                                                           |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i> | Reads and returns the current value of the BUFSET_SUBSAMPLING register for the selected buffer set. The instance must be a valid intel_vvp_vfr_instance fully initialized and configured to enable debug. |
| <i>Arguments</i>   | instance - pointer to the intel_vvp_vfr_instance software driver instance structure<br><br>bufset - the buffer set ID in the range [0..MAX_BUFFER_SETS]                                                   |
| <i>Returns</i>     | Last value written to the BUFSET_SUBSAMPLING register for the selected buffer set if the IP is configured to enable register read-back                                                                    |

#### intel\_vvp\_vfr\_set\_bufset\_cositing

|                    |                                                                                                                                                                                                             |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Prototype</i>   | <pre>int intel_vvp_vfr_set_bufset_cositing(intel_vvp_vfr_instance*instance, uint8_t bufset, uint8_t cositing);</pre>                                                                                        |
| <i>Description</i> | Writes a new value to the BUFSET_COSITING register of the chosen buffer set and selects a new cositing for the set                                                                                          |
| <i>Arguments</i>   | instance - pointer to the intel_vvp_vfr_instance software driver instance structure<br><br>bufset - the buffer set ID in the range [0..MAX_BUFFER_SETS]<br><br>cositing - the new cositing code for the set |
| <i>Returns</i>     | kIntelVvpCoreOk for success, negative error code in case of failures<br><br>kIntelVvpCoreInstanceErr if the instance is invalid                                                                             |

#### intel\_vvp\_vfr\_get\_bufset\_cositing

|                    |                                                                                                                                                                                                        |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Prototype</i>   | <pre>uint8_t<br/>intel_vvp_vfr_get_bufset_cositing(intel_vvp_vfr_instance*instance, uint8_t bufset);</pre>                                                                                             |
| <i>Description</i> | Reads and returns the current value of the BUFSET_COSITING register for the selected buffer set. The instance must be a valid intel_vvp_vfr_instance fully initialized and configured to enable debug. |
| <i>Arguments</i>   | instance - pointer to the intel_vvp_vfr_instance software driver instance structure<br><br>bufset - the buffer set ID in the range [0..MAX_BUFFER_SETS]                                                |
| <i>Returns</i>     | Last value written to the BUFSET_COSITING register for the selected buffer set if the IP is configured to enable register read-back                                                                    |

### intel\_vvp\_vfr\_set\_bufset\_bps

**Prototype**

```
int intel_vvp_vfr_set_bufset_bps(intel_vvp_vfr_instance
*instance, uint8_t bufset, uint8_t bps);
```

**Description**

Writes a new value to the BUFSET\_BPS register of the chosen buffer set and selects a new bits per sample value for the set

**Arguments**

instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]

bps - the new bits per sample value for the set

**Returns**

kIntelVvpCoreOk for success, negative error code in case of failures

kIntelVvpCoreInstanceErr if the instance is invalid

### intel\_vvp\_vfr\_get\_bufset\_bps

**Prototype**

```
uint8_t intel_vvp_vfr_get_bufset_bps(intel_vvp_vfr_instance*
instance, uint8_t bufset);
```

**Description**

Reads and returns the current value of the BUFSET\_BPS register for the selected buffer set. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized and configured to enable debug.

**Arguments**

instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]

**Returns**

Last value written to the BUFSET\_BPS register for the selected buffer set if the IP is configured to enable register read-back

### intel\_vvp\_vfr\_set\_bufset\_field\_count

**Prototype**

```
int
intel_vvp_vfr_set_bufset_field_count(intel_vvp_vfr_instance
*instance, uint8_t bufset, uint16_t field_count);
```

**Description**

Writes a new value to the BUFSET\_FIELD\_COUNT register of the chosen buffer set and selects a new field\_count value for the first buffer of the set

**Arguments**

instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]

field\_count - the new field\_count value for the first buffer of the set

**Returns** kIntelVvpCoreOk for success, negative error code in case of failures  
kIntelVvpCoreInstanceErr if the instance is invalid

#### intel\_vvp\_vfr\_get\_bufset\_field\_count

**Prototype**

```
uint16_t
intel_vvp_vfr_get_bufset_field_count(intel_vvp_vfr_instance*
instance, uint8_t bufset);
```

**Description** Reads and returns the current value of the BUFSET\_FIELD\_COUNT register for the selected buffer set. The instance must be a valid intel\_vvp\_vfr\_instance fully initialized and configured to enable debug.

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

bufset - the buffer set ID in the range [0..MAX\_BUFFER\_SETS]

**Returns** Last value written to the BUFSET\_FIELD\_COUNT register for the selected buffer set if the IP is configured to enable register read-back

#### intel\_vvp\_vfr\_commit\_writes

**Prototype**

```
int intel_vvp_vfr_commit_writes(intel_vvp_vfr_instance*
instance)
```

**Description** Commits all pending writes before processing the next field. The instance must be valid.

**Arguments** instance - pointer to the intel\_vvp\_vfr\_instance software driver instance structure

**Returns** kIntelVvpCoreOk (0) for success, negative error code otherwise



## 49. Video Frame Writer IP

---

### 49.1. About the Video Buffer Writer IP

The IP receives frames or fields from an Altera Streaming Video input and stores them in external memory. The frame writer has a host Avalon memory-mapped interface to allow connection to an external memory interface and an agent Avalon memory-mapped interface to allow run-time control.

The frame writer supports:

- Maximum frame resolutions of 16,384 by 16,384 pixels with 1 to 8 pixels in parallel and any color space
- Writing of progressive frames or interlaced fields
- Configurable memory packing scheme
- Runtime location of frame or field anywhere in memory
- Configurable interrupt on completion of frame, field, or line writes
- Free running or single-shot mode for event capture
- Configurable discarding of broken fields or frames
- Automatic receiving and storing of up to 16,777,215 fields or frames contiguously in memory with no host control

The IP is available as full or lite variants. For more information on full and lite, refer to the *Altera Streaming Video Protocol Specification*. The Video Frame Writer IP takes resolution information from the configurable buffer set registers.

An Avalon memory-mapped interface allows you to configure buffer sets and change the operating mode at run time. You must have this interface for both full and lite variants.

For details about latency and reset behavior for the Video Frame Writer, refer to *Video and Vision IPs Functional Description*.

#### Related Information

[Altera Streaming Video Protocol Specification](#)

#### 49.1.1. Video Frame Writer IP Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family and the number of supported bits per color sample, colors per pixel, and pixels in parallel.



**Table 971. Video Frame Writer IP Performance and Resources**

The table shows ALM usage and  $f_{MAX}$  for a design with an 8K frame writer with 2 pixels in parallel, 10 bits per color sample, 3 color planes and separate control clock.

| Target Device                                 | ALMs  | $f_{MAX}$ (MHz) |
|-----------------------------------------------|-------|-----------------|
| Agilex 7 speed grade 2 (AGFA012R24A2E2V)      | 2,009 | 740             |
| Cyclone 10 GX speed grade 5 (10CX220YF78015G) | 1,693 | 417             |

## 49.2. Video Frame Writer IP Parameters

The IP offers compile-time parameters.

**Table 972. Video Frame Writer IP Parameters**

| Parameter                                                     | Values                          | Description                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------------------------------------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Video Data Format</b>                                      |                                 |                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Bits per color sample</b>                                  | 8 to 16                         | Select the number of bits per color sample.                                                                                                                                                                                                                                                                                                                        |
| <b>Number of color planes</b>                                 | 1 to 4                          | Select the number of color planes per pixel.                                                                                                                                                                                                                                                                                                                       |
| <b>Number of pixels in parallel</b>                           | 1 to 8                          | Select the number of pixels in parallel.                                                                                                                                                                                                                                                                                                                           |
| <b>Maximum Frame Size</b>                                     |                                 |                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Maximum frame height</b>                                   | 32 to 16384                     | Select the maximum height of frames. If you attempt to write fields or frames taller than this, they are cropped to this height.                                                                                                                                                                                                                                   |
| <b>Maximum frame width</b>                                    | 32 to 16384                     | Select the maximum width of frames. If you attempt to write fields or frames wider than this, they are cropped to this width.<br><br>If you use the IP exclusively to write frames with 420 subsampling, optionally, halve the maximum frame width entry in the GUI. Halving the entry optimizes memory footprint because of the more efficient 420 pixel packing. |
| <b>Control</b>                                                |                                 |                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Lite mode</b>                                              | On or off                       | Turn on to operate the frame writer in lite mode.                                                                                                                                                                                                                                                                                                                  |
| <b>Separate clock for control interface</b>                   | On or off                       | Turn on for a separate clock for the control interface.                                                                                                                                                                                                                                                                                                            |
| <b>Debug features</b>                                         | On or off                       | Turn on for debug features.                                                                                                                                                                                                                                                                                                                                        |
| <b>Memory</b>                                                 |                                 |                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Avalon memory mapped host(s) local ports width</b>         | 16, 32, 64, 128, 256, 512, 1024 | Select in bits the width of the Avalon memory-mapped host write port. You must select a width at least as wide as the Altera Streaming Video input tdata width.                                                                                                                                                                                                    |
| <b>Avalon memory mapped host(s) local ports address width</b> | 8 to 32                         | Select in bits the width of the Avalon memory-mapped host write address port. It must be sufficient to fully address the last buffer.                                                                                                                                                                                                                              |
| <b>The depth of the write FIFO</b>                            | 32,64,128,256,512,1024,2048     | Specify the depth of the write FIFO buffer. Each FIFO buffer entry holds one word the width of the specified Avalon memory-mapped local port width. You must specify a FIFO depth of at least twice the specified burst target so that the                                                                                                                         |

*continued...*

| Parameter                                                            | Values                         | Description                                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                      |                                | IP can hold at least 2 bursts at any one time. Increase the FIFO depth to improve resilience to latency on the Avalon memory-mapped interface.                                                                                                                                                                                                    |
| <b>Avalon memory mapped write burst target</b>                       | 2,4,8,16,32,64                 | Select the burst target for writes. Longer bursts provide more efficiency on the bus but require more local storage in the write FIFO buffer.                                                                                                                                                                                                     |
| <b>Packing method</b>                                                | <b>Perfect, color or pixel</b> | Perfect packing minimizes memory footprint of stored frames but increases complexity and therefore size of the frame writer slightly. Color packing leaves spaces in memory between colors if colors do not pack into memory words exactly. Pixel packing leaves spaces in memory between pixels if pixels do not pack into memory words exactly. |
| <b>Separate clock for the Avalon memory-mapped host interface(s)</b> | On or off                      | -                                                                                                                                                                                                                                                                                                                                                 |

Figure 131. Frame Writer IP GUI

The screenshot displays the configuration GUI for the Video Frame Writer Intel FPGA IP. The title bar indicates the component is 'Video Frame Writer Intel FPGA IP' with the identifier 'intel\_vvp\_vfw'. A 'Details' button is located in the top right corner. The configuration is organized into several expandable sections:

- CapInfo**: A section header.
- Video Data Format**: Contains three input fields:
  - Bits per color sample: 10
  - Number of color planes: 3
  - Number of pixels in parallel: 4
- Writer parameters**: Contains two input fields:
  - Maximum field height: 8192
  - Maximum field width: 8192
- Control**: Contains three checkboxes:
  - ☐ Lite mode
  - ☒ Separate clock for control interface
  - ☐ Debug features
- Memory**: Contains four dropdown menus and one checkbox:
  - Avalon memory-mapped local ports width: 128
  - The depth of the write FIFO.: 64
  - Avalon memory-mapped write burst target: 32
  - Packing method: PERFECT
  - ☒ Separate clock for the Avalon memory-mapped host interface(s)

### 49.3. Video Frame Writer IP Functional Description

The IP receives video fields or frames from its Altera Streaming Video input and writes them to external memory via its Avalon memory-mapped interface.

1. Configure the IP with the number of buffers that you want to write to with the `csr_num_buffers` register.
2. Program the base address in memory for the first buffer, the offset between buffers and the offset between individual lines within buffers via the `csr_buffer_base`, `csr_inter_buffer_offset` and `csr_inter_line_offset` registers.
3. Set the `csr_overwrite_broken` register if you want broken frames to be overwritten and then `csr_run` to either free-running mode for continuous operation or single-shot mode.
4. Commit these settings with a write to the `csr_commit` register.

If the IP receives any packets on its Altera Streaming Video input before you configure and commit it, the IP consumes the packets. When consuming, the IP raises `axi4s_vid_in_tready`.

### Register Behavior

Bit [0] of the `csr_status` register goes high when the IP starts writing the first frame. It goes low after the IP finishes writing the last line of the frame. It returns high when the IP starts writing the next frame.

The IP sets the `csr_buffer_available` register after it writes the first frame and a buffer is available in memory. `csr_buffer_start_address` holds the base address of the first frame and `csr_buffer_write_count` increments with each frame the IP writes. If the IP receives an interlaced `f1` field, it sets `csr_buffer_f1_flag`. The IP sets `csr_buffer_field_width` and `csr_buffer_field_height` to the dimensions of the new value. In full mode, the IP extracts the field count from each end of field packet and updates `csr_field_count`.

Acknowledge the buffer by writing to the `csr_buffer_acknowledge` register, which resets `csr_buffer_available`. If you do not acknowledge the buffer, `csr_buffer_available` remains set, the IP keeps writing frames and `csr_buffer_start_address` and associated registers update as normal.

### Latency

The frame writer latency depends on the availability of the external memory interfaces, which may create backpressure for the IP's write interface via the `av_mm_mem_read_host_waitrequest` signal.

The worst-case latency figures are when the frame writer experiences no backpressure. Any backpressure increases these latencies by the same amount of cycles.

The latency figures are for when the host interfaces do not have a separate clock, so all interfaces operate from the same clock.

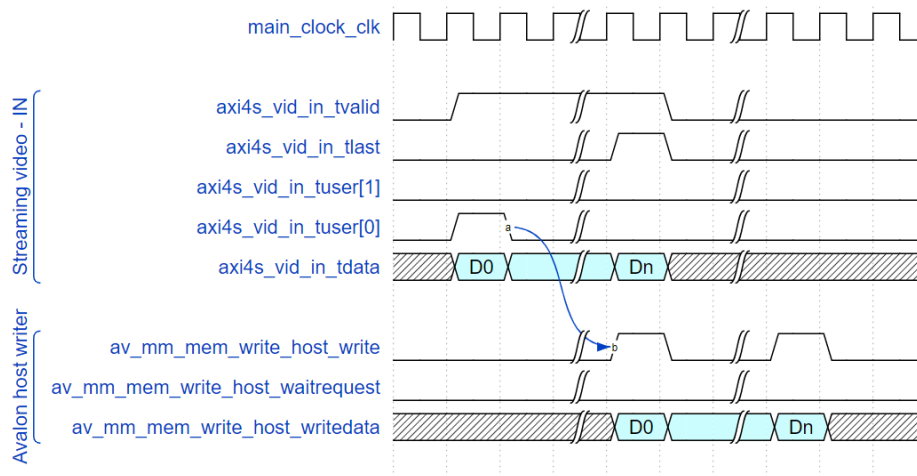
The latency figures are for a typical configuration. Reducing the burst target reduces latency but at the cost of decreasing efficiency and increasing bandwidth consumed on the bus.

**Table 973. Latency**

The latency figures are the same for both full and lite variants of the IP.

| Initiating event                                        | Resultant event                                          | Latency (measured in clock cycles) |
|---------------------------------------------------------|----------------------------------------------------------|------------------------------------|
| axi4s_vid_in_tuser[0] strobe to indicate start of frame | First av_mm_mem_write_host_write strobe of a frame write | 44                                 |

**Figure 132. Latency**



### 49.3.1. Video Frame Writer IP Interfaces

**Table 974. Video Frame Writer IP Interfaces**

| Name                                           | Direction | Width | Description                              |
|------------------------------------------------|-----------|-------|------------------------------------------|
| <b>Clocks and resets</b>                       |           |       |                                          |
| <code>main_clock_clk</code>                    | In        | 1     | AXI4-S processing clock.                 |
| <code>main_reset_rst</code>                    | In        | 1     | AXI4-S processing reset.                 |
| <code>control_clock_clk</code>                 | In        | 1     | Optional control agent interface clock.  |
| <code>control_reset_reset</code>               | In        | 1     | Optional control agent interface reset.  |
| <b>Control interfaces</b>                      |           |       |                                          |
| <code>av_mm_control_agent_address</code>       | In        | 7     | Avalon memory-mapped agent address       |
| <code>av_mm_control_agent_write</code>         | In        | 1     | Avalon memory-mapped agent write.        |
| <code>av_mm_control_agent_writedata</code>     | In        | 32    | Avalon memory-mapped agent write data.   |
| <code>av_mm_control_agent_byteenable</code>    | In        | 4     | Avalon memory-mapped agent byte enable.  |
| <code>av_mm_control_agent_read</code>          | In        | 1     | Avalon memory-mapped agent read.         |
| <code>av_mm_control_agent_readdata</code>      | Out       | 32    | Avalon memory-mapped agent read data.    |
| <code>av_mm_control_agent_readdatavalid</code> | Out       | 1     | Avalon memory-mapped agent read.         |
| <code>av_mm_control_agent_waitrequest</code>   | Out       | 1     | Avalon memory-mapped agent wait request. |
| <i>continued...</i>                            |           |       |                                          |

| Name                                                                                                           | Direction | Width | Description                                  |
|----------------------------------------------------------------------------------------------------------------|-----------|-------|----------------------------------------------|
| <b>Altera Streaming Video interface</b>                                                                        |           |       |                                              |
| axi4s_vid_in_tdata                                                                                             | In        | (147) | AXI4-S data in.                              |
| axi4s_vid_in_tvalid                                                                                            | In        | 1     | AXI4-S data valid.                           |
| axi4s_vid_in_tuser[0]                                                                                          | In        | 1     | AXI4-S start of video frame.                 |
| axi4s_vid_in_tuser[1]                                                                                          | In        | 1     | AXI4-S control or data packet.               |
| axi4s_vid_in_tuser[N-1:2]                                                                                      | In        | (148) | Unused.                                      |
| axi4s_vid_in_tlast                                                                                             | In        | 1     | AXI4-S end of packet.                        |
| axi4s_vid_in_tready                                                                                            | Out       | 1     | AXI4-S data ready                            |
| <b>Avalon memory-mapped host interfaces</b>                                                                    |           |       |                                              |
| Widths for address, data, and burst count buses are as you specify in the GUI. The table shows example widths. |           |       |                                              |
| mem_clock_clk                                                                                                  | In        | 1     | Optional host interface clock.               |
| mem_reset_reset                                                                                                | In        | 1     | Optional host interface reset.               |
| av_mm_mem_write_host_address                                                                                   | Out       | 32    | Avalon memory-mapped host address            |
| av_mm_mem_write_host_write                                                                                     | Out       | 1     | Avalon memory-mapped host write.             |
| av_mm_mem_write_host_burstcount                                                                                | Out       | 5     | Avalon memory-mapped host write burst count. |
| av_mm_mem_write_host_writedata                                                                                 | In        | 32    | Avalon memory-mapped host write data.        |
| av_mm_mem_write_host_waitrequest                                                                               | In        | 1     | Avalon memory-mapped host wait request.      |

### Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

## 49.4. Video Frame Writer IP Registers

Each register is either read-only (RO) or read-write (RW).

**Table 975. Video Frame Writer IP Registers**

In the software API the register names appear with a prefix of INTEL\_VVP, INTEL\_VVP\_CORE, or INTEL\_VVP\_VIDEO\_FRAME\_WRITER as appropriate and with an optional REG suffix.

| Address                           | Register | Access | Description                                                                                         |
|-----------------------------------|----------|--------|-----------------------------------------------------------------------------------------------------|
| <b>Parameterization registers</b> |          |        |                                                                                                     |
| 0x0000                            | VID_PID  | RO     | Read this register for the Video Frame Writer product ID. This register always returns 0x6AF7_0249. |
| <i>continued...</i>               |          |        |                                                                                                     |

- (147) The equation gives all axi4s\_vid\_out\_tdata widths in these interfaces:  

$$\max(2, \text{ceil}((\text{bits per color sample} \times \text{number of color planes})/8)) \times \text{pixels in parallel} \times 8$$
- (148) This equation gives all axi4s\_vid\_out\_tuser widths in these interfaces  $N = \text{ceil}(\text{tdata width} / 8)$

| Address                            | Register               | Access  | Description                                                                                                                           |
|------------------------------------|------------------------|---------|---------------------------------------------------------------------------------------------------------------------------------------|
| 0x0004                             | VERSION                | RO      | Read this register for the IP version information.                                                                                    |
| 0x0008                             | LITE_MODE              | RO      | Read this register to determine if <b>Lite mode</b> is on or off. This register returns 0 when <b>Lite mode</b> is off and 1 when on. |
| 0x000C                             | DEBUG_ENABLED          | RO      | Read this register to determine if <b>Debug features</b> is on.                                                                       |
| 0x0010                             | MAX_HEIGHT             | RO      | Read this register to determine the maximum supported frame height.                                                                   |
| 0x0014                             | MAX_WIDTH              | RO      | Read this register to determine the maximum supported frame width.                                                                    |
| 0x0018                             | BITS_PER_SYMBOL        | RO      | Read this register for the number of bits per symbol configured.                                                                      |
| 0x001C                             | NUMBER_OF_COLOR_PLANES | RO      | Read this register for the number of color planes.                                                                                    |
| 0x0020                             | PIXELS_IN_PARALLEL     | RO      | Read this register for the number of pixels in parallel.                                                                              |
| 0x0024                             | PACKING                | RO      | Read this register for the pixel packing scheme.                                                                                      |
| 0x0028 to 0x00FF                   | RESERVED               |         |                                                                                                                                       |
| <b>Interrupt registers</b>         |                        |         |                                                                                                                                       |
| 0x0100                             | IRQ_CONTROL            | RW      | IRQ control. Set this register to configure interrupts                                                                                |
| 0x0104                             | IRQ_STATUS             | RW      | Read this status register to see if an interrupt fired.                                                                               |
| 0x0108 to 0x011F                   | RESERVED               |         |                                                                                                                                       |
| <b>Control and Debug registers</b> |                        |         |                                                                                                                                       |
| 0x0120                             | IMG_INFO_WIDTH         | RW / RO | Image information width.                                                                                                              |
| 0x0124                             | IMG_INFO_HEIGHT        | RW / RO | Image information height.                                                                                                             |
| 0x0128                             | IMG_INFO_INTERLACE     | RO      | Image information interlace.                                                                                                          |
| 0x012C                             | RESERVED               | RW / RO | Reserved.                                                                                                                             |
| 0x0130                             | IMG_INFO_COLORSPACE    | RW / RO | Image information color space                                                                                                         |
| 0x0134                             | IMG_INFO_SUBSAMPLING   | RW / RO | Image information subsampling.                                                                                                        |
| 0x0138                             | IMG_INFO_COSITING      | RW / RO | Image information cositing.                                                                                                           |
| 0x013C                             | IMG_INFO_FIELD_COUNT   | RW / RO | Field count image information.                                                                                                        |
| 0x0140                             | STATUS                 | RO      | Status register                                                                                                                       |
| 0x0144                             | BUFFER_AVAILABLE       | RO      | Buffer available.                                                                                                                     |
| 0x0148                             | BUFFER_WRITE_COUNT     | RO      | Buffer write count<br>This register is reset after a write to the COMMIT register.                                                    |
| 0x014C                             | BUFFER_START_ADDRESS   | RO      | The start address of the buffer the IP most recently writes.                                                                          |
| 0x0150                             | BUFFER_F1_FLAG         | RO      | Buffer F1 flag.                                                                                                                       |
| 0x0154                             | BUFFER_FIELD_WIDTH     | RO      | Buffer field width,                                                                                                                   |
| 0x0158                             | BUFFER_FIELD_HEIGHT    | RO      | Buffer field height.                                                                                                                  |
| <b>continued...</b>                |                        |         |                                                                                                                                       |

| Address          | Register            | Access | Description                                                                                                                                                                                                                                                                                                                               |
|------------------|---------------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x015C           | RESERVED            |        |                                                                                                                                                                                                                                                                                                                                           |
| 0x0160           | FIELD_COUNT         | RO     | The value from the end-of-field packet of the most recently written buffer.                                                                                                                                                                                                                                                               |
| 0x0164           | COMMIT              | RW     | Write any value to the COMMIT register to commit new RW register values and request an update to the new settings.                                                                                                                                                                                                                        |
| 0x0168           | BUFFER_ACKNOWLEDGE  | RW     | Write to the BUFFER_ACKNOWLEDGE register to reset the BUFFER_AVAILABLE register.<br>BUFFER_ACKNOWLEDGE is the one exception to the need to write to COMMIT after updating a RW register.                                                                                                                                                  |
| 0x016C           | RUN                 | RW     | Set the required operating mode:<br>0 - Frame writer is not running (stopped)<br>1 - Frame writer is free-running<br>2 - Frame writer is not running (stopped)<br>3 - Frame writer is in single-shot mode.<br>Any changes to this register must be followed by a write to the COMMIT register and take effect at the next start of field. |
| 0x0170           | NUM_BUFFERS         | RW     | The number of buffers.<br>Any changes to this register must be followed by a write to the COMMIT register and take effect at the next start of field.                                                                                                                                                                                     |
| 0x0174           | BUFFER_BASE         | RW     | Buffer base address.<br>Any changes to this register must be followed by a write to the COMMIT register and take effect at the next start of field                                                                                                                                                                                        |
| 0x0178           | INTER_BUFFER_OFFSET | RW     | Interbuffer address increment.<br>Any changes to this register must be followed by a write to the COMMIT register and take effect at the next start of field                                                                                                                                                                              |
| 0x017C           | INTER_LINE_OFFSET   | RW     | Interline address increment.<br>Any changes to this register must be followed by a write to the COMMIT register and take effect at the next start of field                                                                                                                                                                                |
| 0x0180 to 0x0193 | RESERVED            |        |                                                                                                                                                                                                                                                                                                                                           |
| 0x0194           | IRQ_LINE_THRESHOLD  | RW     | IRQ line threshold.                                                                                                                                                                                                                                                                                                                       |
| 0x0198           | OVERWRITE_BROKEN    | RW     | Set to overwrite broken fields..                                                                                                                                                                                                                                                                                                          |

### Register Bit Descriptions

**Table 976. VID\_PID**

| Name                                   | Bits | Description                                                                                                                                                                                                  |
|----------------------------------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frame writer version ID and product ID | 31:0 | This register always returns <b>0x6AF7_0249</b> <ul style="list-style-type: none"> <li>15:0 is the product ID and always returns 0x0249</li> <li>31:16 is the vendor ID and always returns 0x6AF7</li> </ul> |

**Table 977. VERSION**

| Name                 | Bits | Description           |
|----------------------|------|-----------------------|
| Register map version | 7:0  | Register map version. |
| <i>continued...</i>  |      |                       |



|                    |       |                                      |
|--------------------|-------|--------------------------------------|
| IP patch revision  | 15:8  | -                                    |
| IP update revision | 23:16 | Updated when the IP version changes. |
| IP major revision  | 31:24 | Updated when the IP version changes. |

**Table 978. LITE\_MODE**

| Name                           | Bits | Description                                 |
|--------------------------------|------|---------------------------------------------|
| Lite mode parameterization bit | 0    | Returns 1 if you turn on <b>lite mode</b> . |
| Unused                         | 31:1 | Unused.                                     |

**Table 979. DEBUG\_ENABLED**

| Name                                | Bits | Description                                      |
|-------------------------------------|------|--------------------------------------------------|
| Debug features parameterization bit | 0    | Returns 1 if you turn on <b>Debug features</b> . |
| Unused                              | 31:1 | Unused.                                          |

**Table 980. MAX\_HEIGHT**

| Name       | Bits | Description                                 |
|------------|------|---------------------------------------------|
| Max height | 31:0 | Returns the maximum supported frame height. |

**Table 981. MAX\_WIDTH**

| Name      | Bits | Description                                |
|-----------|------|--------------------------------------------|
| Max width | 31:0 | Returns the maximum supported frame width. |

**Table 982. BITS\_PER\_SYMBOL**

| Name            | Bits | Description                                                                                     |
|-----------------|------|-------------------------------------------------------------------------------------------------|
| Bits per symbol | 31:0 | Returns the number of bits per symbol that the Altera Streaming Video output is configured for. |

**Table 983. NUMBER\_OF\_COLOR\_PLANES**

| Name                   | Bits | Description                                                                                  |
|------------------------|------|----------------------------------------------------------------------------------------------|
| Number of color planes | 31:0 | Returns the number of color planes that the Altera Streaming Video output is configured for. |

**Table 984. PIXELS\_IN\_PARALLEL**

| Name                         | Bits | Description                                                                                                      |
|------------------------------|------|------------------------------------------------------------------------------------------------------------------|
| Number of pixels in parallel | 31:0 | This register returns the number of pixels in parallel that the Altera Streaming Video output is configured for. |

**Table 985. PACKING**

| Name    | Bits | Description                                                                                                                 |
|---------|------|-----------------------------------------------------------------------------------------------------------------------------|
| Packing | 31:0 | This register returns the packing scheme that the IP uses:<br>0 = Perfect packing<br>1 = Color packing<br>2 = Pixel packing |

**Table 986. IRQ\_CONTROL**

| Name            | Bit | Description                                                                                                                                                                                                                                                                                                                                                                             |
|-----------------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Field_write_irq | 0   | Set the <code>field_write_irq</code> bit of the <code>IRQ_CONTROL</code> register to enable interrupts on completing field writes. Clear to disable interrupts on completing field writes.                                                                                                                                                                                              |
| Line_write_irq  | 1   | Set the <code>line_write_irq</code> bit of the <code>IRQ_CONTROL</code> register to enable interrupts on completion of line writes. Clear the <code>line_write_irq</code> bit to disable interrupts on completion of line writes.<br>Select which line generates the interrupt by programming <code>IRQ_LINE_THRESHOLD</code> with the required line number. Line numbers start from 0. |

**Table 987. IRQ\_STATUS**

| Name            | Bit | Description                                                                                                                                                                                                                         |
|-----------------|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Field_write_irq | 0   | The IP sets the <code>field_write_irq</code> bit of the <code>IRQ_STATUS</code> register if the interrupt fires. Write to the <code>field_write_irq</code> bit of the <code>IRQ_STATUS</code> register to clear down the interrupt. |
| line_write_irq  | 1   | The IP sets the <code>line_write_irq</code> bit of the <code>IRQ_STATUS</code> register if the interrupt fires. Write to the <code>line_write_irq</code> bit of the <code>IRQ_STATUS</code> register to clear down the interrupt    |

**Table 988. IMG\_INFO\_WIDTH**

| Name  | Bits | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| width | 16:0 | The field width that the writer IP uses when writing to memory.<br>If <b>Lite mode</b> is on, this register is RW. Write the expected width in pixels of incoming fields.<br>If <b>Lite mode</b> is off, if debug features are on, this register is RO and contains the field width received via the image information packet.<br>The IP removes extra pixels from received fields wider than <code>IMG_INFO_WIDTH</code> . The IP writes narrower fields leaving pixels from any previous fields in the buffer unmodified.<br>After any changes to this register write to the <code>COMMIT</code> register, which takes effect at the next start of field. |

**Table 989. IMG\_INFO\_HEIGHT**

| Name   | Bits | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|--------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| height | 16:0 | The field height that the writer uses when writing to memory.<br>If <b>Lite mode</b> is on, this register is RW. Write the expected height in lines of incoming fields.<br>If <b>Lite mode</b> is off, if debug features are on, this register is RO and contains the field height received via the image information packet.<br>The IP removes extra lines from fields received that are taller than <code>IMG_INFO_HEIGHT</code> .<br>Shorter fields leave pixels from any previous fields in the buffer unmodified.<br>After any changes to this register write to the <code>COMMIT</code> register, which takes effect at the next start of field. |

**Table 990. IMG\_INFO\_INTERLACE**

| Name      | Bits | Description                                                                                                                                                                                                                                                                                                             |
|-----------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| interlace | 3:0  | If <b>Lite mode</b> is on, the IP stores interlaced fields in the same way as progressive frames by the frame writer, so you do not need to set this nibble.<br>If <b>Lite mode</b> is off, if debug features are on, this register is RO and contains the interlaced nibble received via the image information packet. |

**Table 991. IMG\_INFO\_COLORSPACE**

| Name     | Bits | Description                                                                                                                                                                           |
|----------|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| colspace | 6:0  | Unused If <b>Lite mode</b> is on.<br>If <b>Lite mode</b> is off, if debug features are on, this register is RO and contains the colorspace received via the image information packet. |

**Table 992. IMG\_INFO\_SUBSAMPLING**

| Name        | Bits | Description                                                                                                                                                                            |
|-------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| subsampling | 1:0  | Unused If <b>Lite mode</b> is on.<br>If <b>Lite mode</b> is off, if debug features are on, this register is RO and contains the subsampling received via the image information packet. |

**Table 993. IMG\_INFO\_COSITING**

| Name      | Bits | Description                                                                                                                                                                         |
|-----------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| co-siting | 1:0  | Unused If <b>Lite mode</b> is on.<br>If <b>Lite mode</b> is off, if debug features are on, this register is RO and contains the cositing received via the image information packet. |

**Table 994. IMG\_INFO\_FIELD\_COUNT**

| Name        | Bits | Description                                                                                                                                                                                         |
|-------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| field_count | 5:0  | Unused If <b>Lite mode</b> is on.<br>If <b>Lite mode</b> is off, if debug features are on, this register is RO and contains the <code>field_count</code> received via the image information packet. |

**Table 995. STATUS**

| Name                     | Bit | Description                                                                                                                                                                                                                                                                                                                                                               |
|--------------------------|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Status                   | 0   | Read this register to determine the status of the frame writer. The IP sets the status bit if the write is currently writing a frame, and returns to 0 in between frames.                                                                                                                                                                                                 |
| Pending run-time control | 1   | Read this register to determine if the IP has any pending register writes to commit. This bit goes high when a write occurs to one of the control registers and returns low at the end of the current frame after a write to the <code>COMMIT</code> register. If the frame writer is idle, this bit goes low directly after a write to the <code>COMMIT</code> register. |

**Table 996. BUFFER\_AVAILABLE**

| Name             | Bit | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Buffer available | 0   | After you write a field, the IP sets this bit and sets the <code>BUFFER_START_ADDRESS</code> , <code>BUFFER_FIELD_WIDTH</code> , <code>BUFFER_FIELD_HEIGHT</code> and <code>BUFFER_F1_FLAG</code> registers.<br>Clear the <code>BUFFER_AVAILABLE</code> bit by writing to the <code>BUFFER_ACKNOWLEDGE</code> register. The IP does not update <code>BUFFER_START_ADDRESS</code> , <code>BUFFER_FIELD_WIDTH</code> , <code>BUFFER_FIELD_HEIGHT</code> and <code>BUFFER_F1_FLAG</code> registers until you clear <code>BUFFER_AVAILABLE</code> .<br>For a steady video input, software has the duration of one field (typically around 5-20 ms) to read the buffers and clear <code>BUFFER_AVAILABLE</code> before you lose any buffer information. |

**Table 997. BUFFER\_WRITE\_COUNT**

| Name        | Bits | Description                                                                                                                                                                                                                                                 |
|-------------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Write count | 31:0 | Increments with each buffer the IP writes and resets with a write to the <code>COMMIT</code> register.<br>If you set the <code>OVERWRITE_BROKEN</code> register, this counter does not increment for fields that the IP writes but subsequently overwrites. |

**Table 998. BUFFER\_START\_ADDRESS**

| Name          | Bit  | Description                                                                                                                                                  |
|---------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Start address | 31:0 | Provides the start address of the most recently written buffer.<br>The IP writes the first buffer to the address you specify with <code>BUFFER_BASE</code> . |

**Table 999. BUFFER\_F1\_FLAG**

| Name    | Bit | Description                                                                                                                                                                                                                                                                                                                       |
|---------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| F1 flag | 0   | If <b>Lite mode</b> is on, this bit provides the state of <code>TUSER[1]</code> for the most recently written buffer. The IP set this bit when it writes to interlaced F1 fields.<br>If <b>Lite mode</b> is off, the IP sets this to the binary AND of <code>IntlaceNibble</code> bits 3 and 2 from the image information packet. |

**Table 1000. BUFFER\_FIELD\_WIDTH**

| Name        | Bit  | Description                                                                                                                                                                                      |
|-------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Field width | 31:0 | If <b>Lite mode</b> is on, the IP returns the current value in <code>IMG_INFO_WIDTH</code> .<br>If <b>Lite mode</b> is off, the IP sets these bits to the width in the image information packet. |

**Table 1001. BUFFER\_FIELD\_HEIGHT**

| Name         | Bit  | Description                                                                                                                                                                                        |
|--------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Field height | 31:0 | If <b>Lite mode</b> is on, the IP returns the current value in <code>IMG_INFO_HEIGHT</code> .<br>If <b>Lite mode</b> is off, the IP sets these bits to the height in the image information packet. |

**Table 1002. FIELD\_COUNT**

| Name        | Bit  | Description                                                                                                                                                       |
|-------------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Field count | 15:0 | If <b>Lite mode</b> is on this returns 0.<br>If <b>Lite mode</b> is off, the IP takes the value from the end of field packet of the most recently written buffer. |

**Table 1003. COMMIT**

| Name   | Bit | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commit | 0   | Write any value to the <code>COMMIT</code> register to commit new RW register values and request an update to the new settings.<br>The frame buffer updates to the new RW register settings at the next start of field.<br>The IP starts operating according to the new settings when you write the current. The IP lowers the <code>pending_rtc</code> bit in the status register at this time.<br>The <code>BUFFER_ACKNOWLEDGE</code> register is the only register that does not need to be followed by a write to commit. |

**Table 1004. BUFFER\_ACKNOWLEDGE**

| Name        | Bit | Description                                                                                                                                                                                                                                   |
|-------------|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Acknowledge | 0   | Write to the acknowledge bit to reset the <code>BUFFER_AVAILABLE</code> register.<br>You don't need to follow a write to the <code>BUFFER_ACKNOWLEDGE</code> register with a write to the <code>COMMIT</code> register for it to take effect. |

**Table 1005. RUN**

| Name         | Bits | Description                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Run          | 0    | Set the run bit to start the IP writing fields.<br>Clear the run bit to stop the IP writing at the end of the current field.                                                                                                                                                                                                                                                                                                    |
| Single shot  | 1    | Set the single shot bit to configure single shot mode. The frame writer writes buffers until it reaches <code>NUM_BUFFERS</code> buffers and then switches to consume mode. The IP sets <code>BUFFER_AVAILABLE</code> register after the first field write completes. To perform another single-shot write, clear <code>BUFFER_AVAILABLE</code> by writing to <code>BUFFER_ACKNOWLEDGE</code> . <code>BUFFER_AVAILABLE</code> . |
| continued... |      |                                                                                                                                                                                                                                                                                                                                                                                                                                 |

|  |  |                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  |  | <p>If this bit is clear, the IP operates in continuous free-running mode and after it reaches <code>NUM_BUFFERS</code> buffers, writing continues, starting at <code>BUFFER_BASE</code> again.</p> <p>Use this mode with <code>NUM_BUFFERS</code> set to 1, or a small value.</p> <p>After any changes to this register write to the <code>COMMIT</code> register, which take effect at the next start of field</p> |
|--|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Table 1006. NUM\_BUFFERS**

| Name        | Bits | Description                                                                                                                                                                                                                                                                                                                                               |
|-------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Num_buffers | 23:0 | <p>The number of buffers. When all buffers are full, the frame writer wraps around and begins writing to <code>BUFFER_BASE</code> again unless you set the single shot bit of the <code>RUN</code> register is set.</p> <p>After any changes to this register write to the <code>COMMIT</code> register, which take effect at the next start of field</p> |

**Table 1007. BUFFER\_BASE**

| Name        | Bit  | Description                                                                                                                                                                                                                                                         |
|-------------|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Buffer_base | 31:0 | <p>Frame writing resets to the <code>buffer_base</code> address whenever you commit a write to the <code>RUN</code> register.</p> <p>After any changes to this register write to the <code>COMMIT</code> register, which take effect at the next start of field</p> |

**Table 1008. INTER\_BUFFER\_OFFSET**

| Name                | Bits | Description                                                                                                                                                                                                                                                                                                                                                                                |
|---------------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Inter-buffer offset | 31:0 | <p>Interbuffer address increment. To prevent fields overwriting one another in memory, you must set this so that the spacing between buffers is greater than the size of individual fields in memory.</p> <p>Interbuffer offset must be word aligned</p> <p>After any changes to this register write to the <code>COMMIT</code> register, which take effect at the next start of field</p> |

**Table 1009. INTER\_LINE\_OFFSET**

| Name              | Bits | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-------------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Inter-line offset | 31:0 | <p>Interline address increment.</p> <p>Usually set to the low byte of the next word address above<br/><i>bytes per pixel * IMAGE_WIDTH.</i></p> <p>To prevent lines overwriting one another in memory you must set this bit so it is equal or greater than the line width in memory.</p> <p>Interline offset must be word aligned.</p> <p>After any changes to this register write to the <code>COMMIT</code> register, which take effect at the next start of field</p> |

**Table 1010. IRQ\_LINE\_THRESHOLD**

| Name               | Bit  | Description                                                                                                                                                                                                    |
|--------------------|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IRQ line threshold | 31:0 | <p>Set the IRQ line threshold in combination with setting bit 1 of <code>IRQ_CONTROL</code> for the IP to raise an interrupt every time it writes line number <code>IRQ_LINE_THRESHOLD</code> for a field.</p> |

**Table 1011. OVERWRITE\_BROKEN**

| Name             | Bit | Description                                                                                                                                                                                                                                                                                                      |
|------------------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| overwrite_broken | 0   | <p>If you set the <code>overwrite_broken</code> bit, the IP overwrites fields received where the broken bit is set in the EOF, or where the height does not match the expected height. Also the IP does not set the <code>BUFFER_AVAILABLE</code> bit and other buffer information registers for that field.</p> |

## 49.5. Video Frame Writer IP Software API

The IP has a software driver for software control of the IP at run time. The IP does not fit any of the generic device models provided by the Nios II HAL. It exposes a set of dedicated accessors to the control and status registers. The driver structure inherits the base driver structure so all common methods described in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_vfw_regs.h`

Include file: `intel_vvp_vfw.h`

**Table 1012. Video Frame Writer IP Software API**

| Name                                                  | Description                                                                                                                                                                                 |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>intel_vvp_vfw_init</code>                       | Initialize the Frame Writer instance                                                                                                                                                        |
| <code>intel_vvp_core_*</code>                         | Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i> . Writable when <b>Lite</b> is on. Readable when <b>Lite</b> is off and <b>Debug features</b> is on. |
| <code>intel_vvp_vfw_get_lite_mode</code>              | Returns if <b>Lite mode</b> is on                                                                                                                                                           |
| <code>intel_vvp_vfw_get_debug_enabled</code>          | Returns if <b>Debug features</b> is on                                                                                                                                                      |
| <code>intel_vvp_vfw_get_max_height</code>             | Returns the maximum supported height                                                                                                                                                        |
| <code>intel_vvp_vfw_get_max_width</code>              | Returns the maximum supported width                                                                                                                                                         |
| <code>intel_vvp_vfw_get_bps</code>                    | Returns the number of bits per color sample                                                                                                                                                 |
| <code>intel_vvp_vfw_get_number_of_color_planes</code> | Returns the number of color planes per pixel                                                                                                                                                |
| <code>intel_vvp_vfw_get_pixels_in_parallel</code>     | Returns the number of pixels streamed in parallel                                                                                                                                           |
| <code>intel_vvp_vfw_get_mem_word_packing</code>       | Returns the algorithm used for pixels packing in memory                                                                                                                                     |
| <code>intel_vvp_vfw_is_running</code>                 | Returns if the IP is currently processing video data                                                                                                                                        |
| <code>intel_vvp_vfw_get_commit_status</code>          | Returns if there are uncommitted writes                                                                                                                                                     |
| <code>intel_vvp_vfw_get_status</code>                 | Reads the status register                                                                                                                                                                   |
| <code>intel_vvp_vfw_is_buffer_available</code>        | Returns if a field has been buffered to memory                                                                                                                                              |
| <code>intel_vvp_vfw_get_buffer_write_count</code>     | Returns the number of fields written to memory                                                                                                                                              |
| <code>intel_vvp_vfw_get_buffer_start_addr</code>      | Returns the base address in memory of the last buffer made available                                                                                                                        |
| <code>intel_vvp_vfw_is_buffer_fl</code>               | Returns whether the field in the last buffer made available is a F1 field                                                                                                                   |
| <code>intel_vvp_vfw_get_buffer_width</code>           | Returns the field width for the last buffer made available                                                                                                                                  |
| <code>intel_vvp_vfw_get_buffer_height</code>          | Returns the field height for the last buffer made available                                                                                                                                 |
| <code>intel_vvp_vfw_get_field_count</code>            | Returns the <code>field_count</code> field extracted from the last image info packet (full variant only)                                                                                    |
| <code>intel_vvp_vfw_acknowledge_buffer</code>         | Acknowledges a newly written buffer                                                                                                                                                         |
| <code>intel_vvp_vfw_set_run_mode</code>               | Changes the run mode                                                                                                                                                                        |
| <code>intel_vvp_vfw_get_run_mode</code>               | Returns the run mode                                                                                                                                                                        |
| continued...                                          |                                                                                                                                                                                             |

| Name                                      | Description                                            |
|-------------------------------------------|--------------------------------------------------------|
| intel_vvp_vfw_set_num_buffers             | Sets the number of buffers in use                      |
| intel_vvp_vfw_get_num_buffers             | Returns the number of buffers in use                   |
| intel_vvp_vfw_set_base_addr               | Sets the base address for the first buffer in memory   |
| intel_vvp_vfw_get_base_addr               | Returns the base address of the first buffer in memory |
| intel_vvp_vfw_set_inter_buffer_offset     | Sets the offset between field buffers in memory        |
| intel_vvp_vfw_get_inter_buffer_offset     | Returns the offset between field buffers in memory     |
| intel_vvp_vfw_set_inter_line_offset       | Sets the offset between lines in memory                |
| intel_vvp_vfw_get_inter_line_offset       | Returns the offset between lines in memory             |
| intel_vvp_vfw_set_irq_line_threshold      | Selects the line threshold used to fire the interrupt  |
| intel_vvp_vfw_get_irq_line_threshold      | Returns the line threshold used to fire the interrupt  |
| intel_vvp_vfw_overwrite_broken_fields     | Selects the overwrite behavior for broken fields       |
| intel_vvp_vfw_get_overwrite_broken_fields | Returns if broken fields are overwritten               |
| intel_vvp_vfw_commit_writes               | Commit all outstanding writes                          |

#### enum eIntelVvpVfwPacking

|                    |                                                                                                                                                                                                                                                                                                                                                                                          |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Members</b>     | <p>kIntelVvpVfwPerfectPacking (0) - Perfect sample packing (no bits wasted)</p> <p>kIntelVvpVfwColorPacking (1) - Color sample packing (color samples are not split across memory words)</p> <p>kIntelVvpVfwPixelPacking (2) - Pixel packing (pixels are not split across memory words)</p> <p>kIntelVvpVfwInvalidPacking (-1) - Invalid packing algorithm (used to indicate errors)</p> |
| <b>Description</b> | Enumeration for the configurable sample packing in memory algorithms                                                                                                                                                                                                                                                                                                                     |

#### enum eIntelVvpVfwRunMode

|                    |                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Members</b>     | <p>kIntelVvpVfwStop (0) - Frame Writer is stopped</p> <p>kIntelVvpVfwFreeRunning (1) - Frame Writer proceeds in a round-robin scheme using all available buffers</p> <p>kIntelVvpVfwSingleShot (3) - Frame Writer proceeds and stops once all buffers have been written once</p> <p>kIntelVvpVfwInvalidRunMode (-1) - Invalid value (used to indicate errors)</p> |
| <b>Description</b> | Enumeration for the various write methods                                                                                                                                                                                                                                                                                                                         |

### intel\_vvp\_vfw\_init

**Prototype**

```
int intel_vvp_vfw_init(intel_vvp_vfw_instance *instance,
intel_vvp_core_base base);
```

**Description**

Initializes a Frame Writer instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

**Arguments**

instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

base - base address of the register map

**Returns**

kIntelVvpCoreOk (0) in case of success, a negative error code in case of error

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7).

kIntelVvpCorePidErr if the product\_id does not match with the Frame Writer product ID

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpFrameWriterRegMapVersionErr if the register map is not supported

### intel\_vvp\_vfw\_get\_lite\_mode

**Prototype**

```
bool intel_vvp_vfw_get_lite_mode(intel_vvp_vfw_instance
*instance);
```

**Description**

Returns the value of the LITE\_MODE register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized

**Arguments**

instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

**Returns**

true if the IP is parameterized in lite mode

### intel\_vvp\_vfw\_get\_debug\_enabled

**Prototype**

```
bool intel_vvp_vfw_get_debug_enabled(intel_vvp_vfw_instance
*instance);
```

**Description**

Returns the value of the DEBUG\_ENABLED register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized.



*Arguments* instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

*Returns* true if the IP is parameterized with debug features enabled

#### intel\_vvp\_vfw\_get\_max\_height

*Prototype*

```
uint32_t intel_vvp_vfw_get_max_height(intel_vvp_vfw_instance *instance);
```

*Description* Returns the value of the MAX\_HEIGHT register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

*Returns* The maximum supported height set at configuration

#### intel\_vvp\_vfw\_get\_max\_width

*Prototype*

```
uint32_t intel_vvp_vfw_get_max_width(intel_vvp_vfw_instance *instance);
```

*Description* Returns the value of the MAX\_WIDTH register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

*Returns* The maximum supported width set at configuration

#### intel\_vvp\_vfw\_get\_bps

*Prototype*

```
uint8_t intel_vvp_vfw_get_bps(intel_vvp_vfw_instance *instance);
```

*Description* Returns the value of the BPS register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

*Returns* the number of bits per color sample selected at configuration

#### intel\_vvp\_vfw\_get\_number\_of\_color\_planes

*Prototype*

```
uint8_t
intel_vvp_vfw_get_number_of_color_planes(intel_vvp_vfw_instance *instance);
```

**Description** Returns the value of the NUMBER\_OF\_COLOR\_PLANES register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

**Returns** the number of color planes per pixel selected at configuration

#### intel\_vvp\_vfw\_get\_pixels\_in\_parallel

##### Prototype

```
uint8_t
intel_vvp_vfw_get_pixels_in_parallel(intel_vvp_vfw_instance
*instance);
```

**Description** Returns the value of the PIXELS\_IN\_PARALLEL register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

**Returns** the number of pixels streamed in parallel selected at configuration

#### intel\_vvp\_vfw\_get\_mem\_word\_packing

##### Prototype

```
eIntelVvpVfwPacking
intel_vvp_vfw_get_mem_word_packing(intel_vvp_vfw_instance
*instance);
```

**Description** Returns the value of the WORD\_PACKING register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

**Returns** the algorithm used to pack/unpack pixels and color samples of a field in memory

#### intel\_vvp\_vfw\_is\_running

##### Prototype

```
bool intel_vvp_vfw_is_running(intel_vvp_vfw_instance*
instance);
```

**Description** Reads and returns the running bit of the STATUS register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

**Returns** True is the IP is currently reading a field, false when between two output fields

### intel\_vvp\_vfw\_get\_commit\_status

**Prototype**

```
bool intel_vvp_vfw_get_commit_status(intel_vvp_vfw_instance* instance);
```

**Description**

Reads and returns the commit status bit of the STATUS register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized.

**Arguments**

instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

**Returns**

True if there are uncommitted writes

### intel\_vvp\_vfw\_get\_status

**Prototype**

```
uint32_t intel_vvp_vfw_get_status(intel_vvp_vfw_instance* instance);
```

**Description**

Reads the STATUS register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized.

**Arguments**

instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

**Returns**

The content of the STATUS register

### intel\_vvp\_vfw\_is\_buffer\_available

**Prototype**

```
bool intel_vvp_vfw_is_buffer_available(intel_vvp_vfw_instance* instance);
```

**Description**

Reads the BUFFER\_AVAILABLE register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized.

**Arguments**

instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

**Returns**

True if a new buffer has been written to memory since the last call to intel\_vvp\_vfw\_acknowledge\_buffer

### intel\_vvp\_vfw\_get\_buffer\_write\_count

**Prototype**

```
uint32_t intel_vvp_vfw_get_buffer_write_count(intel_vvp_vfw_instance* instance);
```

**Description**

Reads the BUFFER\_WRITE\_COUNT register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

**Returns** The number of buffers written to memory that were not discarded

#### intel\_vvp\_vfw\_get\_buffer\_start\_addr

**Prototype**

```
uint32_t
intel_vvp_vfw_get_buffer_start_addr(intel_vvp_vfw_instance*
instance);
```

**Description** Reads the BUFFER\_START\_ADDRESS register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

**Returns** The memory address for the last field flagged as available

#### intel\_vvp\_vfw\_is\_buffer\_f1

**Prototype**

```
bool intel_vvp_vfw_is_buffer_f1(intel_vvp_vfw_instance*
instance);
```

**Description** Reads and returns the BUFFER\_F1\_FLAG register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

**Returns** True if the last buffer made available is a F1 field

#### intel\_vvp\_vfw\_get\_buffer\_width

**Prototype**

```
uint32_t
intel_vvp_vfw_get_buffer_width(intel_vvp_vfw_instance*
instance);
```

**Description** Reads and returns the BUFFER\_WIDTH register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

**Returns** The width of the last buffer made available

### intel\_vvp\_vfw\_get\_buffer\_height

*Prototype*

```
uint32_t
intel_vvp_vfw_get_buffer_height(intel_vvp_vfw_instance*
instance);
```

*Description* Reads and returns the BUFFER\_HEIGHT register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

*Returns* The height of the last buffer made available

### intel\_vvp\_vfw\_get\_field\_count

*Prototype*

```
uint32_t
intel_vvp_vfw_get_field_count(intel_vvp_vfw_instance*
instance);
```

*Description* Reads and returns the FIELD\_COUNT register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized and configured for full variant.

*Arguments* instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

*Returns* T

### intel\_vvp\_vfw\_acknowledge\_buffer

*Prototype*

```
int intel_vvp_vfw_acknowledge_buffer(intel_vvp_vfw_instance
*instance);
```

*Description* Writes to the BUFFER\_ACKNOWLEDGE register, freeing space for future buffers

*Arguments* instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

*Returns* kIntelVvpCoreOk for success, negative error code in case of failures  
kIntelVvpCoreInstanceErr if the instance is invalid

### intel\_vvp\_vfw\_set\_run\_mode

*Prototype*

```
int intel_vvp_vfw_set_run_mode(intel_vvp_vfw_instance
*instance, eIntelVvpVfwRunMode run_mode);
```

**Description** Writes a new value to the RUN\_MODE register and chooses the write process used while storing buffers in memory

**Arguments**

- instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure
- run\_mode - the new run\_mode
- kIntelVvpVfwStop to stop the frame writer until another run-mode is selected
- kIntelVvpVfwFreeRunning to store incoming fields using all buffers in round-robin fashion
- kIntelVvpVfwSingleShot to store incoming fields until all buffers have been written once

**Returns**

- kIntelVvpCoreOk for success, negative error code in case of failures
- kIntelVvpCoreInstanceErr if the instance is invalid

#### intel\_vvp\_vfw\_get\_run\_mode

##### Prototype

```
eIntelVvpVfwRunMode
intel_vvp_vfw_get_run_mode(intel_vvp_vfw_instance* instance);
```

**Description** Reads and returns the current value of the RUN\_MODE register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized and configured to enable debug.

**Arguments**

- instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

**Returns** Last value written to the RUN\_MODE register if the IP is configured to enable register read-back

#### intel\_vvp\_vfw\_set\_num\_buffers

##### Prototype

```
int intel_vvp_vfw_set_num_buffers(intel_vvp_vfw_instance
*instance, uint32_t num_buffers);
```

**Description** Writes a new value to the NUM\_BUFFERS register and selects the number of buffers

**Arguments**

- instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure
- num\_buffers - new number of buffers

**Returns**

- kIntelVvpCoreOk for success, negative error code in case of failures
- kIntelVvpCoreInstanceErr if the instance is invalid

### intel\_vvp\_vfw\_get\_num\_buffers

*Prototype*

```
uint32_t
intel_vvp_vfw_get_num_buffers(intel_vvp_vfw_instance*
instance);
```

*Description*

Reads and returns the current value of the NUM\_BUFFERS register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized and configured to enable debug.

*Arguments*

instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

*Returns*

Last value written to the NUM\_BUFFERS register if the IP is configured to enable register read-back

### intel\_vvp\_vfw\_set\_base\_addr

*Prototype*

```
int intel_vvp_vfw_set_base_addr(intel_vvp_vfw_instance
*instance, uint32_t base_addr);
```

*Description*

Writes a new value to the BUFFER\_BASE register and selects a new base address for buffers in memory

*Arguments*

instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

base\_addr - the base address for the first buffer

*Returns*

kIntelVvpCoreOk for success, negative error code in case of failures  
kIntelVvpCoreInstanceErr if the instance is invalid

### intel\_vvp\_vfw\_get\_base\_addr

*Prototype*

```
uint32_t intel_vvp_vfw_get_base_addr(intel_vvp_vfw_instance*
instance);
```

*Description*

Reads and returns the current value of the BUFFER\_BASE register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized and configured to enable debug.

*Arguments*

instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

*Returns*

Last value written to the BUFFER\_BASE register if the IP is configured to enable register read-back

### intel\_vvp\_vfw\_set\_inter\_buffer\_offset

**Prototype**

```
int
intel_vvp_vfw_set_inter_buffer_offset(intel_vvp_vfw_instance
*instance, uint32_t inter_buffer_offset);
```

**Description** Writes a new value to the INTER\_BUFFER\_OFFSET register and selects a new inter-buffer offset for the buffers

**Arguments** instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

inter\_buffer\_offset - the offset, in bytes, between two buffers, or, in other words, the number of bytes allocated to each buffer

**Returns** kIntelVvpCoreOk for success, negative error code in case of failures

kIntelVvpCoreInstanceErr if the instance is invalid

### intel\_vvp\_vfw\_get\_inter\_buffer\_offset

**Prototype**

```
uint32_t
intel_vvp_vfw_get_inter_buffer_offset(intel_vvp_vfw_instance*
instance);
```

**Description** Reads and returns the current value of the INTER\_BUFFER\_OFFSET register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized and configured to enable debug.

**Arguments** instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

**Returns** Last value written to the INTER\_BUFFER\_OFFSET register if the IP is configured to enable register read-back

### intel\_vvp\_vfw\_set\_inter\_line\_offset

**Prototype**

```
int
intel_vvp_vfw_set_inter_line_offset(intel_vvp_vfw_instance
*instance, uint32_t inter_line_offset);
```

**Description** Writes a new value to the INTER\_LINE\_OFFSET register and selects a new inter-line offset for the buffers

**Arguments** instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

inter\_line\_offset - the offset, in bytes, between consecutive lines

**Returns** kIntelVvpCoreOk for success, negative error code in case of failures

kIntelVvpCoreInstanceErr if the instance is invalid



### intel\_vvp\_vfw\_get\_inter\_line\_offset

**Prototype**

```
uint32_t
intel_vvp_vfw_get_inter_line_offset(intel_vvp_vfw_instance*
instance);
```

**Description** Reads and returns the current value of the INTER\_LINE\_OFFSET register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized and configured to enable debug.

**Arguments** instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

**Returns** Last value written to the INTER\_LINE\_OFFSET register if the IP is configured to enable register read-back

### intel\_vvp\_vfw\_set\_irq\_line\_threshold

**Prototype**

```
int
intel_vvp_vfw_set_irq_line_threshold(intel_vvp_vfw_instance
*instance, uint32_t line_threshold);
```

**Description** Writes a new value to the IRQ\_LINE\_THRESHOLD register and selects a new threshold to fire the interrupt

**Arguments** instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

line\_threshold - the new line threshold

**Returns** kIntelVvpCoreOk for success, negative error code in case of failures  
kIntelVvpCoreInstanceErr if the instance is invalid

### intel\_vvp\_vfw\_get\_irq\_line\_threshold

**Prototype**

```
uint32_t
intel_vvp_vfw_get_irq_line_threshold(intel_vvp_vfw_instance*
instance);
```

**Description** Reads and returns the current value of the IRQ\_LINE\_THRESHOLD register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized and configured to enable debug.

**Arguments** instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

**Returns** Last value written to the IRQ\_LINE\_THRESHOLD register for the selected buffer set if the IP is configured to enable register read-back

### intel\_vvp\_vfw\_overwrite\_broken\_fields

*Prototype*

```
int
intel_vvp_vfw_overwrite_broken_fields(intel_vvp_vfw_instance
*instance, bool overwrite_broken);
```

*Description* Writes a new value to the OVERWRITE\_BROKEN register and selects how the Frame Writer handles broken fields

*Arguments* instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

overwrite\_broken - True to ignore and overwrite broken fields, false otherwise

*Returns* kIntelVvpCoreOk for success, negative error code in case of failures

kIntelVvpCoreInstanceErr if the instance is invalid

### intel\_vvp\_vfw\_get\_overwrite\_broken\_fields

*Prototype*

```
bool
intel_vvp_vfw_get_overwrite_broken_fields(intel_vvp_vfw_instance*
instance);
```

*Description* Reads and returns the current value of the OVERWRITE\_BROKEN register. The instance must be a valid intel\_vvp\_vfw\_instance fully initialized and configured to enable debug.

*Arguments* instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

*Returns* Last value written to the OVERWRITE\_BROKEN register if the IP is configured to enable register read-back

### intel\_vvp\_vfw\_commit\_writes

*Prototype*

```
int intel_vvp_vfw_commit_writes(intel_vvp_vfw_instance*
instance)
```

*Description* Commits all pending writes before processing the next field. The instance must be valid.

*Arguments* instance - pointer to the intel\_vvp\_vfw\_instance software driver instance structure

*Returns* kIntelVvpCoreOk (0) for success, negative error code otherwise

## 50. Video Streaming FIFO IP

### 50.1. About the Video Streaming FIFO IP

The IP provides a FIFO buffer storage solution with input and output interfaces compliant with the Altera Streaming Video Protocol.

The IP supports full and lite variants without selecting a parameter. The depth of the FIFO buffer must be a whole power of two. The depth parameter sets the number of beats of data that the FIFO holds. When full, the IP deasserts the input tready signal to hold the input stream and ensure no loss of data. When empty, the IP deasserts the output tvalid signal.

The IP supports both single and dual clocks. With a single clock both the input and output interfaces are on the same clock domain. With dual clocks the input and output interfaces are on different clock domains. Dual clocks allow the IP to be a clock domain crossing bridge compliant to the Altera Streaming Video Protocol.

The IP delivers high operating clock frequencies (in excess of 600 MHz on Agilex 7 family devices), but with relatively high latency around 5-10 clock cycles with a single clock and 10-20 cycles with dual clocks.

#### 50.1.1. Video Streaming FIFO IP Performance and Resources

Altera provides resource and utilization data for guidance. The IP resource utilization depends on the device family and the number of supported bits per color sample, colors per pixel, and pixels in parallel.

The table shows ALM usage and  $f_{MAX}$  for a design with 2 pixels in parallel, 8 bits per color sample, 3 color planes, 128 depth, and a single clock

| Target device                                  | ALMs | M20k | $f_{MAX}$ (MHz) |
|------------------------------------------------|------|------|-----------------|
| Agilex 7 speed grade 2 (AGFA012R24A2E2V)       | 268  | 3    | 781             |
| Arria 10 speed grade 1 (10AS066H1F34E1HG)      | 205  | 3    | 645             |
| Cyclone 10 GX speed grade 5 (10CX220YF78015G)  | 206  | 3    | 599             |
| Stratix 10 GX speed grade 1 (1SX280LN2F43E1VG) | 271  | 3    | 833             |

### 50.2. Video Streaming FIFO IP Parameters

Table 1013. Video Streaming FIFO IP Parameters

| Parameter                    | Value                        | Description                                                           |
|------------------------------|------------------------------|-----------------------------------------------------------------------|
| Bits per color sample        | 8 to 16                      | Select the number of bits per color sample.                           |
| Number of color planes       | 1 to 4                       | Select the number of color planes per pixel.                          |
| Number of pixels in parallel | 1 to 8                       | Select the number of pixels in parallel.                              |
| Dual clock                   | On or off                    | Turn on to operate the FIFO buffer with dual clocks.                  |
| FIFO depth                   | 32, 64, 128, 256, 1024, 2048 | Specify the depth of the FIFO buffer in words (beats of data).        |
| Pipeline ready signals       | On or off                    | Turn on to add extra pipeline registers to the AXI4-S tready signals. |

## 50.3. Video Streaming FIFO IP Interfaces

Table 1014. Video Streaming FIFO IP Interfaces

| Name                                     | Direction        | Width        | Description                                                                                                                                |
|------------------------------------------|------------------|--------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Clocks and resets</b>                 |                  |              |                                                                                                                                            |
| main_clock_clk                           | In               | 1            | AXI4-S processing clock. Only when you turn off <b>Dual clock</b> .                                                                        |
| main_reset_rst                           | In               | 1            | AXI4-S processing reset. Only when you turn off <b>Dual clock</b> .                                                                        |
| in_clock_clk                             | In               | 1            | AXI4-S processing clock for the input interface domain. Only when you turn on <b>Dual clock</b> .                                          |
| in_reset_rst                             | In               | 1            | AXI4-S processing reset for the input interface domain. Only when you turn on <b>Dual clock</b> .                                          |
| out_clock_clk                            | In               | 1            | AXI4-S processing clock for the output interface domain. Only when you turn on <b>Dual clock</b> .                                         |
| out_reset_rst                            | In               | 1            | AXI4-S processing reset for the output interface domain. Only when you turn on <b>Dual clock</b> .                                         |
| <b>Altera Streaming Video interfaces</b> |                  |              |                                                                                                                                            |
| <b>Port name</b>                         | <b>Direction</b> | <b>Width</b> | <b>Description</b>                                                                                                                         |
| axi4s_vid_in_tdata                       | In               |              | AXI4S data in.                                                                                                                             |
| axi4s_vid_in_tvalid                      | In               | 1            | AXI4-S data valid.                                                                                                                         |
| axi4s_vid_in_tuser                       | In               |              | AXI4-S tuser<br>tuser[0] indicates start of video frame when asserted<br>tuser[1] indicates the start of a non-video packet when asserted. |
| axi4s_vid_in_tlast                       | In               | 1            | AXI4-S end of packet.                                                                                                                      |
| axi4s_vid_in_tready                      | Out              | 1            | AXI4-S data ready.                                                                                                                         |
| <i>continued...</i>                      |                  |              |                                                                                                                                            |

(149) The equation gives all tdata widths in these interfaces:  

$$\max(\text{ceil}((\text{bits per color sample} \times \text{number of color planes} \times \text{pixels in parallel}) / 8) \times 8, 16)$$

(150) The equation gives all tuser widths in these interfaces:  

$$N = \text{ceil}(\text{tdata width} / 8)$$

| Name                 | Direction | Width | Description                                                                                                                                              |
|----------------------|-----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| axi4s_vid_out_tdata  | Out       | (149) | AXI4-S data in.                                                                                                                                          |
| axi4s_vid_out_tvalid | Out       | 1     | AXI4-S data valid.                                                                                                                                       |
| axi4s_vid_in_tuser   | Out       | (150) | AXI4-S tuser<br><i>tuser[0]</i> indicates start of video frame when asserted<br><i>tuser[1]</i> indicates the start of a non-video packet when asserted. |
| axi4s_vid_out_tlast  | Out       | 1     | AXI4-S end of packet.                                                                                                                                    |
| axi4s_vid_out_tready | In        | 1     | AXI4-S data ready.                                                                                                                                       |

### Related Information

- [Video and Vision Processing IP Interfaces](#) on page 30
- [Video and Vision Processing IP Interfaces](#) on page 30



## 51. Video Timing Generator IP

---

### 51.1. About the Video Timing Generator IP

The Video Timing Generator IP provides the real-time signals that define a video raster. The IP can generate any raster, including both interlaced and progressive standards. You configure the IP with a single fixed standard at build time. You can reconfigure dynamically at run-time using the optional processor interface.

#### Pixels in Parallel Support

The IP can support any number of pixels in parallel from 1 to 8. The IP has no restriction on raster dimensions versus the number of pixels in parallel. The raster width does not have to be an integer multiple of pixels in parallel.

The full-raster output interface contains multiple control words, one per pixel in parallel. The IP can set individual control words, thus allowing any raster width to be correctly output.

The Legacy Clocked Video Output contains multiple F, V, and H, one per pixel in parallel. The IP can set individual F, V, and H, thus allowing any raster width to be correctly output.

The IP can accept timing parameters that are integer multiples of the pixels in parallel, which reduces the gate count of the synthesized IP.

#### Hard Frame Lock

The hard frame lock allows you to synchronize the output raster to an external timing reference, which you require to genlock the output.

The IP can receive either a timing pulse, or a timing toggle. When the IP detects external timing signal (rising edge only of a pulse, both edges of a toggle) the IP restarts the output raster at your specified pixel location.

The external timing signal might be in a clock domain different to the IP. The period of the external timing signal might contain jitter. The IP places a jitter window, which you specify as a number of clock cycles, around the point when the external timing signal is expected. During this jitter window the IP does not restart the raster.

Diagnostics via processor interface indicate if the raster reset occurs.

Each restart causes a discontinuity in the output timing signals and causes interconnect protocols such as SDI to become invalid for one frame. If the period of the external timing reference and the period of the generated raster do not align, the IP constantly resets and you see no output.

### Soft Frame Lock

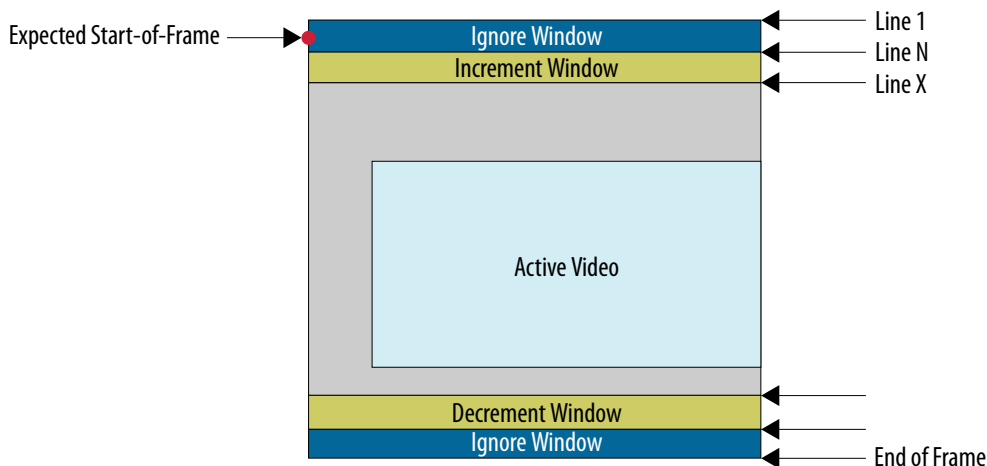
Soft frame lock allows the output raster to track an external timing reference, but the output clock is not locked to the pixel rate. For example, V-by-One interconnect does not require a pixel-accurate clock.

The IP achieves a soft frame lock by automatically adjusting the total height of the raster at run time.

You specify four regions of the raster where the occurrence of the external timing reference affects the height of the raster:

- Ignore region. A small number of lines before and after the expected timing reference where the total height is not changed.
- Increment region. A small number of lines after the ignore region where the IP increases the total height of the raster by one whole line, if the external timing reference occurs.
- Decrement region. A small number of lines before the ignore region where the IP decreases the total height of the raster by one whole line, if the external timing reference occurs.
- Hard frame lock. The remainder of the raster not included in the ignore, increment, or decrement regions. If the external timing reference occurs in this region, the IP restarts the raster at your specified restart pixel location.

**Figure 133. Soft Frame Lock Regions**



### Variable Refresh Rate

When you turn on variable refresh rate:

- The raster stops at the end of the frame and waits (possibly indefinitely) for the external timing reference. When the IP sees the external timing reference, the IP produces one whole frame, starting with pixel (0,0).
- The IP continues to generate whole lines of raster after the main raster completes. When the IP sees the external timing reference, the whole current line completes, and then the IP produces a pixel (0,0).
- You must turn off both hard frame lock and soft frame lock, because these parameters conflict. The resulting behavior, when the IP detects the external timing reference, is undefined

### General-Purpose Pulses and Toggles

The IP can produce up to 8 additional outputs that each provide a pulse or a toggle once per frame.

You can program these general-purpose signals to occur at any fixed point in the raster. Separate parameters specify the first and last pixel of the pulse. If the start and end pixels are the same, the IP generates a single clock pulse. If the end pixel is outside the defined raster, the signal becomes a once-per-frame toggle.

#### 51.1.1. Video Timing Generator IP Features

- Altera Streaming Video full-raster compliant video output interface
- Optional `TREADY` conduit on the streaming full-raster interface
- Legacy clocked video output interface
- Single-fixed standard without the overhead of a processor interface.
- Hard frame lock.
- Soft frame lock.
- Variable refresh rate
- 6-bits to 16-bits per color component
- 1, 2, 3 or 4 color planes per pixel
- 1 to 8 pixels in parallel
- Up to 8 general-purpose pulses or toggles aligned with the raster.

#### 51.1.2. Video Timing Generator IP Performance and Resources

Altera provides resource and utilization data for guidance. Video Timing Generator IP resource usage depends on the device family and number of supported bits per color sample, pixels in parallel, and number of color planes.



**Table 1015. Video Timing Generator IP Performance and Resources**

The table shows ALM usage and  $f_{MAX}$  for an example parameterization of 10 bps, 3 color planes, 2 pixels in parallel and runtime control.

| Target Device                   | ALMs  | M20Ks | $f_{MAX}$ (MHz) |
|---------------------------------|-------|-------|-----------------|
| Agilex 7 (AGFA012R24A2E2V)      | 1,774 | 0     | 675             |
| Arria 10 (10AS066H1F34E1HG)     | 1,532 | 0     | 520             |
| Cyclone 10 GX (10CX220YF672E5G) | 1,531 | 0     | 448             |
| Stratix 10 (1SX280LN2F43E1VG)   | 1,846 | 0     | 540             |

## 51.2. Video Timing Generator IP Parameters

The IP offers compile-time parameters. The IP GUI separates the compile-time parameters into four tabs. After you build the IP, you cannot change the build parameters. However, you can change all other parameters using the processor interface.

These parameters are fixed at build time and can only be changed by recompiling the IP.

**Table 1016. Build Parameters**

|                                                   | Allowed Range        | Description                                                                                                                                                                                |
|---------------------------------------------------|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Video Settings</b>                             |                      |                                                                                                                                                                                            |
| <b>Number of pixels in parallel</b>               | 1 to 8               | Select the number of pixels in parallel                                                                                                                                                    |
| <b>Number of color planes</b>                     | 1 to 4               | Select the number of color planes per pixel                                                                                                                                                |
| <b>Bits per color sample</b>                      | 6 to 16              | Select the number of bits per color sample                                                                                                                                                 |
| <b>Output Type</b>                                | AXI4-S FR or CV      | Select the type of output bus.                                                                                                                                                             |
| <b>AXI4-S FR interface TREADY<sup>(151)</sup></b> | True or False        | Select <b>True</b> to include the $tReady$ signal in the full-raster interface<br>Select <b>False</b> to remove the $tReady$ signal                                                        |
| <b>CV Timing Signals<sup>(152)</sup></b>          | Sync, Blank, or Both | Select which timing signals are included on the output CV bus.                                                                                                                             |
| <b>Control Settings</b>                           |                      |                                                                                                                                                                                            |
| <b>Memory-Mapped Control Interface</b>            | True or False        | Select <b>True</b> to enable the processor interface and associated signals.<br>When <b>False</b> , the IP removes the processor interface, and all processor registers use default values |
| <b>Frequency of CPU Clock<sup>(153)</sup></b>     | 1 to 1000000000      | The frequency, in Hz, of the processor clock.                                                                                                                                              |
| <b>General Settings</b>                           |                      |                                                                                                                                                                                            |
| <i>continued...</i>                               |                      |                                                                                                                                                                                            |

<sup>(151)</sup> This parameter is only available when you select **AXI4-S FR** for **Output Type**

<sup>(152)</sup> This parameter is only available when you select **CV** for **Output Type**

<sup>(153)</sup> This parameter is only available when you select **True** for **Memory-Mapped Control Interface**

|                                      | Allowed Range           | Description                                                                                                                                                                                                                    |
|--------------------------------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Timing Word Alignment</b>         | Any or PIP-Aligned Only | Select <b>Any</b> and there is no restriction on raster dimensions versus the number of pixels in parallel.<br>When <b>PIP-Aligned Only</b> , all timing parameters must be integer multiples of the pixels in parallel value. |
| <b>Hard frame lock support</b>       | True or False           | Select <b>True</b> to turn on hard frame lock.<br>When <b>False</b> , the IP removes all frame lock support.                                                                                                                   |
| <b>Soft frame lock support</b>       | True or False           | Select <b>True</b> to enable soft frame lock support. <sup>(154)</sup><br>When <b>False</b> , the IP removes soft frame lock support.                                                                                          |
| <b>Variable refresh rate support</b> | True or False           | Select <b>True</b> to enable variable refresh rate support<br>When <b>False</b> , the IP removes support for variable refresh rate                                                                                             |
| <b>Horizontal counter bits</b>       | 4 to 16                 | The number of binary bits required to represent the maximum width of raster.<br>For example, for a 4096 wide, set to 13.                                                                                                       |
| <b>Vertical counter bits</b>         | 4 to 16                 | The number of binary bits required to represent the maximum height of raster.<br>For example, for a 2048 high raster, set to 12.                                                                                               |
| <b>Number of pulses</b>              | 0 to 8                  | The number of additional general-purpose pulses that the IP can produce.<br>Each additional pulse increases the gate count of this IP.<br>If 0, the <b>Default Pulse Configuration</b> GUI is off.                             |

Figure 134. Build Parameters GUI

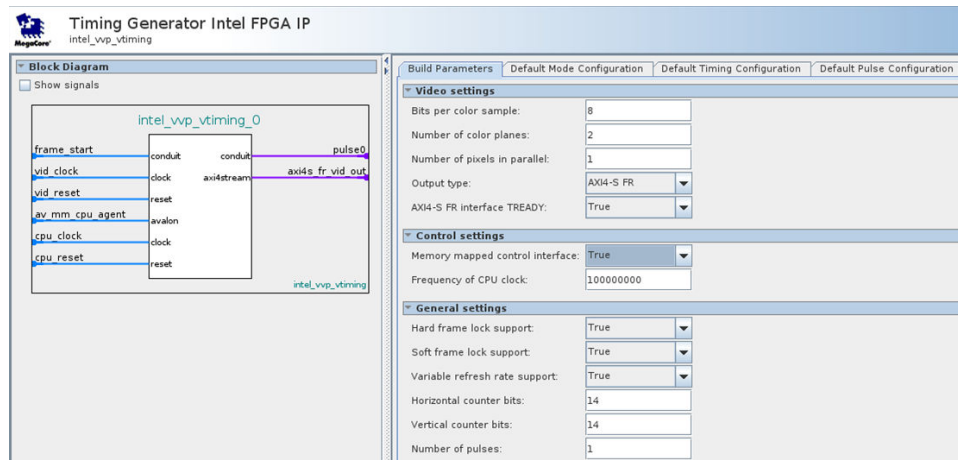


Table 1017. Default Mode Configuration

The Default Mode Configuration parameters apply to the IP when the IP is reset. You can change them during run time using the processor Interface, without recompiling the IP.

|                      | Allowed Range | Description                                                                                                                                                                                                                     |
|----------------------|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Blanking Type</b> | Blank or Sync | Set the style of h and v timing signals on the output.<br>In full-raster variants, the SYNC MODE bit of the full-raster control word is set to '1' to indicate <b>blank</b> style timing and '0' to indicate <b>sync</b> style. |
| continued...         |               |                                                                                                                                                                                                                                 |

(154) Hard Frame Lock must also be **True** for correct operation of soft frame lock

|                                                        | Allowed Range                                      | Description                                                                                                                                                                                                                                                                                                                                         |
|--------------------------------------------------------|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                        |                                                    | If <b>blank</b> style is selected for a clocked video interface, the IP drives only the hBlank and vBlank signals.<br>If <b>sync</b> style is selected for a clocked video interface, the IP drives only the clocked video hSync and CV vSync signals.                                                                                              |
| <b>Black Value, color plane 0</b>                      | 0 to 65535                                         | The initial value of "black" for this color plane                                                                                                                                                                                                                                                                                                   |
| <b>Black Value, color plane 1</b> <sup>(155)</sup>     | 0 to 65535                                         | The initial value of "black" for this color plane                                                                                                                                                                                                                                                                                                   |
| <b>Black Value, color plane 2</b> <sup>(155)</sup>     | 0 to 65535                                         | The initial value of "black" for this color plane                                                                                                                                                                                                                                                                                                   |
| <b>Black Value, color plane 3</b> <sup>(155)</sup>     | 0 to 65535                                         | The initial value of "black" for this color plane                                                                                                                                                                                                                                                                                                   |
| <b>Frame lock mode</b>                                 | Freerunning<br>Frame Lock<br>Variable Refresh Rate | The initial style of frame lock. For <b>Frame Lock</b> and <b>Variable Refresh Rate</b> , you must turn on the appropriate Build Parameters.                                                                                                                                                                                                        |
| <b>Soft frame lock</b> <sup>(156)</sup>                | True or False                                      | Select <b>True</b> , for a soft frame lock. Select <b>False</b> for the hard frame lock.                                                                                                                                                                                                                                                            |
| <b>Frame start signal type</b> <sup>(157)</sup>        | Pulse or Toggle                                    | If you select <b>Pulse</b> , the IP processes the frame start input signal as a pulse. The IP uses the rising edge of the signal to indicate the start of a frame.<br>If you select <b>Toggle</b> , the IP processes the frame start input signal as a toggle and the IP uses both edges of the signal to indicate the start of a frame.            |
| <b>Frame start max allowed jitter</b> <sup>(158)</sup> | 0 to 127                                           | The number of video clock cycles either side of the position of the expected start of frame where an occurrence of the frame start input signal does not cause the raster to restart.<br>If the frame start input signal occurs more than this number of video clock cycles from the point where it is expected, the IP restarts the output raster. |
| <b>Soft lock frame start ignore</b>                    | 0 to 127                                           | When you select <b>True</b> for <b>Soft frame lock</b> , this parameter specifies the number of lines where the frame start input signal is ignored.<br>Unused when you turn off soft lock.                                                                                                                                                         |
| <b>Soft lock frame start adjust</b> <sup>(159)</sup>   | 0 to 127                                           | When you select <b>True</b> for <b>Soft frame lock</b> , this parameter specifies the total number of lines used for soft lock, including the ignore lines.<br>Unused when you select <b>false</b> for <b>Soft frame lock</b> .                                                                                                                     |
| <b>VRR Line Mode</b> <sup>(160)</sup>                  | True or False                                      | If you select <b>Variable Refresh Rate</b> for <b>Frame Lock Mode</b> and select <b>True</b> for this parameter, the IP continues to produce whole lines of blanking after it produces the active pixels until it detects the frame start input signal is detected.                                                                                 |
| <i>continued...</i>                                    |                                                    |                                                                                                                                                                                                                                                                                                                                                     |

<sup>(155)</sup> This parameter is only available if the color plane exists. Refer to the **Build Parameter** tab **Number of color planes**

<sup>(156)</sup> This parameter is only available if you select **Frame lock** for **Frame Lock Mode**

<sup>(157)</sup> This parameter is only available if you select **Frame lock** or **Variable refresh rate** for **Frame Lock Mode**

<sup>(158)</sup> This parameter is only available if you select **Frame lock** for **Frame Lock Mode**

<sup>(159)</sup> This parameter is only available if you select **True** for **Soft Frame Lock**

<sup>(160)</sup> This parameter is only available if you select **Variable refresh rate** for **Frame Lock Mode**

|  | Allowed Range | Description                                                                                                                                                                                                                                                                                                                                                         |
|--|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  |               | <p>If you select <b>Variable Refresh Rate</b> for <b>Frame Lock Mode</b> and select <b>False</b> for this parameter, the IP stops producing a raster when the last active line of a frame completes. The IP resumes output when it detects the frame start input signal.</p> <p>Unused if you do not select <b>Variable Refresh</b> for <b>Frame Lock Mode</b>.</p> |

Figure 135. Default Mode Configuration GUI

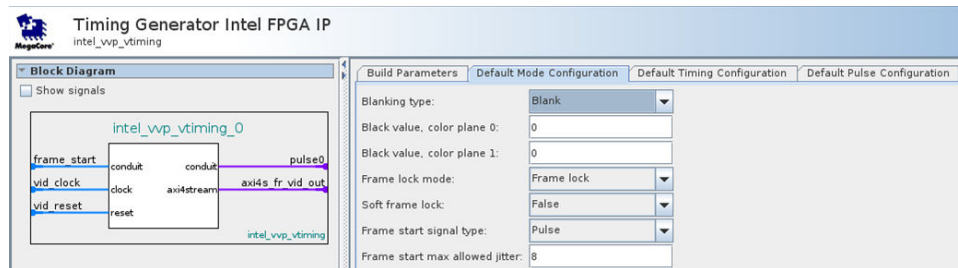


Table 1018. Default Timing Configuration

The Default Timing Configuration parameters apply to the IP when the IP is reset. You can change them at run time using the processor Interface, without recompiling the IP.

| Parameter                          | Allowed Range <sup>(161)</sup><br>( <sup>162</sup> ) | Description                                                                                                                                                                                   |
|------------------------------------|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Dimensions</b>                  |                                                      |                                                                                                                                                                                               |
| <b>Default value for HRESET</b>    | 0 to Hmax                                            | When you select <b>Frame lock</b> for <b>Frame Lock Mode</b> , this parameter is the horizontal coordinate where the IP expects the frame start input signal.<br>Unused if frame lock is off. |
| <b>Default value for VRESET</b>    | 0 to Vmax                                            | When you select <b>Frame lock</b> for <b>Frame Lock Mode</b> , this parameter is the vertical coordinate where the frame start input signal is expected.<br>Unused if frame lock is off.      |
| <b>Default value for HTOTAL</b>    | 0 to Hmax                                            | The width of the current raster, in pixels.                                                                                                                                                   |
| <b>Default value for VTOTAL</b>    | 0 to Vmax                                            | The width of the current raster, in pixels.                                                                                                                                                   |
| <b>Blank Timing</b>                |                                                      |                                                                                                                                                                                               |
| <b>Default value for HB_END</b>    | 0 to Hmax                                            | First pixel of active video after horizontal blanking. Blanking always starts on pixel 0.                                                                                                     |
| <b>Default value for V1B_START</b> | 0 to Vmax                                            | First line of vertical blanking for field 1 (if interlaced) and progressive frames.                                                                                                           |
| <b>Default value for V1B_END</b>   | 0 to Vmax                                            | First line of active video after vertical blanking for field 1 (if interlaced) and progressive frames.                                                                                        |
| <b>Default value for V2B_START</b> | 0 to Vmax                                            | First line of vertical blanking for field 2 (if interlaced).                                                                                                                                  |
| <b>Default value for V2B_END</b>   | 0 to Vmax                                            | First line of active video after vertical blanking for field 2 (if interlaced).                                                                                                               |
| <i>continued...</i>                |                                                      |                                                                                                                                                                                               |

(161)  $V_{max} = (2^{\text{Vertical counter bits}}) - 1$

(162)  $H_{max} = (2^{\text{Horizontal counter bits}}) - 1$

| Parameter                           | Allowed Range <sup>(161)</sup><br>(162) | Description                                                                                                       |
|-------------------------------------|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Field Timing</b>                 |                                         |                                                                                                                   |
| <b>Default value for F1_START</b>   | 0 to Vmax                               | First line where f=0.                                                                                             |
| <b>Default value for F2_START</b>   | 0 to Vmax                               | First line where f=1.                                                                                             |
| <b>Sync Timing</b>                  |                                         |                                                                                                                   |
| <b>Default value for HS_START</b>   | 0 to Hmax                               | First pixel of horizontal sync.                                                                                   |
| <b>Default value for HS_END</b>     | 0 to Hmax                               | First pixel after horizontal sync.                                                                                |
| <b>Default value for V1S_VSTART</b> | 0 to Vmax                               | Vertical coordinate of first pixel of vertical sync for field 1 (if interlaced) and progressive frames.           |
| <b>Default value for V1S_HSTART</b> | 0 to Hmax                               | Horizontal coordinate of first pixel of vsync for field 1 (if interlaced) and progressive frames.                 |
| <b>Default value for V1S_VEND</b>   | 0 to Vmax                               | Vertical coordinate of first active pixel after vertical sync for field 1 (if interlaced) and progressive frames. |
| <b>Default value for V1S_HEND</b>   | 0 to Hmax                               | Horizontal coordinate of first active pixel after vsync for field 1 (if interlaced) and progressive frames.       |
| <b>Default value for V2S_VSTART</b> | 0 to Vmax                               | Vertical coordinate of first pixel of vertical sync for field 2 (if interlaced).                                  |
| <b>Default value for V2S_HSTART</b> | 0 to Hmax                               | Horizontal coordinate of first pixel of vsync for field 2 (if interlaced).                                        |
| <b>Default value for V2S_VEND</b>   | 0 to Vmax                               | Vertical coordinate of first active pixel after vertical sync for field 2 (if interlaced).                        |
| <b>Default value for V2S_HEND</b>   | 0 to Hmax                               | Horizontal coordinate of first active pixel after vsync for field 2 (if interlaced).                              |

<sup>(161)</sup> Vmax =  $(2^{\text{Vertical counter bits}}) - 1$

<sup>(162)</sup> Hmax =  $(2^{\text{Horizontal counter bits}}) - 1$

Figure 136. Default Timing Configuration GUI

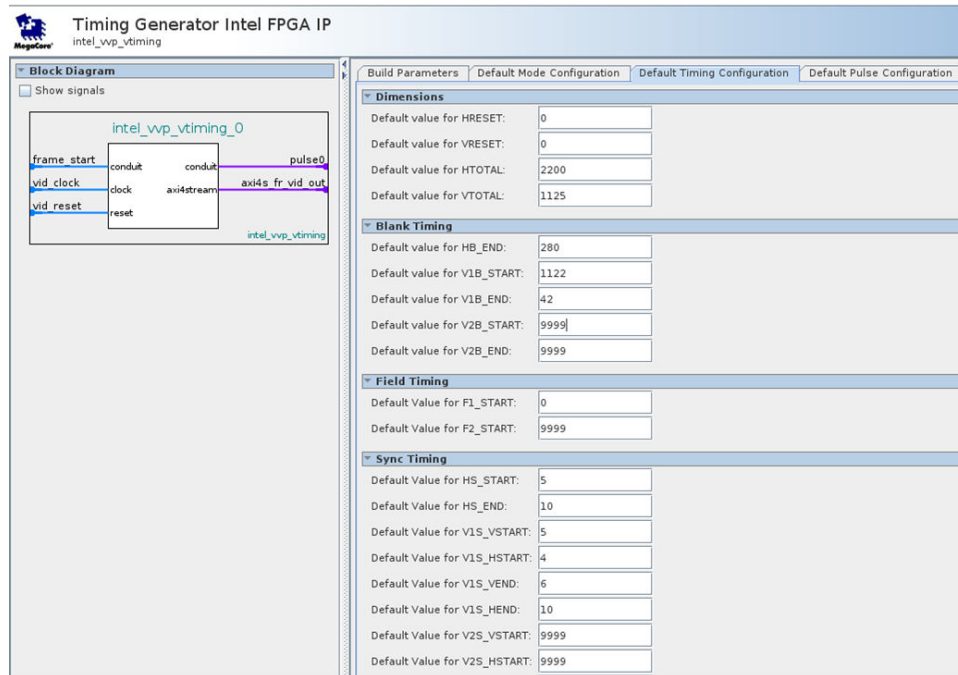


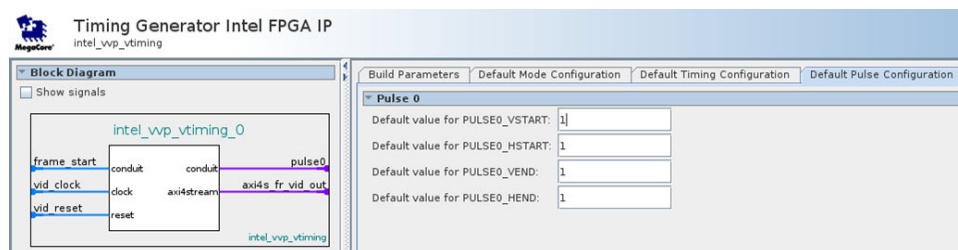
Table 1019. Default Pulse Configuration

The Default Pulse Configuration parameters apply to the IP when the IP is reset. You can change them at run time using the processor interface, without recompiling the IP.

Each output has one set of parameters. If the **Build Parameter** tab **Number of Pulses** is 0, this GUI is not available.

| Parameter                       | Allowed Range | Description                                             |
|---------------------------------|---------------|---------------------------------------------------------|
| Default value for PULSEn_VSTART | 0 to Vmax     | Vertical coordinate of first pixel of pulse             |
| Default value for PULSEn_HSTART | 0 to Hmax     | Horizontal coordinate of first pixel of pulse           |
| Default value for PULSEn_VEND   | 0 to Vmax     | Vertical coordinate of first active pixel after pulse   |
| Default value for PULSEn_HEND   | 0 to Hmax     | Horizontal coordinate of first active pixel after pulse |

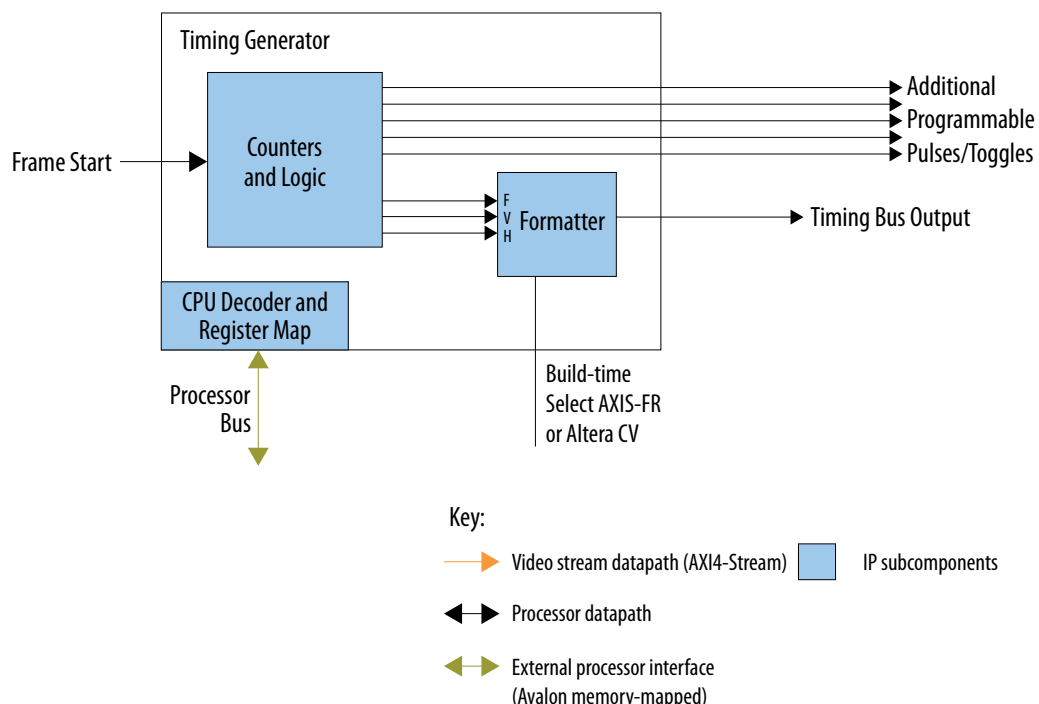
Figure 137. Default Pulse Configuration GUI



### 51.3. Video Timing Generator IP Functional Description

The IP comprises a simple but powerful counter and comparator architecture. Two counters track the real-time horizontal and vertical position of a pixel within the full-raster interface. Multiple software programmable comparators generate timing pulses for the  $f$ ,  $v$ , and  $h$  signals.

**Figure 138. Timing Generator high-level block diagram.**



This processor decoder and register map provide a simple interface to the processor bus. The IP shows all run-time parameters for the video timing through the register map. All run-time parameters default to values provided at build-time.

These counters and logic contain a horizontal pixel counter and vertical line counter. The submodule produces the video timing signals  $f$ ,  $v$ , and  $h$  as specified by the processor registers. The processor specifies additional programmable "pulses" to aid other modules in the system. For example, a programmable pulse can trigger the preload on the SDRAM controller.

This formatter takes the  $f$ ,  $v$ , and  $h$  signals and forms a full-raster bus, or an Altera clocked video bus. You select the type of bus at build time.

#### Output Pixels

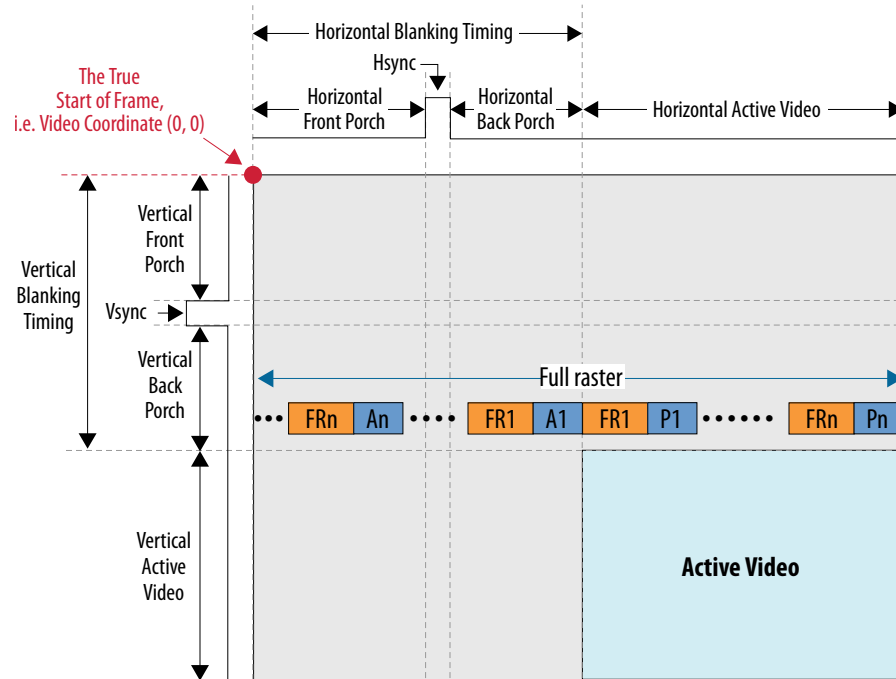
The output timing bus contains space for pixel data. The value of the pixel data can be set by the processor at run time but initially defaults to the value defined at build time.

The IP has a build-time option to include or exclude the  $t_{Ready}$  signal for the full-raster interface. However, the IP does not use this signal. The IP includes it only to allow connection to a full-raster bus that includes this signal. If the  $t_{Ready}$  signal is deasserted, the Video Timing Generator IP continues to produce data.

## Timing

**Figure 139. Timing for a Progressive Video Image**

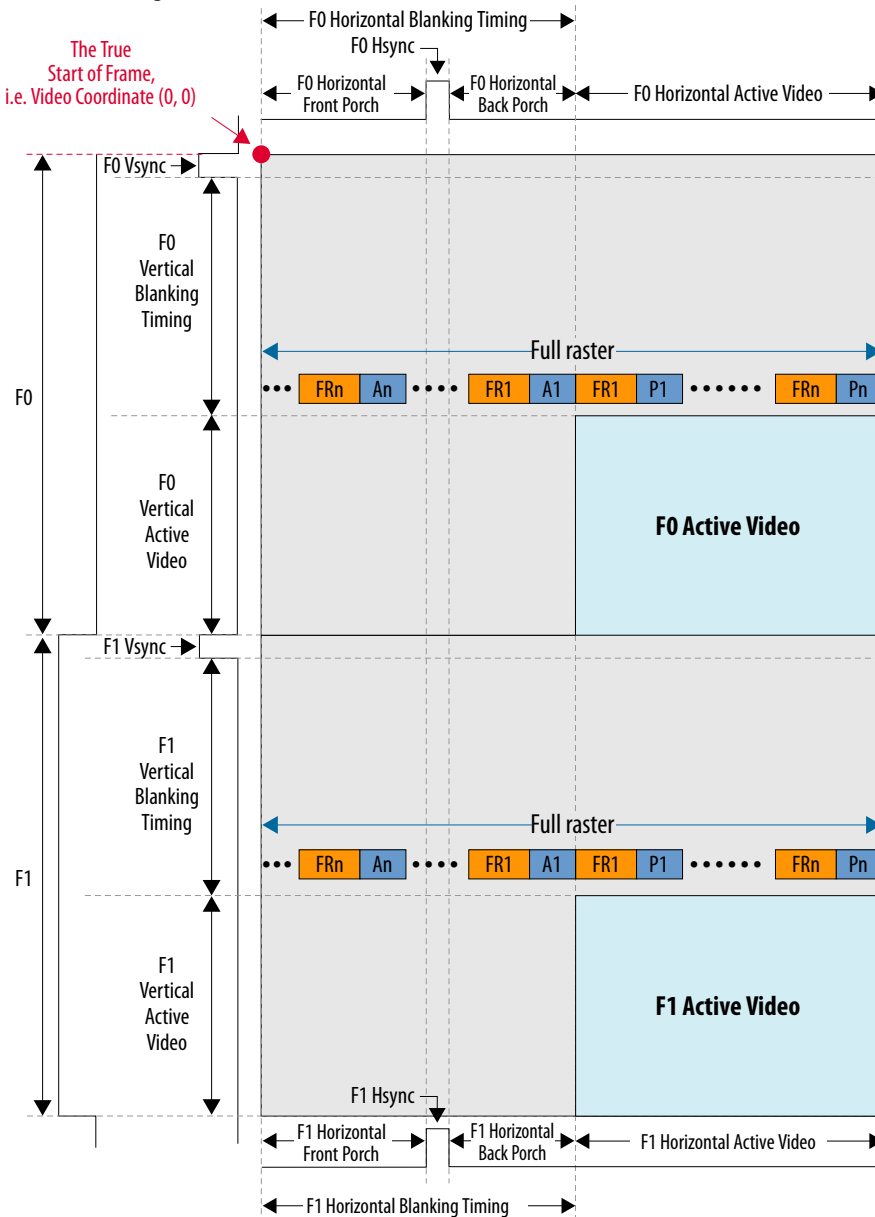
The processor registers can configure the timing signals to generate sync or blank timing for a progressive image





**Figure 140. Timing for an Interlaced Image**

The figure shows the processor registers can configure the timing signals to generate sync or blank timing for an interlaced image.



### Clock Domains

The Timing Generator produces output on the transmit clock for the connectivity IP.

The processor interface operates on the processor clock domain. Drive the processor interface from a known stable clock, such as a dedicated processor clock. Do not drive connectivity IP from the transmit clock as it can be unstable. For example when standards change, which can potentially corrupt the processor interface.

## 51.4. Video Timing Generator IP Interfaces

The IP has one video output interface. You can configure the video output interface at build time to be either full-raster or Altera legacy clocked video interface. The IP has one optional processor interface.

The processor interface is asynchronous to the video output interface. The video clock can be unstable when you select a new standard, which can cause unreliable behavior if you use it for the processor interface.

The Altera Streaming Video full-raster protocol is compatible with AMBA AXI4-Stream interfaces to connect components that exchange video data. The protocols allow interfaces to Altera video IPs or other AXI4-Stream compliant third-party video IPs.

The video clock and processor clock are asynchronous to each other. Internally, the Video Timing Generator IP includes clock domain crossing (CDC) circuits for both single bit and data bus signal cases, which allows safe data exchange between the two asynchronous clock domains. The Video Timing Generator IP also includes an embedded entity `.sdc` file, which provides all the necessary information to the Timing Analyzer. For system integration, when you instantiate the Video Timing Generator IP in a design, the only constraints required are:

- Clock frequency constraints for the video clock (`vid_clock_clk`)
- Clock frequency constraints for the processor clock (`cpu_clock_clk`)

**Table 1020. Video Timing Generator output video interfaces**

| Signal name                                                                              | Direction | Width | Description                                                                                                                                     |
|------------------------------------------------------------------------------------------|-----------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Clocks and resets</b>                                                                 |           |       |                                                                                                                                                 |
| <code>vid_clock_clk</code>                                                               | In        | 1     | Output AXI4-S full-raster processing clock                                                                                                      |
| <code>vid_reset_reset</code>                                                             | in        | 1     | Output AXI4-S full-raster processing reset                                                                                                      |
| <code>cpu_clock_clk</code>                                                               | in        | 1     | Processor interface processing clock                                                                                                            |
| <code>cpu_reset_reset</code>                                                             | in        | 1     | Processor interface processing reset                                                                                                            |
| <b>Control Interface</b>                                                                 |           |       |                                                                                                                                                 |
| This interface is only available if you turn on <b>Memory-Mapped Control Interface</b> . |           |       |                                                                                                                                                 |
| <code>av_mm_cpu_agent_address</code>                                                     | Input     | 7     | Control agent port Avalon memory-mapped address bus. Specifies a word offset into the slave address space.                                      |
| <code>av_mm_cpu_agent_read</code>                                                        | Input     | 1     | Control agent port Avalon memory-mapped read signal. When you assert this signal, the control port drives new data onto the read data bus.      |
| <code>av_mm_cpu_agent_read_data_valid</code>                                             | Output    | 1     | Control agent port Avalon memory-mapped read data valid signal. The IP asserts this signal on the same clock cycle when the read data is valid. |
| <code>av_mm_cpu_agent_readdata</code>                                                    | Output    | 32    | Control agent port Avalon memory-mapped read data bus. These output lines are used for read transfers                                           |
| <code>av_mm_cpu_agent_waitrequest</code>                                                 | Output    | 1     | Control agent port Avalon memory-mapped wait request bus. This signal indicates that the agent is stalling the master transaction.              |
| <code>av_mm_cpu_agent_write</code>                                                       | Input     | 1     | Control agent port Avalon memory-mapped write signal. When you assert this signal, the control port accepts new data from the write data bus.   |
| <i>continued...</i>                                                                      |           |       |                                                                                                                                                 |

| Signal name                                                                                                                                                                                   | Direction | Width      | Description                                                                                                                                                                                                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| av_mm_cpu_agent_writedata                                                                                                                                                                     | Input     | 32         | Control agent port Avalon memory-mapped write data bus. These input lines are used for writing transfers.                                                                                                                                                                                              |
| av_mm_cpu_agent_byteenable                                                                                                                                                                    | Input     | 4          | Control agent port Avalon memory-mapped byteenable bus. These lines indicate which bytes are selected for write and read transactions.                                                                                                                                                                 |
| <b>Altera Streaming Video interfaces</b>                                                                                                                                                      |           |            |                                                                                                                                                                                                                                                                                                        |
| axi4s_fr_vid_out_tdata                                                                                                                                                                        | out       | (163)(164) | AXI4-S data out                                                                                                                                                                                                                                                                                        |
| axi4s_fr_vid_out_tvalid                                                                                                                                                                       | out       | 1          | AXI4-S data valid                                                                                                                                                                                                                                                                                      |
| axi4s_fr_vid_out_tuser[0]                                                                                                                                                                     | out       | 1          | AXI4-S start of video frame                                                                                                                                                                                                                                                                            |
| axi4s_fr_vid_out_tlast                                                                                                                                                                        | out       | 1          | AXI4-S end of packet                                                                                                                                                                                                                                                                                   |
| axi4s_fr_vid_out_tready                                                                                                                                                                       | in        | 1          | AXI4-S data ready                                                                                                                                                                                                                                                                                      |
| <b>Altera CV-Lite Streaming Video Interface</b><br>This interface is only available if you set <b>Output Type</b> to <b>CV</b> on the <b>Build Parameters</b> tab<br>PIP = pixels in parallel |           |            |                                                                                                                                                                                                                                                                                                        |
| cv_vid_out_h                                                                                                                                                                                  | Output    | PIP        | When 1, the video is in a horizontal blanking.                                                                                                                                                                                                                                                         |
| cv_vid_out_v                                                                                                                                                                                  | Output    | PIP        | When 1, the video is in a vertical blanking.                                                                                                                                                                                                                                                           |
| cv_vid_out_h_sync                                                                                                                                                                             | Output    | PIP        | When 1, the video is in a horizontal synchronization period.                                                                                                                                                                                                                                           |
| cv_vid_out_v_sync                                                                                                                                                                             | Output    | PIP        | When 1, the video is in a vertical synchronization period.                                                                                                                                                                                                                                             |
| cv_vid_out_f                                                                                                                                                                                  | Output    | PIP        | When 1, the video is interlaced and in field 1. When 0, the video is either progressive or interlaced and in field 0.                                                                                                                                                                                  |
| cv_vid_out_active                                                                                                                                                                             | Output    | PIP        | When asserted, the video is in an active picture period (not horizontal or vertical blanking). You must drive this signal for correct operation of the IP.                                                                                                                                             |
| cv_vid_out_data                                                                                                                                                                               | Output    | (165)      | Pixel data                                                                                                                                                                                                                                                                                             |
| <b>Internal Timing Generator Conduits</b>                                                                                                                                                     |           |            |                                                                                                                                                                                                                                                                                                        |
| frame_start                                                                                                                                                                                   | Input     | 1          | The internal video timing generator uses this signal to indicate the start of frame. Depending on the generator <b>Frame lock mode</b> , the IP ignores the <code>frame_start</code> signal i.e., freerunning (default configuration)<br>If you select <b>Pulse</b> for <b>Frame start signal type</b> |
| <i>continued...</i>                                                                                                                                                                           |           |            |                                                                                                                                                                                                                                                                                                        |

- (163) The equation gives all full-raster tdata widths:  

$$\max(\text{floor}(((\text{bits per color sample} \times (\text{number of color planes} + 1) \times \text{pixels in parallel}) + 7) / 8) \times 8, 16)$$
- (164) The equation gives all tdata video active only sizes:  

$$\max(\text{floor}(((\text{bits per color sample} \times \text{number of color planes} \times \text{pixels in parallel}) + 7) / 8) \times 8, 16)$$

$$N = \text{ceil}(\text{tdata width} / 8)$$
- (165) The equation gives the data width:  

$$\text{width} = (\text{bits per color sample} \times \text{number of color planes} \times \text{pixels in parallel})$$

| Signal name | Direction | Width | Description                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-------------|-----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|             |           |       | , the IP processes the frame start input signal as a pulse. The IP uses the rising edge of the signal to indicate the start of a frame.<br>If you select <b>Toggle</b> for <b>Frame start signal type</b> , the IP processes the frame start input signal as a toggle and the IP uses both edges of the signal to indicate the start of a frame.                                                                                                   |
| Pulse       | Output    | 1     | The IP can produce up to 8 additional outputs that each provide a pulse or a toggle once per frame.<br>You can program these general-purpose signals to occur at any fixed point in the raster. Separate parameters specify the first and last pixel of the pulse.<br>If the start and end pixels are the same, the IP generates a single clock pulse. If the end pixel is outside the defined raster, the signal becomes a once-per-frame toggle. |

### Related Information

Video and Vision Processing IP Interfaces on page 30

## 51.5. Video Timing Generator IP Registers

The processor interface is optional. You must turn it on at build time on the **Build Parameters** tab in the GUI. Unless stated, all registers are 32-bit wide.

**Table 1021. External Processor Interface**

The Video Timing Generator IP allows run-time configuration of parameters via an Avalon memory-mapped processor register interface.

| Register                          | Offset | Access | Description                                                                                                                                                                                           |
|-----------------------------------|--------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Parameterization Registers</b> |        |        |                                                                                                                                                                                                       |
| VID_PID                           | 0x000  | RO     | Read this register to retrieve the product ID                                                                                                                                                         |
| VERSION                           | 0x004  | RO     | Read this register to retrieve the version information for the IP.                                                                                                                                    |
| NUM_PULSES                        | 0x008  | RO     | The number of additional output pulses generated by the Video Timing Generator.                                                                                                                       |
| OUTPUT_IS_CV                      | 0x00c  | RO     | When 1, the output is legacy Altera Clocked Video.<br>When 0, the output is a full-raster variant.                                                                                                    |
| ALIGNED_ONLY                      | 0x010  | RO     | When 1, the raster is restricted such that all timing parameters for the raster must be integer multiples of the number of pixels in parallel.<br>When 0, the timing parameters have no restrictions. |
| NUMBER_OF_COLOR_PLANES            | 0x014  | RO     | The number of color planes per pixel.                                                                                                                                                                 |
| PIXELS_IN_PARALLEL                | 0x018  | RO     | The number of pixels per clock transferred on the AXI4-S buses.                                                                                                                                       |
| BPS                               | 0x01c  | RO     | The bits per video sample.                                                                                                                                                                            |
| HSIZE                             | 0x020  | RO     | The number of bits to build the horizontal counters and comparators in the Video Timing Generator. Determines the maximum width of raster that the IP can generate.                                   |
| VSIZE                             | 0x024  | RO     | The number of bits to build the vertical counters and comparators inside the Video Timing Generator. Determines the maximum height of raster that the IP can generate.                                |
| <i>continued...</i>               |        |        |                                                                                                                                                                                                       |

| Register              | Offset | Access | Description                                                                                                                                                          |
|-----------------------|--------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CPU_CLK_FREQ_HZ       | 0x028  | RO     | The frequency, in Hz, of the processor interface clock.                                                                                                              |
| BUILD_HARD_FRAME_LOCK | 0x02c  | RO     | If true, the Video Timing Generator includes the hard frame lock function. If false, the Video Timing Generator cannot achieve a hard lock to an external reference. |
| BUILD_SOFT_FRAME_LOCK | 0x030  | RO     | If true, the Video Timing Generator includes the soft frame lock function. If false, the Video Timing Generator cannot achieve a soft lock to an external reference. |
| BUILD_VRR             | 0x034  | RO     | If true, the Video Timing Generator includes the variable refresh rate. If false, the Video Timing Generator cannot generate a variable refresh rate.                |

**Table 1022. Core Processor Register Description**

| Register                       | Offset | Access              | Description                                                                                  |
|--------------------------------|--------|---------------------|----------------------------------------------------------------------------------------------|
| <b>Raster Timing Registers</b> |        |                     |                                                                                              |
| REG_STATUS                     | 0x140  | RO                  | Reserved                                                                                     |
| REG_COMMIT                     | 0x144  | RW                  | Update internal parameters with new video standard.                                          |
| REG_MODE                       | 0x148  | RW                  | Control mode of operation.                                                                   |
| REG_RESET_POS                  | 0x14c  | RW                  | Expected position of the start of frame input signal, relative to the raster                 |
| REG_TOTALS                     | 0x150  | RW                  | Total height and width of the raster                                                         |
| REG_HB_END                     | 0x154  | RW                  | First active pixel of a line                                                                 |
| REG_V1B_POS                    | 0x158  | RW                  | Start and end of vertical blanking for field 1                                               |
| REG_V2B_POS                    | 0x15c  | RW                  | Start and end of vertical blanking for field 2                                               |
| REG_FIELD_STARTS               | 0x160  | RW                  | First lines of field 1 and 2                                                                 |
| REG_HS_POS                     | 0x164  | RW                  | The start and end of horizontal sync                                                         |
| REG_V1S_START                  | 0x168  | RW                  | The horizontal and vertical position of the start of the vertical sync for field 1           |
| REG_V1S_END                    | 0x16c  | RW                  | The horizontal and vertical position of the end of the vertical sync for field 1             |
| REG_V2S_START                  | 0x170  | RW                  | The horizontal and vertical position of the start of the vertical sync for field 2           |
| REG_V2S_END                    | 0x174  | RW                  | The horizontal and vertical position of the end of the vertical sync for field 2             |
| REG_JITTER_CONT                | 0x178  | RW                  | Timing parameters for the hard and soft frame locks                                          |
| REG_BLACK_0                    | 0x17c  | RW                  | The initial value of black for this color plane                                              |
| REG_BLACK_1                    | 0x180  | RW <sup>(166)</sup> | The initial value of black for this color plane                                              |
| REG_BLACK_2                    | 0x184  | RW <sup>(166)</sup> | The initial value of black for this color plane                                              |
| REG_BLACK_3                    | 0x188  | RW <sup>(166)</sup> | The initial value of black for this color plane                                              |
| REG_FRAME_COUNTS               | 0x18c  | RO                  | Returns the total number of frames output, and the number of external frame starts received. |
| REG_FRAME_LENGTH               | 0x190  | RO                  | The number of video clocks between consecutive frame start input signals                     |
| <i>continued...</i>            |        |                     |                                                                                              |

<sup>(166)</sup> This register is RO and returns 0x1234abcd if the color plane does not exist.

| Register                                 | Offset | Access              | Description                                                          |
|------------------------------------------|--------|---------------------|----------------------------------------------------------------------|
| REG_VTOTAL_ADJ                           | 0x194  | RO <sup>(167)</sup> | The total height of the raster after adjustment for soft frame lock. |
| REG_VID_FREQ                             | 0x198  | RO                  | The frequency of the video clock, in Hz.                             |
| REG_GENLOCK_STATS0                       | 0x19c  | RO                  | Diagnostics for hard and soft frame lock                             |
| REG_GENLOCK_STATS1                       | 0x1a0  | RO                  | Diagnostics for soft frame lock.                                     |
| <b>Pulse and Toggle Timing Registers</b> |        |                     |                                                                      |
| REG_PULSE0_START                         | 0x1c0  | RW <sup>(168)</sup> | The horizontal and vertical position of the start of the pulse       |
| REG_PULSE0_END                           | 0x1c4  | RW <sup>(168)</sup> | The horizontal and vertical position of the end of the pulse         |
| REG_PULSE1_START                         | 0x1c8  | RW <sup>(168)</sup> | The horizontal and vertical position of the start of the pulse       |
| REG_PULSE1_END                           | 0x1cc  | RW <sup>(168)</sup> | The horizontal and vertical position of the end of the pulse         |
| REG_PULSE2_START                         | 0x1d0  | RW <sup>(168)</sup> | The horizontal and vertical position of the start of the pulse       |
| REG_PULSE2_END                           | 0x1d4  | RW <sup>(168)</sup> | The horizontal and vertical position of the end of the pulse         |
| REG_PULSE3_START                         | 0x1d8  | RW <sup>(168)</sup> | The horizontal and vertical position of the start of the pulse       |
| REG_PULSE3_END                           | 0x1dc  | RW <sup>(168)</sup> | The horizontal and vertical position of the end of the pulse         |
| REG_PULSE4_START                         | 0x1e0  | RW <sup>(168)</sup> | The horizontal and vertical position of the start of the pulse       |
| REG_PULSE4_END                           | 0x1e4  | RW <sup>(168)</sup> | The horizontal and vertical position of the end of the pulse         |
| REG_PULSE5_START                         | 0x1e8  | RW <sup>(168)</sup> | The horizontal and vertical position of the start of the pulse       |
| REG_PULSE5_END                           | 0x1ec  | RW <sup>(168)</sup> | The horizontal and vertical position of the end of the pulse         |
| REG_PULSE6_START                         | 0x1f0  | RW <sup>(168)</sup> | The horizontal and vertical position of the start of the pulse       |
| REG_PULSE6_END                           | 0x1f4  | RW <sup>(168)</sup> | The horizontal and vertical position of the end of the pulse         |
| REG_PULSE7_START                         | 0x1f8  | RW <sup>(168)</sup> | The horizontal and vertical position of the start of the pulse       |
| REG_PULSE7_END                           | 0x1fc  | RW <sup>(168)</sup> | The horizontal and vertical position of the end of the pulse         |

**Table 1023. REG\_STATUS**

| Name     | Bits | Attribute | Description |
|----------|------|-----------|-------------|
| Reserved | 31:0 | Rsvd      | Reserved    |

**Table 1024. REG\_COMMIT**

| Name          | Bits | Attribute | Description                                                                                                                                      |
|---------------|------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Commit        | 0    | WO        | Writing a 1 to this bit causes all uploaded processor parameters to be committed to the Video Timing Generators at the start of the next raster. |
| VidClkRunning | 1    | RO        | 1 indicates the video clock is running.                                                                                                          |

*continued...*

<sup>(167)</sup> If Soft Mode is off, this register is undefined.

<sup>(168)</sup> This register is RO and returns 0x1234abcd if the pulse does not exist. Refer to **Build Parameter** tab **Number of pulses**

|                     |   |    |                                                                                                                                              |
|---------------------|---|----|----------------------------------------------------------------------------------------------------------------------------------------------|
|                     |   |    | 0 indicates the video clock is not running, or is unstable.                                                                                  |
| UpdateTimingRequest | 2 | RO | 1 indicates the IP issues a commit, but is not completed. The commit logic has seen no start of raster.<br>0 indicates no commit is pending. |

Table 1025. REG\_MODE

| Name                 | Bits | Attribute | Description                                                                                                                    |
|----------------------|------|-----------|--------------------------------------------------------------------------------------------------------------------------------|
| Frame Start is Pulse | 0    | RW        | Sets the type of signal for the frame start input<br>If 1, the input is a short pulse<br>If 0, the input is a per-frame toggle |
| Blank_n_Sync         | 1    | RW        | If 1, the IP generates blank timing<br>If 0, the IP generates sync timing                                                      |
| HardFrameLock        | 2    | RW        | If 1, the soft frame lock is false.<br>If 0, the soft frame lock is true                                                       |
| SoftFrameLock        | 3    | RW        | If 1, and the soft frame lock is false, soft frame lock is on.<br>If 0, soft frame lock is off.                                |
| VRR Enable           | 4    | RW        | If 1, variable refresh rate is on.<br>If 0, variable refresh rate is off.                                                      |
| VRR Line Mode        | 5    | RW        | If variable refresh rate is on, set to 1 to turn on the <b>VVR Line Mode</b> .<br>If set to 0, <b>VVR Line Mode</b> is off.    |
| Reserved             | 31:6 | RO        | Reserved                                                                                                                       |

Table 1026. REG\_RESET\_POS

| Name   | Bits        | Attribute | Description                                                                  |
|--------|-------------|-----------|------------------------------------------------------------------------------|
| hReset | hSize-1:0   | RW        | The horizontal position, in pixels, of the expected frame start input signal |
| vReset | 15+vSize:16 | RW        | The vertical position, in lines, of the expected frame start input signal    |

Table 1027. REG\_TOTALS

| Name   | Bits        | Attribute | Description                         |
|--------|-------------|-----------|-------------------------------------|
| hTotal | hSize-1:0   | RW        | The width, in pixels, of the raster |
| vTotal | 15+vSize:16 | RW        | The height, in lines, of the raster |

Table 1028. REG\_HB\_END

| Name     | Bits      | Attribute | Description                                           |
|----------|-----------|-----------|-------------------------------------------------------|
| HB_END   | hSize-1:0 | RW        | The first active pixel after the horizontal blanking. |
| Reserved | 32:hSize  | RO        | Reserved                                              |

Table 1029. REG\_V1B\_POS

| Name      | Bits        | Attribute | Description                                     |
|-----------|-------------|-----------|-------------------------------------------------|
| V1B Start | vSize-1:0   | RW        | The first line of vertical blanking for field 1 |
| V1B End   | 15+vSize:16 | RW        | The first active line of field 1                |

**Table 1030. REG\_V2B\_POS**

| Name      | Bits        | Attribute | Description                                                                            |
|-----------|-------------|-----------|----------------------------------------------------------------------------------------|
| V2B Start | vSize-1:0   | RW        | The first line of vertical blanking for field 2<br>For a progressive raster, set to -1 |
| V2B End   | 15+vSize:16 | RW        | The first active line of field2<br>For a progressive raster, set to -1                 |

**Table 1031. REG\_FIELD\_STARTS**

| Name          | Bits        | Attribute | Description                                                      |
|---------------|-------------|-----------|------------------------------------------------------------------|
| Field 1 Start | vSize-1:0   | RW        | The first line of field 1                                        |
| Field 2 End   | 15+vSize:16 | RW        | The first line of field 2<br>For a progressive raster, set to -1 |

**Table 1032. REG\_HS\_POS**

| Name        | Bits        | Attribute | Description                               |
|-------------|-------------|-----------|-------------------------------------------|
| HSync Start | hSize-1:0   | RW        | The first pixel of the horizontal sync    |
| HSync End   | 15+hSize:16 | RW        | The first pixel after the horizontal sync |

**Table 1033. REG\_V1S\_START**

| Name          | Bits        | Attribute | Description                                      |
|---------------|-------------|-----------|--------------------------------------------------|
| V1Sync hStart | hSize-1:0   | RW        | The first pixel of the vertical sync for field 1 |
| V1Sync vStart | 15+hSize:16 | RW        | The first line of the vertical sync for field 1  |

**Table 1034. REG\_V1S\_END**

| Name        | Bits        | Attribute | Description                                         |
|-------------|-------------|-----------|-----------------------------------------------------|
| V1Sync hEnd | hSize-1:0   | RW        | The first pixel after the vertical sync for field 1 |
| V1Sync vEnd | 15+hSize:16 | RW        | The last line of the vertical sync for field 1      |

**Table 1035. REG\_V2S\_START**

| Name          | Bits        | Attribute | Description                                                                             |
|---------------|-------------|-----------|-----------------------------------------------------------------------------------------|
| V2Sync hStart | hSize-1:0   | RW        | The first pixel of the vertical sync for field 2<br>For a progressive raster, set to -1 |
| V2Sync vStart | 15+hSize:16 | RW        | The first line of the vertical sync for field 2<br>For a progressive raster, set to -1  |

**Table 1036. REG\_V2S\_END**

| Name        | Bits        | Attribute | Description                                                                                |
|-------------|-------------|-----------|--------------------------------------------------------------------------------------------|
| V2Sync hEnd | hSize-1:0   | RW        | The first pixel after the vertical sync for field 2<br>For a progressive raster, set to -1 |
| V2Sync vEnd | 15+vSize:16 | RW        | The last line of the vertical sync for field 2<br>For a progressive raster, set to -1      |



Table 1037. REG\_JITTER\_CONT

| Name                         | Bits  | Attribute | Description                                                                                                                                                                                                                                                                                                                                         |
|------------------------------|-------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frame Start Max Jitter       | 7:0   | RW        | The number of video clock cycles either side of the position of the expected start of frame where an occurrence of the frame start input signal does not cause the raster to restart.<br>If the frame start input signal occurs more than this number of video clock cycles from the point where it is expected, the IP restarts the output raster. |
| Soft lock frame start ignore | 15:8  | RW        | When you select <b>True</b> for <b>Soft frame lock</b> , this parameter specifies the number of lines where the frame start input signal is ignored.<br>When you select <b>True</b> for <b>Soft frame lock</b> , this parameter is not used.                                                                                                        |
| Soft lock frame start adjust | 23:16 | RW        | When you select <b>True</b> for <b>Soft frame lock</b> , this parameter specifies the total number of lines for soft lock, including the ignore lines.<br>When you select <b>False</b> for <b>Soft frame lock</b> , this parameter is not used.                                                                                                     |
| Reserved                     | 31:24 | RO        | Reserved                                                                                                                                                                                                                                                                                                                                            |

Table 1038. REG\_BLACK\_0

| Name    | Bits          | Attribute | Description                                     |
|---------|---------------|-----------|-------------------------------------------------|
| BLACK_0 | BIT_DEPTH-1:0 | RW        | The initial value of black for this color plane |

Table 1039. REG\_BLACK\_1

| Name    | Bits          | Attribute | Description                                                                                                                     |
|---------|---------------|-----------|---------------------------------------------------------------------------------------------------------------------------------|
| BLACK_1 | BIT_DEPTH-1:0 | RW        | The initial value of black for this color plane<br>This register is RO and returns 0x1234abcd if the color plane does not exist |

Table 1040. REG\_BLACK\_2

| Name    | Bits          | Attribute | Description                                                                                                                     |
|---------|---------------|-----------|---------------------------------------------------------------------------------------------------------------------------------|
| BLACK_2 | BIT_DEPTH-1:0 | RW        | The initial value of black for this color plane<br>This register is RO and returns 0x1234abcd if the color plane does not exist |

Table 1041. REG\_BLACK\_3

| Name    | Bits          | Attribute | Description                                                                                                                     |
|---------|---------------|-----------|---------------------------------------------------------------------------------------------------------------------------------|
| BLACK_3 | BIT_DEPTH-1:0 | RW        | The initial value of black for this color plane<br>This register is RO and returns 0x1234abcd if the color plane does not exist |

Table 1042. REG\_FRAME\_COUNTS

| Name          | Bits  | Attribute | Description                                         |
|---------------|-------|-----------|-----------------------------------------------------|
| Start Counter | 15:0  | RO        | Counts occurrences of the Frame Start Input Signal. |
| Frame Counter | 31:16 | RO        | Counts number of frames generated.                  |

Table 1043. REG\_FRAME\_LENGTH

| Name         | Bits            | Attribute | Description                                                                |
|--------------|-----------------|-----------|----------------------------------------------------------------------------|
| Frame Length | hSize+vSize-1:0 | RO        | The number of video clocks per occurrence of the frame start input signal. |

**Table 1044. REG\_VTOTAL\_ADJ**

| Name       | Bits      | Attribute | Description                                                      |
|------------|-----------|-----------|------------------------------------------------------------------|
| vTotal_Adj | vSize-1:0 | RO        | The total number of lines per frame used by the soft frame lock. |
| Reserved   | 31:vSize  | RO        | Reserved.                                                        |

**Table 1045. REG\_VID\_FREQ**

| Name       | Bits | Attribute | Description                              |
|------------|------|-----------|------------------------------------------|
| VidClkFreq | 31:0 | RO        | The frequency of the video clock, in Hz. |

**Table 1046. REG\_GENLOCK\_STATS0**

| Name         | Bits  | Attribute | Description                                                                                                                                                                                                                                                                                   |
|--------------|-------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reset Count  | 15:0  | RO        | Counts the number of times the raster is reset by the frame start signal. Normally static and only increments when the output standard is changed. If this value is constantly incrementing, it suggests the period of the frame start signal does not match the period of the output raster. |
| Stable Count | 31:16 | RO        | This increments every time the soft frame lock mechanism does not change the vTotal_adj value.                                                                                                                                                                                                |

**Table 1047. REG\_GENLOCK\_STATS1**

| Name               | Bits  | Attribute | Description                                                                          |
|--------------------|-------|-----------|--------------------------------------------------------------------------------------|
| vTotal Inc Counter | 15:0  | RO        | Increments every time the soft frame lock mechanism increments the vTotal_adj value. |
| vTotal Dec Counter | 31:16 | RO        | Increments every time the soft frame lock mechanism decrements the vTotal_adj value. |

**Table 1048. REG\_PULSE0\_START**

| Name           | Bits        | Attribute | Description                          |
|----------------|-------------|-----------|--------------------------------------|
| Pulse 0 hStart | hSize-1:0   | RW        | The first pixel of the pulse         |
| Pulse 0 vStart | 15+vSize:16 | RW        | The first line of the pulse<br>(169) |

**Table 1049. REG\_PULSE0\_END**

| Name         | Bits        | Attribute | Description                           |
|--------------|-------------|-----------|---------------------------------------|
| Pulse 0 hEnd | hSize-1:0   | RW        | The last pixel of the pulse.<br>(169) |
| Pulse 0 vEnd | 15+vSize:16 | RW        | The last line of the pulse.<br>(169)  |

(169) This register is RO and returns 0x1234abcd if the pulse does not exist. Refer to the **Build Parameter** tab **Number of pulses**

**Table 1050. REG\_PULSE1\_START**

| Name           | Bits        | Attribute | Description                           |
|----------------|-------------|-----------|---------------------------------------|
| Pulse 1 hStart | hSize-1:0   | RW        | The first pixel of the pulse<br>(169) |
| Pulse 1 vStart | 15+vSize:16 | RW        | The first line of the pulse<br>(169)  |

**Table 1051. REG\_PULSE1\_END**

| Name         | Bits        | Attribute | Description                           |
|--------------|-------------|-----------|---------------------------------------|
| Pulse 1 hEnd | hSize-1:0   | RW        | The last pixel of the pulse.<br>(169) |
| Pulse 1 vEnd | 15+vSize:16 | RW        | The last line of the pulse.<br>(169)  |

**Table 1052. REG\_PULSE2\_START**

| Name           | Bits        | Attribute | Description                           |
|----------------|-------------|-----------|---------------------------------------|
| Pulse 2 hStart | hSize-1:0   | RW        | The first pixel of the pulse<br>(169) |
| Pulse 2 vStart | 15+vSize:16 | RW        | The first line of the pulse<br>(169)  |

**Table 1053. REG\_PULSE2\_END**

| Name         | Bits        | Attribute | Description                           |
|--------------|-------------|-----------|---------------------------------------|
| Pulse 2 hEnd | hSize-1:0   | RW        | The last pixel of the pulse.<br>(169) |
| Pulse 2 vEnd | 15+vSize:16 | RW        | The last line of the pulse.<br>(169)  |

**Table 1054. REG\_PULSE3\_START**

| Name           | Bits        | Attribute | Description                           |
|----------------|-------------|-----------|---------------------------------------|
| Pulse 3 hStart | hSize-1:0   | RW        | The first pixel of the pulse<br>(169) |
| Pulse 3 vStart | 15+vSize:16 | RW        | The first line of the pulse<br>(169)  |

**Table 1055. REG\_PULSE3\_END**

| Name         | Bits        | Attribute | Description                           |
|--------------|-------------|-----------|---------------------------------------|
| Pulse 3 hEnd | hSize-1:0   | RW        | The last pixel of the pulse.<br>(169) |
| Pulse 3 vEnd | 15+vSize:16 | RW        | The last line of the pulse.<br>(169)  |

**Table 1056. REG\_PULSE4\_START**

| Name           | Bits      | Attribute | Description                  |
|----------------|-----------|-----------|------------------------------|
| Pulse 4 hStart | hSize-1:0 | RW        | The first pixel of the pulse |
| continued...   |           |           |                              |

|                |             |    |                                      |
|----------------|-------------|----|--------------------------------------|
|                |             |    | (169)                                |
| Pulse 4 vStart | 15+vSize:16 | RW | The first line of the pulse<br>(169) |

**Table 1057. REG\_PULSE4\_END**

| Name         | Bits        | Attribute | Description                           |
|--------------|-------------|-----------|---------------------------------------|
| Pulse 4 hEnd | hSize-1:0   | RW        | The last pixel of the pulse.<br>(169) |
| Pulse 4 vEnd | 15+vSize:16 | RW        | The last line of the pulse.<br>(169)  |

**Table 1058. REG\_PULSE5\_START**

| Name           | Bits        | Attribute | Description                           |
|----------------|-------------|-----------|---------------------------------------|
| Pulse 5 hStart | hSize-1:0   | RW        | The first pixel of the pulse<br>(169) |
| Pulse 5 vStart | 15+vSize:16 | RW        | The first line of the pulse<br>(169)  |

**Table 1059. REG\_PULSE5\_END**

| Name         | Bits        | Attribute | Description                           |
|--------------|-------------|-----------|---------------------------------------|
| Pulse 5 hEnd | hSize-1:0   | RW        | The last pixel of the pulse.<br>(169) |
| Pulse 5 vEnd | 15+vSize:16 | RW        | The last line of the pulse.<br>(169)  |

**Table 1060. REG\_PULSE6\_START**

| Name           | Bits        | Attribute | Description                           |
|----------------|-------------|-----------|---------------------------------------|
| Pulse 6 hStart | hSize-1:0   | RW        | The first pixel of the pulse<br>(169) |
| Pulse 6 vStart | 15+vSize:16 | RW        | The first line of the pulse<br>(169)  |

**Table 1061. REG\_PULSE6\_END**

| Name         | Bits        | Attribute | Description                           |
|--------------|-------------|-----------|---------------------------------------|
| Pulse 6 hEnd | hSize-1:0   | RW        | The last pixel of the pulse.<br>(169) |
| Pulse 6 vEnd | 15+vSize:16 | RW        | The last line of the pulse.<br>(169)  |

**Table 1062. REG\_PULSE7\_START**

| Name           | Bits        | Attribute | Description                           |
|----------------|-------------|-----------|---------------------------------------|
| Pulse 7 hStart | hSize-1:0   | RW        | The first pixel of the pulse<br>(169) |
| Pulse 7 vStart | 15+vSize:16 | RW        | The first line of the pulse<br>(169)  |

Table 1063. REG\_PULSE7\_END

| Name         | Bits        | Attribute | Description                           |
|--------------|-------------|-----------|---------------------------------------|
| Pulse 7 hEnd | hSize-1:0   | RW        | The last pixel of the pulse.<br>(169) |
| Pulse 7 vEnd | 15+vSize:16 | RW        | The last line of the pulse.<br>(169)  |

## 51.6. Video Timing Generator IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

### intel\_vvp\_vtiming\_get\_param\_vid\_pid

prototype:

```
int
intel_vvp_vtiming_get_param_vid_pid(intel_vvp_vtiming_instance*
instance);
```

argument: instance, pointer to the intel\_vvp\_vtiming\_instance

return value: The product ID number

### intel\_vvp\_vtiming\_get\_param\_version\_number

prototype:

```
int
intel_vvp_vtiming_get_param_version_number(intel_vvp_vtiming_instance*
instance);
```

argument: instance, pointer to the intel\_vvp\_vtiming\_instance

return value: The IP version number

### intel\_vvp\_vtiming\_get\_param\_num\_pulses

prototype:

```
int
intel_vvp_vtiming_get_param_num_pulses(intel_vvp_vtiming_instance*
instance);
```

argument: instance, pointer to the intel\_vvp\_vtiming\_instance

return value: The NUM\_PULSES parameter

### intel\_vvp\_vtiming\_get\_param\_output\_is\_cv

**prototype:**

```
int
intel_vvp_vtiming_get_param_output_is_cv(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The OUTPUT\_IS\_CV parameter

### intel\_vvp\_vtiming\_get\_param\_aligned\_only

**prototype:**

```
int
intel_vvp_vtiming_get_param_aligned_only(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The ALIGNED\_ONLY parameter

### intel\_vvp\_vtiming\_get\_param\_color\_planes

**prototype:**

```
int
intel_vvp_vtiming_get_param_color_planes(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The NUMBER\_OF\_COLOR\_PLANES parameter

### intel\_vvp\_vtiming\_get\_param\_pip

**prototype:**

```
int
intel_vvp_vtiming_get_param_pip(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The PIXELS\_IN\_PARALLEL parameter

### intel\_vvp\_vtiming\_get\_param\_bps

**prototype:**

```
int
intel_vvp_vtiming_get_param_bps(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The BPS parameter

#### intel\_vvp\_vtiming\_get\_param\_hsize

*prototype:*

```
int
intel_vvp_vtiming_get_param_hsize(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The HSIZE parameter

#### intel\_vvp\_vtiming\_get\_param\_vsize

*prototype:*

```
int
intel_vvp_vtiming_get_param_vsize(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The VSIZE parameter

#### intel\_vvp\_vtiming\_get\_param\_cpu\_clk\_freq\_hz

*prototype:*

```
int
intel_vvp_vtiming_get_param_cpu_clk_freq_hz(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The CPU\_CLK\_FREQ\_HZ parameter

#### intel\_vvp\_vtiming\_get\_param\_build\_hard\_frame\_lock

*prototype:*

```
int
intel_vvp_vtiming_get_param_hard_frame_lock(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The BUILD\_HARD\_FRAME\_LOCK parameter

### intel\_vvp\_vtiming\_get\_param\_build\_soft\_frame\_lock

**prototype:**

```
int
intel_vvp_vtiming_get_param_sof_frame_lock(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The BUILD\_SOFT\_FRAME\_LOCK parameter

### intel\_vvp\_vtiming\_get\_param\_build\_vrr

**prototype:**

```
int
intel_vvp_vtiming_get_param_build_vrr(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The BUILD\_VRR parameter

### intel\_vvp\_vtiming\_get\_frame\_start\_pulse

**prototype:**

```
int
intel_vvp_vtiming_get_frame_start_pulse(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The FRAME START IS PULSE field from REG\_MODE register

### intel\_vvp\_vtiming\_set\_frame\_start\_pulse

**prototype:**

```
int
intel_vvp_vtiming_set_frame_start_pulse(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

in\_val, setting this value forces the IP to generate the SOF as a short pulse, otherwise it is a toggle signal.

**return value:** if successful kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr



### intel\_vvp\_vtiming\_get\_out\_is\_blank

*prototype:*

```
int
intel_vvp_vtiming_get_out_is_blank(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The BLANK\_N\_SYNC field from REG\_MODE register

### intel\_vvp\_vtiming\_set\_out\_is\_blank

*prototype:*

```
int
intel_vvp_vtiming_set_out_is_blank(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

in\_val, setting this value forces the IP to generate blank timing, otherwise it generates sync timing.

*return value:* if successful kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_allow\_hv\_reset

*prototype:*

```
int
intel_vvp_vtiming_get_allow_hv_reset(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The HARDFRAMELOCK field from REG\_MODE register

### intel\_vvp\_vtiming\_set\_allow\_hv\_reset

*prototype:*

```
int
intel_vvp_vtiming_set_allow_hv_reset(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

in\_val, setting this value forces the IP to enable the hard frame lock.

*return value:* if successful kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_use\_soft\_reset

**prototype:**

```
int
intel_vvp_vtiming_get_use_soft_reset(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The SOFTFRAMELOCK field from REG\_MODE register

### intel\_vvp\_vtiming\_set\_use\_soft\_reset

**prototype:**

```
int
intel_vvp_vtiming_set_use_soft_reset(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance  
in\_val, setting this value forces the IP to enable the soft frame lock.

**return value:** if successful kIntelVvpVtimingOk, otherwise  
kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_vrr

**prototype:**

```
int intel_vvp_vtiming_get_vrr(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The VRR ENABLE field from REG\_MODE register

### intel\_vvp\_vtiming\_set\_vrr

**prototype:**

```
int intel_vvp_vtiming_set_vrr(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance  
in\_val, setting this value forces the IP to enable the Variable Refresh Rate.

**return value:** if successful kIntelVvpVtimingOk, otherwise  
kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_vrr\_linemode

*prototype:*

```
int
intel_vvp_vtiming_get_vrr_linemode(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The VRR LINE MODE field from REG\_MODE register

### intel\_vvp\_vtiming\_set\_vrr\_linemode

*prototype:*

```
int
intel_vvp_vtiming_set_vrr_linemode(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

in\_val, setting this value forces the IP to enable the Variable Refresh Rate in line mode.

*return value:* if successful kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_tim\_hreset

*prototype:*

```
int
intel_vvp_vtiming_get_tim_hreset(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The HRESET field from REG\_RESET\_POS register

### intel\_vvp\_vtiming\_set\_tim\_hreset

*prototype:*

```
int
intel_vvp_vtiming_set_tim_hreset(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

in\_val, the expected horizontal position for the frame start input signal.

*return value:* if successful kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_tim\_vreset

*prototype:*

```
int
intel_vvp_vtiming_get_tim_vreset(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The VRESET field from REG\_RESET\_POS register

### intel\_vvp\_vtiming\_set\_tim\_vreset

*prototype:*

```
int
intel_vvp_vtiming_set_tim_vreset(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance  
in\_val, the expected vertical position for the frame start input signal.

*return value:* if successful kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_tim\_htotal

*prototype:*

```
int
intel_vvp_vtiming_get_tim_htotal(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The HTOTAL field from REG\_TOTALS register

### intel\_vvp\_vtiming\_set\_tim\_htotal

*prototype:*

```
int
intel_vvp_vtiming_set_tim_htotal(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance  
in\_val, the width of the raster

*return value:* if successful kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_tim\_vtotal

*prototype:*

```
int
intel_vvp_vtiming_get_tim_vtotal(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The VTOTAL field from REG\_TOTALS register

### intel\_vvp\_vtiming\_set\_tim\_vtotal

*prototype:*

```
int
intel_vvp_vtiming_set_tim_vtotal(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance  
in\_val, the height of the raster

*return value:* if successful kIntelVvpVtimingOk, otherwise  
kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_tim\_hb\_end

*prototype:*

```
int
intel_vvp_vtiming_get_tim_hb_end(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The HB\_END field from REG\_HB\_END register

### intel\_vvp\_vtiming\_set\_tim\_hb\_end

*prototype:*

```
int
intel_vvp_vtiming_set_tim_hb_end(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance  
in\_val, the first active pixel after horizontal blanking

*return value:* if successful kIntelVvpVtimingOk, otherwise  
kIntelVvpVtimingInstanceErr

**intel\_vvp\_vtiming\_get\_tim\_v{INDX}b\_start** where {INDX} goes from 1 to 2

**prototype:**

```
int
intel_vvp_vtiming_get_tim_v{INDX}b_start(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The V{INDX}B START FIELD from REG\_V{INDX}B\_POS register

**intel\_vvp\_vtiming\_set\_tim\_v{INDX}b\_start**

**prototype:**

```
int
intel_vvp_vtiming_set_tim_v{INDX}b_start(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance  
in\_val, the first line of vertical blanking for field {INDX}

**return value:** if successful kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

**intel\_vvp\_vtiming\_get\_tim\_v{INDX}b\_end**

**prototype:**

```
int
intel_vvp_vtiming_get_tim_v{INDX}b_end(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The V{INDX}B END FIELD from REG\_V{INDX}B\_POS register

**intel\_vvp\_vtiming\_set\_tim\_v{INDX}b\_end**

**prototype:**

```
int
intel_vvp_vtiming_set_tim_v{INDX}b_end(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance  
in\_val, the last line of vertical blanking for field {INDX}

**return value:** if successful kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

**intel\_vvp\_vtiming\_get\_tim\_f{INDX}\_start** where {INDX} goes from 1 to 2

**prototype:**

```
int
intel_vvp_vtiming_get_tim_f{INDX}_start(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The FIELD {INDX} START from REG\_FIELD\_STARTS register

**intel\_vvp\_vtiming\_set\_tim\_f{INDX}\_start**

**prototype:**

```
int
intel_vvp_vtiming_set_tim_f{INDX}_start(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance  
in\_val, the first line of field {INDX}

**return value:** if successful kIntelVvpVtimingOk, otherwise  
kIntelVvpVtimingInstanceErr

**intel\_vvp\_vtiming\_get\_tim\_hs\_start**

**prototype:**

```
int
intel_vvp_vtiming_get_tim_hs_start(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The HSYNC START from REG\_HS\_POS register

**intel\_vvp\_vtiming\_set\_tim\_hs\_start**

**prototype:**

```
int
intel_vvp_vtiming_set_tim_hs_start(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance  
in\_val, the first pixel of the horizontal sync

**return value:** if successful kIntelVvpVtimingOk, otherwise  
kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_tim\_hs\_end

*prototype:*

```
int
intel_vvp_vtiming_get_tim_hs_end(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The HSYNC END from REG\_HS\_POS register

### intel\_vvp\_vtiming\_set\_tim\_hs\_end

*prototype:*

```
int
intel_vvp_vtiming_set_tim_hs_end(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance  
in\_val, the first pixel after the horizontal sync

*return value:* if successful kIntelVvpVtimingOk, otherwise  
kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_tim\_v{INDX}s\_vstart where {INDX} goes from 1 to 2

*prototype:*

```
int
intel_vvp_vtiming_get_tim_v{INDX}s_vstart(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The V{INDX}SYNC VSTART from REG\_V{INDX}S\_START register

### intel\_vvp\_vtiming\_set\_tim\_v{INDX}s\_vstart

*prototype:*

```
int
intel_vvp_vtiming_set_tim_v{INDX}s_vstart(intel_vvp_vtiming_instance*
instance, uint32_t in_val);argument:
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance  
in\_val, the first line of the vertical sync for field {INDX}

*return value:* if successful kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr



### intel\_vvp\_vtiming\_get\_tim\_v{INDX}s\_hstart

**prototype:**

```
int
intel_vvp_vtiming_get_tim_v{INDX}s_hstart(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The V{INDX}SYNC HSTART from REG\_V{INDX}S\_START register

### intel\_vvp\_vtiming\_set\_tim\_v{INDX}s\_hstart

**prototype:**

```
int
intel_vvp_vtiming_set_tim_v{INDX}s_hstart(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance  
in\_val, the first pixel of the vertical sync for field {INDX}

**return value:** if successful kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_tim\_v{INDX}s\_vend

**prototype:**

```
int
intel_vvp_vtiming_get_tim_v{INDX}s_vend(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The V{INDX}SYNC VEND from REG\_V{INDX}S\_END register

### intel\_vvp\_vtiming\_set\_tim\_v{INDX}s\_vend

**prototype:**

```
int
intel_vvp_vtiming_set_tim_v{INDX}s_vend(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance  
in\_val, the last line of the vertical sync for field {INDX}

**return value:** if successful kIntelVvpVtimingOk, otherwise  
kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_tim\_v{INDX}s\_hend

**prototype:**

```
int
intel_vvp_vtiming_get_tim_v{INDX}s_hend(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The V{INDX}SYNC HEND from REG\_V{INDX}S\_END register

### intel\_vvp\_vtiming\_set\_tim\_v{INDX}s\_hend

**prototype:**

```
int
intel_vvp_vtiming_set_tim_v{INDX}s_hend(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance  
in\_val, the first pixel after the vertical sync for field 1{INDX}

**return value:** if successful kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_frame\_start\_jitter

**prototype:**

```
int
intel_vvp_vtiming_get_frame_start_jitter(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The FRAME START MAX JITTER from REG\_JITTER\_CONT register

### intel\_vvp\_vtiming\_set\_frame\_start\_jitter

**prototype:**

```
int
intel_vvp_vtiming_set_frame_start_jitter(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance  
in\_val, the number of video clock cycles either side of the position of the expected start of frame

**return value:** if successful kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_frame\_adj\_ignore

prototype:

```
int
intel_vvp_vtiming_get_frame_adj_ignore(intel_vvp_vtiming_instance*
instance);
```

argument: instance, pointer to the intel\_vvp\_vtiming\_instance

return value: The SOFT LOCK FRAME START IGNORE from REG\_JITTER\_CONT register

### intel\_vvp\_vtiming\_set\_frame\_adj\_ignore

prototype:

```
int
intel_vvp_vtiming_set_frame_adj_ignore(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

argument: instance, pointer to the intel\_vvp\_vtiming\_instance

in\_val, when you select **True** for **Soft frame lock**, this parameter specifies the number of lines where the frame start input signal is ignored

return value: if successful kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_frame\_adj\_window

prototype:

```
int
intel_vvp_vtiming_get_frame_adj_window(intel_vvp_vtiming_instance*
instance);
```

argument: instance, pointer to the intel\_vvp\_vtiming\_instance

return value: The SOFT LOCK FRAME START ADJUST from REG\_JITTER\_CONT register

### intel\_vvp\_vtiming\_set\_frame\_adj\_window

prototype:

```
int
intel_vvp_vtiming_set_frame_adj_ignore(intel_vvp_vtiming_instance*
instance, uint32_t in_val);
```

argument: instance, pointer to the intel\_vvp\_vtiming\_instance

in\_val, when you select **True** for **Soft frame lock**, this parameter specifies the total number of lines for soft lock, including the "ignore" lines.

return value: if successful kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_black\_{INDX} where {INDX} goes from 0 to 3

**prototype:**

```
int intel_vvp_vtiming_get_black_{INDX}
(intel_vvp_vtiming_instance* instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** REG\_BLACK\_{INDX} register

### intel\_vvp\_vtiming\_set\_black\_{INDX}

**prototype:**

```
int intel_vvp_vtiming_set_black_{INDX}
(intel_vvp_vtiming_instance* instance, uint32_t in_val);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance  
in\_val, the pixel value for color plane {INDX}.

**return value:** if successful kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_start\_counter

**prototype:**

```
int
intel_vvp_vtiming_get_start_count(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The START COUNTER field from REG\_FRAME\_COUNTS register

### intel\_vvp\_vtiming\_get\_frame\_count

**prototype:**

```
int
intel_vvp_vtiming_get_frame_count(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The FRAME COUNTER field from REG\_FRAME\_COUNTS register

### intel\_vvp\_vtiming\_get\_actual\_pixels\_latched

**prototype:**

```
int
intel_vvp_vtiming_get_actual_pixels_latched(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The FRAME LENGTH field from REG\_FRAME\_LENGTH register

#### `intel_vvp_vtiming_get_tim_vtotal_latched`

*prototype:*

```
int
intel_vvp_vtiming_get_1_second_ltchd(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The VTOTAL\_ADJ field from REG\_VTOTAL\_ADJ register

#### `intel_vvp_vtiming_get_1_second_ltchd`

*prototype:*

```
int
intel_vvp_vtiming_get_1_second_ltchd(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The VIDCLKFREQ field from REG\_VID\_FREQ register

#### `intel_vvp_vtiming_get_hv_reset_count`

*prototype:*

```
int
intel_vvp_vtiming_get_hv_reset(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The RESET COUNT field from REG\_GENLOCK\_STATS0 register

#### `intel_vvp_vtiming_get_stable_count`

*prototype:*

```
int
intel_vvp_vtiming_get_stable_count(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The STABLE COUNT field from REG\_GENLOCK\_STATS0 register

### intel\_vvp\_vtiming\_get\_inc\_count

*prototype:*

```
int
intel_vvp_vtiming_get_inc_count(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The VTOTAL INC COUNTER field from REG\_GENLOCK\_STATS1 register

### intel\_vvp\_vtiming\_get\_dec\_count

*prototype:*

```
int
intel_vvp_vtiming_get_dec_count(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The VTOTAL DEC COUNTER field from REG\_GENLOCK\_STATS1 register

### intel\_vvp\_vtiming\_get\_tim\_pulse{INDX}\_hstart where {INDX} goes from 0 to 7

*prototype:*

```
int
intel_vvp_vtiming_get_tim_pulse{INDX}_hstart(intel_vvp_vtiming_instance*
instance);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance

*return value:* The PULSE {INDX} HSTART field from REG\_PULSE{INDX}\_START register

### intel\_vvp\_vtiming\_set\_tim\_pulse{INDX}\_hstart

*prototype:*

```
intel_vvp_vtiming_set_tim_pulse{INDX}_hstart
(intel_vvp_vtiming_instance* instance, uint32_t in_val);
```

*argument:* instance, pointer to the intel\_vvp\_vtiming\_instance  
in\_val, sets the position of the first pixel of the pulse

*return value:* if successful kIntelVvpVtimingOk, otherwise kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_tim\_pulse{INDX}\_vstart

**prototype:**

```
int
intel_vvp_vtiming_get_tim_pulse{INDX}_vstart(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The PULSE {INDX} VSTART field from REG\_PULSE{INDX}\_START register

### intel\_vvp\_vtiming\_set\_tim\_pulse{INDX}\_vstart

**prototype:**

```
intel_vvp_vtiming_set_tim_pulse{INDX}_vstart
(intel_vvp_vtiming_instance* instance, uint32_t in_val);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

in\_val, sets the position of the first line of the pulse

**return value:** if successful kIntelVvpVtimingOk, otherwise  
kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_tim\_pulse{INDX}\_hend

**prototype:**

```
int
intel_vvp_vtiming_get_tim_pulse{INDX}_hend(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The PULSE {INDX} HEND field from REG\_PULSE{INDX}\_END register

### intel\_vvp\_vtiming\_set\_tim\_pulse{INDX}\_hend

**prototype:**

```
intel_vvp_vtiming_set_tim_pulse{INDX}_hend
(intel_vvp_vtiming_instance* instance, uint32_t in_val);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

in\_val, sets the position of the last pixel of the pulse

**return value:** if successful kIntelVvpVtimingOk, otherwise  
kIntelVvpVtimingInstanceErr

### intel\_vvp\_vtiming\_get\_tim\_pulse{INDX}\_vend

**prototype:**

```
int
intel_vvp_vtiming_get_tim_pulse{INDX}_vend(intel_vvp_vtiming_instance*
instance);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

**return value:** The PULSE {INDX} VEND field from REG\_PULSE{INDX}\_END register

### intel\_vvp\_vtiming\_set\_tim\_pulse{INDX}\_vend

**prototype:**

```
intel_vvp_vtiming_set_tim_pulse{INDX}_vend
(intel_vvp_vtiming_instance* instance, uint32_t in_val);
```

**argument:** instance, pointer to the intel\_vvp\_vtiming\_instance

in\_val, sets the position of the last line of the pulse

**return value:** if successful kIntelVvpVtimingOk, otherwise  
kIntelVvpVtimingInstanceErr

### Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38



## 52. Vignette Correction IP

### 52.1. About the Vignette Correction IP

The Vignette Correction IP compensates for nonuniform intensity across the image because of uneven light gathering limitations of the optical system (vignetting). Usually, vignetting gives a decrease in brightness from the center of the image to the corners. Drop-offs can happen independently in color channels giving also color shading (color variation from the center to the corners). Many factors cause shading: lens geometry, mechanical (aperture, lens mount), sensor size, and pixel crosstalk. The IP corrects shading by using a run-time programmable mesh of coefficients and interpolating coefficients among points for any given pixel.

The IP only supports lite variants. For more information about full and lite variants, refer to the *Altera Streaming Video Protocol Specification*

#### Related Information

[Altera Streaming Video Protocol Specification](#)

#### 52.1.1. Vignette Correction IP Performance and Resources

Altera provides resource and utilization data for guidance only. The IP resource utilization depends on the device family, compilation tool version, compilation seed randomization, and other parameters.

**Table 1064. Vignette Correction IP Performance and Resources**

With **Independent color plane correction mesh gain coefficients** on

| Device                                        | Pixel in Parallel | Bits per Color Sample | Max Resolution | CFA Enable | ALM  | M20k | DSP | f <sub>MAX</sub> (MHz) |
|-----------------------------------------------|-------------------|-----------------------|----------------|------------|------|------|-----|------------------------|
| Agilex 7 speed grade 2 (AGFA012R24A2E2V)      | 1                 | 10                    | 2048           | On         | 1404 | 9    | 7   | 750                    |
|                                               | 1                 | 10                    | 2048           | Off        | 2329 | 7    | 21  | 723                    |
|                                               | 4                 | 16                    | 8192           | On         | 3419 | 9    | 28  | 716                    |
|                                               | 4                 | 16                    | 8192           | Off        | 7692 | 7    | 84  | 613                    |
| Arria 10 speed grade 1 (10AS066H1F34E1HG)     | 1                 | 10                    | 2048           | On         | 1139 | 14   | 7   | 524                    |
|                                               | 1                 | 10                    | 2048           | Off        | 1656 | 19   | 21  | 495                    |
|                                               | 2                 | 10                    | 4096           | On         | 1600 | 19   | 14  | 518                    |
|                                               | 2                 | 10                    | 4096           | Off        | 2793 | 31   | 42  | 510                    |
| Cyclone 10 GX speed grade 5 (10CX220YF672E5G) | 1                 | 10                    | 2048           | On         | 1133 | 14   | 7   | 472                    |
|                                               | 1                 | 10                    | 2048           | Off        | 1657 | 19   | 21  | 440                    |
| continued...                                  |                   |                       |                |            |      |      |     |                        |

| Device                                      | Pixel in Parallel | Bits per Color Sample | Max Resolution | CFA Enable | ALM  | M20k | DSP | f <sub>MAX</sub> (MHz) |
|---------------------------------------------|-------------------|-----------------------|----------------|------------|------|------|-----|------------------------|
| Stratix 10 speed grade 1 (1SX280LN2F43E1VG) | 1                 | 10                    | 2048           | On         | 1426 | 9    | 7   | 583                    |
|                                             | 1                 | 10                    | 2048           | Off        | 2373 | 7    | 21  | 525                    |
|                                             | 2                 | 10                    | 4096           | On         | 2129 | 9    | 14  | 565                    |
|                                             | 2                 | 10                    | 4096           | Off        | 4225 | 7    | 42  | 535                    |

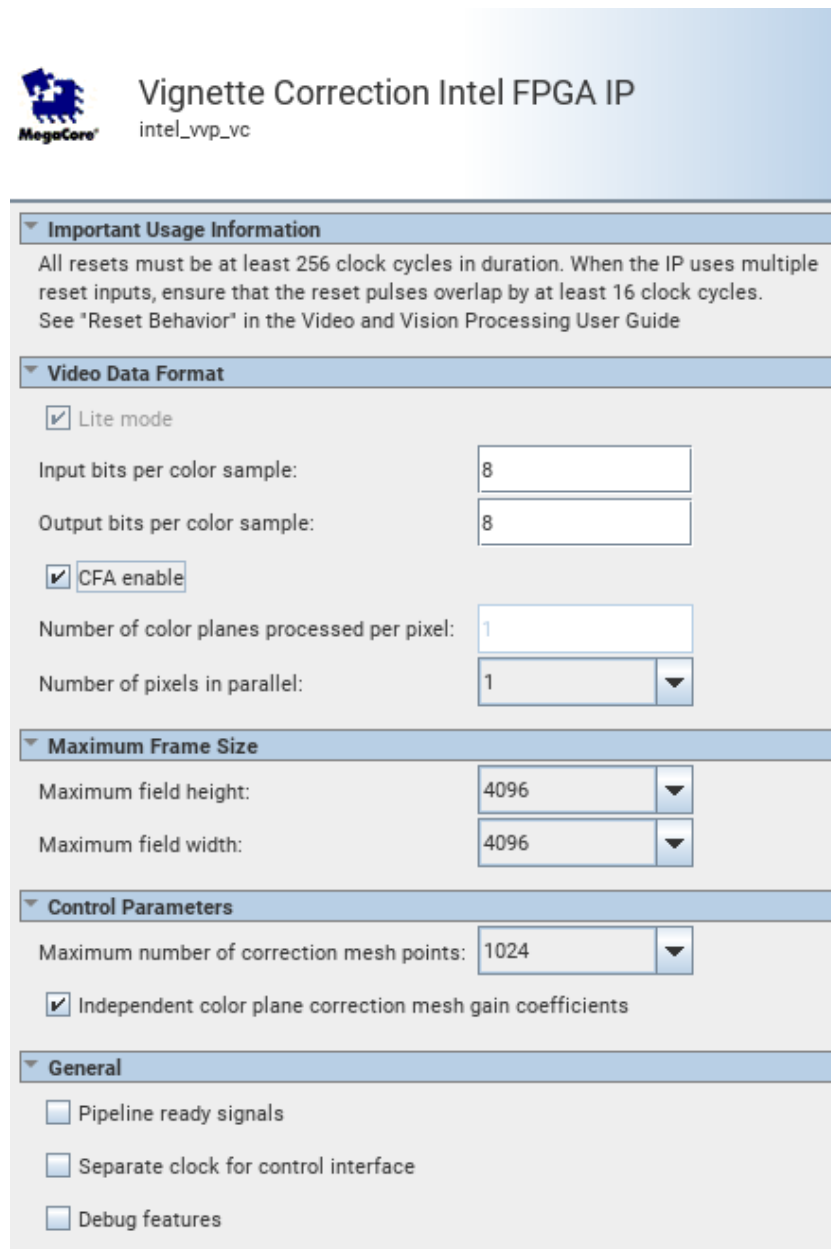
## 52.2. Vignette Correction IP Parameters

The IP offers run-time and compile-time parameters.

**Table 1065. Vignette Correction IP Parameters**

| Parameter                                                        | Values              | Description                                                                                                                                                                                                                                                                                                                                                         |
|------------------------------------------------------------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Video Data Format</b>                                         |                     |                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Lite mode</b>                                                 | On                  | The IP only supports lite mode                                                                                                                                                                                                                                                                                                                                      |
| <b>Input bits per color sample</b>                               | 8 to 16             | Select the number of input bits per color sample.                                                                                                                                                                                                                                                                                                                   |
| <b>Output bits per color sample</b>                              | 8 to 16             | Select the number of output bits per color sample.                                                                                                                                                                                                                                                                                                                  |
| <b>CFA enable</b>                                                | On or off           | Turn on to support 2x2 color filter array mapped to 1 color plane per pixel. Turn off to support simple 1 color plane and multiple color panes per pixel.                                                                                                                                                                                                           |
| <b>Number of color planes processed per pixel</b>                | 1 to 4              | Number of color planes processed per pixel. When <b>CFA enable</b> is on, this parameter is 1. Otherwise, you may select any valid value within the range.                                                                                                                                                                                                          |
| <b>Number of pixels in parallel</b>                              | 1, 2, 4, or 8       | Select the number of pixels in parallel.                                                                                                                                                                                                                                                                                                                            |
| <b>Maximum Frame Size</b>                                        |                     |                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Maximum field height</b>                                      | 1k, 2k, 4k, 8k, 16k | Specify the maximum height of incoming frames                                                                                                                                                                                                                                                                                                                       |
| <b>Maximum field width</b>                                       | 2k, 4k, 8k, 16k     | Specify the maximum width of incoming frames                                                                                                                                                                                                                                                                                                                        |
| <b>Control Parameters</b>                                        |                     |                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Maximum number of correction mesh points</b>                  | 1024, 2048, 4096    | Limits the maximum number of correction mesh points. The IP supports arbitrary V x H rectangular correction meshes, where valid values of V and H range from 8 to 512 in 1 step increments. You must set this parameter to more than or equal to the mesh size V x H you intend to use. You program the values of V and H during runtime via the control interface. |
| <b>Independent color plane correction mesh gain coefficients</b> | On or Off           | Turn on to for independent coefficient meshes for each color channel when <b>CFA enable</b> is on or off. All color channels share a single set of mesh coefficients when off.                                                                                                                                                                                      |
| <b>General</b>                                                   |                     |                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Pipeline ready signal</b>                                     | On or off           | Turn on to add extra pipeline registers to the AXI4-S <code>tready</code> signals                                                                                                                                                                                                                                                                                   |
| <b>Separate clock for control interface</b>                      | On or off           | Turn on for a separate clock for the control agent interface                                                                                                                                                                                                                                                                                                        |
| <b>Debug features</b>                                            | On or off           | Turn on to read back frame information registers and debugging information registers via the control agent interface.                                                                                                                                                                                                                                               |

Figure 141. Vignette Correction IP GUI



**Vignette Correction Intel FPGA IP**  
intel\_vvp\_vc

**Important Usage Information**  
All resets must be at least 256 clock cycles in duration. When the IP uses multiple reset inputs, ensure that the reset pulses overlap by at least 16 clock cycles. See "Reset Behavior" in the Video and Vision Processing User Guide

**Video Data Format**

☒ Lite mode

Input bits per color sample: 8

Output bits per color sample: 8

☒ CFA enable

Number of color planes processed per pixel: 1

Number of pixels in parallel: 1

**Maximum Frame Size**

Maximum field height: 4096

Maximum field width: 4096

**Control Parameters**

Maximum number of correction mesh points: 1024

☒ Independent color plane correction mesh gain coefficients

**General**

☐ Pipeline ready signals

☐ Separate clock for control interface

☐ Debug features

## 52.3. Vignette Correction IP Interfaces

**Table 1066. Vignette Correction Clock and Reset Interfaces**

| Name              | Direction | Width | Description                             |
|-------------------|-----------|-------|-----------------------------------------|
| main_clock_clk    | In        | 1     | AXI4-S processing clock.                |
| main_reset_rst    | In        | 1     | AXI4-S processing reset.                |
| agent_clock_clk   | In        | 1     | Optional control agent interface clock. |
| agent_reset_reset | In        | 1     | Optional control agent interface reset. |

**Table 1067. Vignette Correction Control Interfaces**

| Name                              | Direction | Width | Description                              |
|-----------------------------------|-----------|-------|------------------------------------------|
| av_mm_control_agent_address       | In        | 16    | Avalon memory-mapped agent address       |
| av_mm_control_agent_write         | In        | 1     | Avalon memory-mapped agent write.        |
| av_mm_control_agent_writedata     | In        | 32    | Avalon memory-mapped agent write data.   |
| av_mm_control_agent_byteenable    | In        | 4     | Avalon memory-mapped agent byte enable.  |
| av_mm_control_agent_read          | In        | 1     | Avalon memory-mapped agent read.         |
| av_mm_control_agent_readdata      | Out       | 32    | Avalon memory-mapped agent read data.    |
| av_mm_control_agent_readdatavalid | Out       | 1     | Avalon memory-mapped agent read.         |
| av_mm_control_agent_waitrequest   | Out       | 1     | Avalon memory-mapped agent wait request. |

**Table 1068. Vignette Correction Altera Streaming Video Interfaces**

| Name                 | Direction | Width | Description                                         |
|----------------------|-----------|-------|-----------------------------------------------------|
| <b>Input</b>         |           |       |                                                     |
| axi4s_vid_in_tdata   | In        | (170) | AXI4-S data in.                                     |
| axi4s_vid_in_tvalid  | In        | 1     | AXI4-S data valid.                                  |
| axi4s_vid_in_tuser   | In        | (171) | [0] AXI4-S start of video frame.<br>[N-1:1] Unused. |
| axi4s_vid_in_tlast   | In        | 1     | AXI4-S end of packet.                               |
| axi4s_vid_in_tready  | Out       | 1     | AXI4-S data ready.                                  |
| <b>Output</b>        |           |       |                                                     |
| axi4s_vid_out_tdata  | Out       | (170) | AXI4-S data out.                                    |
| axi4s_vid_out_tvalid | Out       | 1     | AXI4-S data valid.                                  |
| axi4s_vid_out_tuser  | Out       | (171) | [0] AXI4-S start of video frame.                    |
| <i>continued...</i>  |           |       |                                                     |

(170) The equation gives all tdata widths sizes in these interfaces: pixels in parallel × max (round (bits per color sample × number of color planes, 8), 16)

(171) This equation gives all tuser widths sizes in these interfaces:  $N = \text{ceil}(\text{tdata width} / 8)$

| Name                 | Direction | Width | Description           |
|----------------------|-----------|-------|-----------------------|
|                      |           |       | [N-1:1] Unused.       |
| axi4s_vid_out_tlast  | Out       | 1     | AXI4-S end of packet. |
| axi4s_vid_out_tready | In        | 1     | AXI4-S data ready     |

### Related Information

Video and Vision Processing IP Interfaces on page 30

## 52.4. Vignette Correction IP Registers

Each register is either read-only (RO) or read-write (RW). Tables are write-only (WO).

**Table 1069. Vignette Correction IP Registers**

In the software API, the register names appear with a prefix of INTEL\_VVP, INTEL\_VVP\_CORE or INTEL\_VVP\_VC as appropriate and with an optional REG suffix

| Address                    | Register      | Access        |      | Description                                                                                                       |
|----------------------------|---------------|---------------|------|-------------------------------------------------------------------------------------------------------------------|
|                            |               | Lite<br>(172) | Full |                                                                                                                   |
| Parameterization registers |               |               |      |                                                                                                                   |
| 0x00000                    | VID_PID       | RO            | N/A  | Read this register to retrieve the ID of the IP.<br>This register always returns 0x6FA7_0178.                     |
| 0x00004                    | VERSION       | RO            | N/A  | Read this register to retrieve the version information for the IP.                                                |
| 0x00008                    | LITE_MODE     | RO            | N/A  | Read this register to determine if <b>lite mode</b> is on.<br>This register always returns 1.                     |
| 0x0000c                    | DEBUG_ENABLED | RO            | N/A  | Read this register to determine if <b>Debug features</b> are on.<br>This register returns 0 for off and 1 for on. |
| 0x00010                    | BPS_IN        | RO            | N/A  | Read this register to determine the bits per symbol for the input data.                                           |
| 0x00014                    | BPS_OUT       | RO            | N/A  | Read this register to determine the bits per symbol for the output data.                                          |
| 0x00018                    | NUM_COLOR_IN  | RO            | N/A  | Read this register to determine the number of color planes for the input data.                                    |
| 0x0001c                    | NUM_COLOR_OUT | RO            | N/A  | Read this register to determine the number of color planes for the output data.                                   |
| 0x00020                    | PIP           | RO            | N/A  | Read this register to determine the number of pixels in parallel.                                                 |
| 0x00024                    | MAX_WIDTH     | RO            | N/A  | Read this register to determine the maximum supported input field width.                                          |
| 0x00028                    | MAX_HEIGHT    | RO            | N/A  | Read this register to determine the maximum supported input field height.                                         |
| continued...               |               |               |      |                                                                                                                   |

(172) In **lite mode**, registers are RW only if you also turn on **Debug features**, otherwise they are WO.

| Address                                         | Register              | Access        |      | Description                                                                                                                                                                                                                        |
|-------------------------------------------------|-----------------------|---------------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                 |                       | Lite<br>(172) | Full |                                                                                                                                                                                                                                    |
| 0x0002c                                         | CFA_ENABLE            | RO            | N/A  | Read this register to determine if <b>Color filter array</b> is on. When on, the IP operates with a 2x2 color filter array with 4 color channels. When off, the IP operates with 1 color plane and multiple color panes per pixel. |
| 0x00030                                         | MAX_GAIN_MESH_POINTS  | RO            | N/A  | Read this register to determine the maximum number of mesh points.                                                                                                                                                                 |
| 0x00034                                         | PER_COLOR_GAIN_ENABLE | RO            | N/A  | Read this register to determine the per color or shared coefficient tables.                                                                                                                                                        |
| 0x00038 to 0x0011F                              | -                     | -             | -    | Reserved                                                                                                                                                                                                                           |
| <b>Control, debug, and statistics registers</b> |                       |               |      |                                                                                                                                                                                                                                    |
| 0x00120                                         | IMG_INFO_WIDTH        | RW            | N/A  | The expected width of the incoming video fields.                                                                                                                                                                                   |
| 0x00124                                         | IMG_INFO_HEIGHT       | RW            | N/A  | The expected height of the incoming video fields.                                                                                                                                                                                  |
| 0x00128 to 0x0013F                              | -                     | -             | -    | Reserved                                                                                                                                                                                                                           |
| 0x00140                                         | STATUS                | RO            | N/A  | Read this register for information about the IP status.                                                                                                                                                                            |
| 0x00144                                         | FRAME_STATS           | RO            | N/A  | Read this register for some frame statistics.                                                                                                                                                                                      |
| 0x00148                                         | COMMIT                | RW            | N/A  | Write to this register to submit changes to the control and setup registers.                                                                                                                                                       |
| 0x0014C                                         | CONTROL               | RW            | N/A  | Control bits and fields of the IP                                                                                                                                                                                                  |
| 0x00150                                         | BLOCK_PIX_COUNT       | RW            | N/A  | Number of pixel columns in a base zone                                                                                                                                                                                             |
| 0x00154                                         | BLOCK_LINE_COUNT      | RW            | N/A  | Number of pixel lines in a base zone                                                                                                                                                                                               |
| 0x00158                                         | H_NUM_BLOCKS          | RW            | N/A  | Number of horizontal zones                                                                                                                                                                                                         |
| 0x0015C                                         | V_NUM_BLOCKS          | RW            | N/A  | Number of vertical zones                                                                                                                                                                                                           |
| 0x00160                                         | H_RAMP_FRAC_BITS      | RW            | N/A  | Fractional part of the horizontal ramp increments for a base zone size                                                                                                                                                             |
| 0x00164                                         | H_RAMP_P1_FRAC_BITS   | RW            | N/A  | Fractional part of the horizontal ramp increments for an extended zone size                                                                                                                                                        |
| 0x00168                                         | H_RAMP_M1_FRAC_BITS   | RW            | N/A  | Fractional part of the horizontal ramp increments for the last column of zones                                                                                                                                                     |
| 0x0016C                                         | V_RAMP_FRAC_BITS      | RW            | N/A  | Fractional part of the vertical ramp increments for a base zone size                                                                                                                                                               |
| 0x00170                                         | V_RAMP_P1_FRAC_BITS   | RW            | N/A  | Fractional part of the vertical ramp increments for an extended zone size                                                                                                                                                          |
| 0x00174                                         | V_RAMP_M1_FRAC_BITS   | RW            | N/A  | Fractional part of the vertical ramp increments for the last line of zones                                                                                                                                                         |
| 0x00178 to                                      | -                     | -             | -    | Reserved                                                                                                                                                                                                                           |
| <b>continued...</b>                             |                       |               |      |                                                                                                                                                                                                                                    |

(172) In **lite mode**, registers are RW only if you also turn on **Debug features**, otherwise they are WO.

| Address                            | Register            | Access        |      | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------------------|---------------------|---------------|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                    |                     | Lite<br>(172) | Full |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 0x001FF                            |                     |               |      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 0x00200 to<br>0x00200+M-1<br>(173) | CP0_MESH_GAIN_TABLE | WO            | N/A  | The IP uses this table for all color channels if <b>Independent color plane correction mesh gain coefficients</b> parameter is off.                                                                                                                                                                                                                                                                                                                                                                                             |
| 0x00200+M to<br>0x041ff<br>(173)   | -                   | -             | -    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 0x04200 to<br>0x04200+M-1<br>(173) | CP1_MESH_GAIN_TABLE | WO            | N/A  | The IP uses this table only if <b>Independent color plane correction mesh gain coefficients</b> is on and color channel 1 exists. Otherwise, all registers in this table are reserved.                                                                                                                                                                                                                                                                                                                                          |
| 0x04200+M to<br>0x081ff<br>(173)   | -                   | -             | -    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 0x08200 to<br>0x08200+M-1<br>(173) | CP2_MESH_GAIN_TABLE | WO            | N/A  | The IP uses this table only if <b>Independent color plane correction mesh gain coefficients</b> is on and color channel 2 exists. Otherwise, all registers in this table are reserved.                                                                                                                                                                                                                                                                                                                                          |
| 0x08200+M to<br>0x0C1ff<br>(173)   | -                   | -             | -    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 0x0C200<br>to 0x0C200+M-1<br>(173) | CP3_MESH_GAIN_TABLE | WO            | N/A  | The IP uses this table only if <b>Independent color plane correction mesh gain coefficients</b> is on and color channel 3 exists. Otherwise, all registers in this table are reserved.                                                                                                                                                                                                                                                                                                                                          |
| 0x0C200+M to<br>0x101ff<br>(173)   | -                   | -             | -    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 0x10200<br>to<br>0x10200+M-1(173)  | STEP_LUT_TABLE      | WO            | N/A  | This is a complementary table shared across all color channels, enabling the IP to achieve fine grained precision at the corners and edges of the mesh boundaries.<br><br>The IP expects V x H mesh entries to be stored in raster order, left to right and top to bottom. The number of entries V x H must remain less than or equal to the <b>Maximum number of correction mesh points</b> parameter.<br><br>The top left mesh point of any given mesh box is the reference table entry point for the V and H step functions. |
| 0x10200+M to<br>0x201FF<br>(173)   | -                   | -             | -    | Reserved                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

(172) In **lite mode**, registers are RW only if you also turn on **Debug features**, otherwise they are WO.

(173)  $M = 4 \times$  **Maximum number of correction mesh points**

## Register Bit Descriptions

**Table 1070. STATUS**

| Name     | Bits | Description                         |
|----------|------|-------------------------------------|
| Reserved | 31:2 | Reserved                            |
| Commit   | 1    | Commit is pending                   |
| Running  | 0    | When 1, the IP is processing data.. |

**Table 1071. FRAME\_STATS**

| Name     | Bits | Description                     |
|----------|------|---------------------------------|
| Reserved | 31:8 | Reserved                        |
| Checksum | 7:0  | A simple checksum of the frame. |

**Table 1072. CONTROL**

| Name                      | Bits | Description                                                                                                                                                                                                                                                                                                                                                           |
|---------------------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved                  | 31:3 | Reserved. Write 0.                                                                                                                                                                                                                                                                                                                                                    |
| Color filter Array Phase. | 2:1  | Specifies 2x2 color filter order starting from the top left corner of the image.<br><div> <div>00 01 10 11</div> <div>C<sub>0</sub>C<sub>1</sub> C<sub>1</sub>C<sub>0</sub> C<sub>2</sub>C<sub>3</sub> C<sub>3</sub>C<sub>2</sub></div> <div>C<sub>2</sub>C<sub>3</sub> C<sub>3</sub>C<sub>2</sub> C<sub>0</sub>C<sub>1</sub> C<sub>1</sub>C<sub>0</sub></div> </div> |
| Bypass                    | 0    | Set to bypass the IP. When set the IP passes the input image unmodified.                                                                                                                                                                                                                                                                                              |

**Table 1073. BLOCK\_PIX\_COUNT**

| Name     | Bits  | Description                                                                                                                                                                                                                                                                                                                                   |
|----------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved | 31:16 | Reserved.                                                                                                                                                                                                                                                                                                                                     |
| Value    | 15:0  | The mesh box boundary in pixels. The IP determines when to load the next mesh gain coefficient while moving in the horizontal direction by reading this register. The value of this register must be:<br>$\text{floor}((\text{Frame Width} / \text{H\_NUM\_BLOCKS}) / \text{PIP})$ .<br>You can use the H Step LUT to make up the remainders. |

**Table 1074. BLOCK\_LINE\_COUNT**

| Name     | Bits  | Description                                                                                                                                                                                                                                                                                                                                     |
|----------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved | 31:16 | Reserved.                                                                                                                                                                                                                                                                                                                                       |
| Value    | 15:0  | The mesh box boundary in lines. The IP determines when to load the next mesh gain coefficient while moving in the vertical direction by reading this register. The value of this register must be:<br>$\text{floor}(\text{Frame Height} / \text{V\_NUM\_BLOCKS})$<br>You can use the V Step LUT to make up the remainders for a given mesh box. |



Table 1075. H\_NUM\_BLOCKS

| Name     | Bits  | Description                                                                                                                                                                                                                                                                                                                                                 |
|----------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved | 31:16 | Reserved.                                                                                                                                                                                                                                                                                                                                                   |
| Value    | 15:0  | The number of mesh boxes in the horizontal direction, which is the number of horizontal mesh points minus 1. The IP supports arbitrary V x H rectangular mesh blocks, where valid values of H range from 8 to 512 in 1 step increments. The value of V x H must remain less than or equal to the <b>Maximum number of correction mesh points</b> parameter. |

Table 1076. V\_NUM\_BLOCKS

| Name     | Bits  | Description                                                                                                                                                                                                                                                                                                                                             |
|----------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved | 31:16 | Reserved.                                                                                                                                                                                                                                                                                                                                               |
| Value    | 15:0  | The number of mesh boxes in the vertical direction, which is the number of vertical mesh points minus 1. The IP supports arbitrary V x H rectangular mesh blocks, where valid values of V range from 8 to 512 in 1 step increments. The value of V x H must remain less than or equal to the <b>Maximum number of correction mesh points</b> parameter. |

Table 1077. H\_RAMP\_FRAC\_BITS

| Name     | Bits  | Description                                                                                                                                                                                          |
|----------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved | 31:22 | Reserved.                                                                                                                                                                                            |
| Value    | 21:0  | The horizontal ramp fractional bits. This is the fraction of corrective gain for each horizontal pixel step within a mesh box. The value of this register must be:<br>$1 / \text{BLOCK\_PIX\_COUNT}$ |

Table 1078. H\_RAMP\_P1\_FRAC\_BITS

| Name     | Bits  | Description                                                                                                                                                                                                                                            |
|----------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved | 31:22 | Reserved.                                                                                                                                                                                                                                              |
| Value    | 21:0  | An alternative horizontal ramp fractional bits register for removing reminders. The IP uses this register when the H Step LUT indicates a +1 version for a given mesh box. The value of this register must be:<br>$1 / (\text{BLOCK\_PIX\_COUNT} + 1)$ |

Table 1079. H\_RAMP\_M1\_FRAC\_BITS

| Name     | Bits  | Description                                                                                                                                                                                                                                                                                                                                                                    |
|----------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved | 31:22 | Reserved.                                                                                                                                                                                                                                                                                                                                                                      |
| Value    | 21:0  | A third alternative horizontal ramp fractional bits register, for when the IP is processing the final mesh column. It allows fine alignment to the top right mesh gain value. The value of this register must be:<br>$((1 / (\text{BLOCK\_PIX\_COUNT} - 1)) - \text{H\_RAMP\_FRAC\_BITS}) / \text{PIP} + \text{H\_RAMP\_FRAC\_BITS} - k$<br>The k is 1 for PIP>1, otherwise 0. |

Table 1080. V\_RAMP\_FRAC\_BITS

| Name     | Bits  | Description                                                                                                                                                                                     |
|----------|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved | 31:22 | Reserved.                                                                                                                                                                                       |
| Value    | 21:0  | The vertical ramp fractional bits. It is the fraction of corrective gain for each vertical pixel step within a mesh box. The value of this register must be:<br>$1 / \text{BLOCK\_LINE\_COUNT}$ |

**Table 1081. V\_RAMP\_P1\_FRAC\_BITS**

| Name     | Bits  | Description                                                                                                                                                                                                                                        |
|----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved | 31:22 | Reserved.                                                                                                                                                                                                                                          |
| Value    | 21:0  | An alternative vertical ramp fractional bits register for removing reminders. The IP uses this register when the V Step LUT indicates a +1 version for a given mesh box. The value of this register must be: $1 / (\text{BLOCK\_LINE\_COUNT} + 1)$ |

**Table 1082. V\_RAMP\_M1\_FRAC\_BITS**

| Name     | Bits  | Description                                                                                                                                                                                                                                       |
|----------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved | 31:22 | Reserved.                                                                                                                                                                                                                                         |
| Value    | 21:0  | A third alternative vertical ramp fractional bits register, for when the IP is processing the final mesh line. It allows fine alignment to the bottom mesh gain values. The value of this register must be: $1 / (\text{BLOCK\_LINE\_COUNT} - 1)$ |

**Table 1083. CP0\_MESH\_GAIN\_TABLE**

| Name        | Bits  | Description                                                                                                                                                                                                                                                                                         |
|-------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved    | 31:19 | Reserved.                                                                                                                                                                                                                                                                                           |
| Coefficient | 18:0  | Unsigned 8.11 fixed point mesh coefficient table for color channel 0. The IP expects V x H mesh entries to be stored in raster order, left to right and top to bottom. The number of entries V x H must remain less than or equal to the <b>Maximum number of correction mesh points</b> parameter. |

**Table 1084. CP1\_MESH\_GAIN\_TABLE**

| Name        | Bits  | Description                                                                                                                                                                                                                                                                                         |
|-------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved    | 31:19 | Reserved.                                                                                                                                                                                                                                                                                           |
| Coefficient | 18:0  | Unsigned 8.11 fixed point mesh coefficient table for color channel 1. The IP expects V x H mesh entries to be stored in raster order, left to right and top to bottom. The number of entries V x H must remain less than or equal to the <b>Maximum number of correction mesh points</b> parameter. |

**Table 1085. CP2\_MESH\_GAIN\_TABLE**

| Name        | Bits  | Description                                                                                                                                                                                                                                                                                         |
|-------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved    | 31:19 | Reserved.                                                                                                                                                                                                                                                                                           |
| Coefficient | 18:0  | Unsigned 8.11 fixed point mesh coefficient table for color channel 2. The IP expects V x H mesh entries to be stored in raster order, left to right and top to bottom. The number of entries V x H must remain less than or equal to the <b>Maximum number of correction mesh points</b> parameter. |

**Table 1086. CP3\_MESH\_GAIN\_TABLE**

| Name        | Bits  | Description                                                                                                                                                                                                                                                                                         |
|-------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved    | 31:19 | Reserved.                                                                                                                                                                                                                                                                                           |
| Coefficient | 18:0  | Unsigned 8.11 fixed point mesh coefficient table for color channel 3. The IP expects V x H mesh entries to be stored in raster order, left to right and top to bottom. The number of entries V x H must remain less than or equal to the <b>Maximum number of correction mesh points</b> parameter. |

**Table 1087. STEP\_LUT\_TABLE**

| Name                                  | Bits  | Description                                                                                                                                                                                                     |
|---------------------------------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved                              | 31:17 | Reserved.                                                                                                                                                                                                       |
| H comp enable for the PIP instance 8. | 16    | Reserved if PIP<8.<br>(174)                                                                                                                                                                                     |
| H step enable for the PIP instance 8. | 15    | Reserved if PIP<8.<br>(175)                                                                                                                                                                                     |
| H comp enable for the PIP instance 7. | 14    | Reserved if PIP<8.<br>(174)                                                                                                                                                                                     |
| H step enable for the PIP instance 7. | 13    | Reserved if PIP<8.<br>(175)                                                                                                                                                                                     |
| H comp enable for the PIP instance 6. | 12    | Reserved if PIP<8.<br>(174)                                                                                                                                                                                     |
| H step enable for the PIP instance 6. | 11    | Reserved if PIP<8.<br>(175)                                                                                                                                                                                     |
| H comp enable for the PIP instance 5. | 10    | Reserved if PIP<8.<br>(174)                                                                                                                                                                                     |
| H step enable for the PIP instance 5. | 9     | Reserved if PIP<8.<br>(175)                                                                                                                                                                                     |
| H comp enable for the PIP instance 4. | 8     | Reserved if PIP<4.<br>(174)                                                                                                                                                                                     |
| H step enable for the PIP instance 4. | 7     | Reserved if PIP<4.<br>(175)                                                                                                                                                                                     |
| H comp enable for the PIP instance 3. | 6     | Reserved if PIP<4.<br>(174)                                                                                                                                                                                     |
| H step enable for the PIP instance 3. | 5     | Reserved if PIP<4.<br>(175)                                                                                                                                                                                     |
| H comp enable for the PIP instance 2. | 4     | Reserved if PIP<2.<br>(174)                                                                                                                                                                                     |
| H step enable for the PIP instance 2. | 3     | Reserved if PIP<2.<br>(175)                                                                                                                                                                                     |
| H comp enable                         | 2     | Compensates for the fractional pixel distances from the last mesh point for PIP instance 1.<br>(174)                                                                                                            |
| H step enable.                        | 1     | For this mesh point, adds 1 to the pixel count for PIP instance 1, effectively stretching the mesh box by PIP pixels. You must set all entries across a mesh column to the same value for consistency.<br>(175) |
| V step enable                         | 0     | For this mesh point, adds 1 to the line count, stretching the mesh box by 1 pixel line. You must set all entries across a mesh line to the same value for consistency.                                          |

- (174) The IP uses H comp enable to compensate for fractional precision when the width of the frame is not an integer multiple of (H\_NUM\_BLOCKS x PIP). You must set H comp enable to 0 by default. If a final remainder that is not divisible by PIP exists, in the final mesh box column you must set H comp enable to 1 for all PIP instances.

## 52.5. Vignette Correction IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_vc_regs.h`

Include file: `intel_vvp_vc.h`

**Table 1088. Vignette Correction IP Software API**

| Name                                                | Description                                                                                                      |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <code>intel_vvp_vc_init</code>                      | Initialize an IP instance                                                                                        |
| <code>intel_vvp_core_*</code>                       | Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i>                           |
| <code>intel_vvp_vc_get_lite_mode</code>             | Returns if <b>Lite mode</b> is on                                                                                |
| <code>intel_vvp_vc_get_debug_enabled</code>         | Returns if <b>Debug features</b> is on                                                                           |
| <code>intel_vvp_vc_get_bits_per_sample_in</code>    | Returns the number of bits per color sample for the streaming input interface                                    |
| <code>intel_vvp_vc_get_bits_per_sample_out</code>   | Returns the number of bits per color sample value for the streaming output interface                             |
| <code>intel_vvp_vc_get_num_color_planes_in</code>   | Returns the number of color planes of the streaming input interface                                              |
| <code>intel_vvp_vc_get_num_color_planes_out</code>  | Returns the number of color planes of the streaming output interface                                             |
| <code>intel_vvp_vc_get_pixels_in_parallel</code>    | Returns the pixels in parallel of the streaming input and output interfaces                                      |
| <code>intel_vvp_vc_get_max_width</code>             | Returns the maximum number of pixels that the IP supports on the horizontal dimension of an image or video frame |
| <code>intel_vvp_vc_get_max_height</code>            | Returns the maximum number of pixels that the IP supports on the vertical dimension of an image or video frame   |
| <code>intel_vvp_vc_get_cfa_enable</code>            | Returns if the color filter array mode is enabled                                                                |
| <code>intel_vvp_vc_get_max_gain_mesh_points</code>  | Returns the maximum number of gain mesh points                                                                   |
| <code>intel_vvp_vc_get_per_color_gain_enable</code> | Returns if each color channel has its own gain mesh                                                              |
| <code>intel_vvp_vc_is_running</code>                | Reads if the IP is running                                                                                       |
| <code>intel_vvp_vc_commit_is_pending</code>         | Reads if the IP has uncommitted writes                                                                           |
| <code>intel_vvp_vc_get_status</code>                | Reads the status register                                                                                        |
| continued...                                        |                                                                                                                  |

- (175) The IP uses H step enable to remove remainders. When the remainder is greater than or equal to PIP, you must set H step enable for a mesh box column of choice for all PIP instances. However, Altera recommends distributing those mesh box columns across the width of the frame uniformly. Continue until the remainder is less than PIP, where you must set H step enable only for the remaining pixels in the final mesh box column.

| Name                              | Description                                                                                                                       |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| intel_vvp_vc_get_frame_stats      | Reads the frame statistics register                                                                                               |
| intel_vvp_vc_commit               | Commit pending writes                                                                                                             |
| intel_vvp_vc_get_bypass           | Reads the bypass bit from the status register                                                                                     |
| intel_vvp_vc_set_bypass           | Writes the bypass bit of the status register                                                                                      |
| intel_vvp_vc_get_cfa_phase        | Reads the color filter array phase field from the status register                                                                 |
| intel_vvp_vc_set_cfa_phase        | Writes the color filter array phase field of the status register                                                                  |
| intel_vvp_vc_get_block_pix_count  | Reads the number of pixel columns of the mesh box boundary                                                                        |
| intel_vvp_vc_set_block_pix_count  | Writes the number of pixel columns of the mesh box boundary                                                                       |
| intel_vvp_vc_get_block_line_count | Reads the number of lines of the mesh box boundary                                                                                |
| intel_vvp_vc_set_block_line_count | Writes the number of lines of the mesh box boundary                                                                               |
| intel_vvp_vc_get_h_num_blocks     | Reads the number of mesh boxes on the horizontal dimension                                                                        |
| intel_vvp_vc_set_h_num_blocks     | Writes the number of mesh boxes on the horizontal dimension                                                                       |
| intel_vvp_vc_get_v_num_blocks     | Reads the number of mesh boxes on the vertical dimension                                                                          |
| intel_vvp_vc_set_v_num_blocks     | Writes the number of mesh boxes on the vertical dimension                                                                         |
| intel_vvp_vc_get_h_ramp_frac      | Reads the fraction of corrective gain for each horizontal pixel step within a mesh box                                            |
| intel_vvp_vc_set_h_ramp_frac      | Writes the fraction of corrective gain for each horizontal pixel step within a mesh box                                           |
| intel_vvp_vc_get_h_ramp_pl_frac   | Reads the next fraction of corrective gain for each horizontal pixel step within a mesh box contributed from the next mesh point  |
| intel_vvp_vc_set_h_ramp_pl_frac   | Writes the next fraction of corrective gain for each horizontal pixel step within a mesh box contributed from the next mesh point |
| intel_vvp_vc_get_h_ramp_ml_frac   | Reads the fraction of corrective gain for each horizontal pixel step within the last mesh box                                     |
| intel_vvp_vc_set_h_ramp_ml_frac   | Writes the fraction of corrective gain for each horizontal pixel step within the last mesh box                                    |
| intel_vvp_vc_get_v_ramp_frac      | Reads the fraction of corrective gain for each vertical pixel step within a mesh box                                              |
| intel_vvp_vc_set_v_ramp_frac      | Writes the fraction of corrective gain for each vertical pixel step within a mesh box                                             |
| intel_vvp_vc_get_v_ramp_pl_frac   | Reads the next fraction of corrective gain for each vertical pixel step within a mesh box contributed from the next mesh point    |
| intel_vvp_vc_set_v_ramp_pl_frac   | Writes the next fraction of corrective gain for each vertical pixel step within a mesh box contributed from the next mesh point   |
| intel_vvp_vc_get_v_ramp_ml_frac   | Reads the fraction of corrective gain for each vertical pixel step within the last mesh box                                       |
| intel_vvp_vc_set_v_ramp_ml_frac   | Writes the fraction of corrective gain for each vertical pixel step within the last mesh box                                      |
| intel_vvp_vc_update_cp_lut        | Writes the color plane lookup table                                                                                               |
| intel_vvp_vc_update_step_lut      | Reads the step lookup table                                                                                                       |

## intel\_vvp\_vc\_init

### Prototype

```
int
intel_vvp_vc_init(intel_vvp_vc_instance* instance,
intel_vvp_core_base base);
```

**Description** Initialize an IP instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

base - base address of the register map

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7)

kIntelVvpCorePidErr if the product\_id does not match with the expected product ID (0x0178)

kIntelVvpVcRegMapVersionErr if the register map is not supported

## intel\_vvp\_vc\_get\_lite\_mode

### Prototype

```
bool
intel_vvp_vc_get_lite_mode(intel_vvp_vc_instance* instance);
```

**Description** Returns the value of the LITE\_MODE register. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

**Returns** true if the IP is parameterized in lite mode

### intel\_vvp\_vc\_get\_debug\_enabled

*Prototype*

```
bool
intel_vvp_vc_get_debug_enabled(intel_vvp_vc_instance*
instance);
```

*Description* Returns the value of the DEBUG\_ENABLED register. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

*Returns* true if the IP is parameterized with debug features enabled

### intel\_vvp\_vc\_get\_bits\_per\_sample\_in

*Prototype*

```
uint8_t
intel_vvp_vc_get_bits_per_sample_in(intel_vvp_vc_instance*
instance);
```

*Description* Returns the value of the BPS\_IN register. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

*Returns* the input bits per color sample parameter used to generate the IP

### intel\_vvp\_vc\_get\_bits\_per\_sample\_out

*Prototype*

```
uint8_t
intel_vvp_vc_get_bits_per_sample_out(intel_vvp_vc_instance*
instance);
```

*Description* Returns the value of the BPS\_OUT register. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

*Returns* the output bits per color sample parameter used to generate the IP

### intel\_vvp\_vc\_get\_num\_color\_planes\_in

#### Prototype

```
uint8_t
intel_vvp_vc_get_num_color_planes_in(intel_vvp_vc_instance*
instance);
```

**Description** Returns the value of the NUM\_COLOR\_IN register. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

**Returns** the input number of color planes parameter used to generate the IP

### intel\_vvp\_vc\_get\_num\_color\_planes\_out

#### Prototype

```
uint8_t
intel_vvp_vc_get_num_color_planes_out(intel_vvp_vc_instance*
instance);
```

**Description** Returns the value of the NUM\_COLOR\_OUT register. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

**Returns** the output number of color planes parameter used to generate the IP

### intel\_vvp\_vc\_get\_pixels\_in\_parallel

#### Prototype

```
uint8_t
intel_vvp_vc_get_pixels_in_parallel(intel_vvp_vc_instance*
instance);
```

**Description** Returns the value of the PIXELS\_IN\_PARALLEL register. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

**Returns** the pixels in parallel parameter used to generate the IP



### intel\_vvp\_vc\_get\_max\_width

*Prototype*

```
uint32_t
intel_vvp_vc_get_max_width(intel_vvp_vc_instance* instance);
```

*Description* Returns the value of the MAX\_WIDTH register. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

*Returns* the maximum field width parameter used to generate the IP

### intel\_vvp\_vc\_get\_max\_height

*Prototype*

```
uint32_t
intel_vvp_vc_get_max_height(intel_vvp_vc_instance* instance);
```

*Description* Returns the value of the MAX\_HEIGHT register. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

*Returns* the maximum field height parameter used to generate the IP

### intel\_vvp\_vc\_get\_cfa\_enable

*Prototype*

```
bool
intel_vvp_vc_get_cfa_enable(intel_vvp_vc_instance* instance);
```

*Description* Returns the value of the CFA\_ENABLE register. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

*Returns* the color filter array enable parameter used to generate the IP

### intel\_vvp\_vc\_get\_max\_gain\_mesh\_points

*Prototype*

```
uint16_t
intel_vvp_vc_get_max_gain_mesh_points(intel_vvp_vc_instance* instance);
```

**Description** Returns the value of the MAX\_GAIN\_MESH\_POINTS register. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

**Returns** the maximum number of mesh points parameter used to generate the IP

#### intel\_vvp\_vc\_get\_per\_color\_gain\_enable

##### Prototype

```
bool
intel_vvp_vc_get_per_color_gain_enable(intel_vvp_vc_instance*
instance);
```

**Description** Returns the value of the PER\_COLOR\_GAIN\_ENABLE register. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

**Returns** the per color gain enable parameter used to generate the IP

#### intel\_vvp\_vc\_is\_running

##### Prototype

```
bool
intel_vvp_vc_is_running(intel_vvp_vc_instance* instance);
```

**Description** Returns the value of the IP Running bit from the STATUS register. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

**Returns** the running bit from the status register

#### intel\_vvp\_vc\_commit\_is\_pending

##### Prototype

```
bool
intel_vvp_vc_commit_is_pending(intel_vvp_vc_instance*
instance);
```

**Description** Returns the value of the commit pending bit from the STATUS register. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

*Returns* true if the agent interface has pending writes from a previous commit request

### intel\_vvp\_vc\_get\_status

*Prototype*

```
uint32_t
intel_vvp_vc_get_status(intel_vvp_vc_instance* instance);
```

*Description* Returns the value of the STATUS register. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

*Returns* the value of the status register

### intel\_vvp\_vc\_get\_frame\_stats

*Prototype*

```
int
intel_vvp_vc_get_frame_stats(intel_vvp_vc_instance* instance,
uint32_t* stats_out);
```

*Description* Reads the value of the FRAME\_STATS register. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

stats\_out - pointer of a variable to return the statistics register value

*Returns* kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpVcPointerErr if stats\_out is a null pointer

### intel\_vvp\_vc\_commit

*Prototype*

```
int
intel_vvp_vc_commit(intel_vvp_vc_instance* instance);
```

|                    |                                                                                                                                                     |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i> | Commits all pending writes before processing the next field. The instance must be a valid intel_vvp_vc_instance fully initialized.                  |
| <i>Arguments</i>   | instance - pointer to the intel_vvp_vc_instance software driver instance structure                                                                  |
| <i>Returns</i>     | kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error<br><br>kIntelVvpCoreInstanceErr if the instance is a null pointer |

### intel\_vvp\_vc\_get\_bypass

#### Prototype

```
bool
intel_vvp_vc_get_bypass(intel_vvp_vc_instance* instance);
```

|                    |                                                                                                                                      |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i> | Returns the value of the bypass bit from the CONTROL register. The instance must be a valid intel_vvp_vc_instance fully initialized. |
| <i>Arguments</i>   | instance - pointer to the intel_vvp_vc_instance software driver instance structure                                                   |
| <i>Returns</i>     | the value of the bypass bit form the control register                                                                                |

### intel\_vvp\_vc\_set\_bypass

#### Prototype

```
int
intel_vvp_vc_set_bypass(intel_vvp_vc_instance* instance, bool
bypass);
```

|                    |                                                                                                                                                                                                                                |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i> | Writes to the bypass bit of the CONTROL register. The instance must be a valid intel_vvp_vc_instance fully initialized.                                                                                                        |
| <i>Arguments</i>   | instance - pointer to the intel_vvp_vc_instance software driver instance structure<br><br>bypass - new value of the bypass bit                                                                                                 |
| <i>Returns</i>     | kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error<br><br>kIntelVvpCoreInstanceErr if the instance is a null pointer<br><br>kIntelVvpVcCommitPendingErr if a previous commit request is pending |

### intel\_vvp\_vc\_get\_cfa\_phase

#### Prototype

```
uint8_t
intel_vvp_vc_get_cfa_phase(intel_vvp_vc_instance* instance);
```

**Description** Returns the value of the color filter array phase field from the CONTROL register. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

**Returns** the value of the color filter array phase field form the control register

### intel\_vvp\_vc\_set\_cfa\_phase

#### Prototype

```
int
intel_vvp_vc_set_cfa_phase(intel_vvp_vc_instance* instance,
uint8_t cfa_phase);
```

**Description** Writes to the color filter array phase field of the CONTROL register. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

cfa\_phase - new value of the color filter array phase field

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpVcValueErr if the value of cfa\_phase is out of range

kIntelVvpVcCommitPendingErr if a previous commit request is pending

### intel\_vvp\_vc\_get\_block\_pix\_count

#### Prototype

```
uint16_t
intel_vvp_vc_get_block_pix_count(intel_vvp_vc_instance*
instance);
```

**Description** Returns the number of pixel columns of the mesh box boundary. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

**Returns** the number of pixel columns of the mesh box boundary

### intel\_vvp\_vc\_set\_block\_pix\_count

**Prototype**

```
int
intel_vvp_vc_set_block_pix_count(intel_vvp_vc_instance*
instance, uint16_t block_pix_count);
```

**Description** Writes the number of pixel columns of the mesh box boundary. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

block\_pix\_count - new number of pixel columns of the mesh box boundary

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpVcValueErr if the value of block\_pix\_count is out of range

kIntelVvpVcCommitPendingErr if a previous commit request is pending

### intel\_vvp\_vc\_get\_block\_line\_count

**Prototype**

```
uint16_t
intel_vvp_vc_get_block_line_count(intel_vvp_vc_instance*
instance);
```

**Description** Returns the number of lines of the mesh box boundary. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

**Returns** the number of lines of the mesh box boundary

### intel\_vvp\_vc\_set\_block\_line\_count

#### Prototype

```
int
intel_vvp_vc_set_block_line_count(intel_vvp_vc_instance*
instance, uint16_t block_line_count);
```

**Description** Writes the number of lines of the mesh box boundary. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure  
  
block\_line\_count - new number of lines of the mesh box boundary

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error  
  
kIntelVvpCoreInstanceErr if the instance is a null pointer  
  
kIntelVvpVcValueErr if the value of block\_line\_count is out of range  
  
kIntelVvpVcCommitPendingErr if a previous commit request is pending

### intel\_vvp\_vc\_get\_h\_num\_blocks

#### Prototype

```
uint16_t
intel_vvp_vc_get_h_num_blocks(intel_vvp_vc_instance* instance);
```

**Description** Returns the number of mesh boxes on the horizontal dimension. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

**Returns** the number of mesh boxes on the horizontal dimension

### intel\_vvp\_vc\_set\_h\_num\_blocks

#### Prototype

```
int
intel_vvp_vc_set_h_num_blocks(intel_vvp_vc_instance* instance,
uint16_t h_num_blocks);
```

**Description** Writes the number of mesh boxes on the horizontal dimension. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

h\_num\_blocks - new number of mesh boxes on the horizontal dimension

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpVcValueErr if the value of h\_num\_blocks is out of range

kIntelVvpVcCommitPendingErr if a previous commit request is pending

### intel\_vvp\_vc\_get\_v\_num\_blocks

**Prototype**

```
uint16_t
intel_vvp_vc_get_v_num_blocks(intel_vvp_vc_instance* instance);
```

**Description** Returns the number of mesh boxes on the vertical dimension. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

**Returns** the number of mesh boxes on the vertical dimension

### intel\_vvp\_vc\_set\_v\_num\_blocks

**Prototype**

```
int
intel_vvp_vc_set_v_num_blocks(intel_vvp_vc_instance* instance,
uint16_t v_num_blocks);
```

**Description** Writes the number of mesh boxes on the vertical dimension. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

v\_num\_blocks - new number of mesh boxes on the vertical dimension

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer



kIntelVvpVcValueErr if the value of v\_num\_blocks is out of range  
kIntelVvpVcCommitPendingErr if a previous commit request is pending

### intel\_vvp\_vc\_get\_h\_ramp\_frac

#### Prototype

```
uint32_t
intel_vvp_vc_get_h_ramp_frac(intel_vvp_vc_instance* instance);
```

**Description** Returns the fraction of corrective gain for each horizontal pixel step within a mesh box. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

**Returns** the fraction of corrective gain for each horizontal pixel step within a mesh box

### intel\_vvp\_vc\_set\_h\_ramp\_frac

#### Prototype

```
int
intel_vvp_vc_set_h_ramp_frac(intel_vvp_vc_instance* instance,
uint32_t h_ramp_frac);
```

**Description** Writes fraction of corrective gain for each horizontal pixel step within a mesh box. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

h\_ramp\_frac - new fraction of corrective gain for each horizontal pixel step within a mesh box

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpVcValueErr if the value of h\_ramp\_frac is out of range

kIntelVvpVcCommitPendingErr if a previous commit request is pending

### intel\_vvp\_vc\_get\_h\_ramp\_p1\_frac

#### Prototype

```
uint32_t
intel_vvp_vc_get_h_ramp_p1_frac(intel_vvp_vc_instance*
instance);
```

**Description** Returns the next fraction of corrective gain for each horizontal pixel step within a mesh box. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

**Returns** the next fraction of corrective gain for each horizontal pixel step within a mesh box

### intel\_vvp\_vc\_set\_h\_ramp\_p1\_frac

#### Prototype

```
int
intel_vvp_vc_set_h_ramp_p1_frac(intel_vvp_vc_instance*
instance, uint32_t h_ramp_p1_frac);
```

**Description** Writes the next fraction of corrective gain for each horizontal pixel step within a mesh box. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

h\_ramp\_p1\_frac - new next fraction of corrective gain for each horizontal pixel step within a mesh box

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpVcValueErr if the value of h\_ramp\_p1\_frac is out of range

kIntelVvpVcCommitPendingErr if a previous commit request is pending

### intel\_vvp\_vc\_get\_h\_ramp\_m1\_frac

#### Prototype

```
uint32_t
intel_vvp_vc_get_h_ramp_m1_frac(intel_vvp_vc_instance*
instance);
```

**Description** Returns the fraction of corrective gain for each horizontal pixel step within the last mesh box. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

**Returns** the fraction of corrective gain for each horizontal pixel step within the last mesh box

#### intel\_vvp\_vc\_set\_h\_ramp\_m1\_frac

**Prototype**

```
int
intel_vvp_vc_set_h_ramp_m1_frac(intel_vvp_vc_instance*
instance, uint32_t h_ramp_m1_frac);
```

**Description** Writes the fraction of corrective gain for each horizontal pixel step within the last mesh box. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

h\_ramp\_m1\_frac - new fraction of corrective gain for each horizontal pixel step within the last mesh box

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error  
  
kIntelVvpCoreInstanceErr if the instance is a null pointer  
  
kIntelVvpVcValueErr if the value of h\_ramp\_m1\_frac is out of range  
  
kIntelVvpVcCommitPendingErr if a previous commit request is pending

#### intel\_vvp\_vc\_get\_v\_ramp\_frac

**Prototype**

```
uint32_t
intel_vvp_vc_get_v_ramp_frac(intel_vvp_vc_instance* instance);
```

**Description** Returns the fraction of corrective gain for each vertical pixel step within a mesh box. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

**Returns** the fraction of corrective gain for each vertical pixel step within a mesh box

### intel\_vvp\_vc\_set\_v\_ramp\_frac

**Prototype**

```
int
intel_vvp_vc_set_v_ramp_frac(intel_vvp_vc_instance* instance,
uint32_t v_ramp_frac);
```

**Description** Writes fraction of corrective gain for each vertical pixel step within a mesh box. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

v\_ramp\_frac - new fraction of corrective gain for each vertical pixel step within a mesh box

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpVcValueErr if the value of v\_ramp\_frac is out of range

kIntelVvpVcCommitPendingErr if a previous commit request is pending

### intel\_vvp\_vc\_get\_v\_ramp\_p1\_frac

**Prototype**

```
uint32_t
intel_vvp_vc_get_v_ramp_p1_frac(intel_vvp_vc_instance*
instance);
```

**Description** Returns the next fraction of corrective gain for each vertical pixel step within a mesh box. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

**Returns** the next fraction of corrective gain for each vertical pixel step within a mesh box

### intel\_vvp\_vc\_set\_v\_ramp\_p1\_frac

#### Prototype

```
int
intel_vvp_vc_set_v_ramp_p1_frac(intel_vvp_vc_instance*
instance, uint32_t v_ramp_p1_frac);
```

**Description** Writes the next fraction of corrective gain for each vertical pixel step within a mesh box. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

v\_ramp\_p1\_frac - new next fraction of corrective gain for each vertical pixel step within a mesh box

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpVcValueErr if the value of v\_ramp\_p1\_frac is out of range

kIntelVvpVcCommitPendingErr if a previous commit request is pending

### intel\_vvp\_vc\_get\_v\_ramp\_m1\_frac

#### Prototype

```
uint32_t
intel_vvp_vc_get_v_ramp_m1_frac(intel_vvp_vc_instance*
instance);
```

**Description** Returns the fraction of corrective gain for each vertical pixel step within the last mesh box. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

**Returns** the fraction of corrective gain for each vertical pixel step within the last mesh box

### intel\_vvp\_vc\_set\_v\_ramp\_m1\_frac

#### Prototype

```
int
intel_vvp_vc_set_v_ramp_m1_frac(intel_vvp_vc_instance*
instance, uint32_t v_ramp_m1_frac);
```

- Description** Writes the fraction of corrective gain for each vertical pixel step within the last mesh box. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.
- Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure
- v\_ramp\_m1\_frac - new fraction of corrective gain for each vertical pixel step within the last mesh box
- Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
- kIntelVvpCoreInstanceErr if the instance is a null pointer
- kIntelVvpVcValueErr if the value of v\_ramp\_m1\_frac is out of range
- kIntelVvpVcCommitPendingErr if a previous commit request is pending

#### intel\_vvp\_vc\_update\_cp\_lut

**Prototype**

```
int
intel_vvp_vc_update_cp_lut(intel_vvp_vc_instance* instance,
uint8_t cp_lut_idx,
const uint32_t* cp_lut_values);
```

- Description** Writes the color plane lookup table from a vector. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.
- Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure
- cp\_lut\_idx - the color index of the lookup table to write to
- cp\_lut\_values - pointer of a vector of lookup table values
- Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
- kIntelVvpCoreInstanceErr if the instance is a null pointer
- kIntelVvpVcPointerErr if cp\_lut\_values is a null pointer
- kIntelVvpVcParameterErr if cp\_lut\_idx is out of range
- kIntelVvpVcLutNotAvailableErr if the lookup table is not available for this instance

## intel\_vvp\_vc\_update\_step\_lut

### Prototype

```
int
intel_vvp_vc_update_step_lut(intel_vvp_vc_instance* instance,
const uint32_t* step_lut_values);
```

**Description** Writes the step lookup table from a vector. The instance must be a valid intel\_vvp\_vc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_vc\_instance software driver instance structure

step\_lut\_values - pointer of a vector of lookup table values

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpVcPointerErr if step\_lut\_values is a null pointer

kIntelVvpVcLutInvalidNumBlocksErr if the number of horizontal or vertical blocks were misconfigured to zero in a previous function call

### Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38



## 53. Warp IP

---

### 53.1. About the Warp IP

The Warp IP applies a warp (image transform) to a video stream. It processes RGB or YUV video streams at resolutions of up to 3840x2160 at 120 fps or 7680x4320 at 30 fps. It can process one, two, four, eight pixels in parallel.

The Warp IP offers mipmaps that allow arbitrary warps up to a limit of 256:1 downscale ratio. Without mipmaps, the Warp IP can only process arbitrary warps up to a limit of 2:1 for the effective downscale ratio. If, in any region, the warp produces an output image that is downscaled by more than this limit from the input image, an error occurs in the IP software.

Typical applications include:

- Camera lens distortion correction
- Projector system distortion correction

You can turn on **Use Easy warp** for resolutions up to UHD. This parameter offers reduced resource and memory bandwidth requirements. Turn on **Use Easy warp** for mirroring and rotations that are a multiple of 90° (0°, 90°, 180° and 270°).

The IP only supports lite variants. For more information about full and lite variants, refer to the *Altera Streaming Video Protocol Specification*

#### Related Information

[Altera Streaming Video Protocol Specification](#)

#### 53.1.1. Warp IP Features

- Avalon memory-mapped interface for memory access
- Fixed 10 bits per color RGB or YUV 4:4:4
- One, two, four, or eight pixels in parallel
- Subframe latency (transform dependent)
- Software controlled arbitrary warps up to 8k and easy warp for resource efficient mirroring and rotations up to UHD
- Maximum image size of 7680x7680
- Minimum image size of 128x64
- 600 MHz video and processing clock speeds in Agilex 7 devices
- Single memory bounce for reduced memory bandwidth (transform dependent)
- Arbitrary warps with downscaling to 256:1



### 53.1.2. Warp IP Performance and Resource Utilization

Altera provides resource and utilization data for guidance. The designs target an Arria 10 10AX115N2F40I2LG device or an Agilex 7 AGIB027R29A1E2V.

For devices other than Agilex 7 devices, the Warp IP supports clock rates of 300 MHz for the video related clocks `axi4s_vid_in_0_clock`, `axi4s_vid_out_0_clock`, and `core_clock`.

For Agilex 7 devices, the Warp IP supports clock rates of 600 MHz for the video related clocks `axi4s_vid_in_0_clock`, `axi4s_vid_out_0_clock`, and `core_clock`. This allows a single pixel in parallel, single engine configuration to process UHD frames at 60 fps. The Warp IP also supports a configuration of 2 pixels in parallel with one engine. Your design can process UHD frames at 60 fps on Agilex 7 devices with a reduced video clock rate of 300 MHz on the video input and output connections and running the main processing clock at 600 MHz.

**Table 1089. HD frame processing on Arria 10 Device with Double Memory Bounce**

Processing frames of up to 1920x1080 resolution. Altera set the video related clocks `axi4s_vid_in_0_clock`, `axi4s_vid_out_0_clock`, and `core_clock` to a minimum of 150 MHz to allow the IP to process 60 fps. Set these clocks to 300 MHz for frame rates of 120 fps. **Use mipmaps** is off.

| Pixel in Parallel | Use Single Memory Bounce | Number of Engines | Max Video Width <sup>(176)</sup> <sub>(177)</sub> | Memory Buffer Size | ALMs   | Memory Blocks (M20K) | DSP Blocks |
|-------------------|--------------------------|-------------------|---------------------------------------------------|--------------------|--------|----------------------|------------|
| 1                 | Off                      | 1                 | 2048                                              | HD                 | ~6,000 | 191                  | 36         |

**Table 1090. HD frame processing on Arria 10 Device with Single Memory Bounce**

Processing frames of up to 1920x1080 resolution. Altera set the video related clocks `axi4s_vid_in_0_clock`, `axi4s_vid_out_0_clock`, and `core_clock` to a minimum of 150 MHz to allow the IP to process 60 fps. Set these clocks to 300 MHz for frame rates of 120 fps. **Use mipmaps** is off.

| Pixel in Parallel | Use Single Memory Bounce | Cache Blocks per Engine | Number of Engines | Maximum Video Width <sup>(177)</sup> | Memory Buffer Size | ALMs   | Memory Blocks (M20K) | DSP Blocks |
|-------------------|--------------------------|-------------------------|-------------------|--------------------------------------|--------------------|--------|----------------------|------------|
| 1                 | On                       | 256                     | 1                 | 2048                                 | HD                 | ~6,000 | 163                  | 36         |
| 1                 | On                       | 512                     | 1                 | 2048                                 | HD                 | ~6,000 | 211                  | 36         |
| 1                 | On                       | 1024                    | 1                 | 2048                                 | HD                 | ~6,000 | 307                  | 36         |

**Table 1091. UHD Frames at 30 fps on Arria 10 Device with Double Memory Bounce**

Processing frames of up to 3840x2160 resolution at 30 fps. Altera set the video related clocks `axi4s_vid_in_0_clock`, `axi4s_vid_out_0_clock`, and `core_clock` to 300 MHz. **Use mipmaps** is off.

| Pixel in parallel | Use Single Memory Bounce | Number of Engines | Max Video Width <sup>(177)</sup> | Memory Buffer Size | ALMs   | Memory Blocks (M20K) | DSP Blocks |
|-------------------|--------------------------|-------------------|----------------------------------|--------------------|--------|----------------------|------------|
| 1                 | Off                      | 1                 | 4096                             | UHD                | ~6,000 | 273                  | 36         |

<sup>(176)</sup> Same maximum video width for input and output.

<sup>(177)</sup> Same maximum video width for input and output.

**Table 1092. UHD Frames at 60 fps on Arria 10 Device with Double Memory Bounce**

Processing frames of up to 3840x2160 resolution at 60 fps. Altera set the video related clocks `axi4s_vid_in_0_clock`, `axi4s_vid_out_0_clock`, and `core_clock` to 300 MHz. **Use mipmaps** is off.

| Pixel in parallel | Use Single Memory Bounce | Number of Engines | Max Video Width <sup>(177)</sup> | Memory Buffer Size | ALMs    | Memory Blocks (M20K) | DSP Blocks |
|-------------------|--------------------------|-------------------|----------------------------------|--------------------|---------|----------------------|------------|
| 2                 | Off                      | 2                 | 4096                             | UHD                | ~10,000 | 354                  | 72         |

**Table 1093. UHD frames at 60 fps on Arria 10 Device with Single Memory Bounce**

Processing frames of up to 3840x2160 resolution at 60 fps. Altera set the video related clocks `axi4s_vid_in_0_clock`, `axi4s_vid_out_0_clock`, and `core_clock` to 300 MHz. **Use mipmaps** is off.

| Pixel in parallel | Use Single Memory Bounce | Cache Blocks per Engine | Number of Engines | Max Video Width <sup>(177)</sup> | Memory BufferSize | ALMs    | Memory Blocks (M20K) | DSP Blocks |
|-------------------|--------------------------|-------------------------|-------------------|----------------------------------|-------------------|---------|----------------------|------------|
| 2                 | On                       | 256                     | 2                 | 4096                             | UHD               | ~10,000 | 300                  | 72         |
| 2                 | On                       | 512                     | 2                 | 4096                             | UHD               | ~10,000 | 396                  | 72         |
| 2                 | On                       | 1024                    | 2                 | 4096                             | UHD               | ~10,000 | 588                  | 72         |

**Table 1094. One Pixel In Parallel UHD Frames at 60 fps, on Agilex 7 Device with Double Memory Bounce**

Processing frames of up to 3840x2160 resolution at 60 fps. Altera set the video related clocks `axi4s_vid_in_0_clock`, `axi4s_vid_out_0_clock`, and `core_clock` to 600 MHz. **Use mipmaps** is off.

| Pixel in parallel | Use Single Memory Bounce | Number of Engines | Max Video Width <sup>(177)</sup> | Memory Buffer Size | ALMs   | Memory Blocks (M20K) | DSP Blocks |
|-------------------|--------------------------|-------------------|----------------------------------|--------------------|--------|----------------------|------------|
| 1                 | Off                      | 1                 | 4096                             | UHD                | ~8,000 | 249                  | 36         |

**Table 1095. One Pixel In Parallel UHD Frames at 60 fps on Agilex 7 Device with Single Memory Bounce**

Processing frames of up to 3840x2160 resolution at 60 fps. Altera set the video related clocks `axi4s_vid_in_0_clock`, `axi4s_vid_out_0_clock`, and `core_clock` to 600 MHz. **Use mipmaps** is off.

| Pixel in parallel | Use Single Memory Bounce | Cache Blocks per Engine | Number of Engines | Max Video Width <sup>(177)</sup> | Memory Buffer Size | ALMs   | Memory Blocks (M20K) | DSP Blocks |
|-------------------|--------------------------|-------------------------|-------------------|----------------------------------|--------------------|--------|----------------------|------------|
| 1                 | On                       | 256                     | 1                 | 4096                             | UHD                | ~7,000 | 212                  | 36         |
| 1                 | On                       | 512                     | 1                 | 4096                             | UHD                | ~7,000 | 260                  | 36         |
| 1                 | On                       | 1024                    | 1                 | 4096                             | UHD                | ~7,000 | 356                  | 36         |

**Table 1096. One Pixel In Parallel HD frame processing with Use easy warp on Arria 10 Device**

Processing frames of up to 1920x1080 resolution. Altera set the video related clocks `axi4s_vid_in_0_clock` and `axi4s_vid_out_0_clock` to a minimum of 150 MHz to allow the IP to process 60 fps. Set these clocks to 300 MHz for frame rates of 120 fps.

| Pixel in parallel | Maximum Video Width <sup>(177)</sup> | Memory Buffer Size | Use Easy Warp | ALMs   | Memory Blocks (M20K) | DSP Blocks |
|-------------------|--------------------------------------|--------------------|---------------|--------|----------------------|------------|
| 1                 | 2048                                 | HD                 | On            | ~3,000 | 148                  | 0          |

**Table 1097. One Pixel In Parallel UHD frame processing with Use easy warp on Arria 10 Device**

Processing frames of up to 3840 × 2160 resolution. Altera set the video related clocks axi4s\_vid\_in\_0\_clock, axi4s\_vid\_out\_0\_clock to a minimum of 300 MHz to allow the IP to process 30 fps.

| Pixel in parallel | Maximum Video Width <sup>(177)</sup> | Memory Buffer Size | Use Easy Warp | ALMs  | Memory Blocks (M20K) | DSP Blocks |
|-------------------|--------------------------------------|--------------------|---------------|-------|----------------------|------------|
| 1                 | 3840                                 | UHD                | On            | ~3000 | 271                  | 0          |

**Table 1098. Two Pixels In Parallel UHD frame processing with Use easy warp on Arria 10 Device**

Processing frames of up to 3840 × 2160 resolution. Altera set the video related clocks axi4s\_vid\_in\_0\_clock, axi4s\_vid\_out\_0\_clock to a minimum of 300 MHz to allow the IP to process 60 fps.

| Pixel in parallel | Easy Warp | Maximum Video Width <sup>(177)</sup> | Use Easy Warp | Memory Buffer Size | ALMs  | Memory Blocks (M20K) | DSP Blocks |
|-------------------|-----------|--------------------------------------|---------------|--------------------|-------|----------------------|------------|
| 2                 | 1         | 3840                                 | On            | UHD                | ~3000 | 271                  | 0          |

**Table 1099. UHD Frames at 120 fps, on Agilex 7 Device with Double Memory Bounce**

Processing frames of up to 3840x2160 resolution. Altera set the video related clocks axi4s\_vid\_in\_0\_clock, axi4s\_vid\_out\_0\_clock, and core\_clock to 600 MHz with the memory clock av\_mm\_memory\_host\_clock set to 333 MHz. **Use mipmaps** is off. Four engines and four pixels in parallel with the 8K buffer size allow the Warp IP to process the bandwidth necessary for 120 fps.

| Pixel in parallel | Use Single Memory Bounce | Number of Engines | Max Video Width | Memory Buffer Size | ALMs    | Memory Blocks (M20K) | DSP Blocks |
|-------------------|--------------------------|-------------------|-----------------|--------------------|---------|----------------------|------------|
| 4                 | Off                      | 4                 | 4096            | 8K                 | ~23,000 | 629                  | 144        |

**Table 1100. UHD Frames at 120 fps on Agilex 7 Device with Single Memory Bounce**

Processing frames of up to 3840x2160 resolution. Altera set the video related clocks axi4s\_vid\_in\_0\_clock, axi4s\_vid\_out\_0\_clock and core\_clock to 600 MHz. **Use mipmaps** is off. Four engines and four pixels in parallel with the 8K buffer size allow the Warp IP to process the bandwidth necessary for 120 fps.

| Pixel in parallel | Use Single Memory Bounce | Cache Blocks per Engine | Number of Engines | Max Video Width | Memory Buffer Size | ALMs    | Memory Blocks (M20K) | DSP Blocks |
|-------------------|--------------------------|-------------------------|-------------------|-----------------|--------------------|---------|----------------------|------------|
| 4                 | On                       | 256                     | 4                 | 4096            | 8K                 | ~20,000 | 471                  | 144        |
| 4                 | On                       | 512                     | 4                 | 4096            | 8K                 | ~21,000 | 663                  | 144        |
| 4                 | On                       | 1024                    | 4                 | 4096            | 8K                 | ~22,000 | 1047                 | 144        |

**Table 1101. 8K Frames at 30 fps, on Agilex 7 Device with Double Memory Bounce**

Processing frames of up to 7680x4320 resolution. Altera set the video related clocks axi4s\_vid\_in\_0\_clock, axi4s\_vid\_out\_0\_clock, and core\_clock to 600 MHz with the memory clock av\_mm\_memory\_host\_clock set to 333 MHz. **Use mipmaps** is off. Four engines and four pixels in parallel allow the Warp IP to process the bandwidth necessary for 8k at 30 fps.

| Pixel in parallel | Use Single Memory Bounce | Number of Engines | Max Video Width | Memory Buffer Size | ALMs    | Memory Blocks (M20K) | DSP Blocks |
|-------------------|--------------------------|-------------------|-----------------|--------------------|---------|----------------------|------------|
| 4                 | Off                      | 4                 | 7680            | 8K                 | ~24,000 | 737                  | 144        |

**Table 1102. 8K Frames at 30 fps on Agilex 7 Device with Single Memory Bounce**

Processing frames of up to 7680x4320 resolution. Altera set the video related clocks `axi4s_vid_in_0_clock`, `axi4s_vid_out_0_clock` and `core_clock` to 600 MHz. **Use mipmaps** is off. Four engines and four pixels in parallel allow the Warp IP to process the bandwidth necessary for 8K at 30 fps.

| Pixel in parallel | Use Single Memory Bounce | Cache Blocks per Engine | Number of Engines | Max Video Width | Memory Buffer Size | ALMs    | Memory Blocks (M20K) | DSP Blocks |
|-------------------|--------------------------|-------------------------|-------------------|-----------------|--------------------|---------|----------------------|------------|
| 4                 | On                       | 256                     | 4                 | 7680            | 8K                 | ~21,000 | 579                  | 144        |
| 4                 | On                       | 512                     | 4                 | 7680            | 8K                 | ~22,000 | 771                  | 144        |
| 4                 | On                       | 1024                    | 4                 | 7680            | 8K                 | ~23,000 | 1155                 | 144        |

**Table 1103. Extra resources for Agilex 7 Devices with Mipmaps On**

Turning **Use mipmaps** on requires extra resources that depend on the number of pixels in parallel, the maximum video width and the size of the memory buffers.

| Pixel in parallel | Max Video Width | Memory Buffer Size | ALMs   | Memory Blocks (M20K) | DSP Blocks |
|-------------------|-----------------|--------------------|--------|----------------------|------------|
| 1                 | 4096            | UHD                | ~2,000 | 131                  | 18         |
| 4                 | 4096            | 8K                 | ~3,000 | 143                  | 30         |
| 4                 | 7680            | 8K                 | ~4,000 | 259                  | 30         |

**Table 1104. Extra resources for Arria 10 Devices with Mipmaps On**

Turning **Use mipmaps** on requires extra resources that depend on the number of pixels in parallel, the maximum video width and the size of the memory buffers.

| Pixel in parallel | Max Video Width | Memory Buffer Size | ALMs   | Memory Blocks (M20K) | DSP Blocks |
|-------------------|-----------------|--------------------|--------|----------------------|------------|
| 1                 | 2048            | HD                 | ~2,000 | 85                   | 18         |
| 2                 | 4096            | UHD                | ~2,000 | 155                  | 18         |

## 53.2. Warp IP Parameters

The IP offers various compile-time parameters.

**Table 1105. Warp IP Parameters**

| Parameter                           | Values                     | Description                                                                                                                                                                                                                                    |
|-------------------------------------|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Video data format</b>            |                            |                                                                                                                                                                                                                                                |
| <b>Lite mode</b>                    | On                         | The IP only operates in <b>Lite mode</b> , with no support for control packets.                                                                                                                                                                |
| <b>Number of pixels in parallel</b> | 1, 2, 4, or 8              | Number of pixels processed in parallel. When <b>Use easy warp</b> is on, the IP supports only 1 or 2 pixels in parallel.                                                                                                                       |
| <b>Number of color planes</b>       | 3                          | Number of color planes per pixel.                                                                                                                                                                                                              |
| <b>Bits per color sample</b>        | 10                         | Number of bits per color sample                                                                                                                                                                                                                |
| <b>Maximum input video width</b>    | 2048, 3840, 4096, or 7680. | Maximum number of pixels per input line. Configures the depth of line buffers in the video input block.<br>The IP can process image widths of up to 7680. When <b>Use easy warp</b> is on, the maximum video width is limited to 2048 or 3840. |
| <i>continued...</i>                 |                            |                                                                                                                                                                                                                                                |

| Parameter                                                                                                      | Values                     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Maximum output video width</b>                                                                              | 2048, 3840, 4096, or 7680. | Maximum number of pixels per output line. Configures the depth of line buffers in the video output block. When <b>Use easy warp</b> is on, the maximum video width is limited to 2048 or 3840.                                                                                                                                                                                                                                                                                                                                  |
| <b>Configuration Settings</b>                                                                                  |                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Width of memory host data bus</b>                                                                           | 256 or 512                 | Defines the width of the data interface to external memory. Choose this width based on the width of the memory controller you use.                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Use easy warp</b>                                                                                           | On or off                  | Turn on for a limited set of warp operations. Turn off for arbitrary warps. The IP can only process image heights and widths that are a multiple of two when you select 2 pixels in parallel and turn on <b>Use easy warp</b> .                                                                                                                                                                                                                                                                                                 |
| <b>Memory frame buffer size</b>                                                                                | SD, HD, UHD, or 8K         | The amount of memory space the IP allocates to each frame buffer. Each pixel is 4 bytes in size. <ul style="list-style-type: none"> <li>SD is 1024x1024 pixels</li> <li>HD is 2048x2048 pixels</li> <li>UHD is 4096x4096 pixels</li> <li>8K is 8192x8192 pixels</li> </ul>                                                                                                                                                                                                                                                      |
| <b>Enable Debug Registers</b>                                                                                  | On or off                  | Turn on to read back various registers containing debugging information.                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Engine Configuration Settings</b><br>(These parameters are only available when <b>Use easy warp</b> is off) |                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Number of engines</b>                                                                                       | 1, 2 or, 4                 | Number of processing engines to use. Each engine processes one pixel per clock cycle.                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Use mipmaps</b>                                                                                             | On or off                  | When you turn on <b>Use mipmaps</b> , the IP uses a pyramid of automatically downscaled mipmap images to support arbitrary warps with downscaled regions up to 256:1. When you turn off <b>Use mipmaps</b> , arbitrary warps are limited to downscaled regions up to 2:1.                                                                                                                                                                                                                                                       |
| <b>Use single memory bounce</b>                                                                                | On or off                  | Defines how the engines are connected to and use the external memory. When you turn off <b>Use single memory bounce</b> the engines both read and write their video data through the memory (double memory bounce). When you turn on <b>Use single memory bounce</b> the engines only read video data from memory and their output data passes directly to the video output process of the warp IP.<br>The IP can only generate image heights that are a multiple of 8 lines when you turn on <b>Use single memory bounce</b> . |
| <b>Cache blocks per engine</b>                                                                                 | 256, 512 or 1024           | Only available when <b>Use single memory bounce</b> is on. Defines the amount of cache memory that is available to each engine. The amount of cache memory required is a function of the input resolution the IP processes, the required warp and the number of engines you select.                                                                                                                                                                                                                                             |

Figure 142. Warp IP GUI

Details

Warp Intel FPGA IP

intel\_vwp\_warp

▶ Important Usage Information

▼ Video Data Format

☒ Lite mode
 

Bits per color sample: 10 ▼
 Number of color planes: 3 ▼
 Number of pixels in parallel: 1 ▼
 Maximum input video width: 7680 ▼
 Maximum output video width: 7680 ▼

▼ General Configuration Settings

Width of memory host data bus: 512 ▼
 ☐ Use easy warp
 Memory frame buffer size: 8K ▼
 ☒ Enable Debug Registers

▼ Engine Configuration Settings

The number of engines: 1 ▼
 ☒ Use mipmaps
 ☐ Use single memory bounce
 Cache blocks per engine: 512 ▼

Table 1106. Warp IP throughput for different parameters

| Number of pixels in parallel | The number of processing engines to use | f <sub>MAX</sub> (MHz) | Performance                                                             |
|------------------------------|-----------------------------------------|------------------------|-------------------------------------------------------------------------|
| 1                            | 1                                       | 150                    | Image resolutions of up to 1920x1080 at 60 fps.                         |
| 1                            | 1                                       | 300                    | Image resolutions of up to 3840x2160 at 30 fps.                         |
| 2                            | 2                                       | 300                    | Image resolutions of up to 3840x2160 at 60 fps.                         |
| 1                            | 1                                       | 600                    |                                                                         |
| 4                            | 4                                       | 600                    | Image resolutions of up to 3840x2160 at 120 fps or 7680x4320 at 30 fps. |

### 53.3. Warp IP Block Description

The Warp IP accepts RGB or YUV format video input from its Altera Streaming Video interface and directly stores the video data in external memory. The video input process has a pool of four frame buffers that it uses to store the incoming video data. The IP accesses the buffers in a cyclic order.

The video output process generates RGB or YUV format Altera Streaming Video using the warped image data.

The IP generates arbitrary warps based on a transform mesh. If you turn on **Use easy warp**, it offers a fixed set of transforms (rotations and mirroring) without needing a transform mesh. If you turn off **Use easy warp**, the IP processes the buffered input video by the configured number of warp engines to apply the required warp. Three coefficient tables control the warp engines to define the warp that the IP applies. External memory stores the three coefficient tables that are generated using the Warp IP software API.

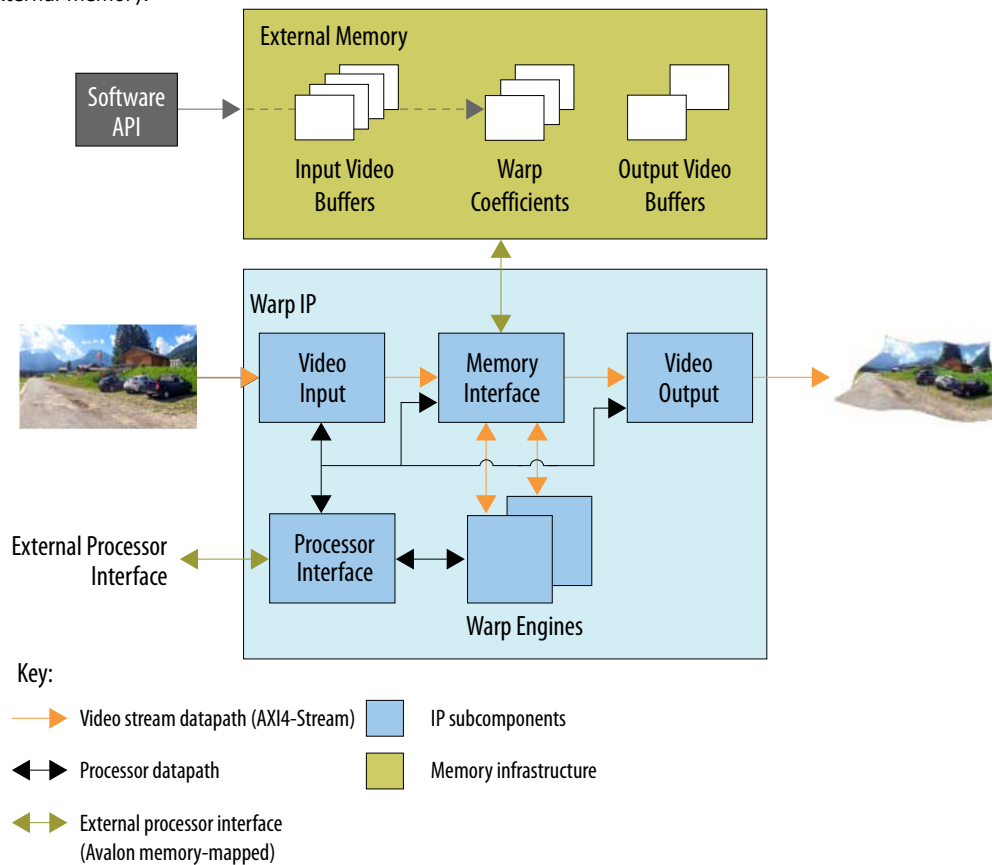
The IP defines the required warp with a backward mapping from the output to the input pixel positions. It represents the warp as a subsampled mesh that defines the mapping in 8x8 or 16x16 regions. For output pixel mappings within the 8x8 or 16x16 positions, the warp engine applies bilinear interpolation. For any input image dimensions (height or width) that are greater than 3840, use a 16x16 mesh.

If you turn off **Use single memory bounce**, the IP operates with two bounces through external memory. The IP buffers the input video and it writes back the resultant warped image to external memory into one of two output video buffers. The IP writes to these dual output buffers alternately. The IP reads the warped image in the output video buffers and passes out of the IP.

If you turn on **Use single memory bounce**, the IP operates with just a single bounce through external memory—the buffering of the input video. The IP transfers the resultant warped image directly to the video output process without passing through external memory.

**Figure 143. Warp IP block diagram (Double Memory Bounce)**

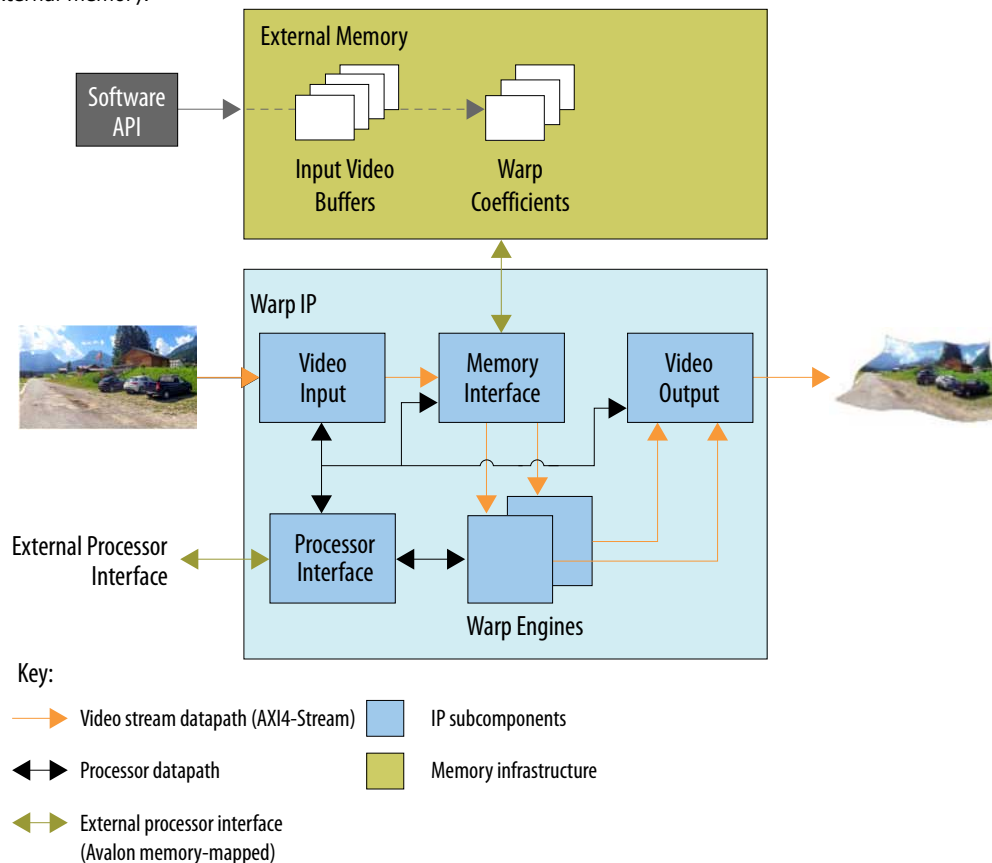
The figure shows a high-level block diagram for the Warp IP with its connection to external memory when **Use single memory bounce** is off. In this configuration, the engines read and write video data through the external memory.





**Figure 144. Warp IP block diagram (Single Memory Bounce)**

The figure shows a high-level block diagram for the Warp IP with its connection to external memory when **Use single memory bounce** is on. In this configuration, the engines only read video data from the external memory. The engines transmit their video data directly to the output process without the data passing through external memory.



**Figure 145. Warp image transform examples**

From left: arbitrary warp with a 5x5 array of control points, four corner warp with some radial distortion.



### Coefficient Tables

Each engine within the Warp IP has read access to its own set of three coefficient tables that define and control the image transform that the IP applies. The three different tables are:

- Mesh coefficients that define the output to input pixel transform
- Fetch coefficients that control the loading of the input image into the cache memory within the engine(s).
- Filter coefficients that control the mapping from the cache memory as the IP generates the interpolated or filtered output pixels.

The format of the mesh coefficients is different to the mesh data that you provide to the software API. The Software API uses 32-bit signed integers for the mesh values; the Warp IP uses a 16-bit offset binary format.

The IP needs just the mesh data to define the warp. The software API uses this mesh data to generate the required coefficient tables.

### Warp Mesh Interpolation

The IP defines the warp transform using either an 8x8 or 16x16 subsampled mesh. This mesh defines the mapping from the output pixel positions to the corresponding input pixel positions. The subsampled mesh requires that only the mappings for the following output pixel positions are defined:

#### 8x8 mesh

```
(0,0), (8,0), (16,0) ... (W, 0)
```

```
(0,8), (8,8), (16,8) ... (W, 8)
```

```
⋮
(0,H), (8, H), (16, H) ... (W, H)
```

where  $W=8*\text{ceil}(\text{image width}/8)$  and  $H=8*\text{ceil}(\text{image height}/8)$

#### 16x16 mesh

```
(0,0), (16,0), (32,0) ... (W, 0)
```

```
(0,16), (16,16), (32,16) ... (W, 16)
```

```
⋮
(0,H), (16, H), (32, H) ... (W, H)
```

where  $W=16*\text{ceil}(\text{image width}/16)$  and  $H=16*\text{ceil}(\text{image height}/16)$

Using either the 8x8 or 16x16 mesh is based on the input image size. If either the input image width or height is greater than 3840, you must use a 16x16 mesh. If both the input image width and height are equal to or less than 3840, you must use an 8x8 mesh.

To generate the output pixel positions that lie in between these 8x8 or 16x16 positions, the Warp IP uses bilinear interpolation.

## Output Pixel Interpolation and Filtering

The IP generates output pixels with the pixel data from the associated input pixel positions as defined by the warp that the IP applies. The IP generates output pixel values with a bicubic interpolation calculation using a 4x4 kernel of the associated input pixel values.

The weightings for the interpolation over the 4x4 kernel are a combination of a bicubic function and a variable low pass filtering function. The software API automatically applies low pass filtering, which it bases on the amount of downscaling that results for that region of the warp.

When **Use mipmaps** is on, the Warp IP software automatically identifies regions of the output image that require the downsampled, mipmap images and the interpolation and filtering settings are chosen accordingly.

## Blank Skip Regions

When you configure the Warp IP to substantially downscale regions of an image, large areas of the output image can map to points outside the input image. These unmapped regions result in the IP producing black.

Because these regions in the output image do not require any processing of the input image by the Warp IP, for efficiency the IP skips the processing associated with these regions. This skipping process is setup automatically by the software API which determines, from the desired warp mapping, which regions the IP skips. You see this behavior only when **Use single memory bounce** is off.

## Easy Warp

When you turn on **Use easy warp**:

- The IP supports rotations of 0°, 90°, 180° and 270° and you can apply a horizontal mirror operation before the selected rotation.
- The IP does not configure any engines.
- The IP applies the required rotational or mirrored transform by the video output block as it reads data from external memory.
- The IP does not require any processing engines, giving resource savings and memory bandwidth savings.

When **Use easy warp** is on, ensure that both **Use mipmaps** and **Use single memory bounce** are off.

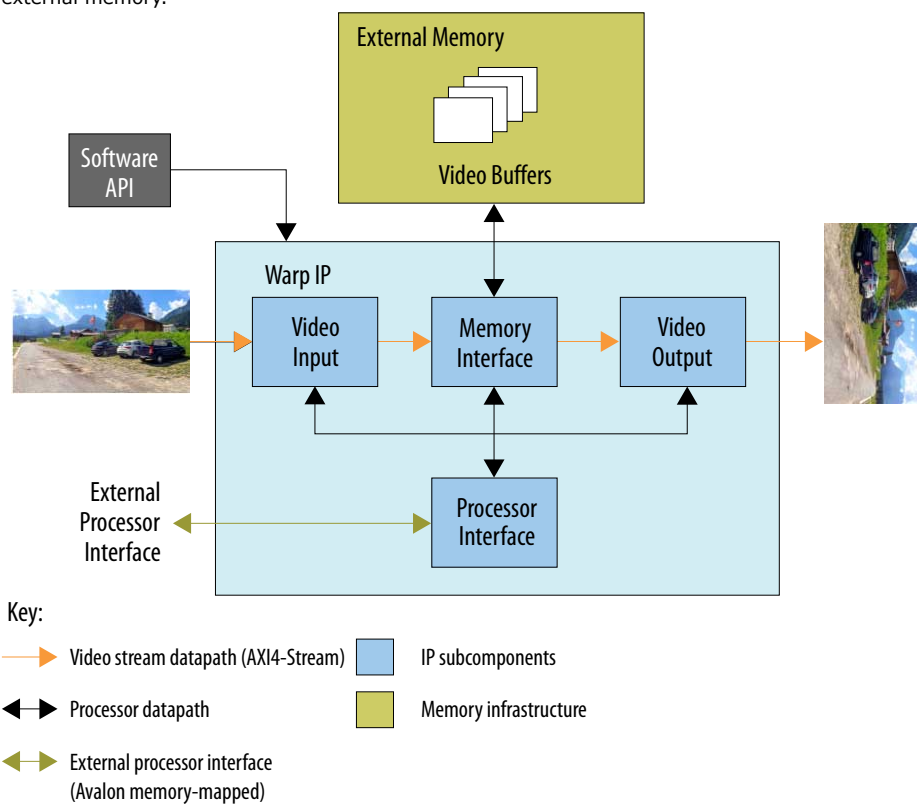
For easy warp rotations of 0 or 180°, the maximum input width dimension must not exceed the maximum output width. You set the maximum width with the **Maximum output video width** parameter. For easy warp rotations of 90° or 270°, the transposing of vertical and horizontal dimensions places restrictions on the input resolution.

**Table 1107. Input Resolution Restrictions for 90° or 270°**

| Maximum Output Video Width | Input Height Restriction |
|----------------------------|--------------------------|
| 2048                       | 1088                     |
| 3840                       | 2176                     |

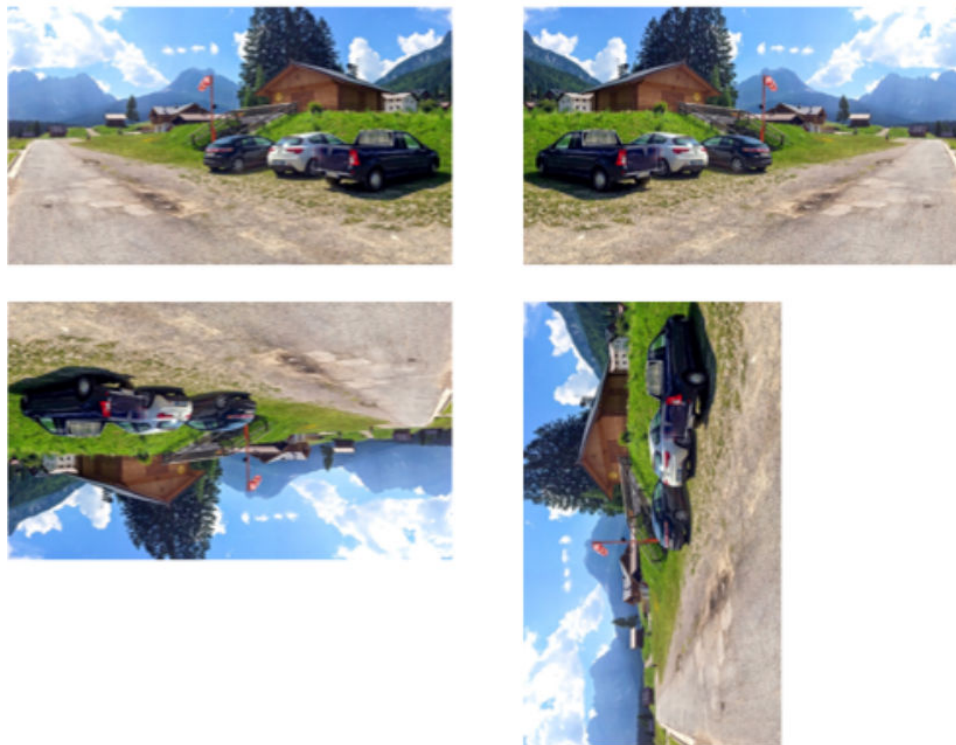
**Figure 146. Warp IP block diagram (Easy Warp)**

The figure shows a high-level block diagram for the Warp IP with **Use Easy warp** with its connection to external memory.



**Figure 147. Easy Warp image transform examples**

From top left clockwise: original image, mirrored, 90° rotate and 180° rotate.



### Single Memory Bounce

Generally, turning on **Use single memory bounce** gives reduced memory bandwidth compared to when it is off. However, single memory bounce may require larger internal RAM. The RAM usage depends on the specific warp transform. For some transforms the IP requires larger amounts of cache. Any increased cache requirement requires increased internal block RAM usage.

With **Use single memory bounce** on, three cache size options are available: 256, 512 and 1024 cache blocks per engine. With **Use single memory bounce** off, the IP has a fixed cache size of 256 cache blocks per engine.

Whether you turn on **Use single memory bounce** depends on the actual transform the IP performs. Altera provides a software tool that determines how much cache the IP needs to process the desired transform. You provide the tool with information such as the input and output resolutions, the transform required, and the number of engines to use. The tool then provides guidance on how much cache to use to process the transform.

For information on the Warp block cache tool, refer to *Block Cache Tool*. For information on the memory bandwidth implications of running with a single memory bounce, refer to *External Memory for Warp IP*.

### Single Memory Bounce Cache Example

An example of cache usage is given based on a 45 degree rotation with UHD input and output resolutions at 60 fps on an Arria 10 device. This throughput requires that the IP uses two engines.

With **Use single memory bounce** off, the block RAM usage by the IP is 365 M20Ks.

With **Use single memory bounce** on, the IP requires 512 cache blocks per engine because of the constraints of the 45 degree rotation. This cache block requirement translates to a block RAM usage of 407 M20Ks.

**Table 1108. Block RAM Usage – UHD 45 Degree Rotate**

| Use single memory bounce | Cache blocks per engine | Total block RAM usage |
|--------------------------|-------------------------|-----------------------|
| On                       | 512                     | 407                   |
| Off                      | 256 (fixed)             | 365                   |

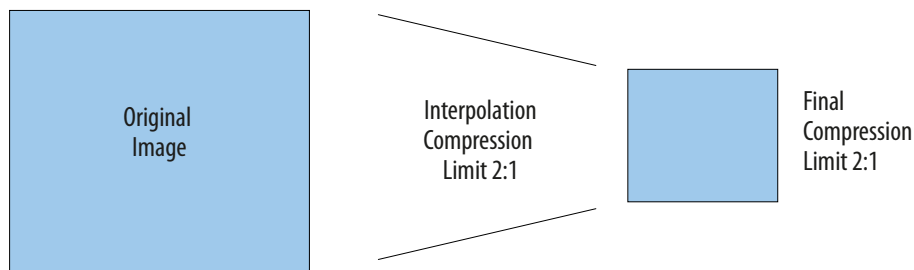
### Transform Compression Limits and Mipmaps

There is a fundamental compression limit of 2:1 in the way the Warp IP applies the required transform in generating the output image. This limit comes from the low-pass filtering performed during the interpolation calculation and from the structure of the cache blocks used in the pixel processing. The limit may be overcome by using downscaled versions of the input image as the source data in the interpolation and filtering stage.

The figures show how you may use in this way the downscaled versions of the input image, available as a pyramid of the various downscale factors.

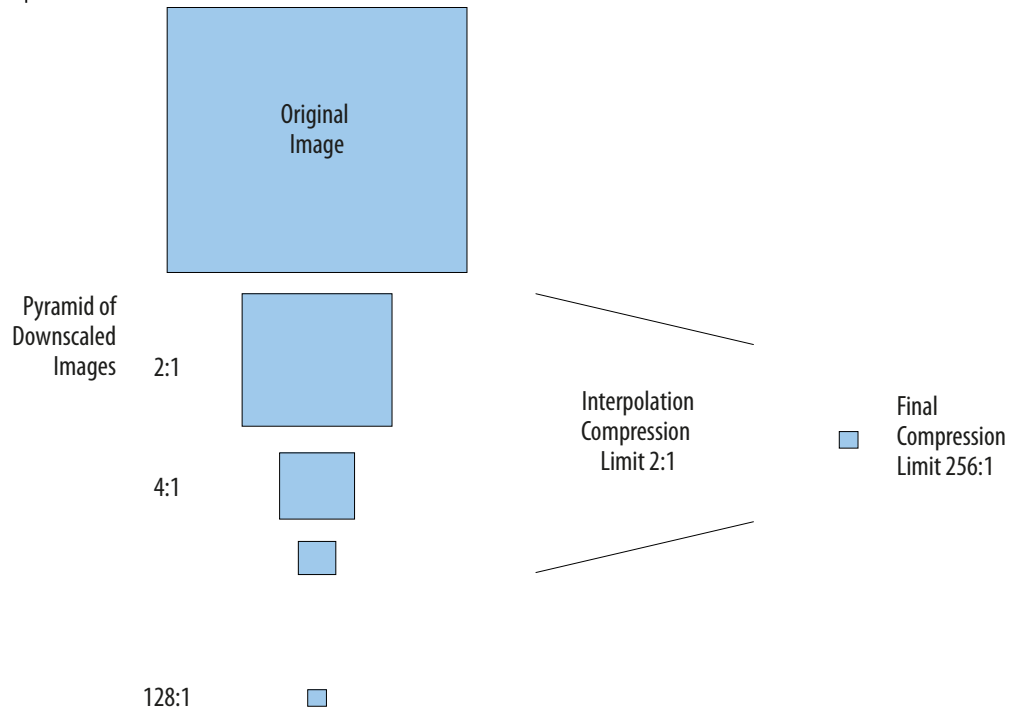
**Figure 148. Transform Compression Limit with Use mipmaps off**

Using only the original image in the application of the transform gives a final compression limit of 2:1.



**Figure 149. Transform Compression Limit with Use mipmaps on**

Using the original image and a pyramid of downscaled images in the application of the transform gives a final compression limit of 256:1.



When **Use mipmaps** is on, the Warp IP automatically generates and uses this pyramid of downscaled input images as required by the transform it performs.

#### Related Information

- [Block Cache Tool](#) on page 1059
- [Warp IP Performance and Resource Utilization](#) on page 1045

### 53.3.1. Block Cache Tool

This tool estimates the minimum suggested size of the Warp IP engine's block cache for the given transform and the number of processing engines. You can use the block cache tool at the design stage to help estimate required FPGA resources and evaluate warps specific to the customer application.

The block cache tool also detects if the configured transform exceeds maximum supported downscale ratio of 256:1, if mipmaps are on or 2:1 otherwise, in any part of the image. These values are the maximum supported compression values of the Warp IP. The block cache tool supports both single bounce and double bounce warp IP configurations.

The block cache tool provides a command line interface that allows you to specify the desired warp by using several available options such as rotation, mirroring, keystone, or by using a warp mesh file generated by the Warp SoC Design Example. The tool then estimates block cache usage for all supported block cache sizes. For each size option the tool prints the result in the following form:

**Table 1109. Tool Results**

| Result                           | Description                                                                                                                                                                                                                                       |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pass                             | Requested warp is possible using this block cache size per processing engine                                                                                                                                                                      |
| Pass, Block cache usage too high | Requested warp might be possible. However the IP does not guarantee the output quality because of high block cache usage. The configuration requires further evaluation on the target system                                                      |
| Fail                             | Requested warp is not possible using this block cache size. The estimation also fails if the warp exceeds a maximum supported local compression of 256:1 or 2:1 depending on the mipmap setting, in which case it fails for all block cache sizes |

The block cache tool can visualize configured transform by warping a reference image and saving it as a bitmap (bmp) file on the disk.

The block cache tool is a command line application and included with the Quartus Prime Software.

### Block Cache Tool Options

The block cache tool options:

```
vidip_bct.exe [option]...
```

**Table 1110. Block Cache Tool Options**

| Option          | Description                                                                                      | Usage                                                                                                              |
|-----------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| -h              | Help                                                                                             | Print the tool usage information.                                                                                  |
| -iw -ih -ow -oh | Input width (pixels)<br>Input height (pixels)<br>Output width (pixels)<br>Output height (pixels) | Set input and output dimensions of the transformed image in pixels. Default image dimensions are 7680x4320 pixels. |
| -r              | Rotation angle                                                                                   | Set rotation angle in degrees. Image is rotated around the center.                                                 |
| -mh             | Mirror horizontally                                                                              | Mirror image along horizontal axis.                                                                                |
| -mv             | Mirror vertically                                                                                | Mirror image along vertical axis.                                                                                  |
| -ho             | Horizontal offset (pixels)                                                                       | Set horizontal offset in pixels.                                                                                   |
| -vo             | Vertical offset (pixels)                                                                         | Set vertical offset in pixels.                                                                                     |
| -hk             | Horizontal keystone angle                                                                        | Set horizontal keystone compensation angle in degrees.                                                             |
| -vk             | Vertical keystone angle                                                                          | Set vertical keystone compensation angle in degrees.                                                               |
| -z              | Zoom                                                                                             | Set image zoom value: less than 1 to zoom out, greater than 1 to zoom in.                                          |
| -pb             | Radial distortion                                                                                | Set radial distortion compensation value in the range: [-0.45..0.45].                                              |
| -f              | Mesh file name                                                                                   | Path to the warp mesh file generated by the Warp SoC Example application                                           |
| continued...    |                                                                                                  |                                                                                                                    |



| Option | Description             | Usage                                                                                                                                               |
|--------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| -e     | Number of warp engines. | Number of the warp processing engines. Supported values: 1 or 2 (default);                                                                          |
| -db    | Double bounce           | Assume double bounce Warp IP configuration. By default, the tool assumes Single bounce configuration. Use this option to override default behavior. |
| -mm    | Use mipmaps             | Use mipmaps to allow warps with downscale ratios to a limit of 256:1                                                                                |
| -s     | Run warp simulator      | Warp a reference image using provided settings and save result on disk. The output file name is <code>warp_output.bmp</code> .                      |

### Examples

This command:

```
vidip_bct.exe -iw 1920 -ih 1080 -ow 3840 -oh 2160 -r 15.0 -s
```

- Sets the input resolution to full HD, output resolution to 4K
- Performs a 15 degree rotation anticlockwise
- Stores warped reference image on the disk.

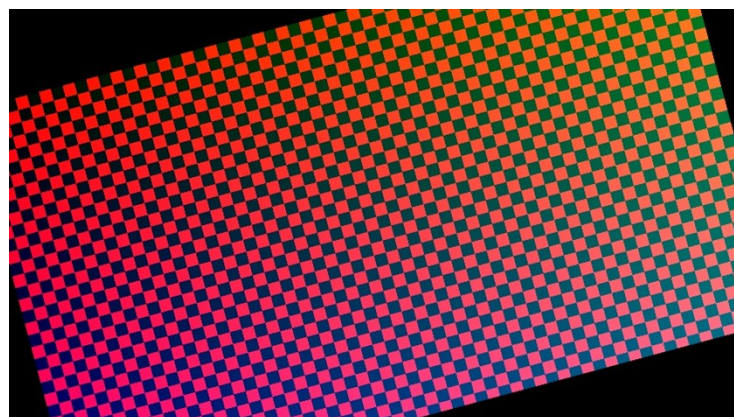
The console output is:

```
intel_vvp_warp block cache tool v2.0
Input resolution: 1920x1080
Output resolution: 3840x2160
Warp engines: 4
Bounce: Single
Mipmaps: No
Generating warp data...

Cache size Result Warnings
256 PASS
512 PASS
1024 PASS
```

The console output indicates the IP can perform the configured warp with all available block cache size options.

**Figure 150. Warped reference image**



This command:

```
vidip_bct.exe -iw 3840 -ih 2160 -ow 3840 -oh 2160 -e 2 -f
flying_flag.owf -mm -s
```

- Uses the warp mesh file generated by the Warp SoC Example Design, named `flying_flag.owf`
- Sets input and output resolutions to 4K
- Sets number of warp engines to 2
- Enables mipmap usage
- Stores warped reference image on the disk.

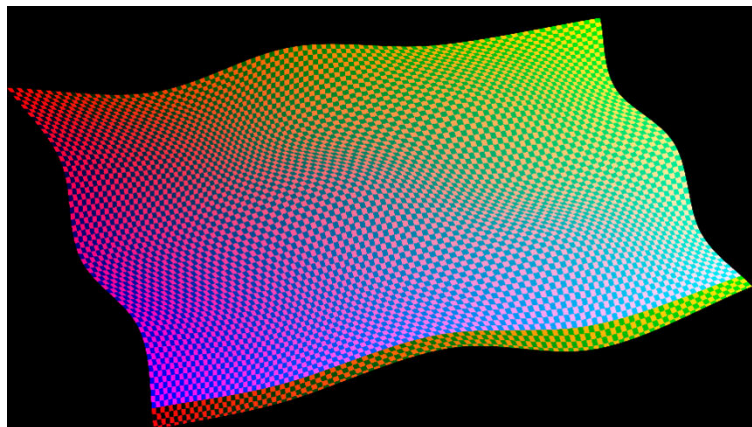
The console output is:

```
intel_vvp_warp block cache tool v2.0
Input resolution: 3840x2160
Output resolution: 3840x2160
Warp engines: 2
Bounce: Single

Generating warp data...
Cache size Result Warnings
256 PASS Block cache usage too high
512 PASS
1024 PASS
```

The console output indicates the IP can perform the warp using block cache sizes of 512 and 1024. With the block cache size of 256, the tool detects a high block cache usage. The IP cannot guarantee the output image quality and requires further evaluation on the actual hardware.

**Figure 151. Warped reference image**



This command:

```
vidip_bct.exe -z 0.45
```

- Uses default input and output resolution of 8K
- Scaled the image down by 0.45

The console output is

```
intel_vvp_warp block cache tool v2.0
Input resolution: 7680x4320
Output resolution: 7680x4320
Warp engines: 4
Bounce: Single
Mipmaps: No
Generating warp data...

Cache size Result Warnings
256 PASS Compression too high
```

The console output indicates the transform exceeds the maximum compression of 2:1 and therefore the output image quality is not guaranteed.

### Related Information

[Warp SoC Design Example](#)

## 53.3.2. Warp IP Interfaces

The Warp IP has four functional interfaces.

The functional interfaces are:

- Altera video stream input interface
- Altera video stream output interface
- Avalon Memory-Mapped compatible Processor interface
- Avalon Memory-Mapped compatible memory interface

### Avalon Memory-Mapped Processor interface

The Warp IP control interface uses a 32bit Avalon Memory-Mapped interface to access control registers.

**Table 1111. Avalon Memory-Mapped Processor interface Signals**

| Signal name                       | Direction | Width | Description                                                                      |
|-----------------------------------|-----------|-------|----------------------------------------------------------------------------------|
| av_mm_control_agent_address       | Input     | 13    | The byte address of the register being accessed.                                 |
| av_mm_control_agent_write         | Input     | 1     | Assert to indicate a write transfer.                                             |
| av_mm_control_agent_byteenable    | Input     | 4     | Enables one or more byte lanes during a write transfer.                          |
| av_mm_control_agent_writedata     | Input     | 32    | Data for write transfers.                                                        |
| av_mm_control_agent_read          | Input     | 1     | Assert to indicate a read transfer.                                              |
| av_mm_control_agent_readdata      | Output    | 32    | Data for read transfers.                                                         |
| av_mm_control_agent_readdatavalid | Output    | 1     | Asserted by the IP to indicate valid read data.                                  |
| av_mm_control_agent_waitrequest   | Output    | 1     | Asserted by the IP to indicate that the host must wait to complete the transfer. |

### Avalon Memory-Mapped Memory interface

The Warp IP memory interface uses a 512-bit Avalon Memory-Mapped interface to access external memory.

| Signal name                     | Direction | Width | Description                                                                            |
|---------------------------------|-----------|-------|----------------------------------------------------------------------------------------|
| av_mm_memory_host_waitrequest   | Input     | 1     | Asserted by the agent to indicate that the Warp IP must wait to complete the transfer. |
| av_mm_memory_host_readdata      | Input     | 512   | Data for read transfers.                                                               |
| av_mm_memory_host_readdatavalid | Input     | 1     | Assert to indicate valid read data.                                                    |
| av_mm_memory_host_response      | Input     | 2     | The response status of the agent.                                                      |
| av_mm_memory_host_burstcount    | Output    | 4     | Indicates the number of transfers in each burst.                                       |
| av_mm_memory_host_writedata     | Output    | 512   | Data for write transfers.                                                              |
| av_mm_memory_host_address       | Output    | 32    | The byte address of the memory location being accessed.                                |
| av_mm_memory_host_write         | Output    | 1     | Asserted to indicate a write transfer.                                                 |
| av_mm_memory_host_read          | Output    | 1     | Asserted to indicate a read transfer.                                                  |
| av_mm_memory_host_byteenable    | Output    | 64    | Enables one or more byte lanes during a write transfer.                                |
| av_mm_memory_host_debugaccess   | Output    | 1     | Not used by the Warp IP.                                                               |

### Clocking

The Warp IP has five clock domains, each with a corresponding reset. For the Agilex 7 family, the video and processing clock domains can run up to 600 MHz. The CPU and memory interface clock domains can run up to 500 MHz. For all other device families the clock domains run up to 300 MHz.

**Table 1112. Clock Domains**

| Clock name                | Description                      |
|---------------------------|----------------------------------|
| av_mm_control_agent_clock | CPU interface clock domain       |
| av_mm_memory_host_clock   | Memory interface clock domain    |
| axi4s_vid_in_0_clock      | Input video stream clock domain  |
| axi4s_vid_out_0_clock     | Output video stream clock domain |
| core_clock                | Processing engine clock domain   |

The CPU interface uses little bandwidth and does not impose a minimum clock frequency.

The video clock frequency depends on the video resolution and frame rate and the Warp IP's number of pixels in parallel. For example, a 600 MHz clock at 1 pixel in parallel supports resolutions up to 3840x2160 at 60 fps. A 300 MHz clock at 2 pixels in parallel supports active video resolutions up to 3840x2160 at 60 fps. A 150 MHz clock at 1 pixel in parallel supports resolutions up to 1920x1080 at 60 fps.

All RTL-based blocks that transfer or receive data from a different clock domain include clock domain crossing (CDC) circuits for both, single bit and data bus signal cases. The CDC circuits safely allow exchange of data between the two asynchronous clock domains. The Warp IP includes an `.sdc` file to constrain these CDC paths.

When **Use easy warp** is on, Altera recommends that you connect the `core_clock` and `av_mm_memory_host_clock` inputs to the same clock source.

## Resets

**Table 1113. Resets associated with clock domains**

All resets are synchronous active-high

| Reset name                | Description                              |
|---------------------------|------------------------------------------|
| av_mm_control_agent_reset | CPU interface clock domain reset.        |
| av_mm_memory_host_reset   | Memory interface clock domain reset.     |
| axi4s_vid_in_0_reset      | Input video stream clock domain reset.   |
| axi4s_vid_out_0_reset     | Output video stream clock domain reset.  |
| core_reset                | Processing engine(s) clock domain reset. |

All the resets in the Warp IP are synchronous. Ensure that, when resetting the Warp IP, all clocks are active at the same time while you apply the resets.

## Interrupts

**Table 1114. Interrupt Signals**

| Signal        | Description                                                                                                                                                                                                                                                                                             |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| interrupt_irq | Active high interrupt triggered at the start of each output frame sent from the axi4s_vid_out_0 interface. The signal is synchronous to the av_mm_control_agent_clock domain.<br>Enable <code>tinterrupt_irq</code> using the interrupt control register and clear using the interrupt status register. |

## Related Information

[Video and Vision Processing IP Interfaces](#) on page 30

### 53.3.3. Warp IP Latency

The IP has three latency ranges that depend on if you turn on **Use Easy warp**, turn on **Use Single Memory bounce**, and if you synchronize the input and output frame rates.

The IP does not support low latency behavior when **Use mipmaps** is on.

The Warp IP latency is:

- A quarter of a frame up to a whole frame, if you lock the input and output frame rates,
- Between one and two frames if you turn on **Use Easy warp** and for normal latency behavior when you turn on **Use single memory bounce**.
- Between two to three frames for normal latency behavior when you turn off **Use single memory bounce**.

The IP does not control the relative timings of the input and output processes. Their timings can be asynchronous relative to each other. The flow control from the video pipeline that is in place downstream of the output determines the exact timings of the output video data from the IP.

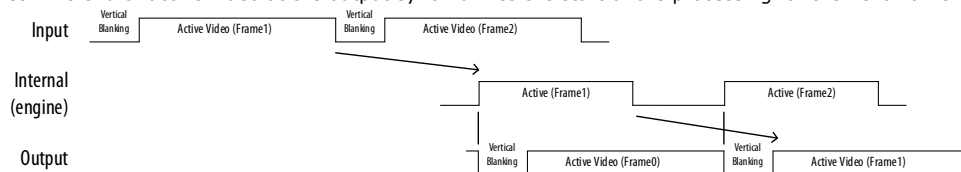
In a simple implementation, the output frames in the system have no timing relationship with the input frames. The IP automatically adapts to their different frame rates by dropping and repeating frames of video data as necessary.

In a sophisticated implementation, you can lock the input and output frame rates with a fixed phase relationship existing between the input and output frame timings. You must implement this frame rate locking as part of the system level design, for example with an external genlock.

The figures show only the vertical blanking periods in active frames of video data. Assume any horizontal blanking periods are within the active video periods.

**Figure 152. Warp IP Latency**

The figure shows the buffering and processing of a video frame (frame1) through the Warp IP from the input, through the engine(s) to the output when **Use single memory bounce** is off. The latency is from two to three frames. The end of active video at the output synchronizes the start of the processing for the next frame.



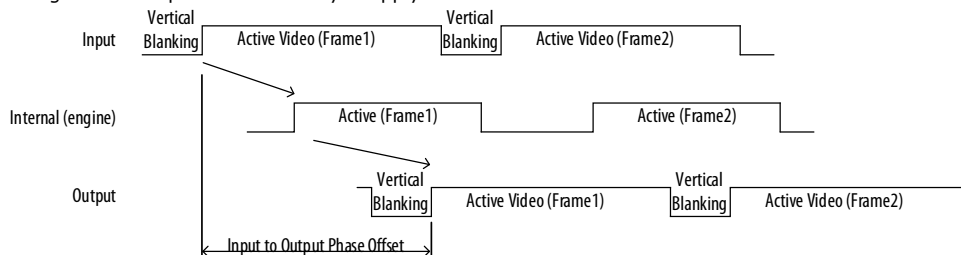
When the IP receives the whole of the input frame (frame 1), the internal processing engines can read the buffered frame from memory. Because the exact timing of the internal processing is synchronized with the timing of the output frames, the IP may see a delay of up to one frame before the internal processing engines use frame 1. When the IP completes the processing of the frame, the output process begins producing the resultant video data.

### Warp IP with Low Latency

**Figure 153. Warp IP with Low Latency**

The figure shows the low latency behavior when the input and output video frame rates and the input to output phase offset are locked.

The figure shows the buffering and processing of frame1 through the Warp IP from the input, through the engines to the output. The latency in this example can range from approximately a quarter to a whole frame depending on the warp transform that you apply.



Locking the input and output frame rates offset allows the IP to use video data as it buffers in memory, before it stores the whole frame. The figure shows a fixed relationship between the phasing of the input and output video frames and the end of active video at the output is no longer coincident with the start of the internal processing.

The engines begin processing the data from frame1 as soon as the IP stores sufficient input data, while it is receiving the frame. Similarly, the output process begins producing the video data associated with frame1 while the IP writes it to memory after processing by the engines. The delay from the start of the internal processing to the point where the output process can start reading the data from memory is a fixed amount based on the resolution of the output image the IP processes. However, the delay from the start of the input frame to the point where the internal processing can start using the frame data is a function of the warp transform. For this reason, the software API provides support in calculating these delays.

### Setting for Low Latency

For low latency behavior, program the correct settings in the IP and ensure the input and output frames have a fixed offset. The video system outside of the IP controls the fixed offset. This offset has a minimum setting based on the warp transform that you require. The `GenerateLatencyParams()` call, in the IP's software API, generates the values for the two settings for low-latency behavior. One of the settings for the IP programs the delay between the internal processing and the generation of active video at the output. The other setting is the delay, in clock cycles, between the input and output frames.

In addition to the required warp mesh, the `GenerateLatencyParams()` call requires the following information to calculate the settings:

`System_clock` – the frequency of `core_clock`, in Hz.

`Video_clock` – the frequency of `axi4s_vid_out_0_clock`, in Hz.

`Full_height` – the full height, in lines, of the resolution being processed including the vertical blanking.

`Frame_rate` – the output frame rate, in hundredths of a hertz.

Knowing the clock rates, the frame rate and the full height of the video frames enables the IP to calculate the number of clocks for each line of video data. Then, by examining the maximum line offsets implied by the required warp transform, the IP calculates how many clock cycles of delay it requires.

The `GenerateLatencyParams()` call returns two values via the structure `WarpLatencyParams`. These two values are:

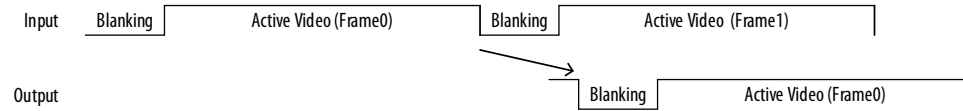
`_total_latency` – the minimum delay, in `axi4s_vid_out_0_clock` clock cycles, between the input and output frames.

`_output_latency` – the value to apply to the IP using the `intel_vvp_warp_set_output_latency()` call. For more information, refer to *Warp IP Software Code Examples*

Calling `intel_vvp_warp_set_output_latency()` with a non-zero `clock_offset` value switches the Warp IP core into the low latency mode of operation. Using a zero `clock_offset` value in the call to `intel_vvp_warp_set_output_latency()` puts the Warp IP in to the normal latency mode.

### Easy Warp IP Latency

Figure 154. Easy Warp IP Latency



When the IP receives the whole of the first input frame (frame 0), it can read the buffered data from memory by the output process of the Warp IP (which gives one frame of latency). If you do not lock the input and output frame rates, you can see up to a frame's delay before the IP produces the output video data, which gives a total of two frames delay maximum.

### Related Information

Warp IP Software Code Examples on page 1116

## 53.3.4. External Memory for Warp IP

The IP requires access to two separate areas of external memory: one for its input and output video buffers and one for its coefficient tables. The processor system running the Warp Software API must be able to access the coefficient tables but does not need access to the buffer area.

### Memory Space Allocation in External Memory

Table 1115. Warp IP Video Buffer Memory Region - double bounce without mipmaps

The table defines how much space is required in external memory by the Warp IP for the video buffer region when you turn off **Use easy warp**, turn off **Use single memory bounce**, and turn off **Use mipmaps**. This space depends on the size of the images to be processed in a system. It is defined by the **Memory frame buffer size** parameter. Six buffers require space in total: four input and two output.

| Buffer Space Configuration  | Region Size (MB) | Memory Region Required | Alignment (multiples of) |
|-----------------------------|------------------|------------------------|--------------------------|
| SD buffer size (1024x1024)  | 24               | 0x0180_0000            | 0x0200_0000              |
| HD buffer size (2048x2048)  | 96               | 0x0600_0000            | 0x0800_0000              |
| UHD buffer size (4096x4096) | 384              | 0x1800_0000            | 0x2000_0000              |
| 8K buffer size (8192x8192)  | 1536             | 0x6000_0000            | 0x8000_0000              |

Table 1116. Warp IP Video Buffer Memory Region - single bounce without mipmaps

The table defines how much space is required in external memory by the Warp IP for the video buffer region when you turn on **Use easy warp** or turn on **Use single memory bounce**, and turn off **Use mipmaps**. This space depends on the size of the images to be processed in a system. It is defined by the **Memory frame buffer size** parameter. Four buffers require space in total.

| Buffer Space Configuration                                  | Region Size (MB) | Memory Region Required | Alignment (multiples of) |
|-------------------------------------------------------------|------------------|------------------------|--------------------------|
| SD buffer size (1024x1024)                                  | 16               | 0x0100_0000            | 0x0100_0000              |
| HD buffer size (2048x2048)                                  | 64               | 0x0400_0000            | 0x0400_0000              |
| UHD buffer size (4096x4096)                                 | 256              | 0x1000_0000            | 0x1000_0000              |
| 8K buffers size (8192x8192)<br>Not supported with easy warp | 1024             | 0x4000_0000            | 0x4000_0000              |



**Table 1117. Warp IP Video Buffer Memory Region – double bounce with mipmaps**

The table defines how much space is required in external memory by the Warp IP for the video buffer region when you turn off **Use easy warp**, turn off **Use single memory bounce** and turn on **Use mipmaps**. This space depends on the size of the images to be processed in a system. It is defined by the **Memory frame buffer size** parameter. Eight buffers require space in total: four for normal input, two for mipmap input and two for output.

| Buffer Space Configuration  | Region Size (MB) | Memory Region Required | Alignment (multiples of) |
|-----------------------------|------------------|------------------------|--------------------------|
| SD buffer size (1024x1024)  | 32               | 0x0200_0000            | 0x0200_0000              |
| HD buffer size (2048x2048)  | 128              | 0x0800_0000            | 0x0800_0000              |
| UHD buffer size (4096x4096) | 512              | 0x2000_0000            | 0x2000_0000              |
| 8K buffer size (8192x8192)  | 2048             | 0x8000_0000            | 0x8000_0000              |

**Table 1118. Warp IP Video Buffer Memory Region - single bounce with mipmaps**

The table defines how much space is required in external memory by the Warp IP for the video buffer region when you turn off **Use easy warp**, turn on **Use single memory bounce** and turn on **Use mipmaps**. This space depends on the size of the images to be processed in a system. It is defined by the **Memory frame buffer size** parameter. Six buffers require space in total: four for normal input and two for mipmap input.

| Buffer Space Configuration  | RegionSize (MB) | Memory Region Required | Alignment (multiples of) |
|-----------------------------|-----------------|------------------------|--------------------------|
| SD buffer size (1024x1024)  | 24              | 0x0180_0000            | 0x0200_0000              |
| HD buffer size (2048x2048)  | 96              | 0x0600_0000            | 0x0800_0000              |
| UHD buffer size (4096x4096) | 384             | 0x1800_0000            | 0x2000_0000              |
| 8K buffer size (8192x8192)  | 1536            | 0x6000_0000            | 0x8000_0000              |

The IP passes the base address of the memory region allocated to the frame buffers to the software API using the `ram_addr` element in the structure.

The memory region that the coefficient tables require is related to the number of warp engines, the resolution of the images, and the type of warp. The IP only requires this memory region when you turn off **Use easy warp**.

**Table 1119. Warp IP Coefficient Tables Memory Region**

The table shows the maximum size of the coefficient table memory region, per engine.

| Warp Engines | Region Size (MB) | Memory Region Required | Alignment (multiples of) |
|--------------|------------------|------------------------|--------------------------|
| 1            | 16               | 0x0100_0000            | 0x0100_0000              |
| 2            | 32               | 0x0200_0000            | 0x0200_0000              |
| 4            | 64               | 0x0400_0000            | 0x0400_0000              |

### Bandwidth to External Memory

The performance of the interface from the Warp IP to the external memory is important for the correct operation of a system using the Warp IP.

The Warp IP generates a substantial amount of memory traffic. It has up to four video streams passing to and from external memory. In addition, each engine has three read streams to access the coefficient tables. All these streams combine to make Warp IP memory accesses complex. The streams affect how much efficiency you can obtain when accessing memory such as DDR4.

The Warp IP memory controller mitigates potential inefficiencies caused by these complex access patterns. It uses burst lengths of 8 beats for all its read and write accesses to improve the burst performance of DDR4 memory. It also attempts to cluster individual read and write bursts together to eliminate some of the issues with read and write turnaround dead time at the DDR4 interface.

These memory access patterns depend on the image transform that you apply. Some complex image transforms may reduce memory traffic because of the skip region functionality. When **Use single memory bounce** is off, one of the worst transforms for generated memory traffic is a unity warp that gives a 1:1 mapping between input and output pixels. When **Use single memory bounce** is on, memory traffic is proportional to the amount of compression in the vertical direction of the transform. The higher the amount of compression in the transform vertically, the higher the memory bandwidth.

The operation of the Warp IP is easier to predict when it is the only user of the DDR4 memory in a system. Ensure the Warp IP is the only high bandwidth user of the DDR4 memory in a system. When other high bandwidth accesses are made to the memory at the same time as the Warp IP, ensure that any interactions don't adversely affect performance.

When **Use mipmaps** is on, the Warp IP generates two separate streams of input video data that the IP stores in memory: the original image data and the downscaled mipmap image data. The Warp IP software selects those regions of the input image where the downscaling is required and for these regions, only the downscaled image data is written to memory. The result is a net reduction in the amount of memory traffic from the input video stream compared to a design where **Use mipmaps** is set to off.

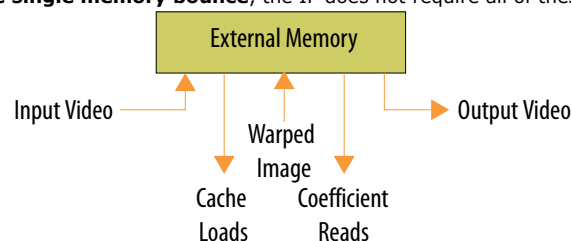
### Total Warp IP Memory Traffic

Three different configurations of the warp IP affect the way it connects to the external memory and hence affect the total memory traffic:

- When **Use easy warp** is on
- **Use easy warp** is off and **Use single memory bounce** is either on or off.

**Figure 155. External Memory Data Streams**

The figure shows a schematic view of all the possible data streams the IP uses. Depending on the settings of **Use easy warp** and **Use single memory bounce**, the IP does not require all of these streams.



**Table 1120. Active Data Streams for Different Configurations**

The table shows which external memory data streams are active for the different configuration settings.

| Data Stream       | Use easy warp on | Use single memory bounce on | Use single memory bounce off |
|-------------------|------------------|-----------------------------|------------------------------|
| Input video       | Active           | Active                      | Active                       |
| Cache loads       |                  | Active                      | Active                       |
| Warped image      |                  |                             | Active                       |
| Coefficient reads |                  | Active                      | Active                       |
| Output video      | Active           |                             | Active                       |

The bandwidth for each data stream is:

- A video stream (input video, warped image and output video)
- A coefficient stream (coefficient reads)
- Cache loads (a scaled version of a video stream).

#### Peak Memory Bandwidth Approximation for Video Streams

The equation calculates the peak bandwidth (in bits per second) for the video streams passing through the warp (where number\_of\_lines includes blanking). Each pixel of data is transferred to or from external memory as a 32bit word.

$$\text{Peak bandwidth} = 32 * \text{pixels\_per\_line} * \text{number\_of\_lines} * \text{frame\_rate}$$

**Table 1121. Video Stream Memory Bandwidth Examples**

The table shows the peak memory bandwidth per video stream for various resolutions and frame rates.

| Resolution             | Frame Rate (fps) | Peak Data Rate (Gbps) |
|------------------------|------------------|-----------------------|
| 7680x4320 (4500 lines) | 30               | 33                    |
| 3840x2160 (2250 lines) | 120              | 33                    |
| 3840x2160 (2250 lines) | 60               | 16.6                  |
| 3840x2160 (2250 lines) | 30               | 8.3                   |
| 1920x1080 (1125 lines) | 60               | 4.2                   |

#### Peak Memory Bandwidth Approximation for Coefficient Streams

You can approximate the memory bandwidth required by the coefficient streams as 9% of a video stream.

**Table 1122. Coefficient Stream Memory Bandwidth Examples**

The table shows the peak memory bandwidth per stream for various resolutions and frame rates.

| Resolution          | Frame Rate (fps) | Peak Data Rate (Gbps) |
|---------------------|------------------|-----------------------|
| 7680x4320           | 30               | 3.0                   |
| 3840x2160           | 120              | 3.0                   |
| <i>continued...</i> |                  |                       |

| Resolution | Frame Rate (fps) | Peak Data Rate (Gbps) |
|------------|------------------|-----------------------|
| 3840x2160  | 60               | 1.5                   |
| 3840x2160  | 30               | 0.75                  |
| 1920x1080  | 60               | 0.4                   |

### Peak Memory Bandwidth Approximation for Cache Loads

The memory bandwidth required by the IP for the cache loads is closely linked to the warp transform that the IP applies. The memory bandwidth is proportional to the bandwidth required by a standard video stream. It is also affected by **Use single memory bounce**.

When **Use single memory bounce** is off, the worst-case bandwidth required for cache loads is experienced for a 1:1 or unity warp and is approximately 25% higher than a normal video stream. The unity warp gives an upper bound for the bandwidth. Other transforms give lower.

When **Use single memory bounce** is on, the amount of vertical compression in the transform that the IP performs affects the bandwidth required for cache loads. A 1:1 or unity warp has a 1:1 vertical compression. The resultant cache load bandwidth is equivalent to the bandwidth for a standard video stream. In contrast, a 53 degree vertical keystone warp has a vertical compression approaching 2:1 in some regions. The maximum cache load bandwidth for this warp is two times the standard video stream bandwidth. The 2:1 ratio provides an upper bound for the cache load bandwidth for recommended operation.

The recommended 2:1 compression limit comes from the limitations of the low pass filtering that takes place in the bicubic pixel interpolation process when generating output pixels. However, you can generate transforms that have a compression ratio of greater than 2:1, which can potentially affect the single memory bounce memory bandwidth and reduce the final picture quality.

When **Use mipmaps** is on, any transform regions that have compressions of greater than 2:1 use the downscaled mipmap image data. The result is lower bandwidth for these regions.

### Memory Interface Bandwidth Requirements

**Table 1123. Memory Interface Bandwidth Requirements – Easy Warp**

The table shows the burst data rate scaling across resolutions and frame rates when **Use easy warp** is set to on. When **Use easy warp** is on, you can approximate the memory bandwidth using two video streams.

| Resolution | Frame Rate (fps) | Maximum Burst Data Rate (Gbps) |
|------------|------------------|--------------------------------|
| 3840x2160  | 60               | 34                             |
| 3840x2160  | 30               | 17                             |
| 1920x1080  | 60               | 8.5                            |

**Table 1124. Memory Interface Bandwidth Requirements – Single Memory Bounce**

The table shows the burst data rate scaling across resolutions and frame rates when **Use single memory bounce** is on. The cache load data rate assumes a 2:1 vertical compression factor, which gives worst-case figures. When **Use single memory bounce** is on, you can approximate the memory bandwidth using one video stream, a coefficient stream, and the cache load stream (with an upper bound of twice the bandwidth of a standard video stream).

| Resolution | Frame Rate (fps) | Maximum Burst Data Rate (Gbps) |
|------------|------------------|--------------------------------|
| 7680x4320  | 30               | 103                            |
| 3840x2160  | 120              | 103                            |
| 3840x2160  | 60               | 51.3                           |
| 3840x2160  | 30               | 25.7                           |
| 1920x1080  | 60               | 8.5                            |

**Table 1125. Memory Interface Bandwidth Requirements - Double Memory Bounce**

The table shows the burst data rate scaling across resolutions and frame rates when **Use single memory bounce** is off. The cache load data rate assumes a 1:1 warp transform, which gives the worst-case figures. You can approximate the memory bandwidth using three video streams, a coefficient stream, and the cache load stream (with a 25% overhead above the bandwidth of a standard video stream).

| Resolution | Frame Rate (fps) | Maximum Burst Data Rate (Gbps) |
|------------|------------------|--------------------------------|
| 7680x4320  | 30               | 140                            |
| 3840x2160  | 120              | 140                            |
| 3840x2160  | 60               | 72                             |
| 3840x2160  | 30               | 36                             |
| 1920x1080  | 60               | 18                             |

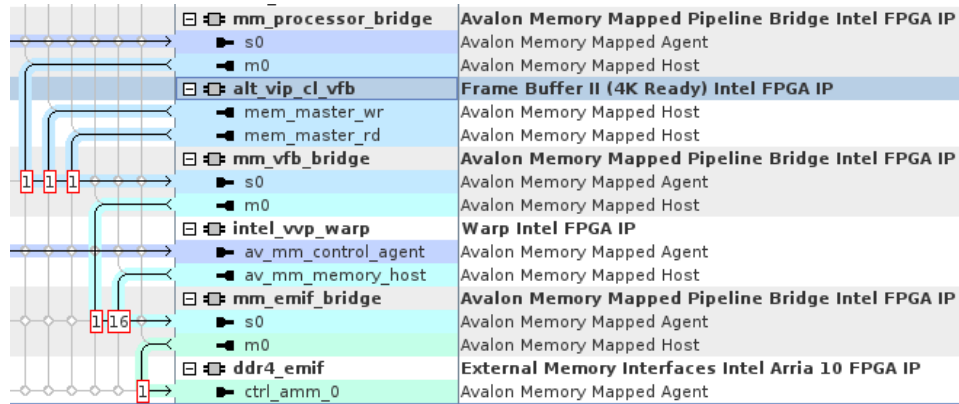
Altera references these burst data rates to the memory interface of the IP. The total data rates available are affected by other factors outside of the IP such as the performance of the interconnect fabric and the efficiency of the memory controller.

#### Example system sharing access to memory at UHD 60 fps

In this example system the Warp IP shares the DDR4 interface with a frame buffer in a system that processes UHD frames at 60 fps. The system runs on an Arria 10 GX Development Kit with the DDR4 EMIF running a 2,133 MHz interface to a DDR4 memory. This system has four memory-mapped hosts accessing the memory: processor, frame buffer read, frame buffer write, and warp.

**Figure 156. Warp and Video Frame Buffer Platform Designer**

The figure shows the Platform Designer connectivity where the Frame Buffer II component is sharing access to the DDR4 EMIF with the Warp IP. The Frame Buffer is part of the same video processing pipeline as the Warp IP. For clarity, the figure only shows the Avalon memory-mapped Interfaces and **Show Arbitration Shares** is on.



For this system to work:

- Configure Frame Buffer to use bursts of 32 beats for read and write.
- Configure Frame Buffer to use read and write FIFO depths of 256
- Set the arbitration weighting at the front end of the DDR4 EMIF to 16:1 in favor of the Warp IP (versus the processor and the Frame Buffer's read and write interfaces connected through the mm\_vfb\_bridge component).
- Set the **Maximum pending read transactions** parameter in the pipelined transfers section of the Avalon memory-mapped ports to be at 8.
- Set **Limit interconnect pipeline stages to** for the domain at the front end of the DDR4 EMIF to 4. This limit helps to meet timing on the memory interface clock domain.

Memory accesses from the processor may adversely affect the performance of the rest of the system. If the performance is affected, increase the arbitration weightings at the mm\_vfb\_bridge component to be 1:2:2 or higher in favor of the Frame Buffer's read and write interfaces. Alternatively, implement a fixed priority arbitration scheme at the mm\_vfb\_bridge to reduce the effect of the processor's memory accesses.

Figure 157. Video Frame Buffer Parameterization

The screenshot shows the parameterization window for the 'Frame Buffer II (4K Ready) Intel FPGA IP'. The window has a title bar with a 'Details' button. Below the title bar, the IP name 'Frame Buffer II (4K Ready) Intel FPGA IP' and its identifier 'alt\_vip\_cl\_vfb' are displayed. The main area is divided into two sections: 'Video Data Format' and 'Memory'. The 'Video Data Format' section includes fields for 'Maximum frame width' (4096), 'Maximum frame height' (2160), 'Bits per color sample' (10), 'Number of color planes' (3), a checked checkbox for 'Color planes transmitted in parallel', a 'Number of pixels in parallel' dropdown (set to 2), and an unchecked checkbox for 'Interlace support'. The 'Memory' section includes a checked checkbox for 'Use separate clock for the Avalon-MM master interface(s)', and five dropdown menus for 'Avalon-MM master(s) local ports width' (512), 'FIFO depth Write' (256), 'Av-MM burst target Write' (32), 'FIFO depth Read' (256), and 'Av-MM burst target Read' (32).

| Video Data Format                                                        |      |
|--------------------------------------------------------------------------|------|
| Maximum frame width:                                                     | 4096 |
| Maximum frame height:                                                    | 2160 |
| Bits per color sample:                                                   | 10   |
| Number of color planes:                                                  | 3    |
| <input checked="" type="checkbox"/> Color planes transmitted in parallel |      |
| Number of pixels in parallel:                                            | 2    |
| <input type="checkbox"/> Interlace support                               |      |

| Memory                                                                                       |     |
|----------------------------------------------------------------------------------------------|-----|
| <input checked="" type="checkbox"/> Use separate clock for the Avalon-MM master interface(s) |     |
| Avalon-MM master(s) local ports width:                                                       | 512 |
| FIFO depth Write:                                                                            | 256 |
| Av-MM burst target Write:                                                                    | 32  |
| FIFO depth Read:                                                                             | 256 |
| Av-MM burst target Read:                                                                     | 32  |

**Figure 158. Maximum Pending Read Transactions**

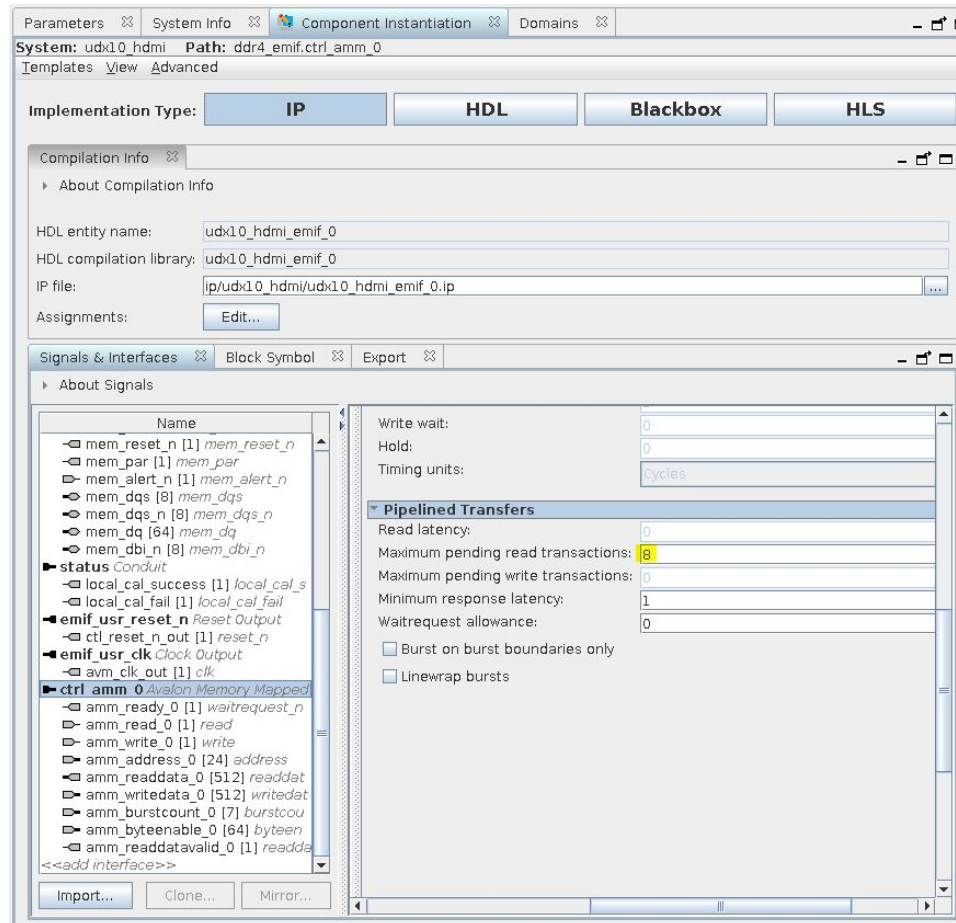
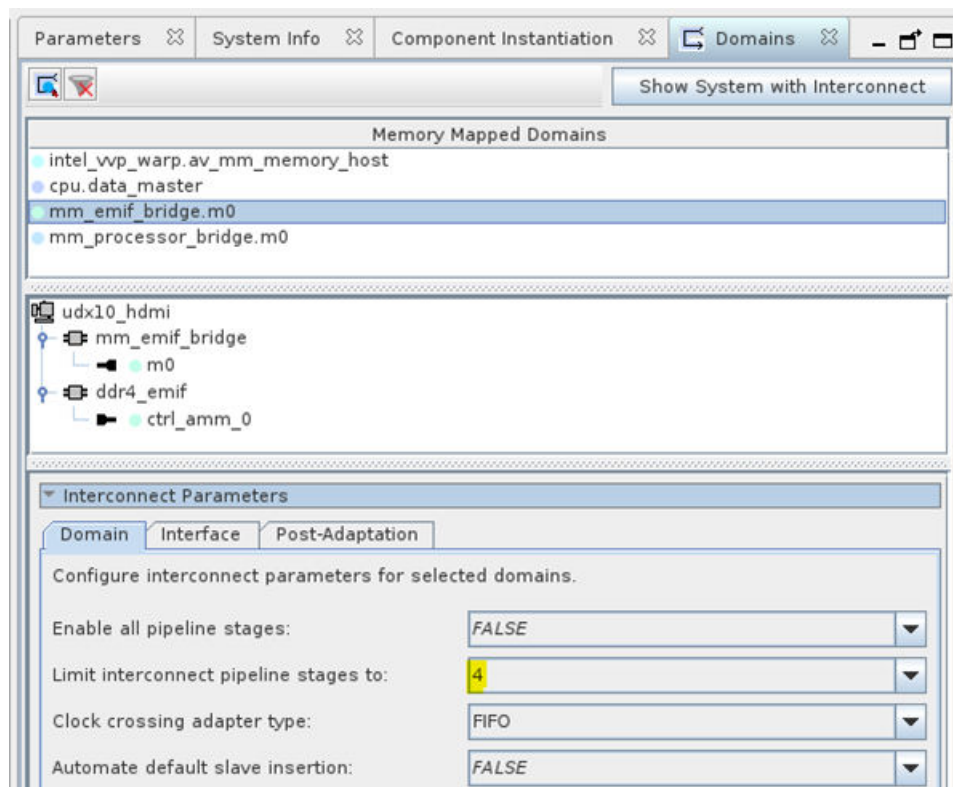




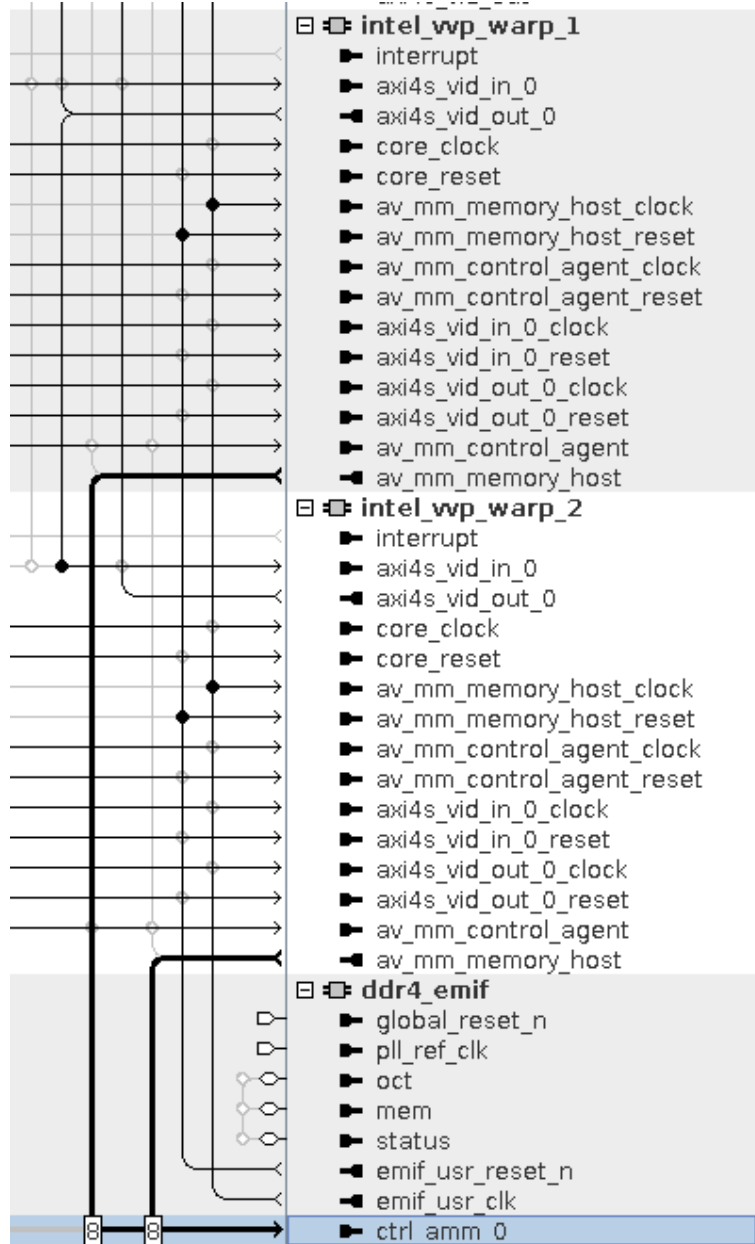
Figure 159. Parameterizing interconnect pipeline stages



## Multiple Warp IPs sharing access to memory

**Figure 160. Multiple Warp IPs sharing access to memory**

The figure shows an example with two Warp IPs that share a DDR4 interface. To match the burst access patterns of the Warp IP, set the arbitration values at the combining interface to 8.



### Example system running at UHD 120 fps or 8K 30 fps

In this example system the Warp IP accesses a DDR4 interface in a system that processes UHD frames at 120 fps or 8K frames at 30 fps. The system runs on an Agilex 7 I-Series Transceiver-SoC Development Kit with the DDR4 EMIF running a 2,666 Mbps interface to a DDR4 memory. This speed of DDR4 interface gives a memory clock speed of 333 MHz.

For this system to work:

- Set the arbitration weighting at the front end of the DDR4 EMIF to 16:1 in favor of the Warp IP (versus the processor).
- Set the maximum pending read transactions parameter in the pipelined transfers section of the Avalon memory-mapped ports to be at 64.
- Set limit interconnect pipeline stages to for the domain at the front end of the DDR4 EMIF to 4. This limit helps to meet timing on the memory interface clock domain.
- Configure the Warp IP with 4 pixels in parallel with 4 engines while using **Memory frame buffer size** set to 8K..
- Run the Warp IP's video interface and core processing clocks at 600 MHz. This speed ensures the stable operation of the video output with the higher bandwidth requirement of 4K120 or 8k30 operation.

## 53.4. Warp IP Registers

As the software API allows you to program and control the warp IP, it only has a limited set of registers.

**Table 1126. Warp IP Registers**

All the registers are 32-bit wide.

| Register Name      | Offset Address | Access Type | Description                                                 |
|--------------------|----------------|-------------|-------------------------------------------------------------|
| vid_pid            | 0x000          | RO          | Warp IP product and vendor ID                               |
| version_number     | 0x004          | RO          | The version for this release of the Warp IP                 |
| Reserved           | 0x008          | -           |                                                             |
| Reserved           | 0x00C          | -           |                                                             |
| pip                | 0x010          | RO          | Indicates the value of pixels in parallel parameter         |
| color_planes       | 0x014          | RO          | Indicates the value of number of color planes parameter     |
| cps                | 0x018          | RO          | Indicates the value of bits per color sample parameter      |
| num_engines        | 0x01C          | RO          | Indicates the value of number of engines parameter          |
| max_input_width    | 0x020          | RO          | Indicates the value of maximum input video width parameter  |
| max_output_width   | 0x024          | RO          | Indicates the value of maximum output video width parameter |
| memory_buffer_size | 0x028          | RO          | Indicates the value of memory frame buffer size parameter   |
| easy_warp          | 0x02C          | RO          | Indicates the value of <b>Use easy warp</b> parameter       |
| continued...       |                |             |                                                             |

| Register Name        | Offset Address | Access Type | Description                                                             |
|----------------------|----------------|-------------|-------------------------------------------------------------------------|
| single_memory_bounce | 0x030          | RO          | Indicates the value of <b>Use single memory bounce</b> parameter        |
| cache_blocks         | 0x034          | RO          | Indicates the value of <b>Cache blocks per engine</b> parameter         |
| mipmaps              | 0x038          | RO          | Indicates the value of <b>Use mipmaps</b> parameter                     |
| Reserved             | 0x03C-0x18C    | RO          | -                                                                       |
| int_control          | 0x190          | RW          | Turns on the interrupt                                                  |
| int_status           | 0x194          | RW1C        | Read interrupt status and clear interrupt                               |
| Reserved             | 0x198-0x33C    | -           | -                                                                       |
| Various Debug        | 0x340-0x53C    | RO          | Use debug register access API as described in the <i>Software API</i> . |

**Table 1127. vid\_pid Register**

| Bits  | Name | Description                                    |
|-------|------|------------------------------------------------|
| 31:16 | VID  | Vendor ID that returns a value of 0x6AF7       |
| 15:0  | PID  | Warp product ID that returns a value of 0x016F |

**Table 1128. version\_number Register**

| Bits | Name           | Description                       |
|------|----------------|-----------------------------------|
| 31:0 | Version Number | The version number of the Warp IP |

**Table 1129. pip Register**

| Bits | Name               | Description                                                 |
|------|--------------------|-------------------------------------------------------------|
| 31:0 | Pixels in Parallel | The pixel in parallel parameter. Returns a value of 1 or 2. |

**Table 1130. color\_planes Register**

| Bits | Name                   | Description                                                 |
|------|------------------------|-------------------------------------------------------------|
| 31:0 | Number of Color Planes | The number of color planes parameter. Returns a value of 3. |

**Table 1131. bps Register**

| Bits | Name                  | Description                                                 |
|------|-----------------------|-------------------------------------------------------------|
| 31:0 | Bits per Color Sample | The bits per color sample parameter. Returns a value of 10. |

**Table 1132. num\_engines Register**

| Bits | Name              | Description                                                 |
|------|-------------------|-------------------------------------------------------------|
| 31:0 | Number of Engines | The number of engines parameter. Returns a value of 1 or 2. |

**Table 1133. max\_input\_width Register**

| Bits | Name                      | Description                                                                           |
|------|---------------------------|---------------------------------------------------------------------------------------|
| 31:0 | Maximum input video width | The maximum input video width parameter. Returns a value of 2048, 3840, 4096 or 7860. |

**Table 1134. max\_output\_width Register**

| Bits | Name                       | Description                                                                            |
|------|----------------------------|----------------------------------------------------------------------------------------|
| 31:0 | Maximum output video width | The maximum output video width parameter. Returns a value of 2048, 3840, 4096 or 7860. |

**Table 1135. memory\_buffer\_size Register**

| Bits | Name                     | Description                                                                                             |
|------|--------------------------|---------------------------------------------------------------------------------------------------------|
| 31:0 | Memory frame buffer size | The memory frame buffer size parameter. Returns a value of 0, 1, 2, or 3. 0 = SD, 1=HD, 2=UHD, and 3=8K |

**Table 1136. easy\_warp Register**

| Bits | Name          | Description                                                    |
|------|---------------|----------------------------------------------------------------|
| 31:0 | Use easy warp | The <b>Use easy warp</b> parameter. Returns a value of 0 or 1. |

**Table 1137. single memory bounce Register**

| Bits | Name                 | Description                                                               |
|------|----------------------|---------------------------------------------------------------------------|
| 31:0 | Single memory Bounce | The <b>Use single memory bounce</b> parameter. Returns a value of 0 or 1. |

**Table 1138. cache\_blocks\_per\_engine Register**

| Bits | Name         | Description                                                                        |
|------|--------------|------------------------------------------------------------------------------------|
| 31:0 | Cache blocks | The <b>Cache blocks per engine</b> parameter. Returns a value of 256, 512 or 1024. |

**Table 1139. mipmaps Register**

| Bits | Name        | Description                                                  |
|------|-------------|--------------------------------------------------------------|
| 31:0 | Use mipmaps | The <b>Use mipmaps</b> parameter. Returns a value of 0 or 1. |

**Table 1140. int\_control Register**

| Bits | Name             | Description                                                                       |
|------|------------------|-----------------------------------------------------------------------------------|
| 0    | Interrupt Enable | Setting this bit to 1 enables the interrupt. Setting to 0 disables the interrupt. |

**Table 1141. int\_status Register**

| Bits | Name             | Description                                                                                                                                                                                           |
|------|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0    | Interrupt Status | Reading from this bit returns the status of the interrupt.<br>Writing a 1 to this bit clears the interrupt. Once triggered, the interrupt remains set until it is cleared by writing a 1 to this bit. |

### Debug and Measurement Registers

When you turn on **Enable Debug Registers** in the configuration settings of the Warp IP, a set of read only registers are available that provide various debug and measurement readings. The registers provide information useful during system bring up and debug and are accessed using the software API.

The available measurements are:

- Input section:
  - ☐ INPUT\_FRAME\_COUNT
  - ☐ INPUT\_FRAME\_PERIOD
  - ☐ INPUT\_MEM\_DIRECT\_WRITES
  - ☐ INPUT\_MEM\_MIPMAP\_WRITES
- Output section
  - ☐ OUTPUT\_FRAME\_COUNT
  - ☐ OUTPUT\_FRAME\_PERIOD
- Engine section
  - ☐ ENGINE\_FRAME\_COUNT
  - ☐ ENGINE\_FRAME\_PERIOD
  - ☐ ENGINE\_BUSY\_CYCLES
  - ☐ ENGINE\_FRAME\_DELAY
  - ☐ ENGINE\_FAILED\_TO\_COMPLETE\_FRAME
  - ☐ ENGINE\_RUN\_STATUS
  - ☐ ENGINE\_CACHE\_LOADS
- General section
  - ☐ MEM\_WR\_QUEUES0
  - ☐ MEM\_WR\_QUEUES1
  - ☐ MEM\_RD\_QUEUES0
  - ☐ MEM\_RD\_QUEUES1
  - ☐ -MEM\_RD\_ACCESSES
  - ☐ MEM\_WR\_ACCESSES
  - ☐ MEM\_STALLED\_ACCESSES

**Table 1142. Registers**

| Name                                                                                                                                                                              | Description                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input Section</b><br>The IP reads the debug registers relating to the input section of the Warp IP using the <code>intel_vvp_warp_get_input_debug_register ()</code> API call. |                                                                                                                                                                                                                                                                                                                                                              |
| INPUT_FRAME_COUNT                                                                                                                                                                 | A count of the number of frames that have started at the input of the Warp IP. The last frame counted may be incomplete.<br>This counter can indicate the video activity at the input of the Warp IP. Sampling this count periodically indicates frame rate. A static count indicates that the upstream pipeline to the Warp IP is not producing video data. |
| INPUT_FRAME_PERIOD                                                                                                                                                                | The number of <code>axi4s_vid_in_0_clock</code> cycles seen between the last two start of frames seen at the input.                                                                                                                                                                                                                                          |
| <i>continued...</i>                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                              |

| Name                                                                                                                                                                             | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                  | Use this counter to indicate the frame rate by comparing its value with the input clock frequency. If the value from this register varies greatly, it indicates an unstable processing pipeline upstream of the Warp IP.                                                                                                                                                                                                                                                                                                                                                                                                             |
| INPUT_MEM_DIRECT_WRITES                                                                                                                                                          | The number of blocks of direct, non-mipmap data the IP writes to memory during the processing of the last frame of input data. A block is defined as a 16x8 region of pixel data.                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| INPUT_MEM_MIPMAP_WRITES                                                                                                                                                          | The number of blocks of mipmap data the IP writes to memory during the processing of the last frame of input data. A block is defined as a 16x8 region of pixel data. This value is only available when <b>Use mipmap</b> is on.                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Output Section</b><br>The debug registers relating to the output section of the Warp IP are read using the <code>intel_vvp_warp_get_output_debug_register ()</code> API call. |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| OUTPUT_FRAME_COUNT                                                                                                                                                               | A count of the number of frames that have started at the output. The last frame counted may be incomplete.<br>This counter can indicate the video activity at the output of the Warp IP. Sampling this count periodically gives an indication of frame rate and a static count indicates that the downstream pipeline from the Warp IP is blocked and not accepting video data.                                                                                                                                                                                                                                                      |
| OUTPUT_FRAME_PERIOD                                                                                                                                                              | The number of <code>axi4s_vid_out_0</code> clock cycles seen between the last two start of frames seen at the output.<br>This counter can indicate the frame rate by comparing its value with the output clock frequency. If the value from this register varies wildly, it indicates an unstable processing pipeline downstream of the Warp IP.                                                                                                                                                                                                                                                                                     |
| <b>Engine Section</b><br>The debug registers relating to the engine section of the Warp IP are read using the <code>intel_vvp_warp_get_engine_debug_register ()</code> API call. |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| ENGINE_FRAME_COUNT                                                                                                                                                               | A count of the number of frames that have started to be processed by the engine being referenced. The processing of the last frame counted may be in progress.<br>This counter is useful for indicating the video activity within the Warp IP. It gives an indication that the Warp IP is set up and that the engine is configured to process correctly.                                                                                                                                                                                                                                                                             |
| ENGINE_FRAME_PERIOD                                                                                                                                                              | The number of <code>core_clock</code> cycles seen between the last two start of frames seen at the engine.<br>This counter is useful for confirming that the <code>core_clock</code> is set correctly given the expected frame rate. It is also useful for comparing against the <code>ENGINE_BUSY_CYCLES</code> value as an indication of how busy the engine is.                                                                                                                                                                                                                                                                   |
| ENGINE_BUSY_CYCLES                                                                                                                                                               | The number of <code>core_clock</code> cycles for which the engine actively processes video data during the last frame. The counter starts when the processing of a frame begins and continues until either the IP completes the frame or encounters the start of the next frame. For the latter the engine fails to complete the processing of the frame in time, refer to the <code>ENGINE_FAILED_TO_COMPLETE_FRAME</code> status indicator.<br>When compared to the <code>ENGINE_FRAME_PERIOD</code> , the value of <code>ENGINE_BUSY_CYCLES</code> gives an indication of how busy an engine is during the processing of a frame. |
| ENGINE_FRAME_DELAY                                                                                                                                                               | The number of <code>core_clock</code> cycles between the start of the last input frame and the start of the engine's processing phase.<br>Use this counter for low latency behavior to determine if you have enough delay when programming the low latency settings. It also indicates the input and output frame synchronization that is a requirement for correct low latency behavior. This delay must be stable for low latency IPs to work correctly. If the IP does not see a stable value, the delay is changing over time, which indicates the input to output frame synchronization may not be operating correctly.         |
| continued...                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

| Name                                                                                                                                                                        | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ENGINE_FAILED_TO_COMPLETE_FRAME                                                                                                                                             | <p>A sticky flag that indicates that the engine has failed to complete the processing of a frame in the required frame period. The flag is cleared each time it is interrogated.</p> <p>The IP sets this to indicate that the engine has not had sufficient time to process the video frame as required and image corruption occurs. The most likely reason is insufficient memory bandwidth available to the engine. Insufficient memory bandwidth gives processing stalls either while waiting for read data to return or stalls while blocked waiting for write data is sent out.</p>                                                                                                                         |
| ENGINE_RUN_STATUS                                                                                                                                                           | <p>This value is only relevant when <b>Use single memory bounce</b> is on.</p> <p>The lower 16 bits of the value indicate the lowest buffering level the IP reaches during the processing of the previous frame. The buffering level is the difference between the write position and the read position of the current cache accesses. If this value is 0, it indicates that the engine stalls during the processing of the frame. This stall can give unstable output video when <b>Use single memory bounce</b> is on.</p> <p>The upper 16 bits of the value is the output line at which the IP reaches the lowest buffering level. They indicate at which point in the transform this output line occurs.</p> |
| ENGINE_CACHE_LOADS                                                                                                                                                          | <p>The number of cache loads the IP produces while the IP processes the previous frame. Each cache load reads a 16x8 pixel block (128 pixels in total). The total number of cache reads indicates the memory bandwidth load that the current warp transform is placing on the memory.</p>                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>General Section</b><br>The debug registers relating to the general section of the Warp IP are read using the <code>intel_vvp_warp_get_debug_register ()</code> API call. |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| MEM_WR_QUEUES0                                                                                                                                                              | <p>Indicates the number of internal requests that queue at any one time for each of the write accesses through the Warp IP's memory controller. The IP stores each queue depth value as a byte in the 32-bit data word returned by the read to this register. The IP has a number of queues depending on the configuration (<a href="#">Separate Queue Values (Write Accesses)</a>).</p>                                                                                                                                                                                                                                                                                                                         |
| MEM_RD_QUEUES0 and MEM_RD_QUEUES1                                                                                                                                           | <p>Indicates the number of internal requests that queue at any one time for each of the read accesses through the Warp IP's memory controller. The IP has two 32 bit registers for access to the read queue depths. Each individual queue depth value is stored as a nibble in the 32 bit data word returned by a read to one of these registers. The IP has a number of queues depending on the configuration (<a href="#">Separate Queue Values (Read Accesses)</a>).</p>                                                                                                                                                                                                                                      |
| MEM_RD_ACCESSES                                                                                                                                                             | <p>The number of active memory read accesses issued by the Warp IP in the last second that were not blocked by the <code>av_mm_memory_host_waitrequest</code> signal. Each access represents a burst of eight cycles of data on the memory bus so the reported value should be multiplied by eight to get the number of actual read data transfer cycles.</p>                                                                                                                                                                                                                                                                                                                                                    |
| MEM_WR_ACCESSES                                                                                                                                                             | <p>The number of active memory write access cycles issued by the Warp IP in the last second that were not blocked by the <code>av_mm_memory_host_waitrequest</code> signal. This value represents the actual number of data transfer cycles that were active on the memory bus.</p>                                                                                                                                                                                                                                                                                                                                                                                                                              |
| MEM_STALLED_ACCESSES                                                                                                                                                        | <p>The number of cycles on the memory bus for which the IP blocks a read or write access cycle because of asserting of the <code>av_mm_memory_host_waitrequest</code> signal.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |



**Table 1143. Separate Queue Values (Write Accesses)**

The table shows the separate queue values for each configuration. The maximum depth value for each write queue is 48. If periodically sampling of the queue depth values finds them to be close to or at this maximum value, it indicates a limitation on write bandwidth from the IP to the external memory because the queues are filling up and blocking throughput.

| Queue          | Use Easy Warp or Use Single Memory Bounce are On | Single Engine        | Dual Engine           |
|----------------|--------------------------------------------------|----------------------|-----------------------|
| Video Input    | MEM_WR_QUEUES0[7:0]                              | MEM_WR_QUEUES0[7:0]  | MEM_WR_QUEUES0[7:0]   |
| Engine 0 Write | N/A                                              | MEM_WR_QUEUES0[15:8] | MEM_WR_QUEUES0[15:8]  |
| Engine 1 Write | N/A                                              | N/A                  | MEM_WR_QUEUES0[23:16] |

**Table 1144. Separate Queue Values (Read Accesses)**

The table shows the separate queue values for each configuration.

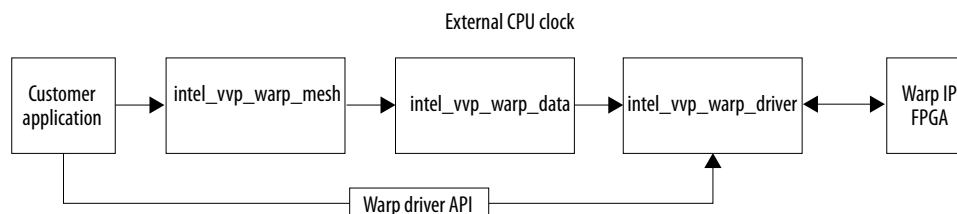
The maximum value for each read queue depth is 12. If periodically sampling of queue depth values finds them close to or at this maximum value, it indicates a limitation on read bandwidth from the IP to the external memory because the queues are filling up and blocking throughput.

| Queue           | Easy Warp           | Single Engine         | Dual Engine           |
|-----------------|---------------------|-----------------------|-----------------------|
| Video Output    | MEM_RD_QUEUES0[3:0] | MEM_RD_QUEUES0[3:0]   | MEM_RD_QUEUES0[3:0]   |
| Engine 0 Video  | N/A                 | MEM_RD_QUEUES0[7:4]   | MEM_RD_QUEUES0[7:4]   |
| Engine 0 Coeff0 | N/A                 | MEM_RD_QUEUES0[11:8]  | MEM_RD_QUEUES0[11:8]  |
| Engine 0 Coeff1 | N/A                 | MEM_RD_QUEUES0[15:12] | MEM_RD_QUEUES0[15:12] |
| Engine 0 Coeff2 | N/A                 | MEM_RD_QUEUES0[19:16] | MEM_RD_QUEUES0[19:16] |
| Engine 1 Video  | N/A                 | N/A                   | MEM_RD_QUEUES0[23:20] |
| Engine 1 Coeff0 | N/A                 | N/A                   | MEM_RD_QUEUES0[27:24] |
| Engine 1 Coeff1 | N/A                 | N/A                   | MEM_RD_QUEUES0[31:28] |
| Engine 1 Coeff2 | N/A                 | N/A                   | MEM_RD_QUEUES1[3:0]   |

## 53.5. Warp IP Software API

The Warp IP software is implemented in C and C++. This topic describes the API functions in the code and provides code examples.

**Figure 161. Warp software architecture**



The software includes:

- `intel_vvp_warp_driver`
- `intel_vvp_warp_data`
- `intel_vvp_warp_mesh`

The `intel_vvp_warp_driver` is the warp IP driver. It provides API for initializing, configuring, and controlling the Warp IP, warp video channel management, and using debug features.

The `intel_vvp_warp_data` is the software component that generates mesh, cache, and filter coefficient data required by the warp IP. Place generated data in a region of RAM accessed by the warp IP. This location and other necessary parameters are passed to the warp driver.

This component requires transformation mesh in a predefined format as the main input data. It also requires some IP parameters e.g. number of available engines, processing block dimensions, block cache size, single memory bounce setting.

The `intel_vvp_warp_mesh` is the software component that generates warp meshes for common affine transformations such as translation, rotation, zoom; perspective transformations e.g. keystone and radial distortion (fisheye) and arbitrary warps defined by a set of curves. Meshes are generated in the format required by the `intel_vvp_warp_data` interface.

You can use this reference software in a number of real-world applications. However, in some more complex cases you might want to use external software to generate transformation mesh. For example, when projecting image onto a complex surface.

You can deploy the software on a Nios V based system. However, because of computational intensity of the mesh and data generating components, Altera recommends an SoC with a dedicated CPU such as an Arria 10 SX device.

#### `intel_vvp_warp_driver`

**Table 1145. `intel_vvp_warp_driver` API reference**

The software driver for Warp IP provides the following set of API functions.

| Name                                                 | Description                                   |
|------------------------------------------------------|-----------------------------------------------|
| <code>intel_vvp_warp_init_instance</code>            | Initialize driver instance                    |
| <code>intel_vvp_warp_create_channel</code>           | Create single engine video processing channel |
| <code>intel_vvp_warp_create_double_channel</code>    | Create dual engine video processing channel   |
| <code>intel_vvp_warp_create_quad_channel</code>      | Create quad engine video processing channel   |
| <code>intel_vvp_warp_create_easy_warp_channel</code> | Create easy warp video processing channel     |
| <code>intel_vvp_warp_add_engine_to_channel</code>    | Add a warp engine to existing channel         |
| <code>intel_vvp_warp_free_channel</code>             | Free video processing channel                 |
| <code>intel_vvp_warp_configure_channel</code>        | Configure video processing channel            |
| <code>intel_vvp_warp_apply_transform</code>          | Apply video transformation                    |
| <code>intel_vvp_warp_bypass</code>                   | Bypass video processing                       |
| <code>intel_vvp_warp_reset_skip_ram</code>           | Reset skip RAM page                           |
| <code>intel_vvp_warp_reset_input_skip_ram</code>     | Reset input skip RAM page                     |
| <code>intel_vvp_warp_check_easy_warp_capable</code>  | Check if a warp channel can perform Easy warp |
| <i>continued...</i>                                  |                                               |

| Name                                        | Description                                     |
|---------------------------------------------|-------------------------------------------------|
| intel_vvp_warp_set_easy_warp                | Set easy warp transformation                    |
| intel_vvp_warp_set_output_latency           | Set warp output latency                         |
| intel_vvp_warp_get_channel_framebuffer_size | Get total size of the frame buffer RAM required |
| intel_vvp_warp_get_debug_register           | Get contents of a warp debug register           |
| intel_vvp_warp_get_input_debug_register     | Get contents of a warp input debug register     |
| intel_vvp_warp_get_output_debug_register    | Get contents of a warp output debug register    |
| intel_vvp_warp_get_engine_debug_register    | Get contents of a warp engine debug register    |
| intel_vvp_warp_get_engine_status            | Get current status flags of the warp engine     |
| intel_vvp_warp_get_output_status            | Get current status flags of the warp output     |

### intel\_vvp\_warp\_init\_instance

```
int intel_vvp_warp_init_instance(intel_vvp_warp_instance_t*
instance, intel_vvp_warp_base_t base)
```

**Description** Initialize intel\_vvp\_warp driver instance

**Arguments** instance – pointer to the intel\_vvp\_warp software driver instance structure

base – hardware access handler. In a bare metal environments such as Nios II it is defined as 32-bit unsigned integer representing physical address of the intel\_vvp\_warp IP on the CPU bus.

**Return value** Zero on success, negative integer otherwise

**intel\_vvp\_warp\_create\_channel**  
**intel\_vvp\_warp\_create\_double\_channel**  
**intel\_vvp\_warp\_create\_quad\_channel**

```
intel_vvp_warp_channel_t*
intel_vvp_warp_create_channel(intel_vvp_warp_instance_t
*instance, uint32_t input_idx, uint32_t engine_idx, uint32_t
output_idx) intel_vvp_warp_channel_t*
intel_vvp_warp_create_double_channel(intel_vvp_warp_instance_t
*instance, uint32_t input_idx, uint32_t engine1_idx, uint32_t
engine2_idx, uint32_t output_idx)
intel_vvp_warp_create_quad_channel(intel_vvp_warp_instance_t
*instance, uint32_t input_idx, uint32_t output_idx)
```

**Description** Create a video processing channel. The functions allocate hardware resources and initialize driver data structures necessary for processing a video stream up to 7680x4320 pixels.

**Arguments** instance – pointer to initialized intel\_vvp\_warp\_instance\_t structure;

input\_idx, engine\_idx, engine2\_idx, output\_idx – indexes of the warp IP input, engines and output to use in this channel.

You can configure a single warp IP to provide multiple input, output, and processing engine blocks. These blocks are grouped into video processing channels arbitrarily by using index of the block.

The indexes must be in the range 0..num\_inputs(num\_outputs or num\_engines respectively), where you can obtain num\_inputs, num\_outputs and num\_engines through the intel\_vvp\_warp\_instance\_t structure after you initialize the driver instance.

**Return value** Valid pointer to initialized intel\_vvp\_warp\_channel\_t structure on success, null pointer otherwise.

#### intel\_vvp\_warp\_create\_easy\_warp\_channel

```
intel_vvp_warp_channel_t*
intel_vvp_warp_create_easy_warp_channel(intel_vvp_warp_instance_t
*instance, uint32_t input_idx, uint32_t output_idx)
```

**Description** Create an easy warp video processing channel. The functions allocate hardware resources and initialize driver data structures necessary for processing a video stream up to 3840x2160 pixels using easy warp configuration.

**Arguments** instance – pointer to initialized intel\_vvp\_warp\_instance\_t structure;

input\_idx, output\_idx – indexes of the warp IP input and output to use in this channel.

**Return value** Valid pointer to initialized intel\_vvp\_warp\_channel\_t structure on success, null pointer otherwise.

#### intel\_vvp\_warp\_add\_engine\_to\_channel

```
int
intel_vvp_warp_add_engine_to_channel(intel_vvp_warp_channel_t*
channel, uint32_t engine_idx)
```

**Description** Add a warp engine to existing channel.

Warp IP allows you to split video processing between multiple engines. This function adds additional engines to already allocated channels and updates driver and channel data structures accordingly

*Arguments* instance – pointer to the intel\_vvp\_warp software driver instance structure  
engine\_idx – indexes of the warp IP engine to add to the channel

*Return value* Zero on success, negative integer otherwise

#### intel\_vvp\_warp\_free\_channel

```
void intel_vvp_warp_free_channel(intel_vvp_warp_channel_t* channel)
```

*Description* Delete video processing channel and release hardware resources and driver data structures allocated for it

*Arguments* channel – a valid pointer to initialized intel\_vvp\_warp\_channel\_t structure

*Return value* None

#### intel\_vvp\_warp\_configure\_channel

```
int intel_vvp_warp_configure_channel(intel_vvp_warp_channel_t* channel, intel_vvp_warp_channel_config_t* cfg)
```

*Description* Configure video processing channel data structures and allocated hardware by providing necessary parameters such as input/output resolutions, frame buffers location etc.

*Arguments* channel – a valid pointer to initialized video processing channel;  
cfg – a valid pointer to initialized intel\_vvp\_warp\_channel\_config\_t structure.

*Return value* Zero on success, negative integer otherwise

#### intel\_vvp\_warp\_apply\_transform

```
int intel_vvp_warp_apply_transform(intel_vvp_warp_channel_t* channel, intel_vvp_warp_data_t* data)
```

*Description* Start warping video by configuring the channel to use provided warp data coefficients

*Arguments* channel – a valid pointer to initialized video processing channel

data – a valid pointer to initialized intel\_vvp\_warp\_data\_t structure.

*Return value* Zero on success, negative integer otherwise

#### intel\_vvp\_warp\_bypass

```
int intel_vvp_warp_bypass(intel_vvp_warp_channel_t* channel,
uint32_t bypass, uint32_t skip_ram_page, uint32_t width,
uint32_t height)
```

*Description* Enable or disable video bypass where the input video displays as is without any processing

*Arguments* channel – a valid pointer to initialized video processing channel  
bypass – a nonzero value to enable video bypass, zero to disable  
skip\_ram\_page – number of skip ram page to use after enabling or disabling bypass. The software uses the same page number for both input and output blank skip logic.  
width, height – override output resolution if necessary. Both width and height have to be nonzero. Typically, use to match the input resolution when enabling video bypass.

*Return value* Zero on success, negative integer otherwise

#### intel\_vvp\_warp\_reset\_skip\_ram

```
void intel_vvp_warp_reset_skip_ram(intel_vvp_warp_channel_t*
channel, uint32_t skip_ram_page)
```

*Description* Reset skip RAM page setting all values to zero.

Skip RAM pages are programmed as part of intel\_vvp\_warp\_apply\_transform(). Some use scenarios may require skip RAM to reset explicitly. This function allows for that

*Arguments* channel – a valid pointer to initialized video processing channel  
skip\_ram\_page – skip RAM page to reset

*Return value* None

### intel\_vvp\_warp\_reset\_input\_skip\_ram

```
void intel_vvp_warp_reset_skip_ram(intel_vvp_warp_channel_t*
channel, uint32_t skip_ram_page)
```

- Description** Reset input skip RAM page setting all values to zero.
- Skip RAM pages are programmed as part of intel\_vvp\_warp\_apply\_transform(). Some scenarios may require skip RAM to reset explicitly, which this function allows for
- Arguments** channel – a valid pointer to initialized video processing channel
- skip\_ram\_page – input skip RAM page to reset
- Return value** None

### intel\_vvp\_warp\_check\_easy\_warp\_capable

```
int
intel_vvp_warp_check_easy_warp_capable(intel_vvp_warp_channel_t*
channel)
```

- Description** Check if the channel can perform easy warp transforms
- Arguments** channel – a valid pointer to initialized video processing channel
- Return value** Zero if the channel can perform easy warp transforms, negative integer otherwise

### intel\_vvp\_warp\_set\_easy\_warp

```
int intel_vvp_warp_set_easy_warp (intel_vvp_warp_channel_t*
channel, uint32_t easy_warp)
```

- Description** Apply an easy warp transformation to the input video
- Arguments** channel – a valid pointer to initialized, easy warp capable video processing channel
- easy\_warp – desired easy warp transformation as 32-bit unsigned integer encoded in the following way:

|          |   |        |          |   |
|----------|---|--------|----------|---|
| 31       | 3 | 2      | 1        | 0 |
| Reserved |   | Mirror | Rotation |   |

- Bits 1:0 Rotation field can have the following values:
  - ☐ 00b – no rotation;
  - ☐ 01b – 90 degrees counter-clockwise rotation
  - ☐ 10b – 180 degrees rotation
  - ☐ 11b – 270 degrees counter-clockwise rotation
- Bit 2: mirrors input video if set. If combined with a rotation the mirror is applied first.

*Return value* Zero on success, negative integer otherwise

#### intel\_vvp\_warp\_set\_output\_latency

```
void intel_vvp_warp_set_output_latency
(intel_vvp_warp_channel_t* channel, uint32_t clock_offset)
```

*Description* Offset the warp output start-of-processing with the start-of-output by the specified number of system clock cycles. For more information, refer to *Warp Low Latency*.

*Arguments* channel – a valid pointer to initialized video processing channel  
clock\_offset – number of system clock cycles by which to offset the output start-of-processing

*Return value* None

#### intel\_vvp\_warp\_get\_channel\_framebuffer\_size

```
void
intel_vvp_warp_get_channel_framebuffer_size
(intel_vvp_warp_channel_t* channel)
```

*Description* Get total size of RAM required for the channel frame buffers.

*Arguments* channel - a valid pointer to initialized video processing channel

*Return* Size of required RAM in bytes

#### intel\_vvp\_warp\_get\_debug\_register

```
uint32_t intel_vvp_warp_get_debug_register (intel_vvp_warp_instance_t*
instance, uint32_t reg_offset)
```

*Description* Read content of a warp debug and measurement register. Refer to *Warp Debug and Measurement Registers*.



**Arguments** instance – pointer to initialized intel\_vvp\_warp\_instance\_t structure

reg\_offset – zero based register offset. Use the following register offset macros for this argument:

| Macro                                 | Value |
|---------------------------------------|-------|
| INTEL_VVP_WARP_DEBUG_MEM_WR_QUEUES0   | 0     |
| INTEL_VVP_WARP_DEBUG_MEM_WR_QUEUES1   | 1     |
| INTEL_VVP_WARP_DEBUG_MEM_RD_QUEUES0   | 2     |
| INTEL_VVP_WARP_DEBUG_MEM_RD_QUEUES1   | 3     |
| INTEL_VVP_WARP_DEBUG_MEM_RD_CYCLES    | 8     |
| INTEL_VVP_WARP_DEBUG_MEM_WR_CYCLES    | 9     |
| INTEL_VVP_WARP_DEBUG_MEM_STALL_CYCLES | 10    |

**Return value** Current content of the warp debug and measurement register specified by the reg\_offset argument

#### intel\_vvp\_warp\_get\_input\_debug\_register

```
uint32_t intel_vvp_warp_get_input_debug_register
(intel_vvp_warp_instance_t* instance, uint32_t input_idx,
uint32_t reg_offset)
```

**Description** Read content of a warp input debug and measurement register. Refer to *Warp Debug and Measurement Registers*.

**Arguments** instance – pointer to initialized intel\_vvp\_warp\_instance\_t structure

input\_idx – index of the warp IP input to access

reg\_offset – zero based register offset. Use the following register offset macros for this argument:

| Macro                                        | Value |
|----------------------------------------------|-------|
| INTEL_VVP_WARP_DEBUG_INPUT_FRAME_COUNT       | 0     |
| INTEL_VVP_WARP_DEBUG_INPUT_FRAME_PERIOD      | 1     |
| INTEL_VVP_WARP_DEBUG_INPUT_MEM_WRITES        | 2     |
| INTEL_VVP_WARP_DEBUG_INPUT_MIPMAP_MEM_WRITES | 3     |

**Return value** Current content of the warp input debug and measurement register specified by the reg\_offset argument

### intel\_vvp\_warp\_get\_output\_debug\_register

```
uint32_t intel_vvp_warp_get_output_debug_register
(intel_vvp_warp_instance_t* instance, uint32_t output_idx,
uint32_t reg_offset)
```

**Description** Read content of a warp output debug and measurement register. Refer to *Warp Debug and Measurement Registers*.

**Arguments** instance – pointer to initialized intel\_vvp\_warp\_instance\_t structure

output\_idx – index of the warp IP output to access

reg\_offset – zero based register offset. Use the following register offset macros for this argument:

| Macro                                    | Value |
|------------------------------------------|-------|
| INTEL_VVP_WARP_DEBUG_OUTPUT_FRAME_COUNT  | 0     |
| INTEL_VVP_WARP_DEBUG_OUTPUT_FRAME_PERIOD | 1     |

**Return value** Current content of the warp output debug and measurement register specified by the reg\_offset argument

### intel\_vvp\_warp\_get\_engine\_debug\_register

```
uint32_t intel_vvp_warp_get_engine_debug_register
(intel_vvp_warp_instance_t* instance, uint32_t engine_idx,
uint32_t reg_offset)
```

**Description** Read content of a warp output debug and measurement register. Refer to *Warp Debug and Measurement Registers*.

**Arguments** instance – pointer to initialized intel\_vvp\_warp\_instance\_t structure

engine\_idx – index of the warp IP engine to access

reg\_offset – zero based register offset. Use the following register offset macros for this argument:

| Macro                                    | Value |
|------------------------------------------|-------|
| INTEL_VVP_WARP_DEBUG_ENGINE_FRAME_COUNT  | 0     |
| INTEL_VVP_WARP_DEBUG_ENGINE_FRAME_PERIOD | 1     |
| INTEL_VVP_WARP_DEBUG_ENGINE_BUSY_CYCLES  | 2     |
| INTEL_VVP_WARP_DEBUG_ENGINE_FRAME_DELAY  | 3     |
| INTEL_VVP_WARP_DEBUG_ENGINE_RUN_STATUS   | 4     |
| INTEL_VVP_WARP_DEBUG_ENGINE_CACHE_LOADS  | 5     |

**Return value** Current content of the warp engine debug and measurement register specified by the `reg_offset` argument

### `intel_vvp_warp_get_engine_status`

```
uint32_t intel_vvp_warp_get_engine_status
(intel_vvp_warp_instance_t* instance, uint32_t engine_idx)
```

**Description** Get current status flags of a warp engine. Refer to *Warp Debug and Measurement Registers*.

**Arguments** `instance` – pointer to initialized `intel_vvp_warp_instance_t` structure

`engine_idx` – index of the warp IP engine to access

**Return value** Current values of the engine status flags. The flags are in a 32 bit integer.

| Bit | Flag                            | Description                                                                                                          |
|-----|---------------------------------|----------------------------------------------------------------------------------------------------------------------|
| 0   | Engine failed to complete frame | A sticky flag indicating that the engine fails to complete the processing of a frame. The flag is cleared when read. |

### `intel_vvp_warp_get_output_status`

```
uint32_t intel_vvp_warp_get_output_status
(intel_vvp_warp_instance_t* instance, uint32_t output_idx)
```

**Description** Get current status flags of the warp output. Refer to *Warp Debug and Measurement Registers*.

**Arguments** `instance` – pointer to initialized `intel_vvp_warp_instance_t` structure

`output_idx` – index of the warp IP output to access

**Return** Current values of the output status flags. The flags are in a 32 bit integer.

| Bit | Flag                           | Description                                                                                                                                          |
|-----|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0   | Output frame has been repeated | A sticky flag indicating that the output frame repeated because the next rendered frame does not complete in time. The IP clears the flag when read. |

## Data structures and types

### intel\_vvp\_warp\_base\_t

**Description** Platform specific type to perform register read and write operations on the warp IP. In bare metal environments such as a Nios V processor, it is defined as 32-bit unsigned integer representing physical address of the warp IP on the CPU bus

### intel\_vvp\_warp\_instance

```
typedef struct intel_vvp_warp_instance
{
 intel_vvp_warp_base_t base;
 uint32_t num_inputs;
 uint32_t num_engines;
 uint32_t num_outputs;
} intel_vvp_warp_instance_t;
```

**Description** Main data structure of the driver

**Members**

- base – warp IP access handler
- num\_inputs – number of available warp inputs
- num\_engines – number of available warp engines
- num\_outputs – number of available warp outputs

### intel\_vvp\_warp\_channel

```
typedef struct intel_vvp_warp_channel
{
 uint32_t in_use;
 uint32_t idx;
 struct intel_vvp_warp_instance* instance;
 uint32_t num_engines;
 uint32_t ram_addr;
} intel_vvp_warp_channel_t
```

**Description** Video processing channel resources and parameters structure

#### Arguments

**Return value** in\_use – non-zero value if the channel is allocated and initialized. Zero otherwise.

idx – index number of the channel

instance – pointer to the parent driver instance the channel belongs to

num\_engines – number of warp engines for the channel

ram\_addr - base address of RAM region allocated for the channel frame buffers

#### intel\_vvp\_warp\_channel\_config

```
typedef struct intel_vvp_warp_channel_config
{
 uint32_t ram_addr;
 intel_vvp_warp_cs_t cs;
 uint32_t width_input;
 uint32_t height_input;
 uint32_t width_output;
 uint32_t height_output;
 uint8_t lfr;
} intel_vvp_warp_channel_config_t
```

**Description** Video processing channel configuration

**Members** ram\_addr - base address of RAM region allocated for the channel frame buffers

cs - video color space. Must be initialized to

- 00b = YUV
- 01b = RGB reduced range
- 10b = RFG full range.

width\_input, height\_input, width\_output, height\_output - input and output video dimensions

lfr - low frame rate fallback. Must be initialized to 0x0

#### intel\_vvp\_warp\_data

```
typedef struct intel_vvp_warp_data
{
 uint32_t num_engines;
 const uint8_t* skip_ram_input;
 const uint8_t* skip_megablock_data;
 uint32_t skip_ram_page;
 uint32_t mipmap_level;
 uint32_t mesh_step;
 intel_vvp_warp_engine_data_t engine_data[0];
} intel_vvp_warp_data_t;
```

**Description** Warp data descriptor

**Members** num\_engines - data for how many engines this set contains. .

skip\_ram\_input- data for input skip RAM. Generated by the intel\_vvp\_warp\_data library. Refer to Warp Data structure.

skip\_megablock\_data - data for the blank skip logic. Generated by the intel\_vvp\_warp\_data library. See WarpData structure.

\_kip\_ram\_page – skip RAM page to use for blank skip logic for this transform. Warp IP provides 2 pages for input and 2 pages for output blank skip logic. You can keep skip blank data for different transforms and switch between the pages as necessary. When a transform is applied the software uses the same page number for both input and output blank skip logic.

mipmap\_level - maximum mipmap level used by the current transform. Must be in the range of 0..7

mesh\_step - mesh step in pixels used by the current transform. Must be 8 or 16.

engine\_data – array of intel\_vvp\_warp\_engine\_data\_t structures containing warp coefficients for each individual engine

#### intel\_vvp\_warp\_engine\_data

```
typedef struct intel_vvp_warp_engine_data
{
 uint32_t mesh_addr;
 uint32_t mesh_size;
 uint32_t filter_addr;
 uint32_t filter_size;
 uint32_t fetch_addr;
 uint32_t fetch_size;
 uint32_t start_h;
 uint32_t start_v;
 uint32_t end_h;
 uint32_t end_v;
 uint32_t mesh_stride;
} intel_vvp_warp_engine_data_t;
```

**Description** Warp coefficient descriptor for an individual warp engine

**Members** mesh\_addr – address of the mesh coefficients in memory

mesh\_size – size of the mesh coefficients

filter\_addr – address of the filter coefficients in memory

filter\_size – size of the filter coefficients

fetch\_addr – address of the fetch coefficients in memory

fetch\_size – size of the fetch coefficients

start\_h, start\_v, end\_h, end\_v – engine processing region. Internally warp engine processes video in 16x8 pixel blocks with the top-left corner being block [0, 0]. The processing region is defined by the start and end block indexes in horizontal and vertical dimensions.

mesh\_stride – number of warp mesh nodes per mesh data row. (in sets of 4 minus 1). Required mesh stride is calculated by the intel\_vvp\_warp\_data library.

### intel\_vvp\_warp\_data

This software component uses transformation mesh in a predefined format to generate mesh, cache, and filter coefficient data required by the warp IP, and blank skip data if supported by the IP configuration.

If two or four engines are available, the software splits workload evenly so that each engine processes a vertical section of the frame. This software component also provides a factory function to create a warp mesh object for the given input and output resolutions and IP configuration.

**Table 1146. intel\_vvp\_warp\_data API reference**

| Name                  | Description                                                          |
|-----------------------|----------------------------------------------------------------------|
| WarpDataGenerator     | Constructor of the intel_vvp_warp_data component                     |
| GenerateData          | Generate warp data                                                   |
| GenerateLatencyParams | Generate recommended minimum latency settings for a given warp       |
| GetLastErrorCode      | Get warning or error code for the most recent call to GenerateData() |
| GetErrorString        | Get a string describing the error code passed as argument            |

**Table 1147. intel\_vvp\_warp\_data Helper functions**

| Name               | Description                                                            |
|--------------------|------------------------------------------------------------------------|
| GetHwContext       | Create object containing warp IP parameters necessary to generate data |
| GenerateDriverData | Create and fill in warp data object required by the warp driver        |

**Table 1148. intel\_vvp\_warp\_data Mesh factory function**

| Name             | Description                                                                                                          |
|------------------|----------------------------------------------------------------------------------------------------------------------|
| WarpMesh::Create | Create mesh object sufficient to hold transform data for the given input and output resolutions and IP configuration |

### WarpDataGenerator

WarpDataGenerator

**Description** Constructs the main object of the intel\_vvp\_warp\_data software component.

**Arguments** None

**Return value** None

### GenerateData

```
WarpDataPtr GenerateData(const WarpDataContext& ctx,
WarpMeshPtr& mesh)
```

|                     |                                                                                                                                                                                                                                                                                                                                                    |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i>  | Generate warp coefficients using provided transformation mesh, video resolution and necessary IP parameters. If two or four warp engines are available, the workload is split evenly such that each engine processes a vertical section of the image                                                                                               |
| <i>Arguments</i>    | ctx – reference to an instance of WarpDataContext object containing information about video resolution and IP parameters required to generate data<br><br>mesh – pointer to the transformation mesh                                                                                                                                                |
| <i>Return value</i> | On success the function returns a valid smart pointer to a WarpData object. If in any region of the frame the requested transform exceeds IP limitations such as maximum downscale ratio or block cache size, the region is rendered blank and the corresponding error code is set. You can retrieve the error code using GetLastErrorCode() call. |

### GenerateLatencyParams

```
WarpLatencyParamsGenerateLatencyParams(const WarpDataContext&
ctx, const WarpDataPtr warp_data, uint32_t system_clock,
uint32_t video_clock, uint32_t full_height, uint32_t
frame_rate) const
```

|                     |                                                                                                                                                                                                                                                                                                                                                                                                                          |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i>  | Generate recommended minimum warp latency settings for a given warp. The generated settings should be used to set up the video pipeline for the low latency processing. Refer to <i>Low Latency</i> .                                                                                                                                                                                                                    |
| <i>Arguments</i>    | ctx– reference to an instance of WarpDataContext object containing information about video resolution and IP parameters required to generate data<br><br>warp_data– a smart pointer to a valid WarpData object<br><br>system_clock – system clock frequency<br><br>video_clock – video clock frequency<br><br>full_height – full output video height (including vertical blanking)<br><br>frame_rate – output frame rate |
| <i>Return value</i> | Initialized WarpLatencyParams object containing recommended minimum latency settings for the given warp                                                                                                                                                                                                                                                                                                                  |

### GetLastErrorCode

```
uint32_t GetLastErrorCode() const
```



**Description** Return error or warning code for the most recent call to GenerateData(). The software indicates if warp data generation fails or an important condition is detected by setting a numerical error code. Use this function to retrieve the most recent error code.

**Arguments** None

**Return value** Possible return values are:

**Table 1149. Return Values**

| Macro               | Value | Description                                          |
|---------------------|-------|------------------------------------------------------|
| Success             | 0     | Data generated successfully                          |
| BlockCacheUsageHigh | 1     | Block cache usage too high. Output might be unstable |
| DataAllocError      | 2     | Unable to allocate memory for the warp data          |
| CompressionTooHigh  | 3     | Software detected compression higher than 2:1        |
| BlockCacheFull      | 4     | Requested warp uses up all available block cache     |
| FetchLogicError     | 5     | Error detected in the block cache control logic      |
| OutOfFetchMem       | 6     | Requested warp requires too many block cache fetches |
| Unknown             | 7     | All other errors                                     |

### GetErrorString

```
std::string GetErrorString(const uint32_t error_code)
const
```

**Description** Return string describing the error code obtained with GetLastErrorCode().

**Arguments** error\_code - integral error code value as returned by GetLastErrorCode()

**Return value** An object of type std::string containing textual description of the error code

### Data structures and types

#### WarpMesh

```
using resolution_t = std::pair<uint32_t, uint32_t>;
class WarpMesh
{
```

```
public:
 static WarpMeshPtr Create(resolution_t& input, resolution_t&
output, WarpHwContextPtr hw_ctx);
 uint32_t GetStep();
 uint32_t GetFractBits();
 uint32_t GetVNodes();
 uint32_t GetHNodes();
 uint32_t GetStreams();
 mesh_node_t* GetRow(uint32_t v);
}
```

**Description** Class containing user-defined transformation mesh.

The transformation mesh defines a mapping between the output and input images. Each entry of the mesh (node) represents a position in the output image and contains the relevant x and y coordinates of the input image encoded as 32-bit integers. The step between mesh nodes and number of fractional bits in the 32-bit values are selected by the software based on the input and output resolution. If any of the dimensions exceeds the value of 3840 the software uses 16x16 mesh step with 3 fractional bits otherwise 8x8 mesh with 4 fractional bits is used. The WarpMesh class allows for multiple mesh streams to be encoded. However only single stream mesh is currently supported by the IP. The class automatically manages internal storage for the mesh data

**Members** Create(resolution\_t& input, resolution\_t& output, WarpHwContextPtr hw\_ctx) – Factory method to create a mesh object.

**Argumenta** sinput, output – input and output frame dimensions.

hw\_ctx - smart pointer to an instance of WarpHwContext object containing relevant IP parameters

**Returns** On success the function returns a valid smart pointer to a WarpMesh object. If requested input or output resolution exceeds current IP settings the null pointer is returned.

uint32\_t GetStep() - Returns distance between mesh nodes

uint32\_t GetFractBits() – Returns number of fractional bits in 32-bit position values

uint32\_t GetVNodes()

uint32\_t GetHNodes() - Returns number of vertical/horizontal nodes in the mesh respectively

mesh\_node\_t\* GetRow(uint32\_t v) - Returns pointer to the mesh row. Access individual nodes in the row using pointer arithmetic or indexing

### mesh\_node\_t

```
typedef struct mesh_node
{
 int32_t _x;
 int32_t _y;
} mesh_node_t
```

**Description** Structure representing individual entry in the mesh

**Members** `_x, _y` – coordinates of the input image this node maps to. the IP represents each value by a 32-bit signed integer with the four least significant bits containing fractional part for subpixel precision.

### WarpMeshPtr

```
using WarpMeshPtr = std::shared_ptr<WarpMesh>
```

**Description** Type alias for a smart pointer wrapper around the mesh class

### WarpDataContext

```
struct WarpDataContext
{
 WarpDataContext(const WarpHwContextPtr hw, uint32_t wi,
uint32_t hi, uint32_t wo, uint32_t ho);
}
```

**Description** Structure containing image dimensions and IP parameters required to generate data.

**Members** `WarpDataContext(const WarpHwContextPtr hw, uint32_t wi, uint32_t hi, uint32_t wo, uint32_t ho)` - Constructor. The constructor uses provided arguments to initialize the data members, which you should not modify.

**Arguments** `hw` – smart pointer to an instance of `WarpHwContext` object containing relevant IP parameters

`wi, hi` – input image width and height

`wo, ho` – output image width and height

### WarpData

```
class WarpData
{
 virtual uint32_t _GetEngines() const;
 virtual const uint8_t* GetSkipRamInputData() const;
 virtual const uint8_t* GetSkipMegablockData() const;
 virtual const_WarpEngineData* GetEngineData(const uint32_t
engine) const;
 virtual uint32_t GetLatencyLines() const;
```

```
virtual uint32_t GetMipmapLevel() const;
virtual uint32_t GetMeshStep() const;
};
```

**Description** Interface class for accessing generated warp coefficients and data

**Members**

GetEngines() – returns the number of warp engine data objects. The value is always less or equal to MAX\_ENGINES

GetSkipRamInputData() – returns pointer to the input skip RAM data

GetSkipMegablockData() – returns pointer to the skip blank logic data

GetEngineData() – returns pointers to individual WarpEngineData objects

GetLatencyLines() – returns latency in image lines of the current transform. This value is used by the GenerateLatencyParams() call to calculate warp latency settings

GetMipmapLevel() – returns maximum mipmap level used by the current transform

GetMeshStep() – returns warp mesh step (8 or 16) used in this data set

### WarpEngineData

```
class WarpEngineData
{
 virtual uint32_t GetMeshEntries() const;
 virtual uint32_t GetFilterEntries() const;
 virtual uint32_t GetFetchEntries() const;

 virtual const mesh_entry_t* GetMeshData() const;
 virtual const filter_entry_t* GetFilterData() const;
 virtual const fetch_entry_t* GetFetchData() const;
};
```

**Description** Interface class for accessing data coefficients of individual warp engine

**Members**

GetMeshEntries() – returns number of mesh entries in the data block

GetFilterEntries() – returns number of filter entries in the data block

GetFetchEntries() – returns number of fetch entries in the data block

GetMeshData() – returns pointer to the mesh data

GetFilterData() – returns pointer to the filter data

GetFetchData() – returns pointer to the fetch data

`mesh_entry_t`, `filter_entry_t`, `fetch_entry_t`

**Description** Type aliases to use when accessing individual data blocks and calculating block data size in bytes.

.

### WarpLatencyParams

```
struct WarpLatencyParams
{
 uint32_t _output_latency;
 uint32_t _total_latency;
};
```

**Description** Structure contains recommended pipeline latency settings for the low latency processing. Refer to *Warp IP Latency*.

**Members** `_output_latency` - the recommended warp output clock offset to be used in `intel_vvp_warp_set_output_latency()` driver call. The value is in `core_clock` clock cycles

`_total_latency` – the recommended minimum offset between the input and output frames. The value is in `axi4s_vid_out_0_clock` clock cycles

### Helper functions

#### GetHwContext

```
WarpHwContextPtr GetHwContext(intel_vvp_warp_channel_t* ch)
```

**Description** Create object containing warp IP parameters necessary to generate data. Function uses warp channel driver structure to extract all necessary IP parameters and create object of the type `WarpHwContext`, required by the `GenerateData()` call

**Arguments** `ch` - valid pointer to initialized `intel_vvp_warp_channel_t` structure object `mesh_set` - reference to a set of transformation meshes

**Return value** On success the function returns a valid smart pointer to a `WarpHwContext` object. On failure a `nullptr` object is returned

## GenerateDriverData

```
WarpDriverDataPtr GenerateDriverData(const WarpDataContext&
ctx, const WarpDataPtr user_data, const uint32_t
coef_base_address, const uint32_t skip_ram_page)
```

|                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i>  | Create and fill in an object of type <code>intel_vvp_warp_data_t</code> required by the warp driver's method <code>intel_vvp_warp_apply_transform()</code> to apply warp.                                                                                                                                                                                                                                                                                                             |
| <i>Arguments</i>    | <p><code>ctx</code> - reference to the <code>WarpDataContext</code> object containing information about video resolution and parameters the IP uses to generate the warp data</p> <p><code>user_data</code> - Warp data as generated by the <code>GenerateData()</code> call</p> <p><code>coef_base_address</code> - base address in the FPGA accessible RAM where the warp is placed.</p> <p><code>skip_ram_page</code> - number of the blank skip ram page to use for this warp</p> |
| <i>Return value</i> | On success the function returns a valid smart pointer to an <code>intel_vvp_warp_data_t</code> object. On failure a <code>nullptr</code> object is returned. Before applying the warp the client must still transfer the warp data to the locations specified in the returned <code>intel_vvp_warp_data_t</code> object                                                                                                                                                               |

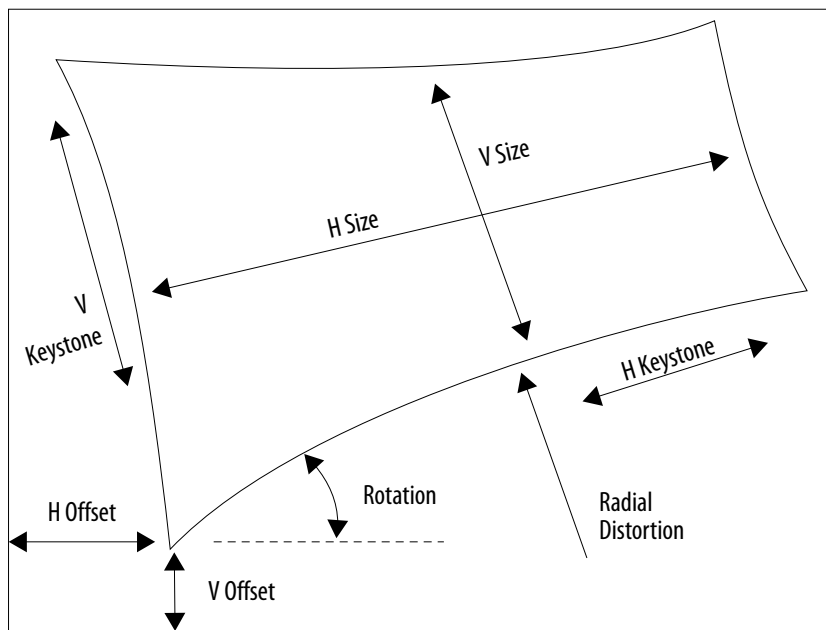
## intel\_vvp\_warp\_mesh

This reference software component allows you to generate sample warp meshes for common affine transformations, perspective transformations, radial distortion compensation and arbitrary transforms defined by a set of curves.

The main object of the component is `WarpConfigurator`. The `WarpConfigurator` provides the following three types of warp configurations that the host application can use independently.:

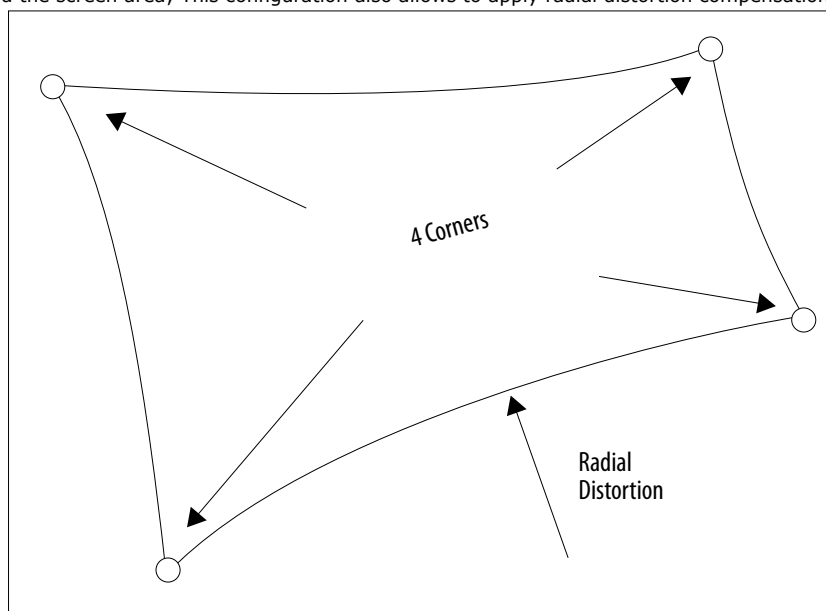
**Figure 162. Fixed warp**

In the fixed warp configuration a set of affine transformations such as translation, rotation, scaling, perspective correction as well as radial distortion (fisheye) compensation are configured individually using corresponding API calls.



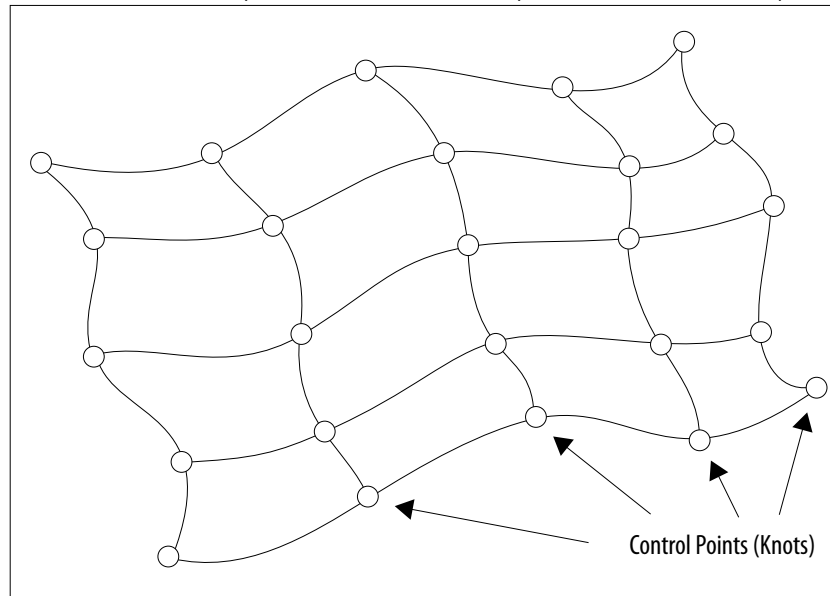
**Figure 163. 4x corner warp**

In the 4x corner configuration affine and perspective transformations are defined using four points positioned in or around the screen area; This configuration also allows to apply radial distortion compensation.



**Figure 164. Arbitrary warp**

Arbitrary transformation is defined by a set of curves controlled by a fixed number of control points (knots).



Warp meshes are tied to a specific input and output video resolution. The user must set these accordingly before generating a mesh.

Offset, sizes and positions are accepted as absolute values in pixels. To allow for sub-pixel precision floating-point type is used. Internally these parameters normalized to the [0..1] range using current output resolution and kept in this form. The warp automatically fits into the new output resolution if it changes.

**Table 1150. intel\_vvp\_warp\_mesh API reference**

| Name                                     | Description                                      |
|------------------------------------------|--------------------------------------------------|
| WarpConfigurator                         | Constructor of the intel_vvp_warp_mesh component |
| SetInputResolution, GetInputResolution   | Set or get input image resolution                |
| SetOutputResolution, GetOutputResolution | Set or get output image resolution               |
| GenerateMeshFromFixed                    | Generate mesh using fixed warp configuration     |
| GenerateMeshFromCorners                  | Generate mesh using 4x corner warp configuration |
| GenerateMeshFromArbitrary                | Generate mesh using arbitrary warp configuration |
| SetHSize, GetHSize                       | Set or get horizontal size of the image          |
| SetVSize, GetVSize                       | Set or get vertical size of the image            |
| SetHOffset, GetHOffset                   | Set or get horizontal offset of the image        |
| SetVOffset, GetVOffset                   | Set or get vertical offset of the image          |
| SetRotate, GetRotate                     | Set or get rotation angle                        |
| SetHMirror, GetHMirror                   | Set or get horizontal mirroring                  |
| continued...                             |                                                  |



| Name                                       | Description                                              |
|--------------------------------------------|----------------------------------------------------------|
| SetVMirror, GetVMirror                     | Set or get vertical mirroring                            |
| SetZoom, GetZoom                           | Set or get zoom factor                                   |
| SetHKeystone, GetHKeystone                 | Set or get horizontal keystone angle                     |
| SetVKeystone, GetVKeystone                 | Set or get vertical keystone angle                       |
| SetFOV, GetFOV                             | Set or get field of view                                 |
| SetVAxisOffset, GetVAxisOffset             | Set or get vertical axis offset                          |
| SetPreRadial, GetPreRadial                 | Set or get radial distortion compensation parameters     |
| SetCorner, GetCorner                       | Set or get corner position                               |
| SetCornerNorm, GetCornerNorm               | Set or get normalized corner position                    |
| SetArbitraryKnotsNum, GetArbitraryKnotsNum | Set or get size of the arbitrary control grid            |
| SetArbitraryKnot, GetArbitraryKnot         | Set or get position of arbitrary control knot            |
| SetArbitraryKnotNorm, GetArbitraryKnotNorm | Set or get normalized position of arbitrary control knot |
| SetMaintainRatio, GetMaintainRatio         | Set or get maintain aspect ratio setting                 |
| SetShrinkToFit, GetShrinkToFit             | Set or get shrink to fit setting                         |

### WarpConfigurator

```
WarpConfigurator(WarpHwContextPtr hw_ctx)
```

**Description** Constructs the main object of the intel\_vvp\_warp\_mesh software component

**Arguments** hw\_ctx - smart pointer to an instance of WarpHwContext object containing relevant IP parameters

**Return value** None

### SetInputResolution, GetInputResolution, SetOutputResolution, GetOutputResolution

```
void SetInputResolution(const uint32_t width, const uint32_t height) void GetInputResolution(uint32_t& width, uint32_t& height) void SetOutputResolution(const uint32_t width, const uint32_t height) void GetOutputResolution(uint32_t& width, uint32_t& height)
```

**Description** Set or get input and output image resolution

**Arguments** width – image width  
height – image height

**Return value** None

### GenerateMeshFromFixed, GenerateMeshFromCorners, GenerateMeshFromArbitrary

```
WarpMeshPtr GenerateMeshFromFixed() WarpMeshPtr
GenerateMeshFromCorners() WarpMeshPtr
GenerateMeshFromArbitrary()
```

|                     |                                                                                                                                                                         |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i>  | Generate transformation mesh using fixed, 4x corners or arbitrary configurations respectively                                                                           |
| <i>Arguments</i>    | None                                                                                                                                                                    |
| <i>Return value</i> | Smart pointer to the object of the type WarpMesh containing generated transformation mesh (please see intel_vvp_data interface section for the description of WarpMesh) |

### SetHSize, GetHSize, SetVSize, GetVSize

```
void SetHSize(float pixels) float GetHSize() void
SetVSize(float pixels) float GetVSize()
```

|                     |                                                                                                                                   |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i>  | Set or get horizontal and vertical size of the image, which is equivalent to scaling the image along the horizontal/vertical axis |
| <i>Arguments</i>    | pixels – horizontal or vertical size of the image accordingly                                                                     |
| <i>Return value</i> | horizontal or vertical size of the image accordingly                                                                              |

### SetHOffset, GetHOffset, SetVOffset, GetVOffset

```
void SetHOffset(float pixels) float GetHOffset() void
SetVOffset(float pixels) float GetVOffset()
```

|                     |                                                                 |
|---------------------|-----------------------------------------------------------------|
| <i>Description</i>  | Set or get horizontal and vertical offsets of the image         |
| <i>Arguments</i>    | pixels – horizontal or vertical offset of the image accordingly |
| <i>Return value</i> | horizontal or vertical offset of the image accordingly          |

### SetRotate, GetRotate

```
void SetRotate(float angle) float GetRotate()
```

|                     |                                                                                       |
|---------------------|---------------------------------------------------------------------------------------|
| <i>Description</i>  | Set or get rotation angle of the image in degrees. Image is rotated around the center |
| <i>Arguments</i>    | angle – rotation angle in degrees                                                     |
| <i>Return value</i> | rotation angle in degrees                                                             |

### SetHMirror, GetHMirror, SetVMirror, GetVMirror

```
void SetHMirror(bool v) bool GetHMirror() void
SetVMirror(bool v) bool GetVMirror()
```

*Description* Set or get image mirroring along horizontal and vertical axis accordingly

*Arguments* v – true to enable mirroring, false to disable

*Return value* true if mirroring enabled, false otherwise

### SetZoom, GetZoom

```
void SetZoom(float zoom) float GetZoom()
```

*Description* Set or get image zoom value

*Arguments* zoom – value > 1 to zoom in, < 1 to zoom out

*Return value* Configured zoom value

### SetHKeystone, GetHKeystone, SetVKeystone, GetVKeystone

```
void SetHKeystone(float angle) float GetHKeystone() void
SetVKeystone(float angle) float GetVKeystone()
```

*Description* Set or get horizontal and vertical keystone compensation angle accordingly

*Arguments* angle - horizontal or vertical angle in degrees by which the projector is tilting relative to the normal to the projection surface.

*Return value* Horizontal or vertical projector tilting angle accordingly

### SetFOV, GetFOV

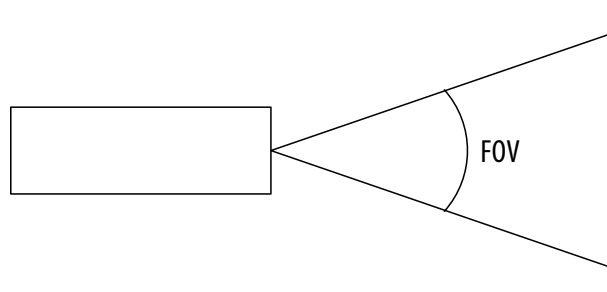
```
void SetFOV(float angle) float GetFOV()
```

*Description* Set or get field of View. Field of view represents projector's beam angle. This value is necessary for correct keystone correction. The default value is 30°

*Arguments* angle - projector's beam angle in degrees

*Return value* Projector's beam angle in degrees

Figure 165. Field of View



#### SetVAxisOffset, GetVAxisOffset

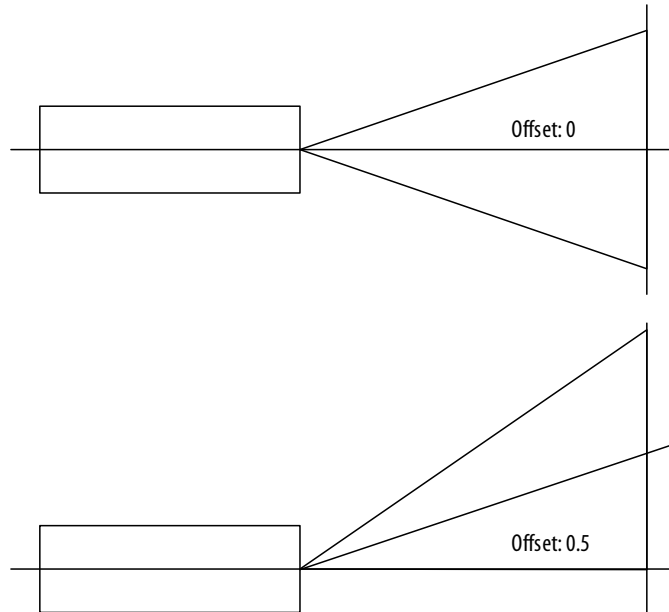
```
void SetVAxisOffset(float v) float GetVAxisOffset()
```

**Description** Set or get vertical offset of the optical axis from the normal to the projection plane. This value is required for correct keystone correction. Some projectors, particularly short throw models have their beam already at an angle to the wall. Take account of this initial angle when calculating keystone correction. The provided value is the absolute offset divided by the image height. The default value is 0

**Arguments** v - value of vertical axis offset

**Return value** value of vertical axis offset

Figure 166. Vertical Offset



#### SetPreRadial, GetPreRadial

```
void SetPreRadial(float x, float y, float k1, float k2) void
GetPreRadial(float& x, float& y, float& k1, float& k2)
```

**Description** Set or get radial distortion compensation parameters. The software uses Brown-Conrady distortion model

**Arguments** x, y – distortion center  
k1, k2 – 1st and 2nd distortion coefficients

**Return value** None

#### SetCorner, GetCorner

```
void SetCorner(const ECornerId& corner, float pixel_x, float
pixel_y) void GetCorner(const ECornerId& corner, float&
pixel_x, float& pixel_y)
```

**Description** Set or get position of an individual corner point of the 4x corner warp configuration

**Arguments** corner – corner identifier  
pixel\_x, pixel\_y – horizontal and vertical coordinate of the corner

**Return value** None

### SetCornerNorm, GetCornerNorm

```
void SetCornerNorm(const ECornerId& corner, float x, float y)
void GetCornerNorm(const ECornerId& corner, float& x, float& y)
```

**Description** Set or get normalized position of an individual corner point of the 4x corner warp configuration

**Arguments** corner – corner identifier  
x,y – horizontal and vertical coordinate of the corner in the range 0..1

**Return** None

### SetArbitraryKnotsNum, GetArbitraryKnotsNum

```
void SetArbitraryKnotsNum(uint32_t num) uint32_t
GetArbitraryKnotsNum()
```

**Description** Set or get number of control points (knots) for the arbitrary warp. Arbitrary warp is defined by a set of curves and control points arranged into NxN grid. This parameter sets the size of this grid. For simplicity the number of control points in both dimensions is the same

**Arguments** num – number of control knots in each dimension

**Return value** number of control knots in each dimension

### SetArbitraryKnot, GetArbitraryKnot

```
void SetArbitraryKnot(const uint32_t idx, const float
pixel_x, const float pixel_y)
void GetArbitraryKnot(const uint32_t idx, float& pixel_x,
float& pixel_y)
```

**Description** Set or get absolute position of individual control points. Points are numbered left to right, top to bottom

**Arguments** idx – zero based point index number  
pixel\_x, pixel\_y – horizontal and vertical coordinate of the point

**Return value** None

#### SetArbitraryKnotNorm, GetArbitraryKnotNorm

```
void SetArbitraryKnotNorm(const uint32_t idx, const float x,
const float y)
void GetArbitraryKnotNorm(const uint32_t idx, float& x, float&
y)
```

**Description** Set or get normalized position of individual control points. Points are numbered left to right, top to bottom

**Arguments** idx– zero based point index number  
  
x,y – horizontal and vertical coordinate of the point in the range 0..1

**Return** None

#### SetMaintainRatio, GetMaintainRatio

```
void SetMaintainRatio(bool val) bool GetMaintainRatio()
```

**Description** Set or get maintain aspect ratio flag. If set the aspect ratio of the input resolution is preserved. As a result the output image may have blank areas along either horizontal or vertical axis

**Arguments** val – true to maintain input aspect ratio, false otherwise

**Return value** Current maintain aspect ratio setting

#### SetShrinkToFit, GetShrinkToFit

```
void SetShrinkToFit(bool val) bool GetShrinkToFit()
```

**Description** Set or get shrink to fit setting. If set the resulting image is scaled down after applying all transforms to make sure it fits into the screen. Otherwise the resulting image is cropped to fit output dimensions. This flag is only relevant for the Fixed warp configuration

**Arguments** val – true to shrink resulting image to fit output dimensions, false otherwise

**Return value** Current shrink to fit setting

## Data structures and types

### ECornerId

```
enum ECornerId {
 ETopLeft = 0,
 ETopRight,
 EBottomLeft,
 EBottomRight,
 ETotalCorners,
};
```

*Description* Enum identifies individual corners in the 4x corner warp configuration

### Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38

## 53.5.1. Using Warp IP Software

1. Initialize `intel_vvp_warp` driver instance
2. Allocate warp video channel
3. Configure the channel by providing video resolution, color space, base address of the framebuffer region in RAM and other required parameters
4. Instantiate `WarpConfigurator` object (`intel_vvp_warp_mesh`), set input, output video resolutions and desired transforms
5. Generate transformation mesh
6. Instantiate `WarpDataGenerator` object (`intel_vvp_warp_data`)
7. Instantiate `WarpDataContext` object (`intel_vvp_warp_data`) and fill it in using input, output video resolution and required IP parameters available through the warp channel structure
8. Generate mesh, data and filter coefficients by calling `WarpDataGenerator::GenerateData()` method and passing the data context object and the transformation mesh
9. Transfer generated coefficients to the designated RAM region accessible by the warp IP
10. Instantiate and fill in an `intel_vvp_warp_data_t` structure object using the addresses of the coefficient data in RAM and other required parameter
11. Apply new warp by calling `intel_vvp_warp_apply_transform()` method of the driver passing the warp channel and warp data structures as parameters.

## 53.5.2. Warp IP Software Code Examples



### 8K UHD Workflow example

This example shows the workflow and basic warp software usage of the C++ source code to generate and apply 15 degree rotation warp. The example is for 7680x4320 video, which requires the processing to be split between four warp engines. The frame buffer and warp coefficient base addresses in the example are arbitrary. Actual values depend on a particular system design.

```
uint32_t FRAMEBUF_BASE_ADDR = 0x80000000;
uint32_t COEF_BASE_ADDR = 0xa0000000;
uint32_t SKIP_RAM_PAGE = 0;
uint32_t VIDEO_WIDTH = 7680;
uint32_t VIDEO_HEIGHT = 4320;

intel_vvp_warp_base_t base = INTEL_VVP_WARP_BASE;

// Warp driver instance
intel_vvp_warp_instance_t wrp0;

// Initialize driver instance
intel_vvp_warp_init_instance(&wrp0, base);

// Create warp channel
intel_vvp_warp_channel_t* ch0 = intel_vvp_warp_create_quad_channel(&wrp0, 0, 0);

// Fill in warp channel configuration structure
intel_vvp_warp_channel_config_t cfg;
cfg.ram_addr = FRAMEBUF_BASE_ADDR; // Frame buffers base address
cfg.cs = ERGB_FULL; // Video colourspace
cfg.width_input = VIDEO_WIDTH; // Video dimensions
cfg.height_input = VIDEO_HEIGHT;
cfg.width_output = VIDEO_WIDTH;
cfg.height_output = VIDEO_HEIGHT;
cfg.lfr = 0; // No low frame rate fallback

// Configure warp channel using the parameters above
intel_vvp_warp_configure_channel(ch0, &cfg);

// Obtain required hardware information
WarpHwContextPtr hw = WarpDataHelper::GetHwContext(ch0);

// Instantiate and initialize mesh generator
WarpConfigurator configurator{ hw };
configurator.SetInputResolution(VIDEO_WIDTH, VIDEO_HEIGHT);
configurator.SetOutputResolution(VIDEO_WIDTH, VIDEO_HEIGHT);
configurator.Reset();
configurator.SetRotate(15.0f);

// Generate mesh
WarpMeshPtr mesh = configurator.GenerateMeshFromFixed();

// Instantiate data generator
WarpDataGenerator data_generator;

WarpDataContext ctx{ hw,
 VIDEO_WIDTH, VIDEO_HEIGHT,
 VIDEO_WIDTH, VIDEO_HEIGHT
};

// Generate warp data using provided hardware configuration and mesh
WarpDataPtr user_data = data_generator.GenerateData(ctx, mesh);

// Allocate and fill in intel_vvp_warp_data_t object required by the warp driver
WarpDataHelper::WarpDriverDataPtr driver_data =
WarpDataHelper::GenerateDriverData(ctx, user_data, COEF_BASE_ADDR,
SKIP_RAM_PAGE);

// Transfer generated warp data to the calculated destination for each engine
```

```
for(uint32_t i = 0; i < driver_data->num_engines; ++i)
{
 const WarpEngineData* ue = user_data->GetEngineData(i);
 intel_vvp_warp_engine_data_t* de = &(driver_data->engine_data[i]);
 memcpy((void*)(de->mesh_addr), ue->GetMeshData(), de->mesh_size);
 memcpy((void*)(de->filter_addr), ue->GetFilterData(), de->filter_size);
 memcpy((void*)(de->fetch_addr), ue->GetFetchData(), de->fetch_size);
}

// Apply warp by passing new warp data set to the driver
intel_vvp_warp_apply_transform(ch0, driver_data.get());
// Release allocated resources
intel_vvp_warp_free_channel(ch0);
```

### Warp mesh usage

Define required warp using the WarpMesh object. The example shows the simplest case of 1:1 (unity) warp for a 8K UHD video.

```
WarpMeshPtr mesh = WarpMesh::Create({7680, 4320}, {7680, 4320}, hw);
uint32_t mesh_step = mesh->GetStep();
uint32_t fract_bits = mesh->GetFractBits();

for(uint32_t v = 0; v < mesh->GetVNodes(); ++v)
{
 mesh_node_t* node = mesh->GetRow(v);

 for(uint32_t h = 0; h < mesh->GetHNodes(); ++h)
 {
 node->_x = (h * mesh_step) << fract_bits; node->_y = (v * mesh_step) <<
 fract_bits;
 }
}
```

Mesh coordinates use the least significant bits as fractional part for subpixel precision. In the example above the fractional part is always 0. Store subpixel positions in the following way:

```
mesh_node_t* node = mesh->GetRow(v);
float k = static_cast<float>(1 << mesh->GetFractBits());
float pos_x = 10.6f;
node->_x = static_cast<int32_t>(roundf(pos_x * k));
```

### Easy warp example

When you turn on **Easy warp** you can rotate the input video to 0, 90, 180 or 270 degrees or mirror the video without the need of transform mesh and associated warp data.

```
uint32_t FRAMEBUF_BASE_ADDR = 0x80000000;
uint32_t VIDEO_WIDTH = 3840;
uint32_t VIDEO_HEIGHT = 2160;
intel_vvp_warp_base_t base = INTEL_VVP_WARP_BASE;

intel_vvp_warp_instance wrp0;

// Initialize driver instance
intel_vvp_warp_init_instance(&wrp0, base);

// Allocate Easy warp channel
intel_vvp_warp_channel_t* ch0 = intel_vvp_warp_create_easy_warp_channel(&wrp0,
0, 0);

assert(intel_vvp_warp_check_easy_warp_capable(ch0) == 0);
```

```
// Configure channel
intel_vvp_warp_channel_config_t cfg;
// Configure in 4K, RGB Full colourspace
cfg.ram_addr = FRAMEBUF_BASE_ADDR;
cfg.cs = ERGB_FULL;
cfg.width_input = VIDEO_WIDTH;
cfg.height_input = VIDEO_HEIGHT;
cfg.width_output = VIDEO_WIDTH;
cfg.height_output = VIDEO_HEIGHT;
cfg.lfr = 0;

intel_vvp_warp_configure_channel(ch0, &cfg);

// Mirror input video
// Enable video bypass, keep original resolution
intel_vvp_warp_bypass(ch0, 1, 0, VIDEO_WIDTH, VIDEO_HEIGHT);

// Configure Easy warp mirror
intel_vvp_warp_set_easy_warp(ch0, 0x4);

// Rotate input video 90 degrees CCW
// Enable video bypass, flip input width and height
intel_vvp_warp_bypass(ch0, 1, 0, VIDEO_HEIGHT, VIDEO_WIDTH);

// Configure Easy warp rotation
intel_vvp_warp_set_easy_warp(ch0, 0x01);

// Release warp channel
intel_vvp_warp_free_channel(ch0);
```

### Warp latency parameters generation example

The example shows how to generate recommended minimum latency parameters for a given video transformation. These parameters are necessary to configure video pipeline for low latency operation.

```
// Example video and system clock
uint32_t EXAMPLE_CLOCK = 300000000;
// UHD video dimensions
uint32_t VIDEO_WIDTH = 3840;
uint32_t VIDEO_HEIGHT = 2160;
uint32_t VIDEO_HEIGHT_FULL = 2250;
// Output frame rate x100
uint32_t OUTPUT_FRAME_RATE = 6000;
// In this example system and video clock are the same
uint32_t system_clock = EXAMPLE_CLOCK;
uint32_t video_clock = EXAMPLE_CLOCK;
// Warp channel
intel_vvp_warp_channel_t* ch0{nullptr};

// Allocate and initialize a warp channel here
// ...
////////////////////////////////////

// Obtain required hardware information
WarpHwContextPtr hw = WarpDataHelper::GetHwContext(ch0);

// Generate a 4K mesh for 5 degree CCW rotation
WarpConfigurator configurator{hw};

configurator.SetInputResolution(VIDEO_WIDTH, VIDEO_HEIGHT);
configurator.SetOutputResolution(VIDEO_WIDTH, VIDEO_HEIGHT);
configurator.Reset();
configurator.SetRotate(5.0f);

WarpMeshPtr mesh = configurator.GenerateMeshFromFixed();
```

```
// Parameters required for warp data generation
WarpDataContext ctx{
 hw,
 VIDEO_WIDTH, VIDEO_HEIGHT,
 VIDEO_WIDTH, VIDEO_HEIGHT
};

WarpDataGenerator data_generator;

WarpDataPtr user_data = data_generator.GenerateData(ctx, mesh);

// Obtain latency params for the configured warp
WarpLatencyParams latency_params = data_generator.GenerateLatencyParams(ctx,
user_data, system_clock, video_clock, VIDEO_HEIGHT_FULL, OUTPUT_FRAME_RATE);

// Upload and apply generated warp data here
// ...
// intel_vvp_warp_apply_transform(ch0, ...);
// ...

// Pass on "output_latency" to the driver
intel_vvp_warp_set_output_latency(ch0, latency_params._output_latency);

// The "total_latency" member represents the recommended minimum offset
// between the input and output frames
// The value is in axi4s_vid_out_0 clock clock cycles
// Use the this parameter to configure the rest of the video pipeline
// as appropriate
//
// latency_params._total_latency;
```

## 54. White Balance Correction IP

### 54.1. About the White Balance Correction IP

The White Balance Correction IP adjusts colors in an image to eliminate color casts. These casts occur because of variations in color temperatures of lighting conditions or differences in light sensitivity of the color filter array. The IP ensures that white objects appear truly white, without unwanted color tinting. The IP operates only on 2x2 Bayer color filter array images.

The IP only supports lite variants. For more information about full and lite variants, refer to the *Altera Streaming Video Protocol Specification*

#### Related Information

[Altera Streaming Video Protocol Specification](#)

#### 54.1.1. White Balance Correction IP Performance and Resources

Altera provides resource and utilization data for guidance only. The IP resource utilization depends on the device family, compilation tool version, compilation seed randomization, and other parameters.

**Table 1151. White Balance Correction IP Performance and Resources**

| Device                                        | Pixels in Parallel | Bit per Color Sample | Max Resolution | ALM | M20k | DSP | f <sub>MAX</sub> (MHz) |
|-----------------------------------------------|--------------------|----------------------|----------------|-----|------|-----|------------------------|
| Agilex 7 speed grade 2 (AGFA012R24A2E2V)      | 1                  | 10                   | 2k             | 420 | 0    | 1   | 884                    |
|                                               | 1                  | 16                   | 2k             | 487 | 0    | 1   | 834                    |
|                                               | 4                  | 10                   | 8k             | 658 | 0    | 4   | 796                    |
|                                               | 4                  | 16                   | 8k             | 769 | 0    | 4   | 790                    |
| Arria 10 speed grade 1 (10AS066H1F34E1HG)     | 1                  | 10                   | 2k             | 336 | 0    | 1   | 638                    |
|                                               | 1                  | 16                   | 2k             | 354 | 0    | 1   | 647                    |
|                                               | 2                  | 10                   | 4k             | 388 | 0    | 2   | 637                    |
|                                               | 2                  | 16                   | 4k             | 429 | 0    | 2   | 630                    |
| Cyclone 10 GX speed grade 5 (10CX220YF672E5G) | 1                  | 10                   | 2k             | 337 | 0    | 1   | 579                    |
|                                               | 1                  | 16                   | 2k             | 351 | 0    | 1   | 546                    |
| Stratix 10 speed grade 1 (1SX280LN2F43E1VG)   | 1                  | 10                   | 2k             | 420 | 0    | 1   | 696                    |
|                                               | 1                  | 16                   | 2k             | 434 | 0    | 1   | 726                    |
|                                               | 2                  | 10                   | 4k             | 509 | 0    | 2   | 699                    |
|                                               | 2                  | 16                   | 4k             | 564 | 0    | 2   | 700                    |

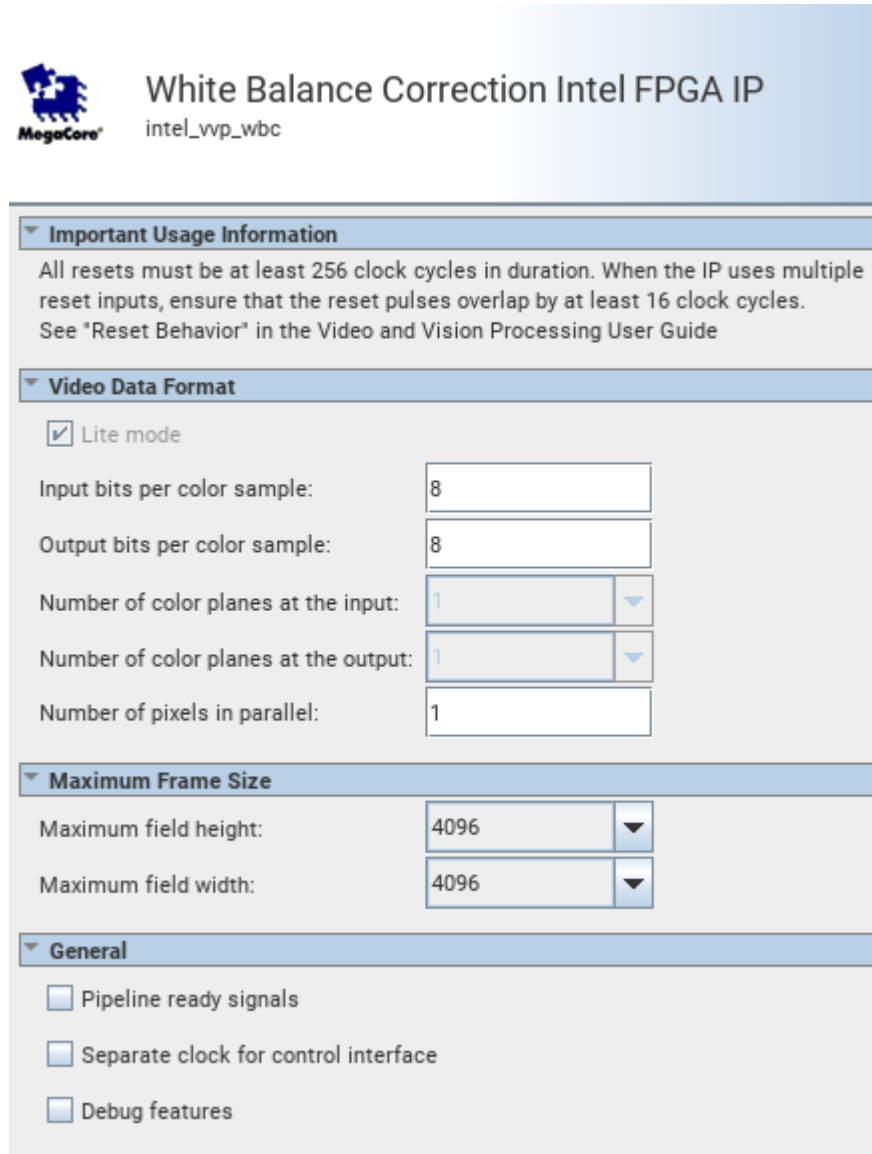
## 54.2. White Balance Correction IP Parameters

The IP offers run-time and compile-time parameters.

**Table 1152. White Balance Correction IP Parameters**

| Parameter                                   | Values              | Description                                                                                                           |
|---------------------------------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Video Data Format</b>                    |                     |                                                                                                                       |
| <b>Lite mode</b>                            | On                  | The IP only supports lite mode                                                                                        |
| <b>Input bits per color sample</b>          | 8 to 16             | Select the number of input bits per color sample.                                                                     |
| <b>Output bits per color sample</b>         | 8 to 16             | Select the number of output bits per color sample.                                                                    |
| <b>Number of color planes at the input</b>  | 1                   | The IP only supports 1 color plane at its input                                                                       |
| <b>Number of color planes at the output</b> | 1                   | The IP only supports 1 color plane at its output                                                                      |
| <b>Number of pixels in parallel</b>         | 1 to 8              | Select the number of pixels in parallel.                                                                              |
| <b>Maximum Frame Size</b>                   |                     |                                                                                                                       |
| <b>Maximum field height</b>                 | 1k, 2k, 4k, 8k, 16k | Specify the maximum height of incoming frames                                                                         |
| <b>Maximum field width</b>                  | 2k, 4k, 8k, 16k     | Specify the maximum width of incoming frames                                                                          |
| <b>General</b>                              |                     |                                                                                                                       |
| <b>Pipeline ready signal</b>                | On or off           | Turn on to add extra pipeline registers to the AXI4-S <code>tready</code> signals                                     |
| <b>Separate clock for control interface</b> | On or off           | Turn on for a separate clock for the control agent interface                                                          |
| <b>Debug features</b>                       | On or off           | Turn on to read back frame information registers and debugging information registers via the control agent interface. |

Figure 167. White Balance Correction IP GUI



The screenshot shows the configuration interface for the White Balance Correction Intel FPGA IP. At the top, there is a MegaCore logo and the text "White Balance Correction Intel FPGA IP" and "intel\_vvp\_wbc". Below this, the interface is divided into several sections:

- Important Usage Information:** A text block stating: "All resets must be at least 256 clock cycles in duration. When the IP uses multiple reset inputs, ensure that the reset pulses overlap by at least 16 clock cycles. See 'Reset Behavior' in the Video and Vision Processing User Guide".
- Video Data Format:**
  - ☒ Lite mode
  - Input bits per color sample: 8
  - Output bits per color sample: 8
  - Number of color planes at the input: 1
  - Number of color planes at the output: 1
  - Number of pixels in parallel: 1
- Maximum Frame Size:**
  - Maximum field height: 4096
  - Maximum field width: 4096
- General:**
  - ☐ Pipeline ready signals
  - ☐ Separate clock for control interface
  - ☐ Debug features

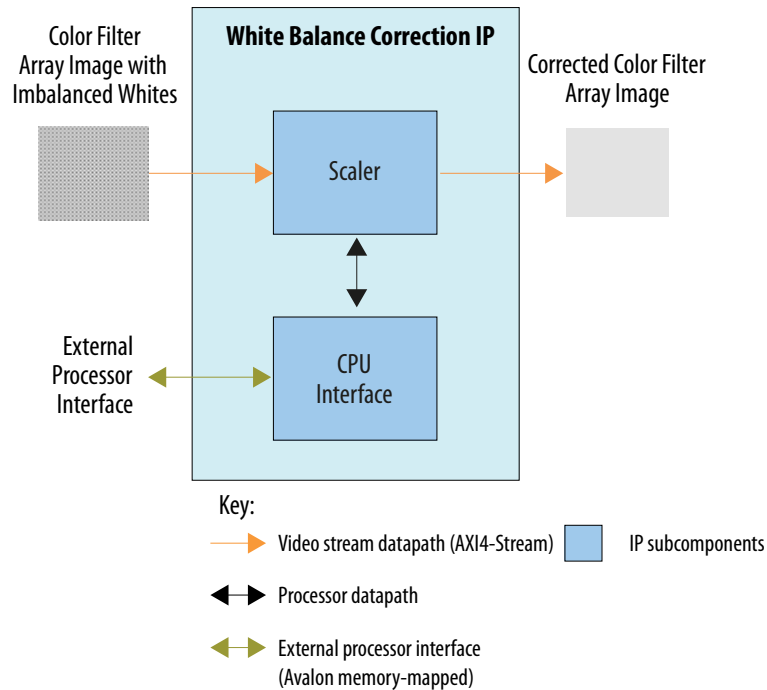
### 54.3. White Balance Correction IP Functional Description

Variations in imaging sensor technology and the lighting conditions can cause color distortions when a camera captures them. The IP scales individual color channels of a 2x2 color filter array input video stream to correct color distortions. With this IP, you write scaler values for each of the 4 color channels of the 2x2 color filter array over the external processor interface. The IP uses these values to calculate the output video stream. The IP supports any arbitrary 2x2 color filter arrays.

The scaler part of the IP multiplies each input pixel value with an arbitrary factor (scaler value) that you program over the external processor interface. You must calculate the correct multiplication factor from the dynamic range of the video stream,

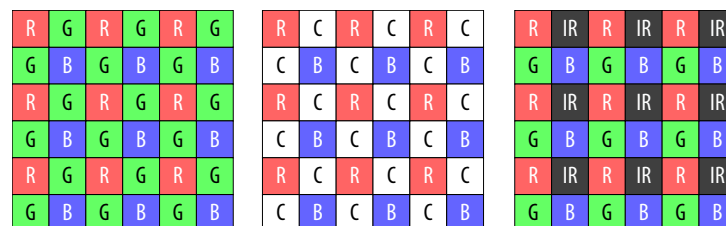
calibration of the imaging sensor, and the color temperature of the light source illuminating the scene. The IP clips the overflowed values to the maximum value of the output video stream. You can use the output of the White Balance Statistics IP to calculate control parameters for white balance correction at run time.

**Figure 168. White Balance Correction IP Block Diagram**



You must set the color filter array phase to its correct value, which depends on the framing, rotation, and flipping options you apply on the original image that an imaging sensor captures. If you do not set the correct color filter array phase, the IP may mix color channels and produce incorrect outputs.

**Figure 169. Examples of 2x2 color filter arrays for a 6x6 section of an image**





### 54.3.1. White Balance Correction IP Interfaces

**Table 1153. White Balance Correction Clock and Reset Interfaces**

| Name              | Direction | Width | Description                                                                                                            |
|-------------------|-----------|-------|------------------------------------------------------------------------------------------------------------------------|
| main_clock_clk    | In        | 1     | AXI4-S processing clock.                                                                                               |
| main_reset_rst    | In        | 1     | AXI4-S processing reset.                                                                                               |
| agent_clock_clk   | In        | 1     | Clock for the memory-mapped control agent interface. Only if you turn on <b>Separate clock for control interface</b> . |
| agent_reset_reset | In        | 1     | Reset for the memory-mapped control agent interface. Only if you turn on <b>Separate clock for control interface</b> . |

**Table 1154. White Balance Correction Control Interfaces**

| Name                              | Direction | Width | Description                              |
|-----------------------------------|-----------|-------|------------------------------------------|
| av_mm_control_agent_address       | In        | 7     | Avalon memory-mapped agent address       |
| av_mm_control_agent_write         | In        | 1     | Avalon memory-mapped agent write.        |
| av_mm_control_agent_writedata     | In        | 32    | Avalon memory-mapped agent write data.   |
| av_mm_control_agent_byteenable    | In        | 4     | Avalon memory-mapped agent byte enable.  |
| av_mm_control_agent_read          | In        | 1     | Avalon memory-mapped agent read.         |
| av_mm_control_agent_readdata      | Out       | 32    | Avalon memory-mapped agent read data.    |
| av_mm_control_agent_readdatavalid | Out       | 1     | Avalon memory-mapped agent read.         |
| av_mm_control_agent_waitrequest   | Out       | 1     | Avalon memory-mapped agent wait request. |

**Table 1155. White Balance Correction Altera Streaming Video Interfaces**

| Name                 | Direction | Width | Description                                         |
|----------------------|-----------|-------|-----------------------------------------------------|
| <b>Input</b>         |           |       |                                                     |
| axi4s_vid_in_tdata   | In        | (178) | AXI4-S data in.                                     |
| axi4s_vid_in_tvalid  | In        | 1     | AXI4-S data valid.                                  |
| axi4s_vid_in_tuser   | In        | (179) | [0] AXI4-S start of video frame.<br>[N-1:1] Unused. |
| axi4s_vid_in_tlast   | In        | 1     | AXI4-S end of packet.                               |
| axi4s_vid_in_tready  | Out       | 1     | AXI4-S data ready.                                  |
| <b>Output</b>        |           |       |                                                     |
| axi4s_vid_out_tdata  | Out       | (178) | AXI4-S data out.                                    |
| axi4s_vid_out_tvalid | Out       | 1     | AXI4-S data valid.                                  |
| <i>continued...</i>  |           |       |                                                     |

(178) The equation gives all tdata widths sizes in these interfaces: pixels in parallel × max (round (bits per color sample × number of color planes, 8), 16)

(179) This equation gives all tuser widths sizes in these interfaces: N = ceil (tdata width / 8)

| Name                 | Direction | Width | Description                                         |
|----------------------|-----------|-------|-----------------------------------------------------|
| axi4s_vid_out_tuser  | Out       | (179) | [0] AXI4-S start of video frame.<br>[N-1:1] Unused. |
| axi4s_vid_out_tlast  | Out       | 1     | AXI4-S end of packet.                               |
| axi4s_vid_out_tready | In        | 1     | AXI4-S data ready                                   |

### Related Information

Video and Vision Processing IP Interfaces on page 30

## 54.4. White Balance Correction IP Registers

Each register is either read-only (RO) or read-write (RW).

**Table 1156. White Balance Correction Registers**

In the software API the register names appear with a prefix of INTEL\_VVP, INTEL\_VVP\_CORE, or INTEL\_VVP\_WBC as appropriate and with an optional REG suffix

| Address                    | Register      | Access     |      | Description                                                                                                          |
|----------------------------|---------------|------------|------|----------------------------------------------------------------------------------------------------------------------|
|                            |               | Lite (180) | Full |                                                                                                                      |
| Parameterization registers |               |            |      |                                                                                                                      |
| 0x0000                     | VID_PID       | RO         | N/A  | Read this register to retrieve the ID of the IP.<br>This register always returns 0x6FA7_017A.                        |
| 0x0004                     | VERSION       | RO         | N/A  | Read this register to retrieve the version information for the IP.                                                   |
| 0x0008                     | LITE_MODE     | RO         | N/A  | Read this register to determine if <b>Lite mode</b> is on.<br>This register always returns 1.                        |
| 0x000C                     | DEBUG_ENABLED | RO         | N/A  | Read this register to determine if <b>Debug features</b> are on.<br>This register returns 0 for off and 1 for on.    |
| 0x0010                     | BPS_IN        | RO         | N/A  | Read this register to determine the <b>Input bits per color symbol</b> .                                             |
| 0x0014                     | BPS_OUT       | RO         | N/A  | Read this register to determine the <b>Output bits per color symbol</b> .                                            |
| 0x0018                     | NUM_COLOR_IN  | RO         | N/A  | Read this register to determine the <b>Number of color planes at the input</b> .<br>This register always returns 1.  |
| 0x001C                     | NUM_COLOR_OUT | RO         | N/A  | Read this register to determine the <b>Number of color planes at the output</b> .<br>This register always returns 1. |
| 0x0020                     | PIP           | RO         | N/A  | Read this register to determine the <b>Number of pixels in parallel</b> .                                            |
| 0x0024                     | MAX_WIDTH     | RO         | N/A  | Read this register to determine the <b>Maximum field width</b> .                                                     |
| continued...               |               |            |      |                                                                                                                      |

(180) Registers are RW only if you also turn on **Debug features**, otherwise they are WO.

| Address                                        | Register            | Access                |      | Description                                                                                   |
|------------------------------------------------|---------------------|-----------------------|------|-----------------------------------------------------------------------------------------------|
|                                                |                     | Lite <sup>(180)</sup> | Full |                                                                                               |
| 0x0028                                         | MAX_HEIGHT          | RO                    | N/A  | Read this register to determine the <b>Maximum field height</b> .                             |
| 0x002C to 0x011F                               | -                   | -                     | -    | Reserved                                                                                      |
| <b>Control, debug and statistics registers</b> |                     |                       |      |                                                                                               |
| 0x0120                                         | IMG_INFO_WIDTH      | RW                    | N/A  | The expected width of the incoming video fields.                                              |
| 0x0124                                         | IMG_INFO_HEIGHT     | RW                    | N/A  | The expected height of the incoming video fields.                                             |
| 0x0028 to 0x013F                               | -                   | -                     | -    | Reserved                                                                                      |
| 0x0140                                         | STATUS              | RO                    | N/A  | Read this register for information about the IP status.                                       |
| 0x0144                                         | FRAME_STATS         | RO                    | N/A  | Read this register for some frame statistics.                                                 |
| 0x0148                                         | COMMIT              | RW                    | N/A  | Write any value to this register to submit changes to the control and color scaler registers. |
| 0x014C                                         | CONTROL             | RW                    | N/A  | Control bits and fields of the IP                                                             |
| 0x0150                                         | CFA_00_COLOR_SCALER | RW                    | N/A  | Color scaler value for color channel 0.                                                       |
| 0x0154                                         | CFA_01_COLOR_SCALER | RW                    | N/A  | Color scaler value for color channel 1.                                                       |
| 0x0158                                         | CFA_10_COLOR_SCALER | RW                    | N/A  | Color scaler value for color channel 2.                                                       |
| 0x015C                                         | CFA_11_COLOR_SCALER | RW                    | N/A  | Color scaler value for color channel 3.                                                       |
| 0x0160 to 0x01FF                               | -                   | -                     | -    | Reserved                                                                                      |

### Register Bit Description

**Table 1157. STATUS**

| Name     | Bits | Description                         |
|----------|------|-------------------------------------|
| Reserved | 31:2 | Reserved.                           |
| Commit   | 1    | Pending commit                      |
| Running  | 0    | When 1, the IP is processing data.. |

**Table 1158. FRAME\_STATS**

| Name     | Bits | Description                     |
|----------|------|---------------------------------|
| Reserved | 31:8 | Reserved.                       |
| Checksum | 7:0  | A simple checksum of the frame. |

<sup>(180)</sup> Registers are RW only if you also turn on **Debug features**, otherwise they are WO.

**Table 1159. CONTROL**

| Name                     | Bits | Description                                                                                                                                                                                                                                                                                                                                                           |
|--------------------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved                 | 31:3 | Reserved. Write 0.                                                                                                                                                                                                                                                                                                                                                    |
| Color filter array phase | 2:1  | Specifies 2x2 color filter order starting from the top left corner of the image.<br><div> <div>00 01 10 11</div> <div>C<sub>0</sub>C<sub>1</sub> C<sub>1</sub>C<sub>0</sub> C<sub>2</sub>C<sub>3</sub> C<sub>3</sub>C<sub>2</sub></div> <div>C<sub>2</sub>C<sub>3</sub> C<sub>3</sub>C<sub>2</sub> C<sub>0</sub>C<sub>1</sub> C<sub>1</sub>C<sub>0</sub></div> </div> |
| Bypass                   | 0    | When set the IP passes the input image unmodified.                                                                                                                                                                                                                                                                                                                    |

**Table 1160. CFA\_00\_COLOR\_SCALER**

| Name     | Bits  | Description                                                                                                                                        |
|----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved | 31:19 | Reserved. Write 0.                                                                                                                                 |
| Value    | 18:0  | Unsigned 8.11 fixed-point color scaler for color channel 0 (C0). MSBs to LSBs correspond to the binary digits 2 <sup>7</sup> to 2 <sup>-11</sup> . |

**Table 1161. CFA\_01\_COLOR\_SCALER**

| Name     | Bits  | Description                                                                                                                                        |
|----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved | 31:19 | Reserved. Write 0.                                                                                                                                 |
| Value    | 18:0  | Unsigned 8.11 fixed-point color scaler for color channel 1 (C1). MSBs to LSBs correspond to the binary digits 2 <sup>7</sup> to 2 <sup>-11</sup> . |

**Table 1162. CFA\_10\_COLOR\_SCALER**

| Name     | Bits  | Description                                                                                                                                        |
|----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved | 31:19 | Reserved. Write 0.                                                                                                                                 |
| Value    | 18:0  | Unsigned 8.11 fixed-point color scaler for color channel 2 (C2). MSBs to LSBs correspond to the binary digits 2 <sup>7</sup> to 2 <sup>-11</sup> . |

**Table 1163. CFA\_11\_COLOR\_SCALER**

| Name     | Bits  | Description                                                                                                                                        |
|----------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved | 31:19 | Reserved. Write 0.                                                                                                                                 |
| Value    | 18:0  | Unsigned 8.11 fixed-point color scaler for color channel 3 (C3). MSBs to LSBs correspond to the binary digits 2 <sup>7</sup> to 2 <sup>-11</sup> . |

## 54.5. White Balance Correction IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_wbc_regs.h`

Include file: `intel_vvp_wbc.h`

**Table 1164. White Balance Correction IP Software API**

| Name                                   | Description                                                                                                      |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------|
| intel_vvp_wbc_init                     | Initialize an IP instance                                                                                        |
| intel_vvp_core_*                       | Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i>                           |
| intel_vvp_wbc_get_lite_mode            | Returns if <b>Lite mode</b> is on                                                                                |
| intel_vvp_wbc_get_debug_enabled        | Returns if <b>Debug features</b> is on                                                                           |
| intel_vvp_wbc_get_bits_per_sample_in   | Returns the number of bits per color sample for the streaming input interface                                    |
| intel_vvp_wbc_get_bits_per_sample_out  | Returns the number of bits per color sample value for the streaming output interface                             |
| intel_vvp_wbc_get_num_color_planes_in  | Returns the number of color planes of the streaming input interface                                              |
| intel_vvp_wbc_get_num_color_planes_out | Returns the number of color planes of the streaming output interface                                             |
| intel_vvp_wbc_get_pixels_in_parallel   | Returns the pixels in parallel of the streaming input and output interfaces                                      |
| intel_vvp_wbc_get_max_width            | Returns the maximum number of pixels that the IP supports on the horizontal dimension of an image or video frame |
| intel_vvp_wbc_get_max_height           | Returns the maximum number of pixels that the IP supports on the vertical dimension of an image or video frame   |
| intel_vvp_wbc_is_running               | Reads if the IP is running                                                                                       |
| intel_vvp_wbc_commit_is_pending        | Reads if the IP has uncommitted writes                                                                           |
| intel_vvp_wbc_get_status               | Reads the status register                                                                                        |
| intel_vvp_wbc_get_frame_stats          | Reads the frame statistics register                                                                              |
| intel_vvp_wbc_commit                   | Commit pending writes                                                                                            |
| intel_vvp_wbc_get_bypass               | Reads the bypass bit from the status register                                                                    |
| intel_vvp_wbc_set_bypass               | Writes the bypass bit of the status register                                                                     |
| intel_vvp_wbc_get_cfa_phase            | Reads the color filter array phase field from the status register                                                |
| intel_vvp_wbc_set_cfa_phase            | Writes the color filter array phase field of the status register                                                 |
| intel_vvp_wbc_get_cfa_00_color_scaler  | Reads the color scaler register associated with the color filter phase 00                                        |
| intel_vvp_wbc_set_cfa_00_color_scaler  | Writes the color scaler register associated with the color filter phase 00                                       |
| intel_vvp_wbc_get_cfa_01_color_scaler  | Reads the color scaler register associated with the color filter phase 01                                        |
| intel_vvp_wbc_set_cfa_01_color_scaler  | Writes the color scaler register associated with the color filter phase 01                                       |
| intel_vvp_wbc_get_cfa_10_color_scaler  | Reads the color scaler register associated with the color filter phase 10                                        |
| intel_vvp_wbc_set_cfa_10_color_scaler  | Writes the color scaler register associated with the color filter phase 10                                       |
| intel_vvp_wbc_get_cfa_11_color_scaler  | Reads the color scaler register associated with the color filter phase 11                                        |
| intel_vvp_wbc_set_cfa_11_color_scaler  | Writes the color scaler register associated with the color filter phase 11                                       |

## intel\_vvp\_wbc\_init

### Prototype

```
int
intel_vvp_wbc_init(intel_vvp_wbc_instance* instance,
intel_vvp_core_base base);
```

**Description** Initialize an IP instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a non-zero error code.

**Arguments** instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

base - base address of the register map

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7)

kIntelVvpCorePidErr if the product\_id does not match with the expected product ID (0x017A)

kIntelVvpWbcRegMapVersionErr if the register map is not supported

## intel\_vvp\_wbc\_get\_lite\_mode

### Prototype

```
bool
intel_vvp_wbc_get_lite_mode(intel_vvp_wbc_instance* instance);
```

**Description** Returns the value of the LITE\_MODE register. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

**Returns** true if the IP is parameterized in lite mode

### intel\_vvp\_wbc\_get\_debug\_enabled

*Prototype*

```
bool
intel_vvp_wbc_get_debug_enabled(intel_vvp_wbc_instance*
instance);
```

*Description* Returns the value of the DEBUG\_ENABLED register. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

*Returns* true if the IP is parameterized with debug features enabled

### intel\_vvp\_wbc\_get\_bits\_per\_sample\_in

*Prototype*

```
uint8_t
intel_vvp_wbc_get_bits_per_sample_in(intel_vvp_wbc_instance*
instance);
```

*Description* Returns the value of the BPS\_IN register. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

*Returns* the input bits per color sample parameter used to generate the IP

### intel\_vvp\_wbc\_get\_bits\_per\_sample\_out

*Prototype*

```
uint8_t
intel_vvp_wbc_get_bits_per_sample_out(intel_vvp_wbc_instance*
instance);
```

*Description* Returns the value of the BPS\_OUT register. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

*Returns* the output bits per color sample parameter used to generate the IP

### intel\_vvp\_wbc\_get\_num\_color\_planes\_in

#### Prototype

```
uint8_t
intel_vvp_wbc_get_num_color_planes_in(intel_vvp_wbc_instance*
instance);
```

- Description** Returns the value of the NUM\_COLOR\_IN register. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.
- Arguments** instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure
- Returns** the input number of color planes parameter used to generate the IP

### intel\_vvp\_wbc\_get\_num\_color\_planes\_out

#### Prototype

```
uint8_t
intel_vvp_wbc_get_num_color_planes_out(intel_vvp_wbc_instance*
instance);
```

- Description** Returns the value of the NUM\_COLOR\_OUT register. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.
- Arguments** instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure
- Returns** the output number of color planes parameter used to generate the IP

### intel\_vvp\_wbc\_get\_pixels\_in\_parallel

#### Prototype

```
uint8_t
intel_vvp_wbc_get_pixels_in_parallel(intel_vvp_wbc_instance*
instance);
```

- Description** Returns the value of the PIXELS\_IN\_PARALLEL register. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.
- Arguments** instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure
- Returns** the pixels in parallel parameter used to generate the IP



#### intel\_vvp\_wbc\_get\_max\_width

*Prototype*

```
uint32_t
intel_vvp_wbc_get_max_width(intel_vvp_wbc_instance* instance);
```

*Description* Returns the value of the MAX\_WIDTH register. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

*Returns* the maximum field width parameter used to generate the IP

#### intel\_vvp\_wbc\_get\_max\_height

*Prototype*

```
uint32_t
intel_vvp_wbc_get_max_height(intel_vvp_wbc_instance* instance);
```

*Description* Returns the value of the MAX\_HEIGHT register. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

*Returns* the maximum field height parameter used to generate the IP

#### intel\_vvp\_wbc\_is\_running

*Prototype*

```
bool
intel_vvp_wbc_is_running(intel_vvp_wbc_instance* instance);
```

*Description* Returns the value of the IP Running bit from the STATUS register. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

*Returns* the running bit from the status register

### intel\_vvp\_wbc\_commit\_is\_pending

#### Prototype

```
bool
intel_vvp_wbc_commit_is_pending(intel_vvp_wbc_instance*
instance);
```

**Description** Returns the value of the commit pending bit from the STATUS register. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

**Returns** true if the agent interface has pending writes from a previous commit request

### intel\_vvp\_wbc\_get\_status

#### Prototype

```
uint32_t
intel_vvp_wbc_get_status(intel_vvp_wbc_instance* instance);
```

**Description** Returns the value of the STATUS register. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

**Returns** the value of the status register

### intel\_vvp\_wbc\_get\_frame\_stats

#### Prototype

```
int
intel_vvp_wbc_get_frame_stats(intel_vvp_wbc_instance*
instance, uint32_t* stats_out);
```

**Description** Reads the value of the FRAME\_STATS register. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

stats\_out - pointer of a variable to return the statistics register value

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer  
kIntelVvpWbcPointerErr if stats\_out is a null pointer

### intel\_vvp\_wbc\_commit

#### Prototype

```
int
intel_vvp_wbc_commit(intel_vvp_wbc_instance* instance);
```

**Description** Commits all pending writes before processing the next field. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

### intel\_vvp\_wbc\_get\_bypass

#### Prototype

```
bool
intel_vvp_wbc_get_bypass(intel_vvp_wbc_instance* instance);
```

**Description** Returns the value of the bypass bit from the CONTROL register. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

**Returns** the value of the bypass bit form the control register

### intel\_vvp\_wbc\_set\_bypass

#### Prototype

```
int
intel_vvp_wbc_set_bypass(intel_vvp_wbc_instance* instance,
bool bypass);
```

**Description** Writes to the bypass bit of the CONTROL register. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

bypass - new value of the bypass bit

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpWbcCommitPendingErr if a previous commit request is pending

### intel\_vvp\_wbc\_get\_cfa\_phase

**Prototype**

```
uint8_t
intel_vvp_wbc_get_cfa_phase(intel_vvp_wbc_instance* instance);
```

**Description** Returns the value of the color filter array phase field from the CONTROL register. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

**Returns** the value of the color filter array phase field form the control register

### intel\_vvp\_wbc\_set\_cfa\_phase

**Prototype**

```
int
intel_vvp_wbc_set_cfa_phase(intel_vvp_wbc_instance* instance,
uint8_t cfa_phase);
```

**Description** Writes to the color filter array phase field of the CONTROL register. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

cfa\_phase - new value of the color filter array phase field

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpWbcValueErr if the value of cfa\_phase is out of range

kIntelVvpWbcCommitPendingErr if a previous commit request is pending

#### intel\_vvp\_wbc\_get\_cfa\_00\_color\_scaler

##### Prototype

```
uint32_t
intel_vvp_wbc_get_cfa_00_color_scaler(intel_vvp_wbc_instance
*instance);
```

**Description** Returns the color scaler register associated with the color filter phase 00. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

**Returns** the value of the color scaler register associated with the color filter phase 00

#### intel\_vvp\_wbc\_set\_cfa\_00\_color\_scaler

##### Prototype

```
int
intel_vvp_wbc_set_cfa_00_color_scaler(intel_vvp_wbc_instance*
instance, uint32_t scaler);
```

**Description** Writes to the color scaler register associated with the color filter phase 00. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

scaler - new value of the color scaler register associated with the color filter phase 00

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpWbcValueErr if the value of scaler is out of range

kIntelVvpWbcCommitPendingErr if a previous commit request is pending

#### intel\_vvp\_wbc\_get\_cfa\_01\_color\_scaler

##### Prototype

```
uint32_t
intel_vvp_wbc_get_cfa_01_color_scaler(intel_vvp_wbc_instance
*instance);
```

- Description** Returns the color scaler register associated with the color filter phase 01. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.
- Arguments** instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure
- Returns** the value of the color scaler register associated with the color filter phase 01

#### intel\_vvp\_wbc\_set\_cfa\_01\_color\_scaler

##### Prototype

```
int
intel_vvp_wbc_set_cfa_01_color_scaler(intel_vvp_wbc_instance*
instance, uint32_t scaler);
```

- Description** Writes to the color scaler register associated with the color filter phase 01. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.
- Arguments** instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure
- scaler - new value of the color scaler register associated with the color filter phase 01
- Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
- kIntelVvpCoreInstanceErr if the instance is a null pointer
- kIntelVvpWbcValueErr if the value of scaler is out of range
- kIntelVvpWbcCommitPendingErr if a previous commit request is pending

#### intel\_vvp\_wbc\_get\_cfa\_10\_color\_scaler

##### Prototype

```
uint32_t
intel_vvp_wbc_get_cfa_10_color_scaler(intel_vvp_wbc_instance
*instance);
```

- Description** Returns the color scaler register associated with the color filter phase 10. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.
- Arguments** instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure

**Returns** the value of the color scaler register associated with the color filter phase 10

#### `intel_vvp_wbc_set_cfa_10_color_scaler`

**Prototype**

```
int
intel_vvp_wbc_set_cfa_10_color_scaler(intel_vvp_wbc_instance*
instance, uint32_t scaler);
```

**Description** Writes to the color scaler register associated with the color filter phase 10. The instance must be a valid `intel_vvp_wbc_instance` fully initialized.

**Arguments** instance - pointer to the `intel_vvp_wbc_instance` software driver instance structure

scaler - new value of the color scaler register associated with the color filter phase 10

**Returns** `kIntelVvpCoreOk` (0) in case of success, a negative error code in case of an error

`kIntelVvpCoreInstanceErr` if the instance is a null pointer

`kIntelVvpWbcValueErr` if the value of scaler is out of range

`kIntelVvpWbcCommitPendingErr` if a previous commit request is pending

#### `intel_vvp_wbc_get_cfa_11_color_scaler`

**Prototype**

```
uint32_t
intel_vvp_wbc_get_cfa_11_color_scaler(intel_vvp_wbc_instance
*instance);
```

**Description** Returns the color scaler register associated with the color filter phase 11. The instance must be a valid `intel_vvp_wbc_instance` fully initialized.

**Arguments** instance - pointer to the `intel_vvp_wbc_instance` software driver instance structure

**Returns** the value of the color scaler register associated with the color filter phase 11

## intel\_vvp\_wbc\_set\_cfa\_11\_color\_scaler

### Prototype

```
int
intel_vvp_wbc_set_cfa_11_color_scaler(intel_vvp_wbc_instance*
instance, uint32_t scaler);
```

- Description** Writes to the color scaler register associated with the color filter phase 11. The instance must be a valid intel\_vvp\_wbc\_instance fully initialized.
- Arguments**
- instance - pointer to the intel\_vvp\_wbc\_instance software driver instance structure
  - scaler - new value of the color scaler register associated with the color filter phase 11
- Returns**
- kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
  - kIntelVvpCoreInstanceErr if the instance is a null pointer
  - kIntelVvpWbcValueErr if the value of scaler is out of range
  - kIntelVvpWbcCommitPendingErr if a previous commit request is pending

### Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38



## 55. White Balance Statistics IP

### 55.1. About the White Balance Statistics IP

The White Balance Statistics IP analyses the pixel values across 7x7 zones within a region of interest. It operates on 2x2 Bayer color filter array input video streams and generates statistics that you can use for white balance correction.

The IP only supports lite variants. For more information about full and lite variants, refer to the *Altera Streaming Video Protocol Specification*

#### Related Information

[Altera Streaming Video Protocol Specification](#)

#### 55.1.1. White Balance Statistics IP Performance and Resources

Altera provides resource and utilization data for guidance only. The IP resource utilization depends on the device family, compilation tool version, compilation seed randomization, and other parameters.

| Device                                        | Pixels in Parallel | Bits per Color Sample | Max Resolution | Precision Bits | ALM   | M20k | DSP | f <sub>MAX</sub> (MHz) |
|-----------------------------------------------|--------------------|-----------------------|----------------|----------------|-------|------|-----|------------------------|
| Agilex 7 speed grade 2 (AGFA012R24A2E2V)      | 1                  | 10                    | 2              | 8              | 2,176 | 4    | 9   | 778                    |
|                                               | 1                  | 16                    | 2              | 10             | 2,809 | 6    | 26  | 728                    |
|                                               | 4                  | 10                    | 8              | 8              | 6,593 | 13   | 36  | 692                    |
|                                               | 4                  | 16                    | 8              | 10             | 8,655 | 23   | 104 | 680                    |
| Arria 10 speed grade 1 (10AS066H1F34E1HG)     | 1                  | 10                    | 2              | 8              | 1,801 | 13   | 9   | 531                    |
|                                               | 1                  | 16                    | 2              | 10             | 2,773 | 20   | 26  | 499                    |
|                                               | 2                  | 10                    | 4              | 8              | 2,711 | 21   | 18  | 526                    |
|                                               | 2                  | 16                    | 4              | 10             | 4,584 | 35   | 52  | 480                    |
| Cyclone 10 GX speed grade 5 (10CX220YF672E5G) | 1                  | 10                    | 2              | 8              | 1,794 | 13   | 9   | 467                    |
|                                               | 1                  | 16                    | 2              | 10             | 2,771 | 20   | 26  | 433                    |
| Stratix 10 speed grade 1 (1SX280LN2F43E1VG)   | 1                  | 10                    | 2              | 8              | 2,406 | 8    | 9   | 563                    |
|                                               | 1                  | 16                    | 2              | 10             | 3,910 | 10   | 26  | 572                    |
|                                               | 2                  | 10                    | 4              | 8              | 3,700 | 13   | 18  | 568                    |
|                                               | 2                  | 16                    | 4              | 10             | 6,576 | 17   | 52  | 542                    |

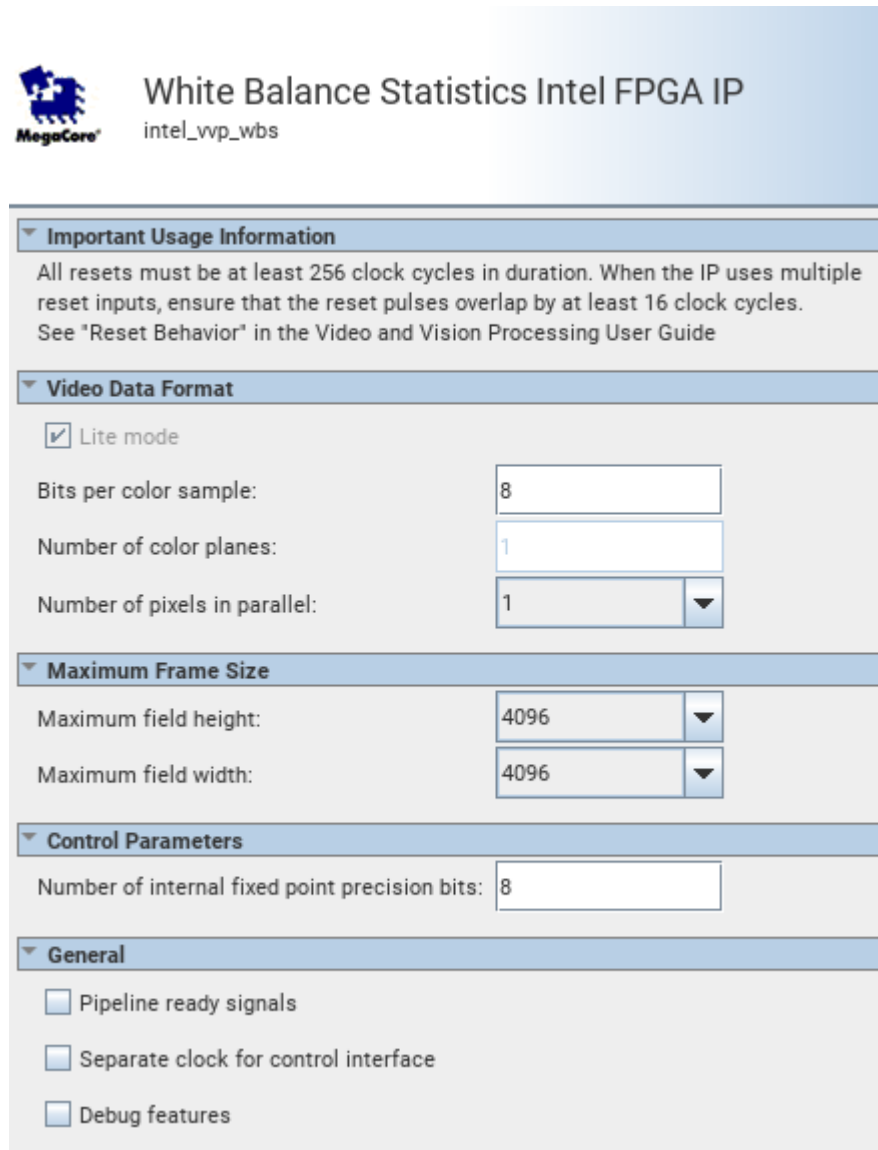
## 55.2. White Balance Statistics IP Parameters

The IP offers run-time and compile-time parameters.

**Table 1165. White Balance Statistics IP Parameters**

| Parameter                                   | Values              | Description                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Video Data Format</b>                    |                     |                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Lite mode</b>                            | On                  | The IP only supports lite mode.                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Bits per color sample</b>                | 8 to 16             | Select the number of bits per color sample.                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Number of color planes</b>               | 1                   | The IP only supports 1 color plane                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Number of pixels in parallel</b>         | 1, 2, 4, or 8       | Select the number of pixels in parallel.                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Maximum Frame Size</b>                   |                     |                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Maximum field height</b>                 | 1k, 2k, 4k, 8k, 16k | Specify the maximum height of incoming frames                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Maximum field width</b>                  | 2k, 4k, 8k, 16k     | Specify the maximum width of incoming frames                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Control Parameters</b>                   |                     |                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Precision bits</b>                       | 8 to 16             | Specify the number of fractional precision bits allocated for fixed-point calculations. The IP limits the maximum total precision to 26 bits for efficiency, where <b>Bits per color sample</b> number of bits constitute the integer part of the fixed-point numbers. Therefore, the upper limit of <b>Precision bits</b> goes down accordingly for <b>Bits per color sample</b> values greater than 10 bits. |
| <b>General</b>                              |                     |                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Pipeline ready signal</b>                | On or off           | Turn on to add extra pipeline registers to the AXI4-S <code>tready</code> signals                                                                                                                                                                                                                                                                                                                              |
| <b>Separate clock for control interface</b> | On or off           | Turn on for a separate clock for the control agent interface                                                                                                                                                                                                                                                                                                                                                   |
| <b>Debug features</b>                       | On or off           | Turn on to read back frame information registers and debugging information registers via the control agent interface.                                                                                                                                                                                                                                                                                          |

Figure 170. White Balance Statistics IP GUI



**White Balance Statistics Intel FPGA IP**  
intel\_vvp\_wbs

**Important Usage Information**  
All resets must be at least 256 clock cycles in duration. When the IP uses multiple reset inputs, ensure that the reset pulses overlap by at least 16 clock cycles. See "Reset Behavior" in the Video and Vision Processing User Guide

**Video Data Format**

☒ Lite mode

Bits per color sample: 8

Number of color planes: 1

Number of pixels in parallel: 1

**Maximum Frame Size**

Maximum field height: 4096

Maximum field width: 4096

**Control Parameters**

Number of internal fixed point precision bits: 8

**General**

☐ Pipeline ready signals

☐ Separate clock for control interface

☐ Debug features

## 55.3. White Balance Statistics IP Interfaces

**Table 1166. White Balance Statistics Clock and Reset Interfaces**

| Name              | Direction | Width | Description                                                                                                            |
|-------------------|-----------|-------|------------------------------------------------------------------------------------------------------------------------|
| main_clock_clk    | In        | 1     | AXI4-S processing clock.                                                                                               |
| main_reset_rst    | In        | 1     | AXI4-S processing reset.                                                                                               |
| agent_clock_clk   | In        | 1     | Clock for the memory-mapped control agent interface. Only if you turn on <b>Separate clock for control interface</b> . |
| agent_reset_reset | In        | 1     | Reset for the memory-mapped control agent interface. Only if you turn on <b>Separate clock for control interface</b> . |

**Table 1167. White Balance Statistics Control Interfaces**

| Name                              | Direction | Width | Description                              |
|-----------------------------------|-----------|-------|------------------------------------------|
| av_mm_control_agent_address       | In        | 10    | Avalon memory-mapped agent address       |
| av_mm_control_agent_write         | In        | 1     | Avalon memory-mapped agent write.        |
| av_mm_control_agent_writedata     | In        | 32    | Avalon memory-mapped agent write data.   |
| av_mm_control_agent_byteenable    | In        | 4     | Avalon memory-mapped agent byte enable.  |
| av_mm_control_agent_read          | In        | 1     | Avalon memory-mapped agent read.         |
| av_mm_control_agent_readdata      | Out       | 32    | Avalon memory-mapped agent read data.    |
| av_mm_control_agent_readdatavalid | Out       | 1     | Avalon memory-mapped agent read.         |
| av_mm_control_agent_waitrequest   | Out       | 1     | Avalon memory-mapped agent wait request. |

**Table 1168. White Balance Statistics Altera Streaming Video Interfaces**

| Name                 | Direction | Width | Description                                         |
|----------------------|-----------|-------|-----------------------------------------------------|
| <b>Input</b>         |           |       |                                                     |
| axi4s_vid_in_tdata   | In        | (181) | AXI4-S data in.                                     |
| axi4s_vid_in_tvalid  | In        | 1     | AXI4-S data valid.                                  |
| axi4s_vid_in_tuser   | In        | (182) | [0] AXI4-S start of video frame.<br>[N-1:1] Unused. |
| axi4s_vid_in_tlast   | In        | 1     | AXI4-S end of packet.                               |
| axi4s_vid_in_tready  | Out       | 1     | AXI4-S data ready.                                  |
| <b>Output</b>        |           |       |                                                     |
| axi4s_vid_out_tdata  | Out       | (181) | AXI4-S data out.                                    |
| axi4s_vid_out_tvalid | Out       | 1     | AXI4-S data valid.                                  |
| <i>continued...</i>  |           |       |                                                     |

(181) The equation gives all tdata widths sizes in these interfaces: pixels in parallel × max (round (bits per color sample × number of color planes, 8), 16)

(182) This equation gives all tuserwidths sizes in these interfaces:  $N = \text{ceil}(\text{tdata width} / 8)$

| Name                 | Direction | Width | Description                                         |
|----------------------|-----------|-------|-----------------------------------------------------|
| axi4s_vid_out_tuser  | Out       | (182) | [0] AXI4-S start of video frame.<br>[N-1:1] Unused. |
| axi4s_vid_out_tlast  | Out       | 1     | AXI4-S end of packet.                               |
| axi4s_vid_out_tready | In        | 1     | AXI4-S data ready.                                  |

### Related Information

Video and Vision Processing IP Interfaces on page 30

## 55.4. White Balance Statistics IP Registers

Each register is either read-only (RO) or read-write (RW).

**Table 1169. White Balance Statistics IP Registers**

In the software API the register names appear with a prefix of INTEL\_VVP, INTEL\_VVP\_CORE, or INTEL\_VVP\_WBS as appropriate and with an optional REG suffix

| Address                    | Register      | Access     |      | Description                                                                                                                                                                         |
|----------------------------|---------------|------------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                            |               | Lite (183) | Full |                                                                                                                                                                                     |
| Parameterization registers |               |            |      |                                                                                                                                                                                     |
| 0x0000                     | VID_PID       | RO         | N/A  | Read this register to retrieve the ID of the IP.<br>This register always returns 0x6FA7_0179.                                                                                       |
| 0x0004                     | VERSION       | RO         | N/A  | Read this register to retrieve the version information for the IP.                                                                                                                  |
| 0x0008                     | LITE_MODE     | RO         | N/A  | Read this register to determine if <b>Lite mode</b> is on.<br>This register always returns 1.                                                                                       |
| 0x000C                     | DEBUG_ENABLED | RO         | N/A  | Read this register to determine if <b>Debug features</b> are on.<br>This register returns 0 for off and 1 for on.                                                                   |
| 0x0010                     | BPS_IN        | RO         | N/A  | Read this register to determine the <b>Bits per color symbol</b> for the input streaming video interface.                                                                           |
| 0x0014                     | BPS_OUT       | RO         | N/A  | Read this register to determine the <b>Bits per color symbol</b> for the output streaming video interface.<br>The value of this register reads the same for its input counterpart.  |
| 0x0018                     | NUM_COLOR_IN  | RO         | N/A  | Read this register to determine the <b>Number of color planes</b> for the input streaming video interface.                                                                          |
| 0x001C                     | NUM_COLOR_OUT | RO         | N/A  | Read this register to determine the <b>Number of color planes</b> for the output streaming video interface.<br>The value of this register reads the same for its input counterpart. |
| 0x0020                     | PIP           | RO         | N/A  | Read this register to determine the <b>Number of pixels in parallel</b> .                                                                                                           |
| 0x0024                     | MAX_WIDTH     | RO         | N/A  | Read this register to determine the <b>Maximum field width</b> .                                                                                                                    |
| 0x0028                     | MAX_HEIGHT    | RO         | N/A  | Read this register to determine the <b>Maximum field height</b> .                                                                                                                   |
| continued...               |               |            |      |                                                                                                                                                                                     |

(183) Registers are RW only if you also turn on **Debug features**, otherwise they are WO.

| Address                                             | Register        | Access                |      | Description                                                                                                                                                                                                                                    |
|-----------------------------------------------------|-----------------|-----------------------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                     |                 | Lite <sup>(183)</sup> | Full |                                                                                                                                                                                                                                                |
| 0x002C                                              | PRECISION_BITS  | RO                    | N/A  | Read this register to determine the <b>Number of internal fixed point precision bits</b> .                                                                                                                                                     |
| 0x0030 to 0x011F                                    | -               | -                     | -    | Reserved.                                                                                                                                                                                                                                      |
| <b>Control, debugging, and statistics registers</b> |                 |                       |      |                                                                                                                                                                                                                                                |
| 0x0120                                              | IMG_INFO_WIDTH  | RW                    | N/A  | The expected width of the incoming video fields.                                                                                                                                                                                               |
| 0x0124                                              | IMG_INFO_HEIGHT | RW                    | N/A  | The expected height of the incoming video fields.                                                                                                                                                                                              |
| 0x0128 to 0x013F                                    | -               | -                     | -    | Reserved.                                                                                                                                                                                                                                      |
| 0x0140                                              | STATUS          | RO                    | N/A  | Read this register for information about the IP status.                                                                                                                                                                                        |
| 0x0144                                              | FRAME_STATS     | RO                    | N/A  | Read this register for some frame statistics.                                                                                                                                                                                                  |
| 0x0148                                              | COMMIT          | RW                    | N/A  | Write any value to this register to submit changes to the control, region of interest, zone and range registers. A pending commit request does not block Freeze Statistics Request of the CONTROL register, which the IP samples continuously. |
| 0x014C                                              | CONTROL         | RW                    | N/A  | Control bits and fields of the IP.                                                                                                                                                                                                             |
| 0x0150                                              | H_START         | RW                    | N/A  | Region of interest horizontal start value                                                                                                                                                                                                      |
| 0x0154                                              | V_START         | RW                    | N/A  | Region of interest vertical start value                                                                                                                                                                                                        |
| 0x0158                                              | H_END           | RW                    | N/A  | Region of interest horizontal end value                                                                                                                                                                                                        |
| 0x015C                                              | V_END           | RW                    | N/A  | Region of interest vertical end value                                                                                                                                                                                                          |
| 0x0160                                              | ZONE_H_COUNT    | RW                    | N/A  | Zone horizontal count value                                                                                                                                                                                                                    |
| 0x0164                                              | ZONE_V_COUNT    | RW                    | N/A  | Zone vertical count value                                                                                                                                                                                                                      |
| 0x0168                                              | CFA_X0_RANGE_LO | RW                    | N/A  | Low range value for the left column of the color filter array.                                                                                                                                                                                 |
| 0x016C                                              | CFA_X0_RANGE_HI | RW                    | N/A  | High range value for the left column of the color filter array.                                                                                                                                                                                |
| 0x0170                                              | CFA_X1_RANGE_LO | RW                    | N/A  | Low range value for the right column of the color filter array.                                                                                                                                                                                |
| 0x0174                                              | CFA_X1_RANGE_HI | RW                    | N/A  | High range value for the right column of the color filter array.                                                                                                                                                                               |
| 0x0178 to 0x01FF                                    | -               | -                     | -    | Reserved                                                                                                                                                                                                                                       |
| 0x0200 - 0x09FC                                     | RESULTS_TABLE   | RO                    | N/A  | White balance statistics values                                                                                                                                                                                                                |
| 0x0A00 to 0x0FFF                                    | -               | -                     | -    | Reserved                                                                                                                                                                                                                                       |

<sup>(183)</sup> Registers are RW only if you also turn on **Debug features**, otherwise they are WO.

## Register Bit Descriptions

**Table 1170. STATUS**

| Name             | Bits | Description                                                          |
|------------------|------|----------------------------------------------------------------------|
| Reserved         | 31:3 | Reserved.                                                            |
| Stats are Frozen | 2    | The IP sets this bit to indicate it stopped updating the statistics. |
| Commit           | 1    | Pending commit                                                       |
| Running          | 0    | When 1 the IP is processing data.                                    |

**Table 1171. FRAME\_STATS**

| Name     | Bits | Description                     |
|----------|------|---------------------------------|
| Reserved | 31:8 | Reserved.                       |
| Checksum | 7:0  | A simple checksum of the frame. |

**Table 1172. CONTROL**

| Name                            | Bits | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|---------------------------------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Freeze Statistics Request       | 31   | Set this bit to 1 for the IP to start sampling a new set of white balance and frame statistics. The IP lowers the <code>Stats are Frozen</code> bit of the <code>STATUS</code> register as a response, and you need to poll <code>Stats are Frozen</code> bit until it is 1 before reading the statistics. You should keep <code>Freeze Statistics Request</code> 1 until it raises <code>Stats are Frozen</code> . You must set <code>Freeze Statistics Request</code> to 0 before starting over with a new sampling request. Deviating from this flow might result in unexpected IP behavior. |
| Reserved                        | 30:5 | Reserved.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Color filter array phase invert | 4:3  | Independently invert the numerator and denominator pixels of the 2x2 CFA phase. For example, if color filter array phase is 00, and color filter array phase invert is 01, the effective color filter array phase is: <div style="background-color: #f0f0f0; padding: 5px; margin: 5px 0;"> <math display="block">\begin{matrix} C_0C_3 \\ C_2C_1 \end{matrix}</math> </div> Likewise, when color filter array phase invert is 10: <div style="background-color: #f0f0f0; padding: 5px; margin: 5px 0;"> <math display="block">\begin{matrix} C_2C_1 \\ C_0C_3 \end{matrix}</math> </div>       |
| Color filter array phase        | 2:1  | Specifies 2x2 color filter order starting from the top left corner of the image. <div style="background-color: #f0f0f0; padding: 5px; margin: 5px 0;"> <math display="block">\begin{matrix} 00 &amp; 01 &amp; 10 &amp; 11 \\ C_0C_1 &amp; C_1C_0 &amp; C_2C_3 &amp; C_3C_2 \\ C_2C_3 &amp; C_3C_2 &amp; C_0C_1 &amp; C_1C_0 \end{matrix}</math> </div>                                                                                                                                                                                                                                          |
| Reserved                        | 0    | Reserved. Write 0.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |

**Table 1173. H\_START**

| Name     | Bits  | Description                                                                                                                                                                                                                                |
|----------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved | 31:16 | Reserved. Write 0.                                                                                                                                                                                                                         |
| Position | 15:0  | Horizontal position of the first pixel in the region of interest. The IP starts counting the pixels from index 0. Set the value of this register to a number that is even and an integer multiple of <b>Number of pixels in parallel</b> . |

**Table 1174. V\_START**

| Name     | Bits  | Description                                                                                                                                                   |
|----------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved | 31:16 | Reserved. Write 0.                                                                                                                                            |
| Position | 15:0  | Vertical position of the first line in the region of interest. The IP starts counts the lines from index 0. Set the value of this register to an even number. |

**Table 1175. H\_END**

| Name     | Bits  | Description                                                                                                                                                                                                                                                                                                                                               |
|----------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved | 31:16 | Reserved. Write 0.                                                                                                                                                                                                                                                                                                                                        |
| Position | 15:0  | Horizontal position of the last pixel in the region of interest incremented by 1. The horizontal size of the region of interest must be even and an integer multiple of <b>Number of pixels in parallel</b> . Therefore, set the value of this register to an odd number and an integer multiple of <b>Number of pixels in parallel</b> incremented by 1. |

**Table 1176. V\_END**

| Name     | Bits  | Description                                                                                                                                                                                          |
|----------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved | 31:16 | Reserved. Write 0.                                                                                                                                                                                   |
| Position | 15:0  | Vertical position of the last line in the region of interest incremented by 1. The vertical size of the region of interest must be even. Therefore, set the value of this register to an odd number. |

**Table 1177. ZONE\_H\_COUNT**

| Name     | Bits  | Description                                                                              |
|----------|-------|------------------------------------------------------------------------------------------|
| Reserved | 31:16 | Reserved. Write 0.                                                                       |
| Value    | 15:0  | Number of horizontal pixels in a zone. Set the value of this register to an even number. |

**Table 1178. ZONE\_V\_COUNT**

| Name     | Bits  | Description                                                                            |
|----------|-------|----------------------------------------------------------------------------------------|
| Reserved | 31:16 | Reserved. Write 0.                                                                     |
| Value    | 15:0  | Number of vertical pixels in a zone. Set the value of this register to an even number. |

**Table 1179. CFA\_X0\_RANGE\_LO**

| Name  | Bits | Description                                                                                                                                                                                                                                                                                                                                   |
|-------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Value | 31:0 | Lower range of the ratio check for the left column of the 2x2 color filter array. The IP marks the whole 2x2 color filter array out of range and discards all statistics regarding the 2x2 pixel group if the ratio calculated for the left column of the 2x2 color filter array is smaller than the value of this register. <sup>(184)</sup> |

(184) The numbering format is fixed-point unsigned integer 16.16, where the bit ranges [31:16] and [15:0] are the integer and fractional parts, respectively. For the integer part, the IP uses the least significant **Bits per color sample** number of bits and assumes the remaining MSBs are 0. For the fractional part, the IP uses the most significant **Number of internal fixed point precision bits** and assumes the remaining LSBs are 0.



**Table 1180. CFA\_X0\_RANGE\_HI**

| Name  | Bits | Description                                                                                                                                                                                                                                                                                                                                    |
|-------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Value | 31:0 | Higher range of the ratio check for the left column of the 2x2 color filter array. The IP marks the whole 2x2 color filter array out of range and discards all statistics regarding the 2x2 pixel group if the ratio calculated for the left column of the 2x2 color filter array is greater than the value of this register. <sup>(184)</sup> |

**Table 1181. CFA\_X1\_RANGE\_LO**

| Name  | Bits | Description                                                                                                                                                                                                                                                                                                                                     |
|-------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Value | 31:0 | Lower range of the ratio check for the right column of the 2x2 color filter array. The IP marks the whole 2x2 color filter array out of range and discards all statistics regarding the 2x2 pixel group if the ratio calculated for the right column of the 2x2 color filter array is smaller than the value of this register. <sup>(184)</sup> |

**Table 1182. CFA\_X1\_RANGE\_HI**

| Name  | Bits | Description                                                                                                                                                                                                                                                                                                                                      |
|-------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Value | 31:0 | Higher range of the ratio check for the right column of the 2x2 color filter array. The IP marks the whole 2x2 color filter array out of range and discards all statistics regarding the 2x2 pixel group if the ratio calculated for the right column of the 2x2 color filter array is greater than the value of this register. <sup>(184)</sup> |

**Table 1183. RESULTS\_TABLE**

| Name                             | Bits  | Description                                                                                                                                                                                                                                                                                                                                                                                                           |
|----------------------------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserved                         | 31:20 | Reserved.                                                                                                                                                                                                                                                                                                                                                                                                             |
| White balance statistics values. | 19:0  | The IP stores the statistics for each zone linearly from zone 1 to zone 49, packing 7x7 zones in raster order, left to right, top to bottom. The IP concatenates the two sum of ratios and the number of counted 2x2 color filter arrays, spanning across multiple registers. Altera provides an API function for reading statistics using zone coordinates and unpacking the statistics into a more accessible form. |

## 55.5. White Balance Statistics IP Software API

The IP includes software for run-time control. The IP does not fit any of the generic device models provided by the Nios II HAL and it exposes a set of dedicated accessors to the control and status registers. The IP driver structure inherits the base driver structure so all common methods defined in *Video and Vision Processing IPs Software API* are applicable.

Register definition header file: `intel_vvp_wbs_regs.h`

Include file: `intel_vvp_wbs.h`

**Table 1184. White Balance Statistics IP Software API**

| Name                                         | Description                                                                            |
|----------------------------------------------|----------------------------------------------------------------------------------------|
| <code>intel_vvp_wbs_init</code>              | Initialize an IP instance                                                              |
| <code>intel_vvp_core_*</code>                | Accessors defined in <i>Video and Vision Processing IPs Software Programming Model</i> |
| <code>intel_vvp_wbs_get_lite_mode</code>     | Returns if <b>Lite mode</b> is on                                                      |
| <code>intel_vvp_wbs_get_debug_enabled</code> | Returns if <b>Debug features</b> is on                                                 |
| <i>continued...</i>                          |                                                                                        |

| Name                                   | Description                                                                                                      |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------|
| intel_vvp_wbs_get_bits_per_sample_in   | Returns the number of bits per color sample for the streaming input interface                                    |
| intel_vvp_wbs_get_bits_per_sample_out  | Returns the number of bits per color sample value for the streaming output interface                             |
| intel_vvp_wbs_get_num_color_planes_in  | Returns the number of color planes of the streaming input interface                                              |
| intel_vvp_wbs_get_num_color_planes_out | Returns the number of color planes of the streaming output interface                                             |
| intel_vvp_wbs_get_pixels_in_parallel   | Returns the pixels in parallel of the streaming input and output interfaces                                      |
| intel_vvp_wbs_get_max_width            | Returns the maximum number of pixels that the IP supports on the horizontal dimension of an image or video frame |
| intel_vvp_wbs_get_max_height           | Returns the maximum number of pixels that the IP supports on the vertical dimension of an image or video frame   |
| intel_vvp_wbs_get_precision_bits       | Returns the number of precision bits                                                                             |
| intel_vvp_wbs_is_running               | Reads if the IP is running                                                                                       |
| intel_vvp_wbs_commit_is_pending        | Reads if the IP has uncommitted writes                                                                           |
| intel_vvp_wbs_stats_are_frozen         | Reads if the IP finished collecting the statistics                                                               |
| intel_vvp_wbs_get_status               | Reads the status register                                                                                        |
| intel_vvp_wbs_get_frame_stats          | Reads the frame statistics register                                                                              |
| intel_vvp_wbs_commit                   | Commit pending writes                                                                                            |
| intel_vvp_wbs_get_bypass               | Reads the bypass bit from the status register                                                                    |
| intel_vvp_wbs_set_bypass               | Writes the bypass bit of the status register                                                                     |
| intel_vvp_wbs_get_cfa_phase            | Reads the color filter array phase field from the status register                                                |
| intel_vvp_wbs_set_cfa_phase            | Writes the color filter array phase field of the status register                                                 |
| intel_vvp_wbs_get_cfa_invert           | Reads the color filter array invert field from the status register                                               |
| intel_vvp_wbs_set_cfa_invert           | Writes the color filter array invert field of the status register                                                |
| intel_vvp_wbs_get_freeze_stats_request | Reads the freeze statistics request bit from the status register                                                 |
| intel_vvp_wbs_set_freeze_stats_request | Writes the freeze statistics request bit of the status register                                                  |
| intel_vvp_wbs_get_h_start              | Reads the horizontal starting coordinate of the region of interest                                               |
| intel_vvp_wbs_set_h_start              | Writes the horizontal starting coordinate of the region of interest                                              |
| intel_vvp_wbs_get_v_start              | Reads the vertical starting coordinate of the region of interest                                                 |
| intel_vvp_wbs_set_v_start              | Writes the vertical starting coordinate of the region of interest                                                |
| intel_vvp_wbs_get_h_end                | Reads the horizontal ending coordinate of the region of interest                                                 |
| intel_vvp_wbs_set_h_end                | Writes the horizontal ending coordinate of the region of interest                                                |
| intel_vvp_wbs_get_v_end                | Reads the vertical ending coordinate of the region of interest                                                   |
| intel_vvp_wbs_set_v_end                | Writes the vertical ending coordinate of the region of interest                                                  |
| intel_vvp_wbs_get_zone_h_count         | Reads the number of horizontal pixels in each zone                                                               |
| <i>continued...</i>                    |                                                                                                                  |

| Name                                     | Description                                                                                   |
|------------------------------------------|-----------------------------------------------------------------------------------------------|
| intel_vvp_wbs_set_zone_h_count           | Writes the number of horizontal pixels in each zone                                           |
| intel_vvp_wbs_get_zone_v_count           | Reads the number of vertical pixels in each zone                                              |
| intel_vvp_wbs_set_zone_v_count           | Writes the number of vertical pixels in each zone                                             |
| intel_vvp_wbs_get_cfa_x0_range_lo        | Reads the lower value of the ratio range checking for the first column in a 2x2 pixel group   |
| intel_vvp_wbs_set_cfa_x0_range_lo        | Writes the lower value of the ratio range checking for the first column in a 2x2 pixel group  |
| intel_vvp_wbs_get_cfa_x0_range_hi        | Reads the upper value of the ratio range checking for the first column in a 2x2 pixel group   |
| intel_vvp_wbs_set_cfa_x0_range_hi        | Writes the upper value of the ratio range checking for the first column in a 2x2 pixel group  |
| intel_vvp_wbs_get_cfa_x1_range_lo        | Reads the lower value of the ratio range checking for the second column in a 2x2 pixel group  |
| intel_vvp_wbs_set_cfa_x1_range_lo        | Writes the lower value of the ratio range checking for the second column in a 2x2 pixel group |
| intel_vvp_wbs_get_cfa_x1_range_hi        | Reads the upper value of the ratio range checking for the second column in a 2x2 pixel group  |
| intel_vvp_wbs_set_cfa_x1_range_hi        | Writes the upper value of the ratio range checking for the second column in a 2x2 pixel group |
| intel_vvp_wbs_get_table_entries          | Reads the white balance statistics in a raw format                                            |
| intel_vvp_wbs_get_formatted_table_result | Reads the white balance statistics as a formatted table                                       |

### intel\_vvp\_wbs\_init

#### Prototype

```
int
intel_vvp_wbs_init(intel_vvp_wbs_instance* instance,
intel_vvp_core_base base);
```

**Description** Initialize an IP instance. The initialization stops early if the vendor ID or product ID read at the base address are not a match or if the register map version is not supported. Otherwise, the function proceeds to read and store the IP compile-time parameterization. The instance is not fully initialized and the application should not use it further if returning a nonzero error code.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

base - base address of the register map

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpCoreVidErr if the vendor id of the core is not the IntelFPGA vendor ID (0x6AF7)

kIntelVvpCorePidErr if the product\_id does not match with the expected product ID (0x0179)

kIntelVvpWbsRegMapVersionErr if the register map is not supported

### intel\_vvp\_wbs\_get\_lite\_mode

#### Prototype

```
bool
intel_vvp_wbs_get_lite_mode(intel_vvp_wbs_instance* instance);
```

**Description** Returns the value of the LITE\_MODE register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

**Returns** true if the IP is parameterized in lite mode

### intel\_vvp\_wbs\_get\_debug\_enabled

#### Prototype

```
bool
intel_vvp_wbs_get_debug_enabled(intel_vvp_wbs_instance*
instance);
```

**Description** Returns the value of the DEBUG\_ENABLED register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

**Returns** true if the IP is parameterized with debug features enabled

### intel\_vvp\_wbs\_get\_bits\_per\_sample\_in

#### Prototype

```
uint8_t
intel_vvp_wbs_get_bits_per_sample_in(intel_vvp_wbs_instance*
instance);
```

**Description** Returns the value of the BPS\_IN register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

*Returns* the input bits per color sample parameter used to generate the IP

#### intel\_vvp\_wbs\_get\_bits\_per\_sample\_out

*Prototype*

```
uint8_t
intel_vvp_wbs_get_bits_per_sample_out(intel_vvp_wbs_instance*
instance);
```

*Description* Returns the value of the BPS\_OUT register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

*Returns* the output bits per color sample parameter used to generate the IP

#### intel\_vvp\_wbs\_get\_num\_color\_planes\_in

*Prototype*

```
uint8_t
intel_vvp_wbs_get_num_color_planes_in(intel_vvp_wbs_instance*
instance);
```

*Description* Returns the value of the NUM\_COLOR\_IN register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

*Returns* the input number of color planes parameter used to generate the IP

#### intel\_vvp\_wbs\_get\_num\_color\_planes\_out

*Prototype*

```
uint8_t
intel_vvp_wbs_get_num_color_planes_out(intel_vvp_wbs_instance*
instance);
```

*Description* Returns the value of the NUM\_COLOR\_OUT register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

**Returns** the output number of color planes parameter used to generate the IP

### `intel_vvp_wbs_get_pixels_in_parallel`

**Prototype**

```
uint8_t
intel_vvp_wbs_get_pixels_in_parallel(intel_vvp_wbs_instance*
instance);
```

**Description** Returns the value of the PIXELS\_IN\_PARALLEL register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

**Returns** the pixels in parallel parameter used to generate the IP

### `intel_vvp_wbs_get_max_width`

**Prototype**

```
uint32_t
intel_vvp_wbs_get_max_width(intel_vvp_wbs_instance* instance);
```

**Description** Returns the value of the MAX\_WIDTH register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

**Returns** the maximum field width parameter used to generate the IP

### `intel_vvp_wbs_get_max_height`

**Prototype**

```
uint32_t
intel_vvp_wbs_get_max_height(intel_vvp_wbs_instance* instance);
```

**Description** Returns the value of the MAX\_HEIGHT register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

**Returns** the maximum field height parameter used to generate the IP

### intel\_vvp\_wbs\_get\_precision\_bits

*Prototype*

```
uint8_t
intel_vvp_wbs_get_precision_bits(intel_vvp_wbs_instance*
instance);
```

*Description* Returns the value of the PRECISION\_BITS register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

*Returns* the precision bits parameter used to generate the IP

### intel\_vvp\_wbs\_is\_running

*Prototype*

```
bool
intel_vvp_wbs_is_running(intel_vvp_wbs_instance* instance);
```

*Description* Returns the value of the IP Running bit from the STATUS register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

*Returns* the running bit from the status register

### intel\_vvp\_wbs\_commit\_is\_pending

*Prototype*

```
bool
intel_vvp_wbs_commit_is_pending(intel_vvp_wbs_instance*
instance);
```

*Description* Returns the value of the commit pending bit from the STATUS register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

*Returns* true if the agent interface has pending writes from a previous commit request

### intel\_vvp\_wbs\_stats\_are\_frozen

#### Prototype

```
bool
intel_vvp_wbs_stats_are_frozen(intel_vvp_wbs_instance*
instance);
```

**Description** Returns the value of the statistics are frozen bit from the STATUS register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

**Returns** the statistics are frozen bit from the status register

### intel\_vvp\_wbs\_get\_status

#### Prototype

```
uint32_t
intel_vvp_wbs_get_status(intel_vvp_wbs_instance* instance);
```

**Description** Returns the value of the STATUS register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

**Returns** the value of the status register

### intel\_vvp\_wbs\_get\_frame\_stats

#### Prototype

```
int
intel_vvp_wbs_get_frame_stats(intel_vvp_wbs_instance*
instance, uint32_t* stats_out);
```

**Description** Reads the value of the FRAME\_STATS register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

stats\_out - pointer of a variable to return the statistics register value

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer



kIntelVvpWbsFreezePendingErr if the last freeze request is pending

kIntelVvpWbsPointerErr if stats\_out is a null pointer

#### intel\_vvp\_wbs\_commit

##### Prototype

```
int
intel_vvp_wbs_commit(intel_vvp_wbs_instance* instance);
```

**Description** Commits all pending writes before processing the next field. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

#### intel\_vvp\_wbs\_get\_bypass

##### Prototype

```
bool
intel_vvp_wbs_get_bypass(intel_vvp_wbs_instance* instance);
```

**Description** Returns the value of the bypass bit from the CONTROL register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

**Returns** the value of the bypass bit form the control register

#### intel\_vvp\_wbs\_set\_bypass

##### Prototype

```
int
intel_vvp_wbs_set_bypass(intel_vvp_wbs_instance* instance,
bool bypass);
```

**Description** Writes to the bypass bit of the CONTROL register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

bypass - new value of the bypass bit

*Returns* kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpWbsCommitPendingErr if a previous commit request is pending

### intel\_vvp\_wbs\_get\_cfa\_phase

*Prototype*

```
uint8_t
intel_vvp_wbs_get_cfa_phase(intel_vvp_wbs_instance* instance);
```

*Description* Returns the value of the color filter array phase field from the CONTROL register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

*Returns* the value of the color filter array phase field form the control register

### intel\_vvp\_wbs\_set\_cfa\_phase

*Prototype*

```
int
intel_vvp_wbs_set_cfa_phase(intel_vvp_wbs_instance* instance,
uint8_t cfa_phase);
```

*Description* Writes to the color filter array phase field of the CONTROL register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

*Arguments* instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

cfa\_phase - new value of the color filter array phase field

*Returns* kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpWbsValueErr if the value of cfa\_phase is out of range

kIntelVvpWbsCommitPendingErr if a previous commit request is pending

### intel\_vvp\_wbs\_get\_cfa\_invert

#### Prototype

```
uint8_t
intel_vvp_wbs_get_cfa_invert(intel_vvp_wbs_instance* instance);
```

**Description** Returns the value of the color filter array invert field from the CONTROL register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

**Returns** the value of the color filter array invert field from the control register

### intel\_vvp\_wbs\_set\_cfa\_invert

#### Prototype

```
int
intel_vvp_wbs_set_cfa_invert(intel_vvp_wbs_instance* instance,
uint8_t cfa_invert);
```

**Description** Writes to the color filter array invert field of the CONTROL register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

cfa\_invert - new value of the color filter array invert field

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpWbsValueErr if the value of cfa\_invert is out of range

kIntelVvpWbsCommitPendingErr if a previous commit request is pending

### intel\_vvp\_wbs\_get\_freeze\_stats\_request

#### Prototype

```
bool
intel_vvp_wbs_get_freeze_stats_request(intel_vvp_wbs_instance*
instance);
```

- Description** Returns the value of the freeze statistics request bit from the CONTROL register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.
- Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure
- Returns** the value of the freeze statistics request bit form the control register

#### intel\_vvp\_wbs\_set\_freeze\_stats\_request

##### Prototype

```
int
intel_vvp_wbs_set_freeze_stats_request(intel_vvp_wbs_instance*
instance, bool freeze_stats);
```

- Description** Writes to the freeze statistics request bit of the CONTROL register. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.
- Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure
- freeze\_stats - new value of the freeze statistics request bit
- Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
- kIntelVvpCoreInstanceErr if the instance is a null pointer

#### intel\_vvp\_wbs\_get\_h\_start

##### Prototype

```
uint16_t
intel_vvp_wbs_get_h_start(intel_vvp_wbs_instance* instance);
```

- Description** Returns the horizontal starting coordinate of the region of interest. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.
- Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure
- Returns** the value of the horizontal starting coordinate of the region of interest

### intel\_vvp\_wbs\_set\_h\_start

#### Prototype

```
int
intel_vvp_wbs_set_h_start(intel_vvp_wbs_instance* instance,
uint16_t h_start);
```

**Description** Writes the horizontal starting coordinate of the region of interest. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

h\_start - new value of the horizontal starting coordinate of the region of interest

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpWbsValueErr if the value of h\_start is out of range

kIntelVvpWbsCommitPendingErr if a previous commit request is pending

### intel\_vvp\_wbs\_get\_v\_start

#### Prototype

```
uint16_t
intel_vvp_wbs_get_v_start(intel_vvp_wbs_instance* instance);
```

**Description** Returns the vertical starting coordinate of the region of interest. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

**Returns** the value of the vertical starting coordinate of the region of interest

### intel\_vvp\_wbs\_set\_v\_start

#### Prototype

```
int
intel_vvp_wbs_set_v_start(intel_vvp_wbs_instance* instance,
uint16_t v_start);
```

- Description** Writes the vertical starting coordinate of the region of interest. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.
- Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure
- v\_start - new value of the vertical starting coordinate of the region of interest
- Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error
- kIntelVvpCoreInstanceErr if the instance is a null pointer
- kIntelVvpWbsValueErr if the value of v\_start is out of range
- kIntelVvpWbsCommitPendingErr if a previous commit request is pending

#### intel\_vvp\_wbs\_get\_h\_end

##### Prototype

```
uint16_t
intel_vvp_wbs_get_h_end(intel_vvp_wbs_instance* instance);
```

- Description** Returns the horizontal ending coordinate of the region of interest. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.
- Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure
- Returns** the value of the horizontal ending coordinate of the region of interest

#### intel\_vvp\_wbs\_set\_h\_end

##### Prototype

```
int
intel_vvp_wbs_set_h_end(intel_vvp_wbs_instance* instance,
uint16_t h_end);
```

- Description** Writes the horizontal ending coordinate of the region of interest. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.
- Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure
- h\_end - new value of the horizontal ending coordinate of the region of interest

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpWbsValueErr if the value of h\_end is out of range

kIntelVvpWbsCommitPendingErr if a previous commit request is pending

#### intel\_vvp\_wbs\_get\_v\_end

**Prototype**

```
uint16_t
intel_vvp_wbs_get_v_end(intel_vvp_wbs_instance* instance);
```

**Description** Returns the vertical ending coordinate of the region of interest. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

**Returns** the value of the vertical ending coordinate of the region of interest

#### intel\_vvp\_wbs\_set\_v\_end

**Prototype**

```
int
intel_vvp_wbs_set_v_end(intel_vvp_wbs_instance* instance,
uint16_t v_end);
```

**Description** Writes the vertical ending coordinate of the region of interest. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

v\_end - new value of the vertical ending coordinate of the region of interest

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpWbsValueErr if the value of v\_end is out of range

kIntelVvpWbsCommitPendingErr if a previous commit request is pending

### intel\_vvp\_wbs\_get\_zone\_h\_count

#### Prototype

```
uint16_t
intel_vvp_wbs_get_zone_h_count(intel_vvp_wbs_instance*
instance);
```

**Description** Returns the number of horizontal pixels in each zone. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

**Returns** the number of horizontal pixels in each zone

### intel\_vvp\_wbs\_set\_zone\_h\_count

#### Prototype

```
int
intel_vvp_wbs_set_zone_h_count(intel_vvp_wbs_instance*
instance, uint16_t zone_h_count);
```

**Description** Writes the number of horizontal pixels in each zone. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

zone\_h\_count - new value of the number of horizontal pixels in each zone

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpWbsValueErr if the value of zone\_h\_count is out of range

kIntelVvpWbsCommitPendingErr if a previous commit request is pending

### intel\_vvp\_wbs\_get\_zone\_v\_count

#### Prototype

```
uint16_t
intel_vvp_wbs_get_zone_v_count(intel_vvp_wbs_instance*
instance);
```

**Description** Returns the number of vertical pixels in each zone. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.



**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

**Returns** the number of vertical pixels in each zone

#### intel\_vvp\_wbs\_set\_zone\_v\_count

**Prototype**

```
int
intel_vvp_wbs_set_zone_v_count(intel_vvp_wbs_instance*
instance, uint16_t zone_v_count);
```

**Description** Writes the number of vertical pixels in each zone. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

zone\_v\_count - new value of the number of vertical pixels in each zone

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpWbsValueErr if the value of zone\_v\_count is out of range

kIntelVvpWbsCommitPendingErr if a previous commit request is pending

#### intel\_vvp\_wbs\_get\_cfa\_x0\_range\_lo

**Prototype**

```
uint32_t
intel_vvp_wbs_get_cfa_x0_range_lo(intel_vvp_wbs_instance*
instance);
```

**Description** Returns the lower value of the ratio range checking for the first column in a 2x2 pixel group. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

**Returns** the lower value of the ratio range checking for the first column in a 2x2 pixel group

### intel\_vvp\_wbs\_set\_cfa\_x0\_range\_lo

#### Prototype

```
int
intel_vvp_wbs_set_cfa_x0_range_lo(intel_vvp_wbs_instance*
instance, uint32_t cfa_x0_range_lo);
```

**Description** Writes the lower value of the ratio range checking for the first column in a 2x2 pixel group. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

cfa\_x0\_range\_lo - new value of the lower range checking for the first column in a 2x2 pixel group

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpWbsValueErr if the value of cfa\_x0\_range\_lo is out of range

kIntelVvpWbsCommitPendingErr if a previous commit request is pending

### intel\_vvp\_wbs\_get\_cfa\_x0\_range\_hi

#### Prototype

```
uint32_t
intel_vvp_wbs_get_cfa_x0_range_hi(intel_vvp_wbs_instance*
instance);
```

**Description** Returns the upper value of the ratio range checking for the first column in a 2x2 pixel group. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

**Returns** the upper value of the ratio range checking for the first column in a 2x2 pixel group

### intel\_vvp\_wbs\_set\_cfa\_x0\_range\_hi

#### Prototype

```
int
intel_vvp_wbs_set_cfa_x0_range_hi(intel_vvp_wbs_instance*
instance, uint32_t cfa_x0_range_hi);
```

|                    |                                                                                                                                                                                                                                                                                                             |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i> | Writes the upper value of the ratio range checking for the first column in a 2x2 pixel group. The instance must be a valid intel_vvp_wbs_instance fully initialized.                                                                                                                                        |
| <i>Arguments</i>   | instance - pointer to the intel_vvp_wbs_instance software driver instance structure<br><br>cfa_x0_range_hi - new value of the upper range checking for the first column in a 2x2 pixel group                                                                                                                |
| <i>Returns</i>     | kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error<br><br>kIntelVvpCoreInstanceErr if the instance is a null pointer<br><br>kIntelVvpWbsValueErr if the value of cfa_x0_range_hi is out of range<br><br>kIntelVvpWbsCommitPendingErr if a previous commit request is pending |

#### intel\_vvp\_wbs\_get\_cfa\_x1\_range\_lo

##### Prototype

```
uint32_t
intel_vvp_wbs_get_cfa_x1_range_lo(intel_vvp_wbs_instance*
instance);
```

|                    |                                                                                                                                                                        |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i> | Returns the lower value of the ratio range checking for the second column in a 2x2 pixel group. The instance must be a valid intel_vvp_wbs_instance fully initialized. |
| <i>Arguments</i>   | instance - pointer to the intel_vvp_wbs_instance software driver instance structure                                                                                    |
| <i>Returns</i>     | the lower value of the ratio range checking for the second column in a 2x2 pixel group                                                                                 |

#### intel\_vvp\_wbs\_set\_cfa\_x1\_range\_lo

##### Prototype

```
int
intel_vvp_wbs_set_cfa_x1_range_lo(intel_vvp_wbs_instance*
instance, uint32_t cfa_x1_range_lo);
```

|                    |                                                                                                                                                                       |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i> | Writes the lower value of the ratio range checking for the second column in a 2x2 pixel group. The instance must be a valid intel_vvp_wbs_instance fully initialized. |
| <i>Arguments</i>   | instance - pointer to the intel_vvp_wbs_instance software driver instance structure                                                                                   |

cfa\_x1\_range\_lo - new value of the lower range checking for the second column in a 2x2 pixel group

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpWbsValueErr if the value of cfa\_x1\_range\_lo is out of range

kIntelVvpWbsCommitPendingErr if a previous commit request is pending

### intel\_vvp\_wbs\_get\_cfa\_x1\_range\_hi

#### Prototype

```
uint32_t
intel_vvp_wbs_get_cfa_x1_range_hi(intel_vvp_wbs_instance*
instance);
```

**Description** Returns the upper value of the ratio range checking for the second column in a 2x2 pixel group. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

**Returns** the upper value of the ratio range checking for the second column in a 2x2 pixel group

### intel\_vvp\_wbs\_set\_cfa\_x1\_range\_hi

#### Prototype

```
int
intel_vvp_wbs_set_cfa_x1_range_hi(intel_vvp_wbs_instance*
instance, uint32_t cfa_x1_range_hi);
```

**Description** Writes the upper value of the ratio range checking for the second column in a 2x2 pixel group. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments** instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

cfa\_x1\_range\_hi - new value of the upper range checking for the second column in a 2x2 pixel group

**Returns** kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpWbsValueErr if the value of cfa\_x1\_range\_hi is out of range

kIntelVvpWbsCommitPendingErr if a previous commit request is pending

### intel\_vvp\_wbs\_get\_table\_entries

#### Prototype

```
int
intel_vvp_wbs_get_table_entries(intel_vvp_wbs_instance*
instance, uint8_t vert, uint8_t horiz,
uint8_t* num_entries, uint32_t*
entry_store);
```

**Description** Reads the white balance statistics in a raw format. The instance must be a valid intel\_vvp\_wbs\_instance fully initialized.

**Arguments**

instance - pointer to the intel\_vvp\_wbs\_instance software driver instance structure

vert - vertical coordinate of the zone

horiz - horizontal coordinate of the zone

num\_entries - pointer of the variable to return the number of 20-bits entries

entry\_store - pointer to the vector to return the white balance statistics of the zone

**Returns**

kIntelVvpCoreOk (0) in case of success, a negative error code in case of an error

kIntelVvpCoreInstanceErr if the instance is a null pointer

kIntelVvpWbsValueErr if at least one of the arguments vert or horiz is out of range

kIntelVvpWbsPointerErr if at least one of the arguments num\_entries or entry\_store is a null pointer

### intel\_vvp\_wbs\_get\_formatted\_table\_result

#### Prototype

```
intel_vvp_wbs_zone_result
intel_vvp_wbs_get_formatted_table_result(
intel_vvp_wbs_instance* instance, uint8_t
vert, uint8_t horiz);
```

|                    |                                                                                                                                                                                           |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Description</i> | Reads the white balance statistics as a formatted table. The instance must be a valid intel_vvp_wbs_instance fully initialized.                                                           |
| <i>Arguments</i>   | <p>instance - pointer to the intel_vvp_wbs_instance software driver instance structure</p> <p>vert - vertical coordinate of the zone</p> <p>horiz - horizontal coordinate of the zone</p> |
| <i>Returns</i>     | <p>for any type of invalid arguments a formatted statistics table filled with all ones</p> <p>for a valid set of arguments a formatted table filled with statistics of the zone</p>       |

### Related Information

[Video and Vision Processing IPs Software Programming Model](#) on page 38



## 56. Design Security

---

You control the overall system security when using video and vision IPs. Altera recommends that you evaluate potential security risks that may exist.

### 56.1. Memory Subsystems

Some video and vision IPs require you to connect them to RAM. Configuring these blocks requires you to set a base address. Other configuration options determine the RAM space the IP requires. These two elements define the address space the IP operates over. The IPs only use the address space defined by these parameters. You should appropriately map modules in the address space of the system. You should implement any access control protections associated with the memory subsystem. The IPs do not perform any system configuration checks. Platform Designer performs some limited system configuration checks, which may or may not be sufficient according to the complexity of requirements.

### 56.2. Considering Design Security

When designing the systems based on video and vision processing IPs always conduct a security review of your final design to ensure it meets your security goals.

You can apply these precautions to production or deployed systems. Not all precautions apply to all designs or IPs.

1. Remove the JTAG interface from your designs.
2. To guarantee video data integrity, restrict access to memory allocated to the frame buffer.
3. Control access to areas of memory to prevent unauthorized transactions or corruption by other IPs in the design.
4. Ensure that you correctly configure the IP via the I<sup>2</sup>C interface and that the input video is valid.
5. Protect the bitstreams for your design using the security features built-in to Quartus Prime.
6. Enable a password for the design's ARM processor.
7. Protect access to your design through development kit ports.
8. Restrict debugging access by tools such as Signal Tap.
9. Encrypt information on SD cards, FPGA bitstreams, and within DDR memory devices.
10. Apply security features to stored video data.

11. Consider using an HDCP encryption scheme.
12. Consider the boot sequence and boot security aspects of your own design.
13. Implement Altera's FPGA bitstream encryption technology to further protect the FPGA design content of your products. For information on FPGA bitstream encryption technology, refer to *Using the Design Security Features in Altera FPGAs*.



## 57. Document Revision History for Video and Vision Processing Suite User Guide

| Document Version | Quartus Prime Version | Changes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2026.03.03       | 25.1                  | <ul style="list-style-type: none"> <li>Added more info to <code>actv_vid_size</code> table in <i>TMO IP Registers</i>.</li> <li>Added <b>Width of memory host data bus</b> parameter to <i>Warp IP Parameters</i>.</li> <li>Updated the <i>Altera Streaming Video lite to Altera Streaming Video full</i> section in the <i>Protocol Converter IP Functional Description</i> topic to improve clarity.</li> <li>Rebranded to Altera.</li> </ul>                                                                          |
| 2025.03.30       | 25.1                  | Added support for Agilex 3 devices                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 2025.09.15       | 25.1                  | Added more info to <code>actv_vid_size</code> table.                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 2024.09.30       | 24.3                  | <ul style="list-style-type: none"> <li>Added Text Box IP</li> <li>Added dithering to Bits per Color Sample Adapter IP</li> <li>Changed Adaptive Noise Reduction IP <b>Number of Color Planes</b> parameter values.</li> </ul>                                                                                                                                                                                                                                                                                            |
| 2024.07.08       | 24.2                  | <ul style="list-style-type: none"> <li>Added <i>Advanced Test Pattern Generator IP</i></li> <li>Added <i>Adaptive Noise Reduction IP</i></li> <li>Added support for Agilex 5 E-series devices</li> <li>Added motion adaptive algorithm to Deinterlacer IP.</li> <li>Updated Warp IP: <ul style="list-style-type: none"> <li><input type="checkbox"/> Added mipmaps</li> <li><input type="checkbox"/> Added more pixels in parallel</li> <li><input type="checkbox"/> Increased maximum image size</li> </ul> </li> </ul> |
| 2024.05.08       | 24.1                  | Corrected <i>Color Plane Manager Registers</i> 0x0120 to 0x0138                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 2024.04.01       | 24.1                  | <ul style="list-style-type: none"> <li>Added Color Plane Manager IP</li> <li>Updated <i>Test Pattern Generator IP Performance and Resources</i></li> <li>Updated <i>Test Pattern Generator IP Parameters</i> and added new parameters.</li> <li>Added zone plate and digital clock to <i>Test Pattern Generator IP Functional Description</i>.</li> <li>Added new registers to <i>Test Pattern Generator IP Registers</i></li> <li>Added new APIs to <i>Test Pattern Generator IP Software APIs</i></li> </ul>           |
| continued...     |                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

| Document Version | Quartus Prime Version | Changes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------------------|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2023.12.18       | 23.4                  | <ul style="list-style-type: none"> <li>Added new IPs: <ul style="list-style-type: none"> <li><input type="checkbox"/> 1D LUT</li> <li><input type="checkbox"/> Black Level Correction</li> <li><input type="checkbox"/> Black Level Statistics</li> <li><input type="checkbox"/> Defect Pixel Correction</li> <li><input type="checkbox"/> Demosaic</li> <li><input type="checkbox"/> Histogram Statistics</li> <li><input type="checkbox"/> Unsharp Mask</li> <li><input type="checkbox"/> Vignette Correction</li> <li><input type="checkbox"/> White Balance Correction</li> <li><input type="checkbox"/> White Balance Statistics</li> </ul> </li> <li>Deleted MAX_WIDTH from <i>Deinterlacer IP Registers</i></li> </ul>                                                                                                                                                                                                                              |
| 2023.10.02       | 23.3                  | <ul style="list-style-type: none"> <li>For the Bits per Color Sample Adapter IP: <ul style="list-style-type: none"> <li><input type="checkbox"/> Added new parameters</li> <li><input type="checkbox"/> Added new control interface</li> <li><input type="checkbox"/> Added <i>Bits per Color Sample Adapter IP Registers</i></li> <li><input type="checkbox"/> Added <i>Bits per Color Sample Adapter IP Software API</i></li> </ul> </li> <li>Updated bilinear information in <i>Chroma Resampling Functional Description</i></li> <li>Added new interfaces to <i>Clocked Video Output IP Interfaces</i></li> <li>Added new interfaces to <i>Video Timing Generator IP Interfaces</i></li> </ul>                                                                                                                                                                                                                                                         |
| 2023.06.26       | 23.2                  | <ul style="list-style-type: none"> <li>Added new IPs: <ul style="list-style-type: none"> <li><input type="checkbox"/> Bits per Color Sample Adapter IP</li> <li><input type="checkbox"/> Video and Vision Monitor IP</li> <li><input type="checkbox"/> Video Frame Reader IP</li> <li><input type="checkbox"/> Video Frame Writer IP</li> </ul> </li> <li>Added weave algorithm to Deinterlacer IP</li> <li>Added <b>Genlock Profiler</b> parameter to Genlock Controller IP.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 2023.04.03       | 23.1                  | <ul style="list-style-type: none"> <li>Added new <b>Edge behavior</b> parameter to <i>Scaler IP Parameters</i></li> <li>Added <i>Filter Behavior at Edge Boundaries</i></li> <li>Updated the product family name to Intel Agilex 7.</li> <li>For the FIR Filter IP: <ul style="list-style-type: none"> <li><input type="checkbox"/> Added new parameter to <i>FIR Filter Parameters</i></li> <li><input type="checkbox"/> Added registers to <i>FIR Filter Registers</i>.</li> <li><input type="checkbox"/> Added <code>intel_vvp_fir_runtime_coeffs_load</code> to <i>FIR Filter Software API</i></li> </ul> </li> <li>Updated <i>Warp IP Performance and Resources</i></li> </ul>                                                                                                                                                                                                                                                                        |
| 2022.12.12       | 22.4                  | <ul style="list-style-type: none"> <li>Added new Genlock Controller IP</li> <li>Corrected Clipper register entries 0x148 and 0x14C</li> <li>Corrected example in <i>External Memory for Warp IP</i>.</li> <li>For the clocked Video Input IP: <ul style="list-style-type: none"> <li><input type="checkbox"/> Added new step to <i>Initializing the Clocked Video Input IP</i></li> <li><input type="checkbox"/> Added new <code>f0f1_fps_cnt</code> register</li> <li><input type="checkbox"/> Added new APIs.</li> </ul> </li> <li>For the FIR Filter IP: <ul style="list-style-type: none"> <li><input type="checkbox"/> Added <i>FIR Filter IP Performance and Resources</i></li> <li><input type="checkbox"/> Changed even to odd in <i>Horizontal Symmetry, Vertical Symmetry and Diagonal Symmetry</i></li> </ul> </li> <li>Added new parameters to the Scaler IP.</li> <li>Added new parameters to the Switch IP and updated registers.</li> </ul> |
| continued...     |                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

| Document Version | Quartus Prime Version | Changes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|------------------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2022.09.30       | 22.3                  | <ul style="list-style-type: none"> <li>Added new IPs: <ul style="list-style-type: none"> <li><input type="checkbox"/> Chroma Key IP</li> <li><input type="checkbox"/> Interlacer IP</li> <li><input type="checkbox"/> Stream Cleaner IP</li> <li><input type="checkbox"/> Switch IP</li> </ul> </li> <li>Added new 420 and 422 chroma parameters to Scaler IP</li> <li>Added single bounce and block cache tool to Warp IP.</li> <li>Changed some values in <i>FIR Filter Parameters</i>.</li> <li>Changed some values in <i>FIR Filter Precision</i></li> <li>Corrected <code>IMG_INFO_INTERLACE</code> bits in <i>Deinterlacer Registers</i></li> </ul>                                                                                                                                                                                                                                                                                                                                                                        |
| 2022.08.08       | 22.2                  | Updated description for Deinterlacer IP <i>IMG_INFO_INTERLACE Register</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 2022.06.25       | 22.2                  | <ul style="list-style-type: none"> <li>Added new IPs: <ul style="list-style-type: none"> <li><input type="checkbox"/> AXI-Stream broadcaster</li> <li><input type="checkbox"/> Clocked video input</li> <li><input type="checkbox"/> Clocked video output</li> <li><input type="checkbox"/> Deinterlacer</li> <li><input type="checkbox"/> Frame cleaner</li> <li><input type="checkbox"/> FIR filter</li> <li><input type="checkbox"/> Generic crosspoint</li> <li><input type="checkbox"/> Genlock signal router</li> <li><input type="checkbox"/> Video timing generator</li> </ul> </li> <li>Updated these APIs: <ul style="list-style-type: none"> <li><input type="checkbox"/> Chroma resampler</li> <li><input type="checkbox"/> Guard bands</li> <li><input type="checkbox"/> Protocol converter</li> <li><input type="checkbox"/> Scaler</li> <li><input type="checkbox"/> Test pattern generator</li> <li><input type="checkbox"/> Video frame buffer</li> </ul> </li> <li>Changed <i>Scaler Parameters</i></li> </ul> |
| 2022.04.04       | 22.1                  | <ul style="list-style-type: none"> <li>Added new data to <i>Warp IP Performance and Resource Utilization</i></li> <li>Added <i>Input Resolution Restrictions for 90° or 270°</i> table to <i>Warp IP Block Description</i></li> <li>Changed Intel Agilex support to final.</li> <li>Updated parameter names in <i>Chroma Resampler Parameters</i> and <i>Scaler Parameters</i></li> <li>Added interlaced video support to <i>About the Video Frame Buffer</i>.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| continued...     |                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

| Document Version | Quartus Prime Version | Changes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2022.02.15       | 21.4                  | <ul style="list-style-type: none"> <li>Added new IPs: <ul style="list-style-type: none"> <li><input type="checkbox"/> Chroma Resampler</li> <li><input type="checkbox"/> Clipper</li> <li><input type="checkbox"/> Clocked Video to Streaming Full-Raster Converter</li> <li><input type="checkbox"/> Color Space Converter</li> <li><input type="checkbox"/> Full-raster to Clocked Video Converter</li> <li><input type="checkbox"/> Full-raster to Streaming Converter</li> <li><input type="checkbox"/> Guard Bands</li> <li><input type="checkbox"/> Mixer</li> <li><input type="checkbox"/> Pixel in Parallel Converter</li> <li><input type="checkbox"/> Scaler</li> <li><input type="checkbox"/> Test Pattern Generator</li> <li><input type="checkbox"/> Video Frame Buffer</li> <li><input type="checkbox"/> Video Streaming FIFO</li> </ul> </li> <li>Added new APIs to Warp IP</li> <li>Removed two features from <i>Warp IP Features</i></li> <li>Added new information to Tone Mapping Operator IP: <ul style="list-style-type: none"> <li><input type="checkbox"/> Four new modes</li> <li><input type="checkbox"/> New <i>Examples of horizontal slider and region of interest</i> figure</li> <li><input type="checkbox"/> Two new register tables</li> <li><input type="checkbox"/> New APIs</li> </ul> </li> <li>Added <i>Control Examples</i></li> <li>Added Intel FPGA streaming video full information to Protocol Converter IP</li> <li>Added <b>Easy warp</b> to Warp IP.</li> </ul> |
| 2021.09.30       | 21.3                  | Added new parameters to 3D LUT IP: <ul style="list-style-type: none"> <li>Optional output alpha channel</li> <li>LUT sizes of 9<sup>3</sup></li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 2021.09.10       | 21.2                  | Initial release.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |